



QSpace tensor library

v4.0

Andreas Weichselbaum
(May 2024)

Open source under the Apache 2.0 license.

Generated by Doxygen 1.10.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	5
2.1 Class List	5
3 File Index	9
3.1 File List	9
4 Class Documentation	11
4.1 __wb_msgbuf Struct Reference	11
4.2 Wb::ARGV Class Reference	11
4.3 Mx::Array< T > Class Template Reference	12
4.4 Wb::Bitset Class Reference	13
4.5 Wb::bitset< T > Class Template Reference	19
4.6 blockSpaceQS< TQ, TD > Class Template Reference	20
4.7 Wb::cc_quad Class Reference	20
4.8 cdata< TD > Class Template Reference	20
4.9 CData< TQ, TD > Class Template Reference	27
4.10 cgc_contract_id< TM > Class Template Reference	31
4.11 cgdStatus Class Reference	37
4.12 cgrType Class Reference	38
4.13 Wb::citerator< T, T2 > Class Template Reference	39
4.14 Wb::citerator< T, T2 * > Class Template Reference	40
4.15 Wb::CleanUp Class Reference	40
4.16 Wb::Clock Class Reference	40
4.17 Wb::ClockSet Class Reference	41
4.18 Wb::counter Class Reference	41
4.19 CPAT< TQ, TD > Class Template Reference	42
4.20 CPAT_Set< TQ, TD > Class Template Reference	43
4.21 cpu_time Class Reference	43
4.22 CRef< TQ > Class Template Reference	44
4.23 CStore< TQ > Class Template Reference	46
4.24 ctrlIdx Class Reference	48
4.25 CG::FileLock Class Reference	55
4.26 genRG_base< TQ, TD > Class Template Reference	55
4.27 genRG_struct< TQ, TD > Class Template Reference	56
4.28 Wb::gpara__ Class Reference	57
4.29 groupIndex< T > Class Template Reference	57
4.30 icFlags Class Reference	58
4.31 indexSparseRef< TD > Class Template Reference	58
4.32 Wb::IOstat Class Reference	59
4.33 is_pointer_< T > Class Template Reference	59

4.34 is_pointer_< T * > Class Template Reference	59
4.35 is_pointer_< T ** > Class Template Reference	60
4.36 is_pointer_< T *** > Class Template Reference	60
4.37 itag_ Class Reference	60
4.38 iTags Class Reference	61
4.39 Wb::iterator< T, T2 > Class Template Reference	69
4.40 Wb::iterator< T, T2 * > Class Template Reference	69
4.41 IterOM_DATA< TD > Class Template Reference	70
4.42 iterSparseCMref< TD > Class Template Reference	70
4.43 Wb::line_stat__ Class Reference	71
4.44 Wb::LogException Struct Reference	71
4.45 Wb::matFile Class Reference	72
4.46 MEM_SET< T > Class Template Reference	72
4.47 MEM_SET< T * > Class Template Reference	73
4.48 MHash< TM > Class Template Reference	73
4.49 Wb::mpfr__< P > Class Template Reference	73
4.50 Wb::MREC Class Reference	76
4.51 mxArray_buf Class Reference	76
4.52 MXPut Class Reference	77
4.53 mxStruct Class Reference	78
4.54 Wb::myname__ Class Reference	78
4.55 NRGData< TQ, TD > Class Template Reference	79
4.56 NRGIndex< TQ, TD > Class Template Reference	81
4.57 Wb::num2Fmt< T > Class Template Reference	81
4.58 opFlags< T > Class Template Reference	82
4.59 OPTS Class Reference	82
4.60 pairPatternCG< T > Class Template Reference	83
4.61 Wb::Path Class Reference	83
4.62 Wb::Profile Class Reference	84
4.63 QBlock< TQ, TD, TE > Class Template Reference	84
4.64 QDir Class Reference	85
4.65 QHash< TQ > Class Template Reference	92
4.66 QHash_ Class Reference	92
4.67 QMap< TQ > Class Template Reference	93
4.68 qset< TQ > Class Template Reference	93
4.69 QSet< TQ > Class Template Reference	100
4.70 QSpace< TQ, TD > Class Template Reference	102
4.71 QType Class Reference	109
4.72 QVec Class Reference	110
4.73 Wb::rand Class Reference	118
4.74 RCStore Class Reference	118
4.75 RIXS_para Class Reference	119

4.76 RSpecData< TS > Class Template Reference	120
4.77 RStore< TQ, TD > Class Template Reference	120
4.78 wblog::SBUF Class Reference	121
4.79 Wb::SigHandler Class Reference	121
4.80 sparseCollector2D< TD > Class Template Reference	122
4.81 sparseIndex2D< T > Class Template Reference	122
4.82 Spectral< TS > Class Template Reference	123
4.83 Wb::sptr< T > Class Template Reference	124
4.84 STAT_DGEMM Class Reference	124
4.85 wblog::stdio_buf Class Reference	125
4.86 Wb::String Class Reference	125
4.87 SVD_Data< TQ, TD > Class Template Reference	131
4.88 DY::Symmetry Class Reference	132
4.89 TDSpectral< TS > Class Template Reference	133
4.90 tensor_ref_ Class Reference	133
4.91 tensorRef_ Class Reference	134
4.92 tensorRefs Class Reference	135
4.93 Wb::termcolor Class Reference	141
4.94 Wb::tmpFile Class Reference	142
4.95 Wb::VersionInfo Class Reference	142
4.96 wbarray< T > Class Template Reference	143
4.97 wbarrRef< T > Class Template Reference	151
4.98 wbcomplex Class Reference	151
4.99 wbdebug_dummy Class Reference	153
4.100 wbhist Class Reference	153
4.101 wbindex Class Reference	153
4.102 wbIndex Class Reference	160
4.103 Wb::WbListMTRACK Class Reference	167
4.104 wblogbuf_struct Class Reference	168
4.105 wblogbuffer Class Reference	168
4.106 wbMatrix< T > Class Template Reference	169
4.107 wbMsg Class Reference	176
4.108 wbperm Class Reference	176
4.109 wbRecs< T > Class Template Reference	184
4.110 wbREF< TX > Class Template Reference	184
4.111 wbsparray< TD > Class Template Reference	184
4.112 wbstring Class Reference	190
4.113 wbtopy Class Reference	198
4.114 WbUtil< T > Class Template Reference	199
4.115 wbvector< T > Class Template Reference	199
4.116 DY::weight_info< TQ > Class Template Reference	206
4.117 DY::Weights< TQ > Class Template Reference	206

4.118 x3map< TQ, TD > Class Template Reference	207
4.119 X3Map< TQ, TD > Class Template Reference	208
4.120 XMAP Class Reference	208
4.121 XMAP_CMat Class Reference	215
4.122 XMAP_Node Class Reference	215
5 File Documentation	217
5.1 clebsch.hh	217
5.2 cpat.hh	272
5.3 dmrho.cc	286
5.4 gathered.cc	293
5.5 gathered.hh	316
5.6 matlib.cc	323
5.7 memlib.cc	326
5.8 memlib.hh	329
5.9 memtrack.hh	332
5.10 mexlib.cc	333
5.11 mexlib.hh	345
5.12 mpsortho.cc	351
5.13 mpsortho.hh	365
5.14 mxlib.hh	367
5.15 nomex.hh	373
5.16 nrgdata.hh	375
5.17 QSpace.hh	405
5.18 QSpace_aux.hh	440
5.19 qspace_lib.cc	443
5.20 spectral.hh	451
5.21 util.cc	471
5.22 util.hh	485
5.23 wbararray.cc	492
5.24 wbararray.hh	540
5.25 wbararray_blas.cc	573
5.26 wbararray_blas.hh	586
5.27 wbbitsset.cc	590
5.28 wbbitsset.hh	591
5.29 wbbblas.hh	593
5.30 wbclock.cc	597
5.31 wbclock.hh	600
5.32 wbcomplex.cc	605
5.33 wbcomplex.hh	605
5.34 wbeigs.hh	610
5.35 wbffft.hh	611

5.36 wbindex.cc	612
5.37 wbindex.hh	631
5.38 wbio.c	646
5.39 wbio.h	649
5.40 wblib.h	651
5.41 wblibx.h	660
5.42 wblog.c	661
5.43 wblog.h	676
5.44 wbMatIO.hh	680
5.45 wbMatrix.cc	682
5.46 wbMatrix.hh	707
5.47 wbMatrix_blas.cc	740
5.48 wbMatrix_blas.hh	748
5.49 wbmpfr.hh	749
5.50 wbmshh	755
5.51 wbomp.cc	757
5.52 wbomp.hh	766
5.53 wbopts.hh	769
5.54 wbperm.cc	774
5.55 wbperm.hh	779
5.56 wbsort.cc	783
5.57 wbsort.hh	787
5.58 wbsparray.cc	788
5.59 wbsparray.hh	837
5.60 wbstring.cc	860
5.61 wbstring.hh	865
5.62 wbutil.hh	871
5.63 wbvector.cc	874
5.64 wbvector.hh	891
5.65 xmap.hh	914
Index	917

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

__wb_msgbuf	11
Wb::ARGV	11
Mx::Array< T >	12
Wb::bitset< T >	19
blockSpaceQS< TQ, TD >	20
Wb::cc_quad	20
cgdStatus	37
cgrType	38
Wb::citerator< T, T2 >	39
Wb::citerator< T, T2 * >	40
Wb::CleanUp	40
Wb::Clock	40
Wb::ClockSet	41
Wb::counter	41
CPAT< TQ, TD >	42
CPAT_Set< TQ, TD >	43
cpu_time	43
CRef< TQ >	44
CStore< TQ >	46
exception	
Wb::LogException	71
CG::FileLock	55
genRG_base< TQ, TD >	55
genRG_struct< TQ, TD >	56
Wb::gpara__	57
groupIndex< T >	57
icFlags	58
indexSparseRef< TD >	58
Wb::IOstat	59
is_pointer_< T >	59
is_pointer_< T * >	59
is_pointer_< T ** >	60
is_pointer_< T *** >	60
itag_	60
Wb::iterator< T, T2 >	69

Wb::iterator< T, T2 * >	69
IterOM_DATA< TD >	70
iterSparseCMref< TD >	70
Wb::line_stat_	71
Wb::matFile	72
MEM_SET< T >	72
MEM_SET< T * >	73
MHash< TM >	73
Wb::mpfr_< P >	73
Wb::MREC	76
mxArray_buf	76
MXPut	77
mxStruct	78
Wb::myname_	78
NRGData< TQ, TD >	79
NRGIndex< TQ, TD >	81
Wb::num2Fmt< T >	81
opFlags< T >	82
OPTS	82
pairPatternCG< T >	83
pairPatternCG< double >	83
Wb::Path	83
Wb::Profile	84
QBlock< TQ, TD, TE >	84
QBlock< TQ, TD, double >	84
QHash< TQ >	92
QHash_	92
QMap< TQ >	93
QSet< TQ >	100
CData< TQ, RTD >	27
CData< TQ, TD >	27
QSpace< TQ, TD >	102
QType	109
Wb::rand	118
RCStore	118
RIXS_para	119
RSpecData< TS >	120
RStore< TQ, TD >	120
wblog::SBUF	121
Wb::SigHandler	121
sparseCollector2D< TD >	122
sparseIndex2D< T >	122
Spectral< TS >	123
Wb::sptr< T >	124
Wb::sptr< double >	124
Wb::sptr< int >	124
Wb::sptr< TD >	124
Wb::sptr< TS >	124
Wb::sptr< wbcomplex >	124
STAT_DGEMM	124
wblog::stdio_buf	125
SVD_Data< TQ, TD >	131
DY::Symmetry	132
TDSpectral< TS >	133
tensor_ref_	133
tensorRef_	134
Wb::termcolor	141
Wb::tmpFile	142

Wb::VersionInfo	142
wbarray< T >	143
wbarray< double >	143
wbarray< int >	143
wbarray< TD >	143
wbarray< TS >	143
wbarray< wbcomplex >	143
wbarrRef< T >	151
wbcomplex	151
wbdebug_dummy	153
wbhist	153
Wb::WbListMTRACK	167
wblogbuf_struct	168
wblogbuffer	168
wbMatrix< T >	169
wbMatrix< const CRef< TQ > * >	169
wbMatrix< CRef< TQ > >	169
wbMatrix< double >	169
wbMatrix< int >	169
wbMatrix< SPIDX_T >	169
wbMatrix< TQ >	169
wbMatrix< TS >	169
wbMatrix< unsigned >	169
wbMatrix< wbcomplex >	169
wbMatrix< widx_t >	169
wbMsg	176
wbRecs< T >	184
wbREF< TX >	184
wbsparray< TD >	184
cdata< RTD >	20
cdata< TD >	20
wbsparray< RTD >	184
wbtop	198
WbUtil< T >	199
wbvector< T >	199
wbvector< char >	199
QDir	85
Wb::Bitset	13
Wb::String	125
wbstring	190
wbvector< double >	199
wbvector< int >	199
wbvector< itag_ >	199
iTags	61
wbvector< mxArray * >	199
wbvector< NRGIndex< TQ, TD > >	199
wbvector< pairPatternCG< double > >	199
wbvector< QBlock< TQ, TD, double > >	199
wbvector< QSpace< TQ, TD > >	199
wbvector< QType >	199
QVec	110
wbvector< RSpecData< TS > >	199
wbvector< RTD >	199
wbvector< size_t >	199
wbvector< SPIDX_T >	199
wbvector< sSPIDX_T >	199
wbvector< TD >	199

wbvector< tensorRef_ >	199
tensorRefs	135
wbvector< TM >	199
cgc_contract_id< TM >	31
wbvector< TQ >	199
qset< TQ >	93
wbvector< TS >	199
wbvector< unsigned >	199
ctrlIdx	48
wbvector< unsigned long >	199
wbvector< warray< TD > * >	199
wbvector< wbMatrix< unsigned > >	199
wbvector< wbSparrowTD >	199
wbvector< wbstring >	199
wbvector< wbvector< CPAT_Set< TQ, TD > > >	199
wbvector< wbvector< QSpace< TQ, TD > > >	199
wbvector< widx_t >	199
wbIndex	160
wbindex	153
wbvector< wperm_t >	199
wbperm	176
wbvector< XMAP_Node >	199
XMAP	208
DY::weight_info< TQ >	206
DY::Weights< TQ >	206
x3map< TQ, TD >	207
X3Map< TQ, TD >	208
XMAP_CMat	215
XMAP_Node	215

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

__wb_msgbuf	11
Wb::ARGV	11
Mx::Array< T >	12
Wb::Bitset	13
Wb::bitset< T >	19
blockSpaceQS< TQ, TD >	20
Wb::cc_quad	20
cdata< TD >	20
CData< TQ, TD >	27
cgc_contract_id< TM >	31
cgdStatus	37
cgrType	38
Wb::citerator< T, T2 >	39
Wb::citerator< T, T2 * >	40
Wb::CleanUp	40
Wb::Clock	40
Wb::ClockSet	41
Wb::counter	41
CPAT< TQ, TD >	42
CPAT_Set< TQ, TD >	43
cpu_time	43
CRef< TQ >	44
CStore< TQ >	46
ctrlIdx	48
CG::FileLock	55
genRG_base< TQ, TD >	55
genRG_struct< TQ, TD >	56
Wb::gpara__	57
groupIndex< T >	57
icFlags	58
indexSparseRef< TD >	58
Wb::IOstat	59
is_pointer_< T >	59
is_pointer_< T * >	59
is_pointer_< T ** >	60

is_pointer_< T *** >	60
itag_	60
iTags	61
Wb::iterator< T, T2 >	69
Wb::iterator< T, T2 * >	69
IterOM_DATA< TD >	70
iterSparseCMref< TD >	70
Wb::line_stat__	71
Wb::LogException	71
Wb::matFile	72
MEM_SET< T >	72
MEM_SET< T * >	73
MHash< TM >	73
Wb::mpfr__< P >	73
Wb::MREC	76
mxArray_buf	76
MXPut	77
mxStruct	78
Wb::myname__	78
NRGData< TQ, TD >	79
NRGIndex< TQ, TD >	81
Wb::num2Fmt< T >	81
opFlags< T >	82
OPTS	82
pairPatternCG< T >	83
Wb::Path	83
Wb::Profile	84
QBlock< TQ, TD, TE >	84
QDir	85
QHash< TQ >	92
QHash_	92
QMap< TQ >	93
qset< TQ >	93
QSet< TQ >	100
QSpace< TQ, TD >	102
QType	109
QVec	110
Wb::rand	118
RCStore	118
RIXS_para	119
RSpecData< TS >	120
RStore< TQ, TD >	120
wblog::SBUF	121
Wb::SigHandler	121
sparseCollector2D< TD >	122
sparseIndex2D< T >	122
Spectral< TS >	123
Wb::sptr< T >	124
STAT_DGEMM	124
wblog::stdio_buf	125
Wb::String	125
SVD_Data< TQ, TD >	131
DY::Symmetry	132
TDSpectral< TS >	133
tensor_ref_	133
tensorRef_	134
tensorRefs	135
Wb::termcolor	141

Wb::tmpFile	142
Wb::VersionInfo	142
wbarray< T >	143
wbarrRef< T >	151
wbcomplex	151
wbdebug_dummy	153
wbhist	153
wbindex	153
wbIndex	160
Wb::WbListMTRACK	167
wblogbuf_struct	168
wblogbuffer	168
wbMatrix< T >	169
wbMsg	176
wbperm	176
wbRecs< T >	184
wbREF< TX >	184
wbsparray< TD >	184
wbstring	190
wbtop	198
WbUtil< T >	199
wbvector< T >	199
DY::weight_info< TQ >	206
DY::Weights< TQ >	206
x3map< TQ, TD >	207
X3Map< TQ, TD >	208
XMAP	208
XMAP_CMat	215
XMAP_Node	215

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

clebsch.hh	217
cpat.hh	272
dmrho.cc	286
gathered.cc	293
gathered.hh	316
matlib.cc	323
memlib.cc	326
memlib.hh	329
memtrack.hh	332
mexlib.cc	333
mexlib.hh	345
mpsortho.cc	351
mpsortho.hh	365
mxlib.hh	367
nomex.hh	373
nrgdata.hh	375
QSpace.hh	405
QSpace_aux.hh	440
qspace_lib.cc	443
spectral.hh	451
util.cc	471
util.hh	485
wbarray.cc	492
wbarray.hh	540
wbarray_blas.cc	573
wbarray_blas.hh	586
wbbitsset.cc	590
wbbitsset.hh	591
wbblas.hh	593
wbclock.cc	597
wbclock.hh	600
wbcomplex.cc	605
wbcomplex.hh	605
wbeigs.hh	610
wbfft.hh	611

wbindex.cc	612
wbindex.hh	631
wbio.c	646
wbio.h	649
wblib.h	651
wblibx.h	660
wblog.c	661
wblog.h	676
wbMatlO.hh	680
wbMatrix.cc	682
wbMatrix.hh	707
wbMatrix_blas.cc	740
wbMatrix_blas.hh	748
wbmpfr.hh	749
wbmsg.hh	755
wbomp.cc	757
wbomp.hh	766
wbopts.hh	769
wbperm.cc	774
wbperm.hh	779
wbsort.cc	783
wbsort.hh	787
wbsparray.cc	788
wbsparray.hh	837
wbstring.cc	860
wbstring.hh	865
wbutil.hh	871
wbvector.cc	874
wbvector.hh	891
xmap.hh	914

Chapter 4

Class Documentation

4.1 `__wb_msgbuf` Struct Reference

Public Attributes

- long **type**
- time_t **time**
- char **text** [128]
- double **val**

The documentation for this struct was generated from the following file:

- `wbmsg.hh`

4.2 `Wb::ARGV` Class Reference

Public Member Functions

- **ARGV** (`va_list *vl=NULL`)

The documentation for this class was generated from the following file:

- `wblib.h`

4.3 Mx::Array< T > Class Template Reference

Public Member Functions

- **Array** (unsigned ndim, size_t *dims, char t_=0, char c_=-1)
- **Array** (size_t d1, size_t d2, char t_=0, char c_=-1)
- **Array** (const [wbvector](#)< size_t > &S, char t_=0, char c_=-1)
- **Array** (const mxArray *a, char t_=0, char c_=-1)
- **Array** (const char *F, int L, const mxArray *a, char t_=0, char c_=-1)
- [wbstring toStr](#) (char vflag=0, unsigned l=64) const
- [wbstring toStrT](#) (char vflag=0) const
- void **init_xstr** ()
- const char * **xStr** () const
- mxArray * **Return** (T *&dd)
- mxArray * **Return** ()
- [Array](#) & **init** (unsigned ndim, size_t *dims, char t_=-1, char c_=-1)
- [Array](#) & **init** (const mxArray *a, char t_=-1, char c_=-1)
- template<class Tb >
mxArray * **copyFromTR** (const Tb *b, char tcheck=0)
- template<class Tb >
int **copy_from** (const Tb *b, char tcheck=0)
- template<class Tb >
size_t **copyTo** (Tb *b, char tcheck=0) const
- size_t **copy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- size_t **rcopy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- size_t **zcopy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- template<class Tx >
size_t **cpy_to_** (T *b, const [wbperm](#) *P, char tcheck, const Tx *ad_) const
- size_t **ncopy_to** (T *b, size_t n, char tcheck=0)
- size_t **copy_to** (CGD_TYPE *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (CGR_TYPE *b, const [wbperm](#) *P, char tcheck) const
- size_t **zcopy_to** ([wbcomplex](#) *b, const [wbperm](#) *P, char ck) const
- size_t **copy_to** ([wbcomplex](#) *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (double *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (int *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (unsigned *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (size_t *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (char *b, const [wbperm](#) *P, char tcheck) const

Public Attributes

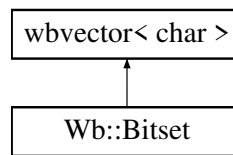
- mxArray * **ax**
- T * **data**
- size_t **len**
- unsigned **rank**
- bool **trans**
- char **cmplx**
- char **mxref**
- std::string **xstr**

The documentation for this class was generated from the following files:

- mxlib.hh
- mexlib.cc

4.4 Wb::Bitset Class Reference

Inheritance diagram for Wb::Bitset:



Public Member Functions

- **Bitset** (unsigned n=0)
- **Bitset** (const [Bitset](#) &b)
- [Bitset](#) & **init** (unsigned n=0)
- [Bitset](#) & **init** (unsigned n, const char *b)
- [Bitset](#) & **setall** (bool t)
- [Bitset](#) & **set** (unsigned i)
- [Bitset](#) & **unset** (unsigned i)
- bool **isset** (unsigned i)
- unsigned **nnz** () const
- bool **operator==** (const [Bitset](#) &b) const
- bool **operator!=** (const [Bitset](#) &b) const
- [wbstring](#) **toStr** () const
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- void **put** (const char *vname, const char *ws="base") const
- void **put** (const char *file, int line, const char *vname, const char *ws="base") const

Public Member Functions inherited from [wbvector< char >](#)

- **wbvector** (size_t l, char x)
- **wbvector** (size_t l, const char *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const char *d0)
- [wbvector](#) & **init** (size_t l, const char *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & **init** (const [wbsparray](#)< char > &S)
- [wbvector](#) & **init** (size_t l1, const char *d1, size_t l2, const char *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const char *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const char &x)

- [wbvector](#) & **initStride** (size_t l, const char *d0, size_t stride)
- [wbvector](#) & **init_stride** (const char *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const char *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< char > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< char > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< char > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const char *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpfr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< char > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const char &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const char &x)
- void **set** (size_t offset, const char *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const char &x)
- [wbvector](#) & **setMax** (const char &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const char &x)
- [wbvector](#) & **add** (const char *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const char &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const char &x) const
- size_t **find1** () const
- size_t **count** (const char &x) const
- size_t **icount** (const char &x) const
- size_t **nnz** () const
- size_t **nnz** (char eps) const
- size_t **numZeros** (char eps=char(1E-14)) const
- size_t **numEQ** (const char &) const
- size_t **numGT** (const char &) const
- size_t **numLT** (const char &) const
- [wbindex](#) & **find** (const char &x, [wbindex](#) &l, char iflag=0) const

- `wbindex` & `find` (const char &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex` & `find` (`wbindex` &l) const
- `wbindex` & `find` (`wbindex` &l, `wbindex` &lx) const
- `wbvector`< size_t > `find` () const
- `wbindex` & `findGT` (const char &x, `wbindex` &l, char iflag=0) const
- `wbindex` & `findGT` (const char &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex` & `findzeros` (`wbindex` &l) const
- `wbindex` & `findRange` (const char &x1, const char &x2, `wbindex` &l, const char *w=NULL) const
- int `findClosestSorted` (char r) const
- int `findClosest` (char r) const
- size_t `findValues` (const `wbvector` &v, `wbindex` &l) const
- `wbvector` & `setp` (const char *x)
- void `reset` ()
- void `cpy` (const char *v)
- void `cpy` (const char *x, size_t m)
- `wbvector` & `save2` (`wbvector` &v)
- void `swap` (`wbvector` &v)
- `wbvector` & `swap` (size_t i, size_t j)
- `wbvector` & `Collect` (char &A1, char &A2)
- `wbvector` & `Collect` (char &A1, char &A2, char &A3)
- `wbvector` & `Skip` (size_t)
- `wbvector` & `Skip` (const WBINDEX &, char rflag=0)
- `wbvector` & `Move2_FE` (const WBINDEX &l, char FE_flag)
- `wbvector` & `Move2front` (const WBINDEX &l)
- `wbvector` & `Move2front` (size_t k)
- `wbvector` & `Move2end` (const WBINDEX &l)
- `wbvector` & `Move2end` (size_t k)
- `wbvector` & `mvLastTo` (size_t k)
- `wbvector` & `mvFirstTo` (size_t k)
- bool `checkSameLength` (const `wbvector` &a, const char *F=NULL, int L=0, const char *istr="") const
- bool `checkSameLength` (const `wbvector` &a, const `wbvector` &b, const char *F=NULL, int L=0, const char *istr="") const
- void `operator+=` (const T_ &a)
- void `operator+=` (const `wbvector` &b)
- void `operator-=` (const T_ &a)
- void `operator-=` (const `wbvector` &b)
- `wbvector` & `operator*=` (const T_ &a)
- `wbvector` & `operator/=` (const T_ &a)
- `wbvector` `operator*` (const T_ &a) const
- `wbvector` `operator+` (const T_ &a) const
- `wbvector` `operator+` (const `wbvector` &b) const
- `wbvector` `operator-` (const T_ &a) const
- `wbvector` `operator-` (const `wbvector` &b) const
- `wbvector` & `Plus` (const char *b)
- `wbvector` & `Plus` (const char *b, char bfac)
- `wbvector` & `Plus` (const `wbvector` &b, char bfac=1)
- void `plus` (const `wbvector` &b, `wbvector` &c, char bfac=1, char cfac=0) const
- `wbvector` & `times` (char fac, `wbvector` &x) const
- `wbvector` & `RevertSigns` ()
- int `isum` () const
- char `sum` () const
- char `sum` (size_t i1, const char *estr) const
- char `sum` (size_t i1, size_t i2) const
- char `cumsum_` (`wbvector` &S, char xflag=0) const
- char `Cumsum` (char zflag=0) const

- char **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< char > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- char **scalarProd** (const [wbvector](#) &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const [wbvector](#) &v, char fac=1) const
- char **normDiff2** (const [wbvector](#) &v, char fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- char **GSPProject** (const char *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const char *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const char &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const
- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const char &r) const
- bool **operator<=** (const [wbvector](#) &v) const

- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- char & **el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (Tl &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< char > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const char *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const
- char **max_** (char x) const
- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const
- char **avg** () const

- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, char *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, char *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getl** (size_t k, **wbvector** &v) const
- **wbvector** & **getl** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getl** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initl** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initl** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const char &offset, const char tflag=0) const

- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- void `add2MxStruct` (mxArray *S, unsigned i, char tflag=0) const
- void `add2MxStruct` (mxArray *S, const char *vname, char tflag=0) const
- `mxArray * mxCreateCell` (unsigned m, unsigned n) const
- void `add2MxCell` (mxArray *S, unsigned i, char tflag=0) const
- void `putg` (const char *vname="ans", char tflag=0) const
- void `putx` (const char *vname="ans", char tflag=0) const
- `wbstring toStr` (int n, const char *sep) const
- `wbstring toStr` () const
- `wbstring toStr` (int n, const char *sep) const
- `wbstring sizeStr` () const
- `wbstring toStrf` (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- `wbstring toStrf` (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Public Attributes

- unsigned `nbits`

Public Attributes inherited from `wbvector< char >`

- char * `data`
- size_t `len`
- char `isref`

Protected Member Functions

- `Bitset & NEW_BITSET` (unsigned n, const char *d=NULL)

Protected Member Functions inherited from `wbvector< char >`

- `wbvector & RENEW` (size_t n, const char *d=NULL, size_t l=-1, char init=1)
- `wbvector & RENEW_VAL` (size_t n, const char &x)

The documentation for this class was generated from the following files:

- `wbbitset.hh`
- `wbbitset.cc`

4.5 Wb::bitset< T > Class Template Reference

Public Member Functions

- `bitset` (const `bitset< T >` &b)
- `bitset & setall` (bool t)
- `bitset & set` (unsigned i)
- `bitset & unset` (unsigned i)
- bool `isset` (unsigned i)
- unsigned `nnz` () const
- bool `operator==` (const `bitset` &b) const
- bool `operator!=` (const `bitset` &b) const
- `wbstring toStr` () const
- `mxArray * toMx` () const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- void `add2MxStruct` (mxArray *S, unsigned i) const
- void `put` (const char *vname, const char *ws="base") const
- void `put` (const char *file, int line, const char *vname, const char *ws="base") const

Public Attributes

- **T data**

The documentation for this class was generated from the following files:

- wbbitsset.hh
- wbbitsset.cc

4.6 blockSpaceQS< TQ, TD > Class Template Reference

The documentation for this class was generated from the following file:

- clebsch.hh

4.7 Wb::cc_quad Class Reference**Public Member Functions**

- **cc_quad** (long double l=0)
- **cc_quad sqrt** () const
- **cc_quad fabs** () const
- **cc_quad abs** () const
- **cc_quad operator+** (const **cc_quad** &b) const
- **cc_quad operator-** (const **cc_quad** &b) const
- **cc_quad operator*** (const **cc_quad** &b) const
- **cc_quad operator/** (const **cc_quad** &b) const
- **cc_quad operator+=** (const **cc_quad** &b)
- **cc_quad operator-=** (const **cc_quad** &b)
- **cc_quad operator*=** (const **cc_quad** &b)
- **cc_quad operator/=** (const **cc_quad** &b)

Public Attributes

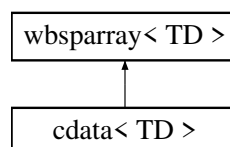
- long double **f**

The documentation for this class was generated from the following file:

- wbmpfr.hh

4.8 cdata< TD > Class Template Reference

Inheritance diagram for cdata< TD >:



Public Member Functions

- [cdata](#) & [init](#) (SPIDX_T l=0)
- [cdata](#) & [init](#) (SPIDX_T d1, SPIDX_T d2, SPIDX_T d, unsigned m=1)
- [cdata](#) & [init](#) (const [wbvector](#)< SPIDX_T > &s)
- [cdata](#) & [init](#) (const [cdata](#) &c)
- template<class TQ >
[cdata](#) & [init](#) (const [CRef](#)< TQ > &r, unsigned l, char full=1)
- template<class T2 >
[cdata](#) & [init](#) (const [wbsparray](#)< T2 > &c)
- [cdata](#) & [init](#) (const char *f, int l, const mxArray *a, unsigned k, const [QType](#) &q, unsigned r=-1)
- [cdata](#) & [operator=](#) (const [cdata](#) &c)
- [cdata](#) & [initScalar](#) (double x=1, unsigned r=-1)
- [cdata](#) & [initIdentity](#) (const [QType](#) &q, SPIDX_T d, TD dval=1)
- [cdata](#) & [initIdentity](#) (const [wbvector](#)< SPIDX_T > &s)
- bool [isScalar](#) () const
- unsigned [len](#) () const
- unsigned [getOM](#) (const char *f=NULL, int l=0, const char *istr=NULL) const
- [cdata](#) & [Conj](#) ()
- [cdata](#) & [permute](#) ([cdata](#) &b, const [wbperm](#) &p0, char iflag=0) const
- [cdata](#) & [Permute](#) (const [wbperm](#) &p0, char iflag=0)
- [cdata](#) & [Kron](#) (const [cdata](#) &b)
- TD [contract](#) (const char *f, int l, const [ctrlidx](#) &ica, const [cdata](#) &b, const [ctrlidx](#) &icb, [cdata](#) &c, [cdata](#) *Cx=NULL, [wbperm](#) *P=NULL, char normalize=1) const
- double [SkipTiny](#) (const char *f=NULL, int l=0)
- TD [NormSignC](#) (const char *f=NULL, int l=0, unsigned m=0, TD eps=CG_EPS1, TD eps2=CG_EPS2)
- void [info](#) () const
- template<class TQ >
[cdata](#)< TD > & [init](#) (const [CRef](#)< TQ > &r, unsigned k, char full)

Public Member Functions inherited from [wbsparray](#)< TD >

- [wbsparray](#) (SPIDX_T d1)
- [wbsparray](#) (SPIDX_T d1, SPIDX_T d2)
- [wbsparray](#) (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- [wbsparray](#) (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- [wbsparray](#) (const [wbvector](#)< SPIDX_T > &d, SPIDX_T l=0)
- [wbsparray](#) (char *sidx, SPIDX_T l=0)
- [wbsparray](#) (const [wbsparray](#) &a)
- [wbsparray](#) (const char *f, int l, const mxArray *a)
- [wbsparray](#) (const mxArray *a)
- [wbsparray](#) & [init](#) (const char *f, int l, const [wbarray](#)< TD > &a, TD eps=0)
- template<class TA >
[wbarray](#)< TA > & [toFull](#) ([wbarray](#)< TA > &a) const
- template<class TA >
[wbvector](#)< TA > & [toFull](#) ([wbvector](#)< TA > &a) const
- template<class DB >
[wbsparray](#) & [init](#) (const [wbsparray](#)< DB > &a)
- SPIDX_T [catRecs](#) (SPIDX_T i0, const [wbsparray](#) &a, SPIDX_T id)
- [wbsparray](#) & [init](#) (const [wbvector](#)< SPIDX_T > &s0, SPIDX_T l=0)
- template<class I2 >
[wbsparray](#) & [init](#) (const [wbvector](#)< I2 > &s_, SPIDX_T l=0)
- [wbsparray](#) & [init](#) ()
- [wbsparray](#) & [init](#) (SPIDX_T d1)

- [wbsparray](#) & **init** (SPIDX_T d1, SPIDX_T d2)
- [wbsparray](#) & **init** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- [wbsparray](#) & **init** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- [wbsparray](#) & **initz** ()
- [wbsparray](#) & **initz** (SPIDX_T d1, SPIDX_T n)
- [wbsparray](#) & **initz** (SPIDX_T d1, SPIDX_T d2, SPIDX_T n)
- [wbsparray](#) & **initz** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T n)
- [wbsparray](#) & **init_rzd** (SPIDX_T r, SPIDX_T n)
- [wbsparray](#) & **init_nnz** (SPIDX_T n=0, char kflag=0)
- [wbsparray](#) & **setRand** (double x=1, char sflag=0)
- [wbsparray](#) & **setRand_data** (double x=1, char sflag=0)
- template<class DB >
 [wbsparray](#) & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const [wbsparray](#)< DB > &B)
- template<class DB >
 [wbsparray](#) & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const [wbsparray](#)< DB > &B)
- template<class DB >
 [wbsparray](#) & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const [wbsparray](#)< DB > &B)
- [wbsparray](#) & **initEmpty** (const [wbsparray](#) &a)
- [wbsparray](#) & **init** (const char *F, int L, const mxArray *a, unsigned k=0)
- [wbsparray](#) & **init2ref** (const [wbsparray](#) &a)
- [wbsparray](#) & **unRef** ()
- [wbsparray](#) & **initCAT** (const char *F, int L, [wbvector](#)< [wbsparray](#) * > &X, [wbvector](#)< SPIDX_T > *i0=NULL, char del=0)
- [wbsparray](#) & **initCAT** (const char *F, int L, [wbvector](#)< [wbsparray](#) > &X, const [wbvector](#)< SPIDX_T > *i0=NULL)
- [wbsparray](#) & **initCAT** (const char *F, int L, const [wbvector](#)< const [wbsparray](#) * > &X, unsigned dim)
- [wbsparray](#) & **initCAT** (const char *F, int L, const [wbvector](#)< [wbsparray](#) > &X, unsigned dim)
- [wbsparray](#) & **Cat** (const char *F, int L, const [wbsparray](#) &B, unsigned dim)
- [wbsparray](#) & **init_kron** (const [wbvector](#)< SPIDX_T > &sa, const [wbvector](#)< SPIDX_T > &sb, SPIDX_T nab, char kflag)
- void **setSize_kron** (const char *F, int L, const [wbvector](#)< SPIDX_T > &sa, const [wbvector](#)< SPIDX_T > &sb, char kflag=0)
- void **setSize_kron** (const char *F, int L, char kflag=0)
- [wbsparray](#) & **setRec** (SPIDX_T i, SPIDX_T j)
- template<class T2 >
 [wbsparray](#) & **setRec** (SPIDX_T i, SPIDX_T i1, SPIDX_T i2, const T2 &x)
- [wbsparray](#) & **setRecP** (SPIDX_T i, const SPIDX_T *I, const TD &x)
- [wbsparray](#) & **setRec_kron** (SPIDX_T I, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2, unsigned kflag=0)
- [wbsparray](#) & **Resize** (SPIDX_T I)
- [wbsparray](#) & **REDSIZE_NNZ** (SPIDX_T I)
- [wbsparray](#) & **Resize** (const [wbvector](#)< SPIDX_T > &S)
- [wbsparray](#) & **operator=** (const [wbsparray](#) &a)
- template<class DB >
 [wbsparray](#) & **operator=** (const [wbsparray](#)< DB > &a)
- [wbsparray](#) & **save2** ([wbsparray](#) &a, char ref=0)
- SPIDX_T **sub2ind** (SPIDX_T k) const
- void **ind2sub** (SPIDX_T k, SPIDX_T *I) const
- void **ind2sub** (SPIDX_T k, [wbindex](#) &I) const
- [wbsparray](#)< TD > & **make2D** (const [wbindex](#) &ic, [wbsparray](#)< TD > &B, char pos=1) const
- [sparseIndex2D](#)< SPIDX_T > & **toIndex2D** (const [wbindex](#) &ic, [sparseIndex2D](#)< SPIDX_T > &a, char pos, char use_rmaj=0) const
- [wbsparray](#) & **initDiag** (SPIDX_T d)
- [wbsparray](#) & **initDiag** (const [wbvector](#)< TD > &dd)
- [wbsparray](#) & **initDiag2** (SPIDX_T d, TD dval=1)

- bool **isDiagMatrix** (TD eps=1E-14) const
- bool **isDiag** (const char *F=0, int L=0) const
- [wbvector](#)< TD > & **getDiag** (const char *F, int L, [wbvector](#)< TD > &dd) const
- TD **getDiag** (const char *F, int L, SPIDX_T k) const
- [wbsparray](#) & **getDiag** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **diag2reg** (const char *F=0, int L=0, unsigned r=-1)
- [wbsparray](#) & **diag2reg** (const char *F, int L, [wbsparray](#) &A, unsigned r=-1) const
- [wbsparray](#) & **toDiag** (const char *F=0, int L=0)
- int **toDiagSafe** (const char *F, int L, TD eps=1E-14)
- [wbsparray](#) & **initIdentity** (SPIDX_T d, TD dval=1)
- [wbsparray](#) & **initIdentity** (SPIDX_T d1, SPIDX_T d2, sSPIDX_T m=0, TD dval=1)
- [wbsparray](#) & **initIdentity** (const [wbvector](#)< SPIDX_T > &S0, int m=-1, sSPIDX_T k=0, char fflag='f')
- [wbsparray](#) & **initIdentityB3** (SPIDX_T d1, SPIDX_T d2, SPIDX_T D, sSPIDX_T i0=0, char fflag='f')
- unsigned **rank** (const char *F=NULL, int L=0) const
- SPIDX_T **dim** () const
- SPIDX_T **dim** (unsigned k) const
- SPIDX_T **dim0** (unsigned k) const
- SPIDX_T **matDim** (unsigned k=1)
- SPIDX_T **maxDim** (unsigned r=-1) const
- TD **norm2** (char tnorm=0) const
- [wbstring](#) **sizeStr** () const
- TD **SkipTinyCols** (TD eps=1E-14)
- TD **SkipTiny** (TD eps=1E-14)
- bool **isVector** (const char *F=0, int L=0) const
- bool **isMatrix** (const char *F=0, int L=0) const
- bool **isRank** (unsigned r) const
- bool **isSMatrix** (const char *F=0, int L=0, SPIDX_T *d=NULL) const
- bool **isSMatrix** (const char *F, int L, SPIDX_T d) const
- bool **isEmpty** () const
- bool **isEmpty_** () const
- **operator bool** () const
- bool **operator!** () const
- bool **isZero** (TD eps=0, char flag=0) const
- SPIDX_T **nnz** () const
- SPIDX_T **nnz** (const TD eps1, const TD eps2=1E9) const
- SPIDX_T **nnz_diag** () const
- SPIDX_T **numel** () const
- TD **norm** (char tnorm=0) const
- template<class T2 >
 [wbvector](#)< T2 > & **norm2vec** (unsigned k, [wbvector](#)< T2 > &A, char tnorm=0) const
- void **swap** ([wbsparray](#) &B)
- [wbsparray](#) & **TimesEI** (const [wbsparray](#) &B, char tnorm=0)
- TD **dotProd** (const char *F, int L, const [wbsparray](#) &B, char tnorm=0) const
- TD **froNorm2** (const [wbsparray](#) &B) const
- double **normDiff2** (const [wbsparray](#) &B) const
- double **normDiff** (const [wbsparray](#) &B) const
- TD **Normalize** (char tnorm=0, char qflag=0)
- TD **NormalizeCol** (SPIDX_T k, char tnorm=0, char qflag=0)
- [wbsparray](#) & **QRdecomp** (const char *F, int L, [wbsparray](#) &Q, char posR=0, TD eps=1E-14)
- [wbsparray](#) & **OrthoNormalizeColsQR** (const char *F=NULL, int L=0, TD eps=1E-14)
- [wbsparray](#) & **OrthoNormalizeColsGS** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, TD eps=1E-14, unsigned np=1)
- [wbsparray](#) & **setCol** (const char *F, int L, SPIDX_T k, const [wbsparray](#) &v)
- [wbsparray](#) & **setCol** (SPIDX_T k, const [wbsparray](#) &v)
- double **sparsity** () const

- int **checkTrailingSingletons** (const char *F=0, int L=0, unsigned *r=NULL) const
- int **SkipTrailingSingletons** (const char *F=0, int L=0, unsigned r=-1)
- int **skipTrailingSingletons** (const char *F, int L, [wbsparray](#) &c, unsigned r=-1) const
- bool **hasSingletons** (const [wbindex](#) &l) const
- [wbsparray](#) & **skipSingletons** (const char *F, int L, const [wbindex](#) &l, [wbsparray](#) &A) const
- [wbsparray](#) & **AddTrailingSingletons** (const char *F, int L, unsigned r)
- [wbsparray](#) & **initScalar** (const TD x=0, unsigned r=-1)
- char **isScalar** (const char *F=0, int L=0, char zflag=0) const
- [wbsparray](#) & **toScalar** (const char *F=0, int L=0, unsigned r=-1)
- TD **getScalar** (const char *F=0, int L=0) const
- char **checkSize** (const char *F=0, int L=0, const char *fct=0, const char *istr=0) const
- char **checkSize2** (const char *F, int L, const [wbsparray](#) &B) const
- void **check_IDX_range** (const char *F=0, int L=0) const
- bool **isProptold** () const
- bool **isProptold** (TD &q, TD eps=1E-14) const
- bool **isIdentityMatrix** (TD eps=1E-14) const
- char **isIdentity** (TD eps=1E-14) const
- char **isIdentity** (const [wbperm](#) &P, TD *dval=NULL, TD eps=1E-14) const
- bool **hasSize** (SPIDX_T d1) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const
- template<class T >
[wbvector](#)< T > & **getSize** ([wbvector](#)< T > &S) const
- char **fitsSize** (const [wbsparray](#) &, char strict=0) const
- char **sameSize** (const [wbsparray](#) &, char strict=0) const
- char **sameSizeUp2Singletons** (const [wbvector](#)< SPIDX_T > &S, char strict=0) const
- char **matchWithSingletons** (const char *F, int L, const [wbvector](#)< SPIDX_T > &S, [wbvector](#)< int > &M) const
- char **sameUptoFac** (const [wbsparray](#) &B, TD *fac=NULL, char lex=-1, TD eps=1E-12) const
- bool **operator==** (const [wbsparray](#) &B) const
- bool **operator!=** (const [wbsparray](#) &B) const
- [wbsparray](#) & **Plus** (const char *F, int L, const [wbsparray](#) &B, TD bfac=1, TD afac=1, TD eps=0)
- TD **sum** () const
- TD **aMin** () const
- TD **aMax** () const
- SPIDX_T **findRecSortedP** (SPIDX_T *idx, SPIDX_T n, char lex=-1)
- [wbsparray](#) & **Eldiv** (const char *F, int L, const [wbvector](#)< TD > &x_)
- [wbsparray](#) & **operator+=** (const [wbsparray](#) &B)
- [wbsparray](#) & **operator-=** (const [wbsparray](#) &B)
- [wbsparray](#) & **Conj** ()
- [wbsparray](#) & **operator*=** (TD x)
- [wbsparray](#) & **operator/=** (TD x)
- const TD & **operator[]** (SPIDX_T i) const
- TD & **operator[]** (SPIDX_T i)
- TD **value** (const SPIDX_T *idx, SPIDX_T n, char lex=-1) const
- [wbstring](#) **info2Str** (const char *istr=0) const
- size_t **memSize** () const
- void **info** (const char *istr="") const
- void **info** (const char *F, int L, const char *="") const
- void **print** (const char *istr="", const char *fmt="") const
- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **printdata** (const char *istr="ans", const char *fmt="") const
- void **sperror_this** (const char *F, int L, const char *fct=0, const char *istr=0) const
- TD **trace** () const

- `template<class T2 >`
`wbvector< T2 > & trace` (unsigned k, `wbvector< T2 > &A`) const
- `bool isSym_aux` (const char *file, int line, const char *fct, const `wbsparray` &B, TD eps, TD *xref, const char symflag='s', const char lflag=1) const
- `bool isHConj` (const `wbsparray` &B, TD eps=1E-12, TD *xref=NULL) const
- `bool isHConj` (TD eps=1E-12, TD *xref=NULL) const
- `bool isAHerm` (const `wbsparray` &B, TD eps=1E-12, TD *xref=NULL) const
- `bool isAHerm` (TD eps=1E-12, TD *xref=NULL) const
- `wbsparray & Sort` (char lex=-1)
- `double Compress` (const char *F=0, int L=0, TD eps=0, char lex=-1, const `wbperm` *P=NULL)
- `bool isCompressed` (char lex=-1, char dir=0) const
- `int splitSparseCM` (const char *F, int L, `wbvector< wbsparray > &S`, TD eps=0, char ref=0, char tnorm=0)
- `wbsparray & reshape` (const `wbvector< SPIDX_T > &S`, `wbsparray` &A) const
- `wbsparray & Reshape` (const `wbvector< SPIDX_T > &S`)
- `wbsparray & Reshape` (SPIDX_T s1, SPIDX_T s2)
- `wbsparray & Reshape` (SPIDX_T s1, SPIDX_T s2, SPIDX_T s3)
- `wbvector< TD > & getCol` (SPIDX_T k, `wbvector< TD > &v`) const
- `wbvector< TD > & getRow` (SPIDX_T k, `wbvector< TD > &v`) const
- `wbsparray & getBlock` (const char *F, int L, SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2, `wbsparray< TD > &B`) const
- `wbsparray & getCol` (SPIDX_T k, `wbsparray< TD > &b`)
- `wbsparray & splitBlock` (const char *F, int L, sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2, `wbsparray< TD > &B`)
- `wbsparray & SplitQR` (sSPIDX_T k, sSPIDX_T k_, `wbsparray< TD > &X`)
- `TD getHouseholderVec` (const char *F, int L, SPIDX_T k, SPIDX_T j, `wbsparray< TD > &b`, TD eps2=0) const
- `wbsparray & IDX_Shift` (sSPIDX_T i1, sSPIDX_T i2)
- `wbsparray< TD > & LProject1` (const `wbsparray< TD > &b`, `wbsparray< TD > &X`, TD xfac=1, char tnorm=0) const
- `SPIDX_T LContractVec_omp` (const `wbsparray< TD > &b`, `wbvector< TD > &x`, char tnorm) const
- `wbsparray & splitLast` (SPIDX_T k, `wbsparray` &a) const
- `wbsparray & select0` (const WBINDEX &, unsigned dim, `wbsparray` &) const
- `wbsparray & Select0` (const WBINDEX &l, unsigned dim)
- `wbsparray & Permute` (const char *s, char iflag=0)
- `wbsparray & Permute` (const `wbperm` &, char iflag=0)
- `wbsparray & permute` (const char *s, `wbsparray` &, char iflag=0) const
- `wbsparray & permute` (const `wbperm` &, `wbsparray` &, char iflag=0) const
- `wbsparray & transpose` (const char *F, int L, `wbsparray` &A) const
- `wbsparray & Transpose` (const char *F=0, int L=0)
- `wbsparray & htranspose` (const char *F, int L, `wbsparray` &A) const
- `wbsparray & hTranspose` (const char *F=0, int L=0)
- `wbsparray & ColPermute` (const `wbperm` &P, char iflag=0)
- `wbsparray & MatPermute` (const `wbperm` &P, char iflag=0)
- `wbsparray & mtimes` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char sflag=0, const TD eps=0)
- `char contract_scalar` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal=`wbperm`(), const TD &afac=1., const TD &cfac=0.) const
- `wbsparray & contract_diag_diag` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac) const
- `char contract_check_2full` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac, char flag, TD eps, SPIDX_T *nnz_← C=NULL) const
- `wbsparray & contract` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal=`wbperm`(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const

- **wbarray** & **contract** (const char *F, int L, const char *ica, const **wbarray** &B, const char *icb, **wbarray** &C, const **wbperm** &pfinal=**wbperm**(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- **wbarray**< TD > & **contract** (const char *F, int L, const **ctrlidx** &ica, const **wbarray** &B, const **ctrlidx** &icb, **wbarray**< TD > &C, const **wbperm** &pfinal, const TD &afac, const TD &cfac) const
- **wbarray** & **contract** (const char *F, int L, unsigned i1, const **wbvector**< TD > &B, **wbarray** &C) const
- **wbarray** & **comm** (const char *F, int L, const **wbarray** &B, **wbarray** &C, char aflag='N', char bflag='N') const
- **wbarray** & **tensorProdX** (const char *F, int L, const **wbarray** &B, **wbarray** &C, char conjA=0, const TD &afac=1, char conjB=0, const TD &cfac=0, const **wbperm** *Pfinal=NULL) const
- **wbarray** & **tensorProd** (const char *F, int L, const **wbarray** &B, **wbarray** &C, char aflag='N', char bflag='N', char kflag=0) const
- **wbarray** & **Kron** (const **wbarray** &B, char aflag='N', char bflag='N', char kflag='k')
- **wbarray** & **kron** (const char *F, int L, const **wbarray** &B, **wbarray** &C, char aflag='N', char bflag='N', char kflag='k') const
- mxArray * **toMxSp** () const
- mxArray * **toMx** (int m=-99) const
- mxArray * **IDtoMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *a, unsigned i, int ma=-99) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *a, unsigned i) const
- mxArray * **toMx0** (int m=-99) const
- mxArray * **mxCreateStructX** (unsigned m, unsigned n, int ma=-99) const
- void **add2MxStructX** (mxArray *a, unsigned i, int ma=-99) const
- mxArray * **toMX** () const
- mxArray * **mxCreateSTRUCT** (unsigned m, unsigned n) const
- mxArray * **add2MxSTRUCT** (mxArray *a, unsigned i) const
- void **put** (const char *vname, unsigned ma=-99, const char *ws="caller") const
- void **put0** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **put** (const char *F, int L, const char *vname, unsigned ma=-99, const char *ws="caller", char flag=0) const
- template<class T2 >
wbvector< T2 > & **toFull** (**wbvector**< T2 > &A) const
- template<class DB >
wbarray< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const **wbarray**< DB > &B)
- template<class DB >
wbarray< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const **wbarray**< DB > &B)
- template<class DB >
wbarray< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const **wbarray**< DB > &B)

Additional Inherited Members

Public Attributes inherited from **wbarray**< TD >

- **wbvector**< SPIDX_T > **SIZE**
- **wbMatrix**< SPIDX_T > **IDX**
- **wbvector**< TD > **D**
- char **isref**

Protected Member Functions inherited from [wbsparray< TD >](#)

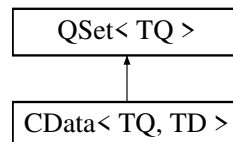
- unsigned **get2DSize** (const char *F, int L, SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const
- unsigned **get2DIndex** ([wbMatrix](#)< SPIDX_T > &IJ, SPIDX_T &d1, SPIDX_T &d2, int m=-1) const

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_aux.cc
- clebsch_io.cc

4.9 CData< TQ, TD > Class Template Reference

Inheritance diagram for CData< TQ, TD >:



Public Member Functions

- **CData** (const [CData](#) &C)
- **CData** (const [QSet](#)< TQ > &Q)
- **CData** (const [CRef](#)< TQ > &R, unsigned I)
- **CData** & **init** ()
- **CData** & **init** (const [QSet](#)< TQ > &Q, bool full=true)
- **CData** & **init** (const [CRef](#)< TQ > &R, unsigned I)
- **CData** & **init** (const [CData](#) &B)
- template<class T2 >
 CData & **init** (const [CData](#)< TQ, T2 > &B)
- **CData** & **operator=** (const [CData](#) &B)
- **CData** & **operator=** (const [QSet](#)< TQ > &Q)
- **CData** & **save2** ([CData](#) &B)
- **CData** & **initAbelian** (const [QSet](#)< TQ > &Q)
- **CData** & **initScalar** (const [QSet](#)< TQ > &Q)
- **CData** & **init3** (const [QType](#) &t_, SPIDX_T D1, SPIDX_T D2, SPIDX_T D, unsigned M=1)
- **CData** & **init3** (const [QType](#) &t_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const [qset](#)< TQ > &J, unsigned M=1)
- **CData** & **init3_abelian** (const [QType](#) &t_, const TQ &J1, const TQ &J2, const TQ &J)
- int **Reflnit** (const char *F, int L, const [CData](#)< TQ, TD > &B, char lflag=0, char bare=0)
- **CData** & **Reflnit_auxtr** (const char *F, int L, const [wbvector](#)< SPIDX_T > &S, const [wbvector](#)< RTD > &cgt)
- int **Reduce2Ref** (const char *F=0, int L=0, char force=0)
- int **LoadRef** (const char *F=0, int L=0, char force=1)
- int **Load_CRef** (const char *F, int L, [CData](#) &Cr, const char *cgd_file) const
- **CData** & **initX3** (const char *F, int L, const [QSet](#)< TQ > &Q, [wbvector](#)< char > &cflags, [wbperm](#) &P, unsigned loadRC=0, unsigned char flag=0)
- int **init_mxCRef** (const char *F, int L, const mxArray *S, unsigned k)
- TQ * **qptr** (unsigned k) const

- bool **olderThan** (const [CData](#) &B) const
- bool **operator!=** (const [CData](#) &B) const
- bool **operator==** (const [CData](#) &B) const
- int **gotDiff** (const [CData](#) &B, char lflag=0) const
- bool **sameAs** (const [CData](#) &B, char lflag=0) const
- bool **operator!=** (const [QSet](#)< TQ > &B) const
- bool **operator==** (const [QSet](#)< TQ > &B) const
- bool **operator!=** (const [cgdStatus](#) &b) const
- bool **operator==** (const [cgdStatus](#) &b) const
- bool **isEmpty** (char check=3) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **sizeable** (unsigned Dmin=1024) const
- bool **isScalar** (char dflag=0) const
- TD **getScalar** (const char *F=NULL, int L=0) const
- unsigned **rank** (const char *F=NULL, int L=0) const
- unsigned **rankS** () const
- bool **isrank** (unsigned r, unsigned *r_=NULL) const
- template<class T >
 [wbvector](#)< T > & **getSize** ([wbvector](#)< T > &S, char bare=0, const [wbperm](#) *cgp=NULL) const
- SPIDX_T **dim** () const
- bool **isReflnit** (char check=1) const
- bool **checkValidStat** (const char *F=0, int L=0) const
- int **checkSameStat** (const char *F, int L, const [CData](#) &B, char lflag=1) const
- bool **sameType** (const [CData](#) &S, const char *F=NULL, int L=0) const
- int **cmpOM** (const [CData](#) &B) const
- template<class DB >
 bool **sameSizeR** (const [CData](#)< TQ, DB > &B, unsigned *r=NULL, const char *F=NULL, int L=0) const
- template<class DB >
 bool **sameSizeR** (const [cdata](#)< DB > &B, unsigned *r=NULL, const char *F=NULL, int L=0) const
- template<class T2 >
 int **sameSizeR** (const char *F, int L, const [wbvector](#)< T2 > &S) const
- bool **isSymmetric** () const
- [CData](#) & **initOM** (const char *F, int L)
- unsigned **getOM** (const char *F=NULL, int L=0) const
- unsigned **gotOM** (const char *F=NULL, int L=0) const
- unsigned **checkOM** (const char *F=NULL, int L=0) const
- int **gotFullOM** (const char *F=NULL, int L=0) const
- int **completeOM_DegQ** (const char *F, int L, const [wbperm](#) &p, unsigned level=0)
- [CData](#) & **Project** (const char *F, int L, [cdata](#)< TD > X, [wbvector](#)< TD > &w)
- double **normDiff** (const char *F, int L, const [CData](#) &S, double eps=1E14) const
- double **norm2** (unsigned k=-1) const
- [wbvector](#)< RTD > & **trace** (const char *F, int L, [wbvector](#)< RTD > &cgt) const
- [wbvector](#)< RTD > **trace** (const char *F=NULL, int L=0) const
- [CData](#) & **trace** (const char *F, int L, [ctrlIdx](#) &i1, [ctrlIdx](#) &i2, [CData](#)< TQ, TD > &X) const
- [CData](#) & **AddMultiplicity** (const char *F, int L, const [cdata](#)< TD > &c)
- [CData](#) & **AddMultiplicity** (const char *F, int L, const [CData](#) &C)
- [cdata](#)< TD > & **getMultiplicity** (unsigned im, [cdata](#)< TD > &c) const
- bool **isBasicCG** (const char *F=NULL, int L=0) const
- SPIDX_T **getBasicCGSize** (unsigned *m=NULL) const
- [wbsparray](#)< TD > & **getBasicCG** (unsigned k, [wbsparray](#)< TD > &a) const
- int **getCG_set** (const char *F, int L, [wbIndex](#) &Idx, [wbsparray](#)< TD > &a) const
- [CData](#) & **reduceto1JSymbol** (const char *F, int L, [CData](#)< TQ, TD > &C) const
- int **checkConsistency** (const char *F=NULL, int L=0) const
- int **checkNormSign** (const char *F=NULL, int L=0, char xflag=0) const
- int **checkQ** (const char *F, int L, const [QSet](#)< TQ > &Q, const char *istr=NULL) const

- bool **checkAdditivityZ** (const char *F, int L, const [wbMatrix](#)< double > &Z1, const [wbMatrix](#)< double > &Z2, const [wbMatrix](#)< double > &Z, TD eps=CG_EPS2) const
- double **SkipTiny** (const char *F=NULL, int L=0)
- [CData](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [CData](#) & **permute** ([CData](#) &B, const [wbperm](#) &P, char iflag=0) const
- [CData](#) & **Conj** ()
- [CData](#) & **Conj** (unsigned i)
- size_t **memSize** () const
- void **info** (const char *istr=NULL, const char *F=NULL, int L=0) const
- [wbstring](#) **dStr** () const
- [wbstring](#) **qStr** () const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (char vflag) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **sizeStr** (char xflag) const
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tst=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tst=0) const
- void **put** (const char *vname, const char *ws="caller") const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **setuser_BUF** (unsigned char c=1)
- void **setuser_BUF** (unsigned char c, unsigned l)
- void **setuser_BUF_ref** (unsigned char c=1)
- bool **gotuser_BUF** () const
- bool **gotuser_BUF_ref** () const

Public Member Functions inherited from [QSet](#)< TQ >

- [QSet](#) (const [QSet](#) &B)
- [QSet](#) (const [QType](#) &t_, const [qset](#)< TQ > &qset_, unsigned iout=0, char isref=0)
- [QSet](#) (const [QType](#) &t_, const [iTags](#) &it, unsigned r=-1)
- [QSet](#) (const [CRef](#)< TQ > &A)
- [QSet](#) (const char *F, int L, const mxArray *a)
- [QSet](#) (const mxArray *a)
- [QSet](#) (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *P=NULL)
- [QSet](#) (const char *F, int L, const char *s)
- [QSet](#) & **init** (const [CRef](#)< TQ > &A)
- [QSet](#) & **init** (const char *F, int L, const mxArray *a, unsigned k=0)
- [QSet](#) & **init** (const [QType](#) t_=QTYPE_UNKNOWN)
- [QSet](#) & **init** (const [QSet](#) &B)
- [QSet](#) & **init2ref** (const [QSet](#) &B)
- [QSet](#) & **init** (const [QType](#) &t_, const TQ *q, const char *qd)
- [QSet](#) & **init** (const [QType](#) &t_, const [qset](#)< gTQ > &qset_, unsigned iout=0, char isref=0)
- [QSet](#) & **init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, unsigned iout=0)
- [QSet](#) & **init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, const [iTags](#) &it)
- [QSet](#) & **init1J** (const [QType](#) &t_, const TQ *q, char iflag=0)
- [QSet](#) & **init2** (const [QType](#) &t_, const TQ *q=NULL)
- [QSet](#) & **init2** (const [QType](#) &t_, const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const char *qds="++")
- [QSet](#) & **init1** (const [QType](#) &t_, TQ *q=NULL)
- [QSet](#) & **init3** (const [QType](#) &t_)
- [QSet](#) & **init3** (const [QType](#) &t_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const [qset](#)< TQ > &J)

- **QSet & init3** (const [QType](#) &t_, const TQ &J1, const TQ &J2, const TQ &J)
- **QSet & init** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlidx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlidx](#) &icb, [wbperm](#) *Pcgd=NULL)
- **QSet & init_str** (const char *F, int L, const char *s)
- **QSet & Conj** ()
- **QSet & Conj** (unsigned i)
- unsigned **rank** (const char *F=NULL, int L=0) const
- **QSet & permute** ([QSet](#) &B, const [wbperm](#) &P, char iflag=0) const
- **QSet & Permute** (const [wbperm](#) &P, char iflag=0)
- void **swap** ([QSet](#) &X)
- **QSet & save2** ([QSet](#) &X)
- **QSet & operator=** (const [QSet](#) &B)
- bool **operator==** (const [QSet](#) &B) const
- bool **operator!=** (const [QSet](#) &B) const
- char **cmp** (const [QSet](#)< TQ > &B) const
- bool **operator<** (const [QSet](#)< TQ > &B) const
- **operator bool** () const
- bool **operator!** () const
- bool **isEmpty** (char check=3) const
- bool **gotRCData** () const
- int **isZero** () const
- unsigned **isStd3** () const
- unsigned **checkStd3** () const
- int **init2opt3** (const [QSet](#)< TQ > &Q, [wbperm](#) &P, [wbvector](#)< char > &cflags)
- **QSet & Convert** (const char *s, [wbvector](#)< char > &cflags)
- [wbstring](#) **sizeStr** () const
- bool **isScalar** () const
- bool **is1JSymbol** () const
- bool **isAbelian** () const
- int **permitsOM** () const
- int **checkQ_abelian** (const char *F, int L) const
- int **checkQ_SU2** (const char *F, int L) const
- **QSet & Sort** ([wbperm](#) *cgp=NULL, char *conj=NULL, char iflag=0)
- bool **isSorted** () const
- size_t **memSize** () const
- [wbstring](#) **QStrS** (const [wbperm](#) *cgp=NULL, char sep='|') const
- [wbstring](#) **QStr** () const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (const char *istr) const
- [wbstring](#) **toTag** () const
- mxArray * **toMx** () const

Public Attributes

- [cdata](#)< TD > **cgd**
- [cgdStatus](#) **cstat**

Public Attributes inherited from [QSet< TQ >](#)

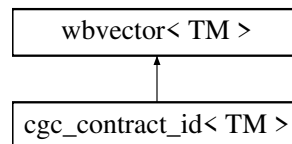
- [QType](#) t
- [qset< TQ >](#) qs
- [QDir](#) qdir

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_aux.cc
- clebsch_io.cc

4.10 cgc_contract_id< TM > Class Template Reference

Inheritance diagram for cgc_contract_id< TM >:

**Public Member Functions**

- `template<class TQ , class TD >`
cgc_contract_id (const [CData](#)< TQ, TD > &A, const [ctrlIdx](#) &ica, const [CData](#)< TQ, TD > &B, const [ctrlIdx](#) &icb)
- `template<class TQ , class TD >`
void extract ([CData](#)< TQ, TD > &A, [ctrlIdx](#) &ica, [CData](#)< TQ, TD > &B, [ctrlIdx](#) &icb, char xflag=1) const
- **size_t memSize** () const
- **void wblog_** (const char *F=0, int L=0, const char *istr=0) const
- **wbstring toStr** (char vflag=7) const

Public Member Functions inherited from [wbvector< TM >](#)

- **wbvector** (size_t l, TM x)
- **wbvector** (size_t l, const TM *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const TM *d0)
- [wbvector](#) & **init** (size_t l, const TM *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< TM > & **init** (const [wbsparray](#)< TM > &S)
- [wbvector](#) & **init** (size_t l1, const TM *d1, size_t l2, const TM *d2)

- `int init (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)`
- `int init (const mxArray *a, const char *istr=NULL, char check_type=1)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `wbvector & initp (size_t l, const TM *d0, size_t n, char init=1)`
- `wbvector & init2val (size_t l, const TM &x)`
- `wbvector & initStride (size_t l, const TM *d0, size_t stride)`
- `wbvector & init_stride (const TM *d0, size_t n, size_t m, size_t D0)`
- `wbvector & init_data (const TM *d0)`
- `wbvector & init_data (const T0 *d0)`
- `wbvector & init_data (const char *F, int L, const T0 *d0)`
- `wbvector< TM > & initT (const wbsparray< T2 > &S)`
- `wbvector< TM > & initT (size_t l, const T2 *v)`
- `wbvector< TM > & initT (const wbvector< T2 > &v)`
- `wbvector< TM > & initT (const char *F, int L, const wbvector< T2 > &v)`
- `wbvector< TM > & initT (const wbsparray< TD > &S)`
- `wbvector< TM > & initT_ (const wbvector< T2 > &v)`
- `wbvector & init2ref (const wbvector &v)`
- `wbvector & init2ref (size_t l, const TM *d0)`
- `wbvector & unRef ()`
- `int init_Struct (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)`
- `int init_base (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)`
- `int init_mpfr (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)`
- `void initDef (size_t n)`
- `void initDef (const wbvector &v)`
- `wbvector & operator= (const wbvector &v)`
- `wbvector< TM > & operator= (const wbvector< TB > &v)`
- `wbvector & set (const TM &x)`
- `wbvector & set (const wbvector< size_t > &l, const TM &x)`
- `void set (size_t offset, const TM *x, size_t n)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `wbvector & setMin (const TM &x)`
- `wbvector & setMax (const TM &x)`
- `wbvector & add (const wbvector< size_t > &l, const TM &x)`
- `wbvector & add (const TM *d)`
- `wbvector & Set (const wbvector< size_t > &l, const wbvector &v)`
- `void Sort (wbperm &p, char dir=+1)`
- `void Sort (char dir=+1)`
- `void sort (wbvector &v2, wbperm &p) const`
- `void sort (wbvector &v2) const`
- `wbperm & pSort (wbperm &p) const`
- `bool isSorted () const`
- `bool isSortedU () const`
- `bool isUnique () const`
- `int contains (const TM &x) const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isFinite () const`
- `size_t find1 (const TM &x) const`
- `size_t find1 () const`
- `size_t count (const TM &x) const`
- `size_t icount (const TM &x) const`
- `size_t nnz () const`

- `size_t nnz` (TM eps) const
- `size_t numZeros` (TM eps=TM(1E-14)) const
- `size_t numEQ` (const TM &) const
- `size_t numGT` (const TM &) const
- `size_t numLT` (const TM &) const
- `wbindex & find` (const TM &x, `wbindex` &l, char iflag=0) const
- `wbindex & find` (const TM &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex & find` (`wbindex` &l) const
- `wbindex & find` (`wbindex` &l, `wbindex` &lx) const
- `wbvector< size_t > find` () const
- `wbindex & findGT` (const TM &x, `wbindex` &l, char iflag=0) const
- `wbindex & findGT` (const TM &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex & findzeros` (`wbindex` &l) const
- `wbindex & findRange` (const TM &x1, const TM &x2, `wbindex` &l, const char *w=NULL) const
- `int findClosestSorted` (TM r) const
- `int findClosest` (TM r) const
- `size_t findValues` (const `wbvector` &v, `wbindex` &l) const
- `wbvector & setp` (const TM *x)
- `void reset` ()
- `void cpy` (const TM *v)
- `void cpy` (const TM *x, `size_t` m)
- `wbvector & save2` (`wbvector` &v)
- `void swap` (`wbvector` &v)
- `wbvector & swap` (`size_t` i, `size_t` j)
- `wbvector & Collect` (TM &A1, TM &A2)
- `wbvector & Collect` (TM &A1, TM &A2, TM &A3)
- `wbvector & Skip` (`size_t`)
- `wbvector & Skip` (const WBINDEX &, char rflag=0)
- `wbvector & Move2_FE` (const WBINDEX &l, char FE_flag)
- `wbvector & Move2front` (const WBINDEX &l)
- `wbvector & Move2front` (`size_t` k)
- `wbvector & Move2end` (const WBINDEX &l)
- `wbvector & Move2end` (`size_t` k)
- `wbvector & mvLastTo` (`size_t` k)
- `wbvector & mvFirstTo` (`size_t` k)
- `bool checkSameLength` (const `wbvector` &a, const char *F=NULL, int L=0, const char *istr="") const
- `bool checkSameLength` (const `wbvector` &a, const `wbvector` &b, const char *F=NULL, int L=0, const char *istr="") const
- `void operator+=` (const T_ &a)
- `void operator+=` (const `wbvector` &b)
- `void operator-=` (const T_ &a)
- `void operator-=` (const `wbvector` &b)
- `wbvector & operator*=` (const T_ &a)
- `wbvector & operator/=` (const T_ &a)
- `wbvector operator*` (const T_ &a) const
- `wbvector operator+` (const T_ &a) const
- `wbvector operator+` (const `wbvector` &b) const
- `wbvector operator-` (const T_ &a) const
- `wbvector operator-` (const `wbvector` &b) const
- `wbvector & Plus` (const TM *b)
- `wbvector & Plus` (const TM *b, TM bfac)
- `wbvector & Plus` (const `wbvector` &b, TM bfac=1)
- `void plus` (const `wbvector` &b, `wbvector` &c, TM bfac=1, TM cfac=0) const
- `wbvector & times` (TM fac, `wbvector` &x) const
- `wbvector & RevertSigns` ()

- `int isum () const`
- `TM sum () const`
- `TM sum (size_t i1, const char *estr) const`
- `TM sum (size_t i1, size_t i2) const`
- `TM cumsum_ (wbvector &S, char xflag=0) const`
- `TM Cumsum (char zflag=0) const`
- `TM cumsum0prod (wbvector< T2 > &b, wbvector< TM > &cs, char xflag) const`
- `wbvector & Eldiv (const wbvector &v) const`
- `wbvector & tensorProd (const wbvector &v2, wbvector &vv) const`
- `TM scalarProd (const wbvector &v) const`
- `TM scalarProd (const TM *v) const`
- `TM norm () const`
- `TM norm2 () const`
- `double normDiff (const wbvector &v, TM fac=1) const`
- `TM normDiff2 (const wbvector &v, TM fac=1) const`
- `double normDiff2_ (const wbvector &v, double fac=1) const`
- `TM GSProject (const TM *v, char isnorm=0, char tflag=0) const`
- `wbvector & Normalize (const char *F=NULL, int L=0)`
- `wbvector & subtract (const TM *d)`
- `void getReal (wbvector< double > &R) const`
- `void getReal (wbvector< double > &R) const`
- `void getImag (wbvector< double > &I) const`
- `void getImag (wbvector< double > &I) const`
- `wbvector & Conj ()`
- `wbvector< wbcomplex > & Conj ()`
- `wbvector & applyFlag (char &flag, const TM &fac, wbvector &a)`
- `bool allEqual () const`
- `bool allEqual (const TM &a) const`
- `bool allZero () const`
- `bool allOnes () const`
- `bool allEqual2 (const TM &a, TM eps=1E-15) const`
- `bool allUnequal (const TM &a) const`
- `bool any () const`
- `bool anyUnequal (const TM &a) const`
- `bool anyUnequal (const wbvector< size_t > &I, const TM &x, size_t *k=NULL) const`
- `bool anyEqual (const TM &a, size_t *i1=NULL) const`
- `bool anyEqual (const wbvector< size_t > &I, const TM &x, size_t *k=NULL) const`
- `bool anyGT (const TM &a) const`
- `bool anyGE (const TM &a) const`
- `bool anyLT (const TM &a) const`
- `bool anyLE (const TM &a) const`
- `bool allGT (const TM &a) const`
- `bool allGE (const TM &a) const`
- `bool allLE (const TM &a) const`
- `bool allLT (const TM &a) const`
- `bool allIn (const TM &a1, const TM &a2) const`
- `bool anySmallVals (const TM eps=1E-15) const`
- `bool isEqual (const TM *v) const`
- `bool isEqual (const wbvector &v) const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool operator== (const wbvector &v) const`
- `bool operator!= (const wbvector &v) const`
- `bool isEqualR (const wbvector &v) const`

- bool **operator[^]** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const TM &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- TM & **el** (size_t i) const
- TM **elx** (size_t i, const TM &x) const
- const TM & **operator[]** (size_t i) const
- TM & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- TM & **last** () const
- TM & **last** (size_t &i) const
- TM & **last** (T1 &i_) const
- TM & **p2last** () const
- TM * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const TM *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const TM *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< TM > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< TM > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const TM &x)
- [wbvector](#) & **Append** (size_t d2, const TM *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const TM *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const TM *v) const
- int **recLessE** (const TM *v) const
- int **recCompare** (const TM *v) const
- TM **max** () const
- TM **max** (size_t *k) const
- TM **max** (unsigned &k) const
- TM **max_** (TM x) const
- TM **min** () const
- TM **min** (size_t *k) const
- TM **min** (unsigned &k) const
- TM **min_** (TM x) const

- TM **aMax** (size_t *k=NULL) const
- TM **aMax** (unsigned &k) const
- TM **aMin** (char zflag=0, size_t *k=NULL) const
- TM **maxneg** (size_t *k=NULL) const
- TM **minpos** (size_t *k=NULL) const
- TM **avg** () const
- TM **std** () const
- void **range** (TM &xmin, TM &xmax) const
- TM **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &l0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &l, TM *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &l)
- void **select** (const WBINDEX &l, TM *r) const
- **wbvector** & **select** (const WBINDEX &l, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &l) const
- **wbvector** & **Select** (const WBINDEX &l)
- **wbvector** & **BlockSelect** (const WBINDEX &l, const WBINDEX &D)
- **wbvector** & **getl** (size_t k, **wbvector** &v) const
- **wbvector** & **getl** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getl** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initl** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initl** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- TM **prod** () const
- TM **prod** (TM x) const
- TM **prod** (size_t i, size_t i2) const
- TM **prod** (size_t i, size_t i2, TM x) const
- TM **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (TM &p1, TM &p2, const TM &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *&a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const

- `mxArray * toMxP` (const char tflag=0) const
- `mxArray * toMx_base` (const char tflag=0) const
- `mxArray * toMx_base` (const char tflag) const
- `mxArray * toMx_base` (const char tflag) const
- `mxArray * toMx_base_d` (const char tflag=0) const
- `mxArray * toMx_offset` (const TM &offset, const char tflag=0) const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- void `add2MxStruct` (mxArray *S, unsigned i, char tflag=0) const
- void `add2MxStruct` (mxArray *S, const char *vname, char tflag=0) const
- `mxArray * mxCreateCell` (unsigned m, unsigned n) const
- void `add2MxCell` (mxArray *S, unsigned i, char tflag=0) const
- void `putg` (const char *vname="ans", char tflag=0) const
- void `putx` (const char *vname="ans", char tflag=0) const
- `wbstring toStr` (int n, const char *sep) const
- `wbstring toStr` () const
- `wbstring toStr` (int n, const char *sep) const
- `wbstring sizeStr` () const
- `wbstring toStrf` (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- `wbstring toStrf` (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from `wbvector< TM >`

- TM * `data`
- `size_t len`
- char `isref`

Protected Member Functions inherited from `wbvector< TM >`

- `wbvector` & `RENEW` (size_t n, const TM *d=NULL, size_t l=-1, char init=1)
- `wbvector` & `RENEW_VAL` (size_t n, const TM &x)

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_aux.cc`

4.11 cgdStatus Class Reference

Public Member Functions

- `cgdStatus` (CGD_TYPE t_)
- `cgdStatus` (double t_)
- `cgdStatus` (const char *F, int L, mxArray *a)
- `template<class TQ >`
`cgdStatus` (const CDATA_TQ &C)
- `cgdStatus` & `init` ()
- `cgdStatus` & `init` (const CGD_TYPE &t_)

- [cgdStatus](#) & **init0** (const CGD_TYPE &t_)
- [cgdStatus](#) & **update_m** ()
- [cgdStatus](#) & **update_m** (const CGD_TYPE &t_)
- [cgdStatus](#) & **operator=** (const [cgdStatus](#) &b)
- [cgdStatus](#) & **init** (const [cgdStatus](#) &b)
- CGR_TYPE **init** (const char *F, int L, const mxArray *a)
- int **sameID** (const [cgdStatus](#) &b) const
- int **cmp** (const char *F, int L, const [cgdStatus](#) &b) const
- bool **olderThan** (const [cgdStatus](#) &b) const
- bool **newerThan** (const [cgdStatus](#) &b) const
- bool **hasID** (char lflag=0) const
- bool **operator==** (const CGD_TYPE t_) const
- bool **operator!=** (const CGD_TYPE t_) const
- bool **operator==** (const [cgdStatus](#) &b) const
- bool **operator!=** (const [cgdStatus](#) &b) const
- bool **isSet** () const
- bool **isEmpty** () const
- int **invalid** () const
- bool **sameAs** (const [cgdStatus](#) &b, char lflag=0) const
- const char * **tstr** () const
- [wbstring](#) **toStr** (char vflag=0) const
- [wbstring](#) **u2Str** (const char *istr=NULL) const
- mxArray * **toMx** () const
- mxArray * **toMx** (CGR_TYPE rt) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- mxArray * **add2MxStruct** (mxArray *S, unsigned k) const

Public Attributes

- CGD_TYPE **t**
- double **ctime**
- double **mtime**
- unsigned **ID**
- unsigned char **user** [4]

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.12 cgrType Class Reference

Public Member Functions

- **cgrType** (CGR_TYPE t_=CGR_DEFAULT)
- **cgrType** (double t_)
- template<class T >
[cgrType](#) & **init** (const char *F, int L, T t_)
- [cgrType](#) & **operator=** (const [cgrType](#) &q)
- [cgrType](#) & **operator=** (const CGR_TYPE &t_)

- bool **operator**< (const CGR_TYPE t_) const
- bool **operator**<= (const CGR_TYPE t_) const
- bool **operator**> (const CGR_TYPE t_) const
- bool **operator**>= (const CGR_TYPE t_) const
- bool **operator**== (const CGR_TYPE t_) const
- bool **operator**!= (const CGR_TYPE t_) const
- bool **operator**< (const [cgrType](#) &q) const
- bool **operator**<= (const [cgrType](#) &q) const
- bool **operator**> (const [cgrType](#) &q) const
- bool **operator**>= (const [cgrType](#) &q) const
- bool **operator**== (const [cgrType](#) &q) const
- bool **operator**!= (const [cgrType](#) &q) const
- const char * **tostr** () const
- [wbstring](#) **toStr** () const

Public Attributes

- CGR_TYPE **t**

The documentation for this class was generated from the following file:

- clebsch.hh

4.13 Wb::citerator< T, T2 > Class Template Reference

Public Member Functions

- **citerator** (const T *p_)
- [citerator](#) & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- const T2 & **operator[]** (size_t k)
- const T2 & **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- const T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.14 Wb::citerator< T, T2 * > Class Template Reference

Public Member Functions

- **citerator** (const T *p_)
- **citerator** & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- const T2 * **operator[]** (size_t k)
- const T2 * **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- const T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.15 Wb::CleanUp Class Reference

Public Member Functions

- void **Check** ()

Public Attributes

- int **i**

The documentation for this class was generated from the following file:

- wblib.h

4.16 Wb::Clock Class Reference

Public Member Functions

- **Clock** (const char *F, int L, char use_tag=1, unsigned u=0)
- **Clock** (const char *istr, char use_tag=0)
- **Clock** (const char *istr, char use_tag, unsigned u, [ClockSet](#) *gcs_=NULL, [Clock](#) **g=NULL, char rflag=1)
- void **done** ()
- void **init** ()
- void **reset** ()
- void **init** (const char *s, char use_tag=1)
- void **start** ()
- void **resume** ()
- int **stop** (const char *F=NULL, int L=0)
- double **gettime** (char cflag=0) const
- void **Add** (const [Clock](#) &t)
- int **Switch** (const char *istr, [Clock](#) **g=NULL, char rflag=1)
- mxArray * **toMx** (const char *F=NULL, int L=0) const
- mxArray * **toMxS** (const char *F=NULL, int L=0) const
- void **info** (const char *istr=NULL, char vflag=0) const

Public Attributes

- double **ttot**
- double **ctot**
- double **tneg**
- CHRONO_CLOCK **tref**
- clock_t **cref**
- size_t **nz**
- size_t **ncall**
- [wbstring](#) **name**
- unsigned **clk_flags**
- unsigned **user**
- [ClockSet](#) * **gcs**

The documentation for this class was generated from the following files:

- wbclock.hh
- wbclock.cc

4.17 Wb::ClockSet Class Reference

Public Member Functions

- void **init** (char vflag=1)
- int **info_u** (unsigned u, char rflag=1)
- int **reset_u** (unsigned u, char iflag=0)
- int **reset** (const char *istr, char use_tag=1)
- unsigned **info** (char vflag=0) const
- [Clock](#) * **use** (const char *istr, char mode)
- [Clock](#) * **use** (const char *F, int L, char mode)
- [Clock](#) * **get** (const char *s, char use_tag=1)
- [Clock](#) * **insert** ([Wb::Clock](#) *clk, char lflag=0)
- int **erase** ([Wb::Clock](#) *clk)
- mxArray * **toMx** () const

Public Attributes

- map< std::string, [Wb::Clock](#) * > **buf**

The documentation for this class was generated from the following files:

- wbclock.hh
- wbclock.cc

4.18 Wb::counter Class Reference

Public Member Functions

- **counter** (const char *F, int L, const char *fmt_)
- [counter](#) & **operator++** ()

Public Attributes

- long **n**
- int **line**
- [wbstring](#) **file**
- [wbstring](#) **fmt**

The documentation for this class was generated from the following file:

- wbutil.hh

4.19 CPAT< TQ, TD > Class Template Reference

Public Member Functions

- **CPAT** (const [CPAT](#)< TQ, TD > &D)
- void **init** (TD oz, const mxArray **, unsigned n, unsigned ic1, unsigned r1, unsigned ic2, unsigned r2, const char twoSite, const char ldir)
- int **getCtrlIndex** (const [QSpace](#)< TQ, TD > &PSI)
- [QSpace](#)< TQ, TD > & **HZTimes** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &HZPsi, const char='N', char disp=1)
- [QSpace](#)< TQ, TD > & **HZApplyDiag** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &EPsi, TD a, TD b, char lflag=0, char disp=1)
- void **check_contraction** (const [QSpace](#)< TQ, TD > &PSI, char disp=1)
- void **initCounter** ()
- [QSpace](#)< TQ, TD > & **HZTimes_tst** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &HZPsi, const char='N', char disp=1)
- void **getQRange** (unsigned k, [wbMatrix](#)< TQ > &QQ, [wbvector](#)< unsigned > &SS, [QSpace](#)< TQ, TD > *B=NULL) const
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base") const
- void **PermIC** (const [wbperm](#) &P)

Public Attributes

- [wbvector](#)< [wbvector](#)< [CPAT_Set](#)< TQ, TD > > > **CP**
- TD **z**
- unsigned **sameCtrlIdx**
- unsigned **nr_check_ctr**
- unsigned **HPsi_counter**

The documentation for this class was generated from the following file:

- cpat.hh

4.20 CPAT_Set< TQ, TD > Class Template Reference

Public Member Functions

- **CPAT_Set** (const [CPAT_Set](#)< TQ, TD > &D)
- int **init** (const mxArray *a, unsigned ic1, unsigned r1, unsigned ic2, unsigned r2, const char twoSite, const char ldir)
- int **getCtrlIndex** (const [QSpace](#)< TQ, TD > &PSI)
- int **getCtrlIndex** (const [wbvector](#)< [QSpace](#)< TQ, TD > > &, const char='N')
- int **getCtrlIndex** (const unsigned i, const [QSpace](#)< TQ, TD > &, char='N')
- void **contract** (const [QSpace](#)< TQ, TD > &PSI, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, char='N') const
- void **contract** (const [wbvector](#)< [QSpace](#)< TQ, TD > > &PSI1, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, char='N') const
- void **contract** (const unsigned k, const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &C, char='N') const
- void **contract_diag** (const [QSpace](#)< TQ, TD > &PSI, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, TD a, TD b, char lflag) const
- void **contract_diag** (const [wbvector](#)< [QSpace](#)< TQ, TD > > &PSI1, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, TD a, TD b, char lflag) const
- void **contract_diag** (const unsigned k, const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &C, TD a, TD b, char lflag) const
- mxArray * **toMx** () const

Public Attributes

- [wbvector](#)< [QSpace](#)< TQ, TD > > **A**
- char **opA**
- unsigned **ic**
- [wbperm](#) **P**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **CQn**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **CQt**
- [wbvector](#)< [wbMatrix](#)< unsigned > > **CIn**
- [wbvector](#)< [wbMatrix](#)< unsigned > > **CIt**

The documentation for this class was generated from the following file:

- cpat.hh

4.21 cpu_time Class Reference

Public Member Functions

- void **init** ()
- [cpu_time](#) **since** ()
- [wbstring](#) **toStr** (char flag='c')

Static Public Member Functions

- static double **time** (char flag)

Public Attributes

- double **tcpu**
- double **tsys**

The documentation for this class was generated from the following files:

- util.hh
- util.cc

4.22 CRef< TQ > Class Template Reference

Public Member Functions

- **CRef** (CGR_TYPE rt_=CGR_DEFAULT)
- **CRef** (const [CRef](#) &R)
- **CRef** (const CDATA_TQ &C, unsigned m=-1)
- **CRef & init** (CGR_TYPE rt_=CGR_DEFAULT)
- **CRef & init** (const [CRef](#) &R, char full=1)
- **CRef & operator=** (const [CRef](#) &R)
- void **swap** ([CRef](#) &R)
- **CRef & save2** ([CRef](#) &R)
- double **safeCpy** (const char *F, int L, const [CRef](#) &B, [CRef](#) &C) const
- **CRef & init_1** (const [CRef](#) &R, unsigned j)
- **CRef & init_wld** (const [CRef](#) &R, unsigned wid=1, unsigned l1=-1, const unsigned *d_=NULL)
- **CRef & init** (const char *F, int L, const mxArray *a, unsigned k, char refC=0, const [QSet](#)< TQ > *Q=NULL, char xflag=0)
- **CRef< TQ > & initIdentityR** (const char *F, int L, const [QType](#) &q, const TQ *qs, unsigned dim=-1, char xflag=0)
- **CRef< TQ > & initIdentity1J** (const char *F, int L, const [QType](#) &q, const TQ *qs, unsigned dim=-1, char xflag=0)
- **CRef< TQ > & Reduce2Identity** (char xflag=0)
- **wbvector< RTD > & trace** (const char *F, int L, [wbvector](#)< RTD > &cgt) const
- **wbvector< double > & trace** (const char *F, int L, [wbvector](#)< double > &cgt) const
- **wbvector< double > trace** (const char *F, int L) const
- double **trace** () const
- double **trace** (const char *F, int L, const [ctrlidx](#) &i1, const [ctrlidx](#) &i2, [CRef](#) *Rt=NULL) const
- **CRef & initAbelian** ()
- **CRef & initCtrScalar** (double x=1)
- **CRef & initBase** (const CDATA_TQ *cgr, unsigned m=-1, const CDATA_TQ *X=NULL)
- **CRef & initBase** (const CDATA_TQ &C, unsigned m=-1)
- int **LoadRef** (const char *F=0, int L=0, char force=1) const
- int **Reduce2Ref** (const char *F=0, int L=0, char force=0) const
- **operator bool** () const
- bool **operator!** () const
- bool **isEmpty** () const
- bool **isSymmetric** (const char *F=0, int L=0) const
- bool **isRefInit** (const char *F=0, int L=0, char check=1) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **isScalar** (char dflag=0) const
- [size_t](#) **isw2** (const char *F=NULL, int L=0) const
- [size_t](#) **isw3** () const

- unsigned **Reduce_w3ld** (unsigned d=-1, char lflag=0)
- int **cgw_check** (const char *F, int L) const
- bool **cgw_exists** (unsigned i, unsigned j)
- int **cgw_check_std3** (const char *F=NULL, int L=-1, unsigned d3=0, unsigned im=0)
- int **checkQ** (const char *F, int L, const **QSet**< TQ > &Q) const
- unsigned **numel** () const
- unsigned **wnumel** () const
- bool **wscalar** (const char *F=NULL, int L=0) const
- bool **wscalar1** (const char *F=NULL, int L=0) const
- double **wel** (unsigned k) const
- bool **wOM** (const char *F=NULL, int L=0) const
- bool **wdim_is** (unsigned m) const
- unsigned **wdim** (const char *F=NULL, int L=1) const
- unsigned **wdim1** (const char *F=NULL, int L=0) const
- unsigned **wdim2** (const char *F=NULL, int L=0) const
- unsigned **wdim12_** (const char *F, int L, unsigned &d2) const
- unsigned **wdim12** (const char *F, int L, unsigned &d2, unsigned d0=1) const
- unsigned **wdim3** (const char *F=NULL, int L=0) const
- double **wget0** () const
- double **wget1** () const
- bool **wSame** (const **wbarray**< double > &cgw_, char lenient=0, double eps=1e-14) const
- template<class T >
wbvector< T > & **getSize** (**wbvector**< T > &S, char bare=0) const
- unsigned **Size** (unsigned k, unsigned r=-1) const
- unsigned **qdim** (unsigned k, const **QType** *qt=NULL) const
- int **checkAbelian** (const char *F=NULL, int L=0) const
- bool **isDiagCSC** (RTD eps=1E-14) const
- bool **isIdentityCG** (**wbvector**< double > *nrm=NULL, double eps=1E-14) const
- int **wisld** () const
- double **norm2** (char checks=1) const
- double **NormSignW** (const char *F=NULL, int L=0, char useExt=1, double eps=CG_SKIP_DEPS1, double eps2=CG_SKIP_DEPS2)
- char **sameUptoFac** (const **CRef** &B, double *fac=NULL, double eps=1E-12) const
- char **sameAs** (const **CRef** &B, double eps=1E-12) const
- char **sameAs_fix** (const char *F, int L, **CRef** &B, double eps=1E-12)
- bool **sameQSet** (const **QSet**< TQ > &Q) const
- bool **sameQSet** (const **CRef** &B) const
- bool **sameQDir** (const char *F, int L, const **QDir** &qd) const
- bool **sameQDir** (const char *F, int L, const **CRef** &B) const
- bool **sameQDir** (const **iTags** &b) const
- int **gotSameCData** (const char *F, int L, const **CRef** &B, char xflag=0) const
- double **NormStd** (const char *F, int L, unsigned r=-1)
- double **normExt** (const char *F=NULL, int L=0) const
- int **SortDegQ** (const char *F=NULL, int L=0, **QSet**< TQ > *QS=NULL)
- bool **isSortedDegQ** (**QSet**< TQ > *QS, **wbperm** &pxt, char &cxt) const
- bool **isSortedDegQ** (**QSet**< TQ > *QS=NULL) const
- char **got3** (const char *F, int L, const **QType** &q, const **qset**< TQ > &J12, const **qset**< TQ > &J3) const
- bool **operator==** (const **CRef** &B)
- bool **operator!=** (const **CRef** &B)
- double **normDiff2** (const char *F, int L, const **CRef** &B) const
- double **normDiff** (const char *F, int L, const **CRef** &B) const
- **wbarray**< double > & **wProd** (const **CRef** &B, **wbarray**< double > &x) const
- int **check** (const char *F, int L) const
- int **check** (const char *F, int L, const **QType** &q, const **QDir** &qdir) const
- unsigned **getOM** (const char *F=NULL, int L=0, char wflag=0) const

- unsigned **checkOM** (const char *F=NULL, int L=0) const
- unsigned **rankS** (char lflag=0) const
- unsigned **rank** (const char *F=NULL, int L=0, char lflag=0) const
- **CRef wget** (unsigned j) const
- **CRef & Permute** (const **wbperm** &P, char iflag=0, char isnew=0)
- **CRef & permute** (const **wbperm** &P, **CRef** &B, char iflag=0, char isnew=0) const
- **CRef & Conj** ()
- int **HConjOpScalar** ()
- bool **gotConj** () const
- bool **gotPerm** (const char *F=NULL, int L=0) const
- bool **anyTrafo** (char pflag=1) const
- bool **anyTrafo** (char &ip, char &ic, char pflag=1) const
- bool **gotnoTrafos** () const
- unsigned **getP** (unsigned k) const
- **wbperm** & **getP** (**wbperm** &p, char iflag=0) const
- char **stat** () const
- bool **affectsQDir** () const
- **qset**< TQ > & **adapt** (**qset**< TQ > &q, char iflag=0) const
- **QSet**< TQ > & **adapt** (**QSet**< TQ > &Q, char iflag=0) const
- **ctrlIdx** & **adapt** (**ctrlIdx** &l, char iflag=0) const
- **QDir** & **get_qdir** (**QDir** &qd) const
- **QDir** **get_qdir** () const
- **wbstring qdir2Str** (char vflag=0) const
- **wbstring toStr** (char lflag=0) const
- **wbstring sizeStr** () const
- **wbstring statStr** (char vflag=0) const
- **wbstring qStr** () const
- **wbstring QStr** () const
- mxArray * **toMx** (char flag=0) const
- mxArray * **toMX** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n, char flag=0) const
- void **add2MxStruct** (mxArray *S, unsigned i, char flag=0) const

Public Attributes

- const CDATA_TQ * **cgb**
- **wbarray**< double > **cgw**
- **wbperm** **cgp**
- char **conj**
- **cgrType** **rtype**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.23 CStore< TQ > Class Template Reference

Public Types

- typedef MAP32::iterator **iMAP32**
- typedef MAP31::iterator **iMAP31**

Public Member Functions

- void **clear** ()
- void **init** ()
- const CDATA_TQ & **getIdentityC** (const char *F, int L, const QType &t, const TQ *qs, unsigned dim=-1, unsigned loadRC=LB_GEN)
- const CDATA_TQ & **getIdentity1J** (const char *F, int L, CRef< TQ > &C, const QType &t, const TQ *qs, unsigned dim=-1, unsigned loadRC=LB_GEN)
- void **add_CData_abelian** (const QType &q, const TQ &J1, const TQ &J2)
- void **add_CData** (const QType &q, const qset< TQ > &J1, const qset< TQ > &J2)
- void **add2BUF** (const char *F, int L, const CDATA_TQ &S, char nflag=0)
- unsigned **add3** (const QType &q, const mxArray *S, char loadC=0)
- unsigned **add2BUF** (const QType &q, const mxArray *S, unsigned k=-1)
- CDATA_TQ & **getBUF** (const char *F, int L, const QSet< TQ > &Q, unsigned loadRC=0)
- size_t **reduceMemUsage** (const char *F=NULL, int L=0, double mfac=-1)
- void **getQfinal_1** (const char *F, int L, const QType &type, const qset< TQ > &J1, const qset< TQ > &J2, wbMatrix< TQ > &J, wbMatrix< const CRef< TQ > * > *ss=NULL, char loadC=0)
- void **getQfinal_v** (const char *F, int L, const QVec &type, const qset< TQ > &q1, const qset< TQ > &q2, wbMatrix< TQ > &QQ, wbMatrix< const CRef< TQ > * > *SS=NULL, char loadC=0)
- int **getQfinal_zdim** (const char *F, int L, const QVec &qvec, const wbMatrix< TQ > &Q1, const wbMatrix< TQ > &Q2, const wbMatrix< TQ > &Q, wbvector< widx_t > &s1, wbvector< widx_t > &s2, wbvector< widx_t > &s, wbvector< unsigned > *m=NULL, wbMatrix< const CRef< TQ > * > *S3=NULL)
- void **getQfinal** (const char *F, int L, const QVec &type, const wbMatrix< TQ > &Q1, const wbMatrix< TQ > &Q2, QMap< TQ > &M, unsigned char cgflag=0)
- void **Info** (const char *F=0, int L=0, char vflag=0) const
- int **checkInit** (const char *F, int L, const QType &t) const
- unsigned **LoadStore** (const char *F, int L, const char *file)
- mxArray * **toMx** (const QType &q=QTYPE_UNKNOWN) const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **put** (const char *vname, const char *ws="caller") const
- int **find_map3** (const QType &q, const qset< TQ > &J1, const qset< TQ > &J2) const
- int **find_map3** (const QType &q, const qset< TQ > &J12) const
- int **valid_mp3_data** (const QType &t, const qset< TQ > &J1, const qset< TQ > &J2, const qset< TQ > &J3, char rflag=0) const
- CDATA_TQ * **BUF_find** (const QSet< TQ > &Q)
- int **Reduce2Ref** (const char *F, int L, const QSet< TQ > &Q, char force=0, unsigned Dmin=1024)

Public Attributes

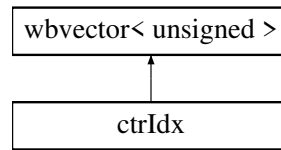
- map< QType, map< qset< TQ >, map< qset< TQ >, CRef< TQ > > > > **map3**
- unordered_map< QSet< TQ >, CDATA_TQ, QHash< TQ > > **BUF**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc
- clebsch_old.cc

4.24 ctrIdx Class Reference

Inheritance diagram for ctrIdx:



Public Member Functions

- **ctrIdx** (unsigned l, unsigned *d, char c=0)
- **ctrIdx** (const char *F, int L, const char *s, char c=0)
- **ctrIdx** (const char *F, int L, const mxArray *a)
- **ctrIdx** (const [ctrIdx](#) &l)
- **ctrIdx** (const [wbvector](#)< unsigned > &l, char c=0)
- **ctrIdx & init** (unsigned l=0, unsigned *d=NULL, char c=0)
- **template**<class T >
 ctrIdx & initT (unsigned l=0, T *d=NULL, char c=0)
- **ctrIdx & init** (const [wbvector](#)< unsigned > &l, char c=0)
- **ctrIdx & init1** (unsigned k, char c=0)
- **ctrIdx & init2** (unsigned k1, unsigned k2, char c=0)
- **template**<class TQ, class TD >
 ctrIdx & initOp (const char *F, int L, [itag_](#) t1, const [QSpace](#)< TQ, TD > &B, const char *sopt)
- **int init** (const char *F, int L, const char *s, unsigned offset=1)
- **ctrIdx & init** (const char *F, int L, const mxArray *a)
- **ctrIdx & init** (const mxArray *a)
- **ctrIdx & Index** (unsigned l, char c=0)
- **int checkUniqueS** (const char *F=0, int L=0, const unsigned r=-1) const
- **ctrIdx & Sort** ([wbperm](#) &P, unsigned r=-1)
- **ctrIdx & sort** ([ctrIdx](#) &l, [wbperm](#) &P, unsigned r=-1) const
- **ctrIdx & Sort_** ([ctrIdx](#) &J, unsigned r=-1)
- **ctrIdx & invert** (unsigned r, [ctrIdx](#) &l) const
- **ctrIdx & Invert** (unsigned r)
- **ctrIdx & Conj** ()
- **int extend2Perm** (widx_t N, [wbperm](#) &P, char toend=0) const
- **wbperm & extend2Perm** (unsigned ra, unsigned ma, const [ctrIdx](#) &icb, unsigned rb, unsigned mb, const [wbperm](#) &pab, char conj, unsigned mc, [wbperm](#) &Pbc, [ctrIdx](#) &kb, [ctrIdx](#) &kcb, char Bflag=1) const
- **ctrIdx & save2** ([ctrIdx](#) &J)
- **mxArray * toMx** () const
- **wbstring toStr** (char xflag=0) const

Public Member Functions inherited from [wbvector](#)< unsigned >

- **wbvector** (size_t l, unsigned x)
- **wbvector** (size_t l, const unsigned *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)

- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const unsigned *d0)
- [wbvector](#) & **init** (size_t l, const unsigned *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< unsigned > & **init** (const [wbsparray](#)< unsigned > &S)
- [wbvector](#) & **init** (size_t l1, const unsigned *d1, size_t l2, const unsigned *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const unsigned *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const unsigned &x)
- [wbvector](#) & **initStride** (size_t l, const unsigned *d0, size_t stride)
- [wbvector](#) & **init_stride** (const unsigned *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const unsigned *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< unsigned > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< unsigned > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< unsigned > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< unsigned > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< unsigned > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< unsigned > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const unsigned *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< unsigned > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const unsigned &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const unsigned &x)
- void **set** (size_t offset, const unsigned *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const unsigned &x)
- [wbvector](#) & **setMax** (const unsigned &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const unsigned &x)
- [wbvector](#) & **add** (const unsigned *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const unsigned &x) const
- bool **isNormal** () const
- bool **isNormal** () const

- `bool isNormal () const`
- `bool isFinite () const`
- `size_t find1 (const unsigned &x) const`
- `size_t find1 () const`
- `size_t count (const unsigned &x) const`
- `size_t icount (const unsigned &x) const`
- `size_t nnz () const`
- `size_t nnz (unsigned eps) const`
- `size_t numZeros (unsigned eps=unsigned(1E-14)) const`
- `size_t numEQ (const unsigned &) const`
- `size_t numGT (const unsigned &) const`
- `size_t numLT (const unsigned &) const`
- `wbindex & find (const unsigned &x, wbindex &l, char iflag=0) const`
- `wbindex & find (const unsigned &x, wbindex &l, wbindex &lx) const`
- `wbindex & find (wbindex &l) const`
- `wbindex & find (wbindex &l, wbindex &lx) const`
- `wbvector< size_t > find () const`
- `wbindex & findGT (const unsigned &x, wbindex &l, char iflag=0) const`
- `wbindex & findGT (const unsigned &x, wbindex &l, wbindex &lx) const`
- `wbindex & findzeros (wbindex &l) const`
- `wbindex & findRange (const unsigned &x1, const unsigned &x2, wbindex &l, const char *w=NULL) const`
- `int findClosestSorted (unsigned r) const`
- `int findClosest (unsigned r) const`
- `size_t findValues (const wbvector &v, wbindex &l) const`
- `wbvector & setp (const unsigned *x)`
- `void reset ()`
- `void cpy (const unsigned *v)`
- `void cpy (const unsigned *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (unsigned &A1, unsigned &A2)`
- `wbvector & Collect (unsigned &A1, unsigned &A2, unsigned &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="") const`
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="") const`
- `void operator+= (const T_ &a)`
- `void operator+= (const wbvector &b)`
- `void operator-= (const T_ &a)`
- `void operator-= (const wbvector &b)`
- `wbvector & operator*= (const T_ &a)`
- `wbvector & operator/= (const T_ &a)`
- `wbvector operator* (const T_ &a) const`
- `wbvector operator+ (const T_ &a) const`
- `wbvector operator+ (const wbvector &b) const`
- `wbvector operator- (const T_ &a) const`

- **wbvector operator-** (const **wbvector** &b) const
- **wbvector** & **Plus** (const unsigned *b)
- **wbvector** & **Plus** (const unsigned *b, unsigned bfac)
- **wbvector** & **Plus** (const **wbvector** &b, unsigned bfac=1)
- void **plus** (const **wbvector** &b, **wbvector** &c, unsigned bfac=1, unsigned cfac=0) const
- **wbvector** & **times** (unsigned fac, **wbvector** &x) const
- **wbvector** & **RevertSigns** ()
- int **isum** () const
- unsigned **sum** () const
- unsigned **sum** (size_t i1, const char *estr) const
- unsigned **sum** (size_t i1, size_t i2) const
- unsigned **cumsum_** (**wbvector** &S, char xflag=0) const
- unsigned **Cumsum** (char zflag=0) const
- unsigned **cumsum0prod** (**wbvector**< T2 > &b, **wbvector**< unsigned > &cs, char xflag) const
- **wbvector** & **Eldiv** (const **wbvector** &v) const
- **wbvector** & **tensorProd** (const **wbvector** &v2, **wbvector** &vv) const
- unsigned **scalarProd** (const **wbvector** &v) const
- unsigned **scalarProd** (const unsigned *v) const
- unsigned **norm** () const
- unsigned **norm2** () const
- double **normDiff** (const **wbvector** &v, unsigned fac=1) const
- unsigned **normDiff2** (const **wbvector** &v, unsigned fac=1) const
- double **normDiff2_** (const **wbvector** &v, double fac=1) const
- unsigned **GSPProject** (const unsigned *v, char isnorm=0, char tflag=0) const
- **wbvector** & **Normalize** (const char *F=NULL, int L=0)
- **wbvector** & **subtract** (const unsigned *d)
- void **getReal** (**wbvector**< double > &R) const
- void **getReal** (**wbvector**< double > &R) const
- void **getImag** (**wbvector**< double > &I) const
- void **getImag** (**wbvector**< double > &I) const
- **wbvector** & **Conj** ()
- **wbvector**< **wbcomplex** > & **Conj** ()
- **wbvector** & **applyFlag** (char &flag, const unsigned &fac, **wbvector** &a)
- bool **allEqual** () const
- bool **allEqual** (const unsigned &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const unsigned &a, unsigned eps=1E-15) const
- bool **allUnequal** (const unsigned &a) const
- bool **any** () const
- bool **anyUnequal** (const unsigned &a) const
- bool **anyUnequal** (const **wbvector**< size_t > &l, const unsigned &x, size_t *k=NULL) const
- bool **anyEqual** (const unsigned &a, size_t *i1=NULL) const
- bool **anyEqual** (const **wbvector**< size_t > &l, const unsigned &x, size_t *k=NULL) const
- bool **anyGT** (const unsigned &a) const
- bool **anyGE** (const unsigned &a) const
- bool **anyLT** (const unsigned &a) const
- bool **anyLE** (const unsigned &a) const
- bool **allGT** (const unsigned &a) const
- bool **allGE** (const unsigned &a) const
- bool **allLE** (const unsigned &a) const
- bool **allLT** (const unsigned &a) const
- bool **allIn** (const unsigned &a1, const unsigned &a2) const
- bool **anySmallVals** (const unsigned eps=1E-15) const
- bool **isEqual** (const unsigned *v) const

- `bool isEqual (const wbvector &v) const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool operator== (const wbvector &v) const`
- `bool operator!= (const wbvector &v) const`
- `bool isEqualR (const wbvector &v) const`
- `bool operator^ (const wbvector &v) const`
- `bool deepEqualP (const wbvector &B) const`
- `char cmp (const wbvector &v) const`
- `bool operator< (const wbvector &v) const`
- `int operator< (const unsigned &r) const`
- `bool operator<= (const wbvector &v) const`
- `bool operator> (const wbvector &v) const`
- `bool operator>= (const wbvector &v) const`
- `unsigned & el (size_t i) const`
- `unsigned elx (size_t i, const unsigned &x) const`
- `const unsigned & operator[] (size_t i) const`
- `unsigned & operator[] (size_t i)`
- `wbvector operator[] (const wbvector< size_t > &l) const`
- `unsigned & last () const`
- `unsigned & last (size_t &i) const`
- `unsigned & last (T1 &i_) const`
- `unsigned & p2last () const`
- `unsigned * ref (size_t i) const`
- `wbvector & setRand (double fac=1., double shift=0.)`
- `wbvector< wbcomplex > & setRand (double fac, double shift)`
- `void setIdx (double shift=0., double fac=1.)`
- `int set2Group (const wbperm &P, const WBINDEX &D, const unsigned *S0=NULL, char iflag=0)`
- `wbvector & Resize_ (size_t n)`
- `wbvector & Shorten2 (size_t n)`
- `wbvector & Resize (size_t n, const unsigned *d=NULL)`
- `wbvector & resize (size_t n, wbvector &X) const`
- `wbvector & cat (const wbvector &v2)`
- `wbvector & Cat (const wbvector &v1, const wbvector &v2)`
- `wbvector & Cat (const wbvector< wbvector > &vv)`
- `wbvector & Cat (const wbvector< wbvector * > &vv)`
- `wbvector & Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)`
- `wbvector & Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3=0, const wbvector< T3 > *v4=0)`
- `wbvector< unsigned > & Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3, const wbvector< T3 > *v4)`
- `wbvector< unsigned > & Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)`
- `wbvector & Append (const wbvector &v2)`
- `wbvector & Append (const unsigned &x)`
- `wbvector & Append (size_t d2, const unsigned *x)`
- `wbvector append (const wbvector &v) const`
- `wbvector & append (const wbvector &v2, wbvector &vout) const`
- `void append (const unsigned *x, size_t d2, wbvector &vout) const`
- `int recLess (const unsigned *v) const`
- `int recLessE (const unsigned *v) const`
- `int recCompare (const unsigned *v) const`
- `unsigned max () const`

- unsigned **max** (size_t *k) const
- unsigned **max** (unsigned &k) const
- unsigned **max_** (unsigned x) const
- unsigned **min** () const
- unsigned **min** (size_t *k) const
- unsigned **min** (unsigned &k) const
- unsigned **min_** (unsigned x) const
- unsigned **aMax** (size_t *k=NULL) const
- unsigned **aMax** (unsigned &k) const
- unsigned **aMin** (char zflag=0, size_t *k=NULL) const
- unsigned **maxneg** (size_t *k=NULL) const
- unsigned **minpos** (size_t *k=NULL) const
- unsigned **avg** () const
- unsigned **std** () const
- void **range** (unsigned &xmin, unsigned &xmax) const
- unsigned **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, unsigned *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, unsigned *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- unsigned **prod** () const
- unsigned **prod** (unsigned x) const
- unsigned **prod** (size_t i, size_t i2) const
- unsigned **prod** (size_t i, size_t i2, unsigned x) const
- unsigned **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (unsigned &p1, unsigned &p2, const unsigned &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const

- `mxArray * toMxP_S (const char tflag=0) const`
- `mxArray * toMx (const char tflag=0) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMxP (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base_d (const char tflag=0) const`
- `mxArray * toMx_offset (const unsigned &offset, const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tflag=0) const`
- `void add2MxStruct (mxArray *S, const char *vname, char tflag=0) const`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tflag=0) const`
- `void putg (const char *vname="ans", char tflag=0) const`
- `void putx (const char *vname="ans", char tflag=0) const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring toStr () const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring sizeStr () const`
- `wbstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const`
- `wbstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const`

Public Attributes

- `char conj`
- `iTags newtags`

Public Attributes inherited from `wbvector< unsigned >`

- `unsigned * data`
- `size_t len`
- `char isref`

Additional Inherited Members

Protected Member Functions inherited from `wbvector< unsigned >`

- `wbvector & RENEW (size_t n, const unsigned *d=NULL, size_t l=-1, char init=1)`
- `wbvector & RENEW_VAL (size_t n, const unsigned &x)`

The documentation for this class was generated from the following files:

- `wbindex.hh`
- `wbindex.cc`

4.25 CG::FileLock Class Reference

Public Member Functions

- template<class TQ >
FileLock (const QType &q_, const qset< TQ > &J1, const qset< TQ > &J2, const char *ext="mp3")
- template<class TQ >
int **init** (const QType &q_, const qset< TQ > &J1, const qset< TQ > &J2, const char *ext="mp3")
- int **GetLock** (const char *F=NULL, int L=0, char flag='W')
- int **ReleaseLock** (const char *F=NULL, int L=0)
- int **ensureFOpen** (const char *F=NULL, int L=0) const

Public Attributes

- int **fid**
- QType **q**
- wstring **fname**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.26 genRG_base< TQ, TD > Class Template Reference

Public Member Functions

- bool **isEmpty** () const
- double **normDiff** (const char *F, int L, const genRG_base &B, TD eps=1E12) const
- void **checkOrthoCommRel** (const char *F, int L, const genRG_base &B) const
- genRG_base & **Sort** ()
- double **SkipTiny** (const char *F=0, int L=0)
- genRG_base & **ApplyQFac** (const wvector< double > &qfac, const warray< double > *JM=NULL)
- genRG_base & **operator=** (const genRG_base &S)
- genRG_base & **init** ()
- SPIDX_T **dim** () const
- void **compareStdSU2** (const char *F=NULL, int L=0) const
- genRG_base & **save2** (genRG_base &S)
- wstring **info** () const
- mxArray * **toMx** () const
- size_t **memSize** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const

Public Attributes

- [QType](#) **q**
- [wbvector](#)< [wbSparrayTD](#) > **Sp**
- [wbvector](#)< [wbSparrayTD](#) > **Sz**
- [qset](#)< [TQ](#) > **J**
- [wbMatrix](#)< double > **Z**
- double **err**
- [wbstring](#) **istr**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_aux.cc
- clebsch_io.cc

4.27 [genRG_struct](#)< [TQ](#), [TD](#) > Class Template Reference

Public Member Functions

- [genRG_struct](#) & **checkInit** (const char *F, int L, const [QType](#) &q0)
- [genRG_struct](#) & **SetupSym** (const char *F, int L, const [QType](#) &q0, [qset](#)< [TQ](#) > *qs=NULL)
- [genRG_struct](#) & **Setup_SUN** (const char *F, int L, [qset](#)< [TQ](#) > *qs=NULL)
- [genRG_struct](#) & **Setup_SpN** (const char *F, int L, [qset](#)< [TQ](#) > *qs=NULL)
- [genRG_struct](#) & **Setup_SON** (const char *F, int L, [qset](#)< [TQ](#) > *qs=NULL)
- [genRG_struct](#) & **Setup_SEN** (const char *F, int L, [qset](#)< [TQ](#) > *qs=NULL)
- void **initSU2** (const [qset](#)< [TQ](#) > &J)
- void **initCommRel** (const char *F, int L, const [genRG_base](#)< [TQ](#), [TD](#) > &R, char vflag=0)
- double **checkCommRel** (const char *F, int L, const [genRG_base](#)< [TQ](#), [TD](#) > &R) const
- void **printStatus** (const char *F, int L, const char *istr=NULL)
- int **get1J_gen** (const char *F, int L, const [qset](#)< [TQ](#) > &q, char flag=0)
- int **getTensorProdReps_gen** (const [qset](#)< [TQ](#) > &q1, const [qset](#)< [TQ](#) > &q2, unsigned flag=TP3_LDM)
- void **getTensorProdReps** (const [qset](#)< [TQ](#) > &q1, const [qset](#)< [TQ](#) > &q2)
- unsigned **genTensorProds** (const [qset](#)< [TQ](#) > &q, unsigned flag=0)
- unsigned **genTensorProds** (unsigned dmax=-1, char sdig=0, char vflag=0)
- void **put** (const char *vname, const char *ws="caller") const
- [mxArray](#) * **toMx** (char bflag=1) const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **wblog_RC_1st_build** (const char *F, int L, const char *fct, int nrep) const

Public Attributes

- [QType](#) **q**
- map< [qset](#)< [TQ](#) >, [genRG_base](#)< [TQ](#), [TD](#) > > **RSet**
- [qset](#)< [TQ](#) > **qdef**
- [wbvector](#)< [pairPatternCG](#)< double > > **CR**
- [wbMatrix](#)< double > **DZ**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.28 Wb::gpara__ Class Reference

Public Member Functions

- void **init** ()

Public Attributes

- char **x**

The documentation for this class was generated from the following file:

- wblib.h

4.29 groupIndex< T > Class Template Reference

Public Member Functions

- **groupIndex** (const [wbvector](#)< T > &P_, const [wbvector](#)< T > &D_, const wperm_t **p=NULL)
- **groupIndex** (const [wbvector](#)< T > &DI, T ng=-1)
- void **init** ()
- **groupIndex** & **init** (const [wbvector](#)< T > &P_, const [wbvector](#)< T > &D_)
- **groupIndex** & **initX** ([wbvector](#)< wperm_t > &P_, [wbvector](#)< T > &D_, const wperm_t **p=NULL)
- **groupIndex** & **init** (const [wbvector](#)< T > &DI, T ng=-1)
- **groupIndex** & **init** (T ng)
- **groupIndex** & **init_g** (T ig, T nz)
- **groupIndex** & **setD** (T i, T nz)
- T **init_ic** ()
- **groupIndex** & **setup** (char full=0)
- T **dim** (T ig)
- T **numGroups** () const
- void **getGroup** (T ig, T *&p0, T &d) const
- T **getGroup** (T ig, T &j, T &j2) const
- T **getGroup** (T ig) const
- template<class T2 >
void **getRecs0** (const [wbMatrix](#)< T2 > &, [wbMatrix](#)< T2 > &, T m) const
- T **groupIter** (T &ig, T &j, T &j2) const
- mxArray * **toMx** () const

Public Attributes

- [wbvector](#)< T > **D**
- [wbperm](#) **P**
- [wbvector](#)< T > **I0**
- [wbvector](#)< T > **Ig**

The documentation for this class was generated from the following files:

- wbindex.hh
- wbindex.cc

4.30 icFlags Class Reference

Public Member Functions

- **icFlags** (const mxArray *ain=NULL)
- **icFlags** & **checkQSpace** (const mxArray *a)
- char **check_arg** (const mxArray *ain)
- **icFlags** & **init** (const mxArray *ain)
- char **set** (const mxArray *a, const char *istr="")
- template<class TQ, class TD >
void **apply** (**QSpace**< TQ, TD > &X, char vflag=1)

Public Attributes

- const mxArray * **a**
- char **conj**
- char **xflag**
- **wbstring** **otags**
- **wbstring** **regex_r**
- **wbstring** **sidx**

The documentation for this class was generated from the following file:

- contractQS.cc

4.31 indexSparseRef< TD > Class Template Reference

Public Member Functions

- **indexSparseRef** (const **wbsparray**< TD > &A_, unsigned M2=1)
- **indexSparseRef** (const char *F, int L, const **wbsparray**< TD > &A_, unsigned M2=1)
- **indexSparseRef** & **init** (const char *F, int L, const **wbsparray**< TD > &A_, unsigned M2=1)
- SPIDX_T **numCols** () const
- SPIDX_T **numRows** () const
- SPIDX_T **nnz** (SPIDX_T k) const
- TD **LContractVec** (const **wbsparray**< TD > &b, SPIDX_T k, char tnorm=0)
- mxArray * **toMx** () const

Public Attributes

- unsigned **M**
- **wbvector**< SPIDX_T > **JA**
- **wbvector**< sSPIDX_T > **JJ**
- const **wbsparray**< TD > * **A**

The documentation for this class was generated from the following file:

- wbsparray.hh

4.32 Wb::IOstat Class Reference

Public Member Functions

- [IOstat](#) & **init** ()
- void **gotread** (size_t s=0)
- void **gotwrite** (size_t s=0)
- bool **isEmpty** ()
- void **print** (const char *istr, const char *hstr=0, const char *xstr1=0, const char *xstr2=0)

Public Attributes

- unsigned **nread**
- unsigned **nwrite**
- size_t **sread**
- size_t **swrite**
- size_t **aux1**
- size_t **aux2**

The documentation for this class was generated from the following files:

- util.hh
- util.cc

4.33 is_pointer_< T > Class Template Reference

Public Member Functions

- char **q** ()

The documentation for this class was generated from the following file:

- wblibx.h

4.34 is_pointer_< T * > Class Template Reference

Public Member Functions

- char **q** ()

The documentation for this class was generated from the following file:

- wblibx.h

4.35 `is_pointer_< T ** >` Class Template Reference

Public Member Functions

- `char q ()`

The documentation for this class was generated from the following file:

- `wbllibx.h`

4.36 `is_pointer_< T *** >` Class Template Reference

Public Member Functions

- `char q ()`

The documentation for this class was generated from the following file:

- `wbllibx.h`

4.37 `itag_` Class Reference

Public Member Functions

- `itag_ (const char *F, int L, const char *s)`
- `itag_ (const char *s)`
- `itag_ (const itag\_ &x)`
- `itag\_ & init (const char *F, int L, const char *s, unsigned n=-99)`
- `itag\_ & init (const char *s=NULL)`
- `itag\_ & init_tag ()`
- `itag\_ & init_qdir (char c)`
- `template<class TQ , class TD > itag\_ & init (const char *F, int L, const wbvector< QSpace< TQ, TD > > &A, wbindex ia)`
- `itag\_ & init (const char *F, int L, const mxArray *a)`
- `itag\_ & operator= (unsigned n)`
- `itag\_ & operator= (const char *s)`
- `bool operator== (const itag\_ &x) const`
- `bool operator!= (const itag\_ &x) const`
- `char sameAs (const itag\_ &x, char lflag=0) const`
- `bool isConj (const itag\_ &x, char full=1) const`
- `bool isConj_x (const itag\_ &x, char lflag) const`
- `int isValid () const`
- `bool isEmpty () const`
- `bool isEmpty (char lenient) const`
- `bool isEmpty_m () const`
- `char getMark () const`
- `operator bool () const`
- `bool operator! () const`

- **operator const char * ()** const
- [itag_](#) & **Set** (const char *s)
- [itag_](#) & **tSet** (const [itag_](#) &x)
- [itag_](#) & **Set_** (const [itag_](#) &x, char keep=0)
- [itag_](#) & **Conj** ()
- [itag_](#) & **setConj** ()
- bool **isConj** () const
- [itag_](#) & **deConj** ()
- [itag_](#) **conj** () const
- [itag_](#) **deconj** () const
- [itag_](#) & **SetFlag** (unsigned l)
- [itag_](#) & **SetFlags** (unsigned l=-1, char check=1)
- [itag_](#) & **UnsetFlag** (unsigned l)
- [itag_](#) & **UnsetFlags** ()
- bool **GotFlag** (unsigned l) const
- unsigned **GotFlags** (unsigned l=1) const
- [itag_](#) & **SetSI** (const char *F, int L, const char *tag, unsigned k, char mflag=6, char nfmt=1)
- [itag_](#) & **AppendChar** (char q, const char *qs=NULL)
- [itag_](#) & **PrependChar** (char q, const char *qs=NULL)
- int **CheckFirstChar** (char q, const char *qs=NULL)
- [itag_](#) & **MarkDual** (char m=QS_MARK_DUAL, bool always=1)
- [itag_](#) & **AddTilde** ()
- [itag_](#) & **SetM** ()
- [itag_](#) & **SetK** ()
- [itag_](#) & **SetD** ()
- unsigned **to_str** (char *s, unsigned len, char cflag=1) const
- [wbstring](#) **toStr** (char cflag=1) const
- mxArray * **toMx** () const

Static Public Member Functions

- static void **Reset** (unsigned char q=-1)
- static void **iter_flag_id** ()

Public Attributes

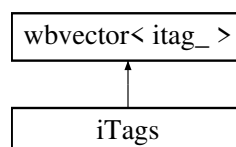
- IDT **t**

The documentation for this class was generated from the following files:

- wbindex.hh
- wbindex.cc

4.38 iTags Class Reference

Inheritance diagram for iTags:



Public Member Functions

- **iTags** (unsigned n=0)
- **iTags** (const char *F, int L, const char *s)
- **iTags** (const char *s)
- unsigned **init** (const char *F, int L, const char *s)
- unsigned **init** (const char *s)
- **iTags** & **init** (unsigned n=0)
- **iTags** & **init** (const **iTags** &b)
- **iTags** & **operator=** (const **iTags** &b)
- **iTags** & **Update** (const **iTags** &b)
- **iTags** & **tSet** (const **iTags** &b)
- unsigned **init** (const char *F, int L, const mxArray *a)
- **iTags** & **init3** ()
- **iTags** & **init** (size_t l1, const **itag_** *d1, size_t l2, const **itag_** *d2)
- unsigned **Init** (const char *F, int L, const char *s, unsigned k=-1, char lflag=-1)
- **iTags** & **Init** (unsigned r, int k, const char **st)
- **iTags** & **initOp** (**itag_** t, const **iTags** &b, const char *ot="op")
- **iTags** & **init_alpha** (unsigned r, unsigned s, const char *tag, char t, unsigned k)
- unsigned **Set** (const char *F, int L, const char *tag, unsigned r=-1, const char *tom=NULL, unsigned m=-1, bool conjOM=1)
- **iTags** & **SetConj** (const char *F, int L, unsigned i)
- **iTags** & **Conj** ()
- **iTags** & **SetFlag** (const char *F, int L, unsigned k, unsigned l)
- **iTags** & **SetFlags** (const char *F, int L, unsigned k, unsigned l=-1, char check=1)
- **iTags** & **UnsetFlag** (const char *F, int L, unsigned k, unsigned l)
- **iTags** & **UnsetFlags** ()
- **iTags** & **init_tags** ()
- bool **operator==** (const **iTags** &b) const
- bool **operator!=** (const **iTags** &b) const
- int **findRegEx** (const char *r) const
- int **RegEx_replace** (const char *F, int L, const char *rstr) const
- int **getCtr** (const char *F, int L, const **iTags** &b, **ctrldx** &ia, **ctrldx** &ib, **iTags** *c=NULL, **wbvector**< char > *ma=NULL, **wbvector**< char > *mb=NULL) const
- int **UpdateItagsCtr** (const char *F, int L, const **ctrldx** &ia, **iTags** &b, const **ctrldx** &ib, const char *tag)
- int **FlagItagsCtr** (const char *F, int L, const **ctrldx** &ia, **iTags** &B, const **ctrldx** &ib, char all=1, char sgn=0)
- bool **sameConj** (const **iTags** &b) const
- bool **isOp** (int r=-1, char lflag=0) const
- bool **isOpX** (unsigned r=-1) const
- bool **isAtensor** (unsigned r=-1) const
- bool **got_qdir** (const char *qdir, char fflag=0) const
- unsigned **SkipLast** (unsigned n=1)
- **wbstring toStr** (char vflag=1) const
- const char * **toStrk** (char *sout, unsigned l, unsigned k, const char *st="(itag?)", char cflag=0) const
- mxArray * **toMx** () const
- int **got_op_labels** (const char *F=NULL, int L=0) const
- unsigned **gotLabels** () const

Public Member Functions inherited from [wbvector< itag_ >](#)

- [wbvector](#) (size_t l, [itag_](#) x)
- [wbvector](#) (size_t l, const [itag_](#) *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & [init](#) (size_t l=0)
- [wbvector](#) & [init](#) (size_t l, const [itag_](#) *d0)
- [wbvector](#) & [init](#) (size_t l, const [itag_](#) *d0, char ref)
- [wbvector](#) & [init](#) (const [wbvector](#) &v)
- [wbvector](#) & [init](#) (size_t n, const [wbvector](#) &v)
- [wbvector](#)< [itag_](#) > & [init](#) (const [wbsparray](#)< [itag_](#) > &S)
- [wbvector](#) & [init](#) (size_t l1, const [itag_](#) *d1, size_t l2, const [itag_](#) *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const [itag_](#) *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const [itag_](#) &x)
- [wbvector](#) & [initStride](#) (size_t l, const [itag_](#) *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const [itag_](#) *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const [itag_](#) *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< [itag_](#) > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< [itag_](#) > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< [itag_](#) > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< [itag_](#) > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< [itag_](#) > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< [itag_](#) > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const [itag_](#) *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)
- [wbvector](#) & [operator=](#) (const [wbvector](#) &v)
- [wbvector](#)< [itag_](#) > & [operator=](#) (const [wbvector](#)< TB > &v)
- [wbvector](#) & [set](#) (const [itag_](#) &x)
- [wbvector](#) & [set](#) (const [wbvector](#)< size_t > &l, const [itag_](#) &x)
- void [set](#) (size_t offset, const [itag_](#) *x, size_t n)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & [setMin](#) (const [itag_](#) &x)
- [wbvector](#) & [setMax](#) (const [itag_](#) &x)
- [wbvector](#) & [add](#) (const [wbvector](#)< size_t > &l, const [itag_](#) &x)
- [wbvector](#) & [add](#) (const [itag_](#) *d)
- [wbvector](#) & [Set](#) (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void [Sort](#) ([wbperm](#) &p, char dir=+1)
- void [Sort](#) (char dir=+1)

- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const [itag_](#) &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const [itag_](#) &x) const
- size_t **find1** () const
- size_t **count** (const [itag_](#) &x) const
- size_t **icount** (const [itag_](#) &x) const
- size_t **nnz** () const
- size_t **nnz** ([itag_](#) eps) const
- size_t **numZeros** ([itag_](#) eps=[itag_](#)(1E-14)) const
- size_t **numEQ** (const [itag_](#) &) const
- size_t **numGT** (const [itag_](#) &) const
- size_t **numLT** (const [itag_](#) &) const
- [wbindex](#) & **find** (const [itag_](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const [itag_](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const [itag_](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const [itag_](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const [itag_](#) &x1, const [itag_](#) &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** ([itag_](#) r) const
- int **findClosest** ([itag_](#) r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const [itag_](#) *x)
- void **reset** ()
- void **cpy** (const [itag_](#) *v)
- void **cpy** (const [itag_](#) *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** ([itag_](#) &A1, [itag_](#) &A2)
- [wbvector](#) & **Collect** ([itag_](#) &A1, [itag_](#) &A2, [itag_](#) &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)

- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const [itag_](#) *b)
- [wbvector](#) & **Plus** (const [itag_](#) *b, [itag_](#) bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, [itag_](#) bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, [itag_](#) bfac=1, [itag_](#) cfac=0) const
- [wbvector](#) & **times** ([itag_](#) fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- [itag_](#) **sum** () const
- [itag_](#) **sum** (size_t i1, const char *estr) const
- [itag_](#) **sum** (size_t i1, size_t i2) const
- [itag_](#) **cumsum_** ([wbvector](#) &S, char xflag=0) const
- [itag_](#) **Cumsum** (char zflag=0) const
- [itag_](#) **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< [itag_](#) > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- [itag_](#) **scalarProd** (const [wbvector](#) &v) const
- [itag_](#) **scalarProd** (const [itag_](#) *v) const
- [itag_](#) **norm** () const
- [itag_](#) **norm2** () const
- double **normDiff** (const [wbvector](#) &v, [itag_](#) fac=1) const
- [itag_](#) **normDiff2** (const [wbvector](#) &v, [itag_](#) fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- [itag_](#) **GSProject** (const [itag_](#) *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const [itag_](#) *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const [itag_](#) &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const [itag_](#) &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const [itag_](#) &a, [itag_](#) eps=1E-15) const
- bool **allUnequal** (const [itag_](#) &a) const
- bool **any** () const
- bool **anyUnequal** (const [itag_](#) &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &I, const [itag_](#) &x, size_t *k=NULL) const
- bool **anyEqual** (const [itag_](#) &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &I, const [itag_](#) &x, size_t *k=NULL) const
- bool **anyGT** (const [itag_](#) &a) const
- bool **anyGE** (const [itag_](#) &a) const

- `bool anyLT (const itag_ &a) const`
- `bool anyLE (const itag_ &a) const`
- `bool allGT (const itag_ &a) const`
- `bool allGE (const itag_ &a) const`
- `bool allLE (const itag_ &a) const`
- `bool allLT (const itag_ &a) const`
- `bool allIn (const itag_ &a1, const itag_ &a2) const`
- `bool anySmallVals (const itag_ eps=1E-15) const`
- `bool isEqual (const itag_ *v) const`
- `bool isEqual (const wbvector &v) const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool operator== (const wbvector &v) const`
- `bool operator!= (const wbvector &v) const`
- `bool isEqualR (const wbvector &v) const`
- `bool operator^ (const wbvector &v) const`
- `bool deepEqualP (const wbvector &B) const`
- `char cmp (const wbvector &v) const`
- `bool operator< (const wbvector &v) const`
- `int operator< (const itag_ &r) const`
- `bool operator<= (const wbvector &v) const`
- `bool operator> (const wbvector &v) const`
- `bool operator>= (const wbvector &v) const`
- `itag_ &el (size_t i) const`
- `itag_ elx (size_t i, const itag_ &x) const`
- `const itag_ &operator[] (size_t i) const`
- `itag_ &operator[] (size_t i)`
- `wbvector operator[] (const wbvector< size_t > &l) const`
- `itag_ &last () const`
- `itag_ &last (size_t &i) const`
- `itag_ &last (T1 &i_) const`
- `itag_ &p2last () const`
- `itag_ *ref (size_t i) const`
- `wbvector &setRand (double fac=1., double shift=0.)`
- `wbvector< wbcomplex > &setRand (double fac, double shift)`
- `void setIdx (double shift=0., double fac=1.)`
- `int set2Group (const wbperm &P, const WBINDEX &D, const itag_ *S0=NULL, char iflag=0)`
- `wbvector &Resize_ (size_t n)`
- `wbvector &Shorten2 (size_t n)`
- `wbvector &Resize (size_t n, const itag_ *d=NULL)`
- `wbvector &resize (size_t n, wbvector &X) const`
- `wbvector &cat (const wbvector &v2)`
- `wbvector &Cat (const wbvector &v1, const wbvector &v2)`
- `wbvector &Cat (const wbvector< wbvector > &vv)`
- `wbvector &Cat (const wbvector< wbvector * > &vv)`
- `wbvector &Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)`
- `wbvector &Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3=0, const wbvector< T3 > *v4=0)`
- `wbvector< itag_ > &Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3, const wbvector< T3 > *v4)`
- `wbvector< itag_ > &Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)`
- `wbvector &Append (const wbvector &v2)`

- [wbvector](#) & **Append** (const [itag_](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [itag_](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const [itag_](#) *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const [itag_](#) *v) const
- int **recLessE** (const [itag_](#) *v) const
- int **recCompare** (const [itag_](#) *v) const
- [itag_](#) **max** () const
- [itag_](#) **max** (size_t *k) const
- [itag_](#) **max** (unsigned &k) const
- [itag_](#) **max_** ([itag_](#) x) const
- [itag_](#) **min** () const
- [itag_](#) **min** (size_t *k) const
- [itag_](#) **min** (unsigned &k) const
- [itag_](#) **min_** ([itag_](#) x) const
- [itag_](#) **aMax** (size_t *k=NULL) const
- [itag_](#) **aMax** (unsigned &k) const
- [itag_](#) **aMin** (char zflag=0, size_t *k=NULL) const
- [itag_](#) **maxneg** (size_t *k=NULL) const
- [itag_](#) **minpos** (size_t *k=NULL) const
- [itag_](#) **avg** () const
- [itag_](#) **std** () const
- void **range** ([itag_](#) &xmin, [itag_](#) &xmax) const
- [itag_](#) **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, [itag_](#) *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, [itag_](#) *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getI** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getI** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- [itag_](#) **prod** () const
- [itag_](#) **prod** ([itag_](#) x) const
- [itag_](#) **prod** (size_t i, size_t i2) const
- [itag_](#) **prod** (size_t i, size_t i2, [itag_](#) x) const
- [itag_](#) **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** ([itag_](#) &p1, [itag_](#) &p2, const [itag_](#) &p0=0) const

- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const [itag_](#) &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< itag_ >](#)

- [itag_](#) * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< itag_ >](#)

- [wbvector](#) & **RENEW** (size_t n, const [itag_](#) *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const [itag_](#) &x)

The documentation for this class was generated from the following files:

- [wbindex.hh](#)
- [wbindex.cc](#)

4.39 Wb::iterator< T, T2 > Class Template Reference

Public Member Functions

- **iterator** (T *p_)
- **iterator** & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- T2 & **operator[]** (size_t k)
- T2 & **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.40 Wb::iterator< T, T2 * > Class Template Reference

Public Member Functions

- **iterator** (T *p_)
- **iterator** & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- T2 * **operator[]** (size_t k)
- T2 * **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.41 IterOM_DATA< TD > Class Template Reference

Public Member Functions

- template<class TQ >
IterOM_DATA (const [QSpace](#)< TQ, TD > &A, unsigned i)
- void **begin** ()
- bool **end** ()
- [IterOM_DATA](#) & **operator++** ()
- [wbarrray](#)< TD > **DATA** ()

Public Attributes

- unsigned **iter**
- unsigned **niter**
- size_t **nS**
- TD * **data0**
- [wbarrray](#)< TD > **X**

The documentation for this class was generated from the following file:

- QSpace.hh

4.42 iterSparseCMref< TD > Class Template Reference

Public Member Functions

- **iterSparseCMref** (const char *F, int L, [wbsparray](#)< TD > &A)
- SPIDX_T **next** ([wbsparray](#)< TD > &a)
- [wbsparray](#)< TD > & **getIter** (SPIDX_T i, [wbsparray](#)< TD > &a, char ref=0) const
- TD **norm2** (SPIDX_T i, char tnorm=0) const
- SPIDX_T **numIter** () const
- bool **gotLast** () const
- SPIDX_T **index** () const
- mxArray * **toMx** () const

Public Attributes

- SPIDX_T **it**
- [wbvector](#)< SPIDX_T > **I0**
- [wbvector](#)< SPIDX_T > **IX**
- [wbMatrix](#)< SPIDX_T > **IDX**

The documentation for this class was generated from the following file:

- wbsparray.hh

4.43 Wb::line_stat__ Class Reference

Public Member Functions

- void **add** (double dt_, double dx)

Public Attributes

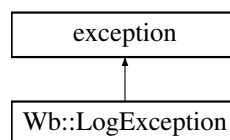
- double **dt**
- double **aux**
- size_t **ncall**

The documentation for this class was generated from the following file:

- wblock.hh

4.44 Wb::LogException Struct Reference

Inheritance diagram for Wb::LogException:



Public Member Functions

- **LogException** (WBLOG_TYPE l=OK, const char *s=NULL)
- **LogException** (const [LogException](#) &e)
- [LogException](#) & **operator=** (const [LogException](#) &e)
- [LogException](#) & **operator++** ()
- [LogException](#) & **operator+=** (const [LogException](#) &e)
- bool **operator!** () const
- **operator bool** () const
- [LogException](#) & **init** ()
- const char * **what** () const throw ()
- [wstring](#) **toStr** () const
- void **report** (const char *F=0, int L=0, const char *func=0)

Public Attributes

- WBLOG_TYPE **type**
- int * **nrefs**
- char **istr** [64]
- unsigned **count**
- int **ith**
- int **nth**

The documentation for this struct was generated from the following files:

- wblog.h
- wblog.c

4.45 Wb::matFile Class Reference

Public Member Functions

- **matFile** (const char *F, int L, const char *fname, const char *rw)
- **matFile** (const [matFile](#) &X)
- void **close** ()
- void **init** ()
- int **Open** (const char *F, int L, const char *FName, const char *RW)
- int **open** (const char *file, int line, const char *FName, const char *RW, char xflag=0)
- [matFile](#) & **put** (const char *file, int line, const char *vname, const mxArray *a)
- [matFile](#) & **put** (const char *vname, const mxArray *a)
- int **existsVar** (const char *vn) const
- int **deleteVar** (const char *vn) const
- int **deleteVar** (const char *f, int L, const char *vn) const

Public Attributes

- MATFile * **mfp**
- [wbstring](#) **F**
- [wbstring](#) **fname**
- [wbstring](#) **rw**
- int **L**

The documentation for this class was generated from the following file:

- wbMatIO.hh

4.46 MEM_SET< T > Class Template Reference

Public Member Functions

- **MEM_SET** (T *data __attribute__((unused)), size_t len __attribute__((unused)), const T *a __attribute__((unused))=NULL)
- **MEM_SET** (int *data, size_t len, const int *a)
- **MEM_SET** (unsigned *data, size_t len, const unsigned *a)
- **MEM_SET** (unsigned long *data, size_t len, const unsigned long *a)
- **MEM_SET** (long *data, size_t len, const long *a)
- **MEM_SET** (char *data, size_t len, const char *a)
- **MEM_SET** (double *data, size_t len, const double *a)
- **MEM_SET** (float *data, size_t len, const float *a)
- **MEM_SET** ([wbcomplex](#) *data, size_t len, const [wbcomplex](#) *a)

The documentation for this class was generated from the following file:

- memlib.hh

4.47 MEM_SET< T * > Class Template Reference

Public Member Functions

- **MEM_SET** (T **data, size_t len, T *const *a=NULL)

The documentation for this class was generated from the following file:

- memlib.hh

4.48 MHash< TM > Class Template Reference

Public Member Functions

- size_t **operator()** (const [cgc_contract_id](#)< TM > &S) const

The documentation for this class was generated from the following file:

- clebsch.hh

4.49 Wb::mpfr__< P > Class Template Reference

Public Member Functions

- **mpfr__** (double val=0)
- **mpfr__** (const char *F, int L, double val)
- **mpfr__** (const [mpfr__](#) &b)
- void **Cleanup** (const char *F=NULL, int L=0) const
- [mpfr__](#) & **init** (const char *F, int L, double val, char lflag=0)
- [mpfr__](#) & **init_s** (const char *F, int L, const char *s, unsigned base=10)
- [mpfr__](#) & **init_d** (const char *F, int L, double val, char dcheck=1)
- [mpfr__](#) & **set_pi** ()
- [mpfr__](#) & **set_log2** ()
- [mpfr__](#) & **set_e** ()
- [mpfr__](#) & **set_nan** ()
- [mpfr__](#) & **set_inf** (int s=+1)
- [mpfr__](#) & **set_zero** (int s=+1)
- int **isnan** () const
- int **isinf** () const
- int **iszero** () const
- int **isnumber** () const
- int **isnonzero** () const
- int **sign** ()
- void **swap** ([mpfr__](#)< P > &b)
- int **cmp** (const [mpfr__](#) &b) const
- int **cmp** (const double &val) const
- int **cmp** (const int &val) const
- int **cmp** (const long &val) const

- int **cmp** (const unsigned &val) const
- int **cmp** (const unsigned long &val) const
- bool **operator>** (const [mpfr__](#) &x) const
- bool **operator>** (const double &x) const
- bool **operator>** (const int &x) const
- bool **operator>** (const long &x) const
- bool **operator>** (const unsigned &x) const
- bool **operator>** (const unsigned long &x) const
- bool **operator<** (const [mpfr__](#) &x) const
- bool **operator<** (const double &x) const
- bool **operator<** (const int &x) const
- bool **operator<** (const long &x) const
- bool **operator<** (const unsigned &x) const
- bool **operator<** (const unsigned long &x) const
- bool **operator>=** (const [mpfr__](#) &x) const
- bool **operator>=** (const double &x) const
- bool **operator>=** (const int &x) const
- bool **operator>=** (const long &x) const
- bool **operator>=** (const unsigned &x) const
- bool **operator>=** (const unsigned long &x) const
- bool **operator<=** (const [mpfr__](#) &x) const
- bool **operator<=** (const double &x) const
- bool **operator<=** (const int &x) const
- bool **operator<=** (const long &x) const
- bool **operator<=** (const unsigned &x) const
- bool **operator<=** (const unsigned long &x) const
- bool **operator!=** (const [mpfr__](#) &x) const
- bool **operator!=** (const double &x) const
- bool **operator!=** (const int &x) const
- bool **operator!=** (const long &x) const
- bool **operator!=** (const unsigned &x) const
- bool **operator!=** (const unsigned long &x) const
- bool **operator==** (const [mpfr__](#) &x) const
- bool **operator==** (const double &x) const
- bool **operator==** (const int &x) const
- bool **operator==** (const long &x) const
- bool **operator==** (const unsigned &x) const
- bool **operator==** (const unsigned long &x) const
- [mpfr__](#) & **operator=** (double val)
- [mpfr__](#) & **operator=** (const [mpfr__](#) &b)
- [mpfr__](#) & **times** (const [mpfr__](#) &b, [mpfr__](#) &c) const
- [mpfr__](#) & **divby** (const [mpfr__](#) &b, [mpfr__](#) &c) const
- [mpfr__](#) & **operator*=** (const [mpfr__](#) &b)
- [mpfr__](#) & **operator/=** (const [mpfr__](#) &b)
- [mpfr__](#) & **operator+=** (const [mpfr__](#) &b)
- [mpfr__](#) & **operator-=** (const [mpfr__](#) &b)
- [mpfr__](#) & **operator*=** (double val)
- [mpfr__](#) & **operator/=** (double val)
- [mpfr__](#) & **operator+=** (double val)
- [mpfr__](#) & **operator-=** (double val)
- [mpfr__](#) **operator*** (const [mpfr__](#) &b) const
- [mpfr__](#) **operator/** (const [mpfr__](#) &b) const
- [mpfr__](#) **operator+** (const [mpfr__](#) &b) const
- [mpfr__](#) **operator-** (const [mpfr__](#) &b) const
- [mpfr__](#) **operator*** (int n) const

- [mpfr__ operator/](#) (int n) const
- [mpfr__ operator+](#) (int n) const
- [mpfr__ operator-](#) (int n) const
- [mpfr__ operator*](#) (unsigned n) const
- [mpfr__ operator/](#) (unsigned n) const
- [mpfr__ operator+](#) (unsigned n) const
- [mpfr__ operator-](#) (unsigned n) const
- [mpfr__ operator*](#) (long n) const
- [mpfr__ operator/](#) (long n) const
- [mpfr__ operator+](#) (long n) const
- [mpfr__ operator-](#) (long n) const
- [mpfr__ operator*](#) (unsigned long n) const
- [mpfr__ operator/](#) (unsigned long n) const
- [mpfr__ operator+](#) (unsigned long n) const
- [mpfr__ operator-](#) (unsigned long n) const
- [mpfr__ & Sqrt](#) ()
- [mpfr__ & Exp](#) ()
- [mpfr__ & Log](#) ()
- [mpfr__ & Log2](#) ()
- [mpfr__ & Atan](#) ()
- [mpfr__ & Tanh](#) ()
- [mpfr__ & Atanh](#) ()
- [mpfr__ & Rand](#) ()
- [mpfr__ & Log10](#) ()
- [mpfr__ & Round](#) ()
- [mpfr__ sqrt](#) () const
- [mpfr__ exp](#) () const
- [mpfr__ log](#) () const
- [mpfr__ log2](#) () const
- [mpfr__ atan](#) () const
- [mpfr__ log10](#) () const
- [mpfr__ round](#) () const
- [operator double](#) () const
- [operator wbcomplex](#) () const
- [operator bool](#) () const
- [double getval_d](#) () const
- [bool operator<](#) (double val) const
- [bool operator<=](#) (double val) const
- [bool operator>](#) (double val) const
- [bool operator>=](#) (double val) const
- [int prec](#) (unsigned base=2) const
- [mpfr__ eps](#) ()
- [mpfr__ & FlipSign](#) ()
- [mpfr__ & Abs](#) ()
- [mpfr__ abs](#) () const
- [mpfr__ fabs](#) () const
- [mpfr__ abs2](#) () const
- [void print](#) (const char *vname="", const char *istr=0, char sep=0) const
- [size_t to_str](#) (char *s, size_t n_, int base=10) const
- [wbstring to_str](#) (int base=10) const
- [void toStr](#) (char *s, unsigned n, int base=10, int nd=-1, char align=0) const
- [wbstring & toStr](#) ([wbstring](#) &sout, int base=10, int nd=-1, char align=0) const

Public Attributes

- `mpfr_t f`

Static Public Attributes

- static unsigned long **num_vars** =0

The documentation for this class was generated from the following files:

- `wbmpfr.hh`
- `wbmpfr.cc`

4.50 Wb::MREC Class Reference

Public Member Functions

- `template<class T >`
`const char * init (const char *F __attribute__((unused)), int L __attribute__((unused)), size_t i, const T *p`
`__attribute__((unused)), size_t l)`
- `void println (size_t i, void *p) const`

Public Attributes

- `size_t id`
- `size_t len`
- `unsigned unit`

The documentation for this class was generated from the following file:

- `memtrack.cc`

4.51 mxArray_buf Class Reference

Public Member Functions

- `mxArray_buf (const char *n=NULL)`
- `void add (void *p, mxArray *a)`
- `mxArray * find (const char *F, int L, void *p)`
- `mxArray * Return (const char *F, int L, void *p)`
- `int erase (void *p)`

Public Attributes

- unordered_map< void *, mxArray * > **BUF**
- char * **name**

The documentation for this class was generated from the following file:

- mexlib.hh

4.52 MXPut Class Reference

Public Member Functions

- **MXPut** (const char *vn=0, const char *ws=0)
- **MXPut** (const char *file, int line, const char *vn="ans", const char *ws=0)
- void **init** ()
- void **init** (const char *file, int line, const char *vn=NULL)
- void **put** (const char *ws=0, const char *vn=0)
- void **save2** (mxArray *&S0)
- mxArray * **toMx** ()
- void **add2Struct** (mxArray *S0)
- **MXPut** & **addFL** ()
- template<class T >
MXPut & **add** (const T &x, const char *fn)
- template<class T >
MXPut & **addP** (const T *x, const char *fn)
- template<> **MXPut** & **addP** (const mxArray *a, const char *fn)
- template<> **MXPut** & **addP** (const char *s, const char *fn)
- template<> **MXPut** & **add** (const double &x, const char *fn)
- template<> **MXPut** & **add** (const float &x, const char *fn)
- template<> **MXPut** & **add** (const long double &x, const char *fn)
- template<> **MXPut** & **add** (const **wbcomplex** &x, const char *fn)
- template<> **MXPut** & **add** (const char &x, const char *fn)
- template<> **MXPut** & **add** (char *const &s, const char *fn)
- template<> **MXPut** & **add** (const char *const &s, const char *fn)
- template<> **MXPut** & **add** (const unsigned &x, const char *fn)
- template<> **MXPut** & **add** (const int &x, const char *fn)
- template<> **MXPut** & **add** (const bool &x, const char *fn)
- template<> **MXPut** & **add** (const long &x, const char *fn)
- template<> **MXPut** & **add** (const unsigned long &x, const char *fn)

Public Attributes

- mxArray * **S**
- char **wsp** [16]
- char * **vname**
- char * **F**
- unsigned **L**

Static Public Attributes

- static const unsigned **wlen** =16

The documentation for this class was generated from the following file:

- mexlib.hh

4.53 mxStruct Class Reference

Public Member Functions

- **mxStruct** (const char *file, int line)
- void **init** (const char *file, int line)
- void **put** (const char *name)
- void **save2Struct** (mxArray *S0, const char *name)
- void **checkNewField** (const char *f=NULL, int l=0)
- **mxStruct** & **operator()** (const char *fn)
- template<class T >
 mxStruct & **operator=** (const T &x)
- **mxStruct** & **operator=** (mxArray *a)
- template<class T >
 mxStruct & **operator=** (const **wbvector**< T > &x)
- template<class T >
 mxStruct & **operator=** (const **wbMatrix**< T > &x)
- template<> **mxStruct** & **operator=** (const **wbperm** &P)
- template<> **mxStruct** & **operator=** (const **wbindex** &I)

Public Attributes

- mxArray * **S**
- **wbstring** **F**
- **wbstring** **fld**
- unsigned **L**

The documentation for this class was generated from the following file:

- mexlib.hh

4.54 Wb::myname__ Class Reference

Public Attributes

- char **data** [64]

The documentation for this class was generated from the following file:

- wblib.h

4.55 NRGData< TQ, TD > Class Template Reference

Public Member Functions

- **NRGData** (const char *s=NULL)
- **NRGData** (const [NRGData](#) &A)
- void **initName** (const char *n)
- void **init** (const char *F, int L, const mxArray *a, [wbvector](#)< [QSpace](#)< TQ, TD > > &C0, int k=0, char force=0, const char *vname0=0)
- void **init** (const char *F, int L, const char *XX, int iter=-1)
- void **initXX** (const char *F, int L, int m=4, int iter=-1)
- void **initX** (const char *F, int L, int m=2, int iter=-1)
- void **init** (char flag=0)
- void **setupMX** (const char *F, int L, int N, char store_=1, char locMX_=1)
- void **PUT** (const char *F, int L, const char *vn, const char *ws="caller")
- void **check_dloc** (const char *F, int L, const char *fun, unsigned dloc, unsigned iter) const
- template<class TD_ >
int **checkOpConsistency** (const char *F, int L, const [NRGData](#)< TQ, TD_ > &B) const
- bool **isQSpaceVec** (const char *nm, int iter=-1)
- bool **checkVec** (const char *F, int L, const char *XX, unsigned R, const char *istr="")
- bool **checkVecVec** (const char *XX, unsigned r)
- void **forceCalc** ()
- void **getRelevantOps** ([wbindex](#) &l) const
- [wbvector](#)< [QSpace](#)< TQ, TD > > & **getOpsXX** (const char *t)
- const [QSpace](#)< TQ, TD > & **getQSpace** (const char *t, unsigned i=0) const
- [QSpace](#)< TQ, TD > & **getQSpace** (const char *t, unsigned i=0)
- const [QSpace](#)< TQ, TD > & **getQSpace** (char t1, char t2, unsigned i=0) const
- [QSpace](#)< TQ, TD > & **getQSpace** (char t1, char t2, unsigned i=0)
- const [QSpace](#)< TQ, TD > & **getQSpace** (char t) const
- [QSpace](#)< TQ, TD > & **getQSpace** (char t)
- void **info** (const char *vstr=NULL) const
- void **displdx** (const char *F, int L) const
- void **info** (const char *F, int L, const char *istr=" * ") const
- void **disp_otype** (const char *F=0, int L=0) const
- const mxArray * **getMxInfo** (const char *n)
- template<class TX >
void **getMxInfo** (const char *F, int L, const char *v, TX &X)
- mxArray * **getMxData** (const char *nm, int iter=-1, char ionly=0)
- const char * **wrng** ()
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base")
- void **put** (const char *F, int L, const char *vname, const char *ws="base")
- template<class T >
int **getPara** (const char *name, T &x, int iter=-1, char i=0)
- bool **opExists** (char tflag=1)
- void **initOp** (unsigned n=1, char calc_=1, char store_=0)
- bool **initOp** (const char *F, int L, const [wbvector](#)< [QSpace](#)< TQ, TD > > &C0, char tflag=1, [NRGData](#)< TQ, TD > *CR=NULL, const char *istr="")
- bool **initOp** (const char *F, int L, const [wbvector](#)< [wbvector](#)< [QSpace](#)< TQ, TD > > &CC, [NRGData](#)< TQ, TD > *CR=NULL, const char *istr="", char tflag=1)
- void **initSym** (const char *tag)
- void **updatePara** (const char *F, int L, const char *tag, mxArray *, int iter=-1) const
- void **updatePara** (const char *F, int L, const char *tag, int iter=-1, const char *tag2=NULL, char all=-1) const

- void **updateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A, const [NRGData](#)< TQ, TD > &B, unsigned nop, int iter=-1)
- template<class TC >
void **updateOpProd** (const char *F, int L, unsigned iter, const [NRGData](#)< TQ, TD > &B, const [NRGData](#)< TQ, TC > &C)
- void **updateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A, unsigned nop, int iter=-1)
- template<class TA >
void **backPropagateOp** (const char *F, int L, const [NRGData](#)< TQ, TA > &A0, const [NRGData](#)< TQ, TA > &A2)
- template<class TD_ >
void **initBG** (const char *F, int L, const [NRGData](#)< TQ, TD_ > &B, const [NRGData](#)< TQ, TD_ > &H1, const [NRGData](#)< TQ, TD_ > &H2, int m=3)
- void **unsetAllRefs** ()
- unsigned **skipOpZeros** (double eps=DEPS, char bflag='b')
- void **saveOp** (const char *, int, int iter=-1, unsigned m=4) const
- void **times** (TD fac, const char *flag)
- void **setupIO** (const char *, int, const char *, const mxArray *, const char *istr=0)
- void **updateInfo** (const char *F, int L, const char *, mxArray *, char keep=-1) const
- void **updateInfoP** (const char *F, int L, const char *vtag, mxArray *, char keep=-1) const
- const char * **getFileName** (unsigned k, char lenient=0) const
- template<class TD >
void **updateOpProd** (const char *F, int L, unsigned iter, const [NRGData](#)< TQ, TS > &B, const [NRGData](#)< TQ, TD > &C)
- template<class TD >
void **initBG** (const char *F, int L, const [NRGData](#)< TQ, TD > &B, const [NRGData](#)< TQ, TD > &H0, const [NRGData](#)< TQ, TD > &H2, int m)
- template<class TD >
void **backPropagateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A0, const [NRGData](#)< TQ, TD > &A2)

Public Attributes

- [wbstring](#) **name**
- [wbvector](#)< [NRGIndex](#)< TQ, TD > > **nrgIdx**
- char **calc**
- char **store**
- int **kloc**
- char **locMX**
- [wbvector](#)< [wbvector](#)< [QSpace](#)< TQ, TD > > > **CI**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **KK**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **KD**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **DK**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **DD**
- [QSpace](#)< TQ, TD > **K**
- [QSpace](#)< TQ, TD > **D**
- [wbvector](#)< char > **issym**
- mxArray * **MX**
- [wbstring](#) **NAME**
- mxArray * **aux**

The documentation for this class was generated from the following file:

- nrgdata.hh

4.56 NRGIndex< TQ, TD > Class Template Reference

Public Member Functions

- **NRGIndex** (const [NRGIndex](#) &l)
- void **init** ()
- void **init** (int i, [NRGData](#)< TQ, TD > *d)
- void **init** (int i, const char *t)
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tst=0) const
- void **put** (const char *vname, const char *ws="base") const

Public Attributes

- int **idx**
- [NRGData](#)< TQ, TD > * **nrg**
- [wbstring](#) **tag**

The documentation for this class was generated from the following file:

- nrgdata.hh

4.57 Wb::num2Fmt< T > Class Template Reference

Public Member Functions

- **num2Fmt** (const char *F, int L, int m_=-2, int p_=-2, const char *t_="")
- int **check_init** ()
- **operator wbstring** () const

The documentation for this class was generated from the following file:

- gathered.hh

4.58 opFlags< T > Class Template Reference

Public Member Functions

- **opFlags** (const [opFlags](#) &f)
- **opFlags** (const char *F, int L, char f)
- **opFlags** (const [opFlags](#) &f1, const [opFlags](#) &f2)
- [opFlags](#)< T > & **operator=** (char f)
- [opFlags](#)< T > & **operator=** (const [opFlags](#)< T > &x)
- **opFlags** (char f)
- **operator char** ()
- [opFlags](#)< T > & **init** (const char *F, int L, char f)
- [opFlags](#)< T > & **init** (char f)
- [opFlags](#)< T > & **init** (const [opFlags](#)< T > &f1, const [opFlags](#)< T > &f2)
- char **trans** () const
- char **conj** () const
- char **idle** () const
- void **applyOrRef** (const char *F, int L, const [wbararray](#)< T > &A0, [wbararray](#)< T > &A, const T fac=1) const
- template<class T2 >
T2 & **applyOrRef** (const char *F, int L, const T2 &A0, const T fac=1) const
- template<class T2 >
const T2 & **applyConjOrRef** (const char *F, int L, const T2 &A0, const T fac=1) const
- template<class T2 >
char **apply** (const char *F, int L, T2 &A, const T fac=1) const
- char **toChar** () const
- mxArray * **toMx** () const
- void **info** (const char *F, int L, const char *istr="")

Public Attributes

- char **p**

The documentation for this class was generated from the following file:

- [wbararray_blas.hh](#)

4.59 OPTS Class Reference

Public Member Functions

- **OPTS** (const mxArray **ain, const unsigned n)
- **OPTS** (const char *fname)
- void **init** (const mxArray **ain, const unsigned n)
- void **init** (const char *fname)
- void **init** ()
- int **len** ()
- template<class T >
bool **getOpt** (const char *, int, const char *vn, T &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *, int, const char *vn, [wbvector](#)< T > &x, char force=0, const char *istr=0)

- `template<class T >`
`bool getOpt (const char *, int, const char *vn, wbMatrix< T > &x, char force=0, const char *istr=0)`
- `bool getOpt (const char *, int, const char *vn, const char *istr=0)`
- `template<class T >`
`bool getOpt (const char *vn, T &x, char force=0, const char *istr=0)`
- `template<class T >`
`bool getOpt (const char *vn, wbvector< T > &x, char force=0, const char *istr=0)`
- `template<class T >`
`bool getOpt (const char *vn, wbMatrix< T > &x, char force=0, const char *istr=0)`
- `bool getOpt (const char *vn, const char *istr=0)`
- `void checkAnyLeft (const char *MAT=NULL, int L=0)`
- `template<> bool getOpt (const char *F, int L, const char *vn, wbstring &x, char force, const char *istr)`
- `template<> bool getOpt (const char *F, int L, const char *vn, mxArray *&a, char force, const char *istr)`
- `template<> bool getOpt (const char *F __attribute__((unused)), int L __attribute__((unused)), const char *vn, itag\_ &t, char force, const char *istr)`

The documentation for this class was generated from the following file:

- `wbopts.hh`

4.60 pairPatternCG< T > Class Template Reference

Public Member Functions

- `pairPatternCG & init (unsigned i0=0, unsigned j0=0, unsigned n=0)`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i) const`

Public Attributes

- unsigned `i`
- unsigned `j`
- `wbvector< unsigned > k`
- `wbvector< T > fac`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_io.cc`

4.61 Wb::Path Class Reference

Public Member Functions

- `Path (const char *s)`
- `Path (const char *F, int L, const char *s)`
- `int init ()`
- `int init (const char *F, int L, const char *s, const char *delim=":", unsigned n_=-1)`
- `int append (const char *F, int L, const char *s, const char *delim=":")`
- `char * operator[] (unsigned i)`
- `const char * operator[] (unsigned i) const`
- `const char * last () const`
- `unsigned numel () const`
- `Wb::iterator< Path, char * > begin ()`
- `Wb::citerator< Path, char * > cbegin () const`
- `void print (const char *istr="") const`

Public Attributes

- [wbstring](#) **buf**
- [wbvector](#)< unsigned > **idx**

The documentation for this class was generated from the following file:

- [wbutil.hh](#)

4.62 Wb::Profile Class Reference**Public Member Functions**

- **Profile** (const char *F, int L, const char *istr=0)
- void **tic** ()
- void **toc** (int L, double x=0)

Public Attributes

- CHRONO_CLOCK **tref**

The documentation for this class was generated from the following file:

- [wbclock.hh](#)

4.63 QBlock< TQ, TD, TE > Class Template Reference**Public Member Functions**

- [QBlock](#) & **init** ()
- [QBlock](#) & **operator=** (const [QBlock](#) &B)
- [QBlock](#) & **init_bare_refA** (const [QBlock](#) &B, unsigned w=0)
- void **toBlockMatrix** ([wbarray](#)< TD > &MM, unsigned K, const [wbperm](#) &P=[wbperm](#)())
- void **initFromBlockMatrix** (const char *F, int L, [QSpace](#)< TQ, TD > &B, const [wbarray](#)< TD > &MM, const [iTags](#) &it, const [wbperm](#) *cgp=NULL) const
- void **updateBlockDim** (unsigned dim2)
- double **get_s2t** (const [wbvector](#)< char > &mark)
- void **print** (const char *F=NULL, int L=0, const [iTags](#) *t=NULL) const
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="caller")

Public Attributes

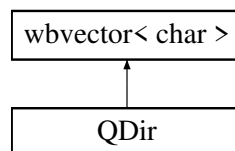
- unsigned **qdim_tot**
- [wbvector](#)< unsigned > **qdim**
- [QSpace](#)< TQ, TD > **A**
- [wbindex](#) **l0**
- [wbindex](#) **lk**
- [wbindex](#) **lt**
- [wbMatrix](#)< TQ > **Q1**
- [wbMatrix](#)< TQ > **Q2**
- [wbMatrix](#)< widx_t > **S1**
- [wbMatrix](#)< widx_t > **S2**
- [wbMatrix](#)< widx_t > **D1**
- [wbMatrix](#)< widx_t > **D2**
- [wbMatrix](#)< int > **idx**
- [wbararray](#)< TD > **U**
- [wbararray](#)< TD > **Vc**
- [wbvector](#)< TE > **S**

The documentation for this class was generated from the following files:

- mpsortho.hh
- mpsortho.cc

4.64 QDir Class Reference

Inheritance diagram for QDir:

**Public Member Functions**

- **QDir** (unsigned l, const char *d=NULL, char ref=0)
- **QDir** (const [iTags](#) &it)
- **QDir** (const char *F, int L, const char *s)
- **QDir** (const char *s)
- **QDir** (const char *F, int L, const mxArray *a)
- **QDir** & **init** (unsigned l=0, const char *d=NULL, char ref=0)
- **QDir** & **operator=** (const char *s)
- **QDir** & **init** (const char *F, int L, const char *s)
- **QDir** & **init** (const char *s)
- **QDir** & **init** (const [iTags](#) &it)
- **QDir** & **init** (const char *F, int L, const mxArray *a)
- **QDir** & **init_iout** (const char *F, int L, unsigned r, unsigned iout)
- unsigned **isSorted** () const
- bool **operator==** (const char *s) const

- bool **operator!=** (const char *s) const
- bool **operator==** (const [QDir](#) &q) const
- bool **operator!=** (const [QDir](#) &q) const
- bool **operator<** (const [QDir](#) &q) const
- [QDir](#) & **Conj** ()
- char & **Conj** (const unsigned &i)
- unsigned **nconj** ()
- unsigned **nconj** (const [ctrlIdx](#) &lx)
- int **find1** () const
- [wstring](#) **toStr** () const
- [wstring](#) **toTag** () const
- mxArray * **toMx** () const

Public Member Functions inherited from [wbvector< char >](#)

- [wbvector](#) (size_t l, char x)
- [wbvector](#) (size_t l, const char *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const char *d0)
- [wbvector](#) & **init** (size_t l, const char *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & **init** (const [wbsparray](#)< char > &S)
- [wbvector](#) & **init** (size_t l1, const char *d1, size_t l2, const char *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const char *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const char &x)
- [wbvector](#) & **initStride** (size_t l, const char *d0, size_t stride)
- [wbvector](#) & **init_stride** (const char *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const char *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< char > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< char > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< char > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const char *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpfr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)

- **wbvector** & **operator=** (const **wbvector** &v)
- **wbvector**< char > & **operator=** (const **wbvector**< TB > &v)
- **wbvector** & **set** (const char &x)
- **wbvector** & **set** (const **wbvector**< size_t > &l, const char &x)
- void **set** (size_t offset, const char *x, size_t n)
- void **set** (const **wbvector**< double > &R, const **wbvector**< double > &l)
- void **set** (const **wbvector**< double > &R, const **wbvector**< double > &l)
- **wbvector** & **setMin** (const char &x)
- **wbvector** & **setMax** (const char &x)
- **wbvector** & **add** (const **wbvector**< size_t > &l, const char &x)
- **wbvector** & **add** (const char *d)
- **wbvector** & **Set** (const **wbvector**< size_t > &l, const **wbvector** &v)
- void **Sort** (**wbperm** &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** (**wbvector** &v2, **wbperm** &p) const
- void **sort** (**wbvector** &v2) const
- **wbperm** & **pSort** (**wbperm** &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const char &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const char &x) const
- size_t **find1** () const
- size_t **count** (const char &x) const
- size_t **icount** (const char &x) const
- size_t **nnz** () const
- size_t **nnz** (char eps) const
- size_t **numZeros** (char eps=char(1E-14)) const
- size_t **numEQ** (const char &) const
- size_t **numGT** (const char &) const
- size_t **numLT** (const char &) const
- **wbindex** & **find** (const char &x, **wbindex** &l, char iflag=0) const
- **wbindex** & **find** (const char &x, **wbindex** &l, **wbindex** &lx) const
- **wbindex** & **find** (**wbindex** &l) const
- **wbindex** & **find** (**wbindex** &l, **wbindex** &lx) const
- **wbvector**< size_t > **find** () const
- **wbindex** & **findGT** (const char &x, **wbindex** &l, char iflag=0) const
- **wbindex** & **findGT** (const char &x, **wbindex** &l, **wbindex** &lx) const
- **wbindex** & **findzeros** (**wbindex** &l) const
- **wbindex** & **findRange** (const char &x1, const char &x2, **wbindex** &l, const char *w=NULL) const
- int **findClosestSorted** (char r) const
- int **findClosest** (char r) const
- size_t **findValues** (const **wbvector** &v, **wbindex** &l) const
- **wbvector** & **setp** (const char *x)
- void **reset** ()
- void **cpy** (const char *v)
- void **cpy** (const char *x, size_t m)
- **wbvector** & **save2** (**wbvector** &v)
- void **swap** (**wbvector** &v)
- **wbvector** & **swap** (size_t i, size_t j)
- **wbvector** & **Collect** (char &A1, char &A2)

- [wbvector](#) & **Collect** (char &A1, char &A2, char &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &I, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &I)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &I)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const char *b)
- [wbvector](#) & **Plus** (const char *b, char bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, char bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, char bfac=1, char cfac=0) const
- [wbvector](#) & **times** (char fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- char **sum** () const
- char **sum** (size_t i1, const char *estr) const
- char **sum** (size_t i1, size_t i2) const
- char **cumsum_** ([wbvector](#) &S, char xflag=0) const
- char **Cumsum** (char zflag=0) const
- char **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< char > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- char **scalarProd** (const [wbvector](#) &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const [wbvector](#) &v, char fac=1) const
- char **normDiff2** (const [wbvector](#) &v, char fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- char **GSProject** (const char *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const char *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()

- [wbvector](#) & **applyFlag** (char &flag, const char &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const
- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const char &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- char & **el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (TI &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)

- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< char > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const char *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const
- char **max_** (char x) const
- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const
- char **avg** () const
- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, char *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, char *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const

- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const char &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< char >](#)

- char * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< char >](#)

- [wbvector](#) & **RENEW** (size_t n, const char *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const char &x)

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc

4.65 QHash< TQ > Class Template Reference

Public Member Functions

- size_t **operator()** (const [QSet](#)< TQ > &S, const char *istr=0) const

The documentation for this class was generated from the following file:

- clebsch.hh

4.66 QHash_ Class Reference

Public Member Functions

- size_t **operator()** (const [qset](#)< double > &x) const

The documentation for this class was generated from the following file:

- whash.cc

4.67 QMap< TQ > Class Template Reference

Public Member Functions

- void **init** ()
- **QMap**< TQ > & **init** (const char *F, int L, const **QVec** &qv, const **wbMatrix**< TQ > &q1, const **wbMatrix**< TQ > &q2, const **wbMatrix**< **wbMatrix**< TQ > > &QQ, const **wbMatrix**< **wbMatrix**< const **CRef**< TQ > * > > *SS=NULL)
- **QMap**< TQ > & **init** (const char *F, int L, const mxArray *a)
- template<class TD >
int **getCGZlist** (const char *F, int L, **wbMatrix**< TQ > &qc, **wbvector**< TD > &dc, **wbvector**< widx_t > *IC=NULL, const **wbvector**< widx_t > *Ix=NULL, const **qset**< TQ > *q3=NULL, **wbIndex** *m3=NULL, const **wbvector**< TD > *cc=NULL, double eps1=1E-10, double eps2=1E-14) const
- void **Skip** (const **wbindex** &l)
- template<class TD >
void **getIdentityQ** (const char *F, int L, const **wbvector**< widx_t > &Sa, const **wbvector**< widx_t > &Sb, **QSpace**< TQ, TD > &A, char vflag=1) const
- bool **isConsistent** (const char *F=NULL, int L=0, const **QVec** *qv=NULL) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tst=0) const
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="caller") const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const

Public Attributes

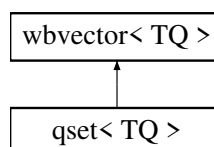
- **QVec** qvec
- **wbMatrix**< TQ > **Q1**
- **wbMatrix**< TQ > **Q2**
- **wbMatrix**< TQ > **Q**
- **wbvector**< widx_t > **D**
- **wbvector**< widx_t > **I1**
- **wbvector**< widx_t > **I2**
- **wbMatrix**< widx_t > **II**
- **wbMatrix**< const **CRef**< TQ > * > **cg3**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.68 qset< TQ > Class Template Reference

Inheritance diagram for qset< TQ >:



Public Member Functions

- **qset** (unsigned l, const TQ *d0=NULL, char ref=0)
- **qset** (const [qset](#)< TQ > &q)
- **qset** (const [QType](#) &Q, const TQ *qq)
- **qset** (const [QVec](#) &Q, const TQ *qq)
- **qset** (const char *F, int L, const mxArray *a)
- **qset** (const char *F, int L, const mxArray *a, const [QVec](#) *Q, char check_type=1)
- **qset** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2)
- **qset** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const [qset](#)< TQ > &q3)
- **qset** (const [qset](#)< TQ > &q1, char op, const [qset](#)< TQ > &q2)
- **qset** & **init** (const [qset](#)< TQ > &q1)
- **qset** & **init** (const char *F, int L, const char *s, [QType](#) *t=NULL)
- **qset** & **init** (const char *F, int L, const mxArray *a, const [QVec](#) *Q=NULL, char check_type=1)
- **qset** & **init** (const [QType](#) &Q, const TQ *qq)
- **qset** & **init** (const [QVec](#) &Q, const TQ *qq)
- **qset** & **init** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2)
- **qset** & **init** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const [qset](#)< TQ > &q3)
- **qset** & **init** ()
- **qset** & **init** (unsigned l, const TQ *d0=NULL)
- **qset** & **init** (unsigned l1, const TQ *d1, unsigned l2, const TQ *d2)
- **qset** & **init** (unsigned l1, const TQ *d1, unsigned l2, const TQ *d2, unsigned l3, const TQ *d3)
- [qset](#)< TQ > & **init** (const TQ q1)
- [qset](#)< TQ > & **init** (const TQ q1, const TQ q2)
- [qset](#)< TQ > & **init** (const TQ q1, const TQ q2, const TQ q3)
- [qset](#)< TQ > & **initStride** (const TQ *d_, unsigned n, unsigned r, unsigned QDIM)
- bool **operator**< (const [qset](#)< TQ > &b) const
- int **checkNQs** (const [QVec](#) &qq, const char *F=NULL, int L=0)
- bool **isConsistent** (const [QVec](#) &qq, int rank=-1, const char *F=NULL, int L=0) const
- unsigned **QDim** (const [QVec](#) &q, int rank=-1) const
- [qset](#)< TQ > & **times** (TQ fac, [qset](#)< TQ > &x) const
- **wbstring toStr** () const
- **wbstring toStr** (const [QType](#) &t __attribute__((unused))) const
- bool **operator**< (const [qset](#)< double > &b) const
- bool **operator**< (const [qset](#)< float > &b) const

Public Member Functions inherited from [wbvector](#)< TQ >

- **wbvector** (size_t l, TQ x)
- **wbvector** (size_t l, const TQ *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const TQ *d0)
- [wbvector](#) & **init** (size_t l, const TQ *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< TQ > & **init** (const [wbsparray](#)< TQ > &S)
- [wbvector](#) & **init** (size_t l1, const TQ *d1, size_t l2, const TQ *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)

- `int init (const mxArray *a, const char *istr=NULL, char check_type=1)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `wbvector & initp (size_t l, const TQ *d0, size_t n, char init=1)`
- `wbvector & init2val (size_t l, const TQ &x)`
- `wbvector & initStride (size_t l, const TQ *d0, size_t stride)`
- `wbvector & init_stride (const TQ *d0, size_t n, size_t m, size_t D0)`
- `wbvector & init_data (const TQ *d0)`
- `wbvector & init_data (const T0 *d0)`
- `wbvector & init_data (const char *F, int L, const T0 *d0)`
- `wbvector< TQ > & initT (const wbsparray< T2 > &S)`
- `wbvector< TQ > & initT (size_t l, const T2 *v)`
- `wbvector< TQ > & initT (const wbvector< T2 > &v)`
- `wbvector< TQ > & initT (const char *F, int L, const wbvector< T2 > &v)`
- `wbvector< TQ > & initT (const wbsparray< TD > &S)`
- `wbvector< TQ > & initT_ (const wbvector< T2 > &v)`
- `wbvector & init2ref (const wbvector &v)`
- `wbvector & init2ref (size_t l, const TQ *d0)`
- `wbvector & unRef ()`
- `int init_Struct (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)`
- `int init_base (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)`
- `int init_mpr (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)`
- `void initDef (size_t n)`
- `void initDef (const wbvector &v)`
- `wbvector & operator= (const wbvector &v)`
- `wbvector< TQ > & operator= (const wbvector< TB > &v)`
- `wbvector & set (const TQ &x)`
- `wbvector & set (const wbvector< size_t > &l, const TQ &x)`
- `void set (size_t offset, const TQ *x, size_t n)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `wbvector & setMin (const TQ &x)`
- `wbvector & setMax (const TQ &x)`
- `wbvector & add (const wbvector< size_t > &l, const TQ &x)`
- `wbvector & add (const TQ *d)`
- `wbvector & Set (const wbvector< size_t > &l, const wbvector &v)`
- `void Sort (wbperm &p, char dir=+1)`
- `void Sort (char dir=+1)`
- `void sort (wbvector &v2, wbperm &p) const`
- `void sort (wbvector &v2) const`
- `wbperm & pSort (wbperm &p) const`
- `bool isSorted () const`
- `bool isSortedU () const`
- `bool isUnique () const`
- `int contains (const TQ &x) const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isFinite () const`
- `size_t find1 (const TQ &x) const`
- `size_t find1 () const`
- `size_t count (const TQ &x) const`
- `size_t icount (const TQ &x) const`
- `size_t nnz () const`
- `size_t nnz (TQ eps) const`
- `size_t numZeros (TQ eps=TQ(1E-14)) const`

- `size_t numEQ` (const TQ &) const
- `size_t numGT` (const TQ &) const
- `size_t numLT` (const TQ &) const
- `wbindex & find` (const TQ &x, `wbindex` &l, char iflag=0) const
- `wbindex & find` (const TQ &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex & find` (`wbindex` &l) const
- `wbindex & find` (`wbindex` &l, `wbindex` &lx) const
- `wbvector< size_t > find` () const
- `wbindex & findGT` (const TQ &x, `wbindex` &l, char iflag=0) const
- `wbindex & findGT` (const TQ &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex & findzeros` (`wbindex` &l) const
- `wbindex & findRange` (const TQ &x1, const TQ &x2, `wbindex` &l, const char *w=NULL) const
- `int findClosestSorted` (TQ r) const
- `int findClosest` (TQ r) const
- `size_t findValues` (const `wbvector` &v, `wbindex` &l) const
- `wbvector & setp` (const TQ *x)
- `void reset` ()
- `void cpy` (const TQ *v)
- `void cpy` (const TQ *x, `size_t` m)
- `wbvector & save2` (`wbvector` &v)
- `void swap` (`wbvector` &v)
- `wbvector & swap` (`size_t` i, `size_t` j)
- `wbvector & Collect` (TQ &A1, TQ &A2)
- `wbvector & Collect` (TQ &A1, TQ &A2, TQ &A3)
- `wbvector & Skip` (`size_t`)
- `wbvector & Skip` (const WBINDEX &, char rflag=0)
- `wbvector & Move2_FE` (const WBINDEX &l, char FE_flag)
- `wbvector & Move2front` (const WBINDEX &l)
- `wbvector & Move2front` (`size_t` k)
- `wbvector & Move2end` (const WBINDEX &l)
- `wbvector & Move2end` (`size_t` k)
- `wbvector & mvLastTo` (`size_t` k)
- `wbvector & mvFirstTo` (`size_t` k)
- `bool checkSameLength` (const `wbvector` &a, const char *F=NULL, int L=0, const char *istr="") const
- `bool checkSameLength` (const `wbvector` &a, const `wbvector` &b, const char *F=NULL, int L=0, const char *istr="") const
- `void operator+=` (const T_ &a)
- `void operator+=` (const `wbvector` &b)
- `void operator-=` (const T_ &a)
- `void operator-=` (const `wbvector` &b)
- `wbvector & operator*=` (const T_ &a)
- `wbvector & operator/=` (const T_ &a)
- `wbvector operator*` (const T_ &a) const
- `wbvector operator+` (const T_ &a) const
- `wbvector operator+` (const `wbvector` &b) const
- `wbvector operator-` (const T_ &a) const
- `wbvector operator-` (const `wbvector` &b) const
- `wbvector & Plus` (const TQ *b)
- `wbvector & Plus` (const TQ *b, TQ bfac)
- `wbvector & Plus` (const `wbvector` &b, TQ bfac=1)
- `void plus` (const `wbvector` &b, `wbvector` &c, TQ bfac=1, TQ cfac=0) const
- `wbvector & times` (TQ fac, `wbvector` &x) const
- `wbvector & RevertSigns` ()
- `int isum` () const
- `TQ sum` () const

- TQ **sum** (size_t i1, const char *estr) const
- TQ **sum** (size_t i1, size_t i2) const
- TQ **cumsum_** (wbvector &S, char xflag=0) const
- TQ **Cumsum** (char zflag=0) const
- TQ **cumsum0prod** (wbvector< T2 > &b, wbvector< TQ > &cs, char xflag) const
- wbvector & **Eldiv** (const wbvector &v) const
- wbvector & **tensorProd** (const wbvector &v2, wbvector &vv) const
- TQ **scalarProd** (const wbvector &v) const
- TQ **scalarProd** (const TQ *v) const
- TQ **norm** () const
- TQ **norm2** () const
- double **normDiff** (const wbvector &v, TQ fac=1) const
- TQ **normDiff2** (const wbvector &v, TQ fac=1) const
- double **normDiff2_** (const wbvector &v, double fac=1) const
- TQ **GSProject** (const TQ *v, char isnorm=0, char tflag=0) const
- wbvector & **Normalize** (const char *F=NULL, int L=0)
- wbvector & **subtract** (const TQ *d)
- void **getReal** (wbvector< double > &R) const
- void **getReal** (wbvector< double > &R) const
- void **getImag** (wbvector< double > &I) const
- void **getImag** (wbvector< double > &I) const
- wbvector & **Conj** ()
- wbvector< wbcomplex > & **Conj** ()
- wbvector & **applyFlag** (char &flag, const TQ &fac, wbvector &a)
- bool **allEqual** () const
- bool **allEqual** (const TQ &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const TQ &a, TQ eps=1E-15) const
- bool **allUnequal** (const TQ &a) const
- bool **any** () const
- bool **anyUnequal** (const TQ &a) const
- bool **anyUnequal** (const wbvector< size_t > &l, const TQ &x, size_t *k=NULL) const
- bool **anyEqual** (const TQ &a, size_t *i1=NULL) const
- bool **anyEqual** (const wbvector< size_t > &l, const TQ &x, size_t *k=NULL) const
- bool **anyGT** (const TQ &a) const
- bool **anyGE** (const TQ &a) const
- bool **anyLT** (const TQ &a) const
- bool **anyLE** (const TQ &a) const
- bool **allGT** (const TQ &a) const
- bool **allGE** (const TQ &a) const
- bool **allLE** (const TQ &a) const
- bool **allLT** (const TQ &a) const
- bool **allIn** (const TQ &a1, const TQ &a2) const
- bool **anySmallVals** (const TQ eps=1E-15) const
- bool **isEqual** (const TQ *v) const
- bool **isEqual** (const wbvector &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const wbvector &v) const
- bool **operator!=** (const wbvector &v) const
- bool **isEqualR** (const wbvector &v) const
- bool **operator^** (const wbvector &v) const
- bool **deepEqualP** (const wbvector &B) const

- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const TQ &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- TQ & **el** (size_t i) const
- TQ & **elx** (size_t i, const TQ &x) const
- const TQ & **operator[]** (size_t i) const
- TQ & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- TQ & **last** () const
- TQ & **last** (size_t &i) const
- TQ & **last** (Tl &i_) const
- TQ & **p2last** () const
- TQ * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const TQ *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const TQ *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< TQ > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< TQ > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const TQ &x)
- [wbvector](#) & **Append** (size_t d2, const TQ *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const TQ *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const TQ *v) const
- int **recLessE** (const TQ *v) const
- int **recCompare** (const TQ *v) const
- TQ **max** () const
- TQ **max** (size_t *k) const
- TQ **max** (unsigned &k) const
- TQ **max_** (TQ x) const
- TQ **min** () const
- TQ **min** (size_t *k) const
- TQ **min** (unsigned &k) const
- TQ **min_** (TQ x) const
- TQ **aMax** (size_t *k=NULL) const
- TQ **aMax** (unsigned &k) const

- TQ **aMin** (char zflag=0, size_t *k=NULL) const
- TQ **maxneg** (size_t *k=NULL) const
- TQ **minpos** (size_t *k=NULL) const
- TQ **avg** () const
- TQ **std** () const
- void **range** (TQ &xmin, TQ &xmax) const
- TQ **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &l0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &l, TQ *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &l)
- void **select** (const WBINDEX &l, TQ *r) const
- **wbvector** & **select** (const WBINDEX &l, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &l) const
- **wbvector** & **Select** (const WBINDEX &l)
- **wbvector** & **BlockSelect** (const WBINDEX &l, const WBINDEX &D)
- **wbvector** & **getl** (size_t k, **wbvector** &v) const
- **wbvector** & **getl** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getl** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initl** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initl** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- TQ **prod** () const
- TQ **prod** (TQ x) const
- TQ **prod** (size_t i, size_t i2) const
- TQ **prod** (size_t i, size_t i2, TQ x) const
- TQ **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (TQ &p1, TQ &p2, const TQ &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const

- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base_d (const char tflag=0) const`
- `mxArray * toMx_offset (const TQ &offset, const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tflag=0) const`
- `void add2MxStruct (mxArray *S, const char *vname, char tflag=0) const`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tflag=0) const`
- `void putg (const char *vname="ans", char tflag=0) const`
- `void putx (const char *vname="ans", char tflag=0) const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring toStr () const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring sizeStr () const`
- `wbstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const`
- `wbstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const`

Additional Inherited Members

Public Attributes inherited from `wbvector< TQ >`

- `TQ * data`
- `size_t len`
- `char isref`

Protected Member Functions inherited from `wbvector< TQ >`

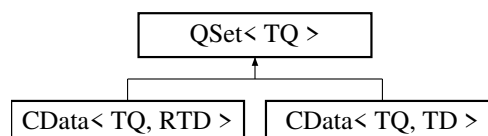
- `wbvector & RENEW (size_t n, const TQ *d=NULL, size_t l=-1, char init=1)`
- `wbvector & RENEW_VAL (size_t n, const TQ &x)`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch.cc`

4.69 QSet< TQ > Class Template Reference

Inheritance diagram for `QSet< TQ >`:



Public Member Functions

- **QSet** (const [QSet](#) &B)
- **QSet** (const [QType](#) &t_, const [qset](#)< TQ > &qs_, unsigned iout=0, char isref=0)
- **QSet** (const [QType](#) &t_, const [iTags](#) &it, unsigned r=-1)
- **QSet** (const [CRef](#)< TQ > &A)
- **QSet** (const char *F, int L, const mxArray *a)
- **QSet** (const mxArray *a)
- **QSet** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *P=NULL)
- **QSet** (const char *F, int L, const char *s)
- **QSet & init** (const [CRef](#)< TQ > &A)
- **QSet & init** (const char *F, int L, const mxArray *a, unsigned k=0)
- **QSet & init** (const [QType](#) t_=QTYPE_UNKNOWN)
- **QSet & init** (const [QSet](#) &B)
- **QSet & init2ref** (const [QSet](#) &B)
- **QSet & init** (const [QType](#) &t_, const TQ *q, const char *qd)
- **QSet & init** (const [QType](#) &t_, const [qset](#)< gTQ > &qs_, unsigned iout=0, char isref=0)
- **QSet & init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, unsigned iout=0)
- **QSet & init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, const [iTags](#) &it)
- **QSet & init1J** (const [QType](#) &t_, const TQ *q, char iflag=0)
- **QSet & init2** (const [QType](#) &t_, const TQ *q=NULL)
- **QSet & init2** (const [QType](#) &t_, const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const char *qds="++")
- **QSet & init1** (const [QType](#) &t_, TQ *q_=NULL)
- **QSet & init3** (const [QType](#) &t_)
- **QSet & init3** (const [QType](#) &t_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const [qset](#)< TQ > &J)
- **QSet & init3** (const [QType](#) &t_, const TQ &J1, const TQ &J2, const TQ &J)
- **QSet & init** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *Pcd=NULL)
- **QSet & init_str** (const char *F, int L, const char *s)
- **QSet & Conj** ()
- **QSet & Conj** (unsigned i)
- unsigned **rank** (const char *F=NULL, int L=0) const
- **QSet & permute** ([QSet](#) &B, const [wbperm](#) &P, char iflag=0) const
- **QSet & Permute** (const [wbperm](#) &P, char iflag=0)
- void **swap** ([QSet](#) &X)
- **QSet & save2** ([QSet](#) &X)
- **QSet & operator=** (const [QSet](#) &B)
- bool **operator==** (const [QSet](#) &B) const
- bool **operator!=** (const [QSet](#) &B) const
- char **cmp** (const [QSet](#)< TQ > &B) const
- bool **operator<** (const [QSet](#)< TQ > &B) const
- **operator bool** () const
- bool **operator!** () const
- bool **isEmpty** (char check=3) const
- bool **gotRCData** () const
- int **isZero** () const
- unsigned **isStd3** () const
- unsigned **checkStd3** () const
- int **init2opt3** (const [QSet](#)< TQ > &Q, [wbperm](#) &P, [wbvector](#)< char > &cflags)
- **QSet & Convert** (const char *s, [wbvector](#)< char > &cflags)
- [wbstring](#) **sizeStr** () const
- bool **isScalar** () const
- bool **is1JSymbol** () const
- bool **isAbelian** () const

- int **permitsOM** () const
- int **checkQ_abelian** (const char *F, int L) const
- int **checkQ_SU2** (const char *F, int L) const
- **QSet & Sort** (**wbperm** *cgp=NULL, char *conj=NULL, char iflag=0)
- bool **isSorted** () const
- size_t **memSize** () const
- **wbstring QStrS** (const **wbperm** *cgp=NULL, char sep='|') const
- **wbstring QStr** () const
- **wbstring toStr** () const
- **wbstring toStr** (const char *istr) const
- **wbstring toTag** () const
- mxArray * **toMx** () const

Public Attributes

- **QType t**
- **qset**< TQ > **qs**
- **QDir qdir**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.70 QSpace< TQ, TD > Class Template Reference

Public Member Functions

- **QSpace** (unsigned n=0)
- **QSpace** (unsigned m, unsigned n, unsigned nq)
- **QSpace** (const **QSpace** &B)
- template<class TQ_, class TD_>
QSpace (const **QSpace**< TQ_, TD_ > &B, char xflag=**WbUtil**< TD_ >::isComplex() ? 1 :0)
- **QSpace** (const char *F, int L, const mxArray *a, char ref=0, char skip_empty=1)
- **QSpace** (const mxArray *a, char ref=0, char skip_empty=1)
- **QSpace & clearQSpace** (char mflag=0)
- void **clearDATA** (char mflag=0)
- unsigned **rank** (const char *F=NULL, int L=0) const
- unsigned **len** () const
- unsigned **nsym** () const
- TQ * **Qref** (unsigned r, unsigned k)
- const TQ * **Qref** (unsigned r, unsigned k) const
- void **init** ()
- void **init** (unsigned m, unsigned n, unsigned nq)
- void **init** (const **wbMatrix**< TQ > &Q, unsigned qdim, const **QVec** *qt=NULL)
- **QSpace & init** (const **QSpace** &B)
- template<class TB_>
QSpace & initT (const **QSpace**< TQ, TB_ > &B)
- template<class TR_>
QSpace & getReal (**QSpace**< TQ, TR_ > &R) const

- [QSpace](#)< TQ, double > **getReal** () const
- template<class TI >
 [QSpace](#) & **getImag** ([QSpace](#)< TQ, TI > &l) const
- [QSpace](#)< TQ, double > **getImag** () const
- void **init** (const char *F, int L, const mxArray *S, char ref, unsigned k, char skip_empty=1)
- void **init** (const char *F, int L, const mxArray *S, char ref=0, char skip_empty=1)
- [QSpace](#) & **setupDATA** (const char *F=NULL, int L=0)
- int **gotCGS** (const char *F=NULL, int L=0, char xflag=0) const
- int **gotCGX** (const char *F=NULL, int L=0) const
- [QSpace](#) & **Enlarge** (unsigned m)
- [QSpace](#) & **Trim** (unsigned n)
- [QSpace](#) & **initScalar** (const TD &x)
- void **init2ref** (const mxArray *S)
- void **init2ref** (const [QSpace](#) &)
- void **initOpZ_WET** (const char *F, int L, const [QVec](#) &qvec, const [wbMatrix](#)< TQ > &Q1, const [wbMatrix](#)< TQ > &Q2, const [wbMatrix](#)< TQ > &Q, const [wbarray](#)< TD > &D3, const double eps1=1E-10, const double eps2=1E-13)
- void **reduceMatEI** (const char *F, int L, const [wbMatrix](#)< TQ > &Q1, const [wbvector](#)< TD > &dd, const [wbMatrix](#)< TQ > &Q2, const [wbvector](#)< TD > &dc, [wbvector](#)< unsigned > &l1, [wbvector](#)< unsigned > &l2, [wbvector](#)< TD > &dr) const
- void **unRef** ()
- void **initIdentity** (const [QSpace](#) &A, char dflag=0)
- void **initIdentity** (const [wbMatrix](#)< TQ > &Q)
- [QSpace](#) & **setupCGR** (const char *F=NULL, int L=0)
- [QSpace](#) & **initIdentityCGS** (const char *F=NULL, int L=0)
- [QSpace](#) & **initIdentityCGS** (const char *F, int L, unsigned k, unsigned r=0)
- template<class TD2 >
 [QSpace](#) & **initIdentityCGS** (const char *F, int L, const [QSpace](#)< TQ, TD2 > &H)
- template<class TA >
 [QSpace](#)< TQ, TD > & **initIdentityCG** (const [wbvector](#)< [QSpace](#)< TQ, TA > > &A, const [wbindex](#) &ia, char zflag=0)
- template<class TA , class TB >
 [QSpace](#)< TQ, TD > & **initIdentityCG** (const [wbvector](#)< [QSpace](#)< TQ, TA > > &A, const [wbindex](#) &ia, const [wbvector](#)< [QSpace](#)< TQ, TB > > &B, const [wbindex](#) &ib, char vflag='v')
- template<class TA , class TB >
 [QSpace](#)< TQ, TD > & **initIdentityCG** (const [wbvector](#)< [QSpace](#)< TQ, TA > > &A, const [wbvector](#)< [QSpace](#)< TQ, TB > > &B, const char *pstr=NULL)
- [QSpace](#) & **initDiagonal** (const [wbMatrix](#)< TQ > &Q, const [wbvector](#)< unsigned > &S, const [wbvector](#)< TD > &D, char rflag)
- [QSpace](#) & **init2DiffOp** (const [QSpace](#) &A, unsigned ia, const [QSpace](#) &B, unsigned ib)
- template<class TB >
 [QSpace](#) & **initQT_safe** (const [QSpace](#)< TQ, TB > &B)
- template<class TA >
 [QSpace](#) & **initQ** (const [QSpace](#)< TQ, TA > &A, char full=1)
- template<class TA , class TB >
 [QSpace](#) & **initQ** (const char *F, int L, const [QSpace](#)< TQ, TA > &A, char full=1, const [QSpace](#)< TQ, TB > *B=NULL)
- void **initQ** (unsigned n, unsigned r, const [QSpace](#) &A)
- void **initQ** (const [wbMatrix](#)< TQ > &Qk, const [QSpace](#) &A)
- void **init_itags** (const char *F, int L, const char *s, unsigned r=-1, char lflag=-1)
- [QSpace](#) & **SetTag** (unsigned k, const [itag_](#) &b)
- [QSpace](#) & **SetTags** (const [itag_](#) &b)
- [QSpace](#) & **SetConjTags** ()
- [QSpace](#) & **SetConjTags** (unsigned k1, unsigned k2)
- [QSpace](#) & **SetConjTag** (unsigned k)
- [QSpace](#) & **SetConjTag** (const char *F, int L, unsigned k)

- [QSpace](#) & **SetFlag** (const char *F, int L, unsigned k, unsigned l)
- [QSpace](#) & **UnsetFlag** (const char *F, int L, unsigned k, unsigned l)
- [QSpace](#) & **UnsetFlags** ()
- [QSpace](#) & **SetFlags** (const char *F, int L, unsigned k, unsigned l=-1)
- [QSpace](#) & **init_tags** ()
- template<class TB >
 unsigned **matchITags** (const char *F, int L, const [QSpace](#)< TQ, TB > &B, [ctrlidx](#) &ia, [ctrlidx](#) &ib) const
- unsigned **RemoveZLabels** (const char *F, int L, [wbMatrix](#)< TQ > &QQ, [wbMatrix](#)< TQ > *Z=NULL) const
- void **RemoveZLabels** (const char *F, int L, [wbMatrix](#)< TQ > *Z)
- void **ExpandDiagonal** (unsigned i1=1, unsigned i2=2)
- size_t **getDataSize** ([wbMatrix](#)< size_t > &S) const
- size_t **getDataSize** ([wbvector](#)< size_t > &S) const
- size_t **getDataSize** () const
- size_t **getCGSSize** () const
- [wbstring](#) **totSize2Str** () const
- [wbstring](#) **itags2Str** () const
- unsigned **map2Vec** (const char *F, int L, [wbvector](#)< TD > &V, const [wbvector](#)< unsigned > &S=[wbvector](#)< unsigned >())
- unsigned **map2Vec** (TD *v, char lflag=0)
- void **ExpandQ** ([CPAT](#)< TQ, TD > &CP, double nrm, [wbvector](#)< unsigned > &xflag, char disp=0)
- void **ResetRec** (unsigned k)
- void **setData** (const TD c)
- void **setRand** (double nrm=0)
- void **markUnequalZero** (const TD m=1, const TD eps=0)
- void **PrependSingletons** (unsigned R)
- void **swap** ([QSpace](#) &B)
- [QSpace](#) & **save2** ([QSpace](#) &B)
- unsigned **Reduce2AbelianOp** (const char *F=NULL, int L=0)
- char **allAbelian** () const
- char **isNonAbelian** () const
- bool **permitsOM** () const
- bool **hasOM** () const
- bool **allAdditive** () const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **isConsistent** (const char *F, int L, unsigned &r, char level=3) const
- bool **isConsistent** (const char *F=NULL, int L=0) const
- bool **isConsistent** (unsigned &r, char level=3) const
- bool **isConsistent_r** (unsigned r, char level=3) const
- bool **isScalar** () const
- bool **isComplex** () const
- bool **isDiagBlock** (unsigned k) const
- bool **isBlockDiagMatrix** (const char *F, int L, char dflag=0) const
- bool **isBlockDiagMatrix** (char dflag=0) const
- bool **isDiagMatrix** (const double eps=1e-14) const
- bool **isIdentityMatrix** (const TD eps=1e-14) const
- char **hasIdentityCGS** (const TD eps=1e-14) const
- bool **isR3Op** () const
- bool **isOp** (int r=-1, char lflag=0) const
- int **isOperator** (unsigned *r_, char xflag=0) const
- bool **isOperator** () const
- bool **isAtensor** (unsigned r=-1) const
- int **checkAbelianOp** () const
- bool **isAbelianOp** () const

- bool **isQSym** (const char *F=0, int L=0, char dflag=0) const
- bool **isHConj** (const char *F=0, int L=0, double eps=1e-14, char vflag=0) const
- bool **isAHerm** (const char *F=0, int L=0, double eps=1e-14, char vflag=0) const
- int **isSingleton** (unsigned k) const
- template<class T2 >
bool **hasSameQ** (const QSpace< TQ, T2 > &B) const
- bool **hasSameQ** (unsigned r1, unsigned k1, unsigned r2, unsigned k2) const
- template<class T2 >
bool **hasSameRank** (const QSpace< TQ, T2 > &B) const
- char **checkQ** (const char *F=NULL, int L=0) const
- template<class TB >
void **checkQ** (const char *F, int L, const QSpace< TQ, TB > &B, char aflag=0) const
- char **checkQ_CGR** (const char *F, int L, C_UVEC *l=NULL) const
- int **check_CGR_cgw** (const char *F=0, int L=0) const
- void **checkScalarCGS** (const char *F=0, int L=0) const
- template<class TB >
bool **sameType** (const QSpace< TQ, TB > &B, int r=-1, const char *istr=NULL) const
- QDir & **getQDir** (QDir &qdir) const
- bool **sameQDir** (const char *F, int L, const QSpace &B) const
- bool **hasQOverlap** (const QSpace &A) const
- template<class TB >
int **hasQOverlap** (unsigned ica, const QSpace< TQ, TB > &B, unsigned icb, const char type=0) const
- int **getQOverlapU** (const char *F, int L, const QSpace &B, wbMatrix< TQ > &QQ, wbMatrix< int > &IQ, char olonly=0) const
- int **getQOverlap** (const char *F, int L, const QSpace &B, wbMatrix< TQ > &QQ, wbMatrix< int > &IQ, wbvector< widx_t > &DQ) const
- bool **findDimQ** (const TQ *q, unsigned &d) const
- bool **findDimQ** (const TQ *q, unsigned k, unsigned &d) const
- bool **operator==** (const QSpace &B) const
- bool **operator!=** (const QSpace &B) const
- template<class T >
void **operator*=** (const T fac)
- template<class T >
QSpace & **times** (const T fac, QSpace &B) const
- QSpace & **operator=** (const QSpace &A)
- void **operator+=** (const QSpace &Q)
- void **operator-=** (const QSpace &Q)
- QSpace & **Cat** (const char *F, int L, const QSpace< TQ, TD > &A, const QSpace< TQ, TD > &B, TD afac=1, TD bfac=1, char uflag=1)
- QSpace & **plus** (const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const
- QSpace & **minus** (const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const
- QSpace & **Plus** (const QSpace &A, TD bfac=1)
- QSpace & **Minus** (const QSpace &A, TD bfac=1)
- QSpace & **plus_plain** (const QSpace &B, QSpace &C, TD bfac=1) const
- int **NormCGW** (char full=0, char skipzeros=1, char rcpy=0)
- TD **norm2** (char checks=1) const
- TD **norm** (char checks=1) const
- TD **normDiff2** (const QSpace &B) const
- TD **scalarProd** (const QSpace &B) const
- void **checkNorm** (const char *F, int L, double nrm, char vflag=1, double eps=1E-12) const
- QSpace & **TimesEI** (const QSpace &B, char conj=0, unsigned r2=-1)
- TD **sumData** () const
- template<class TM >
wbMatrix< TM > & **diag** (wbMatrix< TM > &xd) const
- QSpace & **TensorProd** (const QSpace &B, QSpace &C, const char aflag='N', const char bflag='N') const

- `template<class TC, ENABLE_IF_COMPLEX(TC) >`
`QSpace< TQ, TC > & TensorProd (const QSpace &B, QSpace< TQ, TC > &C, const char aflag='N', const char bflag='N') const`
- `template<class TC, ENABLE_IF_COMPLEX(TC) >`
`QSpace< TQ, TC > & TensorProd (const QSpace< TQ, TC > &B, QSpace< TQ, TC > &C, const char aflag='N', const char bflag='N') const`
- `void TensorProdUnity (const wbMatrix< TQ > &Q, QSpace &C) const`
- `void EigenSymmetric (wbMatrix< double > &Etot) const`
- `void EigenSymmetric (QSpace &Ak, QSpace< TQ, double > &Ek, wbMatrix< double > &Etot) const`
- `void EigenSymmetric (QSpace &Ak, QSpace< TQ, double > &Ek) const`
- `void EigenSymmetric (QSpace &Ak, QSpace &Ad, QSpace< TQ, double > &Ek, QSpace< TQ, double > &Et, wbMatrix< double > &Etot, wbMatrix< unsigned > &DD, int &Nkeep, double Etrunc=0, double *E0=NULL, char mKD=0, const wbperm &P=wbperm(), double eps=0., double b=0., int dmax=-1, const char *sort=NULL) const`
- `void GroupIndices (unsigned K)`
- `int ExpandOM (const char *F, int L, wbvector< unsigned > &iOM, char fflag=0)`
- `int ExpandOM_ (const char *F, int L, unsigned ia, QSpace< TQ, TD > &B, unsigned ib)`
- `int ExpandOM_ (const char *F, int L, unsigned ia, unsigned ib)`
- `void Append (unsigned k)`
- `void Append (const wbvector< TQ > &, const wbarray< TD > &D)`
- `QSpace & Append (const char *F, int L, const QSpace &A, char uflag=1)`
- `int Append2AndDestroy (const char *F, int L, QSpace &A, char unique=0)`
- `template<class TA, ENABLE_IF_COMPLEX(TA) >`
`int Append2AndDestroy (const char *F, int L, QSpace< TQ, TA > &A, char unique=0)`
- `void TransferSpace (const char *F, int L, QSpace &B, unsigned k, const QSpace< TQ, double > &W, double eps)`
- `void TransferSpace (const char *F, int L, QSpace &B, unsigned k)`
- `unsigned MakeUnique ()`
- `bool isZero (const double eps=0., char flag=0) const`
- `unsigned SkipZeroData (double eps=0., char bflag=0, char cgflag=1, char all=0)`
- `unsigned skipZeroOffDiag (double eps=0., char bflag=0)`
- `unsigned SkipEmptyData (const char *F=0, int L=0)`
- `bool gotZeroData (unsigned i, const double eps=0.) const`
- `void Set (unsigned k, const wbvector< TQ > &Q, const wbarray< TD > &D)`
- `void SetQ (unsigned i, unsigned k, const TQ *d)`
- `QSpace & Select (const wbindex &l, char data_only=0)`
- `template<class TM >`
`QSpace & SelectMarked (const wbvector< TM > &mark)`
- `void Sort (const QSpace &A)`
- `void Sort ()`
- `unsigned getOM (wbMatrix< unsigned > &OM, wbvector< unsigned > *om1=NULL, char wflag=0) const`
- `unsigned cgsDimScalar (unsigned k) const`
- `unsigned cgsDim (unsigned k, unsigned r) const`
- `unsigned getDim (unsigned k, widx_t *D=NULL) const`
- `unsigned getDIM (unsigned k) const`
- `void getDim (wbvector< widx_t > &D, wbvector< widx_t > *DD=NULL) const`
- `wbvector< widx_t > getDim () const`
- `void getQDim (unsigned k, wbMatrix< TQ > &Q, wbvector< widx_t > &S, wbMatrix< widx_t > *SC=NULL) const`
- `void getQDim (wbMatrix< TQ > &Q, wbvector< widx_t > &S, wbMatrix< widx_t > *SC=NULL) const`
- `void getDRange (TD &dmin, TD &dmax) const`
- `void getDistQtot (wbMatrix< TQ > &Qtot, wbvector< double > &w) const`
- `widx_t getDQtot ()`
- `wbMatrix< TQ > & getQsub (const wbindex &l, wbMatrix< TQ > &Ql) const`
- `wbMatrix< TQ > & getQsub (const unsigned k, wbMatrix< TQ > &Qk, char iflag=0) const`

- void **getQsub** (const [wbindex](#) &I, [wbMatrix](#)< TQ > &Q1, [wbMatrix](#)< TQ > &Q2) const
- [wbMatrix](#)< TQ > **getQsub** (const unsigned k) const
- [wbMatrix](#)< TQ > & **getQtot** ([wbMatrix](#)< TQ > &Qtot, char uflag=0) const
- [wbMatrix](#)< TQ > & **getQsum** (const [wbindex](#) &J, [wbMatrix](#)< TQ > &Q) const
- [wbMatrix](#)< TQ > & **getQsum** (const [wbMatrix](#)< TQ > &Q0, [wbMatrix](#)< TQ > &Q) const
- [wbMatrix](#)< TQ > & **getQfinal** (const [wbindex](#) &J, [wbMatrix](#)< TQ > &Q) const
- [QSpace](#) & **getSub** (const [wbindex](#) &I, [QSpace](#) &A, char ref=0) const
- [QSpace](#) & **getSubInit** (const [wbindex](#) &I, [QSpace](#) &A) const
- void **saveSub2** ([QSpace](#) &A, const [wbindex](#) &I)
- void **toFull** ([wbarray](#)< TD > &A, [wbvector](#)< [wbMatrix](#)< TQ > > &Q, [wbvector](#)< [wbvector](#)< unsigned > > &S) const
- void **toFullI2** ([wbarray](#)< TD > &A, [wbMatrix](#)< TQ > &Q, [wbMatrix](#)< widx_t > &S) const
- void **ExtendQ** (const [QSpace](#) &B)
- void **takeMainQS** (char disp=1)
- void **setQ** (unsigned k, const [QSpace](#) &B, unsigned k0)
- void **setQRec** (unsigned r, unsigned j, const TQ *q0, const TQ *q2=NULL)
- bool **hasEqualQ** (unsigned i1, unsigned i2) const
- double **maxDiff** (const [QSpace](#) &B) const
- void **PermuteQ** (const [wbperm](#) &P)
- void **permuteQ** (const [wbperm](#) &P, [wbMatrix](#)< TQ > &) const
- [QSpace](#) & **Permute** (const [wbperm](#) &P, char iflag=0, char rcpy=0)
- [QSpace](#) & **permute** (const [wbperm](#) &P, [QSpace](#) &B, char iflag=0) const
- void **permute** (const char *pstr, [QSpace](#) &B, unsigned offset=1) const
- void **Permute** (const char *pstr, unsigned offset=1)
- void **PermuteFirstTo** (unsigned k)
- void **permuteFirstTo** (unsigned k, [QSpace](#) &C)
- void **PermuteLastTo** (unsigned k)
- void **permuteLastTo** (unsigned k, [QSpace](#) &C)
- void **FlipQ** ()
- [QSpace](#) & **trace** (const char *F, int L, [ctrlidx](#) &i1, [ctrlidx](#) &i2, [QSpace](#) &C) const
- TD **trace** () const
- void **contractMat** (unsigned ia, const [QSpace](#) &B, unsigned ib, [QSpace](#) &C) const
- template<class TB, class TC >
int **contract_getldxSet** (const char *F, int L, const [ctrlidx](#) &ica, const [QSpace](#)< TQ, TB > &B, const [ctrlidx](#) &icb, [wbindex](#) &la, [wbindex](#) &lb, [wbvector](#)< widx_t > &Dc, [QSpace](#)< TQ, TC > &C) const
- template<class TB, class TC >
double **contract** (const char *F, int L, const [ctrlidx](#) &ica, const [QSpace](#)< TQ, TB > &B, const [ctrlidx](#) &icb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char preview=0) const
- template<class TB, class TC >
double **contract** (const char *F, int L, unsigned ia, const [QSpace](#)< TQ, TB > &B, unsigned ib, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *F, int L, unsigned ia, const [QSpace](#)< TQ, TB > &B, const char *sb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *F, int L, const char *sa, const [QSpace](#)< TQ, TB > &B, unsigned ib, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *F, int L, const char *sa, const [QSpace](#)< TQ, TB > &B, const char *sb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *sa, const [QSpace](#)< TQ, TB > &B, const char *sb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char preview=0) const
- template<class TB, class TC >
int **contract** (const [ctrlidx](#) &ica, const [QSpace](#)< TQ, TB > &B, const [ctrlidx](#) &icb, [QSpace](#)< TQ, TC > &C, const [wbperm](#) &P=[wbperm](#)(), char preview=0) const

- `template<class TB , class TC >`
`int contract (unsigned ia, const QSpace< TQ, TB > &B, unsigned ib, QSpace< TQ, TC > &C, const wbperm &P=wbperm(), char preview=0) const`
- `mxArray * toMx (bool setcls=0) const`
- `mxArray * save2Mx (char vflag=0)`
- `mxArray * QIDX_toMx () const`
- `mxArray * INFO_toMx () const`
- `mxArray * DATA_toMx () const`
- `mxArray * DATA_save2Mx (char vflag=0)`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, bool setcls=0, char chk=0) const`
- `void save2MxStruct (mxArray *S, unsigned i, char tst=0, char vflag=0)`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tst=0) const`
- `void put (const char *F, int L, const char *vname, const char *ws="caller") const`
- `void put (const char *vname, const char *ws="caller") const`
- `wbstring sizeStrQ (const char *F=NULL, int L=0) const`
- `wbstring sizeStr (char vflag=0) const`
- `void print_SIZE (const char *F=NULL, int L=0) const`
- `void print_rankDATA (const char *F=NULL, int L=0) const`
- `wbstring qStr () const`
- `wbstring qrec2Str (unsigned k) const`
- `void disp_cgs (const char *F, int L, const char *istr) const`
- `void info (const char *vname=NULL, unsigned nlt=0, unsigned nlb=0, unsigned nind=2) const`
- `void print () const`
- `void print (const char *vname, char vflag=3) const`
- `void print_tst_ (const char *F, int L, const char *istr, unsigned k=0) const`
- `void initOType (const char *F, int L, const mxArray *a)`
- `wbstring otype2Str (const char *fmt="%s") const`
- `int ConjOpScalar ()`
- `QSpace & SortDegQ (const char *F=0, int L=0)`
- `QSpace & Conj ()`
- `QSpace & DConj ()`
- `QSpace & RConj ()`
- `QSpace & HConj ()`
- `QSpace & hconj (QSpace &B) const`
- `QSpace & transp (QSpace &B) const`
- `const QSpace & opA (char tflag, QSpace &X) const`
- `template<class TD2 >`
`QSpace< TQ, TD > & initIdentityCGS (const char *F, int L, const QSpace< TQ, TD2 > &H)`

Public Attributes

- `wbMatrix< TQ > QIDX`
- `wbvector< wbarray< TD > * > DATA`
- `wbMatrix< CRef< TQ > > CGR`
- `QVec qtype`
- `unsigned QDIM`
- `QS_TYPES otype`
- `iTags itags`
- `Wb::MTYPE mt`
- `char isref`
- `double ctime`

The documentation for this class was generated from the following files:

- `QSpace.hh`
- `cpat.hh`
- `QSpace.cc`

4.71 QType Class Reference

Public Member Functions

- **QType** (QTYPE_SET t=QTYPE_UNKNOWN, unsigned s=0)
- **QType** (const char *s)
- **QType** (const [QType](#) &q)
- **QType** (const char *F, int L, const mxArray *a)
- **QType** & **init** ()
- **QType** & **initx** (QTYPE_SET t, unsigned m=0)
- **QType** & **init** (QTYPE_SET t, unsigned m=0)
- int **init_s** (const char *s)
- **QType** & **init** (const char *s)
- **QType** & **init** (const char *F, int L, const char *s)
- **QType** & **init** (const char *F, int L, const mxArray *a)
- **QType** & **init** (const [QType](#) &q)
- bool **isUnknown** () const
- bool **isKnown** () const
- bool **validType** (const char *F=0, int L=0) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **isNonAbelian** (const char *F=NULL, int L=0, char lflag=0) const
- bool **isSU2** () const
- bool **isSUN** () const
- bool **isSU** (unsigned N) const
- bool **isSpN** () const
- bool **isSON** () const
- bool **isSEN** () const
- bool **isU1** () const
- bool **isParity** () const
- bool **isAdditive** () const
- int **permitsOM** () const
- int **permitsOM** (unsigned r) const
- unsigned **qlen** () const
- unsigned **qrank** () const
- bool **isLargeD** (unsigned d) const
- bool **isLargeD** (const [wbvector](#)< unsigned > &dd) const
- unsigned **getDimDef** () const
- template<class TQ >
size_t **wdim** (const TQ *qs) const
- template<class TQ >
size_t **wdim** (const [qset](#)< TQ > &qs) const
- unsigned **maxDimLocal** () const
- unsigned **maxDimLocal0** () const
- void **printDimInfo** (const char *F=NULL, int L=0) const
- template<class T >
unsigned **qdim** (const T *q) const
- template<class T >
[wbvector](#)< unsigned > & **QDim** (const T *q, unsigned r, unsigned stride, [wbvector](#)< unsigned > &S) const
- bool **isSelfDual** () const
- template<class TQ >
char **setDual** (TQ *q) const
- template<class TQ >
unsigned **getDual** (const TQ *q0, TQ *q) const

- `template<class TQ >`
`bool isDual (const TQ *q, const TQ *x) const`
- `QType & operator= (const char *s)`
- `QType & operator= (const QType &q)`
- `QType & operator= (QTYPE_SET t)`
- `bool operator< (const QType &q) const`
- `char cmp (const QType &q) const`
- `bool operator> (const QType &q) const`
- `bool operator<= (const QType &q) const`
- `bool operator== (const QType &q) const`
- `bool operator!= (const QType &q) const`
- `wbstring toStr (const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i) const`
- `mxArray * toMx (const char tflag=0) const`
- `template<class TQ >`
`unsigned qdim (const TQ *q) const`
- `template<class TQ >`
`wbvector< unsigned > & QDim (const TQ *q, unsigned r, unsigned stride, wbvector< unsigned > &S) const`

Public Attributes

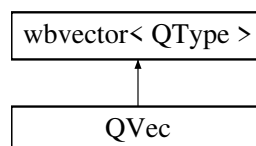
- `QTYPE_SET type`
- `unsigned sub`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch.cc`
- `clebsch_io.cc`

4.72 QVec Class Reference

Inheritance diagram for QVec:



Public Member Functions

- **QVec** (unsigned l, const [QType](#) *d0, char ref=0)
- **QVec** (const char *F, int L, const mxArray *a, unsigned d=0)
- **QVec & init** (const char *F, int L, const mxArray *a, unsigned d=0)
- **QVec & init** (unsigned l=0)
- **QVec & init** (const char *F, int L, const char *s, unsigned d=0)
- unsigned **Qlen** (unsigned n=-1) const
- unsigned **Qrank** () const
- unsigned **Qlen** ([wbvector](#)< unsigned > &dd) const
- unsigned **Qlen** ([wbvector](#)< unsigned > &dd, [wbvector](#)< unsigned > &dz) const
- unsigned **Qpos** ([wbvector](#)< unsigned > &dc) const
- unsigned **Qlenz** () const
- [wbvector](#)< gTQ > & **getScalar** ([wbvector](#)< gTQ > &q, char zflag=0)
- unsigned **maxRank** () const
- template<class TQ >
unsigned **QDim** (const TQ *q) const
- template<class TQ >
unsigned **QDim** (const TQ *qq, unsigned k) const
- template<class TQ >
[wbvector](#)< unsigned > & **QDim** (const TQ *qq, [wbvector](#)< unsigned > &S) const
- template<class TQ >
[wbvector](#)< unsigned > & **QDim** (const TQ *qq, unsigned k, unsigned r, [wbvector](#)< unsigned > &S) const
- template<class TQ >
[wbMatrix](#)< TQ > & **getQsub** (const [wbMatrix](#)< TQ > &Q, const [wbindex](#) &l, [wbMatrix](#)< TQ > &Ql, [wbMatrix](#)< TQ > *Qx=NULL) const
- bool **allU1** () const
- void **ReduceU1** ()
- char **allAbelian** () const
- bool **anyAbelian** () const
- bool **anyParity** () const
- bool **allNonAbelian** () const
- char **isNonAbelian** () const
- bool **allAdditive** () const
- int **permitsOM** () const
- int **permitsOM** (unsigned r) const
- bool **sameAs** (const [QVec](#) &b, char lflag=1) const
- bool **sameType** (const [QVec](#) &b) const
- [wbstring](#) **toStr** (const char vflag=0) const
- int **print_qset** (const char *F, int L, const gTQ *qs, char *s, unsigned n) const

Public Member Functions inherited from [wbvector](#)< [QType](#) >

- **wbvector** (size_t l, [QType](#) x)
- **wbvector** (size_t l, const [QType](#) *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const [QType](#) *d0)
- [wbvector](#) & **init** (size_t l, const [QType](#) *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)

- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< QType > & **init** (const [wbsparray](#)< QType > &S)
- [wbvector](#) & **init** (size_t l1, const [QType](#) *d1, size_t l2, const [QType](#) *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const [QType](#) *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const [QType](#) &x)
- [wbvector](#) & **initStride** (size_t l, const [QType](#) *d0, size_t stride)
- [wbvector](#) & **init_stride** (const [QType](#) *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const [QType](#) *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< QType > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< QType > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< QType > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< QType > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< QType > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< QType > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const [QType](#) *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mprf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< QType > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const [QType](#) &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const [QType](#) &x)
- void **set** (size_t offset, const [QType](#) *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const [QType](#) &x)
- [wbvector](#) & **setMax** (const [QType](#) &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const [QType](#) &x)
- [wbvector](#) & **add** (const [QType](#) *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const [QType](#) &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const [QType](#) &x) const
- size_t **find1** () const

- `size_t count` (const [QType](#) &x) const
- `size_t icount` (const [QType](#) &x) const
- `size_t nnz` () const
- `size_t nnz` ([QType](#) eps) const
- `size_t numZeros` ([QType](#) eps=[QType](#)(1E-14)) const
- `size_t numEQ` (const [QType](#) &) const
- `size_t numGT` (const [QType](#) &) const
- `size_t numLT` (const [QType](#) &) const
- `wbindex & find` (const [QType](#) &x, [wbindex](#) &l, char iflag=0) const
- `wbindex & find` (const [QType](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- `wbindex & find` ([wbindex](#) &l) const
- `wbindex & find` ([wbindex](#) &l, [wbindex](#) &lx) const
- `wbvector< size_t > find` () const
- `wbindex & findGT` (const [QType](#) &x, [wbindex](#) &l, char iflag=0) const
- `wbindex & findGT` (const [QType](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- `wbindex & findzeros` ([wbindex](#) &l) const
- `wbindex & findRange` (const [QType](#) &x1, const [QType](#) &x2, [wbindex](#) &l, const char *w=NULL) const
- `int findClosestSorted` ([QType](#) r) const
- `int findClosest` ([QType](#) r) const
- `size_t findValues` (const [wbvector](#) &v, [wbindex](#) &l) const
- `wbvector & setp` (const [QType](#) *x)
- `void reset` ()
- `void cpy` (const [QType](#) *v)
- `void cpy` (const [QType](#) *x, size_t m)
- `wbvector & save2` ([wbvector](#) &v)
- `void swap` ([wbvector](#) &v)
- `wbvector & swap` (size_t i, size_t j)
- `wbvector & Collect` ([QType](#) &A1, [QType](#) &A2)
- `wbvector & Collect` ([QType](#) &A1, [QType](#) &A2, [QType](#) &A3)
- `wbvector & Skip` (size_t)
- `wbvector & Skip` (const WBINDEX &, char rflag=0)
- `wbvector & Move2_FE` (const WBINDEX &l, char FE_flag)
- `wbvector & Move2front` (const WBINDEX &l)
- `wbvector & Move2front` (size_t k)
- `wbvector & Move2end` (const WBINDEX &l)
- `wbvector & Move2end` (size_t k)
- `wbvector & mvLastTo` (size_t k)
- `wbvector & mvFirstTo` (size_t k)
- `bool checkSameLength` (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- `bool checkSameLength` (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- `void operator+=` (const T_ &a)
- `void operator+=` (const [wbvector](#) &b)
- `void operator-=` (const T_ &a)
- `void operator-=` (const [wbvector](#) &b)
- `wbvector & operator*=` (const T_ &a)
- `wbvector & operator/=` (const T_ &a)
- `wbvector operator*` (const T_ &a) const
- `wbvector operator+` (const T_ &a) const
- `wbvector operator+` (const [wbvector](#) &b) const
- `wbvector operator-` (const T_ &a) const
- `wbvector operator-` (const [wbvector](#) &b) const
- `wbvector & Plus` (const [QType](#) *b)
- `wbvector & Plus` (const [QType](#) *b, [QType](#) bfac)
- `wbvector & Plus` (const [wbvector](#) &b, [QType](#) bfac=1)

- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, [QType](#) bfac=1, [QType](#) cfac=0) const
- [wbvector](#) & **times** ([QType](#) fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- [QType](#) **sum** () const
- [QType](#) **sum** (size_t i1, const char *estr) const
- [QType](#) **sum** (size_t i1, size_t i2) const
- [QType](#) **cumsum_** ([wbvector](#) &S, char xflag=0) const
- [QType](#) **Cumsum** (char zflag=0) const
- [QType](#) **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< [QType](#) > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- [QType](#) **scalarProd** (const [wbvector](#) &v) const
- [QType](#) **scalarProd** (const [QType](#) *v) const
- [QType](#) **norm** () const
- [QType](#) **norm2** () const
- double **normDiff** (const [wbvector](#) &v, [QType](#) fac=1) const
- [QType](#) **normDiff2** (const [wbvector](#) &v, [QType](#) fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- [QType](#) **GSProject** (const [QType](#) *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const [QType](#) *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const [QType](#) &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const [QType](#) &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const [QType](#) &a, [QType](#) eps=1E-15) const
- bool **allUnequal** (const [QType](#) &a) const
- bool **any** () const
- bool **anyUnequal** (const [QType](#) &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const [QType](#) &x, size_t *k=NULL) const
- bool **anyEqual** (const [QType](#) &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const [QType](#) &x, size_t *k=NULL) const
- bool **anyGT** (const [QType](#) &a) const
- bool **anyGE** (const [QType](#) &a) const
- bool **anyLT** (const [QType](#) &a) const
- bool **anyLE** (const [QType](#) &a) const
- bool **allGT** (const [QType](#) &a) const
- bool **allGE** (const [QType](#) &a) const
- bool **allLE** (const [QType](#) &a) const
- bool **allLT** (const [QType](#) &a) const
- bool **allIn** (const [QType](#) &a1, const [QType](#) &a2) const
- bool **anySmallVals** (const [QType](#) eps=1E-15) const
- bool **isEqual** (const [QType](#) *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const

- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const [QType](#) &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [QType](#) & **el** (size_t i) const
- [QType](#) **elx** (size_t i, const [QType](#) &x) const
- const [QType](#) & **operator[]** (size_t i) const
- [QType](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- [QType](#) & **last** () const
- [QType](#) & **last** (size_t &i) const
- [QType](#) & **last** (T1 &i_) const
- [QType](#) & **p2last** () const
- [QType](#) * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [QType](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [QType](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< [QType](#) > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< [QType](#) > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const [QType](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [QType](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const [QType](#) *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const [QType](#) *v) const
- int **recLessE** (const [QType](#) *v) const
- int **recCompare** (const [QType](#) *v) const
- [QType](#) **max** () const
- [QType](#) **max** (size_t *k) const
- [QType](#) **max** (unsigned &k) const
- [QType](#) **max_** ([QType](#) x) const
- [QType](#) **min** () const

- **QType** **min** (size_t *k) const
- **QType** **min** (unsigned &k) const
- **QType** **min_** (**QType** x) const
- **QType** **aMax** (size_t *k=NULL) const
- **QType** **aMax** (unsigned &k) const
- **QType** **aMin** (char zflag=0, size_t *k=NULL) const
- **QType** **maxneg** (size_t *k=NULL) const
- **QType** **minpos** (size_t *k=NULL) const
- **QType** **avg** () const
- **QType** **std** () const
- void **range** (**QType** &xmin, **QType** &xmax) const
- **QType** **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, **QType** *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, **QType** *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- **QType** **prod** () const
- **QType** **prod** (**QType** x) const
- **QType** **prod** (size_t i, size_t i2) const
- **QType** **prod** (size_t i, size_t i2, **QType** x) const
- **QType** **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (**QType** &p1, **QType** &p2, const **QType** &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const

- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMxP (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base_d (const char tflag=0) const`
- `mxArray * toMx_offset (const QType &offset, const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tflag=0) const`
- `void add2MxStruct (mxArray *S, const char *vname, char tflag=0) const`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tflag=0) const`
- `void putg (const char *vname="ans", char tflag=0) const`
- `void putx (const char *vname="ans", char tflag=0) const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring toStr () const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring sizeStr () const`
- `wbstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const`
- `wbstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const`

Additional Inherited Members

Public Attributes inherited from [wbvector< QType >](#)

- [QType](#) * `data`
- `size_t len`
- `char isref`

Protected Member Functions inherited from [wbvector< QType >](#)

- [wbvector](#) & `RENEW (size_t n, const QType *d=NULL, size_t l=-1, char init=1)`
- [wbvector](#) & `RENEW_VAL (size_t n, const QType &x)`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch.cc`

4.73 Wb::rand Class Reference

Public Member Functions

- double **get_dfac** ()
- template<class T >
T & **rand_** (T &x)
- template<class T >
T & **rand_** (T &x, double fac)
- template<class T >
T & **rands** (T &x)
- template<class T >
T & **randb** (T &x)
- template<class T >
void **scale_rand** (T *x, size_t n, double fac)
- template<> double & **rand_** (double &x)
- template<> int & **rand_** (int &x)
- template<> unsigned & **rand_** (unsigned &x)
- template<> long & **rand_** (long &x)
- template<> size_t & **rand_** (size_t &x)
- template<> int & **rand_** (int &x, double fac)
- template<> unsigned & **rand_** (unsigned &x, double fac)
- template<> long & **rand_** (long &x, double fac)
- template<> size_t & **rand_** (size_t &x, double fac)
- template<> double & **randb** (double &x)
- template<> int & **randb** (int &x)
- template<> double & **rands** (double &x)
- template<> int & **rands** (int &x)
- template<> void **scale_rand** (int *x, size_t n, double fac)

The documentation for this class was generated from the following file:

- util.hh

4.74 RCStore Class Reference

Public Member Functions

- int **setupDirs** (const char *F=NULL, int L=0)
- template<class TQ, class TD >
int **save_CData** (const char *F, int L, const CData< TQ, TD > &A)
- template<class TQ, class TD >
int **load_CData** (const char *F, int L, const QSet< TQ > &Q, CData< TQ, TD > &A, char bflag=0)
- template<class TQ >
int **save_Std3** (const char *F, int L, const QType &t, const qset< TQ > &J12, const wvector< double > *c2eps=NULL) const
- template<class TQ >
int **load_Std3** (const char *F, int L, const QType &t, const qset< TQ > &J1, const qset< TQ > &J2, unsigned loadRC=0) const
- template<class TQ, class TD >
int **save_RSet** (const char *F, int L, const genRG_base< TQ, TD > &R, char xflag=0)

- `template<class TQ >`
`int load_RSet (const char *F, int L, const QType &t, const qset< TQ > &J, char vflag=0)`
- `int save_XMap (const char *F, int L, const cgc_contract_id< MTI > &idc) const`
- `int load_XMap (const char *F, int L, const cgc_contract_id< MTI > &idc) const`
- `template<class TQ >`
`int get_file_name (const char *F, int L, wstring &file, const QSet< TQ > &Q, const char *ext=NULL, char mflag=0) const`
- `template<class TQ, class TD >`
`int get_file_name (const char *F, int L, wstring &file, const genRG_base< TQ, TD > &R, const char *ext=NULL, char mflag=0) const`
- `int get_file_name (const char *F, int L, wstring &file, const cgc_contract_id< MTI > &idc, const char *ext=NULL, char mflag=0, QType *t=NULL) const`
- `size_t rlog (const QType &t, const char *F, int L, const char *fmt,...)`
- `size_t rlog (const QType &t, const char *F, int L, char vflag, const char *fmt,...)`
- `size_t vrclog (const QType &t, const char *F, int L, const char *fmt, va_list args)`
- `void Info (const char *F=0, int L=0)`
- `int is_new (const QType &t)`

Public Attributes

- [Wb::Path](#) `root`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_aux.cc`
- `clebsch_io.cc`

4.75 RIXS_para Class Reference

Public Member Functions

- `void init ()`
- `void setE0 (int it, double e0, double e2)`
- `void check (const char *F=0, int L=0, char init=0)`
- [Wb::String](#) `toStr () const`
- `mxArray * toMx () const`

Public Attributes

- [wbcomplex](#) `Einc`
- `double Eoffset`
- `double sigma`
- `double afac`
- `char wrxs`
- `double E0`
- `double E2`
- `unsigned dloc`
- `int iter`

The documentation for this class was generated from the following file:

- `nrgdata.hh`

4.76 RSpecData< TS > Class Template Reference

Public Member Functions

- [RSpecData](#) & **Append** (const double *op, const TS *cp, unsigned n)
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const

Public Attributes

- [wbvector](#)< double > **om**
- [wbvector](#)< TS > **cc**

The documentation for this class was generated from the following file:

- spectral.hh

4.77 RStore< TQ, TD > Class Template Reference

Public Member Functions

- **RStore** (const char *s=NULL)
- size_t **bufsize** (const [QType](#) &q)
- bool **isEmpty** (const [QType](#) &q)
- [genRG_struct](#)< TQ, TD > & **Buf** (const [QType](#) &q)
- [genRG_base](#)< TQ, TD > * **find_RSet** (const [QType](#) &q, const TQ *qptra)
- [genRG_struct](#)< TQ, TD > & **getRSet** (const char *F, int L, const [QType](#) &q, [qset](#)< TQ > *qvec=NULL)
- [genRG_base](#)< TQ, TD > & **getR** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- [genRG_base](#)< TQ, TD > & **getR** (const char *F, int L, const CDATA_TQ &S, unsigned k) const
- [wbMatrix](#)< TQ > **getZ** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- unsigned **qdim** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- mxArray * **toMx** (const [QType](#) &q=QTYPE_UNKNOWN, char bflag=1) const
- unsigned **LoadStore** (const char *F, int L, const char *file)
- unsigned **add** (const [QType](#) &q, const mxArray *S, unsigned k)
- void **Info** (const char *F=0, int L=0) const

Public Attributes

- map< [QType](#), [genRG_struct](#)< TQ, TD > > **buf**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc
- clebsch_old.cc

4.78 wblog::SBUF Class Reference

Public Member Functions

- **SBUF** (unsigned l0=wblog::SLEN)
- **SBUF** & **init** (unsigned l0=0)
- int **cat** (const char *s)
- int **catf** (const char *fmt,...)
- void **error_bounds** (const char *F, int L, const char *fct, const char *fmt)
- void **increase_size** (const char *F, int L, unsigned n)
- void **flush** (FILE *fid=stdout, char fflag=0)
- void **print** (const char *F=0, int L=0, const char *istr=0)
- int **skipEscCols** ()

Public Attributes

- unsigned **l**
- unsigned **slen**
- char * **sbuf**

The documentation for this class was generated from the following files:

- wblog.h
- wblog.c

4.79 Wb::SigHandler Class Reference

Public Member Functions

- **SigHandler** (const char *F, int L)
- int **check911** (char tflag=0)

Static Public Member Functions

- static int **check911_** (const char *F=NULL, int L=0)

Static Public Attributes

- static int **NC0** = 3
- static int **icount** = SigHandler::NC0
- static int **acount** = SigHandler::NC0

The documentation for this class was generated from the following file:

- gathered.hh

4.80 `sparseCollector2D< TD >` Class Template Reference

Public Member Functions

- `sparseCollector2D` (SPIDX_T m=0, SPIDX_T n=0)
- void `clear` ()
- void `init` (SPIDX_T m=0, SPIDX_T n=0)
- `sparseCollector2D` & `add` (const char *F, int L, SPIDX_T i, SPIDX_T j, const TD &a)
- void `getData` (`wbMatrix`< SPIDX_T > &IJ, `wbvector`< TD > &X)
- `mxAarray` * `toMx` () const

The documentation for this class was generated from the following file:

- `wbsparray.hh`

4.81 `sparseIndex2D< T >` Class Template Reference

Public Member Functions

- `sparseIndex2D` & `Setup` (char use_rmaj=0)
- T `cidx` (T j) const
- T `ridx` (T ic) const
- `wbstring` `sizeStr` () const

Public Attributes

- char `rmaj`
- `wbMatrix`< T > `IJ`
- `wbvector`< T > `S`
- `wbperm` `P`
- `wbvector`< T > `cidx`

The documentation for this class was generated from the following files:

- `wbindex.hh`
- `wbindex.cc`

4.82 Spectral< TS > Class Template Reference

Public Member Functions

- void **init** ()
- void **init** (const char *istr, unsigned Nops, double delta=0., double T_=0, unsigned Niter=0)
- void **initBuf** ()
- void **calcSpectralMoments** (const char *F, int L, unsigned n)
- void **Add** (const [wbararray](#)< double > &Om, const [wbararray](#)< TS > &CC, unsigned s, char dblock=0, char mflag=0)
- void **Add2buf** (const [wbararray](#)< double > &Om, const [wbararray](#)< TS > &CC, unsigned s)
- void **crossAddBuf** (const char minor=1, char vflag=1)
- void **savelter** (const char *F, int L, unsigned iter)
- void **getOmega** ([wbvector](#)< double > &om, unsigned &linfit) const
- void **getOmega** ([wbvector](#)< double > &om) const
- void **applyIROPfac** (const char *F, int L, const [wbvector](#)< double > &fac)
- void **getRawData** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, unsigned linfit=0, const char *isbuf="") const
- void **getPARTIAL** ([wbararray](#)< TS > &A1, [wbararray](#)< TS > &A2) const
- void **pairRawData** ()
- void **detailedBalance** (const double T, [wbvector](#)< double > &om, [wbMatrix](#)< TS > &aa) const
- void **dispSumRule** ()
- void **dispSumRule** (const [wbvector](#)< double > &om, const [wbMatrix](#)< TS > &A)
- void **getSmoothSpec** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, char vflag=1)
- void **getSmoothSpec** ([wbMatrix](#)< TS > &A, const [wbvector](#)< double > &om, char vflag=1)
- void **KKreal** ([wbMatrix](#)< [wbcomplex](#) > &GZ, [wbvector](#)< double > &OM, [wbvector](#)< double > &DOM) const

Public Attributes

- [wbMatrix](#)< TS > **Ap**
- [wbMatrix](#)< TS > **An**
- [wbMatrix](#)< TS > **Ap_buf**
- [wbMatrix](#)< TS > **An_buf**
- [wbvector](#)< TS > **Ar**
- [wbMatrix](#)< TS > **A0**
- [wbMatrix](#)< TS > **mspec**
- [wbararray](#)< TS > **AP**
- [wbararray](#)< TS > **AN**
- [wbstring](#) **istr**
- double **Temp**
- double **sigma**
- double **sigma2**
- double **alpha**
- double **Emin**
- double **Emax**
- double **eps**
- unsigned **nlog**

The documentation for this class was generated from the following file:

- spectral.hh

4.83 Wb::sptr< T > Class Template Reference

Public Member Functions

- void **init_def** ()
- T * **malloc_base** (size_t n, char init=1, MTYPE mt=MEM_DEF)
- T * **malloc_data** (size_t n, const T *d0=NULL, MTYPE mt=MEM_DEF)
- int **rm_dref** (char mflag=0)
- unsigned **add_dref** ()
- T * **save_dref** (const char *F=0, int L=0)
- T * **init2ref** (size_t n, const T *d0)
- int **isRef** () const
- **operator bool** () const
- bool **operator!** () const
- int **check_consistency** (char force=1) const
- [wbstring](#) **toStr** () const

Public Attributes

- MTYPE **mtype**
- size_t **len**
- unsigned **nref**
- T * **data**

The documentation for this class was generated from the following files:

- memlib.hh
- memlib.cc

4.84 STAT_DGEMM Class Reference

Public Member Functions

- void **account** (unsigned n0)

Public Attributes

- [wbvector](#)< unsigned > **nn**
- [wbvector](#)< unsigned long > **ss**

The documentation for this class was generated from the following file:

- wbbblas.hh

4.85 wblog::stdio_buf Class Reference

Public Member Functions

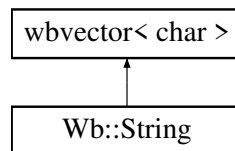
- **stdio_buf** (SBUF &S, FILE *f_, int t_)
- **stdio_buf** (const stdio_buf &b)
- void **print_stdout** (FILE *f=NULL) const

The documentation for this class was generated from the following file:

- wblog.h

4.86 Wb::String Class Reference

Inheritance diagram for Wb::String:



Public Member Functions

- **String** (unsigned n)
- **String** (const char *F, int L, unsigned n)
- int **pushf** (const char *fmt,...)

Public Member Functions inherited from [wbvector< char >](#)

- **wbvector** (size_t l, char x)
- **wbvector** (size_t l, const char *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const char *d0)
- [wbvector](#) & **init** (size_t l, const char *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & **init** (const [wbsparray](#)< char > &S)
- [wbvector](#) & **init** (size_t l1, const char *d1, size_t l2, const char *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const char *d0, size_t n, char init=1)

- [wbvector](#) & **init2val** (size_t l, const char &x)
- [wbvector](#) & **initStride** (size_t l, const char *d0, size_t stride)
- [wbvector](#) & **init_stride** (const char *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const char *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< char > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< char > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< char > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const char *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpfr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< char > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const char &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const char &x)
- void **set** (size_t offset, const char *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const char &x)
- [wbvector](#) & **setMax** (const char &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const char &x)
- [wbvector](#) & **add** (const char *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const char &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const char &x) const
- size_t **find1** () const
- size_t **count** (const char &x) const
- size_t **icount** (const char &x) const
- size_t **nnz** () const
- size_t **nnz** (char eps) const
- size_t **numZeros** (char eps=char(1E-14)) const
- size_t **numEQ** (const char &) const
- size_t **numGT** (const char &) const
- size_t **numLT** (const char &) const

- [wbindex](#) & **find** (const char &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const char &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const char &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const char &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const char &x1, const char &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (char r) const
- int **findClosest** (char r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const char *x)
- void **reset** ()
- void **cpy** (const char *v)
- void **cpy** (const char *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (char &A1, char &A2)
- [wbvector](#) & **Collect** (char &A1, char &A2, char &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const char *b)
- [wbvector](#) & **Plus** (const char *b, char bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, char bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, char bfac=1, char cfac=0) const
- [wbvector](#) & **times** (char fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- char **sum** () const
- char **sum** (size_t i1, const char *estr) const
- char **sum** (size_t i1, size_t i2) const
- char **cumsum_** ([wbvector](#) &S, char xflag=0) const

- char **Cumsum** (char zflag=0) const
- char **cumsum0prod** (wbvector< T2 > &b, wbvector< char > &cs, char xflag) const
- wbvector & **Eldiv** (const wbvector &v) const
- wbvector & **tensorProd** (const wbvector &v2, wbvector &vv) const
- char **scalarProd** (const wbvector &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const wbvector &v, char fac=1) const
- char **normDiff2** (const wbvector &v, char fac=1) const
- double **normDiff2_** (const wbvector &v, double fac=1) const
- char **GSPProject** (const char *v, char isnorm=0, char tflag=0) const
- wbvector & **Normalize** (const char *F=NULL, int L=0)
- wbvector & **subtract** (const char *d)
- void **getReal** (wbvector< double > &R) const
- void **getReal** (wbvector< double > &R) const
- void **getImag** (wbvector< double > &I) const
- void **getImag** (wbvector< double > &I) const
- wbvector & **Conj** ()
- wbvector< wbcomplex > & **Conj** ()
- wbvector & **applyFlag** (char &flag, const char &fac, wbvector &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const wbvector< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const wbvector< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const
- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const wbvector &v) const
- bool **isEmpty** () const
- operator bool () const
- bool **operator!** () const
- bool **operator==** (const wbvector &v) const
- bool **operator!=** (const wbvector &v) const
- bool **isEqualR** (const wbvector &v) const
- bool **operator^** (const wbvector &v) const
- bool **deepEqualP** (const wbvector &B) const
- char **cmp** (const wbvector &v) const
- bool **operator<** (const wbvector &v) const
- int **operator<** (const char &r) const

- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- char & **el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (T1 &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< char > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const char *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const
- char **max_** (char x) const
- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const

- char **avg** () const
- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, char *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, char *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const

- `mxArray * toMx_offset` (const char &offset, const char tflag=0) const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- `void add2MxStruct` (mxArray *S, unsigned i, char tflag=0) const
- `void add2MxStruct` (mxArray *S, const char *vname, char tflag=0) const
- `mxArray * mxCreateCell` (unsigned m, unsigned n) const
- `void add2MxCell` (mxArray *S, unsigned i, char tflag=0) const
- `void putg` (const char *vname="ans", char tflag=0) const
- `void putx` (const char *vname="ans", char tflag=0) const
- `wbstring toStr` (int n, const char *sep) const
- `wbstring toStr` () const
- `wbstring toStr` (int n, const char *sep) const
- `wbstring sizeStr` () const
- `wbstring toStrf` (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- `wbstring toStrf` (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from `wbvector< char >`

- `char * data`
- `size_t len`
- `char isref`

Protected Member Functions inherited from `wbvector< char >`

- `wbvector & RENEW` (size_t n, const char *d=NULL, size_t l=-1, char init=1)
- `wbvector & RENEW_VAL` (size_t n, const char &x)

The documentation for this class was generated from the following files:

- `wbstring.hh`
- `wbstring.cc`

4.87 SVD_Data< TQ, TD > Class Template Reference

Public Member Functions

- `void blockSVD` (const `QSpace< TQ, TD >` &PSI, const unsigned K)
- `unsigned dmrgTruncate` (unsigned Nkmin, unsigned Nkeep, double stol, `itag_` tx, `wbMatrix< double >` &SV, `QSpace< TQ, TD >` &UQ, `QSpace< TQ, TD >` *SQ, `QSpace< TQ, TD >` &VC, `mxArray *Sout=NULL`, char vflag=0)
- `mxArray * toMx` () const
- `void put` (const char *vname, const char *ws="base") const
- `void put` (const char *F, int L, const char *vname, const char *ws="base") const

Public Attributes

- [wbvector](#)< [QBlock](#)< TQ, TD, double > > **QB**

The documentation for this class was generated from the following files:

- mpsortho.hh
- mpsortho.cc

4.88 DY::Symmetry Class Reference

Public Member Functions

- **Symmetry** (const [QType](#) &q_)
- **Symmetry** (const mxArray *a)
- **Symmetry** & **init** (const [QType](#) &q)
- template<class TQ >
size_t **getWeightsFT** (const char *F, int L, const [qset](#)< TQ > &qs, [Weights](#)< TQ > &R)
- template<class TQ >
int **getInnerMultFT** (const [qset](#)< TQ > &qs, const [weight_info](#)< TQ > &w, [Weights](#)< TQ > &R, char vflag=0)
- template<class TQ >
int **Get** (const [qset](#)< TQ > &qk, map< [qset](#)< TQ >, [weight_info](#)< TQ > > &W, char vflag=0)
- template<class TQ >
int **wExpand** (const char *F, int L, [Weights](#)< TQ > &R) const
- mxArray * **toMx** () const

Public Attributes

- [QType](#) **q**
- unsigned **n**
- [wbarray](#)< double > **R**
- [wbMatrix](#)< int > **A**
- [wbvector](#)< int > **n2**
- [wbarray](#)< int > **M**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.89 TDSpectral< TS > Class Template Reference

Public Member Functions

- void **init** (const char *F, int L, const char *istr0, const [wbvector](#)< double > &t0, unsigned Nnrg, unsigned Nop, double Emin=1E-8, double Emax=10., unsigned nl=512, char vflag=1)
- void **checkAvgOm** ()
- [wbarray](#)< TS > **getAvgOm** ()
- void **Add** (const [wbarray](#)< double > &Om, const [wbarray](#)< TS > &CC, unsigned iter, unsigned s)
- void **addPartialFourier** ()
- void **getOmega** ([wbvector](#)< double > &om, unsigned &linfit) const
- void **getOmega** ([wbvector](#)< double > &om) const
- void **applyIROPfac** (const char *F, int L, const [wbvector](#)< double > &fac, char rixs=0)
- void **getSpecData** ([wbvector](#)< double > &om, [wbarray](#)< TS > &A, unsigned linfit=0) const
- void **getSpecData** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, unsigned linfit=0)
- void **mergeNRGData** ()
- void **getSmoothSpec_t** (double sigma, double eps, char bswitch=LOG_GAUSS_BRD, char vflag=1)
- void **Fourier** ([wbvector](#)< double > &om, [wbMatrix](#)< [wbcomplex](#) > &ST, double alpha, double Lambda, char vflag=1)
- void **Fourier** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, const [wbvector](#)< double > &tt, [wbMatrix](#)< [wbcomplex](#) > &at, double alpha=0., char vflag=1)
- void **dispSumRule** ()
- mxArray * **toMx** () const
- int **init** (const mxArray *S, double &emin, double &emax, unsigned &nlog)

Public Attributes

- [wbarray](#)< TS > **AP**
- [wbarray](#)< TS > **AN**
- [wbMatrix](#)< TS > **Ap**
- [wbMatrix](#)< TS > **An**
- [wbMatrix](#)< [wbcomplex](#) > **At**
- [wbvector](#)< double > **tt**
- [wbstring](#) **istr**
- unsigned **raw**
- [wbvector](#)< [RSpecData](#)< TS > > **RAW**

The documentation for this class was generated from the following file:

- spectral.hh

4.90 tensor_ref_ Class Reference

Public Member Functions

- **tensor_ref_** (const [tensor_ref_](#) &b)
- [tensor_ref_](#) & **init** (const [tensor_ref_](#) &b)
- [tensor_ref_](#) & **operator=** (const [tensor_ref_](#) &b)
- bool **operator==** (const [wbarray](#)< double > *A) const
- bool **operator==** (const [wbarray](#)< [wbcomplex](#) > *A) const

- `tensor_ref_ & init` (const `wbarray`< double > &A)
- `tensor_ref_ & init` (const `wbarray`< `wbcomplex` > &A)
- `operator bool` () const
- `bool operator!` () const
- `int check` (const char *F=NULL, int L=0) const
- `int rank` (unsigned r) const
- `bool sameSize` (const `wbvector`< size_t > &S) const
- `double xflops` () const
- `wbstring sizeStr` () const
- `const wbvector`< size_t > * `getSIZE` () const
- `template<class TC >`
`void contract` (const char *F, int L, const `ctrlidx` &ia, const `tensor_ref_` &b, const `ctrlidx` &ib, `wbarray`< TC > &C) const
- `template<>` `void contract` (const char *F, int L, const `ctrlidx` &ia, const `tensor_ref_` &b, const `ctrlidx` &ib, `wbarray`< double > &C) const
- `template<>` `void contract` (const char *F, int L, const `ctrlidx` &ia, const `tensor_ref_` &b, const `ctrlidx` &ib, `wbarray`< `wbcomplex` > &C) const

Public Attributes

- const `wbarray`< double > * **Ar**
- const `wbarray`< `wbcomplex` > * **Az**

The documentation for this class was generated from the following files:

- `wbarray.hh`
- `wbarray.cc`

4.91 tensorRef_ Class Reference

Public Member Functions

- `tensorRef_` (const `tensorRef_` &B)
- `template<class T >`
`tensorRef_` (const `wbarray`< T > &A, const `iTags` *t=NULL)
- `tensorRef_ & operator=` (const `tensorRef_` &B)
- `tensorRef_ & init` (const `tensorRef_` &B)
- `tensorRef_ & save2` (`tensorRef_` &B)
- `template<class T >`
`tensorRef_ & init` (const `wbarray`< T > &A, const `iTags` *t=NULL, char c=0)
- `size_t numel` () const
- `char check` (const char *F=NULL, int L=0) const
- `char Check` (const char *F, int L, unsigned &id_)
- `int UpdateItagsCtr` (const char *F, int L, const `ctrlidx` &ia, `tensorRef_` &b, const `ctrlidx` &ib, const char *tag)
- `template<class T >`
`int contract` (const char *F, int L, const `tensorRef_` &b, `tensorRef_` &c, double *flops, `wbarray`< T > *C) const
- `int contract` (const char *F, int L, const `tensorRef_` &b, `tensorRef_` &c, double &flops) const
- `template<class T >`
`int contract` (const char *F, int L, const `tensorRef_` &b, `tensorRef_` &c, `wbarray`< T > &C) const
- `wbstring toStr` () const
- `const tensorRef_ & print` (const char *istr=NULL, char vflag=1) const

Public Attributes

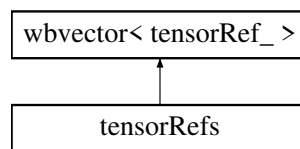
- unsigned **id**
- char **conj**
- [tensor_ref_ R](#)
- [iTags it](#)
- [wbvector](#)< size_t > **S**
- [wbvector](#)< unsigned > **ID**

The documentation for this class was generated from the following files:

- wbararray.hh
- wbararray.cc

4.92 tensorRefs Class Reference

Inheritance diagram for tensorRefs:

**Public Member Functions**

- **tensorRefs** (unsigned l=0)
- char **check** (const char *F, int L) const
- char **Check** (const char *F, int L)
- void **print** ()
- void **print** (const char *F, int L) const
- double **getOptimalCtrOrder** (const char *F, int L, [wbperm](#) &P, [wbperm](#) *pfin=NULL, char useP=0, char vflag=0)

Public Member Functions inherited from [wbvector](#)< [tensorRef_](#) >

- **wbvector** (size_t l, [tensorRef_](#) x)
- **wbvector** (size_t l, const [tensorRef_](#) *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const [tensorRef_](#) *d0)
- [wbvector](#) & **init** (size_t l, const [tensorRef_](#) *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< [tensorRef_](#) > & **init** (const [wbsparray](#)< [tensorRef_](#) > &S)

- [wbvector](#) & [init](#) (size_t l1, const [tensorRef_](#) *d1, size_t l2, const [tensorRef_](#) *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const [tensorRef_](#) *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const [tensorRef_](#) &x)
- [wbvector](#) & [initStride](#) (size_t l, const [tensorRef_](#) *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const [tensorRef_](#) *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const [tensorRef_](#) *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< [tensorRef_](#) > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< [tensorRef_](#) > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< [tensorRef_](#) > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< [tensorRef_](#) > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< [tensorRef_](#) > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< [tensorRef_](#) > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const [tensorRef_](#) *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)
- [wbvector](#) & [operator=](#) (const [wbvector](#) &v)
- [wbvector](#)< [tensorRef_](#) > & [operator=](#) (const [wbvector](#)< TB > &v)
- [wbvector](#) & [set](#) (const [tensorRef_](#) &x)
- [wbvector](#) & [set](#) (const [wbvector](#)< size_t > &l, const [tensorRef_](#) &x)
- void [set](#) (size_t offset, const [tensorRef_](#) *x, size_t n)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & [setMin](#) (const [tensorRef_](#) &x)
- [wbvector](#) & [setMax](#) (const [tensorRef_](#) &x)
- [wbvector](#) & [add](#) (const [wbvector](#)< size_t > &l, const [tensorRef_](#) &x)
- [wbvector](#) & [add](#) (const [tensorRef_](#) *d)
- [wbvector](#) & [Set](#) (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void [Sort](#) ([wbperm](#) &p, char dir=+1)
- void [Sort](#) (char dir=+1)
- void [sort](#) ([wbvector](#) &v2, [wbperm](#) &p) const
- void [sort](#) ([wbvector](#) &v2) const
- [wbperm](#) & [pSort](#) ([wbperm](#) &p) const
- bool [isSorted](#) () const
- bool [isSortedU](#) () const
- bool [isUnique](#) () const
- int [contains](#) (const [tensorRef_](#) &x) const
- bool [isNormal](#) () const
- bool [isNormal](#) () const
- bool [isNormal](#) () const
- bool [isFinite](#) () const
- size_t [find1](#) (const [tensorRef_](#) &x) const
- size_t [find1](#) () const
- size_t [count](#) (const [tensorRef_](#) &x) const
- size_t [icount](#) (const [tensorRef_](#) &x) const

- `size_t nnz ()` const
- `size_t nnz (tensorRef_ eps)` const
- `size_t numZeros (tensorRef_ eps=tensorRef_(1E-14))` const
- `size_t numEQ (const tensorRef_ &)` const
- `size_t numGT (const tensorRef_ &)` const
- `size_t numLT (const tensorRef_ &)` const
- `wbindex & find (const tensorRef_ &x, wbindex &l, char iflag=0)` const
- `wbindex & find (const tensorRef_ &x, wbindex &l, wbindex &lx)` const
- `wbindex & find (wbindex &l)` const
- `wbindex & find (wbindex &l, wbindex &lx)` const
- `wbvector< size_t > find ()` const
- `wbindex & findGT (const tensorRef_ &x, wbindex &l, char iflag=0)` const
- `wbindex & findGT (const tensorRef_ &x, wbindex &l, wbindex &lx)` const
- `wbindex & findzeros (wbindex &l)` const
- `wbindex & findRange (const tensorRef_ &x1, const tensorRef_ &x2, wbindex &l, const char *w=NULL)` const
- `int findClosestSorted (tensorRef_ r)` const
- `int findClosest (tensorRef_ r)` const
- `size_t findValues (const wbvector &v, wbindex &l)` const
- `wbvector & setp (const tensorRef_ *x)`
- `void reset ()`
- `void cpy (const tensorRef_ *v)`
- `void cpy (const tensorRef_ *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (tensorRef_ &A1, tensorRef_ &A2)`
- `wbvector & Collect (tensorRef_ &A1, tensorRef_ &A2, tensorRef_ &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="")` const
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="")` const
- `void operator+= (const T_ &a)`
- `void operator+= (const wbvector &b)`
- `void operator-= (const T_ &a)`
- `void operator-= (const wbvector &b)`
- `wbvector & operator*= (const T_ &a)`
- `wbvector & operator/= (const T_ &a)`
- `wbvector operator* (const T_ &a)` const
- `wbvector operator+ (const T_ &a)` const
- `wbvector operator+ (const wbvector &b)` const
- `wbvector operator- (const T_ &a)` const
- `wbvector operator- (const wbvector &b)` const
- `wbvector & Plus (const tensorRef_ *b)`
- `wbvector & Plus (const tensorRef_ *b, tensorRef_ bfac)`
- `wbvector & Plus (const wbvector &b, tensorRef_ bfac=1)`
- `void plus (const wbvector &b, wbvector &c, tensorRef_ bfac=1, tensorRef_ cfac=0)` const
- `wbvector & times (tensorRef_ fac, wbvector &x)` const

- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- [tensorRef_](#) **sum** () const
- [tensorRef_](#) **sum** (size_t i1, const char *estr) const
- [tensorRef_](#) **sum** (size_t i1, size_t i2) const
- [tensorRef_](#) **cumsum**_ ([wbvector](#) &S, char xflag=0) const
- [tensorRef_](#) **Cumsum** (char zflag=0) const
- [tensorRef_](#) **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< [tensorRef_](#) > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- [tensorRef_](#) **scalarProd** (const [wbvector](#) &v) const
- [tensorRef_](#) **scalarProd** (const [tensorRef_](#) *v) const
- [tensorRef_](#) **norm** () const
- [tensorRef_](#) **norm2** () const
- double **normDiff** (const [wbvector](#) &v, [tensorRef_](#) fac=1) const
- [tensorRef_](#) **normDiff2** (const [wbvector](#) &v, [tensorRef_](#) fac=1) const
- double **normDiff2**_ (const [wbvector](#) &v, double fac=1) const
- [tensorRef_](#) **GSProject** (const [tensorRef_](#) *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const [tensorRef_](#) *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const [tensorRef_](#) &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const [tensorRef_](#) &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const [tensorRef_](#) &a, [tensorRef_](#) eps=1E-15) const
- bool **allUnequal** (const [tensorRef_](#) &a) const
- bool **any** () const
- bool **anyUnequal** (const [tensorRef_](#) &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const [tensorRef_](#) &x, size_t *k=NULL) const
- bool **anyEqual** (const [tensorRef_](#) &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const [tensorRef_](#) &x, size_t *k=NULL) const
- bool **anyGT** (const [tensorRef_](#) &a) const
- bool **anyGE** (const [tensorRef_](#) &a) const
- bool **anyLT** (const [tensorRef_](#) &a) const
- bool **anyLE** (const [tensorRef_](#) &a) const
- bool **allGT** (const [tensorRef_](#) &a) const
- bool **allGE** (const [tensorRef_](#) &a) const
- bool **allLE** (const [tensorRef_](#) &a) const
- bool **allLT** (const [tensorRef_](#) &a) const
- bool **allIn** (const [tensorRef_](#) &a1, const [tensorRef_](#) &a2) const
- bool **anySmallVals** (const [tensorRef_](#) eps=1E-15) const
- bool **isEqual** (const [tensorRef_](#) *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const

- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const [tensorRef_](#) &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [tensorRef_](#) & **el** (size_t i) const
- [tensorRef_](#) **elx** (size_t i, const [tensorRef_](#) &x) const
- const [tensorRef_](#) & **operator[]** (size_t i) const
- [tensorRef_](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#) < size_t > &l) const
- [tensorRef_](#) & **last** () const
- [tensorRef_](#) & **last** (size_t &i) const
- [tensorRef_](#) & **last** (T1 &i_) const
- [tensorRef_](#) & **p2last** () const
- [tensorRef_](#) * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#) < [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [tensorRef_](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [tensorRef_](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3=0, const [wbvector](#) < T3 > *v4=0)
- [wbvector](#) < [tensorRef_](#) > & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3, const [wbvector](#) < T3 > *v4)
- [wbvector](#) < [tensorRef_](#) > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const [tensorRef_](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [tensorRef_](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const [tensorRef_](#) *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const [tensorRef_](#) *v) const
- int **recLessE** (const [tensorRef_](#) *v) const
- int **recCompare** (const [tensorRef_](#) *v) const
- [tensorRef_](#) **max** () const
- [tensorRef_](#) **max** (size_t *k) const
- [tensorRef_](#) **max** (unsigned &k) const
- [tensorRef_](#) **max_** ([tensorRef_](#) x) const
- [tensorRef_](#) **min** () const
- [tensorRef_](#) **min** (size_t *k) const
- [tensorRef_](#) **min** (unsigned &k) const

- [tensorRef_ min_](#) ([tensorRef_ x](#)) const
- [tensorRef_ aMax](#) (size_t *k=NULL) const
- [tensorRef_ aMax](#) (unsigned &k) const
- [tensorRef_ aMin](#) (char zflag=0, size_t *k=NULL) const
- [tensorRef_ maxneg](#) (size_t *k=NULL) const
- [tensorRef_ minpos](#) (size_t *k=NULL) const
- [tensorRef_ avg](#) () const
- [tensorRef_ std](#) () const
- void [range](#) ([tensorRef_](#) &xmin, [tensorRef_](#) &xmax) const
- [tensorRef_ range](#) () const
- [wbvector](#) & [Permute](#) (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & [permute](#) ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) [permute](#) (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & [blockPermute](#) (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & [BlockPermute](#) (const [wbperm](#) &P, char iflag=0)
- int [blockCompare](#) (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void [get](#) (const WBINDEX &l0, [wbvector](#) &v) const
- void [get](#) (const [wbvector](#)< size_t > &l, [tensorRef_](#) *v) const
- void [get](#) (size_t i1, size_t i2, [wbvector](#) &v) const
- void [selectSU](#) (const WBINDEX &l)
- void [select](#) (const WBINDEX &l, [tensorRef_](#) *r) const
- [wbvector](#) & [select](#) (const WBINDEX &l, [wbvector](#) &a) const
- [wbvector](#) [select](#) (const WBINDEX &l) const
- [wbvector](#) & [Select](#) (const WBINDEX &l)
- [wbvector](#) & [BlockSelect](#) (const WBINDEX &l, const WBINDEX &D)
- [wbvector](#) & [getl](#) (size_t k, [wbvector](#) &v) const
- [wbvector](#) & [getl](#) (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & [getl](#) (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & [initl](#) (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & [initl](#) (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & [Flip](#) ()
- void [flip](#) ([wbvector](#) &a) const
- [wbvector](#) [flip](#) () const
- [tensorRef_ prod](#) () const
- [tensorRef_ prod](#) ([tensorRef_ x](#)) const
- [tensorRef_ prod](#) (size_t i, size_t i2) const
- [tensorRef_ prod](#) (size_t i, size_t i2, [tensorRef_ x](#)) const
- [tensorRef_ prod](#) (const T1 *l, size_t n) const
- T2 [prod](#) (const T1 *l, size_t n, T2 &s2) const
- void [prod2](#) ([tensorRef_](#) &p1, [tensorRef_](#) &p2, const [tensorRef_](#) &p0=0) const
- void [info](#) (const char *istr) const
- void [print](#) (const char *istr, char mflag=0) const
- void [printdata](#) (const char *istr=0) const
- void [put](#) (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void [put](#) (const char *vname="ans", char tflag=0, const char *ws="base") const
- void [dispWithMsg](#) (const char *file, int line, const char *msg, const char *vname="") const
- void [mat2mx](#) (mxArray *&a, char dim=2) const
- void [mat2mxs](#) (mxArray *a, char field_nr, char dim=2) const
- mxArray * [toMx_Struct](#) (const char tflag=0) const
- mxArray * [toMxP_S](#) (const char tflag=0) const
- mxArray * [toMx](#) (const char tflag=0) const
- mxArray * [toMx](#) (const char tflag) const
- mxArray * [toMx](#) (const char tflag) const
- mxArray * [toMx](#) (const char tflag) const
- mxArray * [toMx](#) (const char tflag) const

- `mxArray * toMx (const char tflag) const`
- `mxArray * toMxP (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base_d (const char tflag=0) const`
- `mxArray * toMx_offset (const tensorRef\_ &offset, const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tflag=0) const`
- `void add2MxStruct (mxArray *S, const char *vname, char tflag=0) const`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tflag=0) const`
- `void putg (const char *vname="ans", char tflag=0) const`
- `void putx (const char *vname="ans", char tflag=0) const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring toStr () const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring sizeStr () const`
- `wbstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const`
- `wbstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const`

Additional Inherited Members

Public Attributes inherited from [wbvector< \[tensorRef_\]\(#\) >](#)

- `tensorRef\_ * data`
- `size_t len`
- `char isref`

Protected Member Functions inherited from [wbvector< \[tensorRef_\]\(#\) >](#)

- `wbvector & RENEW (size_t n, const tensorRef\_ *d=NULL, size_t l=-1, char init=1)`
- `wbvector & RENEW_VAL (size_t n, const tensorRef\_ &x)`

The documentation for this class was generated from the following files:

- `wbarray.hh`
- `wbarray.cc`

4.93 Wb::termcolor Class Reference

Public Member Functions

- `termcolor (const char *tag)`
- `termcolor (Wb::TCOLS k)`
- `termcolor (unsigned k)`
- `termcolor & init (unsigned k)`

Static Public Member Functions

- static int **ID** (const char *tag)

Public Attributes

- char * **em**
- char * **e1**

The documentation for this class was generated from the following files:

- wblog.h
- wblog.c

4.94 Wb::tmpFile Class Reference

Public Member Functions

- **tmpFile** (const char *F, int L, const char *f=NULL, const char *x=NULL, const char *mode=NULL)
- void **init** (const char *F, int L, const char *f=NULL, const char *x=NULL, const char *mode=NULL)
- int **blogf** (const char *F, int L, const char *fmt,...)
- int **sepline** (const char *w="-", unsigned len=-1)
- void **flush** ()

Public Attributes

- FILE * **fid**
- int **ncall**
- [wbstring](#) **fout**

The documentation for this class was generated from the following files:

- wbio.h
- wbio.c

4.95 Wb::VersionInfo Class Reference

Public Member Functions

- [VersionInfo](#) & **init** ()
- [wbstring](#) **toStr** () const
- void **print** () const
- mxArray * **toMx** () const

Public Attributes

- char **fctn** [32]
- char **tag** [8]
- char **matlab** [16]
- char **mpfr** [16]
- char **compiler** [16]
- char **os** [24]
- char **qspace** [16]
- char **git** [16]
- char **flags** [128]
- char **compiled** [64]

The documentation for this class was generated from the following files:

- wblibx.h
- wblog.c

4.96 wbarry< T > Class Template Reference**Public Member Functions**

- **wbarry** (size_t d1)
- **wbarry** (size_t d1, size_t d2)
- **wbarry** (size_t d1, size_t d2, size_t d3)
- **wbarry** (size_t d1, size_t d2, size_t d3, size_t d4)
- **wbarry** (char *sidx)
- **wbarry** (const [wbvector](#)< unsigned > &S)
- **wbarry** (const [wbvector](#)< size_t > &S, T *d0, char ref=0)
- **wbarry** (const [wbarry](#) &a, char ref=0)
- template<class T2 >
 wbarry (const [wbarry](#)< T2 > &a)
- **wbarry** (const mxArray *a, char ref=0, char vec=0)
- **wbarry** (const char *F, int L, const mxArray *a, char ref=0, char vec=0)
- **wbarry** (const [wbMatrix](#)< T > &a, char ref=0)
- [wbarry](#) & **initX** ()
- [wbarry](#) & **init** ()
- [wbarry](#) & **init** (size_t d1, const T *d=NULL)
- [wbarry](#) & **init** (size_t d1, size_t d2, const T *d=NULL)
- [wbarry](#) & **init** (size_t d1, size_t d2, size_t d3, const T *d=NULL)
- [wbarry](#) & **init** (size_t d1, size_t d2, size_t d3, size_t d4, const T *d=NULL)
- [wbarry](#) & **init2ref** (const [wbarry](#) &a)
- [wbarry](#) & **initScalar** (T x=0, unsigned r=2)
- [wbarry](#) & **init** (const [wbarry](#) &a, char ref=0)
- [wbarry](#) & **set** (const [wbarry](#) &A)
- template<class T_ >
 [wbarry](#) & **set** (const [wbarry](#) &A, T_ fac)
- [wbarry](#) & **init** (const [wbMatrix](#)< T > &a, char ref=0)
- [wbindx](#) & **ind2sub** (size_t k, [wbindx](#) &l) const
- void **adjustMMat** (const char *F, int L, char flag, const T fac, [wbarry](#) &A) const
- void **adjustDMat** (const char *F, int L, char flag, const T fac, [wbarry](#) &A) const
- void **adjustVec** (const char *F, int L, char &flag, [wbarry](#) &A, char pos) const

- void **adjustVec** (const char *F, int L, char &flag, char pos)
- void **adjustCMat** (const char *F, int L, T cfac, size_t s1, size_t s2, char cforce)
- void **setBlock** (const [wbvector](#)< WBINDEX > &D, const [wbindex](#) &IB, const [wbarray](#) &A)
- void **addBlock** (const WBINDEX &l, const [wbarray](#) &A, const char dflag=0)
- [wbarray](#) & **addBlock** (size_t i, size_t j, size_t n, size_t m, [wbarray](#) &a) const
- T **norm2block** (size_t i, size_t j, size_t n, size_t m) const
- [wbarray](#) & **getBlock** (size_t i, size_t j, size_t n, size_t m, [wbarray](#) &a) const
- [wbarray](#) & **BlockDiag** (const [wbvector](#)< [wbarray](#) > &D)
- [wbarray](#) & **blockTrace** (size_t D, [wbarray](#) &a, size_t D2=-1) const
- void **copyStride** (T *dd, size_t stride, T afac=1) const
- void **addStride** (T *dd, size_t stride, T afac=1) const
- template<class T0 >
 [wbarray](#) & **init_T** (const T0 *dd)
- template<class T0 >
 [wbarray](#) & **init_T** (const char *F, int L, const T0 *dd)
- [wbarray](#) & **initT** (const [wbarray](#)< T > &a)
- template<class T0 >
 [wbarray](#) & **initT** (const [wbarray](#)< T0 > &a)
- template<class T0 >
 [wbarray](#) & **initT** (const char *F, int L, const [wbarray](#)< T0 > &a)
- [wbarray](#) & **init** (char *s, T *d=NULL)
- [wbarray](#) & **init** (const WBINDEX &S, T *d=NULL)
- template<class IT >
 [wbarray](#) & **init** (const [wbvector](#)< IT > &S_, T *d=NULL)
- [wbarray](#) & **init_bare** (const WBINDEX &S)
- [wbarray](#) & **init_bare** (size_t d1, size_t d2)
- [wbarray](#) & **init** (const char *F, int L, const mxArray *a, char ref=0, char vec=0)
- [wbarray](#) & **init** (const char *F, int L, const mxArray *a, [wbperm](#) &P0)
- [wbarray](#) & **init** (const mxArray *a, char ref=0, char vec=0)
- [wbarray](#) & **init** (const mxArray *a, [wbperm](#) &P0)
- [wbarray](#) & **init2ref** (const T *x, const WBINDEX &S)
- [wbarray](#) & **init2ref** (const T *x)
- [wbarray](#) & **init2ref** (size_t d1, const T *x)
- [wbarray](#) & **init2ref** (size_t d1, size_t d2, const T *x)
- [wbarray](#) & **init2ref** (size_t d1, size_t d2, size_t d3, const T *x)
- int **isRef** () const
- [wbarray](#) & **Instantiate** (const char *F=NULL, int L=0)
- [wbarray](#) & **unRef** ()
- void **init2Vec** (const T *d0, size_t len, char ref=0)
- void **init2ref** (const [wbarray](#) &A, size_t s1, size_t s2)
- void **initTst** ()
- void **init2Vec** (const [wbvector](#)< T > &a)
- [wbarray](#) & **initIdentity** (size_t d, char dflag=0, T dval=1)
- [wbarray](#) & **initIdentity** (const WBINDEX &S, char dflag=0)
- [wbarray](#) & **Reduce2Id** ()
- [wbarray](#) & **initIdentityB** (size_t d1, size_t d2, size_t k=0)
- [wbarray](#) & **initIdentityB3** (size_t d1, size_t d2, size_t D, size_t i0, T x=1)
- [wbarray](#) & **Expand2Projector** (const T eps=0)
- [wbarray](#) & **ExpandDiagonal** (unsigned i1, unsigned i2)
- [wbarray](#) & **ExpandDiagonal** ()
- void **Diag2Vec** ()
- [wbarray](#) & **Reduce2Diag** (T eps=0)
- [wbarray](#) & **initDiag** (unsigned n, const T *d)
- [wbarray](#) & **initDiag** (const [wbvector](#)< T > &d)
- void **initVec** (size_t d, const T val=0)

- [wbarry](#) & **resize** (const [wbvector](#)< size_t > &S, [wbarry](#) &B) const
- [wbarry](#) & **resizeNumCols** (size_t n, [wbarry](#) &B) const
- void **Resize** (const [wbvector](#)< size_t > &S, const T *d0=NULL)
- void **Resize** (size_t s1, size_t s2)
- [wbarry](#) & **Resize1** (unsigned i, size_t Si)
- [wbarry](#) & **ResizeM** (unsigned r, const [wbvector](#)< size_t > &Sm)
- [wbarry](#) & **Enlarge** (size_t dim, size_t D, size_t istart=0)
- void **swap** ([wbarry](#) &A)
- [wbarry](#) & **save2** ([wbarry](#) &A)
- [wbarry](#) & **save2** ([wbMatrix](#)< T > &)
- [wbarry](#) & **Append2** (const char *F, int L, [wbarry](#) &B, unsigned dim)
- [wbarry](#) & **Append2** ([wbarry](#) &B, unsigned dim)
- void **Append2** (const char *F, int L, [wbarry](#) &B, unsigned dim, [wbarry](#)< double > &W, double eps)
- void **Append2** ([wbarry](#) &B, unsigned dim, [wbarry](#) &W, double eps)
- void **rand** (const UVEC &d)
- void **setRand** (const char pflag=0)
- [wbarry](#) & **Reset** (const T *dref)
- [wbarry](#) & **set** (const T &c)
- [wbarry](#) & **operator=** (const [wbarry](#) &a)
- template<class T0 >
 [wbarry](#)< T > & **operator=** (const [wbarry](#)< T0 > &a)
- void **SetRef** (const widx_t *l, size_t len, const T *d0)
- void **SetRef** (size_t k, const T *d0)
- const T * **ref** (const widx_t *l, size_t n) const
- T * **ref** (const widx_t *l, size_t n)
- T * **col** (size_t k)
- const T * **col** (size_t k) const
- T * **row** (size_t k)
- const T * **row** (size_t k) const
- [wbvector](#)< T > & **getCol** (size_t k, [wbvector](#)< T > &v) const
- [wbarry](#)< T > & **getCol** (size_t k, [wbarry](#)< T > &a) const
- [wbvector](#)< T > & **getRow** (size_t k, [wbvector](#)< T > &v) const
- [wbarry](#)< T > & **getRow** (size_t k, [wbarry](#)< T > &a) const
- [wbvector](#)< T > **getDiag** () const
- T * **ref** (const WBINDEX &l)
- const T * **ref** (const WBINDEX &l) const
- const T * **ref** (widx_t i) const
- T * **ref** (widx_t i)
- const T * **ref** (widx_t i1, widx_t i2) const
- T * **ref** (widx_t i1, widx_t i2)
- const T * **ref** (widx_t i1, widx_t i2, widx_t i3) const
- T * **ref** (widx_t i1, widx_t i2, widx_t i3)
- const T & **operator[]** (size_t i) const
- T & **operator[]** (size_t i)
- const T & **operator()** (size_t i) const
- T & **operator()** (size_t i)
- const T & **operator()** (size_t i, size_t j) const
- T & **operator()** (size_t i, size_t j)
- const T & **operator()** (size_t i, size_t j, size_t k) const
- T & **operator()** (size_t i, size_t j, size_t k)
- const T & **operator()** (const WBINDEX &l) const
- T & **operator()** (const WBINDEX &l)
- const T & **element** (const WBINDEX &l) const
- T & **element** (const WBINDEX &l)
- size_t **numel** () const

- `size_t numOM` (unsigned r0) const
- `wbstring sizeStr` (unsigned stride=0, const char *sep2="") const
- `wbarray & Squeeze` ()
- `int skipSingletons` (const char *F, int L, unsigned r=-1)
- `int skipSingletons` (unsigned r=-1)
- `wbarray & appendSingletons` (const char *F, int L, unsigned r, unsigned m=0)
- `wbarray & appendSingletons` (unsigned r, unsigned m=0)
- `void prependSingletons` (unsigned r)
- `wbarray & ExpandOM` (const char *F, int L, unsigned r0, const `wbvector`< unsigned > &S, const unsigned *sx=NULL)
- `wbarray & ExpandOM` (const char *F, int L, unsigned r0, unsigned m, const unsigned *s, const unsigned *sx=NULL)
- `int ExpandOM` (const char *F, int L, `wbarray` &B, unsigned r0)
- `wbarray & FuseOM` (const char *F, int L, unsigned r, unsigned mc=-1)
- `size_t length` () const
- `size_t size` (unsigned i) const
- `size_t dim` (unsigned i) const
- `size_t dim0` (unsigned i) const
- `size_t dim1` () const
- `size_t dim2` () const
- `void getMatSize` (const char *F, int L, `size_t` &dim1, `size_t` &dim2) const
- `void getMatSize` (`size_t` &dim1, `size_t` &dim2, unsigned m) const
- `void getMatSize` (const `ctrlidx` &ic, `size_t` &dimc, `size_t` &dimk, unsigned r=-1, `size_t` *dom=NULL) const
- unsigned `rank` () const
- bool `isRank` (unsigned r) const
- bool `isRankM` (unsigned r, unsigned m) const
- bool `isOpS` (`size_t` *n=NULL) const
- bool `isOpS` (unsigned &n) const
- `wbarray & SkipTiny_float` (const T eps=1E-14)
- double `SkipTiny_imag` (double eps=1E-14)
- double `SkipTiny` (double eps_=1E-14)
- `wbarray`< T > & `SkipTrailingZeroSpace` (unsigned k, `size_t` n=0, double eps=1e-14)
- bool `isMatrix` () const
- bool `isMatrix` (`size_t` d1, `size_t` d2)
- bool `isSMatrix` (unsigned d=-1) const
- bool `isSquare` () const
- bool `isEmpty` () const
- bool `isComplex` () const
- bool `isZero` (double eps=0., char flag=0) const
- bool `isFinite` () const
- `operator bool` () const
- bool `operator!` () const
- `wbarray & Symmetrize` (const char *F, int L, double *delta=NULL, char cflag=1, char tflag=0, char fflag=0)
- `wbarray & swapRows` (`size_t` i1, `size_t` i2)
- `wbarray & swapCols` (`size_t` j1, `size_t` j2)
- `wbarray & setCol` (`size_t` k, `size_t` k0)
- `wbarray & setCol` (`size_t` k, const T *d, T fac=1, `size_t` stride=1)
- `wbarray & setRow` (`size_t` k, const T *d)
- `wbarray & setCol` (`size_t` k, const `wbvector`< T > &v)
- template<class T2 >
 `wbarray`< T > & `setCol` (`size_t` k, const `wbvector`< T2 > &v)
- `wbvector`< T > & `colNorm2` (`wbvector`< T > &a) const
- T `colNorm2` (`size_t` k) const
- `wbvector`< T > & `normDim2` (unsigned k, `wbvector`< T > &xk) const
- `wbvector`< T > & `normDim` (unsigned k, `wbvector`< T > &xk) const

- unsigned **QRdecomp** (const char *F, int L, [wbarray< T >](#) &Q, [wbarray< T >](#) &R, T eps=1E-15)
- int **Householder** (const char *F, int L, size_t k, T *u, WPERM *P=NULL, T eps=1E-15)
- unsigned **eigTriDiag** (const char *F, int L, [wbvector< T >](#) &E, [wbarray< T >](#) *U=NULL, T eps=1E-15) const
- [wbarray](#) & **ColProject** (size_t k1, size_t k2, char nflag=0, char tnorm=0)
- [wbarray](#) & **ColPermute** (const [wbperm](#) &P, char iflag=0)
- bool **isOrthogonalCol** (size_t k, char tnorm=0) const
- bool **isOrthoCols** (T *x2=NULL, char tnorm=0, T eps=1E-14) const
- [wbarray](#) & **SignCol** (size_t k, const T *d0, char tnorm=0)
- [wbarray](#) & **FlipSignCol** (size_t k)
- [wbarray](#) & **SignConventionCol** (size_t k=-1, double eps=1E-12)
- T **NormalizeCol** (size_t k, char tnorm=0, char qflag=0)
- [wbarray](#) & **NormalizeCols** (const char *F, int L, double *amin=NULL, double *amax=NULL, char tnorm=0)
- [wbarray](#) & **OrthoNormalizeCols** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, double eps=1E-14, unsigned np=1)
- [wbarray](#) & **balanceOp** (const char *F, int L, T &xref, double &xscale)
- bool **isProptold** (T &x, T eps=1E-14) const
- bool **isProptold** (T *x_=NULL, T eps=1E-14) const
- bool **isDiagMatrix** (double eps=1E-14) const
- bool **isIdentityMatrix** (double eps=1E-14) const
- bool **isDiagMatrix** (double *eps) const
- bool **isIdentityMatrix** (double *eps) const
- size_t **nnz** (const T eps=0, WBINDEX *I=NULL) const
- double **sparsity** (const T eps=0) const
- bool **isHConj** (const [wbarray](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isHConj** (double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (const [wbarray](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (double eps=1E-12, double *xref=NULL) const
- char **hasGroupSize** (size_t s1, size_t s2) const
- template<class TI >
int **aMax** (T &x, TI &k) const
- T **aMax** () const
- template<class TI >
T **aMax** (TI &i, TI &j, const [wbarray](#) *B=NULL) const
- double **maxDiff** (const [wbarray](#) &B) const
- T **froNorm2** (const [wbarray](#) &B) const
- T **norm2** () const
- T **norm** () const
- T **hasNorm2** (const T &x2) const
- T **normCol2** (size_t k) const
- T **norm2Recs** (long i1, long i2) const
- T **norm2Cols** (long j1, long j2) const
- [wbvector< T >](#) & **norm2Cols** ([wbvector< T >](#) &x2) const
- [wbvector< T >](#) **normCols** () const
- T **scalarProd** (const [wbarray](#) &B) const
- T **sum** () const
- [wbarray](#) & **sum** (const WBINDEX &I, [wbarray](#) &) const
- [wbarray](#) & **sum** (const char *sidx, [wbarray](#) &A) const
- char **fitsSize** (const [wbarray](#) &, char strict=0) const
- size_t **getSize** (unsigned i, char lflag=1) const
- [wbvector< size_t >](#) & **getSize** (const [wbvector< unsigned >](#) &I, [wbvector< size_t >](#) &S) const
- size_t **getSizeM** (unsigned r)
- bool **sameSize** (const size_t *s, unsigned n) const
- bool **sameSize** (size_t d1, size_t d2) const
- template<class Ts >
bool **sameSizeM** (unsigned r, unsigned m, const Ts *s) const

- `template<class TB >`
`bool sameSize (const wbararray< TB > &, char lflag=2) const`
- `bool sameSize1 (const size_t *s, unsigned n) const`
- `template<class TB >`
`bool hasSameSize (unsigned ia, const wbararray< TB > &B, unsigned ib, char lflag=1) const`
- `char sameUptoFac (const wbararray &B, T *fac=NULL, double eps=1e-12) const`
- `bool sameAs (const wbararray &B, double eps=1e-12) const`
- `bool operator== (const wbararray &B) const`
- `bool operator!= (const wbararray &B) const`
- `T normDiff2 (const wbararray &B, char sflag=0) const`
- `T normDiff (const wbararray &B, char sflag=0) const`
- `T normDiff2 (const wvector< T > &B) const`
- `T normDiff (const wvector< T > &B) const`
- `void TimesEI (const wbararray &, char conj=0)`
- `wbararray & timesEI (const wbararray &B, wbararray &C, T bfac=1, T cfac=0, char conj=0) const`
- `wbararray & timesEI_OM (unsigned r, const wbararray &B, wbararray &C, char conj=0) const`
- `T dotProd (const wbararray &, char conj=0) const`
- `T weightedAvg (const wbararray &) const`
- `void TensorProd (const wbararray &B, const char aflag='N', const char bflag='N', const char kflag=0)`
- `wbararray & tensorProd (const wbararray &, wbararray &, const char aflag='N', const char bflag='N', const char kflag=0) const`
- `wbararray & kron (const wbararray &B, wbararray &X, char aflag='N', char bflag='N') const`
- `wbararray & Kron (const wbararray &B, char aflag='N', char bflag='N')`
- `wbararray & Cat (unsigned dim, const wvector< wbararray * > &ap)`
- `wbararray & Cat (unsigned dim, const wvector< wbararray > &ap)`
- `wbararray & BlockCat (const wbararray< wbararray > &aa, WBINDEX *D1=NULL, WBINDEX *D2=NULL)`
- `wbararray & BlockCat (const wbararray< const wbararray * > &ap, WBINDEX *D1=NULL, WBINDEX *D2=NULL)`
- `void operator+= (T x)`
- `void operator-= (T x)`
- `template<class T_ >`
`wbararray & Times (const T_ a, char rcpy=0)`
- `template<class T_ >`
`wbararray & times (const T_ a, wbararray< T > &X) const`
- `template<class T_ >`
`wbararray & operator*= (const T_ a)`
- `template<class T_ >`
`wbararray & operator/= (const T_ &a)`
- `void operator+= (const wbararray &B)`
- `void operator-= (const wbararray &B)`
- `wbararray & plus (const wbararray &B, wbararray &C, T bfac=1, char iflag=0) const`
- `template<class TB >`
`wbararray & Plus (const wbararray< TB > &B, TB bfac=TB(1), char iflag=0, T afac=T(1))`
- `bool equal (const wbararray &, T eps) const`
- `bool unequal (const wbararray &, T eps) const`
- `bool equal (const wbararray &, double eps, double &maxdiff) const`
- `bool unequal (const wbararray &, double eps, double &maxdiff) const`
- `bool allEqual (const T &x)`
- `bool allLE (const T &x)`
- `char isVector () const`
- `unsigned isScalar () const`
- `bool requiresDataPerm (const wperm &P, char lflag=3) const`
- `void markUnequalZero (const T m=1, const T eps=0)`
- `void toMatrixRef (wbararray &A, const UVEC &I, int pos, wperm &P) const`
- `bool toMatrixRef (const char *F, int L, wbararray &A, const UVEC &I, int pos, char &tflag) const`
- `void toMatrixRef (wbararray &A, unsigned K, const wperm &P0) const`

- void **toMatrixRefM** (wbararray &A, unsigned K, const wbpPerm &P0, unsigned R, wbpPerm pfin=wbpPerm()) const
- void **toMatrixRef** (wbararray &A, unsigned i, int pos, wbpPerm &P) const
- void **toMatrixRef** (wbararray &A, const char *sidx, int pos, wbpPerm &P) const
- wbararray & **Reshape** (const wvector< size_t > &, bool lflag=0)
- wbararray & **Reshape** (size_t s1, size_t s2, bool lflag=0)
- wbararray & **Reshape** (size_t s1, size_t s2, size_t s3, bool lflag=0)
- wbararray< T > & **Reshape1** (const size_t *s, unsigned n)
- wbararray & **Reshape1** (size_t s1, size_t s2)
- wbararray & **Reshape1** (size_t s1, size_t s2, size_t s3)
- void **GroupIndices** (size_t K)
- wbararray & **groupIndices_DREF** (wbararray &A, unsigned k1, unsigned k2) const
- void **GroupIndices** (unsigned k1, unsigned k2)
- void **groupIndices_P** (const WBINDEX &, int, wbpPerm &, size_t *=NULL, size_t *=NULL) const
- wbararray & **transpose** (const char *F, int L, wbararray &A) const
- wbararray & **Transpose** (const char *F=0, int L=0)
- wbararray & **flipLR** (const char *F, int L, wbararray &A) const
- wbararray & **FlipLR** (const char *F=0, int L=0)
- wbararray & **flipUD** (const char *F, int L, wbararray &A) const
- wbararray & **FlipUD** (const char *F=0, int L=0)
- wbararray & **MatPermute** (const wbpPerm &P, char iflag=0)
- wbararray & **Permute** (const char *s, char iflag=0, char rcpy=0)
- wbararray & **Permute** (const wbpPerm &, char iflag=0, char rcpy=0)
- wbararray & **permute** (wbararray &, const char *s, char iflag=0) const
- wbararray & **permute** (wbararray &, const wbpPerm &, char iflag=0) const
- wbararray & **select0** (const WBPERM &, unsigned dim, wbararray &) const
- wbararray & **Select0** (const UVEC &I, unsigned dim)
- wbararray & **select0** (size_t i1, size_t i2, unsigned dim, wbararray &Q) const
- wbararray & **selectSqueeze** (wbararray &A, unsigned p, unsigned dim) const
- T **min** () const
- T **max** () const
- T **aMin** (char zflag=0, size_t *k=NULL) const
- void **info** (const char *="") const
- void **info** (const char *F, int L, const char *="", unsigned k=-1, unsigned nlt=0, unsigned nlb=0) const
- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **print** (const char *istr="", const char *fmt="") const
- int **printdata** (const char *istr="ans", const char *fmt="") const
- void **print_ref** (const char *bstr=" ***", const char *nl="\n") const
- wstring **toStr** () const
- void **contractDiag** (unsigned ic, const wbararray &B, T a, T b, wbararray &C, char lflag=0, char bflag='N') const
- template<class TB, class TC>
 - wbararray< TC > & **contractMat** (const char *F, int L, unsigned i1, const wbararray< TB > &M, unsigned i2, wbararray< TC > &) const
- template<class TB, class TC>
 - wbararray< TC > & **contractMat** (const char *F, int L, unsigned i1, const wbararray< TB > &M, wbararray< TC > &C) const
- wbararray & **ContractMat** (const char *F, int L, unsigned i1, const wbararray &M, unsigned i2=1)
- template<class TB, class TC>
 - wbararray< TC > & **contract** (const char *F, int L, const ctrlIdx &, const wbararray< TB > &B, const ctrlIdx &, wbararray< TC > &C, const wbpPerm &pfinal=wbpPerm(), T afac=1, TC cfac=1) const
- template<class TB, class TC>
 - wbararray< TC > & **contract** (const char *F, int L, const iTags &ta, const wbararray< TB > &B, const iTags &tb, wbararray< TC > &C, iTags &tc, const wbpPerm &pfinal=wbpPerm(), T afac=1, TC cfac=1) const
- template<class TB>
 - wbararray< T > & **Contract** (const wvector< unsigned > &, const wbararray< TB > &, const wvector< unsigned > &, const wbpPerm &pfinal=wbpPerm())

- `template<class TB >`
`wbarray< T > & Contract` (char *, const `wbarray< TB >` &, char *, const `wbperm` &pfinal=`wbperm`())
- `template<class TB, class TC >`
`wbarray< TC > & contract` (const char *F, const int L, const char *, const `wbarray< TB >` &, const char *, `wbarray< TC >` &, const `wbperm` &pfinal=`wbperm`()) const
- `template<class TB, class TC >`
`wbarray< TC > & contract` (char *i1, const `wbarray< TB >` &B, char *i2, `wbarray< TC >` &Q, const `wbperm` &pfinal=`wbperm`()) const
- `template<class TB, class TC >`
`wbarray< TC > & contract` (const `wbvector< unsigned >` &i1, const `wbarray< TB >` &B0, const `wbvector< unsigned >` &i2, `wbarray< TC >` &Q0, const `wbperm` &P) const
- `template<class TB, class TC >`
`wbarray< TC > & contract` (const char *F, int L, unsigned i1, const `wbarray< TB >` &B, unsigned i2, `wbarray< TC >` &C, const `wbperm` &pfinal=`wbperm`()) const
- `template<class TB, class TC >`
`wbarray< TC > & contract` (unsigned i1, const `wbarray< TB >` &B, unsigned i2, `wbarray< TC >` &C, const `wbperm` &pfinal=`wbperm`()) const
- `template<class TB, class TC >`
`wbarray< TC > & comm` (const `wbarray< TB >` &, `wbarray< TC >` &, char aflag='N', char bflag='N') const
- `template<class TB, class TC >`
`wbarray< TC > & acomm` (const `wbarray< TB >` &, `wbarray< TC >` &, char aflag='N', char bflag='N') const
- `wbarray operator+` (const `wbarray` &B) const
- `wbarray operator-` (const `wbarray` &B) const
- `wbarray operator*` (const `wbarray` &B) const
- `T trace` () const
- `wbarray & trace` (unsigned i1, unsigned i2, `wbarray` &C) const
- `wbarray & trace` (const `ctrlidx` &l1, const `ctrlidx` &l2, `wbarray< T >` &C) const
- `mxArray * toMx` () const
- `mxArray * toMx_Struct` () const
- `mxArray * toMx_base` () const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- `void add2MxStruct` (mxArray *S, unsigned i, char tflag=0) const
- `void put` (const char *vname="ans", const char *ws="base") const
- `void put` (const char *F, int L, const char *vname="ans", const char *ws="base") const
- `void getReal` (`wbarray< double >` &R) const
- `void getImag` (`wbarray< double >` &I) const
- `void Conj` ()
- `void set` (const `wbarray< double >` &R, const `wbarray< double >` &I)
- const `wbarray` & `opA` (char tflag, `wbarray` &aux)
- bool `isSym_aux` (const char *file, int line, const char *fct, const `wbarray` &B, double eps, double *xref, const char symflag='s', const char lflag=1) const
- `template<class TB >`
`wbarray< T > & Plus` (const `wbarray< TB >` &B, TB bfac, char iflag, T afac)
- `template<class TB, class TC >`
`wbarray< TC > & contract` (const char *F, const int L, const `ctrlidx` &ica, const `wbarray< TB >` &B, const `ctrlidx` &icb, `wbarray< TC >` &C0, const `wbperm` &P, TA afac, TC cfac) const
- `mxArray * toMx` () const
- `mxArray * toMx` () const

Public Attributes

- `wbvector< size_t > SIZE`
- `Wb::MTYPE mtype`
- `Wb::sptr< T > * sptr`
- `T * data`

Protected Member Functions

- `bool isDiag_aux` (const T eps, const char *task) const
- `bool isDiag_aux` (double *eps, const char *task) const
- `size_t serial_index` (const widx_t *l, size_t n) const

The documentation for this class was generated from the following files:

- `wbarray.hh`
- `wbarray.cc`

4.97 `wbarrRef< T >` Class Template Reference**Public Member Functions**

- `wbarrRef` (const `wbarrRef< T >` &r)
- `wbarrRef< T >` & `operator=` (const `wbarrRef< T >` &r)
- void `print` (const char *vname="ans")

Public Attributes

- double `dc`
- double `fac`
- bool `tflag`
- `wbarray< T >` * `D`

The documentation for this class was generated from the following file:

- `wbarray.hh`

4.98 `wbcomplex` Class Reference**Public Member Functions**

- `wbcomplex` (const `wbcomplex` &z)=default
- `wbcomplex` (double r0, double i0=0.)
- `wbcomplex` & `operator=` (const `wbcomplex` &z)=default
- `wbcomplex` & `operator=` (const double &dbl)
- `operator bool` () const
- bool `operator!` () const
- `operator double` () const
- `wbcomplex` & `init` (double r0, double i0)
- `wbcomplex` & `set` (double r0, double i0)
- `wbcomplex` & `setRand` (double fac=2., double shift=-1.)
- `wbcomplex` & `init_expi` (double ph)
- `mxComplexDouble` & `explicit_copy2` (`mxComplexDouble` &zm) const
- bool `isinf` () const
- bool `isnan` () const

- bool **isfinite** () const
- bool **isnormal** () const
- bool **operator==** (const [wbcomplex](#) &z) const
- bool **operator!=** (const [wbcomplex](#) &z) const
- bool **operator==** (const double &dbl) const
- bool **operator!=** (const double &dbl) const
- bool **operator>** (const [wbcomplex](#) &z) const
- bool **operator>=** (const [wbcomplex](#) &z) const
- bool **operator<** (const [wbcomplex](#) &z) const
- bool **operator<=** (const [wbcomplex](#) &z) const
- [wbcomplex](#) & **operator+=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator-=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator*=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator/=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator*=** (double dbl)
- [wbcomplex](#) & **operator+=** (double dbl)
- [wbcomplex](#) & **operator-=** (double dbl)
- [wbcomplex](#) **operator+** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator-** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator*** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator/** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator+** (const double &dbl) const
- [wbcomplex](#) **operator-** (const double &dbl) const
- [wbcomplex](#) **operator*** (const double &dbl) const
- [wbcomplex](#) **operator/** (const double &dbl) const
- [wbcomplex](#) **operator/=** (double dbl)
- [wbcomplex](#) **times** (const [wbcomplex](#) &) const
- [wbcomplex](#) **ConjProd** (const [wbcomplex](#) &z) const
- void **Conj** ()
- [wbcomplex](#) **conj** () const
- double **abs2** () const
- double **abs** () const
- [wbcomplex](#) **sqrt** () const
- [wbcomplex](#) **sign** () const
- double **arg** () const
- double **arg_degree** ()
- [wbcomplex](#) **round** () const
- [wbstring](#) **toStr** (const char *fmt="%.4g") const
- void **put** (const char *vname="ans", const char *ws="base") const

Public Attributes

- double **r**
- double **i**

The documentation for this class was generated from the following files:

- [wbcomplex.hh](#)
- [wbcomplex.cc](#)

4.99 wbdebug_dummy Class Reference

The documentation for this class was generated from the following file:

- memtrack.cc

4.100 wbhist Class Reference

Public Member Functions

- **wbhist** (const mxArray *a)
- **wbhist** & **init** (const mxArray *a)
- **wbhist** & **add** (const char *F, int L, const mxArray *ax, const mxArray *ay)

Public Attributes

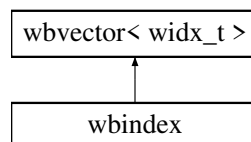
- long unsigned **ndat**
- long unsigned **ncalls**
- **wbvector**< double > **xbdr**
- **wbvector**< double > **xbin**
- **wbvector**< double > **ybin**

The documentation for this class was generated from the following file:

- wbhist.cc

4.101 wbindex Class Reference

Inheritance diagram for wbindex:



Public Member Functions

- **wbindex** (widx_t n=0, char iflag=0)
- **wbindex** (const [ctrlidx](#) &ic)
- **wbindex** (widx_t n, const widx_t *d)
- **wbindex** (const char *s, unsigned offset=0)
- **wbindex** & **init** (widx_t n=0, const widx_t *d=NULL)
- **wbindex** & **init** (const [ctrlidx](#) &ic)
- **wbindex** & **init** (const char *F, int L, const mxArray *a, widx_t offset=0)
- widx_t **init** (const char *F, int L, const mxArray *a, const char *istr, char check_type=1)
- **wbindex** & **init** (const mxArray *a, widx_t offset=0)
- **wbindex** & **initGroup** (const [wbvector](#)< widx_t > &P, [wbvector](#)< widx_t > &D)
- **wbindex** & **initGroup0** (const [wbvector](#)< widx_t > &P, [wbvector](#)< widx_t > &D)
- void **initStr** (const char *s, widx_t offset=0)
- bool **isIndex** (widx_t n=0)
- **wbindex** & **Index** (widx_t n)
- **wbindex** & **Index_ex** (widx_t n, widx_t ix)
- **wbindex** & **Index** (widx_t i1, widx_t i2)
- **wbindex** & **Index** (widx_t i1, widx_t i2, int p)
- **wbindex** & **BlockIndex** (const [wbvector](#)< widx_t > &D, char sub=0)
- **wbindex** & **BlockIndex** (widx_t n, widx_t D)
- **wbindex** & **BlockIndexI** (widx_t n, widx_t D)
- **wbindex** & **IndexTranspose** (widx_t m, widx_t n)
- **wbindex** & **operator=** (const char *s)
- int **extend2Perm** (widx_t N, [wbperm](#) &P, char toend=0) const
- void **toPerm** ([wbperm](#) &P) const
- **wbindex** & **invert** (widx_t N, [wbindex](#) &I2, char uflag=1) const
- **wbindex** & **Invert** (widx_t N, char uflag=1)
- **wbindex** **flipIdx** (widx_t ndim) const
- char **isUnique** (widx_t imax, widx_t offset=0) const
- mxArray * **toMx** (const char tflag=0) const

Public Member Functions inherited from [wbvector](#)< widx_t >

- **wbvector** (size_t l, widx_t x)
- **wbvector** (size_t l, const widx_t *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- **wbvector** & **init** (size_t l=0)
- **wbvector** & **init** (size_t l, const widx_t *d0)
- **wbvector** & **init** (size_t l, const widx_t *d0, char ref)
- **wbvector** & **init** (const [wbvector](#) &v)
- **wbvector** & **init** (size_t n, const [wbvector](#) &v)
- **wbvector**< widx_t > & **init** (const [wbsparray](#)< widx_t > &S)
- **wbvector** & **init** (size_t l1, const widx_t *d1, size_t l2, const widx_t *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- **wbvector** & **initp** (size_t l, const widx_t *d0, size_t n, char init=1)
- **wbvector** & **init2val** (size_t l, const widx_t &x)

- [wbvector](#) & **initStride** (size_t l, const widx_t *d0, size_t stride)
- [wbvector](#) & **init_stride** (const widx_t *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const widx_t *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< widx_t > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< widx_t > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< widx_t > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< widx_t > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< widx_t > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< widx_t > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const widx_t *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpfr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< widx_t > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const widx_t &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const widx_t &x)
- void **set** (size_t offset, const widx_t *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const widx_t &x)
- [wbvector](#) & **setMax** (const widx_t &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const widx_t &x)
- [wbvector](#) & **add** (const widx_t *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const widx_t &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const widx_t &x) const
- size_t **find1** () const
- size_t **count** (const widx_t &x) const
- size_t **icount** (const widx_t &x) const
- size_t **nnz** () const
- size_t **nnz** (widx_t eps) const
- size_t **numZeros** (widx_t eps=widx_t(1E-14)) const
- size_t **numEQ** (const widx_t &) const
- size_t **numGT** (const widx_t &) const
- size_t **numLT** (const widx_t &) const
- [wbindex](#) & **find** (const widx_t &x, [wbindex](#) &l, char iflag=0) const

- [wbindex](#) & **find** (const widx_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const widx_t &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const widx_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const widx_t &x1, const widx_t &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (widx_t r) const
- int **findClosest** (widx_t r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const widx_t *x)
- void **reset** ()
- void **cpy** (const widx_t *v)
- void **cpy** (const widx_t *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (widx_t &A1, widx_t &A2)
- [wbvector](#) & **Collect** (widx_t &A1, widx_t &A2, widx_t &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const widx_t *b)
- [wbvector](#) & **Plus** (const widx_t *b, widx_t bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, widx_t bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, widx_t bfac=1, widx_t cfac=0) const
- [wbvector](#) & **times** (widx_t fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- widx_t **sum** () const
- widx_t **sum** (size_t i1, const char *estr) const
- widx_t **sum** (size_t i1, size_t i2) const
- widx_t **cumsum_** ([wbvector](#) &S, char xflag=0) const
- widx_t **Cumsum** (char zflag=0) const

- `widx_t cumsum0prod (wbvector< T2 > &b, wbvector< widx_t > &cs, char xflag) const`
- `wbvector & Eldiv (const wbvector &v) const`
- `wbvector & tensorProd (const wbvector &v2, wbvector &vv) const`
- `widx_t scalarProd (const wbvector &v) const`
- `widx_t scalarProd (const widx_t *v) const`
- `widx_t norm () const`
- `widx_t norm2 () const`
- `double normDiff (const wbvector &v, widx_t fac=1) const`
- `widx_t normDiff2 (const wbvector &v, widx_t fac=1) const`
- `double normDiff2_ (const wbvector &v, double fac=1) const`
- `widx_t GSProject (const widx_t *v, char isnorm=0, char tflag=0) const`
- `wbvector & Normalize (const char *F=NULL, int L=0)`
- `wbvector & subtract (const widx_t *d)`
- `void getReal (wbvector< double > &R) const`
- `void getReal (wbvector< double > &R) const`
- `void getImag (wbvector< double > &I) const`
- `void getImag (wbvector< double > &I) const`
- `wbvector & Conj ()`
- `wbvector< wbcomplex > & Conj ()`
- `wbvector & applyFlag (char &flag, const widx_t &fac, wbvector &a)`
- `bool allEqual () const`
- `bool allEqual (const widx_t &a) const`
- `bool allZero () const`
- `bool allOnes () const`
- `bool allEqual2 (const widx_t &a, widx_t eps=1E-15) const`
- `bool allUnequal (const widx_t &a) const`
- `bool any () const`
- `bool anyUnequal (const widx_t &a) const`
- `bool anyUnequal (const wbvector< size_t > &l, const widx_t &x, size_t *k=NULL) const`
- `bool anyEqual (const widx_t &a, size_t *i1=NULL) const`
- `bool anyEqual (const wbvector< size_t > &l, const widx_t &x, size_t *k=NULL) const`
- `bool anyGT (const widx_t &a) const`
- `bool anyGE (const widx_t &a) const`
- `bool anyLT (const widx_t &a) const`
- `bool anyLE (const widx_t &a) const`
- `bool allGT (const widx_t &a) const`
- `bool allGE (const widx_t &a) const`
- `bool allLE (const widx_t &a) const`
- `bool allLT (const widx_t &a) const`
- `bool allIn (const widx_t &a1, const widx_t &a2) const`
- `bool anySmallVals (const widx_t eps=1E-15) const`
- `bool isEqual (const widx_t *v) const`
- `bool isEqual (const wbvector &v) const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool operator== (const wbvector &v) const`
- `bool operator!= (const wbvector &v) const`
- `bool isEqualR (const wbvector &v) const`
- `bool operator^ (const wbvector &v) const`
- `bool deepEqualP (const wbvector &B) const`
- `char cmp (const wbvector &v) const`
- `bool operator< (const wbvector &v) const`
- `int operator< (const widx_t &r) const`
- `bool operator<= (const wbvector &v) const`

- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [widx_t](#) & **el** (size_t i) const
- [widx_t](#) **elx** (size_t i, const [widx_t](#) &x) const
- const [widx_t](#) & **operator[]** (size_t i) const
- [widx_t](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- [widx_t](#) & **last** () const
- [widx_t](#) & **last** (size_t &i) const
- [widx_t](#) & **last** (T1 &i_) const
- [widx_t](#) & **p2last** () const
- [widx_t](#) * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setldx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [widx_t](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [widx_t](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< [widx_t](#) > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< [widx_t](#) > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const [widx_t](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [widx_t](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const [widx_t](#) *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const [widx_t](#) *v) const
- int **recLessE** (const [widx_t](#) *v) const
- int **recCompare** (const [widx_t](#) *v) const
- [widx_t](#) **max** () const
- [widx_t](#) **max** (size_t *k) const
- [widx_t](#) **max** (unsigned &k) const
- [widx_t](#) **max_** ([widx_t](#) x) const
- [widx_t](#) **min** () const
- [widx_t](#) **min** (size_t *k) const
- [widx_t](#) **min** (unsigned &k) const
- [widx_t](#) **min_** ([widx_t](#) x) const
- [widx_t](#) **aMax** (size_t *k=NULL) const
- [widx_t](#) **aMax** (unsigned &k) const
- [widx_t](#) **aMin** (char zflag=0, size_t *k=NULL) const
- [widx_t](#) **maxneg** (size_t *k=NULL) const
- [widx_t](#) **minpos** (size_t *k=NULL) const
- [widx_t](#) **avg** () const

- `widx_t std () const`
- `void range (widx_t &xmin, widx_t &xmax) const`
- `widx_t range () const`
- `wbvector & Permute (const wbperm &P, char iflag=0)`
- `wbvector & permute (wbvector &, const wbperm &P, char iflag=0) const`
- `wbvector permute (const wbperm &P, char iflag=0) const`
- `wbvector & blockPermute (const wbperm &P, wbvector &, char iflag=0) const`
- `wbvector & BlockPermute (const wbperm &P, char iflag=0)`
- `int blockCompare (const wbperm &pa, const wbvector &b, const wbperm &pb) const`
- `void get (const WBINDEX &I0, wbvector &v) const`
- `void get (const wbvector< size_t > &I, widx_t *v) const`
- `void get (size_t i1, size_t i2, wbvector &v) const`
- `void selectSU (const WBINDEX &I)`
- `void select (const WBINDEX &I, widx_t *r) const`
- `wbvector & select (const WBINDEX &I, wbvector &a) const`
- `wbvector select (const WBINDEX &I) const`
- `wbvector & Select (const WBINDEX &I)`
- `wbvector & BlockSelect (const WBINDEX &I, const WBINDEX &D)`
- `wbvector & getI (size_t k, wbvector &v) const`
- `wbvector & getI (const wbindex &k, wbvector &v, char uflag=1) const`
- `wbvector & getI (const WBINDEX &k, wbvector &v, char uflag=1) const`
- `wbvector & initI (const wbvector &a, const wbindex &ia)`
- `wbvector & initI (const wbvector &a, const wbindex &ia, const wbvector &b, const wbindex &ib, char uflag=1)`
- `wbvector & Flip ()`
- `void flip (wbvector &a) const`
- `wbvector flip () const`
- `widx_t prod () const`
- `widx_t prod (widx_t x) const`
- `widx_t prod (size_t i, size_t i2) const`
- `widx_t prod (size_t i, size_t i2, widx_t x) const`
- `widx_t prod (const T1 *I, size_t n) const`
- `T2 prod (const T1 *I, size_t n, T2 &s2) const`
- `void prod2 (widx_t &p1, widx_t &p2, const widx_t &p0=0) const`
- `void info (const char *istr) const`
- `void print (const char *istr, char mflag=0) const`
- `void printdata (const char *istr=0) const`
- `void put (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const`
- `void put (const char *vname="ans", char tflag=0, const char *ws="base") const`
- `void dispWithMsg (const char *file, int line, const char *msg, const char *vname="") const`
- `void mat2mx (mxArray *a, char dim=2) const`
- `void mat2mxs (mxArray *a, char field_nr, char dim=2) const`
- `mxArray * toMx_Struct (const char tflag=0) const`
- `mxArray * toMxP_S (const char tflag=0) const`
- `mxArray * toMx (const char tflag=0) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMxP (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base_d (const char tflag=0) const`
- `mxArray * toMx_offset (const widx_t &offset, const char tflag=0) const`

- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tflag=0) const`
- `void add2MxStruct (mxArray *S, const char *vname, char tflag=0) const`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tflag=0) const`
- `void putg (const char *vname="ans", char tflag=0) const`
- `void putx (const char *vname="ans", char tflag=0) const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring toStr () const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring sizeStr () const`
- `wbstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const`
- `wbstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const`

Additional Inherited Members

Public Attributes inherited from [wbvector< widx_t >](#)

- `widx_t * data`
- `size_t len`
- `char isref`

Protected Member Functions inherited from [wbvector< widx_t >](#)

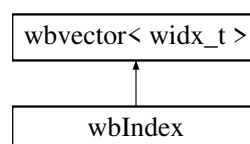
- `wbvector & RENEW (size_t n, const widx_t *d=NULL, size_t l=-1, char init=1)`
- `wbvector & RENEW_VAL (size_t n, const widx_t &x)`

The documentation for this class was generated from the following files:

- `wbindex.hh`
- `wbindex.cc`

4.102 wbIndex Class Reference

Inheritance diagram for `wbIndex`:



Public Member Functions

- **wblIndex** (const [wbvector](#)< widx_t > &S)
- template<class T1 >
 wblIndex (widx_t n, const T1 *S)
- **wblIndex** (const [wblIndex](#) &l)
- [wblIndex](#) & **init** ()
- [wblIndex](#) & **init** (const [wbvector](#)< widx_t > &S, char pp=1)
- template<class T1 >
 [wblIndex](#) & **init** (unsigned n, const T1 *S)
- [wblIndex](#) & **reset** (char pp=1)
- [wblIndex](#) & **set** ([wbvector](#)< widx_t > &l)
- bool **isValid** () const
- void **checkValid** (const char *F, int L) const
- bool **operator++** ()
- bool **operator--** ()
- bool **isEmpty** () const
- bool **gotmore** () const
- bool **isdiag** () const
- widx_t **numel** () const
- widx_t **serial** (const widx_t *S=NULL)
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStr** () const
- mxArray * **toMx** () const

Public Member Functions inherited from [wbvector](#)< widx_t >

- **wbvector** (size_t l, widx_t x)
- **wbvector** (size_t l, const widx_t *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const widx_t *d0)
- [wbvector](#) & **init** (size_t l, const widx_t *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< widx_t > & **init** (const [wbsparray](#)< widx_t > &S)
- [wbvector](#) & **init** (size_t l1, const widx_t *d1, size_t l2, const widx_t *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const widx_t *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const widx_t &x)
- [wbvector](#) & **initStride** (size_t l, const widx_t *d0, size_t stride)
- [wbvector](#) & **init_stride** (const widx_t *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const widx_t *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< widx_t > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< widx_t > & **initT** (size_t l, const T2 *v)

- [wbvector](#)< widx_t > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< widx_t > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< widx_t > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< widx_t > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const widx_t *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< widx_t > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const widx_t &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const widx_t &x)
- void **set** (size_t offset, const widx_t *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const widx_t &x)
- [wbvector](#) & **setMax** (const widx_t &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const widx_t &x)
- [wbvector](#) & **add** (const widx_t *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const widx_t &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const widx_t &x) const
- size_t **find1** () const
- size_t **count** (const widx_t &x) const
- size_t **icount** (const widx_t &x) const
- size_t **nnz** () const
- size_t **nnz** (widx_t eps) const
- size_t **numZeros** (widx_t eps=widx_t(1E-14)) const
- size_t **numEQ** (const widx_t &) const
- size_t **numGT** (const widx_t &) const
- size_t **numLT** (const widx_t &) const
- [wbindex](#) & **find** (const widx_t &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const widx_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const widx_t &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const widx_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const

- [wblindex](#) & **findRange** (const widx_t &x1, const widx_t &x2, [wblindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (widx_t r) const
- int **findClosest** (widx_t r) const
- size_t **findValues** (const [wbvector](#) &v, [wblindex](#) &l) const
- [wbvector](#) & **setp** (const widx_t *x)
- void **reset** ()
- void **cpy** (const widx_t *v)
- void **cpy** (const widx_t *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (widx_t &A1, widx_t &A2)
- [wbvector](#) & **Collect** (widx_t &A1, widx_t &A2, widx_t &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const widx_t *b)
- [wbvector](#) & **Plus** (const widx_t *b, widx_t bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, widx_t bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, widx_t bfac=1, widx_t cfac=0) const
- [wbvector](#) & **times** (widx_t fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- widx_t **sum** () const
- widx_t **sum** (size_t i1, const char *estr) const
- widx_t **sum** (size_t i1, size_t i2) const
- widx_t **cumsum_** ([wbvector](#) &S, char xflag=0) const
- widx_t **Cumsum** (char zflag=0) const
- widx_t **cumsum0prod** ([wbvector](#) < T2 > &b, [wbvector](#) < widx_t > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- widx_t **scalarProd** (const [wbvector](#) &v) const
- widx_t **scalarProd** (const widx_t *v) const
- widx_t **norm** () const
- widx_t **norm2** () const

- double **normDiff** (const [wbvector](#) &v, widx_t fac=1) const
- widx_t **normDiff2** (const [wbvector](#) &v, widx_t fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- widx_t **GSProject** (const widx_t *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const widx_t *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const widx_t &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const widx_t &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const widx_t &a, widx_t eps=1E-15) const
- bool **allUnequal** (const widx_t &a) const
- bool **any** () const
- bool **anyUnequal** (const widx_t &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const widx_t &x, size_t *k=NULL) const
- bool **anyEqual** (const widx_t &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const widx_t &x, size_t *k=NULL) const
- bool **anyGT** (const widx_t &a) const
- bool **anyGE** (const widx_t &a) const
- bool **anyLT** (const widx_t &a) const
- bool **anyLE** (const widx_t &a) const
- bool **allGT** (const widx_t &a) const
- bool **allGE** (const widx_t &a) const
- bool **allLE** (const widx_t &a) const
- bool **allLT** (const widx_t &a) const
- bool **allIn** (const widx_t &a1, const widx_t &a2) const
- bool **anySmallVals** (const widx_t eps=1E-15) const
- bool **isEqual** (const widx_t *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const widx_t &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- widx_t & **el** (size_t i) const
- widx_t **elx** (size_t i, const widx_t &x) const
- const widx_t & **operator[]** (size_t i) const
- widx_t & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const

- `widx_t & last () const`
- `widx_t & last (size_t &i) const`
- `widx_t & last (T1 &i_) const`
- `widx_t & p2last () const`
- `widx_t * ref (size_t i) const`
- `wbvector & setRand (double fac=1., double shift=0.)`
- `wbvector< wbcomplex > & setRand (double fac, double shift)`
- `void setIdx (double shift=0., double fac=1.)`
- `int set2Group (const wbperm &P, const WBINDEX &D, const widx_t *S0=NULL, char iflag=0)`
- `wbvector & Resize_ (size_t n)`
- `wbvector & Shorten2 (size_t n)`
- `wbvector & Resize (size_t n, const widx_t *d=NULL)`
- `wbvector & resize (size_t n, wbvector &X) const`
- `wbvector & cat (const wbvector &v2)`
- `wbvector & Cat (const wbvector &v1, const wbvector &v2)`
- `wbvector & Cat (const wbvector< wbvector > &vv)`
- `wbvector & Cat (const wbvector< wbvector * > &vv)`
- `wbvector & Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)`
- `wbvector & Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3=0, const wbvector< T3 > *v4=0)`
- `wbvector< widx_t > & Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3, const wbvector< T3 > *v4)`
- `wbvector< widx_t > & Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)`
- `wbvector & Append (const wbvector &v2)`
- `wbvector & Append (const widx_t &x)`
- `wbvector & Append (size_t d2, const widx_t *x)`
- `wbvector append (const wbvector &v) const`
- `wbvector & append (const wbvector &v2, wbvector &vout) const`
- `void append (const widx_t *x, size_t d2, wbvector &vout) const`
- `int recLess (const widx_t *v) const`
- `int recLessE (const widx_t *v) const`
- `int recCompare (const widx_t *v) const`
- `widx_t max () const`
- `widx_t max (size_t *k) const`
- `widx_t max (unsigned &k) const`
- `widx_t max_ (widx_t x) const`
- `widx_t min () const`
- `widx_t min (size_t *k) const`
- `widx_t min (unsigned &k) const`
- `widx_t min_ (widx_t x) const`
- `widx_t aMax (size_t *k=NULL) const`
- `widx_t aMax (unsigned &k) const`
- `widx_t aMin (char zflag=0, size_t *k=NULL) const`
- `widx_t maxneg (size_t *k=NULL) const`
- `widx_t minpos (size_t *k=NULL) const`
- `widx_t avg () const`
- `widx_t std () const`
- `void range (widx_t &xmin, widx_t &xmax) const`
- `widx_t range () const`
- `wbvector & Permute (const wbperm &P, char iflag=0)`
- `wbvector & permute (wbvector &, const wbperm &P, char iflag=0) const`
- `wbvector permute (const wbperm &P, char iflag=0) const`
- `wbvector & blockPermute (const wbperm &P, wbvector &, char iflag=0) const`

- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &l0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &l, widx_t *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &l)
- void **select** (const WBINDEX &l, widx_t *r) const
- [wbvector](#) & **select** (const WBINDEX &l, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &l) const
- [wbvector](#) & **Select** (const WBINDEX &l)
- [wbvector](#) & **BlockSelect** (const WBINDEX &l, const WBINDEX &D)
- [wbvector](#) & **getl** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- widx_t **prod** () const
- widx_t **prod** (widx_t x) const
- widx_t **prod** (size_t i, size_t i2) const
- widx_t **prod** (size_t i, size_t i2, widx_t x) const
- widx_t **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (widx_t &p1, widx_t &p2, const widx_t &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const widx_t &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const

- [wbstring toStr](#) (int n, const char *sep) const
- [wbstring toStr](#) () const
- [wbstring toStr](#) (int n, const char *sep) const
- [wbstring sizeStr](#) () const
- [wbstring toStrf](#) (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring toStrf](#) (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Public Attributes

- [wbvector](#)< widx_t > **SIZE**

Public Attributes inherited from [wbvector](#)< widx_t >

- widx_t * **data**
- size_t **len**
- char **isref**

Additional Inherited Members

Protected Member Functions inherited from [wbvector](#)< widx_t >

- [wbvector](#) & **RENEW** (size_t n, const widx_t *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const widx_t &x)

The documentation for this class was generated from the following file:

- wbindex.hh

4.103 Wb::WbListMTRACK Class Reference

Public Member Functions

- template<class T >
void **rm_ptr** (const char *F, const int &L, T *&p)
- template<class T >
void **add_ptr** (const char *F, int L, T *&p, size_t l)
- template<class T >
bool **exists** (T *&p)
- template<class T >
void **tst_exists** (const char *F, int L, T *&p)
- void **checkTotSize** ()
- char * **totStr** (char mflag=0)
- void **printSize** (const char *F, int L, char mflag=0)

Public Attributes

- `map< void *, MREC > M`
- unsigned long long `idx`
- unsigned long long `totsize`
- unsigned long long `maxsize`

The documentation for this class was generated from the following file:

- `memtrack.cc`

4.104 wblogbuf_struct Class Reference

Public Member Functions

- `wblogbuf_struct` (`const char *s`)
- `wblogbuf_struct` (`const char *F`, `int L`, `const char *s`)
- `wblogbuf_struct` (`const wblogbuf_struct &S`)
- `void unset ()`

Public Attributes

- `char * file`
- `int line`
- `char * text`

The documentation for this class was generated from the following file:

- `wbstring.hh`

4.105 wblogbuffer Class Reference

Public Member Functions

- `void push` (`const char *F`, `int L`, `const char *s`)
- `void flush ()`

The documentation for this class was generated from the following file:

- `wbstring.hh`

4.106 `wbMatrix< T >` Class Template Reference

Public Member Functions

- **`wbMatrix`** (`size_t` r=0, `size_t` c=0)
- **`wbMatrix`** (`size_t` r, `size_t` c, `T` *d, `const char` ref=0)
- **`wbMatrix`** (`const` [`wbMatrix`](#) &M)
- `template<class T2 >`
 `wbMatrix` (`const` [`wbMatrix`](#)< T2 > &M)
- **`wbMatrix`** (`const` `mxAarray` *a, `char` tflag=0, `char` ref=0)
- **`wbMatrix`** (`const` `char` *F, `int` L, `const` `mxAarray` *a)
- [`wbMatrix`](#) & **`init`** (`size_t` r=0, `size_t` c=0, `const` `T` *d=NULL)
- [`wbMatrix`](#) & **`init`** (`size_t` r, `size_t` c, `const` `T` *d0, `size_t` stride)
- [`wbMatrix`](#) & **`initIdentity`** (`size_t` n)
- [`wbMatrix`](#) & **`init2val`** (`size_t` r, `size_t` c, `const` `T` &d)
- [`wbMatrix`](#) & **`initDef`** (`size_t` r=0, `size_t` c=0, `T` *d=NULL)
- `template<class T2 >`
 [`wbMatrix`](#)< T > & **`initT`** (`size_t` r, `size_t` c, `T2` *d)
- `template<class T2 >`
 [`wbMatrix`](#)< T > & **`initT`** (`const` [`wbMatrix`](#)< T2 > &B)
- `template<class T2 >`
 [`wbMatrix`](#)< T > & **`initT`** (`const` `char` *F, `int` L, `const` [`wbMatrix`](#)< T2 > &B)
- `template<class TA , class TB >`
 [`wbMatrix`](#)< T > & **`init`** (`const` `char` *F, `int` L, `const` [`wbMatrix`](#)< TA > &B, `const` [`wbvector`](#)< TB > &V, `char` dim=2)
- `template<class T2 >`
 [`wbMatrix`](#)< T > & **`initDim`** (`const` [`wbMatrix`](#)< T2 > &B)
- [`wbMatrix`](#) & **`init`** (`const` [`wbMatrix`](#) &B)
- [`wbMatrix`](#) & **`init`** (`const` `char` *F, `int` L, `const` `mxAarray` *a, `char` tflag=0, `char` ref=0, `char` tcheck=1)
- `void` **`init`** (`const` `mxAarray` *a)
- `void` **`init0`** (`const` `mxAarray` *a)
- `void` **`initTST`** ()
- [`wbMatrix`](#) & **`init2ref`** (`size_t` r, `size_t` c, `const` `T` *d)
- [`wbMatrix`](#) & **`init2ref`** (`const` [`wbvector`](#)< T > &v, `const` `char` tflag=0)
- [`wbMatrix`](#) & **`init2ref`** (`const` [`wbMatrix`](#) &M)
- [`wbMatrix`](#) & **`unRef`** ()
- `void` **`no_ref`** (`const` `char` *F, `int` L, `const` `char` *fct)
- `void` **`swap`** ([`wbMatrix`](#) &B, `char` ref=0)
- [`wbMatrix`](#) & **`save2`** ([`wbMatrix`](#) &A, `char` ref=0)
- `void` **`copyStride`** (`T` *dd, `size_t` stride) `const`
- [`wbMatrix`](#) & **`operator=`** (`const` [`wbMatrix`](#) &B)
- `template<class TB >`
 [`wbMatrix`](#) & **`operator=`** (`const` [`wbMatrix`](#)< TB > &B)
- **`operator bool`** () `const`
- `bool` **`operator!`** () `const`
- `bool` **`operator==`** (`const` `T` &x) `const`
- `bool` **`operator==`** (`const` [`wbMatrix`](#) &B) `const`
- `bool` **`operator!=`** (`const` [`wbMatrix`](#) &B) `const`
- `bool` **`deepEqualP`** (`const` [`wbMatrix`](#) &B) `const`
- `bool` **`anyEqual`** (`const` `T` &x) `const`
- `bool` **`allEqual`** (`const` `T` &x) `const`
- `bool` **`anyUnequal`** (`const` `T` &x) `const`
- `bool` **`anyLT`** (`const` `T` &x) `const`
- `bool` **`anyGT`** (`const` `T` &x) `const`
- `bool` **`col_anyGT`** (`size_t` j, `const` `T` &x) `const`

- [wbMatrix](#) & **operator+=** (const [wbMatrix](#) &B)
- [wbMatrix](#) **operator+** (const [wbMatrix](#) &B) const
- [wbMatrix](#) **operator+** (const T &x) const
- void **operator-=** (const [wbMatrix](#) &B)
- [wbMatrix](#) **operator-** (const [wbMatrix](#) &B) const
- void **operator*=** (const T c)
- [wbMatrix](#) & **operator+=** (const T c)
- void **operator-=** (const T c)
- [wbMatrix](#) **operator*** (const T x) const
- [wbMatrix](#) & **add** (const [wbMatrix](#) &B, const T &c=1, const char zflag=0)
- [wbMatrix](#) & **minus** (const [wbMatrix](#) &B, const T &c=1, const char zflag=0)
- [wbMatrix](#) & **times** (T fac, [wbMatrix](#) &X) const
- [wbMatrix](#) & **rowTimes** (size_t i, T fac)
- [wbMatrix](#) & **colTimes** (size_t j, T fac)
- T **prod** () const
- T **min** () const
- T **max** () const
- T **aMax** () const
- T **aMax** (unsigned &i_, unsigned &j_) const
- T **sum** () const
- T **cumsum0prodRec** ([wbvector](#)< T > &cs, char xflag=0) const
- T **cumsum0prodRec** ([wbvector](#)< T > &cs, unsigned i1, unsigned i2, char xflag=0) const
- T **norm2** () const
- T **norm** () const
- double **normReal** () const
- double **normImag** () const
- T **norm2Cols** (long j1, long j2) const
- T **norm2Recs** (long i1, long i2) const
- [wbvector](#)< T > & **norm2Cols** ([wbvector](#)< T > &x2_) const
- [wbvector](#)< T > **norm2Cols** () const
- [wbvector](#)< T > **normCols** () const
- size_t **maxRec** (char lex=1, size_t *m=NULL) const
- size_t **maxRec_float** (char lex=1, size_t *m=NULL, T xref=-1) const
- T **recMax** (size_t r, size_t *k=NULL) const
- T **colMax** (size_t c, size_t *k=NULL) const
- T **colMax** (size_t c, unsigned &k) const
- [wbMatrix](#) & **NormalizeCol** (size_t k)
- bool **isOrthoRows** (T *x=NULL, char tnorm=0, T eps=1E-14) const
- bool **isOrthoRows** (const [wbMatrix](#) &B, T *x=NULL, char tnorm=0, T eps=1E-14) const
- double **recMaxA** (size_t r, size_t *k=NULL) const
- size_t **skipNanRecs** ([wbindex](#) &l)
- [wbMatrix](#) & **ResizeXRecs** (size_t r)
- [wbMatrix](#) & **Resize** (size_t r, size_t c, const T *dx=NULL)
- [wbMatrix](#) & **resize** (size_t m, size_t n, [wbMatrix](#) &M) const
- [wbMatrix](#) & **Resize2Mult** (const WBINDEX &M)
- [wbMatrix](#) & **transpose** ([wbMatrix](#) &M) const
- [wbMatrix](#) & **Transpose** ()
- [wbMatrix](#) & **Reshape** (size_t r, size_t c)
- template<class T2 >
 [wbMatrix](#)< T > & **AddCols** (size_t n, const T2 *d0=NULL)
- template<class T2 >
 [wbMatrix](#)< T > & **AddRows** (size_t n, const T2 *d0=NULL)
- bool **isEmpty** () const
- bool **isNormal** () const
- bool **isFinite** () const

- bool **isSquare** () const
- bool **isSquare** (size_t d) const
- bool **isScalar** () const
- bool **isVector** () const
- bool **isDiag** () const
- bool **isDiagMatrix** (T eps=1E-14) const
- bool **isIdentity** (double eps, double &maxdiff) const
- bool **isProptold** (T &x, T eps=1E-14) const
- bool **isHConj** (const **wbMatrix** &B, double eps=1E-12, double *xref=NULL) const
- bool **isHConj** (double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (const **wbMatrix** &B, double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (double eps=1E-12, double *xref=NULL) const
- bool **isComplex** () const
- bool **isUnique** () const
- bool **isUniqueSorted** (char dir=0, char lex=1) const
- int **isSorted** (char dir=0, char lex=1, size_t m=-1) const
- bool **quickCheckSorted** (char dir, char lex=1, const char *F=NULL, int L=0, size_t n=2) const
- bool **thisIsInt** () const
- int **getDiff** (const **wbMatrix** &B, **wbindex** &la, **wbindex** *lb=NULL) const
- int **getDiffSorted** (const **wbMatrix** &B, **wbindex** &la, **wbindex** *lb=NULL) const
- size_t **length** () const
- size_t **numel** () const
- void **flipSign** ()
- **wbMatrix** & **Symmetrize** ()
- void **set** (const T &x)
- **wbMatrix** & **setRand** (double fac=1., double shift=0.)
- void **setDiagRand** (double fac=1., double shift=0.)
- double **maxRelDiff** (const **wbMatrix** &M) const
- template<class T2 >
double **normDiff** (const **wbMatrix**< T2 > &M) const
- T **normDiff2** (const **wbMatrix** &M, size_t *k=NULL) const
- T **normDiff** (const **wbMatrix** &M, size_t *k=NULL) const
- bool **sameSize** (const **wbMatrix** B) const
- WBINDEX & **toIndex** (WBINDEX &I, const WBINDEX *S=NULL) const
- WBIDXMAT & **toIndex2D** (const WBINDEX &S, const **wbindex** &ic, WBIDXMAT &IJ, char pos=1) const
- **wbMatrix** & **SkipTiny_float** (T ref __attribute__((unused))=-1)
- void **reset** ()
- size_t **nnz** () const
- size_t **nnz** (T eps) const
- **wbMatrix** & **CAT** (C_UINT d12, const **wbMatrix** **M, C_UINT len)
- **wbMatrix** & **CAT** (C_UINT d12, const **wbvector**< **wbMatrix** > &M0)
- **wbMatrix** & **CAT** (C_UINT d12, **wbvector**< **wbMatrix** const * > M)
- **wbMatrix** & **CAT** (C_UINT d12, const **wbvector**< **wbvector**< T > > &M)
- **wbMatrix** & **CAT** (C_UINT d12, const **wbvector**< **wbvector**< T > const * > &M)
- **wbMatrix** & **cat** (C_UINT d12, const **wbMatrix** &)
- **wbMatrix** & **cat** (C_UINT d12, const **wbMatrix** &, const **wbMatrix** &)
- **wbMatrix** & **cat** (C_UINT d12, const **wbMatrix** &, const **wbMatrix** &, const **wbMatrix** &)
- **wbMatrix** & **Cat** (C_UINT d12, const **wbMatrix** &, const **wbMatrix** &)
- **wbMatrix** & **Cat** (C_UINT d12, const **wbMatrix** &, const **wbMatrix** &, const **wbMatrix** &)
- **wbMatrix** & **Cat** (C_UINT d12, const **wbMatrix** &, const **wbMatrix** &, const **wbMatrix** &, const **wbMatrix** &)
- **wbMatrix**< T > & **CAT** (const unsigned dim, **wbvector**< **wbMatrix**< T > const * > M, **wbvector**< WBINDEX const * > I)
- **wbMatrix** & **Cat** (const unsigned dim, const **wbMatrix** &M1, const **wbindex** &I1, const **wbMatrix** &M2, const **wbindex** &I2)
- void **split** (C_UINT d12, **wbvector**< **wbMatrix** * > &M) const

- void **split** (C_UINT d12, [wbMatrix](#) **M, C_UINT len) const
- void **split** (C_UINT d12, [wbMatrix](#) &M1, [wbMatrix](#) &M2, [wbMatrix](#) &M3) const
- const T & **operator()** (size_t i, size_t j, const T &x) const
- T & **operator()** (size_t i, size_t j, const T &x)
- const T & **operator()** (size_t i, size_t j) const
- T & **operator()** (size_t i, size_t j)
- const T & **el** (size_t i, size_t j) const
- T & **el** (size_t i, size_t j)
- const T & **els** (long i, long j) const
- T & **els** (long i, long j)
- const T * **operator()** (size_t i) const
- T * **operator()** (size_t i)
- const T & **operator[]** (size_t i) const
- T & **operator[]** (size_t i)
- const T * **ref** (size_t i, size_t j=0) const
- T * **ref** (size_t i, size_t j=0)
- const T * **rec** (size_t i) const
- T * **rec** (size_t i)
- void **getDiag** ([wbvector](#)< T > &D) const
- [wbvector](#)< T > **getDiag** () const
- void **appendRows** (size_t n, const T *=NULL)
- void **appendRow** (const [wbvector](#)< T > &v)
- mxArray * **toMx** (char raw=0) const
- mxArray * **toMxP** (char flag=0) const
- mxArray * **toMx_base** (char raw=0) const
- mxArray * **toMx_base_d** (char raw=0) const
- mxArray * **toMx_Struct** () const
- mxArray * **toMxP_S** () const
- mxArray * **toMxP_C** () const
- mxArray * **toMxT** (char raw=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tst=0) const
- void **toMxStruct** (mxArray *S, const char *vname) const
- void **mat2mx** (mxArray *a) const
- void **mat2mxs** (mxArray *a, char field_nr) const
- void **info** (const char *istr="ans") const
- void **print** (const char *istr="", char mflag=0) const
- void **Print** (const char *istr="", char mflag=0) const
- void **recPrint** (size_t i, const char *istr="", char mflag=0) const
- void **printdata** (const char *istr, const char *dbl_fmt=" %8.4g", const char *rsep="\n") const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (const char *fmt, const char *sep=" ", const char *rsep=";", size_t stride=0, const char *sep2="") const
- [wbstring](#) **rec2Str** (size_t k, const char *fmt="", const char *sep=" ", size_t stride=0, const char *sep2="") const
- [wbstring](#) **sizeStr** () const
- void **put** (const char *F, int L, const char *vname="ans", const char *ws="caller", const char raw=0) const
- void **put** (const char *vname="ans", const char *ws="caller", const char raw=0) const
- void **putg** (const char *vname="ans", const char raw=0) const
- void **putx** (const char *vname="ans", const char raw=0) const
- void **put0** (const char *vname="ans", const char *ws="caller") const
- void **getReal** ([wbMatrix](#)< double > &R) const
- void **getImag** ([wbMatrix](#)< double > &I) const
- void **set** (const [wbMatrix](#)< double > &R, const [wbMatrix](#)< double > &I)
- [wbMatrix](#) & **Conj** ()

- void **setRec** (size_t r, const T x)
- void **setRec** (size_t r, const T x1, const T x2)
- void **recSet** (size_t idest, size_t isrc)
- void **recSet** (size_t, const [wbvector](#)< T > &)
- template<class T2 >
void **recSetT** (size_t, const [wbvector](#)< T2 > &)
- void **recSet** (size_t, const [wbMatrix](#) &, size_t)
- void **recSet** (size_t i, const [wbvector](#)< T > &, const [wbvector](#)< T > &)
- void **recSet** (size_t, const T *, T bfac=1)
- void **recSetP** (size_t, const T *, size_t n=-1)
- void **recSetP** (size_t, const T *, size_t, const T *, size_t)
- void **recSetP** (size_t, const T *, const [wbindex](#) &i1, const T *, const [wbindex](#) &i2)
- void **recSetB** (size_t, size_t, size_t D, const T *, const T *=NULL, const T bfac=1)
- void **recSetB** (size_t, const [wbindex](#) &, size_t D, const T *)
- void **recSet** (size_t k, const [wbvector](#)< T > &a, const [wbindex](#) &ia, const [wbvector](#)< T > &b, const [wbindex](#) &ib)
- void **recSet** (size_t k, const T *a, const [wbindex](#) &ia, const T *b, const [wbindex](#) &ib)
- template<class T2 >
void **setCol** (size_t k, const [wbvector](#)< T2 > &v)
- template<class T2 >
void **setCol** (size_t k, const T2 *v, T fac=1, size_t stride=1)
- void **setCol** (size_t k, const T &x)
- template<class T2 >
void **setColT** (size_t k, const [wbvector](#)< T2 > &v)
- template<class T2 >
void **setColT** (size_t k, const T2 *v, T fac=1, size_t stride=1)
- void **setLastCol** (const T &x)
- void **recAddP** (size_t, const T *)
- int **recLess** (size_t, size_t, char lex=1) const
- int **recLesse** (size_t, size_t, char lex=1) const
- int **recEqual** (size_t, size_t) const
- int **recEqual** (size_t j1, const T *d) const
- T **recDiff2** (size_t i1, size_t i2, size_t n=-1) const
- T **recDiff2P** (size_t i1, const T *b, size_t n=-1) const
- bool **recAllEqual** (const T *v)
- bool **recAllEqual** ()
- bool **recIsZero** (size_t r)
- char **recCompare** (size_t, size_t, size_t m=-1, char lex=1, T eps=0) const
- char **recCompare** (size_t, const [wbvector](#)< T > &r2, char lex=1) const
- char **recCompareP** (size_t, const T *, size_t m=-1, char lex=1, T eps=0) const
- void **set2Rec** (size_t i)
- [wbMatrix](#) & **Set2Recs** (const WBINDEX &I)
- [wbvector](#)< T > & **getRec** (size_t j0, [wbvector](#)< T > &v, char ref=0) const
- [wbvector](#)< T > **getRec** (size_t j0) const
- size_t **UnionRecs** (const [wbMatrix](#) &B, [wbindex](#) &Ia)
- bool **gotRecOverlap** (const [wbMatrix](#) &B) const
- bool **gotRecOverlapSA** (const [wbMatrix](#) &B) const
- size_t **findValsCol** (const size_t k, const [wbvector](#)< T > &v, [wbindex](#) &Ia) const
- [wbMatrix](#) & **getRecs** (const WBINDEX &, [wbMatrix](#) &M) const
- [wbMatrix](#) **getRecs** (const WBINDEX &I) const
- [wbMatrix](#) & **getRecs** (size_t j1, size_t j2, [wbMatrix](#) &M, size_t m=0) const
- [wbMatrix](#) **getRecs** (size_t i1, size_t i2, size_t m=0) const
- [wbvector](#)< T > & **getCol** (const size_t c, [wbvector](#)< T > &v) const
- [wbvector](#)< T > **getCol** (const size_t c) const
- [wbMatrix](#) **getCols** (const size_t j1, const size_t j2) const

- **wbMatrix** & **getCols** (size_t j1, size_t j2, **wbMatrix** &M) const
- template<class T1 >
 wbMatrix & **getCols** (size_t n, const T1 *l, **wbMatrix**< T > &M) const
- **wbMatrix** & **getCols** (const **wbperm** &p, **wbMatrix** &M) const
- **wbMatrix** & **getCols** (const **wbindex** &l, **wbMatrix** &M, char iflag=0) const
- **wbMatrix** & **set2Cols** (const size_t j1, const size_t j2)
- **wbMatrix** & **set2Cols** (const **wbindex** &J, char iflag=0)
- **wbMatrix** & **recPermute** (const **wbperm** &P, **wbMatrix** &M, char iflag=0) const
- **wbMatrix** & **recPermute** (const **wbperm** &P, char iflag=0)
- **wbMatrix** & **colPermute** (const **wbperm** &P, **wbMatrix** &M, char iflag=0) const
- **wbMatrix** & **colPermute** (const **wbperm** &P, char iflag=0)
- **wbMatrix** & **FlipRecs** ()
- **wbMatrix** & **FlipCols** ()
- **wbMatrix** & **cols2Front** (const WBINDEX &l1, **wbMatrix** &M) const
- **wbMatrix** & **cols2End** (const WBINDEX &l1, **wbMatrix** &M) const
- void **blockPermute** (const **wbperm** &P, **wbMatrix** &M) const
- void **BlockPermute** (const **wbperm** &P)
- **wbvector**< T > & **recProd** (**wbvector**< T > &p) const
- T **recProd** (size_t) const
- T **recSum** (size_t) const
- **wbvector**< T > & **recSum** (**wbvector**< T > &) const
- **wbvector**< T > & **recSumA** (**wbvector**< T > &) const
- **wbvector**< T > **recSum** () const
- T **colSum** (size_t) const
- **wbMatrix** & **repmat** (size_t m, size_t n, **wbMatrix** &Q, size_t pad1=0, size_t pad2=0) const
- **wbMatrix** & **Repmat** (size_t m, size_t n, size_t pad1=0, size_t pad2=0)
- void **ColKron** (const **wbMatrix** &Q, int d=-1)
- **wbMatrix** & **getBlocks** (size_t j1, size_t j2, const size_t D, **wbMatrix** &M) const
- **wbMatrix** & **getBlock** (size_t j1, const size_t D, **wbMatrix** &M) const
- **wbMatrix** **getBlock** (size_t j1, const size_t D) const
- template<class T2 >
 wbMatrix< T > & **getBlock** (size_t k, size_t D, const **wbMatrix**< T2 > &R, size_t k2, size_t D2, **wbMatrix** &M) const
- **wbMatrix** & **getBlock** (size_t i, size_t j, size_t n, size_t m, **wbMatrix** &M) const
- **wbMatrix** & **blockSum** (size_t D, **wbMatrix** &) const
- void **blockSum** (size_t, size_t, const size_t, **wbMatrix**< double > &) const
- void **blockSum** (const **wbindex** &J, const size_t, **wbMatrix**< double > &) const
- void **setBlock** (size_t k, const size_t D, const T *)
- void **SetBlock** (size_t i0, size_t j0, const **wbMatrix** &M)
- WBIDXMAT & **toBlockIndex** (widx_t D, WBIDXMAT &Il, **wbvector**< **wbMatrix** > *Ql=NULL) const
- template<class T2 >
 void **toBlockIndex** (widx_t D, **wbMatrix**< T2 > &R, widx_t DR, WBIDXMAT *Il=NULL, WBIDXMAT *IS=NULL, WBIDXMAT *SS=NULL, **wbvector**< **wbMatrix** > *Ql=NULL) const
- void **groupRecs** (**wbperm** &P, WBINDEX &d, size_t m=-1, char lex=1, **wbindex** *lg=NULL, **wbMatrix**< T > *X=NULL)
- void **groupRecs** (**wbindex** &lg, widx_t m=-1, char lex=1)
- void **groupRecs** (**groupIndex**< widx_t > &lg, size_t m=-1, char lex=1, **wbindex** *lg=NULL)
- void **groupRecs** (**groupIndex**< widx_t > &lg, size_t m, char lex, const wperm_t **p)
- void **groupRecs** (**wbperm** &P, WBINDEX &d, **wbMatrix** &B, size_t m=-1, char lex=1, **wbindex** *lg=NULL) const
- void **groupRecs** (**wbperm** &P, WBINDEX &d, const WBINDEX *l, char lex=1)
- template<class T2 >
 void **groupRecs_rdeg** (**wbperm** &P, WBINDEX &D, const **wbMatrix**< T2 > &R, WBINDEX *lb=NULL, WBINDEX *l2=NULL, WBINDEX *Sb=NULL, char iflag=0)

- void **groupRecs** ([wbperm](#) &P, WBINDEX &D, size_t nc, WBINDEX &lb, WBINDEX &l2, WBINDEX &Sb, char iflag=0)
- void **groupSortedRecs** (WBINDEX &d, char keepall=0, size_t m=-1, char lex=1)
- void **groupSortedRecs** (WBINDEX &d, const WBINDEX *l)
- void **groupSortedRecs** (WBINDEX &d, size_t mc, char lex, WBINDEX &lb, WBINDEX &l2, WBINDEX &Sb)
- void **makeUnique** ()
- void **makeUnique** ([wbperm](#) &P, WBINDEX &D)
- void **makeUnique** ([wbindex](#) &l)
- void **findUnique** ([wbindex](#) &l) const
- size_t **findUniqueRecSorted1** (size_t n=-1, T eps=T(1E-12)) const
- void **makeUnique_ig** ([wbindex](#) &lu)
- void **makeUnique** ([groupIndex](#)< widx_t > &IG)
- [wbMatrix](#) & **SortRecs** ([wbperm](#) &P, char dir=+1, char lex=1)
- [wbMatrix](#) & **sortRecs** ([wbMatrix](#) &B, [wbperm](#) &P, char dir=+1, char lex=1)
- [wbMatrix](#) & **SortRecs** (char dir=+1, char lex=1)
- [wbMatrix](#) & **sortRecs_float** ([wbperm](#) &P, char dir=+1)
- [wbMatrix](#) & **sortRecs_float** ()
- char **findRecsInSet** (const [wbMatrix](#) &S, [wbvector](#)< int > &l) const
- int **findRecSorted** (const T *r, size_t n=-1, char lex=1) const
- int **findRec** (const T *r, size_t n=-1, char lex=1) const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- WBINDEX & **tolIndex** (WBINDEX &l, const WBINDEX *S) const
- WBIDXMAT & **tolIndex2D** (const WBINDEX &S, const [wbindex](#) &lc, WBIDXMAT &IJ, char pos) const
- template<class T1 >
[wbMatrix](#)< T > & **getCols** (size_t n, const T1 *l, [wbMatrix](#)< T > &M) const
- template<class T2 >
void **groupRecs_rdeg** ([wbperm](#) &P, WBINDEX &D, const [wbMatrix](#)< T2 > &R, [wbvector](#)< widx_t > *lb, [wbvector](#)< widx_t > *l2, [wbvector](#)< widx_t > *Sb, char iflag)
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const

Public Attributes

- T * **data**
- size_t **dim1**
- size_t **dim2**
- char **isdiag**
- char **isref**

Protected Member Functions

- void **NEW_DATA** (T *d=NULL)
- void **RENEW** (const size_t &d1, const size_t &d2, const T *dd=NULL, char iflag=1)

The documentation for this class was generated from the following files:

- wblib.h
- wbMatrix.hh
- wbMatrix.cc

4.107 wbMsg Class Reference

Public Member Functions

- **wbMsg** (const char *file, int line, const char *istr="Message queue", int key_=0)
- int **Rec** (const char *, int, [wbMsgBuf](#) &rbuf, unsigned flag=IPC_NOWAIT)
- int **Snd** (const char *, int, [wbMsgBuf](#) &sbuf, const char *istr, double val=0., unsigned type=1)

Public Attributes

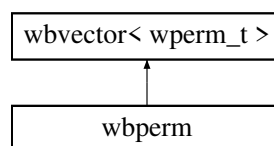
- [wbstring](#) **info**
- int **msqid**
- int **msgflg**
- key_t **key**

The documentation for this class was generated from the following file:

- wbmsg.hh

4.108 wbperm Class Reference

Inheritance diagram for wbperm:



Public Member Functions

- **wbperm** (wperm_t n, char reverse=0)
- **wbperm** (const [wbperm](#) &P, char iflag=0, unsigned r=-1)
- **wbperm** (const [wbperm](#) &P, size_t r)
- template<class T >
 wbperm (const [wbvector](#)< T > &b, wperm_t offset=0)
- **wbperm** (const [wbindex](#) &P)
- **wbperm** (const char *s, wperm_t offset=1)
- wperm_t **el1** (wperm_t i) const
- [wbperm](#) & **operator=** (const char *s)
- int **initStr** (const char *F, int L, const char *s, wperm_t offset=1)
- [wbperm](#) & **init** (const [wbperm](#) &P, char iflag=0, unsigned r=-1)
- [wbperm](#) & **Extend** (wperm_t r)
- [wbperm](#) & **init_trafo** (const [wbperm](#) &p1_, const [wbperm](#) &p2_)
- [wbperm](#) & **init_pad** (const [wbperm](#) &P, wperm_t r)
- [wbperm](#) & **init** (unsigned n=0, char reverse=0)
- [wbperm](#) & **init** (unsigned n, const wperm_t *d, char ref=0)
- [wbperm](#) & **init** (const char *F, int L, const mxArray *a)
- [wbperm](#) & **init** (const char *F, int L, const mxArray *a, wperm_t offset)
- template<class T >
 [wbperm](#) & **init** (const char *F, int L, const [wbvector](#)< T > &b, wperm_t offset=0)
- [wbperm](#) & **initTranspose** (wperm_t r)
- [wbperm](#) & **initPerm2** (wperm_t r)
- [wbperm](#) & **initFirstTo** (wperm_t k, wperm_t N)
- [wbperm](#) & **initLastTo** (wperm_t k, wperm_t N)
- [wbperm](#) & **init2Front** (const WBPPerm &l, wperm_t N)
- [wbperm](#) & **init2End** (const WBPPerm &l, wperm_t N)
- [wbperm](#) & **init2FrontB** (wperm_t m, wperm_t N)
- [wbperm](#) & **init2EndB** (wperm_t m, wperm_t N)
- [wbperm](#) & **init2Front** (wperm_t k, wperm_t N)
- [wbperm](#) & **init2End** (wperm_t k, wperm_t N)
- [wbperm](#) & **Cycle** (wperm_t k1, wperm_t k2)
- [wbperm](#) & **initCycle** (wperm_t n, wperm_t k1, wperm_t k2)
- [wbperm](#) & **Index** (wperm_t n, char reverse=0)
- [wbperm](#) & **Index** (wperm_t i1, wperm_t i2)
- char **isValidPerm** (const char *F, int L, wperm_t r=-1) const
- char **isValidPerm** (wperm_t r=-1) const
- [wbperm](#) & **Complete** (wperm_t l)
- [wbperm](#) & **Invert** ()
- [wbperm](#) & **invert** ([wbperm](#) &iP) const
- [wbperm](#) & **Rotate** (swperm_t l)
- [wbperm](#) & **initRotate** (wperm_t n, swperm_t k)
- [wbperm](#) & **Permute** (const [wbperm](#) &P, char iflag, wperm_t r)
- [wbperm](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- bool **sameAs** (const [wbperm](#) &b) const
- bool **isIdentityPerm** () const
- bool **isIdentityPerm** (const char *F, int L, wperm_t r=-1) const
- bool **isReversePerm** () const
- bool **isCyclic2F** (wperm_t m, wperm_t n=-1, char iflag=0) const
- bool **isOpTranspose** () const
- **operator bool** () const
- bool **operator!** () const
- [wbperm](#) & **times** (const [wbperm](#) &p2, [wbperm](#) &Pout) const
- [wbperm](#) & **flipIdx** ([wbperm](#) &P) const

- [wbperm](#) & [FlipIdx](#) ()
- [wbperm](#) [flipIdx](#) () const
- void [ToRaw](#) ()
- void [ToRaw](#) (wperm_t n)
- [wbstring](#) [toStr](#) () const
- mxArray * [toMx](#) (const char tflag=0) const

Public Member Functions inherited from [wbvector](#)< [wperm_t](#) >

- [wbvector](#) (size_t l, wperm_t x)
- [wbvector](#) (size_t l, const wperm_t *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & [init](#) (size_t l=0)
- [wbvector](#) & [init](#) (size_t l, const wperm_t *d0)
- [wbvector](#) & [init](#) (size_t l, const wperm_t *d0, char ref)
- [wbvector](#) & [init](#) (const [wbvector](#) &v)
- [wbvector](#) & [init](#) (size_t n, const [wbvector](#) &v)
- [wbvector](#)< wperm_t > & [init](#) (const [wbsparray](#)< wperm_t > &S)
- [wbvector](#) & [init](#) (size_t l1, const wperm_t *d1, size_t l2, const wperm_t *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const wperm_t *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const wperm_t &x)
- [wbvector](#) & [initStride](#) (size_t l, const wperm_t *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const wperm_t *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const wperm_t *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< wperm_t > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< wperm_t > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< wperm_t > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< wperm_t > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< wperm_t > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< wperm_t > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const wperm_t *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)
- [wbvector](#) & [operator=](#) (const [wbvector](#) &v)
- [wbvector](#)< wperm_t > & [operator=](#) (const [wbvector](#)< TB > &v)
- [wbvector](#) & [set](#) (const wperm_t &x)
- [wbvector](#) & [set](#) (const [wbvector](#)< size_t > &l, const wperm_t &x)
- void [set](#) (size_t offset, const wperm_t *x, size_t n)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)

- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const wperm_t &x)
- [wbvector](#) & **setMax** (const wperm_t &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const wperm_t &x)
- [wbvector](#) & **add** (const wperm_t *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const wperm_t &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const wperm_t &x) const
- size_t **find1** () const
- size_t **count** (const wperm_t &x) const
- size_t **icount** (const wperm_t &x) const
- size_t **nnz** () const
- size_t **nnz** (wperm_t eps) const
- size_t **numZeros** (wperm_t eps=wperm_t(1E-14)) const
- size_t **numEQ** (const wperm_t &) const
- size_t **numGT** (const wperm_t &) const
- size_t **numLT** (const wperm_t &) const
- [wbindex](#) & **find** (const wperm_t &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const wperm_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const wperm_t &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const wperm_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const wperm_t &x1, const wperm_t &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (wperm_t r) const
- int **findClosest** (wperm_t r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const wperm_t *x)
- void **reset** ()
- void **cpy** (const wperm_t *v)
- void **cpy** (const wperm_t *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (wperm_t &A1, wperm_t &A2)
- [wbvector](#) & **Collect** (wperm_t &A1, wperm_t &A2, wperm_t &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)

- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const wperm_t *b)
- [wbvector](#) & **Plus** (const wperm_t *b, wperm_t bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, wperm_t bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, wperm_t bfac=1, wperm_t cfac=0) const
- [wbvector](#) & **times** (wperm_t fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- wperm_t **sum** () const
- wperm_t **sum** (size_t i1, const char *estr) const
- wperm_t **sum** (size_t i1, size_t i2) const
- wperm_t **cumsum_** ([wbvector](#) &S, char xflag=0) const
- wperm_t **Cumsum** (char zflag=0) const
- wperm_t **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< wperm_t > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- wperm_t **scalarProd** (const [wbvector](#) &v) const
- wperm_t **scalarProd** (const wperm_t *v) const
- wperm_t **norm** () const
- wperm_t **norm2** () const
- double **normDiff** (const [wbvector](#) &v, wperm_t fac=1) const
- wperm_t **normDiff2** (const [wbvector](#) &v, wperm_t fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- wperm_t **GSProject** (const wperm_t *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const wperm_t *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &l) const
- void **getImag** ([wbvector](#)< double > &l) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const wperm_t &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const wperm_t &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const wperm_t &a, wperm_t eps=1E-15) const

- bool **allUnequal** (const wperm_t &a) const
- bool **any** () const
- bool **anyUnequal** (const wperm_t &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const wperm_t &x, size_t *k=NULL) const
- bool **anyEqual** (const wperm_t &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const wperm_t &x, size_t *k=NULL) const
- bool **anyGT** (const wperm_t &a) const
- bool **anyGE** (const wperm_t &a) const
- bool **anyLT** (const wperm_t &a) const
- bool **anyLE** (const wperm_t &a) const
- bool **allGT** (const wperm_t &a) const
- bool **allGE** (const wperm_t &a) const
- bool **allLE** (const wperm_t &a) const
- bool **allLT** (const wperm_t &a) const
- bool **allIn** (const wperm_t &a1, const wperm_t &a2) const
- bool **anySmallVals** (const wperm_t eps=1E-15) const
- bool **isEqual** (const wperm_t *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const wperm_t &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- wperm_t & **el** (size_t i) const
- wperm_t **elx** (size_t i, const wperm_t &x) const
- const wperm_t & **operator[]** (size_t i) const
- wperm_t & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- wperm_t & **last** () const
- wperm_t & **last** (size_t &i) const
- wperm_t & **last** (TI &i_) const
- wperm_t & **p2last** () const
- wperm_t * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const wperm_t *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const wperm_t *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)

- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< wperm_t > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< wperm_t > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const wperm_t &x)
- [wbvector](#) & **Append** (size_t d2, const wperm_t *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const wperm_t *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const wperm_t *v) const
- int **recLessE** (const wperm_t *v) const
- int **recCompare** (const wperm_t *v) const
- wperm_t **max** () const
- wperm_t **max** (size_t *k) const
- wperm_t **max** (unsigned &k) const
- wperm_t **max_** (wperm_t x) const
- wperm_t **min** () const
- wperm_t **min** (size_t *k) const
- wperm_t **min** (unsigned &k) const
- wperm_t **min_** (wperm_t x) const
- wperm_t **aMax** (size_t *k=NULL) const
- wperm_t **aMax** (unsigned &k) const
- wperm_t **aMin** (char zflag=0, size_t *k=NULL) const
- wperm_t **maxneg** (size_t *k=NULL) const
- wperm_t **minpos** (size_t *k=NULL) const
- wperm_t **avg** () const
- wperm_t **std** () const
- void **range** (wperm_t &xmin, wperm_t &xmax) const
- wperm_t **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &l0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &l, wperm_t *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &l)
- void **select** (const WBINDEX &l, wperm_t *r) const
- [wbvector](#) & **select** (const WBINDEX &l, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &l) const
- [wbvector](#) & **Select** (const WBINDEX &l)
- [wbvector](#) & **BlockSelect** (const WBINDEX &l, const WBINDEX &D)
- [wbvector](#) & **getl** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()

- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- `wperm_t` **prod** () const
- `wperm_t` **prod** (`wperm_t` x) const
- `wperm_t` **prod** (`size_t` i, `size_t` i2) const
- `wperm_t` **prod** (`size_t` i, `size_t` i2, `wperm_t` x) const
- `wperm_t` **prod** (const T1 *l, `size_t` n) const
- T2 **prod** (const T1 *l, `size_t` n, T2 &s2) const
- void **prod2** (`wperm_t` &p1, `wperm_t` &p2, const `wperm_t` &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const `wperm_t` &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector](#)< `wperm_t` >

- `wperm_t` * **data**
- `size_t` **len**
- char **isref**

Protected Member Functions inherited from [wbvector< wperm_t >](#)

- [wbvector](#) & **RENEW** (size_t n, const wperm_t *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const wperm_t &x)

The documentation for this class was generated from the following files:

- wbpperm.hh
- wbindex.hh
- wbpperm.cc

4.109 [wbRecs< T >](#) Class Template Reference

Public Member Functions

- **wbRecs** (const [wbMatrix< T >](#) &A, [wbperm](#) &P, char dir_=1, char lex_=1)
- bool **operator()** (const size_t &pa, const size_t &pb) const

The documentation for this class was generated from the following file:

- wbMatrix.hh

4.110 [wbREF< TX >](#) Class Template Reference

Public Member Functions

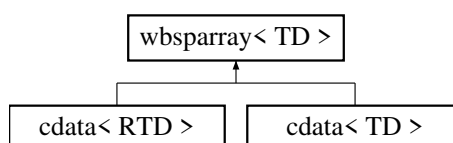
- **wbREF** (const TX &A, char flag)
- TX & **Ref** ()

The documentation for this class was generated from the following file:

- gathered.hh

4.111 [wbsparray< TD >](#) Class Template Reference

Inheritance diagram for [wbsparray< TD >](#):



Public Member Functions

- **wbsparray** (SPIDX_T d1)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- **wbsparray** (const [wbvector](#)< SPIDX_T > &d, SPIDX_T l=0)
- **wbsparray** (char *sidx, SPIDX_T l=0)
- **wbsparray** (const [wbsparray](#) &a)
- **wbsparray** (const char *F, int L, const mxArray *a)
- **wbsparray** (const mxArray *a)
- **wbsparray** & **init** (const char *F, int L, const [wbararray](#)< TD > &A, TD eps=0)
- template<class TA >
 [wbararray](#)< TA > & **toFull** ([wbararray](#)< TA > &A) const
- template<class TA >
 [wbvector](#)< TA > & **toFull** ([wbvector](#)< TA > &A) const
- template<class DB >
 [wbsparray](#) & **init** (const [wbsparray](#)< DB > &a)
- SPIDX_T **catRecs** (SPIDX_T i0, const [wbsparray](#) &a, SPIDX_T id)
- **wbsparray** & **init** (const [wbvector](#)< SPIDX_T > &S0, SPIDX_T l=0)
- template<class I2 >
 [wbsparray](#) & **init** (const [wbvector](#)< I2 > &S_, SPIDX_T l=0)
- **wbsparray** & **init** ()
- **wbsparray** & **init** (SPIDX_T d1)
- **wbsparray** & **init** (SPIDX_T d1, SPIDX_T d2)
- **wbsparray** & **init** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- **wbsparray** & **init** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- **wbsparray** & **initz** ()
- **wbsparray** & **initz** (SPIDX_T d1, SPIDX_T n)
- **wbsparray** & **initz** (SPIDX_T d1, SPIDX_T d2, SPIDX_T n)
- **wbsparray** & **initz** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T n)
- **wbsparray** & **init_rzd** (SPIDX_T r, SPIDX_T n)
- **wbsparray** & **init_nnz** (SPIDX_T n=0, char kflag=0)
- **wbsparray** & **setRand** (double x=1, char sflag=0)
- **wbsparray** & **setRand_data** (double x=1, char sflag=0)
- template<class DB >
 [wbsparray](#) & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const [wbsparray](#)< DB > &B)
- template<class DB >
 [wbsparray](#) & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const [wbsparray](#)< DB > &B)
- template<class DB >
 [wbsparray](#) & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const [wbsparray](#)< DB > &B)
- **wbsparray** & **initEmpty** (const [wbsparray](#) &a)
- **wbsparray** & **init** (const char *F, int L, const mxArray *a, unsigned k=0)
- **wbsparray** & **init2ref** (const [wbsparray](#) &a)
- **wbsparray** & **unRef** ()
- **wbsparray** & **initCAT** (const char *F, int L, [wbvector](#)< [wbsparray](#) * > &X, [wbvector](#)< SPIDX_T > *i0=NULL, char del=0)
- **wbsparray** & **initCAT** (const char *F, int L, [wbvector](#)< [wbsparray](#) > &X, const [wbvector](#)< SPIDX_T > *i0=NULL)
- **wbsparray** & **initCAT** (const char *F, int L, const [wbvector](#)< const [wbsparray](#) * > &X, unsigned dim)
- **wbsparray** & **initCAT** (const char *F, int L, const [wbvector](#)< [wbsparray](#) > &X, unsigned dim)
- **wbsparray** & **Cat** (const char *F, int L, const [wbsparray](#) &B, unsigned dim)
- **wbsparray** & **init_kron** (const [wbvector](#)< SPIDX_T > &sa, const [wbvector](#)< SPIDX_T > &sb, SPIDX_T nab, char kflag)

- void **setSize_kron** (const char *F, int L, const [wbvector](#)< SPIDX_T > &sa, const [wbvector](#)< SPIDX_T > &sb, char kflag=0)
- void **setSize_kron** (const char *F, int L, char kflag=0)
- [wbsparray](#) & **setRec** (SPIDX_T i, SPIDX_T j)
- template<class T2 >
 [wbsparray](#) & **setRec** (SPIDX_T i, SPIDX_T i1, SPIDX_T i2, const T2 &x)
- [wbsparray](#) & **setRecP** (SPIDX_T i, const SPIDX_T *l, const TD &x)
- [wbsparray](#) & **setRec_kron** (SPIDX_T l, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2, unsigned kflag=0)
- [wbsparray](#) & **Resize** (SPIDX_T l)
- [wbsparray](#) & **REDSIZE_NNZ** (SPIDX_T l)
- [wbsparray](#) & **Resize** (const [wbvector](#)< SPIDX_T > &S)
- [wbsparray](#) & **operator=** (const [wbsparray](#) &a)
- template<class DB >
 [wbsparray](#) & **operator=** (const [wbsparray](#)< DB > &a)
- [wbsparray](#) & **save2** ([wbsparray](#) &a, char ref=0)
- SPIDX_T **sub2ind** (SPIDX_T k) const
- void **ind2sub** (SPIDX_T k, SPIDX_T *l) const
- void **ind2sub** (SPIDX_T k, [wbindex](#) &l) const
- [wbsparray](#)< TD > & **make2D** (const [wbindex](#) &ic, [wbsparray](#)< TD > &B, char pos=1) const
- [sparseIndex2D](#)< SPIDX_T > & **toIndex2D** (const [wbindex](#) &ic, [sparseIndex2D](#)< SPIDX_T > &a, char pos, char use_rmaj=0) const
- [wbsparray](#) & **initDiag** (SPIDX_T d)
- [wbsparray](#) & **initDiag** (const [wbvector](#)< TD > &dd)
- [wbsparray](#) & **initDiag2** (SPIDX_T d, TD dval=1)
- bool **isDiagMatrix** (TD eps=1E-14) const
- bool **isDiag** (const char *F=0, int L=0) const
- [wbvector](#)< TD > & **getDiag** (const char *F, int L, [wbvector](#)< TD > &dd) const
- TD **getDiag** (const char *F, int L, SPIDX_T k) const
- [wbsparray](#) & **getDiag** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **diag2reg** (const char *F=0, int L=0, unsigned r=-1)
- [wbsparray](#) & **diag2reg** (const char *F, int L, [wbsparray](#) &A, unsigned r=-1) const
- [wbsparray](#) & **toDiag** (const char *F=0, int L=0)
- int **toDiagSafe** (const char *F, int L, TD eps=1E-14)
- [wbsparray](#) & **initIdentity** (SPIDX_T d, TD dval=1)
- [wbsparray](#) & **initIdentity** (SPIDX_T d1, SPIDX_T d2, sSPIDX_T m=0, TD dval=1)
- [wbsparray](#) & **initIdentity** (const [wbvector](#)< SPIDX_T > &S0, int m=-1, sSPIDX_T k=0, char fflag='f')
- [wbsparray](#) & **initIdentityB3** (SPIDX_T d1, SPIDX_T d2, SPIDX_T D, sSPIDX_T i0=0, char fflag='f')
- unsigned **rank** (const char *F=NULL, int L=0) const
- SPIDX_T **dim** () const
- SPIDX_T **dim** (unsigned k) const
- SPIDX_T **dim0** (unsigned k) const
- SPIDX_T **matDim** (unsigned k=1)
- SPIDX_T **maxDim** (unsigned r=-1) const
- TD **norm2** (char tnorm=0) const
- [wbstring](#) **sizeStr** () const
- TD **SkipTinyCols** (TD eps=1E-14)
- TD **SkipTiny** (TD eps=1E-14)
- bool **isVector** (const char *F=0, int L=0) const
- bool **isMatrix** (const char *F=0, int L=0) const
- bool **isRank** (unsigned r) const
- bool **isSMatrix** (const char *F=0, int L=0, SPIDX_T *d=NULL) const
- bool **isSMatrix** (const char *F, int L, SPIDX_T d) const
- bool **isEmpty** () const
- bool **isEmpty_** () const
- **operator bool** () const

- bool **operator!** () const
- bool **isZero** (TD eps=0, char flag=0) const
- SPIDX_T **nnz** () const
- SPIDX_T **nnz** (const TD eps1, const TD eps2=1E9) const
- SPIDX_T **nnz_diag** () const
- SPIDX_T **numel** () const
- TD **norm** (char tnorm=0) const
- template<class T2 >
 wbvector< T2 > & **norm2vec** (unsigned k, **wbvector**< T2 > &A, char tnorm=0) const
- void **swap** (**wbsparray** &B)
- **wbsparray** & **TimesEI** (const **wbsparray** &B, char tnorm=0)
- TD **dotProd** (const char *F, int L, const **wbsparray** &B, char tnorm=0) const
- TD **froNorm2** (const **wbsparray** &B) const
- double **normDiff2** (const **wbsparray** &B) const
- double **normDiff** (const **wbsparray** &B) const
- TD **Normalize** (char tnorm=0, char qflag=0)
- TD **NormalizeCol** (SPIDX_T k, char tnorm=0, char qflag=0)
- **wbsparray** & **QRdecomp** (const char *F, int L, **wbsparray** &Q, char posR=0, TD eps=1E-14)
- **wbsparray** & **OrthoNormalizeColsQR** (const char *F=NULL, int L=0, TD eps=1E-14)
- **wbsparray** & **OrthoNormalizeColsGS** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, TD eps=1E-14, unsigned np=1)
- **wbsparray** & **setCol** (const char *F, int L, SPIDX_T k, const **wbsparray** &v)
- **wbsparray** & **setCol** (SPIDX_T k, const **wbsparray** &v)
- double **sparsity** () const
- int **checkTrailingSingletons** (const char *F=0, int L=0, unsigned *r=NULL) const
- int **SkipTrailingSingletons** (const char *F=0, int L=0, unsigned r=-1)
- int **skipTrailingSingletons** (const char *F, int L, **wbsparray** &c, unsigned r=-1) const
- bool **hasSingletons** (const **wbindex** &l) const
- **wbsparray** & **skipSingletons** (const char *F, int L, const **wbindex** &l, **wbsparray** &A) const
- **wbsparray** & **AddTrailingSingletons** (const char *F, int L, unsigned r)
- **wbsparray** & **initScalar** (const TD x=0, unsigned r=-1)
- char **isScalar** (const char *F=0, int L=0, char zflag=0) const
- **wbsparray** & **toScalar** (const char *F=0, int L=0, unsigned r=-1)
- TD **getScalar** (const char *F=0, int L=0) const
- char **checkSize** (const char *F=0, int L=0, const char *fct=0, const char *istr=0) const
- char **checkSize2** (const char *F, int L, const **wbsparray** &B) const
- void **check_IDX_range** (const char *F=0, int L=0) const
- bool **isProptold** () const
- bool **isProptold** (TD &q, TD eps=1E-14) const
- bool **isIdentityMatrix** (TD eps=1E-14) const
- char **isIdentity** (TD eps=1E-14) const
- char **isIdentity** (const **wbperm** &P, TD *dval=NULL, TD eps=1E-14) const
- bool **hasSize** (SPIDX_T d1) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const
- template<class T >
 wbvector< T > & **getSize** (**wbvector**< T > &S) const
- char **fitsSize** (const **wbsparray** &, char strict=0) const
- char **sameSize** (const **wbsparray** &, char strict=0) const
- char **sameSizeUp2Singletons** (const **wbvector**< SPIDX_T > &S, char strict=0) const
- char **matchWithSingletons** (const char *F, int L, const **wbvector**< SPIDX_T > &S, **wbvector**< int > &M) const
- char **sameUptoFac** (const **wbsparray** &B, TD *fac=NULL, char lex=-1, TD eps=1E-12) const
- bool **operator==** (const **wbsparray** &B) const
- bool **operator!=** (const **wbsparray** &B) const

- [wbsparray](#) & **Plus** (const char *F, int L, const [wbsparray](#) &B, TD bfac=1, TD afac=1, TD eps=0)
- TD **sum** () const
- TD **aMin** () const
- TD **aMax** () const
- SPIDX_T **findRecSortedP** (SPIDX_T *idx, SPIDX_T n, char lex=-1)
- [wbsparray](#) & **Eldiv** (const char *F, int L, const [wbvector](#)< TD > &x_)
- [wbsparray](#) & **operator+=** (const [wbsparray](#) &B)
- [wbsparray](#) & **operator-=** (const [wbsparray](#) &B)
- [wbsparray](#) & **Conj** ()
- [wbsparray](#) & **operator*=** (TD x)
- [wbsparray](#) & **operator/=** (TD x)
- const TD & **operator[]** (SPIDX_T i) const
- TD & **operator[]** (SPIDX_T i)
- TD **value** (const SPIDX_T *idx, SPIDX_T n, char lex=-1) const
- [wbstring](#) **info2Str** (const char *istr=0) const
- size_t **memSize** () const
- void **info** (const char *istr="") const
- void **info** (const char *F, int L, const char *s="") const
- void **print** (const char *istr="", const char *fmt="") const
- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **printdata** (const char *istr="ans", const char *fmt="") const
- void **sperror_this** (const char *F, int L, const char *fct=0, const char *istr=0) const
- TD **trace** () const
- template<class T2 >
[wbvector](#)< T2 > & **trace** (unsigned k, [wbvector](#)< T2 > &A) const
- bool **isSym_aux** (const char *file, int line, const char *fct, const [wbsparray](#) &B, TD eps, TD *xref, const char symflag='s', const char lflag=1) const
- bool **isHConj** (const [wbsparray](#) &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isHConj** (TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (const [wbsparray](#) &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (TD eps=1E-12, TD *xref=NULL) const
- [wbsparray](#) & **Sort** (char lex=-1)
- double **Compress** (const char *F=0, int L=0, TD eps=0, char lex=-1, const [wbperm](#) *P=NULL)
- bool **isCompressed** (char lex=-1, char dir=0) const
- int **splitSparseCM** (const char *F, int L, [wbvector](#)< [wbsparray](#) > &X, TD eps=0, char ref=0, char tnorm=0)
- [wbsparray](#) & **reshape** (const [wbvector](#)< SPIDX_T > &S, [wbsparray](#) &A) const
- [wbsparray](#) & **Reshape** (const [wbvector](#)< SPIDX_T > &S)
- [wbsparray](#) & **Reshape** (SPIDX_T s1, SPIDX_T s2)
- [wbsparray](#) & **Reshape** (SPIDX_T s1, SPIDX_T s2, SPIDX_T s3)
- [wbvector](#)< TD > & **getCol** (SPIDX_T k, [wbvector](#)< TD > &v) const
- [wbvector](#)< TD > & **getRow** (SPIDX_T k, [wbvector](#)< TD > &v) const
- [wbsparray](#) & **getBlock** (const char *F, int L, SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2, [wbsparray](#)< TD > &B) const
- [wbsparray](#) & **getCol** (SPIDX_T k, [wbsparray](#)< TD > &b)
- [wbsparray](#) & **splitBlock** (const char *F, int L, sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2, [wbsparray](#)< TD > &B)
- [wbsparray](#) & **SplitQR** (sSPIDX_T k, sSPIDX_T k_, [wbsparray](#)< TD > &X)
- TD **getHouseholderVec** (const char *F, int L, SPIDX_T k, SPIDX_T j, [wbsparray](#)< TD > &b, TD eps2=0) const
- [wbsparray](#) & **IDX_Shift** (sSPIDX_T i1, sSPIDX_T i2)
- [wbsparray](#)< TD > & **LProject1** (const [wbsparray](#)< TD > &b, [wbsparray](#)< TD > &X, TD xfac=1, char tnorm=0) const
- SPIDX_T **LContractVec_omp** (const [wbsparray](#)< TD > &b, [wbvector](#)< TD > &x, char tnorm) const
- [wbsparray](#) & **splitLast** (SPIDX_T k, [wbsparray](#) &a) const
- [wbsparray](#) & **select0** (const WBINDEX &, unsigned dim, [wbsparray](#) &) const

- `wbsparray` & `Select0` (const `WBINDEX` &I, unsigned dim)
- `wbsparray` & `Permute` (const char *s, char iflag=0)
- `wbsparray` & `Permute` (const `wbperm` &, char iflag=0)
- `wbsparray` & `permute` (const char *s, `wbsparray` &, char iflag=0) const
- `wbsparray` & `permute` (const `wbperm` &, `wbsparray` &, char iflag=0) const
- `wbsparray` & `transpose` (const char *F, int L, `wbsparray` &A) const
- `wbsparray` & `Transpose` (const char *F=0, int L=0)
- `wbsparray` & `htranspose` (const char *F, int L, `wbsparray` &A) const
- `wbsparray` & `hTranspose` (const char *F=0, int L=0)
- `wbsparray` & `ColPermute` (const `wbperm` &P, char iflag=0)
- `wbsparray` & `MatPermute` (const `wbperm` &P, char iflag=0)
- `wbsparray` & `mtimes` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char sflag=0, const TD eps=0)
- char `contract_scalar` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal=`wbperm`(), const TD &afac=1., const TD &cfac=0.) const
- `wbsparray` & `contract_diag_diag` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac) const
- char `contract_check_2full` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac, char flag, TD eps, `SPIDX_T` *nnz_↵ C=NULL) const
- `wbsparray` & `contract` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal=`wbperm`(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- `wbsparray` & `contract` (const char *F, int L, const char *ica, const `wbsparray` &B, const char *icb, `wbsparray` &C, const `wbperm` &pfinal=`wbperm`(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- `wbarray`< TD > & `contract` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbarray`< TD > &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac) const
- `wbsparray` & `contract` (const char *F, int L, unsigned i1, const `wbvector`< TD > &B, `wbsparray` &C) const
- `wbsparray` & `comm` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char aflag='N', char bflag='N') const
- `wbsparray` & `tensorProdX` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char conja=0, const TD &afac=1, char conjb=0, const TD &cfac=0, const `wbperm` *Pfinal=NULL) const
- `wbsparray` & `tensorProd` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char aflag='N', char bflag='N', char kflag=0) const
- `wbsparray` & `Kron` (const `wbsparray` &B, char aflag='N', char bflag='N', char kflag='k')
- `wbsparray` & `kron` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char aflag='N', char bflag='N', char kflag='k') const
- `mxArray` * `toMxSp` () const
- `mxArray` * `toMx` (int m=-99) const
- `mxArray` * `IDtoMx` () const
- `mxArray` * `mxCreateStruct` (unsigned m, unsigned n) const
- void `add2MxStruct` (`mxArray` *a, unsigned i, int ma=-99) const
- `mxArray` * `mxCreateCell` (unsigned m, unsigned n) const
- void `add2MxCell` (`mxArray` *a, unsigned i) const
- `mxArray` * `toMx0` (int m=-99) const
- `mxArray` * `mxCreateStructX` (unsigned m, unsigned n, int ma=-99) const
- void `add2MxStructX` (`mxArray` *a, unsigned i, int ma=-99) const
- `mxArray` * `toMX` () const
- `mxArray` * `mxCreateSTRUCT` (unsigned m, unsigned n) const
- `mxArray` * `add2MxSTRUCT` (`mxArray` *a, unsigned i) const
- void `put` (const char *vname, unsigned ma=-99, const char *ws="caller") const
- void `put0` (const char *F, int L, const char *vname, const char *ws="caller") const
- void `put` (const char *F, int L, const char *vname, unsigned ma=-99, const char *ws="caller", char flag=0) const
- template<class T2 >
`wbvector`< T2 > & `toFull` (`wbvector`< T2 > &A) const

- `template<class DB >`
`wbsparray< TD > & init (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const wbsparray< DB > &B)`
- `template<class DB >`
`wbsparray< TD > & init (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const wbsparray< DB > &B)`
- `template<class DB >`
`wbsparray< TD > & init (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const wbsparray< DB > &B)`

Public Attributes

- `wbvector< SPIDX_T > SIZE`
- `wbMatrix< SPIDX_T > IDX`
- `wbvector< TD > D`
- `char isref`

Protected Member Functions

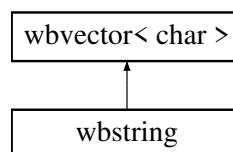
- `unsigned get2DSize (const char *F, int L, SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const`
- `unsigned get2DIndex (wbMatrix< SPIDX_T > &IJ, SPIDX_T &d1, SPIDX_T &d2, int m=-1) const`

The documentation for this class was generated from the following files:

- `wblib.h`
- `wbsparray.hh`
- `wbsparray.cc`

4.112 wbsting Class Reference

Inheritance diagram for wbsting:



Public Member Functions

- `wbsting (unsigned n=0)`
- `wbsting (unsigned n, const char c)`
- `wbsting (const char *s, unsigned n)`
- `wbsting (const wbsting &s)`
- `wbsting (const char *s1)`
- `wbsting (const char *s1, const char *s2, const char *s3=NULL, const char *s4=NULL)`
- `wbsting (const mxArray *a)`
- `wbsting & init (const mxArray *a)`
- `wbsting (const char *F, int L, const mxArray *a)`
- `wbsting & init (unsigned n=0)`

- [wstring](#) & **init** (const char *s)
- [wstring](#) & **init** (const char *s, unsigned n)
- [wstring](#) & **init** (const char *F, int L, const mxArray *a)
- [wstring](#) & **initrep** (const char *s, unsigned n)
- [wstring](#) & **initrep** (char c, unsigned n)
- void **set** (char c)
- [wstring](#) & **operator=** (const char *s)
- bool **gotTerm0** ()
- bool **strchr** (const char *s)
- [wstring](#) & **push** (const char *F, int L, const char *s, char extend=1)
- [wstring](#) & **push** (const char *s, char extend=1)
- [wstring](#) & **pushf** (const char *F, int L, const char *fmt,...)
- [wstring](#) & **cpy** (const char *F, int L, const char *s)
- int **RegEx_replace** (const char *F, int L, const char *pat, const char *rep, char icode=0, char gflag=1)
- int **printf** (const char *F, int L, const char *fmt,...)
- [wstring](#) & **time** ()
- [wstring](#) & **time** (const time_t &t)
- [wstring](#) & **time_sys** ()
- [wstring](#) & **time_sys** (const time_t &t)
- [wstring](#) & **Upper** ()
- [wstring](#) & **Lower** ()
- [wstring](#) **toupper** () const
- [wstring](#) **tolower** () const
- [wstring](#) & **Chomp** ()
- char **getopt** (char c)
- char **getopt** ()
- const char * **basename** (const char c="/") const
- mxArray * **toMx** () const
- size_t **toHash** (unsigned offset=0) const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- template<class T >
char * **init2Fmt** (const T &x __attribute__((unused)), int n=-1, int p=-1)
- template<class T >
[wstring](#) & **operator<<** (const T *x)
- template<class T >
[wstring](#) & **operator<<** (const T &x)
- [wstring](#) **operator+** (const [wstring](#) &s) const
- [wstring](#) **operator+** (const char *s) const
- [wstring](#) & **operator+=** (const char *s)
- [wstring](#) & **operator+=** (const [wstring](#) &s)
- unsigned **cat** (unsigned &k, const char *s)
- [wstring](#) **toStr** (char level=0) const
- char & **operator[]** (unsigned i) const
- unsigned **strlen_** () const
- bool **isEmpty** () const
- char **isPrint** () const
- bool **operator!** () const
- **operator bool** () const
- unsigned **isName** (int L=-1) const
- int **operator==** (const char *s) const
- int **operator!=** (const char *s) const
- int **operator==** (const [wstring](#) &s) const
- int **operator!=** (const [wstring](#) &s) const
- int **operator<** (const [wstring](#) &s) const
- int **operator<=** (const [wstring](#) &s) const

- int **operator>** (const [wstring](#) &s) const
- int **operator>=** (const [wstring](#) &s) const
- int **firstLocation** (const [wstring](#) &, unsigned k=0)
- int **firstLocation** (const char *, unsigned offset=0)
- std::istream & **getline** (std::istream &istr, char c='\n')
- template<> [wstring](#) & **operator<<** (const char *s)
- template<> [wstring](#) & **operator<<** (const char &x)
- template<> [wstring](#) & **operator<<** (const int &x)
- template<> [wstring](#) & **operator<<** (const unsigned &x)
- template<> [wstring](#) & **operator<<** (const long &x)
- template<> [wstring](#) & **operator<<** (const unsigned long &x)
- template<> [wstring](#) & **operator<<** (const double &x)
- template<> [wstring](#) & **operator<<** (const float &x)
- template<> [wstring](#) & **operator<<** (const [wcomplex](#) &x)

Public Member Functions inherited from [wbvector< char >](#)

- **wbvector** (size_t l, char x)
- **wbvector** (size_t l, const char *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const char *d0)
- [wbvector](#) & **init** (size_t l, const char *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & **init** (const [wbsparray](#)< char > &S)
- [wbvector](#) & **init** (size_t l1, const char *d1, size_t l2, const char *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const char *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const char &x)
- [wbvector](#) & **initStride** (size_t l, const char *d0, size_t stride)
- [wbvector](#) & **init_stride** (const char *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const char *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< char > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< char > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< char > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const char *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpfr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)

- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< char > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const char &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const char &x)
- void **set** (size_t offset, const char *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const char &x)
- [wbvector](#) & **setMax** (const char &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const char &x)
- [wbvector](#) & **add** (const char *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const char &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const char &x) const
- size_t **find1** () const
- size_t **count** (const char &x) const
- size_t **icount** (const char &x) const
- size_t **nnz** () const
- size_t **nnz** (char eps) const
- size_t **numZeros** (char eps=char(1E-14)) const
- size_t **numEQ** (const char &) const
- size_t **numGT** (const char &) const
- size_t **numLT** (const char &) const
- [wbindex](#) & **find** (const char &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const char &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const char &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const char &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const char &x1, const char &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (char r) const
- int **findClosest** (char r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const char *x)
- void **reset** ()
- void **cpy** (const char *v)
- void **cpy** (const char *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)

- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (char &A1, char &A2)
- [wbvector](#) & **Collect** (char &A1, char &A2, char &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const char *b)
- [wbvector](#) & **Plus** (const char *b, char bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, char bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, char bfac=1, char cfac=0) const
- [wbvector](#) & **times** (char fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- char **sum** () const
- char **sum** (size_t i1, const char *estr) const
- char **sum** (size_t i1, size_t i2) const
- char **cumsum_** ([wbvector](#) &S, char xflag=0) const
- char **Cumsum** (char zflag=0) const
- char **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< char > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- char **scalarProd** (const [wbvector](#) &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const [wbvector](#) &v, char fac=1) const
- char **normDiff2** (const [wbvector](#) &v, char fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- char **GSProject** (const char *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const char *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const

- [wvector](#) & **Conj** ()
- [wvector](#)< [wcomplex](#) > & **Conj** ()
- [wvector](#) & **applyFlag** (char &flag, const char &fac, [wvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const [wvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const
- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const [wvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wvector](#) &v) const
- bool **operator!=** (const [wvector](#) &v) const
- bool **isEqualR** (const [wvector](#) &v) const
- bool **operator^** (const [wvector](#) &v) const
- bool **deepEqualP** (const [wvector](#) &B) const
- char **cmp** (const [wvector](#) &v) const
- bool **operator<** (const [wvector](#) &v) const
- int **operator<** (const char &r) const
- bool **operator<=** (const [wvector](#) &v) const
- bool **operator>** (const [wvector](#) &v) const
- bool **operator>=** (const [wvector](#) &v) const
- char & **el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wvector](#) **operator[]** (const [wvector](#)< size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (TI &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wvector](#) & **setRand** (double fac=1., double shift=0.)
- [wvector](#)< [wcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wvector](#) & **Resize_** (size_t n)

- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< char > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const char *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const
- char **max_** (char x) const
- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const
- char **avg** () const
- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &l0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &l, char *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &l)
- void **select** (const WBINDEX &l, char *r) const
- [wbvector](#) & **select** (const WBINDEX &l, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &l) const
- [wbvector](#) & **Select** (const WBINDEX &l)

- [wbvector](#) & **BlockSelect** (const WBINDEX &l, const WBINDEX &d)
- [wbvector](#) & **getl** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const char &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< char >](#)

- char * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< char >](#)

- [wbvector](#) & **RENEW** (size_t n, const char *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const char &x)

The documentation for this class was generated from the following files:

- [wbstring.hh](#)
- [wbstring.cc](#)

4.113 wbttop Class Reference

Public Member Functions

- [wbstring](#) **VmSize2Str** () const
- size_t **getVmSize** () const
- [wbstring](#) **VmPeak2Str** () const
- size_t **getVmPeak** () const
- int **runningLarge** (const char *F=NULL, int L=0, double th0=0.80, double fac=1.2)
- void **MemStat** (const char *F=0, int L=0, char lflag=0) const

Public Attributes

- pid_t **pid**
- pid_t **ppid**
- size_t **systemem**

The documentation for this class was generated from the following files:

- [util.hh](#)
- [util.cc](#)

4.114 WbUtil< T > Class Template Reference

Public Member Functions

- **T eps** ()
- [wbcomplex eps](#) ()
- void **adjust_tnorm** (char &tnorm __attribute__((unused)))
- bool **hasConj** ()
- bool **hasNoConj** ()
- bool **isFloat** ()
- bool **isFloat** ()
- bool **isFloat** ()
- bool **isFloat** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- double **eps** ()
- float **eps** ()
- long double **eps** ()

Static Public Member Functions

- static void **adjust_tnorm** (char &tnorm)
- static bool **hasConj** ()
- static bool **hasNoConj** ()
- static constexpr bool **isPOD** ()
- static bool **isFloat** ()
- static bool **isInt** ()
- static constexpr char **isComplex** ()
- static char **isReal** ()

The documentation for this class was generated from the following file:

- util.hh

4.115 wbvector< T > Class Template Reference

Public Member Functions

- **wbvector** (size_t l, T x)
- **wbvector** (size_t l, const T *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- template<class T2 >
 wbvector (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)

- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const T *d0)
- [wbvector](#) & **init** (size_t l, const T *d0, char ref)
- [wbvector](#) & **initp** (size_t l, const T *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const T &x)
- [wbvector](#) & **initStride** (size_t l, const T *d0, size_t stride)
- [wbvector](#) & **init_stride** (const T *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#) & **init_data** (const T *d0)
- template<class T0 >
 [wbvector](#) & **init_data** (const T0 *d0)
- template<class T0 >
 [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< T > & **init** (const [wbsparray](#)< T > &S)
- template<class T2 >
 [wbvector](#)< T > & **initT** (const [wbsparray](#)< T2 > &S)
- template<class T2 >
 [wbvector](#)< T > & **initT** (size_t l, const T2 *v)
- template<class T2 >
 [wbvector](#)< T > & **initT** (const [wbvector](#)< T2 > &v)
- template<class T2 >
 [wbvector](#)< T > & **initT_** (const [wbvector](#)< T2 > &v)
- template<class T2 >
 [wbvector](#)< T > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const T *d0)
- [wbvector](#) & **unRef** ()
- [wbvector](#) & **init** (size_t l1, const T *d1, size_t l2, const T *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init_mpfir** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- template<class TB >
 [wbvector](#)< T > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const T &x)
- [wbvector](#) & **setMin** (const T &x)
- [wbvector](#) & **setMax** (const T &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const T &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const T &x)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const T &x) const

- `bool isNormal () const`
- `bool isFinite () const`
- `size_t find1 (const T &x) const`
- `size_t find1 () const`
- `size_t count (const T &x) const`
- `size_t icount (const T &x) const`
- `size_t nnz () const`
- `size_t nnz (T eps) const`
- `size_t numZeros (T eps=T(1E-14)) const`
- `size_t numEQ (const T &) const`
- `size_t numGT (const T &) const`
- `size_t numLT (const T &) const`
- `wbindex & find (const T &x, wbindex &l, char iflag=0) const`
- `wbindex & find (const T &x, wbindex &l, wbindex &lx) const`
- `wbindex & findGT (const T &x, wbindex &l, char iflag=0) const`
- `wbindex & findGT (const T &x, wbindex &l, wbindex &lx) const`
- `wbindex & findzeros (wbindex &l) const`
- `wbindex & find (wbindex &l) const`
- `wbindex & find (wbindex &l, wbindex &lx) const`
- `wbvector< size_t > find () const`
- `wbindex & findRange (const T &x1, const T &x2, wbindex &l, const char *w=NULL) const`
- `int findClosestSorted (T r) const`
- `int findClosest (T r) const`
- `size_t findValues (const wbvector &v, wbindex &l) const`
- `wbvector & setp (const T *x)`
- `void set (size_t offset, const T *x, size_t n)`
- `void reset ()`
- `void cpy (const T *v)`
- `void cpy (const T *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (T &A1, T &A2)`
- `wbvector & Collect (T &A1, T &A2, T &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="") const`
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="") const`
- `template<class T_>`
`void operator+= (const T_ &a)`
- `template<class T_>`
`void operator-= (const T_ &a)`
- `template<class T_>`
`wbvector & operator*= (const T_ &a)`
- `template<class T_>`
`wbvector & operator/= (const T_ &a)`

- `template<class T_>`
`wbvector operator*` (const T_ &a) const
- `template<class T_>`
`wbvector operator+` (const T_ &a) const
- `template<class T_>`
`wbvector operator-` (const T_ &a) const
- `wbvector operator+` (const `wbvector` &b) const
- `wbvector operator-` (const `wbvector` &b) const
- `void operator+=` (const `wbvector` &b)
- `void operator-=` (const `wbvector` &b)
- `wbvector & Plus` (const T *b)
- `wbvector & Plus` (const T *b, T bfac)
- `wbvector & Plus` (const `wbvector` &b, T bfac=1)
- `void plus` (const `wbvector` &b, `wbvector` &c, T bfac=1, T cfac=0) const
- `wbvector & times` (T fac, `wbvector` &x) const
- `wbvector & RevertSigns` ()
- `int isum` () const
- `T sum` () const
- `T sum` (size_t i1, const char *estr) const
- `T sum` (size_t i1, size_t i2) const
- `T cumsum_` (`wbvector` &S, char xflag=0) const
- `T Cumsum` (char zflag=0) const
- `template<class T2>`
`T cumsum0prod` (`wbvector`< T2 > &b, `wbvector`< T > &cs, char xflag) const
- `wbvector & Eldiv` (const `wbvector` &v) const
- `wbvector & tensorProd` (const `wbvector` &v2, `wbvector` &vv) const
- `T scalarProd` (const `wbvector` &v) const
- `T scalarProd` (const T *v) const
- `T norm` () const
- `T norm2` () const
- `double normDiff` (const `wbvector` &v, T fac=1) const
- `T normDiff2` (const `wbvector` &v, T fac=1) const
- `double normDiff2_` (const `wbvector` &v, double fac=1) const
- `T GSProject` (const T *v, char isnorm=0, char tflag=0) const
- `wbvector & Normalize` (const char *F=NULL, int L=0)
- `wbvector & add` (const T *d)
- `wbvector & subtract` (const T *d)
- `void getReal` (`wbvector`< double > &R) const
- `void getImag` (`wbvector`< double > &I) const
- `void set` (const `wbvector`< double > &R, const `wbvector`< double > &I)
- `wbvector & Conj` ()
- `wbvector & applyFlag` (char &flag, const T &fac, `wbvector` &a)
- `bool allEqual` () const
- `bool allZero` () const
- `bool allOnes` () const
- `bool allEqual` (const T &a) const
- `bool allEqual2` (const T &a, T eps=1E-15) const
- `bool allUnequal` (const T &a) const
- `bool any` () const
- `bool anyUnequal` (const T &a) const
- `bool anyEqual` (const T &a, size_t *i1=NULL) const
- `bool anyGT` (const T &a) const
- `bool anyGE` (const T &a) const
- `bool anyLT` (const T &a) const
- `bool anyLE` (const T &a) const

- `bool allGT (const T &a) const`
- `bool allGE (const T &a) const`
- `bool allLE (const T &a) const`
- `bool allLT (const T &a) const`
- `bool allIn (const T &a1, const T &a2) const`
- `bool anySmallVals (const T eps=1E-15) const`
- `bool isEqual (const T *v) const`
- `bool anyEqual (const wbvector< size_t > &l, const T &x, size_t *k=NULL) const`
- `bool anyUnequal (const wbvector< size_t > &l, const T &x, size_t *k=NULL) const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool operator== (const wbvector &v) const`
- `bool operator!= (const wbvector &v) const`
- `bool isEqual (const wbvector &v) const`
- `bool isEqualR (const wbvector &v) const`
- `bool operator^ (const wbvector &v) const`
- `bool deepEqualP (const wbvector &B) const`
- `char cmp (const wbvector &v) const`
- `bool operator< (const wbvector &v) const`
- `bool operator<= (const wbvector &v) const`
- `bool operator> (const wbvector &v) const`
- `bool operator>= (const wbvector &v) const`
- `int operator< (const T &r) const`
- `T &el (size_t i) const`
- `T elx (size_t i, const T &x) const`
- `const T &operator[] (size_t i) const`
- `T &operator[] (size_t i)`
- `T &last () const`
- `T &last (size_t &i) const`
- `template<class T1 >`
`T &last (T1 &i_) const`
- `T &p2last () const`
- `T * ref (size_t i) const`
- `wbvector &setRand (double fac=1., double shift=0.)`
- `void setIdx (double shift=0., double fac=1.)`
- `int set2Group (const wbperm &P, const WBINDEX &D, const T *S0=NULL, char iflag=0)`
- `wbvector &Resize_ (size_t n)`
- `wbvector &Shorten2 (size_t n)`
- `wbvector &Resize (size_t n, const T *d=NULL)`
- `wbvector &resize (size_t n, wbvector &X) const`
- `wbvector &cat (const wbvector &v2)`
- `wbvector &Cat (const wbvector &v1, const wbvector &v2)`
- `wbvector &Cat (const wbvector< wbvector > &vv)`
- `wbvector &Cat (const wbvector< wbvector * > &vv)`
- `template<class T1, class T2, class T3, class T4 >`
`wbvector &Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)`
- `template<class T1, class T2, class T3, class T4 >`
`wbvector &Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3=0, const wbvector< T3 > *v4=0)`
- `wbvector &Append (const wbvector &v2)`
- `wbvector &Append (const T &x)`
- `wbvector &Append (size_t d2, const T *x)`
- `wbvector append (const wbvector &v) const`

- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const T *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const T *v) const
- int **recLessE** (const T *v) const
- int **recCompare** (const T *v) const
- T **max** () const
- T **max** (size_t *k) const
- T **max** (unsigned &k) const
- T **max_** (T x) const
- T **min** () const
- T **min** (size_t *k) const
- T **min** (unsigned &k) const
- T **min_** (T x) const
- T **aMax** (size_t *k=NULL) const
- T **aMax** (unsigned &k) const
- T **aMin** (char zflag=0, size_t *k=NULL) const
- T **maxneg** (size_t *k=NULL) const
- T **minpos** (size_t *k=NULL) const
- T **avg** () const
- T **std** () const
- void **range** (T &xmin, T &xmax) const
- T **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &l0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &l, T *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &l)
- void **select** (const WBINDEX &l, T *r) const
- [wbvector](#) & **select** (const WBINDEX &l, [wbvector](#) &a) const
- [wbvector](#) & **Select** (const WBINDEX &l)
- [wbvector](#) & **BlockSelect** (const WBINDEX &l, const WBINDEX &D)
- [wbvector](#) **select** (const WBINDEX &l) const
- [wbvector](#) & **getl** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- T **prod** () const
- T **prod** (T x) const
- T **prod** (size_t i, size_t i2) const
- T **prod** (size_t i, size_t i2, T x) const
- template<class T1 >
T **prod** (const T1 *l, size_t n) const
- template<class T1 , class T2 >
T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (T &p1, T &p2, const T &p0=0) const

- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const T &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- bool **isNormal** () const
- bool **isNormal** () const
- template<class T1 , class T2 , class T3 , class T4 >
[wbvector](#)< T > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3,
const [wbvector](#)< T3 > *v4)
- template<class T1 , class T2 , class T3 , class T4 >
[wbvector](#)< T > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4,
size_t l4)
- template<class TD >
[wbvector](#)< T > & **initT** (const [wbsparray](#)< TD > &S)
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &I)
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const

Public Attributes

- `T * data`
- `size_t len`
- `char isref`

Protected Member Functions

- `wbvector` & **RENEW** (`size_t n`, `const T *d=NULL`, `size_t l=-1`, `char init=1`)
- `wbvector` & **RENEW_VAL** (`size_t n`, `const T &x`)

The documentation for this class was generated from the following files:

- `wblib.h`
- `wbvector.hh`
- `wbvector.cc`

4.116 DY::weight_info< TQ > Class Template Reference**Public Member Functions**

- **weight_info** (`const weight_info &w`)
- **weight_info** (`unsigned m_`, `unsigned n`)
- `weight_info` & **init** (`unsigned m_`, `unsigned n`)
- `weight_info` & **init** (`const weight_info &w`)
- `weight_info` & **operator=** (`const weight_info &w`)
- `weight_info` & **save2** (`weight_info &w`)

Public Attributes

- `int m`
- `qset< TQ > p`

The documentation for this class was generated from the following file:

- `clebsch.hh`

4.117 DY::Weights< TQ > Class Template Reference**Public Member Functions**

- **Weights** (`const qset< TQ > &q`)
- `Weights` & **init** (`const qset< TQ > &q`)
- `size_t dim` () `const`
- `void print` (`const char *F`, `int L`, `const QType &q`, `char vflag=0`) `const`
- `mxAarray * toMx` () `const`

Public Attributes

- unsigned **n**
- [qset](#)< TQ > **qm**
- map< [qset](#)< TQ >, [weight_info](#)< TQ > > **W**
- map< [qset](#)< TQ >, const [weight_info](#)< TQ > * > **X**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.118 x3map< TQ, TD > Class Template Reference**Public Member Functions**

- **x3map** (const char *F, int L, const [CRef](#)< TQ > &A_, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B_, const [ctrlIdx](#) &icb, [wbperm](#) &cgp, char xflag=0)
- **x3map** & **init** ()
- int **initCtr** (const char *F, int L, const [CRef](#)< TQ > &A_, const [ctrlIdx](#) &ica_, const [CRef](#)< TQ > &B_, const [ctrlIdx](#) &icb_, [wbperm](#) &cgp, char xflag=1)
- **x3map** & **init** (const char *F, int L, const mxArray *S, unsigned k=0, [ctrlIdx](#) *ica=NULL, [ctrlIdx](#) *icb=NULL, char recal=1)
- bool **isEmpty** () const
- void **print** (const char *F=NULL, int L=0, const char *istr="", char vflag=0) const
- [wbstring](#) **toStr** (char vflag=0) const
- [wbstring](#) **x3Str** (char vflag=0) const
- size_t **memSize** () const
- mxArray * **mxCreatStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned l, const [cgc_contract_id](#)< MTI > *idc=NULL) const
- mxArray * **toMx** (const [cgc_contract_id](#)< MTI > *idc=NULL) const

Public Attributes

- [CData](#)< TQ, RTD > **a**
- [CData](#)< TQ, RTD > **b**
- [CData](#)< TQ, RTD > **c**
- const CDATA_TQ * **cgb**
- [wbperm](#) **pab**
- char **conj**
- int **zflag**
- [cgrType](#) **rtype**
- [wbarray](#)< TD > **x3**
- [wbarray](#)< TD > & **X3**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.119 X3Map< TQ, TD > Class Template Reference

Public Member Functions

- void **clear** ()
- int **init** (const char *F, int L, const mxArray *S, unsigned k=0)
- **x3map**< TQ, TD > & **getXBUF** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc)
- int **contains** (const char *F, int L, const **x3map**< TQ, TD > &x, const [cgc_contract_id](#)< MTI > &idc)
- int **contractCGR** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlidx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlidx](#) &icb, [CRef](#)< TQ > &C, char cg_preview=0)
- [CRef](#)< TQ > & **contractDegQ** (const char *F, int L, const [CRef](#)< TQ > &A, [CRef](#)< TQ > &B)
- unsigned **add** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc, const mxArray *S, unsigned k)
- mxArray * **toMx** () const
- void **Info** (const char *F=0, int L=0) const

Public Attributes

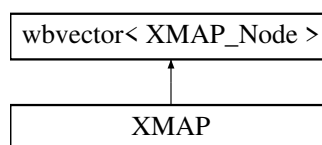
- unordered_map< [cgc_contract_id](#)< MTI >, **x3map**< TQ, TD >, [MHash](#)< MTI > > **XBUF**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.120 XMAP Class Reference

Inheritance diagram for XMAP:



Public Member Functions

- **XMAP** (unsigned n)
- **free** ()
- void **init** (unsigned n)
- template<class T >
void **set** (unsigned k, const wbArray< T > &A, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z)
- void **set** (unsigned k, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z)

Public Member Functions inherited from [wbvector< XMAP_Node >](#)

- [wbvector](#) (size_t l, [XMAP_Node](#) x)
- [wbvector](#) (size_t l, const [XMAP_Node](#) *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & [init](#) (size_t l=0)
- [wbvector](#) & [init](#) (size_t l, const [XMAP_Node](#) *d0)
- [wbvector](#) & [init](#) (size_t l, const [XMAP_Node](#) *d0, char ref)
- [wbvector](#) & [init](#) (const [wbvector](#) &v)
- [wbvector](#) & [init](#) (size_t n, const [wbvector](#) &v)
- [wbvector](#)< [XMAP_Node](#) > & [init](#) (const [wbsparray](#)< [XMAP_Node](#) > &S)
- [wbvector](#) & [init](#) (size_t l1, const [XMAP_Node](#) *d1, size_t l2, const [XMAP_Node](#) *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type __attribute__((unused))=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const [XMAP_Node](#) *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const [XMAP_Node](#) &x)
- [wbvector](#) & [initStride](#) (size_t l, const [XMAP_Node](#) *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const [XMAP_Node](#) *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const [XMAP_Node](#) *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< [XMAP_Node](#) > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< [XMAP_Node](#) > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< [XMAP_Node](#) > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< [XMAP_Node](#) > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< [XMAP_Node](#) > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< [XMAP_Node](#) > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const [XMAP_Node](#) *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)
- [wbvector](#) & [operator=](#) (const [wbvector](#) &v)
- [wbvector](#)< [XMAP_Node](#) > & [operator=](#) (const [wbvector](#)< TB > &v)
- [wbvector](#) & [set](#) (const [XMAP_Node](#) &x)
- [wbvector](#) & [set](#) (const [wbvector](#)< size_t > &l, const [XMAP_Node](#) &x)
- void [set](#) (size_t offset, const [XMAP_Node](#) *x, size_t n)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & [setMin](#) (const [XMAP_Node](#) &x)
- [wbvector](#) & [setMax](#) (const [XMAP_Node](#) &x)
- [wbvector](#) & [add](#) (const [wbvector](#)< size_t > &l, const [XMAP_Node](#) &x)
- [wbvector](#) & [add](#) (const [XMAP_Node](#) *d)
- [wbvector](#) & [Set](#) (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void [Sort](#) ([wbperm](#) &p, char dir=+1)
- void [Sort](#) (char dir=+1)

- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const [XMAP_Node](#) &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const [XMAP_Node](#) &x) const
- size_t **find1** () const
- size_t **count** (const [XMAP_Node](#) &x) const
- size_t **icount** (const [XMAP_Node](#) &x) const
- size_t **nnz** () const
- size_t **nnz** ([XMAP_Node](#) eps) const
- size_t **numZeros** ([XMAP_Node](#) eps=[XMAP_Node](#)(1E-14)) const
- size_t **numEQ** (const [XMAP_Node](#) &) const
- size_t **numGT** (const [XMAP_Node](#) &) const
- size_t **numLT** (const [XMAP_Node](#) &) const
- [wbindex](#) & **find** (const [XMAP_Node](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const [XMAP_Node](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const [XMAP_Node](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const [XMAP_Node](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const [XMAP_Node](#) &x1, const [XMAP_Node](#) &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** ([XMAP_Node](#) r) const
- int **findClosest** ([XMAP_Node](#) r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const [XMAP_Node](#) *x)
- void **reset** ()
- void **cpy** (const [XMAP_Node](#) *v)
- void **cpy** (const [XMAP_Node](#) *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** ([XMAP_Node](#) &A1, [XMAP_Node](#) &A2)
- [wbvector](#) & **Collect** ([XMAP_Node](#) &A1, [XMAP_Node](#) &A2, [XMAP_Node](#) &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const

- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const [XMAP_Node](#) *b)
- [wbvector](#) & **Plus** (const [XMAP_Node](#) *b, [XMAP_Node](#) bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, [XMAP_Node](#) bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, [XMAP_Node](#) bfac=1, [XMAP_Node](#) cfac=0) const
- [wbvector](#) & **times** ([XMAP_Node](#) fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- [XMAP_Node](#) **sum** () const
- [XMAP_Node](#) **sum** (size_t i1, const char *estr) const
- [XMAP_Node](#) **sum** (size_t i1, size_t i2) const
- [XMAP_Node](#) **cumsum_** ([wbvector](#) &S, char xflag=0) const
- [XMAP_Node](#) **Cumsum** (char zflag=0) const
- [XMAP_Node](#) **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< [XMAP_Node](#) > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- [XMAP_Node](#) **scalarProd** (const [wbvector](#) &v) const
- [XMAP_Node](#) **scalarProd** (const [XMAP_Node](#) *v) const
- [XMAP_Node](#) **norm** () const
- [XMAP_Node](#) **norm2** () const
- double **normDiff** (const [wbvector](#) &v, [XMAP_Node](#) fac=1) const
- [XMAP_Node](#) **normDiff2** (const [wbvector](#) &v, [XMAP_Node](#) fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- [XMAP_Node](#) **GSPProject** (const [XMAP_Node](#) *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const [XMAP_Node](#) *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const [XMAP_Node](#) &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const [XMAP_Node](#) &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const [XMAP_Node](#) &a, [XMAP_Node](#) eps=1E-15) const
- bool **allUnequal** (const [XMAP_Node](#) &a) const
- bool **any** () const
- bool **anyUnequal** (const [XMAP_Node](#) &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &I, const [XMAP_Node](#) &x, size_t *k=NULL) const
- bool **anyEqual** (const [XMAP_Node](#) &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &I, const [XMAP_Node](#) &x, size_t *k=NULL) const
- bool **anyGT** (const [XMAP_Node](#) &a) const

- bool **anyGE** (const [XMAP_Node](#) &a) const
- bool **anyLT** (const [XMAP_Node](#) &a) const
- bool **anyLE** (const [XMAP_Node](#) &a) const
- bool **allGT** (const [XMAP_Node](#) &a) const
- bool **allGE** (const [XMAP_Node](#) &a) const
- bool **allLE** (const [XMAP_Node](#) &a) const
- bool **allLT** (const [XMAP_Node](#) &a) const
- bool **allIn** (const [XMAP_Node](#) &a1, const [XMAP_Node](#) &a2) const
- bool **anySmallVals** (const [XMAP_Node](#) eps=1E-15) const
- bool **isEqual** (const [XMAP_Node](#) *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const [XMAP_Node](#) &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [XMAP_Node](#) & **el** (size_t i) const
- [XMAP_Node](#) **elx** (size_t i, const [XMAP_Node](#) &x) const
- const [XMAP_Node](#) & **operator[]** (size_t i) const
- [XMAP_Node](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- [XMAP_Node](#) & **last** () const
- [XMAP_Node](#) & **last** (size_t &i) const
- [XMAP_Node](#) & **last** (T1 &i_) const
- [XMAP_Node](#) & **p2last** () const
- [XMAP_Node](#) * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [XMAP_Node](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [XMAP_Node](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< [XMAP_Node](#) > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< [XMAP_Node](#) > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)

- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const [XMAP_Node](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [XMAP_Node](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const [XMAP_Node](#) *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const [XMAP_Node](#) *v) const
- int **recLessE** (const [XMAP_Node](#) *v) const
- int **recCompare** (const [XMAP_Node](#) *v) const
- [XMAP_Node](#) **max** () const
- [XMAP_Node](#) **max** (size_t *k) const
- [XMAP_Node](#) **max** (unsigned &k) const
- [XMAP_Node](#) **max_** ([XMAP_Node](#) x) const
- [XMAP_Node](#) **min** () const
- [XMAP_Node](#) **min** (size_t *k) const
- [XMAP_Node](#) **min** (unsigned &k) const
- [XMAP_Node](#) **min_** ([XMAP_Node](#) x) const
- [XMAP_Node](#) **aMax** (size_t *k=NULL) const
- [XMAP_Node](#) **aMax** (unsigned &k) const
- [XMAP_Node](#) **aMin** (char zflag=0, size_t *k=NULL) const
- [XMAP_Node](#) **maxneg** (size_t *k=NULL) const
- [XMAP_Node](#) **minpos** (size_t *k=NULL) const
- [XMAP_Node](#) **avg** () const
- [XMAP_Node](#) **std** () const
- void **range** ([XMAP_Node](#) &xmin, [XMAP_Node](#) &xmax) const
- [XMAP_Node](#) **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, [XMAP_Node](#) *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, [XMAP_Node](#) *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getI** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getI** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- [XMAP_Node](#) **prod** () const
- [XMAP_Node](#) **prod** ([XMAP_Node](#) x) const
- [XMAP_Node](#) **prod** (size_t i, size_t i2) const
- [XMAP_Node](#) **prod** (size_t i, size_t i2, [XMAP_Node](#) x) const
- [XMAP_Node](#) **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const

- void **prod2** ([XMAP_Node](#) &p1, [XMAP_Node](#) &p2, const [XMAP_Node](#) &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const [XMAP_Node](#) &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Public Attributes

- char **isref**

Public Attributes inherited from [wbvector< XMAP_Node >](#)

- [XMAP_Node](#) * **data**
- size_t **len**
- char **isref**

Additional Inherited Members**Protected Member Functions inherited from [wbvector< XMAP_Node >](#)**

- [wbvector](#) & **RENEW** (size_t n, const [XMAP_Node](#) *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const [XMAP_Node](#) &x)

The documentation for this class was generated from the following file:

- xmap.hh

4.121 XMAP_CMat Class Reference**Public Attributes**

- [wbCMat](#) * **M**
- unsigned **ic**
- unsigned **i1**
- unsigned **i2**
- [wbcomplex](#) **z**

The documentation for this class was generated from the following file:

- bicgMatC.cc

4.122 XMAP_Node Class Reference**Public Member Functions**

- **free** ()
- void **init** (const mxArray *a)
- [wbvector](#)< unsigned > **size** () const
- unsigned **rank** () const
- bool **sameSize** (const [XMAP_Node](#) &B) const
- template<class T >
void **set** (const [wbArray](#)< T > &A, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z, char ref=1)
- void **set** (unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z, char ref=1)
- template<class T >
void **set** (unsigned k, const [wbArray](#)< T > &A, unsigned ic_, unsigned i1_, unsigned i2_, [wbcomplex](#) z_, char ref)

Public Attributes

- [wbArray](#)< [wbcomplex](#) > * **Z**
- [wbArray](#)< double > * **R**
- char **tflag**
- [wbperm](#) **P**
- unsigned **ic**
- unsigned **i1**
- unsigned **i2**
- unsigned **isref**
- [wbcomplex](#) **z**

The documentation for this class was generated from the following file:

- xmap.hh

Chapter 5

File Documentation

5.1 clebsch.hh

```
00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : clebsch (for abelian and non-abelian symmetries)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_CLEBSCH_GORDAN_HH__
00021 #define __WB_CLEBSCH_GORDAN_HH__
00022
00023 //=====//
00024 // CStore => global CG container: CStore<gTQ> gCS; // CG_GLOBAL
00025 //
00026 //     global container for Clebsch Gordan coefficients
00027 //     - based on CData for single (Q1,Q2->Q) mapping.
00028 //     - based on QVec, qset, cdata below.
00029 //
00030 // cdata : wbararray<double>    // full data arrays!
00031 // cdata : wbSpparray    // switching to sparse // Wb,Dec22,11
00032 //
00033 //     actual storage of ClebschGordan coefficients
00034 //     - of rank 3(+1) for standard CG (3+1=4 in case of multiplicity)
00035 //     - cdata also includes (contracted) derivatives of ANY rank
00036 //
00037 // QVec : wbvector<QType>
00038 //
00039 //     single set of types of quantum numbers with QType
00040 //     - the most basic symmetry unit
00041 //
00042 // qset : wbvector<TQ>
00043 //
00044 //     actual set of quantum numbers interpreted through QVec
00045 //
00046 // Wb,Sep20,09 ; Wb,May01,10
00047 //
00048 // ----- //
00049 // NB! sign convention on CGCs #CGD_NORM_QS #CGD_NORM_EXTERN
00050 //
00051 // using CRef, ensure that CData is simply normalized to 1
00052 // internally within gCS. This ensures that for each OM set
00053 // * overlaps can simply be computed based on cgw data!
00054 // * OM within gCS can be also determined through cdata.norm2().
00055 // return value: number of trailing dimensions to be interpreted
00056 // as OM space.
00057 //
00058 // Changing cgw from vector to // CGW_ARRAY
```

```

00059 //      -> wbMatrix // Wb,Jun08,20
00060 //      -> wbararray // Wb,Apr23,21
00061 //
00062 // * 1st (fast) index contracts onto OM index of CData // wdim1()
00063 // * 2nd (slow) index onto OM index of A.data // wdim2()
00064 //      #CGW_INDEX_ORDER
00065 //
00066 // => normalization convention is  $cgw^T * cgw = 1$  // #CGW_NORM_QS
00067 //      such that cgw represents an isometry
00068 //      where cgw may have have fewer (but not more) columns than rows
00069 //      so it may hold  $cgw * cgw^T \neq 1$ 
00070 //
00071 // Benefits of this normalization
00072 // - OM normalization convention of CData // #CGD_NORM_QS
00073 //      required for simple orthonormal OM basis decomposition
00074 //      remains intact also for  $CData_{actual} = CData * cgw$ 
00075 //      irrespective of direction of legs
00076 // - therefore A.data already carries correct 3j weight factors for simple
00077 //      contraction, such that e.g.,  $A.norm2()$  of some QSpace A
00078 //      is simply carried by  $|A.data|^2$ 
00079 //
00080 // NB! for 'external' purposes only (readability) #CGD_NORM_EXTERN
00081 // and only for rank-2 tensors (i.e. scalar operators,
00082 // which never have OM!), use  $|cgw| = \sqrt{qdim}$ 
00083 // to reflect actual matrix elements in A.data, e.g., see
00084 // QSpace::NormCGW() -> CGR->NormSignW -> normExt()!
00085 // @QSpace/display.m also assumes this convention throughout.
00086 //
00087 // ----- //
00088 // Other (not so useful) attempts to normalize cgw
00089 // (with corresponding inverse factor multiplied onto data{:}):
00090 //
00091 //      |cgw| := sqrt(max(size(cdata))) // former version of CGD_NORM_EXTERN
00092 //
00093 // (+) rank-2 tensor are exact identities (or untaries for 1J symbols)
00094 // (+) rank-3 tensors are mostly normalized to  $\max(|CGC|)$ 
00095 //      e.g. (SS|0) is normalized the same way as (S0|S)
00096 //      whereas their standard normalization [see NormStd()]
00097 //      would be  $\sqrt{d_{out}}$ , here 1 or (2S+1), respectively!
00098 //
00099 // (-) standard rank-3 identity A tensors do not have  $cgd = Id$ 
00100 // (-) does not work specifically well for rank>3 tensors.
00101 //      e.g. consider
00102 //      contract( (1,1|2), 3, (1,1|2), 3 ) (i)
00103 //      vs. contract( (1,1|0), 3, (1,1|0), 3 ) (ii)
00104 //      while the output CData space is the same (1,1,1,1)
00105 //      in either case, already the normalization of the
00106 //      CData requires *different* normalization factors  $1/\sqrt{d3}$ 
00107 //      depending on the *contracted* 3rd index containing a multiplet
00108 //      of dimension d3. This information is no longer available
00109 //      once the CGCs have been contracted!
00110 //
00111 // Wb,Oct16,14 ; Wb,Nov04,14
00112 //
00113 // ===== //
00114 // RStore => global constructor class for non-abelian Lie groups
00115 //
00116 // RStore <TQ,TD> gRS; // global
00117 // * map[QType] := genRG_struct buf[]
00118 //
00119 //      genRG_struct<TQ,TD>
00120 //      * map[qset] := genRG_base<TQ,RTD> RSet[]
00121 //
00122 //      genRG_base<TQ,RTD>
00123 //      * basic symmetry settings such as
00124 //      generators, commutator relations (CR), ...
00125 //
00126 // Wb,Jun07,10
00127 //
00128 // ===== //
00129 /* CHANGE LOG
00130
00131 // Wb,Aug28,20 // PERMIT_RANK1_Q0 ----- //
00132
00133 Permit rank-1 QSpace if scalar, i.e. for q=0 only;
00134 e.g. for SU(2), can contract (S,0|S) at '13'
00135 for any spin-S multiplet, resulting in a non-zero weight
00136 since by DEF_CGD_NORM with d=2S+1 the multiplet dimension,
00137  $trace('S,0;S,'13') = 1/\sqrt{d} * d = \sqrt{d}$ 
00138 [e.g. see @QSpace/getIdentity3.m with --rho flag]
00139 however,  $contract('S,S;S,'13') = 0$ , i.e., rank-1 QSpace is
00140 not permitted for non-zero (non-scalar) symmetry label q!=0.
00141
00142 NB! a rank-1 QSpace may be generated, e.g., by contraction
00143 to obtain expectation values [HAM/calc_SdotS.m] after using
00144 @QSpace/getIdentity3.m
00145

```

```

00146     NB! contract('0,S,S*,'13','S,S*', '12')
00147     = sqrt(d) * trace('S,0;S,'13')
00148     = 1.
00149     since 2nd line acquires additional sqrt(d) factor from 'S,S*'
00150     => X-symbol has x3=1 - ok!
00151
00152     => permit rank-1 QSpace as output, but not necessarily as input
00153     yet with exceptions, such as getDimQS so that at least QSpace
00154     can be displayed using @QSpace/display.m
00155
00156 // Wb, June 2014 ----- //
00157
00158     * removed CG_SU2 (merged with treatment of QTYPE_SUN)
00159     * removed zflag from qset [e.g. nq = n(q) + n(qz)]
00160     * removed zflags from QType and qset
00161     * removed qset::RemoveZLabels (still got QSpace::RemoveZLabels!)
00162     * removed following variables from CData as these are redundant
00163       with data in RStore => genRG_struct => genRG_base:
00164       - qset<TQ> Q1,Q2,Q; // (Q1,Q2) -> Q
00165       - wbMatrix<TQ> Z1,Z2,Z; // (Z1,Z2) -> Z
00166       use gRS container to retrieve these where required
00167     * renamed CData => cdata; CGSpace => CData
00168
00169     => see Archive/clebsch_140627.[ch]* for version including zflag
00170
00171 //=====*/
00172
00173 #ifndef RTD
00174     #ifdef __WB_MPFR_HH__
00175
00176         #define MTI unsigned long
00177         #define RTD Wb::quad
00178
00179     #else
00180
00181         #define MTI unsigned long
00182         #define RTD double
00183
00184     #endif
00185 #endif
00186
00187 #ifndef gTQ
00188     #define gTQ int
00189 #endif
00190
00191 #ifndef gTD
00192     #define gTD double
00193 #endif
00194
00195 #define CData_TQ    CData<TQ,RTD>
00196 #define cdata__    cdata<RTD>
00197 #define cgsparray  wbsparray<RTD>
00198
00199
00200     template <class TQ, class TD> class CData;
00201     template <class TQ> class CRef;
00202
00203     class QVec;
00204
00205     int MAX_OM=9;
00206
00207     int get_CG_VERBOSE(const char *F=0, int L=0);
00208     int CG_VERBOSE=get_CG_VERBOSE();
00209
00210     unsigned get_CG_FIXIT(const char *F=0, int L=0);
00211     unsigned CG_FIXIT=get_CG_FIXIT();
00212
00213     #define FIX_CID3 1
00214     #define FIX_CTR 2
00215
00216     unsigned get_CG_PREVIEW(const char *F=0, int L=0);
00217     unsigned CG_PREVIEW=get_CG_PREVIEW();
00218
00219     int QSP_NUM_THREADS=0;
00220
00221     #define RC_LOAD_ERR_AB_MSG "RC_LOAD_ERROR (A or B)"
00222
00223     wstring RC_LOG;
00224
00225     #define CG_SKIP_DEPS1 1E-12
00226     #define CG_SKIP_DEPS2 1E-14
00227
00228     #define CG_SKIP_REPS 1E-17
00229
00230 #ifdef __WB_MPFR_HH__
00231     #define CG_SKIP_EPS1 1E-16
00232     #define CG_SKIP_EPS2 1E-18

```

```
00233
00234     #define CG_EPS    1E-38
00235     #define CG_EPS0   1E-36
00236     #define CG_EPS1   1E-16
00237     #define CG_EPS2   1E-18
00238
00239 #else
00240
00241     #define CG_EPS    2.22E-16
00242     #define CG_EPS0   1E-15
00243
00244     #define CG_SKIP_EPS1 1E-14
00245     #define CG_SKIP_EPS2 1E-16
00246
00247     #define CG_EPS1   1E-10
00248     #define CG_EPS2   1E-12
00249
00250 #endif
00251
00252     double cg_eps1=CG_EPS1;
00253     double cg_eps2=CG_EPS2;
00254
00255     enum QTYPE_SET {
00256         QTYPE_UNKNOWN,
00257         QTYPE_P,
00258         QTYPE_ZN,
00259         QTYPE_U1,
00260         QTYPE_ASEP,
00261         QTYPE_SUN,
00262         QTYPE_SpN,
00263         QTYPE_SON,
00264         QTYPE_SEN,
00265         QTYPE_NUM_TYPES
00266     };
00267
00268     const char* QTYPE_STR[QTYPE_NUM_TYPES]= {
00269         "?QTYPE?",
00270         "P",
00271         "ZN",
00272         "A",
00273         "!!?",
00274         "SUN",
00275         "Sp2N",
00276         "SO2N",
00277         "SO2N_1"
00278     };
00279
00280     #define CGC_ALL_ABELIAN "A*"
00281
00282     enum CGD_TYPE {
00283         CGD_UNKNOWN,
00284
00285         CGD_ABELIAN,
00286         CGD_IDENTITY,
00287
00288         CGD_REF_INIT,
00289
00290         CGD_BSZ_INIT,
00291
00292         CGD_FROM_CGR,
00293
00294         CGD_FROM_CTR,
00295         CGD_FROM_DEC,
00296
00297         CGD_STD3,
00298
00299         CGD_1JSY_ST3,
00300         CGD_1JSY_GEN,
00301
00302         CGD_STD3_X,
00303         CGD_GEN_X,
00304         CGD_EXPLICIT,
00305
00306         CGD_NUM_TYPES };
00307
00308     #define CID_RANK1_Q0    60
00309
00310     const char* CGD_TYPE_STR[CGD_NUM_TYPES+1] = { "C:??",
00311         "C:iA",
00312         "C:iE",
00313         "C:iR",
00314         "C:iS",
00315         "C:cR",
00316         "C:cX",
00317         "C:cD",
00318         "C:c3",
00319         "C:r1J",
```

```

00320     "C:c1J",
00321     "C:c3x",
00322     "C:cgx",
00323     "C:iX",
00324     "C:##";
00325
00326     enum CGR_TYPE { CGR_DEFAULT,
00327                     CGR_ABELIAN,
00328
00329                     CGR_CTR_SCALAR,
00330
00331                     CGR_CTR_ZERO,
00332
00333     CGR_NUM_TYPES };
00334
00335     const char* CGR_TYPE_STR[CGR_NUM_TYPES]= { "?CGR_TYPE?",
00336         "R:iA",
00337         "R:cS",
00338         "R:c0",
00339     };
00340
00341     namespace Mx {
00342
00343     template <>
00344     size_t Mx::Array<CGD_TYPE>::copy_to(CGD_TYPE* b, const wbpem *P, char tcheck)
00345     const { return rcopy_to(b,P,tcheck); };
00346
00347     template <>
00348     size_t Mx::Array<CGR_TYPE>::copy_to(CGR_TYPE* b, const wbpem *P, char tcheck)
00349     const { return rcopy_to(b,P,tcheck); };
00350
00351     };
00352
00353     template <class T> class qset;
00354     template <class T> class CStore;
00355
00356     template <class T> class QMap;
00357     template <class TQ, class TD> class blockSpaceQS;
00358
00359     #include <map>
00360
00361     namespace CG {
00362
00363     template <class TD> inline
00364     int signFirstVal(const char *F, int L,
00365         const TD *d, SPIDX_T n,
00366         double eps1=CG_EPS1, double eps2=CG_EPS2
00367     );
00368
00369     template <class TD> inline
00370     int rangeSignConvention(const char *F, int L,
00371         TD *d, SPIDX_T n,
00372         double eps1=CG_EPS1, double eps2=CG_EPS2
00373     );
00374
00375     template<class T>
00376     double FixRational(const char *F, int L,
00377         T* d, SPIDX_T n, unsigned niter=0,
00378         double eps1=-1, double eps2=-1
00379     );
00380     };
00381
00382     namespace DY {
00383
00384     template<class TQ>
00385     size_t wdim_A(unsigned n, const TQ *qs);
00386
00387     template<class TQ>
00388     size_t wdim_B(unsigned n, const TQ *qs);
00389
00390     template<class TQ>
00391     size_t wdim_C(unsigned n, const TQ *qs);
00392
00393     template<class TQ>
00394     size_t wdim_D(unsigned n, const TQ *qs);
00395
00396     };
00397
00398     namespace CG {
00399
00400     template <class TQ> inline
00401     unsigned gotQAlpha(const TQ* q, unsigned n) {
00402
00403         if (!WbUtil<TQ>().isInt()) return 0;
00404
00405         if (int(n)<0) wblog(FL,"ERR %s() got n=%d",FCT,n);
00406

```

```

00407     TQ qmax=61;
00408     #if __APPLE__
00409         qmax=35;
00410     #endif
00411
00412     for (unsigned i=0; i<n; ++i) { if (q[i]<0 || q[i]>qmax) return 0; }
00413
00414     return 1;
00415 };
00416
00417 template <> inline
00418 unsigned gotQAlpha(const char* s, unsigned n) {
00419     unsigned i=0;
00420
00421     #if __APPLE__
00422         for (; i<n; ++i) { if (!isalnum(s[i]) || s[i]>90) break; }
00423     #else
00424         for (; i<n; ++i) { if (!isalnum(s[i])) break; }
00425     #endif
00426
00427     if (int(n)<0) { return i; }
00428     else { return (i<n ? (i+1) : 0); }
00429
00430     return 1;
00431 };
00432
00433 template <class TQ> inline
00434 unsigned cstr2qset(const char *s, TQ *q, unsigned n=-1) {
00435     unsigned i=0;
00436     for (; i<n; ++i) {
00437         if (s[i]>='0' && s[i]<='9') { q[i]=s[i]-'0'; } else
00438         if (s[i]>='A' && s[i]<='Z') { q[i]=s[i]-'A'+10; } else
00439         #if !(defined __APPLE__)
00440         if (s[i]>='a' && s[i]<='z') { q[i]=s[i]-'a'+36; } else
00441         #endif
00442             break;
00443     }
00444
00445     if (int(n)<0) { return i; }
00446     else { return (i<n ? (i+1) : 0); }
00447 };
00448
00449 template <class TQ> inline
00450 int qset2cstr(const TQ *q, char *s, unsigned n) {
00451     for (unsigned i=0; i<n; ++i) {
00452         if (q[i]<0) { return (i+1); }
00453         if (q[i]<10) { s[i]='0'+q[i]; } else
00454         if (q[i]<36) { s[i]='A'+q[i]-10; } else
00455         #if !(defined __APPLE__)
00456         if (q[i]<62) { s[i]='a'+q[i]-36; } else
00457         #endif
00458             { return (i+1); }
00459     }
00460     return 0;
00461 };
00462
00463 class QType {
00464 public:
00465     QType(QTYPE_SET t=QTYPE_UNKNOWN, unsigned s=0)
00466         : type(t), sub(s) { if (t || s) validType(FL); };
00467
00468     QType(const char *s) : type(QTYPE_UNKNOWN), sub(0) { init(0,0,s); };
00469     QType(const QType &q) : type(q.type), sub(q.sub) { };
00470
00471     QType(const char *F, int L, const mxArray *a)
00472         : type(QTYPE_UNKNOWN), sub(0) { init(F,L,a); };
00473
00474 ~QType() { init(); };
00475
00476 QType& init() { type=QTYPE_UNKNOWN; sub=0; return *this; };
00477
00478 QType& initx(QTYPE_SET t, unsigned m=0) {
00479     if (!t && !m) { init(); return *this; }
00480     type=t; sub=m; validType(FL);
00481     return *this;
00482 };
00483
00484 QType& init(QTYPE_SET t, unsigned m=0) {
00485     if (!t && !m) { init(); return *this; }

```

```

00494
00495     type=t;
00496     if (type==QTYPE_SpN) { sub=m/2;
00497         if (m%2) wblog(FL,"ERR %s() invalid sub=%d with SpN",FCT,m);
00498     }
00499     else { sub=m; }
00500
00501     validType(FL); return *this;
00502 };
00503
00504 int init_s(const char *s);
00505
00506 QType& init(const char *s) { return init(FL,s); }
00507
00508 QType& init(const char *F, int L, const char *s) {
00509     if (!s || !s[0]) { type=QTYPE_UNKNOWN; sub=0; }
00510     else {
00511         if (init_s(s)) wblog(FL,"ERR invalid QType '%s'",s);
00512         else validType(F_L);
00513     }
00514     return *this;
00515 };
00516
00517 QType& init(const char *F, int L, const mxArray *a) {
00518     wbsttring s(F_L,a);
00519     return init(F_L,s.data);
00520 };
00521
00522 QType& init(const QType &q) {
00523     type=q.type; sub=q.sub; return *this;
00524 };
00525
00526 bool isUnknown() const {
00527     return (type==QTYPE_UNKNOWN && sub==0);
00528 };
00529
00530 bool isKnown() const {
00531     validType();
00532     return (type!=QTYPE_UNKNOWN);
00533 };
00534
00535 bool validType(const char *F=0, int L=0) const;
00536
00537 bool isAbelian(const char *F=NULL, int L=0) const {
00538     if (type<=QTYPE_UNKNOWN || type>=QTYPE_NUM_TYPES) wblog(F_L,
00539         "ERR invalid type %d, '%s'",type,STR_(this));
00540     return ((type && type<QTYPE_ASEP)? 1 : 0);
00541 };
00542
00543 bool isNonAbelian(const char *F=NULL, int L=0, char lflag=0) const {
00544     if (!lflag && !isUnknown()) {
00545         if (type<=QTYPE_UNKNOWN || type>=QTYPE_NUM_TYPES) wblog(F_L,
00546             "ERR invalid type %d, '%s'",type,STR_(this)); }
00547     return ((type && type>QTYPE_ASEP)? 1 : 0);
00548 };
00549
00550 bool isSU2() const { return (type==QTYPE_SUN && sub==1); };
00551 bool isSUN() const { return (type==QTYPE_SUN); };
00552 bool isSU(unsigned N) const { return (type==QTYPE_SUN && sub+1==N); };
00553
00554 bool isSpN() const { return (type==QTYPE_SpN); };
00555 bool isSON() const { return (type==QTYPE_SON); };
00556 bool isSEN() const { return (type==QTYPE_SEN); };
00557
00558 bool isU1() const { return (type==QTYPE_U1 && sub==0); };
00559 bool isParity() const { return (type==QTYPE_P && sub==0); };
00560
00561 bool isAdditive() const {
00562     if (type==QTYPE_U1) {
00563         if (sub) wblog(FL,
00564             "WRN invalid type %s (%d,%d)",STR_(this),type,sub);
00565         return 1;
00566     }
00567     return 0;
00568 };
00569
00570 int permitsOM() const {
00571     return (isAbelian() ? 0 : (sub<=1 ? 1 : 2));
00572 };
00573
00574 int permitsOM(unsigned r) const {
00575     return (isAbelian() || r<3 || (r==3 && sub<=1) ? 0 : 1);
00576 };
00577
00578 unsigned qlen() const {
00579     if (type && type<QTYPE_ASEP) { return 1; }
00580     if (type>=QTYPE_NUM_TYPES) wblog(FL,

```

```

00581         "ERR invalid type `%s' (%d)", STR_(this), type);
00582     return sub;
00583 };
00584
00585 unsigned qrank() const {
00586     if (type && type < QTYPE_ASEP) { return 0; }
00587     if (!type || type >= QTYPE_NUM_TYPES) wblog(FL,
00588         "ERR invalid type `%s' (%d)", STR_(this), type);
00589     if (!sub) wblog(FL,
00590         "ERR %s() got %s @ sub=%d", FCT, STR_(this), sub);
00591     return sub;
00592 };
00593
00594 bool isLargeD(unsigned d) const {
00595     if (type && type < QTYPE_ASEP) { return (d > 1); }
00596     switch (type) {
00597         case QTYPE_SUN: return (d > 12000 || d > (sub != 1 ? exp10(qlen()) : 20));
00598         case QTYPE_SON:
00599         case QTYPE_SEN:
00600         case QTYPE_SpN: return (d > 12000 || d > exp10(qlen()));
00601
00602         case QTYPE_UNKNOWN : return (d > 100);
00603         default:
00604             wblog(FL, "ERR invalid type `%s' (%d)", STR_(this), type);
00605     }
00606     return 1;
00607 };
00608
00609 bool isLargeD(const wvector<unsigned> &dd) const {
00610     for (unsigned i = 0; i < dd.len; ++i) {
00611         if (isLargeD(dd[i])) return 1;
00612     }
00613     return 0;
00614 };
00615
00616 unsigned getDimDef() const {
00617     unsigned d = 1;
00618     if (type && type > QTYPE_ASEP) {
00619         switch (type) {
00620             case QTYPE_SUN: d = sub + 1; return d;
00621             case QTYPE_SpN: d = 2 * sub; return d;
00622             case QTYPE_SON: d = 2 * sub + 1; return d;
00623             case QTYPE_SEN: d = 2 * sub; return d;
00624             default:
00625                 wblog(FL, "ERR invalid type `%s' (%d)", STR_(this), type);
00626         }
00627     }
00628     return d;
00629 };
00630
00631 template<class TQ>
00632 size_t wdim(const TQ *qs) const {
00633     unsigned d = 1;
00634     if (type && type > QTYPE_ASEP) {
00635         switch (type) {
00636             case QTYPE_SUN: d = DY::wdim_A(sub, qs); break;
00637             case QTYPE_SpN: d = DY::wdim_C(sub, qs); break;
00638             case QTYPE_SON: d = DY::wdim_B(sub, qs); break;
00639             case QTYPE_SEN: d = DY::wdim_D(sub, qs); break;
00640             default:
00641                 wblog(FL, "ERR invalid type `%s' (%d)", STR_(this), type);
00642         }
00643     }
00644     return d;
00645 };
00646
00647 template<class TQ>
00648 size_t wdim(const qset<TQ> &qs) const { return wdim(qs.data); }
00649
00650 unsigned maxDimLocal() const {
00651     unsigned d = 1;
00652     if (type) { d = maxDimLocal0(); if (d < 20) d = 20; }
00653     return d;
00654 };
00655
00656 unsigned maxDimLocal0() const {
00657     unsigned d = 2;
00658     if (type > QTYPE_ASEP) {
00659         d = pow(double(getDimDef()), 2.25 + double(sub) / 4.);
00660     }
00661     return d;
00662 };
00663
00664 void printDimInfo(const char *F = NULL, int L = 0) const {
00665     wblog(F, L, " * %s: sub=%d, dim_def=%d, dloc<=%d",
00666         STR_(this), sub, getDimDef(), maxDimLocal());
00667 }

```



```

00668     };
00669
00670     template <class T>
00671     unsigned qdim(const T* q) const;
00672
00673     template <class T>
00674     wbvector<unsigned>& QDim(
00675         const T* q, unsigned r, unsigned stride,
00676         wbvector<unsigned> &S) const;
00677
00678     bool isSelfDual() const {
00679         switch (type) {
00680             case QTYPE_P :
00681             case QTYPE_SpN :
00682                 return 1;
00683
00684             case QTYPE_SEN : return (qlen()%2 ? 0 : 1);
00685
00686             case QTYPE_U1 :
00687             case QTYPE_ZN :
00688                 return 0;
00689
00690             default:
00691                 wblog(FL, "ERR dual not yet defined for '%s'", STR_(this));
00692         }
00693         return 0;
00694     };
00695
00696     template<class TQ>
00697     char setDual(TQ* q) const {
00698         switch (type) {
00699             case QTYPE_P : { return 0; }
00700             case QTYPE_U1 : { q[0]=-q[0]; return 1; }
00701             case QTYPE_ZN : { q[0]=(sub-q[0])%sub; return 1; }
00702
00703             case QTYPE_SUN: {
00704                 unsigned n=qlen(); if (n<2) return 0;
00705                 else {
00706                     unsigned i=0, l=n-1; TQ x;
00707                     for (n/=2; i<n; ++i) { x=q[i]; q[i]=q[l-i]; q[l-i]=x; }
00708                     return 1;
00709                 }
00710             }
00711             case QTYPE_SpN:
00712             case QTYPE_SON: return 0;
00713
00714             case QTYPE_SEN: {
00715                 unsigned n=qlen();
00716                 if (n<3) wblog(FL, "ERR %s() got %s", FCT, STR_(this));
00717                 if ((n%2)==0) { return 0; }
00718                 else {
00719                     unsigned l=n-2; n-=1;
00720                     if (q[l]==q[n]) { return 0; }
00721                     else {
00722                         TQ x=q[l]; q[l]=q[n]; q[n]=x;
00723                         return 1;
00724                     }
00725                 }
00726             }
00727
00728             default:
00729                 wblog(FL, "ERR dual not yet defined for '%s'", STR_(this));
00730         }
00731         return 0;
00732     };
00733
00734     template<class TQ>
00735     unsigned getDual(const TQ* q0, TQ* q) const {
00736         unsigned n=qlen(); memcpy(q, q0, n*sizeof(TQ));
00737         setDual(q); return n;
00738     };
00739
00740     template<class TQ>
00741     bool isDual(const TQ *q, const TQ *x) const {
00742         switch (type) {
00743             case QTYPE_P : { return x[0]== q[0]; }
00744             case QTYPE_U1 : { return x[0]==-q[0]; }
00745             case QTYPE_ZN : { return x[0]==TQ((sub-q[0])%sub); }
00746
00747             case QTYPE_SUN: {
00748                 unsigned i=0, l=qlen();
00749                 if (!l) wblog(FL, "ERR %s() got n=%d", FCT, l);
00750                 for (--l; i<=l; ++i) { if (x[i]!=q[l-i]) return 0; }
00751                 return 1;
00752             }
00753             case QTYPE_SpN:
00754             case QTYPE_SON: {

```

```

00755         if (x!=q) {
00756             unsigned i=0, n=qlen();
00757             for (; i<n; ++i) { if (x[i]!=q[i]) return 0; }
00758         }; return 1;
00759     }
00760     case QTYPE_SEN: {
00761         unsigned n=qlen();
00762         if (n<3) wblog(FL,"ERR %s() got %s",FCT,STR_(this));
00763
00764         if (x==q) {
00765             return (n%2 ? x[n-2]==x[n-1] : 1);
00766         }
00767         else {
00768             unsigned i=0, l=n-(n%2 ? 2 : 0);
00769             for (; i<l; ++i) { if (x[i]!=q[i]) { return 0; } }
00770             if (i<n) {
00771                 if (x[i]!=q[i+1] || x[i+1]!=q[i]) { return 0; }
00772             }; return 1;
00773         }
00774     }
00775
00776     default:
00777         wblog(FL,"ERR dual not yet defined for '%s'",STR_(this));
00778     }
00779     return 0;
00780 };
00781
00782 QType& operator=(const char *s) { return init(FL,s); };
00783 QType& operator=(const QType &q) { return init(q); };
00784
00785 QType& operator=(QTYPE_SET t) {
00786     type=t; sub=0; if (t) validType(FL);
00787     return *this;
00788 };
00789
00790 bool operator<(const QType &q) const {
00791     if (type!=q.type)
00792         return (type<q.type);
00793     else return (sub<q.sub);
00794 };
00795
00796 char cmp(const QType &q) const {
00797     if (type<q.type) return -1;
00798     if (type>q.type) return +1;
00799     if (sub <q.sub ) return -1;
00800     if (sub >q.sub ) return +1; else return 0;
00801 };
00802
00803 bool operator> (const QType &q) const { return !((*this)<=q); };
00804 bool operator<= (const QType &q) const {
00805     if (type!=q.type) return (type<q.type);
00806     else return (sub<=q.sub);
00807 };
00808
00809 bool operator== (const QType &q) const {
00810     return (type==q.type && sub==q.sub);
00811 };
00812 bool operator!= (const QType &q) const {
00813     return (type!=q.type || sub!=q.sub);
00814 };
00815
00816 wstring toStr(const char tflag=0) const {
00817     wstring s(8);
00818     char e=0; s[0]=0;
00819
00820     if (type==QTYPE_U1 ) { s=(tflag ? QTYPE_STR[type] : "U(1)" ); } else
00821     if (type==QTYPE_P ) { s=(tflag ? QTYPE_STR[type] : "Parity"); } else
00822     if (!type && !sub) { s=(tflag ? "???" : "unknown"); }
00823
00824     if (s[0]) {
00825         if (sub) wblog(FL,
00826             "ERR invalid QType %s with sub=%d",s.data,sub);
00827         return s;
00828     }
00829
00830     if (sub<1 || sub>99) { e|=1; }
00831     else if (type==QTYPE_ZN) { if (sub<2) e|=2;
00832         if (tflag)
00833             { sprintf(s.data,s.len,"Z%d", sub); }
00834         else { sprintf(s.data,s.len,"Z(%d)",sub); }
00835     }
00836     else if (type==QTYPE_SUN) {
00837         if (tflag)
00838             { sprintf(s.data,s.len,"SU%d", sub+1); }
00839         else { sprintf(s.data,s.len,"SU(%d)",sub+1); }
00840     }
00841     else if (type==QTYPE_SpN) { if (sub<2) e|=4;

```

```

00842         if (!tflag)
00843             { snprintf(s.data,s.len,"Sp(%d)",2*sub); }
00844         else { snprintf(s.data,s.len,"Sp%d", 2*sub);
00845             if (tflag=='u') { s[1]='P'; }
00846         }
00847     }
00848     else if (type==QTYPE_SON) {
00849         if (!tflag)
00850             { snprintf(s.data,s.len,"SO(%d)",2*sub+1); }
00851         else { snprintf(s.data,s.len,"SO%d", 2*sub+1); }
00852     }
00853     else if (type==QTYPE_SEN) { if (sub<2) e|=8;
00854         if (!tflag)
00855             { snprintf(s.data,s.len,"SO(%d)",2*sub); }
00856         else { snprintf(s.data,s.len,"SO%d", 2*sub); }
00857     }
00858     else {
00859         if (type) wblog(FL,"ERR %s() invalid type=%d",FCT,type);
00860         s=(tflag ? "???" : "unknown");
00861     }
00862
00863     if (e) wblog(FL,
00864         "ERR invalid QType %s with sub=%d", type < QTYPE_NUM_TYPES ?
00865         QTYPE_STR[type] : "(type out of bounds)", sub
00866     );
00867
00868     return s;
00869 };
00870
00871 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00872 void add2MxStruct(mxArray *S, unsigned i) const;
00873
00874 mxArray* toMx(const char tflag=0) const {
00875     const size_t n=32; wbsting s(n);
00876     size_t l=snprintf(s.data,n,"%s",toStr(tflag).data);
00877     if (l>=n) wblog(FL,
00878         "WRN %s() QType too long (%s; %d)",FCT,s.data,l);
00879     return s.toMx();
00880 };
00881
00882 QTYPE_SET type;
00883 unsigned sub;
00884
00885 private:
00886
00887     int atoi(const char *s, int &k, unsigned n) {
00888         if (s[0]!='(') { return Wb::atoi(s,k,n); }
00889         else {
00890             unsigned l=strlen(++s);
00891             if (l && s[l-1]==')') {
00892                 char x[l]; memcpy(x,s,l); x[l-1]=0;
00893                 return Wb::atoi(x,k,n);
00894             }
00895         }
00896         return -6;
00897     }
00898 };
00899
00900 QType SU2("SU2");
00901
00902 bool operator!(const QType &q) {
00903     if (q.type == QTYPE_UNKNOWN) {
00904         if (q.sub) wblog(FL,
00905             "ERR %s() invalid QType %s",FCT,STR(q));
00906         return 1;
00907     }
00908     return 0;
00909 };
00910
00911 map<QType, time_t> load_cstore;
00912
00913 QType load_store_qtype(const char *F, int L, const Wb::matFile &M);
00914 void load_RCStore(const char *F, int L, const QVec &qvec);
00915
00916 class QVec : public wvector<QType> {
00917 public:
00918
00919     QVec() : wvector<QType>() {};
00920
00921     QVec(unsigned l, const QType* d0, char ref=0)
00922         : wvector<QType>(l,d0,ref) {};
00923
00924 #ifndef NOMEX
00925     QVec(const char *F, int L, const mxArray *a, unsigned d=0)
00926         : wvector<QType>() { init(F,L,a,d); };
00927
00928     QVec& init(const char *F, int L, const mxArray *a, unsigned d=0){

```

```

00929         if (!a) { init(); return *this; }
00930         if (mxIsChar(a)) { wbsting s(a); return init(F,L,s.data,d); }
00931         else wblog(FL,"ERR %s() invalid QVec (%s)",FCT,mxGetClassName(a));
00932         return *this;
00933     };
00934 #endif
00935
00936     QVec& init(unsigned l=0) {
00937         wbvector<QType>::init(l); return *this;
00938     };
00939
00940     QVec& init(const char *F, int L, const char *s, unsigned d=0);
00941
00942     unsigned Qlen(unsigned n=-1) const;
00943
00944     unsigned Qrank() const;
00945
00946     unsigned Qlen(wbvector<unsigned> &dd) const;
00947     unsigned Qlen(
00948         wbvector<unsigned> &dd,
00949         wbvector<unsigned> &dz
00950     ) const;
00951
00952     unsigned Qpos(wbvector<unsigned> &dc) const;
00953
00954     unsigned Qlenz() const {
00955         unsigned i=0, n=0;
00956         for (; i<len; ++i) { n+=(data[i].qlen() + data[i].qrank()); }
00957         return n;
00958     }
00959
00960     wbvector<gTQ>& getScalar(wbvector<gTQ> &qs, char zflag=0) {
00961         unsigned l=0, i=0;
00962         for (; i<len; ++i) {
00963             if (data[i].isParity()) {
00964                 if (data[i].qlen()!=1 || data[i].qrank()!=0) wblog(FL,
00965                     "ERR %s() %s @ %d/%d",
00966                     FCT,STR(data[i]),data[i].qlen(),data[i].qrank());
00967                 break;
00968             }
00969         }
00970         if (i==len) {
00971             qs.init(zflag ? Qlenz() : Qlen());
00972         }
00973         else if (!zflag) {
00974             wbvector<unsigned> dd; qs.init(Qlen(dd));
00975             for (i=0; i<len; ++i) {
00976                 if (data[i].isParity()) { qs[l]=1; }; l+=dd[i];
00977             }
00978         }
00979         else {
00980             wbvector<unsigned> dd,dz;
00981             unsigned l1=Qlen(dd,dz), l2=dz.sum(); qs.init(l1+l2);
00982
00983             for (i=0; i<dd.len; ++i) {
00984                 if (data[i].isParity()) { qs[l]=1; }
00985                 l+=dd[i]; if (zflag!='z') l+=dz[i];
00986             }
00987         }
00988         return qs;
00989     };
00990
00991     unsigned maxRank() const {
00992         unsigned r=0, i=0, q=0;
00993         for (; i<len; ++i) { q=data[i].qrank(); if (r<q) { r=q; } }
00994         return r;
00995     };
00996
00997     template <class TQ>
00998     unsigned QDim(const TQ *q) const;
00999
01000     template <class TQ>
01001     unsigned QDim(const TQ *qq, unsigned k) const;
01002
01003     template <class TQ>
01004     wbvector<unsigned>& QDim(
01005         const TQ *qq, wbvector<unsigned> &S
01006     ) const;
01007
01008     template <class TQ>
01009     wbvector<unsigned>& QDim(
01010         const TQ *qq, unsigned k, unsigned r, wbvector<unsigned> &S
01011     ) const;
01012
01013     template <class TQ>
01014     wbMatrix<TQ>& getQsub(
01015         const wbMatrix<TQ>& Q, const wbindex &I,

```

```

01016         wbMatrix<TQ>& QI,
01017         wbMatrix<TQ> *Qx=NULL
01018     ) const;
01019
01020     bool allU1() const {
01021         for (unsigned i=0; i<len; ++i) { if (!data[i].isU1()) return 0; }
01022         return 1;
01023     };
01024
01025     void ReduceU1() {
01026         if (len) {
01027             for (unsigned i=0; i<len; ++i) { if (!data[i].isU1())
01028                 return;
01029             }; init();
01030         }
01031     };
01032
01033     char allAbelian() const {
01034         char q=1;
01035         for (unsigned i=0; i<len; ++i) {
01036             if (!data[i].isAbelian()) { q=0; break; }
01037             if (data[i]!=QTYPE_U1) q|=2;
01038         }
01039         return q;
01040     };
01041
01042     bool anyAbelian() const {
01043         if (!len) return 1;
01044         for (unsigned i=0; i<len; ++i) {
01045             if (data[i].isAbelian()) return 1; }
01046         return 0;
01047     };
01048
01049     bool anyParity() const {
01050         if (len) {
01051             for (unsigned i=0; i<len; ++i) {
01052                 if (data[i].isParity()) return 1; }
01053             }
01054         return 0;
01055     };
01056
01057     bool allNonAbelian() const { return !anyAbelian(); };
01058
01059     char isNonAbelian() const {
01060         if (!len) return 0;
01061         for (unsigned i=0; i<len; ++i) {
01062             if (data[i].isNonAbelian()) { return data[i].permitsOM(); }
01063         }
01064         return 0;
01065     };
01066
01067     bool allAdditive() const {
01068         for (unsigned i=0; i<len; ++i) {
01069             if (!data[i].isAdditive()) return 0;
01070         }
01071         return 1;
01072     };
01073
01074     int permitsOM() const {
01075         int q=0; if (len) { unsigned i=0; int qi;
01076             for (; i<len; ++i) { if (q<(qi=data[i].permitsOM())) { q=qi; } }
01077         }
01078         return q;
01079     };
01080
01081     int permitsOM(unsigned r) const {
01082         for (unsigned i=0; i<len; ++i) { if (data[i].permitsOM(r)) return 1; }
01083         return 0;
01084     };
01085
01086     bool sameAs(const QVec &b, char lflag=1) const {
01087         if (len==b.len || !lflag) { return (*this)==b; }
01088         if (len && !b.len) return allU1();
01089         if (b.len && !len) return b.allU1();
01090         return 0;
01091     };
01092
01093     bool sameType (const QVec &b) const {
01094         if (len!=b.len) return 0;
01095         for (unsigned i=0; i<len; ++i)
01096             if (data[i].type!=b.data[i].type) return 0;
01097         return 1;
01098     };
01099
01100     wbstring toStr(const char vflag=0) const;
01101     int print_gset(
01102         const char *F, int L, const gTQ *qs, char *s, unsigned n) const;

```

```

01103 };
01104
01105 namespace DY {
01106
01107 template <class TQ>
01108 class weight_info {
01109
01110     public:
01111
01112         weight_info() : m(0) {};
01113
01114         weight_info(const weight_info& w) { init(w); };
01115
01116         weight_info(unsigned m_, unsigned n) : m(m_) {
01117             p.init(n); };
01118
01119         weight_info& init(unsigned m_, unsigned n) {
01120             m=m_; p.init(n); return *this;
01121         };
01122
01123         weight_info& init(const weight_info& w) {
01124             m=w.m; p.init(w.p);
01125             return *this;
01126         };
01127
01128         weight_info& operator=(const weight_info& w) { return init(w); }
01129
01130         weight_info& save2(weight_info& w) {
01131             m=w.m; w.m=0; w.p.save2(p);
01132             return *this;
01133         };
01134
01135         int m;
01136
01137         qset<TQ> p;
01138
01139     protected:
01140     private:
01141 };
01142
01143 template <class TQ>
01144 class Weights {
01145
01146     public:
01147         Weights() : n(0) {};
01148
01149         Weights(const qset<TQ> &qs) : n(qs.len), qm(qs) {
01150             if (!n) wblog(FL,"ERR %s() got empty qset",FCT);
01151             weight_info<TQ> &w=W[qs];
01152             w.init(1,n);
01153             X[qs]=&w;
01154         };
01155
01156         Weights& init(const qset<TQ> &qs) {
01157             if (!n=qs.len) wblog(FL,"ERR %s() got empty qset",FCT);
01158             qm.init(qs); W.clear(); X.clear();
01159             weight_info<TQ> &w=W[qs];
01160             w.init(1,n);
01161             X[qs]=&w;
01162             return *this;
01163         };
01164
01165         size_t dim() const {
01166             size_t mtot=0, merr=0;
01167             for (auto I=W.begin(); I!=W.end(); ++I) {
01168                 if (I->second.m>0) { mtot+=I->second.m; }
01169                 else { ++merr; }
01170             }
01171             if (merr) wblog(FL,
01172                 "WRN %s() got %d undetermined weight spaces",FCT,merr);
01173             return mtot;
01174         };
01175
01176         void print(
01177             const char *F, int L, const QType &q,
01178             char vflag=0) const;
01179
01180         mxArray* toMx() const;
01181
01182         unsigned n;
01183         qset<TQ> qm;
01184
01185         map <
01186             qset<TQ>,
01187             weight_info<TQ>
01188         > W;
01189

```

```

01190     map <
01191         qset<TQ>,
01192         const weight_info<TQ>*
01193     > X;
01194
01195 protected:
01196 private:
01197 };
01198
01199 class Symmetry {
01200
01201 public:
01202
01203     Symmetry(const QType &q_) : q(q_) { init(q); };
01204     Symmetry(const mxArray *a) { q.init(FL,a); init(q); };
01205
01206     Symmetry& init(const QType &q);
01207
01208     template <class TQ>
01209     size_t getWeightsFT(
01210         const char *F, int L, const qset<TQ> &qs,
01211         Weights<TQ> &R
01212     );
01213
01214     template <class TQ>
01215     int getInnerMultFT(
01216         const qset<TQ> &qs, const weight_info<TQ> &w,
01217         Weights<TQ> &R, char vflag=0
01218     );
01219
01220     template <class TQ>
01221     int Get(
01222         const qset<TQ> &qk, map < qset<TQ>, weight_info<TQ> > &W,
01223         char vflag=0
01224     );
01225
01226     template <class TQ>
01227     int wExpand(const char *F, int L, Weights<TQ> &R) const;
01228
01229     mxArray* toMx() const;
01230
01231     QType q;
01232     unsigned n;
01233
01234     wbararray<double> R;
01235
01236     wbMatrix<int> A;
01237
01238     wbvector<int> n2;
01239
01240     wbararray<int> M;
01241
01242 protected:
01243 private:
01244
01245 };
01246
01247 };
01248
01249 class QDir: public wbvector<char> {
01250
01251 public:
01252
01253     QDir() : wbvector<char>() {};
01254
01255     QDir(unsigned l, const char* d=NULL, char ref=0)
01256         : wbvector<char>(l,d,ref) {};
01257
01258     QDir(const iTags& it) { init(it); };
01259     QDir(const char *F, int L, const char *s) { init(F,L,s); };
01260     QDir(const char *s) { init(FL,s); };
01261
01262 #ifndef NOMEX
01263     QDir(const char *F, int L, const mxArray *a) { init(F,L,a); };
01264 #endif
01265
01266     QDir& init(unsigned l=0, const char* d=NULL, char ref=0) {
01267         wbvector<char>::init(l,d,ref);
01268         return *this;
01269     };
01270
01271     QDir& operator=(const char *s) { return init(FL,s); };
01272
01273     QDir& init(const char *F, int L, const char *s);
01274     QDir& init(const char *s) { return init(FL,s); };
01275     QDir& init(const iTags& it);
01276

```

```

01277 QDir& init(const char *F, int L,const mxArray *a) {
01278     wbsttring s(F,L,a); init(F,L,s.data);
01279     return *this;
01280 };
01281
01282 QDir& init_iout(const char *F, int L,
01283     unsigned r, unsigned iout
01284 );
01285
01286 unsigned isSorted() const {
01287     unsigned i=0, j=0;
01288
01289     #ifndef WB_SKIP_ASSERT
01290     for (; i<len; ++i) if (!data[i]) {
01291         wblog(FL,"ERR %s() invalid qdir '%s'",FCT,STR_(this));
01292     }; i=0;
01293     #endif
01294
01295     for (; i<len; ++i) { if (data[i]>0) break; };
01296     if (i==len) return 1;
01297     if (i) return 0;
01298
01299     for (; i<len; ++i) { if (data[i]<0) break; }
01300     if (i==len) return (len+1);
01301
01302     for (j=i+1; i<len; ++i) { if (data[i]>0) break; }
01303     if (i==len) return j;
01304
01305     return 0;
01306 };
01307
01308 bool operator==(const char *s) const;
01309 bool operator!=(const char *s) const { return !((*this)==s); };
01310
01311 bool operator==(const QDir &q) const {
01312     return this->wbvector<char>::operator==(q); };
01313 bool operator!=(const QDir &q) const { return !((*this)==q); };
01314
01315 bool operator<(const QDir &q) const {
01316     return wbvector<char>::operator<(q); };
01317
01318 QDir& Conj() {
01319     for (unsigned i=0; i<len; ++i) { data[i]=-data[i]; }
01320     return *this;
01321 };
01322
01323 char& Conj(const unsigned &i) {
01324     if (i>=len) wblog(FL,
01325         "ERR %s() index out of bounds (%d/%d)",FCT,i,len);
01326     return (data[i]=-data[i]);
01327 };
01328
01329 unsigned nconj() {
01330     unsigned i=0, n=0;
01331     for (; i<len; ++i) { if (data[i]<0) ++n; }
01332     return n;
01333 };
01334
01335 unsigned nconj(const ctrIdx &Ix) {
01336     unsigned n=0, i=0;
01337     wbvector<char> q(*this); const unsigned* ix=Ix.data;
01338
01339     for (; i<Ix.len; ++i) {
01340         if (ix[i]>=len) wblog(FL,"ERR %s() index out of bounds"
01341             " (%d/%d: %d/%d)",FCT,i,Ix.len, ix[i],len);
01342         q[ix[i]]=0;
01343     }
01344     if (Ix.conj)
01345         { for (i=0; i<len; ++i) { if (q[i]>0) ++n; }}
01346     else { for (i=0; i<len; ++i) { if (q[i]<0) ++n; }}
01347
01348     return n;
01349 };
01350
01351 int findl() const {
01352     int rval=-1024; unsigned k=0, i=0, o=0, ni=0, no=0;
01353
01354     for (; k<len; ++k) { if (data[k]>0) { ++ni; i=k; } else { ++no; o=k; }}
01355     if (no==1) { rval=-o; } else
01356     if (ni==1) { rval= i; }
01357
01358     return rval;
01359 };
01360
01361 wbsttring toStr() const;
01362 wbsttring toTag() const;
01363

```



```

01364     mxArray* toMx() const { return toStr().toMx(); };
01365
01366     protected:
01367     private:
01368
01369 };
01370
01371 class cgdStatus {
01372
01373     public:
01374
01375     cgdStatus() : t(CGD_UNKNOWN), ctime(0), mtime(0), ID(0) {
01376         memset(user,0,4); };
01377
01378     cgdStatus(CGD_TYPE t_) : cgdStatus() { t=t_; init_time(); };
01379
01380     cgdStatus(double t_) : cgdStatus() {
01381         if (double(t)!=t_ || t>=CGD_NUM_TYPES) wblog(FL,
01382             "ERR %s() invalid CData type %g",FCT,t_);
01383         t=CGD_TYPE(t_);
01384     };
01385
01386     cgdStatus(const char *F, int L, mxArray *a)
01387         : cgdStatus() { t=CGD_UNKNOWN; init(F,L,a); };
01388
01389     template <class TQ>
01390     cgdStatus(const CDATA_TQ& C) { init(C.cstat); }
01391
01392     cgdStatus& init() {
01393         t=CGD_UNKNOWN; ctime=0; mtime=0; ID=0;
01394         memset(user,0,2);
01395         return *this;
01396     };
01397
01398     cgdStatus& init(const CGD_TYPE &t_) {
01399         t=t_; init_time(); return *this;
01400     };
01401
01402     cgdStatus& init0(const CGD_TYPE &t_) {
01403         t=t_; ctime=0; mtime=0; ID=0;
01404         memset(user,0,2);
01405         return *this;
01406     };
01407
01408     cgdStatus& update_m() { init_time('m'); return *this; };
01409     cgdStatus& update_m(const CGD_TYPE &t_) {
01410         t=t_; init_time('m'); return *this;
01411     };
01412
01413     cgdStatus& operator=(const cgdStatus &b) { return init(b); };
01414
01415     cgdStatus& init(const cgdStatus &b) {
01416         if (ID && b.ID && (ID!=b.ID || ctime!=b.ctime)) wblog(FL,
01417             "ERR got initialized cstat with ID mismatch\n%s\n%s",
01418             STR2_(this,'V'),STR2(b,'V'));
01419         t=b.t; ctime=b.ctime; mtime=b.mtime; ID=b.ID;
01420
01421         user[0]=b.user[0];
01422         user[1]=b.user[1];
01423
01424         return *this;
01425     };
01426
01427     CGR_TYPE init(const char *F, int L, const mxArray *a);
01428
01429     int sameID(const cgdStatus &b) const {
01430         return (ID==b.ID && ctime==b.ctime);
01431     };
01432
01433     int cmp(const char *F, int L, const cgdStatus &b) const {
01434         if (ID!=b.ID || ctime!=b.ctime) { if (F) wblog(F,L,
01435             "ERR %s() got CData status mismatch!%N    %s%N<> %s",
01436             FCT, toStr('V').data, b.toStr('V').data);
01437             else return -11;
01438         }
01439         if (mtime<ctime || b.mtime<b.ctime) { if (F) wblog(F,L,
01440             "ERR %s() invalid mtime",FCT);
01441             else return -12;
01442         }
01443         return (mtime<b.mtime ? -1 : (mtime==b.mtime ? 0 : +1));
01444     };
01445
01446     bool olderThan(const cgdStatus &b) const { return (cmp(FL,b)<0); };
01447     bool newerThan(const cgdStatus &b) const { return (cmp(FL,b)>0); };
01448
01449     bool hasID(char lflag=0) const {
01450         if (!lflag)

```

```

01451         return (ctime && mtime && ID);
01452     else return (ctime || mtime || ID);
01453 };
01454
01455 bool operator==(const CGD_TYPE t_) const { return (t==t_); };
01456 bool operator!=(const CGD_TYPE t_) const { return (t!=t_); };
01457
01458 bool operator==(const cgdStatus &b) const {
01459     return (ctime==b.ctime && mtime==b.mtime && ID==b.ID);
01460 };
01461
01462 bool operator!=(const cgdStatus &b) const { return !((*this)==b); };
01463
01464 bool isSet() const { return (ctime || mtime || ID); };
01465 bool isEmpty() const {
01466     return (!ctime && !mtime && !ID && t==CGD_UNKNOWN);
01467 };
01468
01469 int invalid() const {
01470     int e=0;
01471     if ((ctime!=0) ^ (mtime!=0)) { e|=1; }
01472     if (t==CGD_EXPLICIT) {
01473         if (ctime || ID>99) { e|=8; }
01474     }
01475     else {
01476         if ((ctime!=0) ^ (ID!=0)) { e|=2; }
01477         if ((ctime!=0) ^ (t!=CGD_UNKNOWN)) { e|=4; }
01478     }
01479     return e;
01480 };
01481
01482 bool sameAs(const cgdStatus &b, char lflag=0) const {
01483     if (ctime!=b.ctime || ID!=b.ID) { return 0; }
01484
01485     if (lflag>2) {
01486         if (lflag=='l') lflag=1; else
01487         if (lflag=='L') lflag=2; else
01488         wblog(FL,"WRN %s() invalid lflag=%d (using 2='L')",FCT,lflag);
01489     }
01490     if (lflag<2 && mtime!=b.mtime) { return 0; }
01491     if (lflag<1 && t!=b.t) { return 0; }
01492     return 1;
01493 };
01494
01495 const char* tstr() const {
01496     if (t>=CGD_NUM_TYPES) wblog(FL,"ERR %s() "
01497         "cgd type out of bounds (%d/%d)",FCT,t,CGD_NUM_TYPES);
01498     return CGD_TYPE_STR[t];
01499 };
01500
01501 wbsting toStr(char vflag=0) const;
01502 wbsting u2Str(const char *istr=NULL) const;
01503
01504 mxArray* toMx() const;
01505 mxArray* toMx(CGR_TYPE rt) const;
01506
01507 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01508 mxArray* add2MxStruct(mxArray *S, unsigned k) const;
01509
01510 CGD_TYPE t;
01511
01512 double ctime;
01513 double mtime;
01514
01515 unsigned ID;
01516
01517 unsigned char user[4];
01518
01519 protected:
01520 private:
01521
01522     cgdStatus& init_time(char flag=0);
01523     wbsting to_vstr(double t) const;
01524
01525     void setID() {
01526         unsigned i=0, n=3;
01527         for (; i<n; ++i) {
01528             ID= (::rand() & ((1<<20)-1)); if (ID) break;
01529         }
01530         if (!ID) wblog(FL,"ERR %s() got ID=%d (%d/%d)",FCT,ID,i,n);
01531     };
01532 };
01533
01534 bool operator!=(const cgdStatus &q) {
01535     return (q.t==CGD_UNKNOWN && !q.ctime && !q.mtime && !q.ID);
01536 };
01537

```

```

01538 class cgrType {
01539 public:
01540     cgrType(CGR_TYPE t_=CGR_DEFAULT) : t(t_) {};
01541     cgrType(double t_) { init(FL,t_); };
01542     template<class T>
01543     cgrType& init(const char *F, int L, T t_) {
01544         t=CGR_TYPE(t_);
01545         if (T(t)!=t_ || t>=CGR_NUM_TYPES) wblog(F,L,
01546             "ERR %s() invalid CRef type %g",FCT,double(t_))
01547         );
01548         return *this;
01549     };
01550     cgrType& operator=(const cgrType &q) { t=q.t; return *this; };
01551     cgrType& operator=(const CGR_TYPE &t_) { t=t_; return *this; };
01552     bool operator< (const CGR_TYPE t_) const { return (t< t_); };
01553     bool operator<= (const CGR_TYPE t_) const { return (t<=t_); };
01554     bool operator> (const CGR_TYPE t_) const { return (t> t_); };
01555     bool operator>= (const CGR_TYPE t_) const { return (t>=t_); };
01556     bool operator== (const CGR_TYPE t_) const { return (t==t_); };
01557     bool operator!= (const CGR_TYPE t_) const { return (t!=t_); };
01558     bool operator< (const cgrType &q) const { return (q.t< t); };
01559     bool operator<= (const cgrType &q) const { return (q.t<=t); };
01560     bool operator> (const cgrType &q) const { return (q.t> t); };
01561     bool operator>= (const cgrType &q) const { return (q.t>=t); };
01562     bool operator== (const cgrType &q) const { return (q.t==t); };
01563     bool operator!= (const cgrType &q) const { return (q.t!=t); };
01564     const char* tostr() const {
01565         if (t>=CGR_NUM_TYPES) wblog(FL,"ERR %s() "
01566             "cgrType out of bounds (%d/%d)",FCT,t,CGR_NUM_TYPES);
01567         return CGR_TYPE_STR[t];
01568     };
01569     wstring toStr() const { return tostr(); }
01570     CGR_TYPE t;
01571 protected:
01572 private:
01573 };
01574 template <class TQ>
01575 class qset : public wbvector<TQ> {
01576 public:
01577     qset() : wbvector<TQ>() {};
01578     qset(unsigned l, const TQ* d0=NULL, char ref=0)
01579         : wbvector<TQ>(l,d0,ref) {};
01580     qset(const qset<TQ> &q)
01581         : wbvector<TQ>(q.len,q.data) {};
01582     qset(const QType &Q, const TQ *qq)
01583         : wbvector<TQ>() { init(Q,qq); };
01584     qset(const QVec &Q, const TQ *qq)
01585         : wbvector<TQ>() { init(Q,qq); };
01586     qset(const char *F, int L, const mxArray *a) : wbvector<TQ>() {
01587         if (mxIsChar(a)) { wstring s(a); init(F_L,s.data); }
01588         else init(F,L,a,NULL,1);
01589     };
01590     qset(const char *F, int L, const mxArray *a,
01591         const QVec *Q, char check_type=1) : wbvector<TQ>() {
01592         init(F,L,a,Q,check_type);
01593     };
01594     qset(const qset<TQ> &q1, const qset<TQ> &q2)
01595         : wbvector<TQ>() { init(q1,q2); };
01596     qset(const qset<TQ> &q1, const qset<TQ> &q2, const qset<TQ> &q3)
01597         : wbvector<TQ>() { init(q1,q2,q3); };
01598     qset(const qset<TQ> &q1, char op, const qset<TQ> &q2);
01599     qset& init(const qset<TQ> &q1) {
01600         wbvector<TQ>::init(q1);
01601         return *this;
01602     };

```

```

01625     };
01626
01627     qset& init(const char *F, int L, const char *s, QType *t=NULL);
01628     qset& init(
01629         const char *F, int L, const mxArray *a,
01630         const QVec *Q=NULL, char check_type=1
01631     ){
01632         if (mxIsChar(a)) { wbsttring s(a); init(F_L,s.data); }
01633         else wbvector<TQ>::init(F,L,a,NULL,check_type);
01634
01635         if (Q) {
01636             unsigned n=Q->Qlen();
01637             if ((this->len)%n) wblog(FL,
01638                 "ERR %s() got invalid qset.len=%d @ %d",FCT,this->len,n
01639             );
01640         }
01641         return *this;
01642     };
01643
01644     qset& init(const QType &Q, const TQ *qq) {
01645         wbvector<TQ>::init(Q.Qlen(),qq);
01646         return *this;
01647     };
01648
01649     qset& init(const QVec &Q, const TQ *qq) {
01650         init(Q.Qlen(),qq);
01651         return *this;
01652     };
01653
01654     qset& init(const qset<TQ> &q1, const qset<TQ> &q2) {
01655         init(q1.len, q1.data, q2.len, q2.data);
01656         return *this;
01657     };
01658
01659     qset& init(const qset<TQ> &q1, const qset<TQ> &q2, const qset<TQ> &q3) {
01660         init(q1.len, q1.data, q2.len, q2.data, q3.len, q3.data);
01661         return *this;
01662     };
01663
01664     qset& init() { wbvector<TQ>::init(); return *this; };
01665
01666     qset& init(unsigned l, const TQ *d0=NULL) {
01667         wbvector<TQ>::RENEW(l,d0);
01668         return *this;
01669     };
01670
01671     qset& init(
01672         unsigned l1, const TQ *d1,
01673         unsigned l2, const TQ *d2
01674     );
01675
01676     qset& init(
01677         unsigned l1, const TQ* d1,
01678         unsigned l2, const TQ *d2,
01679         unsigned l3, const TQ *d3
01680     );
01681
01682     qset<TQ>& init(const TQ q1) { return init(1U,&q1); }
01683
01684     qset<TQ>& init(const TQ q1, const TQ q2) {
01685         return init(1,&q1,1,&q2);
01686     };
01687     qset<TQ>& init(const TQ q1, const TQ q2, const TQ q3) {
01688         return init(1,&q1,1,&q2,1,&q3);
01689     };
01690
01691     qset<TQ>& initStride(
01692         const TQ* d_, unsigned n, unsigned r, unsigned QDIM
01693     ){
01694         init(n*r); if (d_ && this->len) {
01695             Wb::cpyStride(this->data,d_,n,NULL,r,-1,QDIM);
01696         }
01697         return *this;
01698     };
01699
01700     bool operator<(const qset<TQ> &b) const;
01701
01702     int checkNQs(const QVec &qq, const char *F=NULL, int L=0);
01703
01704     bool isConsistent(
01705         const QVec &qq, int rank=-1, const char *F=NULL, int L=0) const;
01706
01707     unsigned QDim(const QVec &q, int rank=-1) const;
01708
01709     qset<TQ>& times(TQ fac, qset<TQ> &x) const {
01710         x=*this; x*=fac; return x;
01711     };

```

```

01712
01713     wbststring toStr() const { return wbvector<TQ>::toStr(); };
01714
01715     wbststring toStr(const QType &t __attribute__((unused))) const {
01716         if (CG::gotQAlpha(this->data, this->len)) {
01717             wbststring sout(this->len+1);
01718             if (CG::qset2cstr(this->data, sout.data, this->len)) wblog(FL,
01719                 "ERR %s() failed to obtain compact qset string", FCT);
01720             sout.data[this->len]=0;
01721             return sout;
01722         }
01723         return wbvector<TQ>::toStr();
01724     };
01725
01726 };
01727
01728 template <class TQ>
01729 bool qset<TQ>::operator<(const qset<TQ> &b) const {
01730
01731     if (!this->len || !b.len) wblog(FL,
01732         "ERR %s() got empty object (%d/%d)", FCT, this->len, b.len);
01733
01734     return wbvector<TQ>::operator<(b);
01735 };
01736
01737 template <>
01738 bool qset<double>::operator<(const qset<double> &b) const {
01739
01740     if (!len || !b.len) wblog(FL,
01741         "ERR %s() got empty object (%d/%d)", FCT, len, b.len);
01742     if (len!=b.len) wblog(FL,
01743         "ERR %s() got length mismatch (%d/%d)", FCT, len, b.len);
01744
01745     double *d=b.data;
01746     for (unsigned i=0; i<len; i++) {
01747         if (float(data[i])<float(d[i])) return 1;
01748         if (float(data[i])>float(d[i])) return 0;
01749     }
01750     return 0;
01751 };
01752
01753 template <>
01754 bool qset<float>::operator<(const qset<float> &b) const {
01755     wblog(FL, "ERR %s() double precision preferred over '%s' (0x%lX)",
01756         TSTR(float), &b);
01757     return 0;
01758 };
01759
01760 template <class TQ>
01761 class QSet {
01762
01763 public:
01764
01765     QSet() : t(QTYPE_UNKNOWN) {};
01766
01767     QSet(const QSet &B) { t=B.t; qs=B.qs; qdir=B.qdir; };
01768
01769     QSet(const QType &t_, const qset<TQ> &qs_,
01770         unsigned iout=0,
01771         char isref=0
01772     ){
01773         init(t, qs, iout, isref);
01774     };
01775
01776     QSet(const QType &t_, const iTags &it, unsigned r=-1){
01777         if (int(r)<0) { r=it.len; } else
01778             if (r!=it.len) wblog(FL, "ERR %s() rank mismatch (%d/%d)", FCT, r, it.len);
01779         t=t_; qdir.init(it); qs.init(r*t.qlen());
01780     };
01781
01782     QSet(const CRef<TQ> &A) { init(A); };
01783
01784     QSet(const char *F, int L, const mxArray *a) { init(F, L, a); };
01785     QSet(const mxArray *a) { init(0, 0, a); };
01786
01787     QSet(const char *F, int L,
01788         const CRef<TQ> &A, const ctrIdx &ica,
01789         const CRef<TQ> &B, const ctrIdx &icb, wbpemr *P=NULL
01790     ){ init(F, L, A, ica, B, icb, P); };
01791
01792     QSet(const char *F, int L, const char *s) { init_str(F, L, s); };
01793
01794     QSet& init(const CRef<TQ> &A) {
01795         if (!A.cgb) init();
01796         else { init(*A.cgb);
01797             if (A.gotPerm()) Permute(A.cgb);
01798             if (A.gotConj()) Conj();

```

```

01799         }
01800         return *this;
01801     };
01802
01803     QSet& init(const char *F, int L, const mxArray *a, unsigned k=0);
01804
01805     QSet& init(const QType t_ = QTYPE_UNKNOWN) { t=QTYPE_UNKNOWN;
01806         t=t_; if (qs.len ) qs.init();
01807             if (qdir.len) qdir.init();
01808         return *this;
01809     };
01810
01811     QSet& init(const QSet &B) {
01812         t=B.t; qs=B.qs; qdir=B.qdir;
01813         return *this;
01814     };
01815
01816     QSet& init2ref(const QSet &B) {
01817         t=B.t; qs.init2ref(B.qs); qdir.init2ref(B.qdir);
01818         return *this;
01819     };
01820
01821     QSet& init(const QType &t_, const TQ* q, const char *qd);
01822
01823     QSet& init(
01824         const QType &t_, const qset<gTQ>&qs_,
01825         unsigned iout=0,
01826         char isref=0
01827     );
01828
01829     QSet& init(
01830         const QType t_, const TQ* qs_, unsigned r, unsigned N,
01831         unsigned iout=0
01832     );
01833
01834     QSet& init(
01835         const QType t_, const TQ* qs_, unsigned r, unsigned N,
01836         const iTags &it
01837     );
01838
01839     QSet& init1J(const QType &t_, const TQ *q, char iflag=0) {
01840         unsigned m=t_.qlen();
01841         t=t_; qdir.init2val(2, iflag ? -1 : +1);
01842
01843         qs.init(2*m); memcpy(qs.data, q, m*sizeof(TQ));
01844         t.getDual(q, qs.data+m);
01845
01846         return *this;
01847     };
01848
01849     QSet& init2(const QType &t_, const TQ *q=NULL) {
01850         unsigned m=t_.qlen();
01851         t=t_; qdir.init_iout(FL,2,2); qs.init(2*m);
01852         if (q) { unsigned l=m*sizeof(TQ);
01853             memcpy(qs.data, q, l);
01854             memcpy(qs.data+m, q, l);
01855         }
01856         return *this;
01857     };
01858
01859     QSet& init2(const QType &t_,
01860         const qset<TQ> &q1, const qset<TQ> &q2, const char *qds="++") {
01861
01862         unsigned m=t_.qlen(), l=m*sizeof(TQ);
01863         if (q1.len<m || q2.len<m) wblog(FL,
01864             "ERR %s() invalid qset length (%s: %d,%d/%d)",
01865             FCT,STR(t_),q1.len,q2.len,m
01866         );
01867         t=t_; qdir=qds; qs.init(2*m);
01868         memcpy(qs.data, q1.data, l);
01869         memcpy(qs.data+m, q2.data, l);
01870         return *this;
01871     };
01872
01873     QSet& init1(const QType &t_, TQ *q=NULL) {
01874         t=t_; qs.init(t_.qlen(),q_);
01875         qdir.init2val(1,+1);
01876         return *this;
01877     };
01878
01879     QSet& init3(const QType &t_) {
01880         t=t_; qs.init(3*t_.qlen());
01881         qdir.init_iout(FL,3,3); return *this;
01882     };
01883
01884     QSet& init3(const QType &t_,
01885         const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J

```

```

01886     ){
01887         unsigned n=t_.qlen();
01888         if (J1.len!=n || J1.len!=J2.len || J1.len!=J.len) {
01889             if (J1.len!=n || J2.len!=n || J.len!=n) wblog(FL,
01890                 "ERR %s() severe length inconsistency [%d %d %d; %d]", FCT,
01891                 J1.len,J2.len,J.len,n
01892             );
01893         }
01894         qs.init(J1,J2,J); t=t_;
01895         qdir.init_iout(FL,3,3);
01896
01897         return *this;
01898     };
01899
01900     QSet& init3(const QType &t_,
01901         const TQ &J1, const TQ &J2, const TQ &J
01902     ){
01903         unsigned n=t_.qlen();
01904         if (n!=1) wblog(FL,
01905             "ERR %s() got QType %s (%d/1)",FCT,STR(t_),n);
01906         qs.init(3U); qs[0]=J1; qs[1]=J2; qs[2]=J;
01907         qdir.init_iout(FL,3,3); t=t_;
01908         return *this;
01909     };
01910
01911     QSet& init(const char *F, int L,
01912         const CRef<TQ> &A, const ctrIdx &ica,
01913         const CRef<TQ> &B, const ctrIdx &icb, wbperm *Pcgd=NULL
01914     );
01915
01916     QSet& init_str(const char *F, int L, const char *s);
01917
01918     QSet& Conj() { qdir.Conj(); return *this; };
01919     QSet& Conj(unsigned i);
01920
01921     unsigned rank(const char *F=NULL, int L=0) const {
01922         unsigned n=t_.qlen();
01923         if (n*qdir.len!=qs.len) wblog(F_L,
01924             "ERR %s() invalid qs (%d*d == %d?)",FCT,n,qdir.len,qs.len);
01925         return qdir.len;
01926     };
01927
01928     QSet& permute(QSet &B, const wbperm &P, char iflag=0) const;
01929
01930     QSet& Permute(const wbperm &P, char iflag=0) {
01931         QSet<TQ> X; permute(X,P,iflag);
01932         return X.save2(*this);
01933     };
01934
01935     void swap(QSet &X) {
01936         if (this!=&X) {
01937             SWAP(t,X.t); qdir.swap(X.qdir); qs.swap(X.qs);
01938         }
01939     };
01940
01941     QSet& save2(QSet &X) {
01942         if (this!=&X) { this->swap(X); init(); }
01943         return X;
01944     };
01945
01946     QSet& operator=(const QSet &B) { return init(B); };
01947
01948     bool operator==(const QSet &B) const {
01949         return (t==B.t && qs==B.qs && qdir==B.qdir);
01950     };
01951
01952     bool operator!=(const QSet &B) const {
01953         return (t!=B.t || qs!=B.qs || qdir!=B.qdir);
01954     };
01955
01956     char cmp(const QSet<TQ> &B) const {
01957         char i=0;
01958         if ((i=t.cmp(B.t))) { return i; }
01959         if ((i=qdir.cmp(B.qdir))) { return i; }
01960         return (i=qs.cmp(B.qs));
01961     };
01962
01963     bool operator<(const QSet<TQ> &B) const { return cmp(B)<0; }
01964
01965     explicit operator bool() const { return !isEmpty(); }
01966     bool operator! () const { return isEmpty(); }
01967
01968     bool isEmpty(char check=3) const {
01969         if (check>3) {
01970             if (check=='c' ) { check=2; } else
01971             if (check=='C' || check=='!') { check=3; } else
01972                 wblog(FL,"ERR %s() invalid flag=%s",FCT,cSTR(check));

```

```

01973     }
01974
01975     if (qs.len || qdir.len) {
01976         if ( ((check&1) && t==QTYPE_UNKNOWN) ||
01977             ((check&2) && qs.len!=qdir.len*t.qlen()) ) {
01978             wblog(FL,"ERR %s() mismatch itag / QSet %s (%d*%d / %d)",
01979                 FCT,STR_(this), qdir.len, t.qlen(), qs.len);
01980         }
01981         return 0;
01982     }
01983     return (t==QTYPE_UNKNOWN ? 1 : 0);
01984 };
01985
01986 bool gotRCData() const;
01987
01988 int isZero() const;
01989
01990 unsigned isStd3() const;
01991
01992 unsigned checkStd3() const;
01993
01994 int init2opt3(
01995     const QSet<TQ> &Q, wbperm &P, wbvector<char> &cflags);
01996
01997 QSet& Convert(const char *s, wbvector<char> &cflags);
01998
01999 wbstring sizeStr() const;
02000
02001 bool isScalar() const;
02002 bool isJSymbol() const;
02003
02004 bool isAbelian() const { return t.isAbelian(); };
02005 int permitsOM() const { return t.permitsOM(rank(FL)); };
02006
02007 int checkQ_abelian(const char *F, int L) const;
02008
02009 int checkQ_SU2(const char *F, int L) const;
02010
02011 QSet& Sort(wbperm *cgp=NULL, char *conj=NULL, char iflag=0);
02012 bool isSorted() const;
02013
02014 size_t memSize() const {
02015     return ( sizeof(t)
02016         + sizeof(qs) ) + qs .len*sizeof(TQ)
02017         + sizeof(qdir) + qdir.len*sizeof(char)
02018     );
02019 };
02020
02021 wbstring QStrS(const wbperm *cgp=NULL, char sep='|') const;
02022 wbstring QStr() const;
02023
02024 wbstring toStr() const { return toStr(NULL); };
02025 wbstring toStr(const char *istr) const;
02026
02027 wbstring toTag() const;
02028
02029 mxArray* toMx() const;
02030
02031 QType t;
02032 qset<TQ> qs;
02033
02034 QDir qdir;
02035
02036 protected:
02037 private:
02038
02039     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
02040     void add2MxStruct(mxArray *S, unsigned i) const;
02041 };
02042
02043 template <class TQ>
02044 unsigned QSet<TQ>::isStd3() const {
02045
02046     unsigned q=0; { if (qdir!="++-") return q; }
02047
02048     unsigned m=t.qlen(), l=t.maxDimLocal(),
02049         d1=t.qdim(qs.data),
02050         d2=t.qdim(qs.data+ m),
02051         d3=t.qdim(qs.data+2*m), dmax=MAX(1U<<6,l); dmax*=dmax;
02052
02053     if (qdir.len*m!=qs.len) wblog(FL,
02054         "ERR %s %d*%d =? %d",FCT,STR_(this),qdir.len,m,qs.len);
02055
02056     q=(d1*d2<=dmax || d1<=1 || d2<=1) ? 1 : 2;
02057
02058     if (d1>2*d3 && (d2*d3<=dmax || d2<=1 || d3<=1)) q|=4;
02059     if (d2>2*d3 && (d1*d3<=dmax || d1<=1 || d3<=1)) q|=8;

```



```

02060
02061     return q;
02062 };
02063
02064 template <class TQ>
02065 unsigned QSet<TQ>::checkStd3() const {
02066
02067     unsigned q=0; { if (qdir.len!=3) return q; }
02068     unsigned m=t.qlen(), l=t.maxDimLocal(),
02069     d1=t.qdim(qs.data),
02070     d2=t.qdim(qs.data+m),
02071     d3=t.qdim(qs.data+2*m), dmax=MAX(1U<<6,1); dmax*=dmax;
02072
02073     if (qdir.len*m!=qs.len) wblog(FL,
02074         "ERR %s having %d*d=? %d",FCT,STR_(this),qdir.len,m,qs.len);
02075
02076     if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<0);
02077
02078     if (!q) { dmax=1U<<16;
02079         if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<1);
02080     }
02081     if (!q) { dmax=1U<<19;
02082         if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<2);
02083     }
02084
02085     if (d1<=1) q|=(1<<3);
02086     if (d2<=1) q|=(1<<4);
02087     if (d3<=1) q|=(1<<5);
02088
02089     if (d1<d2) {
02090         if (d2<=d3) q|=(3<<6);
02091         else q|=(2<<6);
02092     } else {
02093         if (d1<=d3) q|=(3<<6);
02094         else q|=(1<<6);
02095     }
02096
02097     return q;
02098 };
02099
02100 template <class TQ>
02101 int QSet<TQ>::init2opt3(
02102     const QSet<TQ> &Q, wbperm &iP, wbvector<char> &cflags) {
02103
02104     unsigned i3=Q.checkStd3(), w=(i3>6);
02105     unsigned m=Q.t.qlen();
02106     wbperm P;
02107
02108     if (!(i3&63)) { return -1; }
02109
02110     if (!w || w>3 || m*Q.qdir.len!=Q.qs.len) wblog(FL,
02111         "ERR %s() got %s\n(%d -> %d)",FCT,STR(Q),BITS(i3),BITS(w));
02112     if (w==1) P.init("231"); else
02113     if (w==2) P.init("132"); else
02114     if (w==3) P.init();
02115
02116     Q.permute(*this,P);
02117     Convert("++-",cflags);
02118
02119     if (Wb::cmpRange(qs.data,qs.data+m,m)>0) {
02120         wbperm p2="213"; Permute(p2);
02121         P.Permute(p2); cflags.Permute(p2);
02122     }
02123
02124     P.invert(iP);
02125
02126     return 0;
02127 };
02128
02129 template <class TQ>
02130 QSet<TQ>& QSet<TQ>::Convert(const char *s, wbvector<char> &cflags) {
02131
02132     unsigned i=0, r=(!s || !s[0] ? 0 : strlen(s));
02133
02134     if (r!=qdir.len) wblog(FL,
02135         "ERR %s() size mismatch %s [0/%d]",FCT,STR_(this),qdir.len);
02136     if (!s || !s[0]) { cflags.init(); return *this; }
02137
02138     cflags.init(r);
02139
02140     for (; i<r; ++i) {
02141         if ((s[i]!='+' && qdir[i]<0) || (s[i]!='-' && qdir[i]>0)) {
02142             Conj(i); cflags[i]=1;
02143         }
02144         else if (s[i]!='+' && s[i]!='-')
02145             wblog(FL,"ERR %s() invalid qdir='%s'",FCT,s);
02146     }

```

```

02147
02148     return *this;
02149 };
02150
02151 template <class TQ>
02152 QSet<TQ>& QSet<TQ>::Conj(unsigned i) {
02153     unsigned m=t.qlen();
02154     if (i>=qdir.len || !qdir[i]) wblog(FL,"ERR %s() "
02155     "index out of bounds %s (%d/%d)",FCT,STR_(this),i+1,qdir.len);
02156     if (m*qdir.len!=qs.len) wblog(FL,"ERR %s() "
02157     "invalid %s (%d*%d=? %d)",FCT,STR_(this),m,qdir.len,qs.len);
02158
02159     qdir[i]=--qdir[i];
02160     t.setDual(qs.data+i*m);
02161
02162     return *this;
02163 };
02164
02165
02166 template <class TQ>
02167 wstring QSet<TQ>::sizeStr() const {
02168     unsigned m=t.qlen(), r=qdir.len;
02169
02170     if (r*m!=qs.len) wblog(FL,
02171     "ERR %s: %d != %d*%d",STR_(this),qs.len,r,m);
02172     try {
02173         if (qs.len) { wvector<size_t> dd(r);
02174             for (unsigned i=0; i<r; ++i) { dd[i]=t.qdim(qs.data+i*m); }
02175             return SSTR(dd);
02176         }
02177     }
02178     catch (...) {
02179         wblog(FL,"ERR %s() failed to obtain size(Q)\n%s",FCT,STR_(this));
02180     }
02181     return "";
02182 };
02183
02184 template <class TQ>
02185 class QHash {
02186
02187     public:
02188
02189     size_t operator()(const QSet<TQ> &S, const char *istr=0) const {
02190
02191         unsigned i, j=1, r=S.qdir.len, nq=S.qs.len,
02192             l=1+(1+nq+r)/sizeof(long),
02193             lb=l*sizeof(long);
02194         unsigned long h=5381U, x[l];
02195         char *s=(char*)x;
02196
02197         const TQ *qs=S.qs.data;
02198
02199         if ((!r && nq) || (r && nq&r)) wblog(FL,
02200         "ERR %s() got QSet inconsistency (%d/%d)",FCT,nq,r);
02201         x[l-1]=0;
02202
02203         s[0]=S.t.type;
02204         s[1]=S.t.sub;
02205
02206         for (i=0; i<nq; ++i) {
02207             s[++j]=char(qs[i]);
02208         }; ++j;
02209
02210         if ((j+r)>lb) {
02211             wblog(FL,"ERR %s() string/id out of bounds (%d/(%d*%d))",
02212             FCT,j,l,sizeof(long));
02213         }
02214
02215         memcpy(s+j,S.qdir.data,r);
02216
02217         if (istr && istr[0]) {
02218             unsigned n=strlen(istr); if (n>lb) {
02219                 wblog(FL,"ERR %s() istr `s' out of bounds (%d/%d*%d)",
02220                 FCT,istr,n,l,sizeof(long));
02221             }
02222             for (j=lb-n, i=0; i<n; ++i) { s[j+i] ^= istr[i]; }
02223         }
02224
02225         for (i=0; i<l; ++i) { h ^= ((h<<6) + (h>>2)) + x[i]; }
02226
02227         return h;
02228     };
02229 };
02230
02231 template <class TD>
02232 class cdata : public wbspararray<TD> {
02233

```

```

02234 public:
02235
02236     cdata& init(SPIDX_T l=0) {
02237         wbsparray<TD>::init(l);
02238         return *this;
02239     };
02240
02241     cdata& init(
02242         SPIDX_T D1, SPIDX_T D2, SPIDX_T D,
02243         unsigned M=1
02244     ){
02245         if (M==1)
02246             wbsparray<TD>::init(D1,D2,D);
02247         else wbsparray<TD>::init(D1,D2,D,M);
02248         return *this;
02249     };
02250
02251     cdata& init(const wbvector<SPIDX_T> &S) {
02252         wbsparray<TD>::init(S); return *this;
02253     };
02254
02255     cdata& init(const cdata &C) {
02256         wbsparray<TD>::init(C);
02257         return *this;
02258     };
02259
02260     template <class TQ>
02261     cdata& init(const CRef<TQ> &R, unsigned l, char full=1);
02262
02263     template <class T2>
02264     cdata& init(const wbsparray<T2> &C) {
02265         wbsparray<TD>::init(C);
02266         return *this;
02267     };
02268
02269     cdata& init(
02270         const char *F, int L, const mxArray *a, unsigned k,
02271         const QType &q, unsigned r=-1
02272     );
02273
02274     cdata& operator=(const cdata &C) {
02275         wbsparray<TD>::init(C);
02276         return *this;
02277     };
02278
02279     cdata& initScalar(double x=1, unsigned r=-1) {
02280         wbsparray<TD>::initScalar(x,r);
02281         return *this;
02282     };
02283
02284     cdata& initIdentity(const QType &q, SPIDX_T d, TD dval=1);
02285     cdata& initIdentity(const wbvector<SPIDX_T> &S);
02286
02287     bool isScalar() const {
02288         return wbsparray<TD>::isScalar(FL,'z');
02289     };
02290
02291     unsigned len() const { return this->SIZE.len; };
02292
02293     unsigned getOM(
02294         const char *F=NULL, int L=0, const char *istr=NULL) const;
02295
02296     cdata& Conj() { return *this; };
02297
02298     cdata& permute(cdata &B, const wbperm &P0, char iflag=0) const;
02299     cdata& Permute(const wbperm &P0, char iflag=0) {
02300         cdata X; this->save2(X);
02301         return X.permute(*this,P0,iflag);
02302     };
02303
02304     cdata& Kron(const cdata &B){
02305         wbsparray<TD>::Kron(B,'N','N','k');
02306         return *this;
02307     };
02308
02309     TD contract(
02310         const char *F, int L, const ctrIdx &ica,
02311         const cdata &B, const ctrIdx &icb, cdata &C,
02312         cdata *Cx=NULL, wbperm *P=NULL, char normalize=1
02313     ) const;
02314
02315     double SkipTiny(const char *F=NULL, int L=0);
02316
02317     TD NormSignC(
02318         const char *F=NULL, int L=0, unsigned m=0,
02319         TD eps=CG_EPS1,
02320         TD eps2=CG_EPS2

```

```

02321     );
02322
02323     void info() const {
02324         printf("\n data: %s\n",this->sizeStr().data);
02325     };
02326
02327 protected:
02328 private:
02329 };
02330
02331 template <class TQ, class TD>
02332 class CData : public QSet<TQ> {
02333 public:
02334
02335     CData() : cstat(CGD_UNKNOWN) { init(); };
02336
02337     CData(const CData &C)
02338         : QSet<TQ>((QSet<TQ>&)C), cgd(C.cgd), cstat(C.cstat) {};
02339
02340     explicit CData(const QSet<TQ> &Q) : QSet<TQ>(Q) {};
02341
02342     explicit CData(const CRef<TQ> &R, unsigned l) { init(R,l); }
02343
02344     CData& init() {
02345         this->t.init(); this->qs.init(); this->qdir.init();
02346         cgd.init(); cstat.init();
02347         return *this;
02348     };
02349
02350     CData& init(const QSet<TQ> &Q, bool full=true) {
02351         this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
02352         if (full) {
02353             cgd.init(); cstat.init();
02354         }
02355         return *this;
02356     };
02357
02358     CData& init(const CRef<TQ> &R, unsigned l);
02359
02360     CData& init(const CData &B) {
02361         this->t=B.t; this->qs=B.qs; this->qdir=B.qdir;
02362         cgd=B.cgd; cstat=B.cstat;
02363         return *this;
02364     };
02365
02366     template <class T2>
02367     CData& init(const CData<TQ,T2> &B) {
02368         this->t=B.t; this->qs.init(B.qs); this->qdir=B.qdir;
02369         cgd.wbsparray<TD>::init(B.cgd); cstat=B.cstat;
02370         return *this;
02371     };
02372
02373     CData& operator=(const CData &B) { return init(B); }
02374
02375     CData& operator=(const QSet<TQ> &Q) { return init(Q,0); };
02376
02377     CData& save2(CData &B);
02378
02379     CData& initAbelian(const QSet<TQ> &Q) {
02380         this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
02381         cstat.init(CGD_ABELIAN); cgd.init();
02382         return *this;
02383     };
02384
02385     CData& initScalar(const QSet<TQ> &Q) {
02386         if (!Q.t.isAbelian() && Q.qs.norm2()) {
02387             wblog(FL,"ERR %s() got QSet %s",FCT,STR(Q)); }
02388         this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
02389         cstat.init(CGD_EXPLICIT); cgd.initScalar(1,Q.qdir.len);
02390         return *this;
02391     };
02392
02393     CData& init3(const QType &t_,
02394                 SPIDX_T D1, SPIDX_T D2, SPIDX_T D,
02395                 unsigned M=1
02396     ){
02397         QSet<TQ>::init3(t_);
02398         cgd.init(D1,D2,D, M); cstat=CGD_UNKNOWN;
02399         return *this;
02400     };
02401
02402     CData& init3(const QType &t_,
02403                 const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J,
02404                 unsigned M=1
02405     ){
02406         SPIDX_T D1,D2,D;
02407

```

```

02408         QSet<TQ>::init3(t_, J1, J2, J);
02409         cstat.init(CGD_UNKNOWN);
02410
02411         D1=t_.qdim(J1.data);
02412         D2=t_.qdim(J2.data);
02413         D =t_.qdim(J .data); cgd.init(D1,D2,D,M);
02414
02415         return *this;
02416     };
02417
02418     CData& init3_abelian(const QType &t_,
02419         const TQ &J1, const TQ &J2, const TQ &J
02420     ){
02421         QSet<TQ>::init3(t_, J1, J2, J);
02422         cgd.init();
02423         cstat.init(CGD_ABELIAN);
02424         return *this;
02425     };
02426
02427     int RefInit(
02428         const char *F, int L, const CData<TQ,TD> &B,
02429         char lflag=0, char bare=0);
02430
02431     CData& RefInit_auxtr(
02432         const char *F, int L,
02433         const wbvector<SPIDX_T> &S, const wbvector<RTD> &cgt
02434     );
02435
02436     int Reduce2Ref(const char *F=0, int L=0, char force=0);
02437     int LoadRef (const char *F=0, int L=0, char force=1);
02438
02439     int Load_CRef(
02440         const char *F, int L, CData &Cr, const char *cgd_file) const;
02441
02442     CData& initX3(
02443         const char *F, int L, const QSet<TQ> &Q,
02444         wbvector<char> &cflags, wbperm &P,
02445         unsigned loadRC=0, unsigned char flag=0);
02446
02447     int init_mxCRef(
02448         const char *F, int L, const mxArray *S, unsigned k
02449     );
02450
02451     TQ* qptr(unsigned k) const {
02452         unsigned n=this->t.qlen();
02453         if (this->qs.len%n || k>=this->qs.len/n) wblog(FL,
02454             "ERR %s() index out of bounds (%d/%d <> %d)",
02455             FCT,this->qs.len,n,k);
02456         return (this->qs.data+k*n);
02457     };
02458
02459     bool olderThan(const CData &B) const {
02460         if ((QSet<TQ>&)*this!=(QSet<TQ>&B) wblog(FL,
02461             "ERR %s() got incompatible CData\n  %s\n<> %s",
02462             FCT, STR_(this), STR(B)
02463         );
02464         checkSameStat(FL,B);
02465         return cstat.olderThan(B.cstat);
02466     };
02467
02468     bool operator!=(const CData &B) const { return !((*this)==B); };
02469     bool operator==(const CData &B) const {
02470         return (
02471             this->t==B.t && this->qs==B.qs && this->qdir==B.qdir &&
02472             cgd==B.cgd && cstat.sameAs(B.cstat,0)
02473         );
02474     };
02475
02476     int gotDiff(const CData &B, char lflag=0) const;
02477     bool sameAs(const CData &B, char lflag=0) const {
02478         return !gotDiff(B,lflag);
02479     };
02480
02481     bool operator!=(const QSet<TQ> &B) const {
02482         return (this->QSet<TQ>::operator!=(B));
02483     };
02484     bool operator==(const QSet<TQ> &B) const {
02485         return (this->QSet<TQ>::operator==(B));
02486     };
02487
02488     bool operator!=(const cgdStatus &b) const { return (cstat!=b); };
02489     bool operator==(const cgdStatus &b) const { return (cstat==b); };
02490
02491     bool isEmpty(char check=3) const {
02492         return (QSet<TQ>::isEmpty(check) && cgd.isEmpty());
02493     };
02494

```

```

02495     bool isAbelian(const char *F=NULL, int L=0) const {
02496         if (this->t.isAbelian()) {
02497             if (cstat.t==CGD_ABELIAN) {
02498                 if (cgd.D.len || cgd.SIZE.len) wblog(F_L,
02499                     "ERR %s() invalid abelian CData\n%s",FCT,STR_(this));
02500                 return 1;
02501             }
02502             if (!isScalar() || (cgd.D.len && (cgd.D.len>1 || cgd.D[0]!=1)))
02503                 wblog(F_L,"ERR %s() invalid abelian CData\n%s",STR_(this));
02504             return 1;
02505         }
02506         return 0;
02507     };
02508
02509     bool sizeable(unsigned Dmin=1024) const {
02510         return (cgd.D.len>Dmin);
02511     };
02512
02513     bool isScalar(char dflag=0) const;
02514
02515     TD getScalar(const char *F=NULL, int L=0) const {
02516         if (!isScalar()) wblog(F_L,
02517             "ERR %s() got non-scalar CData\n%s",FCT,STR_(this));
02518         return (cgd.D.len ? cgd.D.data[0] : TD(1));
02519     };
02520
02521     unsigned rank(const char *F=NULL, int L=0) const;
02522
02523     unsigned rankS() const { return cgd.rank(); };
02524
02525     bool isrank(unsigned r, unsigned *r_=NULL) const {
02526         unsigned l=rank(FL); if (r_) { (*r_)=l; }
02527         return (l!=r);
02528     };
02529
02530     template <class T>
02531     wbvector<T>& getSize(
02532         wbvector<T> &S, char bare=0, const wbperm *cgp=NULL) const;
02533
02534     SPIDX_T dim() const;
02535
02536     bool isRefInit(char check=1) const {
02537         if (cstat!=CGD_REF_INIT) { return 0; }
02538         if (check) {
02539             const unsigned r=cgd.SIZE.len, m=cgd.D.len;
02540             if (!r || (m>1 && m!=cgd.SIZE[r-1])) wblog(FL,
02541                 "ERR %s() invalid CData size ref (%s @ D.len=%d)",
02542                 FCT, SSTR(cgd), cgd.D.len
02543             );
02544         }
02545         return 1;
02546     };
02547
02548     bool checkValidStat(const char *F=0, int L=0) const {
02549         if (this->t.isAbelian()) {
02550             if (cstat.hasID('I')) || cstat.t!=CGD_ABELIAN) {
02551                 if (F) wblog(F,L,"ERR %s() "
02552                     "abelian\n%s\n%s",FCT,STR_(this),STR2(cstat,'V'));
02553                 return 0;
02554             }
02555         }
02556         else if (cstat.inValid() || cstat.t<=CGD_ABELIAN) {
02557             if (F) wblog(F,L,"ERR %s() "
02558                 "non-abelian\n%s\n%s",FCT,STR_(this),STR2(cstat,'V'));
02559             return 0;
02560         }
02561         return 1;
02562     };
02563
02564     int checkSameStat(
02565         const char *F, int L, const CData &B, char lflag=1) const {
02566
02567         int e=0;
02568         if (!lflag) { if (cstat.t!=B.cstat.t) e=1; }
02569         if (cstat.ID!=B.cstat.ID || cstat.ctime!=B.cstat.ctime) e|=2;
02570
02571         if (e && F) {
02572             wblog(F_L,"WRN %s() cstat mismatch (e=%d)",FCT,e);
02573             wblog(FL," > %s\n > %s\n... having\n > %s\n > %s",
02574                 STR_(this), STR(B), STR2(cstat,'V'), STR2(B.cstat,'V'));
02575             if (e>1) wblog(FL,"ERR %s() e=%d",FCT,e);
02576         }
02577         return e;
02578     };
02579
02580     bool sameType(const CData &S, const char *F=NULL, int L=0) const;
02581

```

```

02582     int  cmpOM(const CData &B) const;
02583
02584     template <class DB>
02585     bool sameSizeR(
02586         const CData<TQ,DB> &B,
02587         unsigned *r=NULL, const char *F=NULL, int L=0) const;
02588
02589     template <class DB>
02590     bool sameSizeR(
02591         const cdata<DB> &B,
02592         unsigned *r=NULL, const char *F=NULL, int L=0) const;
02593
02594     template <class T2>
02595     int sameSizeR(const char *F, int L, const wvector<T2> &S) const;
02596
02597     bool isSymmetric() const;
02598
02599     CData& initOM(const char *F, int L);
02600
02601     unsigned getOM(const char *F=NULL, int L=0) const {
02602         unsigned m=0;
02603
02604         if (cgd.SIZE) { if (!(m=gotOM(F,L))) { m=1; } } else
02605         if (this->qdir.len==2 && cgd.isDiag()) { m=1; } else
02606         if (cgd.isEmpty() && cstat==CGD_ABELIAN) { m=1; } else
02607         if (!isEmpty()) { wblog(FL,
02608             "WRN %s() got %s -> OM=%d",FCT,STR_(this),m);
02609             this->rank(F_L);
02610             cgd.wbarray<TD>::checkSize(F_LF);
02611         }
02612         return m;
02613     };
02614
02615     unsigned gotOM(const char *F=NULL, int L=0) const {
02616         if (this->qdir.len) {
02617             unsigned l=cgd.SIZE.len, r=this->rank(F_L);
02618             if (l==r+1) {
02619                 SPIDX_T i=cgd.SIZE[r];
02620                 if (!i || (i>1 && !this->t.permitsOM(r))) wblog(FL,
02621                     "ERR %s() invalid OM setting (i=%d,r=%d)\n%s",
02622                     FCT,i,r,STR_(this));
02623                 return i;
02624             }
02625             else if (l && l!=r) wblog(FL,
02626                 "ERR %s() invalid OM setting\n%s",FCT,STR_(this));
02627         };
02628         return 0;
02629     };
02630
02631     unsigned checkOM(const char *F=NULL, int L=0) const {
02632         if (this->qdir.len) {
02633             unsigned r=this->rank(F_L);
02634             if (this->t.permitsOM(r)) {
02635                 if (this->cgd.SIZE.len>r) {
02636                     SPIDX_T i=this->cgd.SIZE[r]; if (!i)
02637                         wblog(FL,"ERR %s() got OM=%d",FCT,i);
02638                     return i;
02639                 }
02640                 return 1;
02641             }
02642             else if (this->cgd.SIZE.len>r) wblog(FL,
02643                 "ERR %s() CData with invalid OM\n%s",FCT,STR_(this));
02644         }
02645         return 0;
02646     };
02647
02648     int gotFullOM(const char *F=NULL, int L=0) const {
02649         unsigned r=this->qdir.len, m=getOM(F_L);
02650
02651         if (isEmpty()) { return -1; }
02652
02653         if (isAbelian(F_L)) { return 1; }
02654         if (r<=2 && m!=1) wblog(FL,"ERR %s() %s with m=%d",FCT,STR_(this),m);
02655
02656         if (r<2) { return -2; }
02657         if (r>3) { return -4; }
02658
02659         if (r==2) {
02660             if (cstat.t!=CGD_IDENTITY && cstat.t!=CGD_1JSY_ST3 &&
02661                 cstat.t!=CGD_1JSY_GEN)
02662                 wblog(FL,"WRN %s() got %s",FCT,STR_(this));
02663             return 2;
02664         }
02665         else {
02666             if (cstat.t==CGD_STD3 || cstat.t==CGD_STD3_X

```

```

02669         || this->t.sub<2>() { return +3; }
02670         if (cstat.t!=CGD_FROM_CTR && cstat.t!=CGD_FROM_DEC &&
02671             cstat.t!=CGD_GEN_X) wblog(FL,
02672             "WRN %s() got %s",FCT,STR_(this));
02673         return -3;
02674     }
02675 };
02676
02677 int completeOM_DegQ(
02678     const char *F, int L, const wbperm &p,
02679     unsigned level=0
02680 );
02681
02682 CData& Project(
02683     const char *F, int L, cdata<TD> X,
02684     wbvector<TD> &w
02685 );
02686
02687 double normDiff(
02688     const char *F, int L, const CData &S,
02689     double eps=1E14
02690 ) const;
02691
02692 double norm2(unsigned k=-1) const;
02693
02694 wbvector<RTD>& trace(
02695     const char *F, int L, wbvector<RTD> &cgt) const;
02696
02697 wbvector<RTD> trace(const char *F=NULL, int L=0) const {
02698     wbvector<RTD> cgt; return trace(F,L,cgt);
02699 };
02700
02701 CData& trace(const char *F, int L,
02702             ctrIdx &i1, ctrIdx &i2, CData<TQ,TD> &X) const;
02703
02704 CData& AddMultiplicity(const char *F, int L, const cdata<TD> &c);
02705
02706 CData& AddMultiplicity(const char *F, int L, const CData &C) {
02707     if (this->t!=C.t || this->qs!=C.qs) wblog(F_L,
02708         "ERR %s() incompatible CData\n[%s] <> [%s] (%s, %s)",
02709         FCT, STR_(this), STR(C), sizeStr().data, C.sizeStr().data
02710     );
02711     return AddMultiplicity(F_L,C.cgd);
02712 };
02713
02714 cdata<TD>& getMultiplicity(unsigned im, cdata<TD> &c) const;
02715
02716 bool isBasicCG(const char *F=NULL, int L=0) const;
02717
02718 SPIDX_T getBasicCGSize(unsigned *m=NULL) const;
02719
02720 wbsparray<TD>&
02721 getBasicCG(unsigned k, wbsparray<TD> &a) const;
02722
02723 int getCG_set(
02724     const char *F, int L, wbIndex &Idx, wbsparray<TD> &a) const;
02725
02726 CData& reducetolJSymbol(const char *F, int L, CData<TQ,TD> &C) const;
02727
02728 int checkConsistency(const char *F=NULL, int L=0) const;
02729 int checkNormSign(const char *F=NULL, int L=0, char xflag=0) const;
02730
02731 int checkQ(const char *F, int L,
02732            const QSet<TQ> &Q, const char* istr=NULL) const;
02733
02734 bool checkAdditivityZ(const char *F, int L,
02735                      const wbMatrix<double> &Z1,
02736                      const wbMatrix<double> &Z2, const wbMatrix<double> &Z,
02737                      TD eps=CG_EPS2
02738 ) const;
02739
02740 double SkipTiny(const char *F=NULL, int L=0);
02741
02742 CData& Permute(const wbperm &P, char iflag=0) {
02743     CData<TQ,TD> X; this->save2(X);
02744     return X.permute(*this,P,iflag);
02745 };
02746
02747 CData& permute(CData &B, const wbperm &P, char iflag=0) const {
02748     QSet<TQ>::permute((QSet<TQ>&)B,P,iflag);
02749
02750     cgd.permute(B.cgd,P,iflag);
02751     B.cstat=cstat;
02752
02753     return B;
02754 };
02755

```



```

02756     CData& Conj() { QSet<TQ>::Conj(); return *this; }
02757     CData& Conj(unsigned i);
02758
02759     size_t memSize() const {
02760         return (
02761             QSet<TQ>::memSize() + cgd.memSize() + sizeof(cstat)
02762         );
02763     };
02764
02765     void info(const char *istr=NULL, const char *F=NULL, int L=0) const;
02766
02767     wstring dStr() const { return this->qdir.toStr(); };
02768     wstring qStr() const { return this->t.toStr(); };
02769
02770     wstring toStr() const { return toStr(0); };
02771     wstring toStr(char vflag) const;
02772
02773     wstring sizeStr() const { return sizeStr(0); };
02774     wstring sizeStr(char xflag) const;
02775
02776     mxArray* toMx() const;
02777
02778     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
02779     void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
02780
02781     mxArray* mxCreateCell(unsigned m, unsigned n) const {
02782         wblog(FL, "ERR %s(%d,%d)", FCT, m, n); return 0; };
02783     void add2MxCell(mxArray *S, unsigned i, char tst=0) const {
02784         wblog(FL, "ERR %s(%lX,%d,%d)", FCT, S, i, tst); };
02785
02786     void put(const char *vname, const char *ws="caller"
02787     ) const { put(0,0,vname,ws); };
02788
02789     void put(const char *F, int L,
02790             const char *vname, const char *ws="caller"
02791     ) const {
02792         mxArray *a=toMx();
02793         int i=mexPutVariable(ws,vname,a);
02794
02795         if (i) wblog(F_L,
02796             "ERR failed to write variable '%s' (%d)",vname,i);
02797         if (F) wblog(F_L, "I/O putting '%s' to %s",vname,ws);
02798         mxDestroyArray(a);
02799     };
02800
02801     #define UBUF_ID 2
02802     #define UBUF_REF 2
02803
02804     void setUser_BUF(unsigned char c=1) { cstat.user[UBUF_ID] = c; };
02805
02806     void setUser_BUF(unsigned char c, unsigned l) { unsigned k=UBUF_ID;
02807         if (l>7 || (!l && c==1)) wblog(FL,
02808             "WRN %s() got u[%d]=%d (%d/%d)",FCT,k,cstat.user[k],l,c);
02809
02810         if (c) { cstat.user[k] |= (c<1); }
02811         else { c=(l<1); cstat.user[k] &= ~c; };
02812     };
02813
02814     void setUser_BUF_ref(unsigned char c=1) {
02815         if (c) { c=(1<UBUF_REF); cstat.user[UBUF_ID] |= c; }
02816         else {
02817             c=(1<UBUF_REF)-1; cstat.user[UBUF_ID] &= c;
02818         };
02819     };
02820
02821     bool gotuser_BUF() const { return cstat.user[UBUF_ID]; };
02822
02823     bool gotuser_BUF_ref() const { unsigned char c=(1<UBUF_REF);
02824         return (cstat.user[UBUF_ID] & c);
02825     };
02826
02827     cdata<TD> cgd;
02828
02829     cgdStatus cstat;
02830
02831 private:
02832 };
02833
02834 template <class TQ>
02835 class CRef {
02836 public:
02837
02838     CRef( CGR_TYPE rt_=CGR_DEFAULT ) : cgb(NULL), conj(0), rtype(rt_) {};
02839
02840     CRef(const CRef &R) { init(R); };
02841     CRef(const CData_TQ &C, unsigned m=-1) : CRef() { initBase(C,m); };

```

```

02843
02844 CRef& init( CGR_TYPE rt_=CGR_DEFAULT ) {
02845     cgb=NULL; conj=0; rtype=rt_; cgp.init(); cgw.init();
02846     return *this;
02847 };
02848
02849 CRef& init(const CRef &R, char full=1) {
02850     if (this!=&R) {
02851         cgb=R.cgb; rtype=R.rtype; conj=R.conj; cgp=R.cgp;
02852         if (full) { cgw=R.cgw; }
02853     }
02854     return *this;
02855 };
02856
02857 CRef& operator=(const CRef &R) { return init(R); }
02858
02859 void swap(CRef &R) {
02860     if (this!=&R) { SWAP(cgb, R.cgb );
02861         cgp.swap(R.cgp); SWAP(conj, R.conj );
02862         cgw.swap(R.cgw); SWAP(rtype,R.rtype);
02863     }
02864 };
02865
02866 CRef& save2(CRef &R) {
02867     if (this!=&R) { R.cgb=cgb; cgb=0;
02868         cgp.swap(R.cgp); R.conj=conj; conj=0;
02869         cgw.swap(R.cgw); R.rtype=rtype; rtype=0;
02870         cgp.init(); cgw.init();
02871     }; return R;
02872 };
02873
02874 double safeCpy(const char *F, int L, const CRef &B, CRef &C) const;
02875
02876 CRef& init_1(const CRef &R, unsigned j) {
02877     unsigned m=R.wdim2(); if (j>=m) wblog(FL,
02878         "ERR %s() index out of bounds (%d/%d)\nhaving %s",FCT,j,m,STR(R));
02879     init(R,0); R.cgw.getCol(j,cgw);
02880     return *this;
02881 };
02882
02883 CRef& init_wId(const CRef &R,
02884     unsigned wid=1, unsigned ll=-1, const unsigned *d=NULL
02885 ){
02886     double w=1;
02887     unsigned m = (wid==1 ? R.wdim1() : R.wdim2());
02888     if (!wid || wid>2) { wblog(FL,"ERR %s() invalid wid=%d",FCT,wid); }
02889
02890     init(R,0);
02891
02892     if (int(ll)>=0) { unsigned d=qdim(ll);
02893         if (d_ && d!=(*d_)) wblog(FL,
02894             "ERR %s() qdim inconsistency (%d/%d)",FCT,d,*d_);
02895         w=::sqrt(double(d));
02896     }
02897     else if (d_) {
02898         if (*d_<1) wblog(FL,"ERR %s() invalid qdim=%d",FCT,*d_);
02899         w=::sqrt(double(*d_));
02900     }
02901
02902     cgw.initIdentity(m,0,w);
02903     return *this;
02904 };
02905
02906 CRef& init(
02907     const char *F, int L, const mxArray *a, unsigned k,
02908     char refC=0, const QSet<TQ> *Q=NULL,
02909     char xflag=0
02910 );
02911
02912 CRef<TQ>& initIdentityR(const char *F, int L,
02913     const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0);
02914
02915 CRef<TQ>& initIdentity1J(const char *F, int L,
02916     const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0);
02917
02918 CRef<TQ>& Reduce2Identity(char xflag=0);
02919
02920 wvector<RTD>& trace(const char *F, int L, wvector<RTD> &cgt) const;
02921
02922 #ifndef QS_SKIP_MPFR
02923 wvector<double>& trace(const char *F, int L, wvector<double> &cgt) const {
02924     wvector<RTD> cgt_; trace(F,L,cgt_);
02925     return cgt.initT(cgt_);
02926 };
02927 #endif
02928
02929 wvector<double> trace(const char *F, int L) const {

```

```

02930         wbvector<double> cgt;
02931         return trace(F,L,cgt);
02932     };
02933
02934     double trace() const;
02935
02936     double trace(const char *F, int L,
02937         const ctrIdx &i1, const ctrIdx &i2, CRef *Rt=NULL) const;
02938
02939     CRef& initAbelian() {
02940         cgb=NULL; rtype=CGR_ABELIAN;
02941         conj=0; cgp.init(); cgw.init();
02942         return *this;
02943     };
02944
02945     CRef& initCtrScalar(double x=1) {
02946         cgb=NULL; rtype=CGR_CTR_SCALAR;
02947         conj=0; cgp.init(); cgw.init(1,1,&x);
02948         return *this;
02949     };
02950
02951     CRef& initBase(const CDATA_TQ *cgr, unsigned m=-1, const CDATA_TQ* X=NULL);
02952     CRef& initBase(const CDATA_TQ &C, unsigned m=-1);
02953
02954     int LoadRef(const char *F=0, int L=0, char force=1) const {
02955         if (!cgb) {
02956             wblog(FL,"WRN %s() got cgb=null\n%s",FCT,STR_(this));
02957             return 0;
02958         }
02959         return ((CDATA_TQ*)cgb)->LoadRef(F,L,force);
02960     };
02961
02962     int Reduce2Ref(const char *F=0, int L=0, char force=0) const {
02963         if (isRefInit(F,L,force)) { return 0; }
02964         if (!cgb) wblog(FL,"ERR %s() got cgb=null\n%s",FCT,STR_(this));
02965         return ((CDATA_TQ*)cgb)->Reduce2Ref(F,L,force);
02966     };
02967
02968     explicit operator bool() const { return !isEmpty(); }
02969     bool operator! () const { return isEmpty(); }
02970
02971     bool isEmpty() const {
02972         if (cgb && !cgb->isEmpty()) { return 0; }
02973
02974         size_t n=cgw.numel();
02975         if (n>1 || cgp.len) wblog(FL,
02976             "ERR %s() invalid abelian cref (%d)",FCT,STR_(this));
02977         return (rtype==CGR_DEFAULT && !n);
02978     };
02979
02980     bool isSymmetric(const char *F=0, int L=0) const;
02981
02982     bool isRefInit(const char *F=0, int L=0, char check=1) const {
02983         if (cgb && cgb->isRefInit(check)) {
02984             if (rtype!=CGR_DEFAULT) wblog(F_L,
02985                 "ERR %s() REF_INIT mismatch\n%s",FCT,STR_(this));
02986             return 1;
02987         }
02988         return 0;
02989     };
02990
02991     bool isAbelian(const char *F=NULL, int L=0) const;
02992
02993     bool isScalar(char dflag=0) const;
02994
02995     size_t isw2(const char *F=NULL, int L=0) const;
02996
02997     size_t isw3() const {
02998         return (cgw.SIZE.len==3 ? cgw.numel() : 0);
02999     };
03000
03001     unsigned Reduce_w3Id(unsigned d=-1, char lflag=0);
03002
03003     int cgw_check(const char *F, int L) const;
03004
03005     bool cgw_exists(unsigned i, unsigned j);
03006     int cgw_check_std3(
03007         const char *F=NULL, int L=-1, unsigned d3=0, unsigned im=0);
03008
03009     int checkQ(const char *F, int L, const QSet<TQ> &Q) const;
03010
03011     unsigned numel() const;
03012     unsigned wnumel() const { return cgw.numel(); }
03013
03014     bool wscalar(const char *F=NULL, int L=0) const;
03015     bool wscalar1(const char *F=NULL, int L=0) const {
03016         return (wscalar(F,L) && cgw[0]==1);

```

```

03017     };
03018
03019     double wel(unsigned k) const;
03020
03021     bool wom(const char *F=NULL, int L=0) const {
03022         return (isw2(F_L)>1); };
03023
03024     bool wdim_is(unsigned m) const {
03025         return (isw2() ? cgw.isSMatrix(m) : 0); };
03026
03027     unsigned wdim(const char *F=NULL, int L=1) const;
03028
03029     unsigned wdim1(const char *F=NULL, int L=0) const {
03030         return isw2(F_L); }
03031     unsigned wdim2(const char *F=NULL, int L=0) const { isw2(F_L);
03032         return cgw.SIZE[1]; }
03033
03034     unsigned wdim12_(const char *F, int L, unsigned &d2) const;
03035
03036     unsigned wdim12(
03037         const char *F, int L, unsigned &d2, unsigned d0=1) const;
03038
03039     unsigned wdim3(const char *F=NULL, int L=0) const;
03040
03041     double wget0() const;
03042     double wget1() const;
03043
03044     bool wSame(
03045         const wbararray<double> &cgw_, char lenient=0, double eps=1e-14) const;
03046
03047     template <class T>
03048     wbvector<T>& getSize(wbvector<T> &S, char bare=0) const;
03049
03050     unsigned Size(
03051         unsigned k,
03052         unsigned r=-1) const;
03053
03054     unsigned qdim(
03055         unsigned k,
03056         const QType *qt=NULL) const;
03057
03058     int checkAbelian(const char *F=NULL, int L=0) const;
03059
03060     bool isDiagCSC(RTD eps=1E-14) const;
03061     bool isIdentityCG(wbvector<double> *nrm=NULL, double eps=1E-14) const;
03062
03063     int wisId() const;
03064
03065     double norm2(char checks=1) const;
03066
03067     double NormSignW(
03068         const char *F=NULL, int L=0,
03069         char useExt=1,
03070         double eps =CG_SKIP_DEPS1,
03071         double eps2=CG_SKIP_DEPS2
03072     );
03073
03074     char sameUptoFac(const CRef &B, double *fac=NULL, double eps=1E-12) const;
03075     char sameAs(const CRef &B, double eps=1E-12) const;
03076     char sameAs_fix(const char *F, int L, CRef &B, double eps=1E-12);
03077
03078     bool sameQSet(const QSet<TQ> &Q) const;
03079     bool sameQSet(const CRef &B) const;
03080     bool sameQDir(const char *F, int L, const QDir &qd) const;
03081
03082     bool sameQDir(const char *F, int L, const CRef& B) const {
03083         QDir qdb; B.get_qdir(qdb);
03084         return sameQDir(F,L,qdb);
03085     };
03086
03087     bool sameQDir(const iTags& b) const;
03088
03089     int gotSameCData(
03090         const char *F, int L, const CRef& B, char xflag=0
03091     ) const;
03092
03093     double NormStd(const char *F, int L, unsigned r=-1);
03094
03095     double normExt(const char *F=NULL, int L=0) const;
03096
03097     int SortDegQ(const char *F=NULL, int L=0, QSet<TQ> *QS=NULL);
03098
03099     bool isSortedDegQ(QSet<TQ> *QS, wbperm &pxt, char &cxt) const;
03100     bool isSortedDegQ(QSet<TQ> *QS=NULL) const {
03101         wbperm pxt; char cxt=0;
03102         return isSortedDegQ(QS,pxt,cxt);
03103     };

```

```

03104
03105     char got3(const char *F, int L,
03106              const QType &q, const qset<TQ> &J12, const qset<TQ> &J3
03107              ) const;
03108
03109     bool operator==(const CRef &B) { return (sameAs(B)==0 ? 1 : 0); };
03110     bool operator!=(const CRef &B) { return (sameAs(B)==0 ? 0 : 1); };
03111
03112     double normDiff2(const char *F, int L, const CRef &B) const;
03113     double normDiff (const char *F, int L, const CRef &B) const {
03114         return SQRT(normDiff2(F,L,B)); };
03115
03116     wbararray<double>& wProd(const CRef &B, wbararray<double> &x) const;
03117
03118     int check(const char *F, int L) const;
03119     int check(const char *F, int L,
03120              const QType &q, const QDir &qdir) const;
03121
03122     unsigned getOM(const char *F=NULL, int L=0, char wflag=0) const;
03123
03124     unsigned checkOM(const char *F=NULL, int L=0) const {
03125         return (cgb ? cgb->checkOM(F,L) : 0);
03126     };
03127
03128     unsigned rankS(char lflag=0) const;
03129     unsigned rank(const char *F=NULL, int L=0, char lflag=0) const;
03130
03131     CRef wget(unsigned j) const {
03132         CRef<TQ> R;
03133         return R.init(*this,j);
03134     };
03135
03136     CRef& Permute(const wbperm &P, char iflag=0, char isnew=0);
03137     CRef& permute(
03138         const wbperm &P, CRef &B, char iflag=0, char isnew=0
03139     ) const { B=(*this); return B.Permute(P,iflag,isnew); };
03140
03141     CRef& Conj() {
03142         if (cgb) { conj=(conj ? 0 : 1); }
03143         return *this;
03144     };
03145
03146     int HConjOpScalar();
03147
03148     bool gotConj() const { return (conj!=0); };
03149
03150     bool gotPerm(const char *F=NULL, int L=0) const {
03151         if (cgp.len) {
03152             if (!cgb) wblog(F_L,"ERR %s() "
03153                 "invalid perm (len=%d; cgb=NULL)",FCT, cgp.len);
03154             if (cgp.len!=cgb->qdir.len) wblog(F_L,"ERR %s() "
03155                 "invalid perm (len=%d/%d)",FCT, cgp.len, cgb->qdir.len);
03156             return !cgp.isIdentityPerm(F_L,cgb->qdir.len);
03157         }
03158         return 0;
03159     };
03160
03161     bool anyTrafo(char pflag=1) const {
03162         if (conj || cgp.len) return 1;
03163         if (pflag && !cgp.isIdentityPerm()) return 1;
03164         return 0;
03165     };
03166
03167     bool anyTrafo(char &ip, char &ic, char pflag=1) const {
03168         if (pflag)
03169             { ip=(cgp.len && !cgp.isIdentityPerm()); }
03170         else { ip=(cgp.len ? 1 : 0); }
03171         ic=(conj!=0);
03172         return (ip || ic);
03173     };
03174
03175     bool gotnoTrafos() const {
03176         return !anyTrafo(); };
03177
03178     unsigned getP(unsigned k) const {
03179         if (cgp.len) {
03180             if (k>=cgp.len) wblog(F_L,
03181                 "ERR %s() index out of bounds (%d/%d)",FCT,k,cgp.len);
03182             return cgp.data[k];
03183         }
03184         return k;
03185     };
03186
03187     wbperm& getP(wbperm &p, char iflag=0) const {
03188         if (!cgp.len)
03189             return p.init(rank(F_L));
03190         else return p.init(cgp,iflag);

```

```

03191     };
03192
03193     char stat() const { char i=0;
03194         if (rtype!=CGR_DEFAULT) i ^= 1;
03195         if (conj) i ^= 2;
03196         if (cgp.len && !cgp.isIdentityPerm()) i ^= 4;
03197         return i;
03198     };
03199
03200     bool affectsQDir() const;
03201
03202     qset<TQ>& adapt(qset<TQ> &qs, char iflag=0) const {
03203         if (cgp.len) qs.BlockPermute(cgp,iflag ? 0 : 'i');
03204         return qs;
03205     };
03206
03207     QSet<TQ>& adapt(QSet<TQ> &Q, char iflag=0) const;
03208
03209     ctrIdx& adapt(ctrIdx &I, char iflag=0) const;
03210
03211     QDir& get_qdir(QDir &qd) const {
03212         if (!cgb) { qd.init(); return qd; }
03213
03214         if (cgp.len)
03215             cgb->qdir.permute(qd,cgp);
03216         else qd=cgb->qdir;
03217
03218         if (conj) qd.Conj();
03219
03220         return qd;
03221     };
03222
03223     QDir get_qdir() const { QDir qd;
03224         return get_qdir(qd);
03225     };
03226
03227     wstring qdir2Str(char vflag=0) const {
03228
03229         if (!cgb || !cgb->qdir.len) return "";
03230
03231         if (vflag<=0) return get_qdir().toStr();
03232         if (vflag==1 || vflag=='v') {
03233             unsigned l, n=2*(cgb->qdir.len)+5; char s[n];
03234             l=snprintf(s,n,"%s => %s",
03235                 cgb->qdir.toStr().data, STR(get_qdir()));
03236             if (l>=n) wblog(FL,
03237                 "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
03238             return s;
03239         }
03240         else {
03241             unsigned l, n=2*(cgb->qdir.len)+5; char s[n];
03242             l=snprintf(s,n,"%s[%s]%s",
03243                 cgb->qdir.toStr().data, (cgp+1).toStr(1,"").data,
03244                 gotConj()?"*":"" );
03245             if (l>=n) wblog(FL,
03246                 "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
03247             return s;
03248         }
03249     };
03250
03251     wstring toStr(char lflag=0) const;
03252
03253     wstring sizeStr() const;
03254
03255     wstring statStr(char vflag=0) const;
03256
03257     wstring qStr() const { return cgb ? cgb->qStr() : ""; };
03258     wstring QStr() const {
03259         return cgb ? cgb->QStrS(&cgp) : "";
03260     };
03261
03262     mxArray* toMx(char flag=0) const;
03263
03264     mxArray* toMX() const { return toMx('f'); };
03265
03266     mxArray* mxCreateStruct(unsigned m, unsigned n, char flag=0) const;
03267     void add2MxStruct(mxAarray *S, unsigned i, char flag=0) const;
03268
03269     const CDATA_TQ *cgb;
03270
03271     warray<double> cgw;
03272
03273     wbperm cgp;
03274
03275     char conj;
03276
03277     cgrType rtype;

```

```

03278 };
03279
03280 #define MAP32 std::map < qset<TQ>, CRef<TQ> >
03281 #define MAP31 std::map < qset<TQ>, MAP32 >
03282
03283 #define MP3_LDC 1
03284 #define MP3_LDR 2
03285
03286 #define TP3_ITER 3
03287 #define TP3_LDM 4
03288 #define TP3_LDR 8
03289 #define TP3_TST 16
03290 #define TP3_LOAD (TP3_LDM|TP3_LDR)
03291
03292 #define LB_LOAD 4
03293 #define LB_UPD 8
03294 #define LB_CALC 16
03295 #define LB_REF 32
03296
03297 #define LB_GEN (LB_LOAD | LB_CALC )
03298 #define LB_GET (LB_LOAD | LB_CALC | LB_UPD )
03299 #define LB_ANY (31<<2)
03300
03301 #define QF_LOADC 128
03302
03303 #define CX3_SAVE 1
03304 #define CX3_FORCE 2
03305
03306 template <class TQ>
03307 class CStore {
03308
03309 public:
03310
03311     CStore() {
03312         BUF.max_load_factor(0.2);
03313     };
03314
03315     void clear() {
03316         map3.clear();
03317         BUF.clear();
03318     };
03319
03320     typedef typename MAP32::iterator iMAP32;
03321     typedef typename MAP31::iterator iMAP31;
03322
03323     void init() {};
03324
03325     const CDATA_TQ& getIdentityC(const char *F, int L,
03326         const QType &t, const TQ *qs, unsigned dim=-1,
03327         unsigned loadRC=LB_GEN
03328     );
03329     const CDATA_TQ& getIdentityIJ(
03330         const char *F, int L, CRef<TQ> &C,
03331         const QType &t, const TQ *qs, unsigned dim=-1,
03332         unsigned loadRC=LB_GEN
03333     );
03334
03335     void add_CData_abelian(
03336         const QType &q, const TQ &J1, const TQ &J2);
03337
03338     void add_CData(
03339         const QType &q, const qset<TQ> &J1, const qset<TQ> &J2);
03340
03341     void add2BUF(const char *F, int L, const CDATA_TQ &S, char nflag=0);
03342
03343     unsigned add3(const QType &q, const mxArray* S, char loadC=0);
03344     unsigned add2BUF(const QType &q, const mxArray* S, unsigned k=-1);
03345
03346     CDATA_TQ& getBUF(
03347         const char *F, int L, const QSet<TQ> &Q, unsigned loadRC=0
03348     );
03349
03350     size_t reduceMemUsage(const char *F=NULL, int L=0, double mfac=-1);
03351
03352     void getQfinal_l(const char *F, int L, const QType &type,
03353         const qset<TQ> &J1, const qset<TQ> &J2, wbMatrix<TQ> &J,
03354         wbMatrix< const CRef<TQ>* > *ss=NULL,
03355         char loadC=0
03356     );
03357
03358     void getQfinal_v(const char *F, int L, const QVec &type,
03359         const qset<TQ> &q1, const qset<TQ> &q2, wbMatrix<TQ> &QQ,
03360         wbMatrix< const CRef<TQ>* > *SS=NULL,
03361         char loadC=0
03362     );
03363
03364     int getQfinal_zdim(

```

```

03365     const char *F, int L, const QVec &qvec,
03366     const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2, const wbMatrix<TQ> &Q,
03367     wbvector<widx_t> &s1, wbvector<widx_t> &s2, wbvector<widx_t> &s,
03368     wbvector<unsigned> *m=NULL, wbMatrix< const CRef<TQ>*> *S3=NULL
03369 );
03370
03371 void getQfinal(const char *F, int L, const QVec &type,
03372     const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2, QMap<TQ> &M,
03373     unsigned char cgflag=0
03374 );
03375
03376 void Info(const char *F=0, int L=0, char vflag=0) const;
03377
03378 int checkInit(const char *F, int L, const QType &t) const;
03379
03380 unsigned LoadStore(const char *F, int L, const char *file);
03381
03382 mxArray* toMx(const QType &q=QTYPE_UNKNOWN) const;
03383
03384 void put(const char *F, int L,
03385     const char *vname, const char *ws="caller"
03386 ) const {
03387     mxArray *a=toMx(); int i=mexPutVariable(ws,vname,a);
03388
03389     if (i) wblog(F,L,
03390         "ERR failed to write variable '%s' to %s (%d)",vname,i,ws);
03391     if (F) wblog(F,L,"I/O putting '%s' to %s",vname,ws);
03392     mxDestroyArray(a);
03393 };
03394
03395 void put(const char *vname, const char *ws="caller") const {
03396     put(0,0,vname,ws);
03397 };
03398
03399 int find_map3(const QType &q, const qset<TQ> &J1, const qset<TQ> &J2
03400 ) const {
03401     qset<TQ> J12(J1,J2);
03402     if (J1.len!=J2.len || J1.len!=q.qlen()) wblog(F,L,
03403         "ERR %s() qset length inconsistency (%d/%d/%d)",
03404         FCT,J1.len,J2.len,q.qlen());
03405     return find_map3(q,J12);
03406 };
03407
03408 int find_map3(const QType &q, const qset<TQ> &J12) const {
03409     if (J12.len!=2*q.qlen()) {
03410         if (!q.isKnown()) wblog(F,L,
03411             "ERR %s() got uninitialized symmetry",FCT); else
03412             wblog(F,L,"ERR %s() "
03413                 "qset length inconsistency (%d/2*%d)",FCT,J12.len,q.qlen()
03414             );
03415     }
03416     auto it1 = map3.find(q);
03417     if (it1!=map3.end()) {
03418         auto it2 = it1->second.find(J12);
03419         if (it2!=it1->second.end()) { return it2->second.size(); }
03420     }
03421     return -1;
03422 };
03423
03424 int valid_mp3_data(const QType &t,
03425     const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J3,
03426     char rflag=0
03427 ) const;
03428
03429 CDATA_TQ* BUF_find(const QSet<TQ> &Q);
03430
03431 int Reduce2Ref(
03432     const char *F, int L, const QSet<TQ> &Q,
03433     char force=0, unsigned Dmin=1024);
03434
03435 map <QType,
03436     map <qset<TQ>,
03437         map <qset<TQ>,
03438             CRef<TQ>
03439         >
03440     >
03441 > map3;
03442
03443 unordered_map <
03444     QSet<TQ>,
03445     CDATA_TQ,
03446     QHash<TQ>
03447 > BUF;
03448
03449 protected:
03450 private:
03451

```



```

03452     iMAP31 get_mp3_data(const char *F, int L,
03453         const QType &q, const qset<TQ> &J1, const qset<TQ> &J2,
03454         char force=0
03455     ){
03456         if (J1.len!=J2.len) wblog(FL,
03457             "ERR qset length inconsistency (%d/%d)",J1.len,J2.len);
03458
03459         iMAP31 iJ12;
03460         qset<TQ> J12(J1,J2);
03461
03462         auto itype = map3.find(q); if (itype==map3.end()) { checkInit(FL,q);
03463             itype = map3.find(q); }
03464
03465         if (itype!=map3.end()) {
03466             iJ12 = itype->second.find(J12);
03467             if (iJ12!=itype->second.end()) {
03468                 if (!iJ12->second.size()) wblog(FL,
03469                     "ERR %s() got empty map3 data (%s)",FCT,STR(q));
03470                 return iJ12;
03471             }
03472         }
03473
03474         if (!force) {
03475             add_CData(q,J1,J2);
03476             return get_mp3_data(F,L,q,J1,J2,'f');
03477         }
03478
03479         wblog(F,L,"ERR failed to find or generate map3 target\n"
03480             "%s [%s] x [%s] => [%s]",STR(q), STR(J1), STR(J2), STR(J12));
03481         return iJ12;
03482     };
03483
03484     void print_Coeff(const char *F, int L, const QType &type,
03485         const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J,
03486         double c
03487     ){
03488         wblog(F,L," * CG coeff. %-8s [%s, %s; %s] = %.6g",
03489             type.toStr(), STR(J1), STR(J2), STR(J),c);
03490     };
03491 };
03492
03493     CStore<gTQ> gCS;
03494
03495     CData<gTQ,double> EMPTY_CGSTORE;
03496
03497     mxArray* map3_CreateStructMatrix(unsigned m, unsigned n);
03498
03499     template <class TQ>
03500     void map3_add2MxStruct(mxArray *S, unsigned k,
03501         const QType &q, const qset<TQ> &J12);
03502
03503     gTQ get_qtot_abelian(
03504         const QType &t, const gTQ *q0, const widx_t n,
03505         const widx_t *idx=NULL, const widx_t stride=1
03506     );
03507
03508     template <class TM>
03509     class cgc_contract_id : public wbvector<TM> {
03510     public:
03511
03512         cgc_contract_id() { wbvector<TM>::init(); };
03513
03514         template <class TQ, class TD>
03515         cgc_contract_id(
03516             const CData<TQ,TD> &A, const ctrIdx &ica,
03517             const CData<TQ,TD> &B, const ctrIdx &icb
03518         );
03519
03520         template <class TQ, class TD>
03521         void extract(
03522             CData<TQ,TD> &A, ctrIdx &ica,
03523             CData<TQ,TD> &B, ctrIdx &icb,
03524             char xflag=1
03525         ) const;
03526
03527         size_t memSize() const {
03528             return ( sizeof(*this) + this->len*sizeof(TM) );
03529         };
03530
03531         void wblog_(const char* F=0, int L=0, const char *istr=0) const {
03532             #ifdef QS_USING_OMP
03533                 Wb::ompGuard myLK(wblog_lock,1);
03534             #endif
03535
03536             CData<gTQ,RTD> a,b; ctrIdx ica,icb;
03537             this->extract(a,ica,b,icb,'l');
03538

```

```

03539
03540     wblog(F_L, "TST cgc_contract_id() %s", istr ? istr : "");
03541     wblog(F_L, "\b a: %-32s @ %s\n b: %-32s @ %s\n %s\n %s\n",
03542           STR(a), STR(ica), STR(b), STR(icb),
03543           STR2(a.cstat, 'V'), STR2(b.cstat, 'V')
03544     );
03545 };
03546
03547 wbststring toStr(char vflag=7) const {
03548     wbststring sout;
03549
03550     CData<gTQ,RTD> a,b; ctrIdx ica,icb;
03551     this->extract(a,ica,b,icb, vflag & 6 ? '1' : 0);
03552
03553     if (vflag&4) {
03554         sout.init(256); snprintf(sout.data,sout.len,
03555             "%-20s @%s #05x | %-20s @%s #05x",
03556             STR2(a,1), STR(ica), a.cstat.ID,
03557             STR2(b,1), STR(icb), b.cstat.ID
03558         );
03559     }
03560     else if (vflag&2) {
03561         sout.init(128); snprintf(sout.data,sout.len,
03562             "%-20s @%s | %-20s @%s",
03563             STR2(a,1), STR(ica), STR2(b,1), STR(icb)
03564         );
03565     }
03566     else {
03567         sout.init(64); snprintf(sout.data,sout.len,
03568             "%s_%s * %s_%s",STR2(a,0), STR(ica), STR2(b,0), STR(icb));
03569     }
03570     return sout;
03571 };
03572
03573 protected:
03574 private:
03575
03576     template <class T1, class T2>
03577     unsigned set_val(const char *F, int L, T1* &s, const T2 &x) {
03578         s[0]=x;
03579         if (T2(s[0])!=x) wblog(F_L, "ERR got non-%s data entry (%g)",
03580             TSTR(T1), double(x)
03581         );
03582         s+=1; return 1;
03583     };
03584
03585     template <class T1, class T2>
03586     unsigned set_vec(const char *F, int L, T1* &s, const wvector<T2> &x) {
03587         s[0]=x.len;
03588         if (unsigned(s[0])!=x.len) wblog(F_L, "ERR unexpected size "
03589             " (out of bounds %s @ %d)", TSTR(T1), x.len
03590         );
03591         set_range(F_L, ++s, x.len, x.data);
03592         return (1+x.len);
03593     };
03594
03595     template <class T1, class T2>
03596     unsigned set_range(const char *F, int L, T1* &s, unsigned n, const T2 *x) {
03597         for (unsigned i=0; i<n; ++i) {
03598             s[i]=x[i];
03599             if (T2(s[i])!=x[i]) wblog(F_L, "ERR got non-%s data entry (%g)",
03600                 TSTR(T1), double(x[i])
03601             );
03602         }
03603         s+=n; return n;
03604     };
03605
03606     template <class TQ, class TD>
03607     unsigned set_CData(const char *F, int L,
03608         char* &s, unsigned n, const CData<TQ,TD> &A, char tflag=1
03609     ){
03610         char *s0=s;
03611         unsigned r=A.rank(), l=4+A.qs.len+r+(4+r)*sizeof(unsigned);
03612
03613         wvector<size_t> S;
03614
03615         if (l>=n) wblog(F_L, "ERR %s() string out of bounds (%d/%d)", FCT, l, n);
03616
03617         if (tflag) {
03618             set_val(F_L, s, A.t.type);
03619             set_val(F_L, s, A.t.sub );
03620         }
03621
03622         set_vec(F_L, s, A.qs);
03623         set_vec(F_L, s, A.qdir);
03624
03625         unsigned *u=(unsigned*)s;

```

```

03626
03627     set_vec(FL,u, A.getSize(S,'b'));
03628
03629     s+=(u-(unsigned*)s)*sizeof(unsigned); l=s-s0;
03630     if (l>=n) wblog(FL,
03631         "ERR %s() string out of bounds (%d/%d)",FCT,l,n
03632     );
03633
03634     return l;
03635 };
03636
03637 template <class T1, class T2>
03638 void get_range(
03639     const char *F, int L, const T1* &s, unsigned n, T2 *x) const {
03640
03641     for (unsigned i=0; i<n; ++i) {
03642         x[i]=s[i];
03643         if (T1(x[i])!=s[i]) wblog(FL,"ERR got non-%s data entry (%g)",
03644             TSTR(T1), double(x[i])
03645         );
03646     }
03647     s+=n;
03648 };
03649
03650 template <class T1, class T2>
03651 void get_vec(
03652     const char *F, int L, const T1* &s, wvector<T2> &x) const {
03653     if (!s || int(s[0])<0) wblog(FL,
03654         "ERR %s() got invalid vec.len=%d",FCT,s?s[0]:-1);
03655     x.init(s[0]); ++s;
03656     get_range(F,L,s,x.len,x.data);
03657 };
03658
03659 template <class T1, class T2>
03660 void get_val(
03661     const char *F, int L, const T1* &s, T2 &x) const {
03662     get_range(F,L,s,1,&x);
03663 };
03664
03665 template <class TQ, class TD>
03666 unsigned get_CData(const char *F, int L,
03667     const char* &s, CData<TQ,TD> &A, char tflag=1) const {
03668
03669     A.init();
03670     if (tflag) {
03671         A.t.initx(QTYPE_SET(s[0]),unsigned(s[1]));
03672         s+=2;
03673     }
03674
03675     get_vec(F,L,s, A.qs);
03676     get_vec(F,L,s, A.qdir);
03677
03678     const unsigned *u = (const unsigned*)s;
03679     get_vec(F,L,u, A.cgd.SIZE);
03680     A.cstat.init0(CGD_BSZ_INIT);
03681
03682     s+=(u-(unsigned*)s)*sizeof(unsigned);
03683
03684     return 0;
03685 };
03686
03687 };
03688
03689 template <class TM>
03690 class MHash {
03691
03692 public:
03693
03694     size_t operator()(const cgc_contract_id<TM> &S) const {
03695
03696         MTI h=5381U, *const x=S.data;
03697         for (unsigned i=0; i<S.len; ++i) {
03698             h ^= ((h<<6) + (h>>2)) + x[i];
03699             h ^= ((h<<6) + (h>>2)) + x[i];
03700         }
03701         return h;
03702     };
03703 };
03704
03705 template <class TQ, class TD>
03706 class x3map {
03707
03708 public:
03709
03710     x3map() : cgb(NULL), conj(0), zflag(0), rtype(CGR_DEFAULT)
03711     #ifndef QS_USING_MPFR
03712     , X3(x3)

```

```

03713     #endif
03714 }
03715
03716 x3map(const char *F, int L,
03717       const CRef<TQ> &A_, const ctrIdx &ica,
03718       const CRef<TQ> &B_, const ctrIdx &icb, wbperm &cgp, char xflag=0
03719       ) : x3map() { initCtr(F_L,A_,ica,B_,icb, cgp, xflag); };
03720
03721 x3map& init() {
03722     a.init(); b.init(); c.init(); pab.init(); x3.init();
03723     #ifdef QS_USING_MPFR
03724         X3.init();
03725     #endif
03726     cgb=NULL; conj=0; zflag=0; rtype=CGR_DEFAULT;
03727     return *this;
03728 };
03729
03730 int initCtr(const char *F, int L,
03731            const CRef<TQ> &A_, const ctrIdx &ica_,
03732            const CRef<TQ> &B_, const ctrIdx &icb_, wbperm &cgp,
03733            char xflag=1
03734            );
03735
03736 x3map& init(
03737     const char *F, int L, const mxArray* S, unsigned k=0,
03738     ctrIdx *ica=NULL, ctrIdx *icb=NULL, char recalc=1
03739 );
03740
03741 bool isEmpty() const {
03742     return (
03743         a.isEmpty() && b.isEmpty() && c.isEmpty() &&
03744         !pab.len && x3.isEmpty()
03745     );
03746 };
03747
03748 void print(
03749     const char *F=NULL, int L=0, const char *istr="", char vflag=0
03750     ) const;
03751
03752 wbsting toStr(char vflag=0) const;
03753 wbsting x3Str(char vflag=0) const;
03754
03755 size_t memSize() const {
03756     return ( sizeof(cgb) + sizeof(char)
03757         + a.memSize() + b.memSize() + c.memSize()
03758         + sizeof(pab) + pab.len*sizeof(widx_t)
03759         + sizeof(x3) + x3.numel()*sizeof(TD)
03760         + sizeof(rtype)
03761     );
03762 };
03763
03764 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
03765
03766 void add2MxStruct(
03767     mxArray *S, unsigned l,
03768     const cgc_contract_id<MTI> *idc=NULL) const;
03769
03770 mxArray* toMx(const cgc_contract_id<MTI> *idc=NULL) const {
03771     mxArray *S=mxCreateStruct(1,1);
03772     add2MxStruct(S,0,idc);
03773     return S;
03774 };
03775
03776 CData<TQ,RTD> a, b, c;
03777
03778 const CData_TQ* cgb;
03779
03780 wbperm pab;
03781
03782 char conj;
03783
03784 int zflag;
03785
03786 cgrType rtype;
03787
03788 wbarry<TD> x3;
03789
03790 #ifdef QS_USING_MPFR
03791     wbarry<RTD> X3;
03792 #else
03793     wbarry<TD> &X3;
03794 #endif
03795
03796 protected:
03797 private:
03798 };
03799

```

```

03800 template <class TQ, class TD>
03801 class X3Map {
03802
03803 public:
03804
03805     X3Map() {
03806         XBUF.max_load_factor(0.2);
03807     };
03808
03809     void clear() {
03810         XBUF.clear();
03811     };
03812
03813     int init(const char *F, int L, const mxArray *S, unsigned k=0);
03814
03815     x3map<TQ,TD>& getXBUF(
03816         const char *F, int L,
03817         const cgc_contract_id<MTI> &idc
03818     );
03819
03820     int contains(const char *F, int L,
03821         const x3map<TQ,TD> &x, const cgc_contract_id<MTI> &idc);
03822
03823     int contractCGR(const char *F, int L,
03824         const CRef<TQ> &A, const ctrIdx &ica,
03825         const CRef<TQ> &B, const ctrIdx &icb, CRef<TQ> &C,
03826         char cg_preview=0
03827     );
03828
03829     CRef<TQ>& contractDegQ(const char *F, int L,
03830         const CRef<TQ> &A, CRef<TQ> &B);
03831
03832     unsigned add(
03833         const char *F, int L, const cgc_contract_id<MTI> &idc,
03834         const mxArray* S, unsigned k
03835     );
03836
03837     mxArray* toMx() const;
03838
03839     void Info(const char *F=0, int L=0) const;
03840
03841     unordered_map <
03842         cgc_contract_id<MTI>,
03843         x3map<TQ,TD>,
03844         MHash<MTI>
03845     > XBUF;
03846
03847 protected:
03848 private:
03849 };
03850
03851     X3Map<gTQ,double> gXS;
03852
03853 template <class TQ>
03854 class QMap {
03855
03856 public:
03857
03858     void init() { qvec.init();
03859         Q1.init(); Q2.init(); Q.init();
03860         I1.init(); I2.init(); II.init(); D.init();
03861     };
03862
03863     QMap<TQ>& init(const char *F, int L,
03864         const QVec &qv, const wbMatrix<TQ> &q1, const wbMatrix<TQ> &q2,
03865         const wbMatrix< wbMatrix<TQ> > &QQ,
03866         const wbMatrix< wbMatrix< const CRef<TQ>* > > *SS=NULL
03867     );
03868
03869     QMap<TQ>& init(const char *F, int L, const mxArray *a);
03870
03871     template<class TD>
03872     int getCGZlist(const char *F, int L,
03873         wbMatrix<TQ> &qc,
03874         wbvector<TD> &dc,
03875         wbvector<widx_t> *IC=NULL,
03876         const wbvector<widx_t>* Ix=NULL,
03877         const qset<TQ> *q3=NULL,
03878         wbIndex *m3=NULL,
03879         const wbvector<TD>* cc=NULL,
03880         double eps1=1E-10,
03881         double eps2=1E-14
03882     ) const;
03883
03884     void Skip(const wbindex &I);
03885
03886     template<class TD>

```

```

03887 void getIdentityQ(const char *F, int L,
03888     const wbvector<widx_t> &Sa, const wbvector<widx_t> &Sb,
03889     QSpace<TQ,TD> &A, char vflag=1
03890 ) const;
03891
03892 bool isConsistent(
03893     const char *F=NULL, int L=0, const QVec *qv=NULL
03894 ) const;
03895
03896 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
03897 void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
03898
03899 mxArray* toMx() const {
03900     mxArray *S=mxCreateStruct(1,1);
03901     add2MxStruct(S,0); return S;
03902 };
03903
03904 void put(const char *vname, const char *ws="caller") const
03905 { put(0,0,vname,ws); }
03906
03907 void put(const char *F, int L,
03908     const char *vname, const char *ws="caller"
03909 ) const {
03910     mxArray *a=toMx(); int i=mexPutVariable(ws,vname,a);
03911
03912     if (i) wblog(F,L,
03913         "ERR failed to write variable '%s' (%d)",vname,i);
03914     if (F) wblog(F,L,"I/O putting '%s' to %s",vname,ws);
03915     mxDestroyArray(a);
03916 };
03917
03918 QVec qvec;
03919
03920 wbMatrix<TQ> Q1,Q2,Q;
03921
03922 wbvector<widx_t> D;
03923
03924 wbvector<widx_t> I1, I2;
03925 wbMatrix<widx_t> II;
03926
03927 wbMatrix< const CRef<TQ>* > cg3;
03928
03929 };
03930
03931 template <class TQ>
03932 bool QMap<TQ>::isConsistent(const char *F, int L, const QVec *qv) const {
03933
03934     unsigned d=qvec.Qlen();
03935
03936     if (Q1.dim2!=d || Q2.dim2!=d) {
03937         if (!F) return 0; else wblog(F,L,
03938             "ERR QMap: invalid qset record length (%d,%d,%d)",
03939             Q1.dim2, Q1.dim2, d);
03940     }
03941     if (qv && (!qvec.sameType(*qv) || d!=(qv->Qlen()))) {
03942         if (!F) return 0; else wblog(F,L,
03943             "ERR QMap: Q type mismatch: %s; %s (%d/%d)",
03944             qv->toStr().data, STR(qvec), qv->Qlen(), d);
03945     }
03946     return 1;
03947 };
03948
03949 template <class T>
03950 class pairPatternCG {
03951 public:
03952
03953     pairPatternCG& init(unsigned i0=0, unsigned j0=0, unsigned n=0) {
03954         i=i0; j=j0; k.init(n); fac.init(n);
03955         return *this;
03956     };
03957
03958     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
03959     void add2MxStruct(mxArray *S, unsigned i) const;
03960
03961     unsigned i,j;
03962     wbvector<unsigned> k;
03963     wbvector<T> fac;
03964
03965 protected:
03966 private:
03967 };
03968
03969 template <class TQ, class TD>
03970 class genRG_base{
03971
03972 public:
03973

```

```

03974     genRG_base() : err(0) {};
03975
03976     bool isEmpty() const {
03977         return (!q && !Sp.len && !Sz.len && !J.len && Z.isEmpty());
03978     };
03979
03980     double normDiff(const char *F, int L,
03981         const genRG_base& B,
03982         TD eps=1E12
03983     ) const;
03984
03985     void checkOrthoCommRel(
03986         const char *F, int L, const genRG_base &B) const;
03987
03988     genRG_base& Sort();
03989     double SkipTiny(const char *F=0, int L=0);
03990
03991     genRG_base& ApplyQFac(
03992         const wvector<double> &qfac, const warray<double> *JM=NULL);
03993
03994     genRG_base& operator=(const genRG_base& S) {
03995         q=S.q; J=S.J; Z=S.Z; Sp=S.Sp; Sz=S.Sz;
03996         err=S.err; istr=S.istr;
03997         return *this;
03998     };
03999
04000     genRG_base& init() {
04001         q=0; J.init(); Z.init(); Sp.init(); Sz.init();
04002         err=0; istr.init();
04003         return *this;
04004     };
04005
04006     SPIDX_T dim() const {
04007         if (!Sz.len || Sz.len!=Sp.len) wblog(FL,
04008             "ERR %s() got empty generators for %s [%s]",
04009             FCT,STR(q),STR(J),Sz.len,Sp.len);
04010         SPIDX_T d=Sz[0].dim();
04011         if (d!=Sp[0].dim()) wblog(FL,"ERR %s() got "
04012             "inconsistent Sp/z dimensions (%d/%d)",FCT,Sp[0].dim(),d);
04013         return d;
04014     };
04015
04016     void compareStdSU2(const char *F=NULL, int L=0) const;
04017
04018     genRG_base& save2(genRG_base& S) {
04019         S.q=q; J.save2(S.J); Z.save2(S.Z); q=0;
04020         Sp.save2(S.Sp); Sz.save2(S.Sz);
04021 #ifdef CG_CHECK_MW_PERM
04022         P0.save2(S.P0);
04023 #endif
04024         S.err=err; istr.save2(S.istr);
04025         return S;
04026     };
04027
04028     wstring info() const {
04029         unsigned n=128; char s[n]; char sep[2]=" ";
04030         if (!q.isAbelian() && J.wvector<TQ>::allIn(0,9)) sep[0]=0;
04031         unsigned l=snprintf(s,n,"%s [%s]",
04032             q.toStr().data, J.toStrf(" ",sep,q.qlen(),";").data);
04033         if (l>=n) wblog(FL,
04034             "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
04035         return s;
04036     };
04037
04038     mxArray* toMx() const {
04039         mxArray *a=mxCreateStruct(1,1);
04040         add2MxStruct(a,0);
04041         return a;
04042     };
04043
04044     size_t memSize() const {
04045         size_t mtot=sizeof(QType) + J.len*sizeof(TQ);
04046         unsigned i=0;
04047         for ( ; i<Sp.len; ++i) mtot+=Sp[i].memSize();
04048         for (i=0; i<Sz.len; ++i) mtot+=Sz[i].memSize();
04049         return mtot;
04050     };
04051
04052     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
04053     void add2MxStruct(mxArray *S, unsigned i) const;
04054
04055     QType q;
04056
04057     wvector< wSparrayTD > Sp;
04058     wvector< wSparrayTD > Sz;
04059
04060     qset<TQ> J;

```

```

04061
04062     wbMatrix<double> Z;
04063
04064 #ifdef CG_CHECK_MW_PERM
04065     wbperm P0;
04066 #endif
04067
04068     double err;
04069     wstring istr;
04070
04071     protected:
04072     private:
04073 };
04074
04075 template <class TQ, class TD>
04076 class genRG_struct {
04077 public:
04078
04079     genRG_struct() {}
04080
04081     genRG_struct& checkInit(const char *F, int L, const QType &q0) {
04082         if (q0.isNonAbelian()) {
04083             unsigned n=RSet.size();
04084             if (n>0) {
04085                 if (!q0.type) wblog(FL,"ERR %s() got %d elements "
04086                     "in buf for '%s'",FCT,n,STR(q));
04087                 return *this;
04088             }
04089             SetupSym(F,L,q0);
04090         }
04091         return *this;
04092     };
04093
04094     genRG_struct& SetupSym(
04095         const char *F, int L, const QType &q0, qset<TQ> *qs=NULL);
04096
04097     genRG_struct& Setup_SUN(const char *F, int L, qset<TQ> *qs=NULL);
04098     genRG_struct& Setup_SpN(const char *F, int L, qset<TQ> *qs=NULL);
04099     genRG_struct& Setup_SON(const char *F, int L, qset<TQ> *qs=NULL);
04100     genRG_struct& Setup_SEN(const char *F, int L, qset<TQ> *qs=NULL);
04101
04102     void initSU2(const qset<TQ> &J);
04103
04104     void initCommRel(const char *F, int L,
04105         const genRG_base<TQ,TD> &R, char vflag=0);
04106
04107     double checkCommRel(
04108         const char *F, int L, const genRG_base<TQ,TD> &R
04109     ) const;
04110
04111     void printStatus(const char *F, int L, const char *istr=NULL);
04112
04113     int getIJ_gen(const char *F, int L, const qset<TQ> &q, char flag=0);
04114
04115     int getTensorProdReps_gen(
04116         const qset<TQ> &q1, const qset<TQ> &q2,
04117         unsigned flag=TP3_LDM);
04118
04119     void getTensorProdReps(
04120         const qset<TQ> &q1, const qset<TQ> &q2
04121     ){
04122         q.validType(FL);
04123         int n=getTensorProdReps_gen(q1,q2,TP3_TST);
04124         if (n>=0) return;
04125
04126         wblog(FL,"ERR %s() "
04127             "failed to access RSets of input spaces (%d)",FCT,n);
04128     };
04129
04130
04131     unsigned genTensorProds(const qset<TQ> &q, unsigned flag=0);
04132     unsigned genTensorProds(unsigned dmax=-1, char sdig=0, char vflag=0);
04133
04134     void put(const char *vname, const char *ws="caller") const
04135     { put(0,0,vname,ws); }
04136
04137     mxArray* toMx(char bflag=1) const;
04138     void put(const char *F, int L,
04139         const char *vname, const char *ws="caller"
04140     ) const;
04141
04142     void wblog_RC_1st_build(
04143         const char *F, int L, const char *fct, int nrep) const {
04144
04145         if (CG_VERBOSE && !q.isSU2()) {
04146             if (q.sub>1) { PRINTF("\n"
04147                 "    RC_STORE %s() first-time build for %s\n"

```



```

04148         "    %d passes, please wait ...\n\n",fct,STR(q),nrep); }
04149     else wblog(FL,
04150         "NB! %s() RC_STORE first-time build for %s",fct,STR(q));
04151     }
04152 };
04153
04154 QType q;
04155
04156 map <qset<TQ>,
04157     genRG_base<TQ,TD>
04158 > RSet;
04159
04160 qset<TQ> qdef;
04161
04162 wbvector< pairPatternCG<double> > CR;
04163
04164 wbMatrix<double> DZ;
04165
04166 protected:
04167 private:
04168 };
04169
04170 template <class TQ>
04171 TQ getTensorProdReps_abelian(const QType &q, const TQ &J1, const TQ &J2) {
04172
04173     TQ J=0;
04174
04175     switch (q.type) {
04176     case QTYPE_U1: J=J1+J2; break;
04177
04178     case QTYPE_ZN: J=int(J1+J2)%q.sub;
04179
04180         if (TQ(int(J1)%q.sub)!=J1 || TQ(int(J2)%q.sub)!=J2)
04181             wblog(FL,"ERR %s() invalid %s labels (%g,%g)",
04182                 FCT, STR(q), double(J1), double(J2));
04183         break;
04184
04185     case QTYPE_P: J=J1*J2;
04186
04187         if (fabs(double(J1))!=1 || fabs(double(J2))!=1)
04188             wblog(FL,"ERR %s() invalid %s labels (%g,%g)",
04189                 FCT,STR(q),double(J1),double(J2));
04190         break;
04191
04192     default: wblog(FL,"ERR %s() got invalid symmetry %s",FCT,STR(q));
04193     }
04194
04195     return J;
04196 };
04197
04198 template <class TQ, class TD>
04199 class RStore {
04200 public:
04201
04202     RStore(const char *s=NULL) {
04203         if (s) {
04204             if (!strcmp(s,"info")) {
04205                 #ifdef __WB_MPFR_HH__
04206                 wblog(FL,"<i> %s() uses type %s (=mpfr@%d) ",
04207                     FCT, TSTR(TD), TD().prec());
04208                 #else
04209                 wblog(FL,"<i> %s() uses type %s",FCT,TSTR(TD));
04210                 #endif
04211             }
04212             else wblog(FL,"ERR %s() invalid usage (%s)",FCT,TSTR(TD));
04213         }
04214     };
04215
04216     size_t bufsize(const QType &q) {
04217         auto it=buf.find(q);
04218         return (it==buf.end() ? 0 : it->second.RSet.size());
04219     };
04220
04221     bool isEmpty(const QType &q) {
04222         return (bufsize(q)>0 ? 1 : 0);
04223     };
04224
04225     genRG_struct<TQ,TD>& Buf(const QType &q) {
04226         auto it=buf.find(q);
04227         if (it!=buf.end()) {
04228             genRG_struct<TQ,TD> &B=(it->second);
04229             if (!B.q.isKnown()) {
04230                 if (B.RSet.size()) wblog(FL,
04231                     "ERR %s() got RSet with unknown qtype (len=%d; %s)",
04232                     FCT,B.RSet.size(),STR(B.q));
04233                 B.checkInit(FL,q);
04234             }

```

```

04235     }
04236     return B;
04237 }
04238 else {
04239     genRG_struct<TQ,TD> &B=buf[q];
04240     B.checkInit(FL,q);
04241     return B;
04242 }
04243 };
04244
04245 genRG_base<TQ,TD>* find_RSet(const QType &q, const TQ *qptr){
04246     genRG_struct<TQ,TD> &B=Buf(q);
04247     qset<TQ> qs(q.qlen(),qptr,'r');
04248
04249     auto it = B.RSet.find(qs);
04250     return (it!=B.RSet.end() ? &(it->second) : NULL);
04251 };
04252
04253 genRG_struct<TQ,TD>& getRSet(
04254     const char *F, int L, const QType &q,
04255     qset<TQ> *qvec=NULL
04256 ){
04257     genRG_struct<TQ,TD> &B=Buf(q);
04258     if (qvec) { (*qvec)=B.qdef; }
04259     return B;
04260 };
04261
04262 genRG_base<TQ,TD>& getR(
04263     const char *F, int L, const QType &qtype, const TQ *q) const;
04264
04265 genRG_base<TQ,TD>& getR(
04266     const char *F, int L, const CDATA_TQ &S,
04267     unsigned k
04268 ) const { return getR(F_L, S.t, S.qptr(k)); };
04269
04270 wbMatrix<TQ> getZ(
04271     const char *F, int L, const QType &qtype, const TQ *q
04272 ) const {
04273     const genRG_base<TQ,TD> &R = getR(F_L,qtype,q);
04274     return R.Z;
04275 };
04276
04277 unsigned qdim(
04278     const char *F, int L, const QType &qtype, const TQ *q
04279 ) const {
04280
04281     if (qtype.isAbelian()) return 1;
04282     const genRG_base<TQ,TD> &R = getR(F_L,qtype,q);
04283
04284     if (qtype.isSU2() && TQ(R.Z.dim1)!=q[0]+1) wblog(FL,"ERR %s() "
04285         "got multiplet dim mismatch (%d/%d)",FCT,R.Z.dim1,int(q[0]+1));
04286
04287     return R.Z.dim1;
04288 };
04289
04290 mxArray* toMx(const QType &q=QTYPE_UNKNOWN, char bflag=1) const;
04291
04292 unsigned LoadStore(const char *F, int L, const char *file);
04293 unsigned add(const QType &q, const mxArray* S, unsigned k);
04294
04295 void Info(const char *F=0, int L=0) const;
04296
04297 map <QType,
04298     genRG_struct<TQ,TD>
04299 > buf;
04300
04301 protected:
04302 private:
04303 };
04304
04305 RStore<gTQ,RTD> gRS;
04306
04307 #ifdef QS_USING_OMP
04308     Wb::ompNLock RCS_lock("RCStore");
04309 #endif
04310
04311 map<QType,int> g_sym;
04312
04313 class RCStore {
04314
04315 public:
04316
04317     RCStore() { };
04318
04319     int setupDirs(const char *F=NULL, int L=0) {
04320
04321         if (root.idx.len) { return -root.idx.len; }

```

```

04322
04323     #ifdef QS_USING_OMP
04324         Wb::ompGuard dLK(RCS_lock);
04325         if (root.idx.len) { return -root.idx.len; }
04326     #endif
04327
04328     int i1,i2=0,e=0;
04329
04330     const char rcs[]="RC_STORE", rsy[]="RC_SYNC";
04331
04332     i1=setup_root_directory(F_L,rcs,root);
04333     if (!i1) wblog(FL,"ERR missing env variable '%s'",rcs);
04334
04335     i2=setup_root_directory(F_L,rsy,root,'+');
04336
04337     if (i1<0 || i2<0) { root.init(); wblog(FL,
04338         "ERR invalid environmental variables RC_STORE%s",
04339         i2<0 ? " or RC_SYNC":"" ); }
04340     if (i2>1) { root.init(); wblog(FL,
04341         "ERR %s() only one RC_SYNC directory allowed",FCT); }
04342
04343     for (auto it=root.begin(); it.end(); ++it) {
04344         if (!strcmp(it.at(),"/home/",7)) { e=1;
04345             wblog(FL,"WRN %s() RC_%s points to HOME (%d/%d)\n"
04346                 "(recommended to use /data or similar, instead)",
04347                 FCT, !it.i || it.i+1<it.n?"STORE":"SYNC", it.i+1,it.n);
04348             root.print("RC_STORE+SYNC"); break;
04349         }
04350     }
04351     if (!e && CG_VERBOSE>6) { root.print("RC_STORE+SYNC"); }
04352
04353     if (!i2 && i1) {
04354         root.append(FL,root.last(),":");
04355     }
04356     return root.idx.len;
04357 };
04358
04359 template <class TQ, class TD>
04360 int save_CData(const char *F, int L, const CData<TQ,TD> &A);
04361
04362 template <class TQ, class TD>
04363 int load_CData(
04364     const char *F, int L, const QSet<TQ> &Q, CData<TQ,TD> &A,
04365     char bflag=0);
04366
04367 template <class TQ>
04368 int save_Std3(
04369     const char *F, int L, const QType &t, const qset<TQ> &J12,
04370     const wbvector<double> *c2eps=NULL) const;
04371
04372 template <class TQ>
04373 int load_Std3(const char *F, int L,
04374     const QType &t, const qset<TQ> &J1, const qset<TQ> &J2,
04375     unsigned loadRC=0) const;
04376
04377 template <class TQ, class TD>
04378 int save_RSet(
04379     const char *F, int L, const genRG_base<TQ,TD> &R, char xflag=0);
04380
04381 template <class TQ>
04382 int load_RSet(
04383     const char *F, int L, const QType &t, const qset<TQ> &J,
04384     char vflag=0);
04385
04386 int save_XMap(
04387     const char *F, int L, const cgc_contract_id<MTI> &idc) const;
04388
04389 int load_XMap(
04390     const char *F, int L, const cgc_contract_id<MTI> &idc) const;
04391
04392 template <class TQ>
04393 int get_file_name(
04394     const char *F, int L, wstring &file,
04395     const QSet<TQ> &Q, const char *ext=NULL, char mflag=0) const;
04396
04397 template <class TQ, class TD>
04398 int get_file_name(
04399     const char *F, int L, wstring &file,
04400     const genRG_base<TQ,TD> &R, const char *ext=NULL, char mflag=0
04401 ) const;
04402
04403 int get_file_name(
04404     const char *F, int L, wstring &file,
04405     const cgc_contract_id<MTI> &idc, const char *ext=NULL, char mflag=0,
04406     QType *t=NULL
04407 ) const;
04408

```

```

04409     size_t rclog(const QType &t,
04410                 const char *F, int L, const char *fmt, ...);
04411
04412     size_t rclog(const QType &t,
04413                 const char *F, int L, char vflag, const char *fmt, ...);
04414
04415     size_t vrclog(const QType &t,
04416                  const char *F, int L, const char *fmt, va_list args);
04417
04418     void Info(const char *F=0, int L=0) {
04419         if (F) wblog(F,L," * %s()\n",FCT);
04420         root.print("RCStore::root");
04421         fprintf(stdout,"\n\n");
04422     }
04423
04424     int is_new(const QType &t) {
04425         return (g_sym[t] ? 0 : 1);
04426     };
04427
04428     Wb::Path root;
04429
04430 protected:
04431 private:
04432
04433     int get_rcs_path(
04434         const char *F, int L, char *path, unsigned len, const QType &t,
04435         const char *sub1=NULL, const char *sub2=NULL, const char *file=NULL,
04436         char mflag=0
04437     ) const;
04438
04439     int setup_root_directory(
04440         const char *F, int L, const char *env, Wb::Path &root,
04441         char append=0
04442     );
04443 };
04444
04445     RCStore gStore;
04446
04447     Wb::IOstat gstatR,gstatM,gstatC,gstatX;
04448
04449     #define RC_MDIR 1
04450     #define RC_SYNC 2
04451     #define RC_CURR 4
04452
04453     #define RC_SAVE (RC_MDIR|RC_CURR)
04454     #define RC_SAVE_S (RC_MDIR|RC_SYNC)
04455
04456     #define RCL_V1 1
04457     #define RCL_V2 2
04458
04459     #define RCL_ALL (RCL_V1|RCL_V2)
04460
04461 int RCStore::setup_root_directory(
04462     const char *F, int L, const char *env, Wb::Path &root, char append
04463 ){
04464     int m=0, k=0, e=0, kmax;
04465     const char *sval;
04466
04467     if (!env || !env[0]) wblog(F,L,
04468         "ERR %s() no variable specified",FCT);
04469
04470     sval=getenv(env); if (!sval || !sval[0]) {
04471         if (!append) { root.init(); }
04472         return m;
04473     }
04474
04475     if (!append)
04476         { m=root.init(F,L,sval,"",1024); }
04477     else { k=root.numel(); m=root.append(F,L,sval,""); }
04478
04479     for (kmax=k+m; k<kmax; ++k) { char *dir=root[k];
04480
04481         if (!dir || !dir[0]) {
04482             if (!e) { PRINTF("\n"); }; e|=1;
04483             PRINTF(" ERR invalid %s entry (empty)\n",env);
04484         }
04485         if (Wb::fexist(dir,'d')) { continue; }
04486
04487         if (k+1<kmax) {
04488             if (!e) { PRINTF("\n"); }; e|=2;
04489             PRINTF(" ERR invalid %s entry (%s) ^1\n",env,dir);
04490         }
04491
04492         unsigned n=strlen(dir), i=n-1, i0=0; int q;
04493
04494         for (; i<n; --i) { if (dir[i]=='/') { i0=i; break; }}
04495         if (i0 && (

```

```

04496         !strncasecmp(dir+i0+1,"RCStore",7) ||
04497         !strncasecmp(dir+i0+1,"RCSync", 6))
04498     ){
04499         dir[i0]=0; q=Wb::fexist(dir,'d');
04500         dir[i0]='/' ;
04501         if (q) {
04502             mode_t p=(S_IRWXU | S_IRWXG | S_IROTH | S_IXOTH);
04503             if ((q=mkdir(dir,p))==0) {
04504                 PRINTF("    creating %-8s root directory (%s)\n",
04505                     env, Wb::repHome(dir).data);
04506             }
04507             else {
04508                 if (!e) { PRINTF("\n"); }; e|=4;
04509                 PRINTF("    ERR invalid %s entry (%s) ^2\n",env,dir);
04510             }
04511         }
04512         else {
04513             if (!e) { PRINTF("\n"); }; e|=8;
04514             PRINTF("    ERR invalid %s entry (%s) ^3\n",env,dir);
04515         }
04516     }
04517     else if (!Wb::fexist(dir,'d')) {
04518         if (!e) { PRINTF("\n"); }; e|=16;
04519         PRINTF("    ERR invalid %s entry (%s) ^4\n",env,dir);
04520     }
04521 }
04522 if (e) { PRINTF("\n");
04523     if (e& 2) { PRINTF("    ^1 all but last entry must exist\n"); }
04524     if (e& 4) { PRINTF("    ^2 failed to create directory\n"); }
04525     if (e& 8) { PRINTF("    ^3 invalid %sdirectory\n",kmax>1?"last ":""); }
04526     if (e&16) { PRINTF("    ^4 %sdirectory must also exist if not named %s*\n",
04527         strncmp(env,"RC_SYNC",8) ? "RCStore":"RCSync", kmax>1?"last ":""); }
04528     if (e> 1) { PRINTF("\n"); }
04529 }
04530
04531 if (e) {
04532     root.init();
04533     wblog(FL,"ERR invalid env %s directory (e=%d)",env,e);
04534 }
04535
04536 return (e ? -m : m);
04537 };
04538
04539 namespace CG {
04540
04541 template <class TQ, class TD>
04542 double getSymmetryStates(const char *F, int L, const QType &q,
04543     const wbvector< wbSpararrayTD > &Sp,
04544     const wbvector< wbSpararrayTD > &Sz,
04545     wbvector< wbSpararrayTD > &U,
04546     wbvector<unsigned> &dd,
04547     wbvector<genRG_base<TQ,TD> > &RR,
04548     wbvector<double> &c2eps,
04549     wbMatrix<unsigned> *iOM=NULL,
04550     char vflag=1
04551 );
04552
04553 template <class TQ>
04554 SPIDX_T findMaxWeight(
04555     const QType &q, const wbMatrix<double> &Z, qset<TQ> *J=NULL, wbperm *P_=NULL
04556 );
04557
04558 template <class TQ, class TD>
04559 void get_SU2mat(TQ s2,
04560     wbvector<double> &sz,
04561     wbsparray<TD> &Sp, wbsparray<TD> &Sz,
04562     wbsparray<TD> &S2, wbsparray<TD> &E
04563 );
04564
04565 int isQSet (const mxArray *S, unsigned k=-1);
04566 int isCData(const mxArray *S, unsigned k=-1);
04567 int isCRef (const mxArray *S, unsigned k=-1);
04568
04569 int MemStat(const char *F=0, int L=0) {
04570     int n=0; const char *hstr="RCStore"; const char *xstr="/full";
04571
04572     wblog(F_L,"%N%s() %33R",FCT,"-");
04573     gRS.Info();
04574     gCS.Info();
04575     gXS.Info();
04576     Wb::MemStat(FL);
04577
04578     if (!gstatR.isEmpty()) gstatR.print("RData", (++n==1)? hstr:0, xstr);
04579     if (!gstatM.isEmpty()) gstatM.print("map3",   (++n==1)? hstr:0, xstr);
04580     if (!gstatC.isEmpty()) gstatC.print("CData",  (++n==1)? hstr:0, xstr);
04581     if (!gstatX.isEmpty()) gstatX.print("XData",  (++n==1)? hstr:0, xstr);
04582

```

```

04583     if (n) { PRINTF("\n"); }
04584     else wblog(FL,"I/O no data read or written yet to RCStore");
04585
04586     return 0;
04587 };
04588
04589 #ifdef QS_USING_OMP
04590
04591     map <size_t, Wb::ompNLock*> lock_map;
04592
04593     class Guard {
04594
04595     public:
04596
04597         Guard() : active(false) {};
04598
04599         template<class TQ>
04600         Guard(const char *F, int L, const QSet<TQ> &Q, const char *tag=NULL,
04601             const char *cond=NULL)
04602             : active(false) { acquire(F_L,Q,tag,cond); };
04603
04604         template<class TM>
04605         Guard(const char *F, int L, const cgc_contract_id<TM> &x,
04606             const char *cond=NULL)
04607             : active(false) { acquire(F_L,x,cond); };
04608
04609         template<class TQ>
04610         Guard(const char *F, int L, const QSet<TQ> &QA, const QSet<TQ> &QB)
04611             : active(false) { acquire(F_L,QA,QB); };
04612
04613         template<class TQ>
04614         Guard(const char *F, int L,
04615             const QSet<TQ> &QA, const QSet<TQ> &QB, const QSet<TQ> &QC)
04616             : active(false) { acquire(F_L,QA,QB,QC); };
04617
04618         ~Guard() {
04619             if (hid.len) { release(); hid.init(); }
04620             active=false;
04621         };
04622
04623         explicit operator bool() const { return (hid.len || active); }
04624         bool operator! () const { return (!hid.len && !active); }
04625
04626         int acquire(const char *F, int L,
04627             const FileLock &flk, const char *tag=NULL, const char *cond=NULL);
04628
04629         template<class TQ>
04630         int acquire(const char *F, int L,
04631             const QSet<TQ> &Q, const char *tag=NULL, const char *cond=NULL);
04632
04633         template<class TM>
04634         int acquire(const char *F, int L,
04635             const cgc_contract_id<TM> &x, const char *cond=NULL);
04636
04637         template<class TQ>
04638         void acquire(const char *F, int L, const QSet<TQ>* Q[], unsigned n);
04639
04640         template<class TQ>
04641         void acquire(const char *F, int L,
04642             const QSet<TQ> &QA, const QSet<TQ> &QB)
04643             { const QSet<TQ>* QQ[2]={ &QA, &QB }; acquire(F_L,QQ,2); };
04644
04645         template<class TQ>
04646         void acquire(const char *F, int L,
04647             const QSet<TQ> &QA, const QSet<TQ> &QB, const QSet<TQ> &QC)
04648             { const QSet<TQ>* QQ[3]={ &QA, &QB, &QC }; acquire(F_L,QQ,3); };
04649
04650         inline int acquire_set(const char *ss[], int ntry=-1) {
04651             if (!hid.len) { return -1; }
04652
04653             int got[hid.len];
04654             Wb::ompNLock *lk[hid.len];
04655
04656             int nlks=acquire_set_iter(got,lk,ss,0);
04657             if (nlks<=0) { if (ntry==1 || !ntry) return nlks;
04658                 nlks=acquire_set_wait(got,lk,ss,ntry);
04659             }
04660
04661             if (nlks<=0) {
04662                 if (ntry<=0 || ntry>99) wblog(FL,
04663                     "ERR %s() failed to qdquire locks (%d/%d)",FCT,nlks,hid.len);
04664                 return nlks;
04665             }
04666
04667             for (unsigned i=0; i<hid.len; ++i) {
04668                 lk[i]->set_istr(ss[i]);
04669                 if (*got < got[i]) { *got=got[i]; }

```

```

04670     }
04671     active=true;
04672
04673     return *got;
04674 };
04675
04676 int acquire_set_iter(
04677     int *got, Wb::ompNLock *lk[], const char *ss[], unsigned iter=0);
04678 int acquire_set_wait(
04679     int *got, Wb::ompNLock *lk[], const char *ss[], int ntry=-1);
04680
04681 void release();
04682
04683 int level() { int l=-1;
04684     for (unsigned i=0; i<hid.len; ++i) {
04685         const Wb::ompNLock *lp = Guard::find(hid[i]);
04686         if (lp && l<(lp->level)) { l=lp->level; }
04687     }
04688     return l;
04689 };
04690
04691 int used() { int l=0;
04692     for (unsigned i=0; i<hid.len; ++i) {
04693         const Wb::ompNLock *lp = Guard::find(hid[i]);
04694         if (lp && l<(lp->level)) {
04695             if (l< lp->level ) { l=lp->level; }
04696             if (l< lp->nactive) { l=lp->nactive; }
04697             if (l< lp->ntot ) { l=lp->ntot; }
04698         }
04699     }
04700     if (!l && active) { l=-1; }
04701     return l;
04702 };
04703
04704 void print(const char *F, int L);
04705
04706 wstring toStr() const;
04707
04708 static wstring toStr(size_t h, const char *istr=NULL);
04709 static int toStr(wstring &s, size_t h, const char *istr=NULL);
04710
04711 template<class TQ>
04712 static wstring Status(const QSet<TQ> &Q, const char *tag=NULL);
04713
04714 template<class TM>
04715 static wstring Status(const cgc_contract_id<TM> &x);
04716
04717 static wstring Status(size_t h1, const char *istr=NULL);
04718
04719 static Wb::ompNLock* find(size_t h);
04720
04721 static int print_CG_locks();
04722
04723 protected:
04724 private:
04725
04726     wvector<size_t> hid;
04727
04728     bool active;
04729
04730     const char* sprintf_locks(
04731         wstring &sout, const int* got, const char *istr[],
04732         const char *indent=NULL);
04733
04734     int deal_with_cond(const char *F, int L, size_t h0, const char *cond);
04735 };
04736
04737 #endif
04738
04739 class FileLock {
04740 public:
04741
04742     FileLock() : fid(0) {};
04743
04744     ~FileLock() {
04745         if (fid) { ReleaseLock(FL); }
04746     };
04747
04748     template<class TQ>
04749     FileLock(
04750         const QType &q_, const qset<TQ> &J1, const qset<TQ> &J2,
04751         const char *ext="mp3"
04752     ) { init(q,J1,J2,ext); }
04753
04754     template<class TQ>

```

```

04757     int init(
04758         const QType &q_, const qset<TQ> &J1, const qset<TQ> &J2,
04759         const char *ext="mp3"
04760     );
04761
04762     int GetLock(const char *F=NULL, int L=0, char flag='W') {
04763         return tryGetLock(F,L,flag);
04764     };
04765
04766     int ReleaseLock(const char *F=NULL, int L=0) {
04767         return tryGetLock(F,L,'u' );
04768     };
04769
04770     int ensureFOpen(const char *F=NULL, int L=0) const {
04771         int q=0;
04772         if (!fname) { q|=1; }
04773         if (fid<3 ) { q|=2; }
04774         if (q && F) {
04775             if (q&1) wblog(FL,"ERR %s() got empty fname (e=%d)",FCT,q);
04776             if (q&2) wblog(FL,"ERR %s() file not yet open "
04777                 "(fid=%d; e=%d)\n%s",FCT,fid, q,fname.data
04778             );
04779         }
04780         return q;
04781     };
04782
04783     int fid;
04784
04785     QType q;
04786     wstring fname;
04787
04788 #ifdef QS_USING_OMP
04789     CG::Guard fLK;
04790 #endif
04791
04792     protected:
04793     private:
04794
04795         int tryGetLock(const char *F, int L, char flag);
04796 };
04797
04798 };
04799
04800 template<class TD>
04801 int BuildKrylovH(
04802     const QType &q,
04803     wbararray<TD> &HK, unsigned nk,
04804     wbSparrayTD &X,
04805     wbvector<TD> &EK,
04806     const wbSparrayTD &HL,
04807     const wbSparrayTD &HR,
04808     const wbvector< wbSparrayTD > SL,
04809     const wbvector< wbSparrayTD > SR,
04810     double &xmin
04811 );
04812
04813 template<class TD>
04814 void GetHPsi(
04815     wbSparrayTD &HX, const wbSparrayTD &X,
04816     const wbSparrayTD &HL,
04817     const wbSparrayTD &HR,
04818     const wbvector< wbSparrayTD > SL,
04819     const wbvector< wbSparrayTD > SR
04820 );
04821
04822 #endif
04823

```

5.2 cpat.hh

```

00001 #ifndef __WB_CPAT_HCC__
00002 #define __WB_CPAT_HCC__
00003
00004 /* ----- *
00005  * class CPAT: class to store contraction pattern in QSpace structure
00006  * ----- *
00007  * contraction pattern (default: twoSite mode)
00008  *
00009  * deals with MatLab cell array of the form { A, i1, i2 [,opA] }
00010  * where A can be a single QSpace or a cell vector of these
00011  * i[12] specifies index onto which to contract onto Psi[12]
00012  * NB! either i1 or i2 is set, the other one must be empty, e.g. []
00013  *

```



```

00014  * single site mode: expects { A, ic, [,opA] } instead as
00015  * contraction pattern where A is always contracted onto "Psi1"
00016  *
00017  * ----- *
00018  * AW (C) Jun 2006
00019  * ----- */
00020
00021 #define CP1 "{ A, ic, [,opA] }"
00022 #define CP2 "{ A, il, i2 [,opA] }"
00023 #define _CP (twoSite ? "{ A, il, i2 [,opA] }" : "{ A, ic, [,opA] }")
00024
00025 template <class TQ, class TD>
00026 class CPAT_Set;
00027
00028 int isCPAT(const mxArray *C, char twoSite=-1);
00029 int isCPattern(const mxArray *C);
00030
00031 unsigned NR_ICTR_CALL=0;
00032
00033 template <class TQ, class TD>
00034 class CPAT {
00035
00036 public:
00037
00038     CPAT() : z(0), sameCtrIdx(0), nr_check_ctr(0), HPsi_counter(0) {};
00039
00040     CPAT(const CPAT<TQ,TD> &D) {
00041         wblog(FL,"ERR do not copy CPAT object!");
00042     };
00043
00044     void init(
00045         TD oz, const mxArray**, unsigned n,
00046         unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00047         const char twoSite, const char ldir
00048     );
00049
00050     int getCtrIndex(const QSpace<TQ,TD> &PSI);
00051
00052     QSpace<TQ,TD>& HZTimes(
00053         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &HZPsi,
00054         const char ='N', char disp=1
00055     );
00056
00057     QSpace<TQ,TD>& HZApplyDiag(
00058         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &EPsi,
00059         TD a, TD b, char iflag=0, char disp=1
00060     );
00061
00062     void check_contraction(const QSpace<TQ,TD> &PSI, char disp=1);
00063     void initCounter() {
00064         HPsi_counter=0;
00065     };
00066
00067     QSpace<TQ,TD>& HZTimes_tst(
00068         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &HZPsi,
00069         const char ='N', char disp=1);
00070
00071     void getQRange(
00072         unsigned k, wbMatrix<TQ> &QQ, wbvector<unsigned> &SS,
00073         QSpace<TQ,TD> *B=NULL
00074     ) const;
00075
00076     mxArray* toMx() const;
00077
00078     void put(const char *vname, const char* ws="base") const {
00079         mxPutAndDestroy(FL,toMx(),vname, ws);
00080     };
00081
00082     void PermIC(const wbperm &P);
00083
00084     wbvector< wbvector< CPAT_Set<TQ,TD> > > CP;
00085     TD z;
00086
00087     unsigned sameCtrIdx;
00088     unsigned nr_check_ctr;
00089     unsigned HPsi_counter;
00090
00091 protected:
00092 private:
00093 };
00094
00095 template <class TQ, class TD>
00096 class CPAT_Set {
00097
00098 public:
00099
00100     CPAT_Set () : opA(0), ic(0) {};

```

```

00101
00102 CPAT_Set (const CPAT_Set<TQ,TD> &D) : opA(0), ic(0) {
00103     wblog(FL,"WRN Do not copy CPAT_Set!");
00104 };
00105
00106 int init(
00107     const mxArray *a,
00108     unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00109     const char twoSite, const char ldir
00110 );
00111
00112 int getCtrIndex(const QSpace<TQ,TD> &PSI);
00113 int getCtrIndex(const wbvector< QSpace<TQ,TD> >&, const char ='N');
00114
00115 int getCtrIndex(const unsigned i, const QSpace<TQ,TD>&, char ='N');
00116
00117 void contract(
00118     const QSpace<TQ,TD> &PSI, wbvector< QSpace<TQ,TD> > &C,
00119     char='N') const;
00120
00121 void contract(
00122     const wbvector< QSpace<TQ,TD> > &PSI1, wbvector< QSpace<TQ,TD> > &C,
00123     char='N') const;
00124
00125 void contract(
00126     const unsigned k, const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &C,
00127     char='N') const;
00128
00129 void contract_diag(
00130     const QSpace<TQ,TD> &PSI, wbvector< QSpace<TQ,TD> > &C,
00131     TD a, TD b, char Iflag) const;
00132
00133 void contract_diag(
00134     const wbvector< QSpace<TQ,TD> > &PSI1, wbvector< QSpace<TQ,TD> > &C,
00135     TD a, TD b, char Iflag) const;
00136
00137 void contract_diag(
00138     const unsigned k, const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &C,
00139     TD a, TD b, char Iflag) const;
00140
00141 mxArray* toMx() const;
00142
00143 wbvector< QSpace<TQ,TD> > A;
00144 char opA;
00145 unsigned ic;
00146
00147 wbperm P;
00148
00149 wbvector< QSpace<TQ,TD> > CQn, CQt;
00150 wbvector< wbMatrix<unsigned> > CIn, CIt;
00151
00152 protected:
00153 private:
00154 };
00155
00156 template <class TQ, class TD>
00157 void CPAT<TQ,TD>::init(
00158     TD oz, const mxArray** aa, unsigned N,
00159     unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00160     const char twoSite, const char ldir
00161 ){
00162     unsigned i,j,e,k,p,m=0,n;
00163     wbvector<unsigned> dd;
00164     mxArray *a;
00165
00166     if (ic1>=r1 || ic2>=r2) wblog(FL,
00167         "ERR Contraction index out of bounds (%d/%d; %d/%d (%d))",
00168         ic1+1, r1, ic2+1, r2, twoSite
00169     );
00170
00171     z=oz;
00172
00173     CP.initDef(N);
00174
00175     for (p=k=0; k<N; k++) {
00176         if (!mxIsCell(aa[k]))
00177             wblog(FL, "ERR Invalid (pair) pattern %s", _CP);
00178
00179         n=isCPAT(aa[k], twoSite);
00180
00181         if (!n) wbdie(FL,str);
00182
00183         CP[p].initDef(n);
00184
00185         if (n==1) {
00186             e=CP[p][0].init(aa[k],ic1,r1,ic2,r2,twoSite,ldir);
00187             if (e) wbdie(FL,str);

```

```

00188         m=CP[p][0].A.len;
00189     }
00190     else {
00191         for (i=0; i<n; i++) {
00192             a=mxGetCell(aa[k],i);
00193             e=CP[p][i].init(a,ic1,r1,ic2,r2,twoSite,ldir);
00194
00195             if (e) {
00196                 printf("\n%s:%d contraction pattern %d:%d\n", FL, k+1, i+1);
00197                 printf("%s\n",str); ExitMsg("");
00198             }
00199
00200             if (i>0) { if (m!=CP[p][i].A.len) wblog(FL,
00201                 "ERR Pattern pair dimension mismatch in A (%d.%d: %d,%d)",
00202                 k+1, i, CP[p][0].A.len, CP[p][i].A.len); }
00203             else m=CP[p][0].A.len;
00204         }
00205     }
00206
00207     dd.init(m);
00208     for (i=0; i<n; i++) {
00209         const wvector< QSpace<TQ,TD> > &Aki=CP[p][i].A;
00210         for (j=0; j<m; j++) if (!Aki[j].isEmpty()) dd[j]++;
00211     }
00212
00213     for (j=0; j<m; j++) if (dd[j]==n) { p++; break; }
00214 }
00215
00216 if (p<N) {
00217     if (p==0) wblog(FL,"WRN eigs: no relevant contraction pattern!");
00218
00219     CP.len=p;
00220 }
00221
00222 NR_ICTR_CALL=0;
00223 }
00224
00225 template <class TQ, class TD>
00226 int CPAT_Set<TQ,TD>::init(
00227     const mxArray *C,
00228     unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00229     const char twoSite, const char ldir
00230 ){
00231
00232     unsigned k,m,w, ra, ica, ic0, N=mxGetNumberOfElements(C);
00233     const char isC=(typeid(TD)==typeid(wbcomplex) ? 'C' : 0);
00234     int r=-1;
00235
00236     mxArray *a; str[0]=0;
00237
00238     if (ic1>=r1 || ic2>=r2) {
00239         sprintf(str, "%s:%d ERR Invalid input (%d/%d; %d/%d (%d))",
00240             FL, ic1+1, r1, ic2+1, r2, twoSite);
00241         return 1;
00242     }
00243
00244     m=isCPattern(C);
00245
00246     if ((char)m!=twoSite+1) wblog(FL,
00247         "ERR invalid input (not a %d-site CPattern; %d,%d)", twoSite+1,m);
00248
00249     a=mxGetCell(C,0);
00250
00251     if (mxIsQSpace(0,0,a,r,isC)>0) {
00252         A.initDef(1);
00253         A[0].init(FL,a,'r');
00254
00255         if (!A[0].isConsistent_r(2)) {
00256             sprintf(str,"%s:%d rank-2 tensor required (arg #1; %s)",FL,_CP);
00257             return 1;
00258         }
00259     }
00260     else {
00261         if (mxIsQSpaceVec(a,2,isC)) { sprintf(str,
00262             "%s:%d rank-2 QSpace (vector) required (arg #1; %s)",FL,_CP);
00263             return 1;
00264         }
00265         mxInitQSpaceVec(FL,a,A,'r');
00266     }
00267
00268     if (twoSite) {
00269         w = (mxIsEmpty(mxGetCell(C,1)) ? 0 : 1) +
00270             (mxIsEmpty(mxGetCell(C,2)) ? 0 : 2);
00271         k=w;
00272
00273         if (w<1 || w>2) {
00274             sprintf(str,"%s:%d Either arg #2 or #3 in CPAT %s",FL,_CP);

```

```

00275         return 1;
00276     }
00277 }
00278 else {
00279     k=1; w=(ldir>0 ? 1 : 2);
00280
00281     if (mxIsEmpty(mxGetCell(C,k))) {
00282         sprintf(str,"%s:%d Invalid arg #%d in CPAT %s",FL,k,CP1);
00283         return 1;
00284     }
00285 }
00286
00287 ra = (w==1 ? r1 : r2);
00288 ica = (w==1 ? ic1 : ic2);
00289
00290 a=mxGetCell(C,k);
00291
00292 if (mxGetNumber(a,ic)) { sprintf(str+strlen(str),
00293     "\n%s:%d Invalid arg #%d in CPAT %s", FL, k+1, _CP);
00294     return 1;
00295 }
00296
00297 if (!ic || ic>ra) { sprintf(str+strlen(str),
00298     "\n%s contraction index out of bounds (%d/%d; %d,%d) in CPAT %s",
00299     SHORT_FL, ic, ra,r1,r2,_CP); return 1;
00300 }
00301
00302 ic0=ic;
00303 ic--;
00304
00305 if (twoSite && ic==ica) { sprintf(str+strlen(str),
00306     "\n%s Index %d overlaps with ctr index in CPAT %s",
00307     SHORT_FL, ic0, _CP); return 1;
00308 }
00309
00310 if (twoSite) {
00311     if (ic>ica) ic--;
00312     if (ldir>0 && w==2) ic+=(r1-1); else
00313     if (ldir<0 && w==1) ic+=(r2-1);
00314 }
00315 else {
00316     if (ic>ica) ic--; else
00317     if (ic==ica) ic=ra-1;
00318 }
00319
00320 opA=0; k=(twoSite ? 3 : 2);
00321
00322 if (k<N) {
00323     a=mxGetCell(C,k);
00324
00325     if (mxGetNumber(a,opA)) { sprintf(str,
00326         "%s:%d Invalid arg #%d in CPAT %s", FL, 3+twoSite, _CP);
00327         return 1;
00328     }
00329     if (N>k+1) { sprintf(str,
00330         "%s:%d Too many arguments for CPAT %s", FL, _CP);
00331         return 1;
00332     }
00333 }
00334
00335 opA=toupper(opA); if (!opA || opA==' ' || opA=='N')
00336
00337 if (!Wb::strchr("NIC*",opA)) { sprintf(str,
00338     "%s:%d ERR invalid opA=%c<%d> in CPAT %s", FL, opA, opA, _CP);
00339     return 1;
00340 }
00341
00342 if (opA!='N') {
00343     char tflag= (opA=='T' || opA=='C'),
00344             cflag= (opA=='C' || opA=='*');
00345     wperm P("2 1");
00346
00347     for (unsigned i=0; i<A.len; i++) {
00348         if (A[i].isEmpty()) continue;
00349         if (tflag) A[i].Permute(P);
00350         if (cflag) A[i].Conj();
00351     }
00352 }
00353
00354 return 0;
00355 }
00356
00357 template <class TQ, class TD>
00358 void CPAT<TQ,TD>::PermIC(const wperm &P) {
00359     unsigned i,j, m=CP.len, n;
00360     wperm iP;
00361 }

```

```

00362     P.invert(iP);
00363
00364     for (i=0; i<m; i++) {
00365         for (n=CP[i].len, j=0; j<n; j++)
00366             CP[i][j].ic = iP[CP[i][j].ic];
00367     }
00368 }
00369
00370 template <class TQ, class TD>
00371 int CPAT<TQ,TD>::getCtrIndex(const QSpace<TQ,TD> &PSI) {
00372
00373     unsigned i,j,n; int m=0;
00374     const QSpace<TQ,TD> PSI2;
00375
00376     if ((++NR_ICTR_CALL)%20==0) wblog(FL,
00377         "%NWRN calling %s already %d times (?)", FCT, NR_ICTR_CALL);
00378
00379     for (i=0; i<CP.len; i++) {
00380         m += CP[i][0].getCtrIndex(PSI);
00381
00382         for (n=CP[i].len, j=1; j<n; j++) {
00383             if (CP[i][0].A.len!=CP[i][j].A.len) wblog(FL,
00384                 "ERR Severe dimension mismatch (%d: %d,%d)", j,
00385                 CP[i][0].A.len, CP[i][j].A.len);
00386
00387             m += CP[i][j].getCtrIndex(CP[i][j-1].CQn, 'N');
00388             m += CP[i][j].getCtrIndex(CP[i][j-1].CQt, 'T');
00389         }
00390     }
00391
00392     return m;
00393 }
00394
00395 template <class TQ, class TD>
00396 int CPAT_Set<TQ,TD>::getCtrIndex(const QSpace<TQ,TD> &PSI) {
00397
00398     unsigned i,m=0,n=A.len;
00399
00400     if (CIn.len!=n) { CIn.initDef(n); m++; }
00401     if (CIt.len!=n) { CIt.initDef(n); m++; }
00402     if (CQn.len!=n) { CQn.initDef(n); m++; }
00403     if (CQt.len!=n) { CQt.initDef(n); m++; }
00404
00405     if (A.len==0) return (m!=0);
00406
00407     P.Index(PSI.rank());
00408     P.mvFirstTo(ic);
00409
00410     for (i=0; i<A.len; i++) {
00411         m += getCtrIndex(i,PSI, 'N');
00412         m += getCtrIndex(i,PSI, 'T');
00413     }
00414
00415     return (m!=0);
00416 }
00417
00418 template <class TQ, class TD>
00419 int CPAT_Set<TQ,TD>::getCtrIndex(
00420     const wvector< QSpace<TQ,TD> > &PSI1,
00421     const char tflag
00422 ){
00423     unsigned i, m=0, n=A.len;
00424
00425     if (!Wb::strchri("NT",tflag)) wblog(FL,
00426         "ERR Invalid flag %c<%d>",tflag,tflag);
00427
00428     if (tflag=='N') {
00429         if (CIn.len!=n) { CIn.initDef(n); m++; }
00430         if (CQn.len!=n) { CQn.initDef(n); m++; }
00431     }
00432     else {
00433         if (CIt.len!=n) { CIt.initDef(n); m++; }
00434         if (CQt.len!=n) { CQt.initDef(n); m++; }
00435     }
00436
00437     if (n==0) return (m!=0);
00438
00439     if (n!=PSI1.len) wblog(FL,
00440         "ERR Severe dimension mismatch (%d,%d)", n, PSI1.len);
00441
00442     for (i=1; i<n; i++) if (PSI1[i].QDIM!=PSI1[0].QDIM)
00443         wblog(FL, "ERR Severe QDIM inconsistency (%d,%d).",
00444             PSI1[i].QDIM, PSI1[0].QDIM);
00445
00446     P.Index(PSI1[0].rank());
00447     P.mvFirstTo(ic);
00448

```

```

00449     for (i=0; i<n; i++)
00450     m += getCtrIndex(i, PSI[i], tflag);
00451
00452     return (m!=0);
00453 }
00454
00455 template <class TQ, class TD>
00456 int CPAT_Set<TQ,TD>::getCtrIndex(
00457     const unsigned i,
00458     const QSpace<TQ,TD> &PSI,
00459     char tflag
00460 ){
00461     int e;
00462     unsigned ica = tflag=='N' ? 2 : (tflag=='T' ? 1 : 0);
00463
00464     QSpace<TQ,TD> &CQi = (tflag=='N' ? CQn[i] : CQt[i]);
00465     wbMatrix<unsigned> &CIi = (tflag=='N' ? CIn[i] : CIt[i]);
00466
00467     QSpace<TQ,TD> CQ0;
00468     const wbMatrix<unsigned> CI0(CIi);
00469
00470     if (!ica) wblog(FL, "ERR Invalid flag %c<%d>",tflag,tflag);
00471
00472     CQ0.QDIM=CQi.QDIM; CQi.QDIM=PSI.QDIM;
00473     CQ0.QIDX=CQi.QIDX;
00474
00475     e=A[i].contract_getIdxSet(ica, PSI, ic+1, CQi.QIDX, CIi);
00476     if (e<0) wbdie(FL,str);
00477
00478     CQi.PermuteQ(P);
00479
00480     return (CQi.QDIM!=CQ0.QDIM || CQi.QIDX!=CQ0.QIDX || CIi!=CI0);
00481 }
00482
00483 template <class TQ, class TD>
00484 QSpace<TQ,TD>& CPAT<TQ,TD>::HZTimes(
00485     const QSpace<TQ,TD> &PSI,
00486     QSpace<TQ,TD> &HZPsi,
00487     const char tflag,
00488     char disp
00489 ){
00490     unsigned i,j,n;
00491
00492     #ifdef WB_CLOCK
00493     Wb::Clock clk("cpat:HZTimes",0);
00494     #endif
00495
00496     wbvector< QSpace<TQ,TD> > X1,X2;
00497
00498     if (!Wb::strchri("NTC",tflag))
00499     wblog(FL,"ERR Invalid flag %c<%d>", tflag, tflag);
00500
00501     check_contraction(PSI,disp);
00502     HPsi_counter++;
00503
00504     HZPsi=PSI;
00505     HZPsi *= (tflag=='C' ? -CONJ(z) : -z);
00506
00507     for (i=0; i<CP.len; i++) {
00508
00509         CP[i][0].contract(PSI, X1, tflag);
00510
00511         for (n=CP[i].len, j=1; j<n; j++) {
00512             X1.save2(X2);
00513             CP[i][j].contract(X2,X1,tflag);
00514         }
00515
00516         for (j=0; j<X1.len; j++)
00517             X1[j].Append2AndDestroy(FL,HZPsi);
00518     }
00519
00520     HZPsi.makeUnique();
00521
00522     return HZPsi;
00523 };
00524
00525 template <class TQ, class TD>
00526 QSpace<TQ,TD>& CPAT<TQ,TD>::HZApplyDiag(
00527     const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &EPsi,
00528     TD a, TD b, char Iflag, char disp
00529 ){
00530     unsigned i,j,n;
00531
00532     wbvector< QSpace<TQ,TD> > X1,X2;
00533
00534     check_contraction(PSI,disp);
00535

```

```

00536     EPsi.init();
00537
00538     for (i=0; i<CP.len; i++) {
00539         CP[i][0].contract_diag(Psi, X1, a,b,Iflag);
00540
00541         for (n=CP[i].len, j=1; j<n; j++) {
00542             X1.save2(X2);
00543             CP[i][j].contract_diag(X2,X1, a,b,Iflag);
00544         }
00545
00546         for (j=0; j<X1.len; j++) {
00547             X1[j].SkipEmptyData();
00548             X1[j].Append2AndDestroy(FL,EPsi);
00549         }
00550     }
00551 }
00552
00553 EPsi.makeUnique();
00554
00555 if (Psi.QIDX!=EPsi.QIDX) {
00556     wblog(FL,
00557         "WRN QIDX different for Psi and EPsi !? (%d/%d)",
00558         EPsi.QIDX.dim1,Psi.QIDX.dim1);
00559     wblog(FL,"TST a=%g, b=%g, '%c'",a,b,Iflag);
00560 #ifdef MATLAB_MEX_FILE
00561     Psi.put("Psi"); EPsi.put("EPsi");
00562 #endif
00563 }
00564
00565 return EPsi;
00566 };
00567
00568 template <class TQ, class TD>
00569 void CPAT<TQ,TD>::check_contraction(
00570     const QSpace<TQ,TD> &Psi, char disp
00571 ) {
00572     if (!nr_check_ctr) getCtrIndex(Psi); else
00573     if (sameCtrIdx<nr_check_ctr) {
00574         unsigned i=getCtrIndex(Psi);
00575         if (i==0) {
00576             sameCtrIdx++;
00577
00578             if (sameCtrIdx==nr_check_ctr && disp>10)
00579                 wblog(FL,"<i> using sameCtrIdx=%d", sameCtrIdx);
00580         }
00581     } else {
00582         if (sameCtrIdx && disp)
00583             wblog(FL,"WRN sameCtrIdx=%d reset to 0.", sameCtrIdx);
00584
00585         sameCtrIdx=0;
00586     }
00587 }
00588 else ARG_CHECK=0;
00589 };
00590
00591 template <class TQ, class TD>
00592 inline void CPAT<TQ,TD>::getQRange(
00593     unsigned k,
00594     wbMatrix<TQ> &QQ,
00595     wbvector<unsigned> &SS,
00596     QSpace<TQ,TD> *B
00597 ) const {
00598
00599     unsigned i,j,l,n,nA,m=CP.len; int e=0;
00600     wbvector<unsigned> S,D;
00601     wbMatrix<TQ> Q;
00602     wbperm P;
00603
00604     for (i=0; i<m; i++) { n=CP[i].len;
00605         for (j=0; j<n; j++) { nA=CP[i][j].A.len; ;
00606             if (CP[i][j].ic!=k) continue;
00607
00608             for (l=0; l<nA; l++) {
00609                 CP[i][j].A[l].getQDim(Q,S);
00610                 QQ.cat(l,Q);
00611                 SS.Append(S);
00612             }
00613         }
00614     }
00615     if (B) {
00616         B->getQDim(k,Q,S);
00617         QQ.cat(l,Q);
00618         SS.Append(S);
00619     }
00620
00621     QQ.groupRecs(P,D);
00622     e=SS.set2Group(P,D,0,'i');

```

```

00623
00624     if (e) wblog(FL,
00625 "ERR QSpace size inconsistency (dim=%d): %s (%d)",k+1,str,e);
00626 }
00627
00628 template <class TQ, class TD>
00629 inline void QSpace<TQ,TD>::ExpandQ(
00630     CPAT<TQ,TD> &CP,
00631     double rnm,
00632     wbvector<unsigned> &xflag,
00633     char disp
00634 ) {
00635     unsigned i,j,k,l,n,nr,s,r=rank(FL), dmin=0;
00636     const char EXISTS=1, OLDSYM=1<<1, XPAND=1<<2, XPAND2=1<<3, rank2=(r<3);
00637     int e=0;
00638
00639     wbvector< wbMatrix<TQ> > QQ;
00640     wbvector< wbvector<unsigned> > SS;
00641     wbvector<unsigned> D(r),S(r),dc;
00642     wbMatrix<unsigned> SA;
00643     wbvector<char> mark;
00644     wbMatrix<TQ> Q1,Q2,QK1,QK2;
00645     QSpace<TQ,TD> AA;
00646     wbindex I1,I2,d1,d2,I(r);
00647     wbperm p1,p2;
00648     TD nl,n2;
00649
00650     QQ.initDef(r);
00651     SS.initDef(r);
00652
00653     for (i=0; i<r; i++) {
00654         CP.getQRange(i,QQ[i],SS[i], this);
00655
00656         S[i]=SS[i].len;
00657
00658         D[i]=SS[i].sum();
00659         if (D[i]>1) {
00660             if (dmin==0) dmin=D[i]; else dmin=MIN(dmin,D[i]); }
00661     }
00662
00663     if (xflag.isEmpty()) {
00664         if (!rank2) {
00665             xflag.init(r);
00666             for (i=0; i<r; i++) if (D[i]<=dmin) xflag[i]=1;
00667         }
00668     }
00669     else if (xflag.len!=r) wblog(FL,
00670 "ERR %s - invalid xflag (dim. mismatch %d/%d)",FCT,xflag.len,r);
00671
00672     if (disp>9)
00673         wblog(FL,"TST D=[%s], expand=[%s] (%d)",
00674             D.toStr().data, xflag.toStr().data, dmin);
00675
00676     s=S.prod(); I.set(0);
00677     AA.init(s,r,QDIM); SA.init(s,r); mark.init(s);
00678
00679     for (l=S.len-1, i=0; i<s; i++) {
00680         for (k=0; k<I.len; k++) {
00681             AA.setQRec(i, k, QQ[k].rec(I[k]));
00682             SA(i,k)=SS[k][I[k]];
00683         }
00684
00685         I[l]++; k=1;
00686         while(I[k]>=S[k] && k>0) { I[k]=0; ++I[--k]; }
00687     }
00688
00689     QIDX.blockSum(QDIM,Q1); Q1.makeUnique();
00690     AA.QIDX.blockSum(QDIM,Q2); Q2.groupRecs(p2,d2);
00691
00692     e=matchIndex(Q1,Q2,I1,I2);
00693     if (e || I1.len!=Q1.dim1) wblog(FL,
00694 "ERR Failed to locate all existing symmetry sectors\n"
00695 " (e=%d; len=%d/%d)", e, I1.len, Q1.dim1);
00696
00697     d2.cumsum_(dc);
00698     for (i=0; i<I2.len; i++) {
00699         l=dc[I2[i]]; n=d2[I2[i]];
00700         for (j=0; j<n; j++) mark[p2[l+j]] |= OLDSYM;
00701     }
00702
00703     if (disp>9) sprintf(str,
00704 "Psi having %d/%d symmetry sector(s)", Q1.dim1, Q2.dim1);
00705
00706     xflag.find(0,I); if (!I.isEmpty()) {
00707         getQsub(I,QK1); QK1.groupRecs(p1,d1);
00708         AA.getQsub(I,QK2); QK2.groupRecs(p2,d2);
00709     }

```



```

00710     e=matchIndex(QK1,QK2,I1,I2);
00711     if (e || I1.len!=QK1.dim1) wblog(FL,
00712         "ERR Failed to locate all existing symmetry sectors (%d/%d)",
00713         I1.len, QK1.dim1);
00714
00715     d2.cumsum_(dc);
00716     for (i=0; i<I2.len; i++) {
00717         l=dc[I2[i]]; n=d2[I2[i]];
00718         for (j=0; j<n; j++) mark[p2[l+j]] |= XPAND;
00719     }
00720 }
00721 else {
00722     for (i=0; i<mark.len; i++) mark[i] |= XPAND;
00723 }
00724
00725 e=matchIndex(QIDX, AA.QIDX, I1, I2);
00726 if (e || I1.len!=QIDX.dim1) wblog(FL,
00727     "ERR Failed to locate all existing data blocks (%d/%d)",
00728     I1.len, QIDX.dim1);
00729
00730 for (i=0; i<I2.len; i++) {
00731     mark[I2[i]] |= EXISTS;
00732     DATA[I1[i]]->save2(*(AA.DATA[I2[i]]));
00733
00734     if (!SA.recEqual(I2[i], AA.DATA[I2[i]]->SIZE.data)) {
00735         SA.recPrint(i,"SA");
00736         AA.DATA[I2[i]]->SIZE.print("SIZE");
00737         wblog(FL, "ERR Severe size inconsistency (%d)", i+1);
00738     }
00739 }
00740
00741 if (rnrnrm<=0) {
00742     for (i=0; i<mark.len; i++)
00743         if (mark[i]>0 && !(mark[i] & OLDSYM)) mark[i]=-mark[i];
00744 }
00745
00746 for (n=i=0; i<mark.len; i++) if (mark[i]>0) n++;
00747
00748 init(n,r,QDIM);
00749 I1.init(n);
00750
00751 for (nr=l=i=0; i<mark.len; i++) {
00752     if (mark[i]<=0) continue;
00753
00754     QIDX.recSetP(l, AA.QIDX.rec(i));
00755
00756     if (AA.DATA[i]->isEmpty()) {
00757         S.init2ref(SA.dim2, SA.rec(i));
00758         DATA[l]->init(S);
00759         if (!(mark[i] & OLDSYM)) {
00760             DATA[l]->setRand('s');
00761             I1[l]=1; nr++;
00762         }
00763     }
00764     else AA.DATA[i]->save2(*DATA[l]);
00765
00766     l++;
00767 }
00768
00769 if (rnrnrm>0 && nr) {
00770     double dbl,fac=0,r2=rnrnrm*rnrnrm*1E-3;
00771     if (fabs(rnrnrm)>1) wblog(FL,"WRN rnrnrm = %g !?",rnrnrm);
00772
00773     for (n1=n2=0,i=0; i<I1.len; i++) {
00774         if (I1[i]) n2+=(DATA[i]->norm2());
00775         else {
00776             dbl=(DATA[i]->norm2()); fac+=dbl;
00777             if (dbl>r2) n1+=dbl;
00778             else {
00779                 DATA[i]->setRand('s');
00780                 I1[i]=2; n2+=(DATA[i]->norm2());
00781             }
00782         }
00783     }
00784
00785     if (n1!=0 && n2!=0) {
00786         r2=rnrnrm*rnrnrm; fac/=(n1+r2);
00787
00788         if (disp>9) wblog(FL,
00789             "TST n1=%g, n2=%g, r2=%g -> fac=%g",n1,n2,r2,fac);
00790
00791         n1=(TD)sqrt(fac*n1);
00792         n2=(TD)sqrt(fac*(r2/n2));
00793
00794         for (i=0; i<I1.len; i++)
00795             if (I1[i]) (*DATA[i])*=n2; else (*DATA[i])*=n1;
00796     }

```

```

00797         else wblog(FL,"WRN rand. norm = %.3g (%.3g) !?",n2,n1);
00798     }
00799
00800     if (xflag.count(0)) {
00801
00802         unsigned itry=0,iTRY=0, ntry=16, nQ=1, m=CP.CP.len, mx=0, m0=QIDX.dim1;
00803         QSpace<TQ,TD> AX;
00804
00805         Q1.init(); Q2.init();
00806
00807         for (iTRY=0; iTRY<ntry && nQ; iTRY++) {
00808             for (itry=0; itry<ntry; itry++) {
00809                 CP.getCtrIndex(*this);
00810
00811                 for (i=0; i<m; i++) {
00812                     j=CP.CP[i].len-1;
00813                     nQ=CP.CP[i][j].CQn.len;
00814
00815                     for (l=0; l<nQ; l++)
00816                         Q1.cat(l,
00817                             CP.CP[i][j].CQn[l].QIDX
00818                         );
00819                 }
00820
00821                 Q1.makeUnique();
00822                 if (Q1==Q2) break; else Q1.save2(Q2);
00823             }
00824
00825             if (itry==ntry) wblog(FL,
00826                 "ERR CP|psi> keeps generating new terms (%d).\n"
00827                 "Symmetries not preserved !?", ntry);
00828             if (Q1.dim2!=QIDX.dim2) wblog(FL,
00829                 "ERR Severe QIDX dimension mismatch (%d/%d)",Q1.dim2,QIDX.dim2);
00830
00831             e=matchIndex(Q1,AA.QIDX,I1,I2);
00832
00833             if (e || I1.len!=Q1.dim1) wblog(FL,
00834                 "ERR Failed to locate all existing data blocks (%d/%d)",
00835                 I1.len, Q1.dim1);
00836
00837             for (nQ=i=0; i<I2.len; i++) if (mark[I2[i]]<=0) nQ++;
00838             if (nQ) {
00839                 l=QIDX.dim1; if (l!=DATA.len) wblog(FL,"%d/%d ???",l,DATA.len);
00840                 QIDX.Resize(l+nQ, QIDX.dim2);
00841                 DATA.Resize(l+nQ);
00842                 for (i=0; i<nQ; i++) DATA[l+i] = new wbararray<TD>;
00843
00844                 for (i=0; i<I2.len; i++) {
00845                     j=I2[i]; if (mark[j]>0) continue;
00846                     if (mark[j]<0) mark[j]=-mark[j]; else
00847                         mark[j] |= XPAND2; mx++;
00848
00849                     QIDX.recSetP(l, AA.QIDX.rec(j));
00850
00851                     if (AA.DATA[j]->isEmpty()) {
00852                         S.init2ref(SA.dim2, SA.rec(j));
00853                         DATA[l]->init(S);
00854                     }
00855                     else {
00856                         wblog(FL,"WRN excuse moi !?");
00857                         AA.DATA[j]->save2(*DATA[l]);
00858                     }
00859
00860                     l++;
00861                 }
00862             }
00863
00864             if (disp>9) wblog(FL,"TST itry=%d, iTRY=%d (%d)",itry,iTRY,ntry);
00865
00866             if (iTRY==ntry) wblog(FL,
00867                 "ERR CP|psi> keeps generating new terms (%d).\n"
00868                 "Symmetries not preserved !?", ntry);
00869
00870             if (disp>9)
00871                 wblog(FL,"TST D=[%s], %d/%d completing to %d+%d blocks",
00872                     getDim().toStr().data, iTRY+1,itry+1,m0,mx);
00873         }
00874
00875         if (disp>9)
00876             wblog(FL,"TST %s -> %d", str, getDQtot());
00877     }
00878
00879     template <class TQ, class TD>
00880     QSpace<TQ,TD>& CPAT<TQ,TD>::HZTimes_tst (
00881         const QSpace<TQ,TD> &PSI,
00882         QSpace<TQ,TD> &HZPsi,
00883         const char tflag,

```

```

00884     char disp
00885 ) {
00886     unsigned i,j,it;
00887     static unsigned call=0;
00888     wbpem P;
00889
00890     if (!(call++)) wblog(FL,"TST %N*** using HZTimes_tst ***%N");
00891
00892     QSpace<TQ,TD> X1,X2,XX;
00893
00894     if (tflag=='C' && ISREAL(TD)) tflag='T';
00895
00896     if (!Wb::strchri("NTC",tflag)) wblog(FL,
00897     "ERR Invalid flag %c<%d>", tflag, tflag);
00898
00899     it=( tflag=='N' ? 2 : ( tflag=='T' ? 1 : -1 ));
00900
00901     HZPsi=PSI;
00902     HZPsi *= (tflag=='C' ? -CONJ(z) : -z);
00903
00904     for (i=0; i<CP.len; i++) {
00905
00906         if (CP[i].len<=1) {
00907             CPAT_Set<TQ,TD> &C1=CP[i][0];
00908             for (j=0; j<C1.A.len; j++) {
00909                 P.initFirstTo(C1.ic, PSI.rank());
00910                 if (tflag!='C')
00911                     C1.A[j].contract(it,PSI, C1.ic+1, X1, P);
00912                 else C1.A[j].opA(tflag,XX).contract(2, PSI, C1.ic+1, X1, P);
00913
00914                 HZPsi+=X1;
00915             }
00916         }
00917         else {
00918             CPAT_Set<TQ,TD> &C1=CP[i][0], &C2=CP[i][1];
00919             if (C1.A.len!=C2.A.len || CP[i].len>2) wblog(FL,"ERR ???");
00920
00921             for (j=0; j<C1.A.len; j++) {
00922                 P.initFirstTo(C1.ic, PSI.rank());
00923                 if (tflag!='C')
00924                     C1.A[j].contract(it,PSI, C1.ic+1, X1, P);
00925                 else C1.A[j].opA(tflag,XX).contract(2, PSI, C1.ic+1, X1, P);
00926
00927                 P.initFirstTo(C2.ic, PSI.rank());
00928                 if (tflag!='C')
00929                     C2.A[j].contract(it,X1 , C2.ic+1, X2, P);
00930                 else C2.A[j].opA(tflag,XX).contract(2, X1 , C2.ic+1, X2, P);
00931
00932                 HZPsi+=X2;
00933             }
00934         }
00935     }
00936
00937     return HZPsi;
00938 };
00939
00940 template <class TQ, class TD>
00941 inline void CPAT_Set<TQ,TD>::contract (
00942     const QSpace<TQ,TD> &PSI,
00943     wvector< QSpace<TQ,TD> > &C,
00944     char tflag
00945 ) const {
00946
00947     unsigned i,n=A.len;
00948     if (C.len!=n) C.initDef(n);
00949
00950     for (i=0; i<n; i++)
00951         contract(i,PSI,C[i],tflag);
00952 }
00953
00954 template <class TQ, class TD>
00955 inline void CPAT_Set<TQ,TD>::contract_diag (
00956     const QSpace<TQ,TD> &PSI,
00957     wvector< QSpace<TQ,TD> > &C,
00958     TD a, TD b, char Iflag
00959 ) const {
00960
00961     unsigned i,n=A.len;
00962     if (C.len!=n) C.initDef(n);
00963
00964     for (i=0; i<n; i++)
00965         contract_diag(i,PSI,C[i],a,b,Iflag);
00966 }
00967
00968 template <class TQ, class TD>
00969 inline void CPAT_Set<TQ,TD>::contract (
00970     const wvector< QSpace<TQ,TD> > &PSI1,

```

```

00971     wbvector< QSpace<TQ,TD> > &C,
00972     char tflag
00973 ) const {
00974
00975     unsigned i,n=A.len;
00976     if (C.len!=n) C.initDef(n);
00977
00978     for (i=0; i<n; i++)
00979         contract(i,PSI1[i],C[i],tflag);
00980 }
00981
00982 template <class TQ, class TD>
00983 inline void CPAT_Set<TQ,TD>::contract_diag(
00984     const wbvector< QSpace<TQ,TD> > &PSI1,
00985     wbvector< QSpace<TQ,TD> > &C,
00986     TD a, TD b, char Iflag
00987 ) const {
00988
00989     unsigned i,n=A.len;
00990     if (C.len!=n) C.initDef(n);
00991
00992     for (i=0; i<n; i++)
00993         contract_diag(i,PSI1[i],C[i],a,b,Iflag);
00994 }
00995
00996 template <class TQ, class TD>
00997 void CPAT_Set<TQ,TD>::contract(
00998     const unsigned k,
00999     const QSpace<TQ,TD> &PSI,
01000     QSpace<TQ,TD> &C,
01001     char tflag
01002 ) const {
01003
01004     unsigned i,n,ia,ib,iu,it;
01005     wbMatrix<unsigned> &Iabu = (tflag=='N' ? CIn[k] : CIt[k]);
01006     wbararray<TD> XX;
01007
01008 #ifdef WB_CLOCK
01009     Wb::Clock clk("cpat:contract",0);
01010 #endif
01011
01012     if (tflag=='C' && ISREAL(TD)) tflag='T';
01013
01014     if (!Wb::strchri("NTC",tflag)) wblog(FL,
01015         "ERR Invalid flag %c<%d>",tflag,tflag);
01016
01017     C.QDIM=PSI.QDIM;
01018     C.QIDX=(tflag=='N' ? CQn[k].QIDX : CQt[k].QIDX);
01019     C.setupDATA();
01020
01021     it=( tflag=='N' ? 2 : ( tflag=='T' ? 1 : -1 ));
01022
01023     for (n=Iabu.dim1, i=0; i<n; i++) {
01024         ia=Iabu(i,0); ib=Iabu(i,1); iu=Iabu(i,2);
01025
01026         if (tflag!='C') {
01027             A[k].DATA[ia]->
01028                 contract(it,*(PSI.DATA[ib]), ic+1, *(C.DATA[iu]), P); }
01029         else {
01030             A[k].DATA[ia]->opA(tflag,XX).
01031                 contract(2, *(PSI.DATA[ib]), ic+1, *(C.DATA[iu]), P);
01032         }
01033     }
01034 }
01035
01036 template <class TQ, class TD>
01037 void CPAT_Set<TQ,TD>::contract_diag(
01038     const unsigned k,
01039     const QSpace<TQ,TD> &PSI,
01040     QSpace<TQ,TD> &C,
01041     TD a, TD b, char Iflag
01042 ) const {
01043
01044     unsigned i,n,ia,ib,iu;
01045     wbMatrix<unsigned> &Iabu = CIn[k];
01046     wbararray<TD> XX;
01047
01048 #ifdef WB_CLOCK
01049     Wb::Clock clk("cpat:contract",0);
01050 #endif
01051
01052     C.QDIM=PSI.QDIM; C.QIDX=CQn[k].QIDX;
01053     C.setupDATA();
01054
01055     for (n=Iabu.dim1, i=0; i<n; i++) {
01056         ia=Iabu(i,0); ib=Iabu(i,1); iu=Iabu(i,2);
01057

```

```

01058         if (A[k].isDiagBlock(ia))
01059             PSI.DATA[ib]->contractDiag(
01060                 ic+1,*A[k].DATA[ia], a,b, *C.DATA[iu], Iflag
01061             );
01062     }
01063 }
01064
01065 template <class TQ, class TD>
01066 mxArray* CPAT<TQ,TD>::toMx() const {
01067     unsigned i,j,l,k, m=CP.len, n;
01068     mxArray *S, *C, *a;
01070
01071     C=mxCreateCellMatrix(m,1);
01072
01073     for (i=0; i<m; i++) {
01074         n=CP[i].len;
01075         S=mxCreateStructMatrix(1,n,0,NULL);
01076
01077         for (j=0; j<n; j++) {
01078             a=CP[i][j].toMx();
01079             k=mxGetNumberOfFields(a);
01080
01081             if (j==0)
01082                 for (l=0; l<k; l++) {
01083                     if (mxAddField(S,mxGetFieldNameByNumber(a,l))<0) wblog(FL,
01084                         "ERR Could not add field `%s'", mxGetFieldNameByNumber(a,l));
01085                 }
01086
01087                 for (l=0; l<k; l++) {
01088                     mxSetFieldByNumber(S, j, l,
01089                         mxDuplicateArray(mxGetFieldByNumber(a,0,l)));
01090                 }
01091
01092                 mxDestroyArray(a);
01093             }
01094             mxSetCell(C,i,S);
01095         }
01096
01097         S=mxCreateStructMatrix(1,1,0,NULL);
01098         mxAddField2Scalar(FL,S,"CP",C);
01099         mxAddField2Scalar(FL,S,"z",numtoMx(z));
01100
01101         return S;
01102     }
01103 }
01104 template <class TQ, class TD>
01105 mxArray* CPAT_Set<TQ,TD>::toMx() const {
01106     mxArray *S, *a;
01107     unsigned i;
01109
01110     S=mxCreateStructMatrix(1,1,0,NULL);
01111
01112     mxAddField2Scalar(FL,S,"A", A.toMx());
01113     mxAddField2Scalar(FL,S,"ic", numtoMx(ic+1));
01114     mxAddField2Scalar(FL,S,"opA", numtoMx(opA));
01115     mxAddField2Scalar(FL,S,"P", (P+1).toMx());
01116
01117     for (i=0; i<CQn.len; ++i) CQn[i].setupDATA();
01118     for (i=0; i<CQt.len; ++i) CQt[i].setupDATA();
01119
01120     mxAddField2Scalar(FL,S,"CQn", CQn.toMx());
01121     mxAddField2Scalar(FL,S,"CQt", CQt.toMx());
01122
01123     a=mxCreateCellMatrix(CIn.len,1);
01124     for (unsigned i=0; i<CIn.len; i++)
01125         mxSetCell(a, i, CIn[i].toMx());
01126     mxAddField2Scalar(FL,S,"CIn",a);
01127
01128     a=mxCreateCellMatrix(CIt.len,1);
01129     for (unsigned i=0; i<CIt.len; i++)
01130         mxSetCell(a, i, CIt[i].toMx());
01131     mxAddField2Scalar(FL,S,"CIt",a);
01132
01133     return S;
01134 }
01135
01136 int isCPAT(const mxArray *CC, char twoSite) {
01137     unsigned i,i0,k,n;
01138
01139     if (!CC || !mxIsCell(CC)) return 0;
01140     if (isCPattern(CC)) return 1;
01141
01142     n=mxGetNumberOfElements(CC);
01143     if (!n) return 0;
01144

```

```

01145     i0=isCPattern(mxGetCell(CC,0));
01146     if (!i0) return 0;
01147
01148     for (k=1; k<n; k++) {
01149         i=isCPattern(mxGetCell(CC,k));
01150         if (!i) return 0;
01151
01152         if (i!=i0) wblog(FL,
01153             "ERR CPAT mismatch twoSite=%d,%d ???", i0, i);
01154     }
01155
01156     if (twoSite>=0) if (1+twoSite!=(char)i0) wblog(FL,
01157         "ERR CPAT is not of type %d-site (%d)", 1+twoSite, i0);
01158
01159     return n;
01160 }
01161
01162 int isCPattern(const mxArray *C) {
01163     unsigned k,n,w;
01164     char twoSite, hasOp;
01165     mxArray *a;
01166
01167     if (!C || !mxIsCell(C)) {
01168         sprintf(str,"%s:%d invalid CP",FL);
01169         return 0;
01170     }
01171
01172     n=mxGetNumberOfElements(C);
01173     if (n<2 || n>4) {
01174         sprintf(str,"%s:%d invalid CP (%d entries)",FL,n);
01175         return 0;
01176     }
01177
01178     a=mxGetCell(C,0);
01179     if (!mxIsStruct(a) || mxIsCell(a)) {
01180         sprintf(str,"%s:%d invalid arg #1",FL);
01181         return 0;
01182     }
01183
01184     a=mxGetCell(C,n-1);
01185     hasOp=mxIsChar(a);
01186
01187     if (hasOp) if (mxGetNumberOfElements(a)!=1) {
01188         sprintf(str,"%s:%d arg #%d not a flag (char)",FL,n);
01189         return 0;
01190     }
01191
01192     k=n-hasOp;
01193     if (k==2) twoSite=0; else
01194     if (k==3) twoSite=1; else {
01195         sprintf(str,"%s:%d invalid CP (k=%d)",FL,k);
01196         return 0;
01197     }
01198
01199     if (twoSite) {
01200         w = (mxIsEmpty(mxGetCell(C,1)) ? 0 : 1) +
01201             (mxIsEmpty(mxGetCell(C,2)) ? 0 : 2);
01202         if (w<1 || w>2) {
01203             sprintf(str, "%s:%d either arg #2 or #3",FL);
01204             return 0;
01205         }
01206     }
01207     else w=1;
01208
01209     a=mxGetCell(C,w);
01210     if (!Mx::IsNumber(a)) {
01211         sprintf(str,"%s:%d arg #%d not a number",FL,w+1);
01212         return 0;
01213     }
01214
01215     return (1+twoSite);
01216 }
01217
01218 #endif
01219

```

5.3 dmrho.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace NRG routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum

```

```

00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_DMRHO_CC__
00021 #define __WB_DMRHO_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 template <class TQ, class TD>
00027 double initRHO(
00028     wbMatrix<double> &rhoNorm,
00029     const double T,
00030     NRGData<TQ,TD> &H,
00031     wbvector<double> E0,
00032     const unsigned dloc,
00033     const unsigned NRho=0,
00034     char vflag=1
00035 );
00036
00037 template <class TQ, class TD>
00038 void getEDData(
00039     const QSpace<TQ,TD> &HK,
00040     wbvector<double> &E, wbvector<unsigned> &D,
00041     wbvector<unsigned> *dz=NULL
00042 );
00043
00044 template <class TQ, class TD>
00045 TD getERange(const QSpace<TQ,TD> &HK, double *emin=NULL, double *emax=NULL);
00046
00047 template <class TQ, class TD>
00048 unsigned getGSDegeneracy(const QSpace<TQ,TD> &H);
00049
00050 template <class TQ, class TD>
00051 bool getBoltzman(
00052     const QSpace<TQ,TD> &H,
00053     wbvector<double> &R, wbvector<unsigned> &D, unsigned iter,
00054     wbvector<unsigned> *DZ=NULL, char useE0=1
00055 );
00056
00057 template <class TD>
00058 bool getBoltzman_base(
00059     const wbvector<TD>& E, const wbvector<unsigned> &dz,
00060     wbvector<TD>& R, unsigned iter, double T=0., char useE0=1
00061 );
00062
00063 template <class TQ, class TD>
00064 double initRho(
00065     QSpace<TQ,TD> &Rho,
00066     const QSpace<TQ,TD> &HK,
00067     const int iter,
00068     double efac=1.
00069 );
00070
00071 template <class TQ, class TD>
00072 void nrgUpdateRHO(
00073     const wbMatrix<double> &R,
00074     NRGData<TQ,TD> &A,
00075     NRGData<TQ,TD> &H,
00076     NRGData<TQ,TD> &RHO,
00077     unsigned NRho,
00078     unsigned dloc,
00079     Wb::SigHandler *sig=NULL
00080 );
00081
00082 template <class TQ, class TD>
00083 void updateRho(
00084     QSpace<TQ,TD> &Rho,
00085     const QSpace<TQ,TD> &AK
00086 );
00087
00088 template <class TQ, class TD, class TX>
00089 void updateOp_SR(QSpace<TQ,TX> &X, const QSpace<TQ,TD> &AK);
00090
00091 template <class TQ, class TD, class TZ>
00092 void initG_SR(

```

```

00093     QSpace<TQ,TZ> &G2, const QSpace<TQ,TD> &H2,
00094     wbcomplex z, unsigned iter
00095 );
00096
00097 template <class TQ, class TD>
00098 void checkRho (const char *F, int L,
00099     const QSpace<TQ,TD> &Rho,
00100     const double traceRho=1.
00101 ){
00102     unsigned i,n=Rho.QIDX.dim1;
00103     double rsum=0, rmin=0;
00104
00105     wbvector< wbvector<double> > E(n);
00106     wbvector<double> rr;
00107     wbarray<TD> U;
00108
00109     char cgflag=Rho.getCGS(FL);
00110
00111     if (cgflag>0 && Rho.qtype.permitsOM(>1) {
00112         cgflag=2;
00113     }
00114
00115     if (!Rho.isConsistent_r(2) || !Rho.isBlockDiagMatrix() || !Rho.isHConj())
00116         wblog(F,L,"ERR %s() inconsistency in Rho QSpace (%d;%d;%d)",
00117             FCT, Rho.isBlockDiagMatrix(), Rho.isHConj());
00118
00119     for (i=0; i<n; i++) {
00120         wbEigenS(*(Rho.DATA[i]),U,E[i]);
00121         if (cgflag>0) {
00122             unsigned d=Rho.cgsDimScalar(i);
00123             if (cgflag<=1) {
00124                 unsigned d0=Rho.qtype.QDim(Rho.QIDX.rec(i)); if (d0!=d)
00125                     wblog(FL,"ERR %s() cgsDim inconsistency! (%d/%d)",FCT,d,d0);
00126             }
00127             E[i]*=d;
00128         }
00129     }
00130     rr.Cat(E);
00131
00132     rmin=rr.min();
00133     rsum=rr.sum();
00134
00135     if (rmin<-EPS) { wblog(F,L,
00136         "ERR eig(rho) is in (%.4g .. %.4g; %.4g)",rmin,rr.max(),rsum);
00137     }
00138     else if (fabs(rsum-traceRho)>EPS) {
00139         char e=(fabs(rsum-traceRho)>1E-8 && rsum>1E-8 && traceRho>1E-8);
00140         if (e) {
00141             printf("\n\n");
00142             MXPput(FL).add(Rho,"Rho").add(rr,"rr").add(traceRho,"t");
00143         }
00144         sprintf(str,"%s tr(rho) inconsistency !?\n"
00145             "%.4g / %.4g @ %.3g (%.2g; %.2g; %.2g)", e ? "ERR":"WRN",
00146             rsum,traceRho,rsum-traceRho,DEPS,DEPS2,EPS); printf("\n");
00147         wblog(F,L,str);
00148     }
00149 };
00150
00151 template <class TQ, class TD>
00152 double initRHO(
00153     wbMatrix<double> &rhoNorm,
00154     const double T,
00155     NRGData<TQ,TD> &H,
00156     wbvector<double> E0,
00157     const unsigned dloc,
00158     const unsigned NRho,
00159     char vflag
00160 ){
00161     unsigned i=0,i1,i2, iter, isneg=0, N=NRG_N; char i0flag=0;
00162     wbvector<double> R;
00163     wbvector<unsigned> D;
00164     double dbl, dfac=1;
00165     double DFAC=DBL_MAX*1E-2;
00166
00167     if (NRho>N) wblog(FL,"ERR NRho out of bounds (%d/%d)");
00168
00169     i1 = (dloc && NRho ? NRho-1 : 0);
00170     i2 = (NRho ? NRho-1 : N-1);
00171
00172     if (int(i1)>int(i2) || int(i1)<0) wblog(FL,
00173         "ERR rho_i = [%d..%d] !?",i1,i2);
00174
00175     rhoNorm.init(N,2);
00176
00177     dbl=E0.min(i);
00178     if (i==0) { i0flag=1;
00179         E0[0]+=E0[1]; dbl=E0.min(i);

```



```

00180     E0[0]-=E0[1];
00181 }
00182
00183 if (i<i2) {
00184     wblog(FL,"WRN Emin set by site %d/%d !?",i+1,N);
00185     MXPut(FL).add(E0,"E0");
00186 }
00187
00188 getBoltzman_base(E0,D,E0,-1,T);
00189
00190 for (iter=i1; iter<=i2; ++iter) {
00191     H.init(FL,"D",iter);
00192     if (getBoltzman(H.D,R,D,iter)) ++isneg;
00193     rhoNorm(iter,1)=R.sum();
00194
00195     if (dloc && iter<i2) continue;
00196
00197     H.init(FL,"K",iter);
00198     if (getBoltzman(H.K,R,D,iter)) ++isneg;
00199     rhoNorm(iter,0)=R.sum();
00200 }
00201
00202 if (isneg) wblog(FL,
00203     "WRN %s() negative (eigen)energies (%d) !?",FCT,isneg);
00204 if (i0flag && (rhoNorm(0,0)>1 || rhoNorm(0,1)>1)) wblog(FL,
00205     "WRN %s() density matrix from first site may be unstable (%.3g,%.3g)",
00206     FCT, rhoNorm(0,0), rhoNorm(0,1));
00207
00208 if (!H.K.isEmpty())
00209     i=getGSDegeneracy(H.K);
00210 else i=getGSDegeneracy(H.D);
00211
00212 if (vflag) {
00213     if (i==1)
00214         wblog(FL," * non-degenerate ground state");
00215     else wblog(FL,"<i> ground state degeneracy k=%d (%d)",iter-1,i);
00216 }
00217
00218 if (!dloc) {
00219     if (!NRho) wblog(FL,"ERR NRho=%d !?", NRho);
00220     wblog(FL,"<i> generate weights for plain NRG (T=%.3g)", T);
00221
00222     for (iter=0; iter<NRho; iter++) {
00223         dbl=rhoNorm.recSum(iter);
00224         if (dbl<1) wblog(FL,"WRN sum_rhoNorm(%d,:)=%.3g",iter+1,dbl);
00225         dbl=1/dbl;
00226         rhoNorm(iter,0)*=dbl;
00227         rhoNorm(iter,1)*=dbl;
00228     }
00229     return 0;
00230 }
00231
00232 if (!NRho)
00233     for (dfac=dloc, iter=i2-1; (int)iter>=0; iter--, dfac*=dloc) {
00234         rhoNorm(iter,1)*=dfac;
00235
00236         if (dfac>DFAC) wblog(FL,"WRN dfac reaches DBL_MAX (%.3g)", dfac);
00237     }
00238
00239     dbl=rhoNorm.sum();
00240     if (dbl<=0.) wblog(FL,"ERR %s() normRH0=%g ??? DIV/0",FCT,dbl);
00241     rhoNorm *= (1/dbl);
00242
00243     if (vflag) {
00244         double rmax;
00245         rhoNorm.recSum(R); rmax=R.max(i);
00246         wblog(FL," * maxRho=%.4g at iter=%d/%d (T=%.4g)",rmax,i+1,N,T);
00247     }
00248
00249     return (-T*log(dbl));
00250 };
00251
00252 template <class TQ, class TD>
00253 void getEData(
00254     const QSpace<TQ,TD> &HK,
00255     wvector<double> &E,
00256     wvector<unsigned> &D,
00257     wvector<unsigned> *DZ
00258 ){
00259     unsigned i,l,m=0,d, e=0, k,K=HK.QIDX.dim1, *dz=NULL;
00260     char cgflag=HK.gotCGS(FL);
00261
00262     if (HK.DATA.len!=K) wblog(FL,
00263         "ERR %s() severe QSpace inconsistency (%d,%d)",
00264         FCT,HK.DATA.len,HK.QIDX.dim1);
00265     if (!HK.isBlockDiagMatrix('d')) wblog(FL,
00266         "ERR %s() Hamiltonian must be (block)diagonal!",FCT);

```

```

00267
00268     D.init(K);
00269     for (l=k=0; k<K; k++) D[k]=HK.DATA[k]->SIZE.max();
00270
00271     E.init(D.sum());
00272     if (DZ && cgflag>0) { DZ->init(E.len); dz=DZ->data; }
00273
00274     if (cgflag>0 && HK.qtype.permitsOM(>1) {
00275         cgflag=2;
00276     }
00277
00278     for (l=k=0; k<K; ++k) {
00279         const wbararray<TD> &Hk = *HK.DATA[k];
00280         const wvector<unsigned> &S = HK.SIZE;
00281         const TD* const E0=Hk.data;
00282
00283         d=D[k]; if (dz) {
00284             m=HK.cgsDimScalar(k);
00285             if (cgflag<=1) {
00286                 unsigned m0=HK.qtype.QDim(HK.QIDX.rec(k)); if (m0!=m)
00287                     wblog(FL,"ERR %s() cgsDim inconsistency! (%d/%d)",FCT,m,m0);
00288             }
00289         }
00290
00291         if (S[0]!=S[l]) { if (S[0]!=1 && S[l]!=1) e++; else {
00292             Wb::cpyRange(E.data+l, E0, d);
00293             if (dz)
00294                 { for (i=0; i<d; ++i,++l) dz[l]=m; }
00295             else { l+=d; }
00296         }}
00297         else { if (S.len!=2) e++; else {
00298             if (d>1 && (fabs(E0[l])>1E-12 || fabs(E0[d])>1E-12)) wblog(FL,
00299                 "WRN HK[%d] not E diagonal (%g,%g)", k+1,E0[l], E0[d]);
00300
00301             for (i=0; i<d; ++i,++l) {
00302                 E[l]=E0[i*d+i]; if (dz) dz[l]=m;
00303             }
00304         }}
00305         if (e) wblog(FL,
00306             "ERR HK[%d] must be rank-2 tensor! (%s)",
00307             k+1, HK.DATA[k]->sizeStr().data
00308         );
00309     }
00310 };
00311
00312 template <class TQ, class TD>
00313 unsigned getGSDegeneracy(const QSpace<TQ,TD> &H) {
00314
00315     wvector<double> E; wvector<unsigned> D,dz;
00316     unsigned i=0, n=0;
00317     double e0, *ee;
00318
00319     getEData(H,E,D,&dz); e0=E.min(); ee=E.data;
00320     for (; i<E.len; ++i) if (fabs(ee[i]-e0)<EPS) n++;
00321
00322     return n;
00323 };
00324
00325 template <class TQ, class TD>
00326 TD getErange(const QSpace<TQ,TD> &H, double *emin, double *emax){
00327
00328     if (!H.isBlockDiagMatrix('d')) {
00329         QSpace<TQ,TD> AK;
00330         QSpace<TQ,double> HK; wbMatrix<double> EE;
00331         H.EigenSymmetric(AK,HK,EE);
00332         return getErange(HK,emin,emax);
00333     }
00334
00335     wvector<double> E; wvector<unsigned> D;
00336     TD e1,e2;
00337
00338     getEData(H,E,D);
00339     e1=E.min(); if (emin) (*emin)=e1;
00340     e2=E.max(); if (emax) (*emax)=e2;
00341     return (e2-e1);
00342 };
00343
00344 template <class TQ, class TD>
00345 bool getBoltzman(
00346     const QSpace<TQ,TD> &H,
00347     wvector<double> &R, wvector<unsigned> &D, unsigned iter,
00348     wvector<unsigned> *DZ, char useE0
00349 ){
00350     wvector<unsigned> dz;
00351     wvector<double> E;
00352     bool e=0;
00353

```

```

00354     getEData(H,E,D,&dz); e=(E<0);
00355
00356     if (getBoltzman_base(E,dz,R,iter,0.,useE0)) { e=1; }
00357     if (DZ) dz.save2(*DZ);
00358
00359     return e;
00360 };
00361
00362 template <class TD>
00363 bool getBoltzman_base(
00364     const wbvector<TD>& E,
00365     const wbvector<unsigned>& dz,
00366     wbvector<TD>& R,
00367     unsigned iter,
00368     double T0,
00369     char useE0
00370 ){
00371     static double T=0.;
00372     static wbvector<double> E0;
00373
00374     unsigned i,e=0;
00375
00376     if (int(iter)<0) { T=T0; E0=R; return 0; }
00377
00378     if (iter>=E0.len) wblog(FL,
00379         "ERR %s() index out of bounds (%d/%d)",FCT,iter+1,E0.len);
00380     if (dz.len && dz.len!=E.len) wblog(FL,
00381         "ERR %s() size mismatch (%d/%d)",FCT,dz.len,E.len);
00382
00383     if (E.isEmpty()) { R.init(); return 0; }
00384
00385     double beta,r, Escal=gEScale(iter);
00386     double Eref=(useE0 ? E0[iter]/Escal : 0.);
00387
00388     if (E<-Eref) { ++e; wblog(FL,
00389         "WRN Eref should be lower bound (%.4g < %.4g) @ %.4g !?",
00390         E.min(), Eref, Escal);
00391     }
00392
00393     if (&R!=&E) R.init(E.len);
00394
00395     if (T>0.) {
00396         beta=Escal/T;
00397         for (i=0; i<E.len; ++i) {
00398             r=exp(-beta*(E[i]+Eref)); if (dz.len) r*=dz[i];
00399             R[i]=r;
00400         }
00401     }
00402     else {
00403         for (i=0; i<E.len; i++)
00404             R[i]=( ABS(E[i]+Eref)<EPS ? (dz.len ? dz[i] : 1) : 0. );
00405
00406         i=unsigned(R.sum());
00407         if (i) wblog(FL," * T=0 ground state degeneracy %d", i);
00408         else wblog(FL," * T=0 got unique ground state");
00409     }
00410
00411     return e;
00412 };
00413
00414 template <class TQ, class TD>
00415 double initRho(
00416     QSpace<TQ,TD> &Rho,
00417     const QSpace<TQ,TD> &H,
00418     const int iter,
00419     double rhofac
00420 ){
00421     unsigned i,k,l,d,n, K=H.QIDX.dim1, has0=0;
00422     wbvector<unsigned> D,dz;
00423     wbvector<double> R;
00424     double Z, *r;
00425
00426     getBoltzman(H,R,D,iter,&dz,0);
00427
00428     Z=R.sum();
00429
00430     if (Z==0.) {
00431         MXPput(FL,"Ix").add(H,"H").add(R,"R").add(iter,"iter").add(dz,"dz");
00432         wblog(FL,"ERR rhoNorm[%d]=%g -> DIV/0 !?", iter+1,Z);
00433     }
00434     if (dz.len && R.len!=dz.len) wblog(FL,
00435         "ERR %s() length mismatch %d/%d",FCT,R.len,dz.len);
00436
00437     R *= (rhofac/Z); r=R.data;
00438
00439     if (dz.len) {
00440         for (i=0; i<R.len; ++i) { if (dz.data[i]>1) r[i]/=dz.data[i]; }

```

```

00441     }
00442
00443     Rho=H;
00444
00445     for (l=k=0; k<K; k++, l+=d) {
00446         wbararray<TD> &Rk = (*Rho.DATA[k]);
00447
00448         d=D[k]; Rk.init(d,d);
00449
00450         for (n=i=0; i<d; i++) {
00451             Rk.data[i*d+i] = r[l+i];
00452             if (!n && r[l+i]>DEPS) n++;
00453         }
00454         if (!n) has0++;
00455     }
00456
00457     if (has0) Rho.SkipZeroData(DEPS2,'b');
00458     return Z;
00459 };
00460
00461 template <class TQ, class TD>
00462 void updateRho(QSpace<TQ,TD> &Rho, const QSpace<TQ,TD> &AK) {
00463
00464     QSpace<TQ,TD> Xk;
00465
00466     if (!Rho.isBlockDiagMatrix()) wblog(FL,
00467         "WRN %s() Rho not blockdiag !?",FCT);
00468
00469     AK.contract(2,Rho,1,Xk);
00470     Xk.contract("3,2",AK,"2,3;*",Rho);
00471 };
00472
00473 template <class TQ, class TD>
00474 void nrgUpdateRHO(
00475     const wbMatrix<double> &R,
00476     NRGData<TQ,TD> &A,
00477     NRGData<TQ,TD> &H,
00478     NRGData<TQ,TD> &RHO,
00479     unsigned NRho,
00480     unsigned dloc,
00481     Wb::SigHandler *sig
00482 ){
00483     unsigned iter, ilast=0, rzero, N=NRG_N; char ie;
00484     const size_t n=16; size_t l=0; char tag[n];
00485     wbvector<double> w; double w2, wtot=0;
00486
00487     QSpace<TQ,TD> Rho;
00488
00489     RHO.K.clearQSpace();
00490
00491     if (R.dim1!=N || R.dim2!=2)
00492         wblog(FL, "ERR %s() size mismatch of rhoNorm (%dx%d; %d %d)",
00493             FCT, R.dim1, R.dim2, N, NRho);
00494
00495     for (iter=N-1; (int)iter>=0; iter--) {
00496         if (sig) { sig->check911(); }
00497
00498         R.getRec(iter,w); rzero=(w<DEPS);
00499         w2=w.sum();
00500
00501         l=snprintf(tag,n,
00502             "%s %2d/%d", !NRho ? "FDM":"DMK", iter,N);
00503         if (l>n) wblog(FL,
00504             "ERR %s() string out of bounds (%s; %d/%d)",FCT,tag,l,n);
00505
00506         RHO.updatePara(FL,"K",iter);
00507
00508         if (rzero && RHO.K.isEmpty()) {
00509             wblog(FL,"%s skip ...%16s\r\\", tag,"");
00510             continue;
00511         }
00512
00513         A.init(FL,"K",iter);
00514         A.init(FL,"D",iter);
00515         H.init(FL,"D",iter);
00516
00517         A.check_dloc(FLF,dloc,iter);
00518
00519         if (!(ie=RHO.K.isEmpty())) {
00520             updateRho(RHO.K, A.K); }
00521
00522         if (!NRho) {
00523             if (H.D.isEmpty()) wblog(FL,
00524                 "%s no discarded space (%.3g) %16s\r\\",tag,wtot,"");
00525             else if (w2!=0) wblog(FL,
00526                 "%s dRho=%.3g (%.3g) %16s\r\\",tag,w2,wtot,"");
00527             else wblog(FL,

```

```

00528         "%s dRho=%.3g %16s\r\\", tag, wtot, "");
00529     }
00530     else wblog(FL, "%-30s\r\\", tag);
00531
00532     if (!rzero) {
00533         if (w[1]) {
00534             initRho(Rho, H.D, iter, w[1]);
00535             updateRho(Rho, A.D);
00536             RHO.K+=Rho; if (!ilast) { ilast=iter; }
00537         }
00538
00539         if (w[0] && (!ilast || ilast==iter)) { H.init(FL, "K", iter);
00540             initRho(Rho, H.K, iter, w[0]);
00541             updateRho(Rho, A.K);
00542             RHO.K+=Rho; if (!ilast) { ilast=iter; }
00543         }
00544
00545         RHO.K.SkipZeroData(DEPS2, 'b');
00546     }
00547
00548     wtot+=w2;
00549
00550     if (!RHO.K.isEmpty()) { try {
00551         checkRho(FL, RHO.K, Wb::addRange(R.ref(iter,-1), N-iter, R.dim2));
00552     }
00553     catch (...) {
00554         wblog(FL, "ERR %s() iter=%d/%d%s", FCT, iter+1, N,
00555             iter+1==N ? "\\nhint: got kept space at last iteration?":""");
00556     }
00557     else {
00558         if (ie && rzero) wblog(FL,
00559             "%s empty RHO (will not contribute) %16s\r\\", tag, "");
00560         else wblog(FL, "WRN %2d/%d got empty RHO !? %16s", iter, N, "");
00561     }
00562     doflush();
00563 }
00564 };
00565
00566 template <class TQ, class TD, class TZ>
00567 void initG_SR(
00568     QSpace<TQ,TZ> &G2, const QSpace<TQ,TD> &H2,
00569     wcomplex z, unsigned iter
00570 ){
00571     unsigned k=0, K=H2.QIDX.dim1;
00572     size_t i,dk; TZ *gk;
00573
00574     double Escale=gEScale(iter);
00575
00576     G2.initT(H2);
00577
00578     for (; k<K; ++k) {
00579         gk=G2.DATA[k]->data;
00580         dk=G2.DATA[k]->dim1();
00581         for (i=0; i<dk; ++i) { gk[i]=1./(z-Escale*gk[i]); }
00582         G2.DATA[k]->ExpandDiagonal();
00583     }
00584 };
00585
00586 template <class TQ, class TD, class TX>
00587 void updateOp_SR(QSpace<TQ,TX> &X, const QSpace<TQ,TD> &AK) {
00588
00589     QSpace<TQ,TX> Xk;
00590
00591     if (!X.isBlockDiagMatrix()) wblog(FL,
00592         "ERR %s() input X not blockdiag !?", FCT);
00593     if (X.itags.len!=2) wblog(FL,
00594         "ERR %s() input X not a scalar op (%s) !?", FCT, IT2STR(X));
00595
00596     AK.contract(2, X, 1, Xk);
00597     Xk.contract("32", AK, "23*", X);
00598 };
00599
00600 #endif
00601

```

5.4 gathered.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace additional routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *

```

```

00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_GATHERED_CC__
00021 #define __WB_GATHERED_CC__
00022
00023 // auxilliary routine for mexlib.* -> Mx::Array
00024 // to deal with type conversions in a controlled way // Wb,May11,19
00025
00026 void setSigHandler(char w) {
00027
00028     static sighandler_t prev_SIGINT;
00029     static sighandler_t prev_SIGABRT;
00030
00031     static int iflag=0;
00032
00033     if (w=='i') {
00034         if (++iflag==1) {
00035             prev_SIGINT = signal(SIGINT, wbSigHandler);
00036             prev_SIGABRT = signal(SIGABRT, wbSigHandler);
00037         }
00038     }
00039     else if (w=='r') {
00040         if (--iflag==0) {
00041             signal(SIGINT, prev_SIGINT);
00042             signal(SIGABRT, prev_SIGABRT);
00043         }
00044         else if (iflag<0) { wblog(gsh_F, gsh_L,
00045             "ERR SigHandler not yet initialized (iflag=%d)",iflag);
00046         }
00047     }
00048     else wblog(gsh_F, gsh_L,"ERR invalid flag=%s",cSTR(w));
00049 };
00050
00051 void wbSigHandler(int sig) {
00052
00053     unsigned i=Wb::SigHandler::icount,
00054             a=Wb::SigHandler::account, n=Wb::SigHandler::NC0;
00055     fflush(0);
00056
00057     if (sig==911) {
00058         if (i!=n || a!=n) {
00059             Wb::SigHandler::icount=Wb::SigHandler::account=n;
00060             wblog(gsh_F, gsh_L,
00061                 "ERR terminating due to %s (%d/%d) -----",
00062                 i<a ? "SIGINT":"SIGABRT", n-MIN(i,a), n
00063             );
00064         }
00065         return;
00066     }
00067
00068     unsigned l=128; char istr[l], s[l6];
00069     istr[0]=0; strcpy(s,"(signal)");
00070
00071     if (sig==SIGINT) {
00072         strcpy(s,"SIGINT"); --Wb::SigHandler::icount;
00073     }
00074     else if (sig==SIGABRT) {
00075         strcpy(s,"SIGABRT"); --Wb::SigHandler::account;
00076         i=a;
00077     }
00078     else { printf("\n");
00079         snprintf(istr,l,"%s ERR terminating due to uncaught signal %d",
00080             shortFL(gsh_F,gsh_L), sig);
00081         ExitMsg(istr); fflush(0); exit(1);
00082     }
00083
00084     if (i<=0) {
00085         snprintf(istr,l,"%s ERR terminating due to %s (%d/%d) %s",
00086             shortFL(gsh_F,gsh_L),s,n,n,"-----");
00087         ExitMsg(istr);
00088         fflush(0); exit(-1);
00089     }
00090
00091     if (i>=3) {
00092         snprintf(istr,l,"--- received %s (%d) %s",
00093             s,i,"-----"); }

```

```

00094     else if (i==2) {
00095         snprintf(istr,1,"--- received %s (%s)",
00096             s,"waiting for natural termination pt"); }
00097     else {
00098         snprintf(istr,1,"WRN next %s will result in immediate stop %s",
00099             s,"-----");
00100     }
00101
00102     time_t curr_time=time(NULL);
00103     struct tm *tblock=localtime(&curr_time);
00104     char t[16]; strftime(t,16,"%T",tblock);
00105
00106     fprintf(stdout,"%-20s %s %s\n",shortFL(gsh_F,gsh_L), t, istr);
00107
00108 };
00109
00110 void dbstop(const char* F, int L) {
00111
00112     unsigned ith=omp_get_thread_num(), nth=omp_get_num_threads();
00113     char sx[8]; sx[0]=0;
00114     if (nth>1) { snprintf(sx,8," (%d/%d)",ith,nth); }
00115
00116     fprintf(stdout,"%s %s %s() raising SIGTRAP%s ...\n\n",
00117         shortFL(F_L),TIME,FCT,sx);
00118     fflush(0);
00119
00120     raise(SIGTRAP);
00121 };
00122
00123 template<class T> inline
00124 int num2int(const char *F, int L, const T &x){
00125     double d=x; int i=round(d);
00126     if (F) {
00127         double e=fabs(d-i); if (e>1E-12) {
00128             wblog(F,L,"ERR %s() got non-int value %g !?",FCT,d); }
00129     }
00130     return i;
00131 };
00132
00133 template<> inline
00134 int num2int(const char *F __attribute__((unused)),
00135     int L __attribute__((unused)), const char &x){ return x; }
00136
00137 template<> inline
00138 int num2int(const char *F __attribute__((unused)),
00139     int L __attribute__((unused)), const unsigned char &x){ return x; }
00140
00141 template<> inline
00142 int num2int(const char *F __attribute__((unused)),
00143     int L __attribute__((unused)), const int &x){ return x; }
00144
00145 template<> inline
00146 int num2int(const char *F __attribute__((unused)),
00147     int L __attribute__((unused)), const long &x){ return x; }
00148
00149 template<> inline
00150 int num2int(const char *F __attribute__((unused)),
00151     int L __attribute__((unused)), const unsigned &x){ return x; }
00152
00153 template<> inline
00154 int num2int(const char *F __attribute__((unused)),
00155     int L __attribute__((unused)), const unsigned long &x){ return x; }
00156
00157 template<class T> inline
00158 unsigned checkInt(const char *F, int L,
00159     const T* x, size_t n, double eps
00160 ){
00161     for (size_t i=0; i<n; ++i) {
00162         if (fabs(x[i]-round(x[i]))>eps) {
00163             if (F) wblog(F_L,
00164                 "ERR %s() got data[%d]=%g (having %s) !?",FCT,
00165                 i+1, double(x[i]), TSTR(T));
00166             return (i+1);
00167         }
00168     }
00169     return 0;
00170 };
00171
00172 template<> inline
00173 unsigned checkInt(
00174     const char *F __attribute__((unused)),
00175     int L __attribute__((unused)),
00176     const int* x __attribute__((unused)),
00177     size_t n __attribute__((unused)),
00178     double eps __attribute__((unused))
00179 ){ return 0; }
00180

```

```

00181 template<> inline
00182 unsigned checkInt(
00183     const char *F      __attribute__((unused)),
00184     int L              __attribute__((unused)),
00185     const unsigned* x  __attribute__((unused)),
00186     size_t n           __attribute__((unused)),
00187     double eps         __attribute__((unused))
00188 ){ return 0; }
00189
00190 template<> inline
00191 unsigned checkInt(
00192     const char *F      __attribute__((unused)),
00193     int L              __attribute__((unused)),
00194     const char* x      __attribute__((unused)),
00195     size_t n           __attribute__((unused)),
00196     double eps         __attribute__((unused))
00197 ){ return 0; }
00198
00199 const char* Wb::basename(const char *data, char c) {
00200
00201     int i, n=strlen(data);
00202     for (i=n-1; i>=0; i--) if (data[i]==c) { i++; break; }
00203
00204     return (data + (i>0 ? i : 0));
00205 };
00206
00207 inline const char* Wb::strchr(const char *s, char c) {
00208
00209     if (!s) wblog(FL, "ERR %s() got invalid null string", FCT);
00210
00211     for (c=tolower(c); *s; ++s) {
00212         if (tolower(*s)==c) return s;
00213     }
00214
00215     return NULL;
00216 };
00217
00218 inline const char* Wb::rstrchr(const char *s1, const char *s2) {
00219     if (!s1 || !s2) wblog(FL, "ERR %s() got null strings (%p/%p)", FCT, s1, s2);
00220     if (!*s2) { return s1; }
00221     if (!*s1) { unsigned n=strlen(s1), m=strlen(s2);
00222         if (n>=m) { unsigned i,j,il=n-1;
00223             for (; il<n; --il) {
00224                 for (i=il, j=m-1; i<n && j<m && s1[i]==s2[j]; --i, --j) {};
00225                 if (j>m) { ++i; return s1+i; }
00226             }
00227         }
00228     }
00229     return NULL;
00230 };
00231
00232 inline const char* Wb::strstri(const char *s1, const char *s2) {
00233
00234     if (!s1 || !s2) wblog(FL,
00235         "ERR %s() got invalid null strings (%p, %p)", FCT, s1, s2);
00236
00237     const char *a, *b;
00238
00239     for(; *s1; ++s1) {
00240         for(a=s1, b=s2; *a && tolower(*a)==tolower(*b); ++a, ++b) {};
00241         if(!*b) return s1;
00242     }
00243
00244     return NULL;
00245 };
00246
00247 inline const char* Wb::strstrw(const char *s1, const char *s2) {
00248
00249     if (!s1 || !s2) wblog(FL,
00250         "ERR %s() got invalid null strings (%p, %p)", FCT, s1, s2);
00251
00252     unsigned i=0, j=0, k=0;
00253
00254     for(; s1[i]; ++i) {
00255         for (j=1, k=0; s1[j] && s1[j]==s2[k]; ++j, ++k) {};
00256         if (!s2[k]) {
00257             if ((!i || !Wb::isword(s1[i-1])) && !Wb::isword(s1[j]))
00258                 return s1+i;
00259         }
00260     }
00261
00262     return NULL;
00263 };
00264
00265 inline const char* Wb::strstrwi(const char *s1, const char *s2) {
00266
00267     if (!s1 || !s2) wblog(FL,

```



```

00268     "ERR %s() got invalid null strings (%p, %p)", FCT, s1, s2);
00269
00270     unsigned i=0, j=0, k=0;
00271
00272     for (; s1[i]; ++i) {
00273         for (j=i, k=0; s1[j] && tolower(s1[j])==tolower(s2[k]); ++j, ++k);
00274         if (!s2[k]) {
00275             if ((!i || !Wb::isword(s1[i-1])) && !Wb::isword(s1[j]))
00276                 return s1+i;
00277         }
00278     }
00279
00280     return NULL;
00281 };
00282
00283 void Wb::shift(char *s, unsigned n, int p, char erase) {
00284
00285     if (unsigned(abs(p))>=n) {
00286         if (erase && n) { unsigned i=0;
00287             if (p>0) { for (; i<n; ++i) { s[i]=' '; } }
00288             else {
00289                 for (; i<n && s[i]; ++i) { s[i]=' '; }
00290                 if (i<n) { s[0]=0; }
00291             }
00292         }
00293     }
00294     else if (p>0) {
00295         unsigned i=n-p-1; for (; i<n; --i) { s[i+p]=s[i]; }
00296         if (erase) { for (i=0; int(i)<p; ++i) { s[i]=' '; } }
00297     }
00298     else if (p<0) {
00299         unsigned i=-p; for (; i<n && s[i]; ++i) { s[i+p]=s[i]; }
00300         if (erase) {
00301             if (n>i) { n=i; }
00302             for (i=i+p+1; i<n && s[i]; ++i) { s[i]=' '; }
00303         }
00304     }
00305 };
00306
00307 void Wb::shift(char *s, unsigned n, int p, const char *s_) {
00308
00309     if (!p) return;
00310     if (!s || !s_) wblog(FL, "ERR %s() invalid input %p / %p", FCT, s, s_);
00311
00312     if (unsigned(abs(p))>=n) { if (n) {
00313         if (p>0) { strncpy(s, s_, n); }
00314         else {
00315             unsigned i=0; for (; i<n && s[i]; ++i) { }
00316             if (i<n) { s[0]=0; }
00317             else { strncpy(s, s_, n); }
00318         }
00319     } }
00320     else if (p>0) {
00321         unsigned i=n-p-1; for (; i<n; --i) { s[i+p]=s[i]; }
00322         for (i=0; int(i)<p; ++i) { s[i]=s_[i]; if (!s[i]) break; }
00323     }
00324     else if (p<0) { unsigned i=-p;
00325         for (; i<n && s[i]; ++i) { s[i+p]=s[i]; }
00326         for (i=n+p; i<n; ++i) { s[i]=s_[i]; if (!s[i]) break; }
00327     }
00328 };
00329
00330 template <class T>
00331 int Wb::charGetNumber(const char *F, int L, const char *s, T &x) {
00332
00333     float fl=0;
00334     int i;
00335
00336     if (!s) {
00337         sprintf(str, "cannot read number from empty string");
00338         return 1;
00339     }
00340
00341     i=sscanf (s, "%f", &fl);
00342     x=(T)fl;
00343
00344     if (!i || fl!=(float)x) {
00345         sprintf(str, "invalid input for type '%s' (%g, %d)",
00346             TSTR(T), fl, i); return 1;
00347     }
00348
00349     return 0;
00350 }
00351
00352 template<>
00353 int Wb::GetEnv (const char *F, int L, const char *name, double &val) {
00354

```

```

00355     char *s, *s2;
00356     double dbl;
00357
00358     s=getenv(name);    if (!s || !s[0]) return -1;
00359     dbl=strtod(s,&s2); if (!s2) return -2;
00360     val=dbl;
00361
00362     if (F) wblog1(F,L," *   getenv %-6s = %g", name, val);
00363
00364     if (*s2) {
00365         wblog1(FL,"WRN %s() got invalid %s='%s' !?",FCT,name,s);
00366         if (isspace(*s2)) {
00367             ++s2; while (*s2 && isspace(*s2)) ++s2;
00368         }
00369     }
00370
00371     return (*s2 ? -3 : 0);
00372 };
00373
00374 template<>
00375 int Wb::GetEnv (const char *F, int L, const char *name, int &val) {
00376
00377     double dbl=0;
00378
00379     int i=GetEnv(F,L,name,dbl); if (i) return i;
00380
00381     i=int(dbl); if (double(i)!=dbl) return -2;
00382     val=i;
00383
00384     return 0;
00385 }
00386
00387 template<>
00388 int Wb::GetEnv (const char *F, int L, const char *name, unsigned &val) {
00389
00390     double dbl=0;
00391     int i=GetEnv(F,L,name,dbl); if (i) return i;
00392
00393     i=unsigned(dbl); if (double(i)!=dbl) return -2;
00394     val=i;
00395
00396     return 0;
00397 };
00398
00399 template<>
00400 int Wb::GetEnv (const char *F, int L, const char *name, char &val) {
00401
00402     char *s=getenv(name); if (!s || !s[0]) return -1;
00403     if (s[1]) { return -2; }
00404     val=s[0];
00405
00406     if (F) wblog1(F,L," *   getenv %-6s = '%c'(%d)", name, val,val);
00407     return 0;
00408 };
00409
00410 template<>
00411 int Wb::GetEnv (const char *F, int L, const char *name, wstring &val) {
00412
00413     if (!name || !name[0]) wblog1(F,L,
00414         "ERR %s() invalid env '%s' !?",FCT,name);
00415
00416     char *s=getenv(name);
00417
00418     if (!s) {
00419         val.init(); return -1;
00420     }
00421     else {
00422         val=s;
00423         if (F) wblog1(F,L," *   getenv %-6s = '%s'",name,val.data);
00424         return 0;
00425     }
00426 };
00427
00428 int Wb::EnvIsSet(const char *F, int L, const char *name) {
00429     int x=0, e=GetEnv(F,L,name,x);
00430     return (!e && x>0 ? 1 : 0);
00431 };
00432
00433 int Wb::get_WB_VERBOSE(const char *F, int L) {
00434
00435     const char *vname="WB_VERBOSE";
00436     int i, k=0xFf;
00437
00438     if ((i=GetEnv(0,0,vname,k))==0) {
00439         if (k<0) wblog1(F,L,
00440             "WRN %s() invalid %s=%d/255",FCT,vname,k);
00441         else if (k!=Wb::envVRB) {

```

```

00442         if ((F && k) || k&8) { unsigned n=32; char s[n];
00443             i=sprintf(s,n,"%s()",myname);
00444             if (i<0) wblog(F_L,"WRN %s() i=%d",FCT,i);
00445             wblog1(PF_L,"ENV %-14s %s = %X->%X",s,vname,Wb::envVRB,k);
00446         }
00447         Wb::envVRB=k;
00448     }
00449 }
00450 else if (int(i)!=-1) {
00451     wblog1(F_L,"WRN %s() invalid %s (e=%d)",FCT,vname,i);
00452 }
00453
00454 { int q=0; i=GetEnv(0,0,"QS_LOG_COLOR",q);
00455     if (i) { q=(Wb::envDKT>1 ? WLC_DARK : WLC_OFF); } else
00456     if (q>=NUM_WBL_COLOR_SCHEME) { q=WLC_DARK; } else
00457     if (q<0) { q=WLC_OFF; }
00458     Wb::useCol=(WBL_COLOR_SCHEME)q;
00459 }
00460
00461     return k;
00462 };
00463
00464 int Wb::got_DBSTOP(const char *F, int L) {
00465
00466     int i=0,x=0,e,q;
00467     const char *ss[]={ "DBSTOP", "ML_DEBUG", "DEBUG" };
00468
00469     for (; i<3; ++i) { q=0;
00470         e=GetEnv(0,0,ss[i],q);
00471         if (!e) { if (q) { x|=(1<<i); } } else
00472         if (e!=-1 && F) wblog1(F_L,
00473             "WRN %s() invalid value for ENV %s (e=%d)",FCT,ss[i],e);
00474     }
00475
00476     if (x!=Wb::envDBG) {
00477         if ((F && x && Wb::envVRB&15) ||
00478             (Wb::envDBG && Wb::envVRB&8)) wblog1(PF_L," * "
00479             "using %-20s = %d -> %d", "Wb::envDBG", Wb::envDBG, x);
00480         Wb::envDBG=x;
00481     }
00482
00483     return x;
00484 };
00485
00486 #ifdef MATLAB_MEX_FILE
00487 int Wb::got_DESKTOP() {
00488
00489     int q=0;
00490     if (Wb::CallMatlab(0,0,"isdeployed")) { q=0; }
00491     else if (Wb::CallMatlab(0,0,"usejava","desktop")) { q=1; }
00492     else if (Wb::CallMatlab(0,0,"usejava","awt")) { q=2; }
00493
00494     if ((Wb::envDKT>=0 && Wb::envDKT!=q) || (Wb::envVRB&8)) { wblog1(F_L,
00495         "ENV %s() Wb::envDKT = %d (%s)",myname, q,
00496         q>1? "terminal":(q? "desktop":"batch"));
00497     }
00498     Wb::envDKT=q;
00499
00500     return q;
00501 };
00502
00503 int Wb::is_DEPLOYED(const char *F, int L) {
00504     return (int)Wb::CallMatlab(F,L,"isdeployed");
00505 };
00506
00507 #else
00508 int Wb::got_DESKTOP() { return 0; }
00509 int Wb::is_DEPLOYED(
00510     const char *F=0 __attribute__((unused)),
00511     int L=0 __attribute__((unused))) { return 1; }
00512 #endif
00513
00514 int Wb::GetNumThreads(const char *F, int L, int &n, const char *name) {
00515
00516     #ifndef QS_USING_OMP
00517         return 0;
00518     #endif
00519
00520     double q=0;
00521     int i=GetEnv(0,0,name,q), ncpu=wbsys::getNumCores();
00522
00523     if (i<0) {
00524         if (i<-1 && F) wblog1(F_L,
00525             "ERR %s() invalid value for env %s (i=%d)",FCT,name,i);
00526         return q;
00527     }
00528     if (q<0 || double(int(q))!=q) wblog1(PF_L,

```

```

00529     "ERR %s() invalid value %g\nfor env %s",FCT,q,name);
00530 if (q>ncpu) wblog1(PF_L,"ERR %s() value exceeds number of cores\n"
00531     "env %s = %g / %d",FCT,name,q,ncpu);
00532
00533     if (n!=q) { i=0;
00534
00535         if (!GetEnv(0,0,"ML_DEBUG",i) && i<1) { q=0; } else
00536         if (!GetEnv(0,0,"DEBUG", i) && i<1) { q=0; } else
00537         if (F && (Wb::envVRB &12)
00538             #ifdef LOAD_CGC_QSPACE
00539             || CG_VERBOSE>6
00540             #endif
00541         )) wblog1(PF_L," * using %-20s = %d -> %g",name,n,q);
00542         n=q;
00543     }
00544
00545     return q;
00546 };
00547
00548 int Wb::strrep(
00549     const char *S0,
00550     const char *t0,
00551     const char *t2,
00552     char *Sout,
00553     size_t N,
00554     char gflag
00555 ){
00556     if (!S0 || !S0[0] || !t0 || !t0[0]) return 0;
00557
00558     int nrep=0;
00559     size_t k, l=0, m0=strlen(t0), m=(t2 ? strlen(t2):0);
00560     const char *s, *s0=S0; char *s2=Sout;
00561
00562     while ((s=strstr(s0,t0))) { k=s-s0; ++nrep;
00563         if (k) {
00564             if (l+k>N) wblog(FI,
00565                 "ERR string out of bounds (%d+%d / %d)",l,k,N);
00566             memcpy(s2,s0,k); l+=k; s2+=k;
00567         }
00568         if (m) {
00569             if (l+m>N) wblog(FI,
00570                 "ERR string out of bounds (%d+%d / %d)",l,m,N);
00571             memcpy(s2,t2,m); l+=m; s2+=m;
00572         }
00573         s0+=(k+m0); if (!gflag) break;
00574     }
00575
00576     if ((k=strlen(s0))) {
00577         if (l+k>N) wblog(FI,
00578             "ERR string out of bounds (%d+%d / %d)",l,k,N);
00579         memcpy(s2,s0,k); l+=k; s2+=k;
00580     }
00581     s2[0]=0;
00582
00583     return nrep;
00584 };
00585
00586 wstring Wb::repHome(const char *file) {
00587
00588     size_t n=2*strlen(file);
00589     const char *s; char F0[n+1], F[n+1];
00590     char *f0=F0, *f=F; strcpy(f,file);
00591
00592     if ((s=getenv("MEX" ))) {
00593         SWAP(f,f0); if (Wb::strrep(f0,s,"$MEX", f,n)) return f; }
00594     if ((s=getenv("HOME" ))) {
00595         SWAP(f,f0); if (Wb::strrep(f0,s,"$HOME",f,n)) return f; }
00596     if ((s=getenv("LMA" ))) {
00597         SWAP(f,f0); if (Wb::strrep(f0,s,"$LMA", f,n)) return f; }
00598     if ((s=getenv("PROJ" ))) {
00599         SWAP(f,f0); if (Wb::strrep(f0,s,"$PROJ",f,n)) return f; }
00600     if ((s=getenv("PROJ_LEO" ))) {
00601         SWAP(f,f0); if (Wb::strrep(f0,s,"$PROJ_LEO",f,n)) return f; }
00602     if ((s=getenv("USER" ))) {
00603         SWAP(f,f0); if (Wb::strrep(f0,s,"$USER",f,n)) return f; }
00604
00605     return f;
00606 };
00607
00608 void Wb::print_backtrace(const char *F, int L, const char *istr) {
00609
00610     unsigned i=0, n=4;
00611     GetEnv(0,0,"QS_NUM_BTRACE",i); if (n<i) { n=i; }
00612
00613     void *array[n];
00614
00615     n=backtrace(array,n);

```

```

00616
00617     char **ss=backtrace_symbols(array,n);
00618
00619     printf("\n# %s: stack backtrace",shortFL(F_L));
00620     if (istr && istr[0]) { printf(" %s",istr); }
00621     printf(" (%d frame%s)\n",n, n!=1? "s":"" );
00622     printf("# hint: may use addr2line to decode this\n\n");
00623
00624     if (ss) {
00625         for (i=0; i<n; ++i) {
00626             printf(" [%d] %s\n", n>10? 2:1, i,ss[i]);
00627         }; printf("\n");
00628         free(ss);
00629     }
00630 };
00631
00632 std::string getName(
00633     const std::type_info &t,
00634     char v
00635 ){
00636
00637     std::string s;
00638
00639     if (t==typeid(float)        ) s=(v? "float"      : "f" ); else
00640     if (t==typeid(double)       ) s=(v? "double"     : "d" ); else
00641     if (t==typeid(bool)         ) s=(v? "bool"        : "b" ); else
00642     if (t==typeid(char)         ) s=(v? "char"        : "c" ); else
00643     if (t==typeid(unsigned char)) s=(v? "uchar"      : "uc" ); else
00644     if (t==typeid(int)          ) s=(v? "int"         : "i" ); else
00645     if (t==typeid(unsigned)     ) s=(v? "unsigned"    : "u" ); else
00646     if (t==typeid(long)         ) s=(v? "long"        : "l" ); else
00647     if (t==typeid(size_t)       ) s=(v? "size_t"      : "st" ); else
00648     if (t==typeid(unsigned long long)) s=(v? "uLong"   : "uL" ); else
00649
00650     if (t==typeid(double*)      ) s=(v? "double*"    : "d*" ); else
00651     if (t==typeid(float*)       ) s=(v? "float*"      : "f*" ); else
00652     if (t==typeid(bool*)        ) s=(v? "bool*"       : "b*" ); else
00653     if (t==typeid(char*)        ) s=(v? "char*"       : "c*" ); else
00654     if (t==typeid(int*)         ) s=(v? "int*"        : "i*" ); else
00655     if (t==typeid(unsigned*)    ) s=(v? "unsigned*"   : "u*" ); else
00656     if (t==typeid(long*)        ) s=(v? "long*"       : "l*" ); else
00657     if (t==typeid(size_t*)      ) s=(v? "size_t*"     : "st*" ); else
00658     if (t==typeid(unsigned long long*)) s=(v? "uLong*" : "uL*" ); else
00659
00660     if (t==typeid(const double*)) s=(v? "c-double*"  : "Kd*"); else
00661     if (t==typeid(const float*))  s=(v? "c-float*"   : "Kf*"); else
00662     if (t==typeid(const bool*))   s=(v? "c-bool*"    : "Kb*"); else
00663     if (t==typeid(const char*))   s=(v? "c-char*"    : "Kc*"); else
00664     if (t==typeid(const int*))    s=(v? "c-int*"     : "Ki*"); else
00665     if (t==typeid(const unsigned*)) s=(v? "c-unsigned*": "Ku*"); else
00666     if (t==typeid(const long*))   s=(v? "c-long*"    : "Kl*"); else
00667     if (t==typeid(const size_t*)) s=(v? "c-size_t*"  : "Kst*"); else
00668
00669     if (t==typeid(wbcomplex)     ) s=(v? "wbcomplex" : "z" ); else
00670     if (t==typeid(wbcomplex*))   s=(v? "wbcomplex*" : "z*" ); else
00671
00672     if (t==typeid(long double)   ) s=(v? "long double" : "dl" ); else
00673     if (t==typeid(long double*)) s=(v? "long double*" : "dl*" ); else
00674
00675     #ifdef __WB_MPPFR_HH__
00676     if (t==typeid(Wb::quad)      ) s=(v? "Wb::quad"   : "q" ); else
00677     if (t==typeid(Wb::qqquad)   ) s=(v? "Wb::qqquad" : "qq" ); else
00678     if (t==typeid(Wb::quad2)    ) s=(v? "Wb::quad2"  : "q2" ); else
00679     if (t==typeid(Wb::quad3)    ) s=(v? "Wb::quad3"  : "q2" ); else
00680
00681     if (t==typeid(Wb::quad*)    ) s=(v? "Wb::quad"   : "q*" ); else
00682     if (t==typeid(Wb::qqquad*)  ) s=(v? "Wb::qqquad" : "qq*"); else
00683     if (t==typeid(Wb::quad2*)   ) s=(v? "Wb::quad2"  : "q2*"); else
00684     if (t==typeid(Wb::quad3*)   ) s=(v? "Wb::quad3"  : "q2*"); else
00685     #endif
00686
00687     s=(char*)(t.name());
00688
00689     return s;
00690 };
00691
00692 bool Wb::isBaseType(const std::type_info &t) {
00693
00694     if ( t==typeid(unsigned)
00695         || t==typeid(double)
00696         || t==typeid(float)
00697         || t==typeid(int)
00698         || t==typeid(char)
00699         || t==typeid(unsigned char)
00700         || t==typeid(long long)
00701         || t==typeid(size_t)
00702         || t==typeid(wbcomplex)

```

```

00703     ) return 1;
00704
00705     return 0;
00706 };
00707
00708 bool isComplexType(const std::type_info &t) {
00709     if ( t==typeid(wbcomplex) ) return 1;
00710
00711     return 0;
00712 };
00713
00714
00715 template<class T>
00716 Wb::num2Fmt<T>::operator wbstring() const {
00717     wbstring s;
00718     get_fmt(s); return s;
00719 };
00720
00721 template<class T>
00722 void Wb::num2Fmt<T>::get_fmt(wbstring &s) const {
00723
00724     unsigned l=0; s.init(12);
00725
00726     if (m<-1 || p<-1 || t[0]<=0) wblog(FL,
00727         "ERR %s<%s> invalid type %d.%d%s",FCT,TSTR(T),m,p,t);
00728
00729     if (m>0) {
00730         if (p>=0)
00731             { l=snprintf(s.data,s.len,"%%d.%d%s",m,p,t); }
00732         else { l=snprintf(s.data,s.len,"%%d%s",m,t); }
00733     }
00734     else {
00735         if (p>=0)
00736             { l=snprintf(s.data,s.len,"%%.%d%s",p,t); }
00737         else { l=snprintf(s.data,s.len,"%%s",t); }
00738     }
00739
00740     if (l>=s.len) wblog(FL,
00741         "ERR %s<%s> string out of bounds (%%d.%d%s; %d/%d)",
00742         FCT,TSTR(T),m,p,t,l,s.len
00743     );
00744 };
00745
00746 template<>
00747 void Wb::num2Fmt<wbcomplex>::get_fmt(wbstring &s) const {
00748
00749     unsigned l=0; s.init(16);
00750
00751     if (m<-1 || p<-1 || t[0]<=0) wblog(FL,
00752         "ERR %s<%s> invalid type %d.%d%s",FCT,TSTR(wbcomplex),m,p,t);
00753
00754     for (int i=0; i<2; ++i) {
00755         l+=snprintf(s.data+l,s.len-l,"%%s",i?"+"+"");
00756         if (m>0) {
00757             if (p>=0)
00758                 { l+=snprintf(s.data+l,s.len-l,"%d.%d%s",m,p,t); }
00759             else { l+=snprintf(s.data+l,s.len-l,"%d%s",m,t); }
00760         }
00761         else {
00762             if (p>=0)
00763                 { l+=snprintf(s.data+l,s.len-l,".%d%s",p,t); }
00764             else { l+=snprintf(s.data+l,s.len-l,"%s",t); }
00765         }
00766     }
00767
00768     if (l+2>=s.len) wblog(FL,
00769         "ERR %s<%s> string out of bounds (%%d.%d%s; %d/%d)",
00770         FCT,TSTR(wbcomplex),m,p,t,l,s.len
00771     );
00772
00773     snprintf(s.data+l,s.len-l,"i");
00774 };
00775
00776 template<class T>
00777 int Wb::num2Fmt<T>::check_init() {
00778     wblog(FL,"ERR %s<> invalid data type `%s'",FCT,TSTR(T));
00779     return 1;
00780 };
00781
00782 template<> inline
00783 int Wb::num2Fmt<double>::check_init() {
00784     if (m<-1) m=8;
00785     if (p<-1) p=5;
00786     if (t[0])
00787         { if (!Wb::strchr("ge",t[0])) return 't'; }
00788     else { strcpy(t,"g"); }
00789     return 0;

```

```

00790 };
00791 template<> inline
00792 int Wb::num2Fmt<float>::check_init() {
00793     return ((num2Fmt<double>*)this)->check_init();
00794 };
00795 template<> inline
00796 int Wb::num2Fmt<wbcomplex>::check_init() {
00797     return ((num2Fmt<double>*)this)->check_init();
00798 };
00799
00800 template<> inline
00801 int Wb::num2Fmt<int>::check_init() {
00802     if (m<-1) m=4;
00803     if (p>=0) return 'p';
00804     if (t[0])
00805         { if (!strchr("ducxX",t[0])) return 't'; }
00806     else { strcpy(t,"d"); }
00807     return 0;
00808 };
00809 template<> inline
00810 int Wb::num2Fmt<unsigned>::check_init() {
00811     return ((num2Fmt<int>*)this)->check_init();
00812 };
00813
00814 template<> inline
00815 int Wb::num2Fmt<long>::check_init() {
00816     if (m<-1) m=4;
00817     if (p>=0) return 'p';
00818     if (t[0])
00819         { if (!strchr("lducxX",t[0])) return 't'; }
00820     else { strcpy(t,"ld"); }
00821     return 0;
00822 };
00823
00824 template<> inline
00825 int Wb::num2Fmt<unsigned long>::check_init() {
00826     return ((num2Fmt<long>*)this)->check_init();
00827 };
00828
00829 template<> inline
00830 int Wb::num2Fmt<char>::check_init() {
00831     if (m<-1) m=4;
00832     if (p>=0) return 'p';
00833     if (t[0])
00834         { if (!strchr("ducxX",t[0])) return 't'; }
00835     else { strcpy(t,"d"); }
00836     return 0;
00837 };
00838
00839 template<> inline
00840 int Wb::num2Fmt<char*>::check_init() {
00841     if (m<-1) m=-1;
00842     if (p>=0) return 'p';
00843     if (t[0])
00844         { if (t[0]!='s') return 't'; }
00845     else { strcpy(t,"s"); }
00846     return 0;
00847 };
00848
00849 template <class T> inline
00850 char* defaultFmt(char *fmt,
00851 const T &x __attribute__((unused)), int m, int p, char t) {
00852
00853     const size_t n=16; size_t l=0;
00854     char dd[n]; dd[0]=0; fmt[0]=0;
00855
00856     if (m>0) {
00857         if (p>=0)
00858             l=snprintf(dd,n,"%d.%d",m,p);
00859         else l=snprintf(dd,n,"%d",m);
00860     }
00861     else if (p>=0) l=snprintf(dd,n,"%d",p);
00862
00863     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00864
00865     if (typeid(T)==typeid(double) || typeid(T)==typeid(float)) {
00866         if (!t) t='g';
00867         sprintf(fmt,"%%%s%c",dd,t);
00868     }
00869     else if (typeid(T)==typeid(wbcomplex)) {
00870         if (!t) t='g';
00871         sprintf(fmt,"%%%s%c%%+%%s%ci",dd,t,dd,t);
00872     }
00873     else if (
00874         typeid(T)==typeid(int) || typeid(T)==typeid(char) ||
00875         typeid(T)==typeid(unsigned)
00876     ){

```

```

00877         if (p>=0) wblog(FL,"WRN %s() got format *.%d* for type '%s'",
00878             FCT, p, TSTR(T));
00879         if (!t) t='d';
00880         sprintf(fmt,"%%%s%c",dd,t);
00881     }
00882     else if (typeid(T)==typeid(char*)) {
00883         if (t && t!='s') wblog(FL,
00884             "ERR %s() got format type '%c'<%d> for type '%s'",
00885             FCT,t,t,TSTR(T)
00886         );
00887         sprintf(fmt,"%%%ss",dd);
00888     }
00889     else wblog(FL,
00890         "ERR %s() unsupported type '%s'",FCT,TSTR(T));
00891
00892     return fmt;
00893 };
00894
00895 wbsting Wb::TimeStamp(char type) {
00896
00897     wbsting s;
00898     unsigned l,n;
00899
00900     #pragma omp critical (got_CPTR_TIME)
00901     { time_t t=time(NULL);
00902       struct tm *tb=localtime(&t);
00903       if (type=='t' || type=='T') {
00904           s.init(32); l=strftime(s.data,s.len,"%T",tb);
00905
00906           if (type=='T') {
00907               using namespace std::chrono;
00908               time_point<system_clock> t=system_clock::now();
00909
00910               l+=snprintf(s.data+l,s.len-l,"%03ld",
00911                   size_t(duration_cast<milliseconds>(t.time_since_epoch()).count())%1000);
00912           }
00913       }
00914       else if (type=='D') {
00915           s.init(32); l=strftime(s.data,s.len,"%d-%b-%Y %Z",tb);
00916       }
00917       else { s=asctime(tb); }
00918     }
00919
00920     n=strlen(s.data);
00921     if (n && s.data[n-1]<32) s.data[n-1]=0;
00922
00923     return s;
00924 }
00925
00926 template <class T1, class T2> inline
00927 void Wb::safeConvert(const char *F, int L, const T1 &x1, T2 &x2) {
00928     x2=T2(x1); if (T1(x2)!=x1) wblog(F_L,
00929         "ERR converting %s -> %s changes value\n%g, %g",
00930         TSTR(T1), TSTR(T2),
00931         double(x1), double(x2)
00932     );
00933 }
00934
00935 template <> inline
00936 void Wb::safeConvert(const char *F, int L, const wbcplx &x1, double &x2) {
00937     if (x1.i!=0.) wblog(F_L,
00938         "ERR cannot cast complex number to real (%.4g%+.4gi)",x1.r,x1.i);
00939     x2=x1.r;
00940 }
00941
00942 template <> inline
00943 void Wb::safeConvert(
00944     const char *F __attribute__((unused)), int L __attribute__((unused)),
00945     const double &x1, wbcplx &x2
00946 ){ x2=wbcplx(x1,0); }
00947
00948 template <class T> inline
00949 bool Wb::isLower(const T* a, const T* b, const size_t n) {
00950     for (size_t i=0; i<n; i++) {
00951         if (a[i]<b[i]) return 1; else
00952         if (a[i]>b[i]) return 0;
00953     }
00954     return 0;
00955 }
00956
00957 template <class T> inline
00958 bool Wb::isEqual(const T* a, const T* b, const size_t n) {
00959     for (size_t i=0; i<n; i++) { if (a[i]!=b[i]) return 0; }
00960     return 1;
00961 }
00962 }
00963

```



```

00964 template <class T> inline
00965 bool Wb::allEqual(const T* a, const size_t n, const T &x) {
00966
00967     for (size_t i=0; i<n; i++) { if (a[i]!=x) return 0; }
00968     return 1;
00969 }
00970
00971 template <class T> inline
00972 bool Wb::anyUnequal(const T* a, const size_t n, const T &x) {
00973     for (size_t i=0; i<n; i++) { if (a[i]!=x) return 1; }
00974     return 0;
00975 };
00976
00977 template <class T> inline
00978 bool Wb::anyEqual(const T* a, const size_t n, const T &x) {
00979     for (size_t i=0; i<n; i++) { if (a[i]==x) return 1; }
00980     return 0;
00981 };
00982
00983
00984 template <class T> inline
00985 void Wb::scale_eps(T &eps, const T* d, size_t n) {
00986     if (double(eps)>0) {
00987         if (double(eps)>1E-8) wblog(FL,
00988             "WRN %s() got eps=%g (ignore)",FCT,double(eps));
00989         else {
00990             T x=getdscale(d,n);
00991             if (x>T(1E-8)) eps*=x;
00992         }
00993     }
00994 };
00995
00996 template <class T>
00997 T Wb::getdscale(const T* data, size_t n) {
00998
00999     if (n) { T d,x=0;
01000         for (size_t i=0; i<n; ++i) { d=ABS(data[i]); if (x<d) x=d; }
01001         return x;
01002     }
01003     return 0;
01004 };
01005
01006 inline
01007 void Wb::update_range_stride(size_t &n, const size_t stride) {
01008     if (stride!=1) {
01009         if (int(stride)<=0 && n) wblog(FL,
01010             "ERR %s() got stride=%ld (n=%ld) !?",FCT,stride,n);
01011         n*=stride;
01012     }
01013 };
01014
01015 template <class T> inline
01016 T Wb::addRange2(const T* d, size_t n, const size_t stride) {
01017     T s=0; Wb::update_range_stride(n,stride);
01018     for (size_t i=0; i<n; i+=stride) { s+=ABS2(d[i]); }
01019     return s;
01020 };
01021
01022 template <class T> inline
01023 T Wb::addRange(const T* d, size_t n, const size_t stride) {
01024     T s=0; Wb::update_range_stride(n,stride);
01025     for (size_t i=0; i<n; i+=stride) { s+=d[i]; }
01026     return s;
01027 };
01028
01029 template <class T> inline
01030 void Wb::addRange(const T* a, T* c, size_t n, const size_t stride) {
01031     Wb::update_range_stride(n,stride);
01032     for (size_t i=0; i<n; i+=stride) { c[i]+=a[i]; }
01033 };
01034
01035 template <class T> inline
01036 void Wb::addRange(
01037     const T* a, const T* b, T* c, size_t n, size_t stride
01038 ){
01039     Wb::update_range_stride(n,stride);
01040     for (size_t i=0; i<n; i+=stride) { c[i]=a[i]+b[i]; }
01041 };
01042
01043 template <class T> inline
01044 void Wb::minusRange(
01045     const T* a, const T* b, T* c, size_t n, size_t stride
01046 ){
01047     Wb::update_range_stride(n,stride);
01048     for (size_t i=0; i<n; i+=stride) { c[i]=a[i]-b[i]; }
01049 };
01050

```

```

01051 template <class T> inline
01052 void Wb::diffRange(
01053     const T* a, const T* b, T* c, size_t n, size_t stride
01054 ){
01055     Wb::update_range_stride(n, stride);
01056     for (size_t i=0; i<n; i+=stride) { c[i]=a[i]-b[i]; }
01057 };
01058
01059 template <class T> inline
01060 void Wb::setRange2avg(T* a, size_t n) {
01061     T x=0; size_t i;
01062     for (i=0; i<n; ++i) { x+=a[i]; }; x*=(T(1)/n);
01063     for (i=0; i<n; ++i) { a[i]=x; }
01064 };
01065
01066 template <class T> inline
01067 size_t Wb::nnzRange(const T* a, size_t n) {
01068     size_t m=0, i=0;
01069     for (; i<n; ++i) { if (a[i]!=0) m++; }
01070     return m;
01071 };
01072
01073 template <class T> inline
01074 T Wb::rangeNormDiff2(
01075     const T* d1, const T* d2, size_t n
01076 ){
01077     if (!n || d1==d2) return 0;
01078
01079     T x2, x2sum=0;
01080
01081     for (size_t i=0; i<n; ++i) {
01082         x2=d1[i]-d2[i]; x2*=CONJ(x2);
01083         x2sum+=x2;
01084     }
01085
01086     return x2sum;
01087 };
01088
01089 template <class T> inline
01090 T Wb::rangeNormDiff2(
01091     const T* d1, const T* d2, size_t n, const T &fac, size_t *k
01092 ){
01093     if (fac==1 && !k) { return rangeNormDiff2(d1,d2,n); }
01094     if (!n || (d1==d2 && fac==1)) { if (k) { (*k)=0; }; return 0; }
01095
01096     T x2, x2sum=0;
01097
01098     if (k) { T x2max=0;
01099         for (size_t i=0; i<n; ++i) {
01100             x2=d1[i]-fac*d2[i]; x2*=CONJ(x2);
01101             x2sum+=x2; if (x2>x2max) { x2max=x2; (*k)=i; }
01102         }
01103     }
01104     else {
01105         for (size_t i=0; i<n; ++i) {
01106             x2=d1[i]-fac*d2[i]; x2*=CONJ(x2);
01107             x2sum+=x2;
01108         }
01109     }
01110
01111     return x2sum;
01112 };
01113
01114 template <class T> inline
01115 T Wb::rangeNorm2(const T* d, size_t n, size_t *k
01116 ){
01117     T x2sum=0;
01118     if (!n) { if (k) { (*k)=0; }; return x2sum; }
01119
01120     if (k) {
01121         T x2, x2max=0; (*k)=0;
01122
01123         for (size_t i=0; i<n; ++i) {
01124             x2=CONJ(d[i])*d[i]; if (x2>x2max) { x2max=x2; (*k)=i; }
01125             x2sum+=x2;
01126         }
01127     }
01128     else {
01129         for (size_t i=0; i<n; ++i) { x2sum+=CONJ(d[i])*d[i]; }
01130     }
01131
01132     return x2sum;
01133 };
01134
01135 template <class T> inline
01136 T Wb::rangeMaxDiff(const T* d1, const T* d2, size_t n, size_t *k) {
01137     size_t imax=0, i=0; T x, xmax=0;

```

```

01138
01139     if (!n || d1==d2) { if (k) (*k)=0;
01140         if (!n) wblog(FL,"WRN %s() got empty input",FCT);
01141         return 0;
01142     }
01143
01144     for (; i<n; i++) {
01145         x=ABS(d1[i]-d2[i]); if (x>xmax) { xmax=x; imax=i; }
01146     }
01147
01148     if (k) (*k)=imax;
01149     return xmax;
01150 };
01151
01152 template <class T, class T2>
01153 void Wb::cpyStride(
01154     T2* d2,
01155     const T* d0,
01156     unsigned n,
01157     const unsigned *idx, unsigned m,
01158     unsigned D2,
01159     unsigned D0,
01160     char add
01161 ){
01162     if (!n || !m) { return; }
01163     if (!idx) { return Wb::cpyStride(d2,d0,n,m,D2,D0, T2(add? 1:0), T(1)); }
01164
01165     unsigned i=0, j; const T *d0_=d0;
01166
01167     if (int(D0)<0) { D0=n; } else if (D0<n) { ++i; }
01168     if (int(D2)<0) { D2=n; } else if (D2<n) { ++i; }
01169
01170     if (i) wblog(FL,
01171         "ERR %s() stride too small (%d,%d/%d)",FCT,D0,D2,n);
01172     if (gotMemOverlap(d0, (m-1)*D0+n, d2, (m-1)*D2+n)) wblog(FL,
01173         "ERR %s() must not copy onto itself",FCT);
01174
01175     for (j=0; j<m; ++j, d2+=D2) { d0 = d0_+ idx[j] * D0;
01176         if (!add) { for (i=0; i<n; ++i) { d2[i] = d0[i]; }}
01177         else { for (i=0; i<n; ++i) { d2[i] += d0[i]; }}
01178     }
01179 };
01180
01181 template <class T, class T2>
01182 void Wb::cpyStride(
01183     T2* d2,
01184     const T* d0,
01185     size_t n,
01186     size_t m,
01187     size_t D2,
01188     size_t D0,
01189     T2 fac2, T fac0
01190 ){
01191     if (!n || !m) { return; }
01192
01193     unsigned j=0;
01194     if (int(D0)<0) { D0=n; } else if (D0<n) { ++j; }
01195     if (int(D2)<0) { D2=n; } else if (D2<n) { ++j; }
01196
01197     if (j) wblog(FL,
01198         "ERR %s() stride too small (%d,%d/%d)",FCT,D0,D2,n);
01199     if (gotMemOverlap(d0, (m-1)*D0+n, d2, (m-1)*D2+n)) wblog(FL,
01200         "ERR %s() must not copy onto itself",FCT);
01201
01202     for (j=0; j<m; ++j, d2+=D2, d0+=D0) {
01203         Wb::cpyRange(d2,d0,n,fac2,fac0);
01204     }
01205 };
01206
01207 template <class T, class TB>
01208 void Wb::cpyRange(T* a, const TB* b, size_t n, T afac, TB bfac) {
01209
01210     size_t i=0;
01211
01212     if (bfac) {
01213         if (!afac) {
01214             if (bfac==TB( 1)) { for (; i<n; ++i) { a[i] = b[i]; }} else
01215             if (bfac==TB(-1)) { for (; i<n; ++i) { a[i] =-b[i]; }}
01216             else { for (; i<n; ++i) { a[i] = b[i]*bfac; }}
01217         }
01218         else if (afac==T(1)) {
01219             if (bfac==TB( 1)) { for (; i<n; ++i) { a[i]+= b[i]; }} else
01220             if (bfac==TB(-1)) { for (; i<n; ++i) { a[i]-= b[i]; }}
01221             else { for (; i<n; ++i) { a[i]+= b[i]*bfac; }}
01222         }
01223     }
01224     if (bfac==TB( 1)) { for (; i<n; ++i) { (a[i]*=afac) += b[i]; }} else

```

```

01225         if (bfac==TB(-1)) { for (; i<n; ++i) { (a[i]*=afac) -= b[i]; }}
01226     else { for (; i<n; ++i) { (a[i]*=afac) += b[i]*bfac; }}
01227     }
01228 }
01229 else if (afac!=T(1)) {
01230     if (afac) { for (; i<n; ++i) { a[i]*=afac; }}
01231     else { T z=0; for (; i<n; ++i) { a[i]=z; }}
01232 }
01233 };
01234
01235 template <class T>
01236 void Wb::cpyRange(T* a, const T* b, size_t n) {
01237     for (size_t i=0; i<n; ++i) { a[i]=b[i]; }
01238 };
01239
01240 template <class T, class Tb>
01241 void Wb::cpyRange(T* a, const Tb* b, size_t n, char tcheck) {
01242     if (!tcheck) {
01243         for (size_t i=0; i<n; ++i) { a[i]=T(b[i]); }}
01244     else {
01245         for (size_t i=0; i<n; ++i) { a[i]=T(b[i]);
01246             if (Tb(a[i])!=b[i]) wblog(FL,
01247                 "ERR %s() rounding error (%g/%g)",FCT,double(a[i]),double(b[i])
01248             );
01249         }
01250     }
01251 }
01252 };
01253
01254 template <>
01255 void Wb::cpyRange(double* a, const double* b, size_t n) {
01256     memcpy(a,b,n*sizeof(double));
01257 };
01258
01259 template <>
01260 void Wb::cpyRange(wbcomplex* a, const wbcomplex* b, size_t n) {
01261     memcpy(a,b,n*sizeof(wbcomplex));
01262 };
01263
01264 template <>
01265 void Wb::cpyRange(int* a, const int* b, size_t n) {
01266     memcpy(a,b,n*sizeof(int));
01267 };
01268
01269 template <>
01270 void Wb::cpyRange(double* a, const wbcomplex* b, size_t n, char tcheck) {
01271     double i2=0;
01272     for (size_t i=0; i<n; ++i) {
01273         a[i]=b[i].r; if (b[i].i) { i2+=(b[i].i*b[i].i); }
01274     }
01275     if (i2) {
01276         sprintf(str,"%s() got |Im(b)|^2 = %.3g",FCT,i2);
01277         wblog(FL,tcheck? "ERR %s":"WRN %s",str);
01278     }
01279 };
01280
01281 template <>
01282 void Wb::cpyRange(wbcomplex* a, const double* b, size_t n, char tcheck) {
01283     for (size_t i=0; i<n; ++i) { a[i]=b[i]; }
01284 };
01285
01286 template <class T>
01287 void Wb::cpyRangeR(T* a, const wbcomplex* z, size_t n) {
01288     for (size_t i=0; i<n; ++i) { a[i]=z[i].r; }
01289 };
01290
01291 template <class T>
01292 void Wb::cpyRangeI(T* a, const wbcomplex* z, size_t n) {
01293     for (size_t i=0; i<n; ++i) { a[i]=z[i].i; }
01294 };
01295
01296 template <class T>
01297 void Wb::cpyRangeA(T* a, const wbcomplex* z, size_t n) {
01298     for (size_t i=0; i<n; ++i) { a[i]=z[i].abs(); }
01299 };
01300
01301 template <class T> inline
01302 bool Wb::uniformRange(const T* d, size_t n) {
01303     for (size_t i=1; i<n; ++i) if (d[i]!=d[0]) return 0;
01304     return 1;
01305 };
01306
01307 template <class T> inline
01308 T Wb::maxRange(const T* d, size_t n) {
01309     T m=d[0];
01310     for (size_t i=1; i<n; ++i) if (d[i]>m) m=d[i];
01311 }

```

```

01312     if (!n) wblog(FL,"ERR %s() got empty range",FCT);
01313
01314     T x=d[0];
01315     for (size_t i=1; i<n; ++i) { if (x<d[i]) { x=d[i]; }}
01316     return x;
01317 };
01318
01319 template <class T> inline
01320 T Wb::minRange(const T* d, size_t n) {
01321
01322     if (!n) wblog(FL,"ERR %s() got empty range",FCT);
01323
01324     T x=d[0];
01325     for (size_t i=1; i<n; ++i) { if (x>d[i]) { x=d[i]; }}
01326     return x;
01327 };
01328
01329 template <class T> inline
01330 T Wb::prodRange(const T* d, size_t n) {
01331     if (!n) wblog(FL,"WRN %s() got empty range (returning 0)",FCT);
01332     return prodRange(d,n,T(0));
01333 };
01334
01335 template <class T> inline
01336 T Wb::prodRange(const T* d, size_t n, T x) {
01337     if (n) { size_t i=1;
01338         if (!d) wblog(FL,"ERR %s() got null space (n=%d)",FCT,n);
01339         for (x=d[0]; i<n; ++i) x*=d[i];
01340     }
01341     return x;
01342 };
01343
01344 template <class TD, class TX> inline
01345 void Wb::timesRange(TD* d, TX x, size_t n, size_t stride) {
01346     if (!n) return;
01347     if (!d) wblog(FL,"ERR %s() got null space (n=%d)",FCT,n);
01348
01349     Wb::update_range_stride(n,stride);
01350     for (size_t i=0; i<n; i+=stride) { d[i]*=x; }
01351 };
01352
01353 template <class T> inline
01354 char Wb::check_conj_flag(const T* a, char &conj) {
01355     if (conj) {
01356         if (!ISCOMPLX(T)) { conj=0; } else {
01357             if (conj>2) {
01358                 if (strchr("2bBcC*",conj)) { conj=2; } else
01359                 if (strchr("1aA", conj)) { conj=1; }
01360             }
01361             if (conj<0 || conj>2) wblog(FL,
01362                 "WRN %s() invalid conj=%s",FCT,cSTR(conj));
01363         }
01364     }
01365     return conj;
01366 };
01367
01368 template <class T> inline
01369 T Wb::dotProd(const T* a, const T* b, size_t n, T x, char conj) {
01370     if (n) {
01371         size_t i=0; x=0;
01372         if (!a || !b) wblog(FL,"ERR %s() got null (%p, %p)",FCT,a,b);
01373
01374         if (check_conj_flag(b,conj)) {
01375             if (conj==1)
01376                 { for (; i<n; ++i) x+=CONJ(a[i])*b[i]; }
01377             else { for (; i<n; ++i) x+=a[i]*CONJ(b[i]); }
01378         }
01379         else { for (; i<n; ++i) x+=(a[i]*b[i]); }
01380     }
01381     return x;
01382 };
01383
01384 template <class T> inline
01385 T Wb::dotProd(const T* a, const T* b, size_t n, char conj) {
01386     T x=T(0);
01387
01388     if (n) { x=dotProd(a,b,n,x,conj); }
01389     else {
01390         wblog(FL,"WRN %s() got empty range",FCT);
01391     }
01392
01393     return x;
01394 };
01395
01396 template <class T> inline
01397 void Wb::TimesElRange(T* a, const T* b, size_t n, char conj) {
01398

```

```

01399     if (!n) { return; }
01400
01401     if (!a || !b) wblog(FL,
01402         "ERR %s() got null space (%p, %p)", FCT, a, b);
01403
01404     if (check_conj_flag(b, conj)) {
01405         if (conj==1)
01406             { for (size_t i=0; i<n; ++i) a[i]=CONJ(a[i])*b[i]; }
01407         else { for (size_t i=0; i<n; ++i) a[i]*=CONJ(b[i]); }
01408     }
01409     else { for (size_t i=0; i<n; ++i) a[i]*=b[i]; }
01410 };
01411
01412 template <class T> inline
01413 void Wb::timesElRange(T* c, const T* a, const T* b, size_t n, char conj) {
01414
01415     if (!n) { return; }
01416     if (!a || !b || !c) wblog(FL,
01417         "ERR %s() got null space (%p, %p, %p)", FCT, a, b, c);
01418     size_t i=0;
01419
01420     if (check_conj_flag(b, conj)) {
01421         if (conj==1)
01422             { for (; i<n; ++i) { c[i] = CONJ(a[i]) * b[i]; }}
01423         else { for (; i<n; ++i) { c[i] = a[i] * CONJ(b[i]); }}
01424     }
01425     else { for (; i<n; ++i) { c[i] = a[i]*b[i]; }}
01426 };
01427
01428 template <class T> inline
01429 void Wb::timesElRange_add(
01430     T* c, const T* a, const T* b, size_t n, T bfac, char conj
01431 ) {
01432     size_t i=0;
01433
01434     if (!n || !bfac) { return; }
01435     if (!a || !b || !c) wblog(FL,
01436         "ERR %s() got null space (%p, %p, %p)", FCT, a, b, c);
01437
01438     if (check_conj_flag(b, conj)) {
01439         if (conj==1) {
01440             if (bfac==T(+1)) { for (; i<n; ++i) c[i]+=CONJ(a[i])*b[i]; } else
01441             if (bfac==T(-1)) { for (; i<n; ++i) c[i]-=CONJ(a[i])*b[i]; }
01442             else { for (; i<n; ++i) c[i]+=CONJ(a[i])*b[i]*bfac; }
01443         }
01444         else {
01445             if (bfac==T(+1)) { for (; i<n; ++i) c[i]+=a[i]*CONJ(b[i]); } else
01446             if (bfac==T(-1)) { for (; i<n; ++i) c[i]-=a[i]*CONJ(b[i]); }
01447             else { for (; i<n; ++i) c[i]+=a[i]*CONJ(b[i])*bfac; }
01448         }
01449     }
01450     else {if (bfac==T(+1)) { for (; i<n; ++i) c[i]+=a[i]*b[i]; } else
01451         if (bfac==T(-1)) { for (; i<n; ++i) c[i]-=a[i]*b[i]; }
01452         else { for (; i<n; ++i) c[i]+=a[i]*b[i]*bfac; }
01453     }
01454 };
01455
01456 template <class T> inline
01457 void Wb::timesElRange_OM(T* c,
01458     const T* a, const T* b,
01459     size_t N, unsigned M, char conj
01460 ) {
01461     if (!N || !M) { wblog(FL,
01462         "WRN %s() got empty NxM=%dx%d - return", FCT, N, M); return; }
01463     if (!a || !b || !c) wblog(FL,
01464         "ERR %s() got null (%p, %p)", FCT, a, b, c);
01465
01466     unsigned m=0; size_t i=0;
01467     check_conj_flag(b, conj);
01468
01469     for (; m<M; ++m) {
01470         if (m) { i=0; a+=N; b+=N;
01471             if (!conj)
01472                 { for (; i<N; ++i) { c[i] += a[i]*b[i]; }} else
01473             if (conj==1)
01474                 { for (; i<N; ++i) { c[i] += CONJ(a[i]) * b[i]; }}
01475             else { for (; i<N; ++i) { c[i] += a[i] * CONJ(b[i]); }}
01476         }
01477         else {
01478             if (!conj)
01479                 { for (; i<N; ++i) { c[i] = a[i]*b[i]; }} else
01480             if (conj==1)
01481                 { for (; i<N; ++i) { c[i] = CONJ(a[i]) * b[i]; }}
01482             else { for (; i<N; ++i) { c[i] = a[i] * CONJ(b[i]); }}
01483         }
01484     }
01485 };

```

```

01486
01487 template<class T> inline
01488 T Wb::overlap(
01489     const T* a, const T* b, size_t n,
01490     size_t stride,
01491     char tnorm __attribute__((unused))
01492 ){
01493     T x2=0;
01494     if (n) {
01495         Wb::update_range_stride(n,stride);
01496         if (!a || !b) wblog(FL,"ERR %s() got null (%p, %p)",FCT,a,b);
01497         for (size_t i=0; i<n; i+=stride) { x2+=CONJ(a[i])*b[i]; }
01498     }
01499     return x2;
01500 };
01501
01502 template<> inline
01503 wbcomplex Wb::overlap(
01504     const wbcomplex* a, const wbcomplex* b, size_t n,
01505     size_t stride, char tnorm
01506 ){
01507     wbcomplex z2=0;
01508     if (n) { size_t i=0;
01509         Wb::update_range_stride(n,stride);
01510         if (!a || !b) wblog(FL,"ERR %s() got null (%p, %p)",FCT,a,b);
01511         if (tnorm) { for (; i<n; i+=stride) { z2+=a[i]*b[i]; } } else
01512         if (a!=b) { for (; i<n; i+=stride) { z2+=a[i].conj()*b[i]; } }
01513         else { double nrm2=0;
01514             for (; i<n; i+=stride) { nrm2+=a[i].abs2(); }
01515             z2=nrm2;
01516         }
01517     }
01518     return z2;
01519 };
01520
01521 template<class T>
01522 T Wb::gs_project_range(
01523     T *a, const T *b,
01524     size_t n, size_t stride, char isnorm, char tnorm
01525 ){
01526     T x=overlap(b,a,n,stride,tnorm);
01527
01528     if (!isnorm) {
01529         T nrm2=overlap(b,b,n,stride,tnorm);
01530         if (std::fabs(double(nrm2))<1E-12) wblog(FL,
01531             "WRN %s() got norm=%.3g !?",FCT,double(nrm2));
01532         x/=nrm2;
01533     }
01534
01535     Wb::update_range_stride(n,stride);
01536     for (size_t i=0; i<n; i+=stride) { a[i]-=x*b[i]; }
01537
01538     return x;
01539 };
01540
01541 template <class T> inline
01542 size_t Wb::replRange(T* a, size_t n, T x, T v) {
01543
01544     size_t i,m=0;
01545
01546     if (!ISNAN(x))
01547         { for (i=0; i<n; ++i) if (a[i]==x) { a[i]=v; ++m; } }
01548     else { for (i=0; i<n; ++i) if (ISNAN(a[i])) { a[i]=v; ++m; } }
01549
01550     return m;
01551 }
01552
01553 inline size_t Wb::countNaN(double* a, size_t m) {
01554     size_t i,n=0;
01555     for (i=0; i<m; i++) { if (std::isnan(a[i])) ++n; }
01556     return n;
01557 };
01558
01559 template <class T>
01560 void Wb::cpyZRange(
01561     const double *R, const double *I, T *Z, size_t n
01562 ){
01563     wblog(FL,"ERR %s not applicable for type '%s'\n(%p,%p,%p,%d)",
01564         FCT, TSTR(T),R,I,Z,n);
01565 };
01566
01567 template <>
01568 void Wb::cpyZRange(
01569     const double *R, const double *I, wbcomplex *Z, size_t n
01570 ){
01571     for (size_t i=0; i<n; i++)
01572         Z[i].set(R[i], I ? I[i] : 0);

```

```

01573     }
01574
01575     template <class T>
01576     void Wb::splitZRange(
01577         const T *Z, double *R, double *I, size_t n
01578     ){
01579         wblog(FL, "ERR %s not applicable for type `%s'",
01580             FCT, TSTR(T));
01581     }
01582
01583     template <>
01584     void Wb::splitZRange(
01585         const wbcomplex *Z, double *R, double *I, size_t n
01586     ){
01587         for (size_t i=0; i<n; i++) {
01588             R[i]=Z[i].r; if (I) I[i]=Z[i].i; }
01589     }
01590
01591     template<>
01592     void Wb::chopTiny_float(double *d, size_t n, double ref) {
01593
01594         size_t i; if (!n) return;
01595
01596         if (ref<0) {
01597             for (ref=0, i=0; i<n; ++i) ref=MAX(ref, ::fabs(d[i]));
01598             if (ref==0) return;
01599         }
01600
01601         if (ref!=0) {
01602             int k=int(std::log(ref)/std::log(2));
01603             if (k>=0) ref=1<<k; else ref=1/(1<<-k);
01604
01605             for (i=0; i<n; ++i) {
01606                 if (d[i]>0.) d[i]=float(d[i]+ref)-ref; else
01607                 if (d[i]<0.) d[i]=float(d[i]-ref)+ref;
01608             }
01609         }
01610         else {
01611             for (i=0; i<n; ++i)
01612                 if (d[i]!=0.) d[i]=double(float(d[i]));
01613         }
01614     };
01615
01616     template<>
01617     double Wb::chopTiny_imag(wbcomplex *z, size_t n, double eps) {
01618
01619         double i2=0, r2=0; size_t k=0;
01620
01621         for (; k<n; ++k) { r2+=(z[k].r*z[k].r); i2+=(z[k].i*z[k].i); }
01622         if (i2) {
01623             if (r2>1) { eps*=std::sqrt(r2); }
01624             if ((i2=std::sqrt(i2))>eps) { i2=-i2; }
01625             else {
01626                 for (k=0; k<n; ++k) { z[k].i=0; }
01627             }
01628         }
01629         return i2;
01630     };
01631
01632     template<>
01633     void Wb::chopTiny_z(wbcomplex *z, size_t n, double eps) {
01634
01635         size_t k=0; double a2, r2=0, i2=0;
01636
01637         for (; k<n; ++k) {
01638             r2+=(z[k].r*z[k].r);
01639             i2+=(z[k].i*z[k].i);
01640         }
01641         a2=r2+i2; if (a2>1) { eps*=std::sqrt(a2); }
01642
01643         if (std::sqrt(i2)<=eps) { for (k=0; k<n; ++k) { z[k].i=0; }} else
01644         if (std::sqrt(r2)<=eps) { for (k=0; k<n; ++k) { z[k].r=0; }}
01645     };
01646
01647     template<class T>
01648     void Wb::invertIndex(
01649         const wbvector<T> &i1, unsigned N,
01650         wbvector<T> &i2, char lflag
01651     ){
01652         wbvector<char> flag(N);
01653         unsigned i,k,n;
01654
01655         for (i=0; i<i1.len; i++) {
01656             k=i1[i]; if (k<N) flag[k]=1; else
01657             wblog(FL, "ERR index out of bounds (%d/%d)", k, N);
01658         }
01659         n=flag.isum();

```



```

01660
01661     if (n!=i1.len && lflag)
01662         wblog(FL, "WRN index set not unique!? (%d/%d)", n, i1.len);
01663
01664     i2.init(N-n);
01665     for (k=i=0; i<N; i++) if (flag[i]==0) i2[k++]=i;
01666 }
01667
01668 void Wb::setRand(wbvector<wbcomplex> &zz, double fac, double shift) {
01669     size_t i=0; fac/=(double)RAND_MAX;
01670
01671     if (shift==0.)
01672         for (; i<zz.len; ++i)
01673             zz.data[i].set(fac*std::rand(), fac*std::rand());
01674     else
01675         for (; i<zz.len; ++i)
01676             zz.data[i].set(fac*std::rand()+shift, fac*std::rand()+shift);
01677 }
01678
01679 template<class T>
01680 void Wb::set2avg(T *dd, const size_t n) {
01681     size_t i; T dbl=0; if (!n) return;
01682     for (i=0; i<n; ++i) { dbl+=dd[i]; } dbl/= n;
01683     for (i=0; i<n; ++i) { dd[i]=dbl; }
01684 };
01685
01686 template<class T>
01687 void Wb::getDiff(const wbvector<T> &xx, wbvector<T> &dx) {
01688     size_t i, n=xx.len; dx.init(n);
01689     for (i=1; i<n; i++) {
01690         dx[i-1] += (dx[i] = xx[i]-xx[i-1]);
01691         if (i>1) dx[i-1] *= 0.5;
01692     }
01693 }
01694
01695 template<class TX, class TY>
01696 TY Wb::IntTrapez(const TX *xx, const TY *yy, const size_t n) {
01697     TY s=0;
01698
01699     for (size_t i=1; i<n; ++i) {
01700         s += 0.5 * (xx[i] - xx[i-1]) * (yy[i] + yy[i-1]);
01701     }
01702
01703     return s;
01704 };
01705
01706 template<class T>
01707 void markSet(
01708     wbvector< wbvector<T>* > &E0,
01709     wbvector< wbvector<char> > &mark,
01710     unsigned Nkmin,
01711     int &Nkeep,
01712     double Etrunc,
01713     wbvector<T> &E,
01714     double eps,
01715     double b,
01716     int dmax,
01717     const char *dir
01718 ){
01719     unsigned i,d,l,N,n=E0.len;
01720     wbvector<char> mm;
01721     wbperm P,iP;
01722
01723     for (N=i=0; i<n; i++) N+=(E0[i]->len);
01724
01725     E.init(N); mark.initDef(n);
01726     if (!N) return;
01727
01728     for (l=i=0; i<n; i++, l+=d) {
01729         d=(E0[i]->len); mark[i].init(d);
01730         memcpy(E.data+l, E0[i]->data, d*sizeof(T));
01731     }
01732
01733     E.Sort(P); P.invert(iP);
01734
01735     if (eps>0) {
01736         unsigned i0=0;
01737
01738         eps *= ABS(E.last()-E[0]);
01739         for (i=1; i<=N; i++) {
01740             if (i==N || ABS(E[i]-E[i-1])>eps) {
01741                 d=i-i0; if (d>1)
01742                     Wb::setRange2avg(E.data+i0,d);
01743                 i0=i;
01744             }
01745         }
01746     }

```

```

01747     wbvector<T> x; E.permute(x,iP);
01748     for (l=i=0; i<n; ++i, l+=d) {
01749         d=E0[i]->len;
01750         memcpy(E0[i]->data, x.data+l, d*sizeof(T));
01751     }
01752 }
01753
01754 if (Nkeep<0) Nkeep=N;
01755 if (Nkeep==0) return;
01756 if (Nkeep>=(int)N && Etrunc<=0) {
01757     for (i=0; i<n; i++) mark[i].set(1);
01758     Nkeep=N; return;
01759 }
01760
01761 markSet(FL,E,mm,Nkmin,Nkeep,Etrunc,b,dmax,dir);
01762
01763 mm.Permute(iP);
01764
01765 for (l=i=0; i<n; i++, l+=d) {
01766     d=mark[i].len;
01767     memcpy(mark[i].data, mm.data+l, d*sizeof(char));
01768 }
01769
01770 };
01771
01772 template<>
01773 void markSet(
01774     wbvector< wbvector<wbcomplex>* > &E0,
01775     wbvector< wbvector<char> > &mark,
01776     unsigned Nkmin,
01777     int &Nkeep,
01778     double Etrunc,
01779     wbvector<wbcomplex> &E,
01780     double eps,
01781     double b,
01782     int dmax,
01783     const char *dir_
01784 ){
01785     unsigned i,d,l,N, n=E0.len;
01786     const char *s=NULL;
01787
01788     wbvector<char> mm;
01789     wbvector<double> ee;
01790     wbperm P,iP;
01791
01792     if (dir_ && dir_[0]) {
01793         if (!strcmp(dir_,"asc", 3)) { s=dir_+3; } else
01794         if (!strcmp(dir_,"desc",4)) { s=dir_+4; } else
01795             wblog(FL,"ERR %s() invalid direction >%s<",ECT,dir_);
01796     }
01797
01798     for (N=i=0; i<n; i++) N+=(E0[i]->len);
01799     if (!N) return;
01800
01801     E.init(N); ee.init(N); mark.initDef(n);
01802
01803     for (l=i=0; i<n; i++, l+=d) {
01804         d=(E0[i]->len); mark[i].init(d);
01805         memcpy(E.data+l, E0[i]->data, d*sizeof(wbcomplex));
01806     }
01807
01808     if (s==NULL || !strcmp(s,"R")) {
01809         Wb::cpyRangeR(ee.data,E.data,N);
01810     }
01811     else if (!strcmp(s,"I")) {
01812         Wb::cpyRangeI(ee.data,E.data,N);
01813     }
01814     else if (!strcmp(s,"A")) {
01815         wbcomplex z=E.min();
01816         wbvector<wbcomplex> x(E); z.i=0; x-=z;
01817         Wb::cpyRangeA(ee.data,x.data,N);
01818     }
01819     else wblog(FL,"ERR markSet() invalid direction >%s<", dir_);
01820
01821     ee.Sort(P); E.Select(P); P.invert(iP);
01822
01823     if (Nkeep<0) Nkeep=N;
01824     if (Nkeep==0) return;
01825
01826     if (Nkeep>=(int)N && Etrunc<=0) {
01827         for (i=0; i<n; ++i) mark[i].set(1);
01828         Nkeep=N; return;
01829     }
01830
01831     markSet(FL,ee,mm,Nkmin,Nkeep,Etrunc,MAX(b,eps),dmax,dir_);
01832
01833     mm.Permute(iP);

```

```

01834
01835     for (l=i=0; i<n; i++, l+=d) {
01836         d=mark[i].len;
01837         memcpy(mark[i].data, mm.data+l, d*sizeof(char));
01838     }
01839 }
01840
01841 template<class T>
01842 void markSet(const char *F, int L,
01843     const wvector<T> &E,
01844     wvector<char> &mark,
01845     unsigned Nkmin,
01846     int &Nkeep,
01847     double Etrunc,
01848     double b,
01849     int dmax,
01850     const char *dir_)
01851 ){
01852
01853     unsigned i,imax,l;
01854     unsigned N=E.len, nx;
01855     double x, xmax=0;
01856     char dir=+1;
01857
01858     if (dir_ && dir_[0]) {
01859         if (!strcmp(dir_, "asc", 3)) dir=+1; else
01860         if (!strcmp(dir_, "desc", 4)) dir=-1; else
01861             wblog(F,L,"ERR %s() invalid direction >%s<",FCT,dir_);
01862     }
01863
01864     mark.init(N);
01865
01866     if (Nkeep<0 || Nkeep>int(N)) Nkeep=N;
01867     if (Nkeep==0) return;
01868
01869     if (Nkeep>=(int)N && Etrunc<=0) { mark.set(1); Nkeep=N; return; }
01870     if (N<1) wblog(FL,"ERR %s() N=%d !?",FCT,N);
01871
01872     if (int(Nkmin)<0) Nkmin=0;
01873
01874     if (dir>0) {
01875         if (Etrunc>0) {
01876             x=Etrunc+E[0];
01877             for (l=1; l<N; ++l) {
01878                 if (E[l-1]>E[l]) wblog(FL,"ERR %s() data not sorted!",FCT);
01879                 if (E[l]>=x && l>Nkmin) break;
01880             }
01881             if (int(l)>Nkeep) l=Nkeep;
01882         }
01883         else { l=unsigned(Nkeep); }
01884
01885         if (l==N) { mark.set(1); Nkeep=N; return; }
01886         if (l==0) { Nkeep=0; return; }
01887
01888         nx=(dmax>=0 ? (unsigned)dmax : MAX(16U,1/5));
01889
01890         if (b>0) {
01891             for (imax=i=1; i<N && i<l+nx; ++i) {
01892                 x=fabs(E[i]-E[i-1]);
01893                 if (x<=b) { if (xmax<x) { xmax=x; imax=i; } }
01894                 else break;
01895             }
01896             if (i>=l+nx) wblog(F,L,
01897                 "WRN %s() reached Nkeep+dmax=%d+%d states\n"
01898                 "(db=%.3g too small !? => %d @ %.3g)", FCT,l,nx,b,imax,xmax);
01899
01900             Nkeep=l=imax;
01901         }
01902         else if (b<0 && nx && l<N) {
01903
01904             double q=10/(-b*nx);
01905             unsigned NX=5/q, i2=MIN(N,l+NX);
01906             i=l-NX; if (int(i)<1) i=1;
01907
01908             for (imax=1; i<i2; ++i) { x=q*int(i-1);
01909                 x=exp(-x*x)*fabs(E[i]-E[i-1]);
01910                 if (x>xmax) { xmax=x; imax=i; }
01911             }
01912             if (i==N) { x=q*int(i-1);
01913                 x=0.1*exp(-x*x)*fabs(E[N-1]-E[0]);
01914                 if (x>xmax) { xmax=x; imax=i; }
01915             }
01916
01917             Nkeep=l=imax;
01918         }
01919
01920         for (i=0; i<l; ++i) mark[i]=1;

```

```

01921     }
01922     else {
01923         if (Etrunc>0) {
01924             unsigned lx=(Nkmin<N ? N-Nkmin : 0);
01925
01926             x=Etrunc;
01927
01928             for (l=N-1; l<N; --l) {
01929                 if (l && E[l-1]>E[l]) wblog(FL,"ERR %s() data not sorted!",FCT);
01930                 if (E[l]<=x && l<lx) break;
01931             }
01932             if (int(N-1-l)>Nkeep) { l = N-1-(unsigned)Nkeep; }
01933         }
01934         else { l=N-1-(unsigned)Nkeep; }
01935
01936         if (l>N) { mark.set(1); Nkeep=N; return; }
01937         if (l==N-1) { Nkeep=0; return; }
01938
01939         nx=(dmax>=0 ? (unsigned)dmax : MAX(16U,(N-1)/5));
01940
01941         if (b>0) {
01942             for (imax=i-1; i<N && (l-i)<nx; --i) {
01943                 x=fabs(E[i+1]-E[i]);
01944                 if (x<=b) { if (xmax<x) { xmax=x; imax=i; } }
01945                 else break;
01946             }
01947             if ((l-i)>=nx) wblog(F,L,
01948                 "WRN %s() reached Nkeep+dmax=%d+%d states (%d/%d)\n"
01949                 "(db=%.3g too small !? => %d @ %.3g)",FCT,
01950                 Nkeep,nx, Nkeep+nx,N, b,N-imax,xmax);
01951             l=imax;
01952         }
01953         else if (b<0 && nx && l<N) {
01954
01955             double q=10/(-b*nx);
01956             unsigned NX=5/q, il=1-NX; if (int(il)<0) il=0;
01957             i=MIN(N-1,1+NX);
01958
01959             for (imax=1; i>il; --i) { x=q*int(i-1);
01960                 x=exp(-x*x)*fabs(E[i+1]-E[i]);
01961                 if (x>xmax) { xmax=x; imax=i; } }
01962             if (i==0) { x=q*int(i-1);
01963                 x=0.1*exp(-x*x)*fabs(E[N-1]-E[0]);
01964                 if (x>xmax) { xmax=x; imax=i-1; } }
01965             l=imax;
01966         }
01967     }
01968 }
01969
01970 if (l>N) i=0;
01971 else { i=l+1; }; Nkeep=(l<N ? N-1-l : 0);
01972
01973 for (; i<N; ++i) mark[i]=1;
01974 }
01975 };
01976
01977 #endif
01978

```

5.5 gathered.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace additional routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_GATHERED_HH__
00021 #define __WB_GATHERED_HH__
00022

```

```

00023  /* ----- */
00024  /* ----- */
00025
00026 namespace Wb {
00027     const char* basename(const char *data, char c='/');
00028
00029     void print_backtrace(
00030         const char *F=NULL, int L=0, const char *istr=NULL);
00031
00032     int GetNumThreads(const char *F, int L, int &n, const char *name);
00033
00034     template<class T>
00035     int GetEnv (const char *F, int L, const char *name, T& val);
00036
00037     int EnvIsSet(const char *F, int L, const char *name);
00038
00039     wstring repHome(const char *file);
00040
00041     inline int INT(const double a) {
00042         int i=:round(a);
00043         if (std::fabs(i-a)>1E-10) wblog(FL,"ERR %s(%g) !?",FCT,a);
00044         return i;
00045     }
00046
00047     const char* strstri (const char *s1, const char *s2);
00048     const char* strstrw (const char *s1, const char *s2);
00049     const char* strstrwi(const char *s1, const char *s2);
00050     const char* strchrri (const char *s, char c);
00051
00052     const char* rstrstr (const char *s1, const char *s2);
00053
00054     int strrep(
00055         const char *S0,
00056         const char *t0,
00057         const char *t2,
00058         char *Sout,
00059         size_t N,
00060         char gflag='g'
00061     );
00062
00063     void shift(char *s, unsigned n, int p, char erase=0);
00064
00065     void shift(char *s, unsigned n, int p, const char *srep);
00066
00067     bool isword(const char &c) { return (
00068         (c>='0' && c<='9') ||
00069         (c>='a' && c<='z') ||
00070         (c>='A' && c<='Z') || c=='_'
00071     ); };
00072
00073     template <class T>
00074     double mod(double a, T b) {
00075         if (b<=0) wblog(FL,"ERR %s() got b=%g",FCT,b);
00076         a=:fmod(a,double(b)); if (a<0) { a+=b; }
00077         return a;
00078     };
00079
00080     template <class T, ENABLE_IF_isINT(T)>
00081     int mod(int a, T b) {
00082         if (b<=0) wblog(FL,"ERR %s() got b=%d",FCT,b);
00083         a %= int(b); if (a<0) { a+=b; }
00084         return a;
00085     };
00086
00087     unsigned strnlen(const char *s, unsigned n) {
00088         for (unsigned i=0; i<n; ++i) if (!s[i]) return i;
00089         return n;
00090     };
00091
00092     template <class T>
00093     bool isLower(const T* a, const T* b, const size_t n);
00094
00095     template <class T>
00096     bool isEqual(const T* a, const T* b, const size_t n);
00097
00098     template <class T>
00099     bool allEqual(const T* a, const size_t n, const T &x);
00100
00101     template <class T>
00102     bool anyEqual(const T* a, const size_t n, const T &x);
00103
00104     template <class T>
00105     bool anyUnqual(const T* a, const size_t n, const T &x);
00106
00107     template <class T> inline
00108     bool anyUnequal(const T* a, const size_t n, const T &x);
00109

```

```

00110     template <class T>
00111     T getsdscalc(const T* d, size_t n);
00112
00113     template <class T> inline
00114     void scale_eps(T &eps, const T* d, size_t n);
00115
00116 }
00117
00118 template <class T, class T2> inline
00119 char GOT_EPS(const T &a, const T2 &eps) {
00120     if (a==0 || (eps!=0 && ABS(a)<eps)) return 1;
00121     return 0;
00122 }
00123
00124 void setSigHandler(char i);
00125 void wbSigHandler(int sig);
00126
00127 char *gsh_F=NULL; int gsh_L=0;
00128
00129 namespace Wb {
00130 class SigHandler {
00131
00132 public:
00133     SigHandler(const char *F, int L) {
00134         if (F && F[0]) {
00135             const char *s=strrchr(F,'/'); if (!s) { s=F; }
00136             gsh_F = new char[strlen(s)+1]; strcpy(gsh_F,s);
00137             gsh_L=L;
00138         }
00139         else {
00140             const char s0[]=__FILE__, *s=strrchr(s0,'/'); if (!s) { s=s0; }
00141             gsh_F = new char[strlen(s)+1]; strcpy(gsh_F,s);
00142             gsh_L=__LINE__;
00143         }
00144         #ifdef DBSTOP
00145         #else
00146             setSigHandler('i');
00147         #endif
00148     };
00149
00150     ~SigHandler() {
00151         #ifdef DBSTOP
00152         #else
00153             setSigHandler('r');
00154         #endif
00155         gsh_L=0; if (gsh_F) { delete [] gsh_F; gsh_F=NULL; }
00156     };
00157
00158     int check911(char tflag=0) { doflush();
00159         if (tflag<=0) {
00160             #ifndef DBSTOP
00161                 wbSigHandler(911);
00162             #endif
00163             return 0;
00164         }
00165         else { return check911_(); }
00166     };
00167
00168     static int check911_(const char *F=NULL, int L=0) {
00169         int i=SigHandler::icount,
00170             a=SigHandler::account, n=SigHandler::NC0;
00171
00172         if (i>=n) { i=0; } else { i=n-i; if (i>9) { i=9; } }
00173         if (a>=n) { a=0; } else { a=n-a; if (a>9) { a=9; } }
00174         if (F) { wblog(F_L,"ERR %s() ",FCT); }
00175         return (i + 10*a);
00176     };
00177
00178     static int NC0, icount, account;
00179
00180 protected:
00181 private:
00182
00183 };
00184
00185 int SigHandler::NC0 = 3;
00186
00187 int SigHandler::icount = SigHandler::NC0;
00188 int SigHandler::account = SigHandler::NC0;
00189
00190 };
00191
00192 template <class TX>
00193 class wbREF {
00194
00195 public:
00196

```

```

00197     wbREF (const TX &A, char flag) : orig(&A), ref(0) {
00198         if (flag) ref=new TX(A); else ref=&A;
00199     };
00200
00201     TX& Ref() { return *ref; }
00202
00203     ~wbREF() { if (ref!=orig) delete ref; }
00204
00205     protected:
00206     private:
00207         const TX *orig, *ref;
00208 };
00209
00210 namespace Wb {
00211
00212 template <class T>
00213 class num2Fmt {
00214
00215     public:
00216         num2Fmt(const char *F, int L,
00217             int m_=-2, int p_=-2, const char *t_="") : m(m_), p(p_) {
00218
00219             if (t_ && t_[0]) {
00220                 if (strlen(t_)>2) wblog(F_L, "ERR %s() invalid t='%s'", FCT, t_);
00221                 strcpy(t, t_);
00222             }
00223             else { memset(t, 0, 3); }
00224
00225             check_init();
00226         };
00227
00228         int check_init();
00229
00230         explicit operator wbstring() const;
00231
00232     private:
00233         char m, p;
00234         char t[3];
00235
00236         void get_fmt(wbstring &s) const;
00237 };
00238
00239 template<class T> inline
00240 int num2int(const char *F, int L, const T &x);
00241
00242 template<class T> inline
00243 unsigned checkInt(
00244     const char *F, int L, const T* x, size_t n,
00245     double eps=1E-14
00246 );
00247
00248 char* strpad(char *s, char c, unsigned n, unsigned w=1) {
00249     unsigned i, l=strlen(s);
00250     for (i=0; i<w; ++i) s[l++]=' ';
00251     for (i=1; i<n; ++i) s[l++] = c;
00252     return s;
00253 };
00254
00255 unsigned strpad_(char *s, unsigned n, char c=' ', int z=1) {
00256     if (int(n)>0) {
00257         unsigned i=0; for (; i<n; ++i) { s[i]=c; }
00258         if (z) s[i]=0;
00259         return i;
00260     }
00261     return 0;
00262 };
00263
00264 template <class T>
00265 int charGetNumber(const char *F, int L, const char *s, T &x);
00266
00267 bool isBaseType(const std::type_info &type_id);
00268 bool isComplexType(const std::type_info &type_id);
00269
00270 wbstring TimeStamp(char type=0);
00271
00272 void inl(unsigned n) { for (unsigned i=0; i<n; i++) printf("\n"); }
00273
00274 void quietErrLog(const char *F, int L, const char *msg, char qflag) {
00275     if (qflag) snprintf(str, 256, "%s [%s:%d]", msg, Wb::basename(F), L);
00276     else wblog(F, L, "ERR %s", msg);
00277 }
00278
00279 template <class T1, class T2> inline
00280 void safeConvert(const char *F, int L, const T1 &x1, T2 &x2);
00281
00282 template <class T1, class T2> inline
00283 int checkUpdate(T1& a, const T2 &r) {

```

```

00284         if (a!=T1(r)) { a=T1(r); return 1; } else return 0;
00285     };
00286
00287     template <class T>
00288     bool uniformRange(const T* d, const size_t n);
00289
00290     template <class T>
00291     T maxRange(const T* d, const size_t n);
00292
00293     template <class T>
00294     T minRange(const T* d, const size_t n);
00295
00296     template <class T>
00297     T addRange2(const T* d, size_t n, const size_t stride=1);
00298
00299     template <class T>
00300     T addRange(const T* d, size_t n, const size_t stride=1);
00301
00302     template <class T>
00303     void addRange(
00304         const T* a, const T* b, T* c, size_t n,
00305         const size_t stride=1
00306     );
00307
00308     template <class T>
00309     void addRange(const T* a, T* c, size_t n, const size_t stride=1);
00310
00311     template <class T>
00312     void minusRange(
00313         const T* a, const T* b, T* c, const size_t n,
00314         size_t stride=1
00315     );
00316
00317     template <class T>
00318     void diffRange(
00319         const T* a, const T* b, T* c, size_t n,
00320         size_t stride=1
00321     );
00322
00323     template <class T>
00324     void setRange2avg(T* a, size_t m);
00325
00326     template <class T>
00327     size_t nnzRange(const T* a, size_t m);
00328
00329     void update_range_stride(size_t &n, const size_t stride);
00330
00331     template <class T, class T2> inline
00332     void cpyStride(
00333         T2* d,
00334         const T* d0,
00335         unsigned n,
00336         const unsigned *idx, unsigned m,
00337         unsigned D2=-1,
00338         unsigned D0=-1,
00339         char add=0
00340     );
00341
00342     template <class T, class T2>
00343     void cpyStride(
00344         T2* d2,
00345         const T* d0,
00346         size_t n,
00347         size_t m,
00348         size_t D2=-1,
00349         size_t D0=-1,
00350         T2 fac2=0, T fac0=1
00351     );
00352
00353     template <class T, class TB>
00354     void cpyRange(T* a, const TB* b, size_t n, T afac, TB bfac);
00355
00356     template <class T>
00357     void cpyRange(T* a, const T* b, size_t n);
00358
00359     template <class T, class Tb>
00360     void cpyRange(T* a, const Tb* b, size_t n, char check_type=1);
00361
00362     template <class T>
00363     void cpyRangeR(T* a, const wbcomplex* z, size_t n);
00364     template <class T>
00365     void cpyRangeI(T* a, const wbcomplex* z, size_t n);
00366     template <class T>
00367     void cpyRangeA(T* a, const wbcomplex* z, size_t n);
00368
00369     template <class T>
00370     T prodRange(const T* d, const size_t n);

```



```

00371     template <class T>
00372     T prodRange(const T* d, const size_t n, T x0);
00373
00374     template <class T>
00375     T dotProd(const T* a, const T* b, size_t n, char conj=0);
00376     template <class T>
00377     T dotProd(const T* a, const T* b, size_t n, T x, char conj=0);
00378
00379     template <class T> inline
00380     void TimesElRange(T* a, const T* b, size_t n, char conj=0);
00381
00382     template <class T> inline
00383     void timesElRange(T* c, const T* a, const T* b, size_t n, char conj=0);
00384
00385     template <class T> inline
00386     char check_conj_flag(const T* a, char &conj);
00387
00388     template <class T> inline
00389     void timesElRange_OM(
00390         T* c,
00391         const T* a, const T* b,
00392         size_t n, unsigned m,
00393         char conj=0);
00394
00395     template <class T>
00396     void timesElRange_add(
00397         T* c, const T* a, const T* b, size_t n, T bfac=T(1), char conj=0);
00398
00399     template <class TD, class TX> inline
00400     void timesRange(TD* d, TX x, const size_t n, size_t stride=1);
00401
00402     template <class T>
00403     T rangeNorm2(const T* d, size_t n, size_t *k=NULL);
00404
00405     template <class T>
00406     T rangeNormDiff2(const T* d1, const T* d2, size_t n,
00407         const T &fac, size_t *k=NULL
00408     );
00409
00410     template <class T>
00411     T rangeNormDiff2(const T* d1, const T* d2, size_t n);
00412
00413     template <class T>
00414     T rangeMaxDiff(const T* d1, const T* d2, size_t n, size_t *k=NULL);
00415
00416     template<class T>
00417     T overlap(
00418         const T *v1, const T *v2, size_t n,
00419         size_t stride=1, char tnorm=0
00420     );
00421
00422     template<class T>
00423     T gs_project_range(
00424         T *v1, const T *v2,
00425         size_t n, size_t stride=1, char isnorm=0, char tnorm=0
00426     );
00427
00428     template <class T>
00429     size_t replRange(T* a, size_t n, T x, T v);
00430
00431     template <class T> inline
00432     size_t replRange(const char *F, int L, T *a, size_t n, T x, T v) {
00433         size_t m=replRange(a,n,x,v);
00434         if (m) wblog(F,L,"Skipped %d NaN's",m);
00435         return m;
00436     };
00437
00438     size_t countNaN(double* a, size_t m);
00439
00440     template<class T>
00441     void chopTiny_float(T *d, size_t n, T dref=-1) { return; };
00442
00443     template<class T>
00444     void chopTiny_z(T *d, size_t n, double eps=1E-14) { return; };
00445
00446     template<class T>
00447     double chopTiny_imag(T *d, size_t n, double eps=1E-14) { return 0; };
00448
00449     template <class T> inline
00450     int recCompare(
00451         const T* a, const T* b, const size_t n, char lex=1
00452     ){
00453         if (lex>0) {
00454             for (size_t i=0; i<n; ++i) {
00455                 if (a[i]<b[i]) return -1; else
00456                 if (a[i]>b[i]) return +1;
00457             }

```

```

00458     }
00459     else {
00460         for (size_t i=n-1; i<n; --i) {
00461             if (a[i]<b[i]) return -1; else
00462             if (a[i]>b[i]) return +1;
00463         }
00464     }
00465     return 0;
00466 };
00467
00468 template <class T> inline
00469 int recCompare(
00470     const T* a, const T* b, const size_t n, char lex,
00471     T eps, int *nwrn=NULL
00472 ){
00473     if (eps<=T(0)) return recCompare(a,b,n,lex);
00474
00475     int nwrn0=0, *nwrn=(nwrn_ ? nwrn_ : &nwrn0);
00476     T x=0.2*eps;
00477
00478     if (lex>0) {
00479         for (size_t i=0; i<n; ++i) {
00480             if (a[i]<b[i]-eps) return -1; else
00481             if (a[i]>b[i]+eps) return +1; else
00482             if (!nwrn[0] && std::fabs(a[i]-b[i])>x) {
00483                 wblog(FL,"WRN %s() got |a-b|/eps=%.3g (%d/%d)",
00484                     FCT, (a[i]-b[i])/eps,i+1,n); ++nwrn[0];
00485             }
00486         }
00487     }
00488     else {
00489         for (size_t i=n-1; i<n; --i) {
00490             if (a[i]<b[i]-eps) return -1; else
00491             if (a[i]>b[i]+eps) return +1; else
00492             if (!nwrn[0] && std::fabs(a[i]-b[i])>x) {
00493                 wblog(FL,"WRN %s() got |a-b|/eps=%.3g (%d/%d)",
00494                     FCT, (a[i]-b[i])/eps,i+1,n); ++nwrn[0];
00495             }
00496         }
00497     }
00498     return 0;
00499 };
00500
00501 template <class T>
00502 int cmpRange(
00503     const T *a, const T *b, const size_t n, char lex=+1
00504 ){
00505     if (lex>0) {
00506         for (size_t i=0; i<n; ++i) { if (a[i]!=b[i]) {
00507             return (a[i]<b[i] ? -1 : +1);
00508         }}
00509         return 0;
00510     }
00511     else {
00512         if (!lex) wblog(FL,"WRN %s() got lex=%d",FCT,lex);
00513         for (size_t i=n-1; i<n; --i) { if (a[i]!=b[i]) {
00514             return (a[i]<b[i] ? -1 : +1);
00515         }}
00516         return 0;
00517     }
00518 };
00519
00520 template <class T> void cpyZRange(
00521     const double *R, const double *I, T *Z, const size_t n);
00522
00523 template <class T> void splitZRange(
00524     const T *Z, double *R, double *I, const size_t n);
00525
00526 template<class T>
00527 void invertIndex(
00528     const wvector<T> &i1, unsigned N,
00529     wvector<T> &i2, char lflag=1
00530 );
00531
00532 void setRand(wvector<wbcomplex> &zz, double fac=1., double shift=0.);
00533
00534 template<class T>
00535 void set2avg(T *dd, const size_t n);
00536
00537 template<class T>
00538 void getDiff(const wvector<T> &xx, wvector<T> &dx);
00539
00540 template<class TX, class TY>
00541 TY IntTrapez(const TX *xx, const TY *yy, const size_t n);
00542
00543 inline bool contains(const char* s, const char &x, const unsigned &m=16) {
00544     if (s)

```

```

00545         for (unsigned i=0; s[i] && i<m; i++) { if (s[i]==x) return 1; };
00546
00547         return 0;
00548     };
00549
00550 };
00551
00552 template<class T>
00553 void markSet(
00554     wvector< wvector<T>* > &E0,
00555     wvector< wvector<char> > &mark,
00556     unsigned Nkmin,
00557     int &Nkeep,
00558     double Etrunc,
00559     wvector<T> &E,
00560     double eps=0.,
00561     double b=0.,
00562     int dmax=-1,
00563     const char *dir=0
00564 );
00565
00566 template<class T>
00567 void markSet(const char *F, int L,
00568     const wvector<T> &E,
00569     wvector<char> &mark,
00570     unsigned Nkmin,
00571     int &Nkeep,
00572     double Etrunc=0,
00573     double b=0.,
00574     int dmax=-1,
00575     const char *dir=0
00576 );
00577
00578 #endif
00579

```

5.6 matlib.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace MEX routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MATLIB_CC__
00021 #define __WB_MATLIB_CC__
00022
00023 /* ----- //
00024  * NB! intended for mex routines that communicate with matlab desktop
00025  * i.e. mex-files, but not programs e.g. with MAIN defined // Wb,Mar31,20
00026  * NB! mexCallMATLAB(), mexDisp, mexPutVariable, mexGetVariablePtr, ...
00027  * ----- */
00028 #ifdef MATLAB_MEX_FILE
00029 /* ----- //
00030  * Safeguard: matlab 'engine' may only be called from the same thread
00031  * that also called the mex routine! see printf -> mexPrintf() error
00032  * in tstmex_assert_error.cc // Wb,May06,19
00033  * ----- */
00034
00035 int Wb::CallMatlab(
00036     int nargout, mxArray *argout[], int nargin, mxArray *argin[],
00037     const char *fctname
00038 ){
00039     if (Wb::my_caller_tid==omp_get_thread_num()) {
00040         return mexCallMATLAB(nargout,argout,nargin,argin,fctname);
00041     }
00042     else { fprintf(stdout,
00043         "mexCallMATLAB() to '%s' only allowed main thread (%d/%d)",
00044         fctname ? fctname : "(function?)", Wb::my_caller_tid,
00045         omp_get_thread_num());

```

```

00046
00047     #ifdef DBSTOP
00048         dbstop(FL);
00049     #endif
00050
00051     return -1;
00052 }
00053 };
00054
00055 double Wb::CallMatlab(
00056     const char *F, int L, const char* cmd, const char *arg1) {
00057
00058     double q=0; int e=0;
00059     mxArray *argout[1], *argin[1]={NULL};
00060
00061     if (!cmd || !cmd[0]) wblog(F_L, "ERR %s() got empty cmd", FCT);
00062
00063     if (Wb::my_caller_tid!=omp_get_thread_num()) {
00064         char s[16]=""; if (arg1) snprintf(s,16, "(' %s' ", arg1);
00065
00066         fprintf(stdout,
00067             "mexCallMATLAB() to %s%s only allowed main thread (%d/%d)",
00068             cmd, s, Wb::my_caller_tid, omp_get_thread_num()
00069         );
00070         #ifdef DBSTOP
00071             dbstop(FL);
00072         #endif
00073
00074         return (q=-1);
00075     }
00076
00077     if (arg1) {
00078         argin[0]=mxCreateString(arg1);
00079         try { e=mexCallMATLAB(1,argout,1,argin,cmd); }
00080         catch (...) { e=-1; }
00081
00082         mxDestroyArray(argin[0]);
00083     }
00084     else {
00085         try { e=mexCallMATLAB(1,argout,0,argin,cmd); }
00086         catch (...) { e=-2; }
00087     }
00088
00089     if (e) { int l=0, n=64; char s[n]; s[0]=0;
00090         if (e<0)
00091             l=snprintf(s,n,"invalid command » %s",cmd);
00092         else l=snprintf(s,n,"» %s",cmd);
00093
00094         if (l<n && arg1) l+=snprintf(s+l,n-l, "(' %s' ", arg1);
00095         if (l<n) l+=snprintf(s+l,n-l,
00096             e>0? " %% returned error e=%d:" : " %% (e=%d)",e);
00097
00098         if (F)
00099             wblog(F,L, "ERR %s %s", FCT, s);
00100         else wblog(FL, "WRN %s %s", FCT, s);
00101
00102         return q=0;
00103     }
00104
00105     mxGetNumber(argout[0],q);
00106     mxDestroyArray(argout[0]);
00107
00108     if (F) { unsigned l, n=32; char s[n];
00109         l=snprintf(s,n,"%s",cmd);
00110         if (arg1 && l<n) l+=snprintf(s+l,n-l, "(' %s' ", arg1);
00111         wblog(F,L, " * %s » %-24s %% %q", FCT, s, q);
00112     }
00113
00114     return q;
00115 };
00116
00117 void mxPutArray(const char *F, int L,
00118     mxArray *a, const char *vn, const char* ws, const char* dstr
00119 ){
00120     int i;
00121
00122     #pragma omp critical (using_MEX_API)
00123     i=mexPutVariable(ws, vn, a);
00124
00125     if (i) wblog(F_L,
00126         "ERR failed to put '%s' to '%s' (%d)",vn,ws,i);
00127
00128     if (dstr!=NULL) {
00129         if (!strcmp(dstr,"destroy")) mxDestroyArray(a);
00130         else if (dstr[0]!=0)
00131             wblog(F_L, "ERR invalid dstr '%s'", dstr);
00132     }

```

```

00133 }
00134
00135 void mexDisp(const mxArray *a, const char *vn) {
00136     if (vn && vn[0]) printf("\n%s = \n\n", vn);
00137     Wb::CallMatlab(0, NULL, 1, (mxArray**) &a, "display");
00138 };
00139
00140 void mexWRN(const char *s) {
00141
00142     unsigned n=32; char tag[n];
00143     snprintf(tag,n, "Wb:MEX:%.24s",myname);
00144
00145     mexWarnMsgIdAndTxt(tag, s && s[0] ? s : "");
00146 };
00147
00148 wbststring sprint_info(const mxArray *a) {
00149     wbststring s(64);
00150     unsigned l=0;
00151
00152     if (!a) { strcpy(s.data, "(null)"); }
00153     else {
00154         unsigned ndim=mexGetNumberOfDimensions(a);
00155         const size_t *dims=mexGetDimensions(a);
00156
00157         l+=snprintf(s.data,s.len, "%s array of type '%s'",
00158             SSTR(wbvector<size_t>(ndim,dims,'r')), mexGetClassName(a)
00159         );
00160     }
00161
00162     if (l>=s.len) wblog(FL,
00163         "WRN %s() string out of bounds (%d/%d)",FCT,l,s.len);
00164     return s;
00165 };
00166
00167 double wbtoc(
00168     const char *F, int L, const char *istr, char restart
00169 ){
00170     double t=-1.; mxArray *a;
00171
00172     if ((F==NULL) ? 1 : (F[0]==0 ? 1 : 0)) {
00173         Wb::CallMatlab(0, NULL, 0, NULL, "tic");
00174         return t;
00175     }
00176
00177     Wb::CallMatlab(1,&a,0,NULL,"toc");
00178     Mx::Array<double>(a).ncopy_to(&t,1);
00179     mxDestroyArray(a);
00180
00181     if (istr) {
00182         if (!istr[0])
00183             wblog(F_L, "elapsed time: %.6f sec", t);
00184         else wblog(F_L, "elapsed time: %.6f sec (%s)", t, istr);
00185     }
00186
00187     if (restart)
00188         Wb::CallMatlab(0, NULL, 0, NULL, "tic");
00189
00190     return t;
00191 };
00192
00193 template<class T>
00194 int getNumGlobal(const char *vn, T &x, const char* ws){
00195
00196     const mxArray *a=mexGetVariablePtr(ws, vn);
00197     if (!a || !Mx::IsNumber(0,0,a)) { return 1; }
00198     else {
00199         double dbl=0;
00200         Mx::Array<double>(a).ncopy_to(&dbl,1);
00201
00202         x=T(dbl);
00203         if ((double)x!=dbl) wblog(FL,
00204             "WRN ()%s invalid value for type '%s' (%g)",FCT,TSTR(T),dbl);
00205
00206         return 0;
00207     }
00208 };
00209
00210 template<>
00211 int getNumGlobal(const char *vn, wbststring &s, const char *ws) {
00212     const mxArray *a=mexGetVariablePtr(ws, vn);
00213
00214     char istr[64];
00215
00216     if (!a || !mxIsChar(a)) return 1;
00217     if (mxGetString(a,istr,63)) return 1;
00218     s=istr;
00219

```

```

00220     return 0;
00221 }
00222
00223 #else
00224
00225 int Wb::CallMatlab(
00226     int nargsout, mxArray *argout[], int nargin, mxArray *argin[],
00227     const char *fctname
00228 ){
00229     wblog(FL,"ERR %s(%d,%p,%d,%p,'%s')\nnot available outside MatLab",
00230     FCT,nargsout,argout,nargin,argin,fctname?fctname:""); return 0;
00231 };
00232
00233 double Wb::CallMatlab(
00234     const char *F, int L, const char* cmd, const char *arg1) {
00235
00236     char s[16]=""; if (arg1) sprintf(s,16,"('%s')",arg1);
00237     wblog(FL,
00238     "ERR %s -> %s%s\nnot available outside MatLab",
00239     FCT, cmd? cmd:"(cmd!?)",s);
00240     return 0;
00241 };
00242
00243 double wbtoc(
00244     const char *F, int L, const char *istr,
00245     char restart __attribute__ ((unused))
00246 ){
00247     wblog(FL,"WRN %s(%s,%s) not available outside MatLab",
00248     FCT,shortFL(F_L),istr ? istr:""); return 0;
00249 }
00250
00251 void mxPutArray(const char *F, int L,
00252     mxArray *a, const char *vn, const char* ws,
00253     const char* dstr
00254 ){
00255     wblog(F_L,"ERR %s(%s,%s,%lX,%s) not available outside MatLab",
00256     FCT,vn?vn:"",ws?ws:"",&a,dstr?dstr:"");
00257 };
00258
00259 template<class T>
00260 int getNumGlobal(
00261     const char *vn __attribute__ ((unused)),
00262     T &x __attribute__ ((unused)),
00263     const char* ws __attribute__ ((unused))
00264 ){
00265     wblog(FL,"ERR %s(%s,%s) not available outside MatLab",
00266     FCT,vn?vn:"",ws?ws:""); return 0;
00267 };
00268
00269 void mexDisp(const mxArray *a, const char *vn) {
00270     wblog(FL,"ERR %s(%s,%lX) not available outside MatLab",
00271     FCT,vn ? vn:"",a);
00272 };
00273
00274 #endif
00275 #endif

```

5.7 memlib.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace memory routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEM_LIB_CC__
00021 #define __WB_MEM_LIB_CC__
00022
00023 // ----- //
00024 // ----- //

```

```

00025
00026 template <class T>
00027 Wb::sptr<T>::toStr() const {
00028
00029     Wbstring s(128);
00030     unsigned l=0;
00031
00032     if (mtype!=Wb::MEM_DEF) {
00033         l+=snprintf(s.data+l,s.len-1,"%s ",Wb::MTYPE_STR[mtype]); }
00034
00035     if (data || len)
00036         { l+=snprintf(s.data+l,s.len-1,"%p %s[%.3g]",data,sTSTR(T),double(len)); }
00037     else { l+=snprintf(s.data+l,s.len-1,"%p",data); }
00038
00039     if (nref) {
00040         l+=snprintf(s.data+l,s.len-1," w/ %d refs",nref); }
00041
00042     return s;
00043 };
00044
00045 template <class T> inline
00046 T* Wb::sptr<T>::malloc_base(size_t n, char init, Wb::MTYPE mt) {
00047
00048     char do_alloc=(n>0);
00049     if (data) {
00050         if (check_consistency(0)) wblog(FL,"ERR Wb::sptr() %s",STR_(this));
00051         if (n!=len || nref) rm_dref();
00052         else {
00053             do_alloc=0;
00054         }
00055     }
00056
00057     if (do_alloc) {
00058         if (data || len) wblog(FL,"ERR %s() %s !?",FCT,STR_(this));
00059         len=n; mtype=mt;
00060
00061         if (mtype==Wb::MEM_DEF) { WB_NEW(data,n); } else
00062         if (mtype==Wb::MEX_RETURN) {
00063             if (typeid(T)==typeid(double)) {
00064                 mxArray *a=Mx::Array<double>(1,&n).Return();
00065                 data=(T*)mxGetDoubles(a);
00066                 P2X.add(data,a);
00067
00068                 nref=0;
00069             }
00070             else if (typeid(T)==typeid(Wbcomplex)) {
00071                 mxArray *a=Mx::Array<Wbcomplex>(1,&n).Return();
00072                 data=(T*)mxGetComplexDoubles(a);
00073                 P2X.add(data,a);
00074             }
00075             else wblog(FL,"ERR %s() got %s",FCT,TSTR(T));
00076
00077             if (WBLOG_MMEX) wblog(FL,"new %s",STR_(this));
00078         }
00079     }
00080     if (n && init) { T z=T();
00081         for (size_t i=0; i<n; ++i) data[i]=z;
00082     }
00083
00084     if ((!len ^ !data) || (nref && !len)) wblog(FL,
00085         "ERR Wb::sptr<%s>=%p -> %s",sTSTR(T),this,STR_(this));
00086
00087     return data;
00088 };
00089
00090 template <class T> inline
00091 T* Wb::sptr<T>::malloc_data(size_t n, const T *d0, Wb::MTYPE mt) {
00092
00093     malloc_base(n,0,mt);
00094     if (n) {
00095         if (d0)
00096             MEM_COPY<T>(data,n,d0);
00097         else MEM_SET<T>(data,n);
00098     }
00099
00100     return data;
00101 };
00102
00103 template <class T> inline
00104 T* Wb::sptr<T>::init2ref(size_t n, const T* d0) {
00105
00106     if (data==d0 && len==n && nref) { return data; }
00107     if (data || len || nref) wblog(FL,"ERR %s() got %s",FCT,STR_(this));
00108     if (!d0 ^ !n) wblog(FL,"ERR %s() got n=%ld with %p",FCT,n,d0);
00109
00110     #pragma omp critical (manage_SPTR)
00111     { data=(T*)d0;

```

```

00112         len=n; mtype=Wb::MEM_REF;
00113         nref=1;
00114     }
00115     return data;
00116 };
00117
00118 template <class T> inline
00119 unsigned Wb::sptr<T>::add_dref() {
00120
00121     if (!data) { check_consistency(); return 0; }
00122
00123     #pragma omp critical (manage_SPTR)
00124     { ++nref; }
00125
00126     if (nref>128) wblog(FL,"WRN %s() %p @ nref=%d",FCT,data,nref);
00127
00128     return nref;
00129 };
00130
00131 template <class T> inline
00132 int Wb::sptr<T>::rm_dref(char mflag) {
00133
00134     int rval=0, e=0;
00135
00136     #pragma omp critical (manage_SPTR)
00137     { try {
00138
00139         if (!len ^ !data) { e|=1; }
00140         if (nref && !len) { e|=2; }
00141
00142         if (!e && data) {
00143             if (mtype==Wb::MEM_DEF) {
00144                 if (nref) { --nref; }
00145                 else {
00146                     WB_DELETE(data);
00147                     rval=1;
00148                 }
00149             }
00150             else if (mtype==Wb::MEM_REF) {
00151                 if (nref==1) { rval=2; }
00152                 else if (nref>1) { --nref; }
00153                 else { e|=16; }
00154             }
00155             else if (mtype==Wb::MEX_RETURN) {
00156                 if (!nref) { rval=3; }
00157                 else { e|=32; }
00158
00159                 if (mflag) {
00160                     mxArray *a=P2X.Return(0,0,data);
00161                     if (a) { mxDestroyArray(a); data=NULL; }
00162                 }
00163                 else if (P2X.erase(data)) { e|=64; }
00164             }
00165             else e|=128;
00166         }
00167     } catch (...) { e|=256; }}
00168
00169     if (e) {
00170         if (e==64)
00171             { wblog(FL,"WRN %s() got %s",FCT,STR_(this)); }
00172         else { wblog(FL,"ERR %s() got e=%d\n%s",FCT,e,STR_(this)); }
00173     }
00174
00175     if (rval) { init_def(); }
00176
00177     if (!data) { mtype=Wb::MEM_DEF; }
00178
00179     return rval;
00180 };
00181
00182 template <class T> inline
00183 T* Wb::sptr<T>::save_dref(const char *F, int L) {
00184
00185     T* dref=data; int e=0;
00186
00187     #pragma omp critical (manage_SPTR)
00188     { if (!len ^ !data) { e|=1; }
00189       if (nref || mtype) { e|=2; }
00190       else if (len) { len=0; data=NULL; }
00191     }
00192
00193     if (e) wblog(FL,"ERR %s() %s got e=%d !?",FCT,STR_(this),e);
00194     return dref;
00195 };
00196
00197 template <class T1, class T2>
00198 bool gotMemOverlap(

```



```

00199     const T1* src, size_t len0,
00200     const T2* dest, size_t len2
00201 ){
00202     if (len0 && len2) {
00203         const char
00204             *L=(char*)src, *R=(char*)(src+len0)-1,
00205             *a=(char*)dest, *b=(char*)(dest+(long(len2)<0 ? len0 : len2))-1;
00206
00207         if (!src || !dest) wblog(FL,
00208             "ERR %s() received null pointer (%p, %p)",FCT,src,dest);
00209         return ((a<=L && b>=L) || (a<=R && b>=R) || (a>=L && b<=R) );
00210     }
00211     return 0;
00212 };
00213
00214 #endif
00215

```

5.8 memlib.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace memory routines and smart pointers
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEM_LIB_HH__
00021 #define __WB_MEM_LIB_HH__
00022
00023 // ----- //
00024 // ----- //
00025
00026 template <class T1, class T2>
00027 bool gotMemOverlap(
00028     const T1* src, size_t len0,
00029     const T2* dest, size_t len2=-1
00030 );
00031
00032 template <class T>
00033 inline void MEM_COPY_BASE(T* data, size_t n, T const* const d=NULL) {
00034
00035     #ifdef MLB_VFLAG
00036         wblog(FL,"TST base init");
00037     #endif
00038     if (n) {
00039         if (!d)
00040             { memset((void*)data,0,n*sizeof(T)); }
00041         else { memcpy((void*)data,d,n*sizeof(T)); }
00042     }
00043 };
00044
00045 template <class T>
00046 inline void MEM_COPY_BASE(
00047     T* data, size_t n, size_t n1, T const* const d1, T const* const d2
00048 ){
00049
00050     #ifdef MLB_VFLAG
00051         wblog(FL,"TST base init %d->%d",n1,n);
00052     #endif
00053     if (n1) {
00054         if (d1==NULL) wblog(FL,"ERR got (null) pointer"); else
00055         if (n1>n) { wblog(FL,"WRN length out of bounds (%d/%d)",n1,n); n1=n; }
00056         memcpy(data,d1,n1*sizeof(T));
00057     }
00058     if (n1<n) {
00059         if (d2) memcpy(          data+n1, d2, (n-n1)*sizeof(T));
00060         else  memset((void*)(data+n1), 0, (n-n1)*sizeof(T));
00061     }
00062 };
00063

```

```

00064 namespace Wb {
00065
00066 enum MTYPE {
00067     MEM_DEF,
00068     MEM_REF,
00069     MEX_RETURN,
00070     MEM_NUM_TYPES
00071 };
00072
00073 const char* MTYPE_STR[MEM_NUM_TYPES] {
00074     "MEM:default",
00075     "MEM:ref",
00076     "MEX:return"
00077 };
00078
00079 template <class T>
00080 class sptr {
00081
00082 public:
00083
00084     sptr() : mtype(MEM_DEF), len(0), nref(0), data(NULL) {};
00085
00086     ~sptr() {
00087         #pragma omp critical (manage_SPTR)
00088         { if (mtype || len || data || nref) wblog(FL,
00089             "WRN %s() got %s",FCT,STR_(this));
00090             init_def();
00091         }
00092     }
00093
00094     void init_def() {
00095         mtype=MEM_DEF; len=nref=0; data=NULL;
00096     }
00097
00098     T* malloc_base(size_t n, char init=1, MTYPE mt=MEM_DEF);
00099     T* malloc_data(size_t n, const T *d0=NULL, MTYPE mt=MEM_DEF);
00100
00101     int rm_dref(char mflag=0);
00102
00103     unsigned add_dref();
00104
00105     T* save_dref(const char *F=0, int L=0);
00106
00107     T* init2ref(size_t n, const T* d0);
00108
00109     int isRef() const {
00110         return (nref || mtype==Wb::MEM_REF);
00111     };
00112
00113     explicit operator bool() const {
00114         check_consistency(); return (len!=0); };
00115     bool operator! () const {
00116         check_consistency(); return (len==0); }
00117
00118     int check_consistency(char force=1) const {
00119         int e=0;
00120
00121         #pragma omp critical (manage_SPTR)
00122         { if (!len ^ !data) { e|=1; }
00123             if (nref && !len) { e|=2; }
00124         }
00125
00126         if (e && force) wblog(FL,
00127             "ERR Wb::sptr() inconsistency (%p @ n=%d, r=%d)",data,len,nref);
00128         return e;
00129     };
00130
00131     wstring toStr() const;
00132
00133     MTYPE mtype;
00134
00135     size_t len;
00136     unsigned nref;
00137
00138     T* data;
00139
00140 protected:
00141 private:
00142 };
00143
00144 };
00145
00146 template <class T, ENABLE_IF_isPOD(T) >
00147 void MEM_COPY(T* data, size_t n, const T* d=NULL) {
00148     MEM_COPY_BASE(data,n,d);
00149 };
00150

```

```

00151 template <class T, ENABLE_IF_noPOD(T) >
00152 void MEM_COPY(T* data, size_t n, const T* d=NULL) {
00153
00154     #ifdef MLB_VFLAG
00155         wblog(FL,"TST class init");
00156     #endif
00157
00158     if (d) { for (size_t i=0; i<n; ++i) data[i]=d[i]; }
00159     else {
00160         const T z=T();
00161         for (size_t i=0; i<n; ++i) data[i]=z;
00162     }
00163 };
00164
00165 template <class T, ENABLE_IF_isPOD(T) >
00166 void MEM_COPY(T* data, size_t n, size_t n1, const T* d1, const T* d2) {
00167     MEM_COPY_BASE(data,n,n1,d1,d2);
00168 };
00169
00170 template <class T, ENABLE_IF_noPOD(T) >
00171 void MEM_COPY(T* data, size_t n, size_t n1, const T* d1, const T* d2) {
00172     #ifdef MLB_VFLAG
00173         wblog(FL,"TST class init %d->%d",n1,n);
00174     #endif
00175
00176     size_t i=0;
00177     if (n1) {
00178         if (d1==NULL) wblog(FL,"ERR got (null) pointer"); else
00179         if (n1>n) { wblog(FL,
00180             "WRN length out of bounds (%d->%d)",n1,n); n1=n; }
00181         for (; i<n1; ++i) data[i]=d1[i];
00182     }
00183     if (d2) { for (; i<n; ++i) { data[i]=d2[i-n1]; }}
00184     else {
00185         const T z=T();
00186         for (; i<n; ++i) data[i]=z;
00187     }
00188 };
00189
00190 template <class T>
00191 void MEM_COPY(T** data, size_t n, T* const* const d=NULL) {
00192     #ifdef MLB_VFLAG
00193         wblog(FL,"TST got pointer space");
00194     #endif
00195     MEM_COPY_BASE(data,n,d);
00196 };
00197
00198 template <class T>
00199 void MEM_COPY(T** data, size_t n, size_t n1,
00200     T* const* const d1, T* const* const d2
00201 ){
00202     #ifdef MLB_VFLAG
00203         wblog(FL,"TST got pointer space");
00204     #endif
00205     MEM_COPY_BASE(data,n,n1,d1,d2);
00206 };
00207
00208 template <class T> inline
00209 void MEM_SET_BASE(T* data, size_t len, const T *a=NULL) {
00210     #ifdef MLB_VFLAG
00211         wblog(FL,"TST base memset");
00212     #endif
00213     if (!a)
00214         { memset((void*)data,0,len*sizeof(T)); }
00215     else { for (size_t i=0; i<len; ++i) data[i]=(*a); }
00216 };
00217
00218 template <class T> inline
00219 void MEM_SET_BASE(T** data, size_t len, T* const *a=NULL) {
00220     #ifdef MLB_VFLAG
00221         wblog(FL,"TST base memset (pointer space)");
00222     #endif
00223     if (!a)
00224         { memset((void*)data,0,len*sizeof(T*)); }
00225     else { for (size_t i=0; i<len; ++i) data[i]=(*a); }
00226 };
00227
00228 template <class T>
00229 class MEM_SET {
00230
00231 public:
00232
00233     MEM_SET(
00234         T* data      __attribute__((unused)),
00235         size_t len    __attribute__((unused)),
00236         const T *a    __attribute__((unused)) = NULL
00237     ){

```

```

00238     };
00239
00240     private:
00241
00242 };
00243
00244 template <class T>
00245 class MEM_SET<T*> {
00246
00247     public:
00248
00249         MEM_SET(T** data, size_t len, T* const *a=NULL) {
00250 #ifdef MLB_VFLAG
00251             wblog(FL, "TST pointer memset");
00252 #endif
00253             MEM_SET_BASE(data, len, a);
00254         }
00255
00256     private:
00257
00258 };
00259
00260 template <> inline MEM_SET<int>::MEM_SET(
00261     int* data, size_t len, const int *a
00262 ){ MEM_SET_BASE(data, len, a); }
00263
00264 template <> inline MEM_SET<unsigned>::MEM_SET(
00265     unsigned* data, size_t len, const unsigned *a
00266 ){ MEM_SET_BASE(data, len, a); }
00267
00268 template <> inline MEM_SET<unsigned long>::MEM_SET(
00269     unsigned long* data, size_t len, const unsigned long *a
00270 ){ MEM_SET_BASE(data, len, a); }
00271
00272 template <> inline MEM_SET<long>::MEM_SET(
00273     long* data, size_t len, const long *a
00274 ){ MEM_SET_BASE(data, len, a); }
00275
00276 template <> inline MEM_SET<char>::MEM_SET(
00277     char* data, size_t len, const char *a
00278 ){ MEM_SET_BASE(data, len, a); }
00279
00280 template <> inline MEM_SET<double>::MEM_SET(
00281     double* data, size_t len, const double *a
00282 ){ MEM_SET_BASE(data, len, a); }
00283
00284 template <> inline MEM_SET<float>::MEM_SET(
00285     float* data, size_t len, const float *a
00286 ){ MEM_SET_BASE(data, len, a); }
00287
00288 template <> inline MEM_SET<wbcomplex>::MEM_SET(
00289     wbcomplex* data, size_t len, const wbcomplex *a
00290 ){ MEM_SET_BASE(data, len, a); }
00291
00292 #endif
00293

```

5.9 memtrack.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace memory routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEM_TRACK_HH__
00021 #define __WB_MEM_TRACK_HH__
00022
00023 // Wb, Jul27, 12
00024

```

```

00025 namespace Wb {
00026     void MemStat(const char *F=0, int L=0, char lflag=0);
00027 };
00028
00029 #if !(defined __WBDEBUG__ || defined __WB_MEM_CHECK__ )
00030
00031 #define WB_NEW(p,n) Wb::NEW(p,n)
00032 #define WB_NEW_1(p) Wb::NEW_1(p);
00033
00034 #define WB_NEW_2(p,x) { p = new x; \
00035     if (!p) wblog(__FILE__,__LINE__,"ERR failed to allocate instance"); \
00036 }
00037
00038 #define WB_DELETE(p) if (p) { delete [] p; p=NULL; }
00039 #define WB_DELETE_1(p) if (p) { delete p; p=NULL; }
00040
00041 namespace Wb {
00042
00043     template<class T>
00044     inline void NEW(T* &p, const size_t &n) {
00045
00046         if (n) {
00047             { try { p = new T[n]; } catch (...) { p=NULL; } }
00048
00049             if (!p) { printf("\n");
00050                 MemStat(FL);
00051                 wblog(FL,"ERR out of memory (%ld * %d = %.1fG) !?",
00052                     n, sizeof(T), n*sizeof(T)/double(1<<30)
00053                 );
00054             }
00055         }
00056         else { p=NULL; }
00057     };
00058
00059     template<class T>
00060     inline void NEW_1(T* &p) {
00061         try { p = new T; } catch (...) { p=NULL; }
00062
00063         if (!p) { printf("\n");
00064             MemStat(FL);
00065             wblog(FL,"ERR failed to allocate instance !?");
00066         }
00067     };
00068
00069 };
00070
00071 #else
00072
00073 #define WB_NEW(p,n) Wb::NEW(__FILE__,__LINE__,p,n);
00074 #define WB_NEW_1(p) Wb::NEW_1(__FILE__,__LINE__,p);
00075
00076 #define WB_NEW_2(p,x) { p = new x; \
00077     if (!p) wblog(__FILE__,__LINE__,"ERR failed to allocate instance"); \
00078 }
00079
00080 #define WB_DELETE(p) \
00081     if (p) { Wb::gML.rm_ptr(__FILE__,__LINE__,p); delete [] p; p=NULL; }
00082
00083 #define WB_DELETE_1(p) \
00084     if (p) { Wb::gML.rm_ptr(__FILE__,__LINE__,p); delete p; p=NULL; }
00085
00086 #include "memtrack.cc"
00087
00088 #endif
00089
00090 #endif
00091

```

5.10 mexlib.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace MEX routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software

```

```

00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEXLIB_CC__
00021 #define __WB_MEXLIB_CC__
00022
00023 // also using mxPutAndDestroy() outside MEX file
00024 // by writing data to file to capture debug data // Wb,Oct25,16
00025
00026 void mxPutAndDestroy(
00027     const char *F, int L, mxArray *a, const char *vn,
00028     const char* ws
00029 ){
00030     int i=0;
00031     if (!ws || !ws[0]) wblog(F, "ERR %s() invalid workspace '%s'", FCT, ws);
00032     if (!strcmp(ws, "caller") || !strcmp(ws, "base")) { i=1; }
00033
00034     #ifndef MATLAB_MEX_FILE
00035         if (i>0) { i=-1; }
00036     #endif
00037     if (i>0 && Wb::my_caller_tid!=omp_get_thread_num()) { i=-2; }
00038     }
00039     if (i<0) {
00040         char tmp[64];
00041         tmpmatFL(F_L, vn && vn[0] && !strcmp(vn, "ans")? vn:ws, tmp, 64);
00042         wblog(F_L, "WRN thread %d/%d cannot put '%s' to %s (i=%d)",
00043             omp_get_thread_num(), omp_get_num_threads(), vn?vn:"", ws, i);
00044
00045         if (!strcmp(tmp, "caller") || !strcmp(tmp, "base")) wblog(F,
00046             "ERR %s() invalid tmp='%s' !?", FCT, tmp);
00047         mxPutAndDestroy(F, L, a, vn, tmp);
00048         return;
00049     }
00050
00051     #ifdef QS_USING_OMP
00052         Wb::ompGuard myLK(mxapi_lock);
00053     #endif
00054
00055     if (i) {
00056         i=mexPutVariable(ws, vn, a);
00057         if (i) wblog(F_L,
00058             "ERR mexPutVariable('%s','%s',..) returned e=%d !?", ws, vn, i);
00059         if (F) wblog(F_L, "I/O putting '%s' to %s", vn, ws);
00060     }
00061     else {
00062         wbsting tmp;
00063         if (!ws || !ws[0] || !strcmp(ws, "tmpfile")) {
00064             tmp.init(64); tmpmatFL(F_L,
00065                 vn && vn[0] && strcmp(vn, "ans")? vn:"", tmp.data, tmp.len);
00066             ws=tmp.data; i=1;
00067         }
00068         else {
00069             unsigned j=0, k=0;
00070             for (; ws[j]; ++j) { if (ws[j]!='.') { k=j+1; } }
00071             if (!k || strcasecmp(ws+k, "mat")) {
00072                 tmp.init(k+6); sprintf(tmp.data, "%s.mat", ws);
00073                 ws=tmp.data;
00074             }
00075         }
00076         if (F) wblog(F_L, "%s saving '%s' to '%s', i?"TMP":"I/O", vn, ws);
00077
00078         char x[2]; strcpy(x, Wb::isFile(ws)? "u":"w");
00079         Wb::matFile f(F_L, ws, x);
00080
00081         matPutVariable(f.mfp, vn, a);
00082     }
00083
00084     mxDestroyArray(a);
00085 };
00086
00087 int Mx::IsNumArray(
00088     const char *F, int L, const mxArray* a,
00089     unsigned r,
00090     char cflag,
00091     char type
00092 ){
00093     int rval=0;
00094
00095     if (!a) {
00096         if (L) {
00097             strcpy(str, "got a=null");
00098             if (F) wblog(F, L, "ERR %s() %s", FCT, str);
00099         }
00100     }

```

```

00101     return (rval=-128);
00102 }
00103
00104 if (mxIsDouble(a) ) { rval|= 2; } else
00105 if (mxIsNumeric(a)) { rval|= 8; } else
00106 if (mxIsChar(a) ) { rval|=16; } else
00107 if (mxIsLogical(a)) { rval|=32; }
00108
00109 if (rval && mxIsComplex(a)) { rval|=4; }
00110
00111 if (type=='d') {
00112     if (!(rval & 2)) {
00113         if (L) {
00114             sprintf(str,"got data of type %s",mxGetClassName(a));
00115             if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00116         }
00117         return (rval=-129);
00118     }
00119 }
00120 else if (type=='*') {
00121     if (!(rval & 58)) {
00122         if (L) {
00123             sprintf(str,"got data of type %s",mxGetClassName(a));
00124             if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00125         }
00126         return (rval=-130);
00127     }
00128 }
00129 else wblog(F,L,"ERR %s() invalid type=%d<%c>",FCT,type,type);
00130
00131 if (mxIsSparse(a)) {
00132     if (L) {
00133         strcpy(str,"got sparse input");
00134         if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00135     }
00136     return (rval=-131);
00137 }
00138
00139 if (!cflag && (rval & 4)) {
00140     if (L) {
00141         strcpy(str,"got complex input");
00142         if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00143     }
00144     return (rval=-132);
00145 }
00146
00147 if (int(r)>0) {
00148     unsigned ra=mxGetNumberOfDimensions(a);
00149     if (ra>r) {
00150         if (ra==2) {
00151             const size_t *D=mxGetDimensions(a);
00152             if (D[0]==1 || D[1]==1) { ra=1; }
00153         }
00154         if (ra>r) {
00155             if (L) {
00156                 sprintf(str,"got invalid rank (r=%d/%d)",ra,r);
00157                 if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00158             }
00159             return (rval=-ra);
00160         }
00161     }
00162 }
00163 rval|=1;
00164
00165 return rval;
00166 };
00167
00168 template <class T>
00169 Mx::Array<T>::Array(const wvector<size_t> &S, char t_, char c_)
00170 : ax(0), data(0), len(0), trans(t_), cmplx(c_), mxref(0) {
00171
00172     if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00173     init(S.len,S.data);
00174 };
00175
00176 template <class T>
00177 int Mx::Array<T>::init_dptra() {
00178
00179     data=0; if (!ax) { return -1; }
00180
00181     if (mxref<=0) wblog(FL,"ERR %s() got ref=%d !?",FCT,mxref);
00182     if (mxIsSparse(ax)) wblog(FL,
00183         "ERR Mx::Array got invalid sparse `%s'",mxGetClassName(ax));
00184
00185     mxClassID t=mxGetClassID(ax); int e=0;
00186
00187     if (!mxIsComplex(ax)) {

```

```

00188         switch (t) {
00189             case mxDOUBLE_CLASS : safe_set_dptra( mxGetDoubles (ax)); break;
00190             case mxCHAR_CLASS   : safe_set_dptra( mxGetChars   (ax)); break;
00191             case mxINT8_CLASS    : safe_set_dptra( mxGetInt8s   (ax)); break;
00192             case mxUINT8_CLASS   : safe_set_dptra( mxGetUint8s  (ax)); break;
00193             case mxINT16_CLASS   : safe_set_dptra( mxGetInt16s  (ax)); break;
00194             case mxUINT16_CLASS  : safe_set_dptra( mxGetUint16s (ax)); break;
00195             case mxINT32_CLASS   : safe_set_dptra( mxGetInt32s  (ax)); break;
00196             case mxUINT32_CLASS  : safe_set_dptra( mxGetUint32s (ax)); break;
00197             case mxINT64_CLASS   : safe_set_dptra( mxGetInt64s  (ax)); break;
00198             case mxUINT64_CLASS  : safe_set_dptra( mxGetUint64s (ax)); break;
00199             case mxLOGICAL_CLASS : safe_set_dptra( mxGetLogicals(ax)); break;
00200             default: e=1; break;
00201         }
00202     }
00203     else {
00204         if (t==mxDOUBLE_CLASS) safe_set_dptra( mxGetComplexDoubles(ax) );
00205         else { e=2; }
00206     }
00207
00208     if (e && !mxIsNumChar(ax))
00209         wblog(FL,"ERR %s got invalid %s '%s' (e=%d)",xStr(),
00210             mxIsSparse(ax) ? "sparse":"type", mxGetClassName(ax),e);
00211     if (e==2 && !data)
00212         wblog(FL,"ERR %s got complex '%s'",xStr(),mxGetClassName(ax));
00213     if (e) wblog(FL,"WRN %s not match found for %s",xStr(),TSTR(T));
00214
00215     if (!e) { mxref+=8; }
00216
00217     return (e<0 ? e : (data ? 0 : 1));
00218 };
00219
00220 template<class T>
00221 Mx::Array<T>& Mx::Array<T>::init(
00222     unsigned ndim, size_t *dims, char t_, char c_) {
00223
00224     if (ax) { mxDestroyArray(ax); ax=NULL; data=NULL; }; mxref=0;
00225
00226     if (t_>=0) { trans=t_; }
00227     if (c_>=0) { cmplx=c_; }
00228     if ((rank=ndim)) {
00229         len=dims[0]; for (unsigned i=1; i<ndim; ++i) { len*=dims[i]; }
00230     } else len=0;
00231
00232     MX_INIT_DATA(ndim,dims);
00233     init_xstr();
00234
00235     if (ax && (cmplx ^ mxIsComplex(ax))) {
00236         if (cmplx) { cmplx=0; wblog(FL,
00237             "WRN %s ignoring cmplx=%d for %s",xStr(),mxGetClassName(ax)); }
00238         else wblog(FL,
00239             "ERR %s complex mismatch (%d/%d)",xStr(),cmplx,mxIsComplex(ax));
00240     }
00241
00242     return *this;
00243 };
00244
00245 template<class T>
00246 Mx::Array<T>& Mx::Array<T>::init(const mxArray *a, char t_, char c_) {
00247
00248     int e=0;
00249     if (ax) { if (mxref<=0) mxDestroyArray(ax); data=NULL; }
00250     mxref=1; len=rank=0;
00251
00252     if (t_>=0) { trans=t_; }
00253     if (c_>=0) { cmplx=c_; }
00254
00255     ax=(mxArray*)a; if (!a) { return *this; }
00256
00257     e=init_dptra();
00258     if (e<0) wblog(FL,
00259         "ERR %s() %s <> %s => e=%d !?",FCT,mxGetClassName(ax),TSTR(T),e);
00260
00261     len=mxGetNumberOfElements(a);
00262     rank=mxGetNumberOfDimensions(a); init_xstr();
00263
00264     if (ax) { bool x=mxIsComplex(ax);
00265         if (cmplx<0) { cmplx=x; } else
00266         if (cmplx ^ x) { const char *s=xStr();
00267             if (cmplx) {
00268                 if (WLOG_TCAST) wblog(FL,
00269                     "WRN %s unsetting cmplx->0 for %s",s,
00270                     mxIsComplex(ax)? "complex ":"", mxGetClassName(ax));
00271                 cmplx=0;
00272             }
00273             else wblog(FL,"ERR %s complex type mismatch (%d/%d)",s,cmplx,x);
00274         }
00275     }

```



```

00275     }
00276
00277     return *this;
00278 };
00279
00280 template <class Ta, class Tb>
00281 size_t copy_row2col_major(
00282     size_t r, const size_t *S, Ta *a, const Tb *b,
00283     char from
00284 ){
00285     if (!r || !S) {
00286         wblog(FL, "WRN %s() r=%d / sz=%p !?", FCT, r, S);
00287         return 0;
00288     }
00289     if (!a || !b) wblog(FL,
00290         "ERR %s() got null pointers (%p/%p) !?", FCT, a, b);
00291
00292     unsigned i=0, l=r-1;
00293     size_t j, k=0, len=S[0], I[r]; for (; i<r; ++i) { I[i]=0; }
00294
00295     for (i=1; i<r; ++i) len*=S[i];
00296
00297     for (k=0; k<len; ++k) {
00298         for (j=I[0], i=1; i<r; ++i) j = j*S[i] + I[i];
00299
00300         if (from)
00301             { a[k]=Ta(b[j]); }
00302         else { a[j]=Ta(b[k]); }
00303
00304         i=0; ++I[0];
00305         while (I[i]>=S[i] && i<l) { I[i]=0; ++I[++i]; }
00306     }
00307
00308     return len;
00309 };
00310
00311 template<class T>
00312 template <class Tb>
00313 int Mx::Array<T>::copy_from(const Tb* b, char tcheck) {
00314
00315     if (!data || !ax) { if (data || b) wblog(FL,
00316         "ERR %s() got null data (%p, %p, %p)", FCT, ax, data, b);
00317         return 0;
00318     }
00319
00320     if (!trans) {
00321         Wb::cpyRange(data, b, len, tcheck);
00322     }
00323     else {
00324         size_t l=
00325             copy_row2col_major(
00326                 rank, mxGetDimensions(ax), data, b,
00327                 'b'
00328             );
00329         if (l!=len) wblog(FL, "ERR %s() %d/%d", FCT, l, len);
00330     }
00331
00332     return (len ? 1 : 0);
00333 };
00334
00335 template<class T>
00336 size_t Mx::Array<T>::rcopy_to(
00337     T* b, const wbpem *P, char ck) const {
00338
00339     size_t l=0;
00340
00341     if (!ax) { if (data || b) wblog(FL,
00342         "ERR %s() got ax=%p (%p, %p)", FCT, ax, data, b);
00343         return l;
00344     }
00345     if (mxIsComplex(ax)) wblog(FL,
00346         "ERR %s got complex `%s'", xStr(), mxGetClassName(ax));
00347
00348     if (data) {
00349         return l=cpy_to_(b, P, ck, data);
00350     }
00351
00352     mxClassID t=mxGetClassID(ax);
00353     switch (t) {
00354         case mxDOUBLE_CLASS : l=cpy_to_(b, P, ck, mxGetDoubles (ax)); break;
00355         case mxCHAR_CLASS   : l=cpy_to_(b, P, ck, mxGetChars   (ax)); break;
00356         case mxINT8_CLASS    : l=cpy_to_(b, P, ck, mxGetInt8s   (ax)); break;
00357         case mxUINT8_CLASS   : l=cpy_to_(b, P, ck, mxGetUint8s  (ax)); break;
00358         case mxINT16_CLASS   : l=cpy_to_(b, P, ck, mxGetInt16s  (ax)); break;
00359         case mxUINT16_CLASS  : l=cpy_to_(b, P, ck, mxGetUint16s (ax)); break;
00360         case mxINT32_CLASS   : l=cpy_to_(b, P, ck, mxGetInt32s  (ax)); break;
00361         case mxUINT32_CLASS  : l=cpy_to_(b, P, ck, mxGetUint32s (ax)); break;

```

```

00362         case mxINT64_CLASS : l=cpy_to_(b,P,ck,mxGetInt64s (ax)); break;
00363         case mxUINT64_CLASS : l=cpy_to_(b,P,ck,mxGetUint64s (ax)); break;
00364         case mxLOGICAL_CLASS: l=cpy_to_(b,P,ck,mxGetLogicals(ax)); break;
00365         default: wblog(FL,"ERR %s unexpected type `%s'",xStr(),mxGetClassName(ax));
00366     }
00367
00368     if (l!=len) wblog(FL,"ERR %s len=%d/%d !?",xStr(),l,len);
00369     return l;
00370 };
00371
00372 namespace Mx {
00373
00374 template<>
00375 size_t Array<wbcomplex>::zcopy_to(
00376     wbcomplex *b, const wbperm *P, char ck) const {
00377
00378     size_t l=0;
00379
00380     if (!ax) { if (data || b) wblog(FL,
00381         "ERR %s() got null data (%p -> %p, %p)",FCT,ax,data,b);
00382         return l;
00383     }
00384
00385     if (data) { return l=cpy_to_(b,P,ck,data); }
00386
00387     mxClassID t=mxGetClassID(ax);
00388     if (t==mxDOUBLE_CLASS) {
00389         if (mxIsComplex(ax))
00390             l=cpy_to_(b,P,ck,(wbcomplex*)mxGetComplexDoubles(ax));
00391         else l=cpy_to_(b,P,ck,mxGetDoubles(ax));
00392     }
00393     else wblog(FL,"ERR %s() %s %s",FCT,
00394         mxIsComplex(ax) ? "complex":"type",mxGetClassName(ax));
00395
00396     if (l!=len) wblog(FL,"ERR %s len=%d/%d !?",xStr(),l,len);
00397     return l;
00398 }
00399
00400 template <>
00401 size_t Mx::Array<wbcomplex>::copy_to(wbcomplex *b, const wbperm *P, char tcheck)
00402 const { return zcopy_to(b,P,tcheck); };
00403
00404 };
00405
00406 template<class T>
00407 template<class Tx>
00408 size_t Mx::Array<T>::cpy_to_(
00409     T* b, const wbperm *P, char ck, const Tx *xd) const {
00410
00411     if (P && trans) wblog(FL,
00412         "ERR %s() got trans=%d together with permutation",FCT,trans);
00413     if (data && (void*)data!=(void*)xd) wblog(FL,
00414         "ERR %s got pointer mismatch %p / %p\n%s",xStr(),data,xd,STR_(this));
00415
00416     if (!trans && !P) {
00417         Wb::cpyRange(b,xd,len,ck);
00418         return len;
00419     }
00420
00421     if (!ax) wblog(FL,"ERR %s() %s",FCT,STR_(this));
00422
00423     if (trans) {
00424         size_t l=
00425             copy_row2col_major(
00426                 rank, mxGetDimensions(ax), b, xd,
00427                 0
00428             );
00429         if (l!=len) wblog(FL,"ERR %s() %ld/%ld",FCT,l,len);
00430     }
00431     else {
00432         size_t i=-1,j;
00433         unsigned k, l=rank-1;
00434         wbIndex I(rank);
00435
00436         const size_t *s0=mxGetDimensions(ax), *p=P->data;
00437
00438         if (!rank || P->len!=rank || len!=(i=mxGetNumberOfElements(ax)))
00439             wblog(FL,"ERR %s() size mismatch (r=%ld/%ld, len=%ld/%ld)",
00440                 FCT, rank, P->len, len, i);
00441
00442         for (i=0; i<len; ++i) {
00443             for (j=I[p[l]], k=l-1; k<l; --k) j = j*s0[p[k]] + I[p[k]];
00444
00445             b[j]=T(xd[i]);
00446
00447             if (ck && Tx(b[j])!=xd[i]) wblog(FL,
00448                 "ERR %s() rounding error (%g/%g)",FCT,double(b[j]),double(xd[i]));

```

```

00449
00450         k=0; ++I[0];
00451         while (I[k]>=s0[k] && k<l) { I[k]=0; ++I[++k]; }
00452     }
00453 }
00454
00455     return len;
00456 };
00457
00458 bool Mx::IsIndex(const mxArray *a, int base) {
00459     if (!a || !mxIsNumeric(a) || mxIsComplex(a)) { return 0; }
00460
00461     const size_t *s=mxGetDimensions(a);
00462     int r=mxGetNumberOfDimensions(a);
00463
00464     if (r!=2 || (s[0]!=1 && s[1]!=1)) { return 0; }
00465
00466     const double *d=mxGetDoubles(a);
00467     if (d) { return IsIndex(d,s[0]*s[1],base); }
00468     else {
00469         wvector<int> x(s[0]*s[1]);
00470         Mx::Array<int>(a).rcopy_to(x.data,0,'!');
00471         return IsIndex(x.data,x.len,base);
00472     }
00473 };
00474
00475 template<class T>
00476 wstring Mx::Array<T>::toStr(char vflag, unsigned l) const {
00477     wstring s(l);
00478
00479     if (ax) {
00480         unsigned i=0;
00481         const size_t *S=mxGetDimensions(ax);
00482
00483         l=snprintf(s.data,s.len,"%s%s ",
00484             mxIsComplex(ax)? "complex ":"",mxGetClassName(ax));
00485         for (; i<rank && l<s.len; ++i) {
00486             l+=snprintf(s.data+l,s.len-l,"%ldx",S[i]); }
00487
00488         if (l<s.len) { if (i) { --l; }
00489             l+=snprintf(s.data+l,s.len-l," [%s",TSTR(T)); }
00490         if (l<s.len) {
00491             l+=snprintf(s.data+l,s.len-l," @ ");
00492             if (!data) { l+=snprintf(s.data+l,s.len-l,"d=null; "); } else
00493             if (vflag) { l+=snprintf(s.data+l,s.len-l,"d=%p; ",data); }
00494         }
00495         if (l<s.len) {
00496             l+=snprintf(s.data+l,s.len-l,"%d,%d,%d; l=%ld]",
00497                 cmplx,trans,mxref,len);
00498         }
00499
00500         if (l>=s.len) wblog(FL,"ERR %s() string out of bounds "
00501             "(%d/%d)%N'%s'",FCT,l,s.len,s.data);
00502     }
00503     else {
00504         if (vflag) snprintf(s.data,s.len,
00505             "ax=%p -> %p (%s; %d,%d,%d; len=%ld)",ax,data,TSTR(T),
00506             cmplx,trans,mxref,len);
00507         else snprintf(s.data,s.len,
00508             "null%s (%s; %d,%d,%d; n=%ld)", data ? " -> d!=0:":"",
00509             TSTR(T), cmplx,trans,mxref,len);
00510     }
00511 };
00512 }
00513
00514     return s;
00515 };
00516
00517 template<class T>
00518 wstring Mx::Array<T>::toStrT(char vflag) const {
00519     wstring s(24);
00520     unsigned l=0;
00521
00522     if (!data || vflag) l=snprintf(s.data,s.len,"%s%s -> %s",
00523         mxIsComplex(ax)? "complex ":"",mxGetClassName(ax),TSTR(T));
00524     else l=snprintf(s.data,s.len,"%s",TSTR(T));
00525
00526     if (l>=s.len) wblog(FL,
00527         "ERR %s() string out of bounds (%d/%d)%N'%s'",FCT,l,s.len,s.data);
00528     return s;
00529 };
00530 };
00531
00532 template<class T>
00533 void Mx::Array<T>::issue_type_error(
00534     const char *F, int L, unsigned ndim, const size_t *dims) {
00535

```

```

00536     if (!ax || !mxGetNumberOfElements(ax)) {
00537         if (ndim) {
00538             size_t l=dims[0]; for (unsigned i=0; i<ndim; ++i) { l*=dims[i]; }
00539             if (l!=len) wblog(F_L,
00540                 "ERR %s length mismatch (%ld/%ld)",xStr(),l,len);
00541             if (l) wblog(F_L,
00542                 "ERR %s failed to allocate memory\n%s (%ld)",xStr(),
00543                 SSTR(wbvector<size_t>(ndim,dims,'r')), len
00544             );
00545         }
00546         if (len || data) wblog(F_L,"ERR %s got null (%d;%p)",xStr(),len,data);
00547     }
00548     else {
00549         wblog(F_L, "ERR %s type mismatch (%s%s/%s)",xStr(),
00550             mxIsComplex(ax) ? "complex ":"",mxGetClassName(ax),TSTR(T));
00551     }
00552 };
00553
00554 template<class T> inline
00555 int mxGetNumber(const mxArray *a, T &d, const char qflag) {
00556
00557     int n=0; d=T(0);
00558     if (!a) { Wb::quietErrLog(FL,"mxArray is NULL",qflag); return 1; }
00559
00560     try {
00561         Mx::Array<T> A(a); n=A.len;
00562         A.ncopy_to(&d,1,1);
00563     }
00564     catch (...) { n=-1; }
00565
00566     if (n==1) { return 0; }
00567     else {
00568         unsigned l,m=64; char msg[m];
00569         l=snprintf(msg,m,"%s %s() invalid input",SHORT_FL,FCT);
00570         if (n>1) { l+=snprintf(msg+l,m-l," (got array, n=%d)",n); } else
00571         if (!n) { l+=snprintf(msg+l,m-l," (empty)"); }
00572         Wb::quietErrLog(FL,msg,qflag);
00573         return 1;
00574     }
00575 };
00576
00577 inline int mxGetString(const mxArray *a, char *s) {
00578     if (mxGetString(a,s,127)) {
00579         sprintf(str,"%s:%d ERR could not read string (%s) !?", FL, s);
00580         return 1;
00581     }
00582     return 0;
00583 }
00584
00585 inline int mxGetString(const mxArray *a, wbststring &s) {
00586     char istr[128]; istr[0]=0;
00587     if (mxGetString(a,istr,127)) {
00588         sprintf(str,"%s:%d ERR could not read string (%s) !?", FL, istr);
00589         return 1;
00590     }
00591     s=istr; return 0;
00592 }
00593
00594 wbststring mxSize2Str(const mxArray *a) {
00595     if (!a) { return "(null)"; }
00596
00597     size_t l=0, n=32; char s[n];
00598     int i=0, r=mxGetNumberOfDimensions(a);
00599     const size_t *S=mxGetDimensions(a);
00600
00601     for (; i<r; ++i) {
00602         l+=snprintf(s+l,n-l,"%s%d", i ? "x":"", unsigned(S[i]));
00603         if (l>n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00604     }
00605     return s;
00606 };
00607
00608 wbststring mxTypeSize2Str(const mxArray *a) {
00609     size_t n=64; char s[n];
00610     if (!a) { strcpy(s,"(null)"); return s; }
00611
00612     size_t l=sprintf(s,"%s: ",mxGetClassName(a));
00613     int r=mxGetNumberOfDimensions(a);
00614     const size_t *S=mxGetDimensions(a);
00615
00616     for (int i=0; i<r; ++i) {
00617         l+=snprintf(s+l,n-l,"%s%d",i?"x":"", (unsigned)S[i]);
00618         if (l>n) wblog(FL,
00619             "ERR %s() string out of bounds (%d/%d)\n`%s'",FCT,l,n,s);
00620     }
00621     return s;
00622 };

```

```

00623
00624 template<class T>
00625 void mxGetVector(const char* F, int L,
00626     const mxArray *S, const char *vn,
00627     wbvector<T> &v
00628 ){
00629     if (!mxIsStruct(S) || !Mx::IsScalar(S)) wblog(F_L,
00630         "ERR input must be scalar structure");
00631
00632     mxArray *a=mxGetField(S,0,vn);
00633     if (!a) wblog(F_L,"ERR field %s does not exist",vn);
00634
00635     mxGetVector(F_L,a,v);
00636 };
00637
00638 double mxGetNumber(
00639     const char *F, int L,
00640     const mxArray *S, const char *vn, const char qflag
00641 ){
00642     mxArray *a;
00643     wbcomplex z;
00644
00645     if (!mxIsStruct(S)) wbdie(F_L,
00646         "need scalar structure on input");
00647     if (mxGetNumberOfElements(S)>1) wbdie(F_L,
00648         "need SINGLE structure on input");
00649
00650     a=mxGetField(S,0,vn);
00651
00652     if (!a) wblog(F_L,
00653         "ERR field %s does not exist in structure", vn);
00654
00655     if (mxGetNumber(a,z,qflag)) wbdie(F_L,str);
00656     if (z.i!=0.) wblog(F_L,"WRN Input number is complex! (%g)", z.i);
00657
00658     return z.r;
00659 }
00660
00661 template<class T>
00662 mxArray* cpyRange2Mx(const T* d, const wbvector<size_t> &S0){
00663
00664     mxArray *a;
00665     unsigned i=0, n=S0.prod(0), r=S0.len, len=MAX(2U,r);
00666     size_t s[len];
00667
00668     for (; i<r; ++i) s[i]=S0[i];
00669     if (i<len) {
00670         if (n) for (; i<len; ++i) s[i]=1;
00671         else for (; i<len; ++i) s[i]=0;
00672     }
00673
00674     a=Mx::Array<double>(len,s).copyFromTR(d);
00675     if (!a && d) wblog(F_L,
00676         "ERR %s() got %p -> %p @ len=%d !?",FCT,d,a,len);
00677
00678     return a;
00679 };
00680
00681 template<>
00682 mxArray* cpyRange2Mx(const char* d0, const wbvector<size_t> &S0){
00683
00684     unsigned i, r=S0.len, n=S0.prod(0), len=MAX(2U,r);
00685     mxArray *a;
00686
00687     size_t s[len];
00688     for (i=0; i<r; i++) s[i]=S0[i];
00689     if (i<2) { if (i) s[1]=1; else s[0]=s[1]=0; }
00690
00691     a=Mx::Array<mxChar>(len,s).Return();
00692
00693     if (a==NULL) {
00694         if (!d0) wblog(F_L,
00695             "ERR %s() invalid usage (%lX, %lX)",__FUNCTION__,d0,a);
00696         else return a;
00697     }
00698     if (!n) return a;
00699
00700     mxChar *d = (mxChar*)mxGetData(a);
00701     if (!d) wblog(F_L,"ERR invalid mxCharArray");
00702
00703     for (i=0; i<n; ++i) { d[i]=mxChar(d0[i]);
00704         if (char(d[i])!=d0[i]) wblog(F_L,
00705             "ERR %s() got data conversion error (%d/%d)",
00706             FCT,int(d[i]),int(d0[i])
00707         );
00708     }
00709

```

```

00710     return a;
00711 };
00712
00713 template<
00714 mxArray* copyRange2Mx(const wbcomplex* z, const wbvector<size_t> &S){
00715
00716     size_t i=0, n=S.prod(0); int isz=0;
00717     mxArray *a;
00718
00719     for (; i<n; ++i) { if (z[i].i!=0) { isz=1; break; } }
00720
00721     if (isz) {
00722         a=Mx::Array<wbcomplex>(S).copyFromTR(z);
00723     }
00724     else {
00725         Mx::Array<double> A(S); if (A.data) {
00726             for (i=0; i<n; ++i) { A.data[i]=z[i].r; }
00727             a=A.Return();
00728         }
00729
00730         if (!a && z) wblog(FL,"ERR %s() failed to allocate data !?",FCT);
00731         return a;
00732     };
00733
00734 mxArray* matGetVariable(
00735     const char *F, int L,
00736     const char *fname, const char *vn, char force
00737 ){
00738     Wb::matFile f;
00739     int i=f.open(F,L,fname,"r", force); if (!i) return NULL;
00740     mxArray *a=matGetVariable(f.mfp,vn);
00741     return a;
00742 };
00743
00744 mxArray* matGetVariableInfo(
00745     const char *F, int L,
00746     const char *fname, const char *vn, char force
00747 ){
00748     Wb::matFile f;
00749     int i=f.open(F,L,fname,"r", force); if (!i) return NULL;
00750     mxArray *a=matGetVariableInfo(f.mfp,vn);
00751     return a;
00752 };
00753
00754 double* mexSetCell2Matrix(mxArray *C, int fid, unsigned d1, unsigned d2) {
00755
00756     double *data;
00757     Mx::Array<double> A(d1,d2); data=A.data;
00758
00759     if (A.data) { mxSetCell(C,fid,A.Return()); } else
00760     if (A.ax || (d1 && d2)) { wblog(FL,"ERR %s() %s",FCT,STR(A)); }
00761
00762     return data;
00763 };
00764
00765 namespace Wb {
00766
00767 template <class TQ, class TD>
00768 mxArray* mxCreateSparse(
00769     const char *F, int L, unsigned d1, unsigned d2,
00770     const wbMatrix<TQ> &IJ,
00771     const wbvector<TD> &D,
00772     const wperm_t *P,
00773     int *info
00774 ){
00775     const char lex=-1;
00776     widx_t n=D.len;
00777     bool gotp=(p!=NULL);
00778
00779     if (D.data && ISCOMPLX_(TD)) wblog(F,L,
00780         "ERR %s() got complex sparse data set (%dx%d; %d)",FCT,d1,d2,D.len);
00781     if (IJ.dim1!=n || (n && IJ.dim2!=2)) wblog(F,L,
00782         "ERR %s() size mismatch (%dx%d : ij=%dx%d @ %d)",
00783         FCT,d1,d2,IJ.dim1,IJ.dim2,D.len);
00784
00785     if (!gotp && !IJ.isSorted(+1,lex)) {
00786         wbMatrix<TQ> IX(IJ);
00787         wperm P;
00788
00789         IX.SortRecs(P,+1,lex);
00790
00791         return mxCreateSparse(F,L,d1,d2,IX,D,P.data);
00792     }
00793
00794     mxArray *a=mxCreateSparse(d1,d2,D.len,mxREAL);
00795     if (!a) wblog(F,L,"ERR %s() "
00796         "failed to allocate sparse matrix (%dx%d; %d)",FCT,d1,d2,D.len);

```

```

00797     if (!n) { return a; }
00798
00799     double *sr=mxGetDoubles(a);
00800     mwIndex *is=mxGetIr(a), *js=mxGetJc(a),k=0;
00801     unsigned i,j=0, nz=0, m=0; TQ *ij=IJ.data;
00802
00803     if (!WbUtil<TQ>::isInt()) {
00804         wblog(FL,"ERR %s() invalid non-integer data type '%s'",FCT,
00805             TSTR(TQ));
00806     }
00807
00808     for (i=0; i<n; ++i, ij+=2) {
00809         const TD& x=(gotp ? D.data[p[i]] : D.data[i]);
00810         if (int(ij[0])<0 || ij[0]>=d1 || int(ij[1])<0 || ij[1]>=d2) wblog(FL,
00811             "ERR %s() index out of bounds (%d,%d) [%dx%d]",
00812             FCT,ij[0],ij[1],d1,d2);
00813         if (x!=0) {
00814             if (i) {
00815                 if (ij[-2]==ij[0] && ij[-1]==ij[1]) {
00816                     sr[k]+=double(x); ++m; continue;
00817                 }
00818                 else if (ij[-1]>ij[1] || (ij[-1]==ij[1] && ij[-2]>ij[0])) {
00819                     wblog(FL,"ERR %s() data not sorted (%d/%d) !?",FCT,i+1,n);
00820                 }
00821                 else ++k;
00822             }
00823             sr[k]=double(x);
00824             is[k]=ij[0];
00825
00826             for (; j<=ij[1]; ++j) js[j]=k;
00827         }
00828         else if (ij[0]!=ij[1]) ++nz;
00829     }
00830
00831     for (++k; j<=d2; ++j) js[j]=k;
00832
00833     if (F) {
00834         if (nz) { wblog(F,L,
00835             "WRN %s() got off-diag zero data (%d/%d) !?",FCT,nz,D.len);
00836         }
00837         if (m) { wblog(F,L,
00838             "WRN %s() got non-unique sparse data (%d/%d)",FCT,m,D.len);
00839         }
00840     }
00841     if (info) (*info)=nz+m;
00842
00843     return a;
00844 };
00845
00846 }
00847
00848 int mxAddField2Scalar(
00849     const char *F, int L,
00850     mxArray* S, const char *vn, mxArray *a
00851 ){
00852     int fid;
00853
00854     if (!mxIsStruct(S)) wblog(F_L,
00855         "ERR scalar input structure required");
00856     if (mxGetNumberOfElements(S)>1) wblog(F_L,
00857         "ERR single input structure required");
00858
00859     fid=mxAddField(S,vn);
00860
00861     if (fid<0) wblog(F_L,
00862         "ERR failed to add field '%s' !? (%d)", vn,fid);
00863
00864     if (a) mxSetFieldByNumber(S,0,fid,a);
00865     return fid;
00866 };
00867
00868 int mxAddField(
00869     const char *F, int L,
00870     mxArray* S, const char *name
00871 ){
00872     int fid = mxAddField(S, name);
00873
00874     if (fid<0) wblog(F,L,
00875         "ERR Failed to add field %s to structure (%d) ???", name, fid);
00876
00877     return fid;
00878 }
00879
00880 void mxAppendStructToStruct(
00881     const char *F, int L,
00882     mxArray *&S0, mxArray *S
00883 ){

```

```

00884     unsigned i,n;
00885     const char* name;
00886
00887     if (!mxIsStruct(S0) || mxGetNumberOfElements(S0)!=1 ||
00888         !mxIsStruct(S) || mxGetNumberOfElements(S)!=1) wblog(F,L,
00889         "ERR %s requires two scalar input structures",__FUNCTION__);
00890
00891     n=mxGetNumberOfFields(S0);
00892
00893     for (i=0; i<n; i++) {
00894         name=mxGetFieldNameByNumber(S0,i);
00895
00896         if (mxGetFieldNumber(S,name)>=0) wblog(F,L,
00897         "WRN overwriting field '%s' in structure", name);
00898
00899         mxAddField2Scalar(F,L,
00900         S, name, mxGetFieldByNumber(S0,0,i)
00901         );
00902
00903         mxSetFieldByNumber(S0,0,i,NULL);
00904     }
00905
00906     mxDestroyArray(S0); S0=NULL;
00907 }
00908
00909 int mxUpdateField(
00910     const char *F, int L,
00911     mxArray* S, const char *name, unsigned k, mxArray *a
00912 ){
00913     int fid=mxGetFieldNumber(S,name);
00914     unsigned n;
00915
00916     if (fid>=0) {
00917         mxArray *x=mxGetFieldByNumber(S,k,fid);
00918         if (x) mxDestroyArray(x);
00919     }
00920     else fid=mxAddField(F,L, S, name);
00921
00922     n=(unsigned)mxGetNumberOfElements(S);
00923
00924     if (k>=n) wblog(F,L,
00925     "ERR %s: index out of bounds (%d/%d)",FCT,k,n);
00926
00927     mxSetFieldByNumber(S,k,fid,a);
00928
00929     return fid;
00930 }
00931
00932 void matPutVariable(const char *F, int L,
00933     const char *file, const char *name, mxArray *a, unsigned keep
00934 ){
00935     Wb::matFile f;
00936
00937     f.Open(F,L,file,"u");
00938
00939     if (matPutVariable(f.mfp, name, a)) wblog(F,L,
00940     "ERR Failed to write parameter '%s' to file '%s'", name, file);
00941
00942     f.close();
00943     if (!keep) mxDestroyArray(a);
00944 }
00945
00946 void mxReplaceField(
00947     const char *F, int L,
00948     mxArray* S, unsigned k, int fid, mxArray *a
00949 ){
00950     if (!S || !mxIsStruct(S)) wblog(F,L,
00951     "ERR %s() got %s",FCT, S ? mxGetClassName(S):(empty));
00952
00953     mxArray *x=mxGetFieldByNumber(S,k,fid);
00954     if (x) mxDestroyArray(x);
00955
00956     mxSetFieldByNumber(S,k,fid,a);
00957 };
00958
00959 void mxSetFieldToNumber(
00960     mxArray *S, unsigned i, unsigned fid, wbcomplex z, char check) {
00961
00962     if (check) { unsigned n;
00963         if (!mxIsStruct(S)) wblog(F,L,"ERR invalid input structure");
00964         if (i>=(n=mxGetNumberOfElements(S))) wblog(F,L,
00965         "ERR index out of bounds (i=%d/%d)",i,n);
00966         if (fid>=(n=mxGetNumberOfFields(S))) wblog(F,L,
00967         "ERR index out of bounds (fid=%d/%d)",fid,n);
00968     }
00969
00970     mxArray *a; size_t sz[]={1,1};

```



```

00971     if (z.i)
00972     { a=Mx::Array<wbcomplex>(2,sz).copyFromTR(&z ); }
00973     else { a=Mx::Array<double>(2,sz).copyFromTR(&z.r); }
00974
00975     mxSetFieldByNumber(S,i,fid,a);
00976 };
00977
00978 int isFlagGlobal(const char *vn) {
00979     double flag=0;
00980     if (getNumGlobal(vn,flag)) return 0;
00981     return (flag!=0);
00982 }
00983
00984 template<class T>
00985 mxArray* vecVec2Mx(
00986     const wbvector< wbvector<T> > &V,
00987     const char tflag
00988 ){
00989     mxArray *C=mxCreateCellMatrix(1,V.len);
00990
00991     for (unsigned i=0; i<V.len; i++)
00992         mxSetCell(C, i, V[i].toMx(tflag));
00993
00994     return C;
00995 }
00996
00997 template<class T>
00998 mxArray* vecMat2Mx(
00999     const wbvector< wbMatrix<T> > &M,
01000     const char rawflag
01001 ){
01002     mxArray *C=mxCreateCellMatrix(1,M.len);
01003
01004     for (unsigned i=0; i<M.len; i++)
01005         mxSetCell(C, i, M[i].toMx(rawflag));
01006
01007     return C;
01008 }
01009
01010 template<class T>
01011 mxArray* vecVec2Mx(
01012     const wbvector< wbvector<T>* > &V,
01013     const char tflag
01014 ){
01015     mxArray *C=mxCreateCellMatrix(1,V.len);
01016     wbvector<T> emptyVec;
01017
01018     for (unsigned i=0; i<V.len; i++)
01019         if (V[i]) mxSetCell(C,i,V[i]->toMx(tflag));
01020     else mxSetCell(C, i, emptyVec.toMx(tflag));
01021
01022     return C;
01023 }
01024
01025 #endif
01026

```

5.11 mexlib.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace MEX routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEXLIB_HH__
00021 #define __WB_MEXLIB_HH__
00022
00023 /* ----- */
00024 /* ----- */

```

```

00025
00026 void mexDisp(const mxArray *a, const char *vname=0);
00027
00028 void mexWRN(const char* s);
00029
00030 wbststring sprint_info(const mxArray *a);
00031
00032 double wbtoc(
00033     const char *F=NULL, int L=0,
00034     const char *istr="", char restart=0
00035 );
00036
00037 void wbtic() { wbtoc(0,0); }
00038
00039 template<class T>
00040 int mxGetNumber(const mxArray *a, T &d, const char qflag=0);
00041
00042 double mxGetNumber(
00043     const char *F, int L,
00044     const mxArray *S, const char *vname, const char qflag=0
00045 );
00046
00047 int getFlag(const mxArray *a) {
00048     double q=0;
00049     Mx::Array<double> A(a);
00050     if (A.len>1) wblog(FL,
00051         "ERR %s() invalid flag (got array, n=%d)",FCT,A.len);
00052     A.ncopy_to(&q,1);
00053     return int(q);
00054 };
00055
00056 int mxGetString(const mxArray *a, char *s);
00057 int mxGetString(const mxArray *a, wbststring &s);
00058
00059 wbststring mxTypeSize2Str(const mxArray *a);
00060 wbststring mxSize2Str(const mxArray *a);
00061
00062 template<class T>
00063 void mxGetVector(const char* F, int L, const mxArray *a, wbstvector<T> &v) {
00064     size_t n=mxGetNumberOfElements(a);
00065     v.init(n);
00066     try {
00067         Mx::Array<T>(a).copyTo(v.data);
00068     }
00069     catch (...) { wblog(FL,"ERR %s()",FCT); }
00070 };
00071
00072 template<class T>
00073 void mxGetVector( const char* F, int L,
00074     const mxArray *S, const char *vname,
00075     wbstvector<T> &v);
00076
00077 mxArray* matGetVariable(const char *F, int L,
00078     const char *fname, const char *vname, char force=0
00079 );
00080 mxArray* matGetVariableInfo(const char *F, int L,
00081     const char *fname, const char *vname, char force=0
00082 );
00083
00084 double* mexSetCell2Matrix(mxArray *a, int fid, unsigned d1, unsigned d2);
00085 void mxSetFieldToNumber(mxArray *S, unsigned i, unsigned fid,
00086     wbstcomplex z, char dblcheck=1);
00087
00088 template<class T>
00089 int getNumGlobal(const char *vname, T &x, const char *ws="global");
00090 int isFlagGlobal(const char *vname);
00091
00092 template<class T>
00093 mxArray* vecVec2Mx(
00094     const wbstvector< wbstvector<T> > &V, const char tflag=0);
00095
00096 template<class T>
00097 mxArray* vecVec2Mx(
00098     const wbstvector< wbstvector<T>* > &V, const char tflag=0);
00099
00100 template<class T>
00101 mxArray* vecMat2Mx(
00102     const wbstvector< wbstMatrix<T> > &M, const char rawflag=0);
00103
00104 template<class T>
00105 mxArray* cpyRange2Mx(const T* d, const wbstvector<size_t> &S0);
00106
00107 int mxAddField2Scalar(
00108     const char *F, int L,
00109     mxArray* S, const char *vname, mxArray *a=NULL
00110 );
00111

```

```

00112 void mxAppendStructToStruct(
00113     const char *F, int L,
00114     mxArray *S0, mxArray *S
00115 );
00116
00117 int mxAddField(
00118     const char *F, int L,
00119     mxArray* S, const char *vname
00120 );
00121
00122 void mxReplaceField(
00123     const char *F, int L,
00124     mxArray* S, unsigned k, int fid, mxArray *a
00125 );
00126
00127 int mxUpdateField(
00128     const char *F, int L,
00129     mxArray* S, const char *name, unsigned k, mxArray *a
00130 );
00131
00132 void matPutVariable(const char *F, int L,
00133     const char *file, const char *name, mxArray *a, unsigned keep=0
00134 );
00135
00136 namespace Wb {
00137
00138     template <class TQ, class TD>
00139     mxArray* mxCreateSparse(
00140         const char *F, int L, unsigned d1, unsigned d2,
00141         const wbMatrix<TQ> &IJ, const wbvector<TD> &D,
00142         const wperm_t *p=NULL, int *info=NULL
00143     );
00144
00145 }
00146
00147 template<class T>
00148 mxArray* numtoMx(const T &x) {
00149     mxArray *a=mxCreateDoubleMatrix(1,1,mxREAL);
00150     mxGetDoubles(a)[0]=double(x); return a;
00151 };
00152
00153 template<>
00154 mxArray* numtoMx(const wbcomplex &z) {
00155     mxArray *a;
00156     if (z.i!=0.) {
00157         a=mxCreateDoubleMatrix(1,1,mxCOMPLEX);
00158         z.explicit_copy2(mxGetComplexDoubles(a)[0]);
00159     }
00160     else a=numtoMx(z.r);
00161     return a;
00162 };
00163
00164 template<>
00165 mxArray* numtoMx(const char &c) {
00166     if (isalpha(c)) {
00167         char s[2]={c,0};
00168         return mxCreateString(s);
00169     }
00170     else return numtoMx(double(c));
00171 };
00172
00173 class mxStruct {
00174 public:
00175
00176     mxStruct(const char *file, int line) : S(NULL) { init(file,line); };
00177     ~mxStruct() {
00178         if (S) {
00179             wblog(FL,"WRN mxStruct() destroyed without further ado");
00180             mxDestroyArray(S);
00181         }
00182     };
00183
00184     void init(const char *file, int line) {
00185         F=file; L=line;
00186         if (S) mxDestroyArray(S);
00187         S=mxCreateStructMatrix(1,1,0,NULL);
00188     };
00189
00190     void put(const char *name) {
00191         if (S) { mxPutAndDestroy(F.data,L,S,name); S=NULL; } else
00192             wblog(FL,"WRN mxStruct::put() not initialized yet - return");
00193         F.init(); L=0;
00194     };
00195
00196     void save2Struct(mxArray *S0, const char *name) {
00197         if (S) {
00198             mxAddField2Scalar(F.data, L, S0, name, S);

```

```

00199         S=NULL;
00200     } else
00201     wblog(FL,"WRN mxStruct::save2Struct() not initialized yet");
00202     F.init(); L=0;
00203 };
00204
00205 void checkNewField(const char *f=NULL, int l=0);
00206
00207 mxStruct& operator()(const char *fn) { fld=fn; return *this; };
00208
00209 template<class T>
00210 mxStruct& operator=(const T&x) {
00211     checkNewField(FL);
00212     mxAddField2Scalar(F.data, L, S, fld.data, numtoMx(x));
00213     return *this;
00214 };
00215
00216 mxStruct& operator=(mxArray *a) {
00217     checkNewField(FL);
00218     mxAddField2Scalar(F.data, L, S, fld.data, a);
00219     return *this;
00220 };
00221
00222 template<class T>
00223 mxStruct& operator=(const wbvector<T>& x) {
00224     checkNewField(FL);
00225     mxAddField2Scalar(F.data, L, S, fld.data, x.toMx());
00226     return *this;
00227 };
00228
00229 template<class T>
00230 mxStruct& operator=(const wbMatrix<T>& x) {
00231     checkNewField(FL);
00232     mxAddField2Scalar(F.data, L, S, fld.data, x.toMx());
00233     return *this;
00234 };
00235
00236 mxArray *S;
00237 wbsting F, fld;
00238 unsigned L;
00239
00240 protected:
00241 private:
00242
00243 };
00244
00245 template<>
00246 mxStruct& mxStruct::operator=(const wbpem &P) {
00247     checkNewField(FL);
00248     mxAddField2Scalar(F.data, L, S, fld.data, P.toMx());
00249     return *this;
00250 }
00251
00252 template<>
00253 mxStruct& mxStruct::operator=(const wbindex &I) {
00254     checkNewField(FL);
00255     mxAddField2Scalar(F.data, L, S, fld.data, I.toMx());
00256     return *this;
00257 }
00258
00259 inline void mxStruct::checkNewField(const char *f, int l) {
00260     int i; if (!f) { f=F.data; l=L; }
00261
00262     if (!S) wblog(f,l,"ERR mxStruct() S not set yet");
00263     if (fld.isEmpty()) wblog(f,l,"ERR mxStruct() field not specified");
00264
00265     i=mxGetFieldNumber(S,fld.data);
00266     if (i>=0) wblog(f,l,"ERR Field '%s' already exists!");
00267 };
00268
00269 class MXPut {
00270 public:
00271
00272     MXPut(const char *vn=0, const char *ws=0)
00273         : S(0), vname(0),F(0), L(0) {
00274         strncpy(wsp, ws && ws[0]? ws:"caller",wlen-1); wsp[wlen-1]=0;
00275         init(0,0,vn);
00276     };
00277
00278     MXPut(const char *file, int line, const char *vn="ans", const char *ws=0)
00279         : S(0), vname(0),F(0), L(0) {
00280         strncpy(wsp, ws && ws[0]? ws:"caller",wlen-1); wsp[wlen-1]=0;
00281         init(file,line,vn);
00282     };
00283
00284 ~MXPut() { put(); }
00285

```

```

00286     void init() {
00287         if (F) { WB_DELETE(F); }; L=0;
00288         if (vname) { WB_DELETE(vname); }
00289         if (S) {
00290             if (mxGetNumberOfFields(S)) { const char *v="I_check_this";
00291                 wblog(F_L,"WRN %s() clearing non-empty S (see %s)",FCT,v);
00292                 mxPutAndDestroy(F,L,S,v,"caller");
00293             }
00294             else { mxDestroyArray(S); }
00295             S=NULL;
00296         }
00297     };
00298
00299     void init(const char *file, int line, const char *vn=NULL) {
00300         if (S || F || vname) init();
00301         if (file && file[0]) { L=line;
00302             WB_NEW(F,strlen(file)+1); strcpy(F,file);
00303         }
00304
00305         if (vn && vn[0])
00306             { WB_NEW(vname,strlen(vn)+1); strcpy(vname,vn); }
00307         else { WB_NEW(vname,4); strcpy(vname,"ans"); }
00308
00309         #pragma omp critical (using_MEX_API)
00310         S=mxCreateStructMatrix(1,1,0,NULL);
00311
00312         if (L) addFL();
00313     };
00314
00315     void put(const char *ws=0, const char *vn=0) {
00316         if (S) {
00317             if (!vname) wblog(F_L,
00318                 "ERR %s() got empty variable name",FCT,vname);
00319             const char s0[]="ans", *s=((vn && vn[0]) ? vn : vname);
00320             if (!s || !s[0]) s=s0;
00321
00322             if (!ws || !ws[0]) {
00323                 if (!wsp[0]) wblog(F_L,"ERR %s() workspace not set",FCT);
00324                 ws=wsp;
00325             }
00326
00327             mxPutAndDestroy(F,L,S,s,ws);
00328             S=NULL;
00329         }
00330         init();
00331     };
00332
00333     void save2(mxArray* &S0) { S0=S; S=NULL; init(); };
00334
00335     mxArray* toMx() { mxArray* S0=S; S=NULL; init(); return S0; };
00336
00337     void add2Struct(mxArray *S0) {
00338         if (S) {
00339             if (!vname) wblog(F_L,
00340                 "ERR %s() got empty variable name !?",FCT,vname);
00341             mxAddField2Scalar(F_L,S0,vname,S); S=NULL;
00342         }
00343         init();
00344     };
00345
00346     MXPut& addFL() {
00347         if (!S) wblog(F_L,"ERR MXPut::%s() S not initialized yet",FCT);
00348         else {
00349             if (!vname) wblog(F_L,
00350                 "ERR %s() got empty variable name !?",FCT,vname);
00351             if (mxGetFieldNumber(S,"FL")<0)
00352                 mxAddField2Scalar(F_L,S,"FL",wbstring(shortFL(F_L)).toMx());
00353             else wblog(F_L,"WRN %s() field FL already set",FCT);
00354         }
00355         return *this;
00356     };
00357
00358     template<class T>
00359     MXPut& add(const T&x, const char *fn) {
00360         if (!S) wblog(F_L,"ERR MXPut::%s() S not initialized yet",FCT);
00361         mxAddField2Scalar(F_L,S,fn,x.toMx());
00362         return *this;
00363     };
00364
00365     template<class T>
00366     MXPut& addP(const T* x, const char *fn) {
00367         if (!S) wblog(F_L,"ERR MXPut::%s() S not initialized yet",FCT);
00368         else wblog(F_L,
00369             "ERR %s() not defined yet for pointer to %s",
00370             FCT,TSTR(T));
00371         return *this;
00372     };

```

```

00373
00374     mxArray *S;
00375     char wsp[16];
00376     char *vname, *F;
00377     unsigned L;
00378
00379     static const unsigned wlen;
00380
00381 protected:
00382 private:
00383 };
00384
00385     const unsigned MXPut::wlen=16;
00386
00387 template<
00388 MXPut& MXPut::addP(const mxArray* a, const char *fn) {
00389     if (!S) wblog(F_L,"ERR MXPut::%s() S not set yet",FCT);
00390     if (!vname) wblog(F_L,"ERR %s() got empty variable name",FCT,vname);
00391     mxAddField2Scalar(F_L,S, fn, (mxArray*)a);
00392     return *this;
00393 };
00394
00395 #ifndef NOMEX
00396 template<
00397 MXPut& MXPut::addP(const char* s, const char *fn) {
00398     return addP(mxCreateString(s),fn);
00399 };
00400 #endif
00401
00402 template<
00403 MXPut& MXPut::add(const double &x, const char *fn) {
00404     return addP(numtoMx(x),fn);
00405 };
00406 template<
00407 MXPut& MXPut::add(const float &x, const char *fn) {
00408     return addP(numtoMx(x),fn);
00409 };
00410 template<
00411 MXPut& MXPut::add(const long double &x, const char *fn) {
00412     return addP(numtoMx(x),fn);
00413 };
00414 template<
00415 MXPut& MXPut::add(const wbcplx &x, const char *fn) {
00416     return addP(numtoMx(x),fn);
00417 };
00418 template<
00419 MXPut& MXPut::add(const char &x, const char *fn) {
00420     return addP(numtoMx(x),fn);
00421 };
00422
00423 template<
00424 MXPut& MXPut::add(char* const &s, const char *fn) {
00425     return add(wbstring(s),fn);
00426 };
00427 template<
00428 MXPut& MXPut::add(const char* const &s, const char *fn) {
00429     return add(wbstring(s),fn);
00430 };
00431
00432 template<
00433 MXPut& MXPut::add(const unsigned &x, const char *fn) {
00434     return addP(numtoMx(x),fn);
00435 };
00436 template<
00437 MXPut& MXPut::add(const int &x, const char *fn) {
00438     return addP(numtoMx(x),fn);
00439 };
00440 template<
00441 MXPut& MXPut::add(const bool &x, const char *fn) {
00442     return addP(numtoMx(x),fn);
00443 };
00444 template<
00445 MXPut& MXPut::add(const long &x, const char *fn) {
00446     return addP(numtoMx(x),fn);
00447 };
00448 template<
00449 MXPut& MXPut::add(const unsigned long &x, const char *fn) {
00450     return addP(numtoMx(x),fn);
00451 };
00452
00453 #ifdef __WB_MPFR_HH__
00454 template<
00455 MXPut& MXPut::add(const Wb::quad &x, const char *fn) {
00456     return addP(numtoMx(x),fn);
00457 };
00458 #endif
00459

```

```

00460 class mxArray_buf {
00461
00462 public:
00463     mxArray_buf(const char *n=NULL) {
00464         BUF.max_load_factor(0.2);
00465         if (n && n[0])
00466             { name = new char[strlen(n)+1]; strcpy(name,n); }
00467         else { name = new char[4]; strcpy(name,"MXB"); }
00468     };
00469
00470 ~mxArray_buf() {
00471     size_t l=BUF.size();
00472     if (l) wblog(FL,"WRN %s() %s got %d entries still",PROG,name,l);
00473     BUF.clear();
00474     if (name) { delete [] name; name=NULL; }
00475 };
00476
00477 void add(void *p, mxArray *a) {
00478     if (p) {
00479         #pragma omp critical (using_mxArray_P2X)
00480         { auto im=BUF.insert({p,a});
00481             if (!im.second) wblog(FL,
00482                 "ERR %s() %s already got entry for %p",PROG,name,p);
00483         }
00484     }
00485     else wblog(FL,"WRN %s() got null pointer",FCT,p);
00486 };
00487
00488 mxArray* find(const char *F, int L, void *p) {
00489     mxArray *a=NULL;
00490
00491     #pragma omp critical (using_mxArray_P2X)
00492     { auto im=BUF.find(p);
00493         if (im!=BUF.end()) { a=im->second; }
00494     }
00495     if (!a && F) wblog(F,L,
00496         "WRN %s() %s missing entry for %p",PROG,name,p);
00497     return a;
00498 };
00499
00500 mxArray* Return(const char *F, int L, void *p) {
00501     mxArray *a=NULL;
00502
00503     #pragma omp critical (using_mxArray_P2X)
00504     { auto im=BUF.find(p);
00505         if (im!=BUF.end()) {
00506             a=im->second; BUF.erase(im->first);
00507         }
00508     }
00509     if (!a && F) wblog(F,L,
00510         "WRN %s() %s missing entry for %p",PROG,name,p);
00511     return a;
00512 };
00513
00514 int erase(void *p) {
00515     int e=0;
00516     #pragma omp critical (using_mxArray_P2X)
00517     { e=BUF.erase(p); }
00518     return e;
00519 };
00520
00521 unordered_map<void*, mxArray*> BUF;
00522
00523 char* name;
00524
00525 protected:
00526 private:
00527
00528 };
00529
00530     mxArray_buf P2X("P2X");
00531
00532 #endif
00533

```

5.12 mpsortho.cc

```

00001 #ifndef __WB_ORTHO_CC__
00002 #define __WB_ORTHO_CC__
00003
00004 // ===== //
00005 // QBlock :: class for block transformations of QSpace data{:}
00006 // e.g. combining/blocking data into fused symmetry sectors

```

```

00007 // ----- //
00008 // toBlockMatrix() operates on this->A
00009 // which is input and needs to be set prior to a call to *this
00010 //
00011 // => for all-abelian symmetries, *this can group multiple symmetry
00012 // sectors into single full matrix MM; typically, however, also
00013 // in the abelian case, either Q1 or Q2 represents a single
00014 // total (fused) symmetry sector, which needs to be taken care of
00015 // in the caller for all-abelian [e.g. using getQsum() on either
00016 // Q1 or Q2 then yields the same result].
00017 //
00018 // => NB! for non-abelian symmetries, this->A must represent single
00019 // well-defined symmetry sector to avoid conflicts with fusion
00020 // rules hence Q1 or Q2 must represent a single leg / index,
00021 // i.e., a single set of symmetry labels, and thus below either
00022 // n1 or n2 must equal 1! Then fusion rules are respected
00023 // automatically (since by construction, all symmetry labels
00024 // from Q1 must be fuseable into Q2 and vice versa, as already
00025 // described by the isometries (up to normalization) via
00026 // CRef -> CData.
00027 //
00028 // Wb,Aug23,08 // Wb,Aug04,15
00029
00030 template <class TQ, class TD, class TE>
00031 void QBlock<TQ,TD,TE>::toBlockMatrix( // ::initBlockMatrix
00032     wbararray<TD> &MM, // output matrix
00033     unsigned K, // group first K dimension and also the rest
00034     const wbpemr &P // permutation on data *before* using K
00035 ) {
00036     char cgflag=A.getCGS(FL), wom=0;
00037     unsigned R=-1, nsym=A.CGR.dim2;
00038     widxt i,j,k,d,l,1l=0, r,s, n1,n2, i1,i2, d1,d2, dim1,dim2;
00039
00040     wbarMatrix<unsigned> OM, OMD, mark;
00041     wbarvector<unsigned> M1,M2, omk;
00042     wbararray<TD> Ak,Mk;
00043     wbararray<double> CGW;
00044     wbindx i1,i2;
00045
00046     groupIndex<widxt> g1, g2;
00047     wbarvector<widxt> s1,s2, cD1, cD2;
00048
00049     if (!A.isConsistent(R)) { wblog(FL,"%s",str); }
00050     if (A.isEmpty()) {
00051         if (K || P) wblog(FL,
00052             "ERR %s() got empty QSpace with K=%d / P.len=%d",FCT,K,P.len);
00053         MM.init(); this->init();
00054         return;
00055     }
00056
00057     if (int(R)<2) wblog(FL,"ERR %s() got invalid QSpace (rank r=%d)",FCT,R);
00058     if (int(K)<0) { K=R/2; } else
00059     if (K>R) wblog(FL,"ERR index out of bounds (k=%d/%d)",K,R);
00060     if (!K || K==R) wblog(FL,"ERR got empty group (k=%d/%d)",K,R);
00061
00062     qdim_tot=1; qdim.init2val(nsym,1);
00063
00064     if (cgflag>0) {
00065         if (K== 1) { wom=2; 1l=0; } else
00066         if (K==(R-1)) { wom=1; 1l=K; }
00067         else {
00068             wblog(FL,"ERR %s() got non-abelian CGR with K=%d/%d (cg=%d)\n"
00069                 "(hint: fuse tensor to rank-2 since non-abelian)",FCT,cgflag,K,R);
00070         }
00071
00072         if (A.CGR) { qdim.init(A.CGR.dim2);
00073             for (i=0; i<A.CGR.dim1; ++i) {
00074                 for (j=0; j<A.CGR.dim2; ++j) {
00075                     A.CGR(i,j).norm2();
00076                     if (i) {
00077                         d = (A.CGR(i,j).cgb ? A.CGR(i,j).qdim(1l) : 1);
00078                         if (d!=qdim[j]) wblog(FL,"ERR %s() "
00079                             "CGR(%d,%d) inconsisteny qdim=%d/%d",FCT,i+1,j+1,d,qdim[j]
00080                             );
00081                     }
00082                     else if (A.CGR(i,j).cgb) {
00083                         qdim_tot *= (qdim[j] = A.CGR(i,j).qdim(1l));
00084                     }
00085                 }
00086             }
00087         else { qdim.init(); }
00088     }
00089
00090     if (K!=1 && K!=(R-1) && (A.itags || A.CGR)) {
00091         int e=0; QDir qdir; A.getQDir(qdir);
00092
00093         if (qdir.len!=R) { e=1; }

```



```

00094     else {
00095         if (P) { qdir.Permute(P); }
00096         for (i=1; i<K; ++i) { if (qdir[i]!=qdir[0]) { e=2; break; } }
00097         for (++i; i<R; ++i) { if (qdir[i]!=qdir[K]) { e=3; break; } }
00098     }
00099     if (e) wblog(FL,
00100 "ERR %s() got mixed qdir=%s (k=%d/%d; e=%d)",FCT,STR(qdir), K,R,e);
00101 }
00102
00103 if (P) {
00104     if (P.len!=R) wblog(FL,
00105 "ERR invalid permutation (len=%d/%d)",P.len,R);
00106     I1.init( K, P.data );
00107     I2.init(R-K, P.data+K);
00108 }
00109 else {
00110     I1.Index(0,K-1);
00111     I2.Index(K,R-1);
00112 }
00113
00114 if (!A.QIDX.isUnique()) wblog(FL,
00115 "ERR %s() got non-unique Q-labels",FCT);
00116
00117 A.getQsub(I1,Q1).groupRecs(g1.P,g1.D); g1.setup('f');
00118 A.getQsub(I2,Q2).groupRecs(g2.P,g2.D); g2.setup('f');
00119
00120 n1=g1.D.len; S1.init(n1,I1.len);
00121 n2=g2.D.len; S2.init(n2,I2.len);
00122
00123 A.getOM(OMD,&M2,2);
00124
00125 if (!A.getOM(OM,&M1,1)) {
00126     D1.init2val(n1,2, 1);
00127     D2.init2val(n2,2, 1);
00128 }
00129 else if (n1==1 && !(n2==1 && wom==1)) {
00130     if (wom!=2) wblog(FL,
00131 "ERR %s() wom=%d/2 (n1=%d, n2=%d, K=%d/%d)",FCT,wom,n1,n2,K,R);
00132     D1.init2val(n1,2, 1);
00133     D2.init2val(n2,2+OM.dim2, 1);
00134     for (l=i=0; i<n2; ++i, l+=d) { d=g2.D[i];
00135         if ((D2(i,1)=M1[g2.P[l]])>1) {
00136             Wb::cpyRange(D2.ref(i,2), OM.rec(g2.P[l]), OM.dim2); }
00137     }
00138 }
00139 else if (n2==1) {
00140     if (wom!=1) wblog(FL,
00141 "ERR %s() wom=%d/1 (n1=%d, n2=%d, K=%d/%d)",FCT,wom,n1,n2,K,R);
00142     D1.init2val(n1,2+OM.dim2, 1);
00143     D2.init2val(n2,2, 1);
00144     for (l=i=0; i<n1; ++i, l+=d) { d=g1.D[i];
00145         if ((D1(i,1)=M1[g1.P[l]])>1) {
00146             Wb::cpyRange(D1.ref(i,2), OM.rec(g1.P[l]), OM.dim2); }
00147     }
00148 }
00149 else { wblog(FL,
00150 "ERR %s() got composite indices for all groups\n"
00151 "in the presence of outer-multiplicity (%s)\n"
00152 "(hint: fuse indices at least for one group since non-abelian)",
00153 FCT, A.qStr().data);
00154 }
00155
00156 mark.init(n1+1,n2+1);
00157 idx.init2val(n1,n2,-1);
00158
00159 for (i=0; i<A.DATA.len; ++i) {
00160     A.DATA[i]->getSize(I1,s1); r=g1.Ig[i];
00161     A.DATA[i]->getSize(I2,s2); s=g2.Ig[i];
00162
00163     idx(r,s)=i;
00164
00165     if (++mark(r,s)>1 && !wom) { wblog(FL,
00166 "ERR %s() block %d not unique\n"
00167 "(got degenerate records in QSpace w/out OM)",FCT,i+1);
00168     }
00169     if (++mark(r,n2)==1) { S1.recSet(r,s1); D1(r,0)=s1.prod(0); }
00170     else if (S1.recCompare(r,s1)) { wblog(FL,
00171 "ERR %s() block size mismatch (%d, r: %d/%d)\n[%s, %s]",
00172 FCT, i, r+1, n1, S1.rec2Str(r,"","x").data, SSTR(s1));
00173     }
00174     if (++mark(n1,s)==1) { S2.recSet(s,s2); D2(s,0)=s2.prod(0); }
00175     else if (S1.recCompare(r,s1)) { wblog(FL,
00176 "ERR %s() block size mismatch (%d, c: %d/%d)\n[%s, %s]",
00177 FCT, i, s+1, n2, S1.rec2Str(r,"","x").data, SSTR(s1));
00178     }
00179 }
00180

```

```

00181     dim1=D1.cumsum0prodRec(cD1,0,1);
00182     dim2=D2.cumsum0prodRec(cD2,0,1);
00183     MM.init(dim1,dim2);
00184
00185     for (k=0; k<A.DATA.len; ++k) {
00186         const CRef<TQ> *Rk=(nsym ? A.CGR.rec(k) : NULL);
00187
00188         r = g1.Ig[k]; s = g2.Ig[k];
00189         i1 = cD1[r]; i2 = cD2[s];
00190         d1 = D1(r,0); d2 = D2(s,0);
00191
00192         if (M1[k]==1) { double cfac=1;
00193             A.DATA[k]->toMatrixRef(Mk,K,P);
00194
00195             for (i=0; i<nsym; ++i) {
00196                 if ((l=Rk[i].wnumel())>1) wblog(FL,"ERR %s() "
00197                     "unexpected CGR(%d,%d) %s",FCT,k+1,i+1,SSTR(Rk[i].cgw));
00198                 if (l==1) { cfac*=Rk[i].wget0(); }
00199             }
00200
00201             if (Mk.SIZE.len!=2 || Mk.SIZE[0]!=d1 || Mk.SIZE[1]!=d2) wblog(FL,
00202                 "ERR %s() size inconsistency %s / %dx%d",FCT,SSTR(Mk),d1,d2);
00203             if (mark(r,s)!=1 || M2[k]!=1) { wblog(FL,"ERR %s() "
00204                 "expecting unique block [mark(%d/%d,%d/%d)=%d] / M2=%d",
00205                 FCT,r+1,n1,s+1,n2,mark(r,s),M2[k]); }
00206             if (i2+Mk.SIZE[1]>dim2) { wblog(FL,
00207                 "ERR %s() index out of bounds:\n"
00208                 "setting M(%d:%d,%d:%d) having size %s",FCT,
00209                 i1+1, i1+Mk.SIZE[0], i2+1, i2+Mk.SIZE[1], SSTR(MM)); }
00210             cfac/=sqrt(double(qdim_tot));
00211
00212             Mk.copyStride( MM.ref(i1,i2), dim1, cfac);
00213         }
00214         else {
00215             if (mark(r,s)!=1) { wblog(FL,"ERR %s() "
00216                 "unexpected unique block [mark(%d/%d,%d/%d)=%d] @ M1=%d",
00217                 FCT,r+1,n1,s+1,n2,mark(r,s),M1[k]); }
00218
00219             if (M1[k]<2) { wblog(FL,"ERR %s() got M1[%d]=%d",FCT,k,M1[k]); }
00220             if (!wom) { wblog(FL,"ERR %s() got wom=%d",FCT,wom); }
00221
00222             for (i=0; i<nsym; ++i) {
00223                 if (i) { CGW.kron(Rk[i].cgw, CGW); }
00224                 else { CGW=Rk[i].cgw; }
00225             }
00226
00227             if (!CGW.isMatrix(M1[k],M2[k])) wblog(FL,"ERR %s() "
00228                 "size mismatch CGW %s / %dx%d",FCT,SSTR(CGW),M1[k],M2[k]);
00229
00230             if (qdim_tot>1) {
00231                 CGW *= sqrt(1./double(qdim_tot));
00232             }
00233             else if (qdim_tot<1) wblog(FL,
00234                 "ERR %s() invalid qdim=%d",FCT,qdim_tot);
00235
00236             A.DATA[k]->contractMat(FL,R+1,CGW,2,Ak);
00237
00238             Ak.toMatrixRefM(Mk,K,P,R, wom==1? "132:");
00239             { wvector<size_t> S(Mk.SIZE);
00240                 if (wom==1)
00241                     { S[0]*=S[1]; S[1]=S[2]; }
00242                 else { S[1]*=S[2]; }
00243                 S.len=2;
00244                 Mk.Reshape(S);
00245             }
00246
00247             if (i1+Mk.SIZE[0]>dim1 || i2+Mk.SIZE[1]>dim2) { wblog(FL,
00248                 "ERR %s() index out of bounds:\n"
00249                 "setting M(%d:%d,%d:%d) having %s",FCT,
00250                 i1+1,i1+Mk.SIZE[0], i2+1,i2+Mk.SIZE[1], SSTR(MM));
00251             }
00252
00253             Mk.copyStride( MM.ref(i1,i2), dim1);
00254         }
00255     }
00256 };
00257
00258 template <class TQ, class TD, class TE>
00259 QBlock<TQ,TD,TE>& QBlock<TQ,TD,TE>::init_bare_refA(
00260     const QBlock &b, unsigned flag) {
00261
00262     if (this==&b) {
00263         if (flag) wblog(FL,"ERR %s() got same object (flag=%d)",FCT,flag);
00264         return *this;
00265     }
00266
00267     if (!strchr("USVX",flag)) wblog(FL,

```

```

00268     "ERR %s() invalid flag %c<%d>",FCT,flag,flag);
00269
00270     I0.init(); Ik.init(); It.init(); U.init(); S.init(); Vc.init();
00271
00272     A.init2ref(b.A);
00273
00274     if (!flag) {
00275         qdim_tot=b.qdim_tot; qdim=b.qdim;
00276         Q1=b.Q1; S1=b.S1; D1=b.D1;
00277         Q2=b.Q2; S2=b.S2; D2=b.D2; idx=b.idx;
00278         return *this;
00279     }
00280
00281     int k=0, e=0;
00282
00283     if (b.S2.dim2==1) { k=2;
00284         b.Q2.blockSum(b.A.QDIM,Q2);
00285         if (b.S2.dim2>1 && b.A.isNonAbelian()) { e=1; }
00286     }
00287     else { k=1;
00288         b.A.getQsum(b.Q1,Q2);
00289     }
00290
00291     if (e || !Q2.recAllEqual() || !Q2.dim1) {
00292         MXPut (FL, "q").add(b, "b").add(Q2, "Q2").add(k, "k");
00293         wblog (FL,
00294             "ERR %s() got inconsistency in block symmetry\n(%s; [%d+%d])",
00295             FCT, SSTR(b.idx), b.S1.dim2, b.S2.dim2
00296             );
00297     }
00298     Q2.dim1=1;
00299
00300     S2.init(1,1);
00301     D2.init2val(1,2, 1);
00302
00303     qdim_tot=b.qdim_tot;
00304     qdim=b.qdim;
00305
00306     if (flag=='U') {
00307         Q1=b.Q1; S1=b.S1; D1=b.D1; D2[0]=S2[0]=b.U.dim(2);
00308
00309         if (b.idx.dim2>1) {
00310             idx.init2val(b.idx.dim1,1, b.A.DATA.len);
00311         }
00312         else idx=b.idx;
00313     }
00314     else if (flag=='V' || flag=='X') {
00315         Q1=b.Q2; S1=b.S2; D1=b.D2; D2[0]=S2[0]=b.Vc.dim(2);
00316
00317         if (b.idx.dim1>1) {
00318             idx.init2val(b.idx.dim2,1, b.A.DATA.len);
00319         }
00320         else b.idx.transpose(idx);
00321     }
00322     else if (flag=='S') { D2[0]=S2[0]=b.S.len;
00323         Q1=Q2; D1=D2; S1=S2;
00324         idx.init(1,1);
00325     }
00326     else wblog (FL, "ERR %s() invalid flag=%d",FCT,flag);
00327
00328 #ifndef WB_SKIP_ASSERT
00329 { wvector<widxt> cD_; widxt dim2_=S2[0],
00330     d1=b.D1.cumsum0prodRec(cD_,0,1),
00331     d2=b.D2.cumsum0prodRec(cD_,0,1);
00332     if (dim2_>d1 && dim2_>d2) wblog (FL,
00333         "ERR %s() unexpected dS=%d (%dx%d)",FCT,dim2_,d1,d2
00334         ); }
00335 #endif
00336
00337     return *this;
00338 };
00339
00340 template <class TQ, class TD, class TE>
00341 void QBlock<TQ,TD,TE>::updateBlockDim(unsigned dim2_) {
00342
00343     if (S2.numel()!=1 || D2.dim1!=1 || D2.dim2<2 || D2[1]!=1)
00344         wblog (FL, "ERR %s() QBlock not yet initialized\n"
00345             "S2: %s, D2: %s",FCT,STR(S2),STR(D2)
00346             );
00347
00348     D2[0]=S2[0]=dim2_;
00349 };
00350
00351 template <class TQ, class TD, class TE>
00352 void QBlock<TQ,TD,TE>::initFromBlockMatrix(const char *F, int L,
00353     QSpace<TQ,TD> &B,
00354     const wbarrray<TD> &MM,

```

```

00355     const iTags &it,
00356     const wbperm *cgp
00357 ) const {
00358
00359     widx_t i,j,k, i0,i1,i2, d1,d2, r,s, dim1, dim2;
00360     wbvector<widx_t> cD1, cD2, S;
00361     wbvector<TQ> Q;
00362
00363     #ifndef WB_SKIP_ASSERT
00364     wbvector<unsigned> Ik(1);
00365     #endif
00366
00367     char cgflag=A.getCGS(F_L);
00368     char cpR=0;
00369     double cfac=1;
00370
00371     if (cgflag>0) {
00372         r=it.len;
00373         s=S1.dim2+S2.dim2;
00374         if (s<2 || s!=r) wblog(FL,
00375             "ERR %s() got rank %d/%d",FCT,s,r);
00376
00377         if (!cgp) {
00378             if (A.itags.sameConj(it)) { cpR=1; }
00379         }
00380         else if (A.itags.len==cgp->len) {
00381             iTags tp(A.itags); tp.Permute(*cgp);
00382             if (tp.sameConj(it)) { cpR=2; }
00383         }
00384         if (!cpR) {
00385             if (it.len!=2 || !it[0].isConj(it[1],0)) wblog(FL,
00386                 "ERR %s() unexpected non-abelian setting\n"
00387                 "r=%d: %s (A: %s); cgp=[%s]", FCT, it.len, STR(it),
00388                 STR(A.itags), STR(cgp ? (*cgp) : wbperm())
00389             );
00390         }
00391     }
00392
00393     if (Q1.dim1!=S1.dim1 || D1.dim1!=S1.dim1 || D1.dim2<2 ||
00394         Q2.dim1!=S2.dim1 || D2.dim1!=S2.dim1 || D2.dim2<2) { wblog(FL,
00395         "ERR block index mismatch (%d/%d/%dx%d; %d/%d/%dx%d)",
00396         Q1.dim1, S1.dim1, D1.dim1, D1.dim2,
00397         Q2.dim1, S2.dim1, D2.dim1, D2.dim2);
00398     }
00399
00400     dim1=D1.cumsum0prodRec(cD1,0,1);
00401     dim2=D2.cumsum0prodRec(cD2,0,1);
00402
00403     if (MM.rank()!=2) wblog(FL,
00404         "ERR %s() got rank-%d object matrix",FCT,MM.rank());
00405     if (dim1!=MM.SIZE[0] || dim2!=MM.SIZE[1]) wblog(FL,
00406         "ERR block size mismatch (%dx%d; %s)",
00407         dim1, dim2, MM.sizeStr().data);
00408     if (!A.QDIM || Q1.dim2%A.QDIM || Q2.dim2%A.QDIM) wblog(FL,
00409         "ERR invalid QDIM (%d,%d / %d)",Q1.dim2,Q2.dim2,A.QDIM);
00410
00411     B.init(D1.colSum(1)*D2.colSum(1), S1.dim2+S2.dim2, A.QDIM);
00412     B.qtype=A.qtype;
00413     B.itags=it;
00414
00415     if (A.CGR.dim2) {
00416         B.setupCGR();
00417     }
00418
00419     for (k=r=0; r<D1.dim1; ++r) {
00420     for ( s=0; s<D2.dim1; ++s, ++k) {
00421         i1 = cD1[r]; i2 = cD2[s];
00422         d1 = D1(r,0); d2 = D2(s,0); if (!d1 || !d2) {
00423             wblog(FL,"WRN %s() got empty %dx%d sub-block",FCT,d1,d2);
00424             continue;
00425         }
00426
00427         if ((i0=idx(r,s))<0) {
00428             double x2=MM.norm2block(i1,i2,d1,d2);
00429             if (x2>CG_SKIP_DEPS1 || !isfinite(x2)) { char s[64];
00430                 snprintf(s,64,"block with norm x2=%.3g\nM(%ld:%ld,%ld:%ld) "
00431                     "having %s",x2,i1+1,i1+d1,i2+1,i2+d2,SSTR(MM));
00432                 if (x2>1e-8)
00433                     { wblog(FL,"ERR %s() got %s",FCT,s); }
00434                 else { wblog(FL,"WRN %s() skipping %s",FCT,s); }
00435             }
00436             continue;
00437         }
00438
00439         if (cpR && i0>=A.CGR.dim1) {
00440             MXPput(FL,"q").add(A,"A").add(B,"B")
00441                 .add(D1,"D1").add(cD1,"cD1").add(D2,"D2").add(cD2,"cD2")

```

```

00442         .add(idx,"idx").add(r+1,"r").add(s+1,"s").add(k+1,"k");
00443         wblog(FL,"ERR %s() index out of bounds:\nid(%d,%d) = %d/%d",
00444             FCT,r+1,s+1,i0+1,A.CGR.dim1);
00445     }
00446
00447     S.init(S1.dim2, S1.rec(r), S2.dim2, S2.rec(s));
00448     Q.init(Q1.dim2, Q1.rec(r), Q2.dim2, Q2.rec(s));
00449
00450     B.QIDX.recSet(k,Q); cfac=1;
00451     if (cpR) {
00452         for (j=0; j<A.CGR.dim2; ++j) {
00453             B.CGR(k,j).init_wId( A.CGR(i0,j),1,-1,&qdim[j]);
00454
00455             if (qdim[j]>1) { cfac*=B.CGR(k,j).NormSignW(FL); }
00456
00457             if (cgp) { B.CGR(k,j).Permute(*cgp); }
00458         }
00459
00460         if (fabs(cfac)<1e-8) wblog(FL,
00461             "WRN %s() got small cfac=%g",FCT,cfac);
00462
00463         #ifndef WB_SKIP_ASSERT
00464             Ik[0]=k;
00465             B.checkQ_CGR(FL,&Ik);
00466         #endif
00467     }
00468     else if (cgflag) {
00469         B.initIdentityCGS(FL);
00470     }
00471
00472     if (D1(r,1)==1 && D2(s,1)==1) {
00473         if (i1+d1>dim1 || i2+d2>dim2) { wblog(FL,"ERR %s() "
00474             "index out of bounds (%d:%d; %d:%d) having %s",
00475             FCT, i1+1,i1+d1, i2+1,i2+d2, SSTR(MM));
00476         }
00477
00478         wbarray<TD> Bk; Bk.init_bare(d1,d2);
00479         Wb::cpyStride(Bk.data, MM.ref(i1,i2), d1,d2,-1,dim1,TD(0),TD(cfac));
00480
00481         Bk.Reshape(S).save2(*B.DATA[k]);
00482     }
00483     else {
00484         unsigned m=0, wom=0; int e=0;
00485         widx_t d1_=d1, d2_=d2;
00486
00487         if (!cpR) wblog(FL,"ERR %s() "
00488             "expecting cpR!=0 with OM=%dx%D",FCT,D1(r,1),D2(s,1));
00489
00490         if (D1(r,1)==1) { wom=2; m=D2(s,1); d2_*=m;
00491             if (D2.dim2!=2+A.CGR.dim2) { e=1; }
00492         } else
00493         if (D2(s,1)==1) { wom=1; m=D1(r,1); d1_*=m;
00494             if (D1.dim2!=2+A.CGR.dim2) { e=2; }
00495         }
00496         else { e=3; }
00497
00498         if (e) { char sx[80]; snprintf(sx,80,"block (%ld/%ld,%ld/%ld) "
00499             "at OM=%ldx%ld)\nhaving %s | %s | cpR=%d, ' %s' (k=%ld)",
00500             r+1,D1.dim1, s+1,D2.dim1, D1(r,1), D2(s,1),
00501             D1.rec2Str(r).data, D2.rec2Str(s).data, cpR,STR(it),k);
00502             if (e<3)
00503                 wblog(FL,"ERR %s() invalid D1/D2 data (e=%d)\n%s",FCT,e,sx);
00504             else wblog(FL,"ERR %s() got multiple OM for\n%s",FCT,sx);
00505         }
00506
00507         if (i1+d1_>dim1 || i2+d2_>dim2) { char sx[32];
00508             if (wom==1)
00509                 { snprintf(sx,32,"(%d*%ld=%ld) x %ld", m,d1,d1_, d2); }
00510             else { snprintf(sx,32,"%ld x (%d*%ld=%ld)", d1, m,d2,d2_); }
00511
00512             wblog(FL,"ERR %s() index out of bounds "
00513                 " (%d:%d/%d; %d:%d/%d)\nhaving block size %s",
00514                 FCT, i1+1,i1+d1_,dim1, i2+1,i2+d2_,dim2, sx
00515             );
00516         }
00517
00518         wbarray<TD> Bk; Bk.init_bare(d1_,d2_);
00519         Wb::cpyStride(Bk.data, MM.ref(i1,i2), d1_,d2_,-1,dim1,TD(0),TD(cfac));
00520
00521         if (wom==1)
00522             { Bk.Reshape(d1,m,d2).Permute("132"); }
00523
00524         S.Append(m);
00525         Bk.Reshape(S).save2(*B.DATA[k]);
00526     }
00527 }
00528

```

```

00529     for (k=i=0; i<B.DATA.len; ++i) {
00530         if (B.DATA[i]->isEmpty()) { continue; }
00531         if (k<i) {
00532             B.QIDX.recSet(k,i);
00533             for (j=0; j<B.CGR.dim2; ++j) { B.CGR(i,j).save2(B.CGR(k,j)); }
00534             B.DATA[k]=B.DATA[i];
00535         }; ++k;
00536     }
00537
00538     if (k<B.DATA.len) {
00539         B.QIDX.Resize(k,B.QIDX.dim2);
00540         B.CGR .Resize(k,B.CGR .dim2);
00541         B.DATA.Resize(k);
00542     }
00543 };
00544
00545 template <class TQ, class TD>
00546 void SVD_Data<TQ,TD>::blockSVD(
00547     const QSpace<TQ,TD> &PSI,
00548     const unsigned K
00549 ){
00550     unsigned i,n, l=0, r=PSI.rank(FL);
00551     wbvector<widxt_t> D,Dc;
00552     wbMatrix<TQ> Qb;
00553     wbindx Ib;
00554     wperm P;
00555
00556     if (PSI.isEmpty()) wblog(FL,"ERR %s() got empty PSI",FCT);
00557     if (!K || K>=r) wblog(FL,
00558         "ERR %s() index specs out of bounds (%d/%d)",FCT,K,r);
00559
00560     if (K!=1) { QDir qdir; PSI.getQDir(qdir);
00561         if (qdir.len!=r) l=1;
00562         else {
00563             for (i=0; i<K; ++i) { if (qdir[i]<=0) { l=2; break; }}
00564             for (++i; i<r; ++i) { if (qdir[i]!=qdir[K]) { l=3; break; }}
00565         }
00566         if (l) wblog(FL,
00567             "ERR %s() got invalid orthonormalization\n"
00568             "'%s' => %s grouped into %d+%d, »\nhint: %s",
00569             FCT, STR(PSI.itags), STR(qdir), K,r-K, (l==2 ?
00570             "only ingoing indices can be fused into outgoing" : (l==3 ?
00571             "2nd/right group is mixed in+out" : "rank mismatch"))
00572         );
00573     }
00574
00575     if (K== 1) { Ib.Index(0,K-1); } else
00576     if (K==r-1) { Ib.Index(K,r-1); }
00577     else {
00578         if (!PSI.allAbelian()) wblog(FL,
00579             "ERR %s() got non-abelian symmetries %s\nwith K=%d/%d",
00580             FCT,PSI.qStr().data,K,r);
00581         Ib.Index(0,K-1);
00582     }
00583
00584     PSI.getQsum(Ib,Qb).groupRecs(P,D);
00585     n=D.len; QB.init(n);
00586
00587     Wb::LogException ex;
00588
00589     int np=MIN(int(n),QSP_NUM_THREADS);
00590     if (np<1) np=1;
00591
00592     D.cumsum_(Dc);
00593
00594     #pragma omp parallel for num_threads(np)
00595     for (i=0; i<n; ++i) { if (!ex) { try {
00596         QBlock<TQ,TD> &b=QB[i];
00597         wbararray<TD> MM;
00598
00599         b.I0.init(D[i],P.data+Dc[i]);
00600         PSI.getSub(b.I0, b.A);
00601
00602         b.toBlockMatrix(MM, K);
00603
00604         wbSVD(MM, b.U, b.S, b.Vc);
00605     }
00606     catch (Wb::LogException &e_) { ex+=e_; }
00607     catch (...) { ++ex; }
00608     }}
00609     ex.report(FLF);
00610 };
00611
00612 template <class TQ, class TD>
00613 unsigned SVD_Data<TQ,TD>::dmrgTruncate(
00614     unsigned Nkmin, unsigned Nkeep, double stol,
00615     itag_ tx,

```

```

00616     wbMatrix<double> &SM,
00617     QSpace<TQ,TD> &UQ,
00618     QSpace<TQ,TD> *SQ,
00619     QSpace<TQ,TD> &VC,
00620     mxArray *Sout,
00621     char vflag
00622 ){
00623
00624     unsigned i,j,i0,d, nk=0, nt=0, nq=QB.len, flag=0;
00625     unsigned QDIM, N;
00626     int NK=Nkeep;
00627
00628     int revert_Uidx=0;
00629
00630     iTags tU,tS,tV;
00631     wbperm P, cgp2;
00632
00633     QBlock<TQ,TD,double> x;
00634     QSpace<TQ,TD> X;
00635
00636     wbvector< wbvector<double>* > > SV;
00637     wbvector< wbvector<char> > > mark;
00638     wbvector<double> Stot;
00639
00640     double snorm, snorm2=0, s2t=0, sfac=1,
00641           xtol2=(stol>0 ? stol*1e-14 : 0);
00642     char cgflag=0;
00643
00644     unsigned l=0, n=128; char istr[n]; istr[0]=0;
00645
00646     if (!nq) wblog(FL,"ERR %s() got empty QB data",FCT);
00647     else {
00648         QDIM=QB[0].A.QDIM;
00649         if (!QDIM) wblog(FL,"ERR QDIM=%d",QDIM);
00650         cgflag=QB[0].A.getCGS(FL);
00651     }
00652
00653     UQ.init(); if (SQ) { SQ->init(); }
00654     VC.init();
00655
00656     for (N=i=0; i<nq; i++) { N+=QB[i].S.len; }
00657
00658     SV.init(nq);
00659
00660     SM.init(N, cgflag? 2:1);
00661
00662     if (!N) wblog(FL,"ERR %s() got empty SVD data",FCT);
00663     else --N;
00664
00665     for (i0=i=0; i<nq; ++i, i0+=d) {
00666         const wbvector<double> &S=QB[i].S; d=S.len;
00667
00668         SV[i]=&(QB[i].S);
00669         snorm2+=S.norm2()*QB[i].qdim_tot;
00670
00671         if (!cgflag) {
00672             for (j=0; j<d; ++j) { SM[i0+j]=S[j]; }
00673         }
00674         else {
00675             double *sm=SM.rec(i0);
00676             for (j=0; j<d; ++j, sm+=2) {
00677                 sm[0]=S[j];
00678                 sm[1]=QB[i].qdim_tot;
00679             }
00680         }
00681     }
00682
00683     SM.SortRecs(P,-1);
00684
00685     if (int(Nkmin)<0) { Nkmin=0; }
00686
00687     snorm=SQRT(snorm2);
00688     markSet(SV,mark,Nkmin,NK,stol,Stot,0.,0.,-1,"desc");
00689
00690     for (i=0; i<nq; ++i) {
00691         QBlock<TQ,TD> &b=QB[i];
00692         s2t+=b.get_s2t(mark[i]);
00693         nk+=b.Ik.len; nt+=(mark[i].len-b.Ik.len);
00694     }
00695
00696     if (nk!=unsigned(NK) || nk+nt!=SM.dim1) wblog(FL,
00697         "WRN size inconsistency (nk=%d/%d, N=%d+%d / %d)",
00698         nk,NK, nk,nt,SM.dim1);
00699
00700     if (SM(N,0)>stol) { if (SM.dim1>Nkeep) {
00701         if (stol) {
00702             l+=snprintf(istr,n,"Smin=%.3g > stol=%.3g (NK=%d/%d/%li)",

```

```

00703         SM(N,0), stol, Nkmin,NK,SM.dim1); flag++;
00704     }
00705     else {
00706         l+=snprintf(istr,n,"using Nkeep=%d (%d/%d/%li; "
00707         "stol=%g; Smin=%.3g)",NK,Nkmin,Nkeep,SM.dim1, stol, SM(N,0));
00708     }
00709     else {
00710         l+=snprintf(istr,n,"keeping all (stol=%g; Smin=%.3g, "
00711         "NK=%d (%d/%d/%li)",stol, SM(N,0), NK,Nkmin,Nkeep,SM.dim1);
00712         xtol2=0;
00713     }
00714 }
00715 else {
00716     if (unsigned(NK)<Nkeep) {
00717         if (Nkmin) {
00718             l+=snprintf(istr,n,"truncation by stol=%.3g or Nkmin=%d "
00719             "(Nkeep %d->%d; Smin=%.3g < %.3g)", stol,
00720             Nkmin,Nkeep,NK, SM(N,0), SM(nk? nk-1:0,0));
00721         }
00722         else {
00723             l+=snprintf(istr,n,"truncation by stol=%.3g "
00724             "(Nkeep %d->%d; Smin=%.3g < %.3g)", stol,
00725             Nkeep,NK, SM(N,0), SM(nk? nk-1:0,0));
00726         }
00727     }
00728     else if (unsigned(NK)<SM.dim1) {
00729         l+=snprintf(istr,n,"truncation by Nkeep ([%d,%d] -> %d/%li; "
00730         "Smin=%.3g < %.3g <= stol=%.3g)", Nkmin,Nkeep,NK,SM.dim1,
00731         SM(N,0), SM(MAX(0,int(NK)-1),0), stol);
00732         flag++;
00733     }
00734 }
00735
00736 if (s2t) {
00737     if (l>=n) wblog(FL,"ERR %s() "
00738     "string out of bounds (%d/%d)%N%N%s%N",FCT,l,n,istr);
00739     else if (snorm2>s2t) {
00740         sfac=snorm/sqrt(snorm2-s2t);
00741         if (fabs(sfac-1)>0.01) { ++flag;
00742             snprintf(istr+l,n-l,
00743             "%sWRN readjusting norm by factor %.4g",l?"\n":"",sfac);
00744             if (fabs(sfac-1)>0.1 && (vflag || Wb::envVRB&8)) {
00745                 wblog(FL,istr+(l?l+1:0));
00746             }
00747         }
00748         for (i=0; i<nq; ++i) QB[i].S*=sfac;
00749     }
00750     else if (snorm2==s2t) { snprintf(istr+l,n-l,
00751     "all states discarded (|SVD|=%.3g / %.3g; Nkeep=%d/%d/%li)",
00752     snorm,stol,NK,Nkeep,SM.dim1);
00753 }
00754     else wblog(FL,"ERR %s() %g/%g",FCT,snorm2,s2t);
00755 }
00756
00757 if (Sout) {
00758     mxAddField2Scalar(FL, Sout, "svd", SM.toMx());
00759     mxAddField2Scalar(FL, Sout, "stol", numtoMx(stol));
00760     mxAddField2Scalar(FL, Sout, "svd2tr", numtoMx(s2t/snorm2));
00761     mxAddField2Scalar(FL, Sout, "sv2tot", numtoMx(snorm2));
00762     mxAddField2Scalar(FL, Sout, "sfac", numtoMx(sfac));
00763     mxAddField2Scalar(FL, Sout, "Nkmin", numtoMx(Nkmin));
00764     mxAddField2Scalar(FL, Sout, "Nkeep", numtoMx(nk));
00765     mxAddField2Scalar(FL, Sout, "Ntot", numtoMx(SM.dim1));
00766     mxAddField2Scalar(FL, Sout, "flag", numtoMx(flag));
00767     mxAddField2Scalar(FL, Sout, "info", mxCreateString(istr));
00768 }
00769 else if (vflag) {
00770     wblog(FL,"<i> sum(SVD) = %g = %g%+.4g",
00771     snorm2, round(snorm2), snorm2-round(snorm2));
00772     wblog(FL,"TST keep %d/%d states (%d)",nk,SM.dim1,Nkeep); if (nt) {
00773         wblog(FL,":x: truncate %d/%d states", nt,SM.dim1); }
00774 }
00775
00776 tx.deConj();
00777
00778 { QB<block<TQ,TD> &b=QB[0];
00779
00780     itag_ tu(tx), tv;
00781
00782     b.A.checkQ(FL);
00783
00784     cgp2.init2End(0,b.S2.dim2+1);
00785
00786     if (b.A.itags.len!=b.S1.dim2+b.S2.dim2) wblog(FL,
00787     "ERR %s() got invalid/empty itags (%s; s=%d+%d)",
00788     FCT, b.A.itags.toStr().data, b.S1.dim2+b.S2.dim2);
00789

```



```

00790     if (b.S2.dim2==1) {
00791         tu.Set_(b.A.itags.last(i));
00792         if (!tu.isConj() && NK) {
00793             if (!b.A.isOp(2,'L')) { revert_Uidx=1; }
00794         }
00795     }
00796     else if (b.S1.dim2==1) {
00797         tu.Set_(b.A.itags[i=0]); tu.Conj();
00798     }
00799     else { i=-1;
00800         if (!b.A.itags[0].isConj()) { tu.Conj(); }
00801     }
00802     tv=tu;
00803     if (SQ && i==1 && b.A.rank()==2) {
00804         tu.tSet(b.A.itags[0]);
00805         if (!b.A.itags[1].isConj() && revert_Uidx) { tu.MarkDual(); }
00806     }
00807
00808     tU.init(b.S1.dim2, b.A.itags.data, 1, &tu);
00809
00810     tS.init(1,&tu,1,&tv);
00811     tS[0].Conj();
00812
00813     tV.init(b.S2.dim2, b.A.itags.data+b.S1.dim2, 1, &tv);
00814     tV.last().Conj();
00815
00816 }
00817
00818 for (i=0; i<nq; ++i) {
00819     QBlock<TQ,TD> &b=QB[i]; if (b.Ik.isEmpty()) continue;
00820     b.U .Select0(b.Ik,1);
00821     b.S .Select(b.Ik);
00822     b.Vc.Select0(b.Ik,1);
00823
00824     if (b.U .rank()!=2) wblog(FL,"ERR U: invalid rank %s",SSTR(b.U));
00825     if (!b.S.len      ) wblog(FL,"ERR got empty S");
00826     if (b.Vc.rank()!=2) wblog(FL,"ERR V: invalid rank %s",SSTR(b.Vc));
00827
00828     x.init_bare_refA(b,'U');
00829     x.initFromBlockMatrix(FL,X,b.U,tU);
00830
00831     if (X.Append2AndDestroy(FL,UQ,'u')) wblog(FL,
00832         "ERR %s() U: QIDX must not overlap\n%s",FCT,istr);
00833
00834     if (SQ) {
00835         wbararray<double> Sf;
00836         Sf.initDiag(b.S.len,b.S.data);
00837
00838         x.init_bare_refA(b,'S');
00839         x.initFromBlockMatrix(FL,X,Sf,tS);
00840
00841         if (X.DATA.len!=1) wblog(FL,
00842             "ERR %s() got S.DATA.len=%d",FCT,X.DATA.len);
00843         X.DATA[0]->Reduce2Diag();
00844
00845         if (X.Append2AndDestroy(FL,*SQ,'u')) wblog(FL,
00846             "ERR %s() S: QIDX must not overlap\n%s",FCT,istr);
00847
00848         x.init_bare_refA(b,'V');
00849         x.initFromBlockMatrix(FL,X,b.Vc,tV,&cgp2);
00850
00851         if (X.Append2AndDestroy(FL,VC,'u')) wblog(FL,
00852             "ERR %s() V: QIDX must not overlap\n%s",FCT,istr);
00853     }
00854     else {
00855         wbararray<TD> VS; Wb::DMatProd(b.Vc,b.S,VS);
00856
00857         if (VS.rank()!=2) wblog(FL,
00858             "ERR VS: invalid rank-%d",VS.rank());
00859
00860         x.init_bare_refA(b,'X');
00861         x.initFromBlockMatrix(FL,X,VS,tV,&cgp2);
00862
00863         X.SkipZeroData(xtol2,'b',0);
00864
00865         if (X.Append2AndDestroy(FL,VC,'u')) wblog(FL,
00866             "ERR %s() VS: QIDX must not overlap\n%s",FCT,istr);
00867     };
00868 }
00869
00870 }
00871
00872 UQ.checkQ(FL); if (SQ) SQ->checkQ(FL);
00873 VC.checkQ(FL);
00874
00875 if (revert_Uidx) {
00876

```

```

00877     unsigned r1=UQ.rank(FL); wbindindex ia(1);
00878     wbvector< QSpace<TQ,TD> >X(1);
00879     QSpace<TQ,TD> U1J;
00880
00881     if (UQ.qtype!=VC.qtype || (!UQ.qtype.allAbelian() &&
00882         (!UQ.CGR.data || !VC.CGR.data))) wblog(FL,"ERR %s() "
00883         "got uninitialized CGR data\n(0x%lx, 0x%lx having %s)",
00884         FCT, UQ.CGR.data, VC.CGR.data, STR(VC.qtype));
00885     if (VC.itags.len!=2) wblog(FL,"ERR %s() "
00886         "unexpected rank-%d for %s",FCT,VC.itags.len, SQ ? "V":"VS");
00887
00888     if (SQ) {
00889         SQ->save2(X[0]); U1J.initIdentityCG(X,ia,'z');
00890         U1J.contract(FL,1,X[0],ia[0]+1,*SQ);
00891     }
00892     else {
00893         ia[0]=VC.rank(FL)-1;
00894         VC.save2(X[0]); U1J.initIdentityCG(X,ia,'z');
00895         X[0].contract(FL,ia[0]+1,U1J,1,VC);
00896     }
00897
00898     UQ.save2(X[0]); {
00899         X[0].contract(FL,r1,U1J,"1*",UQ);
00900     }
00901 }
00902
00903
00904 return flag;
00905 };
00906
00907 char mxGetLRDir(
00908     const char *F, int L,
00909     const mxArray *a, unsigned k
00910 ){
00911     char w=0, e=0; char s[8];
00912
00913     if (mxIsChar(a)) {
00914         if (mxGetString(a,s,8)) {
00915             if (F) wblog(F,L,
00916                 "ERR failed to read lrdir string (arg %d)",k);
00917             return w;
00918         }
00919
00920         if (!strcmp(s,"»")) w+=3; else
00921         if (!strcmp(s,"«")) w-=3; else
00922         if (!strcmp(s,"LR")) w+=2; else
00923         if (!strcmp(s,"RL")) w-=2; else
00924         if (!strcmp(s,"+" )) w+=1; else
00925         if (!strcmp(s,"-" )) w=-1; else {
00926             if (F) wblog(F,L,
00927                 "ERR invalid lrdir (arg %d: '%s')",k,s);
00928             return w;
00929         }
00930     }
00931     else if (Mx::IsNumber(0,0,a)) {
00932         if (mxGetNumber(a,w)) e=1;
00933         else {
00934             if (!w || w<-3 || w>+3) { e=2; }
00935         }
00936         if (e) {
00937             if (F) wblog(F,L,
00938                 "ERR invalid lrdir (arg %d: %d; e=%d)",k,w,e);
00939             w=0; return w;
00940         }
00941     }
00942     else e=3;
00943
00944     if (!w) e=4;
00945     if (e) {
00946         if (F) wblog(F,L,
00947             "ERR invalid lrdir (arg %d; e=%d)",k,e);
00948         return w;
00949     }
00950
00951     return w;
00952 };
00953
00954 template <class TQ, class T1, class T2, class T3>
00955 void twoSiteInit(
00956     const QSpace<TQ,T1> &A1,
00957     const QSpace<TQ,T2> &A2,
00958     QSpace<TQ,T3> &PSI, unsigned &K,
00959     widx_t ic1, widx_t ic2,
00960     char twoSite, char ldir
00961 ){
00962     const unsigned r1=A1.rank(), r2=A2.rank();
00963     wbindindex I1, I2;

```

```

00964     wbpem P;
00965
00966     if (!ldir) wblog(FL,"ERR ldir=%d",ldir);
00967     if ((ldir>0 && ic1>=r1) || (ldir<0 && ic2>=r2)) wblog(FL,
00968         "ERR contraction index out of bounds (%d/%d, %d/%d)",
00969         ic1,r1,ic2,r2);
00970
00971     if (twoSite) {
00972         if (ldir>0) {
00973             A1.contract(ic1+1, A2, ic2+1, PSI);
00974             I1.Index(r1-1); I2.Index(r2-1)+=(r1-1);
00975         }
00976         else {
00977             A2.contract(ic2+1, A1, ic1+1, PSI);
00978             I1.Index(r2-1); I2.Index(r1-1)+=(r2-1);
00979         }
00980     }
00981     else {
00982         if (ldir>0)
00983             { PSI=A1; I1.Index(r1).Skip(ic1); I2.init(1,&ic1); }
00984         else { PSI=A2; I1.Index(r2).Skip(ic2); I2.init(1,&ic2); }
00985     }
00986
00987     if (PSI.isEmpty()) wblog(FL,"WRN %s yields empty QSpace",FCT);
00988
00989     P.Cat(I1,I2);
00990     PSI.Permute(P); K=I1.len;
00991 };
00992
00993 template <class TQ, class T1, class T2, class T3>
00994 void twoSiteFinal(
00995     const QSpace<TQ,T1> &Psi1,
00996     const QSpace<TQ,T2> &Psi2,
00997     QSpace<TQ,T3> &A1,
00998     QSpace<TQ,T3> &A2,
00999     unsigned ic1, unsigned ic2, char twoSite, char ldir
01000 ){
01001     QSpace<TQ,T3> AX;
01002
01003     if (!ldir) wblog(FL,"ERR ldir=%d",ldir);
01004
01005     if (twoSite) {
01006         if (ldir>0) {
01007             A1.PermuteLastTo (ic1);
01008             A2.PermuteFirstTo(ic2);
01009         }
01010         else {
01011             A2.permuteFirstTo(ic1, AX);
01012             A1.permuteLastTo (ic2, A2); AX.save2(A1);
01013         }
01014     }
01015     else {
01016         if (ldir>0) {
01017             A1.PermuteLastTo(ic1);
01018             A2.contract(2, Psi2, ic2+1, AX);
01019             AX.permuteFirstTo(ic2, A2);
01020         }
01021         else {
01022             A2.contract(1, Psi1, ic1+1, AX);
01023             A1.permuteLastTo(ic2, A2);
01024             AX.permuteFirstTo(ic1,A1);
01025         }
01026     }
01027 }
01028
01029 template <class TQ, class TD>
01030 mxArray* orthoQS(
01031     const QSpace<TQ,TD> &PSI,
01032     QSpace<TQ,TD> &A1,
01033     QSpace<TQ,TD> &A2,
01034     unsigned K,
01035     unsigned Nkmin,
01036     unsigned Nkeep,
01037     double stol,
01038     itag_ tx,
01039     const char *info
01040 ){
01041     mxArray *Sout=NULL;
01042     unsigned i=0;
01043     char vflag=0, sflag=0;
01044
01045     SVD_Data<TQ,TD> SVD;
01046     wbMatrix<double> SV;
01047
01048     if (info) {
01049         for (; ++i) {
01050             if (info[i]=='d') { vflag|=1; } else

```

```

01051         if (info[i]=='D') { vflag|=2; } else
01052         if (info[i]=='s') { sflag|=1; } else
01053         if (info[i]=='S') { sflag|=2; } else break;
01054     }
01055     if (i>3 || info[i])
01056         wblog(FL,"ERR %s() invalid flags '%s' (i=%d)",FCT,info,i);
01057 }
01058
01059 if (sflag)
01060     Sout=mxCreateStructMatrix(1,1,0,NULL);
01061
01062     SVD.blockSVD(PSI,K);
01063
01064     i=SVD.dmrgrTruncate(Nkmin,Nkeep,stol,tx,SV, A1,NULL,A2, Sout,vflag);
01065
01066     if (Sout) {
01067         mxAddField2Scalar(FL,Sout,"K", numtoMx(K));
01068         mxAddField2Scalar(FL,Sout,"D1", A1 .getDim().toMx());
01069         mxAddField2Scalar(FL,Sout,"DD", PSI.getDim().toMx());
01070     }
01071     else if (i) wblog(FL,str);
01072
01073     if (sflag>1)
01074         mxAddField2Scalar(FL,Sout,"SVD", SVD.toMx());
01075     return Sout;
01076 };
01077
01078 template <class TQ, class TD>
01079 mxArray* getSVD(
01080     const QSpace<TQ,TD> &PSI,
01081     QSpace<TQ,TD> &U,
01082     QSpace<TQ,TD> &S,
01083     QSpace<TQ,TD> &V,
01084     unsigned K,
01085     unsigned Nkmin,
01086     unsigned Nkeep,
01087     double stol,
01088     itag_ tx,
01089     const char *info
01090 ){
01091     SVD_Data<TQ,TD> SVD;
01092     wbMatrix<double> SV;
01093
01094     mxArray *Sout=NULL;
01095     unsigned i=0; char sflag=0, vflag=0;
01096
01097     if (info) {
01098         for (; ++i) {
01099             if (info[i]=='d') { vflag|=1; } else
01100             if (info[i]=='D') { vflag|=2; } else
01101             if (info[i]=='s') { sflag|=1; } else
01102             if (info[i]=='S') { sflag|=2; } else break;
01103         }
01104         if (i>3 || info[i])
01105             wblog(FL,"ERR %s() invalid flags '%s' (i=%d)",FCT,info,i);
01106     }
01107
01108     if (sflag)
01109         Sout=mxCreateStructMatrix(1,1,0,NULL);
01110
01111     SVD.blockSVD(PSI,K);
01112
01113     i=SVD.dmrgrTruncate(Nkmin,Nkeep,stol,tx,SV, U,&S,V, Sout,vflag);
01114
01115     if (Sout) {
01116         mxAddField2Scalar(FL,Sout,"K", numtoMx(K));
01117         mxAddField2Scalar(FL,Sout,"D1", U.getDim().toMx());
01118         mxAddField2Scalar(FL,Sout,"DD", PSI.getDim().toMx());
01119     }
01120     else if (i) wblog(FL,str);
01121
01122     if (sflag>1)
01123         mxAddField2Scalar(FL,Sout,"SVD", SVD.toMx());
01124     return Sout;
01125 };
01126
01127 template <class TQ, class TD>
01128 mxArray* SVD_Data<TQ,TD>::toMx() const {
01129
01130     const char *fn[]{"Q1","Q2","S1","S2","U","S","Vc","A","I0","Ik"};
01131
01132     mxArray *S=mxCreateStructMatrix(QB.len,1,10,fn);
01133
01134     for (unsigned i=0; i<QB.len; ++i) {
01135         mxSetFieldByNumber(S,i,0, QB[i].Q1.toMx() );
01136         mxSetFieldByNumber(S,i,1, QB[i].Q2.toMx() );
01137         mxSetFieldByNumber(S,i,2, QB[i].S1.toMx() );

```

```

01138     mxSetFieldByNumber(S,i,3, QB[i].S2.toMx() );
01139     mxSetFieldByNumber(S,i,4, QB[i].U .toMx() );
01140     mxSetFieldByNumber(S,i,5, QB[i].S .toMx() );
01141     mxSetFieldByNumber(S,i,6, QB[i].Vc.toMx() );
01142     mxSetFieldByNumber(S,i,7, QB[i].A .toMx() );
01143     mxSetFieldByNumber(S,i,8, QB[i].I0.toMx() );
01144     mxSetFieldByNumber(S,i,9, QB[i].Ik.toMx() );
01145 }
01146
01147 return S;
01148 }
01149
01150 #endif

```

5.13 mpsortho.hh

```

00001 #ifndef __WB_ORTHO_HH__
00002 #define __WB_ORTHO_HH__
00003
00004 #define WB_STOL_SVD 1E-8
00005
00006 /* ----- */
00007 /* ----- */
00008 // change log
00009 // * split off mpsortho.hh from mpsortho.cc
00010 // * transferred former QSpace member functions
00011 //   toBlockMatrix() and initFromBlockMatrix()
00012 //   to class QBlock(), as it fits more naturally there.
00013 // Wb,Aug05,15
00014
00015 template <class TQ, class TD>
00016 mxArray* orthoQS(
00017     const QSpace<TQ,TD> &PSI,
00018     QSpace<TQ,TD> &A1,
00019     QSpace<TQ,TD> &A2, unsigned K,
00020     unsigned Nkmin, unsigned Nkeep,
00021     double stol,
00022     itag_ tx,
00023     const char *info
00024 );
00025
00026 template <class TQ, class TD>
00027 mxArray* getSVD(
00028     const QSpace<TQ,TD> &PSI,
00029     QSpace<TQ,TD> &U,
00030     QSpace<TQ,TD> &S,
00031     QSpace<TQ,TD> &VT, unsigned K,
00032     unsigned Nkmin, unsigned Nkeep, double stol,
00033     itag_ tx,
00034     const char *info
00035 );
00036
00037 char mxGetLRDir(
00038     const char *F, int L, const mxArray *argin, unsigned k
00039 );
00040
00041 template <class TQ, class T1, class T2, class T3>
00042 void twoSiteInit(
00043     const QSpace<TQ,T1> &Psi1,
00044     const QSpace<TQ,T2> &Psi2,
00045     QSpace<TQ,T3> &PSI, unsigned &K,
00046     widx_t ic1, widx_t ic2,
00047     char twoSite, char ldir
00048 );
00049
00050 template <class TQ, class T1, class T2, class T3>
00051 void twoSiteFinal(
00052     const QSpace<TQ,T1> &Psi1,
00053     const QSpace<TQ,T2> &Psi2,
00054     QSpace<TQ,T3> &A1,
00055     QSpace<TQ,T3> &A2,
00056     unsigned ic1, unsigned ic2, char twoSite, char ldir
00057 );
00058
00059 template <class TQ, class TD, class TE=double>
00060 class QBlock {
00061 public:
00062
00063     QBlock() : qdim_tot(1) {};
00064
00065     QBlock& init() { qdim_tot=1; qdim.init();
00066         I0.init(); Ik.init(); It.init(); idx.init();
00067         Q1.init(); S1.init(); D1.init(); A.init(); S.init();

```

```

00068     Q2.init(); S2.init(); D2.init(); U.init(); Vc.init();
00069     return *this;
00070 };
00071
00072 QBlock& operator=(const QBlock &B) {
00073     if (this!=&B) { qdim_tot=B.qdim_tot; qdim=B.qdim;
00074         IO=B.IO; Ik=B.Ik; idx=B.idx;
00075         Q1=B.Q1; S1=B.S1; D1=B.D1; A=B.A; S=B.S;
00076         Q2=B.Q2; S2=B.S2; D2=B.D2; U=B.U; Vc=B.Vc;
00077     }
00078     return *this;
00079 };
00080
00081 QBlock& init_bare_refA(const QBlock &B, unsigned w=0);
00082
00083 void toBlockMatrix(
00084     wbararray<TD> &MM,
00085     unsigned K,
00086     const wbpem &P=wbpem()
00087 );
00088
00089 void initFromBlockMatrix(const char *F, int L,
00090     QSpace<TQ,TD> &B,
00091     const wbararray<TD> &MM,
00092     const iTags &it,
00093     const wbpem *cgp=NULL
00094 ) const;
00095
00096 void updateBlockDim(unsigned dim2);
00097
00098 double get_s2t(const wvector<char> &mark) {
00099     double s2t=0; unsigned i=0;
00100     if (mark.len!=S.len) wblog(FL,"ERR %s() "
00101         "length mismatch (mark.len=%d/%d)",FCT,mark.len,S.len);
00102     mark.find(Ik);
00103     for (; i<mark.len; ++i) { if (!mark[i]) s2t+=S[i]*S[i]; }
00104     if (qdim_tot!=1) { s2t*=qdim_tot; }
00105     return s2t;
00106 };
00107
00108 void print(
00109     const char *F=NULL, int L=0, const iTags *t=NULL
00110 ) const {
00111
00112     wblog(F_L,"%s() QBlock content ===== %N",FCT);
00113     A.print("sub (PSI)"); if (t) {
00114         printf("=> itags: \"%s\\n\",STR_(t)); }
00115         printf("      qdim_tot=%d\\n\\n",qdim_tot);
00116         printf("      Q =[ %s; %s ]\\n",STR(Q1),STR(Q2));
00117         printf("      S =[ %s; %s ]\\n",STR(S1),STR(S2));
00118         printf("      D1=[ %s ]\\n",STR(D1));
00119         printf("      D2=[ %s ]\\n\\n",STR(D2));
00120     };
00121
00122     mxArray* toMx() const {
00123         const char *f[]={ "A","qdim_tot","qdim",
00124             "Q1","S1","D1", "Q2","S2","D2",
00125             "IO","Ik","idx","U", "S", "Vc"
00126         };
00127         unsigned i=0, n=1;
00128         mxArray *q=mxCreateStructMatrix(n,1,15,f);
00129
00130         mxSetFieldByNumber(q,i, 0, A .toMx());
00131         mxSetFieldByNumber(q,i, 1, numtoMx(double(qdim_tot)));
00132         mxSetFieldByNumber(q,i, 2, qdim.toMx());
00133         mxSetFieldByNumber(q,i, 3, Q1.toMx());
00134         mxSetFieldByNumber(q,i, 4, S1.toMx());
00135         mxSetFieldByNumber(q,i, 5, D1.toMx());
00136         mxSetFieldByNumber(q,i, 6, Q2.toMx());
00137         mxSetFieldByNumber(q,i, 7, S2.toMx());
00138         mxSetFieldByNumber(q,i, 8, D2.toMx());
00139         mxSetFieldByNumber(q,i, 9, IO.toMx());
00140         mxSetFieldByNumber(q,i,10, Ik.toMx());
00141         mxSetFieldByNumber(q,i,11, idx.toMx());
00142         mxSetFieldByNumber(q,i,12, U .toMx());
00143         mxSetFieldByNumber(q,i,13, S .toMx());
00144         mxSetFieldByNumber(q,i,14, Vc.toMx());
00145
00146         return q;
00147     };
00148
00149 void put(const char *vname, const char *ws="caller") {
00150     mxArray *a=toMx();
00151     mxPutAndDestroy(FL,a,vname,ws);
00152 };
00153
00154 unsigned qdim_tot;

```

```

00155     wbvector<unsigned> qdim;
00156
00157     QSpace<TQ,TD> A;
00158     wbindex IO;
00159     wbindex Ik;
00160     wbindex It;
00161
00162     wbMatrix<TQ> Q1, Q2;
00163     wbMatrix<widx_t> S1, S2;
00164     wbMatrix<widx_t> D1, D2;
00165
00166     wbMatrix<int> idx;
00167
00168     wbarray<TD> U,Vc;
00169     wbvector<TE> S;
00170
00171     protected:
00172     private:
00173 };
00174
00175 template <class TQ, class TD>
00176 class SVD_Data {
00177     public:
00178         SVD_Data() {};;
00179
00180         void blockSVD(const QSpace<TQ,TD> &PSI, const unsigned K);
00181
00182         unsigned dmrgTruncate(
00183             unsigned Nkmin, unsigned Nkeep, double stol,
00184             itag_ tx,
00185             wbMatrix<double> &SV,
00186             QSpace<TQ,TD> &UQ,
00187             QSpace<TQ,TD> *SQ,
00188             QSpace<TQ,TD> &VC,
00189             mxArray *Sout=NULL, char vflag=0
00190         );
00191
00192         mxArray* toMx() const;
00193
00194         void put(const char *vname, const char *ws="base") const {
00195             put(0,0,vname,ws);
00196         };
00197
00198         void put(const char *F, int L,
00199             const char *vname, const char *ws="base"
00200         ) const {
00201             mxArray *a=toMx();
00202             int i=mexPutVariable(ws,vname,a);
00203             if (i) wblog(F,L,"ERR failed to write variable '%s' (%d)",vname,i);
00204             if (F) wblog(F,L,"I/O putting variable '%s' to '%s'",vname,ws);
00205             mxDestroyArray(a);
00206         };
00207
00208         wbvector< QBlock<TQ,TD,double> > QB;
00209
00210     protected:
00211     private:
00212 };
00213
00214 #endif
00215

```

5.14 mxlib.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace MEX routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019

```

```

00020 #ifndef __WB_MXLIB_HH__
00021 #define __WB_MXLIB_HH__
00022
00023 /* ----- */
00024 // switching to matlab 2018a => got interleaved complex format
00025 // => phasing out mxGetPr_17(FL,) and mxGetPi_17(FL,)
00026
00027 #ifdef MATLAB_MEX_FILE
00028
00029     mxDouble* mxGetPr_17(const char *F, int L, const mxArray *pm) {
00030         wblog(F_L,"ERR mxGetPr() no longer supported in matlab>=2018");
00031         return 0;
00032     };
00033
00034     mxDouble* mxGetPi_17(const char *F, int L, const mxArray *pm) {
00035         wblog(F_L,"ERR mxGetPi() no longer supported in matlab>=2018");
00036         return 0;
00037     };
00038
00039 #endif
00040
00041 #ifndef NOMEX
00042 int isHelpIndicator(const char *s) {
00043     if (s && s[0]!='-') {
00044         if (!strcmp(s,"-?") || !strcmp(s,"-h") || !strcmp(s,"--help"))
00045             return 1;
00046     }
00047     return 0;
00048 };
00049 #endif
00050
00051 int isHelpIndicator(const mxArray *a) {
00052     char s[8];
00053     if (!a || mxGetString(a,s,8)) { return 0; }
00054     return isHelpIndicator(s);
00055 };
00056
00057 int checkHelpVersion(const mxArray *a0, mxArray **argout) {
00058
00059     char s[16];
00060     if (a0 && !mxGetString(a0,s,16)) {
00061         if (isHelpIndicator(s)) { usage(); return 1; }
00062
00063         if (!strcmp(s,"--ping")) { return 2; }
00064
00065         if (!strcmp(s,"--version")) { Wb::VersionInfo I;
00066             if (argout)
00067                 { argout[0]=I.toMx(); }
00068             else { I.print(); }
00069             return 1;
00070         }
00071     }
00072     return 0;
00073 };
00074
00075 inline mxArray* mx_get_field(mxArray *S, unsigned k, int i) {
00076     if (S && i>=0 && int(k)>=0) {
00077         mxArray *a=mxGetFieldByNumber(S,k,i);
00078         if (a && mxGetNumberOfElements(a)) return a;
00079     }
00080     return NULL;
00081 };
00082
00083 void mxPutArray(const char *file, int line,
00084     mxArray *a, const char *vname="ans", const char* ws="base",
00085     const char* dstr="");
00086
00087 void mxPutAndDestroy(const char *file, int line,
00088     mxArray *a, const char *vname="ans", const char* ws="base");
00089
00090 namespace Mx {
00091
00092 int IsEqual(const mxArray *a, const char *s) {
00093
00094     int n=strlen(s)+1;
00095     char sx[n+1]; sx[0]=0;
00096
00097     if (mxGetString(a,sx,n)) { return 0; }
00098     else { return (strcmp(sx,s) ? 0 : 1); }
00099 };
00100
00101 template <class T>
00102 class Array {
00103
00104 public:
00105
00106     Array() : ax(0), data(0),

```



```

00107     len(0), rank(0), trans(0), cmplx(0), mxref(0) { };
00108
00109     Array(unsigned ndim, size_t *dims, char t_=0, char c_=-1)
00110     : ax(0), data(0), len(0), rank(ndim), trans(t_), cmplx(c_), mxref(0) {
00111         if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00112         init(ndim,dims);
00113     };
00114
00115     Array(size_t d1, size_t d2, char t_=0, char c_=-1)
00116     : ax(0), data(0), len(0), rank(2), trans(t_), cmplx(c_), mxref(0) {
00117         size_t dims[2]={d1,d2}; if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00118         init(2,dims);
00119     };
00120
00121     Array(const wvector<size_t> &S, char t_=0, char c_=-1);
00122
00123     Array(const mxArray* a, char t_=0, char c_=-1)
00124     : ax(0), data(0), len(0), rank(0), trans(t_), cmplx(c_), mxref(1) {
00125         if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00126         init(a);
00127     };
00128
00129     Array(const char *F, int L, const mxArray* a, char t_=0, char c_=-1)
00130     : ax(0), data(0), len(0), rank(0), trans(t_), cmplx(c_), mxref(1)
00131     { if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00132
00133         try { init(a); }
00134         catch (...) { wblog(F_L,"ERR %s() ",FCT); }
00135
00136         if (!ax || (!mxref && !data)) wblog(F_L,
00137             "ERR %s() invalid input\n%s %s",FCT,SHORT_FL,STR_(this));
00138     };
00139
00140     wstring toStr(char vflag=0, unsigned l=64) const;
00141     wstring toStrT(char vflag=0) const;
00142
00143     void init_xstr() {
00144         sprintf(str,"Mx::Array<%s>",sTSTR(T));
00145         xstr=str;
00146     };
00147
00148     const char *xStr() const {
00149         return xstr.c_str();
00150     };
00151
00152     ~Array() {
00153         if ((ax || data) && mxref<=0) wblog(FL,
00154             "WRN %s() got ax=%p, data=%p !?",FCT,ax,data);
00155     }
00156
00157     mxArray* Return(T*& dd) { dd=data; return Return(); };
00158
00159     mxArray* Return() {
00160         mxArray *a=ax;
00161         ax=NULL; data=NULL; len=0;
00162         mxref=(mxref>0? 1:0);
00163         return a;
00164     };
00165
00166     Array& init(unsigned ndim, size_t *dims, char t_=-1, char c_=-1);
00167     Array& init(const mxArray *a, char t_=-1, char c_=-1);
00168
00169     template <class Tb>
00170     mxArray* copyFromTR(const Tb* b, char tcheck=0) {
00171
00172         if (!trans && typeid(T)==typeid(Tb) && data && b)
00173             { memcpy(data,b,len*sizeof(T)); }
00174         else { copy_from(b,tcheck); }
00175
00176         return Return();
00177     };
00178
00179     template <class Tb>
00180     int copy_from(const Tb* b, char tcheck=0);
00181
00182     template <class Tb>
00183     size_t copyTo(Tb* b, char tcheck=0) const {
00184         if (!trans && data && b) {
00185             if (mxref<=8) wblog(FL,"WRN %s() ref=%d !?",FCT,mxref);
00186             memcpy(b,data,len*sizeof(T));
00187         }
00188         else { copy_to(b,NULL,tcheck); }
00189         return len;
00190     };
00191
00192     size_t copy_to(T* b, const wperm *P=NULL, char tcheck=0) const;
00193

```

```

00194     size_t rcopy_to(T* b, const wbperm *P=NULL, char tcheck=0) const;
00195     size_t zcopy_to(T* b, const wbperm *P=NULL, char tcheck=0) const;
00196
00197     template <class Tx>
00198     size_t cpy_to_(T* b, const wbperm *P, char tcheck, const Tx *ad_) const;
00199
00200     size_t ncopy_to(T* b, size_t n, char tcheck=0) {
00201         size_t l_=len; if (len>n) len=n;
00202         n=copy_to(b,NULL,tcheck); len=l_;
00203         return n;
00204     };
00205
00206     mxArray *ax;
00207     T *data;
00208     size_t len;
00209     unsigned rank;
00210
00211     bool trans;
00212     char cmplx;
00213     char mxref;
00214
00215     std::string xstr;
00216
00217 protected:
00218 private:
00219
00220     void issue_type_error(
00221         const char *F, int L, unsigned ndim=0, const size_t *dims=NULL);
00222
00223     int init_dptra();
00224
00225     int safe_set_dptra(T *md) { data=md; return 0; }
00226
00227     template <class TM>
00228     int safe_set_dptra(TM *md __attribute__ ((unused))) { return 1; }
00229
00230     void MX_INIT_DATA(unsigned ndim, const size_t *dims);
00231 };
00232
00233 template <> template <>
00234 int Mx::Array<wbcomplex>::safe_set_dptra(mxComplexDouble *md) {
00235     data=(wbcomplex*)md; return 0;
00236 };
00237
00238 template <> template <>
00239 int Mx::Array<char>::safe_set_dptra(int8_T *md) {
00240     unsigned m=sizeof(char), n=sizeof(int8_T);
00241     if (m!=n) wblog(FL,"ERR %s() size mismatch (char: %d/%d)",FCT,m,n);
00242     data=(char*)md; return 0;
00243 };
00244
00245 template <> template <>
00246 int Mx::Array<size_t>::safe_set_dptra(uint64_T *md) {
00247     unsigned m=sizeof(size_t), n=sizeof(uint64_T);
00248     if (m!=n) wblog(FL,"ERR %s() size mismatch (size_t: %d/%d)",FCT,m,n);
00249     data=(size_t*)md; return 0;
00250 };
00251
00252 template <> template <>
00253 int Mx::Array<unsigned long long>::safe_set_dptra(uint64_T *md) {
00254     unsigned m=sizeof(size_t), n=sizeof(uint64_T);
00255     if (m!=n) wblog(FL,"ERR %s() size mismatch (size_t: %d/%d)",FCT,m,n);
00256     data=(unsigned long long*)md; return 0;
00257 };
00258
00259 template <> template <>
00260 int Mx::Array<long long>::safe_set_dptra(int64_T *md) {
00261     unsigned m=sizeof(long long), n=sizeof(int64_T);
00262     if (m!=n) wblog(FL,"ERR %s() size mismatch (long: %d/%d)",FCT,m,n);
00263     data=(long long*)md; return 0;
00264 };
00265
00266 template <>
00267 size_t Mx::Array<double>::copy_to(double * b, const wbperm *P, char tcheck)
00268 const { return rcopy_to(b,P,tcheck); };
00269
00270 template <>
00271 size_t Mx::Array<int>::copy_to(int * b, const wbperm *P, char tcheck)
00272 const { return rcopy_to(b,P,tcheck); };
00273
00274 template <>
00275 size_t Mx::Array<unsigned>::copy_to(unsigned* b, const wbperm *P, char tcheck)
00276 const { return rcopy_to(b,P,tcheck); };
00277
00278 template <>
00279 size_t Mx::Array<size_t>::copy_to(size_t * b, const wbperm *P, char tcheck)
00280 const { return rcopy_to(b,P,tcheck); };

```

```

00281
00282 template <>
00283 size_t Mx::Array<char>::copy_to(char* b, const wbpem *P, char tcheck)
00284 const { return rcopy_to(b,P,tcheck); };
00285
00286 template <>
00287 void Mx::Array<double>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00288     data=0;
00289     #pragma omp critical (using_MEX_API)
00290     { ax=mxCreateNumericArray(ndim,dims, mxDOUBLE_CLASS, mxREAL);
00291       if (ax) { safe_set_dptr(mxGetDoubles(ax)); }
00292     }; if (!data) issue_type_error(FL,ndim,dims);
00293 };
00294
00295 template <>
00296 void Mx::Array<wbcomplex>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00297     data=0;
00298     #pragma omp critical (using_MEX_API)
00299     { ax=mxCreateNumericArray(ndim,dims, mxDOUBLE_CLASS, mxCOMPLEX);
00300       if (ax) { safe_set_dptr(mxGetComplexDoubles(ax)); }
00301     }; if (!data) issue_type_error(FL,ndim,dims);
00302 };
00303
00304 template <>
00305 void Mx::Array<char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00306     data=0;
00307     #pragma omp critical (using_MEX_API)
00308     { ax=mxCreateNumericArray(ndim,dims, mxINT8_CLASS, mxREAL);
00309       if (ax) { safe_set_dptr(mxGetInt8s(ax)); }
00310     }; if (!data) issue_type_error(FL,ndim,dims);
00311 };
00312
00313 template <>
00314 void Mx::Array<signed char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00315     data=0;
00316     #pragma omp critical (using_MEX_API)
00317     { ax=mxCreateNumericArray(ndim,dims, mxINT8_CLASS, mxREAL);
00318       if (ax) { safe_set_dptr(mxGetInt8s(ax)); }
00319     }; if (!data) issue_type_error(FL,ndim,dims);
00320 };
00321
00322 template <>
00323 void Mx::Array<unsigned char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00324     data=0;
00325     #pragma omp critical (using_MEX_API)
00326     { ax=mxCreateNumericArray(ndim,dims, mxUINT8_CLASS, mxREAL);
00327       if (ax) { safe_set_dptr(mxGetUInt8s(ax)); }
00328     }; if (!data) issue_type_error(FL,ndim,dims);
00329 };
00330
00331 template <>
00332 void Mx::Array<mxChar>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00333     data=0;
00334     #pragma omp critical (using_MEX_API)
00335     { ax=mxCreateCharArray(ndim,dims);
00336       if (ax) { safe_set_dptr(mxGetChars(ax)); }
00337     }; if (!data) issue_type_error(FL,ndim,dims);
00338 };
00339
00340 template <>
00341 void Mx::Array<int>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00342     data=0;
00343     #pragma omp critical (using_MEX_API)
00344     { ax=mxCreateNumericArray(ndim,dims, mxINT32_CLASS, mxREAL);
00345       if (ax) { safe_set_dptr(mxGetInt32s(ax)); }
00346     }; if (!data) issue_type_error(FL,ndim,dims);
00347 };
00348
00349 template <>
00350 void Mx::Array<unsigned>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00351     data=0;
00352     #pragma omp critical (using_MEX_API)
00353     { ax=mxCreateNumericArray(ndim,dims, mxUINT32_CLASS, mxREAL);
00354       if (ax) { safe_set_dptr(mxGetUInt32s(ax)); }
00355     }; if (!data) issue_type_error(FL,ndim,dims);
00356 };
00357
00358 template <>
00359 void Mx::Array<long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00360     data=0;
00361     #pragma omp critical (using_MEX_API)
00362     { ax=mxCreateNumericArray(ndim,dims, mxINT64_CLASS, mxREAL);
00363       if (ax) { safe_set_dptr(mxGetInt64s(ax)); }
00364     }; if (!data) issue_type_error(FL,ndim,dims);
00365 };
00366
00367 template <>

```

```

00368 void Mx::Array<long long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00369     data=0;
00370     #pragma omp critical (using_MEX_API)
00371     { ax=mxCreateNumericArray(ndim,dims, mxINT64_CLASS, mxREAL);
00372       if (ax) { safe_set_dp_ptr(mxGetInt64s(ax)); }
00373     }; if (!data) issue_type_error(FL,ndim,dims);
00374 };
00375
00376 template <>
00377 void Mx::Array<size_t>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00378     data=0;
00379     #pragma omp critical (using_MEX_API)
00380     { ax=mxCreateNumericArray(ndim,dims, mxUINT64_CLASS, mxREAL);
00381       if (ax) { safe_set_dp_ptr(mxGetUInt64s(ax)); }
00382     }; if (!data) issue_type_error(FL,ndim,dims);
00383 };
00384
00385 template <>
00386 void Mx::Array<unsigned long long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00387     data=0;
00388     #pragma omp critical (using_MEX_API)
00389     { ax=mxCreateNumericArray(ndim,dims, mxUINT64_CLASS, mxREAL);
00390       if (ax) { safe_set_dp_ptr(mxGetUInt64s(ax)); }
00391     }; if (!data) issue_type_error(FL,ndim,dims);
00392 };
00393
00394 int IsNumArray(
00395     const char *F, int L,
00396     const mxArray* a, unsigned r=2, char cflag=0, char type='d'
00397 );
00398
00399 bool IsDblMat(const char *F, int L,
00400     const mxArray* a, char cflag=0, char dflag='d')
00401 { return Mx::IsNumArray(F,L,a,2,cflag,dflag)>0; }
00402
00403 bool IsDblArr(const char *F, int L,
00404     const mxArray* a, char cflag=0, char dflag='d')
00405 { return Mx::IsNumArray(F,L,a,-1,cflag,dflag)>0; }
00406
00407 bool IsDblVector(const char *F, int L,
00408     const mxArray *a, char cflag=0, char dflag='d')
00409 { return Mx::IsNumArray(F,L,a,1,cflag,dflag)>0; }
00410
00411 bool IsDblScalar(const char *F, int L,
00412     const mxArray *a, char cflag=0, char dflag='d') {
00413     if (Mx::IsNumArray(F,L,a,1,cflag,dflag)<=0) return 0;
00414     return mxGetNumberOfElements(a)==1;
00415 }
00416
00417 bool IsNumber(const char *F, int L,
00418     const mxArray *a, char cflag=0, char dflag='*') {
00419     if (Mx::IsNumArray(F,L,a,1,cflag,dflag)<=0) return 0;
00420     return (mxGetNumberOfElements(a)==1);
00421 }
00422
00423 bool IsScalar(const mxArray *a) {
00424     return (mxGetNumberOfDimensions(a)==2 && mxGetNumberOfElements(a)==1);
00425 };
00426
00427 int IsVector(const mxArray *a) {
00428     int q=0;
00429     if (mxGetNumberOfDimensions(a)==2) {
00430         const size_t *s=mxGetDimensions(a);
00431         if (s[0]==1) { q=1; }
00432         else if (s[1]==1) { q=2; }
00433     }
00434     return q;
00435 };
00436
00437 bool IsIndex(const mxArray *a, int base=0);
00438
00439 template <class T>
00440 bool IsIndex(const T *x, size_t n, int base=0) {
00441     if (!WbUtil<T>().isPOD()) wblog(FL,
00442         "ERR %s() invalid data type `%s'",FCT,TSTR(T));
00443
00444     if (WbUtil<T>().isInt()) {
00445         for (size_t i=0; i<n; ++i) { if (x[i]<base) return 0; }
00446     }
00447     else {
00448         for (size_t i=0; i<n; ++i) {
00449             if (x[i]<base || x[i]!=round(x[i])) return 0; }
00450     }
00451     return 1;
00452 };
00453
00454 };

```

```

00455
00456 namespace Wb {
00457     int CallMatlab(
00458         int nargout, mxArray *argout[], int nargin, mxArray *argin[],
00459         const char *fctname);
00460
00461     double CallMatlab(
00462         const char *F, int L, const char* cmd, const char *arg1=NULL);
00463
00464     double CallMatlab(const char* cmd, const char *arg1=NULL) {
00465         return CallMatlab(0,0,cmd,arg1);
00466     };
00467 };
00468
00469 #ifndef MATLAB_MEX_FILE
00470
00471 #undef mexPutVariable
00472 #define mexPutVariable mexPutVariable_wbx
00473 int mexPutVariable_wbx(const char *ws, const char *vname, const mxArray *a){
00474     wblog(FL,"WRN mexPutVariable(%s,%lX,%s) not available outside MatLab",
00475     vname?vname:"",a,ws?ws:""); return 1;
00476 };
00477
00478 #undef mexCallMATLAB
00479 #define mexCallMATLAB mexCallMATLAB_wbx
00480 int mexCallMATLAB_wbx(
00481     int nargout, mxArray **argout, int nargin, mxArray **argin,
00482     const char *cmd
00483 ){ wblog(FL,"ERR mexCallMATLAB(%d,%lX,%d,%lX,%s) not available outside MatLab",
00484     nargin,argin,nargout,argout,cmd?cmd:""); return 1;
00485 };
00486
00487 #undef mexGetVariable
00488 #define mexGetVariable mexGetVariable_wbx
00489 int mexGetVariable_wbx(const char *ws, const char *vname) {
00490     wblog(FL,"WRN mexGetVariable(%s,%s) not available outside MatLab",
00491     vname?vname:"",ws?ws:""); return 1;
00492 };
00493
00494 #undef mexGetVariablePtr
00495 #define mexGetVariablePtr mexGetVariablePtr_wbx
00496 mxArray* mexGetVariablePtr_wbx(const char *ws, const char *vname) {
00497     wblog(FL,"WRN mexGetVariablePtr(%s,%s) not available outside MatLab",
00498     vname?vname:"",ws?ws:""); return NULL;
00499 };
00500
00501 #undef mexPrintf
00502 #define mexPrintf mexPrintf_wbx
00503 int mexPrintf_wbx(const char *fmt, ...){
00504     wblog(FL,"WRN mexPrintf(%s,...) not available outside MatLab",
00505     fmt?fmt:""); return 1;
00506 };
00507
00508 #undef mexFunctionName
00509 #define mexFunctionName mexFunctionName_wbx
00510
00511 const char* mex_function_name="(program)";
00512
00513 const char* mexFunctionName_wbx(const char *fmt, ...){
00514     wblog(FL,"WRN mexFunctionNamef(%s,...) not available outside MatLab",
00515     fmt?fmt:""); return mex_function_name;
00516 };
00517
00518 const char* mexFunctionName_wbx(){
00519     wblog(FL,"WRN mexFunctionNamef() not available outside MatLab");
00520     return mex_function_name;
00521 };
00522
00523 #endif
00524
00525 #endif
00526

```

5.15 nomex.hh

```

00001 /* ===== */
00002 /* dummy version of bracketing out MEX library if
00003     (i) it is not available and
00004     (ii) also not required in the code per se!
00005     in case any of the mex-routines is required
00006     an error message issued and the program terminates.
00007
00008     NB! return types taken from Matlab R2013a documentation

```

```

00009     Wb,Apr02,13 */
00010  /* ===== */
00011
00012 typedef char mxArray;
00013
00014 // taken from Matlab R2013a documentation
00015 enum mxComplexity {mxREAL=0, mxCOMPLEX};
00016 // #define mxREAL 0
00017 // #define mxCOMPLEX 1
00018
00019 #define mxDOUBLE_CLASS 10
00020
00021 #define MATFile FILE
00022 #define mxChar short
00023 #define mwSize size_t
00024 #define mwIndex int
00025
00026 #define ERRMSG_NOMEX wblog(__FILE__, __LINE__, "ERR NOMEX (%s)", __FUNCTION__)
00027
00028 int wblog(const char* file, int line, const char *fmt, ...);
00029
00030 int mexCallMATLAB(
00031     unsigned nout, mxArray** aout,
00032     unsigned nin, mxArray** ain, const char* fstr)
00033 { ERRMSG_NOMEX; return 0; };
00034
00035 int mxIsChar (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00036 int mxIsDouble (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00037 int mxIsComplex (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00038 int mxIsCell (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00039 int mxIsStruct (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00040 int mxIsSparse (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00041
00042 int mxIsEmpty (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00043 size_t mxGetM (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00044 size_t mxGetN (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00045
00046 int mxGetNzmax (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00047 int* mxGetIr (const mxArray* a) { ERRMSG_NOMEX; return (int*)0; };
00048 int* mxGetJc (const mxArray* a) { ERRMSG_NOMEX; return (int*)0; };
00049
00050 int mxDestroyArray (mxArray* a) { ERRMSG_NOMEX; return 0; };
00051
00052 void mexErrMsgTxt(const char* estr) {
00053     printf("\nmexErrMsgTxt: %s\nExit.\n\n", estr);
00054     exit(66);
00055 };
00056
00057 char* mxGetClassName(const mxArray* a) {
00058     static char ostr[]="(none)";
00059     ERRMSG_NOMEX; return ostr;
00060 };
00061
00062 size_t mxGetNumberOfElements(const mxArray* a) {
00063     ERRMSG_NOMEX; return 0;
00064 };
00065
00066 int mxGetNumberOfFields(const mxArray* a) {
00067     ERRMSG_NOMEX; return 0;
00068 };
00069
00070 unsigned mxGetNumberOfDimensions(const mxArray* a) {
00071     ERRMSG_NOMEX; return 0;
00072 };
00073
00074 const size_t* mxGetDimensions(const mxArray* a) {
00075     ERRMSG_NOMEX; return (const size_t*)NULL;
00076 };
00077
00078 mxArray* mxGetFieldByNumber(const mxArray *a, unsigned i, unsigned j) {
00079     ERRMSG_NOMEX; return (mxArray*)NULL;
00080 };
00081
00082 const char* mxGetFieldNameByNumber(const mxArray *a, int fid) {
00083     ERRMSG_NOMEX; return (const char*)NULL;
00084 };
00085
00086 int mxGetFieldNumber(const mxArray *a, const char *fld) {
00087     ERRMSG_NOMEX; return 0;
00088 };
00089
00090 int mxSetFieldByNumber(mxArray* S, unsigned i, unsigned j, mxArray* a) {
00091     ERRMSG_NOMEX; return 0;
00092 };
00093
00094 mxArray* mxGetField(const mxArray *a, mwIndex idx, const char *fld) {
00095     ERRMSG_NOMEX; return (mxArray*)NULL;

```

```

00096 };
00097
00098 int mxAddField(const mxArray* a, const char *fld) {
00099     ERRMSG_NOMEX; return 0;
00100 };
00101
00102 int mxGetString(const mxArray* a, char *ostr, int len) {
00103     ERRMSG_NOMEX; return 0;
00104 };
00105
00106 mxArray* mxCreateString(const char *s) {
00107     ERRMSG_NOMEX; return (mxArray*)NULL;
00108 };
00109
00110 mxArray* mxCreateDoubleMatrix(size_t d1, size_t d2, char mxreal) {
00111     ERRMSG_NOMEX; return (mxArray*)NULL;
00112 };
00113
00114 mxArray* mxCreateNumericArray(int d, size_t *dims, int cflag, int rflag) {
00115     ERRMSG_NOMEX; return (mxArray*)NULL;
00116 };
00117
00118 mxArray* mxCreateSparse(size_t m, size_t n, size_t nzmax, mxComplexity cflag) {
00119     ERRMSG_NOMEX; return (mxArray*)NULL;
00120 };
00121
00122 mxArray* mxCreateCharArray(int d, size_t *dims) {
00123     ERRMSG_NOMEX; return (mxArray*)NULL;
00124 };
00125
00126 mxArray* mxCreateStructMatrix(size_t d1, size_t d2, int nf, const char** fnames) {
00127     ERRMSG_NOMEX; return (mxArray*)NULL;
00128 };
00129
00130 mxArray* mxCreateCellMatrix(size_t d1, size_t d2) {
00131     ERRMSG_NOMEX; return (mxArray*)NULL;
00132 };
00133
00134 mxArray* mxGetCell(const mxArray* S, unsigned idx) {
00135     ERRMSG_NOMEX; return (mxArray*)NULL;
00136 };
00137
00138 int mxSetCell(mxArray* S, unsigned idx, mxArray* b) {
00139     ERRMSG_NOMEX; return 0;
00140 };
00141
00142 int mexPutVariable(const char *ws, const char *vname, mxArray *a) {
00143     ERRMSG_NOMEX; return 0;
00144 };
00145
00146 int matPutVariable(MATFile *mfp, const char *vname, const mxArray *a) {
00147     ERRMSG_NOMEX; return 0;
00148 };
00149
00150 mxArray* matGetVariable(MATFile *mfp, const char *vname) {
00151     ERRMSG_NOMEX; return (mxArray*)NULL;
00152 };
00153
00154 mxArray* matGetVariableInfo(MATFile *mfp, const char *vname) {
00155     ERRMSG_NOMEX; return (mxArray*)NULL;
00156 };
00157
00158 int matDeleteVariable(MATFile *mfp, const char *name) {
00159     ERRMSG_NOMEX; return 0;
00160 };
00161
00162 MATFile* matOpen(const char *file, const char *mode) {
00163     ERRMSG_NOMEX; return 0;
00164 };
00165
00166 int matClose(MATFile *mfp) {
00167     ERRMSG_NOMEX; return 0;
00168 };
00169

```

5.16 nrgdata.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace NRG routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *

```

```

00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_NRGDATA_HCC__
00021 #define __WB_NRGDATA_HCC__
00022
00023 /* ----- */
00024 // change log
00025
00026 Wb, Feb22, 19: changed internally K, T labels to K, D labels,
00027 besides that T is also temperature, TK is Kondo temperature
00028 TD is a template data type name, etc.
00029
00030 // ----- */
00031
00032 const double DEPS2=1E-6*DBL_EPSILON;
00033
00034 template <class TQ, class TD>
00035 class NRGData;
00036
00037 #define NRG_FLEN 128
00038 const char NRG_EMPTY_STR[]="(NRG) ";
00039
00040 unsigned NRG_N, NRG_ITER;
00041 char NRG_FILE[NRG_FLEN];
00042
00043 unsigned STRICT_ITER0=1;
00044
00045 Wb::ClockSet fdmClocks;
00046
00047 wbvector<double> gES;
00048
00049 inline double gEScale(int iter) {
00050     if (iter<0 || iter>=(int)gES.len) wblog(FL,
00051         "ERR %s() index out of bounds (%d/%d)", FCT, iter, gES.len);
00052     return gES[iter];
00053 };
00054
00055 void wblog_ERR_updateOp(
00056     const char *F, int L, const char *istr, int i, int nop, int iter) {
00057
00058     wblog(F_L,
00059         "ERR failed to update operator (%s-op %d/%d @ iter=%d)",
00060         istr && istr[0] ? istr : "(fdm)", i+1, nop, iter
00061     );
00062 }
00063
00064 template <class TQ, class TD, class TZ>
00065 void WeightE0_dim1(QSpace<TQ, TZ> &X, const QSpace<TQ, TD> &H, double w);
00066
00067 double nrgGetE0(
00068     const wbvector<double> &dE,
00069     const wbvector<double> &ES,
00070     wbvector<double> &E0
00071 ){
00072     if (E0.len!=dE.len) { E0.init(dE.len); }
00073     if (!E0.len) { wblog(FL, "WRN %s() got empty E0 !?", FCT); return 0; }
00074     if (dE.len!=ES.len) wblog(FL,
00075         "ERR %s() length mismatch (%d/%d)", FCT, dE.len, ES.len);
00076
00077     unsigned k=dE.len-1;
00078     double x=0, xl=dE[k]*ES[k]; E0[k]=0;
00079
00080     for (; k; --k) {
00081         x=dE[k-1]*ES[k-1];
00082         E0[k-1]=E0[k]-xl;
00083         xl=x;
00084     }
00085
00086     return x-E0[0];
00087 };
00088
00089 template <class TQ, class TD>
00090 int getSpecDimOps(
00091     const wbvector<QSpace<TQ, TD> > &FF, wbvector<double> &fac
00092 ){
00093     unsigned i=0, r,m, n=FF.len, gotfac=0; widxt nq;

```



```

00094     fac.init2val(n,1);
00095
00096     for (; i<n; ++i) {
00097         r=FF[i].rank(FL); if (r<=2) continue;
00098         if (r>3) wblog(FL,"ERR %s() invalid rank-%g !?",FCT,r);
00099
00100         m=FF[i].getDim(2,&nq);
00101         if (int(nq)<1) wblog(FL,"ERR %s() %d/%d !?",FCT,m,nq);
00102         if (nq>1) { fac[i]=nq; ++gotfac; }
00103     }
00104     return gotfac;
00105 };
00106
00107 class RIXS_para {
00108
00109 public:
00110
00111     RIXS_para()
00112         : Eoffset(0.), sigma(0.), afac(-1.), wrxs(0),
00113           E0(0.), E2(0.), dloc(0), iter(0) {};
00114
00115     void init() {
00116         Einc=0; Eoffset=sigma=E0=E2=0; wrxs=0; dloc=iter=0;
00117         afac=-1;
00118     };
00119
00120     void setE0(int it, double e0, double e2) {
00121         iter=it; E0=e0+Eoffset; E2=e2;
00122     }
00123
00124     void check(const char *F=0, int L=0, char init=0);
00125
00126     Wb::String toStr() const;
00127     mxArray* toMx() const;
00128
00129     wbcomplex Einc;
00130     double Eoffset;
00131     double sigma;
00132     double afac;
00133     char wrxs;
00134
00135     double E0, E2;
00136     unsigned dloc;
00137     int iter;
00138
00139 private:
00140 };
00141
00142     RIXS_para gRX;
00143
00144 void RIXS_para::check(const char *F, int L, char init) {
00145
00146     if (Einc.i<=0) {
00147         if (!Einc.i && init) {
00148             Einc.i=gES.last();
00149         }
00150         else wblog(F_L,
00151             "ERR %s() got imag(Einc)=%g < 0 (RIXS) !?",FCT,Einc.i
00152         );
00153     }
00154     else if (F && init) {
00155         if (Einc.i<1E-2*Einc.r || Einc.i>1E2*Einc.r) wblog(FL,
00156             "WRN %s() got Einc.i / Einc_r = %.3g / %.3g = %.3g",
00157             PROG, Einc.i, Einc.r, Einc.i/Einc.r
00158         );
00159     }
00160
00161     if (sigma<0) wblog(F_L,
00162         "ERR RIXS::%s() invalid sigma=%g<0",FCT,sigma);
00163     if (int(dloc)<=1 || dloc>(1<10)) wblog(F_L,
00164         "ERR RIXS::%s() invalid dloc=%d",FCT,dloc);
00165
00166     if (init) {
00167         if (!sigma) {
00168             if (Einc.r) { sigma = Einc.i/Einc.r; }
00169             else wblog(FL,"ERR %s() got bot sigma and real(Einc) zero !?",FCT);
00170         }
00171         else if (sigma<1E-3 && F) {
00172             wblog(FL,"WRN %s() got sigma = %g",PROG,sigma);
00173         }
00174     }
00175
00176 };
00177
00178 Wb::String RIXS_para::toStr() const {
00179
00180     Wb::String s(64);

```

```

00181     s.pushf("Einc = %.3g",Einc.r);
00182
00183     if (Einc.r) {
00184         double x=Einc.i/Einc.r;
00185         if (x && fabs((x-sigma)/x)<1E-4)
00186             s.pushf(" @ sigma=%.3g",sigma);
00187         else s.pushf(" @ %.3gi, sigma=%g",x,sigma);
00188     }
00189     else { s.pushf(" + %.3gi, sigma=%g",Einc.i,sigma); }
00190
00191     if (wrxs ) { s.pushf(" , mode=%d",wrxs); }
00192     if (afac>=0) { s.pushf(" , afac=%g",afac); }
00193
00194     return s;
00195 };
00196
00197 mxArray* RIXS_para::toMx() const {
00198     const char* fields[]={
00199         "Einc","Eoffset","sigma","wrxs","dloc";
00200
00201     mxArray* a=mxCreateStructMatrix(1,1,5,fields);
00202     mxSetFieldByNumber(a,0,0, numtoMx(Einc ));
00203     mxSetFieldByNumber(a,0,1, numtoMx(Eoffset));
00204     mxSetFieldByNumber(a,0,2, numtoMx(sigma));
00205     mxSetFieldByNumber(a,0,3, numtoMx(wrxs ));
00206     mxSetFieldByNumber(a,0,4, numtoMx(dloc ));
00207
00208     return a;
00209 };
00210
00211 template <class TQ, class TD>
00212 class NRGIndex {
00213
00214     public:
00215
00216         NRGIndex() : idx(-1), nrg(NULL) {};
00217         NRGIndex(const NRGIndex &I) : idx(I.idx), nrg(I.nrg), tag(I.tag) {};
00218
00219         void init() { idx=-1; nrg=NULL; };
00220         void init(int i, NRGData<TQ,TD> *d) { idx=i; nrg=d; }
00221         void init(int i, const char *t) { idx=i; tag=t; }
00222
00223         mxArray* toMx() const;
00224         mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00225         void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
00226
00227         void put (const char *vname, const char *ws="base") const {
00228             mxArray *S=toMx();
00229             mxPutAndDestroy(FL,S,vname,ws);
00230         };
00231
00232         int idx;
00233         NRGData<TQ,TD> *nrg;
00234         wstring tag;
00235
00236     private:
00237 };
00238
00239 template <class TQ, class TD>
00240 mxArray* NRGIndex<TQ,TD>::mxCreateStruct(unsigned m, unsigned n) const {
00241     const char *fields[] = {"idx","nrg","tag"};
00242     return mxCreateStructMatrix(m,n,3,fields);
00243 };
00244
00245 template <class TQ, class TD>
00246 void NRGIndex<TQ,TD>::add2MxStruct(
00247     mxArray *S, unsigned i, char tst __attribute__((unused))
00248 ) const {
00249
00250     mxSetFieldByNumber(S,i,0, numtoMx(idx));
00251     mxSetFieldByNumber(S,i,1, numtoMx((unsigned long) nrg));
00252     mxSetFieldByNumber(S,i,2, mxCreateString(tag.isEmpty() ? tag.data:""));
00253 };
00254
00255 namespace Wb {
00256
00257     template <class TQ, class TD>
00258     int updateOp(const char *F, int L,
00259         const NRGData<TQ,TD> &NRG,
00260         wvector< QSpace<TQ,TD> > &F12,
00261         const QSpace<TQ,TD> &A1,
00262         const QSpace<TQ,TD> &A2,
00263         const wvector< QSpace<TQ,TD> > &FKK,
00264         unsigned iter
00265     );
00266
00267     template<class TT>

```

```

00268 void initLocalOp(const char *F, int L,
00269     const mxArray* a, TT &C,
00270     int k=0, char force=0, char *name=NULL
00271 );
00272
00273 template<class TQ, class TD>
00274 inline void initLocalOp_aux(const char *F, int L,
00275     const mxArray* a, wbvector< QSpace<TQ,TD> > &C
00276 ){ mxInitQSpaceVec(F,L,a,C); }
00277
00278 template<class TQ, class TD>
00279 inline void initLocalOp_aux(const char *F, int L,
00280     const mxArray* a, wbMatrix< QSpace<TQ,TD> > &C
00281 ){ mxInitQSpaceMat(F,L,a,C); }
00282
00283 template<class TQ, class TD>
00284 inline void initLocalOp_aux(const char *F, int L,
00285     const mxArray* a, wbvector< wbvector< QSpace<TQ,TD> > > &C
00286 ){ mxInitQSpaceVecVec(F,L,a,C); }
00287 };
00288
00289 template <class TQ, class TD>
00290 class NRGData {
00291 public:
00292
00293     NRGData(const char *s=NULL)
00294     : name(s), calc(0), store(0), kloc(0), locMX(0), MX(NULL), aux(NULL) {};
00295
00296     NRGData(const NRGData &A)
00297     : name(A.name), nrgIdx(A.nrgIdx), calc(0), store(0), kloc(A.kloc),
00298       locMX(0), MX(NULL), aux(NULL)
00299     {
00300         wblog(FL,"ERR generating copy of NRGData !?");
00301     };
00302
00303 ~NRGData() {
00304     if (MX && !NAME.isEmpty() && !locMX) wblog(FL,
00305         "WRN ** data return '%s' not implemented yet **\n(%lX; %d)",
00306         NAME.data, MX, locMX);
00307     init();
00308 };
00309
00310 void initName(const char *n) { name=n; };
00311
00312 void init(const char *F, int L, const mxArray* a,
00313     wbvector< QSpace<TQ,TD> > &C0,
00314     int k=0, char force=0, const char *vname=0
00315 );
00316
00317 void init(const char *F, int L, const char *XX, int iter=-1);
00318
00319 void initXX(const char *F, int L, int m=4, int iter=-1) {
00320     if (m==1 || m==4) { init(F,L,"KK",iter); } else
00321     if (m==2 || m>4) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
00322     if (m>2) {
00323         init(F,L,"KD",iter);
00324         init(F,L,"DK",iter);
00325         init(F,L,"DD",iter);
00326     }
00327 };
00328
00329 void initX(const char *F, int L, int m=2, int iter=-1) {
00330     if (m==2) { init(F,L,"D",iter); }
00331     else if (m!=1) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
00332     init(F,L,"K",iter);
00333 };
00334
00335 void init(char flag=0) {
00336     if (!flag) { name.init(); NAME.init(); }
00337
00338     nrgIdx.init(); calc=0; store=0;
00339     if (aux) {
00340         mxDestroyArray(aux);
00341         aux=NULL;
00342     }
00343
00344     if (MX && locMX) { mxDestroyArray(MX); }
00345     MX=NULL; locMX=0;
00346
00347     KK.init(); DK.init(); K.clearQSpace();
00348     KD.init(); DD.init(); D.clearQSpace();
00349 };
00350
00351 void setupMX(
00352     const char *F, int L, int N, char store_=1, char locMX_=1
00353 ){
00354     if (MX || locMX) wblog(F_L,

```

```

00355         "ERR %s() already got MX=%p (%d,%d) !?",FCT,MX,locMX,store);
00356     MX=mxCreateStructMatrix(1,N,0,NULL);
00357     store=store_; locMX=locMX_;
00358 };
00359
00360 void PUT(
00361     const char *F, int L, const char *vn, const char *ws="caller"
00362 ){
00363     if (MX && locMX) {
00364         if (!vn || !vn[0]) wblog(FL,"ERR %s() invalid usage",FCT);
00365         int i=mexPutVariable(ws,vn,MX);
00366         if (!i) wblog(FL,"I/O putting %s to %s",vn,ws);
00367         else wblog(FL,"ERR %s() mexPutVariable() returned i=%d !?",FCT,i);
00368     }
00369     else wblog(FL,"WRN %s() failed having %p (%d)",FCT,MX,locMX);
00370 };
00371
00372 void check_dloc(
00373     const char *F, int L, const char *fun,
00374     unsigned dloc, unsigned iter) const;
00375
00376 template<class TD_>
00377 int checkOpConsistency(
00378     const char *F, int L, const NRGData<TQ,TD_> &B) const;
00379
00380 bool isQSpaceVec(const char *nm, int iter=-1);
00381
00382 bool checkVec(const char *F, int L,
00383     const char *XX, unsigned R, const char *istr="");
00384
00385 bool checkVecVec(const char *XX, unsigned r);
00386
00387 void forceCalc();
00388 void getRelevantOps(wbindex &I) const;
00389
00390 wbvector< QSpace<TQ,TD> >& getOpsXX(const char *t) {
00391     if (t && strlen(t)==2)
00392         switch (t[0]) {
00393             case 'K': return (t[1]=='K' ? KK : KD);
00394             case 'D': return (t[1]=='K' ? DK : DD);
00395         }
00396     wblog(FL,"ERR invalid tag `%s'", t ? t : "(null)");
00397     return KK;
00398 };
00399
00400 const QSpace<TQ,TD>& getQSpace(const char *t, unsigned i=0) const;
00401 QSpace<TQ,TD>& getQSpace(const char *t, unsigned i=0) {
00402     return const_cast<QSpace<TQ,TD>&>(
00403         static_cast<const NRGData<TQ,TD>*>(this)->getQSpace(t,i));
00404 }
00405
00406 const QSpace<TQ,TD>& getQSpace(char t1, char t2, unsigned i=0) const {
00407     char t[3]={t1,t2,0}; return getQSpace(t,i);
00408 };
00409 QSpace<TQ,TD>& getQSpace(char t1, char t2, unsigned i=0) {
00410     char t[3]={t1,t2,0}; return getQSpace(t,i);
00411 };
00412
00413 const QSpace<TQ,TD>& getQSpace(char t) const {
00414     switch (t) {
00415         case 'K': return K;
00416         case 'D': return D;
00417     }
00418     wblog(FL,"ERR invalid tag `%c' (%d)",t,t);
00419     return K;
00420 };
00421 QSpace<TQ,TD>& getQSpace(char t) {
00422     return const_cast<QSpace<TQ,TD>&>(
00423         static_cast<const NRGData<TQ,TD>*>(this)->getQSpace(t));
00424 };
00425
00426 void info(const char *vstr=NULL) const;
00427 void dispIdx(const char *F, int L) const;
00428 void info(const char *F, int L, const char *istr=" * ") const;
00429 void disp_otype(const char *F=0, int L=0) const;
00430
00431 const mxArray* getMxInfo(const char *n);
00432
00433 template <class TX>
00434 void getMxInfo(const char *F, int L, const char *v, TX &X) {
00435     const mxArray *a=getMxInfo(v);
00436     if (a) {
00437         X.init(F_L,a);
00438     }
00439     else wblog(FL,
00440         "ERR failed to read '%s' from %s_info.mat",v,wrg());
00441 };

```

```

00442
00443     mxArray* getMxData(
00444     const char *nm, int iter=-1, char ionly=0);
00445
00446     const char* wnrg() {
00447         if (NAME.data) return NAME.data;
00448         return (name.data ? name.data : NRG_EMPTY_STR);
00449     };
00450
00451     mxArray* toMx() const;
00452
00453     void put(const char *vname, const char* ws="base") {
00454         mxPutAndDestroy(FL, toMx(), vname);
00455     };
00456
00457     void put(const char *F, int L, const char *vname, const char* ws="base"
00458     ){ wblog(F,L,"I/O putting '%s' to %s", vname, ws);
00459         mxPutAndDestroy(FL, toMx(), vname);
00460     };
00461
00462     template<class T>
00463     int getPara(const char *name, T &x, int iter=-1, char i=0) {
00464 wblog(FL,"ERR %s() check!",FCT);
00465         mxArray *a=getMxData(iter);
00466         return mxGetNumber(a,x);
00467     }
00468
00469     bool opExists(char tflag=1);
00470
00471     void initOp(unsigned n=1, char calc_=1, char store_=0) {
00472         KK.initDef(n); DK.initDef(n); nrgIdx.initDef(n);
00473         KD.initDef(n); DD.initDef(n); calc=calc_; store=store_;
00474     };
00475
00476     bool initOp(const char *F, int L,
00477         const wvector< QSpace<TQ,TD> > &C0,
00478         char tflag=1,
00479         NRGData<TQ,TD>* CR=NULL,
00480         const char *istr="")
00481     );
00482
00483     bool initOp(const char *F, int L,
00484         const wvector< wvector< QSpace<TQ,TD> > > &CC,
00485         NRGData<TQ,TD>* CR=NULL,
00486         const char *istr="",
00487         char tflag=1
00488     );
00489
00490     void initSym(const char *tag);
00491
00492     void updatePara(
00493         const char *F, int L,
00494         const char *tag, mxArray *, int iter=-1
00495     ) const;
00496
00497     void updatePara(const char *F, int L,
00498         const char *tag, int iter=-1,
00499         const char *tag2=NULL, char all=-1
00500     ) const;
00501
00502     void updateOp(const char *F, int L,
00503         const NRGData<TQ,TD> &A,
00504         const NRGData<TQ,TD> &B, unsigned nop, int iter=-1
00505     );
00506
00507     template<class TC>
00508     void updateOpProd(
00509         const char *F, int L, unsigned iter,
00510         const NRGData<TQ,TD> &B,
00511         const NRGData<TQ,TC> &C
00512     );
00513
00514     void updateOp(const char *F, int L,
00515         const NRGData<TQ,TD> &A, unsigned nop, int iter=-1
00516     ) { updateOp(F,L,A,A,nop,iter); };
00517
00518     template <class TA>
00519     void backPropagateOp(const char *F, int L,
00520         const NRGData<TQ,TA> &A0,
00521         const NRGData<TQ,TA> &A2
00522     );
00523
00524     template<class TD_>
00525     void initBG(
00526         const char *F, int L,
00527         const NRGData<TQ,TD_> &B,
00528         const NRGData<TQ,TD_> &H1, const NRGData<TQ,TD_> &H2,

```

```

00529         int m=3
00530     );
00531
00532     void unsetAllRefs();
00533     unsigned skipOpZeros(double eps=DEPS, char bflag='b');
00534
00535     void saveOp(const char*, int, int iter=-1, unsigned m=4) const;
00536
00537     void times(TD fac, const char *flag);
00538
00539     void setupIO(const char*, int,
00540     const char*, const mxArray*, const char *istr=0);
00541
00542     void updateInfo(
00543         const char *F, int L,
00544         const char*, mxArray *, char keep=-1
00545     ) const;
00546
00547     void updateInfop(
00548         const char *F, int L,
00549         const char *vtag, mxArray *, char keep=-1
00550     ) const;
00551
00552     const char* getFileName(unsigned k, char lenient=0) const {
00553         if (!lenient && k>=NRG_N) wblog(FL,
00554         "ERR %s - index out of bounds (%d/%d)", __FUNCTION__, k, NRG_N);
00555         return getFileName_aux( k);
00556     };
00557
00558     wstring name;
00559
00560     wvector< NRGIndex<TQ,TD> > nrgIdx;
00561
00562     char calc, store;
00563     int kloc;
00564     char locMX;
00565
00566     wvector< wvector< QSpace<TQ,TD> > > CI;
00567
00568     wvector< QSpace<TQ,TD> > KK, KD, DK, DD;
00569
00570     QSpace<TQ,TD> K, D;
00571
00572     wvector<char> issym;
00573
00574     mxArray* MX;
00575     wstring NAME;
00576
00577     mxArray *aux;
00578
00579 private:
00580
00581     const char* getFileNameI() const {
00582         return getFileName_aux(-1);
00583     };
00584
00585     const char* getFileName_aux(int k) const {
00586         int i;
00587         if (k>=0)
00588             i=sprintf(NRG_FILE,NRG_FLEN,"%s_%02d.mat", NAME.data, k);
00589         else i=sprintf(NRG_FILE,NRG_FLEN,"%s_info.mat", NAME.data);
00590         if (i>=NRG_FLEN) wblog(FL,"ERR File name exceeds length (%d/127)",i);
00591         return NRG_FILE;
00592     }
00593 };
00594
00595 template<class TQ, class TD>
00596 void NRGData<TQ,TD>::setupIO(
00597     const char *F, int L, const char *s, const mxArray *a,
00598     const char *istr
00599 ){
00600     if (!s || !s[0]) {
00601         if (!a || (!mxIsCell(a) && !mxIsStruct(a))) wblog(F,L,
00602         "ERR invalid input NRG data (%s)", a ? mxGetClassName(a):"NULL");
00603         NAME.init(); NRG_N=0; NRG_ITER=0;
00604         MX=(mxArray*)a;
00605     }
00606     else {
00607         Wb::matFile f; unsigned n=strlen(s);
00608         Wb::Clock clio("data:I/O",1,0,&dmClocks);
00609
00610         if (!n || n+16>NRG_FLEN) wblog(FL,
00611         "ERR invalid file/variable name '%s' (%d/%d)", s, n, NRG_FLEN);
00612
00613         NRG_N=0; NRG_ITER=0; MX=NULL;
00614         NAME=s;
00615     }

```

```

00616 #ifndef MATLAB_MEX_FILE
00617     mxArray *a=mexGetVariable("caller",s);
00618     if (a && (mxIsStruct(a) || mxIsCell(a))) {
00619         if (istr) wblog(F,L,"<i> I/O NRG%s structure variable: %s",istr,s);
00620     }
00621     MX=a;
00622     return;
00623 }
00624 #endif
00625
00626 if (!f.open(F,L, getFileName(0,'l'),"r"))
00627     wblog(F,L,"ERR cannot open files of type %s_##.mat", s);
00628 else f.close();
00629
00630 if (istr) {
00631     if (strlen(s)<40) {
00632         wblog(F,L,"<i> NRG%s is %s_##.mat",istr,s);
00633     }
00634     else { char s[256];
00635         if (getcwd(s,256)==NULL) wblog(F,L,"ERR failed read cwd");
00636         wblog(F,L,"<i> using NRG%s data ...",istr);
00637         printf("\n   pwd: %s\n   I/O: %s_##.mat\n\n",
00638             Wb::repHome(s).data, Wb::repHome(s).data);
00639     }
00640 }
00641 }
00642 };
00643
00644 template<class TQ, class TD>
00645 void NRGData<TQ,TD>::init(const char *F, int L, const char *XX, int iter) {
00646     char s[64]; unsigned n=strlen(XX), e=0;
00647     mxArray *a;
00648
00649     if (!n || n>2) wblog(F,L,"ERR invalid operator tag '%s'",XX);
00650     if (iter<0) iter=NRG_ITER;
00651
00652     strncpy(s,name.data,60); s[60]=0; strcat(s,XX);
00653     a=getMxData(s,iter);
00654     if (!a) wblog(F,L,"ERR failed to read NRGData '%s' @ iter=%d",s,iter);
00655
00656     if (n==1) {
00657         if (!strcmp(XX,"K")) K.init(F,L,a); else
00658         if (!strcmp(XX,"D")) D.init(F,L,a); else ++e;
00659     }
00660     else {
00661         if (!strcmp(XX,"KK")) mXInitQSpaceVec(F,L,a,KK); else
00662         if (!strcmp(XX,"KD")) mXInitQSpaceVec(F,L,a,KD); else
00663         if (!strcmp(XX,"DK")) mXInitQSpaceVec(F,L,a,DK); else
00664         if (!strcmp(XX,"DD")) mXInitQSpaceVec(F,L,a,DD); else ++e;
00665     }
00666     if (e) wblog(F,L,"ERR invalid operator tag '%s'", XX);
00667 };
00668
00669 template<class TQ, class TD>
00670 void NRGData<TQ,TD>::init(
00671     const char *F, int L, const mxArray* a,
00672     wvector< QSpace<TQ,TD> > &C0,
00673     int k,
00674     char force,
00675     const char *vname0
00676 ){
00677     char nstr[32]; nstr[0]=0;
00678
00679     Wb::initLocalOp(F,L,a,C0,k,force,nstr);
00680
00681     if (vname0 && vname0[0]) { name=vname0; }
00682     else if (nstr[0]) { name=nstr; }
00683 }
00684
00685
00686 template<class TT>
00687 void Wb::initLocalOp(const char *F, int L,
00688     const mxArray* a,
00689     TT &C,
00690     int k,
00691     char force,
00692     char *name
00693 ){
00694     size_t n=12; char istr[n];
00695     if (k)
00696         snprintf(istr,n,"arg %d",k);
00697     else strcpy(istr,"operator");
00698
00699     if (!a || mxIsEmpty(a)) {
00700         if (force) wblog(F,L,"ERR invalid %s%s",istr,a ? " (empty)":"" );
00701         return;
00702     }

```

```

00703     }
00704
00705     if (mxIsChar(a)) {
00706         const unsigned n=32; wbsting aux;
00707
00708         if (name==NULL) { aux.init(n+1); name=aux.data; }
00709         name[0]=0;
00710
00711         if (mxGetString(a,name,n) || name[0]==0) wblog(F,L,
00712             "ERR invalid string of %s%s",istr,name[0] ? ":" : " (empty)");
00713
00714         a=mexGetVariablePtr("caller",name);
00715
00716         if (!a) wblog(F,L,
00717             "ERR Can not read QSpace '%s' (%s) from workspace",name,istr);
00718     }
00719
00720     Wb::initLocalOp_aux(F,L,a,C);
00721
00722     if (force && C.isEmpty()) wblog(F,L,
00723         "ERR invalid QSpace (%s is empty)",istr);
00724 };
00725
00726 template<class TQ, class TD>
00727 void NRGData<TQ,TD>::forceCalc() {
00728
00729     wvector< NRGIndex<TQ,TD> > &I=nrgIdx;
00730     if (I.len==0) return;
00731
00732     for (unsigned i=0; i<I.len; ++i) {
00733         if (I[i].idx<0) continue;
00734         if (I[i].nrg==NULL) I[i].idx=-1;
00735         if (!I[i].tag.isEmpty()) {
00736             I[i].init(); I[i].idx=-1;
00737         }
00738     }
00739
00740     calc=store=1;
00741 }
00742
00743 template<class TQ, class TD>
00744 void NRGData<TQ,TD>::updateOp(
00745     const char *F, int L,
00746     const NRGData<TQ,TD> &A, const NRGData<TQ,TD> &B,
00747     unsigned nop,
00748     int iter
00749 ){
00750     unsigned i=0,j,nrefs=0, m=nrgIdx.len;
00751     const char* tags[4] = { "KK","KD","DK","DD" };
00752     int e=0,k;
00753
00754     wvector< QSpace<TQ,TD> > X, X2;
00755
00756     if (iter<0) iter=NRG_ITER;
00757     if (nrgIdx.len==0) return;
00758
00759     for (; i<m; ++i) { if (nrgIdx[i].idx>=0) ++nrefs; }
00760
00761     unsetAllRefs();
00762
00763     if (nrefs<nrgIdx.len) {
00764         if (nop==4) {
00765             Wb::updateOp(F,L,*this, DD, A.D, B.D, KK, iter);
00766             Wb::updateOp(F,L,*this, DK, A.D, B.K, KK, iter);
00767             Wb::updateOp(F,L,*this, KD, A.K, B.D, KK, iter);
00768             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
00769         }
00770         else if (nop==3) {
00771             Wb::updateOp(F,L,*this, DD, A.D, B.D, KK, iter);
00772             Wb::updateOp(F,L,*this, DK, A.D, B.K, KK, iter);
00773             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
00774
00775             if (KD.len!=DK.len) KD.initDef(DK.len);
00776             for (i=0; i<DK.len; ++i) DK[i].transp(KD[i]);
00777         }
00778         else if (nop==1) {
00779             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
00780         }
00781         else wblog(FL,"ERR %s() invalid nop (%d)",FCT,nop);
00782     }
00783
00784     if (e) wblog(FL,
00785         "ERR %s() operator %s[%d] updates to empty QSpace!",
00786         FCT,name.data,iter
00787     );
00788
00789     if (nrefs) {

```



```

00790     for (i=0; i<nop; ++i) {
00791
00792         wvector< QSpace<TQ,TD> > &C = getOpsXX(tags[i]);
00793         if (C.len!=m) C.initDef(m);
00794
00795         X.init(); X2.init();
00796
00797         for (j=0; j<m; ++j) {
00798             k=nrgIdx[j].idx; if (k<0) continue;
00799
00800             if (nrgIdx[j].nrg) {
00801                 wvector< QSpace<TQ,TD> > &R=nrgIdx[j].nrg->getOpsXX(tags[i]);
00802
00803                 if (k>=(int)R.len) wblog(FL,
00804                     "ERR index out of bounds (%d/%d)",k,R.len);
00805
00806                 C[j].init2ref(R[k]);
00807             }
00808             else if (nrgIdx[j].tag.isEmpty()) {
00809                 if (!X.len) {
00810                     sprintf(str,"%s%s", name.data, tags[i]);
00811                     mxInitQSpaceVecR23(F,L,getMxData(str,iter),X);
00812
00813                     if (k>=int(X.len)) { wblog(FL,
00814                         "ERR index out of bounds (%d/%d; %d)\n"
00815                         "(hint: is stored NRG operator data up to date?)",
00816                         k,X.len,nrgIdx.len);
00817                     }
00818                 }
00819                 if (k>=int(X.len)) wblog(FL,
00820                     "ERR index out of bounds (%d/%d; %d)",k,X.len,nrgIdx.len);
00821
00822                 X[k].save2(C[j]);
00823             }
00824             else {
00825                 if (!X2.len) {
00826                     sprintf(str,"%s%s", nrgIdx[j].tag.data, tags[i]);
00827                     mxInitQSpaceVecR23(F,L, getMxData(str,iter),X2);
00828                 }
00829                 if (k>=(int)X2.len) wblog(FL,
00830                     "ERR index out of bounds (%d/%d)",k,X2.len);
00831
00832                 X2[k].save2(C[j]);
00833             }
00834         }
00835     }
00836 };
00837
00838 template <class TQ, class TS>
00839 template<class TD>
00840 void NRGData<TQ,TS>::updateOpProd(
00841     const char *F, int L, unsigned iter,
00842     const NRGData<TQ,TS> &B,
00843     const NRGData<TQ,TD> &C
00844 ){
00845     unsigned i=0,r,n;
00846
00847     QSpace<TQ,TS> Xd;
00848
00849     char ics[13] = "2 2* 23 23*", *ic=0, *ic_=0;
00850     for (; ics[i]; ++i) { if (ics[i]==' ') ics[i]=0; }
00851
00852     n=B.checkOpConsistency(0,1,C);
00853     if (int(n)<0) wblog(FL,
00854         "ERR got dim/itag inconsistency between B and C !?");
00855     if (!n || B.KK.len!=n || B.DD.len!=n) wblog(FL,
00856         "ERR %s() got empty operators (%d/%d/%d) !?",FCT,B.KK.len,B.DD.len,n);
00857     KK.init(n); KD.init(n); DK.init(n); DD.init(n);
00858
00859     for (i=0; i<n; ++i) {
00860         r=B.KK[i].rank(FL); if (!r) r=B.DD[i].rank(FL);
00861         if (r==2) { ic=ics+0; ic_=ics+3; } else
00862         if (r==3) { ic=ics+6; ic_=ics+9; } else
00863         wblog(FL,"ERR %s() invalid rank r=%d (i=%d)",FCT,r,i);
00864
00865         B.KK[i].contract(F,L,ic,C.KK[i],ic_,KK[i]);
00866         B.KD[i].contract(F,L,ic,C.KD[i],ic_,Xd); KK[i]+=Xd;
00867
00868         B.KK[i].contract(F,L,ic,C.DK[i],ic_,KD[i]);
00869         B.KD[i].contract(F,L,ic,C.DD[i],ic_,Xd); KD[i]+=Xd;
00870
00871         B.DK[i].contract(F,L,ic,C.KK[i],ic_,DK[i]);
00872         B.DD[i].contract(F,L,ic,C.KD[i],ic_,Xd); DK[i]+=Xd;
00873
00874         B.DK[i].contract(F,L,ic,C.DK[i],ic_,DD[i]);
00875         B.DD[i].contract(F,L,ic,C.DD[i],ic_,Xd); DD[i]+=Xd;
00876

```

```

00877         if (KK[i].itags.len>2) wblog(FL,
00878             "ERR %s() KK[%d] '%s' !?",FCT,i+1,IT2STR(KK[i]));
00879         if (DD[i].itags.len>2) wblog(FL,
00880             "ERR %s() DD[%d] '%s' !?",FCT,i+1,IT2STR(DD[i]));
00881     }
00882 };
00883
00884 template <class TQ, class TS, class TD>
00885 void applyBG(const char *F, int L,
00886     QSpace<TQ,TS> &B,
00887     const QSpace<TQ,TD> &H0,
00888     const QSpace<TQ,TD> &H2
00889 ){
00890     unsigned i,j,k,m,n;
00891
00892     wvector<unsigned> I1,I2;
00893     wvector<double> E1,E2;
00894     windex ilh,i2h;
00895
00896     wbararray<TD> Omega;
00897     TD *e1,*e2;
00898
00899     wcomplex z, Einc = gRX.Einc + gRX.E0, dEinc = Einc - gRX.E2;
00900
00901     gRX.check();
00902
00903     double e12, sgm=gRX.sigma/2.;
00904
00905     if (B.isEmpty()) { return; }
00906     matchH12(FL,B,H0,H2,gRX.iter, ilh,E1,I1, i2h,E2,I2);
00907
00908     for (k=0; k<ilh.len; ++k) {
00909         wbararray<TS> &Bk = *B.DATA[k];
00910         TS *xk=Bk.data;
00911
00912         i=ilh[k]; m=I1[i+1]-I1[i];
00913         j=i2h[k]; n=I2[j+1]-I2[j];
00914
00915         if (!Bk.sameSize(m,n)) { Bk.info(FL,"B",k);
00916             MXPut(FL,"i").add(Bk,"B").add(m,"m").add(n,"n").add(k+1,"k");
00917             wblog(FL,"ERR %s() dimension mismatch (%d) !?",FCT,k+1);
00918         }
00919
00920         e1=E1.data+I1[i]; e2=E2.data+I2[j];
00921
00922         for (j=0; j<n; ++j) {
00923             for (i=0; i<m; ++i) { e12 = e2[j] - e1[i];
00924                 z.r = dEinc.r - e12;
00925                 z.i =-Einc.i;
00926
00927                 if (!gRX.wrxs) {
00928                     z/=sqrt(abs(z.i));
00929                 }
00930                 else {
00931                     wblog(FL,"ERR %s() deprecated using BHeisenberg!!",FCT);
00932
00933                     z.i -= sgm*(gRX.E0+gRX.E2);
00934                     if (z.i>=0) wblog(FL,"ERR %s() got z=%g%+gi",FCT,z.r,z.i);
00935
00936                     z/=sqrt(abs(z.i));
00937                 }
00938
00939                 xk[i+m*j]/=z;
00940             }
00941         }
00942     };
00943
00944 template <class TQ, class TS>
00945 template <class TD>
00946 void NRGData<TQ,TS>::initBG(
00947     const char *F, int L,
00948     const NRGData<TQ,TD> &B,
00949     const NRGData<TQ,TD> &H0,
00950     const NRGData<TQ,TD> &H2,
00951     int m
00952 ){
00953     unsigned i;
00954
00955     KD.initT_(B.KD); for (i=0; i<KD.len; ++i) applyBG(F_L,KD[i], H0.K, H2.D);
00956     DK.initT_(B.DK); for (i=0; i<DK.len; ++i) applyBG(F_L,DK[i], H0.D, H2.K);
00957     DD.initT_(B.DD); for (i=0; i<DD.len; ++i) applyBG(F_L,DD[i], H0.D, H2.D);
00958
00959     if (m==3) return; else
00960     if (m!=4) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
00961
00962     KK.initT_(B.KK); for (i=0; i<KK.len; ++i) applyBG(F_L,KK[i], H0.K, H2.K);
00963 };

```

```

00964
00965 template <class TQ, class TD, class TZ>
00966 void WeightE0_dim1(QSpace<TQ,TZ> &X, const QSpace<TQ,TD> &H, double a) {
00967
00968     unsigned i,j,k; widx_t m1,m2,d,D;
00969     double efac,w=1;
00970
00971     wbMatrix<TQ> Qh,Qx;
00972     wbindex ih,ix;
00973
00974     if (X.isEmpty()) { return; }
00975     if ((i=X.rank())<2 || i>3 || H.rank()!=2) wblog(FL,
00976         "ERR %s() got rank-%d data (%d)",FCT,i,H.rank());
00977
00978     if (a<0) { w=0; a=-a; } else
00979     if (!a) wblog(FL,"ERR %s() invalid w=%g",FCT,a);
00980
00981     X.getQsub(0,Qx);
00982     H.getQsub(0,Qh);
00983
00984     matchIndex(Qh,Qx,ih,ix,1,&m1,&m2);
00985     if (m1 || ix.len!=Qx.dim1) wblog(FL,"ERR %s() "
00986         "need unique match (%d/%d; %d/%d)",FCT,m1,m2,ix.len,Qx.dim1);
00987     if (ix.len!=X.DATA.len) wblog(FL,"ERR %s() "
00988         "severe QSpace inconsistency (%d/%d)!",FCT,ix.len,X.DATA.len);
00989
00990     for (k=0; k<ix.len; ++k) {
00991         wbararray<TZ> &xk=*X.DATA[ix[k]];
00992         const wbararray<TD> &ek=*H.DATA[ih[k]];
00993         xk.getMatSize(d,D,1);
00994
00995         if (!ek.isVector()) wblog(FL,
00996             "ERR %s() non-diagonal H (%d: %s)",FCT,k+1,SSTR(ek));
00997         if (d!=ek.dim1()) wblog(FL,"ERR %s() "
00998             "size mismatch (%d: d=%ld/%ld x %ld)",FCT,k+1,d,ek.dim1(),D);
00999
01000         for (i=0; i<d; ++i) {
01001             efac=exp(-a*ek[i]*ek[i]); if (!w) { efac=1-efac; }
01002             efac=sqrt(efac);
01003             for (j=0; j<D; ++j) { xk[i+d*j]*=efac; }
01004         }
01005     }
01006 };
01007
01008 template <class TQ, class TS, class TD>
01009 void matchH12(
01010     const char *F, int L,
01011     QSpace<TQ,TS> &X,
01012     const QSpace<TQ,TD> &H1, const QSpace<TQ,TD> &H2, unsigned iter,
01013     wbindex &i1h, wbvector<double> &E1, wbvector<unsigned> &I1,
01014     wbindex &i2h, wbvector<double> &E2, wbvector<unsigned> &I2,
01015     char redCGR=0
01016 ){
01017     unsigned i,j,e=0; widx_t m1,m2;
01018     wbindex i1,i2;
01019     wbperm P;
01020
01021     wbvector<unsigned> D1,D2;
01022     wbMatrix<TQ> Q1h,Q2h,Q1,Q2;
01023
01024     double Escale = gEScale(iter);
01025
01026     if (X.isEmpty()) {
01027         wblog(F_L,"ERR %N empty matrix elements !?");
01028     }
01029
01030     if (X.rank(F_L)==3 || !X.CGR.isEmpty()) { X.isConsistent(F_L);
01031         if (X.CGR.isEmpty()) {
01032             if (!X.allAbelian()) { wblog(F_L,
01033                 "ERR %s() expecting CG data for rank-3 data (%s)",
01034                 FCT, X.qtype.toStr().data);
01035             }
01036         }
01037         else {
01038             for (i=0; i<X.DATA.len; ++i) {
01039                 wbararray<TS> &D=(*X.DATA[i]);
01040
01041                 if (D.SIZE.len==3) {
01042                     if (D.SIZE[2]!=1) wblog(F_L,
01043                         "ERR expecting operator to have dim3=1 (%s)",SSTR(D));
01044                     if (redCGR) { D.SIZE.len=2; }
01045                 }
01046                 else if (D.SIZE.len!=2) wblog(F_L,
01047                     "ERR invalid rank operator (%s)",SSTR(D));
01048
01049                 if (redCGR) {
01050                     for (j=0; j<X.CGR.dim2; ++j) {

```

```

01051             if (!X.CGR(i,j).isScalar()) wblog(F_L,
01052             "ERR %s() got non-scalar cref (%d,%d): %s",
01053             FCT,i+1,j+1,X.CGR(i,j).toStr().data
01054             );
01055         }
01056     }
01057 }
01058 }
01059
01060 getEDData(H1,E1,D1); E1*=Escale; D1.cumsum_(I1,'x');
01061 getEDData(H2,E2,D2); E2*=Escale; D2.cumsum_(I2,'x');
01062
01063 H1.getQsub(0,Q1h); X.getQsub(0,Q1);
01064 H2.getQsub(0,Q2h); X.getQsub(1,Q2);
01065
01066 i=matchIndex(Q1h,Q1,i1h,i1,1,&m1,&m2); if (m1 || i1.len!=Q1.dim1) ++e;
01067 i=matchIndex(Q2h,Q2,i2h,i2,1,&m1,&m2); if (m1 || i2.len!=Q2.dim1) ++e;
01068
01069 if (e) wblog(F_L,
01070 "ERR %s() need unique and exact match (%d)!",FCT,e);
01071 if (i1.len!=X.DATA.len) wblog(F_L,
01072 "ERR %s() severe QSpace inconsistency (%d %d)!",
01073 FCT, i1.len, X.DATA.len
01074 );
01075
01076 i1.toPerm(P); P.Invert(); i1h.Select(P);
01077 i2.toPerm(P); P.Invert(); i2h.Select(P);
01078 };
01079
01080 template<class TQ, class TD>
01081 void NRGData<TQ,TD>::unsetAllRefs() {
01082     unsigned i,j,m=0;
01083     const char* tags[4] = { "KK","KD","DK","DD" };
01084
01085     for (i=0; i<4; ++i) {
01086         wvector< QSpace<TQ,TD> > &C = getOpsXX(tags[i]);
01087
01088         for (j=0; j<C.len; ++j) {
01089             if (nrgIdx[j].idx<0 || !C[j].isref) continue;
01090             C[j].init(); ++m;
01091         }
01092     }
01093 }
01094
01095 }
01096
01097 template<class TQ, class TD>
01098 unsigned NRGData<TQ,TD>::skipOpZeros(double eps, char bflag) {
01099
01100     wvector< QSpace<TQ,TD> > *XX[] = { &KK, &KD, &DK, &DD };
01101     unsigned i,j,n,r=0;
01102
01103     for (i=0; i<4; ++i) {
01104         n=XX[i]->len;
01105         for (j=0; j<n; ++j)
01106             r += (*XX[i])[j].SkipZeroData(eps,bflag);
01107     }
01108
01109     return r;
01110 }
01111
01112 template<class TQ, class TD>
01113 void NRGData<TQ,TD>::times(TD fac, const char *flag) {
01114
01115     unsigned i,j,n=(flag ? strlen(flag) : 0);
01116
01117     if (n==2) {
01118         if (!strcmp(flag,"XX")) {
01119             const char* tags[4] = { "KK","KD","DK","DD" };
01120             for (i=0; i<4; ++i) {
01121                 wvector< QSpace<TQ,TD> > &X=getOpsXX(tags[i]);
01122                 for (j=0; j<X.len; ++j) X[j]*=fac;
01123             }
01124         }
01125         else {
01126             wvector< QSpace<TQ,TD> > &X=getOpsXX(flag);
01127             for (j=0; j<X.len; ++j) X[j]*=fac;
01128         }
01129     }
01130     else if (n==1) {
01131         if (flag[0]=='X') {
01132             getQSpace('K')*=fac;
01133             getQSpace('D')*=fac;
01134         }
01135         else {
01136             getQSpace(flag[0])*=fac;
01137         }
01138     }

```

```

01138     }
01139     else wblog(FL, "ERR invalid flag `%s'", flag ? flag : "(null)");
01140 };
01141
01142 template<class TQ, class TD>
01143 void NRGData<TQ,TD>::saveOp(
01144     const char *F, int L, int iter, unsigned m) const {
01145
01146     unsigned i=0, n=nrgIdx.len, nrefs=0;
01147     const char* tags[4]={"KK","KD","DK","DD"};
01148
01149     if (!m || m==2 || m>4) wblog(FL,
01150     "ERR value out of bounds (m=%d/%d)",m,4);
01151     if (iter<0) iter=NRG_ITER;
01152
01153     for (; i<n; ++i) { if (nrgIdx[i].idx>=0) { ++nrefs; }}
01154
01155     if (store && (!n || nrefs<n)) { i=0;
01156         if (m==3) { i=1; m=4; }
01157         #if 0
01158         if (WBLOG_IO) { sprintf(str,
01159             "%s() iter=%2d: store=%d, nrefs=%d/%ld, i=%d/%d ",
01160             FCT, iter, store, nrefs,nrgIdx.len, i,m);
01161             wblog(FL, MX? "MX: %s":"MAT %s",str);
01162         }
01163         #endif
01164         for (; i<m; ++i) { updatePara(F,L,tags[i],iter); }
01165     }
01166 };
01167
01168 template<class TQ, class TD>
01169 void NRGData<TQ,TD>::updatePara(
01170     const char *F, int L, const char *tag, mxArray *a, int iter) const {
01171
01172     sprintf(str,"%s%s", name.data, tag);
01173     if (!str[0]) wblog(F,L,"ERR invalid name (empty)");
01174
01175     if (iter<0) iter=NRG_ITER;
01176     if (MX) {
01177         if (!NAME.isEmpty())
01178             mxUpdateField(F,L, MX,str,iter,a);
01179     }
01180     else {
01181         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01182         matPutVariable(F,L,getFileNames(iter),str,a);
01183     }
01184 };
01185
01186 template<class TQ, class TD>
01187 void NRGData<TQ,TD>::updatePara(const char *F, int L,
01188     const char *tag,
01189     int iter,
01190     const char *tag2,
01191     char all
01192 ) const {
01193
01194     unsigned n=strlen(tag), e=0;
01195     mxArray *a=0;
01196
01197     if (n==1) {
01198         if (tag[0]=='K') { a=K.toMx(); } else
01199         if (tag[0]=='D') { a=D.toMx(); } else e|=1;
01200     }
01201     else if (n==2) {
01202         const wvector< QSpace<TQ,TD> > *XX=0;
01203         if (!strcmp(tag,"KK")) { XX=&KK; } else
01204         if (!strcmp(tag,"KD")) { XX=&KD; } else
01205         if (!strcmp(tag,"DK")) { XX=&DK; } else
01206         if (!strcmp(tag,"DD")) { XX=&DD; } else e|=2;
01207
01208         if (!e) {
01209             if (all>0 || (all<0 && store>1)) { a=XX->toMx(); }
01210             else {
01211                 windex I; getRelevantOps(I);
01212                 a=QSpaceVec2Mx(*XX,I);
01213             }
01214         }
01215     }
01216     else { e|=4; }
01217     if (e) wblog(FL,"ERR invalid tag `%s'",tag);
01218
01219     sprintf(str,"%s%s", name.data, tag2 && tag2[0] ? tag2 : tag);
01220     if (!str[0] || !name.isName()) wblog(F,L,"ERR invalid name `%s'",str);
01221
01222     if (iter<0) iter=NRG_ITER;
01223     if (MX) {
01224         mxUpdateField(F,L, MX,str,iter,a);

```

```

01225     }
01226     else {
01227         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01228         matPutVariable(F,L,getFileNames(iter),str,a);
01229     }
01230 };
01231
01232 template<class TQ, class TD>
01233 inline void NRGData<TQ,TD>::getRelevantOps(wbindex &I) const {
01234     unsigned i,k;
01235     I.init(nrgIdx.len);
01236     for (k=i=0; i<nrgIdx.len; ++i) {
01237         if (nrgIdx[i].idx<0) I[k++]=i;
01238     }
01239     I.len=k;
01240 }
01241
01242 template<class TQ, class TD>
01243 const QSpace<TQ,TD>& NRGData<TQ,TD>::getQSpace(
01244     const char *t, unsigned i
01245 ) const {
01246     int d=-1;
01247     if (!t || !index("KD",t[0]) || !index("KD",t[1]) || t[2])
01248         wblog(FL,"ERR invalid tag '%s', t ? t : "(null)");
01249     if (t[0]=='K') {
01250         if (t[1]=='K')
01251             { if (i<KK.len) return KK[i]; else d=KK.len; }
01252         else { if (i<KD.len) return KD[i]; else d=KD.len; }
01253     }
01254     else {
01255         if (t[1]=='K')
01256             { if (i<DK.len) return DK[i]; else d=DK.len; }
01257         else { if (i<DD.len) return DD[i]; else d=DD.len; }
01258     }
01259     wblog(FL,"ERR index out of bounds (%s: %d/%d)",t,i,d);
01260     return K;
01261 };
01262
01263 template<class TQ, class TD>
01264 void NRGData<TQ,TD>::updateInfo(const char *F, int L,
01265     const char *vname, mxArray *a, char keep
01266 ) const {
01267     if (!a) wblog(F,L,
01268         "WRN update info '%s' with NULL mxArray", vname);
01269     if (MX) {
01270         mexPutVariable("caller", vname, a);
01271         if (!keep) mxDestroyArray(a);
01272     }
01273     else {
01274         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01275         matPutVariable(F,L,getFileNames(I),vname,a,keep);
01276     }
01277 }
01278
01279 template<class TQ, class TD>
01280 void NRGData<TQ,TD>::updateInfop(const char *F, int L,
01281     const char *vtag, mxArray *a, char keep
01282 ) const {
01283     if (!name.data || !name.data[0]) wblog(F,L,
01284         "WRN update info '%s' of unnamed operator", vtag);
01285     sprintf(str,"%s%s",
01286         name.data ? name.data : "",
01287         vtag ? vtag : "");
01288     updateInfo(F,L,str,a,keep);
01289 }
01290
01291 template<class TQ, class TD>
01292 bool NRGData<TQ,TD>::opExists(char tflag) {
01293     if (tflag) {
01294         unsigned i, k=(NRG_N>2 ? NRG_N-2 : 0);
01295         const char* tag[3] = { "KK","KD","DK" };
01296         for (i=0; i<3; ++i) {

```

```

01312         sprintf(str,"%s%s",name.data,tag[i]);
01313         if (!isQSpaceVec(str,k)) return 0;
01314     }
01315 }
01316 else {
01317     sprintf(str,"%sKK",name.data);
01318     return isQSpaceVec(str,0);
01319 }
01320
01321 return 1;
01322 }
01323
01324 template<class TQ, class TD>
01325 bool NRGData<TQ,TD>::initOp(const char *F, int L,
01326     const wvector< QSpace<TQ,TD> > &C0,
01327     char tflag,
01328     NRGData<TQ,TD>* CR,
01329     const char *istr
01330 ){
01331     unsigned i,j,m,iter,e,r, nrefs=0;
01332     const char* tag[3] = { "KK","KD","DK" };
01333     unsigned ntag=sizeof(tag)/sizeof(char*);
01334     wvector< QSpace<TQ,TD> > NRG;
01335     mxArray *a;
01336
01337     nrgIdx.initDef(C0.len); kloc=0;
01338
01339     if (C0.len==0) {
01340         init(1); return 0;
01341     }
01342
01343     if (istr && istr[0]) sprintf(str,"%s ",istr); else str[0]=0;
01344
01345     if (!name.isName(99)) wblog(F,L,
01346         "ERR invalid name for %soperator (%d,'%s')",
01347         str, nrgIdx.len, name.data
01348     );
01349
01350     for (i=0; i<C0.len; ++i) {
01351         r=-1;
01352         if (!C0[i].isConsistent(r) || r<2 || r>4) wblog(F,L,
01353             "ERR invalid operator (got rank-%d QSpace) !?\n"
01354             "%s (%s)",r,SHORT_FL,str);
01355         if (C0[i].QDIM!=C0[0].QDIM) wblog(FL,
01356             "ERR invalid operator (QDIM=%d, %d) !?\n%s:%d %s",
01357             C0[i].QDIM, C0[0].QDIM,FL,str);
01358     }
01359
01360     KK.initDef(C0);
01361
01362     for (i=0; i<C0.len; ++i) {
01363         if (nrgIdx[i].idx>=0) continue;
01364         for (j=i+1; j<C0.len; ++j)
01365             if (C0[j]==C0[i]) {
01366                 nrgIdx[j].init(i,this);
01367                 ++nrefs;
01368             }
01369     }
01370
01371     if (CR) {
01372         m=CR->KK.len;
01373         for (i=0; i<C0.len; ++i) {
01374             if (nrgIdx[i].idx>=0) continue;
01375             for (j=0; j<m; ++j) if (C0[i]==CR->KK[j]) {
01376                 nrgIdx[i].init(j,CR);
01377                 ++nrefs;
01378                 break;
01379             }
01380         }
01381         sprintf(str,"%s%s",CR->name.data,"KK_");
01382         a=getMxData(str,0);
01383         if (a && mxIsQSpaceVecR23(FL,a)) {
01384             mxInitQSpaceVecR23(F,L,a,NRG); m=NRG.len;
01385             for (i=0; i<C0.len; ++i) {
01386                 if (nrgIdx[i].idx>=0) continue;
01387                 for (j=0; j<m; ++j) if (C0[i]==NRG[j]) {
01388                     nrgIdx[i].init(j,CR->name.data);
01389                     ++nrefs;
01390                     break;
01391                 }
01392             }
01393         }
01394
01395         if (nrefs==nrgIdx.len) {
01396             return nrefs;
01397         }
01398     }

```

```

01399
01400     e=0;
01401
01402     if (tflag) {
01403         iter=(NRG_N>2 ? NRG_N-2 : 0);
01404         for (i=0; i<ntag; ++i) {
01405             sprintf(str,"%s%s",name.data,tag[i]);
01406             if (!isQSpaceVec(str,iter)) { ++e; break; }
01407         }
01408     }
01409
01410     iter=0;
01411
01412     sprintf(str,"%s%s",name.data,"KK");
01413     if (!isQSpaceVec(str,iter)) ++e;
01414
01415     sprintf(str,"%s%s",name.data,"KK_");
01416     a=getMxData(str,iter); if (!a || !mxIsQSpaceVecR23(FL,a)) ++e;
01417
01418     if (e) {
01419         calc=store=1;
01420         return nrefs;
01421     }
01422
01423     mxInitQSpaceVecR23(F,L,a,NRG);
01424
01425     for (i=0; i<C0.len; ++i) {
01426         if (nrgIdx[i].idx>=0 && nrgIdx[i].tag.isEmpty()) continue;
01427         for (j=0; j<NRG.len; ++j) if (C0[i]==NRG[j]) {
01428             nrgIdx[i].tag.init();
01429             nrgIdx[i].idx=j; ++nrefs;
01430             break;
01431         }
01432     }
01433
01434     calc= (nrefs==C0.len ? 0 : 1);
01435     store=(nrefs==C0.len ? 0 : 1);
01436
01437     return nrefs;
01438 };
01439
01440 template<class TQ, class TD>
01441 bool NRGData<TQ,TD>::initOp(const char *F, int L,
01442     const wbvector< wbvector< QSpace<TQ,TD> > > &CC,
01443     NRGData<TQ,TD>* CR,
01444     const char *istr,
01445     char tflag
01446 ){
01447     unsigned i,j,l,m,iter,q=0,e,r, nrefs=0;
01448     const char* tag[3]={ "KK","KD","DK" };
01449     unsigned ntag=sizeof(tag)/sizeof(char*);
01450     const mxArray *a;
01451
01452     wbvector< wbvector< QSpace<TQ,TD> > > NRG;
01453
01454     nrgIdx.initDef(CC.len);
01455
01456     if (CC.len==0) {
01457         init(1); return 0;
01458     }
01459
01460     if (istr && istr[0])
01461         sprintf(str,"%s ",istr); else str[0]=0;
01462
01463     if (!name.isName(99)) wblog(F,L,
01464         "ERR invalid name for %soperator (%d,'%s')",
01465         str, nrgIdx.len, name.data
01466     );
01467
01468     kloc=-1;
01469
01470     for (i=0; i<CC.len; ++i) {
01471         for (l=j=0; j<CC[i].len; ++j) {
01472             const QSpace<TQ,TD> &x = CC[i][j]; if (x.isEmpty()) continue;
01473             r=-1; l=j; if (!q) q=x.QDIM;
01474
01475             if (!x.isConsistent(r) || r<2 || r>3) wblog(F,L,
01476                 "ERR invalid operator (rank-%d QSpace) !?\n"
01477                 "--> %s %s",r, SHORT_FL, str);
01478             if (x.QDIM!=q) wblog(FL,
01479                 "ERR invalid operator (QDIM=%d, %d) !?\n--> %s %s",
01480                 x.QDIM, q, SHORT_FL, str
01481             );
01482         } kloc=MAX(kloc,(int)l); }
01483
01484     if (kloc<0) wblog(FL,"ERR all empty operators !?");
01485     kloc=MAX(0,kloc-1);

```



```

01486
01487     KK.initDef(CC.len);
01488     CI.initDef(CC.len);
01489     for (i=0; i<CC.len; ++i) {
01490         if (!CC[i].len) wblog(FL,"ERR empty operator set !?");
01491         CI[i].initDef(CC[i]);
01492         KK[i]=CC[i][0];
01493     }
01494
01495     for (i=0; i<CC.len; ++i) { if (nrgIdx[i].idx>=0) continue;
01496         for (j=i+1; j<CC.len; ++j)
01497             if (CC[i].isEqual(CC[j])) { ++nrefs;
01498                 nrgIdx[j].init(i,this);
01499             }
01500     }
01501
01502     if (CR && CR->CI.len) { m=CR->CI.len;
01503         for (i=0; i<CC.len; ++i) {
01504             if (nrgIdx[i].idx>=0) continue;
01505
01506             for (j=0; j<m; ++j)
01507                 if (CC[i].isEqual(CR->CI[j])) {
01508                     nrgIdx[i].init(j,CR);
01509                     ++nrefs; break;
01510                 }
01511         }
01512
01513         sprintf(str,"%s%s",CR->name.data,"CI_");
01514         a=getMxInfo(str);
01515         if (a) { if (mxIsQSpaceVecVec(a,2)) {
01516             mxInitQSpaceVecVec(F,L,a,NRG); m=NRG.len;
01517             for (i=0; i<CC.len; ++i) {
01518                 if (nrgIdx[i].idx>=0) continue;
01519                 for (j=0; j<m; ++j) if (CC[i]==NRG[j]) {
01520                     nrgIdx[i].init(j,CR->name.data);
01521                     ++nrefs; break;
01522                 }
01523             } else wblog(FL,
01524                 "WRN %s::CI_ not a QSpace cell vector !?", CR->name.data);
01525         }
01526
01527         if (nrefs==nrgIdx.len) { return nrefs; }
01528     }
01529
01530     e=0;
01531
01532     if (tflag && NRG_N>2) {
01533         iter=NRG_N-2;
01534         for (i=0; i<ntag; ++i) {
01535             sprintf(str,"%s%s",name.data,tag[i]);
01536             if (!isQSpaceVec(str,iter)) { ++e; break; }
01537         }
01538     }
01539
01540     iter=0;
01541     sprintf(str,"%s%s",name.data,"KK");
01542     if (!isQSpaceVec(str,iter)) ++e;
01543
01544     sprintf(str,"%s%s",name.data,"CI_");
01545     a=getMxData(str,iter); if (!a || !mxIsQSpaceVec(FL,a)) ++e;
01546
01547     if (e) { calc=store=1;
01548         wblog(FL," * calculate all operators %s (%d)",
01549             name.data, nrgIdx.len); return nrefs;
01550     }
01551
01552     mxInitQSpaceVecVec(F,L,a,NRG); m=NRG.len;
01553
01554     for (i=0; i<CC.len; ++i) {
01555         if (nrgIdx[i].idx>=0 && nrgIdx[i].tag.isEmpty()) continue;
01556         for (j=0; j<m; ++j) if (CC[i].isEqual(NRG[j])) {
01557             nrgIdx[i].tag.init();
01558             nrgIdx[i].idx=j; ++nrefs;
01559             break;
01560         }
01561     }
01562
01563     calc = (nrefs==CC.len) ? 0 : 1;
01564     store = (nrefs==CC.len) ? 0 : 1;
01565
01566     return nrefs;
01567 };
01568
01569 template<class TQ, class TD>
01570 void NRGData<TQ,TD>::initSym(const char *tag) {
01571
01572     if (strlen(tag)==2) {

```

```

01573     const wbvector< QSpace<TQ,TD> > &K=getOpsXX(tag);
01574     unsigned i,n=K.len;
01575     issym.init(n);
01576
01577     for (i=0; i<n; ++i) {
01578         if (K[i].isHConj()) issym[i]=+1; else
01579         if (K[i].isAHerm()) issym[i]=-1;
01580     }
01581 }
01582 else if (strlen(tag)==1) {
01583     const QSpace<TQ,TD> &K=getQSpace(tag);
01584     issym.init(1);
01585     if (K.isHConj()) issym[0]=+1; else
01586     if (K.isAHerm()) issym[0]=-1;
01587 }
01588 else wblog(FL,"ERR invalid tag '%s'", tag ? tag : "(null)");
01589 }
01590
01591 template<class TQ, class TD>
01592 void NRGData<TQ,TD>::check_dloc(
01593     const char *F, int L, const char *fun, unsigned dloc, unsigned iter) const {
01594
01595     widxt_t d;
01596     K.getDim(2,&d);
01597
01598     if (iter+1>=NRG_N) {
01599         if (d) wblog(F_L,
01600             "ERR %s() got dloc=%li/%d at k=%d/%d !?", fun,d,dloc,iter,NRG_N);
01601         return;
01602     }
01603
01604     if (d!=dloc && (iter || !D.isEmpty())) {
01605         char s[64]; snprintf(s,64,
01606             "got inconsistent dloc=%li/%d (k=%d/%d)",d,dloc,iter,NRG_N);
01607
01608         if (D.isEmpty())
01609             wblog(F_L,"WRN %s() %s",fun,s);
01610         else wblog(F_L,"ERR %s() %s",fun,s);
01611     }
01612 };
01613
01614 template<class TQ, class TD>
01615 template<class TD_>
01616 int NRGData<TQ,TD>::checkOpConsistency(
01617     const char *F, int L, const NRGData<TQ,TD_> &B) const {
01618
01619     unsigned e=0, n = (KK.len ? KK.len : B.KK.len);
01620     if (!n) n = (DD.len ? DD.len : B.DD.len);
01621     if (!n) { return n; }
01622
01623     if (KK.len && B.KK.len) {
01624         if (KK.len!=B.KK.len) { ++e;
01625             if (L) wblog(F_L,"WRN ... KK.len=%d/%d",KK.len,B.KK.len); }
01626         if (KD.len!=B.KD.len || KD.len!=n) { ++e;
01627             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
01628         if (DK.len!=B.DK.len || DK.len!=n) { ++e;
01629             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
01630         if (DD.len!=B.DD.len || DD.len!=n) { ++e;
01631             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
01632     }
01633     else if (KK.len) {
01634         if (KD.len!=n) { ++e;
01635             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
01636         if (DK.len!=n) { ++e;
01637             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
01638         if (DD.len!=n) { ++e;
01639             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
01640     }
01641     else {
01642         if (B.KD.len!=n) { ++e;
01643             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
01644         if (B.DK.len!=n) { ++e;
01645             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
01646         if (B.DD.len!=n) { ++e;
01647             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
01648     }
01649     if (e) {
01650         if (F) wblog(FL,"ERR %s() got size consistency",FCT);
01651         return -e;
01652     }
01653
01654     return n;
01655 };
01656
01657 template<class TQ, class TD>
01658 bool NRGData<TQ,TD>::isQSpaceVec(const char *nm, int iter) {
01659

```

```

01660     if (iter<0) iter=NRG_ITER;
01661     if (iter>=(int)NRG_N) wblog(FL,
01662         "ERR Index out of bounds (%d/%d)", iter, NRG_N);
01663
01664     const mxArray *a=getMxData(nm,iter,'i');
01665     if (a && mxIsQSpaceVecR23(FL,a) return 1;
01666
01667     return 0;
01668
01669 }
01670
01671 template<class TQ, class TD>
01672 bool NRGData<TQ,TD>::checkVec(
01673     const char *F, int L,
01674     const char *XX, unsigned R, const char *istr
01675 ){
01676     unsigned l,n=128;
01677     char nm[n]; int fid;
01678
01679     char cflag=WbUtil<TD>::isComplex();
01680
01681     l=snprintf(nm,n,"%s%s", name.data, XX);
01682     if (l>=n) wblog(FL,"ERR strlen out of bounds (%d/%d)",l,n);
01683
01684     if (MX) {
01685         const size_t *s=mxGetDimensions(MX);
01686         const int n=mxGetNumberOfDimensions(MX);
01687
01688         fid=mxGetFieldNumber(MX, nm);
01689         if (fid<0) {
01690             if (istr) {
01691                 wblog(F,L,"WRN invalid %s", istr);
01692                 wblog(FL, "ERR field '%s' could not be found",nm);
01693             }
01694             return 1;
01695         }
01696
01697         if (n!=2 || (s[0]!=1 && s[1]!=1)) {
01698             if (istr) {
01699                 wblog(F,L,"WRN invalid %s (%s)",istr,nm);
01700                 wblog(FL, "ERR NRG data must be row structure vector (%d,%d)",
01701                     (int)s[0],(int)s[1]);
01702             }
01703             return 1;
01704         }
01705
01706         if (!NRG_N) NRG_N=unsigned(s[0]*s[1]); else
01707         if (NRG_N!=unsigned(s[0]*s[1])) {
01708             if (istr) {
01709                 wblog(F,L,"WRN invalid %s (%s)",istr,nm);
01710                 wblog(FL, "ERR size mismatch (%d/%d)", NRG_N, int(s[0]*s[1]));
01711             }
01712             return 1;
01713         }
01714
01715         return !mxIsQSpaceVec(MX,fid,R,cflag);
01716     }
01717     else {
01718         int i=mxIsQSpaceVec(F_L,NAME.data,nm,R,0,0,cflag,NRG_N);
01719         if (i<0) {
01720             wblog(F,L,"ERR invalid %s (%s)%N%N%s%N",istr? istr:"!?",nm,istr);
01721             return 1;
01722         }
01723
01724         if (!NRG_N) NRG_N=i; else
01725         if (i!=(int)NRG_N) {
01726             wblog(F,L,"WRN invalid %s (%s)", istr?istr:"!?", nm);
01727             wblog(FL,"ERR size mismatch (%d,%d)", i, NRG_N);
01728             return 1;
01729         }
01730     }
01731
01732     return 0;
01733 };
01734
01735 template<class TQ, class TD>
01736 bool NRGData<TQ,TD>::checkVecVec(const char *XX, unsigned R) {
01737
01738     unsigned l,n=128;
01739     char nm[n]; int fid;
01740
01741     l=snprintf(nm,n,"%s%s",name.data,XX);
01742     if (l>=n) wblog(FL,"ERR strlen out of bounds (%d/%d)",l,n);
01743
01744     if (MX) {
01745         fid=mxGetFieldNumber(MX,nm);
01746         if (fid<0) {

```

```

01747         sprintf(str, "%s:%d field '%s' could not be found.",
01748             FL,nm); return 1;
01749     }
01750
01751     if (!NRG_N) {
01752         const size_t *s=mxGetDimensions(MX);
01753         const int n=mxGetNumberOfDimensions(MX);
01754         if (n!=2 || (s[0]!=1 && s[1]!=1)) { sprintf(str,
01755             "%s:%d NRG data must be row structure vector (%d,%d)",
01756             FL, (int)s[0], (int)s[1]); return 1;
01757         }
01758         NRG_N=unsigned(s[0]*s[1]);
01759     }
01760     return !mxIsQSpaceVEC(MX, fid, R);
01761 }
01762 else {
01763
01764     int i=mxIsQSpaceVEC(FL,NAME.data, nm, R);
01765     if (i<0) return 1;
01766     if (!NRG_N) NRG_N=i;
01767     if (i!=(int)NRG_N) {
01768         sprintf(str,"%s:%d size mismatch (%d,%d)",
01769             FL, i, NRG_N); return 1;
01770     }
01771 }
01772
01773 return 0;
01774 }
01775
01776 template<class TQ, class TD>
01777 mxArray* NRGData<TQ,TD>::getMxData(const char *vs, int iter, char ionly) {
01778     if (iter<0) iter=NRG_ITER;
01779     if (iter>=(int)NRG_N) wblog(FL,
01780         "ERR %s() index out of bound (%d/%d)",FCT,iter,NRG_N);
01781
01782     if (MX) {
01783         int fid=mxGetFieldNumber(MX, vs);
01784         if (fid<0) return NULL;
01785         return mxGetFieldByNumber(MX,iter,fid);
01786     }
01787     else {
01788         Wb::matFile F(FL,getName(iter),"r");
01789         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01790
01791         if (aux) { mxDestroyArray(aux); aux=NULL; }
01792         if (ionly)
01793             { aux = matGetVariableInfo(F.mfp,vs); }
01794         else { aux = matGetVariable(F.mfp,vs); }
01795
01796         F.close(); return aux;
01797     }
01798 };
01799
01800 template<class TQ, class TD>
01801 const mxArray* NRGData<TQ,TD>::getMxInfo(const char *v) {
01802
01803     if (!v || !v[0]) wblog(FL,
01804         "ERR %s() invalid usage (got %s)",FCT,v? "empty":"null");
01805
01806     if (aux) { mxDestroyArray(aux); aux=NULL; }
01807
01808     if (MX) {
01809         return mexGetVariablePtr("caller",v);
01810     }
01811     else {
01812         Wb::matFile F(FL,getNameI(),"r");
01813         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01814
01815         aux=matGetVariable(F.mfp, v);
01816
01817         return aux;
01818     }
01819 };
01820
01821 template<class TQ, class TD>
01822 mxArray* NRGData<TQ,TD>::toMx() const {
01823
01824     MXPut S(0,0);
01825     S.add(name,"name")
01826     .add(calc,"calc").add(store,"store")
01827     .add(kloc,"kloc").add(issym,"issym")
01828     .addP(MXPut(0,0)
01829         .add(CI,"CI").add(nrgIdx,"nrgIdx").add(NAME,"NAME")
01830         .add(Wb::num2Str(MX,"0x%1X"),"MX").add(locMX,"locMX")
01831         .add(NRG_ITER,"ITER")
01832         .add(NRG_N,"N").toMx(),"Idx"
01833     );

```

```

01834
01835     if (KK.len || KD.len || DK.len || DD.len) {
01836         S.add(KK, "KK").add(KD, "KD").add(DK, "DK").add(DD, "DD"); }
01837
01838     if (!K.isEmpty() || !D.isEmpty()) {
01839         S.add(K, "K").add(D, "D"); }
01840
01841     return S.toMx();
01842 };
01843
01844 template<class TQ, class TD>
01845 void NRGData<TQ,TD>::info(const char *vstr) const {
01846
01847     unsigned i,nl=0;
01848     char sx[nrgIdx.len+1], s_[128];
01849
01850     for (i=0; i<nrgIdx.len; ++i) {
01851         if ( nrgIdx[i].idx<0) sx[i]='C';
01852         if (!nrgIdx[i].nrg) sx[i]='R';
01853         else if ( nrgIdx[i].nrg==this)
01854             sx[i]='1';
01855         else sx[i]='2';
01856     }; sx[i]=0;
01857
01858     snprintf(s_,128,"%d/%d; calc=%d (%s; %d); store=%d",
01859     NRG_ITER+1, NRG_N, calc, sx, nrgIdx.len, store);
01860
01861     if (MX) {
01862         printf("\n%s Structure `s' (0x%lX), %s\n",
01863         vstr && vstr[0] ? vstr : (name.data ? name.data : "ans"),
01864         NAME.data ? NAME.data : "",MX,s_);
01865     }
01866     else {
01867         printf("\n%s File structure `s', %s\n",
01868         vstr && vstr[0] ? vstr : (name.data ? name.data : "ans"),
01869         NAME.data ? NAME.data : "",s_);
01870     }
01871
01872     if (!KK.isEmpty()) {
01873         if (!nl) { printf("\n"); ++nl; }
01874         for (i=0; i<KK.len; ++i) {
01875             sprintf(s_,"KK[%d]", i);
01876             KK[i].info(s_);
01877         }
01878     }
01879
01880     if (!KD.isEmpty()) {
01881         if (!nl) { printf("\n"); ++nl; }
01882         for (i=0; i<KD.len; ++i) {
01883             sprintf(s_,"KD[%d]", i);
01884             KD[i].info(s_);
01885         }
01886     }
01887     if (!DK.isEmpty()) {
01888         if (!nl) { printf("\n"); ++nl; }
01889         for (i=0; i<DK.len; ++i) {
01890             sprintf(s_,"DK[%d]", i);
01891             DK[i].info(s_);
01892         }
01893     }
01894     if (!DD.isEmpty()) {
01895         if (!nl) { printf("\n"); ++nl; }
01896         for (i=0; i<DD.len; ++i) {
01897             sprintf(s_,"DD[%d]", i);
01898             DD[i].info(s_);
01899         }
01900     }
01901
01902     nl=0;
01903
01904     if (!K.isEmpty()) { if (!nl) { printf("\n"); ++nl; }; K.info("K"); }
01905     if (!D.isEmpty()) { if (!nl) { printf("\n"); ++nl; }; D.info("D"); }
01906
01907     if (nl) printf("\n");
01908 };
01909
01910 template <class TQ, class TD>
01911 void NRGData<TQ,TD>::dispIdx(const char *F, int L) const {
01912
01913     char s[32];
01914     wblog(F,L,"%NNRGIndex %s (calc=%d, store=%d)%N", name.data, calc, store);
01915
01916     for (unsigned i=0; i<nrgIdx.len; ++i) {
01917         if (!nrgIdx[i].tag.isEmpty())
01918             strncpy(s, nrgIdx[i].tag.data,32);
01919         else if (nrgIdx[i].nrg)
01920             strcpy(s, nrgIdx[i].nrg==this ? "(this)" : "(that)");

```

```

01921         else if (nrgIdx[i].idx>=0)
01922             strcpy(s, "(file)");
01923         else strcpy(s, "(calc)");
01924
01925         printf("%6d: %4d  %s:%1X\n", i, nrgIdx[i].idx, s, nrgIdx[i].nrg);
01926     }
01927
01928     if (nrgIdx.len) printf("\n");
01929 }
01930
01931 template <class TQ, class TD>
01932 void NRGData<TQ,TD>::info(const char *F, int L, const char *istr) const {
01933
01934     unsigned i, nt1=0, nt2=0, nx1=0, nx2=0, nc=0;
01935     wstring mark(nrgIdx.len+1);
01936     wstring S(128);
01937
01938     for (i=0; i<nrgIdx.len; ++i) {
01939
01940         if (nrgIdx[i].nrg && nrgIdx[i].nrg!=this) { ++nx1; mark[i]='x'; } else
01941         if (!nrgIdx[i].tag.isEmpty()) { ++nx2; mark[i]='X'; } else
01942         if (nrgIdx[i].nrg==this) { ++nt1; mark[i]='r'; } else
01943         if (nrgIdx[i].idx>=0) { ++nt2; mark[i]='R'; }
01944         else { ++nc; mark[i]='*'; }
01945     }
01946
01947     if (nc) {
01948         S.pushf(FL, " %s%d/%d op%s",
01949             calc ? "calc " : "", nc, KK.len, KK.len!=1 ? "s":"" );
01950     }
01951     else {
01952         S.pushf(FL, " %d op%s%s", KK.len, KK.len!=1 ? "s":"",
01953             calc ? ", calc!?" : "", all from store or referenced");
01954     }
01955
01956     if (store) S.push(FL, " and store");
01957     else if (nc || calc) {
01958         S.push(FL, " (not stored)");
01959     }
01960
01961     if (kloc          ) S.pushf(FL, " nloc=%d", kloc+1);
01962
01963     if (strcmp(istr,name.data)) {
01964         wblog(F,L, " * %s (%3s): [%s] %s",
01965             name.data, istr, mark.data, S.data[0] ? S.data+2 : "");
01966     }
01967     else {
01968         wblog(F,L, " * %s [%s] %s",
01969             name.data, mark.data, S.data[0] ? S.data+2 : "");
01970     }
01971 };
01972
01973 template <class TQ, class TD>
01974 void NRGData<TQ,TD>::disp_otype(const char *F, int L) const {
01975
01976     wstring s(128); unsigned i=0;
01977
01978     if (name.data) s.cpy(FL,name.data);
01979
01980     if (KK.len) {
01981         s.pushf(FL, "%sKK: ", i?"": "");
01982         for (i=0; i<KK.len; ++i) s.pushf(FL, "%d", KK[i].otype);
01983     }
01984     if (DK.len) {
01985         s.pushf(FL, "%sTK: ", i?"": "");
01986         for (i=0; i<DK.len; ++i) s.pushf(FL, "%d", DK[i].otype);
01987     }
01988     if (KD.len) {
01989         s.pushf(FL, "%sKT: ", i?"": "");
01990         for (i=0; i<KD.len; ++i) s.pushf(FL, "%d", KD[i].otype);
01991     }
01992     if (DD.len) {
01993         s.pushf(FL, "%sTT: ", i?"": "");
01994         for (i=0; i<DD.len; ++i) s.pushf(FL, "%d", DD[i].otype);
01995     }
01996
01997     wblog(F_L, " * NRGData of type %s() %s ", FCT, s);
01998 };
01999
02000 template <class TQ, class TD, class TZ>
02001 void backPropagateOps(const char *F, int L,
02002     wvector< QSpace<TQ,TZ> > &BK,
02003     const wvector< QSpace<TQ,TZ> > &BX,
02004     const QSpace<TQ,TD> &A0,
02005     const QSpace<TQ,TD> &A2
02006 ){
02007     if (!BX.len) return;

```

```

02008     if (!BK.len) BK.init(BX.len); else
02009     if (BK.len!=BX.len) wblog(F_L,
02010         "ERR %s() length mismatch (%d/%d)",FCT,BK.len,BX.len);
02011
02012     unsigned i=0,r;
02013     QSpace<TQ,TZ> Q1,Q2;
02014
02015     for (; i<BX.len; ++i) { r=BX[i].rank(F_L);
02016         if (!r && !i) {
02017             for (; i<BX.len; ++i) { if (BX[i].rank(F_L)) break; }
02018             if (i==BX.len) return;
02019         }
02020         if (r<2 || r>3) wblog(F_L,
02021             "ERR %s() invalid rank r=%d (%d/%d) !?",FCT,r,i+1,BX.len);
02022
02023         A2.contract(F_L,"2*",BX[i],2,Q1);
02024         A0.contract(F_L,"23",Q1,"32",Q2);
02025         BK[i]+=Q2;
02026     }
02027 };
02028
02029 template <class TQ, class TZ>
02030 template <class TD>
02031 void NRGData<TQ,TZ>::backPropagateOp(const char *F, int L,
02032     const NRGData<TQ,TD> &A0,
02033     const NRGData<TQ,TD> &A2
02034 ){
02035     unsigned i=0, e=0;
02036     wvector< QSpace<TQ,TZ> > BK;
02037
02038     A2.check_dloc(F_LF,gRX.dloc,gRX.iter);
02039
02040     backPropagateOps(F,L,BK, KK, A0.K, A2.K);
02041     backPropagateOps(F,L,BK, KD, A0.K, A2.D);
02042     backPropagateOps(F,L,BK, DK, A0.D, A2.K);
02043     backPropagateOps(F,L,BK, DD, A0.D, A2.D);
02044
02045     for (; i<BK.len; ++i) { if (BK[i].isEmpty()) { ++e; }
02046         BK[i]*=(1./gRX.dloc);
02047         BK[i].SkipZeroData(DEPS2,'b');
02048     }
02049     if (e && gRX.iter>kloc) {
02050         MXPut(FL,"Ie").add(*this,"B02").add(gRX.iter,"iter").add(BK,"BK")
02051             .add(A0,"A0").add(A2,"A2");
02052         wblog(FL,"ERR %s() got %d/%d empty BK at iter=%d/%d !?",
02053             FCT,e,BK.len,gRX.iter,kloc);
02054     }
02055
02056     BK.save2(KK);
02057 };
02058
02059 template <class TQ, class TD> inline
02060 void updateOp_L(const char *F, int L,
02061     const QSpace<TQ,TD> &Xin,
02062     const QSpace<TQ,TD> &A1,
02063     const QSpace<TQ,TD> &A2,
02064     QSpace<TQ,TD> &Xout
02065 ){
02066     unsigned rk, r_=-1;
02067     QSpace<TQ,TD> Xk;
02068
02069     if (Xin.isEmpty()) { wblog(F_L,
02070         "ERR %s() got empty L-operator",FCT); return; }
02071     if (int(rk=Xin.isOperator(&r_))<=0) { Xin.info("L-op");
02072         wblog(F_L,"ERR %s() got invalid L-operator (%d/%d)",FCT,rk,r_);
02073     }
02074
02075     Xin.contract(F,L,2,A2,1,Xk);
02076     A1.contract(F,L,"1,3;*",Xk, (rk!=3 ? "1,3":"1,4"), Xout);
02077
02078     if (rk==3) Xout.Permute("1,3,2");
02079
02080     Xout.otype=Xin.otype;
02081     Xout.SkipZeroData();
02082 };
02083
02084 template <class TQ, class TD> inline
02085 void updateOp_s(const char *F, int L,
02086     const QSpace<TQ,TD> &x,
02087     const QSpace<TQ,TD> &A1,
02088     const QSpace<TQ,TD> &A2,
02089     QSpace<TQ,TD> &Xout
02090 ){
02091     unsigned rk, r_=-1;
02092     QSpace<TQ,TD> Xk;
02093
02094     if (x.isEmpty()) { wblog(FL,

```

```

02095     "WRN %s() got empty L-operator",FCT); return; }
02096     if (int(rk=x.isOperator(&r_))<=0) { x.info("local-op");
02097         wblog(FL,"ERR %s() got invalid L-operator (%d/%d)",FCT,rk,r_);
02098     }
02099
02100     A2.contract(F,L,3,x,2, Xk);
02101     A1.contract(F,L,"1,3;*", Xk,"1,3", Xout);
02102
02103     Xout.otype=x.otype;
02104     Xout.SkipZeroData();
02105 };
02106
02107 template <class TQ, class TD>
02108 inline void updateOp_Ls(
02109     const QSpace<TQ,TD> &Xin,
02110     const QSpace<TQ,TD> &A1,
02111     const QSpace<TQ,TD> &A2,
02112     QSpace<TQ,TD> &Xout
02113 ){
02114     unsigned rk, r_=-1;
02115     QSpace<TQ,TD> Xk;
02116
02117     if (Xin.isEmpty()) { wblog(FL,
02118         "WRN %s() got empty L-operator",FCT); return; }
02119     if ((rk=Xin.isOperator(&r_,'x')!=4) { Xin.info("Ls-op");
02120         wblog(FL,"ERR %s() got invalid Ls-operator (%d/%d)",FCT,rk,r_);
02121     }
02122
02123     Xin.contract("3,4",A2,"1,3", Xk);
02124     A1.contract("1,3;*",Xk,"1,2",Xout);
02125
02126     Xout.otype=Xin.otype;
02127     Xout.SkipZeroData();
02128 };
02129
02130 template <class TQ, class TD>
02131 inline void updateOp_sL(
02132     const QSpace<TQ,TD> &Xin,
02133     const QSpace<TQ,TD> &A1,
02134     const QSpace<TQ,TD> &A2,
02135     QSpace<TQ,TD> &Xout
02136 ){
02137     unsigned rk, r_=-1;
02138     QSpace<TQ,TD> Xk;
02139
02140     if (Xin.isEmpty()) { wblog(FL,
02141         "WRN %s() got empty L-operator",FCT); return; }
02142     if ((rk=Xin.isOperator(&r_))!=4) { Xin.info("sL-op");
02143         wblog(FL,"ERR %s() got invalid sL-operator (%d)",FCT,rk,r_);
02144     }
02145
02146     Xin.contract("4,3",A2,"1,3", Xk);
02147     A1.contract("1,3;*",Xk,"2,1",Xout);
02148
02149     Xout.otype=Xin.otype;
02150     Xout.SkipZeroData();
02151 };
02152
02153 template <class TQ, class TD> inline
02154 void updateOp_loc(const char *F, int L,
02155     const NRGData<TQ,TD> &NRG, unsigned i,
02156     const QSpace<TQ,TD> &XL,
02157     const QSpace<TQ,TD> &x,
02158     const QSpace<TQ,TD> &A1,
02159     const QSpace<TQ,TD> &A2,
02160     QSpace<TQ,TD> &Xout,
02161     unsigned iter
02162 ){
02163     bool o1=0, o2=0;
02164     unsigned rk, r_=-1;
02165     QSpace<TQ,TD> Ax,XA;
02166
02167     if (!x.isEmpty()) {
02168         if (int(rk=x.isOperator(&r_))<=0) { x.info("local_op"); wblog(FL,
02169             "ERR %s() got invalid local operator (%d/%d)",FCT,rk,r_); }
02170         if (XL.otype!=x.otype && !XL.isEmpty()) wblog(FL,
02171             "WRN %s() got different operator types (%s; %s)",
02172             FCT, QS_STR[XL.otype],QS_STR[x.otype]);
02173         o2=(rk==3);
02174
02175         wblog(FL," * contracting %s[%d] onto A(%d).%s",
02176             NRG.name.data,i, iter, A2.itags.toStrk(str,8,2,"s"));
02177
02178         A2.contract(F,L,3,x,"-op",Ax);
02179     }
02180     else Ax.init2ref(A2);
02181

```



```

02182     if (iter || !XL.isEmpty()) { r_=-1;
02183     if (int(rk=XL.isOperator(&r_))<=0) { x.info("L-op"); wblog(FL,
02184     "ERR %s() got invalid L-operator (%d/%d)",FCT,rk,r_); }
02185     o1=(rk==3);
02186
02187     if (iter<1) wblog(FL,"TST contracting onto A[%d].%s",
02188     iter, A2.itags.toStrk(str,8,0,"L"));
02189
02190     if (o1 && o2) { o1=o2=0;
02191     XL.contract(F,L,"23",Ax,"14",XA); }
02192     else {
02193     XL.contract(F,L,"-op",Ax,1,XA);
02194     }
02195     }
02196     else XA.init2ref(Ax);
02197
02198     if (o1)
02199     A1.contract(F,L,"13*",XA,"14", Xout,"132");
02200     else A1.contract(F,L,"13*",XA,"13", Xout);
02201
02202     if (o1) Xout.otype=XL.otype; else
02203     if (o2) Xout.otype=x .otype;
02204
02205     Xout.SkipZeroData();
02206 };
02207
02208 template <class TQ, class TD>
02209 int Wb::updateOp(const char *F, int L,
02210     const NRGData<TQ,TD> &NRG,
02211     wbvector< QSpace<TQ,TD> > &F12,
02212     const QSpace<TQ,TD> &A1,
02213     const QSpace<TQ,TD> &A2,
02214     const wbvector< QSpace<TQ,TD> > &FKK,
02215     unsigned iter
02216 ){
02217     unsigned i, j=0, m=0, r_=-1, rk=-1, nop=FKK.len;
02218     int e=0, il=0;
02219
02220     const wbvector< wbvector<QSpace<TQ,TD> > > &CI=NRG.CI;
02221     const wbvector< NRGIndex<TQ,TD> > &nrgIdx=NRG.nrgIdx;
02222
02223     if (A1.isEmpty() || A2.isEmpty()) { F12.initDef(nop); return 0; }
02224
02225     if (nrgIdx.len!=nop) wblog(FL,
02226     "ERR %s() severe size mismatch (nrgIdx: %d,%d)",FCT,nrgIdx.len,nop);
02227
02228     if (iter>0) {
02229     for (i=0; i<nop; ++i) {
02230     if (nrgIdx[i].idx>=0) continue;
02231     if (! (il=FKK[i].isConsistent(FL))
02232     || int(rk=FKK[i].isOperator(&r_))<=0) break;
02233     }
02234     }
02235     else if (CI.isEmpty()) {
02236     for (i=0; i<nop; ++i) {
02237     if (! (il=FKK[i].isConsistent(FL)) ||
02238     int(rk=FKK[i].isOperator(&r_,'x'))<=0
02239     ){ wblog(FL,
02240     "WRN FKK[%d] -> (%d; %d/%d/%d) ",i+1,il,rk,r_,FKK[i].rank());
02241     break;
02242     }
02243     }
02244     }
02245     else {
02246     if (nop!=CI.len) wblog(FL,
02247     "ERR size inconsistency (%d/%d)",nop,CI.len);
02248
02249     for (i=0; i<nop; ++i) { m=CI[i].len;
02250     for (j=0; j<m; ++j) { if (!CI[i][j].isEmpty()) {
02251     if (! (il=CI[i][j].isConsistent(FL)) ||
02252     int(rk=CI[i][j].isOperator(&r_))<=0) break;
02253     }}}
02254     }
02255
02256     if (i<nop || j<m) wblog(F,L,
02257     "ERR invalid rank-%d operator (iter=%d, op=%d/%d)",rk,iter,i+1,nop);
02258     if (A1.rank(F,L)!=3 || A2.rank(F,L)!=3 || A1.QDIM!=A2.QDIM) wblog(F,L,
02259     "ERR %s() severe rank inconsistency (%s; %s)",
02260     FCT, A1.sizeStrQ().data, A2.sizeStrQ().data);
02261
02262     if (F12.len!=nop)
02263     F12.initDef(nop);
02264
02265     if ((int)iter>NRG.kloc) {
02266     for (i=0; i<nop; ++i) {
02267     if (nrgIdx[i].idx>=0) continue;
02268     updateOp_L(F,L, FKK[i], A1, A2, F12[i] );

```

```

02269         wbttop().runningLarge(FL);
02270
02271         if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
02272     }
02273 }
02274 else if (!CI.isEmpty()) {
02275     QSpace<TQ,TD> E;
02276
02277     for (i=0; i<nop; ++i) { if (nrgIdx[i].idx>=0) continue;
02278         try { updateOp_loc(F,L,NRG,i,
02279             FKK[i], iter+2>CI[i].len ? E : CI[i][iter+1],
02280             A1, A2, F12[i],iter);
02281         }
02282         catch (...) { wblog_ERR_updateOp(FL,"loc",i,nop,iter); }
02283
02284         if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
02285     }
02286 }
02287 else {
02288     char Lflag = A1.getDim(1)>1 || A2.getDim(1)>1;
02289     unsigned l=0, tlen=16; char tag[tlen];
02290
02291     wbsting mark(nop,'*');
02292
02293     static time_t last_call_iter0=0;
02294     time_t tnow=time(0);
02295
02296     for (i=0; i<nop; ++i) {
02297         if (nrgIdx[i].idx>=0) continue;
02298         rk=FKK[i].rank(FL);
02299
02300         if (rk==2 || (rk==3 && FKK[i].otype==QS_OPERATOR)) {
02301             if (Lflag &&
02302                 FKK[i].hasQOverlap(1,A1,1,'<')>0 &&
02303                 FKK[i].hasQOverlap(2,A1,1,'<')>0
02304             ){
02305                 try { updateOp_L(F,L,FKK[i],A1,A2, F12[i]); }
02306                 catch (...) { wblog_ERR_updateOp(FL,"L",i,nop,iter); }
02307                 mark[i]='L';
02308             }
02309             else
02310             if (FKK[i].hasQOverlap(1,A1,3,'<')>0 &&
02311                 FKK[i].hasQOverlap(2,A1,3,'<')>0
02312             ){
02313                 try { updateOp_s(F,L,FKK[i], A1, A2, F12[i]); }
02314                 catch (...) { wblog_ERR_updateOp(FL,"s",i,nop,iter); }
02315                 mark[i]='s';
02316             }
02317             else
02318             if (FKK[i].hasQOverlap(1,A1,2,'<')>0 &&
02319                 FKK[i].hasQOverlap(2,A1,2,'<')>0
02320             ){
02321                 mark[i]='R';
02322             }
02323             else wblog(FL,
02324                 "ERR %s() failed to deal input operator with A%d",FCT,iter);
02325         }
02326         else if (rk==4) {
02327             wvector<int> m1,m2;
02328             int mm[]={
02329                 FKK[i].hasQOverlap(1,A1,1,'<')>0,
02330                 FKK[i].hasQOverlap(2,A2,3,'<')>0,
02331                 FKK[i].hasQOverlap(3,A2,1,'<')>0,
02332                 FKK[i].hasQOverlap(4,A2,3,'<')>0,
02333
02334                 FKK[i].hasQOverlap(1,A1,3,'<')>0,
02335                 FKK[i].hasQOverlap(2,A2,1,'<')>0,
02336                 FKK[i].hasQOverlap(3,A2,3,'<')>0,
02337                 FKK[i].hasQOverlap(4,A2,1,'<')>0
02338             };
02339             m1.init(4,mm); m2.init(4,mm+4);
02340
02341             if (m1.allUnequal(0)) {
02342                 try { updateOp_Ls(FKK[i], A1, A2, F12[i]); }
02343                 catch (...) { wblog_ERR_updateOp(FL,"Ls",i,nop,iter); }
02344                 mark[i]='D';
02345             }
02346             else
02347             if (m2.allUnequal(0)) {
02348                 try { updateOp_sL(FKK[i], A1, A2, F12[i]); }
02349                 catch (...) { wblog_ERR_updateOp(FL,"sL",i,nop,iter); }
02350                 mark[i]='d';
02351             }
02352             else {
02353                 if (STRICT_ITER0) { mark[i]='?';
02354                     if (mark.count('?')==1 && last_call_iter0+5<tnow){

```

```

02356         wblog(FL,
02357             "WRN %NFailed to fully associate rank-4 tensor product\n"
02358             "operator with first site. [ %s; %s ]\n",
02359             "A1: [ %s ]; A2: [ %s ]%N",
02360             "NB! Hint: A0 may not be complete due to truncation "
02361             "in NRGWilsonQS already at iter=0 !?%N",
02362             m1.toStr().data, m2.toStr().data,
02363             SSTR(A1.getDim()), SSTR(A2.getDim()));
02364     }}
02365
02366     try { updateOp_Ls(FKK[i], A1, A2, F12[i]); }
02367     catch (...) { wblog_ERR_updateOp(FL,"Ls",i,nop,iter); }
02368 }
02369
02370 else wblog(FL,
02371 "ERR %s() invalid rank-%d operator (iter=%d)",FCT,rk,iter);
02372
02373 if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
02374 }
02375
02376 l=snprintf(tag,tlen,"FDM %2d/%d",iter,NRG_N); {
02377     if (NRG.name.data && NRG.name.data[0] && l<tlen)
02378         l+=snprintf(tag+l,tlen-l, " %s:",NRG.name.data);
02379     if (l>=tlen) wblog(FL,
02380 "ERR %s() string out of bounds (%d/%d)",FCT,l,tlen);
02381 }
02382
02383 if (!nop)
02384     wblog(FL,"%s got empty spectral ops",tag);
02385 else {
02386     unsigned il=0, m=mark.count('*');
02387     for (i=0; i<nop; ++i) { if (mark[i]!='*') { il=i; break; }}
02388     for (; i<nop; ++i) { if (mark[i]!='*') {
02389         if (mark[i]!=mark[il]) break;
02390     }}
02391
02392     if (m==nop) { wblog(FL,
02393 " %s no spectral ops to contract (all referenced)",
02394 tag,mark.data);
02395     }
02396     else if (i==nop && !m) {
02397         char ostr[32];
02398         if (nop>1)
02399             sprintf(ostr,"all %d spectral ops",nop);
02400         else sprintf(ostr,"spectral operator");
02401
02402         if (mark[0]!='R') wblog(FL,
02403 " %s contracting %s onto %c",tag,ostr,mark[0]);
02404         else wblog(FL,
02405 " %s %s already assumed in %c",tag,ostr,mark[0]
02406 );
02407     }
02408     else if (i==nop && m) {
02409         if (mark[il]!='R') wblog(FL,"%s contracting %d/%d "
02410 "spectral ops onto [%s]",tag,nop-m,nop,mark.data);
02411         else wblog(FL,"%s all %d/%d spectral "
02412 "ops already assumed in %s",tag,nop-m,nop,mark.data);
02413     }
02414     else {
02415         wblog(FL,"%s contracting "
02416 "%d/%d spectral ops onto [%s]",tag,nop-m,nop,mark.data);
02417     }
02418 }
02419
02420 last_call_iter0=tnow;
02421 }
02422
02423 if (iter==0) {
02424     for (i=0; i<F12.len; ++i) {
02425         if (F12[i].itags.len>2 && !F12[i].itags[2]) {
02426             F12[i].itags[2].init("op");
02427         }
02428     }
02429 }
02430
02431 wbttop().runningLarge(FL);
02432 return e;
02433 };
02434
02435 template <class TQ, class TD>
02436 unsigned applyZ0(const QSpace<TQ,TD> &Z0, QSpace<TQ,TD> &CK) {
02437
02438     unsigned r=CK.rank(FL);
02439     const QS_TYPES otype=CK.otype;
02440
02441     if (r==2 || r==3) {
02442         if (r==3 && CK.otype!=QS_OPERATOR) wblog(FL,

```

```

02443         "ERR %s() expecting otype==%s for rank-%d op (%s)",
02444         FCT, QS_STR[QS_OPERATOR], r, QS_STR[CK.otype]);
02445     if (Z0.rank(FL) != 2) wblog(FL,
02446         "ERR %s() Z0 of rank-2 required for rank-%d op (%d)",
02447         FCT, r, Z0.rank(FL)
02448     );
02449
02450     if (Z0.hasQOverlap(2, CK, 1, '>') <= 0) {
02451         wblog(FL, "ERR %s() Z0 does not match operator space", FCT);
02452         return 1;
02453     }
02454     Z0.contract(2, CK, 1, CK);
02455     CK.otype = otype;
02456 }
02457 else if (r == 4) {
02458     if (Z0.rank(FL) != 4) wblog(FL,
02459         "ERR tensor Z0 required for tensor operator (rank=%d/%d)",
02460         Z0.rank(FL), r
02461     );
02462
02463     if (Z0.hasQOverlap(3, CK, 1, '>') <= 0 ||
02464         Z0.hasQOverlap(4, CK, 2, '>') <= 0) {
02465         wblog(FL, "ERR %s() Z0 does not match operator space", FCT);
02466         return 2;
02467     }
02468     Z0.contract("3 4", CK, "1 2", CK);
02469     CK.otype = otype;
02470 }
02471 else wblog(FL, "ERR %s() invalid rank=%d.", FCT, r);
02472
02473 return 0;
02474 };
02475
02476 template <class TQ, class TD>
02477 unsigned getDLoc(
02478     NRGData<TQ, TD> &A, Wb::SigHandler *sig=NULL, unsigned *Dk=NULL,
02479     char fflag=0
02480 ){
02481     unsigned iter, N=NRG_N; widx_t d0=0, d=0;
02482     char cgflag=-1;
02483
02484     if (Dk) { (*Dk)=0; }; printf("\r");
02485
02486     for (iter=0; iter<N; ++iter) { if (sig) sig->check911();
02487         printf(" %s: %2d/%d loading AK ... \r", FCT, iter, N);
02488         fflush(0);
02489
02490         A.init(FL, "K", iter);
02491
02492         if (iter==0) cgflag=A.K.gotCGS(FL);
02493
02494         if (A.K.isEmpty()) {
02495             if (iter==N-1) continue; else wblog(FL,
02496                 "ERR %s() empty AK at intermediate iteration %d/%d", FCT, iter, N);
02497         }
02498         else {
02499             unsigned r=-1;
02500             A.K.isConsistent(FL, r);
02501             if (cgflag<=1 && !A.K.QIDX.isUnique()) wblog(FL,
02502                 "ERR invalid AK at iteration %d/%d", iter, N);
02503         }
02504
02505 #ifndef WB_SKIP_ASSERT
02506         if (fflag) { QSpace<TQ, TD> E;
02507             A.K.contract("1, 3;*", A.K, "1, 3", E);
02508             if (!E.isIdentityMatrix(1E-12)) {
02509                 MXPput(FL).add(A, "A").add(E, "E");
02510                 wblog(FL, "ERR NRG[%d].AK not in LR's index order", iter, N);
02511             }
02512         }
02513 #endif
02514
02515         A.K.getDim(2, &d);
02516
02517         if (iter>2 || d0) {
02518             if (d!=d0) {
02519                 MXPput(FL, "a").add(A.K, "K").add(d, "d").add(d0, "d0")
02520                     .add(iter, "iter").add(N, "N");
02521                 wblog(FL, "ERR %s() inconsistent local dimension "
02522                     "(d=%d/%d @ k=%d/%d)", FCT, d, d0, iter, N
02523                 );
02524             }
02525         }
02526         else if (iter==2 || iter+2==N) { d0=d; }
02527         else {
02528             A.init(FL, "D", iter);
02529             if (!A.D.isEmpty()) { d0=d; }

```

```

02530     }
02531
02532     if (Dk) {
02533         d=A.K.getDim(1);
02534         if (*Dk<d) { *Dk=d; continue; }
02535     }
02536     if (!fflag && iter>2) break;
02537 }
02538
02539 if (!d0) wblog(FL,"ERR %s() "
02540 "failed to get local dimension (d=%d; %d/%d)",FCT,d0,iter,N);
02541 return d0;
02542 };
02543
02544 #endif
02545

```

5.17 QSpace.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_QSPACE_COL_MAJOR_HH__
00021 #define __WB_QSPACE_COL_MAJOR_HH__
00022
00023 /* ----- //
00024  QSpace - set of objects (possibly complex matrices)
00025  organized by generalized index set stored as row vectors (recs)
00026  in index matrix. // Wb,Aug08,05 Apr07,06
00027
00028  Introduction of non-abelian QSpace through variable 'qtype'.
00029  see also clebsch.hh // Wb,Sep25,09
00030
00031  NB! QSpace currently only implemented on linux system
00032  (matlab+gcc)
00033
00034  Trying to get it run on a macbook // Wb,Jan18,19
00035  Problems encountered:
00036  * OSX, by default, has case-*IN*sensitive files system (APFS)
00037  encountered trouble with wbcmat.hh + wbCMat.hh
00038  * default "gcc" compiler actually is with Xcode
00039  which is much more picky concerning header files!
00040
00041  // ----- */
00042
00043 #ifdef WB_CLOCK
00044 #define WBC_QSPACE_IO
00045 #endif
00046
00047 enum QS_TYPES {
00048     QS_NONE, QS_OPERATOR, QS_AMATRIX,
00049     QS_NUM_TYPES
00050 };
00051
00052 const char* QS_STR[QS_NUM_TYPES] = {
00053     "", "operator", "A-matrix",
00054 };
00055
00056 template <class TQ, class TD>
00057 class QSpace {
00058
00059 public:
00060
00061     QSpace(unsigned n=0)
00062         : QDIM(n), otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.) {};
00063
00064     QSpace(unsigned m, unsigned n, unsigned nq)

```

```

00065     : QSpace(nq) { init(m,n,nq); };
00066
00067 QSpace(const QSpace &B)
00068     : QIDX(B.QIDX), CGR(B.CGR), qtype(B.qtype), QDIM(B.QDIM),
00069       otype(QS_NONE), itags(B.itags), mt(Wb::MEM_DEF),
00070       isref(0), ctime(B.ctime)
00071 {
00072     if (this!=&B) {
00073         unsigned i=0; setupDATA();
00074         for (; i<B.DATA.len; ++i) { DATA[i]->init(*(B.DATA[i])); }
00075     }
00076 };
00077
00078 template<class TQ_, class TD_>
00079 QSpace(const QSpace<TQ_,TD_> &B,
00080        char xflag = WbUtil<TD_>::isComplex() ? 1 : 0
00081 ): QIDX(B.QIDX), CGR(B.CGR), qtype(B.qtype), QDIM(B.QDIM),
00082    otype(QS_NONE), itags(B.itags), mt(Wb::MEM_DEF),
00083    isref(0), ctime(B.ctime)
00084 {
00085     if (!xflag) wblog(FL,"ERR %s() verify type cast %s -> %s",
00086                     FCT,TSTR(TD_),TSTR(TD_));
00087     setupDATA();
00088     for (unsigned i=0; i<B.DATA.len; ++i) {
00089         DATA[i]->initT(*(B.DATA[i]));
00090     }
00091 };
00092
00093 QSpace( const char *F, int L, const mxArray *a,
00094        char ref=0, char skip_empty=1)
00095     : otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.)
00096     { init(F,L,a,ref,skip_empty); };
00097
00098 QSpace(const mxArray *a, char ref=0, char skip_empty=1)
00099     : otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.)
00100     { init(FL,a,ref,skip_empty); };
00101
00102 ~QSpace() { clearQSpace(); };
00103
00104 QSpace& clearQSpace(char mflag=0) {
00105     if (QIDX.data || DATA.data) { mt=Wb::MEM_DEF; }
00106     QIDX.init(); qtype.init(); itags.init();
00107     clearDATA(mflag); CGR.init(); isref=0; ctime=0.;
00108     return *this;
00109 };
00110
00111 void clearDATA(char mflag=0) {
00112     if (DATA.len) {
00113         if (!isref && !DATA.isref) {
00114             for (unsigned i=0; i<DATA.len; ++i) { if (DATA[i]) {
00115                 if (mflag) { DATA[i]->initX(); }
00116                 WB_DELETE_1(DATA[i]);
00117             } }
00118         }
00119         DATA.init();
00120     }
00121 };
00122
00123 unsigned rank(const char *F=NULL, int L=0) const {
00124     unsigned r=0;
00125     if (QDIM) { r=QIDX.dim2/QDIM;
00126         if (QIDX.dim2!=r*QDIM) wblog(F_L,
00127             "ERR %s() QSpace inconsistency (QIDX: %dx%d @ %d)",
00128             FCT,QIDX.dim1,QIDX.dim2,QDIM);
00129     }
00130     else if (QIDX.dim2) wblog(F_L,
00131         "ERR %s() QSpace inconsistency (QIDX: %dx%d @ %d)",
00132         FCT,QIDX.dim1,QIDX.dim2,QDIM);
00133     return r;
00134 };
00135
00136 unsigned len() const {
00137     if (QIDX.dim1!=DATA.len) wblog(FL,
00138         "ERR qspace inconsistency (QIDX.dim1=%d, len=%d).",
00139         QIDX.dim1, DATA.len);
00140     return QIDX.dim1;
00141 };
00142
00143 unsigned nsym() const {
00144     if (CGR.dim2 && qtype.len!=CGR.dim2) wblog(FL,
00145         "ERR qspace inconsistency (nsym=%d ' %s'; %s !?)",
00146         qtype.len, STR(qtype), SSTR(CGR));
00147     return (qtype.len ? qtype.len : QDIM);
00148 };
00149
00150 TQ* Qref(unsigned r, unsigned k) { unsigned l=k*QDIM;

```

```

00152         if (!QDIM || r>=QIDX.dim1 || (1+QDIM)>QIDX.dim2) wblog(FL,
00153             "ERR %s() index out of bounds (%d/%d; %d*%d/%d)",
00154             FCT,r,QIDX.dim1,k,QDIM,QIDX.dim2
00155         );
00156         return (QIDX.data + r*QIDX.dim2 + 1);
00157     };
00158
00159     const TQ* Qref(unsigned r, unsigned k) const {
00160         unsigned l=k*QDIM;
00161         if (!QDIM || r>=QIDX.dim1 || (1+QDIM)>QIDX.dim2) wblog(FL,
00162             "ERR %s() index out of bounds (%d/%d; %d*%d/%d)",
00163             FCT,r,QIDX.dim1,k,QDIM,QIDX.dim2
00164         );
00165         return (QIDX.data + r*QIDX.dim2 + 1);
00166     };
00167
00168     void init() { clearQSpace(); };
00169
00170     void init(unsigned m, unsigned n, unsigned nq) {
00171         clearQSpace();
00172         QDIM=nq; QIDX.init(m,nq*n);
00173         setupDATA();
00174     };
00175
00176     void init(const wbMatrix<TQ> &Q, unsigned qdim, const QVec *qt=NULL) {
00177         clearQSpace();
00178         QIDX=Q; QDIM=qdim; if (qt) qtype=(*qt); else qtype.init();
00179         setupDATA(FL);
00180     };
00181
00182     QSpace& init(const QSpace &B) {
00183         initQT_safe(B);
00184         for (unsigned i=0; i<B.DATA.len; ++i) {
00185             DATA[i]->init(*(B.DATA[i]));
00186         }
00187         return *this;
00188     };
00189
00190     template<class TB>
00191     QSpace& initT(const QSpace<TQ,TB> &B) {
00192         initQT_safe(B);
00193         for (unsigned i=0; i<B.DATA.len; ++i) {
00194             DATA[i]->initT(*(B.DATA[i]));
00195         }; return *this;
00196     };
00197
00198     template<class TR>
00199     QSpace& getReal(QSpace<TQ,TR> &R) const {
00200         unsigned i=0; R.initQT_safe(*this);
00201         for (; i<DATA.len; ++i) { DATA[i]->getReal(*(R.DATA[i])); }
00202         return R;
00203     };
00204
00205     QSpace<TQ,double> getReal() const {
00206         QSpace<TQ,double> R; unsigned i=0; R.initQT_safe(*this);
00207         for (; i<DATA.len; ++i) { DATA[i]->getReal(*(R.DATA[i])); }
00208         return R;
00209     };
00210
00211     template<class TI>
00212     QSpace& getImag(QSpace<TQ,TI> &I) const {
00213         unsigned i=0; I.initQT_safe(*this);
00214         for (; i<DATA.len; ++i) { DATA[i]->getImag(*(I.DATA[i])); }
00215         return I;
00216     };
00217
00218     QSpace<TQ,double> getImag() const {
00219         QSpace<TQ,double> I; unsigned i=0; I.initQT_safe(*this);
00220         for (; i<DATA.len; ++i) { DATA[i]->getImag(*(I.DATA[i])); }
00221         return I;
00222     };
00223
00224     void init(const char *F, int L,
00225         const mxArray *S, char ref, unsigned k, char skip_empty=1);
00226
00227     void init(const char *F, int L, const mxArray *S,
00228         char ref=0, char skip_empty=1
00229     ) {
00230         unsigned n = (S ? mxGetNumberOfElements(S) : 0);
00231         if (n!=1) {
00232             if (!n) wblog(FL,"ERR %s() got empty array",FCT);
00233             else wblog(FL,"ERR %s() got QSpace array (%d entries)",FCT,n);
00234         }
00235         init(F,L,S,ref,0,skip_empty);
00236     };
00237
00238     QSpace& setupDATA(const char *F=NULL, int L=0){

```

```

00239
00240     if (!QIDX.dim1) { clearDATA(); return *this; }
00241
00242     if (QIDX.dim2 && (!QDIM || QIDX.dim2%QDIM)) wblog(FL,
00243         "ERR invalid QIDX (%dx%d; %d)", QIDX.dim1, QIDX.dim2, QDIM);
00244     clearDATA();
00245
00246     if (isref>1) wblog(FL, "ERR %s() got isref=%d", FCT, isref);
00247
00248     DATA.init(QIDX.dim1);
00249
00250     if (!isref) {
00251         if (mt) { if (mt==Wb::MEX_RETURN) {
00252             if (WBLOG_MMEX) wblog(FL,
00253                 "TST QSpace::init() to %s", Wb::MTYPE_STR[mt]); }
00254             else wblog(FL,
00255                 "ERR %s() unexpected mtype=%s", FCT, Wb::MTYPE_STR[mt]);
00256         }
00257
00258         for (unsigned i=0; i<DATA.len; ++i) {
00259             WB_NEW_1(DATA[i]);
00260             DATA[i]->mtype=mt;
00261         }
00262     }
00263
00264     return *this;
00265 };
00266
00267 int gotCGS(const char *F=NULL, int L=0, char xflag=0) const;
00268
00269 int gotCGX(const char *F=NULL, int L=0) const;
00270
00271 QSpace& Enlarge(unsigned m) {
00272     if (!isConsistent(FL) || !QIDX.dim1) wblog(FL,
00273         "ERR %s() can't enlarge empty QSpace (%d)", FCT, QIDX.dim1);
00274     if (!m) return *this;
00275
00276     unsigned i, n0=QIDX.dim1, n=n0+m;
00277     gotCGS(FL);
00278
00279     QIDX.Resize(n, QIDX.dim2);
00280     DATA.Resize(n);
00281     for (i=n0; i<n; ++i) { WB_NEW_1(DATA[i]); }
00282
00283     CGR.Resize(n, CGR.dim2);
00284     return *this;
00285 };
00286
00287 QSpace& Trim(unsigned n) {
00288     unsigned i, r=-1, n2=QIDX.dim1;
00289     if (!isConsistent(FL, r, 0) || n>QIDX.dim1) wblog(FL,
00290         "ERR %s() can't trim to LARGER QSpace (%d/%d)", FCT, n, QIDX.dim1);
00291     if (n==QIDX.dim1) { return *this; }
00292
00293     gotCGS(FL);
00294
00295     QIDX.Resize(n, QIDX.dim2);
00296
00297     for (i=n; i<n2; ++i) { WB_DELETE_1(DATA[i]); }
00298     DATA.Resize(n);
00299
00300     CGR.Resize(n, CGR.dim2);
00301
00302     return *this;
00303 };
00304
00305 QSpace& initScalar(const TD &x) {
00306     clearQSpace(); DATA.init(1);
00307     QIDX.init(1, 0);
00308
00309     WB_NEW_2(DATA[0], wbararray<TD>(1, 1));
00310     DATA[0]->data[0]=x;
00311
00312     return *this;
00313 };
00314
00315 void init2ref(const mxArray *S) {
00316     init(FL, S, 'r', 0);
00317 };
00318
00319 void init2ref(const QSpace &);
00320
00321 void initOpZ_WET(
00322     const char *F, int L, const QVec &qvec,
00323     const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2,
00324     const wbMatrix<TQ> &Q, const wbararray<TD> &D3,
00325     const double eps1=1E-10,

```



```

00326     const double eps2=1E-13
00327 );
00328
00329 void reduceMatEl(const char *F, int L,
00330     const wbMatrix<TQ> &Q1, const wbvector<TD> &dd,
00331     const wbMatrix<TQ> &Q2, const wbvector<TD> &dc,
00332     wbvector<unsigned> &I1, wbvector<unsigned> &I2,
00333     wbvector<TD> &dr
00334 ) const;
00335
00336 void unRef() {
00337
00338     if (!isref && !qtype.isref && !itags.isref &&
00339         !QIDX.isref && !DATA.isref && !CGR.isref
00340     ) return;
00341
00342     wblog(FL,"WRN %s() check this [%d%d%d%d]",FCT, isref,
00343         QIDX.isref, DATA.isref, CGR.isref, qtype.isref, itags.isref);
00344
00345     QIDX.unRef(); qtype.unRef(); itags.unRef();
00346
00347     if (DATA.isref) DATA.unRef(); else
00348         for (unsigned i=0; i<DATA.len; ++i) DATA[i]->unRef();
00349
00350     if (CGR.isref) CGR.unRef();
00351 };
00352
00353 void initIdentity(const QSpace &A, char dflag=0);
00354 void initIdentity(const wbMatrix<TQ> &Q);
00355
00356 QSpace& setupCGR(const char *F=NULL, int L=0);
00357
00358 QSpace& initIdentityCGS(const char *F=NULL, int L=0);
00359 QSpace& initIdentityCGS(const char *F, int L, unsigned k, unsigned r=0);
00360
00361 template<class TD2>
00362 QSpace& initIdentityCGS(const char *F, int L, const QSpace<TQ,TD2>& H);
00363
00364 template<class TA>
00365 QSpace<TQ,TD>& initIdentityCG(
00366     const wbvector< QSpace<TQ,TA> > &A, const wbindex &ia,
00367     char zflag=0
00368 );
00369
00370 template<class TA, class TB>
00371 QSpace<TQ,TD>& initIdentityCG(
00372     const wbvector< QSpace<TQ,TA> > &A, const wbindex &ia,
00373     const wbvector< QSpace<TQ,TB> > &B, const wbindex &ib,
00374     char vflag='v'
00375 );
00376
00377 template<class TA, class TB>
00378 QSpace<TQ,TD>& initIdentityCG(
00379     const wbvector< QSpace<TQ,TA> > &A,
00380     const wbvector< QSpace<TQ,TB> > &B,
00381     const char *pstr=NULL
00382 ){ wbindex i;
00383     initIdentityCG(A,i,B,i);
00384     if (pstr && pstr[0]) Permute(wbperm(pstr));
00385     return *this;
00386 }
00387
00388 QSpace& initDiagonal(
00389     const wbMatrix<TQ> &Q,
00390     const wbvector<unsigned> &S,
00391     const wbvector<TD> &D,
00392     char rcflag
00393 );
00394
00395 QSpace& init2DiffOp(
00396     const QSpace &A, unsigned ia,
00397     const QSpace &B, unsigned ib
00398 );
00399
00400 template<class TB>
00401 QSpace& initQT_safe(const QSpace<TQ,TB> &B) {
00402
00403     clearQSpace();
00404     QIDX=B.QIDX; QDIM=B.QDIM; qtype=B.qtype; otype=B.otype;
00405     setupDATA(); itags=B.itags; CGR=B.CGR;
00406
00407     if (CGR.dim2) {
00408         if (qtype.len!=CGR.dim2) wblog(FL,
00409             "ERR %s() got CGR size mismatch (%dx%d <> %dx%d)",
00410             FCT,CGR.dim1,CGR.dim2,QIDX.dim1,qtype.len
00411         );
00412     }

```

```

00413         else if (QIDX.data) {
00414             if (DATA.len && !qtype.allAbelian()) wblog(FL,
00415                 "ERR %s() missing CGR data for %s (%dx%d <> %dx%d)",
00416                 FCT,STR(qtype),CGR.dim1,CGR.dim2,QIDX.dim1,qtype.len
00417             );
00418         }
00419         return *this;
00420     };
00421
00422     template <class TA>
00423     QSpace& initQ(const QSpace<TQ,TA> &A, char full=1){
00424         if (isref) wblog(FL,"WRN %s() got QSpace ref!",FCT);
00425         qtype=A.qtype; QDIM=A.QDIM; if (full) {
00426             otype=A.otype; itags=A.itags; }
00427         return *this;
00428     };
00429
00430     template <class TA, class TB>
00431     QSpace& initQ(const char *F, int L,
00432         const QSpace<TQ,TA> &A, char full=1, const QSpace<TQ,TB> *B=NULL
00433     ){
00434         if (B) A.checkQ(F,L,*B);
00435         return initQ(A,full);
00436     };
00437
00438     void initQ(unsigned n, unsigned r, const QSpace &A){
00439         clearQSpace();
00440         QDIM=A.QDIM; qtype=A.qtype; QIDX.init(n,r*QDIM);
00441         setupDATA(FL);
00442         if (!qtype.allAbelian()) setupCGR(FL);
00443     };
00444
00445     void initQ(const wbMatrix<TQ> &Qk, const QSpace &A){
00446         clearQSpace();
00447         QDIM=A.QDIM; qtype=A.qtype; QIDX=Qk;
00448         setupDATA(FL);
00449         if (A.CGR.data) setupCGR(FL);
00450     };
00451
00452     void init_itags( const char *F, int L, const char *s,
00453         unsigned r=-1, char lflag=-1);
00454
00455     QSpace& SetTag(unsigned k, const itag_ &b) {
00456         if ((--k)>=itags.len) wblog(FL,
00457             "ERR %s() index out of bounds (%d/%d)",FCT,k+1,itags.len);
00458         itags.data[k].tSet(b);
00459         return *this;
00460     };
00461
00462     QSpace& SetTags(const itag_ &b) {
00463         unsigned k=rank(FL);
00464         if (b.t && k>2) wblog(FL,
00465             "ERR %s() got '%s' for rank-%d QSpace !?",FCT,STR(b),k);
00466         for (k=0; k<itags.len; ++k) { itags.data[k].tSet(b); }
00467         return *this;
00468     };
00469
00470     QSpace& SetConjTags() {
00471         for (unsigned k=0; k<itags.len; ++k) { itags.data[k].Conj(); }
00472         return *this;
00473     };
00474
00475     QSpace& SetConjTags(unsigned k1, unsigned k2) {
00476         if (!k1 || !k2) wblog(FL,
00477             "ERR %s(%d) expects 1-based index",FCT,k1,k2);
00478         itags.SetConj(0,0,k1-1);
00479         itags.SetConj(0,0,k2-1); return *this;
00480     };
00481
00482     QSpace& SetConjTag(unsigned k) {
00483         if (!k) wblog(FL,"ERR %s(%d) expects 1-based index",FCT,k);
00484         itags.SetConj(0,0,k-1); return *this;
00485     };
00486
00487     QSpace& SetConjTag(const char* F, int L, unsigned k) {
00488         if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
00489         itags.SetConj(F,L,k-1); return *this;
00490     };
00491
00492     QSpace& SetFlag(const char *F, int L, unsigned k, unsigned l) {
00493         if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
00494         itags.SetFlag(F_L,k-1,l);
00495         return *this;
00496     };
00497
00498     QSpace& UnsetFlag(const char *F, int L, unsigned k, unsigned l) {
00499         if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);

```

```

00500         itags.UnsetFlag(F_L,k-1,l);
00501         return *this;
00502     };
00503     QSpace& UnsetFlags() {
00504         itags.UnsetFlags();
00505         return *this;
00506     };
00507     QSpace& SetFlags(const char *F, int L, unsigned k, unsigned l=-1) {
00508         if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
00509         itags.SetFlags(F_L,k-1,l);
00510         return *this;
00511     };
00512
00513     QSpace& init_tags() { itags.init_tags(); return *this; };
00514
00515     template<class TB>
00516     unsigned matchITags(
00517         const char *F, int L, const QSpace<TQ,TB> &B,
00518         ctrIdx &ia, ctrIdx &ib) const;
00519
00520     unsigned RemoveZLabels(
00521         const char *F, int L, wbMatrix<TQ> &QQ, wbMatrix<TQ> *Z=NULL) const;
00522
00523     void RemoveZLabels(const char *F, int L, wbMatrix<TQ> *Z){
00524         if (QDIM==0 && qtype.len) { QDIM=qtype.Qlenz(); }
00525         QDIM=RemoveZLabels(F_L,QIDX,Z);
00526     };
00527
00528     void ExpandDiagonal(unsigned i1=1, unsigned i2=2);
00529
00530     size_t getDataSize(wbMatrix<size_t> &S) const;
00531     size_t getDataSize(wbvector<size_t> &S) const;
00532     size_t getDataSize() const;
00533     size_t getCGSSize() const;
00534     wbstring totSize2Str() const;
00535
00536     wbstring itags2Str() const {
00537         return itags.toStr().data;
00538     };
00539
00540     unsigned map2Vec (const char *F, int L, wbvector<TD> &V,
00541         const wbvector<unsigned> &S=wbvector<unsigned>());
00542
00543     unsigned map2Vec(TD *v, char Iflag=0);
00544
00545     void ExpandQ(
00546         CPAT<TQ,TD> &CP, double nrm, wbvector<unsigned> &xflag, char disp=0);
00547
00548     void ResetRec(unsigned k) {
00549         if (k>=QIDX.dim1) wblog(FL,
00550             "ERR %s() index out of bounds (%d/%d)",FCT,k,QIDX.dim1);
00551         isConsistent(FL);
00552
00553         QIDX.setRec(k,0); DATA[k]->init();
00554         for (unsigned j=0; j<CGR.dim2; ++j) { CGR(k,j).init(); }
00555     };
00556
00557     void setData(const TD c) {
00558         for (unsigned i=0; i<DATA.len; ++i) { DATA[i]->set(c); }
00559     };
00560
00561     void setRand(double nrm=0);
00562
00563     void markUnequalZero(const TD m=1, const TD eps=0) {
00564         for (unsigned i=0; i<DATA.len; ++i)
00565             DATA[i]->markUnequalZero(m,eps);
00566     };
00567
00568     void PrependSingletons(unsigned R);
00569
00570     void swap(QSpace &B) {
00571         if (isref || QIDX.isref || DATA.isref || CGR.isref) wblog(FL,
00572             "ERR must not save referenced data (isref=%d,%d,%d)",
00573             isref, QIDX.isref,DATA.isref);
00574         if (B.isref || B.QIDX.isref || B.DATA.isref || B.CGR.isref) wblog(FL,
00575             "ERR must not save referenced data (isref=%d,%d,%d)",
00576             B.isref, B.QIDX.isref, B.DATA.isref);
00577
00578         DATA .swap(B.DATA );
00579         QIDX .swap(B.QIDX ); SWAP(QDIM, B.QDIM );
00580         CGR .swap(B.CGR ); SWAP(otype,B.otype);
00581         qtype.swap(B.qtype);
00582         itags.swap(B.itags); SWAP(isref,B.isref); SWAP(mt,B.mt);
00583     };
00584
00585     QSpace& save2(QSpace &B) {
00586         if (isref || QIDX.isref || DATA.isref || CGR.isref) wblog(FL,

```

```

00587         "ERR must not save referenced data (isref=%d,%d,%d)",
00588         isref, QIDX.isref, DATA.isref);
00589     for (unsigned i=0; i<DATA.len; ++i) {
00590         if (DATA[i]->isRef()) wblog(FL,"%NERR using save2 with "
00591         "referenced data\n(data[%d] has isref=%d)",
00592         i,DATA[i]->isRef()
00593         );
00594     }
00595     B.clearQSpace();
00596
00597     DATA .save2(B.DATA );
00598     QIDX .save2(B.QIDX ); B.QDIM=QDIM;
00599     CGR .save2(B.CGR );
00600
00601     qtype.save2(B.qtype);
00602     itags.save2(B.itags);
00603     B.otype=otype; otype=QS_NONE;
00604
00605     B.isref=isref; isref=0;
00606     B.ctime=ctime;
00607     B.mt=mt;
00608
00609     return B;
00610 };
00611
00612 unsigned Reduce2AbelianOp(const char *F=NULL, int L=0);
00613
00614 char allAbelian() const {
00615     return qtype.allAbelian();
00616 };
00617
00618 char isNonAbelian() const {
00619     return qtype.isNonAbelian();
00620 };
00621
00622 bool permitsOM() const {
00623     bool q=0;
00624     if (CGR || !qtype.allAbelian()) {
00625         q=qtype.permitsOM(rank(FL)); }
00626     return q;
00627 };
00628
00629 bool hasOM() const {
00630     bool om=0;
00631
00632     #ifndef WB_SKIP_ASSERT
00633     int q=QIDX.isSorted(), pm=permitsOM();
00634     if (! q ) wblog(FL,"WRN %s() got non-sorted QIDX",FCT);
00635     if (!(q&2)) wblog(FL,"WRN %s() got non-unique QIDX "
00636     "%s @ r=%d; %d, %d)",FCT,STR(qtype),rank(),q,pm);
00637     #endif
00638
00639     unsigned i=0, n=CGR.numel();
00640     for (; i<n; ++i) { if (CGR[i].wnumel()>1) { om=1; break; } }
00641     return om;
00642 };
00643
00644 bool allAdditive() const {
00645     return qtype.allAdditive();
00646 };
00647
00648 bool isEmpty() const;
00649
00650 explicit operator bool() const { return !isEmpty(); }
00651 bool operator!() const { return isEmpty(); }
00652
00653 bool isConsistent(
00654     const char *F, int L, unsigned &r, char level=3) const;
00655
00656 bool isConsistent(
00657     const char *F=NULL, int L=0
00658 ) const { unsigned r=-1; return isConsistent(F,L,r,3); };
00659
00660 bool isConsistent(unsigned &r, char level=3) const {
00661     return isConsistent(NULL,0,r,level); };
00662
00663 bool isConsistent_r(unsigned r, char level=3) const {
00664     return isConsistent(NULL,0,r,level);
00665 };
00666
00667 bool isScalar() const {
00668     unsigned r=rank(FL);
00669     if (r || DATA.len!=1) { return 0; }
00670
00671     if (QIDX.dim1!=1 || DATA[0]->numel()!=1) {
00672         wblog(FL,"ERR %s() inconsistency (%d; S=%s)",
00673         FCT,QIDX.dim1,SSTR_(DATA[0]));

```

```

00674     }
00675     return 1;
00676 };
00677
00678 bool isComplex() const {
00679     unsigned i=0, n=DATA.len; isConsistent(FL);
00680     for (; i<n; ++i) { if (DATA[i]->isComplex()) return 1; }
00681     return 0;
00682 };
00683
00684 bool isDiagBlock(unsigned k) const;
00685
00686 bool isBlockDiagMatrix(const char *F, int L, char dflag=0) const;
00687 bool isBlockDiagMatrix(char dflag=0) const {
00688     return isBlockDiagMatrix(NULL,0,dflag); }
00689
00690 bool isDiagMatrix(const double eps=1e-14) const;
00691 bool isIdentityMatrix(const TD eps=1e-14) const;
00692 char hasIdentityCGS(const TD eps=1e-14) const;
00693
00694 bool isR3Op() const {
00695     return (rank(FL)==3 && otype==QS_OPERATOR);
00696 };
00697
00698 bool isOp(int r=-1, char lflag=0) const {
00699     return itags.isOp(r,lflag); };
00700
00701 int isOperator(unsigned *r_, char xflag=0) const;
00702
00703 bool isOperator() const { return (isOperator(NULL,0)>0 ? 1 : 0); };
00704
00705 bool isAtensor(unsigned r=-1) const { return itags.isAtensor(r); }
00706
00707 int checkAbelianOp() const;
00708 bool isAbelianOp() const { return (checkAbelianOp())>0; };
00709
00710 bool isQSym(const char *F=0, int L=0, char dflag=0) const;
00711
00712 bool isHConj(
00713     const char *F=0, int L=0, double eps=1e-14, char vflag=0
00714 ) const;
00715
00716 bool isAHerm(
00717     const char *F=0, int L=0, double eps=1e-14, char vflag=0
00718 ) const;
00719
00720 int isSingleton(unsigned k) const;
00721
00722 template <class T2>
00723 bool hasSameQ(const QSpace<TQ,T2> &B) const;
00724
00725 bool hasSameQ(
00726     unsigned r1, unsigned k1,
00727     unsigned r2, unsigned k2
00728 ) const {
00729     if (memcmp( Qref(r1,k1), Qref(r2,k2), QDIM*sizeof(TQ) )) return 0;
00730     if (DATA[r1]->dim0(k1) != DATA[r2]->dim0(k2)) {
00731         wblog(FL,
00732             "WRN %s()=1, yet DATA size mismatch (%s <> %s; %d,%d)",
00733             FCT, DATA[r1]->sizeStr().data,
00734             DATA[r2]->sizeStr().data, k1+1, k2+1);
00735         return 0;
00736     }
00737     return 1;
00738 };
00739
00740 template <class T2>
00741 bool hasSameRank(const QSpace<TQ,T2> &B) const {
00742     return (QIDX.dim2==B.QIDX.dim2 && QDIM==B.QDIM);
00743 };
00744
00745 char checkQ(const char *F=NULL, int L=0) const;
00746
00747 template <class TB>
00748 void checkQ(
00749     const char *F, int L, const QSpace<TQ,TB> &B,
00750     char aflag=0
00751 ) const;
00752
00753 char checkQ_CGR(const char *F, int L, C_UVEC *I=NULL) const;
00754
00755 int check_CGR_cgw(const char *F=0, int L=0) const;
00756
00757 void checkScalarCGS(const char *F=0, int L=0) const;
00758
00759 template <class TB>
00760 bool sameType(

```

```

00761     const QSpace<TQ,TB> &B, int r=-1, const char *istr=NULL
00762 ) const;
00763
00764 QDir& getQDir(QDir &qdir) const;
00765 bool sameQDir(const char *F, int L, const QSpace &B) const;
00766
00767 bool hasQOverlap(const QSpace &A) const;
00768
00769 template <class TB>
00770 int hasQOverlap(
00771     unsigned ica, const QSpace<TQ,TB> &B, unsigned icb,
00772     const char type=0
00773 ) const;
00774
00775 int getQOverlapU(
00776     const char *F, int L, const QSpace &B,
00777     wbMatrix<TQ> &QQ, wbMatrix<int> &IQ,
00778     char olonly=0
00779 ) const;
00780
00781 int getQOverlap(
00782     const char *F, int L, const QSpace &B,
00783     wbMatrix<TQ> &QQ, wbMatrix<int> &IQ,
00784     wbvector<widx_t> &DQ
00785 ) const;
00786
00787 bool findDimQ(const TQ* q, unsigned &d) const;
00788 bool findDimQ(const TQ* q, unsigned k, unsigned &d) const;
00789
00790 bool operator==(const QSpace &B) const {
00791     if (this!=&B) {
00792         if (!(QIDX==B.QIDX)) return 0;
00793         if (!(qtype==B.qtype)) return 0;
00794         if (!(CGR==B.CGR)) return 0;
00795         if (!DATA.deepEqualP(B.DATA)) return 0;
00796     }
00797     return 1;
00798 };
00799
00800 bool operator!=(const QSpace &B) const { return !((*this)==B); }
00801
00802 template<class T>
00803 void operator*= (const T fac);
00804
00805 template<class T>
00806 QSpace& times(const T fac, QSpace &B) const {
00807     B=*this; B*=fac; return B;
00808 };
00809
00810 QSpace& operator= (const QSpace &A) { return init(A); };
00811
00812 void operator+= (const QSpace &Q);
00813 void operator-= (const QSpace &Q);
00814
00815 QSpace& Cat(const char *F, int L,
00816     const QSpace<TQ,TD> &A, const QSpace<TQ,TD> &B,
00817     TD afac=1, TD bfac=1, char uflag=1
00818 );
00819
00820 QSpace& plus(
00821     const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const;
00822
00823 QSpace& minus(
00824     const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const;
00825
00826 QSpace& Plus (const QSpace &A, TD bfac=1);
00827 QSpace& Minus(const QSpace &A, TD bfac=1);
00828
00829 QSpace& plus_plain(const QSpace &B, QSpace &C, TD bfac=1) const;
00830
00831 int NormCGW(char full=0, char skipzeros=1, char rcpy=0);
00832
00833 TD norm2(char checks=1) const;
00834 TD norm (char checks=1) const { return sqrt(norm2(checks)); }
00835
00836 TD normDiff2(const QSpace &B) const;
00837 TD scalarProd(const QSpace &B) const;
00838
00839 void checkNorm(
00840     const char *F, int L, double nrm, char vflag=1, double eps=1E-12
00841 ) const;
00842
00843 QSpace& TimesEl(const QSpace &B, char conj=0, unsigned r2=-1);
00844
00845 TD sumData() const;
00846
00847 template <class TM>

```

```

00848     wbMatrix<TM>& diag(wbMatrix<TM> &xd) const;
00849
00850     QSpace& TensorProd(const QSpace &B, QSpace &C,
00851         const char aflag='N', const char bflag='N') const;
00852
00853     template <class TC, ENABLE_IF_COMPLEX(TC) >
00854     QSpace<TQ,TC>& TensorProd(const QSpace &B, QSpace<TQ,TC> &C,
00855         const char aflag='N', const char bflag='N'
00856         ) const { QSpace<TQ,TD> C_;
00857         this->TensorProd(B,C_,aflag,bflag);
00858         return C.initT(C_);
00859     };
00860
00861     template <class TC, ENABLE_IF_COMPLEX(TC) >
00862     QSpace<TQ,TC>& TensorProd(const QSpace<TQ,TC> &B, QSpace<TQ,TC> &C,
00863         const char aflag='N', const char bflag='N'
00864         ) const { QSpace<TQ,TC> A; A.initT(*this);
00865         return A.TensorProd(B,C,aflag,bflag);
00866     };
00867
00868     void TensorProdUnity(const wbMatrix<TQ> &Q, QSpace &C) const;
00869
00870     void EigenSymmetric(wbMatrix<double> &Etot) const {
00871         QSpace Ak,Ad;
00872         QSpace<TQ,double> Ek,Et;
00873         wbMatrix<unsigned> DB; int Nkeep=-1;
00874
00875         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
00876     };
00877
00878     void EigenSymmetric(
00879         QSpace &Ak, QSpace<TQ,double> &Ek, wbMatrix<double> &Etot
00880     ) const {
00881         QSpace Ad;
00882         QSpace<TQ,double> Et;
00883         wbMatrix<unsigned> DB; int Nkeep=-1;
00884
00885         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
00886     };
00887
00888     void EigenSymmetric(QSpace &Ak, QSpace<TQ,double> &Ek) const {
00889         QSpace Ad;
00890         QSpace<TQ,double> Et;
00891         wbMatrix<unsigned> DB; int Nkeep=-1;
00892         wbMatrix<double> Etot;
00893
00894         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
00895     };
00896
00897     void EigenSymmetric(
00898         QSpace &Ak, QSpace &Ad,
00899         QSpace<TQ,double> &Ek, QSpace<TQ,double> &Et, wbMatrix<double> &Etot,
00900         wbMatrix<unsigned> &DD, int &Nkeep,
00901         double Etrunc=0, double *E0=NULL,
00902         char mKD=0,
00903         const wbperm &P=wbperm(), double eps=0., double b=0., int dmax=-1,
00904         const char *sort=NULL
00905     ) const;
00906
00907     void GroupIndizes(unsigned K);
00908
00909     int ExpandOM(
00910         const char *F, int L, wbvector<unsigned> &iOM, char fflag=0);
00911
00912     int ExpandOM_(
00913         const char *F, int L, unsigned ia,
00914         QSpace<TQ,TD> &B, unsigned ib);
00915
00916     int ExpandOM_(const char *F, int L, unsigned ia, unsigned ib) {
00917         return ExpandOM_(F,L,ia,*this,ib);
00918     };
00919
00920     void Append(unsigned k);
00921     void Append(const wbvector<TQ> &, const wbarray<TD> &D);
00922
00923     QSpace& Append(const char *F, int L, const QSpace &A, char uflag=1);
00924
00925     int Append2AndDestroy(
00926         const char *F, int L, QSpace &A, char unique=0);
00927
00928     template <class TA, ENABLE_IF_COMPLEX(TA) >
00929     int Append2AndDestroy(
00930         const char *F, int L, QSpace<TQ,TA> &A, char unique=0
00931     ){
00932         QSpace<TQ,TA> X; X.initT(*this);
00933         return X.Append2AndDestroy(F,L,A,unique);
00934     };

```

```

00935
00936 void TransferSpace(const char *F, int L,
00937     QSpace &B, unsigned k,
00938     const QSpace<TQ,double> &W, double eps
00939 );
00940
00941 void TransferSpace(const char *F, int L,
00942     QSpace &B, unsigned k
00943 );
00944
00945 unsigned MakeUnique();
00946
00947 bool isZero(const double eps=0., char flag=0) const;
00948 unsigned SkipZeroData(
00949     double eps=0., char bflag=0, char cgflag=1,
00950     char all=0
00951 );
00952
00953 unsigned skipZeroOffDiag(double eps=0., char bflag=0);
00954
00955 unsigned SkipEmptyData(const char *F=0, int L=0);
00956
00957 bool gotZeroData(unsigned i, const double eps=0.) const;
00958
00959 void Set(unsigned k, const wbvector<TQ> &Q, const wbararray<TD> &D);
00960 void SetQ(unsigned i, unsigned k, const TQ *d);
00961
00962 QSpace& Select(const wbindex &I, char data_only=0);
00963
00964 template <class TM>
00965 QSpace& SelectMarked(const wbvector<TM> &mark){
00966     if (mark.len!=QIDX.dim1) wblog(FL,
00967         "ERR %s() size inconsistency (%d/%d)",FCT,mark.le,QIDX.dim1);
00968     wbindex I; mark.find(I);
00969     return Select(I);
00970 };
00971
00972 void Sort(const QSpace &A);
00973 void Sort() {
00974     wbpem P; QIDX.SortRecs(P); DATA.Select(P);
00975     if (qtype.allAbelian()) {
00976         if (!CGR.isEmpty()) wblog(FL,"ERR %s() "
00977             "invalid all-abelian CGR (%dx%d)",FCT,CGR.dim1,CGR.dim2);
00978     }
00979     else CGR.recPermute(P);
00980 };
00981
00982 unsigned getOM(
00983     wbMatrix<unsigned> &OM, wbvector<unsigned> *oml=NULL,
00984     char wflag=0) const;
00985
00986 unsigned cgsDimScalar(unsigned k) const;
00987 unsigned cgsDim(unsigned k, unsigned r) const;
00988 unsigned getDim(unsigned k, widx_t *D=NULL) const;
00989
00990 unsigned getDIM(unsigned k) const {
00991     widx_t D=0; getDim(k,&D);
00992     return D;
00993 };
00994
00995 void getDim(wbvector<widx_t> &D, wbvector<widx_t> *DD=NULL) const;
00996
00997 wbvector<widx_t> getDim() const {
00998     wbvector<widx_t> D; getDim(D); return D;
00999 }
01000
01001 void getQDim(unsigned k,
01002     wbMatrix<TQ> &Q,
01003     wbvector<widx_t> &S,
01004     wbMatrix<widx_t>*SC=NULL
01005 ) const;
01006
01007 void getQDim(wbMatrix<TQ> &Q,
01008     wbvector<widx_t> &S,
01009     wbMatrix<widx_t>*SC=NULL
01010 ) const;
01011
01012 void getDRange(TD &dmin, TD &dmax) const;
01013
01014 void getDistQtot(wbMatrix<TQ> &Qtot, wbvector<double> &w) const;
01015 widx_t getDQtot() {
01016     wbMatrix<TQ> Q; QIDX.blockSum(QDIM,Q); Q.makeUnique();
01017     return Q.dim1;
01018 };
01019
01020 wbMatrix<TQ>& getQsub(const wbindex &I, wbMatrix<TQ> &QI) const;
01021 wbMatrix<TQ>& getQsub(

```



```

01022     const unsigned k, wbMatrix<TQ> &Qk, char iflag=0) const;
01023
01024     void getQsub(const wbindex &I, wbMatrix<TQ> &QI, wbMatrix<TQ> &Q2) const;
01025
01026     wbMatrix<TQ> getQsub(const unsigned k) const {
01027         wbMatrix<TQ> Qk; getQsub(k,Qk); return Qk;
01028     };
01029
01030     wbMatrix<TQ>& getQtot(wbMatrix<TQ> &Qtot, char uflag=0) const;
01031     wbMatrix<TQ>& getQsum(const wbindex &J, wbMatrix<TQ> &Q) const;
01032     wbMatrix<TQ>& getQsum(const wbMatrix<TQ> &Q0, wbMatrix<TQ> &Q) const;
01033
01034     wbMatrix<TQ>& getQfinal(const wbindex &J, wbMatrix<TQ> &Q) const;
01035
01036     QSpace& getSub(const wbindex &I, QSpace &A, char ref=0) const;
01037
01038     QSpace& getSubInit(const wbindex &I, QSpace &A) const;
01039
01040     void saveSub2(QSpace &A, const wbindex &I);
01041
01042     void toFull(
01043         wbararray<TD> &A,
01044         wbvector< wbMatrix<TQ> > &Q,
01045         wbvector< wbvector<unsigned> > &S
01046     ) const;
01047
01048     void toFull2(
01049         wbararray<TD> &A,
01050         wbMatrix<TQ> &Q,
01051         wbMatrix<widx_t> &S
01052     ) const;
01053
01054     void ExtendQ(const QSpace &B);
01055
01056     void takeMainQS(char disp=1);
01057
01058     void setQ(unsigned k, const QSpace &B, unsigned k0);
01059     void setQRec(unsigned r, unsigned j, const TQ *q0, const TQ *q2=NULL);
01060
01061     bool hasEqualQ(unsigned i1, unsigned i2) const;
01062
01063     double maxDiff(const QSpace &B) const;
01064
01065     void PermuteQ(const wbperm &P);
01066     void permuteQ(const wbperm &P, wbMatrix<TQ>&) const;
01067
01068     QSpace& Permute(const wbperm &P, char iflag=0, char rcpy=0);
01069     QSpace& permute(const wbperm &P, QSpace &B, char iflag=0) const;
01070
01071     void permute (const char *pstr, QSpace &B, unsigned offset=1
01072     ) const {
01073         permute(wbperm(pstr,offset),B);
01074     };
01075
01076     void Permute(const char* pstr, unsigned offset=1) {
01077         Permute(wbperm(pstr,offset));
01078     };
01079
01080     void PermuteFirstTo (unsigned k) {
01081         unsigned r=rank(FL);
01082         if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01083         wbperm P2(r); P2.mvFirstTo(k); Permute(P2);
01084     };
01085     void permuteFirstTo (unsigned k, QSpace &C) {
01086         if (this==&C) { PermuteFirstTo(k); return; }
01087         unsigned r=rank(FL);
01088         if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01089         wbperm P2(r); P2.mvFirstTo(k); permute(P2,C);
01090     };
01091
01092     void PermuteLastTo (unsigned k) {
01093         unsigned r=rank(FL);
01094         if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01095         wbperm P2(r); P2.mvLastTo(k); Permute(P2);
01096     };
01097     void permuteLastTo (unsigned k, QSpace &C) {
01098         if (this==&C) { PermuteLastTo(k); return; }
01099         unsigned r=rank(FL);
01100         if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01101         wbperm P2(r); P2.mvLastTo(k); permute(P2,C);
01102     };
01103
01104     void FlipQ();
01105
01106     QSpace& trace(
01107         const char *F, int L, ctrIdx &i1, ctrIdx &i2, QSpace &C) const;
01108     TD trace() const;

```

```

01109
01110 void contractMat(
01111     unsigned ia, const QSpace &B, unsigned ib,
01112     QSpace &C
01113 ) const;
01114
01115 template <class TB, class TC>
01116 int contract_getIdxSet(const char *F, int L,
01117     const ctrIdx &ica, const QSpace<TQ,TB> &B, const ctrIdx &icb,
01118     wbindx &Ia, wbindx &Ib, wbvector<widx_t> &Dc,
01119     QSpace<TQ,TC> &C
01120 ) const;
01121
01122 template<class TB, class TC>
01123 double contract(
01124     const char *F, int L, const ctrIdx &ica,
01125     const QSpace<TQ,TB> &B, const ctrIdx &icb, QSpace<TQ,TC> &C,
01126     const wbperm &P=wbperm(), char preview=0
01127 ) const;
01128
01129 template<class TB, class TC>
01130 double contract(const char *F, int L,
01131     unsigned ia, const QSpace<TQ,TB> &B, unsigned ib,
01132     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01133 ) const {
01134
01135     unsigned ra=rank(FL), rb=B.rank(FL);
01136     if (!ia || (ia>ra && (QIDX.dim1 || QIDX.dim2))) wblog(FL,
01137         "ERR %s() index out of bounds (ia=%d/%d; %s)",
01138         FCT,ia,ra,SSTR(QIDX));
01139     if (!ib || (ib>rb && (B.QIDX.dim1 || B.QIDX.dim2))) wblog(FL,
01140         "ERR %s() index out of bounds (ib=%d/%d; %s)",
01141         FCT,ib,rb,SSTR(B.QIDX)
01142     );
01143
01144     ctrIdx ica(1,&(--ia)), icb(1,&(--ib));
01145     return contract(F,L,ica,B,icb,C,P,preview);
01146 };
01147
01148 template<class TB, class TC>
01149 int contract(const char *F, int L,
01150     unsigned ia,
01151     const QSpace<TQ,TB> &B, const char *sb,
01152     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01153 ) const {
01154
01155     ctrIdx ica(1,&(--ia)), icb;
01156
01157     if (isEmpty() || B.isEmpty()) { C.init(); return 1; }
01158     if (ia>=itags.len) {
01159         wblog(FL,"ERR %s() index out of bounds (ia=%d/%d; %s)",
01160             FCT,ia+1,itags.len,SSTR(itags));
01161     }
01162
01163     if (sb && isdigit(sb[0]))
01164         { icb.init(F_L,sb); }
01165     else { icb.initOp(FL,itags[ia],B,sb); }
01166
01167     return contract(F,L,ica,B,icb,C,P,preview);
01168 };
01169
01170 template<class TB, class TC>
01171 int contract(const char *F, int L,
01172     const char *sa,
01173     const QSpace<TQ,TB> &B, unsigned ib,
01174     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01175 ) const {
01176
01177     ctrIdx ica, icb(1,&(--ib));
01178
01179     if (isEmpty() || B.isEmpty()) { C.init(); return 1; }
01180
01181     if (ib>=B.itags.len) wblog(FL,
01182         "ERR %s() index out of bounds (ib=%d/%d; %s)",
01183         FCT,ib+1,B.itags.len,SSTR(B.itags));
01184
01185     if (sa && isdigit(sa[0]))
01186         { ica.init(F_L,sa); }
01187     else { ica.initOp(FL,B.itags[ib],*this,sa); }
01188
01189     return contract(F,L,ica,B,icb,C,P,preview);
01190 };
01191
01192 template<class TB, class TC> inline
01193 int contract(const char *F, int L,
01194     const char *sa, const QSpace<TQ,TB> &B, const char *sb,
01195     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0

```

```

01196     ) const {
01197
01198         ctrIdx ica(F_L,sa),  icb(F_L,sb);
01199         return contract(F,L,ica,B,icb,C,P,preview);
01200     };
01201
01202     template<class TB, class TC>
01203     int contract(
01204         const char *sa, const QSpace<TQ,TB> &B, const char *sb,
01205         QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01206     ) const { return contract(FL,sa,B,sb,C,P,preview); };
01207
01208     template<class TB, class TC>
01209     int contract(
01210         const ctrIdx &ica, const QSpace<TQ,TB> &B, const ctrIdx &icb,
01211         QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01212     ) const { return contract(FL,ica,B,icb,C,P,preview); };
01213
01214     template<class TB, class TC>
01215     int contract(
01216         unsigned ia, const QSpace<TQ,TB> &B, unsigned ib,
01217         QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01218     ) const { return contract(FL,ia,B,ib,C,P,preview); };
01219
01220     mxArray* toMx(bool setcls=0) const;
01221
01222     mxArray* save2Mx(char vflag=0);
01223
01224     mxArray* QIDX_toMx() const;
01225     mxArray* INFO_toMx() const;
01226     mxArray* DATA_toMx() const;
01227     mxArray* DATA_save2Mx(char vflag=0);
01228
01229     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01230     void add2MxStruct(mxArray *S, unsigned i, bool setcls=0, char chk=0) const;
01231
01232     void save2MxStruct(mxArray *S, unsigned i, char tst=0, char vflag=0);
01233
01234     mxArray* mxCreateCell(unsigned m, unsigned n) const {
01235         wblog(FL,"ERR %s()",FCT); return 0; };
01236     void add2MxCell(mxArray *S, unsigned i, char tst=0) const {
01237         wblog(FL,"ERR %s()",FCT); };
01238
01239     void put (
01240         const char *F, int L,
01241         const char *vname, const char *ws="caller"
01242     ) const {
01243         wblog(F,L,"I/O putting '%s' to %s", vname, ws);
01244         put(vname,ws);
01245     };
01246
01247     void put(const char *vname, const char *ws="caller") const {
01248         mxArray *S=toMx();
01249         mxPutAndDestroy(0,0,S,vname,ws);
01250     };
01251
01252     wbsttring sizeStrQ(const char *F=NULL, int L=0) const;
01253
01254     wbsttring sizeStr(char vflag=0) const;
01255
01256     void print_SIZE(const char *F=NULL, int L=0) const;
01257     void print_rankDATA(const char *F=NULL, int L=0) const;
01258
01259     wbsttring qStr() const { return qtype.toStr(); };
01260
01261     wbsttring qrec2Str(unsigned k) const {
01262         return QIDX.rec2Str(k,""," ", QDIM,"");
01263     };
01264
01265     void disp_cgs(const char *F, int L, const char *istr) const;
01266
01267     void info(const char *vname=NULL,
01268         unsigned nlt=0, unsigned nlb=0,
01269         unsigned nind=2
01270     ) const;
01271
01272     void print() const { print("",3); };
01273     void print(const char *vname, char vflag=3) const;
01274
01275     void print_tst_(
01276         const char *F, int L, const char *istr, unsigned k=0) const {
01277
01278         if (k>=DATA.len) { wblog(FL,
01279             "WRN %s() index out of bounds (%d/%d)",FCT,k,DATA.len);
01280             return;
01281         }
01282         unsigned i=0, l=0, n=64; char s[n];

```

```

01283     const TD *dk=DATA[k]->data;
01284     l=snprintf(s,n,"%8s",NSTR(dk[0]));
01285     if (CGR.dim2) {
01286         for (; i<CGR.dim2; ++i) { if (l<n) {
01287             l+=snprintf(s+l,n-l," %c %4s",i?'*':'x',RATS(CGR(k,i).wel(0))); }
01288         }
01289         wblog(F_L,"TST %s @%2d/%d: %-32s %s",istr,k+1,DATA.len,s,STR(itags));
01290     };
01291
01292 void initOType(const char *F, int L, const mxArray* a);
01293
01294 wbststring otype2Str(const char *fmt="%s") const;
01295
01296 int ConjOpScalar() {
01297     int e=0;
01298     unsigned r=rank(FL);
01299     if (r!=3) {
01300         if (r!=2) { wblog(FL,"WRN %s() got rank-%d QSpace !?",FCT,r); }
01301         return e;
01302     }
01303     for (unsigned n=CGR.numel(), i=0; i<n; ++i) {
01304         if ((e=CGR[i].HConjOpScalar())) { return e; }
01305     }
01306
01307     if (itags.len!=3) wblog(FL,
01308         "ERR %s() got itags.len=%d/3 !?",FCT,itags.len);
01309     itags[2].Conj();
01310
01311     return e;
01312 };
01313
01314 QSpace& SortDegQ(const char *F=0, int L=0) {
01315     for (unsigned n=CGR.numel(), i=0; i<n; ++i) {
01316         CGR.data[i].SortDegQ(F_L); }
01317     return *this;
01318 };
01319
01320 QSpace& Conj() {
01321     SetConjTags();
01322     return DConj().RConj();
01323 };
01324
01325 QSpace& DConj() {
01326     if (WbUtil<TD>::isComplex()) {
01327         for (unsigned n=DATA.len, i=0; i<n; ++i) { DATA[i]->Conj(); }
01328     }; return *this;
01329 };
01330
01331 QSpace& RConj() {
01332     if (CGR) {
01333         unsigned i=0, n=CGR.numel();
01334         for (; i<n; ++i) { CGR[i].Conj(); }
01335         SortDegQ(FL);
01336     }
01337     return *this;
01338 };
01339
01340 QSpace& HConj() {
01341     QSpace B; this->save2(B);
01342     return B.hconj(*this);
01343 };
01344
01345 QSpace& hconj(QSpace &B) const {
01346     if (!isEmpty()) {
01347         if (!QDIM || QIDX.dim2%QDIM) wblog(FL,
01348             "ERR %s() QSpace inconsistency (%dx%d/%d)",
01349             FCT, QIDX.dim1, QIDX.dim2, QDIM);
01350         unsigned r=QIDX.dim2/QDIM;
01351         if (r%2 && (r!=3 || otype!=QS_OPERATOR)) wblog(FL,"ERR %s() "
01352             "requires even-rank object or operator (%dx%d/%d; %s)",
01353             FCT, QIDX.dim1, QIDX.dim2, QDIM, QS_STR[otype]);
01354         transp(B); B.Conj(); B.otype=otype;
01355         return B;
01356     }
01357     return B.clearQSpace();
01358 };
01359
01360 QSpace& transp(QSpace &B) const {
01361
01362     if (!isEmpty()) { unsigned r=rank(FL);
01363         if (r%2==0) { wblog(FL,"P2.initTranspose(r); permute(P2,B);
01364             P2.initTranspose(r); permute(P2,B);
01365         }
01366         else if (r==3 && otype==QS_OPERATOR) {
01367             permute("2,1,3", B);
01368         }
01369         else wblog(FL,"ERR %s got rank-%d object",FCT,r);

```

```

01370     }
01371     else B.clearQSpace();
01372
01373     return B;
01374 };
01375
01376 const QSpace& opA(char tflag, QSpace& X) const {
01377     if (!strchr("NTC",tflag))
01378         wblog(FL,"ERR %s() invalid flag %c<%d>",FCT,tflag,tflag);
01379
01380     if (tflag=='N' || isEmpty()) return (*this);
01381     else {
01382         wbperm P2("2 1"); unsigned r=rank(FL);
01383
01384         if (r!=2) wblog(FL,
01385             "ERR %s() only applicable to rank-2 tensors (%d)",FCT,r);
01386
01387         permute(P2,X); if (tflag=='C') X.Conj();
01388         return X;
01389     }
01390 };
01391
01392 wbMatrix<TQ> QIDX;
01393
01394 wbvector<wbarray<TD>*> DATA;
01395 wbMatrix< CRef<TQ> > CGR;
01396
01397 QVec qtype;
01398
01399 unsigned QDIM;
01400
01401 QS_TYPES otype;
01402 iTags itags;
01403
01404 Wb::MTYPE mt;
01405
01406 char isref;
01407
01408 double ctime;
01409
01410 protected:
01411 private:
01412
01413     bool isSym_aux(
01414         const char *F, int L, const char *fct,
01415         RTD eps=1E-14, char symflag='s', char vflag=0
01416     ) const;
01417
01418     void recSave2(unsigned k, unsigned i);
01419 };
01420
01421 template <class TD>
01422 class IterOM_DATA {
01423 public:
01424
01425     template <class TQ>
01426     IterOM_DATA(const QSpace<TQ,TD> &A, unsigned i)
01427         : iter(0), niter(1), nS(0), data0(NULL) {
01428
01429         unsigned r, rA=A.rank(FL);
01430         if (i>=A.DATA.len) wblog(FL,
01431             "ERR %s() index out of bounds (%d/%d)",FCT,i,A.DATA.len);
01432         data0=A.DATA[i]->data;
01433         X.SIZE.init2ref(A.DATA[i]->SIZE); r=X.SIZE.len;
01434         if (!rA || r<rA || r>rA+1) wblog(FL,
01435             "ERR %s() rank inconsistency (r=%d/%d)",FCT,r,rA);
01436
01437         if (r>rA) { niter=X.SIZE[rA]; --X.SIZE.len; }
01438         nS=X.SIZE.prod();
01439     };
01440
01441     void begin() { iter=0; }
01442     bool end() { return iter>=niter; }
01443
01444     IterOM_DATA& operator++() { ++iter; return *this; };
01445
01446     wbarray<TD> DATA() {
01447         if (iter>=niter) wblog(FL,
01448             "ERR %s() index out of bounds (%d/%d)",FCT,iter,niter);
01449         return X.init2ref(data0+iter*nS);
01450     };
01451
01452     unsigned iter, niter;
01453     size_t nS;
01454     TD *data0;

```

```

01457     wbararray<TD> X;
01458
01459     protected:
01460     private:
01461 };
01462
01463 template <class TQ, class TD>
01464 template <class T> inline
01465 void QSpace<TQ,TD>::operator*= (const T fac) {
01466     if (fac==T(1)) return;
01467     for (unsigned i=0; i<DATA.len; ++i)
01468         (*DATA[i]) *= fac;
01469 };
01470
01471 template <class TQ, class TD> inline
01472 void QSpace<TQ,TD>::operator+= (const QSpace &B) {
01473     QSpace<TQ,TD> X;
01474     this->plus(B,X); X.save2(*this);
01475 };
01476
01477 template <class TQ, class TD> inline
01478 void QSpace<TQ,TD>::operator-= (const QSpace &B) {
01479     QSpace<TQ,TD> X;
01480     this->plus(B,X,-1); X.save2(*this);
01481 };
01482
01483 template <class TQ, class TD> inline
01484 QSpace<TQ,TD>& QSpace<TQ,TD>::Plus(const QSpace &B, TD bfac) {
01485     QSpace<TQ,TD> X;
01486     this->plus(B,X,bfac); return X.save2(*this);
01487 };
01488
01489 template <class TQ, class TD> inline
01490 QSpace<TQ,TD>& QSpace<TQ,TD>::Minus(const QSpace &B, TD bfac) {
01491     QSpace<TQ,TD> X;
01492     this->plus(B,X,-bfac); return X.save2(*this);
01493 };
01494
01495 template <class TQ, class TD> inline
01496 QSpace<TQ,TD>& QSpace<TQ,TD>::minus(
01497     const QSpace &B, QSpace &C, TD bfac, char vflag
01498 ) const {
01499     plus(B, C, -bfac, vflag); return C;
01500 }
01501
01502 template <class TQ, class TD>
01503 QSpace<TQ,TD>& QSpace<TQ,TD>::plus(
01504     const QSpace &B, QSpace &C, TD bfac, char vflag
01505 ) const {
01506
01507     if (B.isEmpty() || !bfac) { C=(*this); } else
01508     if (this->isEmpty()) { C=B; if (bfac!=1.) { C*=bfac; }} else
01509     if (gotCGS(FL)<2) { plus_plain(B,C,bfac); }
01510     else {
01511         C=B; if (bfac!=1.) { C*=bfac; }
01512         C.Append(FL,*this, vflag<1 ? 'q':vflag);
01513     }
01514
01515     return C;
01516 };
01517
01518 template <class TQ, class TD>
01519 TD QSpace<TQ,TD>::scalarProd(const QSpace<TQ,TD> &B) const {
01520
01521     unsigned i; int i1,i2;
01522     wbMatrix<int> IQ;
01523     wbMatrix<TQ> QQ;
01524     TD x=0;
01525
01526     if (this==&B) return norm2();
01527
01528 #ifdef WB_CLOCK
01529     Wb::Clock clk("QS:scalarProd",1);
01530 #endif
01531
01532     if (isEmpty() || B.isEmpty()) {
01533         wblog(FL,"WRN Overlap with empty object (%d,%d) !?",
01534             isEmpty(), B.isEmpty()); return x;
01535     }
01536
01537     getQOverlapU(FL,B,QQ,IQ,1);
01538
01539     for (i=0; i<IQ.dim1; ++i) {
01540         i1=IQ(i,0); i2=IQ(i,1);
01541         if (i1<0 || i2<0) wblog(FL, "ERR i1=%d, i2=%d !?", i1, i2);
01542
01543         x+=(DATA[i1]->scalarProd(*B.DATA[i2]));

```

```

01544     }
01545
01546     return x;
01547 };
01548
01549 template <class TQ, class TD>
01550 QSpace<TQ,TD>& QSpace<TQ,TD>::TensorProd(
01551     const QSpace<TQ,TD> &B, QSpace<TQ,TD> &C,
01552     const char aflag, const char bflag
01553 ) const {
01554
01555     if (isEmpty() || B.isEmpty()) { C.clearQSpace(); return C; }
01556
01557     if (gotCGS(FL)>0 || B.gotCGS(FL)>0) wblog(FL,
01558         "ERR %s() not implemented for non-abelian symmetries (got %s)",
01559         FCT,STR(qtype));
01560
01561     if (aflag!='N' || bflag!='N') {
01562         QSpace<TQ,TD> AX, BX;
01563         const QSpace<TQ,TD> *Ap=this, *Bp=&B;
01564
01565         if (!strchr("NTC",aflag) || rank(FL)!=2) wblog(FL,
01566             "ERR invalid aflag=%c<%d> (rank-%d)",aflag,aflag,rank(FL));
01567         if (!strchr("NTC",bflag) || B.rank(FL)!=2) wblog(FL,
01568             "ERR invalid bflag=%c<%d> (rank-%d)",bflag,bflag,B.rank(FL));
01569
01570         if (aflag!='N') {
01571             permute("2,1",AX); if (aflag=='C') AX.Conj();
01572             Ap=&AX;
01573         }
01574         if (bflag!='N') {
01575             B.permute("2,1",BX); if (bflag=='C') BX.Conj();
01576             Bp=&BX;
01577         }
01578
01579         return Ap->TensorProd(*Bp,C,'N','N');
01580     }
01581
01582     unsigned i,ia,ib,l,r;
01583     const unsigned n1=QIDX.dim1, n2=B.QIDX.dim1, R=rank(FL), s=QDIM*sizeof(TQ);
01584     const TQ *qa, *qb; TQ *q;
01585
01586     if (R!=B.rank(FL) || QDIM!=B.QDIM) wblog(FL,
01587         "ERR %s() size mismatch (%d,%d)",FCT,R,B.rank(FL));
01588
01589     C.clearQSpace();
01590     C.QIDX.init(n1*n2, 2*QIDX.dim2); C.QDIM=QDIM; C.setupDATA();
01591
01592     C.initQ(FL,*this,'f',&B);
01593
01594     if (itags.len || B.itags.len) { C.itags.init(2*R);
01595         if (!B.itags.len) {
01596             if (itags.len!=R) wblog(FL,
01597                 "ERR %s() invalid itag %s",FCT, STR(itags));
01598             for (i=0; i<R; ++i) {
01599                 C.itags[2*i]=C.itags[2*i+1]=itags[i];
01600             }
01601         }
01602         else if (!itags.len) {
01603             if (B.itags.len!=R) wblog(FL,
01604                 "ERR %s() invalid itag %s",FCT, STR(B.itags));
01605             for (i=0; i<R; ++i) {
01606                 C.itags[2*i]=C.itags[2*i+1]=B.itags[i];
01607             }
01608         }
01609         else {
01610             if (itags.len!=B.itags.len || itags.len!=R || !sameQDir(FL,B))
01611                 wblog(FL,"ERR %s() itag/qdir mismatch (%s <> %s)",FCT,
01612                     STR(itags),STR(B.itags));
01613             for (i=0; i<R; ++i) {
01614                 C.itags[2*i] = itags[i];
01615                 C.itags[2*i+1]=B.itags[i];
01616             }
01617         }
01618     }
01619
01620     for (i=ib=0; ib<n2; ++ib) {
01621         qb=B.QIDX.rec(ib);
01622
01623         for (ia=0; ia<n1; ++ia, ++i) {
01624             qa=QIDX.rec(ia); q=C.QIDX.rec(i);
01625
01626             for (l=r=0; r<R; ++r, l+=QDIM) {
01627                 memcpy(q,qa+l,s); q+=QDIM;
01628                 memcpy(q,qb+l,s); q+=QDIM;
01629             }
01630             DATA[ia]->tensorProd(*B.DATA[ib], *C.DATA[i]);

```

```

01631     }
01632 }
01633
01634 return C;
01635 };
01636
01637 template <class TQ, class TD> inline
01638 void QSpace<TQ,TD>::GroupIndizes(unsigned K) {
01639     unsigned i, r=-1;
01640
01641     if (!isConsistent(r)) wbdie(FL,str);
01642
01643     if (!K || unsigned(r)%K) wblog(FL,
01644 "ERR %s() cannot block QSpace (%d/%d)",FCT,r,K);
01645
01646     QDIM=K*QDIM;
01647
01648     for (i=0; i<DATA.len; ++i)
01649         DATA[i]->GroupIndizes(K);
01650 }
01651
01652 template <class TQ, class TD>
01653 size_t QSpace<TQ,TD>::getDataSize(wbMatrix<size_t> &S) const {
01654
01655     size_t s=0; unsigned i,m,n=DATA.len;
01656
01657     if (n!=QIDX.dim1) wblog(FL,
01658 "ERR size mismatch (%d/%d)",QIDX.dim1,DATA.len);
01659
01660     if (isEmpty()) { S.init(); return s; }
01661
01662     m=DATA[0]->SIZE.len; S.init(n,m);
01663
01664     for (i=0; i<n; ++i) {
01665         const wbvector<size_t> &si=DATA[i]->SIZE;
01666         if (si.len!=m) wblog(FL,"ERR size mismatch (%d/%d)",si.len,m);
01667         S.recSetP(i,si.data);
01668         s+=si.prod(0);
01669     }
01670     return s;
01671 };
01672
01673 template <class TQ, class TD> inline
01674 size_t QSpace<TQ,TD>::getDataSize(wbvector<size_t> &S) const {
01675     size_t s=0; S.init(DATA.len);
01676     for (unsigned i=0; i<DATA.len; ++i) { s+=(S[i]=DATA[i]->SIZE.prod()); }
01677     return s;
01678 };
01679
01680 template <class TQ, class TD> inline
01681 size_t QSpace<TQ,TD>::getDataSize() const {
01682     size_t s=0;
01683     for (unsigned i=0; i<DATA.len; ++i) s+=DATA[i]->SIZE.prod();
01684     return s;
01685 };
01686
01687 template <class TQ, class TD> inline
01688 size_t QSpace<TQ,TD>::getCGSSize() const {
01689     size_t s=0;
01690     if (gotCGS(FL)>0) {
01691         unsigned i,j;
01692         for (i=0; i<CGR.dim1; ++i) {
01693             for (j=0; j<CGR.dim2; ++j) s+=CGR(i,j).numel(); }
01694     }
01695     return s;
01696 };
01697
01698 template <class TQ, class TD> inline
01699 wbsting QSpace<TQ,TD>::totSize2Str() const {
01700
01701     unsigned long ss[3]={
01702         sizeof(TD)*getDataSize(),
01703         sizeof(TQ)*(QIDX.dim1*QIDX.dim2),
01704         sizeof(double)*getCGSSize()
01705     };
01706     char s_[32];
01707     memsize2Str(ss[0]+ss[1]+ss[2],s_);
01708     return s_;
01709 };
01710
01711 template <class TQ, class TD> inline
01712 bool QSpace<TQ,TD>::isZero(const double eps, char flag) const {
01713
01714     for (unsigned i=0; i<DATA.len; ++i)
01715         if (!(DATA[i]->isZero(eps,flag))) return 0;
01716
01717     return 1;

```



```

01718 }
01719
01720 template <class TQ, class TD> inline
01721 void QSpace<TQ,TD>::Set(
01722     unsigned k, const wvector<TQ> &Q, const warray<TD> &D
01723 ){
01724     QIDX.recSet(k,Q);
01725
01726     if (k>=DATA.len)
01727         wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,k,DATA.len);
01728
01729     DATA[k]->init(D);
01730 }
01731
01732 template <class TQ, class TD> inline
01733 void QSpace<TQ,TD>::SetQ(unsigned i, unsigned k, const TQ *d){
01734     if (i>=QIDX.dim1 || k>=rank(FL)) wblog(FL,
01735         "ERR index out of bounds (%d/%d; %d;%d)",i,QIDX.dim1,k,rank(FL));
01736
01737     memcpy(QIDX.data+QIDX.dim2*i+QDIM*k, d, QDIM*sizeof(TQ));
01738 }
01739
01740 template <class TQ, class TD> inline
01741 void QSpace<TQ,TD>::Sort(const QSpace<TQ,TD> &R) {
01742     int e;
01743     windex P,Ir; wperm Pr;
01744
01745     if (QIDX.dim1==0) return;
01746
01747     e=matchIndex(QIDX,R.QIDX, P, Ir);
01748
01749     if (e || P.len!=QIDX.dim1) wblog(FL,
01750         "ERR QSpace R not complete to be used as sorting reference "
01751         "(%d/%d)",P.len, QIDX.dim1);
01752
01753     Ir.Sort(Pr); P.Select(Pr);
01754
01755     QIDX.recPermute(P); if (!CGR.isEmpty())
01756         CGR.recPermute(P);
01757     DATA.Select(P);
01758 };
01759
01760 template <class TQ, class TD> inline
01761 int QSpace<TQ,TD>::isSingleton(unsigned k) const {
01762     unsigned i;
01763
01764     if (k>=rank(FL)) wblog(FL,
01765         "ERR %s() index out of bounds (%d,%d)",FCT,k,rank(FL));
01766
01767     for (i=0; i<DATA.len; ++i)
01768         if (DATA[i]->SIZE[k]!=1) break;
01769
01770     return (i==DATA.len);
01771 }
01772
01773 template <class TQ, class TD> inline
01774 void QSpace<TQ,TD>::getDRange(TD &dmin, TD &dmax) const {
01775     unsigned i,n=DATA.len;
01776
01777     if (!isConsistent()) { info("this"); wbdie(FL,str); }
01778
01779     for (i=0; i<n; ++i) if (DATA[i] && !DATA[i]->isEmpty()) {
01780         dmin=dmax=DATA[i]->data[0];
01781         break;
01782     }
01783
01784     for (i=0; i<n; ++i) if (DATA[i] && !DATA[i]->isEmpty()) {
01785         dmin=MIN(dmin, DATA[i]->min());
01786         dmax=MAX(dmax, DATA[i]->max());
01787     }
01788 }
01789
01790 template <class TQ, class TD> inline
01791 QSpace<TQ,TD> & QSpace<TQ,TD>::getSub(
01792     const windex &I, QSpace<TQ,TD> &A, char ref) const {
01793     if ((void*)this==(void*)&A) { QSpace<TQ,TD> X;
01794         if ( ref) wblog(FL,"ERR %s() requesting ref=%s on self",FCT,cSTR(ref));
01795         if (isref) wblog(FL,"ERR %s() got ref QSpace",FCT);
01796         return getSub(I,X).save2(A);
01797     }
01798
01799     unsigned i,j;
01800     char cgflag=gotCGS(FL);
01801 }

```

```

01805     A.init();
01806     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype; A.itags=itags;
01807
01808     A.isref=(ref ? 1 : 0);
01809
01810     A.setupDATA();
01811
01812     if (cgflag>0) {
01813         if (CGR.dim1!=DATA.len) wblog(FL,
01814             "ERR %s() size mismatch (CGR: %d/%dx%d)",
01815             FCT,DATA.len,CGR.dim1,CGR.dim2);
01816         A.setupCGR();
01817     }
01818
01819     for (i=0; i<I.len; ++i) {
01820         if (I[i]>=DATA.len) wblog(FL,
01821             "ERR %s() index out of bounds (%d,%d)",FCT,I[i],DATA.len);
01822
01823         if (A.isref)
01824             { A.DATA[i]=DATA[I[i]]; }
01825         else { A.DATA[i]->init(*DATA[I[i]]); }
01826
01827         if (cgflag>0) {
01828             for (j=0; j<CGR.dim2; ++j) {
01829                 A.CGR(i,j)=CGR(I[i],j);
01830             }
01831         }
01832     }
01833
01834     return A;
01835 };
01836
01837 template <class TQ, class TD> inline
01838 QSpace<TQ,TD>& QSpace<TQ,TD>::getSubInit(
01839     const wbindex &I,
01840     QSpace<TQ,TD> &A
01841 ) const {
01842
01843     if (this==&A) {
01844         QSpace<TQ,TD> X; A.save2(X);
01845         return X.getSubInit(I,A);
01846     }
01847
01848     unsigned i,j;
01849     char cgflag=gotCGS(FL);
01850
01851     A.init();
01852     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype;
01853     A.setupDATA();
01854
01855     if (cgflag) { A.setupCGR();
01856
01857         for (i=0; i<I.len; ++i) {
01858             if (I[i]>=QIDX.dim1) wblog(FL,
01859                 "ERR %s() index out of bounds (%d/%d)",FCT,I[i]+1,QIDX.dim1);
01860             for (j=0; j<CGR.dim2; ++j) A.CGR(i,j).init(CGR(I[i],j));
01861         }
01862     }
01863
01864     return A;
01865 };
01866
01867 template <class TQ, class TD> inline
01868 void QSpace<TQ,TD>::saveSub2(QSpace<TQ,TD> &A, const wbindex &I) {
01869
01870     if (this==&A) wblog(FL,
01871         "ERR %s() got overlapping input and output space!",FCT);
01872     if (isref) wblog(FL,"ERR %s() called for ISREF!",FCT);
01873
01874     if (!I.len) { A.init(); return; }
01875
01876     unsigned i,j;
01877     char cgflag=gotCGS(FL);
01878
01879     A.init();
01880     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype;
01881     A.setupDATA();
01882     A.setupCGR();
01883
01884     for (i=0; i<I.len; ++i) {
01885         if (I[i]>=DATA.len) wblog(FL,
01886             "ERR %s() index out of bounds (%d,%d)",FCT,I[i],DATA.len);
01887
01888         DATA[I[i]]->save2(*A.DATA[i]);
01889         WB_DELETE_1(DATA[I[i]]);
01890
01891         if (cgflag) {

```

```

01892         for (j=0; j<CGR.dim2; ++j) {
01893             CGR(I[i],j).save2(A.CGR(i,j)); }
01894     }
01895 }
01896
01897 wbindex I2; I.invert(QIDX.dim1,I2);
01898
01899 QIDX.Set2Recs(I2);
01900 DATA.Select(I2); if (cgflag) { CGR.Set2Recs(I2); }
01901 };
01902
01903 template <class TQ, class TD>
01904 void QSpace<TQ,TD>::toFull(
01905     wbararray<TD> &A,
01906     wbvector< wbMatrix<TQ> > &QB,
01907     wbvector< wbvector<unsigned> > &SB
01908 ) const {
01909
01910     if (gotCGS(FL)>0 || gotCGX()) wblog(FL,
01911         "ERR %s() not implemented yet for %d %d",FCT,
01912         gotCGS(FL), gotCGX(FL)
01913     );
01914
01915     widx_t i,j,k,M=QIDX.dim1; unsigned r=-1;
01916     wbvector<widx_t> N,SA,D;
01917     wbararray<unsigned> mark;
01918     wbMatrix<widx_t> G;
01919     wbindex I,J;
01920     wpperm P,iP;
01921
01922     if (!isConsistent(r)) { wblog(FL,"%s",str); }
01923     if (isEmpty()) {
01924         A.init(); QB.init(); SB.init();
01925         return;
01926     }
01927
01928     QB.initDef(r); SB.initDef(r); D.init(r);
01929     I.init(r); P.init(r); iP.init(r); G.init(M,r); N.init(r);
01930
01931     for (k=0; k<r; ++k) {
01932         getQsub(k,QB[k]).groupRecs(P,D); P.invert(iP);
01933
01934         I.BlockIndex(D).Permute(iP);
01935         G.setCol(k,I.data);
01936
01937         N[k]=D.len; SB[k].init(N[k]);
01938     }
01939
01940     mark.init(N+1);
01941
01942     for (i=0; i<M; ++i) {
01943         G.getRec(i,I);
01944
01945         if (mark(I)++) wblog(FL,
01946             "ERR %s() block (%s) not unique (%d)",
01947             FCT,(I+1).toStrf("","").data, mark(I)
01948         );
01949
01950         J.wbvector<widx_t>::operator=(N);
01951
01952         for (k=0; k<r; ++k) {
01953             wbvector<unsigned> &SBk=SB[k];
01954
01955             J[k]=j=I[k]; if (k) J[k-1]=N[k-1];
01956             if (!mark(J)++) {
01957                 SBk[j]=DATA[i]->SIZE[k];
01958             }
01959             else if (SBk[j]!=DATA[i]->SIZE[k]) wblog(FL,
01960                 "ERR %s - block size mismatch (%d: %d,%d)",
01961                 FCT,i+1, SBk[j], DATA[i]->SIZE[k]
01962             );
01963         }
01964     }
01965
01966     SA.init(r); for (k=0; k<r; ++k) SA[k]=SB[k].sum();
01967     A.init(SA);
01968
01969     for (i=0; i<DATA.len; ++i) {
01970         G.getRec(i,I);
01971         A.setBlock(SB, I, *DATA[i]);
01972     }
01973 }
01974
01975 template <class TQ, class TD>
01976 void QSpace<TQ,TD>::toFull12(
01977     wbararray<TD> &A,
01978     wbMatrix<TQ> &QB,

```

```

01979     wbMatrix<widx_t> &SB
01980 ) const {
01981
01982     if (gotCGS(FL)>0 || gotCGX()) wblog(FL,
01983         "ERR %s() not implemented yet for %d %d",FCT,
01984         gotCGS(FL), gotCGX(FL)
01985     );
01986
01987     unsigned i,j,k,r,K,N,M=QIDX.dim1, r_=-1;
01988     wbMatrix<TQ> Q1,Q2;
01989     wbvector<wbvector<widx_t> > S2;
01990     wbvector<widx_t> s,D,SA;
01991     wbMatrix<unsigned> mark; wbMatrix<widx_t> IJ;
01992     wbindex I,I1,I2;
01993     wbarray<TD> a;
01994     wbperm P,iP;
01995
01996     if (!isConsistent(r_)) wblog(FL,"%s",str);
01997     if (isEmpty()) {
01998         A.init(); QB.init(); SB.init();
01999         return;
02000     }
02001
02002     if (r_==3 && getDim(2)==1) { --r_; }
02003
02004     if (r_>2) wblog(FL,"ERR %s even-rank object required (r=%d)", FCT,r_);
02005     r=unsigned(r_);
02006     K=r/2;
02007
02008     I.Index(0,K-1); getQsub(I,Q1);
02009     I.Index(K,r-1); getQsub(I,Q2);
02010
02011     QB.Cat(1,Q1,Q2);
02012
02013     QB.groupRecs(P,D); P.invert(iP);
02014
02015     I.BlockIndex(D); I.Select(iP);
02016
02017     I1.init(M, I.data);
02018     I2.init(M, I.data+M);
02019
02020     N=D.len;
02021     mark.init(N,N+1);
02022     SB.init(N,K);
02023     IJ.init(M,2);
02024
02025     for (k=0; k<M; ++k) {
02026         i=IJ(k,0)=I1[k]; j=IJ(k,1)=I2[k];
02027
02028         if (mark(i,j)++) wblog(FL,
02029             "WRN %s - block (%d,%d) not unique (%d)", FCT, i,j, mark(i,j));
02030
02031         s.init2ref(K, DATA[k]->SIZE.data);
02032         if (mark(i,N)++) {
02033             if (!SB.recEqual(i,s.data)) wblog(FL,
02034                 "ERR %s - block size mismatch (%d: %s != %s)",FCT,k+1,
02035                 SB.rec2Str(i,"","x").data, SSTR(s));
02036         }
02037         else SB.recSet(i,s);
02038
02039         s.init2ref(K, DATA[k]->SIZE.data+K);
02040         if (mark(j,N)++) {
02041             if (!SB.recEqual(j,s.data)) wblog(FL,
02042                 "ERR %s - block size mismatch (%d: %s != %s)",FCT,k+1,
02043                 SB.rec2Str(j,"","x").data, SSTR(s));
02044         }
02045         else SB.recSet(j,s);
02046     }
02047
02048     S2.init(2); SB.recProd(S2[0]); S2[1]=S2[0];
02049     SA.init(2); SA[0]=SA[1]=S2[0].sum();
02050
02051     A.init(SA);
02052
02053     for (i=0; i<DATA.len; ++i) {
02054         I.init2ref(2,IJ.rec(i));
02055
02056         a.init2ref(*DATA[i]);
02057         a.GroupIndices(K);
02058
02059         A.setBlock(S2,I,a);
02060     }
02061 }
02062
02063 template <class TQ, class TD>
02064 void QSpace<TQ,TD>::ExtendQ(const QSpace<TQ,TD> &B) {
02065

```

```

02066     unsigned i,k,m,n=B.QIDX.dim1;
02067     wvector<char> mark;
02068     windex Ia,Ib;
02069
02070     if (B.isEmpty()) return;
02071     if (isEmpty()) {
02072         (*this)=B;
02073         for (i=0; i<DATA.len; ++i) DATA[i]->set(TD(0));
02074         return;
02075     }
02076
02077     if (QIDX.dim2!=B.QIDX.dim2) wblog(FL,
02078     "ERR %s - size mismatch (%d/%d)", FCT, QIDX.dim2, B.QIDX.dim2);
02079
02080     matchIndex(QIDX,B.QIDX,Ia,Ib);
02081
02082     mark.init(n);
02083     for (i=0; i<Ib.len; ++i) mark[Ib[i]]++;
02084
02085     m=mark.count(0);
02086     if (m==0) return;
02087
02088     k=QIDX.dim1;
02089     Append(m);
02090
02091     for (i=0; i<n; ++i) {
02092         if (mark[i]) continue;
02093         QIDX.recSetP(k,B.QIDX.rec(i));
02094         DATA[k]->init(B.DATA[i]->SIZE);
02095         ++k;
02096     }
02097 };
02098
02099 template <class TQ, class TD>
02100 void QSpace<TQ,TD>::takeMainQS(char disp) {
02101
02102     unsigned i,j,n,i0,imax=0;
02103     wvector<widxt_t> d, dc, I;
02104     wperm P;
02105
02106     wbMatrix<TQ> QT;
02107     wvector<TD> Z;
02108     TD z, zmax=0, ztot=0, eps=1E-3;
02109
02110     getQtot(QT);
02111     QT.groupRecs(P,d);
02112
02113     d.cumsum_(dc); Z.init(d.len);
02114
02115     for (i=0; i<d.len; ++i) {
02116         for (z=0, n=d[i], i0=dc[i], j=0; j<n; ++j)
02117             z+=DATA[P[i0+j]]->norm2();
02118
02119         Z[i]=z;
02120
02121         if (z>zmax) { zmax=z; imax=i; }
02122         ztot+=z;
02123     }
02124
02125     if (disp) { char s_[128];
02126         z=fabs((ztot-zmax)/ztot);
02127         snprintf(s_,128,"%s skipping weight %s of total %s (%s%s)\n"
02128         "    Q=[%s]", (ABS(z)>eps ? "WRN" : " * "),
02129         NSTR(ztot-zmax), NSTR(ztot), NSTR(z*100.),
02130         ABS(z)>0.1 ? "% !?" : "%", QT.rec2Str(imax).data
02131         );
02132
02133         wblog(FL,s_);
02134     }
02135
02136     I.init(d[imax], P.data+dc[imax]);
02137
02138     QIDX.Set2Recs(I);
02139     DATA.Select(I);
02140 }
02141
02142 template <class TQ, class TD> inline
02143 wbMatrix<TQ>& QSpace<TQ,TD>::getQsub(
02144     const unsigned k, wbMatrix<TQ> &Qk, char iflag
02145 ) const {
02146
02147     unsigned i, r=rank(FL), s=QDIM*sizeof(TQ), offset=k*QDIM;
02148
02149     if (iflag) {
02150         windex I; I.Index_ex(r,k); getQsub(I,Qk);
02151         return Qk;
02152     }

```

```

02153
02154     if (k>=r) wblog(FL,
02155         "ERR %s() index out of bounds (%dx%d/%d; %d)",
02156         FCT, QIDX.dim1, QIDX.dim2, QDIM, k);
02157
02158     Qk.init(QIDX.dim1, QDIM);
02159
02160     for (i=0; i<QIDX.dim1; ++i)
02161         memcpy(Qk.rec(i), QIDX.rec(i)+offset, s);
02162
02163     return Qk;
02164 };
02165
02166 template <class TQ, class TD> inline
02167 wbMatrix<TQ>& QSpace<TQ,TD>::getQsub(
02168     const wbindex &I,
02169     wbMatrix<TQ> &QI
02170 ) const {
02171     unsigned i,k, r=rank(FL);
02172     TQ *q;
02173
02174     if (&QI==&QIDX) wblog(FL,
02175         "ERR %s() QIN and QOUT space are the same!",FCT);
02176     for (i=0; i<I.len; ++i) if (I[i]>=r) wblog(FL,
02177         "ERR %s() index out of bounds (%d/%d)",FCT,I[i],r);
02178
02179     QI.init(QIDX.dim1, I.len*QDIM);
02180
02181     for (i=0; i<QIDX.dim1; ++i) {
02182         const TQ *q0=QIDX.rec(i); q=QI.rec(i);
02183
02184         for (k=0; k<I.len; ++k, q+=QDIM) {
02185             Wb::cpyRange(q, q0+I[k]*QDIM, QDIM);
02186         }
02187     }
02188
02189     return QI;
02190 }
02191
02192 template <class TQ, class TD> inline
02193 void QSpace<TQ,TD>::getQsub(
02194     const wbindex &I,
02195     wbMatrix<TQ> &QI,
02196     wbMatrix<TQ> &Q2
02197 ) const {
02198
02199     unsigned i, k, r=rank(FL), s=QDIM*sizeof(TQ);
02200     wbindex I2;
02201     TQ *q1,*q2; const TQ *q0;
02202
02203     I.invert(r,I2);
02204
02205     if (&QI==&QIDX || &Q2==&QIDX)
02206         wblog(FL,"ERR getQsub requires distinct QOUT space!");
02207
02208     QI.init(QIDX.dim1, I .len*QDIM);
02209     Q2.init(QIDX.dim1, I2.len*QDIM);
02210
02211     for (i=0; i<QIDX.dim1; ++i) {
02212         q0=QIDX.rec(i); q1=QI.rec(i); q2=Q2.rec(i);
02213
02214         for (k=0; k<I .len; ++k) memcpy(q1+k*QDIM, q0+I [k]*QDIM, s);
02215         for (k=0; k<I2.len; ++k) memcpy(q2+k*QDIM, q0+I2[k]*QDIM, s);
02216     }
02217 }
02218
02219 template <class TQ, class TD> inline
02220 wbMatrix<TQ>& QSpace<TQ,TD>::getQtot(
02221     wbMatrix<TQ> &Qtot,
02222     char uflag
02223 ) const {
02224
02225     unsigned i,k,l,r, r0=rank(FL);
02226     TQ *q; const TQ *q0;
02227
02228     if (&Qtot==&QIDX)
02229         wblog(FL,"ERR getQtot requires distinct QOUT space!");
02230
02231     Qtot.init(QIDX.dim1, QDIM);
02232
02233     for (i=0; i<QIDX.dim1; ++i) {
02234         q0=QIDX.rec(i); q=Qtot.rec(i);
02235         for (l=r0; r<0; ++r)
02236             for (k=0; k<QDIM; ++k, ++l) q[k]+=q0[l];
02237     }
02238
02239     if (uflag) Qtot.makeUnique();

```

```

02240
02241     return Qtot;
02242 }
02243
02244 template <class TQ, class TD> inline
02245 void QSpace<TQ,TD>::getDistQtot(
02246     wbMatrix<TQ> &Q, wbvector<double> &w
02247 ) const {
02248     unsigned i,j,l,d;
02249     wbperm P; wbvector<wid_x_t> D;
02250     double n2;
02251
02252     QIDX.blockSum(QDIM,Q); Q.groupRecs(P,D);
02253     w.init(D.len);
02254
02255     for (l=i=0; i<D.len; ++i, l+=d) { d=D[i];
02256         for (n2=0, j=0; j<d; ++j) n2 += (DATA[P[l+j]]->norm2());
02257         w[i]=sqrt((double)n2);
02258     }
02259 }
02260
02261 template <class TQ, class TD> inline
02262 wbMatrix<TQ>& QSpace<TQ,TD>::getQsum(
02263     const wbindex &J, wbMatrix<TQ> &Q
02264 ) const {
02265
02266     unsigned i,j, r=rank(FL);
02267     const TQ *q0; TQ *q;
02268
02269     if (J.isEmpty()) { Q.init(); return Q; }
02270
02271     for (i=0; i<J.len; ++i) if (J[i]>=r) wblog(FL,
02272         "ERR block index out of range (%d/%d)",J[i],r);
02273
02274     Q.init(QIDX.dim1,QDIM);
02275
02276     if (allAdditive() || J.len==1) {
02277         for (i=0; i<QIDX.dim1; ++i) {
02278             q=Q.rec(i); q0=QIDX.rec(i);
02279             for (j=0; j<J.len; ++j) Wb::addRange(q0+J[j]*QDIM, q, QDIM);
02280         }
02281     }
02282     else if (allAbelian()) {
02283         if (qtype.len!=QDIM) wblog(FL,
02284             "ERR %s() got inconsistent qtype (len=%d/%d; %s)",
02285             FCT,qtype.len,QDIM,qStr().data
02286         );
02287
02288         for (i=0; i<QIDX.dim1; ++i) { q=Q.rec(i); q0=QIDX.rec(i);
02289             for (j=0; j<QDIM; ++j) {
02290                 q[j]=get_qtot_abelian(qtype[j],q0+j,J.len,J.data,QDIM);
02291             }
02292         }
02293     }
02294     else wblog(FL,
02295         "ERR %s() got non-abelian symmetries (%s)",FCT,qStr().data);
02296
02297     return Q;
02298 };
02299
02300 template <class TQ, class TD> inline
02301 wbMatrix<TQ>& QSpace<TQ,TD>::getQsum(
02302     const wbMatrix<TQ> &Q0, wbMatrix<TQ> &Q
02303 ) const {
02304
02305     if (!Q0) { Q.init(); return Q; }
02306     if (Q0.dim2==QDIM) { Q=Q0; return Q; }
02307
02308     if (!QDIM || (Q0.dim2 % QDIM)) wblog(FL,
02309         "ERR size mismatch Q0 (%dx%d /%d)",Q0.dim1,Q0.dim2,QDIM);
02310
02311     unsigned i,j, m=Q0.dim2/QDIM;
02312     const TQ *q0; TQ *q;
02313
02314     Q.init(Q0.dim1,QDIM);
02315
02316     if (allAdditive()) {
02317         for (i=0; i<Q0.dim1; ++i) { q=Q.rec(i); q0=Q0.rec(i);
02318             for (j=0; j<m; ++j) Wb::addRange(q0+j*QDIM, q, QDIM);
02319         }
02320     }
02321     else if (allAbelian()) {
02322         if (qtype.len!=QDIM) wblog(FL,
02323             "ERR %s() got inconsistent qtype (len=%d/%d; %s)",
02324             FCT,qtype.len,QDIM,qStr().data
02325         );
02326     }

```

```

02327         for (i=0; i<Q0.dim1; ++i) { q=Q.rec(i); q0=Q0.rec(i);
02328             for (j=0; j<QDIM; ++j) {
02329                 q[j]=get_qtot_abelian(qtype[j],q0+j,m,NULL,QDIM);
02330             }
02331         }
02332     }
02333     else wblog(FL,"ERR %s() "
02334         "got non-abelian symmetries (%s, m=%d)",FCT,qStr().data,m);
02335
02336     return Q;
02337 };
02338
02339 template <class TQ, class TD> inline
02340 void QSpace<TQ,TD>::FlipQ() {
02341     wbperm P(rank(FL)); P.Flip();
02342     PermuteQ(P);
02343 }
02344
02345 template <class TQ, class TD> inline
02346 void QSpace<TQ,TD>::PermuteQ(const wbperm &P) {
02347     unsigned i,k,r=rank(FL), s=QDIM*sizeof(TQ);
02348     wbMatrix<TQ> Q0(QIDX);
02349     TQ *q0, *qp;
02350
02351     if (!QDIM) wblog(FL,"WRN QDIM=0!");
02352     if (P.isEmpty()) return;
02353     if (isEmpty() || QIDX.dim2==0) return;
02354
02355     if (!P.isValidPerm(r)) wblog(FL,
02356         "ERR %s() invalid permutation [%s] (%d)",FCT,STR(P),r);
02357
02358     for (i=0; i<QIDX.dim1; ++i) {
02359         q0=Q0.rec(i); qp=QIDX.rec(i);
02360         for (k=0; k<P.len; ++k) memcpy(qp+k*QDIM, q0+P[k]*QDIM, s);
02361     }
02362 }
02363 }
02364
02365 template <class TQ, class TD> inline
02366 void QSpace<TQ,TD>::permuteQ(const wbperm &P, wbMatrix<TQ> &Q) const {
02367     if (&Q==&QIDX) {
02368         wblog(FL,"ERR You shall not permuteQ onto itself");
02369     }
02370     else {
02371         unsigned i, k, r=rank(FL), s=QDIM*sizeof(TQ);
02372         const TQ *q0; TQ *qp;
02373
02374         if (P.isEmpty()) { Q=QIDX; return; }
02375
02376         if (!P.isValidPerm(r)) wblog(FL,
02377             "ERR invalid permutation '%s'",STR(P));
02378
02379         Q.init(QIDX.dim1, QIDX.dim2);
02380
02381         for (i=0; i<QIDX.dim1; ++i) {
02382             q0=QIDX.rec(i); qp=Q.rec(i);
02383             for (k=0; k<P.len; ++k) memcpy(qp+k*QDIM, q0+P[k]*QDIM, s);
02384         }
02385     }
02386 }
02387 }
02388
02389 template <class TQ, class TD>
02390 QSpace<TQ,TD>& QSpace<TQ,TD>::Permute(
02391     const wbperm &P_, char iflag,
02392     char rcpy
02393 ) {
02394     unsigned i,n,r=rank(FL);
02395     if ((!r && isEmpty()) || !P_.len) { return *this; }
02396
02397     if (P_.len>r) { wblog(FL,
02398         "ERR %s() invalid permutation '%s'\n(length out of bounds: %d/%d)",
02399         myname,STR(P_),P_.len,r);
02400     }
02401     if (P_.isIdentityPerm(FL)) { return *this; }
02402
02403     wbperm P(P_,iflag,r);
02404     PermuteQ(P);
02405
02406     for (i=0; i<DATA.len; ++i) { DATA[i]->Permute(P,0,rcpy); }
02407     if (itags.len) { itags.Select(P); }
02408
02409     if (CGR) {
02410         for (n=CGR.numel(), i=0; i<n; ++i) {
02411             CGR[i].Permute(P);
02412         }
02413     }

```



```

02414     NormCGW(0,0,rcpy);
02415 }
02416
02417 return *this;
02418 };
02419
02420 template <class TQ, class TD> inline
02421 QSpace<TQ,TD>& QSpace<TQ,TD>::permute(
02422     const wbperm &P_, QSpace<TQ,TD> &B, char iflag
02423 ) const {
02424
02425     unsigned r=rank(FL);
02426     if (P_.isEmpty() || P_.isIdentityPerm(FL,r)) {
02427         B.init(*this); return B;
02428     }
02429
02430     unsigned i,n;
02431     wbperm P(P_,iflag); B.init();
02432
02433     permuteQ(P,B.QIDX);
02434     B.QDIM=QDIM; B.qtype=qtype; B.itags=itags;
02435
02436     if (otype && P.isOpTranspose()) {
02437         B.otype=otype;
02438     }
02439
02440     if (B.itags.len) B.itags.Select(P);
02441
02442     B.setupDATA();
02443     for (n=DATA.len, i=0; i<n; ++i) {
02444         DATA[i]->permute(*B.DATA[i],P);
02445     }
02446
02447     if (CGR.isEmpty()) { B.CGR.init(); }
02448     else {
02449         B.setupCGR(); n=CGR.numel();
02450         if (n!=B.CGR.numel()) wblog(FL,"ERR %s() got CGR size mismatch "
02451             "%s <> %s",FCT,CGR.sizeStr().data,B.CGR.sizeStr().data);
02452         for (i=0; i<n; ++i) {
02453             CGR[i].permute(P,B.CGR[i]);
02454         }
02455     }
02456
02457     return B;
02458 };
02459
02460 template <class TQ, class TD> inline
02461 void QSpace<TQ,TD>::setQ(unsigned k,
02462     const QSpace<TQ,TD> &B, unsigned k0
02463 ) {
02464
02465     unsigned i, s=QDIM*sizeof(TQ), dd0=B.QIDX.dim2, dd2=QIDX.dim2;
02466     TQ *d0 = B.QIDX.data+k0*QDIM, *d2 = QIDX.data+k*QDIM;
02467
02468     if (QIDX.dim1!=B.QIDX.dim1 || QDIM!=B.QDIM) wblog(FL,
02469         "ERR %s() incompatible objects (%d,%d; %d,%d)",
02470         FCT, QIDX.dim1, B.QIDX.dim1, QDIM, B.QDIM);
02471
02472     if (rank(FL)<=k || B.rank(FL)<=k0) wblog(FL,
02473         "ERR %s() index out of bounds (%d,%d; %d,%d)",
02474         FCT, k, rank(FL), k0, B.rank(FL));
02475
02476     for (i=0; i<QIDX.dim1; ++i, d0+=dd0, d2+=dd2)
02477         memcpy(d2,d0,s);
02478 }
02479
02480 template <class TQ, class TD> inline
02481 void QSpace<TQ,TD>::setQRec(
02482     unsigned r, unsigned j, const TQ *q0, const TQ *q2
02483 ) {
02484     if (r>=QIDX.dim1 || QDIM*(j+1)>QIDX.dim2) wblog(FL,
02485         "ERR %s() index out of bounds (%d,%d; %d,%d)",
02486         FCT, r, j, QIDX.dim1, QDIM);
02487
02488     TQ *q = QIDX.rec(r)+j*QDIM;
02489     if (q2==NULL)
02490         for (unsigned i=0; i<QDIM; ++i) q[i]=q0[i];
02491     else for (unsigned i=0; i<QDIM; ++i) q[i]=q0[i]+q2[i];
02492 }
02493
02494 template <class TQ, class TD> inline
02495 bool QSpace<TQ,TD>::hasEqualQ(unsigned i1, unsigned i2) const {
02496
02497     unsigned i, s=QDIM*sizeof(TQ), D=QIDX.dim2, r=rank(FL);
02498     TQ *d1 = QIDX.data+i1*QDIM, *d2 = QIDX.data+i2*QDIM;
02499
02500     if (i1>=r || i2>=r) wblog(FL,

```

```

02501     "ERR %s() index out of bounds (%d,%d; %d)",FCT,i1,i2,r);
02502
02503     for (i=0; i<QIDX.dim1; ++i, d1+=D, d2+=D)
02504         if (memcmp(d1,d2,s)) return 0;
02505
02506     return 1;
02507 };
02508
02509 template <class TQ, class TD>
02510 bool QSpace<TQ,TD>::hasQOverlap(const QSpace<TQ,TD> &A) const {
02511
02512     wbindex Ia, Ib;
02513     if (qtype!=A.qtype) {
02514         wblog(FL,"WRN got objects with different qtype");
02515         return 0;
02516     }
02517
02518     matchIndex(QIDX, A.QIDX, Ia, Ib);
02519     return (Ia.len!=0);
02520 };
02521
02522 template <class TQ, class TD>
02523 template <class TB>
02524 int QSpace<TQ,TD>::hasQOverlap(
02525     unsigned ica, const QSpace<TQ,TB> &B, unsigned icb,
02526     const char type
02527 ) const {
02528
02529     unsigned ra=rank(FL), rb=B.rank(FL); int q=0;
02530     wbMatrix<TQ> QA,QB;
02531     wbindex Ia,Ib,Ja,Jb;
02532
02533     if (!ica || !icb) wblog(FL,
02534         "ERR %s() index must be 1-based (%d,%d)",FCT,ica,icb);
02535     --ica; --icb;
02536
02537     if (QDIM!=B.QDIM) wblog(FL,
02538         "ERR %s() incompatible objects\nQDIM=%d,%d",
02539         FCT, QDIM, B.QDIM);
02540     if (ica>ra || icb>rb) wblog(FL,
02541         "ERR %s() index out of bounds\n%d/%d, %d/%d (QDIM=%d,%d)",
02542         FCT, ica,ra, icb,rb, QDIM, B.QDIM);
02543
02544     getQsub(ica,QA).makeUnique(Ja);
02545     B.getQsub(icb,QB).makeUnique(Jb); matchIndex(QA,QB,Ia,Ib);
02546
02547     if (Ia.len==QA.dim1) { q|=2; }
02548     if (Ib.len==QB.dim1) { q|=4; }
02549
02550     if (type=='<') { if (!(q&2)) { q=0; }} else
02551     if (type=='>') { if (!(q&4)) { q=0; }} else
02552     if (type=='=') { if ( q!=6) { q=0; }} else {
02553         if (type) wblog(FL,"WRN %s() invalid type=%c<d>",FCT,type,type);
02554         if (!Ia.len) { q=0; }
02555     }
02556
02557     if (q && type) {
02558         for (unsigned i=0; i<Ia.len; ++i) {
02559             if (!DATA[Ja[Ia[i]]]->hasSameSize(ica, *B.DATA[Jb[Ib[i]]],icb)) {
02560                 q=-q; break;
02561             }
02562         }
02563     }
02564
02565     return q;
02566 };
02567
02568 template <class TQ, class TD>
02569 mxArray* QSpace<TQ,TD>::QIDX_toMx() const {
02570
02571     unsigned k, r;
02572     mxArray *a;
02573
02574     if (QIDX.isEmpty())
02575         return mxCreateCellMatrix(0,0);
02576
02577     if (!QDIM || QIDX.dim2%QDIM) wblog(FL,
02578         "ERR %s() invalid QSpace (QIDX: %dx%d; %d)",
02579         FCT,QIDX.dim1,QIDX.dim2,QDIM
02580     );
02581
02582     r=QIDX.dim2/QDIM; a=mxCreateCellMatrix(1,r);
02583     if (!a) wblog(FL,"ERR could not allocate cell array (%d)", r);
02584
02585     for (k=0; k<r; ++k)
02586         mxSetCell(a, k, getQsub(k).toMx());
02587

```

```

02588     return a;
02589 }
02590
02591 template <class TQ, class TD>
02592 mxArray* QSpace<TQ,TD>::INFO_toMx() const {
02593
02594     if ((qtype.isEmpty() && CGR.isEmpty()) &&
02595         itags.isEmpty() && otype==QS_NONE)
02596     { return mxCreateCellMatrix(0,0); }
02597
02598     const char *fields[] = { "qtype","otype","itags","ctime","cgr" };
02599
02600     mxArray *S=mxCreateStructMatrix(1,1,5,fields);
02601
02602     mxSetFieldByNumber(S,0,0,qStr().toMx());
02603     mxSetFieldByNumber(S,0,1,otype2Str().toMx());
02604     mxSetFieldByNumber(S,0,2,itags.toMx());
02605     mxSetFieldByNumber(S,0,3,numtoMx(ctime));
02606
02607     mxSetFieldByNumber(S,0,4,CGR.toMx());
02608
02609     return S;
02610 };
02611
02612 template <class TQ, class TD>
02613 mxArray* QSpace<TQ,TD>::DATA_toMx() const {
02614
02615     mxArray *a=mxCreateCellMatrix(DATA.len, DATA.len ? 1 : 0);
02616     if (!a) wblog(FL,"ERR failed to allocate cell array (%d)",DATA.len);
02617     if (DATA.len) {
02618         unsigned i, n=DATA.len; mxArray* x[n];
02619         int np=MAX( QSP_NUM_THREADS, OMP_NUM_THREADS );
02620         if (np>int(n)) { np=(n ? n : 1); }
02621         for (i=0; i<n; ++i) { x[i]=0; }
02622
02623         Wb::LogException ex;
02624
02625         #pragma omp parallel for num_threads(np)
02626         for (i=0; i<n; ++i) { if (!ex) {
02627             try { x[i]=DATA[i]->toMx(); }
02628             catch (Wb::LogException &e_) { ex+=e_; }
02629             catch (...) { ++ex; }
02630         } }
02631         ex.report(FLF);
02632
02633         for (i=0; i<n; ++i) { if (x[i]) mxSetCell(a,i,x[i]); }
02634     }
02635
02636     return a;
02637 };
02638
02639 template <class TQ, class TD>
02640 mxArray* QSpace<TQ,TD>::DATA_save2Mx(char vflag) {
02641
02642     mxArray *a=mxCreateCellMatrix(DATA.len, DATA.len ? 1 : 0);
02643     if (!a) wblog(FL,"ERR failed to allocate cell array (%d)",DATA.len);
02644
02645     if (vflag) { size_t sd=getDataSize();
02646         if (sd<(1<30)) vflag=0; else wblog(FL,
02647             "TST %s() %d entries @ %.3gG ...",FCT,DATA.len,sd/double(1<30));
02648     }
02649
02650     for (unsigned k=0; k<DATA.len; ++k) {
02651         mxSetCell(a, k, DATA[k]->toMx());
02652         if (vflag && k%10==0) PRINTF(" %6d/%1d ... \r",k+1,DATA.len);
02653         DATA[k]->init();
02654     }
02655     if (vflag) PRINTF("\r%60s\r","");
02656
02657     return a;
02658 };
02659
02660 template <class TQ, class TD>
02661 mxArray* QSpace<TQ,TD>::toMx(bool setcls) const {
02662
02663     mxArray *S=mxCreateStruct(1,1);
02664     this->add2MxStruct(S,0,setcls);
02665     return S;
02666 };
02667
02668 template <class TQ, class TD>
02669 mxArray* QSpace<TQ,TD>::save2Mx(char vflag) {
02670
02671     mxArray *S=mxCreateStruct(1,1);
02672     this->save2MxStruct(S,0,0,vflag);
02673     return S;
02674 };

```

```

02675
02676 template <class TQ, class TD>
02677 mxArray* QSpace<TQ,TD>::mxCreatStruct(unsigned m, unsigned n) const {
02678
02679     const char *fields[] = { "Q","data","info" };
02680     return mxCreatStructMatrix(m,n,3,fields);
02681 }
02682
02683 template <class TQ, class TD>
02684 void QSpace<TQ,TD>::add2MxStruct(
02685     mxArray *S, unsigned i,
02686     bool setcls,
02687     char chk
02688 ) const {
02689
02690     mxArray *a;
02691
02692 #ifdef WBC_QSPACE_IO
02693     Wb::Clock clk("QS:toMx",1);
02694 #endif
02695
02696     if (chk) { unsigned s=0; int q=0;
02697     if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s ||
02698         (q=mxGetFieldNumber(S,"qtype"))<0) wblog(FL,
02699             "ERR %s() must follow mxCreatStruct()\n%lx, %d/%d, %d",
02700             FCT,S,i+1,s,q);
02701     }
02702
02703     a=QIDX_toMx(); mxSetFieldByNumber(S,i,0,a);
02704     a=DATA_toMx(); mxSetFieldByNumber(S,i,1,a);
02705     a=INFO_toMx(); mxSetFieldByNumber(S,i,2,a);
02706
02707     if (setcls && Wb::my_caller_tid==omp_get_thread_num()) {
02708         mxSetClassName(S,"QSpace");
02709     }
02710 };
02711
02712 template <class TQ, class TD>
02713 void QSpace<TQ,TD>::save2MxStruct(
02714     mxArray *S, unsigned i, char tst, char vflag
02715 ){
02716     mxArray *a;
02717
02718 #ifdef WBC_QSPACE_IO
02719     Wb::Clock clk("QS:toMx",1);
02720 #endif
02721
02722     if (tst) {
02723         unsigned s=0; int q=0;
02724         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s
02725             || (q=mxGetFieldNumber(S,"qtype"))<0) wblog(FL,
02726                 "ERR %s() must follow mxCreatStruct()\n%lx, %d/%d, %d",
02727                 FCT,S,i+1,s,q);
02728     }
02729
02730     a=QIDX_toMx();          mxSetFieldByNumber(S,i,0,a);
02731     a=DATA_save2Mx(vflag);  mxSetFieldByNumber(S,i,1,a);
02732     a=INFO_toMx();          mxSetFieldByNumber(S,i,2,a);
02733
02734     init();
02735 };
02736
02737 template <class TQ, class TD>
02738 void QSpace<TQ,TD>::init(
02739     const char *F, int L, const mxArray *S,
02740     char ref,
02741     unsigned k,
02742     char skip_empty
02743 ){
02744     unsigned i,j,l,m,n,r,dim1=0,dim2=0; unsigned rk=-1;
02745     mxArray *aq,*ad,*ai;
02746     wbvector< wbMatrix<TQ> > MQ;
02747     wbvector< wbMatrix<TQ>* > mq;
02748     const char cflag = (typeid(TD)==typeid(wbcomplex) ? 'C' : 0);
02749     char refD=0, refC=1, isa=0;
02750
02751 #ifdef WBC_QSPACE_IO
02752     Wb::Clock clk("QS:mXInit",1);
02753 #endif
02754
02755     clearQSpace();
02756
02757     if (!S || mxIsEmpty(S) || mxIsEmptyQSpace(S,k)) return;
02758
02759     aq=mxGetField(S,k,"Q");
02760     ad=mxGetField(S,k,"data");
02761     ai=mxGetField(S,k,"info");

```

```

02762
02763     if (mxIsQSpace(F_L,S,rk,cflag,k)<=0 || !aq || !ad) wblog(F_L,
02764         "ERR invalid QSpace (%d,%d)",aq==0,ad==0);
02765
02766     if (ref) {
02767         if (ref=='r') { refD = ref; } else
02768             wblog(F_L,"ERR %s() got invalid ref=%c<%d>",FCT,ref,ref);
02769     }
02770
02771
02772     m=mxGetM(aq); r=mxGetN(aq);
02773
02774     if ((m!=1 && r!=1 && m && r) || mxGetNumberOfDimensions(aq)>2) {
02775         wblog(F,L,"ERR %s() invalid QSpace\n"
02776             "Q must be blocked into row cell vector (%dx%d; %d)",
02777             FCT,m,r,mxGetNumberOfDimensions(aq));
02778     }
02779
02780     r*=m; MQ.init(r);
02781
02782     for (i=0; i<r; ++i) {
02783         MQ[i].init(mxGetCell(aq,i));
02784
02785         if (i==0) {
02786             dim1=MQ[i].dim1; dim2=MQ[i].dim2;
02787         }
02788         else if (MQ[i].dim1!=dim1 || MQ[i].dim2!=dim2) { wblog(F,L,
02789             "ERR %s() invalid QSpace\nsize mismatch in Q data (%dx%d; %dx%d)",
02790             FCT, MQ[i].dim1, MQ[i].dim2, dim1, dim2);
02791         }
02792     }
02793
02794     mq.init(MQ.len); for (i=0; i<MQ.len; ++i) mq[i] = &MQ[i];
02795
02796     QIDX.CAT(2, (const wbMatrix<TQ>**) mq.data, mq.len);
02797     QDIM=dim2;
02798
02799     if (!QIDX.isUnique()) wblog(F_L,
02800         "ERR %s() invalid input QSpace\nrows in [Q{:}] are not unique",FCT);
02801     setupDATA();
02802
02803     if (ai && !mxIsEmpty(ai)) {
02804         if (mxIsStruct(ai)) {
02805             mxArray *a; QVec qv; unsigned dq=0;
02806
02807             a=mxGetField(ai,0,"qtype");
02808             if (a) qtype.init(F,L,a); else qtype.init();
02809
02810             isa=qtype.allAbelian();
02811             if (isa==1 && qtype.len) qtype.ReduceU1();
02812
02813             initOType(F_L,mxGetField(ai,0,"otype"));
02814
02815             itags.init(F_L,mxGetField(ai,0,"itags"));
02816
02817             if (itags.len && itags.len!=r) wblog(F_L,
02818                 "ERR %s() invalid number of itags (%s; %d/%d)",
02819                 FCT,IT2STR__,itags.len,r);
02820
02821             a=mxGetField(ai,0,"ctime");
02822             if (a && !mxIsEmpty(a)) { mxGetNumber(a,ctime); }
02823
02824             a=mxGetField(ai,0,"cgr");
02825             if (a && !mxIsEmpty(a)) {
02826
02827                 QSet<TQ> Q;
02828                 char xflag=(isa ? 0 : 1);
02829                 wbvector<unsigned> qdc; qtype.Qpos(qdc);
02830
02831                 setupCGR();
02832                 m=mxGetM(a); n=mxGetN(a);
02833
02834                 if (m==1 && !QIDX.dim1) {
02835                     if (!mxIsScalarQSpace(S) || m>1 || r) wblog(F_L,
02836                         "ERR %s() invalid scalar QSpace",FCT);
02837                     QIDX.init(1,0); QDIM=qtype.Qlen(); r=2;
02838                     setupDATA(); setupCGR();
02839                 }
02840                 else
02841                     if (!QDIM || QIDX.dim2%QDIM || QIDX.dim2/QDIM!=r) wblog(F_L,
02842                         "ERR %s() empty QDIM while info.cgr is specified (%d/%d)",
02843                         FCT,m,n);
02844                 else
02845                     if (mxGetNumberOfDimensions(ad)>2 ||
02846                         m!=QIDX.dim1 || m!=CGR.dim1 ||
02847                         n!=qtype.len || n!=CGR.dim2 || qtype.Qlen()!=QDIM)
02848                     wblog(F_L,"ERR %s() invalid dimensions for cell array "

```

```

02849         "info.cgr\n{%s} cgr: %dx%d / %dx%d; QIDX: %dx%d/%d",
02850         FCT, qStr().data, m,n, CGR.dim1, CGR.dim2,
02851         QIDX.dim1, QIDX.dim2, QDIM
02852     );
02853
02854     if (!mxIsCell(a) && !mxIsStruct(a)) wblog(FL,
02855     "ERR %s() cell array expected for info.cgr field (%s)",
02856     FCT,mxGetClassName(a)
02857     );
02858
02859     for (l=j=0; j<n; ++j)
02860     for ( i=0; i<m; ++i, ++l) {
02861         Q.init(qtype[j], QIDX.rec(i)+qdc[j], r, QDIM, itags);
02862         CGR(l,j).init(F_L,a,l,refC,&Q,xflag);
02863     }
02864 }
02865
02866 if (qtype.len) { dq=qtype.Qlen();
02867 if (dq!=QDIM && (QIDX.dim2 || CGR.dim2)) wblog(FL,"ERR "
02868 "incompatible number of symmetry labels (%d/%d; %s; %s)",
02869 QDIM, dq, QIDX.sizeStr().data, CGR.sizeStr().data
02870 );
02871 }
02872 }
02873 else wblog(F,L,"ERR invalid qtype (structure required)");
02874
02875 if (isa) {
02876     if (!CGR.isEmpty()) {
02877         for (i=0; i<CGR.dim1; ++i)
02878         for (j=0; j<CGR.dim2; ++j) { if (CGR(i,j).cgb) {
02879             const cdata__ &c=(CGR(i,j).cgb->cgd);
02880             if (!c.isScalar()) wblog(FL,
02881             "ERR %s() invalid abelian CGC space (%s)",FCT,SSTR(c));
02882             if (c.D.data && c.D.data[0]!=1) wblog(FL,
02883             "ERR %s() invalid abelian CGC coefficient (%.4g)",
02884             FCT,double(c.D.data[0])
02885             );
02886         }}
02887         checkQ_CGR(FL); CGR.init();
02888     }
02889 }
02890 }
02891 }
02892 else qtype.init();
02893
02894 m=mxGetM(ad); n=mxGetN(ad);
02895
02896 if ((m>1 && n>1) || mxGetNumberOfDimensions(ad)>2) wblog(F,L,
02897 "ERR QSpace() cell VECTOR expected for data (%d,%d)",m,n);
02898
02899 m*=n;
02900
02901 if (DATA.len!=m) {
02902     if (m==1 && DATA.len==0 && QIDX.isEmpty() && CGR.isEmpty()) {
02903         QIDX.init(1,0); setupDATA();
02904         if (r<2) r=2;
02905     }
02906     else { wblog(F,L,
02907     "ERR QSpace() dimension mismatch (QIDX: %dx%d, data: %d)",
02908     QIDX.dim1, QIDX.dim2, m);
02909     }
02910 }
02911
02912 for (n=i=0; i<m; ++i) {
02913     DATA[i]->init(mxGetCell(ad,i), refD);
02914     DATA[i]->appendSingletons(FL,r,1);
02915     if (!*DATA[i]) { ++n; }
02916 }
02917
02918 if (n && skip_empty) {
02919     m=SkipEmptyData(FL);
02920 }
02921
02922 };
02923
02924 template <class TQ, class TD> inline
02925 void QSpace<TQ,TD>::init2ref(const QSpace<TQ,TD> &A) {
02926
02927     clearQSpace();
02928
02929     QIDX.init2ref(A.QIDX); QDIM=A.QDIM;
02930     DATA.init2ref(A.DATA);
02931     qtype.init2ref(A.qtype); otype=A.otype;
02932     itags.init2ref(A.itags);
02933
02934     if (!A.CGR.isEmpty()) CGR.init2ref(A.CGR);
02935 }

```

```

02936     isref=1;
02937 };
02938
02939 template <class TQ, class TD> inline
02940 void QSpace<TQ,TD>::setRand(double nrm) {
02941
02942     unsigned i;
02943     for (i=0; i<DATA.len; ++i) DATA[i]->setRand();
02944
02945     if (nrm>0) {
02946         double fac=nrm/sqrt(double(norm2()));
02947         for (i=0; i<DATA.len; ++i) (*DATA[i]) *= fac;
02948     }
02949 }
02950
02951 template <class TQ, class TD>
02952 void QSpace<TQ,TD>::info(const char *vname,
02953     unsigned nlt, unsigned nlb,
02954     unsigned nind
02955 ) const {
02956
02957     wvector<widxt_t> D,DX;
02958     char cgflag=gotCGS();
02959     unsigned r=(QDIM!=0 ? QIDX.dim2/QDIM : 0);
02960     size_t l=0, n=64; char s[n], sp_[nind+1];
02961
02962     unsigned lsz=2*r+2; if (lsz<8) { lsz=8; }
02963
02964     if (QDIM && QIDX.dim2 && QIDX.dim2%QDIM) wblog(FL,
02965         "ERR %s() %d/%d = ?",FCT, QIDX.dim2, QDIM);
02966     if (nind) { memset(sp_, ' ', nind); }; sp_[nind]=0;
02967
02968     l=snprintf(s,n,"QSpace<%s,%s> %s",
02969         STSTR(TQ), STSTR(TD), qtype.len ? qStr().data : "");
02970     if (l<n && itags.len) {
02971         l+=snprintf(s+l,n-l, " %s",STR(itags));
02972     }
02973     if (l>=n) wblog(FL,"WRN %s() string out of bounds (%d/%d)",FCT,l,n);
02974
02975     for (l=0; l<nlt; ++l) { PRINTF("\n"); }
02976     if (vname && vname[0]) {
02977         if (sp_[0] || strlen(vname)>3)
02978             { PRINTF("%s%-3s",sp_,vname); }
02979         else { PRINTF(" %-2s",vname); }
02980     }
02981     else if (sp_[0]) { PRINTF("%s",sp_); }
02982     PRINTF(" %-34s %3ld x (%d x%d)",s,QIDX.diml,r,QDIM);
02983
02984     getDim(D,&DX);
02985
02986     PRINTF("  %-*s",lsz,SSTR(D )); if (cgflag>0) {
02987         PRINTF("  %-*s",lsz,SSTR(DX)); }
02988     PRINTF("%s\n", isref? " *REF*":"" );
02989
02990     for (l=0; l<nlb; ++l) { PRINTF("\n"); }
02991 };
02992
02993 template <class TQ, class TD>
02994 void QSpace<TQ,TD>::print(const char *vname, char vflag) const {
02995
02996     unsigned i=0,j,l,n, N=QIDX.diml, r=rank(FL), m=(N<12? N:4), nstr=256;
02997     char cgflag=gotCGS(FL), qstr[nstr];
02998     const TQ *qs=QIDX.data;
02999
03000     unsigned lsz=2*r+2;
03001     if (lsz<8) { lsz=8; }
03002
03003     if (vflag & 'D') { vflag |=3; }
03004     if (vflag&1) { PRINTF("\n"); }
03005
03006     info(vname,0,0,0);
03007     PRINTF("\n");
03008
03009     QVec qt(qtype);
03010     if (!qt) { qt.init2val(QDIM,QType(QTYPE_U1)); }
03011
03012     for (; i<N; ++i) {
03013         if (i>=m) {
03014             if (N>2*m) { i=N-m; qs=QIDX.rec(i);
03015                 PRINTF("      : ... \n");
03016             }; m=N;
03017         }
03018
03019         if (i<N) {
03020             for (l=j=0; j<r; ++j, qs+=QDIM) {
03021                 if (j && l<nstr) { l+=snprintf(qstr+l,nstr-l, " "); }
03022                 if (l<nstr) { l+=qt.print_qset(FL,qs,qstr+l,nstr-l); }

```

```

03023         else { break; }
03024     }
03025 } else { sprintf(qstr,"!?"); }
03026
03027 PRINTF("%5d. [ %s ] %-*s ", i+1, qstr, lsz,
03028         i<DATA.len ? DATA[i]->sizeStr(r," @").data : "!?");
03029
03030 if (i>=DATA.len) { PRINTF("    !?\n"); continue; }
03031 if (vflag & 64) {
03032     if (!(DATA[i]->prntdata(vname))) { PRINTF("\n"); }
03033 }
03034 else {
03035     if (cgflag>0) {
03036         for (j=0; j<CGR.dim2; ++j) { if (!CGR(i,j).isAbelian()) {
03037             PRINTF(" %-*s", lsz, SSTR(CGR(i,j))); }
03038         }
03039     }
03040
03041     n=DATA[i]->numel();
03042
03043     if (n==0) { PRINTF("    []\n"); } else
03044     if (n==1) { PRINTF(" %9s", Wb::num2Str(DATA[i]->data[0]).data); }
03045     else {
03046         n *= sizeof(TD);
03047         if (n<(1<<10)) { PRINTF("    %5d B", n); } else
03048         if (n<(1<<20)) { PRINTF("    %5.2f kB", n/double(1<<10)); }
03049         else { PRINTF("    %5.2f MB", n/double(1<<20)); }
03050     }
03051     PRINTF("%s\n",DATA[i]->isRef() ? "    *ref:" );
03052 }
03053 }
03054
03055 if (vflag&2) { PRINTF("\n"); }
03056 };
03057
03058 #endif
03059

```

5.18 QSpace_aux.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_QSpace_AUX_HH__
00021 #define __WB_QSpace_AUX_HH__
00022
00023 template <class TQ, class TD> class QSpace;
00024 template <class TQ, class TD> class CPAT;
00025
00026 template <class TQ, class TA, class TB, class TC>
00027 void contractDATA_plain(
00028     const char *F, int L,
00029     const QSpace<TQ,TA> &A, C_UVEC &Ia, const ctrIdx &ica,
00030     const QSpace<TQ,TB> &B, C_UVEC &Ib, const ctrIdx &icb,
00031     QSpace<TQ,TC> &C, unsigned ic
00032 );
00033
00034 template <class TQ, class TA, class TB, class TC>
00035 double contractDATA_group(
00036     const char *F, int L,
00037     const QSpace<TQ,TA> &A, C_UVEC &Ia, const ctrIdx &ica,
00038     const QSpace<TQ,TB> &B, C_UVEC &Ib, const ctrIdx &icb,
00039     QSpace<TQ,TC> &C, unsigned ic,
00040     char preview=0
00041 );
00042
00043 template<class TQ, class TD>

```



```

00044 void mxInitQSpaceVec(
00045     const char *F, int L, const mxArray* C,
00046     wvector< QSpace<TQ,TD> > &Fk,
00047     const char ref=0,
00048     const unsigned *rmin=NULL,
00049     const unsigned *rmax=NULL
00050 );
00051
00052 template<class TQ, class TD>
00053 void mxInitQSpaceVecVec(
00054     const char *F, int L, const mxArray* C,
00055     wvector< wvector< QSpace<TQ,TD> > > &Fk, const char ref=0
00056 );
00057
00058 template<class TQ, class TD>
00059 void mxcInitQSpaceVec(
00060     const char *F, int L, const mxArray* C,
00061     wvector< QSpace<TQ,TD> > &Fk, const char ref=0
00062 );
00063
00064 template<class TQ, class TD>
00065 mxArray* QSpaceVec2Mx(
00066     const wvector< QSpace<TQ,TD> > &Fk
00067 );
00068
00069 template<class TQ, class TD>
00070 mxArray* QSpaceVec2Mx(
00071     const wvector< QSpace<TQ,TD> > &F, windex &I
00072 );
00073
00074 template<class TQ, class TD>
00075 mxArray* QSpaceVecVec2Mx(
00076     const wvector< wvector< QSpace<TQ,TD> > > &F,
00077     const char ref=0
00078 );
00079
00080 template<class TQ, class TD>
00081 mxArray* QSpaceVec2Mxc(
00082     const wvector< QSpace<TQ,TD> > &F,
00083     const char ref=0
00084 );
00085
00086 template<class TQ, class TD>
00087 void putQSpaceVec(
00088     const wvector< QSpace<TQ,TD> > &Fk,
00089     const char *vname, const char *ws="caller"
00090 ){
00091     mxArray *a=Fk.toMx();
00092     mxPutAndDestroy(FL,a,vname);
00093 };
00094
00095 template <class TQ, class TD>
00096 void getDiffOp(
00097     const QSpace<TQ,TD> &A, unsigned ia,
00098     const QSpace<TQ,TD> &B, unsigned ib, const QSpace<TQ,TD> &C
00099 );
00100
00101 template<class TQ, class TD>
00102 void getQall(const wvector< QSpace<TQ,TD> > &F, wMatrix<TQ> &QA);
00103
00104 int mxIsQSpace(
00105     const char *F, int L,
00106     const mxArray *a, unsigned &rank,
00107     char cflag=0,
00108     unsigned k=-1,
00109     const unsigned *rmin=NULL,
00110     const unsigned *rmax=NULL,
00111     const char *istr=NULL
00112 );
00113
00114 inline int mxIsQSpace(
00115     const char *F, int L, const mxArray *a, char cflag=0,
00116     const char *istr=NULL
00117 ){
00118     unsigned r=-1;
00119     return mxIsQSpace(F,L,a,r,cflag,-1,NULL,NULL,istr);
00120 };
00121
00122 inline int mxIsQSpace(
00123     const mxArray *a, char cflag=0, const char *istr=NULL
00124 ){
00125     unsigned r=-1;
00126     return mxIsQSpace(NULL,0,a,r,cflag,-1,NULL,NULL,istr);
00127 };
00128
00129 bool mxIsQSpaceArr(
00130     const char *F, int L,

```

```

00131     const mxArray *S,
00132     unsigned rank=-1,
00133     int arddim=2,
00134     const unsigned *rmin=NULL,
00135     const unsigned *rmax=NULL,
00136     char cflag=0
00137 );
00138
00139 bool mxIsQSpaceVecVec(const mxArray *C, unsigned r=-1, char cflag=0);
00140
00141 inline bool mxIsQSpaceVec(
00142     const char *F, int L,
00143     const mxArray *S,
00144     unsigned rank=-1,
00145     const unsigned *rmin=NULL,
00146     const unsigned *rmax=NULL,
00147     char cflag=0
00148 ){ return mxIsQSpaceArr(F,L,S,rank,1,rmin,rmax,cflag); };
00149
00150 inline bool mxIsQSpaceVecR23(
00151     const char *F, int L,
00152     const mxArray *S,
00153     char cflag=0
00154 ){
00155     unsigned rank=-1, rmin=2, rmax=3;
00156     return mxIsQSpaceArr(F,L,S,rank,1,&rmin,&rmax,cflag);
00157 };
00158
00159 inline bool mxIsQSpaceMat(
00160     const char *F, int L,
00161     const mxArray *S,
00162     unsigned rank=-1,
00163     const unsigned *rmin=NULL,
00164     const unsigned *rmax=NULL,
00165     char cflag=0
00166 ){ return mxIsQSpaceArr(F,L,S,rank,2,rmin,rmax,cflag); };
00167
00168 bool mxIsEmptyQSpace(const mxArray *a, unsigned k=0);
00169
00170 bool mxIsScalarQSpace(const mxArray *a, unsigned k=0);
00171
00172 inline bool mxIsQSpaceScalar(
00173     const char *F, int L,
00174     const mxArray *S,
00175     unsigned rank=-1,
00176     const unsigned *rmin=NULL,
00177     const unsigned *rmax=NULL,
00178     char cflag=0
00179 ){ return mxIsQSpaceArr(F,L,S,rank,0,rmin,rmax,cflag); };
00180
00181 bool mxIsQSpaceVec(const mxArray *S, int fid, unsigned r=-1, char cflag=0);
00182 bool mxIsQSpaceVec(const mxArray *S, int fid, unsigned r=-1, char cflag=0);
00183
00184 int mxIsQSpaceVec(
00185     const char *F, int L,
00186     const char *fname, const char *vname,
00187     unsigned rank=-1,
00188     const unsigned *rmin=NULL,
00189     const unsigned *rmax=NULL,
00190     char cflag=0,
00191     unsigned N=-1
00192 );
00193
00194 int mxIsQSpaceVEC(
00195     const char *F, int L,
00196     const char *fname, const char *vname,
00197     unsigned rank=-1,
00198     const unsigned *rmin=NULL,
00199     const unsigned *rmax=NULL,
00200     char cflag=0
00201 );
00202
00203 bool mxIsQSpaceVecOrEmpty(const mxArray *S, int rank, char cflag=0) {
00204     if (mxIsEmpty(S)) return 1;
00205     return mxIsQSpaceVec(F,L,S,rank,NULL,NULL,cflag);
00206 };
00207
00208 inline int mxIsQSpaceOrEmpty(
00209     const mxArray *a, unsigned rank=-1, char cflag=0, unsigned k=-1
00210 ){ if (mxIsEmpty(a)) return 1;
00211     else return mxIsQSpace(NULL,0,a,rank,cflag,k);
00212 };
00213
00214 inline int mxIsQSpaceOrEmpty(const char *F, int L,
00215     const mxArray *a, unsigned rank=-1, char cflag=0, unsigned k=-1
00216 ){ if (mxIsEmpty(a)) return 1;
00217     else return mxIsQSpace(F,L,a,rank,cflag,k);

```

```

00218 };
00219
00220 QVec qsGetSym(
00221     const char *F, int L, const mxArray *a, unsigned k=-1);
00222
00223 char qsGotCGS(
00224     const char *F, int L, const mxArray *a, unsigned k=-1
00225 ){
00226     QVec qtype=qsGetSym(F_L,a,k);
00227     return qtype.permitsOM();
00228 };
00229
00230 #endif
00231

```

5.19 qspace_lib.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_QSPACE_LIB_HCC__
00021 #define __WB_QSPACE_LIB_HCC__
00022
00023 /* =====
00024  * #include "qspace_lib.cc"
00025  * -----
00026  * contractIndex.cc (library file)
00027  *
00028  * C++ program that gets index sets for generic contractQ()
00029  *
00030  * This is a MEX-file for MATLAB.
00031  * AW (C) Jan 2006
00032  * ===== */
00033
00034 void check_qdim      (C_MEX *C, unsigned &ma, wbvector<widxt_t> &dq);
00035 void check_cidx      (C_MEX *a, const unsigned &adim, UVEC &ica);
00036 void getPerm         (C_MEX *a, const unsigned m, wbvector<unsigned> &P);
00037 char isUInt          (const wbvector<double> &dd);
00038
00039 void getDataSize     (const mxArray *AD, const unsigned nd, const unsigned ma,
00040                     wbMatrix<unsigned> &usa);
00041
00042 void getQsub         (C_MEX *C, unsigned m, C_UVEC &dQ, C_UVEC &I, DMAT &Qc);
00043 void getQall         (C_MEX *A, wbMatrix<double> &Q);
00044 void getQall         (C_MEX *a, const unsigned ma, C_UVEC &dq, wbMatrix<double> &Q);
00045 void getQBlock       (C_MEX *C, C_UINT ic, DMAT &Qc);
00046 void cell2mat        (C_MEX *C, unsigned *idx, unsigned n, DMAT &Qc);
00047
00048 char isUnique        (wbMatrix<double> Q);
00049 char isUniqueSorted  (C_DMAT &Q);
00050
00051 void makeUnique      (DMAT &Q, UVEC &Iu);
00052 void makeUnique      (DMAT &Q);
00053
00054 int matchSortedIdx_old(
00055     C_DMAT &QA, C_DMAT &QB,
00056     wbvector<widxt_t> &Ia, wbvector<widxt_t> &Ib
00057 );
00058
00059 int matchIndex_old(C_DMAT &QA, C_DMAT &QB,
00060     wbvector<widxt_t> &Ia, wbvector<widxt_t> &Ib
00061 );
00062
00063 void QtoCell        (C_DMAT &Q, C_UVEC &dab, mxArray *&S, int fieldnr,
00064                     C_UVEC &P = wbvector<unsigned>());
00065 void Data2vec       (mxArray *AD, C_IMAT sda, wbvector<double> &Avec);
00066 void Sab2Cell       (C_IMAT &SA, C_IMAT &SB, C_UVEC &Ia, C_UVEC &Ib,

```

```

00067         C_UVEC &Iu, C_UVEC &ika, C_UVEC &ikb,
00068         mxArray* C, int fid1, int fid2);
00069
00070 void DtoCell      (C_IMAT &SA, C_IMAT &SB, C_UVEC &Ia, C_UVEC &Ib,
00071                   C_UMAT &IG, C_UVEC &ika, C_UVEC &ikb,
00072                   mxArray* C, int fid1, int fid2);
00073
00074 void check_qdim(
00075     const mxArray *C,
00076     unsigned &ma,
00077     wbvector<widx_t> &dq
00078 ){
00079     mxArray *a;
00080     unsigned k, da;
00081
00082     if (!C || !mxIsCell(C) || mxIsEmpty(C))
00083         usage(FLINE, "Input arguments 1 and 2 must be cell arrays.");
00084
00085     if (mxGetM(C)!=1)
00086         usage(FLINE, "Input arguments 1 and 2 must be cell arrays (row vectors).");
00087
00088     da=mxGetN(C); dq.init(da);
00089
00090     a=mxGetCell(C,0); dq[0]=mxGetN(a);
00091     ma=mxGetM(a);
00092
00093     for (k=1; k<da; k++) {
00094         a=mxGetCell(C,k); dq[k]=mxGetN(a);
00095
00096         if (ma!=(unsigned)mxGetM(a))
00097             usage(FLINE, "Q Space inconsistency in input arguments.");
00098
00099         if (mxGetClassID(a)!=mxDOUBLE_CLASS)
00100             usage(FLINE, "Cell array of type double as input required.");
00101     }
00102 }
00103
00104 void check_cidx(
00105     const mxArray *a, const unsigned &adim, wbvector<unsigned> &ica) {
00106
00107     int e=0;
00108     try {
00109         Mx::Array<unsigned> A(a); ica.init(A.len);
00110         if (A.len) {
00111             ica.init(A.len); A.copyTo(ica.data,'t');
00112         }
00113     }
00114     catch (...) { ++e; }
00115     if (e || !ica.len) usage(FLINE,"input arguments 3 and 4 must be vectors");
00116
00117     unsigned i=0, n=ica.max(); char mark[n];
00118
00119     for (; i<ica.len; ++i) {
00120         if (--ica[i]>n || ++mark[ica[i]]>1) wblog(FL,
00121             "ERR invalid contraction index (unique, 1-based)");
00122     }
00123 };
00124
00125 void getPerm(
00126     const mxArray *a, const unsigned m0, wbvector<unsigned> &P) {
00127
00128     Mx::Array<unsigned> A(a);
00129
00130     if (!A.len) { P.init(); return; }
00131     if (A.len!=m0) {
00132         sprintf(str,"permutation dimension mismatch (%ld,%d)",A.len,m0);
00133         usage(FLINE, str);
00134     }
00135
00136     P.init(A.len); A.copyTo(P.data,'t');
00137     P-=1;
00138
00139     if (!validPerm(P)) {
00140         P.print("P"); wbdie(FLINE,"invalid permutation");
00141     }
00142 };
00143
00144 char isUInt(const wbvector<double> &dd) {
00145
00146     double di;
00147     for (unsigned i=0; i<dd.len; i++) {
00148         di=dd[i];
00149         if (floor(di)!=di) return 0;
00150         if (di<0) return 0;
00151     }
00152     return 1;
00153 }

```

```

00154
00155 void getDataSize(
00156     const mxArray *AD,
00157     const unsigned nd,
00158     const unsigned ma,
00159     wbMatrix<int> &sa
00160 ){
00161     unsigned i,n;
00162     mxArray *a;
00163
00164     if (!mxIsCell(AD))
00165         wblog(FLINE,"ERR Data must be cell structure.");
00166
00167     if (ma!=(n=(unsigned)mxGetNumberOfElements(AD)))
00168         wblog(FLINE,"ERR Dimension mismatch between Q and data (%d,%d)",ma,n);
00169
00170     if (mxGetM(AD)!=1 && mxGetN(AD)!=1)
00171         wblog(FLINE,"ERR Q must be vectorized cell structure");
00172
00173     sa.init(ma,nd); sa.set(1);
00174
00175     for (i=0; i<ma; i++) {
00176         a=mxGetCell(AD,i);
00177
00178         n=mxGetNumberOfDimensions(a);
00179         if (n>nd) wblog(FLINE,"ERR Dimension mismatch in data (%d,%d)",n,nd);
00180
00181         memcpy(sa.rec(i), mxGetDimensions(a), n*sizeof(int));
00182     }
00183
00184     return;
00185 }
00186
00187 void getQsub(
00188     const mxArray *C,
00189     unsigned ma,
00190     const wbvector<unsigned> &dQ,
00191     const wbvector<unsigned> &I,
00192     wbMatrix<double> &Qc
00193 ){
00194     unsigned i,j,k,m,ib,nq,nC=mxGetNumberOfElements(C);
00195     double *qik;
00196
00197     for (m=ib=0; ib<I.len; ib++) m += dQ[I[ib]];
00198
00199     Qc.init(ma,m);
00200
00201     for (k=ib=0; ib<I.len; ib++) {
00202         if (I[ib]>=nC) wbdie(FLINE,"index out of bounds");
00203
00204         Mx::Array<double> A(FL,mxGetCell(C,I[ib]));
00205         nq=dQ[I[ib]];
00206
00207         for (i=0; i<ma; ++i) { qik=Qc.rec(i)+k;
00208             for (j=0; j<nq; ++j) { qik[j]=A.data[i+ma*j]; }
00209
00210             k+=nq;
00211         }
00212     };
00213
00214 void getQall(
00215     const mxArray *A,
00216     wbMatrix<double> &Q
00217 ){
00218     unsigned ma;
00219     int iQ;
00220
00221     mxArray *AQ;
00222     wbvector<widx_t> dqa;
00223
00224     if (!mxIsStruct(A)) usage(FLINE,
00225         "Input argument must be structure {Q [,data,...]}.");
00226
00227     if (mxGetNumberOfElements(A)!=1) usage(FLINE,
00228         "Input argument must be SINGLE structure {Q [,data,...]}.");
00229
00230     iQ=mxGetFieldNumber(A,"Q");
00231     if (iQ<0) usage(FLINE,
00232         "Input argument must be structure of the type {Q [,data,...]}.");
00233
00234     AQ=mxGetFieldByNumber(A,0,iQ);
00235
00236     check_qdim(AQ, ma, dqa);
00237
00238     getQall(AQ, ma, dqa, Q);
00239
00240     return;

```

```

00241 }
00242
00243 void getQall(
00244     const mxArray *C,
00245     unsigned ma,
00246     const wbvector<unsigned> &dq,
00247     wbMatrix<double> &Q
00248 ){
00249     unsigned i, j, k, D, ib, d2;
00250     double *q;
00251
00252     D=dq.sum(); Q.init(ma,D);
00253
00254     if (dq.len!=(unsigned)mxGetNumberOfElements(C))
00255         wbdie(FLINE, "Dimension mismatch.");
00256
00257     for (k=ib=0; ib<dq.len; ++ib) {
00258         Mx::Array<double> A(FL,mxGetCell(C,ib));
00259         d2=dq[ib];
00260
00261         for (i=0; i<ma; ++i) { q=Q.rec(i)+k;
00262             for (j=0; j<d2; ++j) { q[j]=A.data[i+ma*j]; }}
00263
00264         k+=d2;
00265     }
00266 };
00267
00268 void getQBlock(
00269     const mxArray *C,
00270     const unsigned ic,
00271     wbMatrix<double> &Qc
00272 ){
00273     unsigned i,j; double *qi;
00274
00275     if (ic>=mxGetNumberOfElements(C)) wblog(FL,
00276         "ERR %s() index out of bounds (%d/%d)",FCT,ic,j);
00277
00278     Mx::Array<double> A(FL,mxGetCell(C,ic));
00279     if (A.rank!=2) wblog(FL,"ERR %s() invalid rank-%d object",FCT,A.rank);
00280
00281     const size_t *sz=mxGetDimensions(A.ax);
00282     Qc.init(sz[0],sz[1]);
00283
00284     for (i=0; i<sz[0]; ++i) { qi=Qc.rec(i);
00285         for (j=0; j<sz[1]; ++j) { qi[j]=A.data[i+sz[0]*j]; }}
00286
00287     return;
00288 };
00289
00290 void cell2mat(
00291     const mxArray *C,
00292     const wbMatrix<unsigned> &I,
00293     wbMatrix<double> &M
00294 ){
00295     unsigned i,j,k,d1,d2,N,r0,s0,r,s;
00296     double *d;
00297     mxArray *a;
00298
00299     wbvector<unsigned> D1(I.dim1), D2(I.dim2);
00300     wbMatrix<double*> dd(I.dim1, I.dim2);
00301
00302     N=(unsigned)mxGetNumberOfElements(C);
00303
00304     for (i=0; i<I.dim1; i++)
00305         for (j=0; j<I.dim2; j++) {
00306             k=I(i,j);
00307             if (k>=N) wbdie(FLINE, "Index out of bounds.");
00308             a=mxGetCell(C,k);
00309
00310             if (i==0) D2[j]=(unsigned)mxGetN(a);
00311             else if (D2[j]!=(unsigned)mxGetN(a))
00312                 wbdie(FLINE, "Dimension mismatch.");
00313
00314             if (j==0) D1[i]=(unsigned)mxGetM(a);
00315             else if (D1[i]!=(unsigned)mxGetM(a))
00316                 wbdie(FLINE, "Dimension mismatch.");
00317
00318             dd(i,j)=Mx::Array<double>(FL,a).data;
00319         }
00320
00321     M.init(D1.sum(), D2.sum());
00322
00323     for (r0=i=0; i<I.dim1; i++) {
00324         d1=D1[i];
00325         for (s0=j=0; j<I.dim2; j++) {
00326             d2=D2[j]; d=dd(i,j);
00327

```

```

00328         for (k=s=0; s<d2; s++)
00329             for ( r=0; r<d1; r++) M(r0+r,s0+s)=d[k++];
00330
00331         s0+=d2;
00332     }
00333     r0+=d1;
00334 }
00335 };
00336
00337 char isUnique(wbMatrix<double> Q) {
00338
00339     wbperm iS;
00340     Q.SortRecs(iS);
00341     return isUniqueSorted(Q);
00342 }
00343
00344 char isUniqueSorted(const wbMatrix<double> &Q) {
00345
00346     unsigned j;
00347
00348     for (j=1; j<Q.dim1; j++)
00349         if (Q.recEqual(j-1,j)) return 0;
00350
00351     return 1;
00352 }
00353
00354 void makeUnique(wbMatrix<double> &Q, wbvector<unsigned> &Iu) {
00355
00356     unsigned i,k,m,ig,d, ng;
00357     wbvector<widx_t> dQ;
00358     wbperm iS;
00359
00360     Q.groupRecs(iS,dQ); ng=dQ.len;
00361
00362     Iu.init(iS.len+ng);
00363
00364     for (k=ig=0; ig<ng; ig++) {
00365         d=dQ[ig];
00366         Iu[k+ig]=d; m=ig+1;
00367         for (i=0; i<d; i++, k++) Iu[k+m]=iS[k];
00368     }
00369 }
00370
00371 void makeUnique(wbMatrix<double> &Q) {
00372     wbperm iS; wbvector<widx_t> dQ;
00373     Q.groupRecs(iS,dQ);
00374 }
00375
00376 int matchIndex_old(
00377     const wbMatrix<double> &QA,
00378     const wbMatrix<double> &QB,
00379     wbvector<widx_t> &Ia,
00380     wbvector<widx_t> &Ib
00381 ){
00382     wbMatrix<double> Q1(QA), Q2(QB);
00383     wbvector<widx_t> ix1, ix2;
00384     wbperm is1, is2;
00385
00386     Q1.SortRecs(is1);
00387     Q2.SortRecs(is2);
00388
00389     matchSortedIdx_old(Q1,Q2,ix1,ix2);
00390
00391     is1.get(ix1,Ia);
00392     is2.get(ix2,Ib);
00393
00394     return 0;
00395 }
00396
00397 int matchSortedIdx_old(
00398     const wbMatrix<double> &QA,
00399     const wbMatrix<double> &QB,
00400     wbvector<widx_t> &Ia,
00401     wbvector<widx_t> &Ib
00402 ){
00403
00404     unsigned i,j,k, ng, ia,ib, ig, iga, igb, m1,m2, mtot=0;
00405     unsigned *p1, *p2, na=QA.dim1, nb=QB.dim1;
00406     char c;
00407
00408     if (QA.dim2!=QB.dim2)
00409         wbdie(FLINE, "Dimension mismatch");
00410
00411     wbvector<unsigned> gra(2*na), grb(2*nb);
00412
00413     for (ng=iga=igb=ia=ib=0; ia<na && ib<nb;) {
00414         c=QA.recCompareP(ia, QB.rec(ib));

```

```

00415         if (c<0) ia++; else
00416         if (c>0) ib++;
00417         else {
00418             p1=gra.data+(iga++); gra[iga++]=(ia++);
00419             p2=grb.data+(igb++); grb[igb++]=(ib++);
00420
00421             m1=m2=1; ng++;
00422             while (ia<na) {
00423                 if (QA.recEqual(ia-1, ia)) { gra[iga++]=(ia++); m1++; }
00424                 else break;
00425             }
00426             while (ib<nb) {
00427                 if (QB.recEqual(ib-1, ib)) { grb[igb++]=(ib++); m2++; }
00428                 else break;
00429             }
00430             mtot+=m1*m2; (*p1)=m1; (*p2)=m2;
00431         }
00432     }
00433
00434     Ia.init(mtot); Ib.init(mtot);
00435
00436     for (k=ig=iga=igb=0; ig<ng; ig++) {
00437         m1=gra[iga]; p1=gra.ref(iga+1); iga+=(m1+1);
00438         m2=grb[igb]; p2=grb.ref(igb+1); igb+=(m2+1);
00439         for (i=0; i<m1; i++)
00440             for (j=0; j<m2; j++,k++) {
00441                 Ia[k]=p1[i];
00442                 Ib[k]=p2[j];
00443             }
00444     }
00445
00446     return 0;
00447 }
00448
00449 int matchUSortedIdx(
00450     const wbMatrix<double> &QA,
00451     const wbMatrix<double> &QB,
00452     wbvector<unsigned> &Ia,
00453     wbvector<unsigned> &Ib
00454 ){
00455     unsigned n,ia,ib; char c;
00456
00457     if (QA.dim2!=QB.dim2)
00458         wbdie(FLINE, "Dimension mismatch");
00459
00460     #ifdef __WBDEBUG__
00461         if (!QA.isUniqueSorted('A') || !QB.isUniqueSorted('A'))
00462             wbdie(FL,"ERR Input must be unique increasing set of records.");
00463     #endif
00464
00465     n=MIN(QA.dim1,QB.dim1); Ia.init(n); Ib.init(n);
00466
00467     for (n=ia=ib=0; ia<QA.dim1 && ib<QB.dim1 ;) {
00468
00469         c=QA.recCompareP(ia, QB.rec(ib));
00470
00471         if (c<0) ia++; else
00472         if (c>0) ib++;
00473         else {
00474             Ia[n]=ia;
00475             Ib[n]=ib; ia++; ib++; n++;
00476         }
00477     }
00478
00479     Ia.Resize(n); Ib.Resize(n);
00480
00481     return 0;
00482 }
00483
00484 void Sab2Cell(
00485     const wbMatrix<int> &SA,
00486     const wbMatrix<int> &SB,
00487     const wbvector<unsigned> &Ia,
00488     const wbvector<unsigned> &Ib,
00489     const wbvector<unsigned> &Id,
00490     const wbvector<unsigned> &ika,
00491     const wbvector<unsigned> &ikb,
00492     mxArray* ST, int fid_SAB, int fid_Sab
00493 ){
00494     unsigned i,k,l,ia,ib,ng=Id.len,d2=ika.len+ikb.len;
00495     wbvector<int> ss, sk(d2);
00496     double *d, *SD, *sd;
00497
00498     mxSetFieldByNumber(ST,0,fid_SAB, Mx::Array<double>( 2,ng).Return(SD));
00499     mxSetFieldByNumber(ST,0,fid_Sab, Mx::Array<double>(d2,ng).Return(sd));
00500
00501

```



```

00502     for (l=k=0; k<ng; k++) {
00503
00504         ia=Ia[l]; ib=Ib[l]; l+=Id[k];
00505
00506         if (ia>=SA.dim1 || ib>=SB.dim1)
00507             wbdie(FLINE,"Index out of bounds.");
00508
00509         ss.init(SA.dim2, SA.rec(ia)); ss.get(ika, sk.data);
00510         ss.init(SB.dim2, SB.rec(ib)); ss.get(ikb, sk.data+ika.len);
00511
00512         i=k*2;
00513         SD[i ]=(double)sk.prod(unsigned(0),ika.len-1);
00514         SD[i+1]=(double)sk.prod(ika.len,sk.len-1);
00515
00516         d=sd+k*d2;
00517
00518         for (i=0; i<d2; i++) d[i]=(double)sk[i];
00519     }
00520
00521     return;
00522 }
00523
00524 void QtoCell(
00525     const wbMatrix<double> &Q,
00526     const wbvector<unsigned> &dab,
00527     mxArray *aS,
00528     int field_nr,
00529     const wbvector<unsigned> &P
00530 ){
00531     unsigned i,j,k,m,d1=Q.dim1,d2,n=dab.len;
00532     double *dd;
00533     mxArray *C;
00534
00535     wbvector<unsigned> D2(n);
00536
00537     if (field_nr>=0) {
00538         mxSetFieldByNumber(S,0,field_nr, mxCreateCellMatrix(1,n));
00539         C=mxGetFieldByNumber(S,0,field_nr);
00540     } else { S=mxCreateCellMatrix(1,n); C=S; }
00541
00542     if (Q.dim1==0 || dab.len==0) {
00543         for (i=0; i<n; i++)
00544             mxSetCell(C,i,mxCreateDoubleMatrix(0,dab[i],mxREAL));
00545         return;
00546     }
00547
00548     D2[0]=0; for (j=1; j<n; j++) D2[j]=D2[j-1]+dab[j-1];
00549
00550     if (D2[j-1]+dab[j-1]!=Q.dim2) wbdie(FLINE, "Dimension mismatch.");
00551     if (P.len && P.len!=n) wbdie(FLINE, "Invalid permutation.");
00552
00553     for (k=0; k<n; k++) {
00554         d2=dab[k];
00555
00556         mxSetCell(C,k, Mx::Array<double>(d1,d2).Return(dd));
00557         m = P.len ? D2[P[k]] : D2[k];
00558
00559         for (j=0; j<d2; ++j)
00560             for (i=0; i<d1; ++i) { dd[d1*j+i]=Q(i,m+j); }
00561     }
00562 };
00563
00564 void Data2vec(
00565     mxArray *AD, const wbMatrix<int> sda, wbvector<double> &Avec) {
00566
00567     unsigned i,d,k,N;
00568     wbvector<int> SD;
00569
00570     if (sda.dim1==0 || sda.dim2==0) { Avec.init(); return; }
00571
00572     SD.init(sda.dim1);
00573     for (i=0; i<SD.len; i++) SD[i]=sda.recProd(i);
00574
00575     N=(unsigned)SD.sum();
00576
00577     Avec.init(N);
00578     for (k=i=0; i<SD.len; ++i) { d=(unsigned)SD[i];
00579         Mx::Array<double> A(FL,mxGetCell(AD,i));
00580         memcpy(Avec.data+k, A.data, d*sizeof(double));
00581         k+=d;
00582     }
00583 };
00584
00585 void QI2Cell(
00586     const wbMatrix<double> &Q,
00587     const mxArray *AQ,
00588     mxArray* C,

```

```

00589     int field_nr,
00590     const wbvector<widx_t> &P
00591 ){
00592     unsigned i,m,M;
00593     wbvector<widx_t> iP, dqa, iv;
00594     wbindex iq,ir;
00595
00596     wbMatrix<double> Qref;
00597
00598     if (Q.data==NULL) {
00599         mxSetFieldByNumber(C, 0, field_nr,
00600             mxCreateDoubleMatrix(0,0,mxREAL)); return;
00601     }
00602
00603     getIPerm(P,iP);
00604
00605     check_qdim(AQ,m,dqa);
00606     getQsub (AQ,m,dqa, iP, Qref);
00607
00608     M=Q.dim1; iv.init(M);
00609
00610     if (Q.dim2==Qref.dim2) {
00611         matchIndex(Q,Qref,iq,ir); m=iq.len;
00612
00613         for (i=0; i<m; i++) iv[iq[i]]=ir[i]+1;
00614     }
00615
00616     mxSetFieldByNumber(C,0,field_nr,
00617         Mx::Array<double>(M,1).copyFromTR(iv.data)
00618     );
00619 };
00620
00621 void DtoCell(
00622     const wbMatrix<int> &SA,
00623     const wbMatrix<int> &SB,
00624     const wbvector<unsigned> &Ia,
00625     const wbvector<unsigned> &Ib,
00626     const wbMatrix<unsigned> &IG,
00627     const wbvector<unsigned> &ika,
00628     const wbvector<unsigned> &ikb,
00629     mxArray* ST, int fid1, int fid2
00630 ){
00631     unsigned k,l,ia,ib,ng=IG.dim1;
00632     wbvector<size_t> uu, sk(ika.len+ikb.len);
00633
00634     mxArray *D=mxCreateCellMatrix(ng,1);
00635     double *sd;
00636
00637     mxSetFieldByNumber(ST,0,fid1, D);
00638     mxSetFieldByNumber(ST,0,fid2, Mx::Array<double>(ng,2).Return(sd));
00639
00640     for (k=0; k<ng; ++k) {
00641
00642         l=IG(k,1); ia=Ia[l]; ib=Ib[l];
00643
00644         if (ia>=SA.dim1 || ib>=SB.dim1)
00645             wbdie(FLINE,"Index out of bounds.");
00646
00647         uu.initT(SA.dim2, SA.rec(ia)); uu.get(ika, sk.data);
00648         uu.initT(SB.dim2, SB.rec(ib)); uu.get(ikb, sk.data+ika.len);
00649
00650         sd[k] =(double)sk.prod(unsigned(0),ika.len-1);
00651         sd[k+ng]=(double)sk.prod(ika.len,sk.len-1);
00652
00653         mxSetCell(D,k, Mx::Array<double>(sk).Return());
00654     };
00655 };
00656
00657 void Sab2Site2Cell(
00658     const wbMatrix<int> &SA,
00659     const wbMatrix<int> &SB,
00660     const wbvector<unsigned> &Ia,
00661     const wbvector<unsigned> &Ib,
00662     const wbMatrix<unsigned> &IG,
00663     const wbvector<unsigned> &ika,
00664     const wbvector<unsigned> &ikb,
00665     const wbvector<unsigned> &ikP,
00666     mxArray* ST, int fid_Sab, int fid_SAB
00667 ){
00668     unsigned k,l,ia,ib, ng=IG.dim1, dk=ika.len+ikb.len; int *s1,*s2;
00669     wbMatrix<int> sd(ng,dk);
00670     wbvector<int> uu;
00671     double *SD;
00672
00673     mxSetFieldByNumber(ST,0,fid_SAB, Mx::Array<double>(2,ng).Return(SD));
00674
00675     for (k=0; k<ng; ++k) {

```

```

00676
00677     l=IG(k,1); ia=Ia[l]; ib=Ib[l];
00678
00679     if (ia>=SA.dim1 || ib>=SB.dim1)
00680         wbdie(FLINE,"Index out of bounds.");
00681
00682     s1=sd.rec(k); s2=s1+ika.len;
00683
00684     uu.init(SA.dim2, SA.rec(ia)); uu.get(ika, s1);
00685     uu.init(SB.dim2, SB.rec(ib)); uu.get(ikb, s2);
00686
00687     SD[2*k ]=(double)Wb::prodRange(s1,ika.len);
00688     SD[2*k+1]=(double)Wb::prodRange(s2,ikb.len);
00689 }
00690
00691 sd.colPermute(ikP);
00692 sd.mat2mxs(ST,fid_Sab);
00693 };
00694
00695 #endif
00696

```

5.20 spectral.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace NRG routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SPECTRAL_HCC__
00021 #define __WB_SPECTRAL_HCC__
00022
00023 // default smoothing: norm-preserving log-gauss broadening
00024 #define LOG_GAUSS_BRD 2
00025
00026 namespace Wb {
00027
00028 template <class TS>
00029 void Fourier(
00030     const wbvector<double> &om,
00031     const wbMatrix<TS> &A,
00032     const wbvector<double> &tt,
00033     wbMatrix<wbcomplex> &az,
00034     char domFlag=0,
00035     double alpha=0.,
00036     char vflag=1
00037 );
00038
00039 template <class TS>
00040 void dispSumRule(
00041     const wbvector<double> &om,
00042     const wbMatrix<TS> &A
00043 );
00044
00045 };
00046
00047 void KKimag(
00048     const wbMatrix<double> &Rp,
00049     const wbMatrix<double> &Rn,
00050     const wbvector<double> &OM,
00051     wbMatrix<double> &Ip,
00052     wbMatrix<double> &In
00053 );
00054
00055 template <class TS>
00056 class Spectral {
00057 public:
00058
00059     Spectral()

```

```

00060     : sigma(0.6), sigma2(0.5), alpha(0.25), Emin(1E-8), Emax(10.),
00061     eps(1E-6), nlog(128), l0(0), fac(1.), Delta(0.) {};
```

```

00062
00063 void init() {
00064     Ap.init(); An.init(); Ap_buf.init(); An_buf.init();
00065     Ar.init(); A0.init(); mspec.init();
00066     istr.init();
00067 };
00068
00069 void init(const char *istr, unsigned Nops,
00070     double delta=0., double T_=0, unsigned Niter=0
00071 );
00072
00073 void initBuf() {
00074     if (Ap.dim1!=An.dim1) wblog(FL,
00075         "ERR severe dimension mismatch (%d,%d).", Ap.dim1, An.dim1);
00076
00077     Ap_buf.init(Ap.dim1, Ap.dim2);
00078     An_buf.init(An.dim1, An.dim2);
00079 };
00080
00081 void calcSpectralMoments(const char *F, int L, unsigned n) {
00082     if (n) {
00083         if (n>4) wblog(F,L,"WRN %s with n=%d !?",FCT,n);
00084         else wblog(F,L,"<i> calculate %d spectral moment(s)",n);
00085         mspec.init(Ap.dim1,n+1);
00086     }
00087 };
00088
00089 void Add(
00090     const wbararray<double> &Om, const wbararray<TS> &CC, unsigned s,
00091     char dblock=0,
00092     char mflag=0
00093 );
00094
00095 void Add2buf(const wbararray<double> &Om, const wbararray<TS> &CC, unsigned s);
00096
00097 void crossAddBuf(const char minor=1, char vflag=1);
00098
00099 void saveIter(const char *F, int L, unsigned iter);
00100
00101 void getOmega(wbvector<double> &om, unsigned &linfit) const;
00102 void getOmega(wbvector<double> &om) const {
00103     unsigned linfit=0; getOmega(om,linfit);
00104 };
00105
00106 void applyIROPfac(const char *F, int L, const wbvector<double> &fac);
00107
00108 void getRawData(
00109     wbvector<double> &om, wbMatrix<TS> &A,
00110     unsigned linfit=0, const char *isbuf=""
00111 ) const;
00112
00113 void getPARTIAL(
00114     wbararray<TS> &A1, wbararray<TS> &A2) const;
00115
00116 void pairRawData();
00117
00118 void detailedBalance(
00119     const double T, wbvector<double> &om, wbMatrix<TS> &aa) const;
00120
00121 void dispSumRule();
00122 void dispSumRule(const wbvector<double> &om, const wbMatrix<TS> &A);
00123
00124 void getSmoothSpec(
00125     wbvector<double> &om, wbMatrix<TS> &A, char vflag=1);
00126
00127 void getSmoothSpec(
00128     wbMatrix<TS> &A, const wbvector<double> &om, char vflag=1);
00129
00130 void KKreal(
00131     wbMatrix<wbcomplex> &GZ,
00132     wbvector<double> &OM,
00133     wbvector<double> &DOM
00134 ) const;
00135
00136 wbMatrix<TS> Ap, An;
00137 wbMatrix<TS> Ap_buf, An_buf;
00138 wbvector<TS> Ar;
00139 wbMatrix<TS> A0;
00140 wbMatrix<TS> mspec;
00141
00142 wbararray<TS> AP, AN;
00143
00144 wbstring istr;
00145 double Temp;
00146
```

```

00147     double sigma, sigma2, alpha, Emin, Emax, eps;
00148     unsigned nlog;
00149
00150 protected:
00151 private:
00152
00153     double l0, fac, Delta;
00154
00155     double OM2IDX(const double &om) const {
00156         return ((log(fabs(om))-10)*fac);
00157     }
00158
00159     inline unsigned om2idx(double om) const {
00160         if (Delta==0) { if (om==0.) return 0; }
00161         else {
00162             if (om> Delta) om-=Delta; else
00163             if (om< -Delta) om+=Delta; else return 0;
00164         }
00165
00166         int k = (int)floor(OM2IDX(om));
00167         if (k<1) k=1; else
00168         if (k>=(int)Ap.dim2) { k=Ap.dim2-1; }
00169
00170         return (unsigned)k;
00171     }
00172
00173     inline double idx2om(const unsigned &k) const {
00174         return exp( (double(k)+0.5)/fac + 10 );
00175     }
00176
00177     int crossAddBuf_aux(
00178         TS *a, const TS *b, const unsigned M,
00179         const char minor=1, char vflag=1
00180     );
00181
00182     TS foldLogGaussian(
00183         const double om, const TS *aa, const unsigned n
00184     );
00185
00186     TS foldGaussian(
00187         const double om, const double b,
00188         const double *oo, const TS *aa, const unsigned n
00189     );
00190 };
00191
00192 template <class TS>
00193 class RSpecData {
00194 public:
00195     RSpecData() { };
00196
00197     RSpecData& Append(const double *op, const TS *cp, unsigned n) {
00198         om.Append(n,op);
00199         cc.Append(n,cp); return *this;
00200     };
00201
00202     mxArray* mxCreateStruct(unsigned m, unsigned n) const {
00203         const char *flds[]={"om","cc"};
00204         return mxCreateStructMatrix(m,n,2,flds);
00205     };
00206     void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const {
00207         mxSetFieldByNumber(S,i,0,om.toMx(tflag));
00208         mxSetFieldByNumber(S,i,1,cc.toMx(tflag));
00209     };
00210
00211     wbvector<double> om;
00212     wbvector<TS> cc;
00213
00214 protected:
00215 private:
00216
00217 };
00218
00219 template <class TS>
00220 class TDSpectral {
00221 public:
00222
00223     TDSpectral() : raw(0), l0(0), fac(1.), check_avg_om(0) {};
00224
00225     void init(const char *F, int L,
00226         const char *istr0, const wbvector<double> &t0,
00227         unsigned Nnrg, unsigned Nop,
00228         double Emin=1E-8, double Emax=10., unsigned nl=512,
00229         char vflag=1
00230     );
00231
00232     void checkAvgOm() {

```

```

00234     check_avg_om=1;
00235     AW.init(2,AP.dim(2),AP.dim(3));
00236 };
00237 wbararray<TS> getAvgOm();
00238
00239 void Add(
00240     const wbararray<double> &Om, const wbararray<TS> &CC,
00241     unsigned iter, unsigned s
00242 );
00243
00244 void addPartialFourier();
00245
00246 void getOmega(wbvector<double> &om, unsigned &linfit) const;
00247 void getOmega(wbvector<double> &om) const {
00248     unsigned linfit=0; getOmega(om,linfit);
00249 }
00250
00251 void applyIROPfac(
00252     const char *F, int L, const wbvector<double> &fac, char rixs=0);
00253
00254 void getSpecData(
00255     wbvector<double> &om, wbararray<TS> &A,
00256     unsigned linfit=0
00257 ) const;
00258
00259 void getSpecData(
00260     wbvector<double> &om, wbMatrix<TS> &A,
00261     unsigned linfit=0
00262 );
00263
00264 void mergeNRGData();
00265
00266 void getSmoothSpec_t(double sigma, double eps,
00267     char bswitch = LOG_GAUSS_BRD, char vflag=1);
00268
00269 void Fourier(
00270     wbvector<double> &om, wbMatrix<wbcomplex> &ST,
00271     double alpha, double Lambda, char vflag=1
00272 );
00273
00274 void Fourier(
00275     wbvector<double> &om, wbMatrix<TS> &A,
00276     const wbvector<double> &tt,
00277     wbMatrix<wbcomplex> &at,
00278     double alpha=0., char vflag=1
00279 );
00280
00281 void dispSumRule();
00282
00283 wbararray<TS> AP,AN;
00284 wbMatrix<TS> Ap,An;
00285 wbMatrix<wbcomplex> At;
00286 wbvector<double> tt;
00287 wbstring istr;
00288
00289 mxArray *toMx() const;
00290 int init(
00291     const mxArray *S,
00292     double &emin, double &emax, unsigned &nlog
00293 );
00294
00295 unsigned raw;
00296 wbvector< RSpecData<TS> > RAW;
00297
00298 protected:
00299 private:
00300
00301     double l0,fac;
00302     unsigned N;
00303
00304     char check_avg_om;
00305     wbararray<TS> AW;
00306
00307     double OM2IDX(const double &om) const {
00308         return ((log(fabs(om))-l0)*fac);
00309     }
00310
00311     inline unsigned om2idx(const double &om) const {
00312         if (om==0.) return 0;
00313
00314         int k = (int)floor(OM2IDX(om));
00315         if (k<1) k=1; else if (k>=(int)N) k=N-1;
00316
00317         return (unsigned)k;
00318     }
00319
00320     inline double idx2om(const unsigned &k) const {

```

```

00321         return exp( ((double)k+0.5)/fac + 10 );
00322     }
00323
00324     void initLG(
00325         wbMatrix<double> &wlg, unsigned n, double sigma,
00326         char bswitch = LOG_GAUSS_BRD, char vflag=1
00327     );
00328
00329     void foldLogGaussian(
00330         const wbMatrix<double> &wlg,
00331         TS *aa, TS a0, unsigned i0
00332     );
00333 };
00334
00335 template <class TS>
00336 void Spectral<TS>::init (
00337     const char *istr0,
00338     unsigned Nops,
00339     double d,
00340     double T,
00341     unsigned Niter
00342 ){
00343     unsigned Nom;
00344     double l2;
00345
00346     if (Emin<=0. || Emax<=0. || Emin>=0.1*Emax || nlog==0) wblog(FL,
00347         "ERR %s() invalid range E = [%g .. %g @ %d] %s", myname, Emin, Emax, nlog,
00348         (Emin>0 && Emax>0 && Emin<Emax) ? " (require Emin<Emax/10)":"");
00349     if (d<0) wblog(FL, "ERR invalid Delta (%g neg!)", d);
00350
00351     if (eps<Emin) wblog(FL, "NB! eps=%g < min(|om|)=%g", eps, Emin);
00352     if (alpha!=0.25) {
00353         if (alpha==0.5) wblog(FL,
00354             "WRN %s() using symmetric log-Gaussian broadening", FCT);
00355         else wblog(FL, "WRN %s() using "
00356             "log-Gaussian with shift alpha=%g !?", FCT, alpha);
00357     }
00358
00359     Delta=d;
00360
00361     l0 =log(Emin);
00362     l2 =log(Emax);
00363     fac=log10(M_E)*double(nlog);
00364     Nom=(unsigned)((l2-l0)*fac)+1;
00365
00366     Ap.init(Nops,Nom);
00367     An.init(Nops,Nom); Ar.init(Nops);
00368
00369     if (int(Niter)>0) {
00370         AP.init(Nom,Nops,Niter);
00371         AN.init(Nom,Nops,Niter);
00372     }
00373
00374     istr=istr0; Temp=T;
00375 };
00376
00377 template <class TS>
00378 void Spectral<TS>::Add(
00379     const wbararray<double> &Om,
00380     const wbararray<TS> &CC,
00381     unsigned s,
00382     char dblock,
00383     char mflag
00384 ){
00385     unsigned i,k, n=Om.SIZE.prod();
00386     double x, emin=0.1*idx2om(0);
00387     TS q, r=0;
00388
00389     double bfac=(!mflag && Temp>0 ? 10/Temp : 0);
00390
00391     double zfac=(Temp>0 ? 1/Temp : 0);
00392
00393     if (!Om.sameSize(CC)) {
00394         Om.info("Om"); CC.info("CC");
00395         wblog(FL, "ERR size mismatch between Om and CC!");
00396     }
00397     if (dblock) {
00398         if (Om.SIZE.len!=2 || Om.SIZE[0]!=Om.SIZE[1]) wblog(FL,
00399             "ERR %s() quadratic block expected (%s)",
00400             FCT, Om.sizeStr().data);
00401     }
00402
00403     if (s>=Ap.dim1 || s>=Ar.len) wblog(FL,
00404         "ERR index out of bounds (%d/%d/%d)", s, Ap.dim1, Ar.len);
00405
00406     for (i=0; i<n; ++i) { x=Om.data[i]; k=om2idx(x);
00407         if (x<0. || (x<=emin && s%2))

```

```

00408         An(s,k)+=CC.data[i];
00409     else Ap(s,k)+=CC.data[i];
00410
00411     if (x!=0) {
00412         q=CC.data[i]/x;
00413         if (bfac>0) { x=bfac*fabs(x); if (x<10) {
00414             double w=exp(-x*x);
00415             q*=(1-w);
00416         }}
00417         r+=q;
00418     }
00419     else if (mflag && !(s%2)) {
00420         r+=zfacs*CC.data[i];
00421     }
00422 }
00423
00424 Ar[s]+=r;
00425
00426 if (mspec.dim2) {
00427     double dbl; unsigned j;
00428
00429     if (mspec.dim1!=Ap.dim1) {
00430         wblog(FL,
00431             "WRN mspec must be initialized AFTER ASpec!\n"
00432             "resize (%d->%d x %d)", mspec.dim1, Ap.dim1, mspec.dim2
00433         );
00434         mspec.Resize(Ap.dim1,mspec.dim2);
00435     }
00436
00437     for (i=0; i<n; i++)
00438     for (dbl=CC.data[i], j=0; j<mspec.dim2; j++) {
00439         if (j) dbl*=Om.data[i];
00440         mspec(s,j)+=dbl;
00441     }
00442 }
00443 };
00444
00445 template <class TS>
00446 void Spectral<TS>::Add2buf(
00447     const wbararray<double> &Om, const wbararray<TS> &CC,
00448     unsigned s
00449 ){
00450     unsigned i,k,n=Om.SIZE.prod();
00451
00452     if (!Om.sameSize(CC)) {
00453         Om.info("Om"); CC.info("CC");
00454         wblog(FL,"ERR Size mismatch between Om and CC!");
00455     }
00456
00457     if (s>Ap_buf.dim1)
00458         wblog(FL, "ERR Index out of bounds (%d,%d)",s, Ap_buf.dim1);
00459
00460     for (i=0; i<n; i++) {
00461         k=om2idx(Om.data[i]);
00462         if (Om.data[i]<0.)
00463             An_buf(s,k)+=CC.data[i];
00464         else Ap_buf(s,k)+=CC.data[i];
00465     }
00466 };
00467
00468 template <class TS>
00469 void Spectral<TS>::crossAddBuf(
00470     const char minor,
00471     char vflag
00472 ){
00473     unsigned s, Nom=Ap.dim2;
00474     int i;
00475
00476     if (!Ap.sameSize(Ap_buf) || !An.sameSize(An_buf) ||
00477         !Ap.sameSize(An))
00478     ){
00479         Ap.info("Ap"); Ap_buf.info("Ap_buf");
00480         An.info("An"); An_buf.info("An_buf");
00481         wblog(FL,"ERR Spectral - severe size mismatch.");
00482     }
00483
00484     for (s=0; s<Ap.dim1; s++) { i=0;
00485
00486         i+=crossAddBuf_aux(Ap.rec(s), Ap_buf.rec(s), Nom,minor,vflag);
00487         i+=crossAddBuf_aux(An.rec(s), An_buf.rec(s), Nom,minor,vflag);
00488
00489         if (i>1) {
00490             wblog(FL,"WRN empty buffer (%d/%d) !?",s,Ap.dim1);
00491             continue;
00492         }
00493     }
00494 }

```



```

00495 };
00496
00497 template <class TS>
00498 int Spectral<TS>::crossAddBuf_aux(
00499     TS *a,
00500     const TS *b,
00501     const unsigned Nom,
00502     const char minor,
00503     char vflag
00504 ){
00505     unsigned i, i1, i2, amin, amax, bmin, bmax;
00506     double t, tfac;
00507
00508     if (minor<1 || minor>2) wblog(FL,
00509         "ERR invalid minor flag %d", minor);
00510
00511     for (i=0; i<Nom; ++i) { if (a[i]!=0.) break; }; amin=i;
00512     for (i=Nom-1; i>=amin; --i) { if (a[i]!=0.) break; }; amax=i;
00513     for (i=0; i<Nom; ++i) { if (b[i]!=0.) break; }; bmin=i;
00514     for (i=Nom-1; i>=bmin; --i) { if (b[i]!=0.) break; }; bmax=i;
00515
00516     if (bmin>bmax) {
00517         return vflag;
00518     }
00519
00520     if (amin>amax) {
00521         for (i=bmin; i<=bmax; i++) a[i]=b[i];
00522         return 0;
00523     }
00524
00525     i1=MAX(amin,bmin); i2=MIN(amax,bmax);
00526
00527     if (i1==i2) { a[i1]=0.5*(a[i1]+b[i2]); return 0; }
00528
00529     tfac = 1. / double(i2-i1);
00530
00531     for (i=0; i<Nom; ++i) {
00532         if (i<i1 || i>i2) a[i]=b[i];
00533         else {
00534             t = tfac * double(i-i1);
00535             if (minor==1)
00536                 a[i] = (1.-t)*a[i] + t*b[i];
00537             else a[i] = t*a[i] + (1.-t)*b[i];
00538         }
00539     }
00540
00541     return 0;
00542 };
00543
00544 template <class TS>
00545 void Spectral<TS>::saveIter(const char *F, int L, unsigned iter){
00546
00547     unsigned n=Ap.numel();
00548
00549     if (AP.SIZE.len!=3 || AP.SIZE!=AN.SIZE || n!=AP.SIZE[0]*AP.SIZE[1])
00550         wblog(F_L,
00551             "ERR %s() invalid initialization of AP/AN [%s; %s; %dx%d]",
00552             FCT,AP.sizeStr().data, AN.sizeStr().data, Ap.dim1, Ap.dim2
00553         );
00554     if (iter>=AP.SIZE[2]) wblog(F_L,
00555         "ERR %s() iter out of bounds AP/AN [%s; %d/%d]",
00556         FCT,AP.sizeStr().data, iter+1, AP.SIZE[2]
00557     );
00558
00559     memcpy(AP.data+iter*n, Ap.data, n*sizeof(Ap.data[0]));
00560     memcpy(AN.data+iter*n, An.data, n*sizeof(An.data[0]));
00561 };
00562
00563 template <class TS>
00564 void Spectral<TS>::getOmega(wbvector<double> &om, unsigned &linfit) const {
00565
00566     unsigned i, Nom=Ap.dim2;
00567     double dbl;
00568
00569     if (linfit) linfit=unsigned(fac);
00570     if (linfit>Nom) wblog(FL,"ERR Nom<linfit (%d,%d) !?",Nom,linfit);
00571
00572     om.init(2*Nom);
00573
00574     dbl = idx2om(linfit) / (linfit+0.5);
00575
00576     for (i=0; i<linfit; ++i) om[Nom-1-i] = -(om[Nom+i] = dbl*(i+0.5));
00577     for ( ; i<Nom; ++i) om[Nom-1-i] = -(om[Nom+i] = idx2om(i) );
00578 }
00579
00580 template <class TS>
00581 void Spectral<TS>::applyIROPfac(

```

```

00582     const char *F, int L, const wbvector<double> &fac) {
00583
00584     unsigned i1,i2,s,m, n=fac.len;
00585     double x;
00586
00587     if (2*n!=Ap.dim1 || 2*n!=An.dim1 || 2*n!=Ar.len) wblog(FL,
00588         "ERR %s() size mismatch (%d,%d,%d/2*%d)",
00589         FCT,Ap.dim1,An.dim1,Ar.len,n);
00590
00591     for (s=0; s<n; ++s) { if (fac[s]!=1) {
00592         if (fac[s]<1 || fac[s]!=round(fac[s])) wblog(FL,
00593             "ERR %s() got fac[%d]=%g !?",FCT,s+1,fac[s]);
00594         wblog(FL," * applying IROP factor 1/%g to op(%d)",fac[s],s+1);
00595
00596         x=1/fac[s]; i1=2*s; i2=2*s+1;
00597         Ap.rowTimes(i1,x); An.rowTimes(i1,x); Ar[i1]*=x;
00598         Ap.rowTimes(i2,x); An.rowTimes(i2,x); Ar[i2]*=x;
00599     }}
00600
00601     if (!Ap_buf.isEmpty() || !An_buf.isEmpty()) {
00602         if (!Ap_buf.sameSize(Ap) || !An_buf.sameSize(Ap)) wblog(FL,
00603             "ERR %s() severe size mismatch %dx%d <> %dx%d <> %dx%d !?", FCT,
00604             Ap_buf.dim1,Ap_buf.dim2,An_buf.dim1,An_buf.dim2,Ap.dim1,Ap.dim2
00605         );
00606         for (s=0; s<n; ++s) { if (fac[s]>1) { x=1/fac[s]; i1=2*s; i2=2*s+1;
00607             Ap_buf.rowTimes(i1,x); An_buf.rowTimes(i1,x);
00608             Ap_buf.rowTimes(i2,x); An_buf.rowTimes(i2,x);
00609         }}
00610     }
00611
00612     if (!AP.isEmpty() || !AN.isEmpty()) {
00613         if (AP.SIZE.len!=3 || !AP.sameSize(AN) || AP.SIZE[1]!=2*n)
00614             wblog(FL,
00615                 "ERR %s() size mismatch (%s; %s; %dx%d) !?", FCT,
00616                 AP.sizeStr().data, AN.sizeStr().data, Ap.dim1,Ap.dim2
00617             );
00618
00619         TS *ap=AP.data, *an=AN.data;
00620         unsigned n2=2*AP.SIZE[0]; m=n*AP.SIZE[2];
00621
00622         for (i1=0; i1<m; ++i1, ap+=n2, an+=n2) { s=i1%n;
00623             if (fac[s]>1) { x=1/fac[s];
00624                 Wb::timesRange(ap, x, n2);
00625                 Wb::timesRange(an, x, n2);
00626             }
00627         }
00628     }
00629
00630     if (!A0.isEmpty() || !mspec.isEmpty()) { wblog(FL,
00631         "WRN IROPfac yet not applied to A0 and mspec (%d,%d)",
00632         A0.isEmpty(), mspec.isEmpty());
00633     wblog(FL," * A0: %dx%d",A0.dim1,A0.dim2);
00634     wblog(FL," * sp: %dx%d",mspec.dim1,mspec.dim2);
00635 }
00636 };
00637
00638 template <class TS>
00639 void Spectral<TS>::getRawData(
00640     wbvector<double> &om, wbMatrix<TS> &A, unsigned linfit,
00641     const char *isbuf
00642 ) const {
00643     unsigned i,s,Nom=Ap.dim2;
00644
00645     getOmega(om, linfit);
00646
00647     A.init(Ap.dim1, 2*Nom);
00648
00649     if (A.dim2!=om.len || !Ap.sameSize(An)) wblog(FL,
00650         "ERR severe dimension mismatch (%d,%d).", A.dim2, om.len);
00651
00652     if (isbuf && isbuf[0]) {
00653         if (strcmp(isbuf,"buf"))
00654             wblog(FL, "ERR invalid flag `%s'", isbuf);
00655
00656         if (!Ap.sameSize(Ap_buf)) {
00657             if (Ap_buf.isEmpty())
00658                 wblog(FL, "ERR buffer not set !?"); else
00659                 wblog(FL, "ERR severe size mismatch !?");
00660         }
00661
00662         for (s=0; s<Ap.dim1; s++) { A(s,Nom)=NAN;
00663             for (i=0; i<Nom; ++i) {
00664                 A(s,Nom-i-1) = An_buf(s,i);
00665                 A(s,Nom+i) = Ap_buf(s,i);
00666             }
00667         }
00668     }

```

```

00669     else {
00670         for (s=0; s<Ap.dim1; s++) { A(s,Nom)=NAN;
00671             for (i=0; i<Nom; ++i) {
00672                 A(s,Nom-i-1) = An(s,i);
00673                 A(s,Nom+i) = Ap(s,i);
00674             }
00675         }
00676     }
00677
00678     if (linfit) {
00679         for (s=0; s<A.dim1; s++) {
00680             Wb::set2avg(A.rec(s)+Nom, linfit);
00681             Wb::set2avg(A.rec(s)+Nom-linfit, linfit);
00682         }
00683     }
00684 };
00685
00686 template <class TS>
00687 void Spectral<TS>::pairRawData() {
00688     unsigned i,j,k, m=Ap.dim1, n=Ap.dim2;
00689
00690     if (!Ap.sameSize(An)) wblog(FI, "ERR severe size mismatch!");
00691     if (Ap.dim1%2) wblog(FI,
00692         "ERR Ap has odd number of rows !? (%d)", Ap.dim1);
00693
00694     for (i=0; i<m; i+=2)
00695         for (k=i/2, j=0; j<n; j++) {
00696             Ap(k,j)=Ap(i,j)+Ap(i+1,j);
00697             An(k,j)=An(i,j)+An(i+1,j);
00698         }
00699
00700     Ap.Resize(m/2,n);
00701     An.Resize(m/2,n);
00702
00703     if (mspec.dim1==0) return;
00704
00705     m=mspec.dim1; n=mspec.dim2;
00706
00707     if (m%2) wblog(FI,
00708         "ERR mspec has odd number of rows !? (%d)", m);
00709
00710     for (i=0; i<m; i+=2)
00711         for (k=i/2, j=0; j<n; j++) mspec(k,j)=mspec(i,j)+mspec(i+1,j);
00712
00713     mspec.Resize(m/2,n);
00714 }
00715
00716 template <class TS>
00717 void Spectral<TS>::getPARTIAL(
00718     wbararray<TS> &A1, wbararray<TS> &A2
00719 ) const {
00720
00721     if (AP.isEmpty() && AN.isEmpty()) {
00722         A1.init(); A2.init(); return;
00723     }
00724
00725     if (AP.SIZE.len!=3 || AP.SIZE!=AN.SIZE || AP.SIZE[1]%2) wblog(FI,
00726         "ERR %s() invalid initialization of AP/AN [%s; %s; %dx%d]",
00727         FCT,AP.sizeStr().data, AN.sizeStr().data, Ap.dim1, Ap.dim2
00728     );
00729
00730     int i,j, m=AP.SIZE[1]/2, n=AP.SIZE[0], K=AP.SIZE[2], n1=n-1, n2=2*n*m;
00731     const TS *ap=AP.data, *an=AN.data;
00732     TS *a1, *a2;
00733
00734     A1.init(2*n,m,K); a1=A1.data;
00735     A2.init(2*n,m,K); a2=A2.data;
00736
00737     for (K*=m, j=0; j<K; ++j) {
00738         if (j>=m) {
00739             for (i=0; i<n; ++i) { a1[i]=an[n1-i] - an[n1-i-n2]; }; a1+=n; an+=n;
00740             for (i=0; i<n; ++i) { a1[i]=ap[i] - ap[i-n2]; }; a1+=n; ap+=n;
00741             for (i=0; i<n; ++i) { a2[i]=an[n1-i] - an[n1-i-n2]; }; a2+=n; an+=n;
00742             for (i=0; i<n; ++i) { a2[i]=ap[i] - ap[i-n2]; }; a2+=n; ap+=n;
00743         }
00744         else {
00745             for (i=0; i<n; ++i) { a1[i]=an[n1-i]; }; a1+=n; an+=n;
00746             for (i=0; i<n; ++i) { a1[i]=ap[i]; }; a1+=n; ap+=n;
00747             for (i=0; i<n; ++i) { a2[i]=an[n1-i]; }; a2+=n; an+=n;
00748             for (i=0; i<n; ++i) { a2[i]=ap[i]; }; a2+=n; ap+=n;
00749         }
00750     }
00751 };
00752
00753 template <class TS>
00754 void Spectral<TS>::detailedBalance(
00755     const double T,

```

```

00756     wbvector<double> &om, wbMatrix<TS> &aa
00757 ) const {
00758
00759     unsigned i,j,k1,k2,m,n;
00760     wbvector<double> f1,f2;
00761     wbMatrix<TS> ar;
00762
00763     if (T<0) wblog(FL,"ERR T=%g !?", T);
00764
00765     getRawData(om,ar);
00766
00767     n=Ap.dim2; f1.init(2*n);
00768
00769     if (T==0) {
00770         for (i=0; i<n; i++) f1[i]=1;
00771     }
00772     else {
00773         double beta=1./T;
00774         for (i=0; i<om.len; i++) f1[i]=1./(1.+exp(beta*om[i]));
00775     }
00776
00777     f1.flip(f2);
00778
00779     m=Ap.dim1; n=ar.dim2;
00780     aa.init(2*m,n);
00781
00782     for (i=0; i<m; ++i) { k1=2*i; k2=k1+1;
00783         for (j=0; j<n; j++) {
00784             aa(k1,j)=ar(i,j)*f2[j];
00785             aa(k2,j)=ar(i,j)*f1[j];
00786         }
00787     }
00788 }
00789
00790 template <class TS>
00791 void Spectral<TS>::dispSumRule() {
00792
00793     unsigned i; double w=0;
00794     wbvector<TS> sp=Ap.recSum(),sn=An.recSum();
00795     Wb::termcolor Tc(Wb::TCOLS::BLUE);
00796
00797     wblog(FL,"r%60s%N"
00798         "    sum %sp_raw : %s%N"
00799         "    + %sn_raw : %s ", "",
00800         istr.data, sp.toStrf("%8.5f").data,
00801         istr.data, sn.toStrf("%8.5f").data
00802     );
00803     sp+=sn;
00804
00805     if (!A0.isEmpty()) {
00806         if (A0.dim1!=sp.len) wblog(FL,
00807             "ERR size mismatch (%d/%dx%d)", sp.len,A0.dim1,A0.dim2);
00808         sp+=A0.recSum();
00809     }
00810
00811     if (sp.len) { w=REAL(sp[0]); }
00812     for (i=0; i<sp.len; ++i) { if (ABS(sp[i]-w)>1E-2) break; }
00813
00814     if (i==sp.len && fabs(w)>1E-4 && fabs(w-round(w))<1E-4) { sp=w;
00815         wblog(FL,"r%60s\r    %s\\", "",Tc.e1);
00816         printf("total%+7g: %s",-w,sp.toStrf("%8.2g").data);
00817         printf("%s\n",Tc.em);
00818     }
00819     else {
00820         printf("          %stotal : ",Tc.e1);
00821         for (i=0; i<sp.len; ++i) { w=round(REAL(sp[i]));
00822             if (ABS(sp[i]-w)<1E-5) {
00823                 if (w!=0)
00824                     printf("%2g%+5.0E ",w,REAL(sp[i])-w);
00825                 else printf("%8.1E ",REAL(sp[i]));
00826             }
00827             else { printf("%8.5f ",REAL(sp[i])); }
00828         }; printf("%s\n",Tc.em);
00829     }
00830 };
00831
00832 template <class TS>
00833 void Spectral<TS>::dispSumRule(
00834     const wbvector<double> &om, const wbMatrix<TS> &A
00835 ){
00836     wbvector<TS> s(A.dim1);
00837     unsigned i; double m=0;
00838
00839     if (om.len!=A.dim2) wblog(FL,
00840         "ERR Dimension mismatch (%d,%d)", om.len, A.dim2);
00841
00842     for (unsigned i=0; i<A.dim1; ++i)

```

```

00843     s[i] = Wb::IntTrapez(om.data, A.rec(i), om.len);
00844
00845     wblog(FL, "\r%60s\r      "
00846           "int AA(om)   : %s", "", s.toStrf("%8.5f").data);
00847
00848     if (!A0.isEmpty()) {
00849         if (A0.dim1!=s.len) wblog(FL,
00850             "ERR size mismatch (%d/%dx%d)", s.len, A0.dim1, A0.dim2);
00851         s+=A0.recSum();
00852     }
00853
00854     if (s.len) {
00855         for (m=round(REAL(s[0])), i=0; i<s.len; ++i) {
00856             if (ABS(s[i]-m)>1E-2) break;
00857         }
00858         if (m && i==s.len) { s-=m;
00859             char ms[8]; Wb::termcolor Tc(Wb::TCOLS::BLUE);
00860             if (m) {
00861                 i=snprintf(ms, 8, "%+3g", -m);
00862                 if (int(i)<0) wblog(FL, "WRN %s() i=%d", FCT, i);
00863             }
00864             else { *ms=0; }
00865             wblog(FL, "\r%60s\r      %s*   total%-3s: %s%s",
00866                 "", Tc.el, ms, s.toStrf("%8.2g").data, Tc.em);
00867         }
00868     }
00869 };
00870
00871 template <class TS>
00872 void Spectral<TS>::getSmoothSpec(
00873     wbvector<double> &om, wbMatrix<TS> &A, char vflag
00874 ){
00875     unsigned linfit=0;
00876
00877     getOmega(om, linfit);
00878     getSmoothSpec(A, om, vflag);
00879 };
00880
00881 template <class TS>
00882 void Spectral<TS>::getSmoothSpec(
00883     wbMatrix<TS> &A, const wbvector<double> &omega,
00884     char vflag
00885 ){
00886     unsigned i, s, Nom=Ap.dim2;
00887     wbvector<double> OM(Nom), I;
00888     double x, dbl, om, aom;
00889     TS w1=0., w2=0.;
00890
00891     if (fac<=0 || sigma<=0 || sigma2<=0 || eps<=0 || Nom<4)
00892         wblog(FL, "ERR %s()\n"
00893             "fac=%0.3g, sigma=%0.3g, sigma2=%0.3g, eps=%0.3g, Nom=%d !?",
00894             FCT, fac, sigma, sigma2, eps, Nom);
00895
00896     for (i=0; i<OM.len; i++) OM[i]=idx2om(i);
00897
00898     if (vflag) {
00899         double w1=omega.min(), w2=omega.max(), w0=omega.aMin();
00900         if (w1==w2)
00901             sprintf(str, "%0.3g .. %0.3g", w0, w2);
00902         else sprintf(str, "%0.3g .. %0.3g .. %0.3g", w1, w0, w2);
00903
00904         wblog(FL, " * "
00905             "%s() %g bins/dec (len=%d)\n"
00906             "sigma=%0.3g [eps=%0.3g @ %0.3g]\n"
00907             "|omega| range: %s",
00908             myname, fac/log10(M_E), Nom, sigma, eps, sigma2, str);
00909         if (fabs((w0-idx2om(0))/w0)>0.01 ||
00910             fabs((w2-idx2om(Nom-1))/w2)>0.01) wblog(FL,
00911             "|emin...emax|= %0.3g .. %0.3g (bins)", idx2om(0), idx2om(Nom-1));
00912         if (vflag && (Ap.dim1<8 || vflag>1)) { dispSumRule(); }
00913     }
00914
00915     A.init(Ap.dim1, omega.len);
00916
00917     for (s=0; s<Ap.dim1; ++s)
00918         for (i=0; i<omega.len; ++i) { om=omega[i]; aom=fabs(om);
00919             if (aom>eps) x=1.;
00920             else {
00921                 if (aom==0.) x=0.;
00922                 else {
00923                     dbl = (log(aom)-log(eps))/sigma;
00924                     x = exp(-dbl*dbl);
00925                     if (x<1E-10) x=0.;
00926                 }
00927             }
00928         }
00929 }

```

```

00930
00931     if (x!=0.)
00932         w1 = foldLogGaussian(aom, om>0 ? Ap.rec(s) : An.rec(s), Nom);
00933
00934     if (x!=1.) {
00935         double b=MAX(sigma2*eps,om*exp(-sigma*sigma/4));
00936
00937         w2 = foldGaussian( om, b, OM.data, Ap.rec(s), Nom)
00938             + foldGaussian(-om, b, OM.data, An.rec(s), Nom);
00939     }
00940
00941     A(s,i) = x*w1 + (1.-x)*w2;
00942 }
00943
00944 if (vflag && (Ap.dim1<8 || vflag>1)) dispSumRule(omega,A);
00945 };
00946
00947 template <class TS>
00948 TS Spectral<TS>::foldLogGaussian(
00949     const double om, const TS *aa, const unsigned n
00950 ){
00951     double k, dbl, efac;
00952     TS xa=0.;
00953
00954     if (sigma<=0. || fac<=0.) wblog(FL,
00955         "ERR sigma=%g, fac=%g !?", sigma, fac);
00956
00957     k=OM2IDX(om)-0.5;
00958     efac=1./(fac*sigma);
00959
00960     double dx=alpha*sigma;
00961
00962     for (unsigned i=0; i<n; ++i) {
00963         dbl=efac*(k-double(i))-dx;
00964         xa += aa[i]*exp(-dbl*dbl);
00965     }
00966     xa /= (sigma*sqrt(M_PI)*om);
00967
00968     if (alpha!=0.25) {
00969         xa *= exp(+sigma*sigma*(alpha-0.25));
00970     }
00971
00972     return xa;
00973 };
00974
00975 template <class TS>
00976 TS Spectral<TS>::foldGaussian(
00977     const double om, const double b,
00978     const double *oo, const TS *aa, const unsigned n
00979 ){
00980     double dbl, efac;
00981     TS xa=0.;
00982
00983     if (b<=0.) wblog(FL, "ERR b=%g !?", b);
00984
00985     efac = 1./b;
00986
00987     for (unsigned i=0; i<n; i++) {
00988         dbl=efac*(om-oo[i]);
00989         xa += aa[i]*exp(-dbl*dbl);
00990     }
00991
00992     xa *= 1. / (b*sqrt(M_PI));
00993
00994     return xa;
00995 };
00996
00997 template <class TS>
00998 void Spectral<TS>::KKreal(
00999     wbMatrix<wbcomplex> &GZ, wbvector<double> &OM, wbvector<double> &DOM
01000 ) const {
01001
01002     unsigned i,j,s, d=Ap.dim1, Nom;
01003     wbMatrix<double> II, RR;
01004     double doi, doj;
01005
01006     getRawData(OM, II); Nom=OM.len;
01007     Wb::getDiff(OM,DOM);
01008
01009     #if 0
01010         wblog(FL,"TST Get smooth data ...");
01011
01012         getSmoothSpec(OM, II, 1E-6, 0.3, 0); Nom=OM.len;
01013         Wb::getDiff(OM,DOM);
01014
01015         for (s=0; s<d; s++)
01016             for (i=0; i<Nom; i++) II(s,i) *= DOM[i];

```

```

01017 #endif
01018
01019     RR.init(d,Nom);
01020
01021     for (i=0; i<Nom; ++i)
01022     for (j=i+1; j<Nom; ++j) {
01023
01024         doi=DOM[i]/(OM[j]-OM[i]);
01025         doj=DOM[j]/(OM[i]-OM[j]);
01026
01027         for (s=0; s<d; s++) {
01028             RR(s,i) += II(s,j)*doi;
01029             RR(s,j) += II(s,i)*doj;
01030         }
01031     }
01032
01033 #if 0
01034     wbMatrix<double> Ip2, In2;
01035     wblog(FL,"TST backtransform real->imag");
01036
01037     RR*=(1/M_PI); RR.put("RR","base",'r');
01038
01039     KKimag(Rp,Rn,OM,Ip2,In2);
01040
01041     Ip2.put("Ip","base",'r');
01042     In2.put("In","base",'r');
01043
01044     wblog(FL,"ERR");
01045 #endif
01046
01047     II *= M_PI;
01048
01049     GZ.set(RR,II);
01050 }
01051
01052 void KKimag(
01053     const wbMatrix<double> &RR,
01054     const wbvector<double> &OM,
01055     wbMatrix<double> &II
01056 ){
01057     unsigned i,j,s, d=RR.dim1, Nom=RR.dim2;
01058     double doi, doj;
01059     wbvector<double> DOM;
01060
01061     Wb::getDiff(OM, DOM); II.init(d,Nom);
01062
01063     if (OM.len!=Nom) wblog(FL,
01064         "ERR severe dimension mismatch (%d,%d) !?", OM.len, Nom);
01065
01066     for (i=0; i<Nom; ++i)
01067     for (j=i+1; j<Nom; ++j) {
01068         doi=DOM[i]/(OM[i]-OM[j]); doj=DOM[j]/(OM[j]-OM[i]);
01069         for (s=0; s<d; s++) {
01070             II(s,i) += RR(s,j)*doi;
01071             II(s,j) += RR(s,i)*doj;
01072         }
01073     }
01074
01075     II *= (1./M_PI);
01076 }
01077
01078 template <class TS>
01079 void TDSpectral<TS>::init(const char *F, int L, const char *istr0,
01080     const wbvector<double> &t0,
01081     unsigned Nnrg, unsigned Nop,
01082     double Emin, double Emax, unsigned nl, char vflag
01083 ){
01084     double l2;
01085
01086     if (Emin<=0. || Emax<=0. || Emin>=0.1*Emax || nl==0) wblog(F,L,
01087         "ERR invalid parameter set [%g %g %d]%s", Emin, Emax, nl,
01088         (Emin>0 && Emax>0 && Emin<Emax)? " (require Emin<Emax/10)":"");
01089
01090     l0 = log(Emin);
01091     l2 = log(Emax);
01092
01093     fac= log10(M_E)*double(nl);
01094
01095     N = (unsigned)((l2-l0)*fac)+1;
01096
01097     AP.init(N,Nop,Nnrg);
01098     AN.init(N,Nop,Nnrg); if (check_avg_om) {
01099     AW.init(2,Nop,Nnrg); }
01100
01101     if (vflag) {
01102         wblog(F_L,"%3s TDSpectral: %.2g .. %.2g (%d/dec; 2x%d)",
01103             istr0 ? istr0 : "<i>", Emin, Emax, nl, N);

```

```

01104     }
01105
01106     istr=istr0; tt=t0;
01107 };
01108
01109 template <class TS>
01110 wbararray<TS> TDSpectral<TS>::getAvgOm() {
01111
01112     if (AW.rank()!=3 || AW.dim(1)!=2) wblog(FL,
01113         "ERR invalid AW data structure [%s] !?",AW.sizeStr().data);
01114
01115     unsigned i=0, n=AW.numel();
01116     wbararray<TS> aw(AW); if (aw.isEmpty()) return aw;
01117     TS *a=AW.data, *b=aw.data;
01118
01119     for (; i<n; i+=2) { b[i] = ((a[i+1]!=0.) ? a[i]/a[i+1] : a[i]); }
01120
01121     return aw;
01122 };
01123
01124 template <class TS>
01125 void TDSpectral<TS>::Add(
01126     const wbararray<double> &Om,
01127     const wbararray<TS> &CC, unsigned iter, unsigned s
01128 ){
01129     static int iter_last=-1;
01130     unsigned i,k,n=Om.numel();
01131     TS *ap, *an;
01132
01133     if (!Om.sameSize(CC)) {
01134         Om.info("Om"); CC.info("CC");
01135         wblog(FL,"ERR size mismatch between Om and CC!");
01136     }
01137
01138     if (s>=AP.dim(2) || iter>=AP.dim(3)) wblog(FL,
01139         "ERR index out of bounds (op=%d/%d; k=%d/%d)",
01140         s+1,AP.dim(2),iter+1,AP.dim(3));
01141
01142     ap=&AP(0,s,iter);
01143     an=&AN(0,s,iter);
01144
01145     for (i=0; i<n; ++i) {
01146         k=om2idx(Om.data[i]);
01147         if (Om.data[i]<0.)
01148             an[k]+=CC.data[i];
01149         else ap[k]+=CC.data[i];
01150     }
01151
01152     if (raw) {
01153         if (iter_last!=(int)iter) {
01154             wblog(FL,"%NTST iter=%d",iter);
01155             if (RAW.len) addPartialFourier();
01156             RAW.initDef(AP.dim(2));
01157             wblog(FL,"TST done");
01158         }
01159
01160         RAW[s].Append(Om.data, CC.data, n);
01161
01162         iter_last=iter;
01163     }
01164
01165     if (check_avg_om) {
01166         if (AW.rank()!=3 || AW.dim(1)!=2 ||
01167             AW.dim(2)!=AP.dim(2) || AW.dim(3)!=AP.dim(3)) wblog(FL,
01168             "ERR size mismatch for AW [%s] ? [%s]",
01169             AW.sizeStr().data, AP.sizeStr().data
01170         );
01171         AW(0,s,iter)+=CC.dotProd(Om);
01172         AW(1,s,iter)+=CC.sum();
01173     }
01174 };
01175
01176 template <class TS> inline
01177 void TDSpectral<TS>::addPartialFourier() {
01178
01179     if (!RAW.len) return;
01180
01181     unsigned s=0, m=RAW.len;
01182     wbMatrix<wbcomplex> at;
01183     wbMatrix<TS> aa;
01184
01185     if (m!=AP.dim(2)) wblog(FL,
01186         "ERR severe size mismatch (%d/%d)",m,AP.dim(2));
01187
01188     if (At.isEmpty()) At.init(AP.dim(2),tt.len);
01189
01190     for (s=0; s<m; ++s) {

```



```

01191         const RSpecData<TS> &R=RAW[s];
01192         if (R.om.len!=R.cc.len) wblog(FL,
01193             "ERR RAW[%d].len=%d/%d !?",s,R.om.len,R.cc.len);
01194
01195         aa.init2ref(1,R.cc.len,R.cc.data);
01196
01197         Wb::Fourier(R.om,aa,tt,at);
01198         At.recAddP(s,at.data);
01199     }
01200
01201     RAW.init();
01202 };
01203
01204 template <class TS>
01205 void TDSpectral<TS>::getOmega(
01206     wvector<double> &om, unsigned &linfit
01207 ) const {
01208
01209     unsigned i;
01210     double dbl;
01211
01212     if (linfit) linfit=unsigned(fac);
01213     if (linfit>N) wblog(FL,"ERR linfit>N (%d,%d) !?", N, linfit);
01214
01215     om.init(2*N);
01216
01217     dbl = idx2om(linfit) / (linfit+0.5);
01218
01219     for (i=0; i<linfit; i++) om[N-1-i] = -(om[N+i] = dbl*(i+0.5));
01220     for ( ; i<N; i++) om[N-1-i] = -(om[N+i] = idx2om(i) );
01221 }
01222
01223 template <class TS>
01224 void TDSpectral<TS>::applyIROPfac(
01225     const char *F, int L, const wvector<double> &fac, char rixs) {
01226
01227     unsigned i,k=0, n=fac.len;
01228     double x;
01229
01230     for (i=0; i<n; ++i) { if (fac[i]!=1) {
01231         if (fac[i]<1 || fac[i]!=round(fac[i])) wblog(FL,
01232             "ERR %s() got fac[%d]=%g !?",FCT,i+1,fac[i]);
01233         else {
01234             wblog(FL," * applying IROP factor 1/%g%s to op(%d)",
01235                 fac[i], rixs? "^2":"",i+1);
01236             ++k;
01237         }
01238     }}
01239     if (!k) return;
01240
01241     if (!Ap.isEmpty() || !An.isEmpty()) {
01242         if (n!=Ap.dim1 || n!=An.dim1) wblog(FL,
01243             "ERR %s() size mismatch (%d,%d/2*%d)",FCT,Ap.dim1,An.dim1,n);
01244
01245         for (i=0; i<n; ++i) {
01246             if (fac[i]!=1) { x=1/fac[i]; if (rixs) { x*=x; }
01247                 Ap.rowTimes(i,x);
01248                 An.rowTimes(i,x);
01249             }
01250         }
01251     }
01252
01253     if (!AP.isEmpty() || !AN.isEmpty()) {
01254         if (AP.SIZE.len!=3 || !AP.sameSize(AN) || AP.SIZE[1]!=n)
01255             wblog(FL,"ERR %s() size mismatch (%s; %s /%d) !?",
01256                 FCT, SSTR(AP), SSTR(AN), n);
01257
01258         TS *ap=AP.data, *an=AN.data;
01259         unsigned n0=AP.SIZE[0], m=n*AP.SIZE[2];
01260
01261         for (i=0; i<m; ++i, ap+=n0, an+=n0) { k=i%n;
01262             if (fac[k]>1) { x=1/fac[k]; if (rixs) { x*=x; }
01263                 Wb::timesRange(ap, x, n0);
01264                 Wb::timesRange(an, x, n0);
01265             }
01266         }
01267     }
01268 };
01269
01270 template <class TS>
01271 void TDSpectral<TS>::getSpecData(
01272     wvector<double> &om, wbararray<TS> &A,
01273     unsigned linfit
01274 ) const {
01275     unsigned i,r,s, d1=AP.dim(2), d2=AP.dim(3);
01276
01277     getOmega(om,linfit);

```

```

01278     A.init(2*N,d1,d2);
01279
01280     if (om.len!=2*N || !AP.sameSize(AN)) wblog(FL,
01281         "ERR severe dimension mismatch (%d,%d).",2*N,om.len);
01282
01283     for (r=0; r<d1; r++)
01284     for (s=0; s<d2; s++) {
01285         for (i=0; i<N; i++) {
01286             A(N-i-1,r,s) = AN(i,r,s);
01287             A(N+i ,r,s) = AP(i,r,s);
01288         }
01289     }
01290
01291     if (linfit) {
01292         for (r=0; r<d1; r++)
01293         for (s=0; s<d2; s++) {
01294             Wb::set2avg(A.ref(r,s)+N, linfit);
01295             Wb::set2avg(A.ref(r,s)+N-linfit, linfit);
01296         }
01297     }
01298 };
01299
01300 template <class TS>
01301 void TDSpectral<TS>::getSpecData(
01302     wbvector<double> &om, wbMatrix<TS> &A,
01303     unsigned linfit
01304 ){
01305
01306     unsigned i,r;
01307
01308     getOmega(om,linfit);
01309
01310     if (Ap.isEmpty()) mergeNRGData();
01311     A.init(Ap.dim1,2*N);
01312
01313     if (A.dim2!=om.len || !Ap.sameSize(An)) wblog(FL,
01314         "ERR Severe dimension mismatch (%d,%d).",A.dim2,om.len);
01315
01316     for (r=0; r<A.dim1; r++)
01317     for (i=0; i<N; i++) {
01318         A(r,N-i-1) = An(r,i);
01319         A(r,N+i ) = Ap(r,i);
01320     }
01321
01322     if (linfit) {
01323         for (r=0; r<A.dim1; r++) {
01324             Wb::set2avg(A.rec(r)+N, linfit);
01325             Wb::set2avg(A.rec(r)+N-linfit, linfit);
01326         }
01327     }
01328 }
01329
01330 template <class TS>
01331 void TDSpectral<TS>::Fourier(
01332     wbvector<double> &om,
01333     wbMatrix<wbcomplex> &ST,
01334     double alpha, double Lambda, char vflag
01335 ){
01336     unsigned k,it,ik,is,io,l=0,no, nt=tt.len, ns=AP.dim(2), nk=AP.dim(3);
01337
01338     wbararray<wbcomplex> TKS(nt,nk,ns), KST;
01339     wbMatrix<double> aTK(nt,nk);
01340     wbararray<TS> AR;
01341     double ek,oi,*at; TS kso,*ap;
01342     wbcomplex z,*st;
01343
01344     if (vflag) wblog(FL,
01345         "<i> %s (damping alpha=%g, Lambda=%g)",FCT,alpha,Lambda);
01346
01347     getSpecData(om,AR);
01348     ap=AR.data; no=om.len;
01349
01350     if (alpha==0) aTK.set(1); else
01351     for (ik=0; ik<nk; ++ik) {
01352         ek=pow(Lambda, -double(ik)/2.) * 0.5*(Lambda+1);
01353         for (it=0; it<nt; it++)
01354             aTK(it,ik)=exp(-fabs(alpha*ek*tt[it]));
01355     }
01356
01357     for (ik=0; ik<nk; ik++)
01358     for (is=0; is<ns; is++)
01359     for (io=0; io<no; io++, l++) if (ap[l]!=0.) { kso=ap[l]; oi=om[io];
01360         for (it=0; it<nt; it++)
01361             TKS(it,ik,is)+=kso*expi(oi*tt[it]);
01362     }
01363
01364     TKS.permute(KST,"2 3 1");

```

```

01365     ST.init(ns,nt);
01366
01367     for (is=0; is<ns; ++is)
01368     for (it=0; it<nt; ++it) {
01369         st=KST.ref(is,it); z=0;
01370         if (alpha==0)
01371             for (k=0; k<nk; ++k) z+=st[k];
01372         else for (at=aTK.rec(it), k=0; k<nk; ++k) z+=at[k]*st[k];
01373         ST(is,it)=z;
01374     }
01375
01376 };
01377
01378 template <class TS>
01379 void TDSpectral<TS>::Fourier(
01380     wbvector<double> &om, wbMatrix<TS> &A,
01381     const wbvector<double> &tt,
01382     wbMatrix<wbcomplex> &at,
01383     double alpha, char vflag
01384 ){
01385     if (Ap.isEmpty()) { mergeNRGData(); dispSumRule(); }
01386     getSpecData(om,A);
01387
01388     Wb::Fourier(om,A,tt,at,0,alpha,vflag);
01389 };
01390
01391 template <class TS>
01392 void Wb::Fourier(
01393     const wbvector<double> &om, const wbMatrix<TS> &A,
01394     const wbvector<double> &tt,
01395     wbMatrix<wbcomplex> &az,
01396     char domflag,
01397     double alpha,
01398     char vflag
01399 ){
01400     unsigned i,j,l,m,n,b1,b2, nom, nt=tt.len, D=512, stop=0;
01401     wbMatrix<wbcomplex> ez;
01402     wbcomplex z;
01403     double f,ot;
01404
01405     wbvector< wbMatrix<wbcomplex> > AZ;
01406     wbvector<double> dom(om.len);
01407
01408     m=A.dim1; nom=om.len;
01409
01410     if (nom!=A.dim2) wblog(FL,
01411         "ERR %s - size mismatch (%d/%d)",nom,A.dim2);
01412     if (!tt.len || !m || !nom) {
01413         wblog(FL,"WRN Empty data [%d %d %d] - return",tt.len,m,nom);
01414         return;
01415     }
01416     if (nom<2) wblog(FL,"ERR %s - omega too short (%d/2)",nom);
01417
01418     n=om.len-1;
01419     for (i=1; i<n; i++) dom[i]=0.5*(om[i+1]-om[i-1]);
01420     dom[0]=om[1]-om[0]; dom[n]=om[n]-om[n-1];
01421
01422     for (i=0; i<dom.len; i++) if (dom[i]<=0) { wblog(FL,
01423         "WRN omega not strictly ascending !?"); break; }
01424
01425     for (i=0; i<tt.len; i++) if (tt[i]<0) { wblog(FL,
01426         "WRN negative tt values !?"); break; }
01427
01428     if (domflag) {
01429         wbMatrix<double> B(A);
01430
01431         for (i=0; i<B.dim1; ++i)
01432         for (j=0; j<B.dim2; ++j) B(i,j)*=dom[j];
01433
01434         Wb::Fourier(om,B,tt,az,0,alpha,vflag);
01435         return;
01436     }
01437
01438     if (vflag) wblog(FL," * %s() spectral data "
01439         "(alpha=%.4g, %s)",FCT,alpha, domflag? "func":"disc");
01440
01441     AZ.initDef(nt<D ? 1 : nt/D);
01442
01443     for (l=b1=0; !stop; l++,b1+=D) {
01444         b2=b1+D-1; if (b2+D>nt) { b2=nt-1; stop=1; }
01445         n=b2-b1+1;
01446
01447         if (n*nom*sizeof(double)/(1<20)>900) {
01448             wblog(FL,"WRN trying to allocate %.6g MB !?",
01449                 n*nom*sizeof(double)/double(1<20));
01450         }
01451     }

```

```

01452     ez.init(nom,n);
01453     for (i=0; i<nom; i++)
01454         for (j=0; j<n; j++) { ot=om[i]*tt[b1+j];
01455             ez(i,j).init_expi(ot);
01456
01457             if (alpha>0) {
01458                 f=::exp(-::fabs(alpha*dom[i]*tt[b1+j]));
01459                 ez(i,j)*=f;
01460             }
01461         }
01462     Wb::MatProd(A,ez,AZ[l]);
01463 }
01464
01465 az.CAT(2,AZ);
01466 };
01467
01468 template <class TS>
01469 void TDSpectral<TS>::mergeNRGData() {
01470
01471     unsigned i,k, d1=AP.dim(1), d2=AP.dim(2), d3=AP.dim(3);
01472     unsigned s=d1*d2, S=s*d3;
01473     TS *Dp,*Dn,*dp,*dn;
01474
01475     if (!AP.sameSize(AN)) wblog(FL, "ERR severe size mismatch!");
01476
01477     AP.init(d2,d1); dp=AP.data; Dp=AP.data;
01478     AN.init(d2,d1); dn=AN.data; Dn=AN.data;
01479
01480     for (i=0; i<S; i++) { k=i%S;
01481         dp[k]+=Dp[i];
01482         dn[k]+=Dn[i];
01483     }
01484 };
01485
01486 template <class TS>
01487 void TDSpectral<TS>::dispSumRule() {
01488
01489     unsigned i=0; double m=0;
01490     wbvector<TS> sp,sn;
01491     wbararray<TS> x;
01492
01493     AP.sum("1,3",x); sp.init(x.numel(),x.data);
01494     AN.sum("1,3",x); sn.init(x.numel(),x.data);
01495
01496     wblog(FL," * om>0: %s\nom<0: %s",
01497         sp.toStrf("%8.5f").data, sn.toStrf("%8.5f").data);
01498
01499     sp+=sn;
01500
01501     if (sp.len) {
01502         m=ROUND(REAL(sp[0]));
01503         for (i=0; i<sp.len; i++) if (ABS(sp[i]-m)>1E-2) break;
01504     }
01505
01506     Wb::termcolor Tc(Wb::TCOLS::BLUE);
01507     if (i==sp.len) { sp-=TS(m); wblog(FL,
01508         " * %stotal %s%s (%g)",Tc.el,sp.toStrf("%8.2g").data,Tc.em,m); }
01509     else wblog(FL,
01510         " * %stotal %s%s", Tc.el,sp.toStrf("%8.5f").data,Tc.em);
01511 };
01512
01513 template <class TS>
01514 void Wb::dispSumRule(
01515     const wbvector<double> &om,
01516     const wbMatrix<TS> &A
01517 ){
01518     unsigned i; double m=0;
01519     wbvector<double> s(A.dim1);
01520     Wb::termcolor Tc(Wb::TCOLS::BLUE);
01521
01522     if (om.len!=A.dim2) wblog(FL,
01523         "ERR Dimension mismatch (%d,%d)", om.len, A.dim2);
01524
01525     for (unsigned i=0; i<A.dim1; i++)
01526         s[i] = Wb::IntTrapez(om.data, A.rec(i), om.len);
01527
01528     wblog(FL,"SUM %s", s.toStrf("%8.5f").data);
01529
01530     if (s.len) {
01531         for (m=ROUND(s[0]), i=1; i<s.len; ++i) {
01532             if (ABS(s[i]-m)>1E-2) { break; }
01533         }
01534         if (i==s.len) { s-=m;
01535             wblog(FL,"%s * total-%g = %s%s",
01536                 Tc.el,m,s.toStrf("%8.2g").data,Tc.em);
01537         }
01538     }
}

```

```

01539 }
01540
01541 template <class TS>
01542 void TDSpectral<TS>::getSmoothSpec_t(
01543     double sigma, double eps, char bswitch, char vflag
01544 ) {
01545     unsigned i,s;
01546     wbMatrix<double> Ap0(Ap), An0(An), wlg;
01547
01548     if (fac<=0 || sigma<=0 || N<4) wblog(FL,
01549         "ERR invalid paramters (fac=%g, sigma=%g, N=%d)", fac,sigma,N);
01550     if (fac*sigma<1) wblog(FL,
01551         "WRN sigma < discrete log spacing (%.3g / %.3g)", sigma,1/fac);
01552     if (vflag) wblog(FL,
01553         "<i> %s() %dx%d bins (sigma=%g)",myname,AP.dim(2),N,sigma);
01554
01555     if (Ap.isEmpty()) { mergeNRGData(); };
01556     if (vflag) { dispSumRule(); }
01557
01558     Ap.set(0); An.set(0);
01559     initLG(wlg,N,sigma,bswitch,vflag);
01560
01561     for (s=0; s<Ap.dim1; ++s)
01562     for (i=0; i<Ap.dim2; ++i) {
01563         foldLogGaussian(wlg, Ap.rec(s), Ap0(s,i), i);
01564         foldLogGaussian(wlg, An.rec(s), An0(s,i), i);
01565     }
01566
01567     if (eps) {
01568         if (vflag>1) {
01569             wblog(FL, " * %s() eps=%.3g/%.3g",myname,eps,exp(10)); }
01570
01571         unsigned j,l, i0=0, i1=0, i2=0;
01572         unsigned nE=(1<12);
01573
01574         double eps0=0.05*eps, eps1=1.5*eps, x1=5., xfac=x1/double(nE);
01575         double x=1./sqrt(M_PI), dbl, lfac=1/xfac;
01576
01577         wbvector<double> om(Ap.dim2), dom(om.len);
01578         wbvector<double> gauss_lin(2*nE+1), erf_om(om.len);
01579
01580         if (om.len<10) wblog(FL,"ERR %s() om.len = %d !?",FCT,om.len);
01581         for (i=0; i<om.len; ++i) {
01582             om[i]=idx2om(i);
01583
01584             if (!i0 && i && om[i]>eps0) { i0=i; }
01585
01586             if (!i1 && i && (om[i]-om[i-1])>eps1) { i1=i; }
01587
01588             if (i>1) {
01589                 dom[i-1] = 0.5*( om[i] - om[i-2] ); }
01590         }
01591         dom[i-1] = 2*dom[i-2];
01592         dom[0] = 0.5*( om[0] + om[1] );
01593
01594         if (i1) { if (vflag) {
01595             sprintf(str, "(%.1g <) |om| < %.1g (%.0f%%)",om[i0],om[i1],
01596                 100*((i1-i0)/double(om.len)));
01597             wblog(FL, "--> Gaussian broadening for %s",str);
01598         }}
01599         else {
01600             i1=om.len-1; sprintf(str, "(%.3g; eps=%.3g)",om[i1],eps);
01601             wblog(FL, "WRN Gaussian broadening for all omega %s",str);
01602         }
01603
01604         i2=i1+10; if (i2>=om.len) { i2=om.len-1; }
01605
01606         for (i=0; i<erf_om.len; ++i) {
01607             erf_om[i] = (0.5*( std::erf( om[i]/eps ) + 1 ));
01608         }
01609
01610         for (i=0; i<gauss_lin.len; ++i) {
01611             dbl = i*xfac - x1;
01612             gauss_lin[i] = x*exp(-dbl*dbl);
01613         }
01614         dom*=(1/eps);
01615
01616         Ap0=Ap; An0=An;
01617         for (s=0; s<Ap.dim1; ++s)
01618         for (i=0; i<i1; ++i) { Ap(s,i) = An(s,i) = 0; }
01619
01620         for (s=0; s<Ap.dim1; ++s) {
01621             for (x=0., i=0; i<i0; ++i) { x += (Ap0(s,i) + An0(s,i)); }
01622             for (i=0; i<i2; ++i) {
01623                 Ap(s,i) += ( dbl = x*(erf_om[i] - (i? erf_om[i-1] : 0.5)) );
01624                 An(s,i) += dbl;
01625             }
01626         }
01627     }

```

```

01626     }
01627
01628     for (s=0; s<Ap.dim1; ++s)
01629     for (i=i0; i<i1; ++i) {
01630         for (j=0; j<i2; ++j) {
01631             dbl = (om[j]-om[i])/eps;
01632             if (fabs(dbl)<x1 ) { l = (dbl + x1) * lfac + 0.5;
01633                 if (l<gauss_lin.len) {
01634                     Ap(s,j) += Ap0(s,i) * (dbl = gauss_lin[l] * dom[j]);
01635                     An(s,j) += An0(s,i) * dbl;
01636                 }}
01637
01638             dbl = (om[j]+om[i])/eps;
01639             if (fabs(dbl)<x1 ) { l = (dbl + x1) * lfac + 0.5;
01640                 if (l<gauss_lin.len) {
01641                     An(s,j) += Ap0(s,i) * (dbl = gauss_lin[l] * dom[j]);
01642                     Ap(s,j) += An0(s,i) * dbl;
01643                 }}
01644         }
01645     }
01646 }
01647
01648 };
01649
01650 template <class TS>
01651 void TDSpectral<TS>::initLG(
01652     wbMatrix<double> &wlg,
01653     unsigned n, double sigma, char bswitch, char vflag
01654 ){
01655     unsigned i,j;
01656     double dbl,efac,s=0;
01657
01658     if (sigma<=0. || fac<=0.) wblog(FL,"ERR sigma=%g, fac=%g",sigma,fac);
01659
01660     efac=1./(fac*sigma); wlg.init(2,n);
01661
01662     switch (bswitch) {
01663     case 0:
01664         strcpy(str,"symmetric in log. space");
01665         s=0; break;
01666     case 1:
01667         strcpy(str,"symmetric log-gauss");
01668         s=sigma/2.;
01669         break;
01670     case 2:
01671         strcpy(str,"norm preserving log-gauss");
01672         s=sigma/4.;
01673         break;
01674     default:
01675         wblog(FL,"ERR invalid bswitch=%d",bswitch);
01676     }
01677     if (vflag) wblog(FL,"--> %s() using %s",FCT,str);
01678
01679     for (i=0; i<n; i++) {
01680         dbl=efac*double(i)+s; wlg(0,i)=exp(-dbl*dbl);
01681         dbl=efac*double(i)-s; wlg(1,i)=exp(-dbl*dbl);
01682     }
01683
01684     wlg*=(1./wlg.sum());
01685
01686     for (i=0; i<2; ++i)
01687     for (j=0; j<n; ++j) if (fabs(wlg(i,j))<1E-20) wlg(i,j)=0;
01688 };
01689
01690 template <class TS> inline
01691 void TDSpectral<TS>::foldLogGaussian(
01692     const wbMatrix<double> &wlg,
01693     TS *aa, TS a0, unsigned i0
01694 ){
01695     unsigned i, n=wlg.dim2;
01696     const double *wp=wlg.rec(0), *wn=wlg.rec(1);
01697     double wi, nrm=0.;
01698
01699     for (i=0; i<n; ++i) {
01700         nrm += (i0>i ? wp[i0-i] : wn[i-i0]); }
01701     nrm=a0/nrm;
01702
01703     for (i=0; i<n; ++i) {
01704         wi=( i0>i ? wp[i0-i] : wn[i-i0] );
01705         if (wi) aa[i]+=nrm*wi;
01706     }
01707 };
01708
01709 template <class TS>
01710 mxArray* TDSpectral<TS>::toMx() const {
01711     mxArray *S=mxCreateStructMatrix(1,1,0,NULL);
01712

```

```

01713     mxAddField2Scalar(FL,S,"name",istr.toMx());
01714     mxAddField2Scalar(FL,S,"AP", AP.toMx());
01715     mxAddField2Scalar(FL,S,"AN", AN.toMx());
01716     mxAddField2Scalar(FL,S,"fac",numtoMx(fac));
01717     mxAddField2Scalar(FL,S,"l0", numtoMx(l0 ));
01718
01719     if (raw) {
01720         mxAddField2Scalar(FL,S,"RAW",RAW.toMx()); }
01721
01722     return S;
01723 };
01724
01725 template <class TS>
01726 int TDSpectral<TS>::init(const mxArray *S,
01727     double &emin, double &emax, unsigned &nlog
01728 ){
01729     if (!mxIsStruct(S) || !Mx::IsScalar(S) ||
01730         mxGetFieldNumber(S,"name")<0 ||
01731         mxGetFieldNumber(S,"AP" )<0 ||
01732         mxGetFieldNumber(S,"AN" )<0 ||
01733         mxGetFieldNumber(S,"fac" )<0 ||
01734         mxGetFieldNumber(S,"l0" )<0
01735     ) return 1;
01736
01737     istr.init(mxGetField(S,0,"name" ));
01738     AP .init(mxGetField(S,0,"AP"));
01739     AN .init(mxGetField(S,0,"AN"));
01740     mxGetNumber(mxGetField(S,0,"fac"),fac);
01741     mxGetNumber(mxGetField(S,0,"l0"), l0 );
01742
01743     if (AP.SIZE!=AN.SIZE || AP.isEmpty() || AN.isEmpty()) return 2;
01744     if (fac<=0) return 3;
01745
01746     N=AP.dim(1);
01747
01748     emin=exp(l0);
01749     emax=idx2om(N);
01750     nlog=(unsigned)(fac/log10(M_E) + 0.5);
01751
01752     wblog(FL,
01753         "<i> TDSpectral `s' from RAW data: %dx%d bins [%d]",
01754         istr.data, AP.dim(1), AP.dim(2), AP.dim(3));
01755
01756     return 0;
01757 };
01758
01759 #endif

```

5.21 util.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace utility routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_UTIL_CC__
00021 #define __WB_UTIL_CC__
00022
00023 /* ----- */
00024 // NB! this requires
00025 //     fabs() => [std]::fabs
00026 //     sqrt() => [std]::sqrt(), etc.
00027 // since Wb::fabs() is only defined for MPFR (!)
00028 // and so compiler complains that MPFR routine got double // Wb,Jul05,22
00029 /* ----- */
00030
00031 int Wb::Rational(
00032     double &x0,
00033     long &P, long &Q,

```

```

00034     double *dx,
00035     unsigned *nmax,
00036     wvector<double>* aa,
00037     unsigned niter,
00038     long pqmax,
00039
00040     double eps,
00041
00042     double eps2,
00043
00044     char vflag
00045 ){
00046     unsigned i; int err=0;
00047     long p[niter+2], q[niter+2];
00048     double e,xi,x=0, x=x0, a[niter+1];
00049
00050     if (eps<eps2 || eps2<=0) wblog(FL,
00051         "ERR %s() invalid eps=%g, eps2=%g",FCT,eps,eps2);
00052
00053     if (pqmax<=0) { pqmax=long(MIN(1E12, 1/eps)); }
00054     if (pqmax<=0 || pqmax>(1L<<50)) {
00055         wblog(FL,"ERR %s() pqmax=%ld !?",FCT,pqmax);
00056     }
00057
00058     if (::fabs(x0)>pqmax || (x!=0 && ::fabs(1/x0)>pqmax)) {
00059         if (vflag) wblog(FL,"WRN %s() double out of bounds long int",FCT);
00060         if (dx) { (*dx)=0; }
00061         if (nmax) { (*nmax)=0; }; if (aa) { aa->init(); }
00062         return -1;
00063     }
00064
00065     p[0]=1; p[1]=a[0]=xi::round(x);
00066     q[0]=0; q[1]=1;
00067
00068     if (vflag) printf(
00069         "\n %s() finding rational approximation for %.16g ... \n"
00070         "\n%6d: %4g | %8ld / %8ld\n",FCT,x, 0,a[0],p[1],q[1]);
00071
00072     if (::fabs(xi-x)<eps) {
00073         P=p[1]; Q=q[1];
00074         if (dx) { (*dx)=::fabs(xi-x); }
00075         if (nmax) { (*nmax)=0; }; if (aa) { aa->init(1,a); }
00076         x0=xi; return 0;
00077     }
00078
00079     if (::fabs(x)>1) {
00080         eps *=::fabs(x);
00081     }
00082
00083     r=::fabs(xi-x0);
00084
00085     for (i=0; i<niter; ++i) {
00086         if ((e=::fabs(x-a[i]))<eps || r<eps2) break;
00087
00088         x=1./(x-a[i]); a[i+1]=::round(x);
00089         p[i+2]=a[i+1]*p[i+1]+p[i];
00090         q[i+2]=a[i+1]*q[i+1]+q[i];
00091
00092         xi=double(p[i+2])/q[i+2];
00093         r=::fabs(xi-x0);
00094
00095         if (vflag) {
00096             printf("%6d: %4g | %8ld / %8ld = %20.16g @ %8.3g (%.3g)\n",
00097                 i+1, a[i+1], p[i+2],q[i+2], xi,r, x-a[i+1]);
00098         }
00099         if (labs(p[i+2])>pqmax || labs(q[i+2])>pqmax) {
00100             ++i; err=1; break;
00101         }
00102     }
00103     P=p[i+1]; Q=q[i+1]; if (Q<0) { P=-P; Q=-Q; }
00104     if (i>=niter) err=2;
00105
00106     if (vflag) {
00107         if (i<niter) printf("\n "
00108             "Rational representation: %ld/%ld\n",P,Q);
00109         else printf("\n "
00110             "Failed to find rational representation (%d @ %g)\n\n",
00111             niter,eps
00112         );
00113     }
00114
00115     if (dx) (*dx)=r;
00116     if (aa) aa->init(i,a);
00117     if (nmax) (*nmax)=i;
00118
00119     if (!err) x0=xi;
00120     return err;

```



```

00121 };
00122
00123 template<class T>
00124 double Wb::FixRational(
00125     const char *F __attribute__((unused)),
00126     int L __attribute__((unused)),
00127     T *d __attribute__((unused)),
00128     unsigned n __attribute__((unused)),
00129     char rflag __attribute__((unused)),
00130     unsigned niter __attribute__((unused)),
00131     long pqmax __attribute__((unused)),
00132     double eps __attribute__((unused)),
00133     double eps2 __attribute__((unused)),
00134     double *rz __attribute__((unused)),
00135     double *ra __attribute__((unused)),
00136     char vflag __attribute__((unused))
00137 ) {
00138     return 0;
00139 };
00140
00141 template<>
00142 double Wb::FixRational(
00143     const char *F, int L, double *A, unsigned n,
00144     char rflag, unsigned niter,
00145     long pqmax,
00146     double eps,
00147     double eps2,
00148     double *r2z,
00149     double *r2m,
00150     char vflag
00151 ) {
00152     unsigned i,m,m2, nmax=0, nmissd=0; int e1,e2; long p,q, p2,q2;
00153     double x,Ai,Ai2,a2,d2,a=0,r=0,r2=0,r2max=0,z2=0,err2=0, epsi=eps;
00154
00155     for (i=0; i<n; ++i) {
00156         x=::fabs(A[i]); if (a<x) { a=x; }
00157
00158         x=::fabs(A[i]-::round(A[i])); if (r<x) { r=x; }
00159         if (x<eps) { d2=(x*x);
00160             A[i]=::round(A[i]);
00161             if (A[i]) { err2+=d2; } else { z2+=d2; }
00162             if (r2max<d2) { r2max=d2; }
00163         }
00164     }
00165     if (r2z) { (*r2z)=z2; } else { err2+=z2; }
00166
00167     if (r<eps || int(niter)<1) {
00168         if (r2m) { (*r2m)=r2max; }
00169         return err2;
00170     }
00171     if (a>1) { epsi*=a; }
00172
00173     for (i=0; i<n; ++i) { a=Ai=A[i]; e2=-1;
00174         e1=Rational(a,p,q,&r,&m,NULL,niter,pqmax,eps,eps2,vflag>1);
00175
00176         if (rflag && (e1 || abs(p)>999 || abs(q)>999)) {
00177             if ( (a2=Ai2=Ai*Ai) > eps) {
00178                 e2=Rational(a2,p2,q2,&r2,&m2,NULL,niter,pqmax,
00179                     a2<0.01 ? eps : ::fabs(Ai) : eps,
00180                     a2<0.01 ? eps2 : ::fabs(Ai) : eps2, vflag>1);
00181             }
00182         }
00183
00184         if (!e1 && !e2) {
00185             double q1=::fabs(A[i]*A[i]-Ai2), q2=::fabs(a2-Ai2);
00186             if (q1>q2) { if (vflag>1) {
00187                 PRINTF(" ==> picking sqrt() over plain (e=%3g / %3g)\n",
00188                     ::sqrt(q2), ::sqrt(q1)); }
00189                 e1=-2;
00190             }
00191         }
00192
00193         if (!e1) {
00194             if (nmax<m) { nmax=m; }
00195             if (r>epsi) { if (q<0) { p=-p; q=-q; }; wblog(F_L,
00196                 "WRN fixing number %.16g = %ld/%ld (%d; %3g)",Ai,p,q,m,r);
00197             }
00198             A[i]=a;
00199             d2=r*r; err2+=d2; if (r2max<d2) { r2max=d2; }
00200             continue;
00201         }
00202         if (e2) {
00203             nmissd++; continue;
00204         }
00205
00206         if (nmax<m2) { nmax=m2; }
00207         a=::sqrt(a2); if (Ai<0) { a=-a; }

```

```

00208         r:::fabs(a-Ai);
00209
00210         if (r>epsi) {
00211             wblog(F_L,"WRN fixing %.16g = %ssqrt(%ld/%ld) (%.2g)",
00212                 A[i], A[i]<0 ? "-":"", p2,q2,r);
00213             wblog(FL,"TST %s() pqmax=%ld, eps=%g, eps2=%g,niter=%d",
00214                 FCT,pqmax,eps,eps2,niter);
00215             double a2=Ai*Ai;
00216             Rational(a2,p2,q2,&r2,&m2,NULL,niter,pqmax,eps,eps2,'v');
00217         }
00218         d2=r*r; err2+=d2; if (r2max<d2) { r2max=d2; }
00219         A[i]=a;
00220     }
00221
00222     if (r2m) { (*r2m)=r2max; }
00223
00224     if (nmissd && vflag) wblog(F_L,
00225         "WRN %s() %d/%d values unaltered (%d)\r\\",FCT,nmissd,n,nmax);
00226
00227     return err2;
00228 };
00229
00230 wbsting Wb::rat2Str(
00231     const char *F, int L, double d, unsigned niter,
00232     long pqmax,
00233     double eps,
00234     double eps2,
00235     char vflag
00236 ){
00237     wbsting sout(32);
00238     long p;
00239
00240     if (::fabs(d-(p=:round(d)))<eps) {
00241         if (sout.printf(FL,"%.16g",d)>8) {
00242             sout.printf(FL,"%s%ld", p || d>=0 ? "":"-", p);
00243         }; return sout;
00244     }
00245
00246     if (int(niter)<1) {
00247         sout.printf(FL,"%.5g",d);
00248         return sout;
00249     }
00250
00251     int e1, e2=-1; unsigned m,m2; long q, p2,q2;
00252     double r,r2, d_ =d, d2, d2_, a=:fabs(d); if (a>1) { eps*=a; }
00253
00254     e1=Rational(d,p,q,&r,&m,NULL,niter,pqmax,eps,eps2,vflag>1);
00255
00256     if (e1 || abs(p)>999 || abs(q)>999) {
00257         if ((d2_=d_+d_)>eps) { d2=d2_;
00258             e2=Rational(d2,p2,q2,&r2,&m2,NULL,niter,pqmax,
00259                 d2<0.01 ? eps *a : eps,
00260                 d2<0.01 ? eps2*a : eps2, vflag>1
00261             );
00262         }
00263     }
00264
00265     if (!e1 && !e2) {
00266         double q1=:fabs(d*d-d2_), q2=:fabs(d2-d2_);
00267         if (q1>q2) { if (vflag>1) {
00268             PRINTF(" ==> picking sqrt() over plain (e=%.3g / %.3g)\n",
00269                 ::sqrt(q2), ::sqrt(q1)); }
00270             e1=-2;
00271         }
00272     }
00273
00274     if (!e1) {
00275         sout.printf(FL,"%ld/%ld",p,q);
00276         return sout;
00277     }
00278     if (e2) {
00279         sout.printf(FL,"%.5g",d);
00280         return sout;
00281     }
00282
00283     wbsting sx(24);
00284
00285     char sgn[]="-"; if (d>=0) { sgn[0]=0; }
00286     sout.data[0]=0;
00287
00288     if (q2==1) {
00289         sout.printf(FL,"%s%s", sgn,Wb::surdStrf(sx,"%ld",p2));
00290     }
00291     else if (p2==1) {
00292         sout.printf(FL,"%s1/%s",sgn,Wb::surdStrf(sx,"%ld",q2));
00293     }
00294     else { r=:sqrt(double(q2));
00295         if (::fabs(r-:round(r))<eps) {

```

```

00295         sout.printf(FL, "%s%s/%g", sgn, Wb::surdStrf(sx, "%ld", p2), r);
00296     }
00297 }
00298 if (!sout.data[0]) { r::sqrt(double(p2));
00299     if (::fabs(r-::round(r))<eps)
00300         { sout.printf(FL, "%s%g/%s", sgn, r, Wb::surdStrf(sx, "%ld", q2)); }
00301     else { sout.printf(FL, "%s%s", sgn, Wb::surdStrf(sx, "(%ld/%ld)", p2, q2)); }
00302 }
00303 return sout;
00304 };
00305
00306 template<>
00307 double Wb::SkipZeros(double *d, unsigned n, double eps){
00308     unsigned i=0; double s2=0;
00309     for (; i<n; ++i) {
00310         if (::fabs(d[i])<eps) { s2+=(d[i]*d[i]); d[i]=0; }
00311     }
00312     return s2;
00313 };
00314
00315 template<>
00316 double Wb::SkipZeros(long double *d, unsigned n, double eps){
00317     unsigned i=0; double s2=0;
00318     for (; i<n; ++i) {
00319         if (::fabs(d[i])<eps) { s2+=(d[i]*d[i]); d[i]=0; }
00320     }
00321     return s2;
00322 };
00323
00324 template<>
00325 double Wb::SkipZeros(wbcomplex *z, unsigned n, double eps){
00326     unsigned i=0; double s2=0;
00327     for (; i<n; ++i) {
00328         if (::fabs(z[i].r)<eps) { s2+=(z[i].r*z[i].r); z[i].r=0; }
00329         if (::fabs(z[i].i)<eps) { s2+=(z[i].i*z[i].i); z[i].i=0; }
00330     }
00331     return s2;
00332 };
00333
00334 inline wbstring Wb::size2Str(
00335     double n,
00336     char p
00337 ){
00338     if (!n) { return "0"; }
00339     unsigned l=0, m=16; size_t x;
00340     char s[m], ustr[8], fmt[m];
00341     x=size_t(1)<<30; if (n>x) { strcpy(ustr, "GB"); } else {
00342     x=size_t(1)<<20; if (n>x) { strcpy(ustr, "MB"); } else {
00343     x=size_t(1)<<10; if (n>x) { strcpy(ustr, "kB"); } else {
00344     x=size_t(1);
00345         strcpy(ustr, "bytes"); }}}
00346     if (p<1 || p>9) {
00347         if (p=='c') { p=2; }
00348         else p=3;
00349     }
00350     if (p>5) { snprintf(fmt, 16, "%%.%dg %s", p); }
00351     else { snprintf(fmt, 16, "%%.%dg%1.1s", p); }
00352     l=snprintf(s, m, fmt, n/x, ustr);
00353     if (l>=m) wblog(FL,
00354         "ERR %s() string out of bounds (%d/%d)", FCT, l, m);
00355     return s;
00356 };
00357
00358 inline const char* Wb::filename(const char *s) {
00359     if (s==NULL) { wblog(FL, "WRN %s() got NULL string", FCT); }
00360     else {
00361         for (int i=strlen(s)-1; i>=0; --i) {
00362             if (s[i]=='/' || s[i]=='\\') return (s+i+1); }
00363     }
00364     return s;
00365 };
00366
00367 size_t Wb::getFileSize(const char* f) {
00368     struct stat S;
00369     if (!f || !f[0] || stat(f, &S)!=0) { return 0; }
00370     return S.st_size;

```

```

00382 };
00383
00384 #ifdef __APPLE__
00385
00386 double Wb::GET_mtime(const char *F, int L, const char *f, const struct stat &S) {
00387
00388     if (S.st_mtimespec.tv_nsec>1E9) wblog(F,L,
00389         "WRN %s got mtime %ld @ %.3g", f?f:"", f?"()":"",
00390         S.st_mtimespec.tv_sec, 1E-9*S.st_mtimespec.tv_nsec
00391     );
00392     return (S.st_mtimespec.tv_sec + 1E-9*double(S.st_mtimespec.tv_nsec));
00393 };
00394
00395 #else
00396
00397 double Wb::GET_mtime(const char *F, int L, const char *f, const struct stat &S) {
00398
00399     if (S.st_mtim.tv_sec!=S.st_mtime || S.st_mtim.tv_nsec>1E9)
00400         wblog(F,L,"WRN %s() got difference in mtime (%ld @ %.3g)",
00401             f?f:"",S.st_mtim.tv_sec, S.st_mtime, 1E-9*S.st_mtim.tv_nsec
00402         );
00403     return S.st_mtim.tv_sec + 1E-9*double(S.st_mtim.tv_nsec);
00404 };
00405
00406 #endif
00407
00408 #ifdef __APPLE__
00409
00410 double Wb::GET_atime(const char *F, int L, const char *f, const struct stat &S) {
00411
00412     if (S.st_atimespec.tv_nsec>1E9) wblog(F,L,
00413         "WRN %s got atime %ld @ %.3g", f?f:"", f?"()":"",
00414         S.st_atimespec.tv_sec, 1E-9*S.st_atimespec.tv_nsec
00415     );
00416     return S.st_atimespec.tv_sec + 1E-9*double(S.st_atimespec.tv_nsec);
00417 };
00418
00419 #else
00420
00421 double Wb::GET_atime(const char *F, int L, const char *f, const struct stat &S) {
00422
00423     if (S.st_atim.tv_sec!=S.st_atime || S.st_atim.tv_nsec>1E9)
00424         wblog(F,L,"WRN %s() got difference in atime (%ld @ %.3g)",
00425             f?f:"",S.st_atim.tv_sec, S.st_atime, 1E-9*S.st_atim.tv_nsec
00426         );
00427     return S.st_atim.tv_sec + 1E-9*double(S.st_atim.tv_nsec);
00428 };
00429
00430 #endif
00431
00432 #ifdef __APPLE__
00433
00434 double Wb::GET_ctime(const char *F, int L, const char *f, const struct stat &S) {
00435
00436     if (S.st_ctimespec.tv_nsec>1E9) wblog(F,L,
00437         "WRN %s got ctime %ld @ %.3g", f?f:"", f?"()":"",
00438         S.st_ctimespec.tv_sec, 1E-9*S.st_ctimespec.tv_nsec
00439     );
00440     return S.st_ctimespec.tv_sec + 1E-9*double(S.st_ctimespec.tv_nsec);
00441 };
00442
00443 #else
00444
00445 double Wb::GET_ctime(const char *F, int L, const char *f, const struct stat &S) {
00446
00447     if (S.st_ctim.tv_sec!=S.st_ctime || S.st_ctim.tv_nsec>1E9)
00448         wblog(F,L,"WRN %s() got difference in ctime (%ld @ %.3g)",
00449             f?f:"",S.st_ctim.tv_sec, S.st_ctime, 1E-9*S.st_ctim.tv_nsec
00450         );
00451     return S.st_ctim.tv_sec + 1E-9*double(S.st_ctim.tv_nsec);
00452 };
00453
00454 #endif
00455
00456 double Wb::getFileInfo(const char* f, char w) {
00457
00458     struct stat S;
00459     if (!f || !f[0] || stat(f, &S)!=0) { return 0; }
00460
00461     switch (w) {
00462     case 'm': return GET_mtime(F,f,S);
00463     case 'a': return GET_atime(F,f,S);
00464     case 'c': return GET_ctime(F,f,S);
00465
00466     case 'M': return S.st_mtime;
00467     case 'A': return S.st_atime;
00468     case 'C': return S.st_ctime;

```

```

00469
00470     default: wblog(FL,"ERR %s() invalid w=%c<%d>",FCT,w,w);
00471 }
00472
00473     return 0;
00474 };
00475
00476 bool Wb::fexist(const char *f, char type) {
00477
00478     if (!f || !f[0]) wblog(FL,"ERR %s() invalid (empty) file name",FCT);
00479
00480     struct stat S;
00481     if (stat(f, &S)!=0) { return 0; }
00482
00483     if (type=='f') return (S.st_mode & S_IFREG); else
00484     if (type=='d') return (S.st_mode & S_IFDIR); else
00485     return 1;
00486 };
00487
00488 bool Wb::isFile(const char *f) {
00489     if (f && f[0]) {
00490         struct stat S;
00491         if (stat(f,&S)==0) { return (S.st_mode & S_IFREG); }
00492     }
00493     return 0;
00494 };
00495
00496 bool Wb::isDir(const char *f) {
00497     if (f && f[0]) {
00498         struct stat S;
00499         if (stat(f,&S)==0) { return (S.st_mode & S_IFDIR); }
00500     }
00501     return 0;
00502 };
00503
00504 wstring Wb::hostname(unsigned len) {
00505
00506     wstring s(MAX(16U,len));
00507     int e=gethostname(s.data,s.len);
00508
00509     if (e) { unsigned i=0;
00510         if (errno==ENAMETOOLONG) {
00511             for (; i<s.len; ++i) {
00512                 if (s.data[i]=='.') { s.data[i]=0; break; }
00513             }
00514         }
00515         else {
00516             for (; i<s.len; ++i) {
00517                 if (!isprint(s.data[i])) { s.data[i]=0; break; }
00518             }
00519             if (i<4) { strcpy(s.data,"(host!)"); } else
00520             if (i>=s.len) {
00521                 s.data[s.len-1]=0;
00522             }
00523         }
00524         wblog(FL,"WRN %s() received e=%d (%s)",FCT,errno,s.data);
00525     }
00526     return s;
00527 };
00528
00529 wstring Wb::hostid(char pflag) {
00530
00531     wstring hid=hostname(pflag ? 24 : 16);
00532
00533     unsigned i=0, k=0, n=hid.len; char *s=hid.data;
00534     if (!n || !s[0] || !isalpha(s[0])) {
00535         wblog(FL,"WRN %s() received hostname `%s'",FCT,s);
00536         return hid;
00537     }
00538
00539     for (; i<n; ++i) {
00540         if (s[i]=='-') { k=i+1; } else
00541         if (s[i]=='.') { if (i>4 && k+3<i) { s[i]=0; n=i; break; } } else
00542         if (!s[i]) { n=i; break; }
00543     }
00544     if (i<n) { n=i; } else
00545     if (i==hid.len) { s[i-1]=0;
00546         wblog(FL,"WRN %s() received `%s'",FCT,s);
00547         return hid;
00548     }
00549     if (k && k+1<n) {
00550         for (i=k; i<n; ++i) { s[i-k]=s[i]; }
00551         s[i-k]=0; i-=k;
00552     }
00553
00554     if (pflag && i+2<hid.len) { char fmt[]="/%d";
00555         if (pflag=='x' || pflag=='X') { fmt[2]=pflag; }

```

```

00556     snprintf(s+i, hid.len-i, fmt, getpid());
00557 }
00558 return hid;
00559 };
00560
00561 template <class TI> inline
00562 TI Wb::sub2ind(const TI *s, const TI *I, unsigned n){
00563     if (n) {
00564         unsigned j=n-1; TI k=I[j];
00565         for (--j; j<n; --j) k=k*s[j]+I[j];
00566         return k;
00567     }
00568     else { wblog(FL, "ERR %s() got null set !?", FCT); return -1; }
00569 };
00570
00571 template <class T1, class T2> inline
00572 void Wb::ind2sub(T1 k, const T1 *s, T2 *I, unsigned n) {
00573
00574     unsigned x,i=0;
00575     for (; i<n; ++i) {
00576         if (s[i]) { x=k/s[i]; I[i]=k-x*s[i]; k=x; }
00577         else {
00578             wblog(FL, "ERR %s() got null-size %s", FCT,
00579                 SSTR(wbvector<T1>(n,s,'r')));
00580         }
00581     }
00582     if (k) { wblog(FL,
00583         "ERR %s() index out of bounds\n(%d => [%s; %s])", FCT,
00584         (wbvector<T2>(n,I,'r')+1).toStr().data,
00585         SSTR(wbvector<T1>(n,s,'r')));
00586     }
00587 };
00588
00589 int Wb::atoi(const char *s, int &iout,
00590     unsigned n,
00591     char white
00592 ){
00593     if (!s || !s[0]) { return -1; }
00594     unsigned i=0; if (int(n)<=0) n=9;
00595
00596     if (white) {
00597         for (; s[i]; ++i) { if (s[i]!=' ') break; }
00598         if (i) {
00599             if (!s[i]) { return -2; }
00600             s+=i; i=0;
00601         }
00602     }
00603
00604     if (white<0) {
00605         if (s[0]=='+' || s[0]=='-') { if (!s[++i]) return -3; }
00606     }
00607
00608     for (; s[i] && i<n; ++i) {
00609         if (!isdigit(s[i])) return -4;
00610     }
00611     if (i==n && isdigit(s[i])) return -5;
00612
00613     iout=::atoi(s);
00614     return i;
00615 };
00616
00617 int Wb::isUIntString(const char *s, char white) {
00618     if (!s) { return -1; }
00619
00620     if (white) {
00621         unsigned i=0, n=0, m=0;
00622
00623         for (; s[i]; ++i) {
00624             if (isdigit(s[i])) ++m; else
00625             if (isspace(s[i])) { if (m) { ++n; m=0; }}
00626             else return -(100+n);
00627         }
00628         return (m ? n+1 : n);
00629     }
00630     else {
00631         unsigned i=0, n=0;
00632
00633         while (isspace(*s)) ++s;
00634         while (isdigit(s[n])) ++n;
00635         for (i=n; isspace(s[i]); ++i) {};
00636
00637         if (!s[i])
00638             return n;
00639         else return -(10+n);
00640     }
00641 };
00642

```

```

00643 int Wb::dstrlen_utf8(const char *s) {
00644     int q=0;
00645     if (!s || !s[0]) { return q; }
00646
00647     for (int i=0, l=strlen(s); i<l && s[i]; ++i) {
00648         if (s[i]<0 || s[i]>127) {
00649             if ((s[i] & 0xE0) == 0xC0) { q+=1; } else
00650             if ((s[i] & 0xF0) == 0xE0) { q+=2; } else
00651             if ((s[i] & 0xF8) == 0xF0) { q+=3; } else
00652             { q=0; break; }
00653         }
00654     }
00655     return q;
00656 };
00657
00658 template <class T>
00659 wstring Wb::bits(const T &x, char compact) {
00660     wstring s_;
00661     int i, j=0, n=sizeof(T), l=9*n; { s_.init(l); if (!l) return s_; }
00662     char *s; unsigned char c;
00663     const char *sx = (char*)(&x);
00664
00665     if (compact) {
00666         for (i=0; i<n; ++i) { if (sx[i]) j=i; }
00667         n=j+1; l=9*n;
00668     }
00669     s=s_.data+l-1; s[0]=0;
00670
00671     for (i=0; i<n; ++i, s-=9) { c=sx[i];
00672         for (j=1; j<=8; ++j, c>=1) { s[-j]=(c&1 ? '1' : '-'); }
00673         if (i+1<n) s[-j]=' ';
00674     }
00675
00676     return s_;
00677 };
00678
00679 void Wb::IOstat::print(const char *istr,
00680     const char *hstr, const char *xstr1, const char *xstr2
00681 ) {
00682     if (hstr && hstr[0]) printf("\n I/O stats for %-9s "
00683         "#read%-6s size_read  #write%-5s size_write\n",
00684         hstr, xstr1?xstr1:"", xstr2?xstr2:"");
00685
00686     char s[64];
00687     snprintf(s, 30, "%4ld", aux1);
00688     snprintf(s+30, 30, "%4ld", aux2);
00689
00690     printf("      %-19s %6d %4s %10s %6d %4s %10s\n", istr ? istr:"",
00691         nread, aux1 ? s : "", size2Str(sread).data,
00692         nwrite, aux2 ? s+30 : "", size2Str(swrite).data
00693     );
00694 };
00695
00696 bool Wb::is_finite(const wcomplex *x, size_t n) {
00697     for (size_t i=0; i<n; ++i) { if (!x[i].isfinite()) { return 0; } }
00698     return 1;
00699 };
00700
00701 wstring cpu_time::toStr(char flag) {
00702
00703     size_t l=0, n=32; char tstr[n];
00704
00705     cpu_time dt=since();
00706     double t=(flag=='c' ? dt.tcpu : dt.tsys);
00707
00708     if (t<0) {
00709         wblog(FL, "WRN %s() got negative time (%ld)", FCT, long(t)); t=-t; }
00710
00711     if (t<60) {
00712         l=snprintf(tstr, n, "%.4g sec", t); }
00713     else if (t<1E14) {
00714         int s,m,h,d; long x=t,
00715         fac=24*3600; d=t/fac; x-=d*fac;
00716         fac=3600; h=x/fac; x-=h*fac;
00717         fac=60; m=x/fac; x-=m*fac; s=x; x-=s;
00718
00719         if (d)
00720             l=snprintf(tstr, n, "%d-%02d:%02d:%02d", d, h, m, s);
00721         else if (h || m>9)
00722             l=snprintf(tstr, n, "%02d:%02d:%02d", h, m, s);
00723         else l=snprintf(tstr, n, "%02d:%02d", m, s);
00724
00725         if (t>2E9) l=-1;
00726     }
00727     else {
00728         l=snprintf(tstr, n, "(WRN %.3g yrs (%ld sec))",
00729             t/(3600*24*365), long(t));

```

```

00730     l=-1;
00731 }
00732
00733 if (l>=n) {
00734     long tl=(flag=='c' ? dt.tcpu : dt.tsys);
00735     double yrs=double(tl)/(3600*24*365);
00736     wblog(FL,
00737         "WRN cpu_time() %s '%s'\n%c' dt=%ld => %.12g yrs (%d) !?\n"
00738         "%15.12g %15.12g %% now [CPU/SYS]\n"
00739         "%15.12g %15.12g %% then",
00740         int(l)<0 ? "possibly invalid CPU time":"string out of bounds\n",
00741         tstr, flag,tl,yrs,l, dt.time('c'), dt.time('s'), tcpu, tsys
00742     );
00743 }
00744
00745 return tstr;
00746 };
00747
00748 #ifdef __APPLE__
00749
00750 double Mac::getMemSize(const char *tag) {
00751
00752     mach_msg_type_number_t count = HOST_VM_INFO_COUNT;
00753     vm_statistics_data_t S;
00754
00755     if (host_statistics(
00756         mach_host_self(),
00757         HOST_VM_INFO,
00758         (host_info_t)&S,
00759         &count
00760     )) wblog(FL, "ERR %s() %s", FCT, strerror(errno));
00761
00762     long n = getpagesize(),
00763          sused = S.wire_count + S.active_count,
00764          sfree = S.inactive_count + S.free_count;
00765
00766     if (!tag || !tag[0]) wblog(FL, "ERR %s() got empty tag !?", FCT);
00767
00768     if (!strcmp(tag, "total")) { return n*(sused+sfree); }
00769     if (!strcmp(tag, "free" )) { return n*(sfree); }
00770     if (!strcmp(tag, "usage")) {
00771         return 100*double(sused)/(sfree+sused);
00772     }
00773
00774     wblog(FL, "ERR %s() tag '%s' not yet defined", FCT, tag);
00775     return 0;
00776 };
00777
00778 size_t Mac::getProcMemSize(pid_t p, char res) {
00779
00780     struct task_basic_info S;
00781     mach_msg_type_number_t len = TASK_BASIC_INFO_COUNT;
00782
00783     task_t tid = MACH_PORT_NULL;
00784     task_for_pid(current_task(), p ? p : getpid(), &tid);
00785
00786     if (task_info(
00787         tid, TASK_BASIC_INFO, (task_info_t)&S, &len)
00788     ){ wblog(FL, "ERR %s() %s", FCT, strerror(errno)); }
00789
00790     return (res ? S.resident_size : S.virtual_size);
00791 };
00792
00793 size_t Wb::getProcSize(const char *tag, const pid_t &p) {
00794     return Mac::getProcMemSize(p);
00795 };
00796
00797 #else
00798
00799 size_t Wb::getProcSize(const char *tag, const pid_t &p) {
00800     const int n=32; char f[n];
00801     int i=snprintf(f,n,"/proc/%d/status",p);
00802     if (i>=n) wblog(FL,
00803         "ERR %s() string out of bounds (%s; %d)", FCT, f,n);
00804     return Wb::getProcSize(tag,f);
00805 };
00806
00807 size_t Wb::getProcSize(const char *tag, const char *f) {
00808
00809     char *s, *line=NULL, u[8];
00810     size_t val; unsigned i=0, m=0; size_t len=0; ssize_t n;
00811
00812     static unsigned ncall=0;
00813
00814     FILE *fid; ++ncall;
00815     if (!f || !f[0] || !(fid=fopen(f,"r"))) {
00816         wblog(FL, "WRN failed to access %s '%s' (%d)", f?f:"", tag?tag:"", ncall);

```



```

00817     return 0;
00818 }
00819
00820 while ((n=getline(&line,&len,fid))>=0) { ++i;
00821     if (n) line[n-1]=0;
00822     if ((s=strstr(line,tag)) { u[0]=0; ++m;
00823         sscanf(s,"%s %lu %s",&val,u);
00824         if (strlen(u)==2 && u[1]=='B') {
00825             if (u[0]=='k' || u[0]=='K') { val*=(1<<10); } else
00826             if (u[0]=='M') { val*=(1<<20); } else
00827             if (u[0]=='G') { val*=(1<<30); }
00828             else wblog(FL,"WRN %s() got unit '%s'",FCT,u);
00829         }
00830         else wblog(FL,
00831             "WRN %s() invalid unit >%s< (%d)",FCT,u,strlen(u));
00832         break;
00833     }
00834 }
00835
00836 if (line) { free(line); line=NULL; }
00837 fclose(fid); fid=NULL;
00838
00839 if (!m) wblog(FL,"WRN %s() '%s' not found in %s",FCT,tag,f);
00840
00841 return val;
00842 };
00843
00844 long Wb::getCpuInfo(const char *tag, const char *f) {
00845
00846     char *s, *line=NULL;
00847     int m=0, val=0, i=0, vmax=-1; size_t len=0; ssize_t n;
00848
00849     FILE *fid=fopen(f,"r");
00850     if (fid==NULL) {
00851         wblog(FL,"WRN failed to access %s '%s'",f?f:" ",tag?tag:"");
00852         return 0;
00853     }
00854
00855     if (!strcmp(tag,"processor")) {
00856         while ((n=getline(&line,&len,fid))>=0) { ++i;
00857             if (n) line[n-1]=0;
00858             if ((s=strstr(line,tag)) { ++m;
00859                 sscanf(s,"%s : %d",&val); if (vmax<val) vmax=val;
00860             }
00861         }
00862
00863         if (vmax+1!=m) wblog(FL,
00864             "WRN %s() inconsistent ncores = %d/%d !?",FCT,vmax,m);
00865         val=m;
00866     }
00867     else wblog(FL,"ERR %s() invalid tag %s -> %s",FCT,f,tag);
00868
00869     if (line) { free(line); line=NULL; }
00870     fclose(fid); fid=NULL;
00871
00872     if (!m) wblog(FL,"WRN %s() '%s' not found in %s",FCT,tag,f);
00873
00874     return val;
00875 };
00876
00877 #endif
00878
00879 #ifdef __APPLE__
00880
00881 size_t wbsys::getMemTot() { return Mac::getMemSize("total"); };
00882 size_t wbsys::getMemFree() { return Mac::getMemSize("free"); };
00883
00884 int wbsys::getNumCores() {
00885     int n=1;
00886
00887     size_t l=4;
00888     int e=sysctlbyname("hw.physicalcpu",&n,&l,NULL,0);
00889
00890     if (e || n<1) { wblog(FL,
00891         "ERR %s() sysctl returned e=%d (n=%d)\n(%s)",
00892         FCT,e,n, e? strerror(errno):"" );
00893     }
00894     return n;
00895 };
00896
00897 #else
00898 size_t wbsys::getMemTot() { return Wb::getProcSize("MemTotal"); };
00899 size_t wbsys::getMemFree() { return Wb::getProcSize("MemFree"); };
00900 size_t wbsys::getSwapTot() { return Wb::getProcSize("SwapTotal"); };
00901 size_t wbsys::getSwapFree() { return Wb::getProcSize("SwapFree"); };
00902 int wbsys::getNumCores() { return Wb::getCpuInfo("processor"); };
00903 #endif

```

```

00904
00905 void Wb::ResSummary(const char *F, int L, const char *istr) {
00906
00907     static int do_cleanup=1;
00908     if (F==NULL && L) { do_cleanup=L; return; }
00909
00910     #ifndef __APPLE__
00911     if (do_cleanup) { do_cleanup=0; wbttop PS;
00912         wbststring s1(PS.VmSize2Str()), s2(PS.VmPeak2Str());
00913         if (istr) { sprintf(str,"%s() ",istr); } else { str[0]=0; }
00914
00915         wblog(F_L," * %sVMEM size: %s / %s", str, s1.data, s2.data);
00916         wblog(F_L," * %scputime : %s",str, STR2(gCPUTime,'c'));
00917     }
00918     #else
00919     if (do_cleanup) { do_cleanup=0;
00920         if (istr) { sprintf(str,"%s() ",istr); } else { str[0]=0; }
00921         wblog(F_L," * %sVMEM size / cputime (skipped for OSX)", str);
00922     }
00923     #endif
00924 };
00925
00926 void Wb::MemStat(const char *F, int L, char lflag) {
00927     wbttop().MemStat(F,L,lflag);
00928 };
00929
00930 inline wbststring wbsys::MemTot2Str() {
00931     return Wb::size2Str(getMemTot()); };
00932
00933 inline wbststring wbsys::MemFree2Str() {
00934     return Wb::size2Str(getMemFree()); };
00935
00936 inline wbststring wbsys::SwapTot2Str() {
00937     return Wb::size2Str(getSwapTot()); };
00938
00939 inline wbststring wbsys::SwapFree2Str() {
00940     return Wb::size2Str(getSwapFree()); };
00941
00942 #ifdef __APPLE__
00943
00944 char wbsys::checkSwapSpace(const char *F, int L) {
00945
00946     static double xref=0.25;
00947
00948     int e, mib[2] = { CTL_VM, VM_SWAPUSAGE };
00949     struct xsw_usage S; size_t l=sizeof(S);
00950
00951     if ((e=sysctl(mib,2,&S,&l,NULL,0))) wblog(F_L,
00952         "ERR %s() sysctl returned %d\n(%s)",FCT,e,strerror(errno));
00953
00954     if (S.xsu_total<=0 || double(S.xsu_avail)/S.xsu_total>xref) { return 0; }
00955
00956     wblog(F_L,"WRN free swap space: %s / %s @ %.3g",
00957         Wb::size2Str(S.xsu_avail).data,
00958         Wb::size2Str(S.xsu_total).data, xref); xref-=0.05;
00959     return 1;
00960 };
00961
00962 #else
00963
00964 char wbsys::checkSwapSpace(const char *F, int L) {
00965     static double xref=0.25;
00966     double x=getSwapTot(); if (x<=0) return 0;
00967     x=getSwapFree()/x; if (x>xref) { return 0; }
00968
00969     xref-=0.05; wblog(F_L,"WRN free swap space: %s / %s",
00970         SwapFree2Str().data, SwapTot2Str().data);
00971     return 1;
00972 };
00973 #endif
00974
00975 inline wbststring wbttop::VmSize2Str() const {
00976     return Wb::size2Str(getVmSize()); };
00977
00978 inline wbststring wbttop::VmPeak2Str() const {
00979     return Wb::size2Str(getVmPeak()); };
00980
00981 int wbttop::runningLarge(const char *F, int L,
00982     double th0,
00983     double fac
00984 ){
00985     static double thr=th0;
00986     int r=0; size_t s0, M;
00987
00988     s0=wbsys::getMemTot();
00989     M=getVmPeak();
00990

```

```

00991     if (M<thr*s0) { wbsys::checkSwapSpace(F_L); }
00992     else {
00993         wblog(F_L,"%s() %s is using %s",FCT,myname,VmPeak2Str().data);
00994         thr*=fac; r=1;
00995     }
00996     doflush(); wblogBuf.flush();
00997
00998     return r;
00999 };
01000
01001 void wbttop::MemStat(const char *F, int L, char lflag) const {
01002     static size_t vm_size_max=0;
01003     static size_t vm_size_last=0;
01004
01005     if (lflag=='m' || lflag=='M') {
01006         size_t q=getVmSize();
01007         if (vm_size_max*(lflag=='m' ? 1.10 : 1)<q)
01008             { vm_size_max=q; }
01009         else { lflag=0; }
01010     }
01011     else if (!lflag && getVmPeak()>0.66*system) {
01012         size_t q=getVmSize();
01013         if (!vm_size_last ||
01014             fabs((q-vm_size_last)/double(vm_size_last))>0.05
01015         ){ vm_size_last=q; lflag=2; }
01016     }
01017
01018     if (lflag) {
01019         double gfac=1/double(1<<30);
01020         wblog(F_L,"SYS %s() %.1fG (%.1f/%.0fG)",FCT,
01021             gfac*getVmSize(), gfac*getVmPeak(), gfac*system);
01022         doflush();
01023     }
01024 };
01025
01026 template <class T>
01027 size_t Wb::findfirst_sorted(const char *F, int L,
01028     const T* d0, const T* dd, size_t m, size_t N, size_t M, char lex
01029 ){
01030     if (!N || !M || m>=M) { if (F) wblog(F,L,
01031         "ERR %s() got empty/invalid input (%dx%d; %d",FCT,N,M,m);
01032         return -1;
01033     }
01034
01035     int q=Wb::cmpRange(dd,d0,m,lex);
01036     if (q>=0) {
01037         if (q>0) { if (F) wblog(F,L,
01038             "ERR %s() empty match (<first)",FCT);
01039             return -1;
01040         }
01041         return 0;
01042     }
01043
01044     size_t i1=0, i2=N-1, i=(i2+i1)/2;
01045     int q2=Wb::cmpRange(dd+i2*M,d0,m,lex);
01046
01047     if (q2<0) { if (F) wblog(F,L,
01048         "ERR %s() empty match (>last)",FCT);
01049         return N;
01050     }
01051
01052     while (i>i1) {
01053         q=Wb::cmpRange(dd+i*M,d0,m,lex);
01054         if (q<0) { i1=i; }
01055         else {
01056             if (q>0 && q!=q2) wblog(F_L,"ERR %s() "
01057                 "got unsorted data (upper i=%d: %d/%d) !?",FCT,i,q,q2);
01058             q2=q; i2=i;
01059         }
01060         i=(i1+i2)/2;
01061     }
01062
01063     if (q2) { if (F) wblog(F,L,
01064         "ERR %s() empty match (i=%d/%d @ %d,%d)",FCT,i,N,q,q2);
01065         return -1;
01066     }
01067     return i2;
01068 };
01069
01070 template <class T>
01071 size_t Wb::findlast_sorted(const char *F, int L,
01072     const T* d0, const T* dd, size_t m, size_t N, size_t M, char lex
01073 ){
01074     if (!N || !M || m>=M) { if (F) wblog(F,L,
01075         "ERR %s() got empty/invalid input (%dx%d; %d",FCT,N,M,m);
01076         return -1;
01077     }

```

```

01078
01079     int q1=Wb::cmpRange(dd,d0,m,lex);
01080
01081     if (q1>0) { if (F) wblog(F,L,
01082         "ERR %s() empty match (<first)",FCT);
01083         return -1;
01084     }
01085
01086     size_t i1=0, i2=N-1, i=(i2+i1)/2;
01087     int q=Wb::cmpRange(dd+i2*M,d0,m,lex);
01088
01089     if (q<=0) {
01090         if (q<0) { if (F) wblog(F,L,
01091             "ERR %s() empty match (>last)",FCT);
01092             return N;
01093         }
01094         return i2;
01095     }
01096
01097     while (i>i1) {
01098         q=Wb::cmpRange(dd+i*M,d0,m,lex);
01099         if (q>0) { i2=i; }
01100         else {
01101             if (q<0 && q!=q1) wblog(F_L,"ERR %s() "
01102                 "got unsorted data (lower i=%d: %d/%d) !?",FCT,i,q,q1);
01103             q1=q; i1=i;
01104         }
01105         i=(i1+i2)/2;
01106     }
01107
01108     if (q1) { if (F) wblog(F_L,
01109         "ERR %s() empty match (i=%d/%d @ %d,%d)",FCT,i,N,q1,q);
01110         return -1;
01111     }
01112     return i1;
01113 };
01114
01115 void wb_srand() {
01116
01117     struct timespec tnow;
01118     clock_gettime(CLOCK_REALTIME,&tnow);
01119
01120     unsigned long low, high;
01121     __asm__ __volatile__ ("rdtsc" : "=a" (low), "=d" (high) : );
01122
01123     ::srand(::rand() ^ unsigned(getpid()) ^ high ^ low);
01124     ::srand(::rand() ^ tnow.tv_sec ^ tnow.tv_nsec);
01125 };
01126
01127 #ifdef QS_USING_MPFR
01128
01129 namespace Wb {
01130
01131 template<> inline
01132 quad& rand::rand_(quad &x) { return (x.Rand()); };
01133
01134 template<> inline
01135 quad& rand::randb(quad &x) {
01136     x.Rand(); x*=2; x-=1; return x;
01137 };
01138
01139 template<> inline
01140 quad& rand::rands(quad &x) {
01141     x.Rand(); x*=2; x-=1;
01142     return (x.Atanh());
01143 };
01144
01145 };
01146
01147 template <> inline
01148 Wb::quad WbUtil<Wb::quad>::eps() {
01149     Wb::quad x;
01150     return x.eps();
01151 };
01152
01153 #endif
01154
01155 template <> inline
01156 wbcplx WbUtil<wbcplx>::eps() { return DBL_EPSILON; };
01157
01158 #endif
01159

```

5.22 util.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace utility routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_UTIL_HH__
00021 #define __WB_UTIL_HH__
00022
00023 /* ----- */
00024 namespace Wb {
00025 /* ----- */
00026
00027     int Rational(
00028         double &x,
00029         long &P, long &Q,
00030         double *dx=NULL,
00031         unsigned *nmax=NULL,
00032         wvector<double>* aa=NULL,
00033         unsigned niter=6,
00034         long pqmax=-1,
00035         double eps=1E-8,
00036         double eps2=1E-14,
00037         char vflag=0
00038     );
00039
00040     template<class T>
00041     double FixRational(
00042         const char *F, int L, T *d, unsigned n, char rflag=0,
00043         unsigned niter=6,
00044         long pqmax=-1,
00045         double eps=1E-8,
00046         double eps2=1E-14,
00047         double *rz=NULL, double *ra=NULL, char vflag=0
00048     );
00049
00050     wstring rat2Str(
00051         const char *F, int L, double d, unsigned niter=6,
00052         long pqmax=99999, double eps=1E-8, double eps2=1E-14, char vflag=0
00053     );
00054
00055     template<class T>
00056     double SkipZeros(T *d, unsigned n, double eps=1E-14){ return 0; };
00057
00058     void ResSummary(const char *F, int L, const char *istr=NULL);
00059
00060 #ifndef __WB_MEM_TRACK_HH__
00061     void MemStat(const char *F=0, int L=0, char lflag=0);
00062 #else
00063     void MemStat(const char *F, int L, char lflag);
00064 #endif
00065
00066 #ifndef __APPLE__
00067     long  getCpuInfo(const char *tag, const char *f="/proc/cpuinfo");
00068     size_t getProcSize(const char *tag, const char *f="/proc/meminfo");
00069 #endif
00070     size_t getProcSize(const char *tag, const pid_t &p);
00071     size_t getProcSize(const char *tag, const char *f);
00072
00073     wstring hostname(unsigned len=16);
00074     wstring hostid(char pflag=0);
00075
00076     wstring size2Str(double n, char p=4);
00077     const char* filename(const char *s);
00078
00079     size_t getFileSize(const char* f);
00080     double getFileInfo(const char* f, char w='m');
00081
00082     double GET_mtime(const char *F, int L, const char *f, const struct stat &S);
00083     double GET_atime(const char *F, int L, const char *f, const struct stat &S);
00084     double GET_ctime(const char *F, int L, const char *f, const struct stat &S);
00085

```

```

00086 bool fexist(const char *f, char type='*');
00087 bool isFile(const char *f);
00088 bool isDir(const char *f);
00089
00090 int mkdir(
00091     const char *F, int L, const char *path,
00092     char priv=0
00093 ){
00094     mode_t p=(S_IRWXU
00095         | S_IXGRP
00096         | S_IXOTH
00097     );
00098     if (!priv) p |= (
00099         S_IRGRP
00100         | S_IROTH
00101     );
00102
00103     int e;
00104
00105     #pragma omp critical (manage_WB_FILE_IO)
00106     { e=:mkdir(path,p); }
00107
00108     if (e && F) wblog(F_L,
00109         "ERR %s() failed to create directory (e=%d)\n%s",FCT,e,path);
00110     return e;
00111 };
00112
00113 template <class TI>
00114 TI sub2ind(const TI *s, const TI *I, unsigned n);
00115
00116 template <class T1, class T2> inline
00117 void ind2sub(T1 k, const T1 *s, T2 *I, unsigned n);
00118
00119 int atoi(const char *s, int &iout, unsigned n, char white=0);
00120
00121 int isUIntString(const char *s, char white=1);
00122
00123 template <class T>
00124 size_t findfirst_sorted(const char *F, int L,
00125     const T* d0, const T* dd, size_t m, size_t N, size_t M,
00126     char lex=1
00127 );
00128
00129 template <class T>
00130 size_t findlast_sorted(const char *F, int L,
00131     const T* d0, const T* dd, size_t m, size_t N, size_t M,
00132     char lex=1
00133 );
00134
00135 template <class T0, class T> inline
00136 size_t findfirst_sorted(const char *F, int L,
00137     const T0* d0, const T* dd, size_t m, size_t N, size_t M, char lex=1
00138 ){
00139     T d2[m];
00140     for (unsigned i=0; i<m; ++i) d2[i]=T(d0[i]);
00141     return findfirst_sorted(F,L,d2,dd,m,N,M,lex);
00142 };
00143
00144 template <class T0, class T> inline
00145 size_t findlast_sorted(const char *F, int L,
00146     const T0* d0, const T* dd, size_t m, size_t N, size_t M, char lex=1
00147 ){
00148     T d2[m];
00149     for (unsigned i=0; i<m; ++i) d2[i]=T(d0[i]);
00150     return findlast_sorted(F,L,d2,dd,m,N,M,lex);
00151 };
00152
00153 int dstrlen_utf8(const char *s);
00154
00155 template <class T>
00156 wbsting bits(const T &x, char compact=1);
00157
00158 class IOstat {
00159 public:
00160     IOstat()
00161         : nread(0), nwrite(0), sread(0), swrite(0), aux1(0), aux2(0) {};
00162
00163     IOstat& init() {
00164         nread=nwrite=0; sread=swrite=aux1=aux2=0;
00165         return *this;
00166     }
00167
00168     void gotread (size_t s=0) { nread +=1; sread +=s; }
00169     void gotwrite(size_t s=0) { nwrite+=1; swrite+=s; }
00170
00171     bool isEmpty() {

```

```

00173         return (!nread && !nwrite && !sread && !swrite && !aux1 && !aux2);
00174     };
00175
00176     void print(const char *istr,
00177               const char *hstr=0, const char *xstr1=0, const char *xstr2=0);
00178
00179     unsigned nread, nwrite;
00180     size_t sread, swrite, aux1, aux2;
00181
00182     protected:
00183     private:
00184 };
00185
00186 template <class T, class T2>
00187 class iterator {
00188     public:
00189
00190         iterator(T* p_) : i(0), p(p_) {
00191             if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
00192             n=p->numel();
00193         };
00194
00195         iterator& operator++() { ++i; return *this; }
00196         bool end() { return i>=n; }
00197
00198         size_t iter () const { return i; }
00199         size_t numel() const { return n; }
00200
00201         T2& operator[](size_t k) {
00202             if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
00203             return (*p)[k];
00204         }
00205         T2& at() {
00206             if (i>=n) wblog(FL,
00207                 "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
00208             return (*p)[i];
00209         };
00210
00211         size_t i, n;
00212         T* p;
00213
00214     protected:
00215     private:
00216 };
00217
00218 template <class T, class T2>
00219 class iterator<T, T2*> {
00220     public:
00221
00222         iterator(T* p_) : i(0), p(p_) {
00223             if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
00224             n=p->numel();
00225         };
00226
00227         iterator& operator++() { ++i; return *this; }
00228         bool end() { return i>=n; }
00229
00230         size_t iter () const { return i; }
00231         size_t numel() const { return n; }
00232
00233         T2* operator[](size_t k) {
00234             if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
00235             return (*p)[k];
00236         }
00237         T2* at() {
00238             if (i>=n) wblog(FL,
00239                 "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
00240             return (*p)[i];
00241         };
00242
00243         size_t i, n;
00244         T* p;
00245
00246     protected:
00247     private:
00248 };
00249
00250 template <class T, class T2>
00251 class citerator {
00252     public:
00253
00254         citerator(const T* p_) : i(0), p(p_) {
00255             if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
00256             n=p->numel();
00257         };
00258
00259         citerator& operator++() { ++i; return *this; }

```

```

00260     bool end() { return i>=n; }
00261
00262     size_t iter () const { return i; }
00263     size_t numel() const { return n; }
00264
00265     const T2& operator[](size_t k) {
00266         if (!p) wblog(FL,"ERR %s() object not yet initialized !?",FCT);
00267         return (*p)[k];
00268     }
00269     const T2& at() {
00270         if (i>=n) wblog(FL,
00271             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,n);
00272         return (*p)[i];
00273     };
00274
00275     size_t i,n;
00276     const T* p;
00277
00278 protected:
00279 private:
00280 };
00281
00282 template <class T, class T2>
00283 class citerator<T,T2*> {
00284 public:
00285
00286     citerator(const T* p_) : i(0), p(p_) {
00287         if (!p_) wblog(FL,"ERR %s() got p=NULL !?",FCT);
00288         n=p->numel();
00289     };
00290
00291     citerator& operator++() { ++i; return *this; }
00292     bool end() { return i>=n; }
00293
00294     size_t iter () const { return i; }
00295     size_t numel() const { return n; }
00296
00297     const T2* operator[](size_t k) {
00298         if (!p) wblog(FL,"ERR %s() object not yet initialized !?",FCT);
00299         return (*p)[k];
00300     }
00301     const T2* at() {
00302         if (i>=n) wblog(FL,
00303             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,n);
00304         return (*p)[i];
00305     };
00306
00307     size_t i,n;
00308     const T* p;
00309
00310 protected:
00311 private:
00312 };
00313
00314 class rand {
00315 public:
00316
00317     rand() {};
00318
00319     double get_dfac() { return dfac; }
00320
00321     template <class T>
00322     T& rand_(T &x);
00323
00324     template <class T>
00325     T& rand_(T &x, double fac);
00326
00327     template <class T>
00328     T& rands(T &x);
00329
00330     template <class T>
00331     T& randb(T &x);
00332
00333     template <class T>
00334     void scale_rand(T *x, size_t n, double fac) {
00335         for (size_t i=0; i<n; ++i) { x[i]*=fac; }
00336     }
00337
00338 protected:
00339 private:
00340
00341     static double dfac;
00342 };
00343
00344 double rand::dfac = (1.-1./double(long(1)«52)) / double(RAND_MAX);
00345
00346

```



```

00347 template<> inline
00348 double& rand::rand_(double &x) { return (x=dfac*::rand()); };
00349
00350 template<> inline
00351 int& rand::rand_(int &x) { return (x=(100*dfac)*::rand()); };
00352
00353 template<> inline
00354 unsigned& rand::rand_(unsigned&x) { return (x=(100*dfac)*::rand()); };
00355
00356 template<> inline
00357 long& rand::rand_(long &x) { return (x=(100*dfac)*::rand()); };
00358
00359 template<> inline
00360 size_t& rand::rand_(size_t &x) { return (x=(100*dfac)*::rand()); };
00361
00362 template<> inline
00363 int& rand::rand_(int&x, double fac) {
00364     return (x=(fac*dfac)*::rand());
00365 };
00366
00367 template<> inline
00368 unsigned& rand::rand_(unsigned &x, double fac) {
00369     return (x=(fac*dfac)*::rand());
00370 };
00371
00372 template<> inline
00373 long& rand::rand_(long &x, double fac) {
00374     return (x=(fac*dfac)*::rand());
00375 };
00376
00377 template<> inline
00378 size_t& rand::rand_(size_t &x, double fac) {
00379     return (x=(fac*dfac)*::rand());
00380 };
00381
00382 template<> inline
00383 double& rand::randb(double &x) { return (x=(2*dfac)*::rand()-1.); }
00384
00385 template<> inline
00386 int& rand::randb(int &x) { return (x=(199*dfac)*::rand()-99); }
00387
00388 template<> inline
00389 double& rand::rands(double &x) {
00390     x=(2*dfac*::rand()-1);
00391     if (fabs(x)>=1) wblog(FL,"ERR %s() got x=%.4g (1%+.4g) !?",FCT,x,x-1);
00392     return (x=atanh(x));
00393 };
00394
00395 template<> inline
00396 int& rand::rands(int &x) {
00397     double d=dfac*::rand();
00398     if (fabs(d)>=1) wblog(FL,"ERR %s() got d=%.4g (1%+.4g) !?",FCT,d,d-1);
00399     return (x=(100*atanh(d)+0.5));
00400 };
00401
00402 template<> inline
00403 void rand::scale_rand(int *x, size_t n, double fac) {
00404     fac/=double(99);
00405     for (size_t i=0; i<n; ++i) { x[i]=((double(x[i]))*fac); }
00406 };
00407
00408 template <class T,
00409 typename std::enable_if< std::is_floating_point<T>::value , T>::type* = nullptr >
00410 bool is_finite(const T *x, size_t n) {
00411     for (size_t i=0; i<n; ++i) { if (!::isfinite(x[i])) { return 0; }}
00412     return 1;
00413 };
00414
00415 bool is_finite(const wcomplex *x, size_t n);
00416
00417 template <class T,
00418 typename std::enable_if< is_integral<T>::value , T>::type* = nullptr >
00419 bool is_finite(const T *x, size_t n) { return 1; }
00420
00421 };
00422
00423 #ifdef __APPLE__
00424
00425 namespace Mac {
00426     double getMemSize(const char *tag);
00427     size_t getProcMemSize(
00428         pid_t p=0,
00429         char res=1
00430     );
00431 };
00432
00433 #endif

```

```

00434
00435     void wb_srand();
00436
00437 template <class T>
00438 class WbUtil {
00439     public:
00440
00441         static void adjust_tnorm(char &tnorm);
00442
00443         static bool hasConj();
00444         static bool hasNoConj();
00445
00446         static constexpr bool isPOD() {
00447             return std::is_trivially_copyable<T>::value;
00448         };
00449
00450         static bool isFloat();
00451         static bool isInt();
00452
00453         static constexpr char isComplex();
00454         static char isReal() {
00455             char q=isComplex(); if (q>=0) q=( q ? 0 : 1);
00456             return q;
00457         }
00458
00459         T eps();
00460
00461     protected:
00462     private:
00463
00464 };
00465
00466 template <class T> inline
00467 void WbUtil<T>::adjust_tnorm(char &tnorm){ tnorm=0; };
00468
00469 template <> inline
00470 void WbUtil<wbcomplex>::adjust_tnorm(char &tnorm __attribute__ ((unused))){};
00471
00472 template <class T> inline bool WbUtil<T>::hasConj(){ return 0; };
00473 template <> inline bool WbUtil<wbcomplex>::hasConj(){ return 1; };
00474
00475 template <class T> inline bool WbUtil<T>::hasNoConj(){ return 1; };
00476 template <> inline bool WbUtil<wbcomplex>::hasNoConj(){ return 0; };
00477
00478 template <class T> inline
00479 constexpr char ISCOMPLX_(T){ return 0; };
00480
00481 template <> inline
00482 constexpr char ISCOMPLX_(wbcomplex){ return 1; };
00483
00484 template <> inline
00485 constexpr char ISCOMPLX_(void){ return -1; };
00486
00487 #ifdef MATLAB_MEX_FILE
00488 template <> inline
00489 constexpr char ISCOMPLX_(mxComplexDouble){ return 2; };
00490 #endif
00491
00492 #ifdef _COMPLEX_H
00493 template <> inline
00494 constexpr char ISCOMPLX_(complex){ return 3; };
00495 #endif
00496
00497 template <class T> inline
00498 bool WbUtil<T>::isFloat(){ return 0; };
00499
00500 template <> inline bool WbUtil<double>::isFloat(){ return 1; };
00501 template <> inline bool WbUtil<long double>::isFloat(){ return 1; };
00502 template <> inline bool WbUtil<float>::isFloat(){ return 1; };
00503 template <> inline bool WbUtil<wbcomplex>::isFloat(){ return 1; };
00504
00505 template <class T> inline
00506 bool WbUtil<T>::isInt(){ return 0; }
00507
00508 template <> inline bool WbUtil<int>::isInt(){ return 1; }
00509 template <> inline bool WbUtil<char>::isInt(){ return 1; }
00510 template <> inline bool WbUtil<long>::isInt(){ return 1; }
00511
00512 template <> inline bool WbUtil<unsigned>::isInt(){ return 1; }
00513 template <> inline bool WbUtil<unsigned char>::isInt(){ return 1; }
00514 template <> inline bool WbUtil<unsigned long>::isInt(){ return 1; }
00515
00516 template <> inline double WbUtil<double>::eps(){ return DBL_EPSILON; }
00517 template <> inline float WbUtil<float>::eps(){ return FLT_EPSILON; }
00518
00519 template <> inline
00520 long double WbUtil<long double>::eps(){ return LDBL_EPSILON; }

```

```

00521
00522 #include <sys/time.h>
00523 #include <sys/resource.h>
00524
00525 class cpu_time {
00526
00527 public:
00528
00529     cpu_time() { tcpu=time('c'); tsys=time('s'); };
00530
00531     void init() { tcpu=time('c'); tsys=time('s'); };
00532
00533     cpu_time since() {
00534         cpu_time dt; dt.tcpu-=tcpu; dt.tsys-=tsys;
00535         return dt;
00536     };
00537
00538     static double time(char flag) {
00539         struct rusage result;
00540         getrusage(RUSAGE_SELF,&result);
00541         if (flag=='c') {
00542             return (
00543                 double(result.ru_utime.tv_sec
00544                     + 1e-6 * result.ru_utime.tv_usec
00545                 );
00546             } else {
00547                 return (
00548                     double(result.ru_stime.tv_sec
00549                         + 1e-6 * result.ru_stime.tv_usec
00550                     );
00551             }
00552         };
00553
00554         wstring toStr(char flag='c');
00555         double tcpu, tsys;
00556
00557     protected:
00558     private:
00559 };
00560
00561 cpu_time gCPUTime;
00562
00563 namespace wbsys {
00564
00565     wstring MemTot2Str();
00566     wstring MemFree2Str();
00567     wstring SwapTot2Str();
00568     wstring SwapFree2Str();
00569
00570     size_t getMemTot();
00571     size_t getMemFree();
00572     size_t getSwapTot();
00573     size_t getSwapFree();
00574
00575     char checkSwapSpace(const char *F=NULL, int L=0);
00576
00577     int getNumCores();
00578 };
00579
00580 class wbtopy {
00581
00582 public:
00583
00584     wbtopy() {
00585         pid=getpid(); ppid=getppid();
00586         sysmem=wbsys::getMemTot();
00587     };
00588
00589     wstring VmSize2Str() const;
00590     size_t getVmSize() const { return Wb::getProcSize("VmSize",pid); };
00591
00592     wstring VmPeak2Str() const;
00593     size_t getVmPeak() const { return Wb::getProcSize("VmPeak",pid); };
00594
00595     int runningLarge(const char *F=NULL, int L=0,
00596         double th0=0.80,
00597         double fac=1.2
00598     );
00599
00600     void MemStat(const char *F=0, int L=0, char lflag=0) const;
00601
00602     pid_t pid, ppid;
00603     size_t sysmem;
00604
00605     protected:
00606     private:
00607 };

```

```

00608
00609 #endif
00610

```

5.23 wbarrray.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbarrray (array class, col-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_COL_MAJOR_CC__
00021 #define __WB_ARRAY_COL_MAJOR_CC__
00022
00023 //-----//
00024 // wbarrray - N-D Array class
00025 //
00026 //     generic N-dimensional class that allows permutations
00027 //     and reshaping; index order is column-major.
00028 //
00029 // Wb,Sep14,09 ; Wb,Aug11,05
00030 //-----//
00031
00032 template<class T>
00033 void wbarrray<T>::setRand(const char pnflag) {
00034     static int firstcall=1; size_t i,s=numel(); double fac;
00035
00036     if (WbUtil<T>::isFloat())
00037         fac = 1. / (double)RAND_MAX;
00038     else fac = 100. / (double)RAND_MAX;
00039
00040     if (firstcall) { wb_srand(); firstcall=0; }
00041     for (i=0; i<s; ++i) data[i] = (T)(fac*::rand());
00042
00043     if (pnflag) {
00044         fac *= (0.5*(double)RAND_MAX);
00045         for (i=0; i<s; ++i) data[i] -= fac;
00046     }
00047 }
00048
00049 template<class T>
00050 double wbarrray<T>::SkipTiny(double eps_) {
00051     T zero=T(0), x2=zero, eps=T(eps_);
00052     if (eps!=zero) { size_t i=0, n=numel(); T a;
00053         for (; i<n; ++i) { if (data[i]!=zero) {
00054             a=ABS(data[i]); if (a<eps) { x2+=NORM2(a); data[i]=zero; }
00055         } }
00056     }
00057     return double(x2);
00058 };
00059
00060 template<> inline
00061 double wbarrray<wbcomplex>::SkipTiny(double eps) {
00062     double a,x2=0;
00063     for (size_t s=numel(), i=0; i<s; ++i) {
00064         a=fabs(data[i].r); if (a>0 && a<eps) { x2+=a*a; data[i].r=0; }
00065         a=fabs(data[i].i); if (a>0 && a<eps) { x2+=a*a; data[i].i=0; }
00066     }
00067     return std::sqrt(x2);
00068 }
00069
00070
00071 template<> inline
00072 wbarrray<double>& wbarrray<double>::SkipTiny_float(double ref) {
00073     Wb::chopTiny_float(data,numel(),ref);
00074     return *this;
00075 };
00076
00077 template<> inline

```

```

00078 wbararray<wbcomplex>& wbararray<wbcomplex>::SkipTiny_float(wbcomplex ref) {
00079     Wb::chopTiny_float((double*)data,2*numel(),ref.r);
00080     return *this;
00081 };
00082
00083 template<> inline
00084 double wbararray<wbcomplex>::SkipTiny_imag(double eps) {
00085     return Wb::chopTiny_imag(data,numel(),eps);
00086 };
00087
00088 template<class T>
00089 wbvector<T>& wbararray<T>::normDim2(unsigned id, wbvector<T> &xk) const {
00090
00091     size_t i, n, N=numel(); if (!N) { return xk.init(); }
00092     unsigned k, r=SIZE.len, l=r-1;
00093     const size_t *s=SIZE.data; T *x2;
00094     WBINDEX I(r);
00095
00096     if (id>=r) { wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,id,r); }
00097     n=SIZE[id]; xk.init(n); x2=xk.data;
00098
00099     for (i=0; i<N; ++i) {
00100         x2[I[id]] += NORM2(data[i]);
00101
00102         k=0; ++I[0];
00103         while (I[k]>=s[k] && k<l) { I[k]=0; ++I[++k]; }
00104     }
00105     return xk;
00106 };
00107
00108
00109 template<class T>
00110 wbararray<T>& wbararray<T>::SkipTrailingZeroSpace(
00111     unsigned k,
00112     size_t n,
00113     double eps
00114 ) {
00115     wbvector<T> x2; normDim2(k,x2); if (!x2) { return *this; }
00116
00117     size_t N=SIZE[k];
00118
00119     if (long(n)<0) { n=0; }
00120     if (n<N) {
00121         size_t i=N-1;
00122         double eps2=x2.max(); if (eps2<1) { eps2=1; }
00123         eps2*=(eps*eps);
00124
00125         for (; i<N; --i) { if (x2[i]>eps2) { break; } }
00126         if ((++i)<N) { if (i<n) { i=n; }
00127             wbvector<size_t> S(SIZE); S[k]=i;
00128             Resize(S);
00129         }
00130     }
00131     return *this;
00132 };
00133
00134 template<class T> inline
00135 T wbararray<T>::normDiff2(const wbararray<T> &B, char sflag) const {
00136
00137     T x2=0;
00138     if (sameSize(B)) {
00139         x2=Wb::rangeNormDiff2(data, B.data, numel());
00140         return x2;
00141     }
00142
00143     if (!sflag || SIZE.len!=2 || B.SIZE.len!=2) {
00144         if (sflag) wblog(FL,"ERR %s() size mismatch\n"
00145             "(sflag=%d for matrices only: %s / %s",FCT,sflag,SSTR_(this),SSTR(B));
00146         else wblog(FL,"ERR size mismatch (%s; %s)",SSTR_(this),SSTR(B));
00147     }
00148
00149     char b1=1, b2=1;
00150     size_t i,j, d1=SIZE[0], d2=SIZE[1], D1=B.SIZE[0], D2=B.SIZE[1];
00151     const T* a=data, *b=B.data;
00152
00153     if (d1>D1) { SWAP(d1,D1); b1=0; }
00154     if (d2>D2) { SWAP(d2,D2); b2=0; }
00155
00156     for (j=0; j<D2; ++j) {
00157         if (j<d2) {
00158             for (i=0; i<d1; ++i) { x2+=NORM2(a[i]-b[i]); }
00159             if (b1)
00160                 { for (; i<D1; ++i) { x2+=NORM2(b[i]); } }
00161             else { for (; i<D1; ++i) { x2+=NORM2(a[i]); } }
00162             a+=SIZE[0]; b+=B.SIZE[0];
00163         }
00164         else if (b2) {

```

```

00165         if (j==d2) { D1=B.SIZE[0]; }
00166         for (i=0; i<D1; ++i) { x2+=NORM2(b[i]); }; b+=D1;
00167     }
00168     else {
00169         if (j==d2) { D1=SIZE[0]; }
00170         for (i=0; i<D1; ++i) { x2+=NORM2(a[i]); }; a+=D1;
00171     }
00172 }
00173 return x2;
00174 };
00175
00176 template<class T> inline
00177 T wbararray<T>::normDiff2(const wbvector<T> &b) const {
00178     if (!isVector() || numel()!=b.len) wblog(FL,
00179         "ERR size mismatch (%s; %d)",SSTR_(this),b.len);
00180     return Wb::rangeNormDiff2(data, b.data, numel());
00181 };
00182
00183 template<class T>
00184 wbararray<T>& wbararray<T>::plus(
00185     const wbararray &B, wbararray &C, T bfac, char iflag
00186 ) const {
00187     if (isEmpty() && B.isEmpty()) { return C.init(); }
00189     if (C.isRef()) wblog(FL,"ERR %s() got ref (considered const)",FCT);
00191
00192     if (SIZE==B.SIZE) { if (!bfac) { C=*this; } else {
00193         size_t i=0, n=numel(); const T *b=B.data; T* c;
00194
00195         C.SIZE=SIZE;
00196         C.NEW_DATA(SIZE.prod(0),NULL,0,0);
00197         c=C.data;
00198
00199         if (bfac==T(+1)) for (; i<n; ++i) { c[i] = data[i] + b[i]; } else
00200         if (bfac==T(-1)) for (; i<n; ++i) { c[i] = data[i] - b[i]; }
00201         else for (; i<n; ++i) { c[i] = data[i] + b[i]*bfac; }
00202     } }
00203     else if (!B && !bfac) { C.init(*this); }
00204     else if (!iflag || (SIZE.len && B.SIZE.len)) {
00205         wblog(FL,"ERR %s() size mismatch %s / %s",FCT,SSTR_(this),SSTR(B));
00206     }
00207     else {
00208         if (SIZE.len) { C=*this; }
00209         else {
00210             if (!bfac) { C.init(B.SIZE); } else
00211             if (bfac==T(1)) { C.init(B); }
00212             else {
00213                 size_t i=0, n=B.numel(); const T* b=B.data; T* c;
00214                 C.SIZE=B.SIZE;
00215                 C.NEW_DATA(B.SIZE.prod(0),NULL,0,0);
00216                 c=C.data;
00217
00218                 if (bfac==T(-1))
00219                     { for (; i<n; ++i) { c[i] =-b[i]; } }
00220                 else { for (; i<n; ++i) { c[i] = b[i]*bfac; } }
00221             }
00222         }
00223     }
00224     return C;
00225 };
00226
00227 template<class T>
00228 template<class TB>
00229 wbararray<T>& wbararray<T>::Plus(
00230     const wbararray<TB> &B, TB bfac,
00231     char iflag,
00232     T afac
00233 ){
00234     if (isEmpty()) {
00235         if (B.isEmpty()) { return *this; }
00236         if (iflag || afac==T(0)) {
00237             if (iflag && afac && afac!=T(1)) { wblog(FL,
00238                 "ERR %s() got empty A with afac=%s",FCT,NSTR(afac));
00239             }
00240             if (bfac==TB(1)) { return init(B); } else
00241             if (!bfac) { return init(B.SIZE); }
00242             else {
00243                 SIZE=B.SIZE;
00244                 NEW_DATA(B.SIZE.prod(0),NULL,0,0);
00245             }
00246         }
00247     }
00248     else { iflag=0; }
00249
00250     if (isRef()) wblog(FL,"ERR %s() got ref (considered const)",FCT);
00251     if (SIZE!=B.SIZE) wblog(FL,

```

```

00252         "ERR %s() size mismatch %s / %s",FCT,SSTR_(this), SSTR(B));
00253
00254     Wb::cpyRange(data, B.data, numel(), iflag? T(0) : afac, bfac);
00255
00256     return *this;
00257 };
00258
00259 template<class T> inline
00260 char wbararray<T>::sameUptoFac(
00261     const wbararray<T> &B, T* fac_, double eps
00262 ) const {
00263
00264     size_t i, n=numel(); T a,fac=1;
00265
00266     if (SIZE!=B.SIZE) return 1;
00267     if (!n) return 0;
00268
00269     a=this->aMax(&i);
00270     if (a<=eps) {
00271         if (a==0 || B.aMax(&i)>eps) { if (fac_) (*fac_)=0; return 0; }
00272         else return 1;
00273     }
00274     if (ABS(B.data[i])<=eps) return 2;
00275
00276     fac=data[i]/B.data[i]; if (fac_) (*fac_)=fac;
00277
00278     for (i=0; i<n; ++i) {
00279         a=ABS(data[i]-fac*B.data[i]);
00280         if (a>eps) return 3;
00281     }
00282     return 0;
00283 };
00284
00285 template<class T> inline
00286 bool wbararray<T>::sameAs(const wbararray &B, double eps) const {
00287     if (this!=&B) {
00288         if (SIZE!=B.SIZE) { return 0; }
00289         if (data!=B.data) {
00290             if (eps) {
00291                 for (size_t i=0, n=numel(); i<n; ++i) {
00292                     if (ABS(data[i]-B.data[i])>eps) { return 0; }
00293                 }
00294             } else { return (!memcmp(data, B.data, numel()*sizeof(T))); }
00295         }
00296     }
00297     return 1;
00298 };
00299
00300 template<class T> inline
00301 wbvector<T>& wbararray<T>::getCol(size_t k, wbvector<T> &v) const {
00302     const T* x=col(k);
00303     return v.init(SIZE[0],x);
00304 };
00305
00306 template<class T> inline
00307 wbvector<T>& wbararray<T>::getRow(size_t k, wbvector<T> &v) const {
00308     const T* x=row(k);
00309     return v.initStride(SIZE[1],x,SIZE[0]);
00310 };
00311
00312 template<class T> inline
00313 wbararray<T>& wbararray<T>::getCol(size_t k, wbararray<T> &a) const {
00314     const T* x=col(k);
00315     size_t s[2]={ SIZE[0], 1 }; a.SIZE.init(2,s); a.NEW_DATA(s[0],x);
00316     return a;
00317 };
00318
00319 template<class T> inline
00320 wbararray<T>& wbararray<T>::getRow(size_t k, wbararray<T> &a) const {
00321     const T* x=row(k);
00322     size_t s[2]={ 1, SIZE[1] }; a.SIZE.init(2,s);
00323
00324     a.NEW_DATA(s[1],NULL,0,0);
00325     Wb::cpyStride(a.data, x,1,NULL,s[1],-1,SIZE[0],0);
00326
00327     return a;
00328 };
00329
00330 template<class T> inline
00331 wbvector<T> wbararray<T>::getDiag() const {
00332
00333     wbvector<T> x;
00334
00335     if (!SIZE.len) { return x; }
00336     if (SIZE.len!=2) wblog(FL,
00337         "ERR %s() invalid matrix %s",FCT,SSTR_(this));
00338

```

```

00339     size_t i=0, j=0, m=SIZE[0]+1, n=MIN(SIZE[0],SIZE[1]);
00340     if (n) { x.init(n);
00341         for (; i<n; ++i, j+=m) { x[i]=data[j]; }
00342     }
00343     return x;
00344 };
00345
00346 template<class T>
00347 void wbararray<T>::getMatSize(
00348     const char *F, int L, size_t &dim1, size_t &dim2) const {
00349
00350     size_t i, r=SIZE.len, r2=r/2;
00351     const size_t *const &s=SIZE.data;
00352
00353     if (r%2) wblog(F,L,
00354         "ERR %s() even rank object required (%s)",FCT,SSTR_(this));
00355     if (r) {
00356         dim1=s[0]; dim2=s[r2];
00357         for (i=1; i<r2; i++) { dim1*=s[i]; dim2*=s[i+r2]; }
00358     } else { dim1=dim2=0; }
00359 };
00360
00361 template<class T>
00362 void wbararray<T>::getMatSize(size_t &dim1, size_t &dim2, unsigned m) const {
00363
00364     unsigned i=0, r=SIZE.len;
00365     const size_t *const &s=SIZE.data;
00366
00367     if (m>r) { m=r; }
00368     if (m) { dim1=s[0]; for (++i; i<m; ++i) { dim1*=s[i]; } }
00369     else { dim1=(r?1:0); }
00370
00371     if (i<r) { dim2=s[i]; for (++i; i<r; ++i) { dim2*=s[i]; } }
00372     else { dim2=(r?1:0); }
00373 };
00374
00375 template<class T>
00376 void wbararray<T>::getMatSize(const ctrIdx &ic,
00377     size_t &dimc,
00378     size_t &dimk,
00379     unsigned r, size_t *dom
00380 ) const {
00381
00382     unsigned i,j;
00383
00384     if (!ic) wblog(FL,"ERR %s() got empty ctrIdx %s",FCT,STR(ic));
00385     if (int(r)<0) {
00386         if (dom) wblog(FL,"ERR %s() invalid usage (%d,%p)",FCT,r,dom);
00387         r=SIZE.len;
00388     }
00389     else if (r>SIZE.len) {
00390         sprintf(str,"%s() rank out of bounds (r=%d/%ld)",FCT,r,SIZE.len);
00391         wblog(FL,dom ? "ERR %s":"WRN %s", str);
00392         r=SIZE.len;
00393     }
00394
00395     wvector<char> mm(r);
00396     dimc=dimk=1;
00397
00398     for (j=0; j<ic.len; ++j) { i=ic[j];
00399         if (i>=r) { char s[128];
00400             sprintf(s,127,"%s() ctrIdx out of bounds (%s/%d)",FCT,STR(ic),r);
00401             if (dom || i>99)
00402                 wblog(FL,"ERR %s",s);
00403             else wblog(FL,"WRN %s",s);
00404         }
00405         else {
00406             if (++mm[i]!=1) wblog(FL,
00407                 "ERR %s() non-unique ctrIdx (%s/%d)",FCT,STR(ic),r);
00408             dimc *= SIZE[i];
00409         }
00410     }
00411
00412     for (i=0; i<r; ++i) { if (!mm[i]) { dimk *= SIZE[i]; } }
00413
00414     if (dom) { for (*dom=1; i<SIZE.len; ++i) { *dom *= SIZE[i]; } }
00415 };
00416
00417 template<class T>
00418 char wbararray<T>::fitsSize(const wbararray<T> &B, char strict) const {
00419
00420     unsigned i=0, n=MIN(SIZE.len,B.SIZE.len);
00421     const SPIDX_T *s1=SIZE.data, *s2=B.SIZE.data;
00422
00423     for (; i<n; ++i) { if (s1[i]>s2[i]) return 0; }
00424
00425     if (SIZE.len==B.SIZE.len) { return 1; }

```



```

00426     if (strict) { return 0; }
00427
00428     if (SIZE.len<B.SIZE.len) {
00429         for (n=B.SIZE.len; i<n; ++i) { if (!s2[i]) return 0; }
00430         return 2;
00431     }
00432     else {
00433         for (n=SIZE.len; i<n; ++i) { if (s1[i]>1) return 0; }
00434         return 3;
00435     }
00436 };
00437
00438 template<class T>
00439 template<class TB>
00440 bool wbararray<T>::sameSize(const wbararray<TB> &B,
00441     char lflag
00442 ) const {
00443
00444     if (!lflag) { return (SIZE==B.SIZE); }
00445     if (numel()!=B.numel()) { return 0; }
00446     if (!data) { return 1; }
00447
00448     unsigned i=0, j=0, ra=SIZE.len, rb=B.SIZE.len;
00449     const size_t *sa=SIZE.data, *sb=B.SIZE.data;
00450
00451     if (!ra || !rb) wblog(FL,"ERR %s() got rank r=%d/%d",FCT,ra,rb);
00452
00453     if (lflag=='l') { lflag=3; } else
00454     if (lflag=='L') { lflag=7; } else
00455     if (lflag<1 || lflag>7) { wblog(FL,
00456         "ERR %s() invalid lflag=%s",FCT,cSTR(lflag)); }
00457
00458     if (lflag&2) {
00459         while (ra) { if (sa[ra-1]==1) { --ra; } else { break; }}
00460         while (rb) { if (sb[rb-1]==1) { --rb; } else { break; }}
00461     }
00462
00463     if (lflag&1) {
00464         for (; i<ra; ++i) { if (sa[i]!=1) { break; }}
00465         for (; j<rb; ++j) { if (sb[j]!=1) { break; }}
00466     }
00467
00468     if (lflag&4) {
00469         while (i<ra && j<rb) {
00470             if (sa[i]!=sb[j]) { return 0; }
00471             for (++i; i<ra; ++i) { if (sa[i]!=1) break; }
00472             for (++j; j<rb; ++j) { if (sb[j]!=1) break; }
00473         }
00474         if (i<ra || j<rb) { return 0; }
00475     }
00476     else {
00477         ra-=i; rb-=j; if (ra!=rb) { return 0; }
00478         sa+=i; sb+=j;
00479         for (i=0; i<ra; ++i) { if (sa[i]!=sb[i]) return 0; }
00480     }
00481
00482     return 1;
00483 };
00484
00485 template<class T> inline
00486 bool wbararray<T>::sameSize(const size_t *s, unsigned n) const {
00487
00488     unsigned i=0, m=MIN(n,unsigned(SIZE.len));
00489     for (i=0; i<m; ++i) { if (s[i]!=SIZE.data[i]) return 0; }
00490     for (; i<n; ++i) { if (s[i]!=1) return 0; }
00491     return 1;
00492 };
00493
00494 template<class T> inline
00495 bool wbararray<T>::sameSize1(const size_t *s2, unsigned n2) const {
00496
00497     const size_t *s1=SIZE.data;
00498     unsigned i=0, j=0, n1=SIZE.len;
00499
00500     while (1) {
00501         for (; i<n1 && s1[i]==1; ++i) {};
00502         for (; j<n2 && s2[j]==1; ++j) {};
00503         if (i==n1 || j==n2 || s1[i]!=s2[j]) break;
00504         ++i; ++j;
00505     }
00506
00507     return ((i<n1 || j<n2 || (!n1)^(!n2)) ? 0 : 1);
00508 };
00509
00510 template<class T> inline
00511 bool wbararray<T>::equal(const wbararray<T> &B, T eps) const {
00512     size_t i,s=numel(); if (SIZE!=B.SIZE) { return 0; }

```

```

00513     if (eps)
00514     { for (i=0; i<s; ++i) { if (fabs(data[i]-B.data[i])>eps) return 0; }}
00515     else { if (memcmp(data,B.data,s*sizeof(T)) { return 0; }}
00516     return 1;
00517 };
00518
00519 template<class T> inline
00520 bool wbararray<T>::unequal (const wbararray<T> &B, T eps) const {
00521     return !equal(B,eps);
00522 };
00523
00524 template<class T> inline
00525 bool wbararray<T>::equal (
00526     const wbararray<T> &B, double eps, double &maxdiff) const {
00527
00528     if (SIZE!=B.SIZE) return 0;
00529
00530     size_t i,s=numel(); double d;
00531
00532     for (maxdiff=0, i=0; i<s; i++) {
00533         d = ABS( data[i] - B.data[i] );
00534         if (maxdiff<d) maxdiff=d;
00535     }
00536
00537     return (maxdiff<eps);
00538 };
00539
00540 template<class T> inline
00541 bool wbararray<T>::unequal (
00542     const wbararray<T> &B, double eps, double &maxdiff) const {
00543     return !(*this).equal(B,eps,maxdiff);
00544 };
00545
00546 template<class T>
00547 bool wbararray<T>::isDiag_aux(const T eps, const char* task) const {
00548
00549     size_t i=0;
00550     wIndex I(SIZE);
00551
00552     char isDiag=!strcmp(task,"isDiag");
00553     char isIdty=!strcmp(task,"isIdty");
00554
00555     if (!isDiag && !isIdty) wblog(FL,"ERR %s() invalid task '%s'",FCT,task);
00556     if (SIZE.len%2) wblog(FL,"ERR %s() for even-rank objects only (%s)",
00557         task,SSTR_(this));
00558
00559     while (++i) {
00560         if (!I.isdiag()) {
00561             if (ABS(data[i])>eps) return 0;
00562         }
00563         else {
00564             if (isIdty && (ABS(data[i]-T(1)))>eps) return 0;
00565         }
00566         ++i;
00567     }
00568
00569     return 1;
00570 };
00571
00572 template<class T>
00573 bool wbararray<T>::isDiag_aux(double *epsp, const char* task) const {
00574     if (!epsp) wblog(FL,"ERR %s() got null eps",FCT);
00575     if (!task) wblog(FL,"ERR %s() got null task",FCT);
00576
00577     size_t i,k, s=numel(), r=SIZE.len, l=r-1;
00578     WINDEX I(r); bool is=1; double a, eps=0, eref=*epsp;
00579
00580     char isDiag=!strcmp(task,"isDiag");
00581     char isIdty=!strcmp(task,"isIdty");
00582     if (!isDiag && !isIdty) wblog(FL,"ERR invalid task '%s'", task);
00583
00584     if (!isRank(2)) { info("this"); wblog(FL,
00585         "ERR %s only applies to rank-2 tensors (%s).",task,SSTR_(this)); }
00586
00587     for (i=0; i<s; i++) {
00588         if (I[0]!=I[1]) { a=ABS(data[i]);
00589             if (is && eref<a) is=0;
00590             if (eps<a) eps=a;
00591         }
00592         else if (isIdty) { a=ABS(data[i]-T(1));
00593             if (is && eref<a) is=0;
00594             if (eps<a) eps=a;
00595         }
00596
00597         k=0; I[0]++;
00598         while (I[k]>=SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
00599     }

```

```

00600
00601     *epsp=eps; return is;
00602 };
00603
00604 template<class T>
00605 bool wbarry<T>::isProptoId(T &x, const T eps) const {
00606
00607     if (!data) { return 0; }
00608
00609     size_t i=0;
00610     windex I(SIZE);
00611
00612     if (SIZE.len%2) wblog(FL,
00613         "ERR %s() for even-rank objects only (%s)",FCT,SSTR_(this));
00614     if (!x && data) { x=data[0]; }
00615
00616     while (++I) {
00617         if (!I.isdiag())
00618             { if (ABS(data[i] )>eps) return 0; }
00619         else { if (ABS(data[i]-x)>eps) return 0; }; ++i;
00620     }
00621
00622     if (i && !data) wblog(FL,
00623         "ERR %s() invalid empty array (len=%d)",FCT,i);
00624
00625     return 1;
00626 };
00627
00628 template<class T>
00629 bool wbarry<T>::isOpS(size_t *n) const {
00630
00631     size_t i,r=SIZE.len, r2=r/2;
00632
00633     if (r==0) return 1;
00634     if (r%2) return 0;
00635
00636     for (i=0; i<r2; i++) if (SIZE[i]!=SIZE[i+r2]) return 0;
00637
00638     if (n) { (*n)=Wb::prodRange(SIZE.data,r2); }
00639
00640     return 1;
00641 }
00642
00643 template<class T>
00644 wbarry<T>& wbarry<T>::balanceOp(
00645     const char *F, int L, T &dref, double &dscale
00646 ){
00647     size_t i,j,l,n=0;
00648
00649     if (!isOpS(&n)) wblog(F_L,
00650         "ERR %s() requires operator (%s)",FCT,SSTR_(this));
00651
00652     if (n<=1) { dref=0; dscale=1; return *this; }
00653     dref=0; dscale=0;
00654
00655     for (l=i=0; i<n; i++, l+=(n+1)) dref+=data[l];
00656     dref /= n;
00657
00658     for (l=i=0; i<n; i++)
00659         for (j=0; j<n; j++, l++) { if (i==j) data[l]-=dref;
00660             dscale += ABS2(data[l]);
00661         }
00662     dscale = std::sqrt(dscale)/n;
00663
00664     if (dscale!=0) (*this)*=(1./dscale);
00665     else dscale=1;
00666
00667     return *this;
00668 }
00669
00670 template<class T>
00671 wbarry<T>& wbarry<T>::Symmetrize(
00672     const char *F, int L,
00673     double *delta,
00674     char cflag,
00675     char tflag,
00676     char fflag
00677 ){
00678     size_t i,j,n=0; T d1,d2; double e,E=0,dmax=0;
00679
00680     if (!isOpS(&n)) wblog(F,L,
00681         "ERR %s() (generalized) square matrix required (%s)",
00682         FCT,SSTR_(this));
00683     if (tflag && !delta) return *this;
00684
00685     if (cflag && typeid(T)!=typeid(wbcomplex)) cflag=0;
00686

```

```

00687     for (i=0; i<n; i++)
00688     for (j=i; j<n; j++) { d1=data[i+j*n]; d2=data[j+i*n];
00689         dmax=MAX(MAX(dmax,ABS(d1)),ABS(d2));
00690         if (i!=j) {
00691             e=(cflag ? ABS(d1-CONJ(d2)) : ABS(d1-d2));
00692             E=MAX(E,e);
00693         }
00694     }
00695
00696     dmax=MAX(1.,dmax);
00697     e=E; if (dmax!=0) e/=dmax;
00698
00699     if (delta) {
00700         if (dmax!=0) {
00701             if (e>(*delta)) { tflag=1;
00702 #ifdef MATLAB_MEX_FILE
00703                 this->put(FL,"M_");
00704 #endif
00705                 sprintf(str,"%s() non-symmetric operator (%d)\n"
00706                     "eps=%.3g/%.3g @ dmax=%.3g",FCT,cflag,e,*delta,dmax);
00707
00708                 if (fflag) wblog(F,L,"ERR %s",str);
00709                 else wblog(F,L,"WRN %s",str);
00710             }
00711             *delta=(e);
00712         }
00713         else if (e==0) *delta=0; else *delta=1;
00714     }
00715
00716     if (tflag==0) {
00717         if (cflag) {
00718             for (i=0; i<n; i++)
00719                 for (j=i+1; j<n; j++) {
00720                     d1=0.5*(data[i+j*n]+CONJ(data[j+i*n]));
00721                     data[i+j*n]=d1; data[j+i*n] = CONJ(d1);
00722                 }
00723         }
00724         else {
00725             for (i=0; i<n; i++)
00726                 for (j=i+1; j<n; j++) {
00727                     d1=0.5*(data[i+j*n]+data[j+i*n]);
00728                     data[i+j*n] = data[j+i*n] = d1;
00729                 }
00730         }
00731     }
00732
00733     return *this;
00734 };
00735
00736 template<class T>
00737 wbararray<T>& wbararray<T>::swapRows(size_t i1, size_t i2) {
00738
00739     if (rank()!=2) wblog(FL,
00740         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00741     if (i1>=SIZE[0] || i2>=SIZE[0]) wblog(FL,"ERR %s() "
00742         "index out of bounds (%d,%d; %s)",FCT,i1+1,i2+1,SSTR_(this));
00743
00744     if (i1!=i2) { size_t dim1=SIZE[0], dim2=SIZE[1]; if (dim2) {
00745         T x, *d1=data+i1, *d2=data+i2;
00746         for (size_t j=0; j<dim2; ++j, d1+=dim1, d2+=dim1) {
00747             x=(*d1); *d1=(*d2); *d2=x;
00748         }
00749     }}
00750     return *this;
00751 };
00752
00753 template<class T>
00754 wbararray<T>& wbararray<T>::swapCols(size_t j1, size_t j2) {
00755
00756     if (rank()!=2) wblog(FL,
00757         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00758     if (j1>=SIZE[1] || j2>=SIZE[1]) wblog(FL,"ERR %s() "
00759         "index out of bounds (%d,%d; %s)",FCT,j1+1,j2+1,SSTR_(this));
00760
00761     if (j1!=j2) { size_t dim1=SIZE[0]; if (dim1) {
00762         T x, *d1=data+dim1*j1, *d2=data+dim1*j2;
00763         for (size_t i=0; i<dim1; ++i) {
00764             x=d1[i]; d1[i]=d2[i]; d2[i]=x;
00765         }
00766     }}
00767     return *this;
00768 };
00769
00770 template<class T>
00771 wbararray<T>& wbararray<T>::setCol(size_t k, size_t k0) {
00772     if (rank()!=2) wblog(FL,
00773         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));

```

```

00774     if (k>=SIZE[1] || k0>=SIZE[1]) wblog(FL,
00775         "ERR %s() index out of bounds (%d,%d; %s)",
00776         FCT,k+1,k0+1,SSTR_(this));
00777
00778     size_t dim1=SIZE[0];
00779     if (k!=k0 && dim1) {
00780         MEM_COPY<T>(ref(OU,k),dim1,ref(OU,k0));
00781     }
00782
00783     return *this;
00784 };
00785
00786 template<class T>
00787 wbarry<T>& wbarry<T>::setCol(
00788     size_t k, const T* v, T fac, size_t stride) {
00789
00790     if (rank()!=2) wblog(FL,
00791         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00792     if (k>=SIZE[1]) wblog(FL,
00793         "ERR %s() index out of bounds (%d; %s)",FCT,k+1,SSTR_(this));
00794
00795     size_t i=0, dim1=SIZE[0]; T *d=data+k*dim1;
00796     if (dim1 && d!=v) {
00797         if (fac== 1) { for (; i<dim1; ++i, v+=stride) { d[i]= (*v); }} else
00798             if (fac== -1) { for (; i<dim1; ++i, v+=stride) { d[i]= -( *v); }}
00799             else { for (; i<dim1; ++i, v+=stride) { d[i]= fac*( *v); }}
00800     }
00801     return *this;
00802 };
00803
00804 template<class T>
00805 wbarry<T>& wbarry<T>::setCol(size_t k, const wbvector<T> &v) {
00806
00807     if (rank()!=2) wblog(FL,
00808         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00809     if (k>=SIZE[1]) wblog(FL,
00810         "ERR %s() index out of bounds (%d: %s)",FCT,k+1,SSTR_(this));
00811
00812     size_t dim1=SIZE[0];
00813     if (v.len!=dim1) wblog(FL,"ERR size mismatch (%d/%d)",v.len,dim1);
00814     if (dim1) MEM_COPY<T>(data+k*dim1, dim1, v.data);
00815     return *this;
00816 };
00817
00818 template<class T>
00819 template<class T2>
00820 wbarry<T>& wbarry<T>::setCol(size_t k, const wbvector<T2> &v) {
00821
00822     if (rank()!=2) wblog(FL,
00823         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00824     if (k>=SIZE[1]) wblog(FL,
00825         "ERR %s() index out of bounds (%d: %s)",FCT,k+1,SSTR_(this));
00826
00827     size_t dim1=SIZE[0]; T* d=data+k*dim1;
00828     if (dim1) {
00829         size_t i=0, n=(v.len<dim1 ? v.len: dim1);
00830         for (; i<n; ++i) { d[i]=T(v.data[i]); }
00831         for (; i<dim1; ++i) { d[i]=T(0); }
00832     }
00833     return *this;
00834 };
00835
00836 template<class T>
00837 wbarry<T>& wbarry<T>::setRow(size_t k, const T* d0) {
00838
00839     if (rank()!=2) wblog(FL,
00840         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00841     if (k>=SIZE[0]) wblog(FL,
00842         "ERR %s() index out of bounds (%d; %s)",FCT,k+1,SSTR_(this));
00843
00844     size_t i=0, dim1=SIZE[0], dim2=SIZE[1];
00845     T* d=data+k;
00846     for (; i<dim2; i++, d+=dim1) d[i]=d0[i];
00847
00848     return *this;
00849 };
00850
00851 template<class T>
00852 wbvector<T>& wbarry<T>::colNorm2(wbvector<T> &a) const {
00853
00854     if (rank()!=2) wblog(FL,
00855         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00856
00857     size_t i,j, dim1=SIZE[0], dim2=SIZE[1];
00858     const T* d0=data; T x2;
00859
00860     a.init(dim2);

```

```

00861     for (j=0; j<dim2; j++, d0+=dim1) {
00862         for (x2=T(), i=0; i<dim1; i++) x2+=(CONJ(d0[i])*d0[i]);
00863         a[j]=x2;
00864     }
00865     return a;
00866 };
00867
00868 template<class T>
00869 T wbararray<T>::colNorm2(size_t k) const {
00870     size_t j=0, dim1=SIZE[0], dim2=SIZE[1];
00871     const T* d=data+k*dim1;
00872     T a=0;
00873
00874     if (rank()!=2) wblog(FL,
00875         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00876     if (k>=dim2) wblog(FL,
00877         "ERR %s() index out of bounds (%d/%d; %s)",FCT,k,dim2,SSTR_(this));
00878
00879     for (; j<dim2; ++j) { a+=CONJ(d[j])*d[j]; }
00880     return a;
00881 };
00882
00883
00884 template<class T>
00885 T wbararray<T>::NormalizeCol(size_t k, char tnorm, char qflag) {
00886     if (rank()!=2) wblog(FL,
00887         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00888
00889     const size_t dim1=SIZE[0], dim2=SIZE[1];
00890     if (k>=dim2) wblog(FL,
00891         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
00892
00893     T *d=data+dim1*k, x2=Wb::overlap(d,d,dim1,1,tnorm);
00894     x2=SQRT(x2);
00895
00896     if (x2!=0) Wb::timesRange(d,1/x2,dim1);
00897     else if (!qflag) wblog(FL,"ERR %s() got vector with norm 0!",FCT);
00898
00899     return x2;
00900 };
00901
00902
00903 template<class T>
00904 wbararray<T>& wbararray<T>::ColProject(
00905     size_t k1, size_t k2, char nflag, char tnorm
00906 ){
00907     if (rank()!=2) wblog(FL,
00908         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00909
00910     const size_t dim1=SIZE[0], dim2=SIZE[1];
00911     if (k1>=dim2 || k2>=dim2) wblog(FL,
00912         "ERR %s() index out of bounds (%d,%d/%d)",FCT,k1,k2,dim2);
00913     if (k1==k2) wblog(FL,"WRN %s() got k1=k2=%d",FCT,k1+1);
00914
00915     gs_project_range(
00916         data+dim1*k1, data+dim1*k2, dim1, 1,
00917         nflag, tnorm
00918     );
00919     return *this;
00920 };
00921
00922
00923 template<class T>
00924 bool wbararray<T>::isOrthogonalCol(size_t k0, char tnorm) const {
00925     if (rank()!=2) wblog(FL,
00926         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00927
00928     const size_t dim1=SIZE[0], dim2=SIZE[1];
00929     if (k0>=dim2) wblog(FL,
00930         "ERR %s() index out of bounds (%d/%d)",FCT,k0,dim2);
00931     if (!dim1) return 0;
00932
00933     const T *d=data, *d0=data+dim1*k0;
00934     T nrm2=Wb::overlap(d0,d0,dim1,1,tnorm);
00935
00936     for (size_t k=0; k<dim2; k++, d+=dim1) {
00937         if (k!=k0 && ABS(Wb::overlap(d,d0,dim1,1,tnorm)/nrm2)>1E-10)
00938             return 0;
00939     }
00940
00941     return 1;
00942 };
00943
00944
00945 template<class T>
00946 bool wbararray<T>::isOrthoCols(T *x2, char tnorm, T eps) const {
00947

```

```

00948     bool q=0;
00949     const size_t *s=SIZE.data;
00950
00951     if (SIZE.len==2 && s[1] && s[0]>=s[1]) {
00952         wbarry<T> E;
00953         Wb::MatProd(*this,*this,E, tnorm? 'T':'C');
00954
00955         if (x2)
00956             { q=E.isProptoId(*x2,eps); }
00957         else { q=E.isDiagMatrix(); }
00958     }
00959
00960     return q;
00961 };
00962
00963 template<class T>
00964 unsigned wbarry<T>::QRdecomp(
00965     const char *F, int L, wbarry<T> &Q, wbarry<T> &R, T eps
00966 ){
00967     if (isEmpty()) { Q.init(); R.init(); return 0; }
00968     if (rank()!=2 || !numel()) wblog(F_L,
00969         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00970
00971     size_t dim1=SIZE[0], dim2=SIZE[1], n=MIN(dim1,dim2);
00972
00973     if (dim1<=1 || dim2<=1) {
00974         Q.init(dim1,n); R.init(n,dim2);
00975
00976         if (!dim1 || !dim2) return n;
00977         if (dim1==1) { Q[0]=1; R=*this; return n; }
00978         if (dim2==1) { Q=*this;
00979             if ((R[0]=this->norm())!=0) { Q*=(T(1)/R[0]); }
00980             else { Q[0]=1; }
00981             return n;
00982         }
00983     }
00984
00985     if (allEqual(0)) {
00986         Q.init(dim1,1); R.init(1,dim2); Q[0]=1;
00987         return 1;
00988     }
00989
00990     size_t i,j,k=0;
00991     WbPERM P;
00992     wbarry<T> U(dim1,n);
00993
00994     R=*this;
00995
00996     for (k=0; k<n; ++k) {
00997         if (R.Householder(F_L,k,U.col(k),&P,eps)<0) break;
00998     }
00999
01000     if (!R.isFinite() || !U.isFinite()) wblog(FL,"ERR %s() ",FCT);
01001
01002     if (k<n) {
01003         if (!k) wblog(FL,"ERR %s() resulted in k=%d/%d !?",FCT,k,n);
01004         R.Resize(k,dim2); n=k;
01005     }
01006     else if (n<dim1) {
01007         R.Resize(n,dim2);
01008     }
01009
01010     Q.initIdentityB(dim1,n);
01011
01012     T zero=0, *x;
01013     const T *u=U.col(n-1);
01014
01015     for (k=n; k>0; u-=dim1) { --k; x=Q.data;
01016         for (j=0; j<n; ++j, x+=dim1) {
01017             Wb::householder(u+k,x+k,dim1-k,eps);
01018         }
01019     }
01020
01021     x=Q.data;
01022     for (j=0; j<n; ++j, x+=dim1) {
01023         for (i=0; i<dim1; ++i) { if (ABS(x[i])>eps) {
01024             if (x[i]<zero) { T *y=R.data+j;
01025                 for (i=0; i<dim1; ++i) { x[i]=-x[i]; }
01026                 for (k=0; k<dim2; ++k, y+=n) { y[0]=-y[0]; }
01027             }
01028             break;
01029         }}
01030     }
01031
01032     return n;
01033 };
01034

```

```

01035 template<class T>
01036 int wbaray<T>::Householder(
01037     const char *F, int L, size_t k,
01038     T *u, WPERM *P,
01039     T eps
01040 ){
01041     if (rank()!=2) wblog(F_L,
01042         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01043
01044     size_t i=0, j=0, k2=k, dim1=SIZE[0], dim2=SIZE[1];
01045     if (k>=MIN(dim1,dim2)) wblog(F_L,"ERR %s() "
01046         "index out of bounds (%d; %s)",FCT,k+1,SSTR_(this));
01047
01048     T nrm, x2=0., x0max=0., *x;
01049
01050     for (; i<k; ++i) u[i]=0.;
01051
01052     if (P) {
01053         if (!k) P->init(dim2); else
01054         if (P->len!=dim2) wblog(F_L,"ERR %s() "
01055             "got invalid P for (k=%d: %d/%d) !?",FCT,k+1,P->len,dim2);
01056         T x2max=0.;
01057
01058         for (x=data, j=0; j<dim2; ++j, x+=dim1) {
01059             if (!(P->data[j])) {
01060                 for (x2=0., i=k; i<dim1; ++i) { x2 += (CONJ(x[i])*x[i]); }
01061                 if (x2max<x2) { x2max=x2; k2=j; }
01062             }
01063             else {
01064                 if (P->data[j]>k) wblog(F_L,
01065                     "ERR %s() invalid P=[%s] (%d/%d/%d)",FCT,STR_(P),j,k,dim2);
01066                 if (x0max<(x2=ABS(x[P->data[j]-1]))) { x0max=x2; }
01067             }
01068             }; x2=x2max;
01069
01070             x=data+k2*dim1;
01071
01072             for (i=k; i<dim1; ++i) { u[i]=x[i]; };
01073         }
01074     else {
01075         for (x=data, j=0; j<k; ++j, x+=dim1) {
01076             if (x[j]<0) wblog(F_L,"ERR %s() x[%d]=%d",FCT,j,x[j]);
01077             if (x0max<(x2=ABS(x[j]))) { x0max=x2; }
01078         }
01079         for (x2=0., i=k; i<dim1; ++i) { u[i]=x[i];
01080             x2+=(CONJ(x[i])*x[i]);
01081         }
01082     }
01083
01084     eps*=(x0max<x2 ? x2 : x0max);
01085
01086     nrm=SQRT(x2); if (nrm<=eps) {
01087         if (!P && k+1<dim2) wblog(F_L,
01088             "ERR %s() check cols (%d/%d)",FCT,k,dim2);
01089         return -1;
01090     }
01091
01092     u[k] += (x[k]>0 ? +nrm : -nrm);
01093
01094     nrm=T(1)/SQRT( x2 - CONJ(x[k])*x[k] + CONJ(u[k])*u[k] );
01095     if (!isfinite(nrm)) wblog(F_L,"ERR %s() nrm=%g",FCT,nrm);
01096
01097     for (i=k; i<dim1; ++i) { u[i]*=nrm; }
01098
01099     if (P) {
01100         for (x=data, j=0; j<dim2; ++j, x+=dim1) {
01101             if (!(P->data[j])) {
01102                 Wb::householder(u+k,x+k,dim1-k,eps); }
01103             }
01104         P->data[k2]=(k+1);
01105     }
01106     else {
01107         for (x=data+k*dim1, j=k; j<dim2; ++j, x+=dim1) {
01108             Wb::householder(u+k,x+k,dim1-k,eps);
01109         }
01110     }
01111
01112     return k2;
01113 };
01114
01115 template<class T> inline
01116 void Wb::householder(const T *u, T *a, size_t n, const T& eps) {
01117
01118     size_t i=0;
01119     T ua=0.; for (; i<n; ++i) { ua += (CONJ(u[i])*a[i]); }
01120
01121     if (ua) { ua*=2.;

```



```

01122         for (i=0; i<n; ++i) { a[i] -= (u[i]*ua); }
01123         if (eps>0.) { ua=0.;
01124             for (i=0; i<n; ++i) ua += CONJ(a[i])*a[i];
01125             if (ua<eps) { for (i=0; i<n; ++i) a[i]=0.; }
01126         }
01127     }
01128 };
01129
01130 template<class T>
01131 unsigned wbarry<T>::eigTriDiag(
01132     const char *F, int L, wvector<T> &E, warray<T> *U, T eps
01133 ) const {
01134
01135     unsigned D;
01136
01137     if (!isMatrix()) wblog(F_L,
01138         "ERR %s() matrix expected (%s)",FCT,SSTR_(this));
01139
01140     D=dim(1); E.init(D); if (U) U->initIdentity(D);
01141     if (D<=1) {
01142         if (D) { E.data[0]=data[0]; }
01143         return 0;
01144     }
01145
01146     wvector<T> d_(D), e_(D);
01147
01148     int i,k,m,l; unsigned iter=0;
01149     T s,r,p,g,f,da,ea,c,b, *d=d_.data, *e=e_.data;
01150
01151     for (l=D-1, i=0; i<l; ++i) {
01152         if (ABS(data[(i+1)+i*D] - data[i+(i+1)*D])>eps) wblog(F_L,
01153             "ERR %s() got non-symmetric matrix\nM(%d,%d+1): %.4g / %.4g !?",
01154             FCT,i+1,i+2, double(data[(i+1)+i*D]), double(data[i+(i+1)*D]));
01155         if (i>1 && (ABS(data[i])>eps || ABS(data[i*D])>eps)) wblog(F_L,
01156             "ERR %s() got non-triangular matrix\nM(%d,1): %.4g !?",
01157             FCT,i+1, double(data[i]));
01158
01159         d[i]=data[i*D+i]; e[i]=0.5*(data[i*D+i+1] + data[(i+1)*D+i]);
01160     }; d[i]=data[i*D+i];
01161
01162     for (l=0; l<int(D); ++l) { iter=0;
01163         do {
01164             for (m=l; m<int(D-1); ++m) {
01165                 ea=ABS(e[m]); da=ABS(d[m]) + ABS(d[m+1]);
01166                 if (T(ea+da)==da) break;
01167             }
01168             if (m!=l) {
01169                 if (++iter > 30) wblog(F_L,
01170                     "ERR %s() too many iterations (iter=%d) !?",FCT,iter);
01171                 g=(d[l+1] - d[l]) / (T(2)*e[l]);
01172                 r=NR::pythag(g,T(1));
01173                 g=d[m]-d[l]+e[l]/(g+NR::sign(r,g));
01174                 s=c=1; p=0;
01175
01176                 for (i=m-1; i>=1; --i) {
01177                     f=s*e[i];
01178                     b=c*e[i]; e[i+1]=(r=NR::pythag(f,g));
01179                     if (r==0) { d[i+1]-=p; e[m]=0; break; }
01180
01181                     s=f/r;
01182                     c=g/r;
01183                     g=d[i+1]-p;
01184                     r=(d[i]-g)*s + 2*c*b;
01185                     d[i+1]=g + (p=s*r);
01186                     g=c*r-b;
01187
01188                     if (U) { warray<T> &z=(*U);
01189                         for (k=0; k<int(D); ++k) { f=z(k,i+1);
01190                             z(k,i+1) = s*z(k,i) + c*f;
01191                             z(k,i) = c*z(k,i) - s*f;
01192                         }
01193                     }
01194                 }
01195
01196                 if (r==0 && i>=1) continue;
01197                 d[l]-=p; e[l]=g; e[m]=0;
01198             }
01199         } while (m!=l);
01200     }
01201
01202     wperm P;
01203     d_.sort(E,P);
01204     if (U) U->ColPermute(P);
01205
01206     return 0;
01207 };
01208

```

```

01209 template<class T>
01210 wbararray<T>& wbararray<T>::ColPermute(const wperm &P, char iflag){
01211
01212     if (rank()!=2) wblog(FL,
01213         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01214     if (P.len!=SIZE[1]) wblog(FL,
01215         "ERR %s() size mismatch (%d/%d)",FCT,P.len,SIZE[1]);
01216
01217     size_t i=0, dim1=SIZE[0], dim2=SIZE[1];
01218     wbararray<T> X(*this);
01219
01220     for (; i<P.len; i++) {
01221         if (P[i]>=dim2) wblog(FL,
01222             "ERR %s() index out of bounds (%d/%d)",FCT,P[i],dim2);
01223         if (iflag==0)
01224             MEM_COPY<T>(data+i*dim1, dim1, X.data+P[i]*dim1);
01225         else MEM_COPY<T>(data+P[i]*dim1, dim1, X.data+i*dim1);
01226     }
01227
01228     return *this;
01229 };
01230
01231 template<class T>
01232 wbararray<T>& wbararray<T>::SignCol(size_t k, const T *d0, char tnorm) {
01233
01234     if (rank()!=2) wblog(FL,
01235         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01236
01237     const size_t dim1=SIZE[0], dim2=SIZE[1];
01238     if (k>=dim2) wblog(FL,
01239         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01240
01241     T *d=data+dim1*k, x=Wb::overlap(d0,d,dim1,1,tnorm);
01242     if (x<0) {
01243         for (size_t i=0; i<dim1; i++) { d[i]=-d[i]; }
01244     }
01245
01246     return *this;
01247 };
01248
01249 template<class T>
01250 wbararray<T>& wbararray<T>::FlipSignCol(size_t k) {
01251
01252     if (rank()!=2) wblog(FL,
01253         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01254
01255     const size_t dim1=SIZE[0], dim2=SIZE[1];
01256     if (k>=dim2) wblog(FL,
01257         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01258
01259     T *d=data+dim1*k;
01260     for (size_t i=0; i<dim1; i++) { d[i]=-d[i]; }
01261
01262     return *this;
01263 };
01264
01265 template<class T>
01266 wbararray<T>& wbararray<T>::SignConventionCol(
01267     size_t k0,
01268     double deps
01269 ){
01270
01271     if (rank()!=2) wblog(FL,
01272         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01273
01274     const size_t dim1=SIZE[0], dim2=SIZE[1];
01275     size_t i,k,k1,k2; T *d, eps=
01276         ((typeid(T)!=typeid(double) && typeid(T)!=typeid(float)) ? 0 : deps);
01277
01278     if (k>=dim2) wblog(FL,
01279         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01280
01281     if (int(k0)>=0) { k1=k0; k2=k0+1; }
01282     else { k1=0; k2=dim2; }
01283
01284     for (k=k1; k<k2; k++) { d=data+dim1*k;
01285         for (i=0; i<dim1; i++) {
01286             if (d[i]> eps) break;
01287             if (d[i]<=-eps) {
01288                 for (i=0; i<dim1; i++) { d[i]=-d[i]; }
01289                 break;
01290             }
01291         }
01292     }
01293
01294     return *this;
01295 };

```

```

01296
01297 template<class T>
01298 wbarry<T>& wbarry<T>::NormalizeCols(
01299     const char *F, int L, double *amin, double *amax, char tnorm
01300 ){
01301     if (!SIZE.len) return *this;
01302
01303     size_t i,j,m, n=SIZE[0];
01304     T *d=data, z=0; double a;
01305     wbvector<T> nn;
01306
01307     if (SIZE.len>1) for (m=SIZE[1], i=2; i<SIZE.len; i++) m*=SIZE[i];
01308     else m=1;
01309
01310     nn.init(m);
01311
01312     for (j=0; j<m; j++, d+=n, z=0) {
01313         z=Wb::overlap(d,d,n,1,tnorm); z=SQRT(z);
01314
01315         a=ABS(z); if (a==0) {
01316             wblog(F,L,"ERR |sum^2| of column returns %.3g!%s",a,
01317                 tnorm && (typeid(T)==typeid(wbcomplex)) ? " (using t-norm)":"");
01318         }
01319         nn[j]=z;
01320     }
01321
01322     if (amin) {
01323         a=nn.aMin(); if ((*amin)>0 && a<(*amin))
01324             wblog(F,L,"WRN %s() nMin=%.3g (%.3g)",FCT,a,*amin);
01325         (*amin)=a;
01326     }
01327
01328     if (amax) {
01329         a=nn.aMax(); if ((*amax)>0 && a>(*amax))
01330             wblog(F,L,"WRN %s() nMax=%.3g (%.3g)",FCT,a,*amax);
01331         (*amax)=a;
01332     }
01333
01334     for (d=data, j=0; j<m; j++, d+=n)
01335         for (z=T(1)/nn[j], i=0; i<n; i++) { d[i]*=z; }
01336
01337     return *this;
01338 };
01339
01339 template<class T>
01340 wbarry<T>& wbarry<T>::OrthoNormalizeCols(
01341     const char *F, int L,
01342     char tnorm,
01343     char qflag,
01344     double eps,
01345     unsigned np
01346 ){
01347     if (!SIZE.len) return *this;
01348
01349     size_t i,j,k,l,p,m, n=SIZE[0];
01350     T z, *d=0, *v=data; double a;
01351
01352     char cflag=(typeid(T)==typeid(wbcomplex));
01353     char xflag=(qflag=='x' || qflag=='X');
01354
01355     if (SIZE.len<2) m=1; else
01356         for (m=SIZE[1], i=2; i<SIZE.len; i++) m*=SIZE[i];
01357
01358     if (!m || !n) return *this;
01359
01360     for (l=k=0; k<m; ++k, ++l, v+=n) {
01361         for (p=0; p<np; ++p) {
01362             for (d=data, j=0; j<l; ++j, d+=n) {
01363                 z=Wb::overlap(d,v,n,1,tnorm); if (z!=T(0)) {
01364                     for (i=0; i<n; ++i) v[i]-=z*d[i]; }
01365             }
01366         }
01367
01368         z=Wb::overlap(v,v,n,1,tnorm);
01369         a=ABS(z);
01370
01371         if (a>eps) { a=SQRT(1/a);
01372             if (xflag && l<k)
01373                 { for (i=0; i<n; ++i) d[i]=v[i]*a; }
01374             else { for (i=0; i<n; ++i) v[i]*=a; }
01375         }
01376         else {
01377             if (xflag) --l;
01378             else if (qflag) {
01379                 for (i=0; i<n; ++i) v[i]=0;
01380             }
01381             else {
01382                 wblog(F,L,"ERR |sum^2| of column returns %.3g!%s",

```

```

01383         a, tnorm && cflag ? " (using t-norm)" : "");
01384     }
01385 }
01386 }
01387
01388 if (xflag && l<k) {
01389     if (SIZE.len!=2) wblog(F_L,"ERR %s() qflag='%c' must be "
01390         "used with rank-2 tensors (%d)",FCT,qflag,qflag,SIZE.len);
01391     if (!l) wblog(F_L,"ERR %s() got all null-vectors (%g)",FCT,k);
01392     SIZE[1]=1;
01393 }
01394
01395 return *this;
01396 };
01397
01398 template<class T>
01399 T wbararray<T>::norm2Cols(long j1, long j2) const {
01400
01401     T x2=0; long i,j, dim1,dim2;
01402     if (SIZE.len!=2) wblog(F_L,
01403         "ERR %s() got size %s array",FCT,SSTR_(this));
01404     dim1=SIZE[0]; dim2=SIZE[1];
01405
01406     if (j1<0) { j1+=dim2; }
01407     if (j2<0) { j2+=dim2; }; if (j1>j2 || !dim1) { return x2; }
01408     if (j1<0 || j2>=dim2) wblog(F_L,
01409         "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,j1,j2,dim2);
01410
01411     for (j=j1; j<=j2; ++j)
01412         for (i=0; i<dim1; ++i) { x2+=NORM2(data[i+j*dim1]); }
01413     return x2;
01414 };
01415
01416 template<class T>
01417 T wbararray<T>::norm2Recs(long i1, long i2) const {
01418
01419     T x2=0; long i,j,dim1,dim2;
01420     if (SIZE.len!=2) wblog(F_L,
01421         "ERR %s() got size %s array",FCT,SSTR_(this));
01422     dim1=SIZE[0]; dim2=SIZE[1];
01423
01424     if (i1<0) { i1+=dim1; }
01425     if (i2<0) { i2+=dim1; }; if (i1>i2 || !dim2) { return x2; }
01426     if (i1<0 || i2>=dim1) wblog(F_L,
01427         "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,i1,i2,dim1);
01428
01429     for (j=0; j<dim2; ++j)
01430         for (i=i1; i<=i2; ++i) { x2+=NORM2(data[i+j*dim1]); }
01431     return x2;
01432 };
01433
01434 template<class T>
01435 bool wbararray<T>::isSym_aux(
01436     const char *F, int L, const char *fct,
01437     const wbararray<T> &B, double eps, double* xref,
01438     const char symflag,
01439     const char lflag
01440 ) const {
01441
01442     WBINDEX S1,S2;
01443     size_t i,j,k,dim1,dim2, r=SIZE.len, r2=r/2;
01444     double x;
01445
01446     const wbararray<T> &A = (*this);
01447     bool issame = (&A==&B);
01448     wbararray<T> A2,B2;
01449
01450     if (r%2 || r!=B.SIZE.len) {
01451         if (lflag) sprintf(str,
01452             "%s %s() only applies to even-rank objects (%ld;%ld)",
01453             shortFL(F,L),fct, r, B.SIZE.len);
01454         return 0;
01455     }
01456     if (isEmpty() && B.isEmpty()) return 1;
01457
01458     dim1=dim2=1;
01459     for (i=0; i<r2; i++) { j=i+r2; dim1*=SIZE[i]; dim2*=SIZE[j];
01460         if (SIZE[i]!=B.SIZE[j] || SIZE[j]!=B.SIZE[i]) {
01461             if (lflag) sprintf(str,
01462                 "%s %s() size mismatch [%s; %s].", SHORT_FL, fct,
01463                 SSTR_(this), SSTR(B));
01464             return 0;
01465         }
01466     }
01467
01468     A.groupIndices_DREF(A2,r2,r2);
01469     B.groupIndices_DREF(B2,r2,r2);

```

```

01470
01471     if (xref) *xref=0;
01472     if (eps<0) {
01473         eps=-eps; x=A2.aMax(); if (x>1) eps*=x;
01474     }
01475
01476     if (symflag=='s') {
01477         if (eps==0) {
01478             for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01479                 for (j=k; j<dim2; j++) {
01480                     if (A2(i,j)!=CONJ(B2(j,i))) {
01481                         if (xref) {
01482                             x=ABS(A2(i,j)-CONJ(B2(j,i)));
01483                             *xref = MAX(*xref,x);
01484                         }
01485                         else return 0;
01486                     }
01487                 }}
01488             }
01489         else {
01490             for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01491                 for (j=k; j<dim2; j++) {
01492                     x=ABS(A2(i,j)-CONJ(B2(j,i)));
01493                     if (xref) *xref=MAX(*xref,x); else if (x>eps) return 0;
01494                 }}
01495             }
01496         }
01497     else if (symflag=='a') {
01498         if (eps==0) {
01499             for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01500                 for (j=k; j<dim2; j++) {
01501                     if (A2(i,j)!=-CONJ(B2(j,i))) {
01502                         if (xref) {
01503                             x = ABS(A2(i,j) + CONJ(B2(j,i)));
01504                             *xref = MAX(*xref,x);
01505                         }
01506                         else return 0;
01507                     }
01508                 }}
01509             }
01510         else {
01511             for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01512                 for (j=k; j<dim2; j++) {
01513                     x = ABS(A2(i,j) + CONJ(B2(j,i)));
01514                     if (x>eps) { if (xref) *xref=MAX(*xref,x); else return 0; }
01515                 }}
01516             }
01517         }
01518     else wblog(FL,"ERR invalid symflag=%c<%d>",symflag,symflag);
01519
01520     if (xref && (*xref)>eps) return 0;
01521
01522     return 1;
01523 }
01524
01525 template<class T>
01526 char wbarry<T>::hasGroupSize(size_t s1, size_t s2) const {
01527
01528     size_t i,s=1,s0=1;
01529
01530     for (i=0; i<SIZE.len; i++) {
01531         s*=SIZE[i]; if (s0==s1 && (!s || s!=s1)) break;
01532         s0=s;
01533     }
01534     for (s=1; i<SIZE.len; i++) s*=SIZE[i];
01535
01536     return ((s0!=s1 || s!=s2) ? 0 : 1);
01537 };
01538
01539 template<class T> inline
01540 bool wbarry<T>::isComplex() const { return 0; };
01541
01542 template<> inline
01543 bool wbarry<wbcomplex>::isComplex() const {
01544     for (size_t s=numel(), i=0; i<s; i++)
01545         if (data[i].i!=0.) return 1;
01546
01547     return 0;
01548 };
01549
01550 template<class T> inline
01551 bool wbarry<T>::isZero(double eps, char bflag) const {
01552
01553     size_t i, s=numel(); if (!s) { return 1; }
01554
01555     if (eps==0) {
01556         for (i=0; i<s; ++i) { if (data[i]!=0.) return 0; }

```

```

01557     }
01558     else {
01559         if (bflag) {
01560             double e2=eps*eps, n2=0.;
01561             for (i=0; i<s; ++i) {
01562                 n2+=NORM2(data[i]); if (n2>e2) { return 0; }
01563             }
01564         }
01565         else {
01566             for (i=0; i<s; ++i) { if (ABS(data[i])>eps) { return 0; }}
01567         }
01568     }
01569     return 1;
01570 }
01571 }
01572
01573 template<class T> inline
01574 size_t wbararray<T>::nnz(const T eps, WBINDEX *I) const {
01575     size_t i,n, s=numel(); T a;
01576     for (i=n=0; i<s; ++i) {
01577         a=data[i]; if (a<0) { a=-a; };
01578         if (a>eps) { ++n; }
01579     }
01580     if (I) {
01581         I->init(n);
01582         for (i=n=0; i<s; ++i) {
01583             a=data[i]; if (a<0) a=-a;
01584             if (a>eps) I->data[n++]=i;
01585         }
01586     }
01587     return n;
01588 }
01589
01590 template<class T> inline
01591 void wbararray<T>::SetRef (
01592     const widx_t *I, size_t len, const T* d0
01593 ){
01594     size_t i,k,s, S=numel(), r=SIZE.len, l=len-1;
01595     size_t nz=r-len;
01596     if (len>r || r==0 || (len && data==NULL)) wblog(FL,
01597         "ERR %s() index out of bounds ([%s]) for %s !?",FCT,
01598         (WBINDEX(len,I)+1).toStr().data, SSTR_(this));
01599     for (i=0; i<len; i++) if (I[i]>=SIZE[i+nz]) wblog(FL,
01600         "ERR %s() index out of bounds (%s; %s)",FCT,
01601         (WBINDEX(len,I)+1).toStr().data, SSTR_(this));
01602     if (!nz || !len) wblog(FL,"ERR invalid reference (len=%d/%d)",len,r);
01603     for (s=SIZE[0], i=1; i<nz; i++) s*=SIZE[i];
01604     for (k=I[l], i=l-1; i<l; i--) { k = k*SIZE[i+nz] + I[i]; }
01605     k*=s;
01606     if (k+s>S || d0==NULL) wblog(FL,
01607         "ERR index out of bounds ( %d+%d / %d; 0x%lX)\n"
01608         "having I=[%s] for %s array",k,s,S,d0,
01609         (WBINDEX(len,I)+1).toStr().data, SSTR_(this)
01610     );
01611     MEM_COPY<T>(data+k,s,d0);
01612 }
01613
01614 template<class T> inline
01615 wbararray<T>& wbararray<T>::Squeeze() {
01616     size_t r=SIZE.len; if (r>1) {
01617         unsigned i=0, l=0; size_t *s=SIZE.data;
01618         for (; i<r; ++i) { if (s[i]!=1) {
01619             if (l!=i) { s[l]=s[i]; }
01620             ++l;
01621         }}
01622         if (l<r) {
01623             SIZE.len=(l? l:1);
01624         }
01625     }
01626     return *this;
01627 }
01628
01629 template<class T> inline
01630 int wbararray<T>::skipSingletons(const char *F, int L,
01631     unsigned r2
01632 ) {
01633     int m=0;

```

```

01644     unsigned i, r=SIZE.len;
01645
01646     if (r2==r || (!r && r2<=0)) { return m; }
01647     if (!r) { return (m=-r2); }
01648
01649     if (int(r2)<=0) { r2=-r2; if (!r2) { r2=1; }
01650     if (r2<r) {
01651         for (i=r-1; i>=r2; --i) { if (SIZE[i]!=1) break; }
01652
01653         m=r-1-i;
01654         if (m) { SIZE.Shorten2(i+1); }
01655     }
01656     else if (r2<r) { int e=0;
01657         for (i=r2; i<r; ++i) { if (SIZE[i]!=1) { ++e; if (F) wblog(F_L,
01658             "ERR %s() non-singleton at %d/%d in [%s]",FCT,i+1,r2,STR(SIZE));
01659         }
01660         if (e) { m=-e; }
01661         else { m=r-r2; SIZE.Shorten2(r2); }
01662     }
01663     else { m=-r2;
01664         if (F) wblog(F_L,"ERR %s() rank out of bounds (%d/%d)",FCT,r2,r);
01665     }
01666     return m;
01667 };
01668
01669 template<class T> inline
01670 wbararray<T>& wbararray<T>::appendSingletons(
01671     const char *F, int L, unsigned r, unsigned m) {
01672
01673     unsigned i, l=SIZE.len;
01674     if (l<r) {
01675         SIZE.Resize(r); for (i=l; i<r; ++i) { SIZE[i]=1; }
01676         if (!l) {
01677             wblog(F_L,"WRN adding singletons to empty space (%d/%d)",r,l);
01678             SIZE[0]=0;
01679         }
01680     }
01681     else if (l>r) {
01682         size_t *s=SIZE.data;
01683
01684         if (l==2 && r==1 && s[0]==1) {
01685             s[0]=s[1]; SIZE.len=1;
01686         }
01687         else {
01688             if (r>2) { r+=m; } else
01689             if (!r) wblog(F_L,"ERR %s() got r=%d+%d",FCT,r,m);
01690
01691             if (l>r) {
01692                 for (i=r; i<l; ++i) { if (s[i]!=1) { wblog(F_L,
01693                     "ERR %s() got r=%d->%d for %s",FCT,l,r,SSTR_(this));
01694                 }
01695             }
01696             SIZE.len=r;
01697         }
01698     }
01699 }
01700
01701 return *this;
01702 };
01703
01704 template<class T> inline
01705 void wbararray<T>::prependSingletons(unsigned r) {
01706
01707     if (!SIZE.len) wblog(F_L,"ERR %s() called on empty array",FCT);
01708
01709     if (r>SIZE.len) {
01710         size_t i, l=SIZE.len-1, m=r-SIZE.len;
01711         SIZE.Resize(r);
01712         for (i=l; i<r; --i) SIZE[i+m]=SIZE[i];
01713         for (i=0; i<m; ++i) SIZE[i]=1;
01714     }
01715     else if (r<SIZE.len) wblog(F_L,
01716         "ERR %s() invalid extended rank !? (%d->%d)",FCT,SIZE.len,r);
01717 };
01718
01719 template<class T> inline
01720 wbararray<T>& wbararray<T>::ExpandOM(const char *F, int L, unsigned r0,
01721     const wvector<unsigned> &S,
01722     const unsigned *sx
01723 ){
01724     if (!SIZE) {
01725         if (!S && !sx) { return *this; }
01726         wblog(F_L,"ERR %s() got empty vector with S=%s",FCT,STR(S));
01727     }
01728     if (!S) { if (!sx) {
01729         if (int(r0)<=0 || r0==SIZE.len) { return *this; }
01730         wblog(F_L,"ERR %s() got empty S=[%s] for %s",FCT,STR(S),SSTR_(this));

```

```

01731     }
01732     if (sx && S.isEqual(sx)) { sx=NULL; }
01733
01734     unsigned i=0, l=SIZE.len;
01735     size_t M=1;
01736
01737     if (int(r0)<=0) { r0=l-S.len;
01738         if (int(r0)<2) wblog(FL,
01739             "ERR %s() requesting %d/%d OM indices",FCT,S.len,l);
01740     }
01741     else if (r0+S.len<l) wblog(F_L,
01742         "ERR %s() rank out of bounds %d+%d / %d",FCT,r0,S.len,l);
01743
01744     M=( r0<l ? numOM(r0) : 1 );
01745
01746     if (M!=S.prod()) wblog(FL,
01747         "ERR %s() got OM size mismatch %s @ %d <> %s",FCT,
01748         SSTR_(this), r0, SSTR(S));
01749
01750     if (r0<=2 && M>1) wblog(FL,"ERR %s() "
01751         "got OM for rank-%d array %s (l=%d)",FCT,r0,SSTR_(this), S.len);
01752
01753     if (sx) { M=1;
01754         for (; i<S.len; ++i) { M*=sx[i]; if (sx[i]!=S[i]) {
01755             if (sx[i]<S[i]) { wblog(FL,"ERR %s() "
01756                 "cannot zero-pad to smaller size\nS=[%s] -> [%s]",
01757                 FCT,STR(S),STR(wbvector<unsigned>(S.len,sx,'r')));
01758             }
01759         }
01760         if (M>1 && r0<=2) { wblog(FL,"ERR %s() "
01761             "got OM for rank-%d array %s (M=%ld)",FCT,r0,SSTR_(this),M);
01762         }
01763     }
01764
01765     if (l!=r0+1 && l!=r0+S.len) { unsigned j=0;
01766         for (i=r0; i<l && j<S.len; ++i, ++j) { if (SIZE[i]!=S[j]) {
01767             while (S[j]==1 && ++j<S.len) {
01768                 if (SIZE[i]==S[j]) break;
01769             }
01770         }
01771         for (; i<l; ++i) { if (SIZE[i]!=1) break; }
01772         for (; j<S.len; ++j) { if (S[j]!=1) break; }
01773
01774         if (i<l || j<S.len) { wblog(FL,
01775             "WRN %s() got OM size reshape %s @ %d <> %s",
01776             FCT, SSTR_(this), r0, SSTR(S));
01777         }
01778     }
01779
01780     wbvector<size_t> SX; SIZE.resize(r0+S.len,SX);
01781     for (i=0; i<S.len; ++i) { SX[r0+i]=S[i]; }
01782     Reshape(SX);
01783
01784     if (sx) {
01785         for (i=0; i<S.len; ++i) { SX[r0+i]=sx[i]; }
01786         Resize(SX);
01787     }
01788
01789     return *this;
01790 };
01791
01792 template<class T> inline
01793 int wbararray<T>::ExpandOM(
01794     const char *F, int L, wbararray<T> &B,
01795     unsigned r
01796 ) {
01797
01798     int rval=0, eq=0;
01799
01800     if (isEmpty() || B.isEmpty()) { return (rval=-1); }
01801
01802     if (!r) { unsigned R=MAX(SIZE.len,B.SIZE.len);
01803         if (!isScalar() || !B.isScalar() || R>2) { wblog(FL,
01804             "ERR %s() unexpected empty/scalar QSpaces (%s / %s, r=%d)",
01805             FCT,SSTR_(this),SSTR(B),R);
01806         }
01807         if ( SIZE.len<R) { appendSingletons(R); }
01808         if (B.SIZE.len<R) { B.appendSingletons(R); }
01809         return rval;
01810     }
01811
01812     if ((eq=SIZE.isEqual(B.SIZE))) {
01813         #ifndef WB_SKIP_ASSERT
01814         if (r<=2 && r<SIZE.len) { unsigned i=r;
01815             for (; i<SIZE.len; ++i) { if (SIZE[i]!=1) { wblog(FL,
01816                 "ERR %s() got OM space %s for rank r=%d",FCT,SSTR_(this),r);
01817             }

```



```

01818     }
01819     #endif
01820     return rval;
01821 }
01822
01823 if (SIZE.len<r || B.SIZE.len<r) wblog(F_L,"ERR %s() "
01824 "rank out of bounds (r=%d/%d/%d)",FCT,r,SIZE.len,B.SIZE.len);
01825 if (memcmp(SIZE.data,B.SIZE.data,r*sizeof(SIZE[0])) wblog(F_L,
01826 "ERR %s() size mismatch %s / %s (r=%d)",FCT,SSTR_(this),SSTR(B),r);
01827
01828 unsigned i, R=MAX(SIZE.len,B.SIZE.len);
01829
01830 if (SIZE.len!=B.SIZE.len) {
01831     if ( SIZE.len==r) { appendSingletons(R); rval|=1; } else
01832     if (B.SIZE.len==r) { B.appendSingletons(R); rval|=2; }
01833     else {
01834         wblog(F_L,"ERR %s() rank mismatch (r=%d/%d)",
01835             FCT,SIZE.len,B.SIZE.len);
01836     }
01837     eq=SIZE.isEqual(B.SIZE);
01838 }
01839
01840 if (r<3) {
01841     for (i=r; i<R; ++i) { if ( SIZE[i]!=1) { break; }}
01842     if (i<R || !eq) wblog(FL,"ERR %s() "
01843 "got OM space %s / %s for rank r=%d",FCT,SSTR_(this),SSTR(B),r);
01844 }
01845 else if (!eq) {
01846     wvector<size_t> S(SIZE); unsigned q=0;
01847     for (i=r; i<SIZE.len; ++i) {
01848         if (S[i]<B.SIZE[i]) { S[i]=B.SIZE[i]; ++q; }}
01849     if (q) { this->Resize(S); rval|=4; }
01850
01851     S=B.SIZE; q=0;
01852     for (i=r; i<SIZE.len; ++i) {
01853         if (S[i]<SIZE[i]) { S[i]=SIZE[i]; ++q; }}
01854     if (q) { B.Resize(S); rval|=8; }
01855 }
01856
01857 return rval;
01858 };
01859
01860 template<class T>
01861 template<class TI> inline
01862 int wbararray<T>::aMax(T &x, TI &k) const {
01863     int e=0; size_t n=numel(); x=0; k=0;
01864     if (n) { size_t i=0; T a;
01865         for (; i<n; ++i) { if (x<(a=ABS(data[i]))) { x=a; k=i; }}
01866     }
01867     else { k=-1; e=1; }
01868     return e;
01869 };
01870
01871 template<class T>
01872 template<class TI> inline
01873 T wbararray<T>::aMax(TI &i_, TI &j_, const wbararray *B) const {
01874     T x=0; i_=j_=0;
01875     if (SIZE.len!=2) wblog(FL,
01876 "ERR %s() invalid usage (s=%s)",FCT,SSTR_(this));
01877
01878     if (!B) {
01879         size_t k; int e=aMax(x,k);
01880         if (e)
01881             { wblog(FL, "WRN %s() got empty array (e=%d)",FCT,e); }
01882         else { i_=k*SIZE[0]; j_=k/SIZE[0]; }
01883         return x;
01884     }
01885
01886     if (B->SIZE.len!=2) wblog(FL,
01887 "ERR %s() invalid usage (s=%s)",FCT,SSTR_(B));
01888
01889     TI na=SIZE[0], nb=B->SIZE[0],
01890     ma=SIZE[1], mb=B->SIZE[1], n=MIN(na,nb), m=MIN(ma,mb), i,j;
01891
01892     if (n && m) { T a, b;
01893         for (i=0; i<n; ++i) {
01894             for (j=0; j<m; ++j) {
01895                 a=ABS( data[i+j*na]);
01896                 b=ABS(B->data[i+j*nb]); if (a<b) { a=b; }
01897                 if (x<a) { x=a; i_=i; j_=j; }
01898             }
01899         }
01900     } else { wblog(FL, "WRN %s() got empty array (%dx%d)",FCT,n,m); }
01901
01902     return x;
01903 };
01904

```

```

01905 template<class T> inline
01906 double wbararray<T>::maxDiff (const wbararray<T> &B) const {
01907     size_t i,s=numel();
01908     double dmax=0;
01909
01910     if (SIZE!=B.SIZE) {
01911         sprintf(str,"%s:%d data SIZE mismatch ([%s, %s])",
01912             FL,SSTR_(this),SSTR(B));
01913         return NAN;
01914     }
01915
01916     for (i=0; i<s; i++)
01917         dmax=MAX(dmax, ABS(data[i]-B.data[i]));
01918
01919     return dmax;
01920 };
01921
01922 template<class T> inline
01923 void wbararray<T>::TimesEl(const wbararray<T> &B, char conj) {
01924
01925     if (!sameSize(B)) wblog(FL,
01926         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
01927
01928     Wb::TimesElRange(data,B.data,numel(),conj);
01929 };
01930
01931 template<class T>
01932 wbararray<T>& wbararray<T>::timesEl(
01933     const wbararray &B, wbararray &C, T bfac, T cfac, char conj) const {
01934
01935     if (!sameSize(B)) wblog(FL,
01936         "ERR %s() size mismatch\n%s <> %s",FCT,SSTR_(this),SSTR(B));
01937
01938     if (!cfac || C.isEmpty()) {
01939         if (bfac) { C=*this;
01940             Wb::TimesElRange(C.data,B.data,numel(),conj);
01941             if (bfac!=T(1)) { C*=bfac; }
01942         }
01943         else C.init(SIZE);
01944     }
01945     else if (!sameSize(C)) { wblog(FL,
01946         "ERR %s() size mismatch\n%s <> %s",FCT,SSTR_(this),SSTR(C)); }
01947     else {
01948         if (cfac!=T(1)) { C*=cfac; }
01949         if (bfac) {
01950             Wb::timesElRange_add(C.data, data,B.data,numel(),bfac,conj);
01951         }
01952     }
01953     return C;
01954 };
01955
01956 template<class T>
01957 wbararray<T>& wbararray<T>::timesEl_OM(
01958     unsigned r, const wbararray &B, wbararray &C, char conj
01959 ) const {
01960
01961     if (!r || r>SIZE.len) wblog(FL,
01962         "ERR %s() invalid rank (r=%d/%d)",FCT,r,SIZE.len);
01963     if (!sameSize(B)) wblog(FL,
01964         "ERR %s() size mismatch %s / %s",FCT,SSTR_(this),SSTR(B));
01965
01966     unsigned i=0; size_t N=1, M=1;
01967     const size_t *sa=SIZE.data, *sb=B.SIZE.data;
01968
01969     bool self=( &C==this || &C==&B || C.data==data || C.data==B.data);
01970
01971     for (; i<r; ++i) {
01972         if (sa[i]!=sb[i]) wblog(FL,"ERR %s() "
01973             "size mismatch %s / %s (i=%d/%d)",FCT,SSTR_(this),SSTR(B),i+1,r);
01974         N*=sa[i];
01975     }
01976     for (; i<SIZE.len; ++i) { M*=sa[i]; }
01977
01978     if (!self) {
01979         C.init_bare(wbvector<size_t>(r,SIZE.data,'r'));
01980     }
01981
01982     Wb::timesElRange_OM(C.data, data,B.data,N,M,conj);
01983
01984     if (self)
01985         { C.SIZE.len=r; }
01986
01987     return C;
01988 };
01989
01990 template<class T> inline
01991 T wbararray<T>::dotProd(const wbararray<T> &B, char conj) const {

```

```

01992     T x=T(0);
01993     if (!sameSize(B)) wblog(FL,
01994         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
01995
01996     return Wb::dotProd(data,B.data,numel(),x,conj);
01997 };
01998
01999 template<class T> inline
02000 T wbararray<T>::weightedAvg(const wbararray<T> &B) const {
02001     size_t i,s=numel(); T w,x=0,n2=0;
02002
02003     if (!sameSize(B)) wblog(FL,
02004         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
02005
02006     for (i=0; i<s; i++) { w=B.data[i]; x+=(data[i]*w); n2+=w*CONJ(w); }
02007
02008     if (n2<=0) wblog(FL,
02009         "ERR weighted avg with total weight %.4g",n2);
02010     return x/std::sqrt(n2);
02011 };
02012
02013 template<class T>
02014 wbararray<T>& wbararray<T>::tensorProd(
02015     const wbararray<T> &B_, wbararray<T> &C,
02016     const char aflag0, const char bflag0,
02017     const char kflag
02018 ) const {
02019
02020     if (this==&C || &B_==&C) {
02021         wbararray<T> X; tensorProd(B_,X,aflag0,bflag0,kflag);
02022         return X.save2(C);
02023     }
02024
02025     wbararray<T> A,B;
02026     opFlags<T> aflag(aflag0), bflag(bflag0);
02027     size_t ra=SIZE.len, rb=B_.SIZE.len;
02028
02029     aflag.applyOrRef(FL,*this,A);
02030     bflag.applyOrRef(FL,B_, B);
02031
02032     if (ra!=rb) {
02033         if (ra>2 || rb>2) wblog(FL,
02034             "ERR %s() rank mismatch (r=%d/%d)",FCT,ra,rb);
02035         if (!ra || !rb) { return C.init(); }
02036
02037         if (ra==1) { A.ExpandDiagonal(); ra= SIZE.len; } else
02038         if (rb==1) { B.ExpandDiagonal(); rb=B.SIZE.len; }
02039
02040         if (ra!=rb) wblog(FL,
02041             "ERR %s() rank mismatch (%d/%d)",FCT,ra,rb);
02042     }
02043
02044     const size_t r=A.SIZE.len, l=r-1, *sa=A.SIZE.data, *sb=B.SIZE.data;
02045     WBINDEX Ia(r), Ib(r), Ja(r), Jb(r), S;
02046
02047     size_t i=0,k=0,ia,ib,N;
02048
02049     if (kflag) {
02050         S.init(r);
02051         for (; i<r; ++i) { S[i]=sa[i]*sb[i]; }
02052     }
02053     else {
02054         S.init(2*r);
02055         for (; i<r; ++i) { S[k++]=sa[i]; S[k++]=sb[i]; }
02056     }
02057
02058     C.init(S); N=S.prod(); Ja[l]=Jb[l]=0;
02059
02060     for (k=l,i=0; i<N; ++i) {
02061         for (--k; k<r; --k) {
02062             Ja[k] = ( Ja[k+1] + Ia[k+1] ) * sa[k];
02063             Jb[k] = ( Jb[k+1] + Ib[k+1] ) * sb[k];
02064         }
02065
02066         ia = Ja[0] + Ia[0];
02067         ib = Jb[0] + Ib[0];
02068         C.data[i] = A.data[ia] * B.data[ib];
02069
02070         k=0;
02071         if ( (++Ia[k]) >= sa[k] ) { Ia[k]=0; ++Ib[k]; }
02072         while (Ib[k]>=sb[k] && k<l) {
02073             Ib[k]=Ia[k]=0; ++k;
02074             if ( (++Ia[k]) >= sa[k] ) { Ia[k]=0; ++Ib[k]; }
02075         }
02076     }
02077
02078     return C;

```

```

02079 };
02080
02081 template<class T>
02082 wbararray<T>& wbararray<T>::Cat (
02083     unsigned dim,
02084     const wbvector< wbararray<T> > &aa
02085 ){
02086     wbvector< const wbararray<T>* > ap(aa.len);
02087     for (unsigned i=0; i<aa.len; i++) ap[i]=&aa[i];
02088     Cat(dim,ap); return *this;
02089 };
02090
02091 template<class T>
02092 wbararray<T>& wbararray<T>::Cat (
02093     unsigned dim,
02094     const wbvector< wbararray<T>* > &ap
02095 ){
02096     if (ap.len==0) { init(); return *this; }
02097     if (ap.len==1) { *this=(*ap[0]); return *this; }
02098
02099     size_t i,j=0,k,n, e=0, N, *s2=0;
02100     const size_t r=ap[0]->SIZE.len, l=r-1, *sref=ap[0]->SIZE.data;
02101     const WBINDEX &S0=ap[0]->SIZE;
02102     WBINDEX S(S0), I(r), IO(ap.len), idx;
02103
02104     if (dim<1) wblog(FL,"ERR invalid index (not 1-based; %d/%d)",dim,r);
02105     dim--;
02106
02107     if (dim<r) N=sref[dim];
02108
02109     for (i=1; i<ap.len; i++) {
02110         if (ap[i]==NULL || ap[i]->data==NULL) continue;
02111
02112         if (r != ap[i]->SIZE.len) e++; else {
02113             s2=ap[i]->SIZE.data;
02114             for (j=0; j<r; j++) if (j!=dim && s2[j]!=sref[j]) e++;
02115         }
02116         if (e) wblog(FL,"ERR size mismatch %d/%d: %s <> %s (%d)",
02117             i+1,ap.len, SSTR_(ap[0]),SSTR_(ap[i]),dim+1);
02118
02119         IO[i]=N; if (dim<r) N+=s2[dim];
02120     }
02121
02122     if (dim>=r) { T *dd;
02123         S.Resize(dim+1); S[dim]=ap.len; init(S); dd=data; n=ap[0]->numel();
02124         for (i=0; i<ap.len; i++, dd+=n) {
02125             MEM_COPY<T>(dd,n,ap[i]->data);
02126         }
02127         return *this;
02128     }
02129
02130     idx.init(N);
02131     for (k=i=0; i<ap.len; i++) {
02132         if (ap[i]==NULL || ap[i]->data==NULL) continue;
02133         for (n=ap[i]->SIZE.data[dim], j=0; j<n; j++, k++) idx[k]=i;
02134     }
02135
02136     S[dim]=N; init(S); n=numel();
02137
02138     for (i=0; i<n; i++) {
02139         widx_t q=idx[I[dim]];
02140         const size_t *s=ap[q]->SIZE.data;
02141
02142         for (j=0,k=l; k<r; k--) {
02143             if (j) j*=s[k];
02144             j += (k!=dim ? I[k] : (I[k]-IO[q]));
02145         }
02146
02147         data[i]=ap[q]->data[j];
02148
02149         k=0; I[0]++;
02150         while(I[k]>=SIZE[k] && k<l) { I[k]=0; ++I[++k]; }
02151     }
02152
02153     return *this;
02154 };
02155
02156 template<class T>
02157 wbararray<T>& wbararray<T>::BlockCat (
02158     const wbararray< const wbararray<T>* > &ap,
02159     WBINDEX *D1_, WBINDEX *D2_
02160 ){
02161     size_t i,j,m,n,i0,j0,d1,d2,M,N;
02162     WBINDEX D1,D2;
02163
02164     if (ap.isEmpty()) {
02165         init(); if (D1_) D1_->init(); if (D2_) D2_->init();

```

```

02166     return *this;
02167 }
02168
02169 if (ap.rank()!=2) wblog(FL,
02170     "ERR %s() invalid input matrix (%s)",FCT,SSTR(ap));
02171
02172 m=ap.SIZE[0]; n=ap.SIZE[1]; D1.init(m); D2.init(n);
02173 for (i=0; i<m; i++)
02174     for (j=0; j<n; j++) if (ap(i,j) && !ap(i,j)->isEmpty()) {
02175         const wbararray<T> &a=(*ap(i,j));
02176         if (a.rank()!=2) wblog(FL,
02177             "ERR %s() invalid block rank (%d,%d): %s",FCT,i+1,j+1,SSTR(a));
02178
02179         if (D1[i]) {
02180             if (D1[i]!=a.SIZE[0]) wblog(FL,
02181                 "ERR %s() block-size mismatch (%d,%d: %s <> %dx..",
02182                 FCT, i+1, j+1, SSTR(a), D1[i]);
02183         }
02184         else D1[i]=a.SIZE[0];
02185
02186         if (D2[j]) {
02187             if (D2[j]!=a.SIZE[0]) wblog(FL,
02188                 "ERR %s() block-size mismatch (%d,%d: %s <> ..x%d",
02189                 FCT, i+1, j+1, SSTR(a), D2[j]);
02190         }
02191         else D2[j]=a.SIZE[0];
02192     }
02193
02194 if (D1_) D1_->init();
02195 if (D2_) D2_->init();
02196
02197 M=D1.sum(); N=D2.sum();
02198 init(M,N); if (isEmpty()) return *this;
02199
02200 for (j0=j=0; j<n; j++, j0+=d2) { d2=D2[j];
02201     for (i0=i=0; i<m; i++, i0+=d1) { d1=D1[i];
02202         ap(i,j)->copyStride(ref(i0,j0),M);
02203     }}
02204
02205 return *this;
02206 };
02207
02208 template<class T>
02209 wbararray<T>& wbararray<T>::blockTrace(
02210     size_t D1,
02211     wbararray<T> &a, size_t D2
02212 ) const {
02213
02214     if (rank()!=2) wblog(FL,
02215         "ERR %s() only applies to rank-2 tensors (%d)",FCT,rank());
02216
02217     size_t i, dim1=SIZE[0], dim2=SIZE[1], m=(D1 ? dim1/D1 : 0);
02218     if (int(D2)<0) { D2=D1; }
02219
02220     if (!D1 || !D2 || dim1%D1 || dim2%D2 || (dim2/D2)!=m) wblog(FL,
02221         "ERR %s() invalid block specs (%d/%d; %d/%d; %d)",
02222         D1,dim1, D2,dim2, m);
02223
02224     a.init(D1,D2);
02225     for (i=0; i<m; ++i) { addBlock(i*D1, i*D2, D1, D2, a); }
02226
02227     return a;
02228 };
02229
02230 template<class T>
02231 wbararray<T>& wbararray<T>::BlockDiag(const wbvector< wbararray<T> > &D) {
02232
02233     size_t i,d1=0,d2=0, i0=0, j0=0, D1=0, D2=0;
02234
02235     for (i=0; i<D.len; i++) {
02236         if (!D[i].isMatrix()) wblog(FL,
02237             "ERR %s() matrices expected (%s)",FCT,SSTR(D[i]));
02238         D1+=D[i].SIZE[0];
02239         D2+=D[i].SIZE[1];
02240     }
02241
02242     init(D1,D2);
02243     for (i=0; i<D.len; i++, i0+=d1,j0+=d2) {
02244         d1=D[i].SIZE[0];
02245         d2=D[i].SIZE[1];
02246         Wb::copyStride(ref(i0,j0), D[i].data,d1,d2,D1);
02247     }
02248
02249     return *this;
02250 };
02251
02252 template<class T>

```

```

02253 wbararray<T>& wbararray<T>::addBlock(
02254     size_t i, size_t j,
02255     size_t n, size_t m,
02256     wbararray<T> &a
02257 ) const {
02258
02259     if (rank()!=2) wblog(FL,
02260         "ERR %s() only applies to rank-2 tensors (%s)",FCT,SSTR_(this));
02261     size_t dim1=SIZE[0], dim2=SIZE[1];
02262
02263     if (i+n>dim1 || j+m>dim2) wblog(FL,
02264         "ERR %s() index out of bounds (%d:%d/%d; %d:%d/%d)",
02265         i+1,i+n,dim1,j+1,j+m,dim2);
02266
02267     if (a.isEmpty()) a.init(n,m);
02268     Wb::cpyStride(a.data, ref(i,j), n,m, -1,dim1,'+');
02269
02270     return a;
02271 };
02272
02273 template<class T>
02274 T wbararray<T>::norm2block(
02275     size_t i0, size_t j0,
02276     size_t n, size_t m
02277 ) const {
02278
02279     T x2=0;
02280
02281     if (rank()!=2) wblog(FL,
02282         "ERR %s() only applies to rank-2 tensors (%s)",FCT,SSTR_(this));
02283     size_t dim1=SIZE[0], dim2=SIZE[1];
02284
02285     if (i0+n>dim1 || j0+m>dim2) wblog(FL,
02286         "ERR %s() index out of bounds (%d:%d/%d; %d:%d/%d)",
02287         i0+1,i0+n,dim1, j0+1,j0+m,dim2);
02288
02289     if (m && n) {
02290         size_t i,j; const T* d = data + i0 + j0*dim1;
02291         for (j=0; j<m; ++j, d+=dim1) {
02292             for (i=0; i<n; ++i) { x2+=NORM2(d[i]); }
02293         }
02294
02295     return x2;
02296 };
02297
02298 template<class T>
02299 wbararray<T>& wbararray<T>::getBlock(
02300     size_t i, size_t j,
02301     size_t n, size_t m,
02302     wbararray<T> &a
02303 ) const {
02304
02305     if (rank()!=2) wblog(FL,
02306         "ERR %s() only applies to rank-2 tensors (%d)",FCT,rank());
02307
02308     size_t dim1=SIZE[0], dim2=SIZE[1];
02309
02310     if (i+n>dim1 || j+m>dim2) wblog(FL,
02311         "ERR %s() index out of bounds (%d..%d/%d; %d..%d/%d)",
02312         i+1,i+n,dim1,j+1,j+m,dim2);
02313
02314     a.init(n,m); Wb::cpyStride(a.data, ref(i,j),n,m,-1,dim1);
02315     return a;
02316 };
02317
02318 template<class T>
02319 void wbararray<T>::copyStride(T* dd, size_t stride, T afac) const {
02320
02321     if (SIZE.len!=2) wblog(FL,"ERR %s() "
02322         "only intended for rank-2 tensors (r=%d)",FCT,rank());
02323
02324     size_t i, j=0, dim1=SIZE[0], dim2=SIZE[1];
02325     T *d0=data;
02326
02327     if (afac==T(1)) {
02328         for (; j<dim2; ++j, dd+=stride, d0+=dim1) {
02329             MEM_COPY<T>(dd,dim1,d0);
02330         }
02331     }
02332     else if (afac==T(-1)) {
02333         for (; j<dim2; ++j, dd+=stride, d0+=dim1) {
02334             for (i=0; i<dim1; ++i) { dd[i]= -d0[i]; }
02335         }
02336     }
02337     else if (afac) {
02338         for (; j<dim2; ++j, dd+=stride, d0+=dim1) {
02339             for (i=0; i<dim1; ++i) { dd[i]=afac*d0[i]; }
02340         }
02341     }

```

```

02340     else {
02341         for (; j<dim2; ++j, dd+=stride) {
02342             for (i=0; i<dim1; ++i) { dd[i]=0; }
02343         }
02344     };
02345
02346 template<class T>
02347 void wbararray<T>::addStride(T* dd, size_t stride, const T afac) const {
02348
02349     if (SIZE.len==2) wblog(FL,"ERR %s() "
02350         "only intended for rank-2 tensors (r=%d)",FCT,rank());
02351
02352     size_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1];
02353     T *a=data;
02354
02355     if (afac==T(1)) {
02356         for (; j<dim2; ++j, a+=dim1, dd+=stride) {
02357             for (i=0; i<dim1; ++i) { dd[i] += a[i]; }
02358         }
02359     } else if (afac==T(-1)) {
02360         for (; j<dim2; ++j, a+=dim1, dd+=stride) {
02361             for (i=0; i<dim1; ++i) { dd[i] -= a[i]; }
02362         }
02363     } else if (afac) {
02364         for (; j<dim2; ++j, a+=dim1, dd+=stride) {
02365             for (i=0; i<dim1; ++i) { dd[i]+=(afac*a[i]); }
02366         }
02367     };
02368
02369 template<class T> inline
02370 wbararray<T>& wbararray<T>::Reshape(const wbvector<size_t> &S, bool lflag) {
02371
02372     if (numel()!=S.prod(0)) wblog(FL,
02373         "ERR %s() length mismatch (%s => %s)",FCT, SSTR_(this), SSTR(S));
02374
02375     if (!lflag) {
02376         unsigned i=0, j=0, e=0;
02377         const size_t *s1=SIZE.data, *s2=S.data; size_t sb;
02378
02379         while (!e) {
02380             for (; i<SIZE.len; ++i) { if (SIZE[i]!=1) { break; } }
02381             for (; j<S.len; ++j) { if (S[j] !=1) { break; } }
02382             if (i>=SIZE.len || j>=S.len) { break; }
02383
02384             if (s1[i]==s2[j]) { ++i; ++j; }
02385             else if (s1[i]<s2[j]) {
02386                 for (sb=s1[i++]; i<SIZE.len; ++i) {
02387                     if ((sb==s1[i]) >= s2[j]) {
02388                         if (sb==s2[j]) { ++i; ++j; } else { ++e; }
02389                         break;
02390                     }
02391                 }
02392             }
02393             else if (s1[i]>s2[j]) {
02394                 for (sb=s2[j++]; j<S.len; ++j) {
02395                     if ((sb==s2[j]) >= s1[i]) {
02396                         if (sb==s1[i]) { ++i; ++j; } else { ++e; }
02397                         break;
02398                     }
02399                 }
02400             }
02401         }
02402         if (e || i<SIZE.len || j<S.len) wblog(FL,"WRN %s() "
02403             "got non-trivial reshape %s -> %s",FCT,SSTR_(this),SSTR(S));
02404     }
02405     SIZE=S; return *this;
02406 };
02407
02408 template<class T>
02409 void wbararray<T>::GroupIndizes(size_t K) {
02410     size_t i,n;
02411
02412     if (!K || SIZE.len%K) wblog(FL,
02413         "ERR %s() invalid block specs: mod(%d,%d)",FCT,SIZE.len,K);
02414
02415     n=SIZE.len/K;
02416
02417     for (i=0; i<n; ++i)
02418         SIZE[i]=Wb::prodRange(SIZE.data+i*K, K);
02419
02420     SIZE.Resize(n);
02421 }
02422
02423 template<class T> inline
02424 void wbararray<T>::groupIndizes_P(
02425     const WBINDEX &I, int pos, wperm &P,

```

```

02427     size_t *s1, size_t *s2
02428 ) const {
02429
02430     size_t i,j,j1=0,j2=0,l, m=I.len, n=SIZE.len;
02431     wperm_t *p; widx_t *idx;
02432     char mark[n ? n : 1];
02433
02434     if (m>n) wblog(FL,
02435         "ERR %s() invalid group index (len=%d/%d)",FCT,m,n);
02436     if (!n) {
02437         if (s1) { *s1=0; }
02438         if (s2) { *s2=0; }; return;
02439     }
02440     P.init(n); p=P.data; idx=I.data;
02441
02442     if (pos==1) { j1=0; j2=m; } else
02443     if (pos==2) { j1=n-m; j2=0; }
02444     else wblog(FL,"ERR %s() invalid pos=%d",FCT,pos);
02445
02446     memset(mark,0,n*sizeof(char));
02447
02448     for (j=j1, i=0; i<m; ++i, ++j) { l=p[j]=idx[i];
02449         if (l>n) wblog(FL,"ERR %s() index not unique (%d/%d)",FCT,l,n);
02450         if (++mark[l]>1) wblog(FL,
02451             "ERR %s() index not unique (%d/%d)",FCT,l,n);
02452     }
02453     for (j=j2, i=0; i<n; ++i) { if (!mark[i]) { p[j++]=i; }}
02454
02455     if (s1 || s2) {
02456         for (j1=j2=1, i=0; i<n; ++i) {
02457             if (mark[i])
02458                 { j1*=SIZE[i]; }
02459             else { j2*=SIZE[i]; }
02460         }
02461         if (s1) { *s1 = (pos==1 ? j1 : j2); }
02462         if (s2) { *s2 = (pos==1 ? j2 : j1); }
02463     }
02464 };
02465
02466 template<class T>
02467 wbararray<T>& wbararray<T>::transpose(const char *F, int L, wbararray<T> &A) const {
02468
02469     unsigned r=rank(); wperm P;
02470
02471     if (r%2) wblog(F_L,"ERR %s() "
02472         "applies to even-rank arrays only! (%s)",FCT,SSTR_(this));
02473
02474     P.initTranspose(r);
02475     permute(A,P);
02476
02477     return A;
02478 };
02479
02480 template<class T>
02481 wbararray<T>& wbararray<T>::flipLR(const char *F, int L, wbararray &B) const {
02482
02483     if (rank()!=2) wblog(F_L,
02484         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02485     if (&B==this || B.data==data) wblog(F_L,
02486         "ERR %s() requires different objects!",FCT);
02487     B.init(SIZE);
02488
02489     if (SIZE[1]) {
02490         widx_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1];
02491         const T *a=data; T *b=B.data+dim1*(dim2-1);
02492         for (; j<dim2; ++j, a+=dim1, b-=dim1) {
02493             for (i=0; i<dim1; ++i) b[i]=a[i];
02494         }
02495     }
02496
02497     return B;
02498 };
02499
02500 template<class T>
02501 wbararray<T>& wbararray<T>::FlipLR(const char *F, int L) {
02502
02503     if (rank()!=2) wblog(F_L,
02504         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02505
02506     widx_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1], d2=dim2/2;
02507
02508     if (dim2>1) {
02509         T x, *a=data, *b=data+dim1*(dim2-1);
02510         for (; j<d2; ++j, a+=dim1, b-=dim1) {
02511             for (i=0; i<dim1; ++i) {
02512                 x=a[i]; a[i]=b[i]; b[i]=x;
02513             }

```



```

02514     }
02515 }
02516
02517 return *this;
02518 };
02519
02520 template<class T>
02521 wbararray<T>& wbararray<T>::flipUD(const char *F, int L, wbararray &B) const {
02522
02523     if (rank()!=2) wblog(F_L,
02524         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02525     if (&B==this || B.data==data) wblog(F_L,
02526         "ERR %s() requires different objects!",FCT);
02527     B.init(SIZE);
02528
02529     if (SIZE[0]) {
02530         widxt i=0, j=0, dim1=SIZE[0], dim2=SIZE[1], l=dim1-1;
02531         const T *a=data; T *b=B.data;
02532         for (; j<dim2; ++j, a+=dim1, b+=dim1) {
02533             for (i=0; i<dim1; ++i) b[i]=a[l-i];
02534         }
02535     }
02536
02537     return B;
02538 };
02539
02540 template<class T>
02541 wbararray<T>& wbararray<T>::FlipUD(const char *F, int L) {
02542
02543     if (rank()!=2) wblog(F_L,
02544         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02545
02546     widxt i=0, j=0, dim1=SIZE[0], dim2=SIZE[1], d2=dim1/2, l=dim1-1;
02547
02548     if (dim1>1) {
02549         T x, *a=data;
02550         for (; j<dim2; ++j, a+=dim1) {
02551             for (i=0; i<d2; ++i) {
02552                 x=a[i]; a[i]=a[l-i]; a[l-i]=x;
02553             }
02554         }
02555     }
02556
02557     return *this;
02558 };
02559
02560 template<class T>
02561 wbararray<T>& wbararray<T>::MatPermute(const wperm &P, char iflag){
02562
02563     if (isIdentityPerm(P)) return *this;
02564     if (!isSMatrix()) wblog(F_L,
02565         "ERR %s() expects square matrix (%s)",FCT,SSTR_(this));
02566
02567     wbararray<T> X(*this);
02568     const wperm_t *p=P.data;
02569     size_t i,j, dim1=SIZE[0], dim2=SIZE[1];
02570
02571     if (!iflag) {
02572         for (i=0; i<dim1; i++)
02573             for (j=0; j<dim2; j++) { data[i+dim1*j]=X.data[p[i]+dim1*p[j]]; }
02574     }
02575     else {
02576         for (i=0; i<dim1; i++)
02577             for (j=0; j<dim2; j++) { data[p[i]+dim1*p[j]]=X.data[i+dim1*j]; }
02578     }
02579
02580     return *this;
02581 };
02582
02583 template<class T>
02584 wbararray<T>& wbararray<T>::Permute(
02585     const wperm &P, char iflag, char rcpy) {
02586
02587     if (!P) { return *this; }
02588
02589     if (P.len>SIZE.len) { appendSingletons(P.len); }
02590
02591     if (!requiresDataPerm(P)) {
02592         SIZE.Permute(P,iflag);
02593         return *this;
02594     }
02595
02596     if (isRef()) {
02597         if (rcpy=='c' || rcpy=='C' || rcpy=='i' || rcpy=='I') {
02598             wbararray<T> X;
02599             return permute(X,P,iflag).save2(*this);
02600         }

```

```

02601         else if (rcpy!='r' && rcpy!='R') {
02602             wblog(PFL, "ERR %s() got ref (rcpy=%s)", FCT, cSTR(rcpy));
02603         }
02604     }
02605
02606     wbararray<T> X(*this);
02607     X.permute(*this, P, iflag);
02608
02609     return *this;
02610 };
02611
02612 template<class T>
02613 wbararray<T>& wbararray<T>::Permute(const char* sidx, char iflag, char rcpy) {
02614     wperm P(sidx);
02615     if (P.isEmpty()) wblog(FL, "WRN got empty permutation '%s'", sidx);
02616     return Permute(P, iflag, rcpy);
02617 };
02618
02619 template<class T>
02620 wbararray<T>& wbararray<T>::permute(
02621     wbararray<T> &A,
02622     const char* sidx,
02623     char iflag
02624 ) const {
02625     wperm P(sidx);
02626
02627     if (P.isEmpty()) {
02628         wblog(FL, "WRN got empty permutation '%s'", sidx);
02629         return A.init(*this);
02630     }
02631     return permute(A, P, iflag);
02632 };
02633
02634 template<class T>
02635 wbararray<T>& wbararray<T>::permute(
02636     wbararray<T> &A,
02637     const wperm &P0,
02638     char iflag
02639 ) const {
02640     if (!P0) { return A=(*this); }
02641     if (&A==this) {
02642         wbararray<T> X;
02643         return permute(X, P0, iflag).save2(A);
02644     }
02645
02646     size_t i, j, k, *Sp, *S, r=SIZE.len, l=r-1, s;
02647     wvector<size_t> Sp_;
02648     WBINDEX I_; widx_t *I;
02649     wperm P; wperm_t *p;
02650
02651     #ifndef WB_CLOCK
02652         Wb::Clock clk("arr:permute", 0);
02653     #endif
02654
02655     if (iflag!=0 && iflag!='I') wblog(FL,
02656         "ERR %s() invalid iflag '%c'<%d>", FCT, iflag, iflag);
02657
02658     P.init(P0, iflag, SIZE.len);
02659
02660     if (!requiresDataPerm(P)) {
02661         A=(*this); A.SIZE.Select(P);
02662         return A;
02663     }
02664
02665     p=P.data;
02666
02667     SIZE.select(P, Sp_); A.init(Sp_); Sp=Sp_.data; s=numel();
02668     I_.init(SIZE.len); I=I_.data; S=SIZE.data;
02669
02670     for (i=0; i<s; ++i) {
02671         for (j=I[p[l]], k=l-1; k<r; --k) j = j*Sp[k] + I[p[k]];
02672         A.data[j]=data[i];
02673
02674         k=0; ++I[0];
02675         while (I[k]>=S[k] && k<l) { I[k]=0; ++I[++k]; }
02676     }
02677
02678     return A;
02679 }
02680
02681 template<class T>
02682 wbararray<T>& wbararray<T>::select0(
02683     const WPERM &P,
02684     unsigned dim,

```

```

02688     wbararray<T> &A
02689 ) const {
02690
02691     if (&A==this) {
02692         wbararray<T> X(A); X.select0(P,dim,A);
02693         return A;
02694     }
02695
02696     const unsigned r=SIZE.len, l=r-1; size_t i,j,k,d;
02697     WBINDEX S,I;
02698
02699     if (dim>=SIZE.len) wblog(FL,
02700         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02701
02702     S=SIZE; d=SIZE[dim]; S[dim]=P.len;
02703
02704     for (i=0; i<P.len; i++) if (P[i]>=d) wblog(FL,
02705         "ERR %s() index out of bounds (%d/%d)",FCT,P[i],d);
02706
02707     A.init(S); I.init(SIZE.len);
02708     widx_t *ip=I.data, *s=S.data; const wperm_t *p=P.data;
02709
02710     for (d=A.numel(), i=0; i<d; i++) {
02711         for (j=0, k=1; k<r; k--) {
02712             if (j) j*=SIZE[k];
02713             j+=(k!=dim ? ip[k] : p[ip[k]]);
02714         }
02715
02716         A.data[i]=data[j];
02717
02718         k=0; ip[0]++;
02719         while(ip[k]>=s[k] && k<l) { ip[k]=0; ++ip[++k]; }
02720     }
02721
02722     return A;
02723 }
02724
02725 template<class T>
02726 wbararray<T>& wbararray<T>::selectSqueeze(
02727     wbararray<T> &A, unsigned p,
02728     unsigned dim
02729 ) const {
02730
02731     if (&A==this) {
02732         wbararray<T> X(A); X.selectSqueeze(A,p,dim);
02733         return A;
02734     }
02735
02736     const unsigned r=SIZE.len, l=r-1; size_t i,j,k,s;
02737     WBINDEX S,I;
02738
02739     if (dim>=SIZE.len) wblog(FL,
02740         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02741
02742     if (p>=SIZE[dim]) wblog(FL,
02743         "ERR %s() index out of bounds (%d/%d)",FCT,p,SIZE[dim]);
02744
02745     if (SIZE.len==1) {
02746         A.init(1); A.data[0]=data[p];
02747         return A;
02748     }
02749
02750     S=SIZE; S[dim]=1; A.init(S); s=A.numel(); I.init(SIZE.len);
02751
02752     for (i=0; i<s; i++) {
02753         for (j=0, k=1; k<r; k--) {
02754             if (j) j*=SIZE[k];
02755             j+=(k!=dim ? I[k] : p);
02756         }
02757
02758         A.data[i]=data[j];
02759
02760         k=0; I[0]++;
02761         while(I[k]>=S[k] && k<l) { I[k]=0; ++I[++k]; }
02762     }
02763
02764     for (i=dim+1; i<S.len; i++) A.SIZE[i-1]=A.SIZE[i];
02765     A.SIZE.len--; A.SIZE[A.SIZE.len]=0;
02766
02767     return A;
02768 }
02769
02770 template<class T>
02771 void wbararray<T>::setBlock(
02772     const wbvector< WBINDEX > &DB,
02773     const wbindex &IB,
02774     const wbararray<T> &A

```

```

02775 ){
02776     size_t i,j,ik,k,s, r=SIZE.len, l=r-1;
02777     wbinindex I,I1(r),I2(r);
02778
02779     if (DB.len!=r || IB.len!=r) wblog(FL,
02780         "ERR %s() size mismatch (r: %d %d / %d)",FCT,DB.len,IB.len,r);
02781
02782     if (A.SIZE.len!=r) {
02783         for (i=r; i<A.SIZE.len; ++i) { if (A.SIZE[i]!=1) break; }
02784         if (i!=A.SIZE.len) wblog(FL,
02785             "ERR %s() size mismatch (r=%d / %d)",FCT,A.SIZE.len,r);
02786     }
02787
02788     for (k=0; k<r; ++k) {
02789         const WBINDEX &Dk=DB[k]; ik=IB[k];
02790
02791         if (ik>=Dk.len) wblog(FL,
02792             "ERR %s() block index out of bounds (%d: %d/%d)",
02793             FCT,k+1, ik+1, Dk.len);
02794         if (Dk[ik] != A.SIZE[k]) wblog(FL,
02795             "ERR %s() block size mismatch (%d: %d/%d)",
02796             FCT,k+1, Dk[ik], A.SIZE[k]);
02797
02798         I1[k]=Wb::addRange(Dk.data,ik);
02799         I2[k]=I1[k]+Dk[ik];
02800
02801         if (I2[k]>SIZE[k]) wblog(FL,
02802             "ERR %s() index out of bounds (%d: %d/%d)",
02803             FCT,k+1, I2[k], SIZE[k]);
02804     }
02805
02806     I=I1; s=A.numel();
02807
02808     for (i=0; i<s; ++i) {
02809         for (j=I[1],k=1-l; k<r; k--) j = j*SIZE[k] + I[k];
02810
02811         data[j]=A.data[i];
02812
02813         k=0; I[0]++;
02814         while(I[k]>=I2[k] && k<l) { I[k]=I1[k]; ++I[++k]; }
02815     }
02816 }
02817
02818 template<class T>
02819 void wbarrray<T>::addBlock(
02820     const WBINDEX &I0,
02821     const wbarrray<T> &A,
02822     const char dflag
02823 ){
02824     size_t i,j,k, s=A.numel(), r=SIZE.len, l=r-1;
02825     unsigned ra=(dflag ? r/2 : r), la=ra-1;
02826     const widxt_t *S=A.SIZE.data;
02827     WBINDEX I(ra);
02828
02829     if (I0.len!=r || A.SIZE.len!=ra || (dflag && r%2)) wblog(FL,
02830         "ERR %s() size mismatch (%d/%d, %s; %c<%d>)",
02831         FCT,I0.len, A.SIZE.len, SSTR_(this), dflag
02832     );
02833     for (i=0; i<r; i++) if (I0[i]+S[i]>=ra ? i-ra : i)>SIZE[i])
02834         wblog(FL,"ERR %s() index out of bounds (%d: %s - %s %s)",
02835             FCT,i+1,I0.toStr().data,SSTR(A),SSTR_(this)
02836     );
02837
02838     for (i=0; i<s; i++) {
02839         for (j=0,k=1; k<r; k--) { if (j) j*=SIZE[k];
02840             j+=(I0[k]+I[k<ra ? k : k-ra]);
02841         }
02842
02843         data[j] += A.data[i];
02844
02845         k=0; I[0]++;
02846         while(I[k]>=S[k] && k<la) { I[k]=0; ++I[++k]; }
02847     }
02848 };
02849
02850 template<class T>
02851 T wbarrray<T>::min() const {
02852     size_t i,n=numel(); T x;
02853
02854     if (n==0) {
02855         wblog(FLINE,"WRN min() of empty wbarrray !?");
02856         memset(&x,0,sizeof(T)); return x;
02857     }
02858
02859     for (x=data[0], i=1; i<n; i++)
02860         if (x>data[i]) x=data[i];
02861

```

```

02862     return x;
02863 }
02864
02865 template<class T>
02866 T wbararray<T>::max() const {
02867     size_t i,n=numel(); T x;
02868
02869     if (n==0) {
02870         wblog(FLINE,"WRN max() of empty wbararray !?");
02871         memset(&x,0,sizeof(T)); return x;
02872     }
02873
02874     for (x=data[0], i=1; i<n; i++)
02875         if (x<data[i]) x=data[i];
02876
02877     return x;
02878 }
02879
02880 template<class T>
02881 T wbararray<T>::aMin(char zflag, size_t *k_) const {
02882
02883     size_t i=0, k=-1,n=numel(); T a,x=0;
02884
02885     if (!n) {
02886         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got empty array",FCT); }
02887         return 0;
02888     }
02889
02890     if (zflag==0) {
02891         x=ABS(data[i++]); k=0; if (x!=0) {
02892             for (; i<n; ++i) { a=ABS(data[i]); if (x>a) {
02893                 x=a; k=i; if (x==0) break;
02894             }}
02895         }
02896     }
02897     else {
02898         for (; i<n; ++i) {
02899             a=ABS(data[i]); if (a) { x=a; k=i; break; }
02900         }
02901         for (; i<n; ++i) { a=ABS(data[i]);
02902             if (a && x>a) { x=a; k=i; }
02903         }
02904     }
02905     if (k_) (*k_)=k;
02906     return x;
02907 };
02908
02909 template<class T>
02910 wstring wbararray<T>::toStr() const {
02911     wstring s;
02912     unsigned l=2; size_t n=numel();
02913     if (n==1) { s.init(24+4*SIZE.len);
02914         l=snprintf(s.data,s.len, SIZE.len==2? "%s (%s)" : "%s [%s]",
02915             STR(wbvector<T>(numel(),data,'r')), SSTR_(this));
02916     }
02917     else if (n || SIZE.len) { s.init(5 + 10*(numel()+SIZE.len));
02918         l=snprintf(s.data,s.len,"%s [%s]",
02919             STR(wbvector<T>(numel(),data,'r')), SSTR_(this));
02920     }
02921     else { s="[]"; }
02922
02923     if (l>=s.len) wblog(FL,
02924         "WRN %s() string out of bounds (%d/%d)",FCT,l,s.len);
02925     return s;
02926 };
02927
02928 template<class T>
02929 void wbararray<T>::print_ref (const char *sb, const char *nl) const {
02930
02931     if (isRef() || mtype || (sptr && sptr->mtype)) {
02932         int l=0, n=32; char str[n]; str[0]=0;
02933
02934         if (sptr) {
02935             l+=snprintf(str,n,"%s",STR_(sptr));
02936             if (mtype!=sptr->mtype && l<n)
02937                 l+=snprintf(str+l,n-l," / %s !?",Wb::MTYPE_STR[mtype]);
02938         }
02939         else { l+=snprintf(str,n,"%s",Wb::MTYPE_STR[mtype]); }
02940
02941         printf(" %s%s%s", sb?sb:"", str, sb?sb:"", nl?nl:"");
02942
02943         if (l>=n) { printf("\n");
02944             wblog(FL,"WRN %s() string out of bounds (%d/%d)",FCT,l,n);
02945         }
02946     }
02947     else if ((nl && nl[0])) { printf("%s",nl); }
02948

```

```

02949     if (sptr && data!=sptr->data) {
02950         if (!nl && !nl[0]) { PRINTF("\n"); }
02951         wblog(FL, "WRN %s() data=%p / %p !?",FCT,data,sptr->data);
02952     }
02953 };
02954
02955 template<class T>
02956 void wbararray<T>::info(const char *F, int L,
02957     const char* istr, unsigned k,
02958     unsigned nlt, unsigned nlb
02959 ) const {
02960
02961     if (nlt) {
02962         printf("\n"); --nlt; if (nlt>1) {
02963             printf("\n"); --nlt; }
02964
02965         if (nlt) { --nlt;
02966             printf("%-20s %-8s %-8s %s\n",
02967                 "",
02968                 "name","size","data type");
02969         }
02970         if (nlt>4) nlt=4;
02971         for (unsigned i=0; i<nlt; ++i) { printf("\n"); }
02972     }
02973
02974     char s[32];
02975     if (int(k)>=0)
02976         { snprintf(s,32,"%s[%d]",istr?istr:"arr",k+1); }
02977     else { snprintf(s,32,"%s",istr?istr:""); }
02978
02979     #pragma omp critical (__ensure_sequential_wblog__)
02980     { printf("%-20s %-8s %-8s %s",shortFL(F_L),s,SSTR_(this),TSTR(T));
02981         print_ref();
02982     }
02983
02984     if (nlb) {
02985         if (nlb>10) nlb=10;
02986         for (unsigned i=0; i<nlb; ++i) { printf("\n"); }
02987     }
02988 };
02989
02990 template<class T>
02991 void wbararray<T>::info(const char* istr) const {
02992
02993     unsigned l=0, n=16; char s[n];
02994     size_t b=numel()*sizeof(T);
02995
02996     if (b<(1<10)) l=snprintf(s,n,"%ld bytes ",b); else
02997     if (b<(1<20)) l=snprintf(s,n,"%3g kB",b/double(1<10)); else
02998         l=snprintf(s,n,"%3g MB",b/double(1<20));
02999     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
03000
03001     #pragma omp critical (__ensure_sequential_wblog__)
03002     { printf(" %-12s %-10s %12s @ %12p %s",
03003         istr? istr:"", SSTR_(this), s, data, TSTR(T));
03004         print_ref();
03005     }
03006 };
03007
03008 template<class T>
03009 void wbararray<T>::print(
03010     const char *F, int L, const char* istr, const char* fmt0
03011 ) const {
03012
03013     if (F) {
03014         wbststring tstr;
03015         if (typeid(T)==typeid(double)) tstr="double";
03016         else tstr.init(((char*)(typeid(T).name())));
03017
03018         wblog(F,L,"%s = [%s] %s array \\",
03019             istr[0]? istr:"ans", SSTR_(this), tstr.data);
03020         print_ref();
03021     }
03022     else info(istr);
03023
03024     printdata(istr, fmt0);
03025 };
03026
03027 template<class T>
03028 int wbararray<T>::printdata(
03029     const char *istr,
03030     const char *fmt0
03031 ) const {
03032
03033     unsigned i,j,k, r=SIZE.len, l=r-1, s=numel();
03034     wbststring fmt;
03035

```

```

03036     if (!fmt0[0]) {
03037         if (typeid(T)==typeid(double) || typeid(T)==typeid(float))
03038             fmt=" %12.5g";
03039         else fmt.init2Fmt((T)0, 6);
03040     } else fmt=fmt0;
03041
03042     if (s>1) {
03043         if (SIZE.len<2) {
03044             printf("\n");
03045             for (i=0; i<s; i++) printf(fmt.data,data[i]);
03046             if (s) printf("\n\n");
03047         }
03048         else {
03049             WBINDEX I(r);
03050
03051             wbararray<T> A; wbpem P(r); P[0]=1; P[1]=0;
03052             permute(A,P);
03053
03054             for (k=2, i=0; i<s; i++) {
03055                 if (k) { if (i) printf("\n");
03056                     if (k>1) {
03057                         if (r>2) {
03058                             printf("\n  %s(:,:",istr);
03059                             for (j=2; j<r; j++) printf(",%ld",I[j]+1);
03060                             printf(") = [\n\n");
03061                         }
03062                         else printf("%s = [\n",istr);
03063                     }
03064                 }
03065
03066                 printf(fmt.data,A.data[i]);
03067
03068                 k=0; I[0]++;
03069                 while (I[k]>=A.SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
03070             }
03071             printf("\n];\n");
03072         }
03073     }
03074     else if (s==1) {
03075         printf("  %s = ",istr);
03076         printf(fmt.data,data[0]); printf("\n");
03077     }
03078     else printf("\n");
03079     return (int)s;
03080 }
03081 };
03082
03083 template<>
03084 int wbararray<wbcomplex>::printdata(
03085     const char *istr, const char *fmt0
03086 ) const {
03087     unsigned i,j,k, r=SIZE.len, l=r-1, s=numel();
03088     wstring fmt;
03089
03090     fmt=((fmt0 && fmt0[0]) ? fmt0 : " %8.4g%+8.4gi");
03091
03092     if (s>1) {
03093         if (SIZE.len<2) {
03094             printf("\n");
03095             for (i=0; i<s; i++) printf(fmt.data,data[i].r,data[i].i);
03096             if (s) printf("\n\n");
03097         }
03098         else {
03099             WBINDEX I(r);
03100
03101             wbararray<wbcomplex> A; wbpem P(r); P[0]=1; P[1]=0;
03102             permute(A,P);
03103
03104             for (k=2, i=0; i<s; i++) {
03105                 if (k) { if (i) printf("\n");
03106                     if (k>1) {
03107                         if (r>2) {
03108                             printf("\n  %s(:,:",istr);
03109                             for (j=2; j<r; j++) printf(",%ld",I[j]+1);
03110                             printf(") = [\n\n");
03111                         }
03112                         else printf("%s = [\n",istr);
03113                     }
03114                 }
03115
03116                 printf(fmt.data,A.data[i].r,A.data[i].i);
03117
03118                 k=0; I[0]++;
03119                 while (I[k]>=A.SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
03120             }
03121             printf("\n];\n");
03122         }

```

```

03123     }
03124 }
03125 else if (s==1) {
03126     printf(" %s = ",istr);
03127     printf(fmt.data,data[0].r,data[0].i); printf("\n");
03128 }
03129 else printf("\n");
03130
03131 return (int)s;
03132 };
03133
03134 template<class T>
03135 template<class TB, class TC>
03136 wbarray<TC>& wbarray<T>::comm(
03137     const wbarray<TB> &B, wbarray<TC> &C, char aflag, char bflag
03138 ) const {
03139
03140     if ((void*)&C==(void*)this || (void*)&C==(void*)&B) {
03141         wbsparray<TC> X; comm(B,X,aflag,bflag);
03142         return X.save2(C);
03143     }
03144
03145     if (!isMatrix() || !B.isMatrix() || SIZE[0]!=B.SIZE[1] ||
03146         SIZE[0]!=SIZE[1] || B.SIZE[0]!=B.SIZE[1]) wblog(FL,
03147         "ERR %s() invalid operators for [A,B] (%s; %s)",
03148         FCT,SSTR_(this), SSTR(B)
03149     );
03150
03151     Wb::MatProd(*this,B,C,aflag,bflag);
03152     Wb::MatProd(B,*this,C,bflag,aflag,-1.,1.);
03153
03154     return C;
03155 };
03156
03157 template<class T>
03158 template<class TB, class TC>
03159 wbarray<TC>& wbarray<T>::acomm(
03160     const wbarray<TB> &B, wbarray<TC> &C, char aflag, char bflag
03161 ) const {
03162
03163     if (!isMatrix() || !B.isMatrix() ||
03164         SIZE[0]!=SIZE[1] || B.SIZE[0]!=B.SIZE[1]) wblog(FL,
03165         "ERR %s() invalid operators for [A,B] (%s; %s)",
03166         FCT,SSTR_(this), SSTR(B)
03167     );
03168
03169     Wb::MatProd(*this,B,C,aflag,bflag);
03170     Wb::MatProd(B,*this,C,bflag,aflag,+1.,1.);
03171
03172     return C;
03173 };
03174
03175 template<class T>
03176 template<class TB, class TC>
03177 wbarray<TC>& wbarray<T>::contractMat(
03178     const char *F, int L, unsigned i1,
03179     const wbarray<TB> &M, unsigned i2,
03180     wbarray<TC> &X
03181 ) const {
03182
03183     unsigned r=SIZE.len, l=r-1;
03184     wvector<size_t> S; wperm P;
03185     wbarray<T> A;
03186     wbarray<T> MA;
03187
03188     if (!M.isMatrix()) wblog(FL,
03189         "ERR %s() rank 2 object required (%d)",FCT,M.SIZE.len);
03190     if (i1<1 || i1>SIZE.len || i2<1 || i2>2) wblog(FL,
03191         "ERR %s() invalid contraction indices [%d %d, %d %d]", i1,i2,
03192         FCT,rank(), M.rank());
03193
03194     if (i1==1) {
03195         toMatrixRef(MA,i1-1,1,P); SIZE.select(P,S); l=0;
03196         if (i2==2)
03197             { Wb::MatProd(M,MA,X); S[l]=M.SIZE[0]; }
03198         else { Wb::MatProd(M,MA,X,'T'); S[l]=M.SIZE[1]; }
03199     }
03200     else {
03201         toMatrixRef(MA,i1-1,2,P); SIZE.select(P,S);
03202         if (i2==1)
03203             { Wb::MatProd(MA,M,X); S[l]=M.SIZE[1]; }
03204         else { Wb::MatProd(MA,M,X,'N','T'); S[l]=M.SIZE[0]; }
03205     }
03206
03207     X.Reshape(S).Permute(P,'I');
03208
03209     return X;

```



```

03210 };
03211
03212 template<class T>
03213 template<class TB> inline
03214 wbarry<T>& wbarry<T>::Contract(char *idx1,
03215     const wbarry<TB> &B, char *idx2,
03216     const wperm &pfinal
03217 ) {
03218     wbarry<T> A(*this);
03219
03220     WBINDEX i1, i2;
03221     i1 = Str2Idx(idx1,1);
03222     i2 = Str2Idx(idx2,1);
03223
03224     A.contract(i1, B, i2, *this, pfinal);
03225     return *this;
03226 }
03227
03228 template<class T>
03229 template<class TB, class TC> inline
03230 wbarry<TC>& wbarry<T>::contract(const char *F, int L,
03231     const char *idx1, const wbarry<TB> &B, const char *idx2,
03232     wbarry<TC> &Q, const wperm &pfinal
03233 ) const {
03234
03235     wbvector<unsigned> i1, i2;
03236     i1 = Str2Idx(idx1,1);
03237     i2 = Str2Idx(idx2,1);
03238
03239     return contract(F,L, i1,B,i2,Q,pfinal);
03240 }
03241
03242 template<class T>
03243 template<class TB> inline
03244 wbarry<T>& wbarry<T>::Contract(const wbvector<unsigned> &i1,
03245     const wbarry<TB> &B, const wbvector<unsigned> &i2,
03246     const wperm &pfinal
03247 ) {
03248     wbarry<T> A(*this); A.contract(i1, B, i2, *this, pfinal);
03249     return *this;
03250 }
03251
03252 template<class T>
03253 template<class TB, class TC>
03254 wbarry<TC>& wbarry<T>::contract(
03255     const char *F, int L, const iTags &ta,
03256     const wbarry<TB> &B, const iTags &tb, wbarry<TC> &C, iTags &tc,
03257     const wperm &pfinal, T afac, TC cfac
03258 ) const {
03259
03260     ctrIdx ia,ib;
03261
03262     ta.getCtr(F_L,tb,ia,ib,&tc);
03263     contract(F_L,ia,B,ib,C,pfinal,afac,cfac);
03264
03265     tc.Permute(pfinal);
03266     return C;
03267 };
03268
03269 template<class TA>
03270 template<class TB, class TC>
03271 wbarry<TC>& wbarry<TA>::contract(
03272     const char *F, const int L,
03273     const ctrIdx &ica, const wbarry<TB> &B, const ctrIdx &icb,
03274     wbarry<TC> &C0, const wperm &P,
03275     TA afac,
03276     TC cfac
03277 ) const {
03278
03279     if ((void*)(&C0)==(void*)this || (void*)(&C0)==(void*)(&B)) {
03280         wbarry<TC> Ci;
03281         if (cfac) {
03282             bool q1=((void*)(&C0)==(void*)this),
03283                 q2=((void*)(&C0)==(void*)(&B));
03284             wblog(FL,"ERR %s() got cfac=%s with self-reference (%s)",
03285                 FCT, NSTR(cfac), q1 && q2 ? "A=B=C" : (q1 ? "A=C" : "B=C"));
03286         }
03287         contract(F,L,ica,B,icb,Ci,P,afac,cfac);
03288         return Ci.save2(C0);
03289     }
03290
03291 #ifdef WB_CLOCK
03292     Wb::Clock clk("arr:contract",0);
03293 #endif
03294
03295     unsigned i,s; size_t one=1;
03296     char aflag, bflag;

```

```

03297     wbvector<size_t> S1,S2;
03298     wbararray<TA> MA;
03299     wbararray<TB> MB;
03300     wbararray<TC> Ci;
03301     wbpem p1,p2;
03302
03303     wbvector<unsigned> i1(ica);
03304     wbvector<unsigned> i2(icb);
03305
03306     char gotC_=1;
03307     if (C0.isEmpty()) {
03308         if (isEmpty() || B.isEmpty()) { return C0; }
03309         if (cfac && cfac!=TC(1)) wblog(FL,
03310             "ERR %s() got cfac=%s with empty C",FCT,NSTR(cfac));
03311         if (C0.mtype && C0.mtype!=Wb::MEX_RETURN) wblog(FL,
03312             "WRN %s() unexpected mt=%s",FCT,Wb::MTYPE_STR[C0.mtype]);
03313         SWAP(Ci.mtype,C0.mtype);
03314         gotC_=0;
03315     }
03316
03317     if (i1.isEmpty() || i2.len!=i1.len) wblog(F,L,
03318         "ERR %s() index set length mismatch [%s] / [%s]",FCT,STR(i1),STR(i2));
03319     for (s=SIZE.len, i=0; i<i1.len; ++i) { if (i1[i]>=s) wblog(F,L,
03320         "ERR %s() index out of bounds (A @ [%s]/%d)",FCT,STR(i1+1),s); }
03321     for (s=B.SIZE.len, i=0; i<i2.len; ++i) { if (i2[i]>=s) wblog(F,L,
03322         "ERR %s() index out of bounds (B @ [%s]/%d)",FCT,STR(i2+1),s); }
03323     for (i=0; i<i1.len; ++i) {
03324         if (SIZE[i1[i]]!=B.SIZE[i2[i]]) wblog(F,L,
03325             "ERR wbararray::%s() incompatible data size\n[%s] @ %d <> [%s] @ %d",
03326             FCT,SSTR_(this),i1[i]+1,SSTR(B),i2[i]+1); }
03327
03328     toMatrixRef(FL,MA,i1,2,aflag);     SIZE.getI(i1,S1);
03329     B.toMatrixRef(FL,MB,i2,1,bflag);    B.SIZE.getI(i2,S2);
03330
03331     if ((i=S1.len+S2.len)<2) {
03332         if (!i) { S2.init2val(2,one); }
03333     }
03334
03335     if (!afac) {
03336         WBINDEX S(S1,S2); if (P) S.Permute(P);
03337         if (gotC_) {
03338             if (C0.SIZE!=S) wblog(F,L,
03339                 "ERR %s() size mismatch %s / %s",FCT,SSTR(S),SSTR(C0));
03340             C0*=cfac;
03341         }
03342         else {
03343             Ci.init(S).save2(C0);
03344         }
03345         return C0;
03346     }
03347
03348     char isPtrans = P.isCyclic2F(S1.len,S2.len,'i');
03349
03350     if (aflag<0 || aflag>1) wblog(FL,
03351         "ERR %s() invalid aflag=%d !?",FCT,aflag);
03352     if (bflag<0 || bflag>1) wblog(FL,
03353         "ERR %s() invalid bflag=%d !?",FCT,bflag);
03354
03355     if (isPtrans) { aflag=!aflag; bflag=!bflag; }
03356
03357     if (ica.conj && ISCOMPLX_(TA)) {
03358         if (aflag)
03359             { aflag='C'; }
03360         else { aflag='N'; MA.Instantiate().Conj(); }
03361     } else { aflag=(aflag ? 'T':'N'); }
03362
03363     if (icb.conj && ISCOMPLX_(TB)) {
03364         if (bflag)
03365             { bflag='C'; }
03366         else { bflag='N'; MB.Instantiate().Conj(); }
03367     } else { bflag=(bflag ? 'T':'N'); }
03368
03369     if (!isPtrans) {
03370         Wb::MatProd(MA,MB,Ci, aflag, bflag);
03371         Ci.Reshape(UVEC(S1,S2)).Permute(P);
03372     }
03373     else {
03374         Wb::MatProd(MB,MA,Ci, bflag, aflag);
03375         Ci.Reshape(UVEC(S2,S1));
03376     }
03377
03378     if (gotC_) {
03379         if (!Ci.sameSize(C0)) wblog(F,L,
03380             "ERR %s() size mismatch\n%s cannot add %s / %s",
03381             FCT,SHORT_FL,SSTR(Ci),SSTR(C0));
03382         return C0.Plus(Ci, TC(afac), 0, cfac);
03383     }

```

```

03384     else {
03385         if (cfac && cfac!=TC(1)) wblog(FL,
03386             "WRN %s() got cfac=%s with empty input",FCT,NSTR(cfac));
03387         if (afac!=TA(1)) { Ci*=afac; }
03388         return Ci.save2(C0);
03389     }
03390 };
03391
03392 template<class T>
03393 wbararray<T>& wbararray<T>::trace(
03394     unsigned i1, unsigned i2, wbararray<T> &C) const {
03395
03396     size_t i,j,k,d,q,s,r,l;
03397     WBINDEX S,I0,I;
03398
03399     if (&C==this) wblog (FL,
03400         "ERR %s() output space coincides with *this",FCT);
03401
03402     if (i1==i2 || !i1 || !i2 || i1>SIZE.len || i2>SIZE.len ||
03403         SIZE[i1-1]!=SIZE[i2-1]) {
03404         wblog (FL,"ERR %s() invalid trace indizes (%d,%d; %s)",
03405             FCT,i1, i2, SSTR_(this)); return C;
03406     }
03407
03408     i1--; i2--;
03409
03410     d=SIZE[i1];
03411
03412     S.init(SIZE.len-2);
03413     for (k=i0; i<SIZE.len; ++i) if (i!=i1 && i!=i2) S[k++]=SIZE[i];
03414     if (S.len==0) { S.init(1); S[0]=1; }
03415
03416     if (C.isEmpty()) C.init(S); else {
03417         if (S!=C.SIZE) wblog(FL,
03418             "ERR %s() size mismatch of I/O array\n%s: [%s] vs. [%s]",
03419             FCT,SHORT_FL,SSTR(C), STR(S)
03420         );
03421     }
03422
03423     I.init(S.len); s=S.prod(0); I0.init(SIZE.len);
03424     r=SIZE.len-1; l=S.len-1;
03425
03426     for (i=0; i<s; ++i) {
03427         for (q=k=0; k<SIZE.len; ++k) if (k!=i1 && k!=i2) I0[k]=I[q++];
03428         for (q=0; q<d; ++q) {
03429             I0[i1]=I0[i2]=q;
03430             for (j=I0[r],k=r-1; k<r; k--) j = j*SIZE[k] + I0[k];
03431
03432             C.data[i]+=data[j];
03433         }
03434
03435         k=0; I[0]++;
03436         while(I[k]>=S[k] && k<l) { I[k]=0; ++I[++k]; }
03437     }
03438
03439     return C;
03440 }
03441
03442 template<class T>
03443 wbararray<T>& wbararray<T>::trace(
03444     const ctrIdx &I1, const ctrIdx &I2, wbararray<T> &C) const {
03445
03446     size_t i,j,k,l, kc,kx, lc,lx,l0, Nc,Nx, *sx, *sc;
03447     const size_t *s0=SIZE.data;
03448
03449     unsigned r0=SIZE.len, *mx;
03450     const unsigned *i1=I1.data, *i2=I2.data;
03451
03452     WBINDEX Sx, Sc, I0, Ix, Ic; widx_t *i0, *ix, *ic;
03453     wvector<unsigned > Mx;
03454
03455     if (&C==this) wblog (FL,
03456         "ERR %s() output space coincides with *this !?",FCT);
03457     if (I1.len!=I2.len || 2*I1.len>r0) wblog (FL,
03458         "ERR %s() invalid index sets (%s <> %s)",FCT,STR(I1),STR(I2));
03459
03460     Mx.init(r0); mx=Mx.data;
03461     Sx.init(I1.len); sx=Sx.data;
03462
03463     for (i=0; i<I1.len; ++i) {
03464         j=i1[i]; if (j>=r0 || mx[j]++) { break; }
03465         j=i2[i]; if (j>=r0 || mx[j]++) { break; }
03466
03467         sx[i]=s0[i1[i]]; if (sx[i]!=s0[i2[i]]) { break; }
03468     }
03469     if (i<I1.len) wblog (FL,
03470         "ERR %s() invalid index set\nhaving %s with %s <> %s",

```

```

03471     FCT,SSTR_(this),STR(I1),STR(I2)
03472 );
03473
03474 if (!I1.len) {
03475     if (C.isEmpty()) C=(*this); else C+=(*this);
03476     return C;
03477 }
03478
03479 k=r0-2*I1.len; Sc.init(k?k:1); sc=Sc.data;
03480 if (!k) { sc[0]=1; }
03481 else { for (l=i=0; i<r0; ++i) { if (!mx[i]) sc[l++]=s0[i]; }}
03482
03483 Nx=Sx.prod(0); lx=Sx.len-1;
03484 Nc=Sc.prod(0); lc=Sc.len-1; l0=r0-1;
03485
03486 if (C.isEmpty()) C.init(Sc); else {
03487     if (Sc!=C.SIZE) wblog(FL,
03488         "ERR %s() size mismatch of I/O array\n%s: [%s] vs. [%s]",
03489         FCT,SHORT_FL,SSTR(C), STR(Sc)
03490     );
03491 }
03492
03493 I0.init(r0); i0=I0.data;
03494 Ic.init(Sc.len); ic=Ic.data;
03495 Ix.init(Sx.len); ix=Ix.data;
03496
03497 for (kc=0; kc<Nc; ++kc) {
03498     for (l=i=0; i<r0; ++i) if (!mx[i]) i0[i]=ic[l++];
03499
03500     Ix.set(0);
03501     for (kx=0; kx<Nx; ++kx) {
03502         for (i=0; i<Ix.len; ++i) {
03503             i0[i1[i]] = i0[i2[i]] = ix[i];
03504         }
03505         for (k=i0[l0], i=l0-1; i<l0; i--) { k = k*s0[i] + i0[i]; }
03506
03507         C.data[kc] += data[k];
03508
03509         i=0; ++ix[0];
03510         while (ix[i]>=sx[i] && i<lx) { ix[i]=0; ++ix[++i]; }
03511     }
03512
03513     i=0; ic[0]++;
03514     while (ic[i]>=sc[i] && i<lc) { ic[i]=0; ++ic[++i]; }
03515 }
03516
03517 return C;
03518 };
03519
03520 template<class T>
03521 wbararray<T> TestContract(
03522     const wbararray<T> &A, const WBINDEX i1,
03523     const wbararray<T> &B, const WBINDEX i2,
03524     const wperm &pfinal
03525 ){
03526     WBINDEX Si,Sj,Sk, I,J,K, Ia,Ib,Ic, Iq; WPERM Pa,Pb;
03527     size_t i,j,k, si,sj,sk, li,lj,lk, q, ra=A.SIZE.len, rb=B.SIZE.len;
03528     wbararray<T> C;
03529
03530     T dbl;
03531
03532     if (i1.isEmpty() || i2.len!=i1.len) wblog(FL,
03533         "ERR %s() invalid index set {[%s], [%s]}",
03534         FCT,i1.toStr().data, i2.toStr().data);
03535     if (i1.anyGE(ra) || i2.anyGE(rb)) wblog(FL,
03536         "ERR %s() index out of range\n[%s; %d], [%s; %d]",
03537         FCT,i1.toStr().data,ra,i2.toStr().data,rb);
03538
03539     return A;
03540
03541     for (k=0; k<i1.len; ++k) {
03542         if (A.SIZE[i1[k]]!=B.SIZE[i2[k]]) wblog(FL,
03543             "ERR %s() incompatible objects {[%s], [%s]}\n[%s] <> [%s] at %d",
03544             FCT,i1.toStr().data, i2.toStr().data, SSTR(A), SSTR(B), k);
03545     }
03546
03547     Si=A.SIZE.Skip(i1); si=Si.len>0 ? Si.prod(0) : 1; li=Si.len-1;
03548     Sj=A.SIZE [i1]; sj=Sj.len>0 ? Sj.prod(0) : 1; lj=Sj.len-1;
03549     Sk=B.SIZE.Skip(i2); sk=Sk.len>0 ? Sk.prod(0) : 1; lk=Sk.len-1;
03550
03551     Pa = Index(0,ra-1).Move2end (i1);
03552     Pb = Index(0,rb-1).Move2front(i2);
03553
03554     Ia.init(ra); Ib.init(rb); C.init(Si.append(Sk));
03555
03556     if (C.numel()==0) C.init(1);
03557

```

```

03558     I.init(Si.len);
03559     for (i=0; i<si; ++i) {
03560
03561         K.init(Sk.len);
03562         for (k=0; k<sk; ++k) {
03563
03564             J.init(Sj.len);
03565             for (dbl=0, j=0; j<sj; ++j) {
03566                 Iq=I.append(J); for (q=0; q<Iq.len; ++q) Ia[Pa[q]]=Iq[q];
03567                 Iq=J.append(K); for (q=0; q<Iq.len; ++q) Ib[Pb[q]]=Iq[q];
03568
03569                 if (j==sj-1) Ic=I.append(K);
03570
03571                 dbl += A(Ia) * B(Ib);
03572
03573                 if (J.len>0) {
03574                     q=0; J[0]++;
03575                     while(J[q]>=Sj[q] && q<lj) { J[q]=0; ++J[++q]; }
03576                 }
03577             }
03578             C(Ic) = dbl;
03579
03580             if (K.len>0) {
03581                 q=0; K[0]++;
03582                 while(K[q]>=Sk[q] && q<lk) { K[q]=0; ++K[++q]; }
03583             }
03584         }
03585         if (I.len>0) {
03586             q=0; I[0]++;
03587             while(I[q]>=Si[q] && q<li) { I[q]=0; ++I[++q]; }
03588         }
03589     }
03590
03591     if (pfinal) C.Permute(pfinal);
03592
03593     return C;
03594 };
03595
03596 int tensor_ref_::check(const char *F, int L) const {
03597     int q=0;
03598     if (Ar) {
03599         if (!Az) { q=1; }
03600         else { char s[64]; q=-1;
03601             snprintf(s,64,"%s() multiple refs set (%p/%p)",FCT,Ar,Az);
03602             if (F) wblog(F,L,"ERR %s",s);
03603             else wblog(F_L,"WRN %s",s);
03604         }
03605     }
03606     else if (Az) { q=2; }
03607
03608     if (q<=0 && F) wblog(FL,"ERR %s() invalid array ref (q=%d)",FCT,q);
03609     return q;
03610 };
03611
03612 int tensor_ref_::rank(unsigned r) const {
03613     int q=-1;
03614     if (Ar) {
03615         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03616         q=(Ar->SIZE.len==r);
03617     }
03618     else if (Az) { q=(Az->SIZE.len==r); }
03619     return q;
03620 };
03621
03622 bool tensor_ref_::sameSize(const wvector<size_t> &S) const {
03623     bool q=0;
03624     if (Ar) {
03625         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03626         q=(Ar->SIZE.isEqual(S));
03627     }
03628     else if (Az) { q=(Az->SIZE.isEqual(S)); }
03629     return q;
03630 };
03631
03632 wstring tensor_ref_::sizeStr() const {
03633
03634     wstring s;
03635     if (Ar) {
03636         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03637         s=SSTR_(Ar);
03638     }
03639     else if (Az) { s=SSTR_(Az); }
03640     else { s="[]"; }
03641
03642     return s;
03643 };
03644

```

```

03645 const wvector<size_t>* tensor_ref_::getSIZE() const {
03646
03647     const wvector<size_t> *s=NULL;
03648     if (Ar) {
03649         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03650         s=&(Ar->SIZE);
03651     }
03652     else if (Az) { s=&(Az->SIZE); }
03653
03654     return s;
03655 };
03656
03657 template<>
03658 void tensor_ref_::contract(
03659     const char *F, int L, const ctrIdx &ia,
03660     const tensor_ref_ &b, const ctrIdx &ib, warray<double> &C
03661 ) const {
03662
03663     if (Az || b.Az) wblog(FL,
03664         "ERR %s() got complex array refs (%p/%p)",FCT,Az,b.Az);
03665     if (!Ar || !b.Ar) wblog(FL,
03666         "ERR %s() got NULL array refs (%p/%p)",FCT,Ar,b.Ar);
03667     Ar->contract(F_L,ia,*b.Ar,ib,C,wperm(),1.,0.);
03668 };
03669
03670 template<>
03671 void tensor_ref_::contract(
03672     const char *F, int L, const ctrIdx &ia,
03673     const tensor_ref_ &b, const ctrIdx &ib, warray<wbcomplex> &C
03674 ) const {
03675
03676     wperm P; wbcomplex zero(0);
03677     check(F_L); b.check(F_L);
03678
03679     if (Ar) {
03680         if (b.Ar)
03681             { Ar->contract(F_L,ia,*b.Ar,ib,C, P,1.,zero); }
03682         else { Ar->contract(F_L,ia,*b.Az,ib,C, P,1.,zero); }
03683     }
03684     else {
03685         if (b.Ar)
03686             { Az->contract(F_L,ia,*b.Ar,ib,C, P,1.,zero); }
03687         else { Az->contract(F_L,ia,*b.Az,ib,C, P,1.,zero); }
03688     }
03689 };
03690
03691 char tensorRef_::check(const char* F, int L) const {
03692     char q=0, r;
03693     if (!(S && !R) || !it) { q|=1; if (F) wblog(F_L,
03694         "ERR %s() got uninitialized data (%d/%d)",FCT,S.len,it.len); }
03695     if (S.len && S.len!=it.len) { q|=2; if (F) wblog(F_L,
03696         "ERR %s() length mismatch (S: %d/%d)",FCT,S.len,it.len); }
03697     if (!(r=R.rank(it.len))) { q|=2; if (F) wblog(F_L,
03698         "ERR %s() length mismatch (A: %d/%d)",FCT,r,it.len); }
03699     if (ID.len && ID.len!=it.len) { q|=4; if (F) wblog(F_L,
03700         "ERR %s() length mismatch (ID: %d/%d)",FCT,ID.len,it.len); }
03701     if (S.len && r>0 && !R.sameSize(S)) { q|=8; if (F) wblog(F_L,
03702         "ERR %s() SIZE mismatch %s / %s",FCT,SSTR(S),SSTR(R)); }
03703     return q;
03704 };
03705
03706 char tensorRef_::Check(const char* F, int L, unsigned &k) {
03707     if (!id) { id=(++k); }
03708     if (!R) { if (F) wblog(FL,"ERR %s() got NULL array reference",FCT); }
03709     else if (!S.len) { S.init2ref(*R.getSIZE()); }
03710
03711     return check(F_L);
03712 };
03713
03714 template<class T>
03715 int tensorRef_::contract(const char *F, int L,
03716     const tensorRef_ &b, tensorRef_ &c,
03717     double *flops,
03718     warray<T> *C
03719 ) const {
03720
03721     int nc=0;
03722
03723     if (this==&c || &b==&c) { tensorRef_ x;
03724         nc=contract(F,L,b,x,flops,C);
03725         x.save2(c); return nc;
03726     }
03727
03728     unsigned i,k,l=0;
03729
03730     ctrIdx ia, ib;
03731     wvector<char> ma, mb;

```

```

03732
03733     if ((i=(check() | b.check())) wblog(F_L,
03734         "ERR %s() %s input\n%s\n%s", FCT,
03735         i&l ? "got empty":"inconsistent", STR_(this),STR(b));
03736
03737     nc=this->it.getCtr(
03738         flops? NULL : (F? F : __FILE__),
03739         flops? 1 : (L? L : __LINE__),
03740         b.it, ia,ib,
03741         &c.it, &ma, &mb
03742     );
03743
03744     if (nc<1 && !flops) wblog(F_L,
03745         "ERR %s() invalid contraction (empty match)",FCT);
03746
03747     if (ma.len!=S.len || mb.len!=b.S.len) wblog(F_L,"ERR %s() "
03748         "length mismatch %d/%d ; %d/%d",FCT,ma.len,S.len,mb.len,b.S.len);
03749     for (i=0; i<ia.len; ++i) { if (S[ia[i]]!=b.S[ib[i]]) { wblog(F_L,
03750         "ERR %s() size mismatch\n%-40s @ %s\n%-40s @ %s",
03751         FCT, STR_(this), STR(ia), STR(b), STR(ib));
03752     }}
03753
03754     l=c.it.len;
03755     c.S.init(l); c.ID.init(l);
03756
03757     k=(id<<8);
03758     for (l=i=0; i<S.len; ++i) { if (!ma[i]) {
03759         if (c.it[l]!= it[i]) wblog(F_L,"ERR %s() %d/%d: %s / %s",
03760             FCT,i+1, S.len,STR(c.it[l]), STR(it[i]));
03761         c.ID[l]=(ID.len ? ID[i] : k+i);
03762         c.S[l]= S[i]; ++l;
03763     }}
03764
03765     k=(b.id<<8);
03766     for (i=0; i<b.S.len; ++i) { if (!mb[i]) {
03767         if (c.it[l]!=b.it[i]) wblog(F_L,"ERR %s() %d/%d: %s / %s",
03768             FCT,i+1,b.S.len,STR(c.it[l]),STR(b.it[i]));
03769         c.ID[l]=k+i;
03770         c.S[l]=b.S[i]; ++l;
03771     }}
03772     if (l!=c.S.len) wblog(F_L,"ERR %s() %d/%d",FCT,l,c.S.len);
03773
03774     if (flops) { double f=R.xflops()*b.R.xflops();
03775         if (f==2) { f=4; }
03776
03777         for (i=0; i<ia.len; ++i) { f *= S[ia[i]]; }
03778         for (i=0; i<c.S.len; ++i) { f *= c.S[i]; }
03779         *flops = f;
03780     }
03781
03782     if (C) {
03783         ia.conj= conj;
03784         ib.conj=b.conj;
03785
03786         R.contract(F_L,ia,b.R,ib,*C);
03787         c.R.init(*C);
03788     }
03789
03790     return nc;
03791 };
03792
03793 double tensorRefs::getOptimalCtrOrder(const char *F, int L,
03794     wperm &P,
03795     wperm *pfin,
03796     char useP,
03797     char vflag
03798 ) {
03799
03800     double fmin=-1, flops=0, f;
03801
03802     if (len<2 || len>8) wblog(F_L,
03803         "ERR %s() got %d tensors (max. 8 supported)",FCT,len);
03804
03805     Check(F_L);
03806
03807     unsigned ip=0,i0=0, j; int nc; wperm_t *p;
03808     wbMatrix<wperm_t> PP;
03809     tensorRef_ R, Rmin;
03810
03811     if (vflag) {
03812         if (vflag=='v') vflag=1; else
03813         if (vflag=='V') vflag=3; else
03814         if (vflag>8) wblog(F_L,"WRN %s() got vflag=%s",FCT,cSTR(vflag));
03815     }
03816     if (useP) {
03817         if (P) { if (P.len!=len) wblog(F_L,
03818             "ERR %s() size mismatch (%d/%d)",FCT,P.len,len);

```

```

03819     } else { useP=0; }
03820 }
03821 if (useP)
03822     { PP.init(1,len,P.data); }
03823 else { Wb::getPerms(PP,len); }
03824 p=PP.data;
03825
03826 for (; ip<PP.dim1; ++ip, p+=PP.dim2) {
03827     if (p[0]>p[1]) { continue; }
03828
03829     nc=data[p[0]].contract(FL,data[p[1]], R,f);
03830     if (!nc) { continue; }
03831     flops=f;
03832
03833     for (j=2; j<len; ++j) {
03834         nc=R.contract(FL,data[p[j]], R,f);
03835         if (!nc) { break; }
03836         flops+=f;
03837     }
03838
03839     if (!nc) { continue; }
03840
03841     if (!ip || (fmin>flops)) { fmin=flops; R.save2(Rmin); i0=ip; }
03842
03843     if (vflag>1) { PRINTF(
03844         "%2d [%s] %-64s @%8ld flops\n",
03845         ip, STR(wbperm().init(PP.dim2,p,'r')), STR(R), size_t(flops));
03846     }
03847 }
03848 if (vflag) { print();
03849     wblog(FL,"--> %s() picking ip=%d/%ld\n",FCT,i0,PP.dim1);
03850 }
03851
03852 P.init(PP.dim2,PP.rec(i0));
03853
03854 if (pfin) {
03855     Rmin.ID.Sort(*pfin);
03856 }
03857
03858 return fmin;
03859 };
03860
03861 void tensorRefs::print(const char *F, int L) const {
03862
03863     if (F || !len) wblog(F_L,
03864         "--- tensorRefs listing%s,len? "": " (empty)");
03865
03866     if (len) { unsigned i=0; char istr[16];
03867         PRINTF("\n");
03868         for (; i<len; ++i) {
03869             snprintf(istr,16," %2d/%ld :",i+1,len);
03870             data[i].print(istr, F?'v':0);
03871         }
03872         PRINTF("\n");
03873     }
03874 };
03875
03876 template<class T> inline
03877 bool wbararray<T>::requiresDataPerm(const wbperm &P, char lflag) const {
03878
03879     if (!P.len) { return 0; }
03880
03881     const wperm_t *p=P.data; const size_t *s=SIZE.data;
03882     unsigned i,j, j_=-1;
03883
03884     if (P.len!=SIZE.len) {
03885         if (lflag<0 || lflag>3) wblog(FL,
03886             "WRN %s() got lflag=%s",FCT,cSTR(lflag));
03887         if (!(lflag & (P.len<SIZE.len ? 1 : 2))) { wblog(FL,"ERR %s() "
03888             "invalid permutation (len=%d/%d; %d)",FCT,P.len,SIZE.len,lflag);
03889         }
03890     }
03891     if (numel()<=1) { return 0; }
03892
03893     for (i=0; i<P.len; ++i) { if ((j=p[i])<SIZE.len && s[j]>1) {
03894         if (j_<j || int(j_)<0)
03895             { j_=j; }
03896         else { return 1; }
03897     }}
03898
03899     return 0;
03900 };
03901
03902 template<class T> inline
03903 void wbararray<T>::toMatrixRef(
03904     wbararray<T> &A,
03905     const wbvector<unsigned> &I,

```



```

03906     int pos,
03907     wbpperm &P
03908 ) const {
03909
03910     size_t s1,s2;
03911     groupIndices_P(I,pos,P,&s1,&s2);
03912
03913     if (!requiresDataPerm(P)) { A.init2ref(*this); }
03914     else { permute(A,P); }
03915
03916     A.Reshape(s1,s2);
03917 };
03918
03919 template<class T> inline
03920 bool wbararray<T>::toMatrixRef(const char *F, int L,
03921     wbararray<T> &A,
03922     const wvector<unsigned> &I,
03923     int pos,
03924     char &tflag
03925 ) const {
03926
03927     size_t i,s1,s2; wbpperm P;
03928
03929     int k[2]={pos, pos==1 ? 2 : 1};
03930
03931     for (i=0; i<2; ++i) {
03932         groupIndices_P(I,k[i],P,&s1,&s2);
03933
03934         if (!requiresDataPerm(P)) {
03935             tflag = (pos==k[i] ? 0 : 1);
03936             A.init2ref(*this);
03937             A.Reshape(s1,s2);
03938             return 0;
03939         }
03940     }
03941
03942     if (&A==this) wblog(F_L,"ERR %s() Wb::overlapping output space",FCT);
03943
03944     toMatrixRef(A,I,pos,P); tflag=0;
03945
03946     return 1;
03947 };
03948
03949 template<class T>
03950 void wbararray<T>::toMatrixRef(
03951     wbararray<T> &A,
03952     unsigned K,
03953     const wbpperm &P0
03954 ) const {
03955
03956     size_t i,s1,s2; wbpperm P;
03957
03958     if (K>SIZE.len) wblog(FL,
03959         "ERR %s() index out of bounds (%d/%d)",FCT,K,SIZE.len);
03960
03961     if (!P0.len) { P.Index(SIZE.len); }
03962     else {
03963         if (P0.len!=SIZE.len) wblog(FL,
03964             "ERR %s() rank mismatch (len=%d/%d)",FCT,P0.len,SIZE.len);
03965         P=P0;
03966     }
03967
03968     for (s1=1,i=0; i<K; ++i) { s1*=SIZE[P[i]]; }
03969     for (s2=1; i<P.len; ++i) { s2*=SIZE[P[i]]; }
03970
03971     if (!requiresDataPerm(P))
03972         { A.init2ref(*this); }
03973     else { permute(A,P); }
03974
03975     A.Reshape(s1,s2);
03976 };
03977
03978 template<class T>
03979 void wbararray<T>::toMatrixRefM(
03980     wbararray<T> &A,
03981     unsigned K,
03982     const wbpperm &P0,
03983     unsigned R,
03984     wbpperm pfin
03985 ) const {
03986
03987     size_t i,m=1,s1,s2; wbpperm P;
03988     char use_ref=0;
03989
03990     if (K>R) wblog(FL,
03991         "ERR %s() index out of bounds (K=%d/%d/%d)",FCT,K,R,SIZE.len);
03992     if (R<2 || R>SIZE.len) wblog(FL,

```

```

03993     "ERR %s() invalid rank (R=%d/%d; K=%d)", FCT, R, SIZE.len, K);
03994     if (SIZE.len>R+1) wblog(FL, "WRN %s() "
03995         "got multi-dimensional OM (%s @ %d; K=%d)", FCT, SSTR_(this), R, K);
03996
03997     if (!P0.len) { P.Index(R); }
03998     else {
03999         if (P0.len!=R) wblog(FL,
04000             "ERR %s() rank mismatch (len=%d/%d/%d)", FCT, P0.len, R, SIZE.len);
04001         P=P0;
04002     }
04003
04004     for (s1=1,i=0; i<K; ++i) { s1*=SIZE[P[i]]; }
04005     for (s2=1; i<P.len; ++i) { s2*=SIZE[P[i]]; }
04006     for ( ; i<SIZE.len; ++i) { m *=SIZE[ i ]; }
04007
04008     use_ref=( requiresDataPerm(P) ? 0:1 );
04009     if (pfin.len) {
04010         if (pfin.len!=3) wblog(FL, "ERR %s() invalid pfin='%s'", FCT, STR(pfin));
04011         if (pfin[0]!=0 || m!=1) { use_ref=0; }
04012     }
04013
04014     if (use_ref)
04015         { A.init2ref(*this); }
04016     else { permute(A,P); }
04017
04018     A.Reshape(s1,s2,m);
04019
04020     if (pfin.len) {
04021         A.Permute(pfin);
04022     }
04023 };
04024
04025 template<class T> inline
04026 wbararray<T>& wbararray<T>::save2(wbMatrix<T> &M) {
04027
04028     if (M.data) { M.init(); }
04029
04030     if (SIZE.len!=2) {
04031         if (SIZE.len) wblog(FL, "ERR invalid rank-%d", SIZE.len);
04032         return M;
04033     }
04034
04035     check_consistency(FL);
04036     if (sptr) { M.data=sptr->save_dref(FL); }
04037
04038     M.dim1=SIZE[1]; M.dim2=SIZE[0]; SIZE.init();
04039
04040     return M;
04041 };
04042
04043 #ifdef QS_USING_MPFR
04044
04045 template<> inline
04046 mxArray* wbararray<Wb::quad>::toMx() const {
04047
04048     const char *fields[]={"S","data"};
04049     mxArray *a=mxCreateStructMatrix(1,1,2,fields);
04050
04051     wbvector<Wb::quad> X(numel(),data,'r');
04052
04053     mxSetFieldByNumber(a,0,0, SIZE.toMx());
04054     mxSetFieldByNumber(a,0,1, X.toMx());
04055
04056     return a;
04057 };
04058
04059 template<> inline
04060 wbararray<Wb::quad>& wbararray<Wb::quad>::init(
04061     const char *F, int L, const mxArray *S,
04062     char ref __attribute__ ((unused)),
04063     char vec __attribute__ ((unused))
04064 ){
04065     if (!S || !mxIsStruct(S)) wblog(FL, "ERR %s() "
04066         "got invalid input `%s' / mpfr", FCT, S? mxGetClassName(S):"null");
04067
04068     wbvector<size_t> sz(FL,mxGetFieldByNumber(S,0,0));
04069     wbvector<Wb::quad> X(mxGetFieldByNumber(S,0,1));
04070
04071     if (X.len!=sz.prod(0)) wblog(F_L,
04072         "ERR %s() size mismatch %d/%d", FCT, (int)X.len, (int)sz.prod(0));
04073     if (X.len) init(sz,X.data);
04074
04075     return *this;
04076 };
04077
04078 #endif
04079

```

```

04080 template<class T>
04081 void cell2mat (
04082     const wbMatrix< wbarrRef<T> > &C,
04083     wbararray<T> &M,
04084     WBINDEX &D1, WBINDEX &D2
04085 ){
04086     size_t i,j,m,r,s,d1=0,d2=0,i0,j0;
04087     T *d0, *dd;
04088     double dc,fac;
04090
04091     D1.init(C.dim1);
04092     D2.init(C.dim2);
04093
04094     for (i=0; i<C.dim1; ++i)
04095     for (j=0; j<C.dim2; ++j) { if (C(i,j).D==NULL) continue;
04096
04097         WBINDEX &S = C(i,j).D->SIZE;
04098
04099         if (S.len==2) { d1=S[0]; d2=S[1]; } else
04100         if (S.len==1) { d1=d2=S[0]; }
04101         else wblog(FL,
04102             "ERR %s() requires 1/2-D arrays (got rank-%d).",FCT,S.len);
04103
04104         if (C(i,j).tflag) SWAP(d1,d2);
04105
04106         if (D2[j]==0) D2[j]=d2; else
04107         if (d2!=D2[j]) wblog(FL,
04108             "ERR %s() dimension mismatch (%d,%d: %d/%d).",FCT,i,j,d2,D2[j]);
04109
04110         if (D1[i]==0) D1[i]=d1; else
04111         if (d1!=D1[i]) wblog(FL,
04112             "ERR %s() dimension mismatch (%d,%d: %d/%d).",FCT,i,j,d1,D1[i]);
04113     }
04114
04115     M.init(D1.sum(),D2.sum());
04116
04117     for (j0=j=0; j<C.dim2; ++j, j0+=d2) { d2=D2[j];
04118     for (i0=i=0; i<C.dim1; ++i, i0+=d1) { d1=D1[i];
04119
04120         if (C(i,j).D==NULL) continue;
04121
04122         d0=(C(i,j).D->data); dd=M.ref(i0,j0);
04123         fac=C(i,j).fac;
04124         dc =C(i,j).dc;
04125
04126         if (C(i,j).D->SIZE.len==1) {
04127             if (C(i,j).tflag) wblog(FL,
04128                 "WRN %s() tflag irrelevant for diag-matrix (%d,%d)",
04129                 FCT,i+1,j+1);
04130             if (d1!=d2) wblog(FL,"ERR d1 != d2 !? (%,%d)",d1,d2);
04131
04132             for (r=0; r<d1; ++r, dd+=(M.dim1+1))
04133                 dd[r]+= (dc+fac*d0[r]);
04134
04135             continue;
04136         }
04137
04138         if (C(i,j).tflag==0) {
04139
04140             if (fac==1.) {
04141                 for (s=0; s<d2; ++s, dd+=M.dim1, d0+=d1)
04142                     for (r=0; r<d1; ++r) dd[r]+= d0[r];
04143             }
04144             else if (fac==-1.) {
04145                 for (s=0; s<d2; ++s, dd+=M.dim1, d0+=d1)
04146                     for (r=0; r<d1; ++r) dd[r]-= d0[r];
04147             }
04148             else if (fac!=0.) {
04149                 for (s=0; s<d2; ++s, dd+=M.dim1, d0+=d1)
04150                     for (r=0; r<d1; ++r) dd[r]+= fac*d0[r];
04151             }
04152         }
04153         else {
04154
04155             if (fac==1.) {
04156                 for (s=0; s<d2; ++s, dd+=M.dim1)
04157                     for (r=0; r<d1; ++r) dd[r]+= d0[s+r*d2];
04158             }
04159             else if (fac==-1.) {
04160                 for (s=0; s<d2; ++s, dd+=M.dim1)
04161                     for (r=0; r<d1; ++r) dd[r]-= d0[s+r*d2];
04162             }
04163             else if (fac!=0.) {
04164                 for (s=0; s<d2; ++s, dd+=M.dim1)
04165                     for (r=0; r<d1; ++r) dd[r]+= fac*d0[s+r*d2];
04166             }
04167         }
04168     }
04169 }

```

```

04167     }
04168     if (dc!=0.) {
04169         if (d1!=d2) wblog(FL,
04170             "WRN %s() constant diagonal shift \n"
04171             "requires square block (%d,%d: %d,%d)",FCT,i+1,j+1,d1,d2);
04172
04173         dd=M.ref(i0,j0); m=MIN(d1,d2);
04174
04175         for (r=0; r<m; ++r, dd+=(M.dim1+1))
04176             dd[r]+=dc;
04177     }
04178     }}
04179 };
04180
04181 template<class T>
04182 void wbararray<T>::getReal(wbararray<double> &R) const {
04183     R.initT(*this);
04184 };
04185
04186 template<>
04187 void wbararray<wbcomplex>::getReal(wbararray<double> &R) const {
04188     size_t i=0, n=numel();
04189     R.init(SIZE); for (; i<n; ++i) { R.data[i]=(double)data[i].r; }
04190 };
04191
04192 template<class T>
04193 void wbararray<T>::getImag(wbararray<double> &I) const {
04194     wblog(FL,"ERR wbararray::getImag not defined for type %s",TSTR(T));
04195 };
04196
04197 template<>
04198 void wbararray<double>::getImag(wbararray<double> &I) const {
04199     I.init(SIZE);
04200 };
04201
04202 template<>
04203 void wbararray<wbcomplex>::getImag(wbararray<double> &I) const {
04204     size_t i=0, n=numel();
04205     I.init(SIZE); for (; i<n; ++i) { I.data[i]=data[i].i; }
04206 };
04207
04208 template<class T>
04209 void wbararray<T>::Conj() { return; }
04210
04211 template<>
04212 void wbararray<wbcomplex>::Conj() {
04213     size_t i=0, n=numel();
04214     for (; i<n; ++i) { data[i].i = -data[i].i; }
04215 };
04216
04217 template<class T>
04218 void wbararray<T>::set(
04219     const wbararray<double> &R, const wbararray<double> &I) {
04220     wblog(FL,"ERR wbararray::set(R,I) not defined for type %s",TSTR(T));
04221 };
04222
04223 template<>
04224 void wbararray<wbcomplex>::set(
04225     const wbararray<double> &R_, const wbararray<double> &I_) {
04226
04227     size_t i=0, n;
04228     const double *R=R_.data, *I=I_.data;
04229
04230     if (R) { init(R_.SIZE); n=numel();
04231         if (I) {
04232             if (!I_.sameSize(R_)) wblog(FL,"ERR %s() "
04233                 "dimension mismatch %s / %s",FCT,SSTR(R_),SSTR(I_));
04234             for (; i<n; ++i) { data[i].set(R[i],I[i]); }
04235         }
04236         else { for (; i<n; ++i) { data[i].set(R[i],0.); }}
04237     }
04238     else if (I) {
04239         init(I_.SIZE); n=numel();
04240         for (; i<n; ++i) { data[i].set(0.,I[i]); }
04241     }
04242     else { init(); }
04243 };
04244
04245 #endif
04246

```

5.24 wbararray.hh

```
00001 /* -----
```

```

00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbararray (array class, col-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_COL_MAJOR_HH__
00021 #define __WB_ARRAY_COL_MAJOR_HH__
00022
00023 // #define CHECK_ELEMENT_RANGE
00024
00025 namespace Wb {
00026     template<class T>
00027     wbsttring sizeStrM(
00028         unsigned r, const T* sd, unsigned m, const T* sm,
00029         const char *sepl="x", const char *sepM="_");
00030
00031     template<class T> inline
00032     void householder(const T *u, T *x, size_t n, const T& eps);
00033 };
00034
00035 template <class T>
00036 class wbararray {
00037 public:
00038     wbararray() : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { };
00039
00040     wbararray(size_t d1) : wbararray() { init(d1); };
00041     wbararray(size_t d1, size_t d2) : wbararray() { init(d1,d2); };
00042     wbararray(size_t d1, size_t d2, size_t d3) : wbararray() { init(d1,d2,d3); };
00043     wbararray(size_t d1, size_t d2, size_t d3, size_t d4)
00044         : wbararray() { init(d1,d2,d3,d4); };
00045
00046     wbararray(char *sidx) : wbararray() { init(Str2Idx(sidx)); };
00047     wbararray(const wvector<unsigned> &S) : wbararray() { init(S); };
00048
00049     wbararray(const wvector<size_t> &S, T *d0, char ref=0) : wbararray() {
00050         if (ref)
00051             { INIT2REF(d0,&S); }
00052         else { NEW(S,d0); }
00053     };
00054
00055     wbararray(const wbararray &a, char ref=0)
00056         : wbararray() { init(a,ref); };
00057
00058     template<class T2>
00059     wbararray(const wbararray<T2> &a) : wbararray() {
00060         NEW(a.SIZE); init_T(a.data);
00061     };
00062
00063     wbararray(const mxArray* a, char ref=0, char vec=0)
00064         : wbararray() { init(a,ref,vec); };
00065
00066     wbararray(const char *F, int L, const mxArray* a,
00067         char ref=0,
00068         char vec=0)
00069         : wbararray() { init(F,L,a,ref,vec); };
00070
00071     wbararray(const wMatrix<T> &a, char ref=0)
00072         : wbararray() { init(a,ref); };
00073
00074     virtual ~wbararray() {
00075         DELETE_DATA();
00076     };
00077
00078     wbararray& initX() {
00079         DELETE_DATA('!'); SIZE.init();
00080         return *this;
00081     };
00082
00083     wbararray& init() {
00084         DELETE_DATA();
00085         SIZE.init(); return *this;
00086     };
00087
00088 };

```

```

00089
00090 wbararray& init(size_t d1, const T* d=NULL) {
00091     if (d1) { NEW(wbvector<size_t>(1,d1,'r'),d); } else { init(); }
00092     return *this;
00093 };
00094
00095 wbararray& init(size_t d1, size_t d2, const T* d=NULL) {
00096     size_t s[2]= {d1,d2}; NEW(wbvector<size_t>(2,s,'r'),d);
00097     return *this;
00098 };
00099
00100 wbararray& init(size_t d1, size_t d2, size_t d3, const T* d=NULL) {
00101     size_t s[3]= {d1,d2,d3}; NEW(wbvector<size_t>(3,s,'r'),d);
00102     return *this;
00103 };
00104
00105 wbararray& init(size_t d1, size_t d2, size_t d3, size_t d4, const T* d=NULL) {
00106     size_t s[4]= {d1,d2,d3,d4}; NEW(wbvector<size_t>(4,s,'r'),d);
00107     return *this;
00108 };
00109
00110 wbararray& init2ref(const wbararray &a) {
00111     return INIT2REF(a.data,&a.SIZE);
00112 };
00113
00114 wbararray& initScalar(T x=0, unsigned r=2) {
00115     if (r==2) { init(1,1,&x); } else
00116     if (r==1) { init(1); data[0]=x; } else
00117     if (!r) wblog(FL,"ERR %s() got r=%d (x=%g)",FCT,r,double(x));
00118     else { wbvector<size_t> S; NEW(S.init2val(r,1)); data[0]=x; }
00119     return *this;
00120 };
00121
00122 wbararray& init(const wbararray &a, char ref=0) {
00123     if (this!=&a) {
00124         if (!ref) { NEW(a.SIZE, a.data); }
00125         else { DELETE_DATA(); SIZE=a.SIZE;
00126             a.check_consistency(FL);
00127             if (a.sptr) {
00128                 (sptr=a.sptr)->add_dref();
00129                 data=a.data;
00130             }
00131         }
00132     }
00133     return *this;
00134 };
00135
00136 wbararray& set(const wbararray &A) { return init(A); }
00137
00138 template<class T_>
00139 wbararray& set(const wbararray &A, T_ fac) {
00140     if (fac==T_(1) ) { init(A); }
00141     else if (!A || !fac) { NEW(A.SIZE); }
00142     else {
00143         size_t i=0, n=A.SIZE.prod(0);
00144         const T *a=A.data;
00145         SIZE=A.SIZE; NEW_DATA(n,NULL,0,0);
00146
00147         if (fac==T_(-1))
00148             { for (; i<n; ++i) data[i]= -a[i]; }
00149         else { for (; i<n; ++i) data[i]=fac*a[i]; }
00150     }
00151     return *this;
00152 };
00153
00154 wbararray& init(const wbMatrix<T> &a, char ref=0) {
00155     if (ref) {
00156         size_t s[]={ a.dim2, a.dim1 }; SIZE.init(2,s);
00157         INIT2REF(a.data);
00158     }
00159     else {
00160         size_t i,j, l=0; init(a.dim1,a.dim2);
00161         for (i=0; i<a.dim1; ++i)
00162             for (j=0; j<a.dim2; ++j, ++l) { data[i+a.dim1*j]=a.data[l]; }
00163     }
00164     return *this;
00165 };
00166
00167 wbindex& ind2sub(size_t k, wbindex &I) const {
00168     const size_t *s=SIZE.data, n=SIZE.len;
00169     size_t i,r, e=0, *idx;
00170
00171     I.init(n); idx=I.data;
00172     for (i=0; i<n; ++i) {
00173         if (!s[i]) wblog(FL,
00174             "ERR wbararray::%s() got size %s",FCT,SSTR_(this));
00175     }

```

```

00176         r=k/s[i]; idx[i]=k-r*s[i]; k=r;
00177         if (idx[i]>=s[i]) e++;
00178     }
00179
00180     if (e || k) wblog(F,L,
00181         "ERR wbararray::%s() out of bounds (%d => [%s; %s])",
00182         FCT, STR(I+1), SSTR_(this)
00183     );
00184
00185     return I;
00186 };
00187
00188 void adjustMMat(
00189     const char* F, int L,
00190     char flag, const T fac, wbararray &A
00191 ) const {
00192
00193     if (!isMatrix()) wblog(F,L,
00194         "ERR %d - rank-2 tensor required (%d)",FCT,SIZE.len);
00195     if (!strchr("NTC",flag)) wblog(F,L,
00196         "ERR %d - invalid flag %c<%d>",FCT,flag,flag);
00197
00198     size_t s=numel();
00199
00200     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
00201
00202     if (fac!=T(1) || flag!='N') { size_t i;
00203         if (flag!='N') permute(A,"2 1"); else A=(*this);
00204         if (flag=='C') for (i=0; i<s; i++) A[i]=CONJ(A[i]);
00205         if (fac!=T(1)) for (i=0; i<s; i++) A[i]*=fac;
00206     }
00207     else { A.init2ref(*this); }
00208 }
00209
00210 void adjustDMat(
00211     const char* F, int L,
00212     char flag, const T fac, wbararray &A
00213 ) const {
00214
00215     if (SIZE.len!=1) wblog(F,L,
00216         "ERR %d - 1D representation required (%d)",FCT,SIZE.len);
00217     if (!strchr("NTC",flag)) wblog(F,L,
00218         "ERR %d - invalid flag %c<%d>",FCT,flag,flag);
00219
00220     size_t n=SIZE[0];
00221
00222     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
00223
00224     if (fac!=T(1) || flag=='C') { size_t i; A=(*this);
00225         if (flag=='C') for (i=0; i<n; i++) A[i]=CONJ(A[i]);
00226         if (fac!=T(1)) for (i=0; i<n; i++) A[i]*=fac;
00227     }
00228     else { A.init2ref(*this); }
00229 }
00230
00231 void adjustVec(
00232     const char* F, int L, char &flag, wbararray &A, char pos
00233 ) const { A=(*this); A.adjustVec(F,L,flag,pos); };
00234
00235 void adjustVec(const char* F, int L, char &flag, char pos) {
00236     if (SIZE.len!=1) wblog(F,L,
00237         "ERR %s() vector expected (%d)",FCT,SIZE.len);
00238     if (!strchr("NTC",flag)) wblog(F,L,
00239         "ERR %s() invalid flag %c<%d>",FCT,flag,flag);
00240
00241     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
00242     if (flag=='C') {
00243         for (size_t s=numel(), i=0; i<s; ++i)
00244             data[i]=CONJ(data[i]);
00245     }
00246     flag='N';
00247
00248     SIZE.Resize(2);
00249     if (pos==1) { SIZE[1]=SIZE[0]; SIZE[0]=1; } else
00250     if (pos==2) { SIZE[1]=1; } else
00251     wblog(F,L,"ERR %s() invalid pos=%d",FCT,pos);
00252 };
00253
00254 void adjustCMat(const char* F, int L,
00255     T cfac, size_t s1, size_t s2, char cforce
00256 ){
00257     if (cfac!=T(0)) {
00258         if (data==NULL) {
00259             if (cforce) wblog(F,L,
00260                 "WRN C = A*B + c*[ ] with c=%s !?", Wb::num2Str(cfac).data);
00261         }
00262         else if (!isMatrix() || SIZE[0]!=s1 || SIZE[1]!=s2) {

```

```

00263         wblog(F,L,"ERR %s() dimension mismatch: C=(%s) =? (%d,%d).",
00264             FCT, SSTR_(this), s1, s2); return;
00265     }
00266     else {
00267         if (cfac!=T(1)) {
00268             size_t i, s=SIZE.prod(0);
00269             for (i=0; i<s; i++) data[i]*=cfac;
00270         }
00271         return;
00272     }
00273 }
00274 init();
00275 };
00276
00277 void setBlock(
00278     const wbvector< WBINDEX > &D,
00279     const wbindex &IB,
00280     const wbararray &A
00281 );
00282
00283 void addBlock(
00284     const WBINDEX &I,
00285     const wbararray &A, const char dflag=0
00286 );
00287
00288 wbararray& addBlock(
00289     size_t i, size_t j,
00290     size_t n, size_t m,
00291     wbararray &a
00292 ) const;
00293
00294 T norm2block(
00295     size_t i, size_t j,
00296     size_t n, size_t m
00297 ) const;
00298
00299 wbararray& getBlock(
00300     size_t i, size_t j,
00301     size_t n, size_t m,
00302     wbararray &a
00303 ) const;
00304
00305 wbararray& BlockDiag(const wbvector< wbararray > &D);
00306
00307 wbararray& blockTrace(
00308     size_t D,
00309     wbararray &a, size_t D2=-1
00310 ) const;
00311
00312 void copyStride(T* dd, size_t stride, T afac=1) const;
00313 void addStride (T* dd, size_t stride, T afac=1) const;
00314
00315 template<class T0>
00316 wbararray& init_T(const T0 *dd) {
00317     if (SIZE.len) {
00318         for (size_t n=SIZE.prod(), i=0; i<n; ++i) { data[i]=T(dd[i]); }
00319         return *this;
00320     };
00321
00322     template<class T0>
00323     wbararray& init_T(const char *F, int L, const T0 *dd) {
00324         for (size_t n=SIZE.prod(), i=0; i<n; ++i) { data[i]=T(dd[i]);
00325             if (T0(data[i])!=dd[i]) wblog(F,L,
00326                 "ERR type conversion changes value (%g->%g) !?",
00327                 double(dd[i]), double(data[i])
00328             );
00329         }
00330         return *this;
00331     };
00332
00333     wbararray& initT(const wbararray<T> &a) { return init(a); };
00334
00335     template<class T0>
00336     wbararray& initT(const wbararray<T0> &a) {
00337         NEW(a.SIZE); init_T(a.data);
00338         return *this;
00339     };
00340
00341     template <class T0>
00342     wbararray& initT(const char *F, int L, const wbararray<T0> &a) {
00343         NEW(a.SIZE); init_T(F_L,a.data); return *this;
00344     };
00345
00346     wbararray& init(char *s, T* d=NULL) {
00347         init(Str2Idx(s), d); return *this;
00348     };
00349

```



```

00350     wbararray& init(const WBINDEX &S, T* d=NULL) {
00351         return NEW(S,d);
00352     };
00353
00354     template <class IT>
00355     wbararray& init(const wbvector<IT> &S_, T* d=NULL) {
00356         WBINDEX S(S_.len);
00357         for (size_t i=0; i<S.len; ++i) { S.data[i]=size_t(S_.data[i]); }
00358         return NEW(S,d);
00359     };
00360
00361     wbararray& init_bare(const WBINDEX &S) {
00362         size_t len=S.prod(0);
00363         if (len)
00364             { SIZE=S; NEW_DATA(len,NULL,0,0); }
00365         else { init(); }
00366         return *this;
00367     };
00368
00369     wbararray& init_bare(size_t d1, size_t d2) {
00370         wbvector<size_t> S(2); S[0]=d1; S[1]=d2;
00371         return init_bare(S);
00372     };
00373
00374     wbararray& init(const char *F, int L,
00375         const mxArray *a, char ref=0, char vec=0);
00376
00377     wbararray& init(const char *F, int L,
00378         const mxArray *a, wbpperm &P0);
00379
00380     wbararray& init(const mxArray *a, char ref=0, char vec=0) {
00381         return init(FL,a,ref,vec); }
00382     wbararray& init(const mxArray *a, wbpperm &P0) {
00383         return init(FL,a,P0); }
00384
00385     wbararray& init2ref(const T *x, const WBINDEX &S) {
00386         SIZE=S; return INIT2REF(x);
00387     };
00388     wbararray& init2ref(const T *x) {
00389         return INIT2REF(x);
00390     };
00391
00392     wbararray& init2ref(size_t d1, const T *x) {
00393         SIZE.init(1,&d1);
00394         return INIT2REF(x);
00395     };
00396     wbararray& init2ref(size_t d1, size_t d2, const T *x) {
00397         size_t s[] = {d1,d2}; SIZE.init(2,s);
00398         return INIT2REF(x);
00399     };
00400     wbararray& init2ref(size_t d1, size_t d2, size_t d3, const T *x) {
00401         size_t s[] = {d1,d2,d3}; SIZE.init(3,s);
00402         return INIT2REF(x);
00403     };
00404
00405     int isRef() const { return (sptr && sptr->isRef()); };
00406
00407     wbararray& Instantiate(const char *F=NULL, int L=0) {
00408         if (isRef() && data) {
00409             if (!sptr) wblog(F_L,"ERR %s() got null sptr !?",FCT);
00410             T *d0=data; NEW_DATA(SIZE.prod(0),d0);
00411         }
00412         else if (F) wblog(F,L,
00413             "WRN %s() no need to instantiate %s",SSTR_(this));
00414
00415         return *this;
00416     };
00417
00418     wbararray& unRef() { return Instantiate(); };
00419
00420     void init2Vec(const T *d0, size_t len, char ref=0) {
00421         SIZE.init(1,&len);
00422         if (data!=d0) { NEW_DATA(len,d0,ref); }
00423     };
00424
00425     void init2ref(const wbararray &A, size_t s1, size_t s2) {
00426         if (s1*s2!=A.SIZE.prod(0)) wblog(FL,
00427             "ERR %s() cannot reshape array %s into %dx%d",
00428             SSTR_(this), s1, s2
00429         );
00430
00431         SIZE.init(2); SIZE[0]=s1; SIZE[1]=s2;
00432         INIT2REF(A.data);
00433     };
00434
00435     void initTst();
00436

```

```

00437 void init2Vec(const wbvector<T> &a) {
00438     WBINDEX S(1); S[0]=a.len;
00439     NEW(S, a.data);
00440 };
00441
00442 wbarray& initIdentity(size_t d, char dflag=0, T dval=1);
00443 wbarray& initIdentity(const WBINDEX &S, char dflag=0);
00444 wbarray& Reduce2Id();
00445
00446 wbarray& initIdentityB(
00447     size_t d1, size_t d2, size_t k=0);
00448
00449 wbarray& initIdentityB3(
00450     size_t d1, size_t d2, size_t D, size_t i0, T x=1);
00451
00452 wbarray& Expand2Projector(const T eps=0);
00453 wbarray& ExpandDiagonal(unsigned i1, unsigned i2);
00454 wbarray& ExpandDiagonal();
00455
00456 void Diag2Vec();
00457 wbarray& Reduce2Diag(T eps=0);
00458
00459 wbarray& initDiag(unsigned n, const T *d){
00460     init(n,n);
00461     for (size_t i=0; i<n; ++i) data[i+i*n]=d[i];
00462     return *this;
00463 };
00464
00465 wbarray& initDiag(const wbvector<T> &d){
00466     return initDiag(d.len, d.data); };
00467
00468 void initVec(size_t d, const T val=0) {
00469     init(d); if (val) set(val);
00470 };
00471
00472 wbarray& resize(const wbvector<size_t> &S, wbarray &B) const;
00473
00474 wbarray& resizeNumCols(size_t n, wbarray &B) const {
00475     if (!n && isEmpty()) { return B.init(); }
00476     if (SIZE.len!=2) wblog(FL,
00477         "ERR %s() for matrices only (got %s)",FCT,SSTR_(this));
00478     wbvector<size_t> S(SIZE); S[1]=n;
00479     return resize(S,B);
00480 };
00481
00482 void Resize(const wbvector<size_t> &S, const T* d0=NULL) {
00483     if (d0) { NEW(S,d0); } else
00484     if (S!=SIZE) {
00485         wbarray<T> X;
00486         resize(S,X).save2(*this);
00487     };
00488 };
00489
00490 void Resize(size_t s1, size_t s2) {
00491     size_t s[2]={s1,s2}; wbarray X;
00492     if (SIZE.len!=2) wblog(FL,"ERR rank-2 required (%d)",SIZE.len);
00493     resize(wbvector<size_t>(2,s),X).save2(*this);
00494 };
00495
00496 wbarray& Resize1(unsigned i, size_t Si) {
00497     if (i>=SIZE.len) wblog(FL,
00498         "ERR %s() index out of bounds (%d/%d)",FCT,i,SIZE.len);
00499     if (SIZE[i]!=Si) {
00500         wbvector<size_t> S(SIZE); wbarray X; S[i]=Si;
00501         resize(S,X).save2(*this);
00502     }
00503     return *this;
00504 };
00505
00506 wbarray& ResizeM(unsigned r, const wbvector<size_t> &Sm);
00507
00508 wbarray& Enlarge(
00509     size_t dim,
00510     size_t D,
00511     size_t istart=0
00512 );
00513
00514 void swap(wbarray &A) {
00515     SWAP(sptr, A.sptr );
00516     SWAP(data, A.data );
00517     SWAP(mtype,A.mtype); SIZE.swap(A.SIZE);
00518 };
00519
00520 wbarray& save2(wbarray &A) {
00521     if (this!=&A) {
00522         if (A.mtype) wblog(FL,"ERR %s() got %s / %s",
00523             FCT, Wb::MTYPE_STR[mtype], Wb::MTYPE_STR[A.mtype]);

```

```

00524         this->swap(A); if (data!=A.data) { init(); }
00525     }
00526     return A;
00527 };
00528
00529 wbararray& save2(wbMatrix<T>&);
00530
00531 wbararray& Append2(
00532     const char *F, int L, wbararray &B, unsigned dim);
00533 wbararray& Append2(wbararray &B, unsigned dim){
00534     return Append2(0,0,B,dim);
00535 }
00536
00537 void Append2(
00538     const char *F, int L,
00539     wbararray &B, unsigned dim,
00540     wbararray<double> &W, double eps
00541 );
00542 void Append2(wbararray &B, unsigned dim, wbararray &W, double eps){
00543     return Append2(0,0,B,dim,W,eps);
00544 };
00545
00546 void rand(const UVEC &d) { init(d); setRand(); };
00547 void setRand(const char pnflag=0);
00548
00549 wbararray& Reset(const T *dref) {
00550     size_t n=SIZE.prod();
00551     if (dref) { memcpy(data,dref,sizeof(T)*n); }
00552     else { MEM_SET<T>(data,n); }
00553     return *this;
00554 };
00555
00556 wbararray& set(const T& c) {
00557     size_t n=SIZE.prod();
00558     if (c)
00559         { for (size_t i=0; i<n; ++i) data[i]=c; }
00560     else { MEM_SET<T>(data,n); }
00561     return *this;
00562 };
00563
00564 wbararray& operator= (const wbararray &a) { return init(a); };
00565
00566 template <class T0>
00567 wbararray<T>& operator= (const wbararray<T0> &a) {
00568     if ((void*)this!=(void*)&a) {
00569         NEW(a.SIZE);
00570         for (size_t n=a.numel(), i=0; i<n; ++i) data[i]=T(a.data[i]);
00571     }
00572     return *this;
00573 };
00574
00575 void SetRef(const widx_t *I, size_t len, const T* d0);
00576 void SetRef(size_t k, const T* d0) { SetRef(&k,1,d0); };
00577
00578 const T* ref(const widx_t *I, size_t n) const {
00579     if (!n) return NULL;
00580     return data+serial_index(I,n);
00581 };
00582 T* ref(const widx_t *I, size_t n) {
00583     if (!n) return NULL;
00584     return data+serial_index(I,n);
00585 };
00586
00587 T* col(size_t k) {
00588     if (SIZE.len!=2 || k>=SIZE[1]) wblog(FL,
00589         "ERR %s() index out of bounds (%s; %d)",FCT,SSTR_(this),k);
00590     return (data+k*SIZE[0]);
00591 };
00592 const T* col(size_t k) const {
00593     if (SIZE.len!=2 || k>=SIZE[1]) wblog(FL,
00594         "ERR %s() index out of bounds (%s; %d)",FCT,SSTR_(this),k);
00595     return (data+k*SIZE[0]);
00596 };
00597
00598 T* row(size_t k) {
00599     if (SIZE.len!=2 || k>=SIZE[0]) wblog(FL,
00600         "ERR %s() index out of bounds (%d / %s)",FCT,k,SSTR_(this));
00601     return (data+k);
00602 };
00603 const T* row(size_t k) const {
00604     if (SIZE.len!=2 || k>=SIZE[0]) wblog(FL,
00605         "ERR %s() index out of bounds (%d / %s)",FCT,k,SSTR_(this));
00606     return (data+k);
00607 };
00608
00609 wbvector<T>& getCol(size_t k, wbvector<T> &v) const;
00610 wbararray<T>& getCol(size_t k, wbararray<T> &a) const;

```

```

00611
00612     wbvector<T>& getRow(size_t k, wbvector<T> &v) const;
00613     wbarray<T>& getRow(size_t k, wbarray<T> &a) const;
00614
00615     wbvector<T> getDiag() const;
00616
00617     T* ref(const WBINDEX &I) { return ref(I.data,I.len); };
00618     const T* ref(const WBINDEX &I) const {
00619         return ref(I.data,I.len);
00620     };
00621
00622     const T* ref(widx_t i) const { return ref(&i,1); }
00623     T* ref(widx_t i) { return ref(&i,1); }
00624
00625     const T* ref(widx_t i1, widx_t i2) const {
00626         widx_t I[]={i1,i2}; return ref(I,2);
00627     };
00628     T* ref(widx_t i1, widx_t i2) {
00629         widx_t I[]={i1,i2}; return ref(I,2);
00630     };
00631
00632     const T* ref(widx_t i1, widx_t i2, widx_t i3) const {
00633         widx_t I[]={i1,i2,i3}; return ref(I,3);
00634     };
00635     T* ref(widx_t i1, widx_t i2, widx_t i3) {
00636         widx_t I[]={i1,i2,i3}; return ref(I,3);
00637     };
00638
00639     const T& operator[] (size_t i) const {
00640         return data[i];
00641     };
00642     T& operator[] (size_t i) {
00643         return data[i];
00644     }
00645
00646     const T& operator() (size_t i) const {
00647         if (SIZE.len!=1) wblog(FL,
00648             "ERR %s requires vector type (%s)",FCT,SSTR_(this));
00649         return data[i];
00650     };
00651     T& operator() (size_t i) {
00652         if (SIZE.len!=1) wblog(FL,
00653             "ERR %s requires vector type (%s)",FCT,SSTR_(this));
00654         return data[i];
00655     };
00656
00657     const T& operator() (size_t i, size_t j) const {
00658         if (SIZE.len!=2) wblog(FL,
00659             "ERR %s(i,j) requires matrix type (%s)",FCT,SSTR_(this));
00660         return data[ i + j*SIZE[0] ];
00661     };
00662     T& operator() (size_t i, size_t j) {
00663         if (SIZE.len!=2) wblog(FL,
00664             "ERR %s(i,j) requires matrix type (%s)",FCT,SSTR_(this));
00665         return data[ i + j*SIZE[0] ];
00666     };
00667
00668     const T& operator() (size_t i, size_t j, size_t k) const {
00669         if (SIZE.len!=3) wblog(FL,
00670             "ERR %s(i,j,k) requires 3D-object (%s)",FCT,SSTR_(this));
00671         return data[ i + SIZE[0] * (j + SIZE[1]*k) ];
00672     };
00673     T& operator() (size_t i, size_t j, size_t k) {
00674         if (SIZE.len!=3) wblog(FL,
00675             "ERR %s(i,j,k) requires 3D-object (%s)",FCT,SSTR_(this));
00676         return data[ i + SIZE[0] * (j + SIZE[1]*k) ];
00677     };
00678
00679     const T& operator() (const WBINDEX &I) const {
00680         if (I.len!=SIZE.len) wblog(FL,
00681             "ERR %s() invalid index [%s]",FCT,STR(I));
00682         return data[serial_index(I.data,I.len)];
00683     };
00684     T& operator() (const WBINDEX &I) {
00685         if (I.len!=SIZE.len) wblog(FL,
00686             "ERR invalid index [%s] having %s",STR(I),SSTR_(this));
00687         return data[serial_index(I.data,I.len)];
00688     };
00689
00690     const T& element(const WBINDEX &I) const {
00691         if (I.len!=SIZE.len) wblog(FL,"ERR %s() invalid index [%s] "
00692             "having %s",FCT,STR(I),SSTR_(this));
00693         for (size_t i=0; i<I.len; i++) if (I.data[i]>=SIZE.data[i])
00694             wblog(FL,"ERR %s() index out of bounds (%s; %s)",
00695                 FCT,STR(I),SSTR_(this));
00696         return data[serial_index(I.data,I.len)];
00697     };

```

```

00698     T& element(const WBINDEX &I) {
00699         if (I.len!=SIZE.len) wblog(FL,"ERR %s() invalid index [%s] "
00700             "having %s",FCT,STR(I),SSTR_(this));
00701         for (size_t i=0; i<I.len; i++) if (I.data[i]>=SIZE.data[i])
00702             wblog(FL,"ERR %s() index out of bounds (%s; %s)",
00703                 FCT,STR(I),SSTR_(this));
00704         return data[serial_index(I.data,I.len)];
00705     };
00706
00707     size_t numel() const {
00708         size_t N=0;
00709         if (SIZE.len) {
00710             const size_t *s=SIZE.data; size_t i=0;
00711             N=s[i]; for (++i; i<SIZE.len; ++i) { N*=s[i]; }
00712         }
00713         return N;
00714     };
00715
00716     size_t numOM(unsigned r0) const {
00717         size_t M=1;
00718         if (!SIZE.len) { return (M=0); }
00719         if (r0<2) wblog(FL,"ERR %s() invalid r0=%d/%d",FCT,r0,SIZE.len);
00720         if (r0<SIZE.len) {
00721             const size_t *s=SIZE.data; size_t i=r0;
00722             M=s[i]; for (++i; i<SIZE.len; ++i) { M*=s[i]; }
00723         }
00724         if (r0==2 && M>1) wblog(FL,
00725             "ERR %s() got OM=%d for %s @ r0=%d",FCT,M,SSTR_(this),r0);
00726     }
00727     return M;
00728 };
00729
00730     wstring sizeStr(unsigned stride=0, const char *sep2="") const {
00731         if (SIZE.len)
00732             { return SIZE.toStrf("", "x", stride, sep2); }
00733         else { return wstring(""); }
00734     };
00735
00736     wbararray& Squeeze();
00737
00738     int skipSingletons(const char *F, int L, unsigned r=-1);
00739     int skipSingletons(unsigned r=-1) {
00740         return skipSingletons(FL,r);
00741     };
00742
00743     wbararray& appendSingletons(
00744         const char *F, int L, unsigned r, unsigned m=0);
00745     wbararray& appendSingletons(unsigned r, unsigned m=0) {
00746         return appendSingletons(0,0,r,m); };
00747
00748     void prependSingletons(unsigned r);
00749
00750     wbararray& ExpandOM(const char *F, int L, unsigned r0,
00751         const wvector<unsigned> &S,
00752         const unsigned *sx=NULL);
00753
00754     wbararray& ExpandOM(const char *F, int L, unsigned r0,
00755         unsigned m, const unsigned *s, const unsigned *sx=NULL
00756     ){ return ExpandOM(F,L,r0, wvector<unsigned>(m,s,'r'), sx); };
00757
00758     int ExpandOM(const char *F, int L, wbararray &B, unsigned r0);
00759
00760     wbararray& FuseOM(const char *F, int L, unsigned r, unsigned mc=-1) {
00761         if (!SIZE.len) { return *this; }
00762
00763         unsigned i, m=1;
00764         for (i=r; i<SIZE.len; ++i) { m*=SIZE[i]; }
00765
00766         if (r<2 && m>1) wblog(F_L,
00767             "ERR %s() requesting OM m=%d for rank r=%d/%d",FCT,m,r,SIZE.len);
00768         if (r>SIZE.len) wblog(F_L,
00769             "ERR %s() rank out of bounds (r=%d/%d; m=%d)",FCT,r,SIZE.len,m);
00770         if (int(mc)>=0 && m!=mc && (m!=1 || mc>1)) wblog(F_L,"ERR %s() "
00771             "got OM mismatch %s @ %d/%d (r=%d)",FCT,SSTRM(SIZE,r),m,mc,r);
00772
00773         if (m==1 && r)
00774             { SIZE.len=r; }
00775         else { SIZE[r]=m; SIZE.len=r+1; }
00776
00777         return *this;
00778     };
00779
00780     size_t length() const {
00781         size_t l=0; const size_t *s=SIZE.data;
00782         for (unsigned i=0; i<SIZE.len; ++i) {
00783             if (l<s[i]) { l=s[i]; } else
00784                 if (!s[i]) { l=0; break; }

```

```

00785     }
00786     return 1;
00787 }
00788
00789 size_t size(unsigned i) const { return SIZE[i]; };
00790
00791 size_t dim (unsigned i) const {
00792     if (i==0 || i>SIZE.len) wblog(FL,
00793         "ERR index out of bounds (%d/%d)", i-1, SIZE.len);
00794     return SIZE[i-1];
00795 };
00796 size_t dim0(unsigned i) const {
00797     if (i>=SIZE.len) wblog(FL,
00798         "ERR index out of bounds (%d/%d)", i, SIZE.len);
00799     return SIZE[i];
00800 };
00801
00802 size_t dim1() const {
00803     size_t D=0;
00804     if (SIZE.len==1) { D=SIZE[0]; } else
00805     if (SIZE.len) {
00806         unsigned i=0, n=SIZE.len, z=0, l=-1; const size_t *s=SIZE.data;
00807         for (; i<n; ++i) {
00808             if (s[i]>1) {
00809                 if (l>i) { l=i; } else
00810                     wblog(FL, "ERR %s() invalid vector %s", FCT, SSTR_(this));
00811             }
00812             else if (!s[i]) { ++z; }
00813         }
00814
00815         if (!z) { D=s[l<n ? l : 0]; } else
00816         if (n>2 || l<n) {
00817             wblog(FL, "ERR %s() invalid vector %s", FCT, SSTR_(this));
00818         }
00819     }
00820     return D;
00821 };
00822
00823 size_t dim2() const {
00824     if (!SIZE.len) return 0;
00825     if (SIZE.len!=2) wblog(FL,
00826         "ERR %s() got rank-%d array (%s) !?", FCT, SIZE.len, SSTR_(this));
00827     if (SIZE[0]!=SIZE[1]) wblog(FL,
00828         "ERR %s() got non-square matrix (%s) !?", FCT, SSTR_(this));
00829     return SIZE[0];
00830 };
00831
00832 void getMatSize(const char *F, int L, size_t &dim1, size_t &dim2) const;
00833 void getMatSize(size_t &dim1, size_t &dim2, unsigned m) const;
00834 void getMatSize(const ctrIdx &ic, size_t &dimc, size_t &dimk,
00835     unsigned r=-1, size_t *dom=NULL) const;
00836
00837 unsigned rank() const { return SIZE.len; };
00838
00839 bool isRank(unsigned r) const {
00840     bool q=1;
00841     if (r!=SIZE.len) { unsigned i=r;
00842         for (; i<SIZE.len; ++i) { if (SIZE[i]!=1) break; }
00843         if (i!=SIZE.len) { q=0; }
00844     }
00845     return q;
00846 };
00847
00848 bool isRankM(unsigned r, unsigned m) const {
00849     bool q=1;
00850     if (SIZE.len==r) { return q; }
00851     if (r>2 && m) { if (SIZE.len==r+1 || SIZE.len==r+m ) { return q; }}
00852     return (q=0);
00853 };
00854
00855 bool isOpS(size_t *n=NULL) const;
00856 bool isOpS(unsigned &n) const { bool q;
00857     size_t N; q=isOpS(&N); n=N;
00858     if (size_t(n)!=N) wblog(FL,
00859         "ERR %s() unsigned out of bounds (%d/%ld)", FCT, n, N);
00860     return q;
00861 };
00862
00863 wbarray< SkipTiny_float(const T eps=1E-14) { return *this; };
00864 double SkipTiny_imag (double eps=1E-14) { return 0; };
00865
00866 double SkipTiny(double eps_=1E-14);
00867
00868 wbarray<T>& SkipTrailingZeroSpace(
00869     unsigned k,
00870     size_t n=0,
00871     double eps=1e-14);

```

```

00872
00873     bool isMatrix() const { return SIZE.len==2; }
00874     bool isMatrix(size_t d1, size_t d2) {
00875         return (SIZE.len==2 && SIZE[0]==d1 && SIZE[1]==d2);
00876     };
00877
00878     bool isSMatrix(unsigned d=-1) const {
00879         bool q=0;
00880         if (SIZE.len==2 && SIZE[0]==SIZE[1] &&
00881             (int(d)<0 || d==SIZE[0])) { q=1; }
00882         return q;
00883     };
00884
00885     bool isSquare() const {
00886         if (SIZE.len%2) return 0;
00887         else {
00888             for (unsigned i=0, n=SIZE.len/2; i<n; ++i) {
00889                 if (SIZE[i]!=SIZE[n+i]) return 0; }
00890             }
00891         return 1;
00892     };
00893
00894     bool isEmpty() const { return (data==NULL); };
00895     bool isComplex() const;
00896     bool isZero(double eps=0., char flag=0) const;
00897
00898     bool isFinite() const {
00899         bool q=1; size_t i=0, n=numel();
00900         for (; i<n; ++i) { if (!isfinite(data[i])) { q=0; break; } }
00901         return q;
00902     };
00903
00904     explicit operator bool() const { return !isEmpty(); }
00905     bool operator! () const { return isEmpty(); }
00906
00907     wbararray& Symmetrize(
00908         const char *F, int L,
00909         double *delta=NULL, char cflag=1, char tflag=0, char fflag=0
00910     );
00911
00912     wbararray& swapRows(size_t i1, size_t i2);
00913     wbararray& swapCols(size_t j1, size_t j2);
00914
00915     wbararray& setCol(size_t k, size_t k0 );
00916     wbararray& setCol(size_t k, const T* d, T fac=1, size_t stride=1);
00917     wbararray& setRow(size_t k, const T* d);
00918
00919     wbararray& setCol(size_t k, const wvector<T> &v);
00920     template<class T2>
00921     wbararray<T>& setCol(size_t k, const wvector<T2> &v);
00922
00923     wvector<T>& colNorm2(wvector<T> &a) const;
00924
00925     T colNorm2(size_t k) const;
00926
00927     wvector<T>& normDim2(unsigned k, wvector<T> &xk) const;
00928     wvector<T>& normDim (unsigned k, wvector<T> &xk) const {
00929         normDim2(k,xk);
00930         for (size_t n=xk.len, i=0; i<n; ++i) { xk[i]=SQRT(xk[i]); }
00931         return xk;
00932     };
00933
00934     unsigned QRdecomp(
00935         const char *F, int L, wbararray<T> &Q, wbararray<T> &R,
00936         T eps=1E-15);
00937
00938     int Householder(
00939         const char *F, int L,
00940         size_t k, T *u,
00941         WBPPerm *P=NULL,
00942         T eps=1E-15);
00943
00944     unsigned eigTriDiag(
00945         const char *F, int L, wvector<T> &E,
00946         wbararray<T> *U=NULL, T eps=1E-15) const;
00947
00948     wbararray& ColProject(
00949         size_t k1, size_t k2, char nflag=0, char tnorm=0);
00950
00951     wbararray& ColPermute(const wbpPerm &P, char iflag=0);
00952
00953     bool isOrthogonalCol(size_t k, char tnorm=0) const;
00954     bool isOrthoCols(T *x2=NULL, char tnorm=0, T eps=1E-14) const;
00955
00956     wbararray& SignCol(size_t k, const T *d0, char tnorm=0);
00957     wbararray& FlipSignCol(size_t k);
00958     wbararray& SignConventionCol(

```

```

00959     size_t k=-1,
00960     double eps=1E-12
00961 );
00962
00963 T NormalizeCol(size_t k, char tnorm=0, char qflag=0);
00964 wbararray& NormalizeCols(
00965     const char *F, int L,
00966     double *amin=NULL, double *amax=NULL, char tnorm=0
00967 );
00968
00969 wbararray& OrthoNormalizeCols(
00970     const char *F=NULL, int L=0, char tnorm=0,
00971     char qxflag=0, double eps=1E-14, unsigned np=1
00972 );
00973
00974 wbararray& balanceOp(
00975     const char *F, int L, T &xref, double &xscale
00976 );
00977
00978 bool isProptoId(T &x, T eps=1E-14) const;
00979
00980 bool isProptoId(T *x_=NULL, T eps=1E-14) const {
00981     bool q=0; T x=T(0);
00982     if (isProptoId(x,eps)) { q=1; if (x_) { (*x_)=x; } }
00983     return q;
00984 };
00985
00986 bool isDiagMatrix(double eps=1E-14) const {
00987     return isDiag_aux(eps,"isDiag"); }
00988
00989 bool isIdentityMatrix(double eps=1E-14) const {
00990     if (data && ABS(data[0]-double(1))<eps) {
00991         if ((SIZE.len%2)==0) {
00992             if (SIZE.allEqual(1)) { return 1; }
00993             else { T one(1);
00994                 return isProptoId(one,eps);
00995             }
00996         }
00997     }
00998     return 0;
00999 };
01000
01001 bool isDiagMatrix(double *eps) const {
01002     return isDiag_aux(eps,"isDiag"); }
01003
01004 bool isIdentityMatrix(double *eps) const {
01005     return isDiag_aux(eps,"isIdty"); }
01006
01007 size_t nnz(const T eps=0, WBINDEX *I=NULL) const;
01008
01009 double sparsity(const T eps=0) const {
01010     size_t n=numel();
01011     return (n ? nnz(eps)/double(n) : 0);
01012 };
01013
01014 bool isHConj(
01015     const wbararray &B, double eps=1E-12, double* xref=NULL
01016 ) const { return isSym_aux(FLF, B, eps, xref, 's'); };
01017
01018 bool isHConj(
01019     double eps=1E-12, double* xref=NULL
01020 ) const { return isSym_aux(FLF,*this,eps,xref,'s'); };
01021
01022 bool isAHerm(
01023     const wbararray &B, double eps=1E-12, double* xref=NULL
01024 ) const { return isSym_aux(FLF, B, eps, xref, 'a'); };
01025
01026 bool isAHerm(
01027     double eps=1E-12, double* xref=NULL
01028 ) const { return isSym_aux(FLF,*this,eps,xref,'a'); };
01029
01030 char hasGroupSize(size_t s1, size_t s2) const;
01031
01032 template<class TI>
01033 int aMax(T &x, TI &k) const;
01034
01035 T aMax() const {
01036     T x; size_t k; int e=aMax(x,k);
01037     if (e) wblog(FL,"WRN %s() got empty array (e=%d)",FCT,e);
01038     return x;
01039 };
01040
01041 template<class TI>
01042 T aMax(TI &i, TI &j, const wbararray *B=NULL) const;
01043
01044 double maxDiff (const wbararray &B) const;
01045 T froNorm2(const wbararray &B) const;

```



```

01046     T norm2() const;
01047     T norm() const { return Sqrt(norm2()); };
01048
01049     T hasNorm2(const T &x2) const;
01050
01051     T normCol2(size_t k) const;
01052
01053     T norm2Recs(long i1, long i2) const;
01054     T norm2Cols(long j1, long j2) const;
01055
01056     wbvector<T>& norm2Cols(wbvector<T> &x2) const;
01057     wbvector<T> normCols() const;
01058
01059     T scalarProd(const wbararray &B) const;
01060     T sum() const;
01061
01062     wbararray& sum(const WBINDEX &I, wbararray&) const;
01063     wbararray& sum(const char *sidx, wbararray &A) const {
01064         sum(Str2Idx(sidx,1), A);
01065         return A;
01066     };
01067
01068     char fitsSize(const wbararray &, char strict=0) const;
01069
01070     size_t getSize(unsigned i, char lflag=1) const {
01071         size_t s=0;
01072         if (i<SIZE.len) { s=SIZE[i]; } else
01073             if (lflag && SIZE.len) {
01074                 for (i=0; i<SIZE.len; ++i) { if (!SIZE[i]) { break; }; }
01075                 if (i==SIZE.len) { s=1; }
01076             }
01077         return s;
01078     };
01079
01080     wbvector<size_t>& getSize(
01081         const wbvector<unsigned> &I, wbvector<size_t> &S
01082     ) const { return SIZE.select(I,S); };
01083
01084     size_t getSizeM(unsigned r) {
01085         size_t M=1;
01086         if (r>2 && SIZE.len==r+1) { M=SIZE[r]; }
01087         else if (SIZE.len!=r) wblog(FL,
01088             "ERR %s() unexpected rank r=%d/%d",FCT,r,SIZE.len);
01089         return M;
01090     };
01091
01092     bool sameSize(const size_t *s, unsigned n) const;
01093
01094     bool sameSize(size_t d1, size_t d2) const {
01095         size_t s[2]={d1,d2}; return sameSize(s,2);
01096     };
01097
01098     template <class Ts>
01099     bool sameSizeM(unsigned r, unsigned m, const Ts *s) const {
01100         bool rval=0; unsigned n=r+m;
01101         if (n==SIZE.len) { unsigned i=0;
01102             for (; i<m; ++i) { if (SIZE[r+i]!=s[i]) { break; }; }
01103             if (i>=m) { rval=1; }
01104         }
01105         return rval;
01106     };
01107
01108     template<class TB>
01109     bool sameSize(const wbararray<TB> &, char lflag=2) const;
01110
01111     bool sameSize1(const size_t *s, unsigned n) const;
01112
01113     template<class TB>
01114     bool hasSameSize(
01115         unsigned ia, const wbararray<TB> &B, unsigned ib, char lflag=1
01116     ) const {
01117         size_t sa=getSize(ia,lflag), sb=B.getSize(ib,lflag);
01118         return (sa==sb);
01119     };
01120
01121     char sameUptoFac(
01122         const wbararray &B, T *fac=NULL, double eps=1e-12
01123     ) const;
01124
01125     bool sameAs(const wbararray &B, double eps=1e-12) const;
01126
01127     bool operator== (const wbararray &B) const { return sameAs(B); };
01128     bool operator!= (const wbararray &B) const { return !sameAs(B); };
01129
01130     T normDiff2(const wbararray &B, char sflag=0) const;
01131     T normDiff (const wbararray &B, char sflag=0) const {
01132         return Sqrt(normDiff2(B,sflag)); };

```

```

01133
01134     T normDiff2(const wbvector<T> &B) const;
01135     T normDiff (const wbvector<T> &B) const { return Sqrt(normDiff2(B)); };
01136
01137     void TimesEl(const wbararray &, char conj=0);
01138     wbararray& timesEl(
01139         const wbararray &B, wbararray &C, T bfac=1, T cfac=0, char conj=0) const;
01140     wbararray& timesEl_OM(unsigned r,
01141         const wbararray &B, wbararray &C, char conj=0) const;
01142
01143     T dotProd(const wbararray &, char conj=0) const;
01144     T weightedAvg(const wbararray &) const;
01145
01146     void TensorProd(const wbararray &B,
01147         const char aflag='N', const char bflag='N', const char kflag=0
01148     ){
01149         wbararray X(*this);
01150         X.tensorProd(B,*this,aflag,bflag,kflag);
01151     };
01152
01153     wbararray& tensorProd(
01154         const wbararray &, wbararray &,
01155         const char aflag='N', const char bflag='N', const char kflag=0
01156     ) const;
01157
01158     wbararray& kron(
01159         const wbararray &B, wbararray &X, char aflag='N', char bflag='N'
01160     ) const { return tensorProd(B,X,aflag,bflag,'k'); };
01161
01162     wbararray& Kron(const wbararray &B, char aflag='N', char bflag='N') {
01163         wbararray X; tensorProd(B,X,aflag,bflag,'k');
01164         return X.save2(*this);
01165     };
01166
01167     wbararray& Cat(unsigned dim, const wbvector< wbararray* > &ap);
01168     wbararray& Cat(unsigned dim, const wbvector< wbararray > &ap);
01169
01170     wbararray& BlockCat (
01171         const wbararray< wbararray > &aa,
01172         WBINDEX *D1=NULL, WBINDEX *D2=NULL
01173     ){
01174         wbararray< const wbararray* > ap(aa.SIZE);
01175         for (size_t n=aa.numel(), i=0; i<n; ++i) { ap[i]=&aa[i]; }
01176         return BlockCat(ap,D1,D2);
01177     };
01178
01179     wbararray& BlockCat (
01180         const wbararray< const wbararray* > &ap,
01181         WBINDEX *D1=NULL, WBINDEX *D2=NULL
01182     );
01183
01184     void operator+= (T x) {
01185         if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
01186         for (size_t n=numel(), i=0; i<n; ++i) { data[i]+=x; }
01187     };
01188
01189     void operator-= (T x) {
01190         if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
01191         for (size_t n=numel(), i=0; i<n; ++i) { data[i]-=x; }
01192     };
01193
01194     template <class T_>
01195     wbararray& Times(const T_ a, char rcpy=0) {
01196         if (a!=T_(1)) {
01197             if (!rcpy || !isRef()) { return (*this)*=a; }
01198             else {
01199                 wbararray<T> X; this->times(a,X);
01200                 X.save2(*this);
01201             }
01202         }; return *this;
01203     };
01204
01205     template <class T_>
01206     wbararray& times(const T_ a, wbararray<T> &X) const {
01207         if (&X==this) { X*=a; } else
01208         if (a==T_(1)) { X.init(*this); } else
01209         if (!a) { X.init(SIZE); } else { X.set(*this,a); }
01210         return X;
01211     };
01212
01213     template <class T_>
01214     wbararray& operator*=(const T_ a) {
01215         if (a!=T_(1)) { size_t i=0, n=numel();
01216             if (isRef()) wblog(FL,"ERR %s() got ref (considered const)",FCT);
01217             if (!a) { MEM_SET<T>(data,n); } else
01218             if (a==T_(-1))
01219                 { for (; i<n; ++i) { data[i] = -data[i]; } }

```

```

01220         else { for (; i<n; ++i) { data[i] *= a; }}
01221     }
01222     return *this;
01223 };
01224
01225 template <class T_>
01226 wbararray& operator/=(const T_ &a) {
01227     if (SIZE && a!=T_(1)) { size_t i=0, n=numel();
01228         if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
01229         if (!a) wblog(FL,"ERR %s() got div/0",FCT);
01230
01231         if (a==T_(-1)) for (; i<n; ++i) { data[i] = -data[i]; } else
01232         if (n<4 || WbUtil<T>::isInt()) { for (; i<n; ++i) { data[i]/=a; }}
01233         else {
01234             T x=1/T(a);
01235             if (fabs(double(x*a-1))>1E-12)
01236                 { for (; i<n; ++i) { data[i]/=a; }}
01237             else { for (; i<n; ++i) { data[i]*=x; }}
01238         }
01239     }
01240     return *this;
01241 };
01242
01243 void operator+= (const wbararray &B) { Plus(B); };
01244 void operator-= (const wbararray &B) { Plus(B,-1); };
01245
01246 wbararray& plus(
01247     const wbararray &B, wbararray &C, T bfac=1, char iflag=0) const;
01248
01249 template<class TB>
01250 wbararray& Plus(
01251     const wbararray<TB> &B, TB bfac=TB(1),
01252     char iflag=0, T afac=T(1));
01253
01254 bool equal (const wbararray&, T eps) const;
01255 bool unequal(const wbararray&, T eps) const;
01256 bool equal (const wbararray&, double eps, double& maxdiff) const;
01257 bool unequal(const wbararray&, double eps, double& maxdiff) const;
01258
01259 bool allEqual(const T &x) {
01260     for (size_t n=numel(), i=0; i<n; ++i) { if (data[i]!=x) return 0; }
01261     return 1;
01262 };
01263 bool allLE(const T &x) {
01264     for (size_t n=numel(), i=0; i<n; ++i) { if (data[i]>x) return 0; }
01265     return 1;
01266 };
01267
01268 char isVector() const {
01269     if (SIZE.len==1) return 1;
01270     if (SIZE.len!=2 || (SIZE[0]!=1 && SIZE[1]!=1)) return 0;
01271     if (SIZE[0]==1 && SIZE[1]==1) return -1;
01272     else return 2;
01273 };
01274
01275 unsigned isScalar() const {
01276     unsigned rval=0;
01277     if (SIZE.len && SIZE.allEqual(1)) { rval=SIZE.len; }
01278     return rval;
01279 };
01280
01281 bool requiresDataPerm(const wbpPerm &P, char lflag=3) const;
01282
01283 void markUnequalZero(const T m=1, const T eps=0) {
01284     size_t i=0, n=numel(); T z(0);
01285     if (eps==z)
01286         { for (; i<n; ++i) data[i] = (data[i]!=T(0) ? m:z); }
01287     else { for (; i<n; ++i) data[i] = (ABS(data[i])>eps ? m:z); }
01288 };
01289
01290 void toMatrixRef(
01291     wbararray &A,
01292     const UVEC &I,
01293     int pos,
01294     wbpPerm &P
01295 ) const;
01296
01297 bool toMatrixRef(const char *F, int L,
01298     wbararray &A,
01299     const UVEC &I,
01300     int pos,
01301     char &tflag
01302 ) const;
01303
01304 void toMatrixRef(
01305     wbararray &A, unsigned K, const wbpPerm &P0
01306 ) const;

```

```

01307
01308 void toMatrixRefM(
01309     wbararray &A, unsigned K, const wbperm &P0, unsigned R,
01310     wbperm pfin=wbperm()
01311 ) const;
01312
01313 void toMatrixRef(
01314     wbararray &A, unsigned i, int pos, wbperm &P
01315 ) const { return toMatrixRef( A, UVEC(1,&i), pos, P); };
01316
01317 void toMatrixRef(
01318     wbararray &A, const char* sidx, int pos, wbperm &P
01319 ) const { return toMatrixRef( A, Str2Idx(sidx,1), pos, P); };
01320
01321 wbararray& Reshape(const wbvector<size_t> &, bool lflag=0);
01322
01323 wbararray& Reshape(size_t s1, size_t s2, bool lflag=0) {
01324     WBINDEX S(2); S.data[0]=s1; S.data[1]=s2;
01325     return Reshape(S,lflag);
01326 };
01327 wbararray& Reshape(size_t s1, size_t s2, size_t s3, bool lflag=0) {
01328     WBINDEX S(3); S.data[0]=s1; S.data[1]=s2; S.data[2]=s3;
01329     return Reshape(S,lflag);
01330 };
01331
01332 wbararray<T>& Reshapel(const size_t *s, unsigned n) {
01333     if (!sameSize1(s,n)) wblog(FL,
01334         "ERR %s() size mismatch (%s <> %s)",FCT,
01335         SSTR_(this), SSTR(wbvector<size_t>(n,s)));
01336     SIZE.init(n,s); return *this;
01337 };
01338 wbararray& Reshapel(size_t s1, size_t s2) {
01339     size_t s[2]={s1,s2}; return Reshapel(s,2);
01340 };
01341 wbararray& Reshapel(size_t s1, size_t s2, size_t s3) {
01342     size_t s[3]={s1,s2,s3}; return Reshapel(s,3);
01343 };
01344
01345 void GroupIndizes(size_t K);
01346
01347 wbararray& groupIndizes_DREF(
01348     wbararray &A, unsigned k1, unsigned k2
01349 ) const {
01350     size_t i, s1=1, s2=1;
01351
01352     if (k1+k2!=SIZE.len) wblog(FL,
01353         "ERR invalid index group %d+%d=%d !?",k1,k2,SIZE.len);
01354
01355     k2=SIZE.len;
01356
01357     for (i=0; i<k1; ++i) { s1*=SIZE[i]; }
01358     for ( ; i<k2; ++i) { s2*=SIZE[i]; }
01359
01360     A.SIZE.init(2); A.SIZE[0]=s1; A.SIZE[1]=s2;
01361
01362     if (sptr!=A.sptr) {
01363         A.INIT2REF(data);
01364     }
01365
01366     return A;
01367 };
01368
01369 void GroupIndizes(unsigned k1, unsigned k2) {
01370     GroupIndizes_DREF(*this,k1,k2);
01371 };
01372
01373 void groupIndizes_P(
01374     const WBINDEX&, int, wbperm&,
01375     size_t* =NULL, size_t* =NULL
01376 ) const;
01377
01378 wbararray& transpose(const char *F, int L, wbararray &A) const;
01379 wbararray& Transpose(const char *F=0, int L=0) {
01380     wbararray X; transpose(F,L,X);
01381     return X.save2(*this);
01382 };
01383
01384 wbararray& flipLR(const char *F, int L, wbararray &A) const;
01385 wbararray& FlipLR(const char *F=0, int L=0);
01386
01387 wbararray& flipUD(const char *F, int L, wbararray &A) const;
01388 wbararray& FlipUD(const char *F=0, int L=0);
01389
01390 wbararray& MatPermute(const wbperm &P, char iflag=0);
01391
01392 wbararray& Permute(const char* s, char iflag=0, char rcpy=0);
01393 wbararray& Permute(const wbperm&, char iflag=0, char rcpy=0);

```

```

01394
01395 wbararray& permute(wbararray&, const char* s, char iflag=0) const;
01396 wbararray& permute(wbararray&, const wbpPerm&, char iflag=0) const;
01397
01398 wbararray& select0(
01399     const WBPPerm &, unsigned dim, wbararray&) const;
01400 wbararray& Select0(const UVEC &I, unsigned dim) {
01401     wbararray A(*this); A.select0(I, dim, *this);
01402     return *this;
01403 };
01404
01405 wbararray& select0(
01406     size_t i1, size_t i2, unsigned dim, wbararray &Q
01407 ) const {
01408     if (dim>=SIZE.len) wblog(FL,
01409         "ERR dim out of bounds (%d/%d)", dim+1, SIZE.len);
01410     if (i1>=SIZE[dim] || i2>=SIZE[dim]) wblog(FL,
01411         "ERR index out of bounds (%d,%d/%d)", i1, i2, SIZE[dim]);
01412
01413     if (i2<i1) {
01414         Q.init(); Q.SIZE=SIZE; Q.SIZE[dim]=0;
01415         return Q;
01416     }
01417
01418     wbindex I; I.Index(i1, i2);
01419     return select0(I, dim, Q);
01420 };
01421
01422 wbararray& selectSqueeze(
01423     wbararray &A, unsigned p,
01424     unsigned dim) const;
01425
01426 T min() const;
01427 T max() const;
01428
01429 T aMin(char zflag=0, size_t *k=NULL) const;
01430
01431 void info(const char* = "") const;
01432 void info(const char *F, int L,
01433     const char* = "", unsigned k=-1, unsigned nlt=0, unsigned nlb=0) const;
01434
01435 void print(
01436     const char *F, int L,
01437     const char *istr="", const char *fmt=""
01438 ) const;
01439
01440 void print(const char *istr="", const char *fmt="") const {
01441     print(0, 0, istr, fmt);
01442 };
01443
01444 int printdata(const char *istr="ans", const char *fmt="") const;
01445 void print_ref(const char *bstr=" ***", const char *nl="\n") const;
01446
01447 wstring toStr() const;
01448
01449 void contractDiag(
01450     unsigned ic, const wbararray &B,
01451     T a, T b, wbararray &C, char iflag=0, char bflag='N'
01452 ) const;
01453
01454 template<class TB, class TC>
01455 wbararray<TC>& contractMat(
01456     const char *F, int L, unsigned i1,
01457     const wbararray<TB>&M, unsigned i2,
01458     wbararray<TC>&) const;
01459
01460 template<class TB, class TC>
01461 wbararray<TC>& contractMat(
01462     const char *F, int L, unsigned i1,
01463     const wbararray<TB>&M, wbararray<TC> &C,
01464 ) const { return contractMat(F, L, i1, M, 1, C); };
01465
01466 wbararray& ContractMat(const char *F, int L,
01467     unsigned i1, const wbararray &M, unsigned i2=1
01468 ){ wbararray A;
01469     return contractMat(F, L, i1, M, i2, A).save2(*this);
01470 };
01471
01472 template<class TB, class TC>
01473 wbararray<TC>& contract(const char *F, int L,
01474     const ctrIdx&, const wbararray<TB> &B,
01475     const ctrIdx&, wbararray<TC> &C,
01476     const wbpPerm &pfinal=wbpPerm(),
01477     T afac=1, TC cfac=1
01478 ) const;
01479
01480 template<class TB, class TC>

```

```

01481     wbararray<TC>& contract(
01482         const char *F, int L, const iTags &ta,
01483         const wbararray<TB> &B, const iTags &tb, wbararray<TC> &C, iTags &tc,
01484         const wbperm &pfinal=wbperm(),
01485         T afac=1, TC cfac=1
01486     ) const;
01487
01488     template<class TB>
01489     wbararray<T>& Contract(
01490         const wbvector<unsigned>&, const wbararray<TB>&,
01491         const wbvector<unsigned>&,
01492         const wbperm &pfinal=wbperm()
01493     );
01494
01495     template<class TB>
01496     wbararray<T>& Contract(
01497         char*, const wbararray<TB>&, char*,
01498         const wbperm &pfinal=wbperm()
01499     );
01500
01501     template<class TB, class TC>
01502     wbararray<TC>& contract(const char *F, const int L,
01503         const char*, const wbararray<TB>&, const char*, wbararray<TC>&,
01504         const wbperm &pfinal=wbperm()
01505     ) const;
01506
01507     template<class TB, class TC>
01508     wbararray<TC>& contract(
01509         char *i1, const wbararray<TB> &B, char *i2, wbararray<TC> &Q,
01510         const wbperm &pfinal=wbperm()
01511     ) const { return contract(FL,i1,B,i2,Q,pfinal); };
01512
01513     template<class TB, class TC>
01514     wbararray<TC>& contract(
01515         const wbvector<unsigned> &i1, const wbararray<TB> &B0,
01516         const wbvector<unsigned> &i2,
01517         wbararray<TC> &Q0, const wbperm &P
01518     ) const { return contract(FL,i1,B0,i2,Q0,P); };
01519
01520     template<class TB, class TC>
01521     wbararray<TC>& contract(const char *F, int L,
01522         unsigned i1, const wbararray<TB>&B, unsigned i2,
01523         wbararray<TC>&C, const wbperm &pfinal=wbperm()
01524     ) const {
01525
01526         if (!i1 || !i2) wblog(F_L,
01527             "ERR %s() expecting 1-based indices (%d,%d)",FCT,i1,i2);
01528         --i1; --i2;
01529         wbvector<unsigned> I1(1,&i1), I2(1,&i2);
01530
01531         return contract(I1,B,I2,C, pfinal);
01532     };
01533
01534     template<class TB, class TC>
01535     wbararray<TC>& contract(
01536         unsigned i1, const wbararray<TB>&B, unsigned i2,
01537         wbararray<TC>&C, const wbperm &pfinal=wbperm()
01538     ) const { return contract(FL,i1,B,i2,C,pfinal); }
01539
01540     template<class TB, class TC>
01541     wbararray<TC>& comm(
01542         const wbararray<TB>&, wbararray<TC>&, char aflag='N', char bflag='N'
01543     ) const;
01544
01545     template<class TB, class TC>
01546     wbararray<TC>& acomm(
01547         const wbararray<TB>&, wbararray<TC>&, char aflag='N', char bflag='N'
01548     ) const;
01549
01550     wbararray operator+(const wbararray &B) const {
01551         wbararray C;
01552         return plus(B,C);
01553     };
01554
01555     wbararray operator-(const wbararray &B) const {
01556         wbararray C;
01557         return plus(B,C,-1);
01558     };
01559
01560     wbararray operator*(const wbararray &B) const {
01561         wbararray C;
01562         wbvector<unsigned> ica(1), icb(1); ica[0]=1; icb[0]=0;
01563         return contract(FL,ica,B,icb,C);
01564     };
01565
01566     T trace() const;
01567     wbararray& trace(unsigned i1, unsigned i2, wbararray &C) const;

```

```

01568     wbararray& trace(const ctrIdx &I1, const ctrIdx &I2, wbararray<T> &C) const;
01569
01570     mxArray* toMx() const { return toMx_Struct(); }
01571     mxArray* toMx_Struct() const;
01572     mxArray* toMx_base() const;
01573
01574     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01575     void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const;
01576
01577     void put(const char *vname="ans", const char* ws="base") const {
01578         mxArray *a=toMx();
01579         mxPutAndDestroy(FL,a,vname,ws);
01580     };
01581
01582     void put(const char *F, int L,
01583             const char *vname="ans", const char* ws="base"
01584     ) const {
01585         mxArray *a=toMx();
01586         wblog(F,L,"I/O putting array '%s' to %s.",vname,ws);
01587         mxPutAndDestroy(F,L,a,vname,ws);
01588     };
01589
01590     void getReal(wbararray<double> &R) const;
01591     void getImag(wbararray<double> &I) const;
01592     void Conj();
01593
01594     void set(const wbararray<double> &R, const wbararray<double> &I);
01595
01596     const wbararray& opA(char tflag, wbararray& aux) {
01597         if (!isMatrix()) wblog(FL,
01598             "ERR cannot use opA() with rank-%d object.", SIZE.len);
01599
01600         if (tflag=='N') { return *this; } else
01601         if (tflag=='T' || tflag=='C') {
01602             permute(aux, wbpem("2 1")); if (tflag=='C') aux.Conj();
01603             return aux;
01604         }
01605         wblog(FL, "ERR invalid flag %c<%d>", tflag, tflag);
01606         return *this;
01607     };
01608
01609     bool isSym_aux(
01610         const char *file, int line, const char *fct,
01611         const wbararray &B, double eps, double* xref,
01612         const char symflag='s',
01613         const char lflag=1
01614     ) const;
01615
01616     wbvector<size_t> SIZE;
01617
01618     Wb::MTYPE mtype;
01619
01620     Wb::sptr<T> *sptr;
01621
01622     T *data;
01623
01624 protected:
01625
01626     bool isDiag_aux(const T eps, const char* task) const;
01627     bool isDiag_aux(
01628         double *eps,
01629         const char* task) const;
01630
01631     size_t serial_index(const widx_t *I, size_t n) const;
01632
01633 private:
01634
01635     void print_rec (UVEC&, size_t, const char*, const char*) const;
01636
01637     void make2D() {
01638         if (SIZE.len<2) {
01639             SIZE.Resize(2); if (SIZE[0]) { SIZE[1]=1; }
01640         }
01641         else if (SIZE.len>2) wblog(FL,
01642             "ERR make2D() - invalid usage (got rank-%d)",SIZE.len);
01643     };
01644
01645     void MemErrMsg(const char* file, int line, size_t s) {
01646         wblog(file,line, "ERR Out of memory? (%dx%d)", s, sizeof(T));
01647     };
01648
01649     int check_consistency(const char *F=0, int L=0) const {
01650         int e=0;
01651
01652         if (sptr) {
01653             if (!data || sptr->data!=data ) { e|=1; }
01654             if (sptr->check_consistency(0)) { e|=2; }

```

```

01655         if (!SIZE.len || SIZE.min()==0) { e|=4; }
01656
01657         if (e && F) wblog(F,L,
01658             "ERR Wb::sptr = %s\n%p / %p (r=%ld, e=%d)",
01659             STR_(sptr), sptr, data, SIZE.len, e
01660         );
01661
01662         if (mtype!=sptr->mtype) {
01663             if (mtype || sptr->mtype!=Wb::MEM_REF) wblog(F,L,
01664                 "WRN Wb::sptr mtype = %s / %s !?",
01665                 Wb::MTYPE_STR[mtype], Wb::MTYPE_STR[sptr->mtype]);
01666         }
01667     }
01668     else {
01669         if (SIZE.len && SIZE.min()) { e|=8; }
01670         if (data) { e|=16; }
01671
01672         if (e && F) wblog(F,L,
01673             "ERR Wb::sptr inconsistency: %p / %p", sptr, data
01674         );
01675     }
01676     return e;
01677 };
01678
01679 void DELETE_DATA(char mflag=0) {
01680
01681     if (sptr) { check_consistency(FL);
01682         if (sptr->rm_dref(mflag)) {
01683             WB_DELETE_1(sptr);
01684         }
01685         else { sptr=NULL; }
01686         data=NULL;
01687
01688         mtype=Wb::MEM_DEF;
01689     }
01690     else if (data) wblog(FL, "ERR %s() %p / %p !?", FCT, data, sptr);
01691 };
01692
01693 wbarray& NEW_DATA(size_t n, const T* d0=NULL, char ref=0, char init=1) {
01694     if (!n) {
01695         DELETE_DATA(); return *this;
01696     }
01697
01698     if (!ref) {
01699         if (!sptr) WB_NEW_1(sptr);
01700
01701         if (mtype && WBLOG_MMAX) { wblog(FL,
01702             " | %s ... %p -> %s", Wb::MTYPE_STR[mtype],
01703             this, Wb::size2Str(n*sizeof(T)).data);
01704         }
01705
01706         if (d0 || init)
01707             { data=sptr->malloc_data(n,d0,mtype); }
01708         else { data=sptr->malloc_base(n, 0,mtype); }
01709     }
01710     else {
01711         RENEW_SPTR();
01712         data=(sptr->init2ref(n,d0));
01713     }
01714     return *this;
01715 };
01716
01717 void RENEW_SPTR() {
01718     if (sptr) { DELETE_DATA(); }
01719     WB_NEW_1(sptr);
01720 };
01721
01722 wbarray& INIT2REF(const T *d0, const wvector<size_t> *S=NULL) {
01723     if (S) { SIZE=(*S); }
01724     size_t n=SIZE.prod(0);
01725
01726     if (d0) { if (!sptr || !isRef() || d0!=data) {
01727         RENEW_SPTR();
01728         data=(sptr->init2ref(n,d0));
01729     }}
01730     else if (n) wblog(FL,
01731         "ERR data/size inconsistency: %p / %p => %s", data, d0, SSTR_(this));
01732
01733     return *this;
01734 };
01735
01736 wbarray& NEW(const wvector<size_t> &S, const T* d0=NULL) {
01737     SIZE=S;
01738     NEW_DATA(SIZE.prod(0), d0);
01739     return *this;
01740 };
01741

```



```

01742     wbararray& NEW(const wbvector<size_t> &S, size_t n, const T* d0=NULL) {
01743         SIZE.init(n,S); NEW_DATA(SIZE.prod(0),d0);
01744         return *this;
01745     };
01746
01747     T& dummy() const {
01748         static T d; memset(&d,0,sizeof(T));
01749         return d;
01750     };
01751
01752 };
01753
01754 template<class T>
01755 wbstring Wb::sizeStrM(
01756     unsigned r, const T* sd,
01757     unsigned m, const T* sm, const char *sep, const char *sepM
01758 ) {
01759     unsigned i=0, j, l=0, n=0, ndims=r+m;
01760     char *s; size_t x;
01761     wbstring sout;
01762
01763     if (r>1 || m>1)
01764         { n+=(ndims-(r && m ? 2 : 1))*(sep ? strlen(sep) : 1); }
01765     if (m) { n+=(sepM ? strlen(sepM) : 1); }
01766
01767     for (; i<ndims; ++i) {
01768         x=(i<r ? sd[i] : sm[i-r]); if (x<0) { x=-x; ++n; }
01769         for (j=0; j<64; ++j) { if (!(x>>1)) break; }
01770         n+=(1+ceil(3*(double(i)/10)));
01771     }; if (n<8) n=8;
01772
01773     sout.init(n); s=sout.data;
01774
01775     for (i=0; i<r && l<n; ++i) {
01776         l+=snprintf(s+l,n-l,"%s%d",i? (sep? sep:" "):"", long(sd[i]));
01777     }
01778     for (i=0; i<m && l<n; ++i) {
01779         l+=snprintf(s+l,n-l,"%s%d", i? (sep? sep:" "): (sepM? sepM:"|"),
01780             long(sm[i]));
01781     }
01782
01783     if (l>=n) wblog(FL,
01784         "WRN %s() string out of bounds (%d/%d)\n'%s'",FCT,l,n,s);
01785     return sout;
01786 };
01787
01788 class tensor_ref_ {
01789 public:
01790     tensor_ref_() : Ar(NULL), Az(NULL) {};
01791     tensor_ref_(const tensor_ref_ &b) : Ar(b.Ar), Az(b.Az) {};
01792
01793     tensor_ref_& init(const tensor_ref_ &b) {
01794         Ar=b.Ar; Az=b.Az;
01795         return *this;
01796     };
01797     tensor_ref_& operator=(const tensor_ref_ &b) { return init(b); };
01798
01799     bool operator==(const wbararray<double> *A) const {
01800         return (Ar && Ar==A); };
01801     bool operator==(const wbararray<wbcomplex> *A) const {
01802         return (Az && Az==A); };
01803
01804     tensor_ref_& init(const wbararray<double> &A) { Ar=&A; Az=NULL;
01805         return *this; };
01806     tensor_ref_& init(const wbararray<wbcomplex> &A) { Az=&A; Ar=NULL;
01807         return *this; };
01808
01809     explicit operator bool() const { return (Ar || Az); };
01810     bool operator! () const { return (!Ar && !Az); };
01811
01812     int check(const char *F=NULL, int L=0) const;
01813     int rank(unsigned r) const;
01814
01815     bool sameSize(const wbvector<size_t> &S) const;
01816
01817     double xflops() const { return (Az ? 2 : 1); };
01818
01819     wbstring sizeStr() const;
01820     const wbvector<size_t>* getSIZE() const;
01821
01822     template<class TC>
01823     void contract(
01824         const char *F, int L, const ctrIdx &ia,
01825         const tensor_ref_ &b, const ctrIdx &ib, wbararray<TC> &C) const;
01826
01827     const wbararray<double> *Ar;

```

```

01829     const wbararray<wbcomplex> *Az;
01830
01831     protected:
01832     private:
01833 };
01834
01835 class tensorRef_ {
01836 public:
01837     tensorRef_ () : id(0), conj(0) {};
01838
01839     tensorRef_ (const tensorRef_ &B) : id(B.id), conj(B.conj), R(B.R) {
01840         it=B.it; S=B.S; ID=B.ID; };
01841
01842     template<class T>
01843     tensorRef_(const wbararray<T> &A, const iTags *t_=NULL) {
01844         init(A,t_); }
01845
01846     tensorRef_& operator=(const tensorRef_ &B) { return init(B); };
01847
01848     tensorRef_& init(const tensorRef_ &B) {
01849         id=B.id; conj=B.conj; R=B.R; it=B.it; S=B.S; ID=B.ID;
01850         return *this;
01851     };
01852
01853     tensorRef_& save2(tensorRef_ &B) {
01854         B.id=id; B.conj=conj; B.R=R;
01855         S.save2(B.S); it.save2(B.it); ID.save2(B.ID);
01856         return B;
01857     };
01858
01859     template<class T>
01860     tensorRef_& init(const wbararray<T> &A, const iTags *t_=NULL, char c=0) {
01861         R.init(A);
01862         S.init2ref(A.SIZE); ID.init(); id=0; conj=c;
01863
01864         if (!t_) { it.init(); }
01865         else {
01866             if (t_>len!=S.len) { unsigned r=t_>len;
01867                 if (r>S.len || r<2 || (r==2 && A.numOM(r)!=1)) { wblog(FL,
01868                     "ERR %s() unexpected rank r=%d/%d (%s; %s)",
01869                     FCT, r,S.len, SSTR(A), STR_(t_));
01870                 }
01871             }
01872             it.init(*t_);
01873         }
01874         return *this;
01875     };
01876
01877     size_t numel() const { return S.prod(0); };
01878
01879     char check(const char* F=NULL, int L=0) const;
01880     char Check(const char* F, int L, unsigned &id_);
01881
01882     int UpdateItagsCtr(const char *F, int L,
01883         const ctrIdx &ia, tensorRef_ &b,
01884         const ctrIdx &ib, const char *tag
01885     ) {
01886         int q=it.UpdateItagsCtr(F,L,ia,b.it,ib,tag);
01887
01888         conj=ia.conj;
01889         b.conj=ib.conj;
01890
01891         return q;
01892     };
01893
01894     template<class T>
01895     int contract(const char *F, int L,
01896         const tensorRef_ &b, tensorRef_ &c, double *flops,
01897         wbararray<T> *C) const;
01898
01899     int contract(const char *F, int L,
01900         const tensorRef_ &b, tensorRef_ &c, double &flops
01901     ) const { return contract(F,L,b,c,&flops, (wbararray<double>*)NULL); }
01902
01903     template<class T>
01904     int contract(const char *F, int L,
01905         const tensorRef_ &b, tensorRef_ &c, wbararray<T> &C
01906     ) const { return contract(F,L,b,c,NULL, &C); }
01907
01908     wbsttring toStr() const {
01909         wbsttring s(ID.len ? 128 : 256);
01910         unsigned l, n=s.len;
01911
01912         l=snprintf(s.data,s.len,"%-16s", SSTR(S)); if (l<n) {
01913             l+=snprintf(s.data+l,n-l,ID.len ? "%-24s ":"%s",STR(it)); }
01914         for (unsigned i=0; i<ID.len && l<n; ++i) {
01915             l+=snprintf(s.data+l,n-l,"%s%d.%d",i?" ":"",ID[i]>8,ID[i]&255); }

```

```

01916
01917         if (l>=n) wblog(FL, "ERR %s() "
01918             "string out of bounds (%d/%d)%N%N%s%N%N", FCT, l, n, s.data);
01919         return s;
01920     };
01921
01922     const tensorRef_& print(const char* istr=NULL, char vflag=1) const {
01923         char q=(vflag ? check() : 0);
01924         PRINTF("%s %-50s%s\n", istr? istr: " ", STR_(this),
01925             q ? (q&1 ? "empty data" : "length mismatch") : "");
01926         return *this;
01927     };
01928
01929     unsigned id;
01930
01931     char conj;
01932
01933     tensor_ref_ R;
01934
01935     iTags it;
01936     wbvector<size_t> S;
01937
01938     wbvector<unsigned> ID;
01939
01940     protected:
01941     private:
01942 };
01943
01944 class tensorRefs : public wbvector<tensorRef_> {
01945     public:
01946         tensorRefs(unsigned l=0) : wbvector<tensorRef_>(l) {};
01947
01948         char check(const char *F, int L) const {
01949             char q=0;
01950             for (unsigned i=0; i<len; ++i) { q|=data[i].check(F,L); }
01951             return q;
01952         };
01953
01954         char Check(const char *F, int L) {
01955             char q=check(F,L);
01956             if (!q) {
01957                 unsigned i=0, id=0;
01958                 for ( ; i<len; ++i) { if (id<data[i].id) { id=data[i].id; } }
01959                 for (i=0; i<len; ++i) { data[i].Check(F_L,id); }
01960             }
01961             return q;
01962         };
01963
01964         void print() { print(NULL,0); }
01965         void print(const char *F, int L) const;
01966
01967         double getOptimalCtrOrder(
01968             const char *F, int L, wbperm &P,
01969             wbperm *pfin=NULL, char useP=0, char vflag=0);
01970
01971     protected:
01972     private:
01973 };
01974
01975 template <class T>
01976 class wbarrRef {
01977     public:
01978
01979         wbarrRef() : dc(0.), fac(1.), tflag(0), D(NULL) {};
01980
01981         wbarrRef(const wbarrRef<T> &r) {
01982             memcpy(this, &r, sizeof(r));
01983         };
01984
01985         wbarrRef<T>& operator=(const wbarrRef<T> &r) {
01986             memcpy(this, &r, sizeof(r));
01987             return *this;
01988         };
01989
01990         void print(const char *vname="ans") {
01991             printf("\nwbararray %s\n", vname);
01992             printf("... wbararray : 0x%lX (%s%s)\n", D,
01993                 D && D->SIZE.len==1 ? "vec of length ":"", D? SSTR_(D): "");
01994             printf("... dc : %g\n", dc);
01995             printf("... fac : %g\n", fac);
01996             printf("... tflag : %d\n", tflag);
01997         };
01998
01999         double dc;
02000         double fac;
02001         bool tflag;
02002

```

```

02003     wbararray<T> *D;
02004
02005     protected:
02006     private:
02007 };
02008
02009     namespace NR {
02010
02011     template<class T> inline
02012     T pythag(T a, T b) {
02013
02014         if (a<0) a=-a;
02015         if (b<0) b=-b;
02016
02017         if (a>b)
02018             { T x=b/a; return a*SQRT(T(1)+x*x); } else if (b==0) return 0;
02019         else { T x=a/b; return b*SQRT(T(1)+x*x); }
02020     };
02021
02022     template<class T> inline
02023     T sign(const T &a, const T &b) {
02024         return (b>0 ? (a>0 ? a : -a) : (a>0 ? -a : a));
02025     };
02026
02027 };
02028
02029     template<class T>
02030     wbararray<T> TestContract(
02031         const wbararray<T> &A, C_UVEC i1,
02032         const wbararray<T> &B, C_UVEC i2, const wbperm &pfinal=wbperm());
02033
02034     template<class T>
02035     void cell2mat(
02036         const wbMatrix< wbarRef<T> > &C,
02037         wbararray<T> &M, WBINDEX &D1, WBINDEX &D2
02038     );
02039
02040     template<class T> inline
02041     size_t wbararray<T>::serial_index(const widxt *I, size_t len) const {
02042
02043         size_t i,k, r=SIZE.len, l=r-1, nz=r-len;
02044
02045         if (len==0) return -1;
02046         if (!r || len>r || (len && data==NULL)) wblog(FL,
02047             "ERR %s[%s] having %s array",FCT,
02048             STR(WBINDEX(len,I)+1), SSTR_(this));
02049
02050         #ifdef CHECK_ELEMENT_RANGE
02051         for (i=0; i<len; i++) if (I[i]>=SIZE[i+nz]) {
02052             wblog(FL,"ERR %s[%s] index out of bounds (%s)",FCT,
02053                 STR(WBINDEX(len,I)+1), SSTR_(this));
02054         }
02055         #endif
02056
02057         for (k=I[l-nz], i=l-1; i<r; i--) {
02058             k = k*SIZE[i] + ((i>=nz) ? I[i-nz] : 0);
02059         }
02060
02061         #ifdef CHECK_ELEMENT_RANGE
02062         if (k>=numel()) wblog(FL,
02063             "ERR %s() index out of bounds (%d; %s)",FCT,k,SSTR_(this));
02064         #endif
02065
02066         return k;
02067     };
02068
02069     template<class T>
02070     mxArray* wbararray<T>::toMx_Struct() const {
02071
02072         size_t i,dim1,dim2,r=SIZE.len, s=numel();
02073         WBINDEX I(r);
02074         T A;
02075
02076         if (r==2) { dim1=SIZE[0]; dim2=SIZE[1]; } else
02077         if (r==1) { dim1=SIZE[0]; dim2=(dim1 ? 1 : 0); } else
02078         if (r==0) { dim1=dim2=0; }
02079         else wblog(FL,"ERR %s() not implemented yet for rank>2 (%d)",SIZE.len);
02080
02081         mxArray *S=A.mxCreateStruct(dim1,dim2);
02082
02083         for (i=0; i<s; ++i) data[i].add2MxStruct(S,i);
02084
02085         return S;
02086     };
02087
02088     template<class T>
02089     mxArray* wbararray<T>::mxCreateStruct(unsigned m, unsigned n) const {

```

```

02090     return mxCreateCellMatrix(m,n);
02091 }
02092
02093 template<class T>
02094 void wbararray<T>::add2MxStruct(mxArray *S, unsigned i, char tflag) const {
02095
02096     if (tflag) { size_t s=0;
02097         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s) wblog(FL,
02098             "ERR %s() must follow mxCreateStruct()\n%lx, %d/%d",FCT,S,i+1,s);
02099     }
02100
02101     mxSetCell(S,i,toMx());
02102 };
02103
02104 template <class T> inline
02105 mxArray* wbararray<T>::toMx_base() const {
02106
02107     mxArray *a=NULL;
02108
02109     if (sptr && sptr->mtype) { check_consistency(FL);
02110         if (sptr->mtype==Wb::MEX_RETURN) {
02111             size_t l=P2X.BUF.size();
02112             a=P2X.Return(0,0,data);
02113
02114             if (a) {
02115                 size_t s1=numel(), s2=mxGetNumberOfElements(a);
02116                 if (s1==s2) {
02117                     if (WBLOG_MMEX) wblog(FL,
02118                         "2Mx %s (PSX @ %d)",STR_(sptr),l);
02119                     mxSetDimensions(a,SIZE.data,SIZE.len);
02120                 }
02121                 else wblog(FL,
02122                     "TST %s() numel=%ld/%ld %p/%p (PSX @ %ld)",
02123                     FCT,s1,s2,mxGetData(a),data,l
02124                 );
02125             }
02126             else {
02127                 wblog(FL,"WRN sptr=%s not / no longer in P2X @ %ld",STR_(sptr),l);
02128             }
02129         }
02130         else {
02131             if (sptr->mtype!=Wb::MEM_REF)
02132                 wblog(FL,"TST %s() got %s",FCT,Wb::MTYPE_STR[sptr->mtype]);
02133         }
02134     }
02135
02136     if (!a) { a=cpyRange2Mx(data,SIZE); }
02137
02138     return a;
02139 };
02140
02141 template <> mxArray* wbararray<unsigned>::toMx() const { return toMx_base(); }
02142 template <> mxArray* wbararray<int>::toMx() const { return toMx_base(); }
02143 template <> mxArray* wbararray<char>::toMx() const { return toMx_base(); }
02144 template <> mxArray* wbararray<double>::toMx() const { return toMx_base(); }
02145 template <> mxArray* wbararray<wbcomplex>::toMx() const { return toMx_base(); }
02146
02147 template<class T>
02148 wbararray<T>& wbararray<T>::resize(
02149     const wvector<size_t> &S, wbararray<T> &B) const {
02150
02151     if (SIZE.len!=S.len) wblog(FL,
02152         "ERR length mismatch (%d/%d)",SIZE.len,S.len);
02153     if (this==&B) wblog(FL,"WRN %s() got call onto self",FCT);
02154
02155     if (S==SIZE) { B=(*this); return B; }
02156     else if (!S) { B.init(); return B; }
02157     else { B.NEW(S); }
02158
02159     size_t i,j,k, r=S.len, l=r-1, *s1=SIZE.data, *s2=B.SIZE.data;
02160     WBINDEX I(r); T *b=B.data;
02161
02162     size_t s[r];
02163     for (i=0; i<r; ++i) { s[i] = MIN( S[i], SIZE[i] ); }
02164
02165     while (I[l]<s[l]) {
02166         for (i=j=I[l], k=l-1; k<r; --k) {
02167             i = i*s1[k] + I[k];
02168             j = j*s2[k] + I[k];
02169         }
02170
02171         b[j]=data[i];
02172
02173         k=0; I[0]++;
02174         while (I[k]>=s[k] && k<l) { I[k]=0; ++I[++k]; }
02175     }
02176

```

```

02177     return B;
02178 };
02179
02180 template<class T>
02181 wbararray<T>& wbararray<T>::ResizeM(unsigned r, const wvector<size_t> &Sm) {
02182
02183     unsigned i=0, m=Sm.len;
02184     wvector<size_t> S(SIZE);
02185     wbararray<T> X;
02186
02187     if (r+m!=S.len) wblog(FL,
02188         "ERR %s() size mismatch %d+%d / %d",FCT,r,m,S.len);
02189     if (int(r)<=2 && Sm.anyUnequal(1)) wblog(FL,
02190         "ERR %s() unexpected OM %s",FCT,SSTRM_(r,S.data,m,Sm.data));
02191
02192     for (; i<m; ++i) {
02193         if (S[r+i]>Sm[i]) wblog(FL,"WRN %s() "
02194             "reducing OM (%d/%d+%d): %d->%d",FCT,i+1,r,m,S[r+i],Sm[i]);
02195         S[r+i]=Sm[i];
02196     }
02197     return resize(S,X).save2(*this);
02198 };
02199
02200 template<class T>
02201 wbararray<T>& wbararray<T>::Enlarge(
02202     size_t dim,
02203     size_t D,
02204     size_t istart
02205 ){
02206     WBINDEX S(SIZE);
02207     wbararray<T> X;
02208     wIndex I(S);
02209
02210     if (!dim || dim>S.len) wblog(FL,
02211         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim,S.len);
02212     if (D<S[--dim]) wblog(FL,
02213         "ERR %s() envalid new size (%d: %d/%d)",FCT,dim+1,S[dim],D);
02214     if (istart+S[dim]>=D) wblog(FL,
02215         "ERR %s() index out of bounds (%d+%d/%d)",FCT,istart+1,S[dim],D);
02216
02217     S[dim]=D; X.init(S);
02218     if (istart) {
02219         for (size_t i=0; ++I; ++i) {
02220             I.data[dim]+=istart; X(I)=data[i];
02221             I.data[dim]-=istart;
02222         }
02223     } else { for (size_t i=0; ++I; ++i) X(I)=data[i]; }
02224
02225     X.save2(*this);
02226     return *this;
02227 };
02228
02229 template<class T>
02230 wbararray<T>& wbararray<T>::Append2(
02231     const char *F, int L,
02232     wbararray<T> &B,
02233     unsigned dim
02234 ){
02235     size_t i=0, D=0, ib=0;
02236
02237     if (B.isEmpty()) { save2(B); return B; }
02238
02239     if (SIZE.len!=B.SIZE.len) wblog(F_L,
02240         "ERR %s() incompatible objects (%s <> %s)",FCT,SSTR_(this),SSTR(B));
02241     if (dim>=SIZE.len) wblog(F_L,
02242         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02243
02244     for (i=0; i<SIZE.len; ++i) {
02245         if (i!=dim) { if (SIZE[i]!=B.SIZE[i]) wblog(F_L,
02246             "ERR %s() size mismatch (%s <> %s @ %d/%d: %d)",
02247             FCT,SSTR_(this),SSTR(B),i+1,SIZE.len,dim+1);
02248         }
02249         else { D=SIZE[i]+B.SIZE[i]; ib=B.SIZE[i]; }
02250     }
02251
02252     WBINDEX S(SIZE); S[dim]=D;
02253     wIndex Ia(SIZE), Ib(B.SIZE);
02254     size_t &ia=Ia.data[dim];
02255     wbararray<T> X(S);
02256
02257     for (i=0; ++Ib; ++i) X(Ib)=B.data[i];
02258     for (i=0; ++Ia; ++i) {
02259         ia+=ib; X(Ia)=data[i];
02260         ia-=ib;
02261     }
02262
02263     X.save2(B); init();

```

```

02264     return B;
02265 };
02266
02267 template<class T>
02268 void wbararray<T>::Append2(
02269     const char *F, int L,
02270     wbararray<T> &B, unsigned dim,
02271     wbararray<double> &W, double eps
02272 ){
02273     size_t i,n, D=0, ib=0;
02274     const double *w=W.data;
02275
02276     if (this->isEmpty()) return;
02277     if (dim>=SIZE.len) wblog(F_L,
02278         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02279
02280     if (B.isEmpty()) {
02281         WBINDEX S(SIZE); S[dim]=0;
02282         B.init(S);
02283     }
02284
02285     if (SIZE.len!=B.SIZE.len) wblog(F_L,
02286         "ERR %s() incompatible objects (%s <> %s)",
02287         FCT,SSTR_(this),SSTR(B));
02288     for (i=0; i<SIZE.len; ++i) { if (i!=dim) {
02289         if (SIZE[i]!=B.SIZE[i]) wblog(F_L,
02290             "ERR %s() size mismatch (%s <> %s; %d; %d/d)",
02291             FCT,SSTR_(this),SSTR(B),dim+1,i+1,SIZE.len);
02292     }}
02293
02294     wbindex J(SIZE[dim]);
02295     size_t m=0, *j=J.data; n=W.numel();
02296
02297     if (!W.isVector() || n!=SIZE[dim]) wblog(F_L,
02298         "ERR %s() invalid weights W (%s <> %s @ %d)",
02299         FCT, SSTR(W), SSTR_(this), dim+1);
02300     for (i=0; i<n; ++i) { if (w[i]<eps) { j[m++]=i; }}
02301
02302     if (!m) return;
02303     if (m==n) { Append2(F_L,B,dim); return; }
02304
02305     J.len=m;
02306     D=m+B.SIZE[dim]; ib=B.SIZE[dim];
02307
02308     WBINDEX S(SIZE); S[dim]=m;
02309     wbindex Ix(S), Ib(B.SIZE); S[dim]=D;
02310     wbindex Ia(S.len),J2;
02311     wbararray<T> X(S);
02312
02313     size_t &ia=Ia.data[dim], &ix=Ix.data[dim];
02314
02315     for (i=0; ++Ib; ++i) { X(Ib)=B.data[i]; }
02316     for (i=0; ++Ix; ++i) { Ia.setp(Ix.data); ia=j[ia];
02317         ix+=ib; X(Ix)=(*this)(Ia);
02318         ix-=ib;
02319     }
02320
02321     X.save2(B);
02322     J.invert(SIZE[dim],J2); Select0(J2,dim);
02323 };
02324
02325 template<class T>
02326 wbararray<T>& wbararray<T>::init(
02327     const char *F, int L, const mxArray *a, char ref, char vec) {
02328
02329     if (!a) { init(); return *this; }
02330
02331     unsigned r=mxGetNumberOfDimensions(a);
02332     const size_t *sz=mxGetDimensions(a);
02333
02334     if (!mxIsNumChar(a)) wblog(F,L,
02335         "ERR got rank-%d array %s",r,mxGetClassName(a));
02336
02337     wvector<size_t> S(r,sz); init();
02338
02339     if (vec && r==2) {
02340         if (S[1]==1) { S.len=1; } else
02341         if (S[0]==1) { S[0]=S[1]; S.len=1; }
02342     }
02343
02344     Mx::Array<T> A(a);
02345
02346     if (ref && A.data) {
02347         INIT2REF((T*)A.data,&S);
02348     }
02349     else {
02350         init(S); A.copy_to(data);

```

```

02351         if (ref && WBLOG_TCAST) wblog(FL,
02352             "WRN ignoring ref (%s)",A.toStrT().data);
02353     }
02354
02355     if (data && !Wb::is_finite(data,1)) wblog(FL,
02356         "WRN %s() encountered nan or inf in mex input data",FCT);
02357
02358     return *this;
02359 };
02360
02361 template<class T>
02362 wbararray<T>& wbararray<T>::init(
02363     const char *F, int L, const mxArray *a, wbpem &P
02364 ){
02365     if (!a || mxIsEmpty(a)) { init(); return *this; }
02366     if (P.isIdentityPerm()) { return init(F,L,a); }
02367
02368     unsigned i, r=mxGetNumberOfDimensions(a);
02369     const size_t *sz=mxGetDimensions(a);
02370     size_t len=1;
02371
02372     if (!mxIsNumeric(a)) wblog(F,L,
02373         "ERR got rank-%d array %s",r,mxGetClassName(a));
02374     if (!r || !sz || P.len!=r) wblog(FL,
02375         "ERR %s() got %d/%d @ sz=%p !?",FCT,P.len,r,sz);
02376
02377     SIZE.init(r);
02378     for (i=0; i<r; ++i) { SIZE[i]=sz[P[i]]; len*=sz[i]; }
02379
02380     NEW_DATA(len,NULL,0,0);
02381
02382     Mx::Array<T>(a).copyTo(data,P);
02383
02384     return *this;
02385 };
02386
02387 template<class T> inline
02388 void wbararray<T>::initTst() {
02389
02390     size_t i,j,k, r=SIZE.len, l=r-1, s=numel(), x;
02391     WBINDEX I(r);
02392
02393     for (i=0; i<s; i++) {
02394         for (j=I[l], k=l-1; k<r; k--) j = j*SIZE[k] +I[k];
02395         for (x=I[0]+1, k=1; k<r; k++) x = x*10 + (I[k]+1);
02396
02397         data[j]=(T)x;
02398
02399         k=0; I[0]++;
02400         while(I[k]>=SIZE[k] && k<l) { I[k]=0; ++I[++k]; }
02401     }
02402 }
02403
02404 template<class T> inline
02405 wbararray<T>& wbararray<T>::initIdentity(size_t d, char dflag, T dval) {
02406
02407     if (dflag) { init(d).set(dval); }
02408     else {
02409         init(d,d);
02410         for (size_t i=0; i<d; ++i) {
02411             data[i+d*i]=dval;
02412         }
02413     }
02414     return *this;
02415 };
02416
02417 template<class T> inline
02418 wbararray<T>& wbararray<T>::initIdentity(
02419     const WBINDEX &S, char dflag
02420 ){
02421     size_t i,d,m=S.len/2;
02422     size_t const* const s=S.data;
02423
02424     if (S.len%2) wblog(FL,
02425         "ERR %s() requires even rank object (%s)",FCT,SSTR_(this));
02426     for (i=0; i<m; i++) if (s[i]!=s[i+m]) wblog(FL,
02427         "ERR %s() requires symmetric object (%s)",FCT,SSTR_(this));
02428     if (m>1 && dflag) wblog(FL,
02429         "ERR %s() rank-2 tensor required with dflag (%s)",FCT,SSTR_(this));
02430
02431     if (!m) { init(); return *this; }
02432     if (dflag) { init(s[0]).set(T(1)); return *this; }
02433     if (m==1 && s[0]==1) { init(S).set(T(1)); return *this; }
02434
02435     init(S); for (d=s[0], i=1; i<m; i++) d*=s[i];
02436     for (i=0; i<d; i++) data[i+d*i]=1;
02437

```



```

02438     return *this;
02439 };
02440
02441 template<class T> inline
02442 wbararray<T>& wbararray<T>::Reduce2Id() {
02443     if (SIZE.len==2) {
02444         size_t i, j, *s=SIZE.data; T *x=data;
02445
02446         for (j=0; j<s[1]; ++j) {
02447             for (i=0; i<s[0]; ++i, ++x) { *x = (i==j ? 1 : 0); }
02448         }
02449     } else if (SIZE.len) wblog(FL,
02450 "ERR %s() matrix expected (got %s)",FCT,SSTR_(this));
02451     return *this;
02452 };
02453 };
02454
02455 template<class T> inline
02456 wbararray<T>& wbararray<T>::initIdentityB(
02457     size_t d1, size_t d2, size_t k
02458 ){
02459     init(d1,d2); if (d1 && d2) {
02460         if (int(k)>=0) {
02461             if (k<d2) {
02462                 size_t j=k, dx=d1+1; T *x=data+j*d1;
02463                 for (; j<d2 && (j-k)<d1; ++j, x+=dx) { x[0]=1; }
02464             }
02465             else wblog(FL,
02466 "WRN %s() got all-zero block (%dx%d w/ k=%d)",FCT,d1,d2, k);
02467         }
02468         else { k=-k;
02469             if (k<d1) {
02470                 size_t i=k, dx=d1+1; T *x=data+i;
02471                 for (; i<d1 && (i-k)<d2; ++i, x+=dx) { x[0]=1; }
02472             }
02473             else wblog(FL,
02474 "WRN %s() got all-zero block (%dx%d w/ k=%d)",FCT,d1,d2,-k);
02475         }
02476     }
02477     return *this;
02478 };
02479
02480 template<class T> inline
02481 wbararray<T>& wbararray<T>::initIdentityB3(
02482     size_t d1, size_t d2, size_t D,
02483     size_t i0,
02484     T one
02485 ){
02486     size_t i=0, d12=d1*d2, d12_=d12+1;
02487
02488     if (i0+d12>D) wblog(FL,"ERR %s() "
02489 "index out of bounds (%ld*%ld) x %ld @ %ld",FCT,d1,d2,D,i0+1);
02490
02491     init(d1,d2,D);
02492     for (T *x = data + i0*d12; i<d12; ++i, x+=d12_) { (*x)=one; }
02493
02494     return *this;
02495 };
02496
02497 template<class T> inline
02498 void wbararray<T>::Diag2Vec() {
02499     size_t i,m;
02500     WBINDEX S(1);
02501
02502     if (!isSMatrix()) wblog(FL,
02503 "ERR Calling %s for rank-%d object (%s).",
02504     FCT, SIZE.len, SSTR_(this));
02505
02506     m=S[0]=SIZE[0];
02507     for (i=1; i<m; i++) data[i]=data[i+i*m];
02508
02509     Resize(S);
02510 };
02511
02512 template<class T> inline
02513 wbararray<T>& wbararray<T>::Reduce2Diag(T eps) {
02514
02515     if (!SIZE.len) { return *this; }
02516     else if (!isSMatrix()) wblog(FL,
02517 "ERR %s() symmetric matrix required\n"
02518 "(got rank-%d object; %s)",FCT,rank(),SSTR_(this)
02519 );
02520
02521     unsigned i, j=0, n=SIZE[0];
02522     wbararray<T> X(n,1); T *d0=data;
02523
02524     X.init(n,1);

```

```

02525     for (; j<n; ++j, d0+=n) {
02526         X.data[j]=d0[j];
02527         for (i=0; i<n; ++i) { if (i!=j) {
02528             if (ABS(d0[i])>eps) wblog(FL,
02529                 "ERR %s() got off-diagonal matrix element (%g/%g)",
02530                 FCT,double(d0[i]),double(eps)
02531             );
02532         }}
02533     }
02534
02535     return X.save2(*this);
02536 };
02537
02538 template<class T>
02539 void wbararray<T>::contractDiag(
02540     unsigned ic, const wbararray<T> &B,
02541     T a, T b, wbararray<T> &C, char Iflag, char bflag
02542 ) const {
02543
02544     if (B.SIZE.len!=2) wblog(FL,"ERR invalid rank-%d",SIZE.len);
02545     if (ic<1 || ic>SIZE.len) wblog(FL,
02546         "ERR invalid contraction index (ic=%d)",ic);
02547     if (!strchr("NTCc",bflag)) wblog(FL,
02548         "ERR invalid flag %c<%d>",bflag,bflag);
02549
02550     size_t i,k, s=numel(), r=SIZE.len, l=r-1;
02551     size_t n=B.SIZE.min(), m=B.SIZE[0];
02552     WBINDEX I_(r);
02553     wvector<T> h_(n);
02554     wbararray<T> X;
02555
02556     size_t *I=I_.data, *S=SIZE.data;
02557     T *h=h_.data;
02558
02559     ic--;
02560     if (n!=SIZE[ic]) wblog(FL,"ERR dimension mismatch (%d/%d)",n,SIZE[ic]);
02561     if (n==0) return;
02562
02563     for (i=0; i<n; i++) h[i]=B.data[i+i*m];
02564     if (bflag=='C' || bflag=='c') h_.Conj();
02565
02566     X=*this; if (a!=1) X*=a;
02567
02568     for (i=0; i<s; i++) {
02569         if (Iflag)
02570             X.data[i]/=(b-h[I[ic]]+1E-33);
02571         else X.data[i]*=(b-h[I[ic]]);
02572
02573         k=0; I[0]++;
02574         while(I[k]>=S[k] && k<l) { I[k]=0; ++I[++k]; }
02575     }
02576
02577     if (C.isEmpty()) X.save2(C); else {
02578         if (C.SIZE!=SIZE) wblog(FL,"ERR size mismatch in C: %s <> %s",
02579             SSTR_(this), SSTR(C));
02580         C+=X;
02581     }
02582 };
02583
02584 template<class T>
02585 wbararray<T>& wbararray<T>::ExpandDiagonal() {
02586
02587     size_t i,n=numel();
02588     wbararray<T> X;
02589
02590     if (!isVector())
02591         wblog(FL,"ERR %s() got %s array",FCT,SSTR_(this));
02592
02593     save2(X); init(n,n);
02594     for (i=0; i<n; ++i) data[i+n*i]=X.data[i];
02595
02596     return *this;
02597 };
02598
02599 template<class T>
02600 wbararray<T>& wbararray<T>::ExpandDiagonal(unsigned i1, unsigned i2) {
02601
02602     if (i1>=SIZE.len || i2>=SIZE.len || i1==i2) wblog(FL,
02603         "ERR %s() invalid indices (%d %d; %d)",FCT,i1+1,i2+1,SIZE.len);
02604     if (SIZE[i1]!=1 && SIZE[i2]!=1) wblog(FL,
02605         "ERR %s() array should be diagonal in (%s; %d %d)",
02606         FCT, SSTR_(this), i1+1, i2+1);
02607     if (SIZE[i1]==1 && SIZE[i2]==1) { return *this; }
02608
02609     if (SIZE[i1]!=1) SWAP(i1,i2);
02610
02611     size_t i,j,k,*s0,*s, r=SIZE.len, l=r-1;

```

```

02612     WBINDEX S=SIZE;
02613     wbararray<T> X;
02614     windex I(r);
02615
02616     save2(X); s0=X.SIZE.data;
02617     S[i1]=S[i2]; init(S); s=SIZE.data;
02618
02619     for (i=0; I[1]<s0[1]; i++) {
02620         for (j=0,k=1; k<r; k--) {
02621             if (j) { j*=s[k]; }
02622             j += (k!=i1 ? I[k] : I[i2]);
02623         }
02624         data[j] = X.data[i];
02625
02626         k=0; I[0]++;
02627         while(I[k]>=s0[k] && k<1) { I[k]=0; ++I[++k]; }
02628     }
02629     return *this;
02630 };
02631
02632 template<class T>
02633 wbararray<T>& wbararray<T>::Expand2Projector(T eps) {
02634
02635     size_t i,j,m,n=numel();
02636     WBINDEX S(2);
02637     wbvector<char> mark(n);
02638
02639     if (n==0) { if (SIZE.len>1) SIZE[0]=0; return *this; }
02640     if (!isVector()) wblog(FL,
02641         "ERR %s() array must be 1-dim vector (got %s)",
02642         FCT, SSTR_(this)
02643     );
02644
02645     for (m=i=0; i<n; i++) if (ABS(data[i])>eps) { mark[i]=1; m++;}
02646
02647     S[0]=n; S[1]=m; init(S);
02648
02649     for (i=j=0; i<n; i++)
02650         if (mark[i]) { data[i+(j++)*n]=1; }
02651
02652     return *this;
02653 };
02654
02655 template<class T> inline
02656 T wbararray<T>::froNorm2(const wbararray<T> &B) const {
02657     T x=0;
02658
02659     if (!sameSize(B)) wblog(FL,"ERR %s() size mismatch (%s/%s)",
02660         FCT, SSTR_(this), SSTR(B));
02661
02662     for (size_t n=numel(), i=0; i<n; i++) x+=CONJ(data[i])*B.data[i];
02663     return x;
02664 };
02665
02666 template<class T> inline
02667 T wbararray<T>::norm2() const {
02668     T d=0;
02669     for (size_t n=numel(), i=0; i<n; ++i) d+=data[i]*data[i];
02670     return d;
02671 };
02672
02673 template<> inline
02674 wbcomplex wbararray<wbcomplex>::norm2() const {
02675     double d=0;
02676
02677     for (size_t n=numel(), i=0; i<n; ++i)
02678         d+=data[i].abs2();
02679
02680     return d;
02681 };
02682
02683 template<class T> inline
02684 T wbararray<T>::hasNorm2(const T &x2) const {
02685     T q2=0; size_t i=0, n=numel();
02686     for (; i<n; ++i) {
02687         if ((q2+=NORM2(data[i]))>x2) { return q2; }
02688     }
02689     return -q2;
02690 };
02691
02692 template<class T> inline
02693 T wbararray<T>::normCol2(size_t k) const {
02694
02695     if (SIZE.len!=2) wblog(FL,
02696         "ERR %s() invalid matrix %s",FCT,SSTR_(this));
02697     if (k>=SIZE[1]) wblog(FL,
02698         "ERR %s() index out of bounds (%d/%d)",FCT,k,SIZE[1]);

```

```

02699
02700     size_t i=0, dim1=SIZE[0];
02701     const T *d=data+k*dim1; T d2=0;
02702
02703     for (i=0; i<dim1; ++i) { d2+=ABS2(d[i]); }
02704     return d2;
02705 };
02706
02707 template<class T> inline
02708 wbvector<T>& wbararray<T>::norm2Cols(wbvector<T> &x2_) const {
02709
02710     if (!SIZE.len) { if (x2_.len) { x2_.init(); }; return x2_; }
02711     if (SIZE.len!=2) wblog(FL,
02712         "ERR %s() invalid matrix %s",FCT,SSTR_(this));
02713
02714     x2_.init(SIZE[1]); {
02715         T* x2=x2_.data; size_t i,j, l=0, dim1=SIZE[0], dim2=SIZE[1];
02716         for (j=0; j<dim2; ++j)
02717             for (i=0; i<dim1; ++i, ++l) { x2[j]+=NORM2(data[l]); }
02718     }
02719     return x2_;
02720 };
02721
02722 template<class T> inline
02723 wbvector<T> wbararray<T>::normCols() const {
02724
02725     wbvector<T> x_; norm2Cols(x_);
02726     if (x_) {
02727         size_t i=0; T *x=x_.data;
02728         for (; i<x_.len; ++i) { x[i]=SQRT(x[i]); }
02729     }
02730     return x_;
02731 };
02732
02733 template<class T> inline
02734 T wbararray<T>::scalarProd(const wbararray<T> &B) const {
02735     T x=0; size_t i=0, n=numel();
02736
02737     if (!sameSize(B)) wblog(FL,
02738         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
02739     for (; i<n; i++) { x+=data[i]*B.data[i]; }
02740     return x;
02741 }
02742
02743 template<> inline
02744 wbcomplex wbararray<wbcomplex>::scalarProd(
02745     const wbararray<wbcomplex> &B
02746 ) const {
02747
02748     wbcomplex x=0; size_t i=0, n=numel();
02749
02750     if (!sameSize(B)) wblog(FL,
02751         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
02752     for (; i<n; ++i) { x+=data[i]*B.data[i].conj(); }
02753
02754     return x;
02755 };
02756
02757 template<class T> inline
02758 T wbararray<T>::sum() const {
02759     T x=0;
02760
02761     for (size_t n=numel(), i=0; i<n; i++)
02762         x+=data[i];
02763
02764     return x;
02765 }
02766
02767 template<class T> inline
02768 wbararray<T>& wbararray<T>::sum(
02769     const WBINDEX &K,
02770     wbararray<T>&A
02771 ) const {
02772
02773     size_t i,j,k,q, r=SIZE.len, l=r-1, s=numel();
02774     WBINDEX S=SIZE;
02775     wbvector<char> mark(r);
02776     char *m=mark.data;
02777     wbindex I;
02778
02779     for (k=i=0; i<K.len; i++) {
02780         if (K[i]<SIZE.len) { mark[K[i]]++; k++;
02781             if (mark[K[i]]>1) wblog(FL,
02782                 "ERR %s() index not unique (%s; %d)",
02783                 FCT,STR(K+1),SIZE.len);
02784             }
02785         else wblog(FL,

```

```

02786         "ERR %s() index out of bounds (%s; %d/%d)",
02787         FCT, STR(K+1), K[i], SIZE.len);
02788     }
02789
02790     for (k=i=0; i<mark.len; i++) if (!mark[i]) S[k++]=SIZE[i];
02791     if (k) S.len=k;
02792     else { A.init(1,1); A[0]=Wb::addRange(data,s); return A; }
02793
02794     A.init(S); I.init(SIZE.len);
02795     for (q=l; q<r; q--) if (!m[q]) break;
02796
02797     for (i=0; i<s; i++) {
02798         for (j=I[q], k=q-1; k<r; k--) if (!m[k]) j=j*SIZE[k]+I[k];
02799
02800         A.data[j]+=data[i];
02801
02802         k=0; I[0]++;
02803         while(I[k]>=SIZE[k] && k<l) { I[k]=0; ++I[++k]; }
02804     }
02805
02806     return A;
02807 }
02808
02809 template<class T> inline
02810 T wbarry<T>::trace() const {
02811     T x=0;
02812
02813     if (SIZE.len%2) wblog(FL,
02814         "ERR %s() got object with odd rank %d",FCT,SIZE.len);
02815     if (!isSquare()) wblog(FL,
02816         "WRN %s() of non-square array %s",FCT,SSTR_(this));
02817
02818     if (SIZE.len) {
02819         size_t n=SIZE[0], m;
02820
02821         if (SIZE.len==2) { m=SIZE[1]; }
02822         else {
02823             unsigned i=1, r2=SIZE.len/2; m=SIZE[r2];
02824             for (; i<r2; ++i) {
02825                 n*=SIZE[i];
02826                 m*=SIZE[r2+i];
02827             }
02828         }
02829
02830         if (m>n) { m=n; }
02831         for (size_t i=0; i<m; ++i) { x+=data[i+i*n]; }
02832     }
02833
02834     return x;
02835 };
02836
02837 #endif
02838

```

5.25 wbarry_blas.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbarry (array class, col-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_BLAS_CC__
00021 #define __WB_ARRAY_BLAS_CC__
00022
00023 /* ===== */
00024 // NB! call DZGEMM after all checks are made (such as size, rank, ...)
00025 // C = (alpha=afac) * op(A)*op(B) + (beta=cfac)*C
00026 // e.g., also C must already be fully allocated // DZGEMM_C_ALLOCATED
00027 /* ===== */

```

```

00028
00029 template<class TA, class TB, class TC >
00030 void DZGEMM(
00031     const wbararray<TA> &A,
00032     const wbararray<TB> &B, wbararray<TC> &C,
00033     const unsigned k,
00034     char aflag='N', char bflag='N',
00035     TA afac=1, TC cfac=0
00036 );
00037
00038 #ifdef QS_USING_MPFR
00039
00040 template<> inline
00041 void DZGEMM(
00042     const wbararray<Wb::quad> &A,
00043     const wbararray<Wb::quad> &B, wbararray<Wb::quad> &C,
00044     const unsigned k, char aflag, char bflag,
00045     Wb::quad afac, Wb::quad cfac
00046 ){
00047     wblog(FL,"ERR %s() not yet defined for data type Wb::quad");
00048     wblog(FL,"ERR %s() %p %p %p, %d %d %d, %p %p",
00049         FCT,&A, &B, &C, k, aflag, bflag, &afac, &cfac);
00050 };
00051
00052 #endif
00053
00054 template<> inline
00055 void DZGEMM(
00056     const wbararray<double> &A,
00057     const wbararray<double> &B, wbararray<double> &C,
00058     const unsigned k,
00059     char aflag, char bflag,
00060     double afac, double cfac
00061 ){
00062     if (A.SIZE.len!=2 || B.SIZE.len!=2 || C.SIZE.len!=2) wblog(FL,
00063         "ERR %s() requires matrices (got: %s * %s = %s)",
00064         FCT, SSTR(A), SSTR(B), SSTR(C));
00065
00066     unsigned
00067         a1=A.SIZE[0],
00068         b1=B.SIZE[0],
00069         c1=C.SIZE[0], c2=C.SIZE[1];
00070
00071     if (!a1 || !b1 || !c1 || !c2) {
00072         if (A.SIZE[1]==b1 && B.SIZE[1]==c2 && a1==c1) {
00073             if (cfac!=double(1)) C*=cfac;
00074             return;
00075         }
00076         wblog(FL,"ERR %s() got empty matrices (A: %s, B: %s; %c%c)",
00077             FCT,SSTR(A),SSTR(B),aflag,bflag); return;
00078     }
00079
00080 #ifdef WB_CLOCK
00081     unsigned a2=A.SIZE[1], b2=B.SIZE[1];
00082
00083     stat_dgemm.account(a1*a2*(a2==b1 ? b2 : b1));
00084
00085     Wb::Clock clk("dgemm",1);
00086 #endif
00087
00088     dgemm(
00089         aflag, bflag, (pINT)c1, (pINT)c2, (pINT)k,
00090         afac, A.data, (pINT)a1, B.data, (pINT)b1,
00091         cfac, C.data, (pINT)c1
00092     );
00093 };
00094
00095 template<> inline
00096 void DZGEMM(
00097     const wbararray<wbcomplex> &A,
00098     const wbararray<wbcomplex> &B, wbararray<wbcomplex> &C,
00099     const unsigned k,
00100     char aflag, char bflag,
00101     wbcomplex afac, wbcomplex cfac
00102 ){
00103     if (A.SIZE.len!=2 || B.SIZE.len!=2) wblog(FL,
00104         "ERR %s() matrices required (%s * %s)",FCT,
00105         A.sizeStr().data, B.sizeStr().data);
00106
00107     unsigned
00108         a1=A.SIZE[0],
00109         b1=B.SIZE[0],
00110         c1=C.SIZE[0], c2=C.SIZE[1];
00111
00112     if (!a1 || !b1 || !c1 || !c2) {
00113         if (A.SIZE[1]==b1 && B.SIZE[1]==c2 && a1==c1) {
00114             if (cfac!=double(1)) C*=cfac;

```

```

00115         return;
00116     }
00117     wblog(FL,"WRN %s() got empty matrices (A: %s, B: %s)",
00118         FCT, A.sizeStr().data, B.sizeStr().data); return;
00119 }
00120
00121 #ifdef WB_CLOCK
00122     unsigned a2=A.SIZE[1], b2=B.SIZE[1];
00123
00124     stat_dgemm.account(a1*a2*(a2==b1 ? b2 : b1));
00125     Wb::Clock clk("zgemm",1);
00126 #endif
00127
00128     zgemm (
00129         aflag, bflag, (pINT)c1, (pINT)c2, (pINT)k,
00130         afac, A.data, (pINT)a1, B.data, (pINT)b1,
00131         cfac, C.data, (pINT)c1
00132     );
00133
00134 #ifdef WB_CLOCK
00135     { Wb::Clock *clk;
00136       if (clk.stop())
00137         { Wb::Clock("zgeNX",1,0,&Wb::Clocks,&clk,0); }
00138       else { Wb::Clock("zgeNN",1,0,&Wb::Clocks,&clk,0); }
00139
00140       clk.ncall+=2;
00141       clk.tcpu+= (long unsigned)
00142         (double(a1*a2*b2)*(4E-9*CLOCKS_PER_SEC)+1.5);
00143     }
00144 #endif
00145 };
00146
00147 template<> inline
00148 void DZGEMM(
00149     const wbarray<wbcomplex> &A,
00150     const wbarray<double> &B0, wbarray<wbcomplex> &C,
00151     const unsigned k,
00152     char aflag, char bflag,
00153     wbcomplex afac, wbcomplex cfac
00154 ){
00155     wbarray<wbcomplex> B; B.set(B0, wbarray<double>());
00156     DZGEMM(A,B,C,k,aflag,bflag,afac,cfac);
00157 };
00158
00159 template<> inline
00160 void DZGEMM(
00161     const wbarray<double> &A0,
00162     const wbarray<wbcomplex> &B, wbarray<wbcomplex> &C,
00163     const unsigned k,
00164     char aflag, char bflag,
00165     double afac, wbcomplex cfac
00166 ){
00167     wbarray<wbcomplex> A; A.set(A0, wbarray<double>());
00168     DZGEMM(A,B,C,k,aflag,bflag,wbcomplex(afac,0),cfac);
00169 };
00170
00171 template<> inline
00172 void DZGEMM(
00173     const wbarray<double> &A,
00174     const wbarray<double> &B, wbarray<wbcomplex> &C,
00175     const unsigned k,
00176     char aflag, char bflag, double afac, wbcomplex cfac
00177 ){
00178     wbarray<double> C0(C.SIZE);
00179     DZGEMM(A,B,C0,k,aflag,bflag,afac);
00180
00181     C.Plus(C0,double(1),0,cfac);
00182 };
00183
00184 template<class T>
00185 void Wb::VMatProd(
00186     const wbarray<T> &A0, const wbarray<T> &B0, wbarray<T> &C,
00187     char aflag, char bflag, const T afac, const T cfac,
00188     const char cforce
00189 ){
00190     char isvec=0; wbarray<T> A,B;
00191
00192     if (A0.SIZE.len==1) {
00193         isvec++; A0.adjustVec(FL,aflag,A,1);
00194     }
00195     else A.init2ref(A0);
00196
00197     if (B0.SIZE.len==1) {
00198         isvec++; B0.adjustVec(FL,bflag,B,2);
00199     }
00200     else B.init2ref(B0);
00201

```

```

00202     Wb::MatProd(A,B,C,aflag,bflag,afac,cfac,cforce);
00203
00204     if (isvec) { C.skipSingletons(); }
00205 };
00206
00207 template<class T>
00208 void Wb::MatVProd(
00209     const wbararray<T> &A, const wbvector<T> &v, wbararray<T> &x,
00210     char aflag, char bflag, const T afac, const T cfac,
00211     const char cforce
00212 ){
00213     wbararray<T> B;
00214     opFlags<T> Aflag(aflag), Bflag(bflag);
00215
00216     if (Bflag.conj()) { B.init(v.len,1,v.data).Conj(); }
00217     else {
00218         if (Bflag.trans()) wblog(FL,
00219             "WRN %s() ignoring bflag=%c",FCT,bflag);
00220         B.init2ref(v.len,1,v.data);
00221     }
00222
00223     Wb::MatProd(A,B,x,aflag,'N',afac,cfac,cforce);
00224     x.skipSingletons();
00225 };
00226
00227 template<class T>
00228 T Wb::VMatVprod(
00229     const wbararray<T> &v1, const wbararray<T> &M, const wbararray<T> &v2
00230 ){
00231     unsigned i,dim1,dim2;
00232     const T *p1=v1.data, *p2=v2.data, *dd=M.data;
00233     T x=0;
00234
00235     M.getMatSize(FL,dim1,dim2);
00236
00237     if (v1.numel()!=dim1 || v2.numel()!=dim2) wblog(FL,
00238         "ERR %s() severe size mismatch: %s x %s x %s",FCT,
00239         v1.sizeStr().data, M.sizeStr().data, v2.sizeStr().data);
00240
00241     for (i=0; i<dim2; i++, dd+=dim1) {
00242         x+=Wb::overlap(p1,dd,dim1)*p2[i];
00243     }
00244
00245     return x;
00246 };
00247
00248 template<class T>
00249 T Wb::VMatVprod(
00250     const wbvector<T> &v1, const wbararray<T> &M, const wbvector<T> &v2
00251 ){
00252     unsigned i,dim1,dim2;
00253     const T *p1=v1.data, *p2=v2.data, *dd=M.data;
00254     T x=0;
00255
00256     M.getMatSize(FL,dim1,dim2);
00257
00258     if (v1.len!=dim1 || v2.len!=dim2) wblog(FL,
00259         "ERR %s() severe size mismatch: < %d | %s | %d>",
00260         FCT, v1.len, M.sizeStr().data, v2.len);
00261
00262     for (i=0; i<dim2; i++, dd+=dim1) {
00263         x+=Wb::overlap(p1,dd,dim1)*p2[i];
00264     }
00265
00266     return x;
00267 };
00268
00269 template<class TA, class TB, class TC>
00270 wbararray<TC> & Wb::MatProd(
00271     const wbararray<TA> &A, const wbararray<TB> &B, wbararray<TC> &C,
00272     char aflag, char bflag, TA afac, TC cfac,
00273     char cforce
00274 ){
00275     if ((void*)&A==(void*)&C || (void*)&B==(void*)&C) {
00276         wbararray<TC> XC; if (cfac!=TC(0)) XC.init(C);
00277         Wb::MatProd(A,B,XC,aflag,bflag,afac,cfac,cforce);
00278         XC.save2(C); return C;
00279     }
00280
00281     unsigned a1,a2,b1,b2;
00282     char iflag=0;
00283
00284     #ifdef WB_CLOCK
00285         Wb::Clock clk("MatProd",1);
00286     #endif
00287
00288     if (A.rank()!=2 || B.rank()!=2) wblog(FL,

```



```

00289     "ERR %s() rank-2 tensors required (%s; %s)", FCT, SSTR(A), SSTR(B));
00290
00291     a1=A.SIZE[0]; a2=A.SIZE[1]; if (aflag!='N') SWAP(a1,a2);
00292     b1=B.SIZE[0]; b2=B.SIZE[1]; if (bflag!='N') SWAP(b1,b2);
00293
00294     if (a2!=b1) wblog(FL,
00295         "ERR %s() size mismatch %dx%d * %dx%d", FCT, a1, a2, b1, b2);
00296
00297     if (cfac!=TC(0)) {
00298         if (C.data==NULL) { iflag=1;
00299             if (cforce) wblog(FL,
00300                 "WRN C = A*B + c*[] with c=%s", toStr(double(cfac)).data);
00301         }
00302         else if (!C.isMatrix() || C.SIZE[0]!=a1 || C.SIZE[1]!=b2) {
00303             wblog(FL, "ERR %s() dimension mismatch: C=(%s) =? (%d,%d).",
00304                 FCT, SSTR(C), a1, b2); iflag=1;
00305         }
00306     } else iflag=1;
00307
00308     if (iflag) {
00309         C.init(a1,b2);
00310     }
00311
00312     DZGEMM(A,B,C,a2,aflag,bflag,afac,cfac);
00313
00314     return C;
00315 };
00316
00317 template<class TA, class TB, class TC>
00318 void Wb::DMatProd(
00319     const wbarry<TA> &A0, const wbarry<TB> &B0, wbarry<TC> &C,
00320     char aflag, char bflag, const TA afac, const TC cfac,
00321     const char cforce
00322 ){
00323     if (A0.SIZE.len==2 && B0.SIZE.len==2) {
00324         Wb::MatProd(A0,B0,C, aflag,bflag, afac,cfac, cforce);
00325         return;
00326     }
00327
00328 #ifdef WB_CLOCK
00329     Wb::Clock clk("MatProd",1);
00330 #endif
00331
00332     if (A0.SIZE.len<1 || A0.SIZE.len>2 || B0.SIZE.len<1 || B0.SIZE.len>2)
00333         wblog(FL, "ERR %s() rank <= 2 required (got %d,%d)", FCT,
00334             A0.SIZE.len, B0.SIZE.len);
00335
00336     unsigned i,j,k,sa,a1,a2,sb,b1,b2;
00337     wbarry<TA> A;
00338     wbarry<TB> B;
00339
00340     if (A0.SIZE.len==1 && B0.SIZE.len==2) {
00341         A0.adjustDMat(FL,aflag, afac, A);
00342         B0.adjustMMat(FL,bflag, 1. , B);
00343
00344         sa=A.SIZE[0]; b1=B.SIZE[0]; b2=B.SIZE[1]; sb=b1*b2;
00345         if (sa!=b1) wblog(FL,
00346             "ERR %s() sever size mismatch (%d * %dx%d)",
00347             FCT, sa, b1, b2);
00348
00349         C.adjustCMat (FL,cfac,b1,b2,cforce);
00350
00351         if (C.isEmpty()) { C=B;
00352             for (k=j=0; j<b2; j++)
00353                 for ( i=0; i<b1; i++, k++) C.data[k]*=A.data[i];
00354         }
00355         else {
00356             for (k=j=0; j<b2; j++)
00357                 for ( i=0; i<b1; i++, k++) C.data[k]+=A.data[i]*B.data[k];
00358         }
00359     }
00360     else
00361     if (A0.SIZE.len==2 && B0.SIZE.len==1) {
00362
00363         TB bfac; Wb::safeConvert (FL,afac,bfac);
00364
00365         A0.adjustMMat(FL,aflag, 1. , A);
00366         B0.adjustDMat(FL,bflag, bfac, B);
00367
00368         a1=A.SIZE[0]; a2=A.SIZE[1]; sa=a1*a2; sb=B.SIZE[0];
00369         if (a2!=sb) wblog(FL,
00370             "ERR %s() sever size mismatch (%dx%d * %d)",
00371             FCT, a1, a2, sb);
00372
00373         C.adjustCMat (FL,cfac,a1,a2,cforce);
00374         if (C.isEmpty()) { C=A;
00375             for (k=j=0; j<a2; j++)

```

```

00376         for ( i=0; i<a1; i++, k++) C.data[k]*=B.data[j];
00377     }
00378     else {
00379         for (k=j=0; j<a2; j++)
00380             for ( i=0; i<a1; i++, k++) C.data[k]+=A.data[k]*B.data[j];
00381     }
00382 }
00383 else wblog(FL, "ERR !?");
00384 };
00385
00386 template<>
00387 wbararray<double>& wbInverse(const char *F, int L, wbararray<double> &M) {
00388
00389     if (M.rank()!=2 || M.SIZE[1]>M.SIZE[0]) wblog(F_L,
00390         "ERR %s() invalid rank-2 tensor (%s)", FCT, M.sizeStr().data);
00391
00392     pINT e=0, l=-1, m=M.SIZE[0], n=M.SIZE[1], N=m*n;
00393     wbvector<pINT> ipiv(MIN(m,n));
00394     wbvector<double> work(1);
00395
00396     if (n<m) wblog(FL,
00397         "ERR DGETRI() requires dim1>=dim2 (%dx%d)", m,n);
00398
00399     dgetrf(m,n,M.data,m,ipiv.data,e);
00400     if (e) wblog(FL, "ERR DGETRF() returned e=%d !?", FCT,e);
00401
00402     dgetri(n,M.data,m,ipiv.data,work.data,1,e);
00403     if (e) wblog(FL, "ERR DGETRI() returned e=%d !?", FCT,e);
00404
00405     l=MIN(pINT(work[0]),N); work.init(1);
00406     dgetri(n,M.data,m,ipiv.data,work.data,1,e);
00407     if (e) {
00408         if (e<0)
00409             wblog(FL, "ERR DGETRI() got invalid argument #&d", -e);
00410         else wblog(FL, "ERR DGETRI() got singular matrix (i=%d)", e);
00411     }
00412
00413     return M;
00414 };
00415
00416 template<>
00417 wbararray<double>& wbInverse(
00418     const char *F, int L,
00419     const wbararray<double>&M, wbararray<double> &Minv
00420 ){ Minv=M; return wbInverse(F,L,Minv); };
00421
00422 inline void wbEigenS(
00423     const wbararray<double> &M, wbararray<double> &V, wbvector<double> &E
00424 ){
00425     unsigned n, r=M.SIZE.len, r2=r/2;
00426     pINT ni=0, q=0;
00427     double nd=0;
00428
00429     if (r==0) { V.init(); E.init(); return; }
00430
00431     if (r%2) wblog(FL, "ERR %s() even-rank required (%d)", FCT,r);
00432     if (!M.isHConj(-1E-12)) {
00433         MXPut(FL, "ans").add(M, "M").add(V, "V").add(E, "E").add(r, "r");
00434         wblog(FL, "ERR %s() got non-symmetric %s matrix",
00435             FCT, M.sizeStr().data);
00436     }
00437
00438     n=Wb::prodRange(M.SIZE.data, r2);
00439
00440     V=M; E.init(n); if (n==0) return;
00441
00442     if (r2>1) {
00443         V.SIZE[r2]=n; V.SIZE.len=r2+1;
00444     }
00445
00446     dsyevd('V', 'L', n, V.data, n, E.data, &nd, -1, &ni, -1, q);
00447     if (nd<1 || ni<1 || q) wblog(FL,
00448         "ERR DSYEVD returned lwork=%g/%d (e=%d) !?", nd, ni, q);
00449
00450     wbvector<double> wd((pINT)nd);
00451     wbvector<pINT> wi(ni);
00452
00453     dsyevd(
00454         'V',
00455         'L',
00456         n,
00457         V.data,
00458         n,
00459         E.data,
00460         wd.data, wd.len,
00461         wi.data, wi.len,
00462         q

```

```

00463     );
00464
00465     if (q>0) { pINT q_=q; V.Reset(M.data);
00466     dsyeupd('V','U',n,V.data,n,E.data,wd.data,wd.len,wi.data, wi.len,q);
00467
00468     if (!q) wblog(FL,
00469     "WRN DSYEVD() ok only for uplo=L->U (i=%d/%d) !?",q_,n);
00470     else wblog(FL,
00471     "ERR DSYEVD() failed to converge (info=%d/%d) !?\n"
00472     "(D=%d @ l=%ld/%ld)",q_,q,n,E.len,wd.len,wi.len);
00473     }
00474
00475     if (q) {
00476     MXPut(FL,0,"tmpfile").add(M,"M").add(q,"q");
00477     wblog(FL,"ERR DSYEVD() returned "
00478     "info=%d (D=%d; l=%ld/%ld) !?",q,n,wd.len,wi.len);
00479     }
00480 };
00481
00482 inline void wbEigenS(
00483     const wbarray<wbcomplex> &M, wbarray<wbcomplex> &V, wbvector<double> &E
00484 ) {
00485     unsigned n, r=M.SIZE.len, r2=r/2;
00486     pINT ni=0, q=0;
00487     wbcomplex nz=0; double nr=0;
00488
00489     if (r==0) { V.init(); E.init(); return; }
00490
00491     if (r%2) wblog(FL,"ERR %s() even-rank required (%d)",FCT,r);
00492     if (!M.isHConj(-1E-12)) {
00493     MXPut(FL,"ans").add(M,"M").add(V,"V").add(E,"E").add(r,"r");
00494     wblog(FL,"ERR %s() got non-hermitian %s matrix",FCT, SSTR(M));
00495     }
00496
00497     n=Wb::prodRange(M.SIZE.data,r2);
00498
00499     V=M; E.init(n);
00500     if (n<=1) {
00501     if (n) {
00502     double x=V.SkipTiny_imag(1E-12);
00503     if (x<0) wblog(FL,"ERR %s() got complex data @ %.3g",FCT,x);
00504     E.data[0]=V.data[0]; V.data[0]=1;
00505     }
00506     return;
00507     }
00508
00509     if (r2>1) {
00510     V.SIZE[r2]=n; V.SIZE.len=r2+1;
00511     }
00512
00513     zheevd('V','U',n,V.data,n,E.data,&nz,-1,&nr,-1,&ni,-1,q);
00514     if (nz.r<1 || nr<1 || ni<1 || q) {
00515     MXPut(FL,0,"tmpfile").add(M,"M").add(q,"q");
00516     wblog(FL,"ERR ZHEEVD returned lwork=%g/%g/%d (e=%d)",nz.r,nr,ni,q);
00517     }
00518
00519     wbvector<wbcomplex> wz(nz.r);
00520     wbvector<double> wr(nr);
00521     wbvector<pINT> wi(ni);
00522
00523     zheevd(
00524     'V',
00525     'U',
00526     n,
00527     V.data,
00528     n,
00529     E.data,
00530     wz.data, wz.len,
00531     wr.data, wr.len,
00532     wi.data, wi.len,
00533     q
00534     );
00535
00536     if (!q) { return; }
00537     if (q<0) wblog(FL,"ERR %s() ZHEEVD returned e=%d",FCT,q);
00538
00539     V=M; E.init(n);
00540
00541     wblog(FL,"WRN ZHEEVD returned i=%d / falling back to ZHEEV",q);
00542     MXPut(FL,0,"tmpfile").add(M,"M").add(q,"q");
00543
00544     if (n<128) { ni=2*n-1; }
00545     else {
00546     ni=ilaenv(1,"zhetrd","U", (pINT)n, (pINT)n, (pINT)n, (pINT)n);
00547     if (ni>n) wblog(FL,"WRN %s() got lwork=(%d+2)*%d",FCT,ni,n);
00548     ni=(ni+1)*n;
00549     }

```

```

00550     if (size_t(ni)>wz.len) { wz.init(ni); }
00551
00552     nr=3*n-2; if (nr>wr.len) { wr.init(nr); }
00553
00554     zheev(
00555         'V',
00556         'U',
00557         n,
00558         V.data,
00559         n,
00560         E.data,
00561         wz.data, ni,
00562         wr.data,
00563         q
00564     );
00565
00566     if (q) wblog(FL, "ERR %s() ZHEEV returned e=%d", FCT, q);
00567 };
00568
00569 void wbEigen_CS (
00570     wbararray<wbcomplex> &M,
00571     wbararray<wbcomplex> &V,
00572     wbvector<wbcomplex> &E,
00573     char wjob,
00574     char issym,
00575     char tnorm,
00576     char qflag
00577 ){
00578     unsigned n=0, r=M.SIZE.len, r2=r/2;
00579     char jobvl='N', jobvr='V', vflag=1; pINT q=0;
00580
00581     if (wjob) { wjob=toupper(wjob); jobvr=jobvl='N';
00582         switch (wjob) {
00583             case 'L': jobvl='V'; break;
00584             case 'R': jobvr='V'; break;
00585             case 'N': vflag=0; break;
00586             default : wblog(FL, "ERR invalid wjob=%c<%d>", wjob, wjob);
00587         }
00588     }
00589
00590     if (r==0) { V.init(); E.init(); return; }
00591
00592     if (!M.isOpS(n)) wblog(FL,
00593         "ERR %s() (generalized) square matrix required (%s)",
00594         FCT, M.sizeStr().data);
00595
00596     E.init(n); if (vflag) V.init(n,n);
00597
00598     if (n<=0) {
00599         if (n) { E[0]=M[0]; V[0]=1; }
00600         return;
00601     }
00602
00603     if (r2>1) {
00604         V.SIZE[r2]=n; V.SIZE.len=r2+1;
00605     }
00606
00607     if (issym) { tnorm=1;
00608         double x=1E-12;
00609         if (issym=='T') M.Symmetrize(FL, &x, 0);
00610         else if (issym=='H') M.Symmetrize(FL, &x, 1);
00611         else wblog(FL, "ERR invalid issym=%c<%d>", issym, issym);
00612     }
00613     if (x>1E-8) {
00614 #ifdef MATLAB_MEX_FILE
00615         M.put("M_");
00616 #endif
00617         wblog(FL, "ERR operator not symmetric (%.3g)", x);
00618     }
00619     else if (x>1E-12) wblog(FL,
00620         "WRN operator not quite symmetric (%.3g)", x);
00621 }
00622
00623 wbvector<wbcomplex> aux;
00624 wbvector<double> aux2(2*n);
00625
00626 unsigned lwork=MAX(2U, n/4)*n;
00627
00628 if (n>127) {
00629     zgeev (
00630         jobvl, jobvr, n, M.data, n, E.data, V.data, n, V.data, n,
00631         aux.data, -1, aux2.data, q);
00632     lwork=(unsigned)aux[0].r; {
00633         unsigned b=lwork/n; if (b<2 || b>n)
00634             wblog(FL, "WRN %s() got lwork %g*<%d>", FCT, double(lwork)/n, n);
00635     }
00636 }

```

```

00637
00638     aux.init(lwork);
00639
00640 #ifdef WB_CLOCK
00641     Wb::Clock clk("zggev",1);
00642 #endif
00643
00644 { wbararray<wbcomplex> X(M);
00645     zggev (
00646         jobvl,
00647         jobvr,
00648         n,
00649         X.data,
00650         n,
00651         E.data,
00652         V.data,
00653         jobvl=='V' ? n : 1,
00654         V.data,
00655         jobvr=='V' ? n : 1,
00656         aux.data,
00657         aux.len,
00658         aux2.data,
00659         q
00660     ); }
00661
00662     if (q) {
00663         MXPput(FL,0,"tmpfile").add(M,"M").add(q,"q");
00664         wblog(FL,"ERR %s() CGEEV returned %d (%d/%d)",FCT,q,aux.len,n);
00665     }
00666
00667     if (tnorm) {
00668         wbEigen_CS_regen(M,V,E,wjob);
00669
00670         wbararray<wbcomplex> X; double eps=10E-8;
00671         Wb::MatProd(V,V,X,'T');
00672
00673         if (!X.isIdentityMatrix(&eps)) {
00674 #ifdef MATLAB_MEX_FILE
00675             printf("\n"); M.put(FL,"M_"); V.put(FL,"V_"); E.put(FL,"E_",'t');
00676             wbvector<wbcomplex> Es(E); Es.Sort(); Es.put("Es_"); X.put("X_");
00677 #endif
00678             if (qflag)
00679                 wblog(FL,"WRN eig() U matrix not quite orthogonal (%.3g)",eps); else
00680                 wblog(FL,"ERR eig() U matrix not quite orthogonal (%.3g)",eps);
00681         }
00682     }
00683 }
00684
00685 template<class T>
00686 void wbEigen_CS_regen(
00687     const wbararray<T> &M0,
00688     wbararray<T> &V,
00689     wbvector<T> &E,
00690     char wjob
00691 ){
00692     if (E.len==0) return;
00693     if (!V.isRank(2) || E.len!=V.SIZE[1] || V.SIZE[0]!=V.SIZE[1])
00694         wblog(FL,"ERR invalid usage (%s; %d)",V.sizeStr().data,E.len);
00695
00696     unsigned i,i0=0, n=E.len, count=0;
00697     WBINDEX S(2); S[0]=n;
00698     double a, eps=1E-8, nmin=0, nmax=0;
00699
00700     wbpem P; E.Sort(P); V.Select0(P,1);
00701
00702     V.NormalizeCols(FL,&nmin,&nmax,'T');
00703
00704     if (nmin>1+1E-14) wblog(FL,
00705         "ERR zggev() orthogonal matrix with t-norm = %.3g",nmin); else
00706     if (nmin<1E-4) wblog(FL,
00707         "ERR zggev() orthogonal matrix with t-norm = %.3g",nmin); else
00708     if (nmin<0.01) wblog(FL,
00709         "WRN zggev() orthogonal matrix with t-norm = %.3g",nmin);
00710
00711     for (i=1; i<n; i++) {
00712         a=ABSDIFF_F(E[i],E[i-1]);
00713
00714         if (i+1<n) { if (a<eps) continue; }
00715         else { if (a<eps) i++; }
00716
00717         if (i>i0+1) { T eref; double escale, x=eps;
00718
00719             wblog(FL,"TST fixing degenerate block: %d:%d (%d)  \r\\",i0+1,i,n);
00720
00721             wbararray<T> U,X,M2,u2; wbvector<T> e2;
00722
00723             S[1]=i-i0; U.init2ref(V.ref(i0),S);

```

```

00724         Wb::MatProd(M0,U,X); Wb::MatProd(U,X,M2,'T');
00725
00726         M2.Symmetrize(FL,&x,0);
00727         if (x>eps) wblog(FL,
00728             "WRN matrix not quite symmetric (x=%.4g) !?",x);
00729         M2.balanceOp(FL,eref,escale);
00730
00731         wbEigen_CS (M2,u2,e2,wjob,0,0);
00732
00733         e2*=escale; e2+=eref;
00734         Wb::cpyRange(E.data+i0,e2.data,e2.len);
00735
00736         Wb::MatProd(U,u2,X);
00737         memcpy(U.data,X.data,sizeof(T)*X.numel());
00738
00739         U.OrthoNormalizeCols(FL,'T',T(1E-14),2);
00740         count++;
00741     }
00742     i0=i;
00743 }
00744
00745 };
00746
00747 template<class T>
00748 void wbEigen_CS_regen_trial(
00749     const wbararray<T> &M0,
00750     wbararray<T> &V,
00751     wbvector<T> &E,
00752     char wjob
00753 ){
00754     if (E.len==0) return;
00755     if (!V.isRank(2) || E.len!=V.SIZE[1] || V.SIZE[0]!=V.SIZE[1])
00756         wblog(FL,"ERR invalid usage (%s; %d)",V.sizeStr().data,E.len);
00757
00758     unsigned i,j,k,i0=0, n=E.len; double a, eps=1E-8;
00759
00760     wbperm P; E.Sort(P); V.Select0(P,1);
00761     wbcomplex *vr,*v,z,z2;
00762     unsigned count=0;
00763
00764     wbararray<T> U,X,M2,u2; wbvector<T> e2;
00765     double eps0, eps2;
00766
00767     Wb::MatProd(M0,V,X); Wb::MatProd(V,X,M2,'T');
00768     eps0=eps; M2.isDiagMatrix(&eps0);
00769     wbararray<T> V0(V);
00770
00771     for (i=1; i<n; i++) {
00772         a=ABS(E[i]-E[i-1]);
00773         if (a<eps) {
00774
00775             for (j=i0; j<i; j++) { vr=V.ref(j); v=V.ref(i);
00776                 z=z2=0; for (k=0; k<n; k++) { z+=vr[k]*v[k]; z2+=(vr[k]*vr[k]); }
00777                 z=z/z2; for (k=0; k<n; k++) v[k]-=z*vr[k];
00778
00779                 z=z2=0; for (k=0; k<n; k++) { z+=vr[k]*v[k]; z2+=(vr[k]*vr[k]); }
00780                 z=z/z2; for (k=0; k<n; k++) v[k]-=z*vr[k];
00781             }
00782
00783             wblog(FL,"TST fixing degenerate block: %d-%d (%.3g)",i0,j,std::sqrt(z2.abs())); count++;
00784
00785             }
00786         else i0=i;
00787     }
00788
00789     if (count) {
00790         Wb::MatProd(M0,V,X); Wb::MatProd(V,X,M2,'T');
00791         eps2=eps; M2.isDiagMatrix(&eps2);
00792         wblog(FL,"TST fixing degenerate block: %.6g => %.6g",eps0,eps2);
00793         V0.put("V0_"); V.put("V_"); M0.put("M_"); E.put("E_");
00794         wblog(FL,"ERR");
00795     }
00796 };
00797
00798 template<class T> inline
00799 void GESVD_old(
00800     wbararray<T> &A,
00801     wbararray<T> &U,
00802     wbvector<double> &S,
00803     wbararray<T> &Vd
00804 ){
00805     wblog(FL,"ERR %s() not defined for datatype %s",
00806         FCT,getName(typeid(T)).c_str());
00807 };
00808
00809 template<> inline
00810 void GESVD_old(

```

```

00811     wbarry<double> &A,
00812     wbarry<double> &U,
00813     wbvector<double> &S,
00814     wbarry<double> &Vd
00815 ){
00816     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00817     pINT i;
00818
00819     pINT lwork=MAX3(1U, 3*MIN(dim1,dim2)+MAX(dim1,dim2), 5*MIN(dim1,dim2));
00820
00821     if (MIN(dim1,dim2)>=8) {
00822         int e=0; double w;
00823         dgesvd(
00824             'S','S', dim1, dim2, A.data, dim1, S.data,
00825             U.data, dim1, Vd.data, k,
00826             &w, -1, i);
00827         i=(pINT)w;
00828
00829         if (i<lwork) e=1; else
00830         if (i>dim1*dim2 && i>(1<<20)) {
00831             unsigned m=max(dim1,dim2); e=(i<(m*m) ? 2 : 3);
00832         }
00833         if (e) { wblog(FL,
00834             "WRN dgesvd() lwork=%d/%d (%.4g; %dx%d; e=%d)",
00835             i,lwork,double(i)/double(dim1*dim2),dim1,dim2,e);
00836         }
00837         lwork=i;
00838     }
00839
00840     wbvector<double> W(lwork);
00841
00842     dgesvd(
00843         'S','S', dim1, dim2, A.data, dim1, S.data,
00844         U.data, dim1,
00845         Vd.data, k,
00846         W.data, W.len, i
00847     );
00848
00849     if (i) { if (i<0) { i=-i; wblog(FL,
00850         "ERR %s() dgesvd: invalid argument %d\nhint: %s",FCT,i,
00851         i==5 || i==8 ? "check input data for NAN or INF values":"??");
00852     }
00853     else { wblog(FL,
00854         "ERR %s() dgesvd returned e=%d\n"
00855         "hint: check input data for NAN or INF values",FCT,i);
00856     }}
00857 };
00858
00859 template<> inline
00860 void GESVD_old(
00861     wbarry<wbcomplex> &A,
00862     wbarry<wbcomplex> &U,
00863     wbvector<double> &S,
00864     wbarry<wbcomplex> &Vd
00865 ){
00866     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00867     wbvector<wbcomplex> W, RW(5*MIN(dim1,dim2));
00868     pINT i, lwork=MAX(1U, 2*MIN(dim1,dim2)+MAX(dim1,dim2));
00869
00870     if (MIN(dim1,dim2)>=8) { wbcomplex w;
00871         zgesvd(
00872             'S','S', dim1, dim2, A.data, dim1, S.data,
00873             U.data, dim1, Vd.data, k,
00874             &w, -1, RW.data, i
00875         );
00876
00877         i=unsigned(w.r);
00878         if (i<lwork || (i>10*dim1*dim2 && lwork>1E6)) {
00879             unsigned m=MIN(dim1,dim2); wblog(FL,"WRN zgesvd(%dx%d) "
00880             "lwork = %d -> %d (~%d*%d)",dim1,dim2,lwork,i,i/m,m);
00881         }
00882         lwork=i;
00883     }
00884
00885     W.init(lwork);
00886
00887     zgesvd(
00888         'S','S', dim1, dim2, A.data, dim1, S.data,
00889         U.data, dim1,
00890         Vd.data, k,
00891         W.data, W.len, RW.data, i
00892     );
00893
00894     if (i) {
00895         if (i<0) { i=-i;
00896             wblog(FL,"ERR %s() zgesvd: invalid argument %d\nhint: %s",FCT,i,
00897             i==5 || i==8 ? "check input data for NAN or INF values":"??");

```

```

00898     }
00899     else wblog(FL, "ERR %s() zgesvd returned i=%d", FCT, i);
00900 }
00901 };
00902
00903 template<class T> inline
00904 int GESVD(
00905     wbararray<T> &A,
00906     wbararray<T> &U, wvector<double> &S, wbararray<T> &Vd,
00907     char lflag=0
00908 );
00909
00910 template<> inline
00911 int GESVD(
00912     wbararray<double> &A,
00913     wbararray<double> &U,
00914     wvector<double> &S,
00915     wbararray<double> &Vd, char lflag
00916 ){
00917     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00918     pINT q, ni=8*MIN(dim1,dim2);
00919     double nd;
00920
00921     wvector<double> wd;
00922     wvector<pINT> wi(ni);
00923
00924 #ifdef WB_CLOCK
00925     Wb::Clock clk("dgesvd",1);
00926 #endif
00927
00928     dgesdd('S', dim1, dim2, A.data, dim1, S.data, U.data, dim1, Vd.data, k,
00929         &nd, -1, wi.data, q);
00930     if (nd<1 || q) wblog(FL,
00931         "ERR DGESDD() returned lwork=%g (e=%d) !?", nd, q);
00932     wd.init(nd);
00933
00934     dgesdd(
00935         'S', dim1, dim2, A.data, dim1, S.data,
00936         U.data, dim1,
00937         Vd.data, k,
00938         wd.data, wd.len, wi.data,
00939         q
00940     );
00941
00942     if (q) {
00943         if (q<0) { q=-q; wblog(FL,
00944             "ERR DGESDD() invalid argument %d\n%s", q, q==5 || q==8 ?
00945             "hint: check input data for NAN or INF values" : ""); }
00946         else if (lflag) {
00947             wblog(FL, "WRN DGESDD() did not converge (info=%d) !?", q);
00948         }
00949         else {
00950             MXPput(FL, 0, "tmpfile").add(A, "A").add(q, "q");
00951             wblog(FL, "ERR DGESDD() did not converge (info=%d) !?", q);
00952         }
00953     }
00954     return q;
00955 };
00956
00957 template<> inline
00958 int GESVD(
00959     wbararray<wbcomplex> &A,
00960     wbararray<wbcomplex> &U,
00961     wvector<double> &S,
00962     wbararray<wbcomplex> &Vd, char lflag
00963 ){
00964     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00965     pINT q, n1=MIN(dim1,dim2), n2=MAX(dim1,dim2);
00966
00967     pINT nd = n1*(3*n1+6 >= 2*n2 ? 5*n1+7 : 2*(n1+n2)+1);
00968
00969 #ifdef WB_CLOCK
00970     Wb::Clock clk("zgesvd",1);
00971 #endif
00972
00973     wbcomplex nz;
00974     wvector<wbcomplex> wz;
00975     wvector<double> wd(nd);
00976     wvector<pINT> wi(8*n1);
00977
00978     if (!dim1 || !dim2) wblog(FL,
00979         "ERR %s() got %dx%d matrix!?", FCT, dim1, dim2);
00980
00981     zgesdd('S', dim1, dim2, A.data, dim1, S.data, U.data, dim1, Vd.data, k,
00982         &nz, -1, wd.data, wi.data, q);
00983     if (nz.r<1 || q) wblog(FL,
00984         "ERR ZGESDD() returned lwork=%g (e=%d) !?", nz.r, q);

```



```

00985     wz.init(nz.r);
00986
00987     zgesdd(
00988         'S', dim1, dim2, A.data, dim1, S.data, U.data, dim1, Vd.data, k,
00989         wz.data, wz.len, wd.data, wi.data,
00990         q
00991     );
00992
00993     if (q) {
00994         if (q<0) { q=-q; wblog(FL,
00995             "ERR ZGESDD() invalid argument %d\n%s", q, q==5 || q==8 ?
00996             "hint: check input data for NAN or INF values" : ""); }
00997         else if (lflag) {
00998             wblog(FL, "WRN ZGESDD() failed to converge (info=%d) !?", q);
00999         }
01000         else {
01001             wblog(FL, "ERR ZGESDD() failed to converge (info=%d) !?", q);
01002         }
01003     }
01004     return q;
01005 };
01006
01007 template<class T>
01008 void wbsVD(
01009     const wbarray<T> &A,
01010     wbarray<T> &U, wbvector<double> &S,
01011     wbarray<T> &Vc,
01012     const WBINDEX &I,
01013     char bare
01014 ){
01015     unsigned dim1, dim2, k, r=A.SIZE.len, l=I.len;
01016     int itry=0, ntry=2, q=0;
01017     wbarray<T> X, Vd;
01018     wbpem P;
01019
01020     static unsigned ncall=0;
01021     static int nout=0; ++ncall;
01022
01023     for (; itry<ntry; ++itry) {
01024         if (l==0) {
01025             if (r!=2) wblog(FL,
01026                 "ERR %s() rank-2 required, use index otherwise (%d)", FCT, r);
01027             X.init2ref(A);
01028         }
01029         else {
01030             if (l>=r) wblog(FL,
01031                 "ERR %s() invalid dimensions (%s; %d/%d)", FCT, I.toStr().data, l, r);
01032             A.toMatrixRef(X, I, l, P);
01033         }
01034
01035         if (X.isRef()) X.Instantiate();
01036
01037         dim1=X.SIZE[0]; dim2=X.SIZE[1]; k=MIN(dim1, dim2);
01038
01039         U.init(dim1, k); S.init(k); Vd.init(k, dim2);
01040         if (!dim1 || !dim2) return;
01041
01042         if (itry==0) {
01043             q=GESVD(X, U, S, Vd, 'l');
01044             if (!q) break;
01045         }
01046         else {
01047             char vn[8]; snprintf(vn, 8, "Isvd%d", (++nout)%10);
01048             MXPput Ix(FL, vn, "tmpfile");
01049             Ix.add(ncall, "ncall").add(q, "q").add(X, "A");
01050
01051             wblog(FL, "WRN GESVD returned i=%d / falling back to ZGESVD", q);
01052             GESVD_old(X, U, S, Vd);
01053             Ix.add(U, "U").add(S, "S").add(Vd, "Vd");
01054         }
01055     }
01056 }
01057
01058 #ifdef SVD_BUG_SAFEGUARD
01059 { double a=SQRT(A.norm2()), s=SQRT(S.norm2());
01060     double e=fabs(a-s)/(a*sqrt(double(dim1)*double(dim2)));
01061
01062     if (e>1E-10) { char s_[128];
01063         if (l==0) { X.init2ref(A); } else { A.toMatrixRef(X, I, l, P); }
01064         if (X.isRef()) X.Instantiate();
01065
01066         X.Transpose();
01067         dim2=X.SIZE[0]; dim1=X.SIZE[1]; k=MIN(dim1, dim2);
01068
01069         Vd.init(dim2, k); S.init(k); U.init(k, dim1);
01070
01071         GESVD(X, Vd, S, U); Vd.Transpose(); U.Transpose();

```

```

01072
01073     s=SQRT(S.norm2());
01074     a=fabs(a-s)/(a*sqrt(double(dim1)*double(dim2)));
01075     if (ISCOMPLX_(T))
01076         strcpy(s_,"ZGESVD");
01077     else strcpy(s_,"DGESVD");
01078     snprintf(s_+10,118,"%dx%d @%.3g -> %.2g",dim1,dim2,e,a);
01079
01080     if (a<1E-10)
01081         wblog(FL,"WRN fixed %s bug: %s (ok)", s_,s_+10);
01082     else wblog(FL,"ERR failed to fix %s bug\n%s",s_,s_+10);
01083 }
01084 }
01085 #endif
01086
01087 if (l) {
01088     WBINDEX q,Q; A.SIZE.select(P,Q);
01089     q.init(l+1, Q.data ); q[l]=S.len; U.Reshape(q);
01090     q.init(r-l+1,Q.data+l-1); q[0]=S.len; Vd.Reshape(q);
01091 }
01092
01093 if (bare) { Vd.save2(Vc); }
01094 else {
01095     P.initFirstTo(Vd.SIZE.len-1,Vd.SIZE.len);
01096     Vd.permute(Vc,P);
01097 }
01098 };
01099
01100 #endif
01101

```

5.26 wbarray_blas.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbarray (array class, col-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_BLAS_HH__
00021 #define __WB_ARRAY_BLAS_HH__
00022
00023 // ----- //
00024 // also link wbbblas.hh for wrapper routines to Fortran library
00025 // extern "C" { ... }
00026 // Wb,Sep16,09
00027 // ----- //
00028
00029 namespace Wb {
00030
00031 template<class TA, class TB, class TC>
00032 wbarray<TC>& MatProd(
00033     const wbarray<TA> &A, const wbarray<TB> &B,
00034     wbarray<TC> &C,
00035     char aflag='N', char bflag='N',
00036     TA afac=1., TC cfac=0., char cforce=0
00037 );
00038
00039 template<class T>
00040 void VMatProd(
00041     const wbarray<T> &A0, const wbarray<T> &B0, wbarray<T> &C,
00042     char aflag='N', char bflag='N', const T afac=1., const T cfac=0.,
00043     const char cforce=1
00044 );
00045
00046 template<class T>
00047 void MatVProd(
00048     const wbarray<T> &A, const wbvector<T> &v, wbarray<T> &x,
00049     char aflag='N', char bflag='N', const T afac=1., const T cfac=0.,
00050     const char cforce=1

```

```

00051 );
00052
00053 template<class T> T VMatVprod(
00054 const wbarry<T> &v1, const wbarry<T> &M, const wbarry<T> &v2);
00055
00056 template<class T> T VMatVprod(
00057 const wbvector<T> &v1, const wbarry<T> &M, const wbvector<T> &v2);
00058
00059 template<class TA, class TB, class TC>
00060 void DMatProd(
00061     const wbarry<TA> &A0, const wbarry<TB> &B0, wbarry<TC> &C,
00062     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
00063     const char cforce=1
00064 );
00065
00066 template<class TA, class TB, class TC>
00067 void DMatProd(
00068     const wbvector<TA> &A0, const wbarry<TB> &B0, wbarry<TC> &C,
00069     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
00070     const char cforce=1
00071 ){
00072     wbarry<TA> A; A.init2Vec(A0.data,A0.len,'r');
00073     DMatProd(A,B0,C,aflag,bflag,afac,cfac,cforce);
00074 };
00075
00076 template<class TA, class TB, class TC>
00077 void DMatProd(
00078     const wbarry<TA> &A0, const wbvector<TB> &B0, wbarry<TC> &C,
00079     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
00080     const char cforce=1
00081 ){
00082     wbarry<TB> B; B.init2Vec(B0.data,B0.len,'r');
00083     DMatProd(A0,B,C,aflag,bflag,afac,cfac,cforce);
00084 };
00085
00086 };
00087
00088 template<class T>
00089 wbarry<T>& wbInverse(
00090     const char *F, int L,
00091     const wbarry<T>&M, wbarry<T> &Minv
00092 ){
00093     wblog(FL,"ERR %s() not yet defined for T=%s",FCT,TSTR(T));
00094     return Minv;
00095 }
00096
00097 template<class T>
00098 wbarry<T>& wbInverse(const char *F, int L, wbarry<T>&M);
00099
00100 void wbEigenS(
00101     const wbarry<double>&M,
00102     wbarry<double>&U, wbvector<double> &E
00103 );
00104
00105 void wbEigenS(
00106     const wbarry<wbcomplex>&M,
00107     wbarry<wbcomplex>&U, wbvector<double> &E
00108 );
00109
00110 void wbEigen_CS(
00111     wbarry<wbcomplex> &M,
00112     wbarry<wbcomplex>&U, wbvector<wbcomplex> &E,
00113     char wjob='R', char issym='H', char tnorm=1, char qflag=0
00114 );
00115
00116 template<class T>
00117 void wbEigen_CS_regen(
00118     const wbarry<T> &M0, wbarry<T> &V, wbvector<T> &E,
00119     char wjob
00120 );
00121
00122 #define SVD_BUG_SAFEGUARD
00123
00124 template<class T>
00125 void wbsVD(
00126     const wbarry<T> &A,
00127     wbarry<T> &U, wbvector<double> &S, wbarry<T> &Vc,
00128     const WBINDEX &I=WBINDEX(),
00129     char bare=0
00130 );
00131
00132 template <class TQ>
00133 void wbEigenS (
00134     const wbsparray<double>&M,
00135     wbsparray<double>&U, wbvector<double> &E
00136 ){
00137     wbarry<double> Mf,Uf; M.toFull(Mf);

```

```

00138         wbEigenS(Mf,Uf,E); U.init(FL,Uf,1E-14);
00139     };
00140
00141     template<class T>
00142     class opFlags {
00143     public:
00144
00145         opFlags() : p(0), type(0) {};
00146         opFlags(const opFlags &f) : p(f.p), type(0) {};
00147
00148         opFlags(const char *F, int L, char f)
00149             : p(0), type(0) { init(F,L,f); };
00150
00151         opFlags(const opFlags &f1, const opFlags &f2)
00152             : p(0), type(0) { init(f1,f2); };
00153
00154         opFlags<T>& operator= (char f) { init(f); return *this; }
00155         opFlags<T>& operator= (const opFlags<T> &x) { p=x.p; return *this; }
00156
00157         opFlags(char f) : p(0), type(0) { init(FL,f); };
00158         operator char() { return p; }
00159
00160         opFlags<T>& init(const char *F, int L, char f);
00161         opFlags<T>& init(char f) { return init(0,0,f); };
00162
00163         opFlags<T>& init(const opFlags<T> &f1, const opFlags<T> &f2){
00164             p=(f1.p ^ f2.p); return *this; };
00165
00166         char trans() const { return (p & TRANS_PAT); }
00167         char conj () const { return (p & CONJ_PAT); }
00168         char idle () const { return (!(p & TRANS_PAT) && !(p & CONJ_PAT)); }
00169
00170         void applyOrRef(
00171             const char *F, int L,
00172             const wbararray<T> &A0, wbararray<T> &A, const T fac=1
00173         ) const;
00174
00175         template <class T2>
00176         T2& applyOrRef(const char *F, int L, const T2 &A0, const T fac=1) const;
00177
00178         template <class T2>
00179         const T2& applyConjOrRef(const char *F, int L, const T2 &A0, const T fac=1) const;
00180
00181         template <class T2>
00182         char apply(const char *F, int L, T2 &A, const T fac=1) const;
00183
00184         char toChar() const {
00185             char c, tflag=(p & TRANS_PAT), cflag=(p & CONJ_PAT);
00186             if (tflag)
00187                 c=(cflag ? 'C' : 'T');
00188             else c=(cflag ? 'c' : 'N');
00189             return c;
00190         }
00191
00192         mxArray *toMx() const { return numtoMx(this->toChar()); };
00193
00194         void info(const char *F, int L, const char *istr="") {
00195             wblog(F,L,"<i> opFlags<%s> %s: %d -> [%d %d]",
00196                 TSTR(T), istr && istr[0] ? istr : "",
00197                 p, trans(), conj());
00198         };
00199
00200         char p;
00201
00202     private:
00203
00204         T type;
00205
00206         static char TRANS_PAT;
00207         static char CONJ_PAT;
00208     };
00209
00210     template <class T> char opFlags<T>::TRANS_PAT = 1;
00211     template <class T> char opFlags<T>::CONJ_PAT = 2;
00212
00213     template <class T> inline
00214     opFlags<T>& opFlags<T>::init(
00215         const char *file, int line, char f
00216     ){
00217         p=0;
00218
00219         if (f>16) {
00220             if (!strchr("ENCtC",f)) wblog(__FILE__, "ERR invalid flag %c<%d>",f,f);
00221             if (strchr("CT",f)) p |= TRANS_PAT;
00222             if (f=='E') type=(T)'E';
00223
00224             if (typeid(T)==typeid(wbcomplex))

```

```

00225     if (strchr("Cc",f)) p |= CONJ_PAT;
00226 }
00227 else {
00228     p=f;
00229     if (p & CONJ_PAT && typeid(T)!=typeid(wbcomplex))
00230         p &= ~CONJ_PAT;
00231 }
00232
00233 return *this;
00234 };
00235
00236 template <class T> inline
00237 void opFlags<T>::applyOrRef(
00238     const char *F, int L,
00239     const wbarry<T> &A0, wbarry<T> &A, const T fac
00240 ) const {
00241
00242     unsigned r=A0.SIZE.len;
00243     char p2=p;
00244
00245     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
00246         wblog(F,L,"ERR invalid flags (%d)",p);
00247
00248     if (!r) { A.init(); return; }
00249     if (r>2 && r%2) wblog(F,L,"ERR invalid rank-%d object (odd)",r);
00250
00251     if (r==1) p2 &= ~TRANS_PAT;
00252
00253     if (p2 || fac!=T(1)) {
00254         if (p2 & TRANS_PAT) {
00255             wbp perm P; P.initTranspose(r);
00256             A0.permute(A,P);
00257         } else A=A0;
00258
00259         if (p2 & CONJ_PAT) A.Conj();
00260
00261         if (fac!=T(1)) A*=fac;
00262     }
00263     else A.init2ref(A0);
00264 };
00265
00266 template <class T>
00267 template <class T2> inline
00268 T2& opFlags<T>::applyOrRef(
00269     const char *F, int L, const T2 &A, const T fac
00270 ) const {
00271
00272     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
00273         wblog(F_L,"ERR invalid flags (%d)",p);
00274
00275     if (!p && fac==1) return A;
00276
00277     T2 *a = new T2;
00278     if (p & TRANS_PAT) { A.transpose(F_L,*a); } else { (*a)=A; }
00279     if (p & CONJ_PAT) a->Conj();
00280     if (fac!=1) (*a)*=fac;
00281
00282     return (*a);
00283 };
00284
00285 template <class T>
00286 template <class T2> inline
00287 const T2& opFlags<T>::applyConjOrRef(
00288     const char *F, int L, const T2 &A, const T fac
00289 ) const {
00290
00291     char c=(p & CONJ_PAT);
00292     if (p && (p & ~TRANS_PAT & ~CONJ_PAT))
00293         wblog(F_L,"ERR invalid flags (%d)",p);
00294
00295     if (!c && fac==1) return A;
00296     else {
00297         T2 *a = new T2(A);
00298         if (c) a->Conj();
00299         if (fac!=1) (*a)*=fac;
00300         return (*a);
00301     }
00302 };
00303
00304 template <class T>
00305 template <class T2> inline
00306 char opFlags<T>::apply(
00307     const char *F, int L, T2 &A, const T fac
00308 ) const {
00309
00310     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
00311         wblog(F_L,"ERR invalid flags (%d)",p);

```

```

00312
00313     if (p || fac!=1) {
00314         if (p & TRANS_PAT) A.Transpose(F_L);
00315         if (p & CONJ_PAT) A.Conj();
00316         if (fac!=1) A*=fac;
00317         return 1;
00318     }
00319     return 0;
00320 };
00321
00322 #endif
00323

```

5.27 wbbits.cc

```

00001 #ifndef __WB_BITSET_CC__
00002 #define __WB_BITSET_CC__
00003
00004 // ----- //
00005 namespace Wb {
00006 // ----- //
00007
00008 template <class T>
00009 wstring bitstr(const T &x) { // NB! i is 0-based
00010     Bitset B(sizeof(T)); memcpy(B.data, (void*)&x, B.len);
00011     return B.toStr();
00012 };
00013
00014 wstring Bitset::toStr() const {
00015     wstring s(nbits+len+1, ' ');
00016     unsigned i=0, l=0;
00017     s[l++]='['; s[s.len-2]=']';
00018     for (unsigned k=0; k<len; ++k) {
00019         const char &b=data[k]; char r=1;
00020         for (; r && i<nbits; r<<=1, ++i) { s[l++]=(b&r ? '1' : '-'); }
00021         if (!r && i<nbits) s[l++]='|';
00022     }
00023     if (l>s.len) wblog(FL,
00024         "ERR %s() index out of bounds (%d/%d)", FCT, l, s.len);
00025
00026     return s;
00027 };
00028
00029 mxArray* Bitset::toMx() const {
00030     mxArray *a=mxCreateStruct(1,1);
00031     add2MxStruct(a,0);
00032     return a;
00033 };
00034
00035 mxArray* Bitset::mxCreateStruct(unsigned m, unsigned n) const {
00036     const char *fields[]={"nbits", "bset", "data"};
00037     return mxCreateStructMatrix(m,n,3,fields);
00038 };
00039
00040 void Bitset::add2MxStruct(mxArray *S, unsigned i) const {
00041     mxSetFieldByNumber(S,i,0, numtoMx(nbits));
00042     mxSetFieldByNumber(S,i,1, toStr().toMx());
00043     wvector<unsigned> v(len);
00044     for (unsigned j=0; j<len; ++j) v.data[j]=(unsigned char)data[j];
00045     mxSetFieldByNumber(S,i,2, v.toMx());
00046 };
00047
00048 void Bitset::put(const char *file, int line,
00049     const char *vname, const char *ws
00050 ) const {
00051
00052     mxArray *a=toMx();
00053     int i=mexPutVariable(ws,vname,a);
00054
00055     if (i) wblog(__FL__,
00056         "ERR failed to write variable '%s' (%d)", vname,i);
00057     if (file) wblog(file,line,
00058         "I/O putting variable '%s' to %s", vname,ws);
00059     mxDestroyArray(a);
00060 };
00061
00062 template <class T>
00063 wstring bitset<T>::toStr() const {
00064     unsigned len=sizeof(T);
00065
00066     wstring s(9*len+1, ' ');
00067     unsigned i=1, l=0; T r=1;
00068

```

```

00069     s[l++]='['; s[s.len-2]=' '];
00070     for (; r; r<=1, ++i) {
00071         s[l++]=( data&r ? '1' : '-');
00072         if (i%8==0 && l+2<s.len) { s[l++]='|'; }
00073     }
00074
00075     return s;
00076 };
00077
00078 template <class T>
00079 mxArray* bitset<T>::toMx() const {
00080     mxArray *a=mxCreateStruct(1,1);
00081     add2MxStruct(a,0);
00082     return a;
00083 };
00084
00085 template <class T>
00086 mxArray* bitset<T>::mxCreateStruct(unsigned m, unsigned n) const {
00087     const char *fields[]={"nbits","bset","data"};
00088     return mxCreateStructMatrix(m,n,3,fields);
00089 };
00090
00091 template <class T>
00092 void bitset<T>::add2MxStruct(mxArray *S, unsigned i) const {
00093     mxSetFieldByNumber(S,i,0, numtoMx(8*sizeof(T)));
00094     mxSetFieldByNumber(S,i,1, toStr().toMx());
00095     mxSetFieldByNumber(S,i,2, numtoMx(data));
00096 };
00097
00098 template <class T>
00099 void bitset<T>::put(const char *file, int line,
00100     const char *vname, const char *ws
00101 ) const {
00102
00103     mxArray *a=toMx();
00104     int i=mexPutVariable(ws,vname,a);
00105
00106     if (i) wblog(__FL__,
00107         "ERR failed to write variable '%s' (%d)",vname,i);
00108     if (file) wblog(file,line,
00109         "I/O putting variable '%s' to %s",vname,ws);
00110     mxDestroyArray(a);
00111 };
00112
00113 };
00114
00115 #endif
00116

```

5.28 wbbitsset.hh

```

00001 #ifndef __WB_BITSET_HH__
00002 #define __WB_BITSET_HH__
00003
00004 // ----- //
00005 namespace Wb {
00006 // ----- //
00007
00008 // arbitrary length bitset
00009 class Bitset;
00010
00011 template <class T> class bitset;
00012
00013 template <class T>
00014 void SetBit(T &x, unsigned i) {
00015     unsigned nbits=8*sizeof(T);
00016     if (i>=nbits) wblog(FL,
00017         "ERR %s() bit index out of range (%d/%d)",FCT,i,nbits);
00018     T r=1; r<=i; x|=r;
00019 };
00020
00021 template <class T> inline
00022 T setbit(const T &x0, unsigned i) { T x=x0; SetBit(x,i); return x; };
00023
00024 template <class T>
00025 void UnsetBit(T &x, unsigned i) {
00026     unsigned nbits=8*sizeof(T);
00027     if (i>=nbits) wblog(FL,
00028         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00029     T r=1; r=~(r<i)); x&=r;
00030 };
00031
00032 template <class T> inline

```

```

00033 T unsetbit(const T &x0, unsigned i) { T x=x0; UnsetBit(x,i); return x; };
00034
00035 template <class T>
00036 bool issetbit(const T &x, unsigned i) {
00037     unsigned nbits=8*sizeof(T);
00038     if (i>=nbits) wblog(FL,
00039         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00040     T r=1; r<<=i;
00041     return (x&r ? 1 : 0);
00042 };
00043
00044 template <class T>
00045 wstring bitstr(const T &x);
00046
00047 class Bitset: public wvector<char> {
00048
00049     public:
00050
00051     Bitset(unsigned n=0 ) : nbits(0) { NEW_BITSET(n); };
00052     Bitset(const Bitset &b)
00053     : wvector<char>(), nbits(0) { NEW_BITSET(b.nbits, b.data); };
00054
00055     Bitset& init(unsigned n=0) { return NEW_BITSET(n); };
00056     Bitset& init(unsigned n, const char* b) { return NEW_BITSET(n,b); };
00057
00058     Bitset& setall(bool t) {
00059         memset(data, t ? 255:0, len);
00060         if (t && len) { (data[len-1])>=(8*len-nbits); }
00061         return *this;
00062     };
00063
00064     Bitset& set(unsigned i) {
00065         if (i>=nbits) wblog(FL,
00066             "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00067         unsigned k=i/8; data[k] |= (1<<(i-8*k));
00068         return *this;
00069     };
00070
00071     Bitset& unset(unsigned i) {
00072         if (i>=nbits) wblog(FL,
00073             "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00074         unsigned k=i/8; data[k] &= ~(1<<(i-8*k));
00075         return *this;
00076     };
00077
00078     bool isset(unsigned i) {
00079         if (i>=nbits) wblog(FL,
00080             "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00081         unsigned k=i/8;
00082         return (data[k] & (1<<(i-8*k)) ? 1 : 0);
00083     };
00084
00085     unsigned nnz() const {
00086         unsigned i,n=0;
00087         for (i=0; i<len; ++i) {
00088             const char &b=data[i];
00089             for (char r=1; r; r<<=1) { if (b&r) ++n; }
00090         }
00091         return n;
00092     };
00093
00094     bool operator==(const Bitset &b) const {
00095         if (len!=b.len || nbits!=b.nbits) return 0;
00096         if (data!=b.data) {
00097             for (unsigned i=0; i<len; i++)
00098                 if (data[i]!=b.data[i]) return 0;
00099         }
00100         return 1;
00101     };
00102
00103     bool operator!=(const Bitset &b) const { return !((*this)==b); };
00104
00105     wstring toStr() const;
00106
00107     mxArray* toMx() const;
00108     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00109     void add2MxStruct(mxArray *S, unsigned i) const;
00110
00111     void put(const char *vname, const char *ws="base"
00112     ) const { put(0,0,vname,ws); };
00113
00114     void put(const char *file, int line,
00115         const char *vname, const char *ws="base"
00116     ) const;
00117
00118     unsigned nbits;
00119

```



```

00120     protected:
00121
00122     Bitset& NEW_BITSET(unsigned n, const char* d=NULL) {
00123         nbits=n; if (n) RENEW(((n-1)/8)+1,d);
00124         return *this;
00125     }
00126
00127 };
00128
00129 template <class T>
00130 class bitset {
00131
00132     public:
00133
00134     bitset() : data(0) {};
00135     bitset(const bitset<T> &b) : data(0) { data=b.data; };
00136
00137     bitset& setall(bool t) {
00138         memset(&data, t ? 255:0, sizeof(T));
00139         return *this;
00140     };
00141
00142     bitset& set(unsigned i) {
00143         T r=1; r<=i;
00144         if (r) { data |= r; } else wblog(FL,
00145             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
00146         return *this;
00147     };
00148
00149     bitset& unset(unsigned i) {
00150         T r=1; r<=i;
00151         if (r) { data &=(~r); } else wblog(FL,
00152             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
00153         return *this;
00154     };
00155
00156     bool isset(unsigned i) {
00157         T r=1; r<=i;
00158         if (!r) wblog(FL,
00159             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
00160         return (data&r ? 1 : 0);
00161     };
00162
00163     unsigned nnz() const {
00164         unsigned n=0;
00165         for (T r=1; r; r<=1) { if (data&r) ++n; }
00166         return n;
00167     };
00168
00169     bool operator==(const bitset &b) const { return (data==b.data); };
00170     bool operator!=(const bitset &b) const { return (data!=b.data); };
00171
00172     wbsttring toStr() const;
00173
00174     mxArray* toMx() const;
00175     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00176     void add2MxStruct(mxArray *S, unsigned i) const;
00177
00178     void put(const char *vname, const char *ws="base"
00179 ) const { put(0,0,vname,ws); };
00180
00181     void put(const char *file, int line,
00182         const char *vname, const char *ws="base"
00183 ) const;
00184
00185     T data;
00186
00187     protected:
00188
00189 };
00190
00191 };
00192
00193 #endif
00194

```

5.29 wbbblas.hh

```

00001 #ifndef __WB_BLAS_HH__
00002 #define __WB_BLAS_HH__
00003
00004 // ----- //
00005 // NB! matlab's mwblas/mwlapack have their own header files

```

```

00006 // specifically, starting with matlab-R2009, the LDA type
00007 // (e.g. in DGEMM) is no longer int but ptrdiff_t
00008 // for compatibility reasons with 64bit linux (*@&$)!
00009 //
00010 // trying to avoid matlabs blas.h and lapack.h as it defines
00011 // other troublesome stuff, eg. second as wrapper for fortran _second,
00012 // but the latter is defined/used with maps!)*&#
00013
00014 // #define pINT int          // for matlab <= R2008
00015 // #define pINT ptrdiff_t // for matlab >= R2009
00016 // NB! all int, also info variable, replaced by pINT (!)
00017
00018 extern "C" {
00019
00020 #define dgemm dgemm_
00021 void dgemm (
00022
00023     const char &,
00024     const char &,
00025     const pINT &,
00026     const pINT &,
00027     const pINT &,
00028     const double &,
00029     double [],
00030     const pINT &,
00031     double [],
00032     const pINT &,
00033     const double &,
00034     double [],
00035     const pINT &
00036 );
00037
00038 #define zgemm zgemm_
00039 void zgemm (
00040
00041     const char &,
00042     const char &,
00043     const pINT &,
00044     const pINT &,
00045     const pINT &,
00046     const wbcplx &,
00047     wbcplx [],
00048     const pINT &,
00049     wbcplx [],
00050     const pINT &,
00051     const wbcplx &,
00052     wbcplx [],
00053     const pINT &
00054 );
00055
00056 #define ilaenv ilaenv_
00057
00058 int ilaenv(
00059     const pINT &,
00060     const char *,
00061     const char *,
00062     const pINT &,
00063     const pINT &,
00064     const pINT &,
00065     const pINT &
00066 );
00067
00068 #define dgetri dgetri_
00069 void dgetri(
00070     const pINT &,
00071     double [],
00072     const pINT &,
00073     pINT [],
00074     double [],
00075     const pINT &,
00076     pINT &
00077 );
00078
00079 #define dgetrf dgetrf_
00080 void dgetrf(
00081     const pINT &,
00082     const pINT &,
00083     double [],
00084     const pINT &,
00085     pINT [],
00086     pINT &
00087 );
00088
00089 #define dsyevd dsyevd_
00090
00091 void dsyevd(
00092     const char &,

```

```

00093  const char    &,
00094  const pINT    &,
00095          double [],
00096  const pINT    &,
00097          double [],
00098          double [],
00099  const pINT    &,
00100          pINT   [],
00101  const pINT    &,
00102          pINT   &
00103  );
00104
00105  #define zheev zheev_
00106
00107  void zheev (
00108      const char    &,
00109      const char    &,
00110      const pINT    &,
00111      wbcomplex[],
00112      const pINT    &,
00113      double [],
00114      wbcomplex[], const pINT &,
00115      double [],
00116      pINT   &
00117  );
00118
00119  #define zheevd zheevd_
00120
00121  void zheevd(
00122      const char    &,
00123      const char    &,
00124      const pINT    &,
00125      wbcomplex[],
00126      const pINT    &,
00127      double [],
00128      wbcomplex[], const pINT &,
00129      double [], const pINT &,
00130      pINT   [], const pINT &,
00131      pINT   &
00132  );
00133
00134  #define dgeev dgeev_
00135  void dgeev (
00136
00137      const char    &,
00138      const char    &,
00139      const pINT    &,
00140      double [],
00141      const pINT    &,
00142      double [],
00143      double [],
00144      double [],
00145      const pINT    &,
00146      double [],
00147      const pINT    &,
00148      double [],
00149      const pINT    &,
00150      pINT   &
00151  );
00152
00153  #define zgeev zgeev_
00154  void zgeev (
00155
00156      const char    &,
00157      const char    &,
00158      const pINT    &,
00159      wbcomplex [],
00160      const pINT    &,
00161      wbcomplex [],
00162      wbcomplex [],
00163      const pINT    &,
00164      wbcomplex [],
00165      const pINT    &,
00166      wbcomplex [],
00167      const pINT    &,
00168      double  [],
00169      pINT   &
00170  );
00171
00172  #define dgesvd dgesvd_
00173
00174  void dgesvd (
00175      const char    &,
00176      const char    &,
00177      const pINT    &,
00178      const pINT    &,
00179      double [],

```

```

00180  const pINT    &,
00181          double [],
00182          double [],
00183  const pINT    &,
00184          double [],
00185  const pINT    &,
00186
00187          double [],
00188  const pINT    &,
00189  pINT    &
00190 );
00191
00192 #define dgesdd dgesdd_
00193
00194 void dgesdd (
00195     const char    &,
00196     const pINT    &,
00197     const pINT    &,
00198     double [],
00199     const pINT    &,
00200     double [],
00201     double [],
00202     const pINT    &,
00203     double [],
00204     const pINT    &,
00205
00206     double [],
00207     const pINT    &,
00208     pINT    [],
00209     pINT    &
00210 );
00211
00212 #define zgesvd zgesvd_
00213
00214 void zgesvd (
00215     const char    &,
00216     const char    &,
00217     const pINT    &,
00218     const pINT    &,
00219     wcomplex[],
00220     const pINT    &,
00221     double [],
00222     wcomplex[],
00223     const pINT    &,
00224     wcomplex[],
00225     const pINT    &,
00226     wcomplex[],
00227     const pINT    &,
00228     wcomplex[],
00229     pINT    &
00230 );
00231
00232 #define zgesdd zgesdd_
00233
00234 void zgesdd (
00235     const char    &,
00236     const pINT    &,
00237     const pINT    &,
00238     wcomplex[],
00239     const pINT    &,
00240     double [],
00241     wcomplex[],
00242     const pINT    &,
00243     wcomplex[],
00244     const pINT    &,
00245     wcomplex[],
00246     const pINT    &,
00247     double [],
00248     pINT    [],
00249     pINT    &
00250 );
00251
00252 }
00253
00254 class STAT_DGEMM {
00255 public:
00256     STAT_DGEMM() { nn.init(32); ss.init(32); }
00257     ~STAT_DGEMM() {
00258         unsigned i; double fac=nn.max(); fac=1./(fac!=0 ? fac:1);
00259         if (nn.max()<1) { return; }
00260
00261         wblog(FL,"%NDGEMM statistics");
00262         printf("\n idx_n  avg.dim      count      cost (^2.5)\n\n");
00263         for (i=0; i<nn.len; i++) if (nn[i])
00264             printf(" %3d %8.2f %12d %9g\n", i,
00265                 pow(double(ss[i])/nn[i], 1/3.), nn[i],
00266                 pow(double(ss[i])/nn[i], 2.5/3) * nn[i]*fac);

```

```

00267     printf("\n");
00268 };
00269
00270 void account(unsigned n0) {
00271     unsigned i,n=n0;
00272     for (i=0; i<nn.len; i++) { n*=1; if (n==0) break; }
00273     if (i>=nn.len) i=nn.len-1;
00274     nn[i]++; ss[i]+=n0;
00275 };
00276
00277 wbvector<unsigned> nn;
00278 wbvector<unsigned long> ss;
00279
00280 private:
00281
00282 };
00283
00284 #ifdef WB_CLOCK
00285     STAT_DGEMM stat_dgemm;
00286 #endif
00287
00288 #endif
00289

```

5.30 wbclock.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace clock routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_CLOCK_CC__
00021 #define __WB_CLOCK_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025 // see also $JUL/WbModules::sec2str() // Wb,Sep22,16
00026 // tags: sec2str, int2time, num2time, tstamp
00027
00028 inline wbstring Wb::sec2Str(double t) {
00029     wbstring st(16);
00030     unsigned l;
00031
00032     if (t<100) { l=snprintf(st.data,st.len,"%0.5g",t); } else
00033     if (t<3600) {
00034         double s=fmod(t,60), m=(t-s)/60;
00035         if (::fabs(s-::round(s))<1e-6)
00036             l=snprintf(st.data,st.len,"%02.0f:%02f", m,s); }
00037     else { l=snprintf(st.data,st.len,"%02.0f:%02.3f",m,s); }
00038 }
00039 else {
00040     double s=fmod(t,60);
00041     unsigned q=::round((t-s)/60), m,h,d;
00042
00043     m=q%60; q=(q-m)/60;
00044     h=q%24; d=(q-h)/24;
00045
00046     if (d)
00047         l=snprintf(st.data,st.len,"%d-%02d:%02d:%02.0f",d,h,m,s);
00048     else l=snprintf(st.data,st.len, "%02d:%02d:%02.0f", h,m,s);
00049 }
00050
00051 if (l>=st.len) wblog(FL,"WRN %s() string out of bounds "
00052     "(len=%d/%d)\nstr = %s",FCT,l,st.len,st.data);
00053 return st;
00054 };
00055
00056 void Wb::pause(const char *F, int L, double tsec) {
00057     if (tsec<=0) { return; }

```

```

00058
00059     if (F) wblog(FL, " * %s() for %g secs", FCT, tsec); else
00060     if (tsec>99) wblog(FL, "WRN %s() for %g secs !?", FCT, tsec);
00061
00062     timespec t; double s=floor(tsec), n=floor((tsec-s)*1E9);
00063     t.tv_sec=s; t.tv_nsec=n;
00064
00065     nanosleep(&t,0);
00066 };
00067
00068 void Wb::get_Clock_name(wbstring &name, const char *s, char use_tag) {
00069
00070     if (use_tag) {
00071         unsigned l, n=16+(s ? strlen(s) : 0); char sx[n];
00072         l=snprintf(sx,n,"%12s:",
00073             #ifdef PROG_TAG
00074             PROG_TAG
00075             #elif defined(myname)
00076             myname
00077             #else
00078             "???"
00079             #endif
00080         );
00081         if (s && l<n) {
00082             if (l && !strcmp(sx,s,l))
00083                 { wblog(FL, "WRN %s() ' %s :? %s'", FCT, sx, s); }
00084             else { l+=snprintf(sx+l,n-l,"%s",s); }
00085         }
00086         name=sx;
00087     }
00088     else { name=(s ? s : ""); }
00089 };
00090
00091 Wb::Clock* Wb::ClockSet::insert(Wb::Clock* clk, char lflag) {
00092
00093     if (!clk) { wblog(FL, "ERR %s() got null clock", FCT); }
00094     if (!clk->name) { wblog(FL, "WRN %s() got empty clock name", FCT); }
00095
00096     auto q=buf.insert({ clk->name.data, clk });
00097
00098     if (!q.second && !lflag) wblog(FL,
00099         "ERR %s() got existing clock entry \f'%s'", FCT, clk->name.data);
00100     if (q.first->second!=clk) wblog(FL,
00101         "ERR %s() clk inconsistency %p / %p", FCT, q.first->second, clk);
00102     return q.first->second;
00103 };
00104
00105 int Wb::ClockSet::erase(Wb::Clock* clk) {
00106     if (!clk) { wblog(FL, "ERR %s() got null clock", FCT); }
00107     if (!clk->name) {
00108         wblog(FL, "ERR %s() got empty clock name", FCT);
00109     }
00110
00111     return buf.erase(clk->name.data);
00112 };
00113
00114 Wb::Clock* Wb::ClockSet::use(const char *istr, char mode) {
00115     Wb::Clock* clk;
00116
00117     if (!istr) wblog(FL, "ERR %s() got null clock string", FCT);
00118     if (!istr[0]) wblog(FL, "WRN %s() got empty clock string", FCT);
00119
00120     auto i=buf.find(istr);
00121     if (i!=buf.end()) { clk=i->second; }
00122     else {
00123         clk = new Wb::Clock(istr, mode&2, 0, NULL, NULL, 0);
00124         auto q=buf.insert({ istr, clk });
00125         if (!q.second) wblog(FL, "WRN %s() failed to insert clock", FCT);
00126     }
00127
00128     if (mode>1) {
00129         if (mode&1) { clk->resume(); }
00130         if (mode>3) wblog(FL, "WRN %s() got mode = %s", FCT, cSTR(mode));
00131     }
00132     return clk;
00133 };
00134
00135 void Wb::ClockSet::init(char vflag) {
00136     unsigned n=buf.size();
00137
00138     if (vflag>3) {
00139         if (vflag=='v') { vflag=1; } else
00140         if (vflag=='V') { vflag=2; } else
00141         if (vflag=='q') { vflag=0; } else
00142         wblog(FL, "WRN %s() got vflag=%s", FCT, cSTR(vflag));
00143     }
00144

```

```

00145     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00146         if (!vflag) { i->second->init(); }
00147         delete i->second;
00148     }
00149     if (n) { PRINTF("\n"); }
00150 };
00151
00152 unsigned Wb::ClockSet::info(char vflag) const {
00153     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00154         i->second->info(NULL,vflag);
00155     }
00156     return buf.size();
00157 };
00158
00159 int Wb::ClockSet::info_u(unsigned u, char rflag) {
00160     unsigned m=0;
00161
00162     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00163         Wb::Clock &clk = *(i->second);
00164         if ((u && (clk.user & u)) || u==clk.user) { ++m;
00165             clk.info(); if (rflag) {
00166                 clk.reset(); }
00167         }
00168     }
00169     return m;
00170 };
00171
00172 int Wb::ClockSet::reset_u(unsigned u, char iflag) {
00173     unsigned m=0;
00174
00175     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00176         Wb::Clock &clk = *(i->second);
00177         if ((u && (clk.user & u)) || u==clk.user) { ++m;
00178             if (iflag) { clk.info(); }
00179             clk.reset();
00180         }
00181     }
00182     return m;
00183 };
00184
00185 Wb::Clock* Wb::ClockSet::get(const char *istr, char use_tag) {
00186     wstring s;
00187     get_Clock_name(s,istr,use_tag);
00188
00189     auto i=buf.find(s.data);
00190     return ( i!=buf.end() ? i->second : NULL);
00191 };
00192
00193 int Wb::ClockSet::reset(const char *istr, char use_tag) {
00194
00195     int q=0;
00196     Wb::Clock *clk=get(istr,use_tag);
00197     if (clk) { clk->reset(); ++q; }
00198     return q;
00199 };
00200
00201 mxArray* Wb::ClockSet::toMx() const {
00202     unsigned i=0, n=buf.size();
00203     mxArray* C=mxCreateCellMatrix(n,3);
00204
00205     for (auto it=buf.begin(); it!=buf.end(); ++it, ++i) {
00206         const Wb::Clock &ci=*(it->second);
00207         mxSetCell(C, i, ci.name.toMx());
00208         mxSetCell(C, n+i, numtoMx(ci.gettime('c')));
00209         mxSetCell(C, 2*n+i, numtoMx(ci.gettime( )));
00210     }
00211     return C;
00212 };
00213
00214 mxArray* Wb::Clock::toMxS(const char *F, int L) const {
00215     return MXPut(F,L)
00216         .add(name,"istr")
00217         .add(sec2Str(gettime('c')),"cpu")
00218         .add(sec2Str(gettime( )), "wall")
00219         .toMx();
00220 };
00221
00222 mxArray* Wb::Clock::toMx(const char *F, int L) const {
00223     return MXPut(F,L).add(name,"istr")
00224         .addP(numtoMx(gettime('c')),"cpu")
00225         .addP(numtoMx(gettime( )), "wall")
00226         .toMx();
00227 };
00228
00229 void Wb::save_and_clear_Profiling() {
00230
00231     mxArray *S=mxCreateStructMatrix(1,1,0,NULL);

```

```

00232
00233     for (auto I=gwb_Profs.begin(); I!=gwb_Profs.end(); ++I) {
00234         unsigned j=0, n=I->second.size(); double x[4];
00235         WbMatrix<double> M(n,4);
00236         for (auto J=I->second.begin(); J!=I->second.end(); ++J, ++j) {
00237             x[0]=J->first;
00238             x[1]=J->second.ncall;
00239             x[2]=J->second.dt;
00240             x[3]=J->second.aux; if (x[1]>1) { x[2]/=x[1]; x[3]/=x[1]; }
00241             M.recSetP(j,x);
00242         }
00243         mxAddField2Scalar(FL,S,I->first.c_str(),M.SortRecs().toMx());
00244     }
00245
00246     unsigned i=0, l=0,n=128; char fout[n]; fout[0]=fout[n-1]=0;
00247     const char *s;
00248
00249     s=getenv("Wb_PROFILE_FOUT");
00250     if (s && s[0])
00251         { l=snprintf(fout,n,"%s",s); }
00252     else {
00253         const mxArray *a=mexGetVariablePtr("caller","profile_fout");
00254         if (a && mxGetString(a,fout,n-1)) { l=-1; }
00255     }
00256     if (l>=n) { fout[n-1]=0; wblog(FL,
00257         "WRN %s() string out of bounds (%d/%d) !?\ncheck %s",FCT,l,n,
00258         int(n)>0? "variable `profile_fout`":"env `WB_PROFILE_FOUT`");
00259     fprintf(stdout,"\n%s (%ld)\n",fout,strlen(fout)); fout[0]=0;
00260 }
00261
00262     if (fout[0]) { Wb::matFile F;
00263         if (F.open(FL,fout,"w",0)) {
00264             for (n=mxGetNumberOfFields(S), i=0; i<n; ++i) {
00265                 F.put(FL,
00266                     mxGetFieldNameByNumber(S,i),
00267                     mxGetFieldByNumber(S,0,i)
00268                 );
00269             }
00270             F.close();
00271         }
00272         else {
00273             wblog(FL,"WRN %s() invalid file name\n%s",FCT,fout);
00274             fout[0]=0;
00275         }
00276     }
00277
00278     static int ncall=0;
00279
00280     if (1 || ++ncall==1) { strcpy(fout,"Iprof"); }
00281     else { sprintf(fout,"Iprof%02d",ncall); }
00282
00283     mxPutAndDestroy(FL,S,fout,"base");
00284 };
00285
00286 #endif
00287

```

5.31 wbclock.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace clock routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_CLOCK_HH__
00021 #define __WB_CLOCK_HH__
00022
00023 #include <set>
00024

```



```

00025 namespace Wb {
00026     void pause(const char *F, int L, double tsec);
00027     void pause(double tsec) { pause(0,0,tsec); }
00028     wstring sec2Str(double t);
00029 };
00030
00031 #define CHRONO_NOW std::chrono::high_resolution_clock::now
00032
00033 #define CHRONO_CLOCK std::chrono::time_point<std::chrono::high_resolution_clock>
00034 #define CHRONO_DIFF2SEC(A) (1E-9 * \
00035 double(std::chrono::duration_cast<std::chrono::nanoseconds>(A).count()))
00036
00037 #define WBCLK_USE_TAG 1
00038
00039 namespace Wb {
00040
00041 void get_Clock_name(wstring &name, const char *s, char use_tag);
00042
00043 class ClockSet {
00044 public:
00045     ~ClockSet() { init(); }
00046
00047     void init(char vflag=1);
00048
00049     int info_u(unsigned u, char rflag=1);
00050     int reset_u(unsigned u, char iflag=0);
00051
00052     int reset(const char *istr, char uset_tag=1);
00053
00054     unsigned info(char vflag=0) const;
00055
00056     Clock* use(const char *istr, char mode);
00057
00058     Clock* use(const char *F, int L, char mode) {
00059         return use(shortFL(F,L),mode);
00060     };
00061
00062     Clock* get(const char *s, char use_tag=1);
00063
00064     Clock* insert(Wb::Clock* clk, char lflag=0);
00065
00066     int erase(Wb::Clock* clk);
00067
00068     mxArray* toMx() const;
00069
00070     map<std::string, Wb::Clock*> buf;
00071
00072 protected:
00073 private:
00074 };
00075
00076 ClockSet Clocks;
00077
00078 class Clock {
00079 public:
00080     Clock()
00081         : ttot(0), ctot(0), tneg(0), tref(), cref(0), nz(0), ncall(0),
00082           clk_flags(0), user(0), gcs(NULL) {};
00083
00084     Clock(const char *F, int L, char use_tag=1, unsigned u=0)
00085         : Clock(L>0? shortFL(F,L) : F, L>0 ? use_tag : L, u, &Clocks) { };
00086
00087     Clock(const char *istr, char use_tag=0)
00088         : Clock(istr,use_tag,0,&Clocks) { };
00089
00090     Clock(const char *istr, char use_tag, unsigned u,
00091           ClockSet *gcs=NULL,
00092           Clock **g=NULL,
00093           char rflag=1
00094     ) : Clock() { user=u;
00095         if (istr) {
00096             if (use_tag) { clk_flags|=WBCLK_USE_TAG; }
00097
00098             init(istr,use_tag); if (rflag) { resume(); }
00099             gcs=gcs_; if (!gcs && g) { gcs=&Wb::Clocks; }
00100             if (gcs) {
00101                 Clock *x=gcs->use(name.data,0);
00102                 if (g) { (*g)=x; }
00103             }
00104         }
00105         else if (g || gcs) wblog(FL,"ERR %s() invalid usage",FCT);
00106     };
00107
00108 };
00109
00110 };
00111

```

```

00112 ~Clock() { done(); }
00113
00114 void done() {
00115     if (ncall) { stop(); }
00116     if (gcs) {
00117         Clock *g = gcs->use(name.data,0);
00118         g->Add(*this);
00119     }
00120     else if (WBLOG_CLK) { info(); }
00121     init();
00122 }
00123
00124 void init() {
00125     ttot=ctot=tneg=0; tref=CHRONO_CLOCK();
00126     nz=ncall=clk_flags=user=0; cref=0; gcs=NULL;
00127     name.init();
00128 };
00129
00130 void reset() {
00131     ttot=ctot=tneg=0; tref=CHRONO_CLOCK();
00132     nz=ncall=0; cref=0;
00133 };
00134
00135 void init(const char *s, char use_tag=1) {
00136     reset();
00137     get_Clock_name(name,s,use_tag);
00138 };
00139
00140 void start() {
00141     cref=clock(); ctot=ttot=tneg=0; nz=0; ncall=1;
00142     tref=CHRONO_NOW();
00143 };
00144
00145 void resume() {
00146     if (!ncall) { start(); } else
00147     if ((ncall%2)==0) {
00148         cref=clock(); ++ncall;
00149         tref=CHRONO_NOW();
00150     }
00151 };
00152
00153 int stop(const char *F=NULL, int L=0) {
00154     if (ncall%2) { double tw,tc; ++ncall;
00155         ttot+=(tw=get_dt_wall());
00156         ctot+=(tc=double(clock()-cref)/CLOCKS_PER_SEC);
00157
00158         if (tw<=0 || tc<=0) { ++nz; if (tw) tneg-=tw; }
00159         return 1;
00160     }
00161     else {
00162         if (!ncall) wblog(F,L,
00163             "WRN clock '%s' not yet started",name.data? name.data:"");
00164         return 0;
00165     }
00166 };
00167
00168 double gettime(char cflag=0) const;
00169
00170 void Add(const Clock &t) {
00171     if (ncall%2) { wblog(FL,"WRN %s() clock still running",FCT); }
00172     ttot += t.ttot;
00173     ctot += t.ctot; ncall+=2;
00174 };
00175
00176 int Switch(const char *istr, Clock **g=NULL, char rflag=1) {
00177     if (!name.data || !gcs) wblog(FL,
00178         "ERR %s() invalid usage (not yet initialized)",FCT);
00179     int q=stop(); Clock *x;
00180
00181     reset();
00182
00183     init(istr,clk_flags&1); if (rflag) { resume(); }
00184
00185     x=gcs->use(name.data,0);
00186     if (g) { (*g)=x; }
00187
00188     return q;
00189 };
00190
00191 mxArray* toMx (const char *F=NULL, int L=0) const;
00192 mxArray* toMxS(const char *F=NULL, int L=0) const;
00193
00194 void info(const char *istr=NULL, char vflag=0) const;
00195
00196 double ttot, ctot, tneg;
00197
00198 CHRONO_CLOCK tref;

```

```

00199     clock_t   cref;
00200
00201     size_t   nz;
00202
00203     size_t   ncall;
00204
00205     wstring   name;
00206
00207     unsigned clk_flags;
00208
00209     unsigned user;
00210
00211     ClockSet *gcs;
00212
00213 protected:
00214 private:
00215
00216     double get_dt_wall() const {
00217         return ( CHRONO_DIFF2SEC( CHRONO_NOW() - tref ) );
00218     };
00219
00220     double getTickFreq() const;
00221 };
00222
00223 };
00224
00225 double Wb::Clock::gettime(char cflag) const {
00226
00227     double dt=0;
00228
00229     if (cflag) {
00230         dt=ctot; if (ncall%2) {
00231             dt+=double(clock()-cref)/CLOCKS_PER_SEC; }
00232         }
00233     else {
00234         dt=ttot; if (ncall%2) {
00235             dt+=get_dt_wall(); }
00236         }
00237
00238     return dt;
00239 };
00240
00241 void Wb::Clock::info(const char *istr, char vflag) const {
00242
00243     static int first_call=1;
00244
00245     if (vflag>8) {
00246         if (vflag=='v') { vflag=1; } else
00247         if (vflag=='V') { vflag=2; } else
00248             wblog(FL,"WRN %s() unexpected vflag=%s",FCT,cSTR(vflag));
00249     }
00250     if (!ncall && vflag<2) { return; }
00251
00252     unsigned l=0, n=16; char s[n];
00253     double tc=gettime('c'), tw=gettime();
00254
00255     if (nz) {
00256         double p0=1-nz/(double)(ncall/2);
00257         l=snprintf(s,16,"%5.1f%%", 100*p0);
00258     } else s[0]=0;
00259
00260     if (clk_flags && l<n) {
00261         l+=snprintf(s+l,n-l,"%d", clk_flags); }
00262
00263     if (first_call) { first_call=0;
00264         PRINTF("\n    %-28s Count    CPU-time    Wall-time        / call\n",
00265             "Wb::Clocks");
00266     }
00267
00268     PRINTF(" %c %-26s%8ld %10s %10s %9.3g %s\n",
00269         (ncall%2) ? '*' : ' ', istr? istr:name.data, ncall/2,
00270         sec2Str(tc).data, sec2Str(tw).data,
00271         tw/(ncall/2), s
00272     );
00273 };
00274
00275 double Wb::Clock::getTickFreq() const {
00276
00277     unsigned i,j,m=512,n=32;
00278     clock_t c1,c2,nc=0;
00279
00280     CHRONO_CLOCK t1,t2;
00281     double dt=0;
00282
00283     for (i=j=0; i<n; ++i) {
00284         t1=CHRONO_NOW(); c1=clock(); while (c1==clock()) { if (++j>m) break; };
00285         t2=CHRONO_NOW(); c2=clock();

```

```

00286
00287     if (i) {
00288         nc+=(c2-c1);
00289         dt+=CHRONO_DIFF2SEC( t2-t1 );
00290     }
00291 }
00292
00293 return (dt/nc);
00294 };
00295
00296 namespace Wb {
00297     class Profile;
00298
00299     class line_stat__ {
00300     public:
00301         line_stat__() : dt(0), aux(0), ncall(0) {};
00302
00303         void add(double dt_, double dx) { dt+=dt_; aux+=dx; ++ncall; };
00304
00305         double dt, aux;
00306         size_t ncall;
00307
00308     protected:
00309     private:
00310     };
00311
00312     void save_and_clear_Profiling();
00313 };
00314
00315 map< std::string, map<int, Wb::line_stat__ > > gwb_Profs;
00316
00317 namespace Wb {
00318     class Profile {
00319     public:
00320
00321         Profile(const char *F, int L, const char *istr=0) {
00322             unsigned i=0, l=0, n=32; char fl[32];
00323             if (!F || !F[0] || !L) wblog(FL,"ERR %s() invalid empty F !?",FCT);
00324
00325             for (; F[i]; ++i) { if (F[i]=='/') { l=i+1; } }
00326             for (i=0; F[l] && i<n; ++l, ++i) {
00327                 if (isalpha(F[l])) { fl[i]=F[l]; } else
00328                     if ((F[l]=='.' || F[l]=='_') && i) { fl[i]='_'; } else { fl[i]='X'; }
00329             }; l=i;
00330
00331             if (l<n) { l+=snprintf(fl+l,n-l,"%03d",L); }
00332             if (l<n && istr && istr[0]) { l+=snprintf(fl+l,n-l,"%s",istr); }
00333             if (l>n) { fl[n-l]=0; wblog(FL,
00334                 "ERR %s() string out of bounds (%d/%d)\n%s",FCT,l,n,fl);
00335             }
00336             std::string fl_=fl;
00337
00338             #pragma omp critical (gwb_Profs__)
00339             { auto im=gwb_Profs.find(fl_);
00340               if (im==gwb_Profs.end()) {
00341                 gwb_Profs[fl_]=map<int, Wb::line_stat__ >();
00342                 if ((im=gwb_Profs.find(fl_))==gwb_Profs.end())
00343                     { wblog(FL,"ERR %s() map entry failed !?",FCT); }
00344               }
00345               M=&(im->second);
00346             }
00347             tic();
00348         };
00349
00350         void tic() { tref=CHRONO_NOW(); };
00351         void toc(int L, double x=0) {
00352             double dt=CHRONO_DIFF2SEC( CHRONO_NOW() - tref );
00353
00354             #pragma omp critical (gwb_ProfL__)
00355             { auto im=M->find(L);
00356               if (im==M->end()) { (*M)[L]=Wb::line_stat__();
00357                 if ((im=M->find(L))==M->end())
00358                     { wblog(FL,"ERR %s() map entry failed !?",FCT); }
00359               }
00360               im->second.add(dt,x);
00361             }
00362         };
00363
00364         CHRONO_CLOCK tref;
00365
00366     protected:
00367     private:
00368
00369         map<int, Wb::line_stat__> *M;
00370     };
00371
00372 };

```

```

00373
00374 #endif
00375

```

5.32 wbcomplex.cc

```

00001 #ifndef __WB_COMPLEX_CC__
00002 #define __WB_COMPLEX_CC__
00003
00004 /* ----- */
00005 /* ----- */
00006 // complex number class
00007 // Wb,Dec20,01
00008
00009 wbstring wbcomplex::toStr(const char *fmt) const {
00010     wbstring zstr(32), zfmt(16);
00011     unsigned k,n;
00012
00013     double in=abs(), rn=0;
00014     if (in) { rn=r/in; in=i/in;
00015         if (std::fabs(rn)<1E-12) rn=0;
00016         if (std::fabs(in)<1E-12) in=0;
00017     }
00018     else { in=i; rn=i; }
00019
00020     if (!rn) {
00021         if (in) {
00022             zfmt.printf(FL,"%si",fmt);
00023             zstr.printf(FL,zfmt.data,i); return zstr;
00024         }
00025         else {
00026             zstr.printf(FL,fmt,r); return zstr;
00027         }
00028     }
00029     else if (!in) { zstr.printf(FL,fmt,r); return zstr; }
00030
00031     for (n=strlen(fmt), k=0; k<n; ++k) { if (fmt[k]=='+' ) break; }
00032     if (k<n) {
00033         zfmt.printf(FL,"%s%s",fmt,fmt);
00034     }
00035     else {
00036         zfmt.printf(FL,"%s+%s",fmt,fmt);
00037         zfmt[n]='%'; zfmt[n+1]='+';
00038     }
00039
00040     zstr.printf(FL,zfmt.data,r,i);
00041     return zstr;
00042 };
00043
00044 inline wbstring toStr(const wbcomplex &z, const char* fmt) {
00045     return z.toStr(fmt);
00046 }
00047
00048 inline wbstring toStr(const double &d, const char* fmt) {
00049
00050     size_t l=0, n=32; char s[n];
00051     l=snprintf(s,n,fmt,d);
00052     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00053     return s;
00054 };
00055
00056 #endif
00057

```

5.33 wbcomplex.hh

```

00001 #ifndef __WB_COMPLEX_HCC__
00002 #define __WB_COMPLEX_HCC__
00003
00004 class wbcomplex;
00005
00006 /* ----- */
00007 /* ----- */
00008 // complex number class
00009 // Wb,Dec20,01
00010
00011 class wbcomplex {
00012
00013     public:

```

```

00014
00015     wbcomplex() : r(0.), i(0.) {};
00016
00017     wbcomplex(const wbcomplex &z) = default;
00018
00019     wbcomplex(double r0, double i0=0.) : r(r0), i(i0) {};
00020
00021     ~wbcomplex() = default;
00022
00023     wbcomplex& operator= (const wbcomplex &z) = default;
00024
00025     wbcomplex& operator= (const double &dbl) {
00026         r=dbl; i=0.; return *this;
00027     };
00028
00029     explicit operator bool() const { return (r || i); };
00030     bool operator! () const { return (!r && !i); };
00031
00032     operator double() const {
00033         if (i!=0) wblog(FL,
00034             "ERR got double(%g%+.4gi) type conversion !?",r,i);
00035         return r;
00036     };
00037
00038     wbcomplex& init(double r0, double i0) { r=r0; i=i0; return *this; };
00039     wbcomplex& set(double r0, double i0) { r=r0; i=i0; return *this; };
00040
00041     wbcomplex& setRand(double fac=2., double shift=-1.);
00042
00043     wbcomplex& init_expi(double ph) {
00044         r=std::cos(ph); i=std::sin(ph);
00045         return *this;
00046     };
00047
00048     mxComplexDouble& explicit_copy2(mxComplexDouble &zm) const {
00049         zm.real=r; zm.imag=i;
00050         return zm;
00051     };
00052
00053     bool isinf() const { return (std::isinf(r) || std::isinf(i)); };
00054     bool isnan() const { return (std::isnan(r) || std::isnan(i)); };
00055
00056     bool isfinite() const { return (std::isfinite(r) && std::isfinite(i)); };
00057     bool isnormal() const { return (std::isnormal(r) && std::isnormal(i)); };
00058
00059     bool operator== (const wbcomplex &z) const {
00060         return (float(r)==float(z.r) && float(i)==float(z.i));
00061     };
00062     bool operator!= (const wbcomplex &z) const {
00063         return (float(r)!=float(z.r) || float(i)!=float(z.i));
00064     };
00065
00066     bool operator== (const double &dbl) const {
00067         return (i==0. && float(r)==float(dbl)); };
00068     bool operator!= (const double &dbl) const {
00069         return (i!=0. || float(r)!=float(dbl)); };
00070
00071     bool operator> (const wbcomplex &z) const {
00072         return (r>z.r || (r==z.r && i>z.i) ? 1 : 0); };
00073
00074     bool operator>= (const wbcomplex &z) const {
00075         return (r>z.r || (r==z.r && i>=z.i) ? 1 : 0); };
00076
00077     bool operator< (const wbcomplex &z) const {
00078         return (r<z.r || (r==z.r && i<z.i) ? 1 : 0); };
00079
00080     bool operator<= (const wbcomplex &z) const {
00081         return (r<z.r || (r==z.r && i<=z.i) ? 1 : 0); };
00082
00083     wbcomplex& operator+= (const wbcomplex &z) {
00084         r+=z.r; i+=z.i; return *this; };
00085     wbcomplex& operator-= (const wbcomplex &z) {
00086         r-=z.r; i-=z.i; return *this; };
00087     wbcomplex& operator*= (const wbcomplex &z) {
00088         double r_;
00089         r_ = r*z.r - i*z.i;
00090         i = r*z.i + i*z.r; r=r_; return *this;
00091     };
00092     wbcomplex& operator/= (const wbcomplex &z);
00093
00094     wbcomplex& operator*= (double dbl) { r*=dbl; i*=dbl; return *this; };
00095     wbcomplex& operator+= (double dbl) { r+=dbl; return *this; };
00096     wbcomplex& operator-= (double dbl) { r-=dbl; return *this; };
00097
00098     wbcomplex operator+ (const wbcomplex &z) const {
00099         return wbcomplex(r+z.r, i+z.i); };
00100     wbcomplex operator- (const wbcomplex &z) const {

```

```

00101     return wbcomplex(r-z.r, i-z.i); };
00102     wbcomplex operator* (const wbcomplex &z) const {
00103         return wbcomplex( r*z.r-i*z.i, i*z.r+r*z.i); };
00104     wbcomplex operator/ (const wbcomplex &z) const;
00105
00106     wbcomplex operator+ (const double &dbl) const {
00107         return wbcomplex(r+dbl,i); };
00108     wbcomplex operator- (const double &dbl) const {
00109         return wbcomplex(r-dbl,i); };
00110     wbcomplex operator* (const double &dbl) const {
00111         return wbcomplex(r+dbl, i+dbl); };
00112     wbcomplex operator/ (const double &dbl) const {
00113         if (dbl==0.) wblog(FL, "ERR DIV/0! (%g+%gi)/%g", r, i, dbl);
00114         return wbcomplex(r/dbl, i/dbl);
00115     };
00116
00117     wbcomplex operator/= (double dbl) {
00118         if (dbl==0.) wblog(FL, "ERR DIV/0! (%g+%gi)/%g", r, i, dbl);
00119         r/=dbl; i/=dbl; return *this;
00120     };
00121
00122     wbcomplex times(const wbcomplex &) const;
00123
00124     wbcomplex ConjProd(const wbcomplex &z) const {
00125         return wbcomplex(r+z.r+i*z.i, -i*z.r+r*z.i);
00126     };
00127
00128     void Conj() { i=-i; };
00129     wbcomplex conj() const { return wbcomplex(r,-i); };
00130
00131     double abs2() const { return double(r*r+i*i); };
00132     double abs() const;
00133
00134     wbcomplex sqrt() const;
00135
00136     wbcomplex sign() const {
00137         wbcomplex z;
00138         double a=abs(); if (a) { z.init(r/a, i/a); }
00139         return z;
00140     };
00141
00142     double arg() const;
00143     double arg_degree() { return (180./M_PI)*arg(); }
00144
00145     wbcomplex round() const {
00146         return wbcomplex( ::round(r), ::round(i) );
00147     };
00148
00149     wstring toStr(const char *fmt="%.4g") const;
00150     void put (const char *vname="ans", const char *ws="base") const;
00151
00152     double r,i;
00153
00154     protected:
00155
00156     private:
00157 };
00158
00159 inline wbcomplex wbcomplex::operator/ (const wbcomplex &b) const {
00160     double x, den;
00161
00162     if (fabs(b.r)>=fabs(b.i)) {
00163         if (b.r==0) { wblog(FL, "WRN DIV/0"); return wbcomplex(Inf,Inf); }
00164         x=b.i/b.r;
00165         den = b.r + x*b.i;
00166         return wbcomplex((r+x*i)/den, (i-x*r)/den);
00167     }
00168     else {
00169         x=b.r/b.i;
00170         den = b.i + x*b.r;
00171
00172         return wbcomplex((x*r+i)/den, (x*i-r)/den);
00173     }
00174 };
00175
00176 inline wbcomplex& wbcomplex::operator/= (const wbcomplex &z) {
00177
00178     double r_, dbl=1./z.abs2();
00179     if (z==0.) wblog(FL, "ERR DIV/0! (%g+i%g)/(%g+i%g)", r, i, z.r, z.i);
00180
00181     r_ = (r * z.r + i * z.i) * dbl;
00182     i = (i * z.r - r * z.i) * dbl; r = r_; return *this;
00183 };
00184
00185 inline wbcomplex wbcomplex::times(const wbcomplex &z) const {
00186     double a,b; a=r*z.r; b=i*z.i;
00187     wbcomplex zz(a-b, (r+i)*(z.r+z.i)-a-b);

```

```

00188     return zz;
00189 };
00190
00191 inline wbcomplex& wbcomplex::setRand(double fac, double shift) {
00192     static char first_call=1;
00193
00194     if (first_call) { wb_srand(); first_call=0; }
00195     fac/=(double)RAND_MAX;
00196
00197     if (shift==0.)
00198         { r=fac*rand(); i=fac*rand(); }
00199     else { r=fac*rand()+shift; i=fac*rand()+shift; }
00200
00201     return *this;
00202 };
00203
00204 inline double wbcomplex::abs() const {
00205     double a=0.;
00206     double x=fabs(r), y=fabs(i);
00207
00208     if (!x) { a=y; } else
00209     if (!y) { a=x; } else if (x>y)
00210         { a=y/x; a = x*std::sqrt(1.+a*a); }
00211     else { a=x/y; a = y*std::sqrt(1.+a*a); }
00212
00213     return a;
00214 };
00215
00216 inline wbcomplex wbcomplex::sqrt() const {
00217     wbcomplex z;
00218     if (!r && !i) { return z; }
00219
00220     double ar=fabs(r), ai=fabs(i), w, x;
00221     if (ar>=ai)
00222         { x=ai/ar; w=std::sqrt(ar)*std::sqrt(0.5*(1.+std::sqrt(1+x*x))); }
00223     else { x=ar/ai; w=std::sqrt(ai)*std::sqrt(0.5*(x +std::sqrt(1+x*x))); }
00224
00225     if (r>=0.) { z.init( w, i/(2.*w) ); } else
00226     if (i>=0.) { z.init( i/(2.*w), w ); } else { z.init(-i/(2.*w),-w ); }
00227
00228     return z;
00229 };
00230
00231 inline double wbcomplex::arg() const {
00232     double phi=0.;
00233
00234     if (!i) { if (r<0) { phi+=M_PI; }; } else
00235     if (!r) { phi=( i>0? +0.5 : -0.5 )*M_PI; }
00236     else {
00237         phi=atan(i/r);
00238         if (r<0) { phi+=(i>0 ? M_PI : -M_PI); }
00239     }
00240     return phi;
00241 };
00242
00243 void wbcomplex::put(const char *vname, const char *ws) const {
00244
00245     mxArray *a=NULL;
00246     if (i)
00247         { a=Mx::Array<wbcomplex>(1,1).copyFromTR(this); }
00248     else { a=Mx::Array<double>(1,1).copyFromTR(&r ); }
00249
00250     int e=mexPutVariable(ws,vname,a);
00251     if (e==1) wblog(FL,
00252         "ERR failed to put variable %s to workspace '%s'",vname,ws);
00253
00254     mxDestroyArray(a);
00255 };
00256
00257 inline wbcomplex operator+(const wbcomplex &z) { return z; }
00258 inline wbcomplex operator-(const wbcomplex &z) {
00259     return wbcomplex(-z.r, -z.i);
00260 };
00261 inline wbcomplex operator*(double d, const wbcomplex &z) {
00262     return wbcomplex(z.r*d, z.i*d);
00263 };
00264 inline wbcomplex operator+(double d, const wbcomplex &z) {
00265     return wbcomplex(d+z.r, +z.i);
00266 };
00267 inline wbcomplex operator-(double d, const wbcomplex &z) {
00268     return wbcomplex(d-z.r, -z.i);
00269 };
00270 inline wbcomplex operator/(double d, const wbcomplex &z) {
00271     return wbcomplex(d,0)/z;
00272 };
00273
00274 inline bool isnan(const wbcomplex &z) { return z.isnan(); };

```



```

00275
00276 inline bool isfinite(const wbcomplex &z) { return z.isfinite(); };
00277 inline bool isnormal(const wbcomplex &z) { return z.isnormal(); };
00278
00279 template <class T> inline T real(const T &d) { return d; };
00280 template <class T> inline T imag(const T &d) { return 0; };
00281 template <class T> inline T conj(const T &d) { return d; };
00282
00283 template <class T> inline T CONJ(const T &d) { return d; };
00284 template <class T> inline T ABS (const T &a) { return (a<0 ? -a : a); };
00285 template <class T> inline T ABS2(const T &d) { return d*d; };
00286 template <class T> inline T NORM (const T &a){ return (a<0 ? -a : a); };
00287 template <class T> inline T NORM2(const T &d){ return d*d; };
00288
00289 inline double real(const wbcomplex &z) { return z.r; };
00290 inline double imag(const wbcomplex &z) { return z.i; };
00291 inline wbcomplex conj(const wbcomplex &z) {
00292     return wbcomplex(z.r,-z.i);
00293 };
00294
00295 inline double ROUND(const double &d) { return ::round(d); }
00296 inline wbcomplex ROUND(const wbcomplex &d) { return d.round(); }
00297
00298 #ifdef __WB_MPFR_HH__
00299 template <unsigned P> inline
00300 Wb::mpfr__<P> ROUND(const Wb::mpfr__<P> &q) { return q.round(); };
00301 #endif
00302
00303 template <class T> inline
00304 T Sqrt(const T &d) { return std::sqrt(d); }
00305
00306 template <> inline
00307 wbcomplex Sqrt(const wbcomplex &d) { return d.sqrt(); }
00308
00309 #ifdef __WB_MPFR_HH__
00310 template <unsigned P> inline
00311 Wb::mpfr__<P> Sqrt(const Wb::mpfr__<P> &q) { return q.sqrt(); };
00312 #endif
00313
00314 template <> inline
00315 double ABS(const double &a) { return std::fabs(a); };
00316 template <> inline
00317 double NORM(const double &a) { return std::fabs(a); };
00318
00319 template <> inline
00320 wbcomplex CONJ(const wbcomplex &a) { return a.conj(); };
00321
00322 inline double ABS (const wbcomplex &a) { return a.abs(); };
00323 inline double ABS2 (const wbcomplex &a) { return a.abs2(); };
00324 inline double NORM (const wbcomplex &a) { return a.abs(); };
00325 inline double NORM2(const wbcomplex &a) { return a.abs2(); };
00326
00327 inline bool ISINF(const double &a) { return std::isinf(a); }
00328 inline bool ISINF(const wbcomplex &a) {
00329     return (std::isinf(a.r) || std::isinf(a.i)); }
00330
00331 inline bool ISNAN(const double &a) { return std::isnan(a); }
00332 inline bool ISNAN(const wbcomplex &a) {
00333     return (std::isnan(a.r) || std::isnan(a.i)); }
00334
00335 inline double REAL(const double &a) { return a; }
00336 inline double REAL(const wbcomplex &a) { return a.r; }
00337
00338 inline double IMAG(const double &a __attribute__((unused))) { return 0; }
00339 inline double IMAG(const wbcomplex &a) { return a.i; }
00340
00341 #ifdef __WB_MPFR_HH__
00342
00343 template <unsigned P> inline
00344 Wb::mpfr__<P> ABS(const Wb::mpfr__<P> &a) { return a.abs(); };
00345 template <unsigned P> inline
00346 Wb::mpfr__<P> NORM(const Wb::mpfr__<P> &a) { return a.abs(); };
00347
00348 template <unsigned P> inline
00349 Wb::mpfr__<P> ABS2(const Wb::mpfr__<P> &a) { return a.abs2(); };
00350 template <unsigned P> inline
00351 Wb::mpfr__<P> NORM2(const Wb::mpfr__<P> &a) { return a.abs2(); };
00352
00353 #endif
00354
00355 template <class T> inline double ABSDIFF_F(
00356     const T &d1, const T &d2
00357 ){ return double(fabs(float(d1)-float(d2))); };
00358
00359 template <> inline double ABSDIFF_F(
00360     const wbcomplex &d1, const wbcomplex &d2
00361 ){

```

```

00362     float dr=float(d1.r)-float(d2.r), di=float(d1.i)-float(d2.i);
00363     return std::sqrt(double(dr*dr+di*di));
00364 };
00365
00366 inline wbcomplex exp(const wbcomplex &z) {
00367     double r=exp(z.r);
00368     return wbcomplex(r*cos(z.i), r*sin(z.i));
00369 };
00370
00371 inline wbcomplex expi(const double &x) {
00372     return wbcomplex(cos(x), sin(x));
00373 };
00374
00375 inline wbstring toStr(const wbcomplex &z, const char* fmt="%.4g");
00376 inline wbstring toStr(const double &d, const char* fmt="%.4g");
00377
00378 #endif
00379

```

5.34 wbeigs.hh

```

00001 #ifndef __WB_EIGS_HH__
00002 #define __WB_EIGS_HH__
00003
00004 // ----- //
00005 // ----- //
00006
00007 #define AUPDFUN dsaupd
00008 #define EUPDFUN dseupd
00009 #define AUPDSTR "DSAUPD"
00010 #define EUPDSTR "DSEUPD"
00011
00012 extern "C" {
00013
00014 #define dsaupd dsaupd_
00015 void dsaupd (
00016
00017     const int    &,
00018     const char   &,
00019     const int    &,
00020     const char[2] ,
00021     const int    &,
00022     const double &,
00023     double[],
00024     const int    &,
00025     double[],
00026     const int    &,
00027     const int[11] ,
00028     const int[11] ,
00029     double[],
00030     double[],
00031     const int    &,
00032
00033     const int    &
00034 );
00035
00036 #define dseupd dseupd_
00037 void dseupd (
00038
00039     const int    &,
00040     const char   &,
00041     const int[]  ,
00042     const double[],
00043     const double[],
00044     const int    &,
00045     const double &,
00046
00047     const char   &,
00048     const int    &,
00049     const char[2] ,
00050     const int    &,
00051     const double &,
00052     double[],
00053     const int    &,
00054     double[],
00055     const int    &,
00056     const int[11] ,
00057     const int[11] ,
00058     double[],
00059     double[],
00060     const int    &,
00061     const int    &
00062 );

```

```

00063
00064 }
00065
00066 char* info2str(int info) {
00067     char s[64];
00068     switch (info) {
00069         case -1: strcpy(s, "N<=0"); break;
00070         case -2: strcpy(s, "NEV<=0"); break;
00071         case -3: strcpy(s, "NCV<=NEV || NCV>N"); break;
00072         case -4: strcpy(s, "MAXIT<0"); break;
00073         case -5: strcpy(s, "invalid WHICH"); break;
00074         case -6: strcpy(s, "invalid BMAT"); break;
00075         case -7: strcpy(s,
00076             "length of private work array WORKL not sufficient"); break;
00077         case -8: strcpy(s, "error return from trid. eigenvalue "
00078             "calculation (from lapack::dsteqr)"); break;
00079         case -9: strcpy(s, "starting vector is zero"); break;
00080         case -10: strcpy(s, "invalid MODE=IPARAM[6]"); break;
00081         case -11: strcpy(s, "IPARAM[6]==1 && BMAT=='G' are incompatible"); break;
00082         case -12: strcpy(s, "invalid IPARAM[0] (must be equal to 0 or 1)"); break;
00083         case -13: strcpy(s, "NEV && WHICH=='BE' are incompatible."); break;
00084         case -9999: strcpy(s, "could not build an Arnoldi factorization."); break;
00085         case -15: strcpy(s, "HOWMNY must be one of 'A' or 'S' if RVEC==true"); break;
00086         case -16: strcpy(s, "HOWMNY = 'S' not yet implemented"); break;
00087         case -17: strcpy(s, "data inconsistency between DSAUPD and DSEUPD"); break;
00088         default: sprintf(s, "unknown info=%d", info);
00089     }
00090
00091     return s;
00092 }
00093
00094 #endif
00095

```

5.35 wbfft.hh

```

00001
00002 #ifndef __WB_FFTW_C__
00003 #define __WB_FFTW_C__
00004
00005 #include <rfftw.h>
00006
00007 namespace Wb {
00008
00009 int FFT2D (
00010     wbMatrix<double> &X2,
00011     wbMatrix<double> &F2,
00012     char dir
00013 ){
00014
00015     int info=-1, thesame=(X2==&F2);
00016     wbvector<int> dim(2);
00017
00018     if (dir=='F') {
00019         if (X2.isEmpty()) wblog(FL,
00020             "ERR %s - invalid dim [%d,%d]", FCT, X2.dim1, X2.dim2);
00021
00022         dim[0] = X2.dim1;
00023         dim[1] = X2.dim2;
00024
00025         if (!thesame) F2=X2;
00026
00027         F2.Resize(dim[0], dim[1]+2);
00028
00029         info=FFT_ND(dim, F2[0], dir);
00030     }
00031     else
00032     if (dir=='B') {
00033         if (F2.isEmpty()) wblog(FL,
00034             "ERR %s - invalid dim [%d,%d]", FCT, F2.dim1, F2.dim2);
00035
00036         dim[0] = F2.dim1;
00037         dim[1] = F2.dim2-2;
00038
00039         if (!thesame) X2 = F2;
00040
00041         info = FFT_ND (dim, X2[0], dir);
00042
00043         X2.Resize(dim[0], dim[1]);
00044     }
00045     else
00046     wblog(FL, "ERR %s - invalid dir=%c<%d>", FCT, dir, dir);
00047

```

```

00048     return info;
00049 }
00050
00051 int FFT_ND (
00052     wvector<int> &dim,
00053     double *zz,
00054     char dir
00055 ){
00056
00057     unsigned i; int s=dim.prod(), info=-1;
00058
00059     if (dim.len==0 || s<=0) wblog(FL,
00060         "ERR %s - invalid dimensions [%s]",FCT,dim.toStr().data);
00061
00062     if (dim.last()%2) wblog(FL,
00063         "ERR %s - invalid dimensions [%s]\n"
00064         "leading/last dimension must be even!",FCT,dim.toStr().data);
00065
00066     if (dir=='F') {
00067
00068         rfftwnd_plan plan;
00069
00070         plan = rfftwnd_create_plan (
00071             dim.len, dim.data, FFTW_FORWARD, FFTW_ESTIMATE | FFTW_IN_PLACE);
00072
00073         rfftwnd_one_real_to_complex (plan, zz, NULL);
00074
00075         rfftwnd_destroy_plan(plan);
00076         info=0;
00077     }
00078
00079     else if (dir=='B') {
00080
00081         double fact; int ntot, nlast=dim.last();
00082         rfftwnd_plan plan;
00083
00084         plan = rfftwnd_create_plan (
00085             dim.len, dim.data, FFTW_BACKWARD, FFTW_ESTIMATE | FFTW_IN_PLACE);
00086
00087         rfftwnd_one_complex_to_real (plan, (fftw_complex*)zz, NULL);
00088
00089         for (ntot=1,i=0; i<dim.len; i++) ntot *= dim[i];
00090         fact = 1./ntot;
00091         ntot = (ntot / nlast) * (nlast+2);
00092         for (i=0; i<ntot; i++) zz[i] *= fact;
00093
00094         rfftwnd_destroy_plan(plan);
00095         info = 0;
00096     }
00097     else wblog(FL,"ERR %s - invalid dir=%c<%d>",dir,dir);
00098
00099     return info;
00100 }
00101
00102 }
00103
00104 #endif
00105

```

5.36 wbindx.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace index routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_INDEX_CC__
00021 #define __WB_INDEX_CC__
00022

```

```

00023 /* ----- */
00024 /* ----- */
00025 // if toend==0 : add towards the end, i.e. P=[I,...] (default)
00026 // else P=[...,I].
00027
00028 int wbindex::extend2Perm(widx_t N, wbpperm &P, char toend) const {
00029     widx_t i,k;
00030     wbvector<char> mark(N);
00031
00032     for (i=0; i<len; ++i) {
00033         if (data[i]>=N) { sprintf(str,
00034             "%s:%d %s() index out of bounds (%ld/%ld)",FLF,data[i],N);
00035             return 1;
00036         }
00037         if (mark[data[i]]++) { sprintf(str,
00038             "%s:%d %s() index not unique",FLF);
00039             return 1;
00040         }
00041     }
00042 }
00043
00044 P.init(N);
00045
00046 if (!toend) {
00047     memcpy(P.data,data,len*sizeof(widx_t));
00048
00049     for (k=len, i=0; i<N; ++i)
00050         if (!mark[i]) P[k++]=i;
00051 }
00052 else {
00053     memcpy(P.data+N-len,data,len*sizeof(widx_t));
00054
00055     for (k=i=0; i<N; ++i)
00056         if (!mark[i]) P[k++]=i;
00057 }
00058
00059 return 0;
00060 };
00061
00062 wbindex& wbindex::invert(widx_t n, wbindex &I, char uflag) const {
00063     wbvector<char> M(n); char *m=M.data;
00064     widx_t i,l;
00065
00066     for (i=0; i<len; i++) { l=data[i];
00067         if (l<n) m[l]=1; else wblog(FL,
00068             "ERR %s() index out of bounds (%d: %d/%d)",FCT,i,l,n);
00069     }
00070
00071     l=M.nnz();
00072     if (uflag && l!=len) wblog(FL,
00073         "ERR %s() index not unique (%d/%d)",FCT,l,len);
00074
00075     I.init(n-l);
00076     if (I.len) {
00077         for (l=i=0; i<n; ++i) if (!m[i]) I.data[l++]=i;
00078     }
00079
00080     return I;
00081 };
00082
00083
00084 wbindex wbindex::flipIdx(widx_t ndim) const {
00085     wbindex a(len);
00086
00087     if (ndim==0) {
00088         if (len==0) return *this; else { dbstop(FL);
00089             wblog(FL,"ERR wbindex - invalid ndim=%d (%d)", ndim, len); }
00090     }
00091
00092     for (widx_t m=ndim-1, i=0; i<len; i++) {
00093         if (i>m) wblog(FL,"ERR wbindex - invalid ndim=%d (%d)",ndim,i);
00094         a.data[i]=m-data[i];
00095     }
00096     return a;
00097 };
00098
00099 inline char wbindex::isUnique(widx_t imax, widx_t offset) const {
00100     wbvector<char> mark(imax);
00101
00102     if (offset==0) {
00103         for (widx_t i=0; i<len; i++) {
00104             if (data[i]<imax) { if ((++mark[data[i]])>1) return 0; }
00105             else return 0;
00106         }
00107     }
00108     else {
00109         widx_t i=0, i0=offset; imax+=offset;

```

```

00110         for (; i<len; i++) {
00111             if (data[i]>=i0 && data[i]<imax) {
00112                 if ((++mark[data[i]-i0])>1) return 0;
00113             }
00114             else return 0;
00115         }
00116     }
00117
00118     return 1;
00119 };
00120
00121 inline wbindex& wbindex::init(const ctrIdx &ic) {
00122     initT(ic.len,ic.data);
00123     return *this;
00124 };
00125
00126 inline wbindex& wbindex::init(
00127     const char *F, int L, const mxArray *a, widx_t offset
00128 ){
00129     if (!a) { init(); return *this; }
00130
00131     const size_t *sz=mxGetDimensions(a);
00132
00133     Mx::Array<double> A(a);
00134     wbvector<double> x;
00135
00136     if (A.rank!=2 || (sz[0]!=1 && sz[1]!=1)) wblog(F_L,
00137         "ERR %s() numeric vector required\n%s",FCT,STR(A));
00138
00139     init(A.len); if (!A.len) { return *this; }
00140     if (A.data)
00141         { x.init(A.len,A.data,'r'); }
00142     else { x.init(A.len); A.rcopy_to(x.data,0,'!'); }
00143
00144     for (widx_t i=0; i<A.len; ++i) { data[i]=widx_t(x.data[i]);
00145
00146         if (double(data[i])!=x.data[i]) wblog(F_L,
00147             "ERR %s() type mismatch (%d: %d %g)",i+1,data[i],x.data[i]);
00148
00149         if (offset) {
00150             if (data[i]>=offset) { data[i]-=offset; }
00151             else if (offset==1) wblog(F_L,"ERR %s() "
00152                 "1-based index required (%d: %d)",FCT,i+1,data[i]);
00153             else wblog(F_L,"ERR %s() "
00154                 "index out of bounds (%d: %d/%d)",FCT,i+1,data[i],offset
00155             );
00156         }
00157     }
00158
00159     return *this;
00160 };
00161
00162 template <class T>
00163 template <class T2>
00164 void groupIndex<T>::getRecs0(
00165     const wbMatrix<T2> &AB, wbMatrix<T2> &X, T m
00166 ) const {
00167
00168     widx_t i0=0; if (swidx_t(m)<0) { m=-m; i0=AB.dim2-m; }
00169
00170     if (swidx_t(i0)<0 || i0+m>AB.dim2) wblog(FL,
00171         "ERR %s() size out of bounds (%d+%d/%d)",FCT,i0+m,AB.dim2);
00172
00173     X.init(D.len,m); if (!D.len || !m) return;
00174
00175     if (I0.len!=D.len+1) wblog(FL,
00176         "ERR %s() unexpected size %d/%d",FCT,I0.len-1,D.len);
00177     if (AB.dim1!=P.len) wblog(FL,"ERR %s() size mismatch "
00178         "%dx%d <> %dx%d",FCT,P.len, m, AB.dim1, AB.dim2);
00179
00180     for (widx_t i=0; i<D.len; ++i) {
00181         X.recSetP(i,AB.ref(I0.data[i],i0));
00182     }
00183 };
00184
00185 int ctrIdx::init(const char *F, int L, const char *s, unsigned offset) {
00186
00187     int i, n=Str2Idx(FL,s,(wbvector<unsigned>&)(*this),offset,'x');
00188
00189     if (!n) {
00190         if (F) wblog(F,L,"ERR %s() got empty/null string !?",FCT);
00191         return -1;
00192     }
00193
00194     if (n<0) { int m=0; i=-n-1;
00195
00196         for (; s[i]; ++i) { if (!isspace(s[i])) {

```

```

00197         if (CTR_IS_SEP_ANY(s[i])) { ++m; } else break;
00198     }
00199     if (m<=1 && CTR_IS_CONJ(s[i])) { conj=1;
00200         for (++i; s[i]; ++i) { if (!isspace(s[i])) break; }
00201     }
00202
00203     if (s[i]) { if (F) wblog(F,L,"ERR %s() "
00204         "invalid ctr-index set '%s' (%d: %c<%d>)",FCT,s,i+1,s[i],s[i]);
00205         return -2;
00206     }
00207 }
00208
00209 if (!len) { if (F) wblog(F,L,
00210     "ERR %s() got empty ctr-index set '%s' !?",FCT,s);
00211 }
00212 else if (!wbvector<unsigned>::isUnique()) { if (F) wblog(F,L,
00213     "ERR %s() invalid index set '%s' [%s]",
00214     FCT,STR_(this),wbvector<unsigned>::toStr().data);
00215     return -3;
00216 }
00217 else if (F) {
00218     if (len>32 || anyGT(62)) wblog(F,L,"WRN "
00219         "got ctr-index [%s] (len=%d)",wbvector<unsigned>::toStr().data,len);
00220 }
00221
00222 return len;
00223 };
00224
00225 ctrIdx& ctrIdx::init(const char *F, int L, const mxArray *a) {
00226
00227     if (!a) wblog(F,L,"ERR %s() got null mxArray",FCT);
00228     if (mxIsChar(a)) {
00229         wbsting s(a); int e=init(F,L,s.data);
00230         if (e<=0) wblog(F,L,
00231             "ERR %s() '%s' => [%s] @ e=%d !?",FCT,s.data,STR_(this),e);
00232     }
00233     else {
00234         wbvector<unsigned>::init(F,L,a);
00235         if (anyEqual(0)) wblog(F,L,
00236             "ERR %s() contraction index must be 1-based",FCT);
00237         (*this)--l;
00238         conj=0;
00239     }
00240     return *this;
00241 };
00242
00243 template<class TQ, class TD>
00244 ctrIdx& ctrIdx::initOp(const char *F, int L,
00245     itag_ t1, const QSpace<TQ,TD> &B, const char *sopt) {
00246
00247     unsigned l=0, r=-1;
00248
00249     if (!sopt || (l=strlen(sopt))>8 || strcmp(sopt,"-op",3)) wblog(F,L,
00250         "ERR %s() invalid ctr-string '%s'",FCT,sopt?sopt:"");
00251
00252     if (B.isOperator(&r)<=0 || r>3 || r!=B.itags.len) wblog(F,L,
00253         "ERR %s() got '%s' with QSpace %s !?",FCT,sopt,STR(B.itags));
00254
00255     init(l);
00256     if (sopt[l-1]=='*') { data[0]=0; conj=1; } else { data[0]=1; }
00257
00258     newtags.initOp(t1,B.itags,sopt+1);
00259
00260     return *this;
00261 };
00262
00263 int ctrIdx::extend2Perm(widx_t N, wbperm &P, char toend) const {
00264
00265     widx_t i,k;
00266     wbvector<char> mark(N);
00267
00268     for (i=0; i<len; ++i) {
00269         if (data[i]>=N) { sprintf(str,
00270             "%s:%d %s() index out of bounds (%d/%ld)",FLF,data[i],N);
00271             return 1;
00272         }
00273         if (mark[data[i]]++) { sprintf(str,
00274             "%s:%d %s() index not unique",FLF);
00275             return 1;
00276         }
00277     }
00278
00279     P.init(N);
00280
00281     if (!toend) {
00282         for (k=0; k<len; ++k) { P.data[k]=data[k]; }
00283         for (i=0; i<N; ++i) { if (!mark[i]) P[k++]=i; }

```

```

00284     }
00285     else {
00286         for (k=i=0; i<N; ++i) { if (!mark[i]) P[k++]=i; }
00287         for (i=0; i<len; ++i) { P.data[i+k]=data[i]; }
00288     }
00289
00290     return 0;
00291 };
00292
00293 wbperm&
00294 ctrIdx::extend2Perm(    unsigned ra, unsigned ma,
00295     const ctrIdx &icb__, unsigned rb, unsigned mb,
00296     const wbperm &pab, char conj, unsigned mc,
00297     wbperm &Pbc,
00298     ctrIdx &kb,
00299     ctrIdx &kcb,
00300     char Bflag
00301 ) const {
00302
00303     ctrIdx ka,
00304             kca;
00305
00306     unsigned i=0, l=0, m=0, rca=ra-len, rcb=rb-icb__.len, rc=rca+rcb;
00307
00308     ctrIdx ica,icb; wbperm p,ipab;
00309     if (Bflag)
00310         { icb__.sort(icb,p); this->wbvector<unsigned>::permute(ica,p); }
00311     else { this->sort(ica,p); icb__.wbvector<unsigned>::permute(icb,p); }
00312
00313     if (pab.len)
00314         ipab.init(pab,'i');
00315     else ipab.init(rc);
00316
00317     if (len!=icb.len || ra<len || rb<icb.len) wblog(FL,"ERR %s() "
00318         "invalid ctrIdx data (%d/%d, %d/%d) !?",FCT,ra,len,rb,icb.len);
00319     if (ra>127 || rb>127 || rc>127) wblog(FL,
00320         "ERR %s() got rank %d/%d/%d !?",FCT,ra,rb,rc);
00321     if (ipab.len!=rc) wblog(FL,"ERR %s() "
00322         "invalid permutation pab=[%s] (%d)",FCT,STR(pab),rc);
00323
00324     ica.invert(ra,ka);
00325     icb.invert(rb,kb);
00326
00327     kca.initT(rca, ipab.data, !conj);
00328     kcb.initT(rcb, ipab.data+rca, !conj);
00329
00330     if (Bflag) {
00331         wbperm Pa(ra+(mb?1:0)+(mc?1:0)); m=ra;
00332
00333         for (; l<len; ++l) { Pa[l]=ica[l]; }
00334         if (mb) Pa[l++]=(m++);
00335
00336         kca.Sort(p); ka.Permute(p);
00337         for (i=0; i<ka.len; ++i, ++l) { Pa[l]=ka[i]; }
00338         if (mc) Pa[l++]=(m++);
00339
00340         if (l!=Pa.len || !Pa.isValidPerm()) wblog(FL,
00341             "ERR %s() got invalid Pa=[%s] (%d/%d)",FCT,STR(Pa),l,Pa.len);
00342         Pa.invert(Pbc);
00343     }
00344     else {
00345         wbperm Pb(rb+(ma?1:0)+(mc?1:0)); m=rb;
00346
00347         for (; l<len; ++l) { Pb[l]=icb[l]; }
00348         if (ma) Pb[l++]=(m++);
00349
00350         kcb.Sort(p); kb.Permute(p);
00351         for (i=0; i<kb.len; ++i, ++l) { Pb[l]=kb[i]; }
00352         if (mc) Pb[l++]=(m++);
00353
00354         if (l!=Pb.len || !Pb.isValidPerm()) wblog(FL,
00355             "ERR %s() got invalid Pb=[%s] (%d/%d)",FCT,STR(Pb),l,Pb.len);
00356
00357         Pb.invert(Pbc);
00358         ka.save2(kb);
00359         kca.save2(kcb);
00360     }
00361
00362     return Pbc;
00363 };
00364
00365 int ctrIdx::checkUniqueS(const char *F, int L, const unsigned r) const {
00366
00367     if (len) {
00368         unsigned i=1;
00369         for (; i<len; ++i) {
00370             if (data[i-1]>=data[i]) {

```



```

00371         if (data[i-1]>data[i]) { if (F) wblog(F,L,
00372             "ERR %s() ctrIdx not sorted [%s]",FCT,STR_(this));
00373             return 1;
00374         }
00375         if (data[i-1]==data[i]) wblog(F_L,
00376             "ERR %s() ctrIdx not sorted [%s]",FCT,STR_(this));
00377     }
00378 }
00379
00380     if (int(r)>0) {
00381         if (data[i-1]>=r) wblog(F_L,
00382             "ERR %s() ctrIdx out of bounds (%s; %d) !?",FCT,STR_(this),r);
00383     }
00384 }
00385
00386     return 0;
00387 };
00388
00389 ctrIdx& ctrIdx::invert(unsigned r, ctrIdx &I) const {
00390
00391     wvector<char> M(r); char *m=M.data;
00392     unsigned i=0, l;
00393
00394     for (; i<len; ++i) { l=data[i];
00395         if (l>=r) wblog(F_L,
00396             "ERR %s() index out of bounds (%d: %d/%d)",FCT,i,l,r);
00397         if ((++m[l])>1) wblog(F_L,
00398             "ERR %s() index not unique: %s",FCT,toStr().data
00399             );
00400     }
00401
00402     I.init(r-len);
00403     if (I.len) {
00404         for (l=i=0; i<r; ++i) { if (!m[i]) I.data[l++]=i; }
00405     }
00406
00407     I.conj=conj;
00408
00409     return I;
00410 };
00411
00412 wstring ctrIdx::toStr(char xflag) const {
00413     wstring s;
00414
00415     if (!xflag && this->anyGT(60)) {
00416         unsigned l, n=4*len+4; s.init(n);
00417
00418         l=snprintf(s.data,n,"%s%s",
00419             ((wvector<unsigned>&)(*this)+1).toStrf("%d","").data,
00420             conj? " ;*:" : "");
00421         if (l>=n) wblog(F_L,
00422             "ERR %s() string out of bounds (%d/%d)",FCT,l,n
00423             );
00424     }
00425     else {
00426         unsigned i=0, n=len+(conj ? 2 : 1); s.init(n);
00427         for (; i<len; ++i) {
00428             if (data[i]< 9) { s.data[i]='1'+ data[i]; } else
00429             if (data[i]<35) { s.data[i]='A'+(data[i]- 9); }
00430             else { s.data[i]='a'+(data[i]-35); }
00431         }
00432         if (conj) { s.data[i++]='*'; }
00433         s.data[i]=0;
00434     }
00435
00436     return s;
00437 };
00438
00439 itag_& itag_::init(const char* F, int L, const char *s, unsigned n) {
00440
00441     if (!s || !s[0] || !n) { t=0; return *this; }
00442
00443     unsigned i=0, l=0, n_=ITAG_LEN;
00444     char *q=(char*)(&t); t=0;
00445
00446     if (int(n)<0 || n>n_) n=n_;
00447
00448     while (s[i]!=' ') { ++i; }
00449     if (i) wblog(F_L,"WRN %s() ignoring leading blanks (%d)",FCT,i);
00450
00451     for (; i<n && s[i]; ++i) {
00452         if (IS_CHAR_ITAG(s[i])) { q[l++]=s[i]; }
00453         else break;
00454     }
00455
00456     while (s[i]==CC_ITAG) { ++i;
00457         q[0] ^= char(128);

```

```

00458     }
00459
00460     if (i<n && s[i]) {
00461         if (i>=n_)
00462             sprintf(str,"invalid itag '%s' (len=%li/%d)",s,strlen(s),n);
00463         else sprintf(str,"invalid itag '%s' (len=%li/%d; i=%d)",s,strlen(s),n,i);
00464         if (F) wblog(FL,"ERR %s",str); else wblog(F_L,"WRN %s",str);
00465         t=0; return *this;
00466     }
00467     if (!t && F) wblog(F,L,"WRN %s() invalid itag '%s'",FCT,s);
00468
00469     return *this;
00470 };
00471
00472 template<class TQ, class TD>
00473 itag_& itag_::init(const char *F, int L,
00474     const wvector< QSpace<TQ,TD> > &A,
00475     windex ia
00476 ){
00477     t=0; if (!A.len) return *this;
00478
00479     unsigned k=0, l=0; itag_ x;
00480
00481     if (!ia.len) {
00482         unsigned rk,r_=-1;
00483         for (; k<A.len; ++k) {
00484             if (int(rk=A[k].isOperator(&r_,'x'))<=0) wblog(FL,
00485                 "ERR %s() invalid rank-%d operator \n(%d: %d/%d, '%s')",
00486                 FCT,A[k].itags.len,k+1,rk,r_,A[k].otype2Str().data
00487             );
00488         }
00489         ia.Index(r_); k=0;
00490     }
00491     if (!ia.len) wblog(FL,"ERR %s() got empty index set !?",FCT);
00492
00493     for (; k<A.len; ++k) {
00494         const iTags &it=A[k].itags;
00495         for (l=0; l<ia.len; ++l) {
00496             if (ia[l]>=it.len) wblog(F_L,
00497                 "ERR %s() index out of bounds (%d/%d)",FCT,ia[l],it.len);
00498             (x=it[ia[l]]).deConj();
00499
00500             if (!t) t=x.t; else
00501                 if (x.t && x.t!=t) wblog(F_L,"ERR %s() got itags "
00502                     "mismatch (%s [%s]; %s)",FCT, A[k].itags.toStr().data,
00503                     (ia+1).toStr().data, this->toStr().data
00504                 );
00505         }
00506     }
00507     return *this;
00508 };
00509
00510 bool itag_::isEmpty(char lenient) const {
00511
00512     if (!lenient) { return (t ? 0 : 1); }
00513     if (lenient==1 || lenient=='l') {
00514         return isEmpty();
00515     }
00516
00517     char mflag=(lenient=='m' ? 1 : (lenient=='M' ? 2 : 0));
00518     if (mflag==1) {
00519         if (t & ~(IDT)255) { return 0; }
00520     }
00521     else {
00522         unsigned i=(mflag ? 1 : 0);
00523         const char *s=(const char*)&t;
00524         for (; i<ITAG_LEN; ++i) { if (s[i]&127) return 0; }
00525
00526         if (lenient==2 || lenient=='L') { return 1; }
00527     }
00528     if (mflag) {
00529         char c=((char&t) & 127;
00530         return (!c || IS_CHAR_MARK(c) ? 1 : 0);
00531     }
00532     wblog(FL,"WRN %s() invalid flag '%s'",FCT,cSTR(lenient));
00533     return isEmpty();
00534 };
00535
00536 bool itag_::isEmpty_m() const {
00537
00538     if (t & ~(IDT)255) { return 0; }
00539     else {
00540         char c=((char&t) & 127;
00541         return (!c || IS_CHAR_MARK(c) ? 1 : 0);
00542     }
00543 };
00544

```

```

00545 itag_& itag_::SetFlag(unsigned l) {
00546     if (l>=ITAG_LEN) wblog(FL,
00547         "ERR %s() index out of bounds (%d/%d)",FCT,l,ITAG_LEN);
00548     ((char*)&t)[l] |= 128;
00549     return *this;
00550 };
00551
00552 itag_& itag_::SetFlags(unsigned l, char check) {
00553     char *s = (char*)&t;
00554
00555     if (l==1) { s[0] |= 128; return *this; }
00556
00557     if (int(l)<0) { unsigned q=(int(l)==-1);
00558         l=( itag_::flag_id & ~((unsigned char)3) );
00559         if (q) { itag_::iter_flag_id(); }
00560
00561         if (int(l)<=0) wblog(FL,
00562             "ERR %s() invalid usage (l=%d / %d; %d)",FCT,l,itag_::flag_id,q);
00563     }
00564     else if (int(l)<=0) {
00565         wblog(FL,"ERR %s() invalid usage (l=%d)",FCT,l);
00566     }
00567
00568     unsigned i=0, n=ITAG_LEN;
00569
00570     if (l>=(1U<n)) wblog(FL,
00571         "ERR %s() index out of bounds (%d/%d)",FCT,l,1<n);
00572     if (l&1) wblog(FL,"WRN %s() got flags %d",FCT,l);
00573
00574     for (; i<n; ++i) {
00575         if (l & (1<i)) {
00576             if (check && (s[i] & 128)) wblog(FL,
00577                 "WRN %s() flag %d already set (%s)",FCT,i,STR_(this));
00578             s[i] |= 128;
00579         }
00580         else if (i) {
00581             if (check && (s[i] & 128)) wblog(FL,
00582                 "WRN %s() unsetting flag %d (%s)",FCT,i,STR_(this));
00583             s[i] &= 127;
00584         }
00585     }
00586     return *this;
00587 };
00588
00589 itag_& itag_::SetSI(const char *F, int L,
00590     const char *tag, unsigned k, char mflag, char nfmt) {
00591     unsigned l, n=ITAG_LEN+1;
00592     char s[n];
00593     if (!tag) wblog(F_L,"ERR %s() got null tag",FCT);
00594
00595     if (nfmt==2) { l=snprintf(s,n,"%s%02d",tag,k); }
00596     else if (nfmt<=1) { l=snprintf(s,n,"%s%d",tag,k); }
00597     else {
00598         unsigned m=8; char fmt[m];
00599         l=snprintf(fmt,m,"%s%s%0d",nfmt);
00600         if (l>=m) wblog(FL,
00601             "ERR %s() fmt string out of bounds (%s; %d/%d)",FCT,fmt,l,m);
00602         l=snprintf(s,n,fmt,tag,k);
00603     }
00604
00605     if (mflag & ~char(7)) wblog(F_L,
00606         "ERR %s() invalid mflag=%s",FCT,cSTR(mflag));
00607
00608     if (mflag&3) {
00609         char m=getMark();
00610         if (m && l<n) { s[l]=m; if (++l<n) { s[l]=0; } }
00611     }
00612     if (mflag&1 || ((mflag&2) && isConj())) {
00613         if (l+1<n) { s[l]=CC_ITAG; s[l+1]=0; }
00614         else { ++l; }
00615     }
00616     if (l>=n) { s[n-1]=0; wblog(F_L,"ERR %s() "
00617         "string out of bounds (%s, %d/%d/%d)",FCT,s,l,n,ITAG_LEN); }
00618
00619     return init(s);
00620 };
00621
00622 #ifndef NOMEX
00623
00624 itag_& itag_::init(const char* F, int L, const mxArray *a) {
00625     if (!a) { t=0; return *this; }
00626     if (mxIsChar(a)) { wbsting s(a); init(F,L,s.data); }
00627     else wblog(F_L,
00628         "ERR %s() invalid QSpace.itag upon input (%s)",

```

```

00632         FCT,mxGetClassName(a)
00633     );
00634     return *this;
00635 };
00636
00637 #endif
00638
00639 char itag_::sameAs(const itag_ &B, char lflag) const {
00640
00641     char q=0;
00642     IDT c128=128, a=t, b=B.t;
00643     bool cflag = (a & c128) != (b & c128);
00644
00645     a &= ~c128;
00646     b &= ~c128;
00647
00648     if (a==b) { q=(a ? 3 : 4); } else
00649     if (lflag) {
00650         if (!b) { if (! isEmpty_m()) { q|=1; }} else
00651         if (!a) { if (!B.isEmpty_m()) { q|=2; }}
00652     }
00653
00654     if (cflag) { q=-q; }
00655
00656     return q;
00657 };
00658
00659 unsigned itag_::to_str(char *sout, unsigned len,
00660     char cflag
00661 ) const {
00662
00663     unsigned e=0, l=0, i=0, m=0, n=ITAG_LEN;
00664     const char *st=(const char *)(&t), c128=char(128);
00665
00666     if (int(len)<=int(n)) wblog(FL,
00667         "ERR %s() string out of bounds (%d/%d)",FCT,len,n);
00668
00669     for (; i<n && st[i]; ++i) { sout[i]=st[i];
00670         if (sout[i]<0) { sout[i]+=c128; m|=(1<i); }
00671         if (sout[i])
00672             { if (!IS_CHAR_ITAG(sout[i])) { e+=10; }}
00673         else { break; }
00674     }
00675
00676     for (l=i, ++i; i<n; ++i) { if (st[i]) {
00677         if (st[i]==c128) { m|=(1<i); } else { e+=100; }
00678     }}
00679
00680     if (e) { sout[l]=0; wblog(FL,
00681         "WRN %s() got non-char itag (%s; e=%d) !?",FCT,sout,e); }
00682     if ((m&1) && !cflag) { m-=1; }
00683
00684     if (m>1 && l<len) {
00685         if (!(m&2))
00686             { l+=snprintf(sout+l,len-l,"%d",m&2); }
00687         else { l+=snprintf(sout+l,len-l,"%d",m&1); }
00688     }
00689     if (m&1 && l<len) { sout[l++]=CC_ITAG; }
00690
00691     if (l>=len) { sout[len-1]=0; wblog(FL,
00692         "ERR %s() string out of bounds (%s / %d)",FCT,sout,l); }
00693     sout[l]=0;
00694
00695     return l;
00696 };
00697
00698 int itag_::isValid() const {
00699
00700     unsigned l=0, i=0, n=ITAG_LEN, nx=2;
00701     const char *s=(const char *)(&t), x128=~char(128);
00702     char x;
00703
00704     for (; i<n; ++i) { x=(s[i] & x128);
00705         if (x) { if (!IS_CHAR_ITAG(x)) return 0; } else break;
00706         if (s[i]<0 && i>=nx) return 0;
00707     }; l=i;
00708
00709     for (; i<n; ++i) { if (s[i]) {
00710         if (i<nx) { if (s[i] & x128) return 0; }
00711         else return 0;
00712     }}
00713
00714     return (l ? 2: 1);
00715 };
00716
00717 bool itag_::isConj_x(const itag_ &x, char lflag) const {
00718

```

```

00719     if (lflag<=0) { return isConj(x); }
00720     if (lflag>3) {
00721         if (lflag=='l') lflag=1; else
00722             if (lflag=='L') lflag=3; else
00723             wblog(FL,"ERR %s() invalid lflag=%d !?",FCT,lflag);
00724     }
00725
00726     unsigned i=1;
00727     const char *a=(const char*)(&t), *b=(const char*)(&x.t);
00728
00729     if (!(lflag & 2)) {
00730         if ((a[0]^char(128)) != b[0]) return 0;
00731         for (; i<ITAG_LEN; ++i) { if (a[i]!=b[i]) break; }
00732     }
00733     else {
00734         if (tolower(a[0]^char(128)) != tolower(b[0])) return 0;
00735         for (; i<ITAG_LEN; ++i) { if (tolower(a[i])!=tolower(b[i])) break; }
00736     }
00737
00738     if (i<ITAG_LEN) {
00739         if (!(lflag & 1) ||
00740             (a[i] && a[i]!='~') || (b[i] && b[i]!='~')) return 0;
00741         for (++i; i<ITAG_LEN; ++i) { if (a[i] || b[i]) return 0; }
00742     }
00743
00744     return 1;
00745 };
00746
00747 itag_& itag_::AppendChar(char q, const char *qs) {
00748
00749     unsigned i=0, n=ITAG_LEN;
00750     char *s=(char*)(&t);
00751
00752     if (!IS_CHAR_ITAG(q)) wblog(FL,"ERR %s() invalid char %s",FCT,cSTR(q));
00753
00754     for (; i<n; ++i) { if (!(s[i] & char(127))) { break; }}
00755
00756     if (!i) { s[i]+=q; }
00757     else {
00758         char c=(s[i-1] & char(127));
00759         if (qs && strchr(qs,c)) { --i; }
00760         else if (i<n) {
00761             if (IS_CHAR_MARK(c)) {
00762                 s[i] = (s[i-1] & char(127)) + (s[i] & char(128));
00763                 --i;
00764             }
00765         }
00766         else wblog(FL,"ERR %s() itag out of bounds (%s)",FCT,STR(this));
00767
00768         s[i] = q + (s[i] & char(128));
00769     }
00770
00771     return *this;
00772 };
00773
00774 itag_& itag_::MarkDual(char m, bool always) {
00775
00776     unsigned i=0, n=ITAG_LEN;
00777     char *s=(char*)(&t);
00778
00779     if (!IS_CHAR_ITAG(m)) wblog(FL,"ERR %s() invalid char %s",FCT,cSTR(m));
00780
00781     for (; i<n; ++i) { if (!(s[i] & 127)) { break; }}
00782
00783     if (i || always) {
00784         if (i && (s[i-1] & 127)==m) {
00785             s[i-1] &= 128;
00786         }
00787         else if (i<n) {
00788             s[i] = (s[i] & 128) + m;
00789         }
00790         else { wblog(FL,
00791             "ERR %s() itag string out of bounds (%s)\n(length must "
00792             "not exceed %d chars for %s)",FCT,STR(this),n-1,myname);
00793         }
00794     }
00795     return *this;
00796 };
00797
00798 itag_& itag_::PrependChar(char q, const char *qs) {
00799
00800     unsigned i=0, n=ITAG_LEN;
00801     char *s=(char*)(&t), x=(s[0] & 127);
00802
00803     if (!IS_CHAR_ITAG(q)) wblog(FL,
00804         "ERR %s() invalid char %s",FCT,cSTR(q));
00805

```

```

00806     if (!qs || !strchr(qs,x)) {
00807         for (; i<n; ++i) { if (!( s[i] & char(127) )) { break; }}
00808         if (i==n) wblog(FL,"ERR %s() itag out of bounds (%s)",FCT,STR_(this));
00809         for (; i; --i) {
00810             s[i] = (s[i-1] & 127) + (s[i] & 128);
00811         }
00812     }
00813     s[i] = q + (s[i] & 128);
00814
00815     return *this;
00816 };
00817
00818 int itag_::CheckFirstChar(char q, const char *qs) {
00819     char *s=(char*)(&t), x=(s[0] & 127);
00820     if (!IS_CHAR_ITAG(q)) wblog(FL,"ERR %s() invalid char %s",FCT,cSTR(q));
00821     if (qs && strchr(qs,x)) {
00822         s[0] = q + (s[0] & 128);
00823         return 1;
00824     }
00825     return 0;
00826 };
00827
00828 unsigned iTags::init(const char *F, int L, const char *s) {
00829     if (s && s[0]) {
00830         unsigned i,l=0, i0=0, n=0;
00831         for (i=0; s[i]; ++i) {
00832             if (!IS_CHAR_ITAGS(s[i])) wblog(F_L,
00833                 "ERR %s() invalid itag >%s<\n"
00834                 "[required: printable string with no space]",FCT,s);
00835             if (IS_ITAGS_SEP(s[i])) ++l;
00836         }
00837         wbvector<itag_>::init(++l);
00838         for (l=0, n=i, i=0; i<=n; ++i) {
00839             if (IS_ITAGS_SEP(s[i]) || i==n) {
00840                 data[l].init(F_L,s+i0,i-i0);
00841                 ++l; i0=i+1;
00842             }
00843         }
00844     }
00845     else wbvector<itag_>::init();
00846     return len;
00847 };
00848
00849 iTags& iTags::init_alpha(
00850     unsigned r, unsigned s,
00851     const char *tag,
00852     char t,
00853     unsigned k
00854 ){
00855     unsigned i=0, l, n=10; char st[n];
00856     init(r+s);
00857     for (; i<len; ++i) {
00858         l=snprintf(st,n,"%s%c%d",tag ? tag : QS_ITAG_,t+i,k);
00859         if (i>=r && ++l<n) { st[l-1]=CC_ITAG; st[l]=0; }
00860         if (l>=n) wblog(FL,
00861             "ERR %s() string out of bounds (%s: %d/%d)",FCT,st,l,n);
00862         data[i].init(st);
00863     }
00864     return *this;
00865 };
00866
00867 #ifdef LOAD_CGC_QSPACE
00868 unsigned iTags::init_qdir(const char *s) {
00869     QDir qdir(FL,s); init(qdir.len);
00870     for (unsigned i=0; i<qdir.len; ++i) {
00871         if (qdir[i]<0) data[i].Conj();
00872     }
00873     return qdir.len;
00874 };
00875 #endif
00876
00877 #ifndef NOMEX
00878 unsigned iTags::init(const char *F, int L, const mxArray *a) {
00879

```

```

00893     if (!a || mxIsEmpty(a)) { init(); return len; }
00894
00895     if (mxIsChar(a)) { wbsttring sz(a); init(sz.data); }
00896     else if (mxIsCell(a)) {
00897         unsigned i=0, n=mxGetNumberOfElements(a);
00898         wvector<itag_>::init(n);
00899         for (; i<n; ++i) data[i].init(F_L,mxGetCell(a,i));
00900     }
00901     else wblog(FL,"ERR invalid QSpace.itag upon input");
00902
00903     return len;
00904 };
00905 #endif
00906
00907 iTags& iTags::initOp(
00908     itag_ t, const iTags &b, const char *ot) {
00909
00910     if (!b.isOp()) wblog(FL,"ERR %s() invalid usage (%s)",FCT,STR(b));
00911     init(b);
00912
00913     if (!t.isEmpty()) { t.deConj();
00914         for (unsigned i=0; i<2; ++i) {
00915             if (data[i].isConj()) { (data[i]=t).Conj(); }
00916             else { data[i]=t; }
00917         }
00918     }
00919
00920     if (len>2 && data[2].isEmpty()) {
00921         unsigned i=0;
00922         char s[10]; strncpy(s, ot?ot:"op", 8); s[8]=0;
00923
00924         for (; s[i]; ++i) { if (s[i]==CC_ITAG) { s[i]=0; break; }}
00925
00926         if (data[2].isConj()) { s[i]=CC_ITAG; s[i+1]=0; }
00927         data[2].init(s);
00928     }
00929
00930     return *this;
00931 };
00932
00933 unsigned iTags::Init(
00934     const char *F, int L, const char *tag, unsigned k, char lflag
00935 ){
00936     size_t l=0, n=64; char s[n]; s[0]=0;
00937     iTags ix(*this);
00938
00939     if (!tag) wblog(F_L,"ERR %s() invalid tag (empty)",FCT);
00940     if (!strcmp(tag,"nrg:",4)) {
00941         const char *t=tag+4;
00942         if (strlen(t)!=2) wblog(FL,"ERR %s() invalid tag '%s'",FCT,t);
00943
00944         if (int(k)<0) wblog(FL,
00945             "ERR %s() invalid k=%d for '%s'",FCT,k,t);
00946
00947         if (t[0]=='H' && strchr("KD",t[1])) {
00948             l=snprintf(s,n,"%c%02d;%c%02d*",t[1],k,t[1],k);
00949         }
00950         else if (t[0]=='A' && strchr("KD",t[1])) {
00951             if (k)
00952                 l=snprintf(s,n,"K%02d;%c%02d*;s%02d",k-1,t[1],k,k); }
00953             else { l=snprintf(s,n, "L00;%c%02d*;s%02d", t[1],k,k); }
00954         }
00955         else wblog(FL,"ERR %s() invalid tag '%s'",FCT,t);
00956     }
00957     else if (!strncasecmp(tag,"op:",3) && int(k)<0) {
00958         int i=0, n1=0, n2=-1; const char *t=tag+3;
00959         unsigned r=-k;
00960
00961         if (lflag<0) {
00962             lflag = (tag[0]=='O'? 1:0) + (tag[1]=='P'? 2:0);
00963         }
00964
00965         if (r<2 || (lflag ? r>4 : r>3)) wblog(FL,
00966             "ERR %s() invalid rank-%d operator",FCT,r);
00967
00968         for (; t[i]; ++i) {
00969             if (t[i]!=':') {
00970                 n1=i++; while (t[i]) { ++i; }; n2=i-n1-1;
00971                 if (n1>8 || n2>8) wblog(FL,"ERR %s() "
00972                     "itag string out of bounds ('%s' %d,%d/8)",FCT,t,n1,n2);
00973                 break;
00974             }
00975         }
00976         if (n2>=0) { char fmt[20];
00977             if (r>3) wblog(FL,
00978                 "ERR %s() got rank-%d QSpace (l=%d)",FCT,r,lflag);
00979

```

```

00980         if (r==2) {
00981             snprintf(fmt,20,"%%d.%ds;%%d.%ds*",n1,n1,n1,n1);
00982             l=snprintf(s,n,fmt,t,t);
00983         }
00984         else {
00985             snprintf(fmt,20,"%%d.%ds;%%d.%ds;%%s*",n1,n1,n1,n1);
00986             l=snprintf(s,n,fmt,t,t,t+n1+1);
00987         }
00988     }
00989     else if (r==3)
00990     { l=snprintf(s,n,"%s;%s;op*",t,t); }
00991     else { l=snprintf(s,n,"%s;%s*",t,t); }
00992
00993     if (r==3 && !data[2].isConj() && l>1 && l<n) { s[--l]=0; }
00994 }
00995 else wblog(F_L,"ERR %s() unexpected tag '%s'",FCT,tag);
00996 if (l>n) wblog(FL,"ERR %s() string out of bounds (%s/%d)",FCT,l,n);
00997 n=0;
00998
00999 init(F,L,s);
01000
01001 if (len<Ix.len) {
01002     for (unsigned i=0; i<len; ++i) {
01003         if (data[i].isConj(Ix.data[i],0)) { ++n; }
01004         else { Ix[i]=data[i]; }
01005     }
01006     if (!n) { Ix.save2(*this); }
01007 }
01008 else if (Ix.len && !Ix.sameConj(*this)) {
01009     n=99;
01010 }
01011
01012 if (n) wblog(FL,"ERR inconsistent conj labels "
01013             "%s => %s; e=%d",STR(Ix), STR_(this), n);
01014 return len;
01015 };
01016 };
01017
01018 iTags & iTags::Init(unsigned r, int k, const char **ss) {
01019     unsigned i=0, l, n=9; char st[n+1];
01020     init(r);
01021
01022     for (; i<r; ++i) { l=snprintf(st,n,"%s", ss[i]);
01023         if (k>=0 && l<n) {
01024             if (l>1 && st[l-2]==CC_ITAG) { --l;
01025                 l+=snprintf(st+1,n-l,"%d%c",k,CC_ITAG)-1; }
01026             else l+=snprintf(st+1,n-l,"%d",k);
01027         }
01028         if (l>=n) { st[n]=0; wblog(FL,
01029             "ERR %s() string out of bounds (%s /%d)",FCT,st,n); }
01030         data[i].init(st);
01031     }
01032 }
01033
01034 return *this;
01035 };
01036
01037 bool iTags::isOp(int r, char lflag) const {
01038     bool rval=0; if (r>=0) {
01039         if (r<2 || r>4) wblog(FL,"WRN %s() unexpected rank r=%d",FCT,r);
01040         if (len!=unsigned(r)) { return rval; }
01041     }
01042     else if (len<2 || len>4) { return rval; }
01043
01044     if (len==4) {
01045         for (unsigned i=2; i<4; ++i) {
01046             if (!data[i].isConj() ||
01047                 !data[i].isConj(data[i-2], lflag=='L'? 0:1)) {
01048                 return rval; }
01049         }
01050         return (rval=1);
01051     }
01052 }
01053
01054 if (!lflag) { unsigned i=1;
01055     for (; i<len; ++i) { if (!data[i].isConj()) { return rval; } }
01056 }
01057 else if (lflag!='L' && lflag!='L') {
01058     wblog(FL,"WRN %s() unexpected lflag=%s",FCT,cSTR(lflag));
01059 }
01060
01061 rval = data[0].isConj(data[1], lflag=='L'? 0:1);
01062 return rval;
01063 };
01064
01065 bool iTags::isOpX(unsigned r) const {
01066     if (len<2 || len%2 || (int(r)>=0 && len!=2*r)) return 0;

```



```

01067     for (unsigned n=len/2, i=0; i<n; ++i) {
01068         if (!data[i].isConj(data[i+n]) || !data[i+n].isConj()) return 0;
01069     }
01070     return 1;
01071 };
01072
01073 bool iTags::isAtensor(unsigned r) const {
01074     if (int(r)>=0) {
01075         if (r<3) wblog(FL,"ERR %s() r=%d !?",FCT,r);
01076         if (len!=r) return 0;
01077     }
01078     else if (len<3) { return 0; }
01079
01080     if (data[2].isConj()) return 0;
01081     if (data[0].isConj() && data[1].isConj()) return 0;
01082     return 1;
01083 };
01084
01085 unsigned iTags::Set(const char* F, int L,
01086     const char *tag, unsigned r,
01087     const char *tom,
01088     unsigned m,
01089     bool conjOM
01090 ){
01091     unsigned rval=0, i,l;
01092     if (!len && int(r)<=0 && int(m)<=0) { return rval; }
01093
01094     if (int(r)<0) {
01095         if (int(m)<=0) { r=len; } else
01096             if (m+2<len) { r=len-m; } else
01097                 wblog(FL,"ERR %s() requesting %d OM legs for rank-%d",FCT,m,len-m);
01098     } else
01099         if (r>len) wblog(FL,
01100             "ERR %s() rank out of bounds (%d/%d)",FCT,r,len);
01101
01102     if (int(m)<0) { if (tom) { m=len-r; } }
01103     else {
01104         if (m && r< 2) wblog(FL,"ERR %s() "
01105             "requesting %d OM leg%s for rank-%d",FCT,m,m==1?"":"s",r);
01106
01107         if ((l=r+m)<len) wblog(FL,
01108             "ERR %s() asking for lower rank (%d+%d / %d)",FCT,r,m,len);
01109         if (l>len) { this->Resize(l); }
01110     }
01111
01112     if (!tag || (r<len && int(m)>0 && !tom)) wblog(FL,
01113         "ERR %s() got null strings (%p / %p)",FCT,tag,tom);
01114     if (int(m)<0) { m=0; }
01115
01116     for (i=0; i<r; ++i) {
01117         if (data[i].isEmpty('m')) { ++rval;
01118             data[i].SetSI(FL,tag,i+1);
01119         }
01120     }
01121
01122     for (i=0; i<m; ++i) {
01123         if (!data[r+i]) { ++rval;
01124             data[r+i].SetSI(FL,tom,i+1, conjOM ? 1 : 0);
01125         }
01126         else wblog(FL,
01127             "ERR %s() non-empty OM itags (%s; %d+%d)",FCT,STR_(this),r,m);
01128     }
01129
01130     return rval;
01131 };
01132
01133 wstring iTags::toStr(char vflag) const {
01134
01135     unsigned k, n=ITAG_LEN;
01136     size_t l=0, N=(n*len>28 ? 2*n*len+8 : 64);
01137     char s[N];
01138
01139     if (vflag) { s[0]='{' ; s[1]=' ' ; l=2; }
01140     s[N-1]=0;
01141
01142     for (k=0; k<len; ++k) {
01143         if (l+4>N) wblog(FL,"ERR %s() "
01144             "string out of bounds (%s; %d*%d @ %d/%d)",FCT,s,len,n,l,N);
01145         if (k) { s[l++]=',' ; if (vflag) { s[l++]=' ' ; } }
01146         l+=data[k].to_str(s+l,N-1);
01147     }
01148     if (l>=N || (vflag && l+6>N)) wblog(FL,"ERR %s() "
01149         "string out of bounds (%s; %d*%d @ %d/%d)",FCT,s,len,n,l,N);
01150     if (vflag) { s[l]=' ' ; s[l+1]='}'; l+=2; }
01151     s[l]=0;
01152
01153     return s;

```

```

01154 };
01155
01156 int iTags::getCtr(const char *F, int L, const iTags &B,
01157   ctrIdx &ia, ctrIdx &ib,
01158   iTags *C,
01159   wvector<char> *ma_, wvector<char> *mb_
01160 ) const {
01161
01162   int nc=0;
01163   wvector<char> ma(len), mb(B.len);
01164   unsigned i,j, it=0, l=0, cflag=(ia.conj ^ ib.conj);
01165   int q, e=0;
01166
01167   for (; it<2; ++it) {
01168     q=(cflag ? SGN(+1) : SGN(-1));
01169
01170     for (i=0; i< len; ++i) {
01171       for (j=0; j<B.len; ++j) {
01172         if (SGN(data[i].sameAs(B.data[j]))==q) {
01173           if (!ma[i] && !mb[j])
01174             { ma[i]=j+1; mb[j]=i+1; ++nc; }
01175           else { ++e; }
01176         }
01177       }
01178     }
01179     if (nc || e) { break; }
01180     else { cflag=!cflag; }
01181   }
01182
01183   if (e) { unsigned n=64, e1=0, e2=0; char s[n];
01184     for (i= 0; i< len; ++i) {
01185       for (j=i+1; j< len; ++j) { if ( data[i]== data[j]) { ++e1; }}}
01186     for (i= 0; i<B.len; ++i) {
01187       for (j=i+1; j<B.len; ++j) { if (B.data[i]==B.data[j]) { ++e2; }}}
01188
01189     if (e1) {
01190       l=snprintf(s,n,"got non-unique itag%s in A",e1>1 || e2>1? "s":""");
01191       if (e2 && l<n) { l+=snprintf(s+l,n-l," and B"); }
01192     }
01193     else if (e2)
01194       { l=snprintf(s,n,"got non-unique itag%s in B",e2>1? "s":"""); }
01195     else { l=snprintf(s,n,"got non-unique itags"); }
01196
01197     wblog(F_L,"ERR %s() %s (e=%d, %d/%d)\n A: %s\n B: %s",
01198       FCT,s,e,e1,e2,STR_(this),STR(B));
01199   }
01200
01201   ia.wvector<unsigned>::init(nc);
01202   ib.wvector<unsigned>::init(nc);
01203
01204   if (nc) {
01205     for (i=0; i<len; ++i) { if (ma[i]) {
01206       ia[l]=i;
01207       ib[l]=ma[i]-1; ++l;
01208     }}
01209
01210     if (it) {
01211       wblog(F_L,"WRN contract::match() check missing conj flag\n"
01212         "A: %-15s -> %s\n"
01213         "B: %-15s -> %s", STR_(this), STR(ia), STR(B), STR(ib));
01214       mexWRN("applying conj(A) flag");
01215       ia.Conj();
01216     }
01217   }
01218   else if (F || !L) { wblog(F_L,
01219     "ERR %s() empty set of matching itags\n %s%s\n X %s%s", FCT,
01220     STR_(this), ia.conj ? "x":""",
01221     STR(B), ib.conj ? "x":""");
01222   }
01223
01224   if (C) { C->init( len + B.len - 2*nc );
01225   if (C->len) { l=0; itag_ *c=C->data;
01226
01227     for (i=0; i<len; ++i) { if (!ma[i]) {
01228       c[l]= data[i]; if (ia.conj) { c[l].Conj(); }; ++l;
01229     }}
01230     for (i=0; i<B.len; ++i) { if (!mb[i]) {
01231       c[l]=B.data[i]; if (ib.conj) { c[l].Conj(); }; ++l;
01232     }}
01233     if (l!=C->len) wblog(F_L,"ERR %s() %d/%d",FCT,l,C->len);
01234   }}
01235
01236   if (ma_) { ma.save2(*ma_); }
01237   if (mb_) { mb.save2(*mb_); }
01238
01239   return nc;
01240 };

```

```

01241
01242 int iTags::UpdateItagsCtr(const char *F, int L,
01243     const ctrIdx &ia, iTags &B,
01244     const ctrIdx &ib,
01245     const char *tag
01246 ){
01247     int q, s=0, e=0; unsigned i=0, k=0, l;
01248
01249     if (!ia || ia.len!=ib.len) wblog(F_L,
01250         "ERR %s() invalid usage (ia.len=%d/%d)",FCT,ia.len,ib.len);
01251     if (ia.len>15) wblog(F_L,
01252         "ERR %s() contracting %d>15 indices not supported",FCT,ia.len);
01253
01254     if (ia.conj) { this->Conj(); }
01255     if (ib.conj) { B.Conj(); }
01256
01257     for (; i<ia.len; ++i) {
01258         if (ia[i]>= len) wblog(F_L,
01259             "ERR %s() index out of bounds (A: %s / %d)",FCT,STR(ia), len);
01260         if (ib[i]>=B.len) wblog(F_L,
01261             "ERR %s() index out of bounds (B: %s / %d)",FCT,STR(ib),B.len);
01262
01263         itag_ &a=this->data[ia[i]], &b=B.data[ib[i]];
01264         q=a.sameAs(b,'1');
01265
01266         if (!q && (l=strlen(QS_ITAG_)) {
01267             if (!strcmp((const char*)a,QS_ITAG_,l)) { a.init_tag(); } else
01268             if (!strcmp((const char*)b,QS_ITAG_,l)) { b.init_tag(); }
01269             else { l=0; }
01270
01271             if (!l || !(q=a.sameAs(b,'1')) {
01272                 e+=l; continue;
01273             }
01274         }
01275
01276         if (i) { if (SGN(q)!=s) {
01277             e+=(1<<4); continue;
01278         } }
01279         else { s=SGN(q); }
01280
01281         if ((q=abs(q))==3) {
01282             if (a.isEmpty_m())
01283                 { q|=4; }
01284             else { continue; }
01285         }
01286
01287         if (q==1) { b.tSet(a); } else
01288         if (q==2) { a.tSet(b); } else
01289         if (q& 4) {
01290             unsigned n=9; char s[n];
01291             if (!tag) wblog(F_L,"ERR %s() got null tag (%p)",FCT,tag);
01292             l=snprintf(s,n,"%s%d",tag,++k);
01293             if (q>4 && l+1<n) {
01294                 char c=((char&)a)&127;
01295                 if (c) { s[l]=c; s[++l]=0; }
01296             }
01297             if (l>=n) { s[n-1]=0; wblog(F_L,
01298                 "ERR %s() string out of bounds (%s: %d/%d)",FCT,s,l,n); }
01299             a.Set(s); b.Set(s);
01300         }
01301         else wblog(F_L,"ERR %s() q=%d",FCT,q);
01302     }
01303
01304     if (s>=0) { e|=(1<<8); }
01305
01306     if (e && (F || L)) {
01307         sprintf(str,"%s' @ %s / '%s' @ %s (e=%d)",
01308             STR_(this), STR(ia), STR(B), STR(ib), e);
01309         if (e&15) wblog(F_L,"ERR %s() itag mismatch %s",FCT,str);
01310         else if (unsigned(e)>4&15) + 1 >= ia.len)
01311             wblog(F_L,"ERR %s() missing conj-flag %s", FCT,str);
01312         else wblog(F_L,"ERR %s() itag conj mismatch %s",FCT,str);
01313     }
01314
01315     if (!e) {
01316         FlagItagsCtr(F_L,ia,B,ib,0,-1);
01317     }
01318
01319     return e;
01320 };
01321
01322 int iTags::FlagItagsCtr(
01323     const char *F, int L, const ctrIdx &ia,
01324     iTags &B,
01325     const ctrIdx &ib,
01326     char all,
01327     char sgn

```

```

01328 ) {
01329
01330     int rval=0;
01331     unsigned i,j,l, id=0;
01332     wvector<char> ma(len), mb(B.len);
01333
01334     if (!ia.len || !ib.len) {
01335         if (!len && !B.len && !ia.len && !ib.len) { return rval; }
01336     }
01337
01338     if (sgn)
01339         { sgn=SGN(sgn); }
01340     else { sgn=((!ia.conj) ^ (!ib.conj) ? SGN(+1) : SGN(-1)); }
01341
01342     for (i=0; i<ma.len; ++i) {
01343         if (!data[i].isEmpty()) { ma[i]=1; }
01344         for (j=i+1; j<ma.len; ++j) {
01345             if (data[i]==data[j]) {
01346                 if (!(ma[i]&2)) { ma[i]=2; id+=1; }
01347                 if (!(ma[j]&2)) { ma[j]=2; id+=1; }
01348             }
01349         }
01350     }
01351     for (i=0; i<mb.len; ++i) {
01352         if (!B.data[i].isEmpty()) { mb[i]=1; }
01353         for (j=i+1; j<mb.len; ++j) {
01354             if (B.data[i]==B.data[j]) {
01355                 if (!(mb[i]&2)) { mb[i]=2; id+=256; }
01356                 if (!(mb[j]&2)) { mb[j]=2; id+=256; }
01357             }
01358         }
01359     }
01360
01361     if (ia.len!=ib.len) wblog(FL,
01362         "ERR %s() invalid pair of ctr-indices %s / %s",FCT,STR(ia),STR(ib));
01363     for (i=0; i<ia.len; ++i) {
01364         if (ia[i]>ma.len || ma[ia[i]]>3) wblog(F_L,
01365             "ERR %s() invalid ctr-index ica = %s (r=%d)",FCT,STR(ia),ma.len);
01366         ma[ia[i]]+=((1+ib[i])<<2);
01367     }
01368     for (i=0; i<ib.len; ++i) {
01369         if (ib[i]>mb.len || mb[ib[i]]>3) wblog(F_L,
01370             "ERR %s() invalid ctr-index icb = %s (r=%d)",FCT,STR(ib),mb.len);
01371         mb[ib[i]]+=((1+ia[i])<<2);
01372     }
01373
01374     for (j=0; j<B.len; ++j) { if (mb[j]<4 && ((mb[j]&1) || all)) {
01375     for (i=0; i< len; ++i) { if (ma[i]<4 && ((ma[i]&1) || all) &&
01376         SGN(data[i].sameAs(B.data[j]))==sgn) {
01377         B.data[j].SetFlags(-1);
01378         if (mb[j]&2) { mb[j]-=2; id-=256; }
01379         ++rval; break;
01380     } }
01381 }}
01382
01383     if (int(id)<=0) {
01384         if (id) wblog(FL,"ERR %s() got id=%d !?",FCT,id);
01385         return rval;
01386     }
01387
01388     for (i=0; i<ia.len; ++i) { j=ia[i]; l=ib[i];
01389         if (l>=B.len) wblog(FL,
01390             "ERR %s() index out of bounds (%d/%d)",FCT,l,B.len);
01391         if (ma[j]&2 || mb[l]&2) {
01392             if (SGN(data[j].sameAs(B.data[l]))!=sgn) wblog(FL,
01393                 "ERR %s() invalid ctr-indices %s / %s (sgn=%d)",
01394                 FCT,STR(this),STR(B),sgn);
01395
01396             data[j].SetFlags(-2); if (ma[j]&2) { ma[j]-=2; id-= 1; }
01397             B.data[l].SetFlags(-1); if (mb[l]&2) { mb[l]-=2; id-=256; }
01398             rval+=2;
01399         }
01400     }
01401
01402     if (int(id)<=0) {
01403         if (id) wblog(FL,"ERR %s() got id=%d !?",FCT,id);
01404         return rval;
01405     }
01406
01407     for (i=0; i<len; ++i) {
01408         for (j=i+1; j<len; ++j) { if (data[i]==data[j]) {
01409             if (ma[i]>=4 || ma[j]>=4) {
01410                 wblog(FL,"ERR %s() A: (%d,%d) %d/%d",FCT, i,ma[i],ma[j]); }
01411             if ((ma[j]&1) || all) { data[j].SetFlags(-1); }
01412         } }
01413     }
01414

```

```

01415     for (i=0; i<B.len; ++i) {
01416         for (j=i+1; j<B.len; ++j) { if (B.data[i]==B.data[j]) {
01417             if (mb[i]>=4 || mb[j]>=4) {
01418                 wblog(FL,"ERR %s() B: (%d,%d) %d/%d",FCT, i,mb[i],mb[j]); }
01419             if ((mb[j]&1) || all) { B.data[j].SetFlags(-1); }
01420         }}
01421     }
01422
01423 #ifndef WB_SKIP_ASSERT
01424     if (all) { ctrIdx ia_,ib_;
01425         this->getCtr(F,L,B,ia_,ib_);
01426         if (ia_.len!=ib_.len) { wblog(F_L,"ERR %s() ctr-index mismatch\n"
01427             "    A: %s | %s -> %s\n"
01428             "    B: %s | %s -> %s", FCT,
01429             STR_(this),STR(ia),STR(ia_), STR(B),STR(ib),STR(ib_));
01430         }
01431     }
01432 #endif
01433     return rval;
01434 };
01435
01436 int iTags::findRegEx(const char *pat) const {
01437     regex_t r; int e, k=-1;
01438
01439     unsigned i, m=0, l=ITAG_LEN+2;
01440     char s[l+1]; s[l]=0;
01441     wbsttring ptx;
01442
01443     if (!pat || !pat[0]) wblog(FL,
01444         "ERR %s() got invalid regexp (empty)",FCT);
01445
01446     if (strstr(pat,"\\d") && strlen(pat)<32) { unsigned j=0; str[0]=0;
01447         for (i=0; pat[i]; ++i) {
01448             if (pat[i]=='\\' && pat[i+1]=='d') { strcpy(str+j,"[0-9]"); j+=5; ++i; }
01449             else if (pat[i]=='[') {
01450                 if (strstr(pat+i,"\\d")) wblog(FL,
01451                     "WRN failed to translate non-posix \\d -> [0-9]\n"
01452                     "in itag regexp '%s'",FCT,pat);
01453                 strcpy(str+j,pat+i); break;
01454             }
01455             else { str[j++]=pat[i]; }
01456         }
01457         str[j]=0; ptx=str;
01458         pat=ptx.data;
01459     }
01460
01461     if ((e=regcomp(&r,pat, REG_EXTENDED | REG_NOSUB)) {
01462         return -1;
01463     }
01464
01465     for (i=0; i<len; ++i) {
01466         data[i].to_str(s,l);
01467         if (!regexexec(&r,s,0,NULL,0)) {
01468             if (!m || !data[k].sameAs(data[i])) { k=i; ++m; }
01469         }
01470     }
01471     regfree(&r);
01472
01473     if (m>1) {
01474         return -m;
01475     }
01476     return (k+1);
01477 };
01478
01479 int iTags::RegEx_replace(const char *F, int L, const char *rstr) const {
01480     int nrep=0;
01481     if (!rstr || !rstr[0]) { return nrep; }
01482     if (strncmp(rstr,"s/",2) || !rstr[2]) wblog(F_L,
01483         "ERR %s() invalid regex '%s' (expecting s/*/*/*)",FCT,rstr);
01484
01485     unsigned j, i=0, i1=0, i2=0, e=0;
01486     char icafe=0, gflag=0; const char *s=rstr+2;
01487
01488     for (; s[i]; ++i) { if (s[i]=='/') { i1=(++i); break; }}
01489     for (; s[i]; ++i) { if (s[i]=='/') { i2=(++i); break; }}
01490     for (; s[i]; ++i) {
01491         if (s[i]=='i') { ++icafe; } else
01492         if (s[i]=='g') { ++gflag; } else { break; }
01493     }
01494     if (!i1 || !i2 || s[i] || icafe>1 || gflag>1) wblog(F_L,
01495         "ERR %s() invalid regex '%s' (expecting s/*/*/*)",FCT,rstr);
01496
01497     char rst_[strlen(rstr)+1];

```

```

01502     strcpy(rst_,rstr+2); rst_[i1-1]=rst_[i2-1]=0;
01503
01504     wbsting t_(ITAG_LEN+1);
01505     char c, *t=t_.data; t[ITAG_LEN]=0;
01506
01507     for (i=0; i<len; ++i) { char *s=(char *) (data+i);
01508         memcpy(t,s,ITAG_LEN);
01509         for (j=0; j<ITAG_LEN; ++j) { t[j] &= 127; }
01510
01511         if (t_.RegEx_replace(F_L,rst_,rst_+i1,icase,gflag)) { ++nrep;
01512             t=t_.data;
01513             for (j=0; j<ITAG_LEN && t[j]; ++j) {
01514                 c=(t[j]&127); if (c!=t[j]) { ++e; }
01515                 s[j] = (s[j]&128) + c;
01516             }
01517             if (e || (j==ITAG_LEN && t[j])) wblog(FL,"ERR %s() "
01518                 "new itag out of bounds (%s; %d/%d)",FCT,t,strlen(t),ITAG_LEN);
01519             for (; j<ITAG_LEN; ++j) { s[j] &= 128; }
01520         }
01521     }
01522     return nrep;
01523 };
01524
01525 template <class T>
01526 sparseIndex2D<T>& sparseIndex2D<T>::Setup(char use_rmaj) {
01527
01528     if (IJ.dim2!=2) wblog(FL,
01529         "ERR %s() got invalid IJ (%s)",FCT,IJ.sizeStr().data);
01530     if (S.len!=2) wblog(FL,
01531         "ERR %s() got invalid S (len=%d)",FCT,S.len);
01532
01533     const T *s=S.data;
01534
01535     if (IJ.dim1<=1) { P.init(IJ.dim1);
01536         cIdx.init(s[1]+1);
01537         if (IJ.dim1) { T *idx=IJ.data;
01538             if (idx[0]>=s[0] || idx[1]>=s[1]) wblog(FL,
01539                 "ERR %s() index out of bounds (%ld,%ld) having (%dx%d) !?",
01540                 FCT,idx[0],idx[1],s[0],s[1]);
01541             for (T i=idx[1]+1; i<=s[1]; ++i) cIdx[i]=1;
01542         }
01543         else P.init();
01544         return *this;
01545     }
01546
01547     T j=0, l=0, lmax=IJ.dim1-1, *idx=IJ.data;
01548
01549     if (!use_rmaj || s[0]>128 || (s[0]>1 && s[1]<(1<<24))) {
01550         cIdx.init(s[1]+1); rmaj=0;
01551         IJ.SortRecs(P,+1, -1);
01552
01553         while (1) {
01554             for (; j<=idx[1]; ++j) { cIdx[j]=1; }
01555             for (; l<lmax && idx[1]==idx[3]; ++l, idx+=2) {
01556                 if (idx[0]>=idx[2]) wblog(FL,
01557                     "ERR %s() IJ not sorted or unique!",FCT);
01558                 ++l; break; }
01559
01560             if (idx[1]>idx[3]) wblog(FL,"ERR %s() IJ(:,2) not sorted!",FCT);
01561             if (idx[0]>=s[0]) wblog(FL,
01562                 "ERR %s() index out of bounds (%ld/%ld) !?",FCT,idx[0],s[0]);
01563             ++l; idx+=2;
01564         }
01565     }
01566     else {
01567         cIdx.init(s[0]+1); rmaj=1;
01568         IJ.SortRecs(P,+1, +1);
01569
01570         while (1) {
01571             for (; j<=idx[0]; ++j) { cIdx[j]=1; }
01572             for (; l<lmax && idx[0]==idx[2]; ++l, idx+=2) {
01573                 if (idx[1]>=idx[3]) wblog(FL,
01574                     "ERR %s() IJ not sorted or unique!",FCT);
01575                 ++l; break; }
01576
01577             if (idx[0]>idx[2]) wblog(FL,"ERR %s() IJ(:,2) not sorted!",FCT);
01578             if (idx[1]>=s[1]) wblog(FL,
01579                 "ERR %s() index out of bounds (%ld/%ld) !?",FCT,idx[1],s[1]);
01580             ++l; idx+=2;
01581         }
01582     }
01583
01584     for (; j<cIdx.len; ++j) { cIdx[j]=1; }
01585
01586     return *this;
01587 };
01588

```

```

01589 template <class T>
01590 inline T sparseIndex2D<T>::cidx(T j) const {
01591 #ifndef WB_SKIP_ASSERT
01592     if (j>=cIdx.len) wblog(FL,
01593         "ERR %s() index out of bounds (%d/%d)",FCT,j,cIdx.len);
01594 #endif
01595     return cIdx.data[j];
01596 };
01597
01598 template <class T>
01599 inline T sparseIndex2D<T>::ridx(T ic) const {
01600 #ifndef WB_SKIP_ASSERT
01601     if (ic>=IJ.dim1) wblog(FL,
01602         "ERR %s() index out of bounds (%d/%d)",FCT,ic,IJ.dim1);
01603 #endif
01604     return IJ.data[2*ic];
01605 };
01606
01607 #endif
01608

```

5.37 wbindex.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace index routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_INDEX_HH__
00021 #define __WB_INDEX_HH__
00022
00023 // ----- //
00024 // index class
00025 // Wb, Apr19, 06
00026 // ----- //
00027
00028 class wbindex : public wbvector<widx_t> {
00029 public:
00030
00031     wbindex(widx_t n=0, char iflag=0) {
00032         if (iflag) Index(n); else init(n);
00033     };
00034
00035     wbindex(const ctrIdx& ic) { init(ic); };
00036
00037     explicit wbindex(widx_t n, const widx_t* d)
00038     : wbvector<widx_t>(n,d) {};
00039
00040     explicit wbindex(const char* s, unsigned offset=0) {
00041         int i=Str2Idx(s,*this,offset);
00042         if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
00043     };
00044
00045     wbindex& init(widx_t n=0, const widx_t *d=NULL) {
00046         wbvector<widx_t>::init(n,d);
00047         return *this;
00048     };
00049
00050     wbindex& init(const ctrIdx &ic);
00051
00052     wbindex& init(
00053         const char *F, int L, const mxArray *a, widx_t offset=0);
00054
00055     widx_t init(
00056         const char *F, int L, const mxArray *a,
00057         const char *istr, char check_type=1
00058     ){ return wbvector<widx_t>::init(F,L,a,istr,check_type); };
00059
00060

```

```

00061     wbindex& init(const mxArray *a, widx_t offset=0) {
00062         return init(0,0,a,offset);
00063     };
00064
00065     wbindex& initGroup(
00066         const wbvector<widx_t> &P, wbvector<widx_t> &D
00067     ){
00068         widx_t i,d, k=0, ig=0, m=D.sum();
00069         if (m!=P.len) wblog(FL,
00070             "ERR %s() invalid group index (%d/%d)",FCT,m,P.len);
00071         init(P.len+D.len);
00072
00073         for (; ig<D.len; ++ig) {
00074             data[k+ig]=d=D[ig]; m=ig+1;
00075             for (i=0; i<d; ++i, ++k) data[k+m]=P[i];
00076         }
00077         return *this;
00078     };
00079
00080
00081     wbindex& initGroup0(
00082         const wbvector<widx_t> &P, wbvector<widx_t> &D
00083     ){
00084         widx_t l=0, i=0; init(D.len);
00085         for (; i<D.len; ++i) {
00086             if (l>=P.len) wblog(FL, "ERR %s() index out of bounds",FCT,l,P.len);
00087             data[i]=P[l]; l+=D[i];
00088         }
00089         return *this;
00090     };
00091
00092     void initStr(const char* s, widx_t offset=0) {
00093         int i=Str2Idx(s,*this,offset);
00094         if (i<=0) wblog(FL, "ERR %s() invalid index '%s' (%d)",FCT,s,i);
00095     };
00096
00097     bool isIndex(widx_t n=0) {
00098         if (n) {
00099             if (swidx_t(n)>0) {
00100                 for (widx_t i=0; i<len; ++i) {
00101                     if (data[i]!=(n+i)) return 0; }
00102             }
00103             else {
00104                 n=(1-n-len); if (swidx_t(n)<0) return 0;
00105                 for (widx_t i=0; i<len; ++i) {
00106                     if (data[i]!=(n+i)) return 0; }
00107             }
00108         }
00109         else {
00110             for (widx_t i=0; i<len; ++i) { if (data[i]!=i) return 0; }
00111         }
00112         return 1;
00113     };
00114
00115     wbindex& Index(widx_t n) {
00116         init(n);
00117         if (n) { for (widx_t i=0; i<n; ++i) data[i]=i; }
00118         return *this;
00119     };
00120
00121     wbindex& Index_ex(widx_t n, widx_t ix) {
00122         widx_t i,k;
00123
00124         if (ix>=n) wblog(FL,
00125             "WRN index to be excluded of no relevance (%d/%d)",ix,n);
00126
00127         init(ix<n ? n-1 : n); if (n==0) return *this;
00128
00129         for (k=i=0; i<n; i++) if (i!=ix) data[k++]=i;
00130
00131         return *this;
00132     };
00133
00134     wbindex& Index(widx_t i1, widx_t i2) {
00135         if (i1>i2) { init(); return *this; }
00136         init(i2-i1+1);
00137         for (widx_t i=0; i<len; ++i) data[i]=i+i1;
00138
00139         return *this;
00140     };
00141
00142     wbindex& Index(widx_t i1, widx_t i2, int p) {
00143         if (i1>i2) { init(); return *this; }
00144
00145         widx_t n=i2-i1+1; p=(-p)%int(n);
00146         if (!p || n==1) { return Index(i1,i2); }
00147         if (p<0) p+=n;

```



```

00148
00149     init(n); {
00150         widx_t i, ix=i1+p, *d=data;
00151         for (i=ix; i<=i2; ++i, ++d) { (*d)=i; }
00152         for (i=i1; i< ix; ++i, ++d) { (*d)=i; }
00153     }
00154     return *this;
00155 };
00156
00157 wbindex& BlockIndex(const wbvector<widx_t> &D, char sub=0) {
00158     widx_t i,j,d,l; init(D.sum());
00159
00160     if (sub==0) {
00161         for (l=i=0; i<D.len; ++i)
00162             for (d=D[i], j=0; j<d; ++j,++l) data[l]=i;
00163     }
00164     else {
00165         for (l=i=0; i<D.len; ++i)
00166             for (d=D[i], j=0; j<d; ++j,++l) data[l]=j;
00167     }
00168
00169     return *this;
00170 };
00171
00172 wbindex& BlockIndex(widx_t n, widx_t D) {
00173     widx_t i,j,l; init(n*D);
00174
00175     for (l=i=0; i<n; ++i)
00176         for (j=0; j<D; ++j,++l) data[l]=i;
00177
00178     return *this;
00179 };
00180
00181 wbindex& BlockIndexI(widx_t n, widx_t D) {
00182     widx_t i,j,l; init(n*D);
00183
00184     for (l=j=0; j<D; ++j)
00185         for (i=0; i<n; ++i,++l) data[l]=i;
00186
00187     return *this;
00188 };
00189
00190 wbindex& IndexTranspose(widx_t m, widx_t n) {
00191     widx_t i,j,l;
00192     init(n*m); if (!n || !m) return *this;
00193
00194     for (l=j=0; j<m; ++j) {
00195         data[l++]=j;
00196         for (i=1; i<n; ++i,++l) data[l]=data[l-1]+m;
00197     }
00198
00199     return *this;
00200 };
00201
00202 wbindex& operator= (const char* s) {
00203     int i=Str2Idx(s,*this,0);
00204     if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
00205     return *this;
00206 };
00207
00208 int extend2Perm(widx_t N, wbperm &P, char toend=0) const;
00209
00210 void toPerm(wbperm &P) const {
00211     P.initT(len, data);
00212
00213     if (!P.isValidPerm()) { print("this");
00214         wblog(FL, "WRN No valid perm."); }
00215 };
00216
00217 wbindex& invert(widx_t N, wbindex &I2, char uflag=1) const;
00218
00219 wbindex& Invert(widx_t N, char uflag=1) {
00220     wbindex X; save2(X);
00221     return X.invert(N,*this,uflag);
00222 };
00223
00224 wbindex flipIdx(widx_t ndim) const;
00225 char isUnique(widx_t lmax, widx_t offset=0) const;
00226
00227 mxArray* toMx(const char tflag=0) const {
00228     return toMx_offset(1,tflag);
00229 };
00230
00231 protected:
00232 private:
00233
00234 };

```

```

00235
00236 wbpem::wbpem (const wbindx &P) {
00237     RENEW(P.len,P.data);
00238     if (!isValidPerm()) { dispInvalidPerm(FL); }
00239 };
00240
00241 class wbindx : public wbvector<widx_t> {
00242
00243 public:
00244
00245     wbindx() : wbvector<widx_t>() {};
00246     wbindx(const wbvector<widx_t> &S) { init(S); };
00247
00248     template<class TI>
00249     wbindx(widx_t n, const TI *S) { init(n,S); };
00250
00251     wbindx(const wbindx &I) : wbvector<widx_t>() {
00252         if (!I.isEmpty()) {
00253             wbvector<widx_t>::init(I.len); SIZE=I.SIZE;
00254             if (len) --data[0];
00255         }
00256     };
00257
00258     wbindx& init() {
00259         wbvector<widx_t>::init(); SIZE.init();
00260         return *this;
00261     };
00262
00263     wbindx& init(const wbvector<widx_t> &S, char pp=1) {
00264         const widx_t* const &s=S.data;
00265         for (widx_t k=0; k<S.len; k++) if (!s[k]) {
00266             wbvector<widx_t>::init(); SIZE.init();
00267             return *this;
00268         }
00269
00270         wbvector<widx_t>::init(S.len); SIZE=S;
00271
00272         if (len) {
00273             if (pp)
00274                 { --data[0]; }
00275             else { data[len-1]=SIZE[len-1]; }
00276         }
00277         return *this;
00278     };
00279
00280     template<class TI>
00281     wbindx& init(unsigned n, const TI *S) {
00282         for (unsigned k=0; k<n; ++k) if (!S[k]) {
00283             wbvector<widx_t>::init(); SIZE.init();
00284             return *this;
00285         }
00286         wbvector<widx_t>::init(n); SIZE.initT(n,S);
00287         if (len) { --data[0]; }
00288
00289         return *this;
00290     };
00291
00292     wbindx& reset(char pp=1) {
00293         if (len!=SIZE.len) wblog(FL,
00294             "ERR %s() severe size mismatch (%d/%d)",FCT,len,SIZE.len);
00295
00296         if (len) { wbvector<widx_t>::set(0);
00297             if (pp)
00298                 { --data[0]; }
00299             else { data[len-1]=SIZE[len-1]; }
00300         }
00301         return *this;
00302     };
00303
00304     wbindx& set(wbvector<widx_t> &I) {
00305         widx_t k=0, *const &i=I.data, *const &s=SIZE.data;
00306         if (I.len!=len) wblog(FL,
00307             "ERR wbindx::%s() size mismatch (%d/%d)",FCT,I.len,len);
00308         for (; k<len; k++) if (i[k]>=s[k]) {
00309             wblog(FL,"ERR wbindx::%s() index out of bounds (%d: %d/%d)",
00310                 FCT,k+1,i[k],s[k]);
00311             data[k]=i[k];
00312         }
00313         return *this;
00314     };
00315
00316     bool isValid() const {
00317         if (len!=SIZE.len) return 0;
00318         for (widx_t i=0; i<len; ++i) if (data[i]>=SIZE.data[i]) return 0;
00319         return 1;
00320     };
00321

```

```

00322 void checkValid(const char *F, int L) const {
00323     if (!isValid()) wblog(F_L,
00324         "ERR %s() invalid hyperindex\n%s", FCT, STR_(this));
00325 };
00326
00327 bool operator++() {
00328     widx_t k=0, l=len-1, *const &s=SIZE.data;
00329
00330     if (len!=SIZE.len) wblog(F_L,
00331         "ERR wbIndex::++ size mismatch (len=%d/%d)", len, SIZE.len);
00332     if (!len) { return 0; }
00333
00334     k=0; ++data[0];
00335     while (data[k]==s[k] && k<l) { data[k]=0; ++data[++k]; }
00336
00337     if (data[k]>=s[k]) {
00338         if (k!=l) wblog(F_L, "WRN wbIndex::++ "
00339             "premature stop (%d/%d: %s)", k+1, len, STR_(this));
00340         return 0;
00341     }
00342     return 1;
00343 };
00344
00345 bool operator--() {
00346     widx_t k=0, l=len-1, *const &s=SIZE.data;
00347
00348     if (len!=SIZE.len) wblog(F_L,
00349         "ERR wbIndex::-- size mismatch (%d/%d)", len, SIZE.len);
00350     if (!len) { return 0; }
00351
00352     k=0; data[0]--;
00353     while (swidx_t(data[k])==-1 && k<l) { data[k]=s[k]-1; --data[++k]; }
00354
00355     if (data[k]>=s[k]) {
00356         if (k!=l) wblog(F_L, "WRN wbIndex::-- "
00357             "premature stop (%d/%d: %s)", k+1, len, STR_(this));
00358         return 0;
00359     }
00360     return 1;
00361 };
00362
00363 bool isEmpty() const {
00364     if (len!=SIZE.len) wblog(F_L, "ERR %d/%d", len, SIZE.len);
00365     if (!len) return 1;
00366     else {
00367         const widx_t *s=SIZE.data;
00368         for (widx_t i=0; i<len; i++) if (!s[i]) return 1;
00369     }
00370     return 0;
00371 };
00372
00373 bool gotmore() const {
00374     if (len!=SIZE.len) wblog(F_L, "ERR %d/%d", len, SIZE.len);
00375     for (widx_t i=0; i<len; ++i) {
00376         if (data[i]+1<SIZE.data[i]) return 1; else
00377         if (data[i]>=SIZE.data[i]) wblog(F_L,
00378             "WRN %s() index out of bounds (%s)", FCT, STR_(this)
00379         );
00380     }
00381     return 0;
00382 };
00383
00384 bool isdiag() const {
00385     if (len%2) wblog(F_L, "ERR %s() applies to "
00386         "even-rank objects only (%s)", FCT, SSTR_(this));
00387     for (widx_t r=len/2, i=0; i<r; ++i) {
00388         if (data[i]!=data[i+r]) { return 0; }
00389     }
00390     return 1;
00391 };
00392
00393 widx_t numel() const { return SIZE.prod(0); };
00394
00395 widx_t serial(const widx_t *S=NULL) {
00396     widx_t l, k=len-1;
00397     const widx_t *s=(S ? S : SIZE.data);
00398
00399     if (!len) wblog(F_L, "ERR wbIndex::%s() is empty", FCT);
00400     for (l=data[k--]; k<len; --k) { l = l*s[k] + data[k]; }
00401     return l;
00402 };
00403
00404 wbststring sizeStr() const { return SIZE.toStrf("", "x"); };
00405 wbststring toStr() const {
00406     char s[128]; snprintf(s, 128, "[%s] %s",
00407         wbstvector<widx_t>::toStr().data, SSTR(SIZE));
00408     return s;

```

```

00409     };
00410
00411     mxArray* toMx() const {
00412         const char *fields[]={"S","idx"};
00413         mxArray *S=mxCreateStructMatrix(1,1,2,fields);
00414
00415         mxSetFieldByNumber(S,0,0, SIZE.toMx());
00416         mxSetFieldByNumber(S,0,1, ((*this)+1).toMx());
00417         return S;
00418     };
00419
00420     wbvector<widx_t> SIZE;
00421
00422 protected:
00423 private:
00424
00425 };
00426
00427 template <class T=widx_t>
00428 class groupIndex {
00429
00430 public:
00431
00432     groupIndex() {};
00433
00434     groupIndex(
00435         const wbvector<T> &P_, const wbvector<T> &D_,
00436         const wperm_t **p=NULL
00437     ){ P=P_; D=D_; setup(); if (p) (*p)=P.data; };
00438
00439     groupIndex(const wbvector<T> &DI, T ng=-1) { init(DI,ng); }
00440
00441     void init() { P.init(); D.init(); IO.init(); };
00442
00443     groupIndex& init(
00444         const wbvector<T> &P_, const wbvector<T> &D_
00445     ){ P=P_; D=D_; setup(); return *this; };
00446
00447     groupIndex& initX(
00448         wbvector<wperm_t> &P_, wbvector<T> &D_,
00449         const wperm_t **p=NULL
00450     ){
00451         P_.save2(P); D_.save2(D); setup(); if (p) (*p)=P.data;
00452         return *this;
00453     };
00454
00455     groupIndex& init(const wbvector<T> &DI, T ng=-1) {
00456         if (DI.isEmpty()) { init(); return *this; }
00457
00458         if (long(ng)<0) {
00459             T n=0,l=0;
00460             for (; l<DI.len; ++n, l+=(1+DI[l]));
00461             if (l!=DI.len) wblog(FL,
00462                 "ERR %s() invalid group index (%d/%d)",FCT,l,DI.len);
00463             ng=n;
00464         }
00465         P.init(DI.len-ng); D.init(ng);
00466
00467         T i,d,k=0,l=0; T *dp=P.data;
00468         while (l<DI.len) { d=DI.data[l++];
00469             for (i=0; i<d; ++i, ++k, ++l) { dp[k]=DI.data[l]; }
00470         }
00471         if (k!=P.len || l!=DI.len) wblog(FL,
00472             "ERR %s() %d/%d %d/%d",FCT,k,P.len,l,DI.len);
00473         setup(); return *this;
00474     };
00475
00476     groupIndex& init(T ng) {
00477         D.init(ng); IO.init(ng+1); P.init();
00478         return *this;
00479     };
00480
00481     groupIndex& init_g(T ig, T nz) {
00482         if (ig>D.len || ig+1>=IO.len) wblog(FL,"ERR %s() "
00483             "index out of bounds (%d/%d,%d)",FCT,ig,D.len,IO.len-1);
00484         if (ig==0) IO[ig]=0;
00485         IO[ig+1]=nz; D[ig]=nz-IO[ig];
00486         return *this;
00487     };
00488
00489     groupIndex& setD(T i, T nz) {
00490         if (i>=D.len) wblog(FL,"ERR %s() "
00491             "index out of bounds (%d/%d)",FCT,i,D.len);
00492         D[i]=nz; return *this;
00493     };
00494
00495     T init_ic() {

```

```

00496         if (I0.len!=D.len+1) wblog(FL,"ERR %s() "
00497             "I0 not yet initialized !? (%d/%d+1)",FCT,I0.len,D.len+1);
00498         T i=0; I0[0]=0;
00499         for (; i<D.len; ++i) { I0[i+1]=I0[i]+D[i]; }
00500         return I0[i];
00501     };
00502
00503     groupIndex& setup(char full=0) {
00504         T n=D.cumsum_(I0,'N'); if (n!=P.len) wblog(FL,
00505             "ERR %s() severe size mismatch (%d/%d)",FCT,n,P.len);
00506         if (full) { Ig.init(P.len);
00507             for (T l=0, j=0, d=0, i=0; i<D.len; ++i, l+=d) {
00508                 for (d=D[i], j=0; j<d; ++j) { Ig[P[l+j]]=i; }
00509             }
00510         return *this;
00511     };
00512
00513     T dim(T ig) {
00514         if (ig>=D.len) wblog(FL,
00515             "ERR %s() index out of bounds (%d/%d)",FCT,ig,D.len);
00516         return D.data[ig];
00517     };
00518
00519     T numGroups() const { return D.len; };
00520
00521     void getGroup(T ig, T* &p0, T &d) const {
00522         if (ig>=D.len || I0.len!=D.len+1) wblog(FL,
00523             "ERR %s() index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00524         p0=P.data+I0.data[ig];
00525         d=D.data[ig];
00526     };
00527
00528     T getGroup(T ig, T &j, T &j2) const {
00529         if (long(ig)==-1) return 0;
00530         if (ig>=D.len || I0.len!=D.len+1) wblog(FL,
00531             "ERR %s() index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00532         j =I0.data[ig ];
00533         j2=I0.data[ig+1]; return ig+1;
00534     };
00535
00536     T getGroup(T ig) const {
00537         if (long(ig)==-1) return 0;
00538         if (ig>=I0.len) wblog(FL,"ERR %s() "
00539             "index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00540         return I0.data[ig];
00541     };
00542
00543     template <class T2>
00544     void getRecs0(const wbMatrix<T2> &, wbMatrix<T2> &, T m) const;
00545
00546     T groupIter(T &ig, T &j, T &j2) const {
00547         if (long(ig)>=0) { ++ig; } else { ig=0;
00548             if (I0.len!=D.len+1) wblog(FL,"ERR %s() "
00549                 "index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00550         }
00551         if (ig<D.len) {
00552             j =I0.data[ig ];
00553             j2=I0.data[ig+1];
00554         }
00555         else {
00556             ig=-1;
00557         }
00558
00559         return ig+1;
00560     };
00561
00562     mxArray* toMx() const {
00563         const char *fields[]={"P","D","I0","pp"};
00564         mxArray *S=mxCreateStructMatrix(1,1,4,fields);
00565
00566         mxSetFieldByNumber(S,0,0,(P+1).toMx());
00567         mxSetFieldByNumber(S,0,1,D.toMx());
00568         mxSetFieldByNumber(S,0,2,(I0+1).toMx());
00569
00570         wbvector<T> p;
00571         mxArray *c=mxCreateCellMatrix(1,D.len);
00572
00573         for (T i=0; i<D.len; ++i) {
00574             p.init(D[i],P.data+I0[i]);
00575             mxSetCell(c,i,p.toMx());
00576         }
00577         mxSetFieldByNumber(S,0,3,c);
00578
00579         return S;
00580     };
00581
00582     wbvector<T> D;

```

```

00583     wbpem P;
00584
00585     wvector<T> IO;
00586
00587     wvector<T> Ig;
00588
00589     protected:
00590     private:
00591 };
00592
00593 #define IDT unsigned long
00594 #define ITAG_LEN sizeof(IDT)
00595
00596 #define CC_ITAG '*'
00597
00598 #define QS_MARK_DUAL '\\"
00599 #define IS_CHAR_MARK(c) ((c)>=32 && (c)<= QS_MARK_DUAL)
00600
00601 #define IS_ITAGS_SEP(c) ((c)==' ' || (c)==';' || (c)=='|')
00602 #define IS_CHAR_ITAGS(c) ((c)>32 && (c)<127)
00603 #define IS_CHAR_ITAG(c) (((c)>32 && (c)<127) && (c)!=CC_ITAG && !IS_ITAGS_SEP(c))
00604
00605 #define IT2STR(a) (a).itags.toStr().data
00606 #define IT2STR__ itags.toStr().data
00607
00608 class itag_ {
00609
00610 public:
00611
00612     itag_() : t(0) {};
00613     itag_(const char *F, int L, const char *s) { init(F,L,s); };
00614     itag_(const char *s) { init(FL,s); };
00615     itag_(const itag_ &x) : t(x.t) {};
00616
00617     itag_& init(const char* F, int L, const char *s, unsigned n=-99);
00618
00619     itag_& init(const char *s=NULL) { return init(FL,s); };
00620
00621     itag_& init_tag() {
00622         char *s = (char*)&t, c128=char(128);
00623         unsigned k=0, n=ITAG_LEN; for (; k<n; ++k) { s[k]&=c128; }
00624         return *this;
00625     };
00626
00627     itag_& init_qdir(char c) { t=0;
00628         if (c=='-' || c==CC_ITAG) Conj();
00629         else if (c!='+') wblog(FL,"ERR %s() invalid c=%c%d>",FCT,c,c);
00630         return *this;
00631     };
00632
00633     template<class TQ, class TD>
00634     itag_& init(const char *F, int L,
00635         const wvector< QSpace<TQ,TD> > &A, windex ia);
00636
00637 #ifndef NOMEX
00638     itag_& init(const char* F, int L, const mxArray *a);
00639 #endif
00640
00641     itag_& operator=(unsigned n) { t=n; return *this; }
00642     itag_& operator=(const char *s) { return init(FL,s); }
00643
00644     bool operator==(const itag_ &x) const { return t==x.t; };
00645     bool operator!=(const itag_ &x) const { return t!=x.t; };
00646
00647     char sameAs(const itag_ &x, char lflag=0) const;
00648
00649     bool isConj(const itag_ &x, char full=1) const {
00650         IDT c128=128;
00651         return (full ? (t^c128)==x.t : (t&c128) ^ (x.t & c128) );
00652     };
00653
00654     bool isConj_x(const itag_ &x, char lflag) const;
00655     int isValid() const;
00656
00657     bool isEmpty() const {
00658         return (t & ~(IDT)(128) ? 0 : 1);
00659     };
00660
00661     bool isEmpty(char lenient) const;
00662     bool isEmpty_m() const;
00663
00664     char getMark() const {
00665         char c=((char&t) & 127);
00666         if (c && !IS_CHAR_MARK(c)) { c=0; }
00667         return c;
00668     };
00669

```

```

00670     explicit operator bool() const { return (t!=0); };
00671     bool operator! () const { return !t; };
00672
00673     explicit operator const char*() const {
00674         return (const char*)(&t);
00675     };
00676
00677     itag_& Set(const char *s) { char c=isConj();
00678         init(FL,s);
00679         if (isConj()) wblog(FL,
00680             "ERR %s() unexpected usage (%s; %d)",FCT,STR_(this),c);
00681         if (c) Conj();
00682         return *this;
00683     };
00684
00685     itag_& tSet(const itag_&x) {
00686         if (isConj() ^ x.isConj()) { t=x.t; Conj(); } else { t=x.t; }
00687         return *this;
00688     };
00689
00690     itag_& Set_(const itag_&x, char keep=0) {
00691         if (isEmpty() && !keep) { t=x.t; } else
00692         if (isConj() ^ x.isConj()) { Conj(); }
00693         return *this;
00694     };
00695
00696     itag_& Conj() {
00697         ((char&t) ^= char(128);
00698         return *this;
00699     };
00700
00701     itag_& setConj() {
00702         ((char&t) |= char(128);
00703         return *this;
00704     };
00705
00706     bool isConj() const {
00707         return (((char&t)<0);
00708     };
00709
00710     itag_& deConj() {
00711         char &c=((char&t)); if (c<0) { c+=char(128); }
00712         return *this;
00713     };
00714
00715     itag_ conj() const { itag_ x(*this); return x.Conj(); }
00716     itag_ deconj() const { itag_ x(*this); return x.deConj(); }
00717
00718     itag_& SetFlag (unsigned l);
00719
00720     itag_& SetFlags(unsigned l=-1, char check=1);
00721
00722     itag_& UnsetFlag(unsigned l) {
00723         if (l>=ITAG_LEN) wblog(FL,
00724             "ERR %s() index out of bounds (%d/%d)",FCT,l,ITAG_LEN);
00725         ((char*)(&t))[l] &= 127;
00726         return *this;
00727     };
00728
00729     itag_& UnsetFlags() {
00730         char *s = (char*)(&t);
00731         for (unsigned l=1; l<ITAG_LEN; ++l) { if (s[l]<0) {
00732             s[l]+=char(128);
00733         } }
00734         return *this;
00735     };
00736
00737     bool GotFlag(unsigned l) const {
00738         if (l>=ITAG_LEN) wblog(FL,
00739             "ERR %s() index out of bounds (%d/%d)",FCT,l,ITAG_LEN);
00740         return ( ((char*)(&t))[l] < 0 );
00741     };
00742
00743     unsigned GotFlags(unsigned l=1) const {
00744         unsigned q=0;
00745         const char *s = (char*)(&t);
00746         for (; l<ITAG_LEN; ++l) { if (s[l]<0) { q|=(1<<l); } }
00747         return q;
00748     };
00749
00750     itag_& SetSI( const char *F, int L,
00751         const char *tag, unsigned k,
00752         char mflag=6,
00753         char nfmt=1);
00754
00755     itag_& AppendChar(char q, const char *qs=NULL);
00756     itag_& PrependChar(char q, const char *qs=NULL);

```

```

00757     int CheckFirstChar(char q, const char *qs=NULL);
00758
00759     itag_& MarkDual(char m=QS_MARK_DUAL, bool always=1);
00760
00761     itag_& AddTilde() { return AppendChar('~',"KD~"); };
00762     itag_& SetM() { return AppendChar('~',"KD~"); };
00763
00764     itag_& SetK() {
00765         if (CheckFirstChar('K',"AEX")>0)
00766             { return *this; }
00767         else { return AppendChar('K',"KD~"); }
00768     };
00769     itag_& SetD() {
00770         if (CheckFirstChar('D',"AEX")>0)
00771             { return *this; }
00772         else { return AppendChar('D',"KD~"); }
00773     };
00774
00775     unsigned to_str(char *s, unsigned len, char cflag=1) const;
00776
00777     wstring toStr(char cflag=1) const {
00778         wstring s(ITAG_LEN+4);
00779         to_str(s.data,s.len,cflag);
00780         return s;
00781     };
00782
00783     mxArray* toMx() const { return toStr().toMx(); };
00784
00785     IDT t;
00786
00787     static void Reset(unsigned char q=-1) {
00788         if (char(q)<0) {
00789             if (itag_::flag_id & 2) { return; }
00790             else { q=4; }
00791         }
00792         if (q&1) wblog(FL,"ERR %s() invalid q=%d",FCT,q);
00793         if (q<4) { q+=4; }
00794         itag_::flag_id=q;
00795     };
00796
00797     static void iter_flag_id() {
00798         #pragma omp critical (iter_flag_id__)
00799         if ((itag_::flag_id+=4)<4) { itag_::flag_id+=4; }
00800     };
00801
00802     protected:
00803     private:
00804
00805         static unsigned char flag_id;
00806     };
00807
00808     unsigned char itag_::flag_id = 4;
00809
00810     class iTags : public wbvector<itag_> {
00811     public:
00812
00813         iTags(unsigned n=0) : wbvector<itag_>(n) {};
00814         iTags(const char *F, int L, const char *s) { init(F,L,s); };
00815         iTags(const char *s) { init(FL,s); };
00816
00817         unsigned init(const char* F, int L, const char *s);
00818         unsigned init(const char *s) { return init(FL,s); };
00819
00820         iTags& init(unsigned n=0) {
00821             return (iTags&)wbvector<itag_>::init(n);
00822         };
00823         iTags& init(const iTags &b) {
00824             return (iTags&)wbvector<itag_>::init(b);
00825         };
00826         iTags& operator=(const iTags &b) {
00827             return (iTags&)wbvector<itag_>::init(b);
00828         };
00829     };
00830
00831     iTags& Update(const iTags &b) {
00832         if (len!=b.len) { wblog(FL,
00833             "ERR %s() itag length mismatch (%d/%d) '%s' / '%s'",
00834             FCT,len,b.len,STR_(this),STR(b));
00835         }
00836         for (unsigned i=0; i<len; ++i) {
00837             if (!data[i].isEmpty()) { if (data[i]!=b.data[i]) {
00838                 wblog(FL,"ERR %s() itag mismatch '%s' / '%s'",
00839                     FCT,STR_(this),STR(b),len,b.len);
00840             } }
00841             else if (data[i].isConj() ^ b.data[i].isConj()) {
00842                 wblog(FL,"ERR %s() itag conj mismatch '%s' / '%s'",
00843                     FCT,STR_(this),STR(b),len,b.len);

```



```

00844     }
00845 }
00846 return init(b);
00847 };
00848
00849 iTags& tSet(const iTags &b) {
00850     if (len!=b.len) { wblog(FL,
00851         "ERR %s() itag length mismatch (%d/%d) '%s' / '%s'",
00852         FCT,len,b.len,STR_(this),STR(b));
00853     }
00854     for (unsigned i=0; i<len; ++i) { data[i].tSet(b.data[i]); }
00855     return *this;
00856 }
00857
00858 #ifdef LOAD_CGC_QSPACE
00859 unsigned init_qdir(const char *s);
00860 #endif
00861 #ifndef NOMEX
00862 unsigned init(const char* F, int L, const mxArray *a);
00863 #endif
00864
00865 iTags& init3() {
00866     wbvector<itag_>::init(3); data[2].Conj();
00867     return *this;
00868 };
00869
00870 iTags& init(size_t l1, const itag_* d1, size_t l2, const itag_* d2) {
00871     wbvector<itag_>::init(l1,d1,l2,d2);
00872     return *this;
00873 };
00874
00875 unsigned Init(const char* F, int L,
00876     const char *s,
00877     unsigned k=-1,
00878     char lflag=-1);
00879
00880 iTags& Init(unsigned r, int k, const char **st);
00881
00882 iTags& initOp(itag_ t, const iTags &b, const char *ot="op");
00883
00884 iTags& init_alpha(
00885     unsigned r, unsigned s, const char *tag,
00886     char t,
00887     unsigned k);
00888
00889 unsigned Set(const char* F, int L, const char *tag,
00890     unsigned r=-1,
00891     const char *tom=NULL,
00892     unsigned m=-1,
00893     bool conjOM=1
00894 );
00895
00896 iTags& SetConj(const char* F, int L, unsigned i) {
00897     if (len) {
00898         if (i>=len) wblog(F_L,
00899             "ERR %s() index out of bounds (%d/%d)",FCT,i,len);
00900         data[i].Conj();
00901     }; return *this;
00902 };
00903
00904 iTags& Conj() {
00905     for (unsigned i=0; i<len; ++i) { data[i].Conj(); }
00906     return *this;
00907 };
00908
00909 iTags& SetFlag(const char* F, int L, unsigned k, unsigned l) {
00910     if (len) {
00911         if (k>=len) wblog(F_L,
00912             "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
00913         data[k].SetFlag(l);
00914     }; return *this;
00915 };
00916
00917 iTags& SetFlags(
00918     const char* F, int L, unsigned k, unsigned l=-1, char check=1) {
00919     if (len) {
00920         if (k>=len) wblog(F_L,
00921             "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
00922         data[k].SetFlags(l,check);
00923     }; return *this;
00924 };
00925
00926 iTags& UnsetFlag(const char* F, int L, unsigned k, unsigned l) {
00927     if (len) {
00928         if (k>=len) wblog(F_L,
00929             "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
00930         data[k].UnsetFlag(l);

```

```

00931     }; return *this;
00932 };
00933
00934 iTags& UnsetFlags() {
00935     for (unsigned k=0; k<len; ++k) {
00936         data[k].UnsetFlags();
00937     }; return *this;
00938 };
00939
00940 iTags& init_tags() {
00941     for (unsigned k=0; k<len; ++k) data[k].init_tag();
00942     return *this;
00943 };
00944
00945 bool operator==(const iTags &b) const {
00946     if (len!=b.len) return 0;
00947     for (unsigned i=0; i<len; ++i) {
00948         if (data[i].sameAs(b.data[i], 'l')<=0) return 0;
00949     }
00950     return 1;
00951 };
00952 bool operator!=(const iTags &b) const {
00953     return !((*this)==b);
00954 };
00955
00956 int findRegEx(const char *r) const;
00957 int RegEx_replace(const char *F, int L, const char *rstr) const;
00958
00959 int getCtr(
00960     const char *F, int L, const iTags &b,
00961     ctrIdx &ia, ctrIdx &ib, iTags *c=NULL,
00962     wbvector<char> *ma_=NULL, wbvector<char> *mb_=NULL
00963 ) const;
00964
00965 int UpdateItagsCtr(const char *F, int L,
00966     const ctrIdx &ia, iTags &b,
00967     const ctrIdx &ib,
00968     const char *tag
00969 );
00970
00971 int FlagItagsCtr(
00972     const char *F, int L, const ctrIdx &ia,
00973     iTags &B, const ctrIdx &ib,
00974     char all=1,
00975     char sgn=0
00976 );
00977
00978 bool sameConj(const iTags &b) const {
00979     if (len!=b.len) { return 0; }
00980     for (unsigned i=0; i<len; ++i) {
00981         if (data[i].isConj(b.data[i],0)) { return 0; }
00982     }
00983     return 1;
00984 };
00985
00986 bool isOp(int r=-1, char lflag=0) const;
00987
00988 bool isOpX(unsigned r=-1) const;
00989
00990 bool isAtensor(unsigned r=-1) const;
00991
00992 bool got_qdir(const char *qdir, char fflag=0) const {
00993     unsigned i=0, n=(qdir? strlen(qdir) : 0);
00994     if (n<2) wblog(FL,
00995         "ERR %s() invalid qdir=%s !?",FCT,qdir?qdir:"(null)");
00996     if (len<n || (fflag && len!=n)) return 0;
00997
00998     for (; i<n; ++i) {
00999         if (qdir[i]=='+' ) { if ( data[i].isConj()) return 0; } else
01000         if (qdir[i]=='-' ) { if (!data[i].isConj()) return 0; }
01001         else wblog(FL,"ERR %s() invalid qdir='%s' !?",FCT,qdir);
01002     }
01003     return 1;
01004 };
01005
01006 unsigned SkipLast(unsigned n=1) {
01007     if (n<len) { len-=n; }
01008     else {
01009         if (n>len) wblog(FL,
01010             "ERR %s() range out of bounds (%d/%d) !?",FCT,n,len);
01011         init();
01012     }
01013     return len;
01014 };
01015
01016 wbstring toStr(char vflag=1) const;
01017

```

```

01018     const char* toStrk(
01019         char *sout, unsigned l, unsigned k,
01020         const char *st="(itag?)", char cflag=0
01021     ) const {
01022         if (k>=len || data[k].isEmpty())
01023             { sprintf(sout,l,"%s",st?st:""); }
01024         else { sprintf(sout,l,"%s",data[k].toStr(cflag).data); }
01025         return sout;
01026     };
01027
01028     mxArray* toMx() const {
01029         if (len) {
01030             mxArray* a=mxCreateCellMatrix(1,len);
01031             for (unsigned i=0; i<len; ++i) {
01032                 mxSetCell(a,i,data[i].toMx());
01033             }
01034             return a;
01035         }
01036         return mxCreateCellMatrix(0,0);
01037     };
01038
01039     int got_op_labels(const char *F=NULL, int L=0) const {
01040         int q=0, x=0;
01041         unsigned i=0, l=len/2;
01042         for (; i<l; ++i) {
01043             if (!x=data[i].isValid()) wblog(F,L,
01044                 "ERR %s() got invalid itags (%s)",FCT,STR_(this));
01045             if (data[i].isConj() || data[i].conj()!=data[i+1]) return 0;
01046             if (q>0) { if (q!=x) q=-1; } else
01047                 if (!q) { q=x; }
01048         }
01049         return q;
01050     };
01051
01052     unsigned gotLabels() const {
01053         unsigned m=0;
01054         for (int q, k=0; k<int(len); ++k) {
01055             q=data[k].isValid();
01056             if (q>1) { ++m; } else if (!q) return 0;
01057         }
01058         return m;
01059     };
01060
01061 protected:
01062 private:
01063
01064     unsigned checkUnique(const char *F=NULL, int L=0) const {
01065         if (len) {
01066             unsigned i,j;
01067             for (i=0; i<len; ++i)
01068                 for (j=i+1; j<len; ++j) if (data[i].t==data[j].t) {
01069                     if (F) wblog(F,L,
01070                         "WRN got non-unique itags (%s)",STR_(this));
01071                     return 0;
01072                 }
01073         }
01074         return len;
01075     };
01076
01077 };
01078
01079 #define CTR_IS_NUM(c)      (((c)>='1' && (c)<='9') || ((c)>='a' && (c)<='z'))
01080
01081 #define CTR_IS_SEP_ANY(c) (isspace(c) || (c)=='\'' || (c)=='\';')
01082 #define CTR_IS_CONJ(c)    ((c)=='*')
01083 #define CTR_IS_OTHER(c)   (CTR_IS_SEP_ANY(c) || CTR_IS_CONJ(c))
01084
01085 #define CTR_IS_VALID(c)    (CTR_IS_NUM(c) || CTR_IS_OTHER(c))
01086
01087 class ctrIdx : public wbvector<unsigned> {
01088 public:
01089     ctrIdx() : conj(0) {};
01090
01091     ctrIdx(unsigned l, unsigned *d, char c=0)
01092         : wbvector<unsigned>(l,d), conj(c) {};
01093
01094     ctrIdx(const char *F, int L, const char *s, char c=0)
01095         : conj(c) { init(F,L,s); };
01096
01097     ctrIdx(const char *F, int L, const mxArray *a)
01098         : conj(0) { init(F,L,a); };
01099
01100     ctrIdx(const ctrIdx &I)
01101         : wbvector<unsigned>(I), conj(I.conj), newtags(I.newtags) {};
01102
01103
01104

```

```

01105     ctrIdx(const wbvector<unsigned> &I, char c=0)
01106     : wbvector<unsigned>(I), conj(c) {};
01107
01108     ctrIdx& init(unsigned l=0, unsigned *d=NULL, char c=0) {
01109         wbvector<unsigned>::init(l,d); conj=c;
01110         return *this;
01111     };
01112
01113     template <class T>
01114     ctrIdx& initT(unsigned l=0, T* d=NULL, char c=0) {
01115         wbvector<unsigned>::initT(l,d); conj=c;
01116         return *this;
01117     };
01118
01119     ctrIdx& init(const wbvector<unsigned> &I, char c=0) {
01120         wbvector<unsigned>::init(I); conj=c;
01121         return *this;
01122     };
01123
01124     ctrIdx& init1(unsigned k, char c=0) {
01125         wbvector<unsigned>::init(1);
01126         data[0]=k; conj=c;
01127         return *this;
01128     };
01129
01130     ctrIdx& init2(unsigned k1, unsigned k2, char c=0) {
01131         wbvector<unsigned>::init(2);
01132         if (k1==k2) wblog(FL,"ERR %s() got k=(%d,%d) !?",FCT,k1,k2);
01133         data[0]=k1; data[1]=k2; conj=c;
01134         return *this;
01135     };
01136
01137     template<class TQ, class TD>
01138     ctrIdx& initOp(const char *F, int L,
01139         itaq_ t1, const QSpace<TQ,TD> &B, const char *sopt);
01140
01141     int init(const char *F, int L, const char *s, unsigned offset=1);
01142
01143     ctrIdx& init(const char *F, int L, const mxArray *a);
01144     ctrIdx& init(const mxArray *a) { return init(FL,a); };
01145
01146     ctrIdx& Index(unsigned l, char c=0) {
01147         wbvector<unsigned>::init(l); conj=c;
01148         for (unsigned i=0; i<l; ++i) data[i]=i;
01149         return *this;
01150     };
01151
01152     int checkUniqueS(const char *F=0, int L=0, const unsigned r=-1) const;
01153
01154     ctrIdx& Sort(wbperm &P, unsigned r=-1) {
01155         if (len) {
01156             if (!isSorted())
01157                 { wbvector<unsigned>::Sort(P); }
01158             else { P.init(); }
01159             if (int(r)>=0) checkUniqueS(FL,r);
01160         }
01161         else { P.init(); }
01162         return *this;
01163     };
01164
01165     ctrIdx& sort(ctrIdx &I, wbperm &P, unsigned r=-1) const {
01166         if (len) {
01167             if (!isSorted())
01168                 { wbvector<unsigned>::sort(I,P); }
01169             else { I.init(*this); P.init(); }
01170             if (int(r)>=0) I.checkUniqueS(FL,r);
01171         }
01172         else { I.init(); P.init(); }
01173         return I;
01174     };
01175
01176     ctrIdx& Sort_(ctrIdx &J, unsigned r=-1) {
01177         if (len!=J.len) wblog(FL,
01178             "ERR %s() length mismatch (%s/%s) !?",FCT,STR_(this),STR(J));
01179         if (len) {
01180             if (!isSorted()) {
01181                 wbperm P; wbvector<unsigned>::Sort(P);
01182                 J.wbvector<unsigned>::Permute(P);
01183             }
01184             if (int(r)>=0) { checkUniqueS(FL,r); }
01185         }
01186         return *this;
01187     };
01188
01189     ctrIdx& invert(unsigned r, ctrIdx &I) const;
01190     ctrIdx& Invert(unsigned r) {
01191         ctrIdx I; save2(I);

```

```

01192         return I.invert(r,*this);
01193     };
01194
01195     ctrIdx& Conj() {
01196         conj=(conj ? 0 : 1);
01197         return *this;
01198     };
01199
01200     int extend2Perm(widx_t N, wbpem &P, char toend=0) const;
01201
01202     wbpem& extend2Perm( unsigned ra, unsigned ma,
01203         const ctrIdx &icb, unsigned rb, unsigned mb,
01204         const wbpem &pab, char conj, unsigned mc,
01205         wbpem &Pbc,
01206         ctrIdx &kb,
01207         ctrIdx &kcb,
01208         char Bflag=1
01209     ) const;
01210
01211     ctrIdx& save2(ctrIdx &J) {
01212         wbvector<unsigned>::save2(J);
01213         J.conj=conj; return J;
01214     };
01215
01216     mxArray* toMx() const { return toStr().toMx(); };
01217
01218     wbsting toStr(char xflag=0) const;
01219
01220     char conj;
01221
01222     iTags newtags;
01223
01224     protected:
01225     private:
01226
01227 };
01228
01229 bool isCtrIdx(const mxArray *a) {
01230
01231     unsigned i=0, r=60;
01232
01233     if (Mx::IsDblVector(0,0,a)) { wbvector<double> x(FL,a);
01234         if (mxGetM(a)!=1 || x.len>r) return 0;
01235
01236         for (; i<x.len; ++i) {
01237             if (x[i]<0 || double(int(x[i]))!=x[i] || x[i]>r) return 0;
01238         }
01239         if (!x.wbvector<double>::isUnique()) return 0;
01240         return 1;
01241     }
01242     else if (mxIsChar(a)) { ctrIdx idx; wbsting s(a);
01243         if (idx.init(0,0,s.data)<=0) return 0;
01244         for (; i<idx.len; ++i) { if (idx[i]>r) return 0; }
01245         if (!idx.wbvector<unsigned>::isUnique()) return 0;
01246         return 1;
01247     }
01248     return 0;
01249 };
01250
01251 template <class T=SPIDX_T>
01252 class sparseIndex2D {
01253
01254     public:
01255
01256         sparseIndex2D() : rmaj(0) {};
01257
01258         sparseIndex2D& Setup(char use_rmaj=0);
01259
01260         T cidx(T j) const;
01261         T ridx(T ic) const;
01262
01263         wbsting sizeStr() const {
01264             wbsting s;
01265             if (S.len) { s=S.toStrf("","x"); } else { s="[]"; }
01266             return s;
01267         };
01268
01269         char rmaj;
01270         wbMatrix<T> IJ;
01271         wbvector<T> S;
01272
01273         wbpem P;
01274
01275         wbvector<T> cIdx;
01276
01277     protected:
01278     private:

```

```

01279
01280 };
01281
01282 #endif
01283

```

5.38 wbio.c

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wblog (logging routines)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_WBIO_C__
00021 #define __WB_WBIO_C__
00022
00023 /* ----- */
00024 /* ----- */
00025 // generate temporary file $TMPDIR/<p=myprog><f="">-<pid>.<x=log>
00026 // in $TMPDIR (must be defined in environment)
00027 // Wb,Apr30,20
00028
00029 wbststring Wb::tmpfile(const char *F, int L,
00030     const char *t,
00031     const char *x,
00032     const char *f
00033 ) {
00034
00035     wbststring fout(64);
00036
00037     unsigned l; const char *p=getenv("TMPDIR");
00038     if (!p) wblog(F_L,"ERR TMPFILE not defined (null)"); else
00039     if (p[0]!='/') wblog(F_L,"ERR invalid TMPFILE '%s'",p);
00040
00041     l=snprintf(fout.data,fout.len,"%s/%s-%d%s", p,
00042         f && f[0] ? f : myname, t? t:"", getpid(), x? x : ".log");
00043
00044     if (l>=fout.len) wblog(F_L,
00045         "ERR %s() string out of bounds (%d/%d)",FCT,l,fout.len);
00046
00047     return fout;
00048 };
00049
00050 void Wb::tmpFile::init(const char *F, int L,
00051     const char *f,
00052     const char *x,
00053     const char *mode
00054 ) {
00055
00056     if (fid) wblog(F_L,
00057         "ERR %s() already got initialized fid=%d",FCT,fid);
00058
00059     fout=tmpfile(F_L,f,x);
00060     fid=fopen(fout.data, mode && mode[0] ? mode : "a+");
00061
00062     if (!fid) wblog(F_L,
00063         "ERR %s() failed to open file\n`s'",FCT,fout.data);
00064     else {
00065         fprintf(fid,"\n%-18s %5s %s open %s\n\n",
00066             shortFL(PF_L),"", Wb::TimeStamp('T').data, Wb::TimeStamp('D').data);
00067     }
00068
00069     if (F) wblog(PF_L,"I/O opening tmpfile `%s'",fout.data);
00070 };
00071
00072 int Wb::tmpFile::sepline(const char *x, unsigned n) {
00073     unsigned l=0;
00074
00075     if (!n || !ncall) { return l; }

```

```

00076     if (int(n)<0) { n=72; }
00077     else if (int(n)>90) n=90;
00078
00079     unsigned m=(x ? strlen(x) : 0); char s[n+1];
00080
00081     if (m<=1) { memset(s, m? x[0] : '-', n); }
00082     else {
00083         while (l+m<=n) { memcpy(s+l,x,m); l+=m; }
00084         if (l<n) { memcpy(s+l,x,n-l); }
00085     }
00086
00087     if (!omp_in_parallel()) { this->flush(); }
00088
00089     l=n; s[l]=0; {
00090         if (!fid) wblog(FL,"ERR %s() tmpFile not yet opened !?",FCT);
00091         #pragma omp critical (__tmpFile_IO__)
00092         { fprintf(fid,"%s\n",s); }
00093     }
00094
00095     ncall=0;
00096
00097     return l;
00098 };
00099
00100 int Wb::tmpFile::blogf(const char *F, int L, const char *fmt, ...) {
00101
00102     va_list args; Wb::ARGV wd(&args);
00103     va_start(args,fmt); unsigned l=0;
00104
00105     char sx[8];
00106     snprintf(sx,8,"%d/%d",omp_get_thread_num(), omp_get_num_threads());
00107
00108     if ((++ncall)<1) { ncall=1; }
00109
00110     if (fid) {
00111         #pragma omp critical (__tmpFile_IO__)
00112         { l+=fprintf(fid,"%-18s %5s %s ",
00113             shortFL(F_L,18), sx, Wb::TimeStamp('T').data);
00114             l+=vfprintf(fid,fmt,args);
00115             l+=fprintf(fid,"\n");
00116         }
00117     }
00118     else {
00119         fprintf(stdout,"\n%s %s "
00120             "ERR %s() tmpFile closed / not yet opened !?\n\n",shortFLT, FCT);
00121     }
00122
00123     return l;
00124 };
00125
00126 void WbPrintMatrixC (
00127     const wbMatrix<wbcomplex> &M, const char *istr, int space
00128 ){
00129     unsigned i,j,l=0,n=64; char dstr[n], sfmt[64];
00130
00131     int rflag, iflag;
00132
00133     printf ("%s%s", istr, istr[0] ? "\n" : "");
00134
00135     if (space<1) sprintf (sfmt, " %s");
00136     else             sprintf (sfmt, " %%%ds", space);
00137
00138     for (i=0; i<M.dim1; i++) {
00139         for (j=0; j<M.dim2; j++) {
00140
00141             rflag = std::isnan(M(i,j).r);
00142             if (!rflag) {
00143                 if (M(i,j).r!=0.) {
00144                     rflag = (fabs(M(i,j).r)==1.) ? 1 : 2;
00145                     if (M(i,j).r<0) rflag = -rflag;
00146                 }
00147             }
00148
00149             iflag = std::isnan(M(i,j).i);
00150             if (!iflag) {
00151                 if (M(i,j).i!=0.) {
00152                     iflag = (fabs(M(i,j).i)==1.) ? 1 : 2;
00153                     if (M(i,j).i<0) iflag = -iflag;
00154                 }
00155             }
00156
00157             if (rflag) {
00158                 l+=snprintf(dstr+l,n-l,"% g",M(i,j).r);
00159                 if (iflag) {
00160                     if (abs(iflag)>1)
00161                         l+=snprintf(dstr+l,n-l,"%+gi", M(i,j).i);
00162                     else l+=snprintf(dstr+l,n-l,"%ci", iflag==1? '+':'-');

```

```

00163         }
00164     }
00165     else {
00166         if (iflag)
00167             if (abs(iflag)>1)
00168                 l=snprintf(dstr,n,"% gi", M(i,j).i);
00169             else l=snprintf(dstr,n,"%ci", iflag==1? ' ':'-');
00170         else { l=snprintf(dstr,n,"%g", M(i,j).r); }
00171     }
00172     printf(sfmt,dstr);
00173 }
00174 printf ("\n");
00175 }
00176 printf ("\n");
00177 };
00178
00179 template<class T>
00180 int Str2Idx(
00181     const char *F, int L,
00182     const char* s_, wvector<T> &idx,
00183     T offset,
00184     char cmpct
00185 ){
00186     if (!s_) {
00187         wblog(F_L,"ERR %s() got null input !?",FCT);
00188         return -1;
00189     }
00190     if (!s_[0]) { idx.init(); return 0; }
00191
00192     int i,j, n=0, e=0, l=strlen(s_),
00193         ms=1,
00194         mx=0,
00195         ma=0;
00196
00197     char c, sx=0, bflag=0;
00198     char s[l+1], *s1=NULL, *s2=s;
00199     long int x;
00200
00201     strcpy(s,s_);
00202
00203     for (i=0; i<l && !e; ++i) { if (!isspace(c=s_[i])) {
00204         if (isalnum(c)) { ++ma; ms=0;
00205             if ((++n)>1 && !sx) { sx=''; }
00206             for (++i; i<l; ++i) { if (isalnum(s_[i])) ++ma; else break; }
00207             --i; continue;
00208         }
00209         else { ++mx; s[i]=' '; }
00210
00211         if (c==' ' || c=='\n') { ++ms;
00212             if (n && ms==1)
00213                 { if (sx) { if (sx!=c) e=__LINE__; } else { sx=c; } }
00214             else { e=__LINE__; }
00215         }
00216         else if (c=='[') { if (n || bflag&1) e=__LINE__; else bflag|=1; }
00217         else if (c==']') { if (bflag&2) e=__LINE__; else { bflag|=2;
00218             for (++i; i<l && s_[i]; ++i) {
00219                 if (!isspace(s_[i])) { e=__LINE__; break; }
00220             }
00221         } }
00222         else { e=__LINE__; }
00223     } }
00224
00225     if (!n && !mx) { idx.init(); return 0; }
00226
00227     if (bflag && bflag!=3) {
00228         if (e) { if (e>0) e=-e; } else
00229             e=__LINE__;
00230     }
00231
00232     if (e<0) { if (F) wblog(F,L,
00233         "ERR %s() invalid '%s' (i=%d/%d; b=%d)",FCT,s_,i,l,bflag);
00234         return -i;
00235     }
00236
00237     if (e) { l=i; }
00238
00239     if (n==1 && cmpct) { n=ma; idx.init(n);
00240         for (j=i=0; i<l; ++i) {
00241             if (isalnum(s[i])) { c=s[i];
00242                 if (unsigned(j)>=idx.len) wblog(F,
00243                     "ERR %s() j=%d/%d !?",FCT,l,idx.len);
00244                 if (c>='0' && c<='9') { idx[j++]=c-'0'; } else
00245                 if (c>='A' && c<='Z') { idx[j++]=c-'A'+10; } else
00246                 if (c>='a' && c<='z') { idx[j++]=c-'a'+36; }
00247                 else { e=__LINE__; ++i; break; }
00248             }
00249         }

```



```

00250     }
00251     else { idx.init(n);
00252
00253         for (n=0; n<=int(idx.len); ++n) { s1=s2;
00254             x=strtol(s1,&s2,0); if (s1==s2 || n==int(idx.len)) { break; }
00255
00256             if ((long int)T(x)!=x) wblog(F_L,
00257                 "ERR %s() invalid '%s' %g/%ld !?",FCT,s_,double(T(x)),x);
00258             idx[n]=T(x);
00259         }
00260         if (n!=int(idx.len)) { e=__LINE__; }
00261         if (!e) {
00262             if (s1!=s2) { l=s2-s; e=__LINE__; } else {
00263                 for (; *s2; ++s2) { if (!isspace(*s2)) { l=s2-s; e=__LINE__; }}}
00264             }
00265         }
00266
00267         if (e<0) { if (F) wblog(F,L,
00268             "ERR %s() invalid '%s' (%d/%d;%d)",FCT,s_,i,l,bflag);
00269             return -l;
00270         }
00271
00272         if (offset && e>=int(0)) {
00273             for (i=0; i<int(idx.len); ++i) {
00274                 if (idx[i]>=offset) { idx[i]-=offset; } else { e=__LINE__; }}
00275             }
00276
00277         if (e<0) {
00278             if (F) wblog(F,L,"ERR %s() invalid '%s' "
00279                 "(e=%d; %d/%d, %d @ %d)",FCT,s_,-e,idx.len,l,bflag,offset);
00280             return -l;
00281         }
00282
00283         if (e) {
00284             return -l;
00285         }
00286
00287         return idx.len;
00288     };
00289
00290 template<class T>
00291 int Str2Idx(
00292     const char* s, wvector<T> &I, T offset, char cmpct
00293 ){ return Str2Idx(FL,s,I,offset,cmpct); };
00294
00295 template<class T>
00296 int Str2Idx(
00297     const char* F, int L,
00298     wvector<T> &I, const mxArray *a, T offset, char cmpct
00299 ){
00300     if (!a) wblog(FL,"ERR %s() got null mxArray",FCT);
00301
00302     if (mxIsChar(a)) { wstring s(a);
00303         int n=Str2Idx(0,0,s.data,I,offset,cmpct);
00304         if (n<0) wblog(F_L,
00305             "ERR %s() '%s' => [%s] @ e=%d !?",FCT,s.data,STR(I),n);
00306         return I.len;
00307     }
00308     else {
00309         I.init(F,L,a);
00310         if (offset) {
00311             if (I.anyLT(offset)) wblog(FL,
00312                 "ERR %s() index incompatible with offset=%d",FCT,offset);
00313             I-=offset;
00314         }
00315         return 0;
00316     }
00317 };
00318
00319 #endif
00320

```

5.39 wbio.h

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wblog (logging routines)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at

```

```

00010 *
00011 *      http://www.apache.org/licenses/LICENSE-2.0
00012 *
00013 * Unless required by applicable law or agreed to in writing, software
00014 * distributed under the License is distributed on an "AS IS" BASIS,
00015 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016 * See the License for the specific language governing permissions and
00017 * limitations under the License.
00018 * ----- */
00019
00020 #ifndef __WB_WBIO_H__
00021 #define __WB_WBIO_H__
00022
00023 /* ----- */
00024 // tags: tmpfile, tempfile, tempmat, TMPDIR // Wb,Jun06,19
00025
00026 namespace Wb {
00027
00028 wbsting tmpfile(const char *F, int L,
00029     const char *t=NULL, const char *x=NULL, const char *f=NULL);
00030
00031 class tmpFile {
00032 public:
00033     tmpFile() : fid(NULL), ncall(-1) { };
00034
00035     tmpFile(const char *F, int L,
00036         const char *f=NULL, const char *x=NULL, const char *mode=NULL)
00037         : fid(NULL), ncall(-1) { init(F,L,f,x,mode); };
00038
00039     ~tmpFile() { if (fid) {
00040         wblog(PFL,
00041             "I/O closing tmpfile '%s'",fout.data? fout.data:"(null!)");
00042         fprintf(fid,"\n%-18s %s %s close %s\n\n",
00043             shortFL(PFL),"", Wb::TimeStamp('T').data, Wb::TimeStamp('D').data);
00044         fclose(fid); fid=NULL;
00045     } };
00046
00047     void init(const char *F, int L,
00048         const char *f=NULL, const char *x=NULL, const char *mode=NULL);
00049
00050     int blogf(const char *F, int L, const char *fmt, ...);
00051
00052     int sepline(const char *w="-", unsigned len=-1);
00053
00054     void flush() { if (fid) fflush(fid); };
00055
00056     FILE *fid;
00057     int ncall;
00058     wbsting fout;
00059
00060 protected:
00061 private:
00062 };
00063
00064 };
00065
00066 template<class T>
00067 int Str2Idx(
00068     const char *F, int L,
00069     const char* s, wbvector<T> &idx, T offset=0, char cmpct=0
00070 );
00071
00072 template<class T>
00073 int Str2Idx(
00074     const char* s, wbvector<T> &idx, T offset=0, char cmpct=0
00075 );
00076
00077 template<class T> int Str2Idx(
00078     const char* s, wbvector<T> &idx, unsigned offset=0, char cmpct=0
00079 ){ return Str2Idx(s,idx,T(offset),cmpct); };
00080
00081 wbvector<unsigned> Str2Idx(
00082     const char *F, int L, const char* s, unsigned offset=0
00083 ){
00084     wbvector<unsigned> I;
00085     int e=Str2Idx(F,L,s,I,offset);
00086     if (e<0) wblog(PFL,
00087         "ERR %s() invalid '%s' (%d;%d)",FCT,s?s:"null",-e,offset);
00088     return I;
00089 };
00090
00091 wbvector<unsigned> Str2Idx(const char* s, unsigned offset=0) {
00092     return Str2Idx(FL,s,offset);
00093 };
00094
00095 template<class T>
00096 int Str2Idx(

```

```

00097     const char* F, int L,
00098     wvector<T> &I, const mxArray *a, T offset=0, char cmpct=0
00099 );
00100
00101 template<class T>
00102 int Str2Idx(
00103     wvector<T> &I, const mxArray *a, T offset=0, char cmpct=0
00104 ){ return Str2Idx(FL,I,a,offset,cmpct); };
00105
00106 void WbPrintMatrixC (
00107     const wbMatrix<wbcomplex> &M, const char *istr="", int space=10);
00108
00109 #endif

```

5.40 wblib.h

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  *
00004  * Copyright 2022 Andreas Weichselbaum
00005  *
00006  * Licensed under the Apache License, Version 2.0 (the "License");
00007  * you may not use this file except in compliance with the License.
00008  * You may obtain a copy of the License at
00009  *
00010  *     http://www.apache.org/licenses/LICENSE-2.0
00011  *
00012  * Unless required by applicable law or agreed to in writing, software
00013  * distributed under the License is distributed on an "AS IS" BASIS,
00014  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00015  * See the license for the specific language governing permissions and
00016  * limitations under the License.
00017  * ----- */
00018
00019 #ifndef __WB_LIB_MEX_HCC__
00020 #define __WB_LIB_MEX_HCC__
00021
00022 /* ----- //
00023  header file for mex programs
00024  A. Weichselbaum (C) Jan 2006
00025  // ----- */
00026
00027 #define PP_STR__(a) #a
00028 #define PP_STRFY(a) PP_STR__(a)
00029
00030 #ifndef QS_USING_OMP
00031     #define QS_USING_OMP 1
00032 #elif QS_USING_OMP<=0
00033     #undef QS_USING_OMP
00034 #endif
00035
00036 #if __linux__
00037 #elif __unix__
00038     #warning got unix
00039 #elif __APPLE__
00040 #else
00041     #error GOT UNSUPPORTED OPERATING SYSTEM
00042 #endif
00043
00044 #include <assert.h>
00045 #include <execinfo.h>
00046
00047 #include <time.h>
00048 #include <signal.h>
00049 #include <unistd.h>
00050 #include <fcntl.h>
00051 #include <regex.h>
00052 #include <regex>
00053
00054 #include <stdio>
00055 #include <stdlib>
00056 #include <chrono>
00057
00058 #include <exception>
00059
00060 #include <cmath>
00061
00062 #include <ctype.h>
00063 #include <climits>
00064 #include <stdarg.h>
00065 #include <string.h>
00066
00067 #include <string>

```

```

00068 #include <iostream>
00069 #include <typeinfo>
00070
00071 #include <sys/ipc.h>
00072 #include <sys/msg.h>
00073 #include <sys/types.h>
00074 #include <sys/stat.h>
00075
00076 #include <map>
00077 #include <queue>
00078 #include <unordered_map>
00079 #include <algorithm>
00080 #include <vector>
00081
00082 #ifdef QS_USING_OMP
00083 #include <omp.h>
00084 #else
00085     int omp_get_max_threads() { return 1; }
00086     int omp_get_num_threads() { return 1; }
00087     int omp_get_thread_num() { return 0; }
00088     int omp_in_parallel() { return 0; }
00089     int omp_get_level() { return 0; }
00090 #endif
00091
00092 #if __APPLE__
00093     #ifndef expl0
00094         inline double expl0(double x) { return pow(double(10),x); }
00095     #endif
00096     #ifndef sighandler_t
00097         #define sighandler_t sig_t
00098     #endif
00099
00100     #include <mach/mach_host.h>
00101     #include <mach/task.h>
00102
00103     #include <sys/sysctl.h>
00104     #undef setbit
00105     #undef isset
00106
00107 #endif
00108
00109     unsigned ARG_CHECK=1;
00110
00111     #define STRLEN 1023
00112     char str[STRLEN+1];
00113
00114     #define MX_CHECK_HELPER_NARGS(n1,n2,m) \
00115         if (nargin>0 && \
00116             checkHelpVersion(argin[0], nargsout ? nargsout : NULL)) { return; }; \
00117         if (nargin<n1 || (n2>=0 && nargin>n2)) { \
00118             strcpy(str,"ERR invalid number of input args"); unsigned l=strlen(str); \
00119             if (nargin<n1) \
00120                 { snprintf(str+1,32," (%d / %d required)",nargin,n1); } \
00121             else { snprintf(str+1,32," (%d / %d %s)",nargin,n2, \
00122                 n1==n2? "expected":"at most"); }; \
00123             if (nargin || nargsout) wblog(FL,str); else usage(FL,str); \
00124         }; \
00125         if (m>=0 && nargsout>m) { \
00126             sprintf(str,"ERR invalid number of output args (%d)",nargsout); \
00127             if (nargin || nargsout) wblog(FL,str); else usage(FL,str); \
00128         }
00129
00130     template <class T> class wbvector;
00131     template <class T> class wbMatrix;
00132     template <class T> class wbararray;
00133     template <class TD> class wbsparray;
00134     template <class TQ, class TD> class QSpace;
00135     template <class T> class sparseIndex2D;
00136
00137     class wbcomplex;
00138     class wbstring;
00139     class bitset;
00140     class wbperm;
00141     class wbindex;
00142
00143     class iTags;
00144     class ctrIdx;
00145
00146     namespace CG {
00147         class FileLock;
00148     }
00149
00150     namespace Wb {
00151         class Clock;
00152         class ClockSet;
00153     }
00154

```

```

00155 using namespace std;
00156
00157 #ifdef MATLAB_MEX_FILE
00158 class MXPut;
00159 #endif
00160
00161 #define ENABLE_IF_COMPLEX(T__) typename \
00162     std::enable_if< WbUtil<T__>::isComplex(), T__ >::type* = nullptr
00163
00164 #define ENABLE_IF_isPOD(T__) typename \
00165     std::enable_if< WbUtil<T__>::isPOD(), T__ >::type* = nullptr
00166
00167 #define ENABLE_IF_noPOD(T__) typename \
00168     std::enable_if< !WbUtil<T__>::isPOD(), T__ >::type* = nullptr
00169
00170 #define ENABLE_IF_isINT(T__) typename \
00171     std::enable_if<std::is_integral<T__>::value>* = nullptr
00172
00173 #if 1
00174     #define widx_t size_t
00175     #define swidx_t long
00176     #define wperm_t size_t
00177     #define swperm_t long
00178 #else
00179     #define widx_t unsigned
00180     #define swidx_t int
00181     #define wperm_t unsigned
00182     #define swperm_t int
00183 #endif
00184
00185 #if 1
00186     #define SPIDX_T size_t
00187     #define sSPIDX_T long
00188 #else
00189     #define SPIDX_T unsigned
00190     #define sSPIDX_T int
00191 #endif
00192
00193 #define WBINDEX wbvector<widx_t>
00194 #define WBPPerm wbvector<wperm_t>
00195 #define WBIDXMAT wbMatrix<widx_t>
00196
00197 #define UVEC wbvector<unsigned>
00198 #define C_UVEC const wbvector<unsigned>
00199 #define DMAT wbMatrix<double>
00200 #define C_DMAT const wbMatrix<double>
00201 #define UMAT wbMatrix<unsigned>
00202 #define C_UMAT const wbMatrix<unsigned>
00203 #define IMAT wbMatrix<int>
00204 #define C_IMAT const wbMatrix<int>
00205 #define C_TMAT const wbMatrix<T>
00206 #define C_MEX const mxArray
00207 #define C_UINT const unsigned
00208 #define C_CHAR const char
00209
00210 #define DMAT0 wbMatrix<double>()
00211
00212 #define __FL__ (file ? file : __FILE__), (line ? line : __LINE__)
00213
00214 #define F_L (F ? F : __FILE__), (F ? L : __LINE__)
00215 #define F_LF (F ? F : __FILE__), (F ? L : __LINE__), __FUNCTION__
00216 #define F_L_F (F ? F : __FILE__), (F ? L : __LINE__), (fct ? fct : __FUNCTION__)
00217
00218 #define FSTR __FILE__ ": "
00219 #define FLINE __FILE__, __LINE__
00220 #define FL __FILE__, __LINE__
00221 #define FLF __FILE__, __LINE__, __FUNCTION__
00222 #define FCT __FUNCTION__
00223 #define FCL __FUNCTION__, __LINE__
00224
00225 std::string getName(const std::type_info &type_id, char vflag=1);
00226
00227 #define PFL shortFL(__FILE__, __LINE__, -1, ""), WBL_GOT_SHORTFL
00228 #define PF_L shortFL(F?F:__FILE__, F?L:__LINE__, -1, ""), WBL_GOT_SHORTFL
00229
00230 #ifndef TIME
00231 #define TIME Wb::TimeStamp('T').data
00232 #endif
00233
00234 #ifndef shortFLT
00235 #define shortFLT shortFL(__FILE__, __LINE__), Wb::TimeStamp('T').data
00236 #define SHORT_FL shortFL(__FILE__, __LINE__)
00237 #endif
00238
00239 #ifndef PSTr
00240 #ifdef QS_USING_OMP
00241     #define PSTr Wb::ompID2Str().data

```

```

00242     #else
00243         #define PSTR ""
00244     #endif
00245 #endif
00246
00247 #ifndef Inf
00248 #define Inf FP_INFINITE
00249 #endif
00250
00251 #if !defined(MAX)
00252     template <class T> inline
00253     const T& MAX(const T&a, const T&b) { return ((b)>(a) ? (b) : (a)); }
00254 #endif
00255
00256     template <class T> inline
00257     const T& MAX3(const T&a, const T&b, const T&c) {
00258         if ((b)>(a))
00259             { return ((c)>(b) ? (c) : (b)); }
00260         else { return ((c)>(a) ? (c) : (a)); }
00261     }
00262
00263 #if !defined(MIN)
00264     template <class T> inline
00265     const T& MIN(const T&a, const T&b) { return ((b)<(a) ? (b) : (a)); };
00266 #endif
00267
00268     template <class T> inline
00269     const T& MIN3(const T&a, const T&b, const T&c) {
00270         if ((b)<(a))
00271             { return ((c)<(b) ? (c) : (b)); }
00272         else { return ((c)<(a) ? (c) : (a)); }
00273     };
00274
00275 #if !defined(SWAP)
00276     template <class T>
00277     inline void SWAP(T &a, T &b) { T x=a; a=b; b=x; }
00278 #endif
00279
00280 #if !defined(NSTR)
00281 #define NSTR(a) Wb::num2Str(a).data
00282 #endif
00283 #if !defined(NSTRf)
00284 #define NSTRf(a,f) Wb::num2Str(a,f).data
00285 #endif
00286
00287 #if !defined(RSTR)
00288 #define RSTR(a,n) wbsting().initrep(a,n).data
00289 #endif
00290
00291 #if !defined(SEC2STR)
00292 #define SEC2STR(a) Wb::sec2Str(a).data
00293 #endif
00294
00295 #if !defined(TSTR)
00296 #define TSTR(a) getName(typeid(a),1).c_str()
00297 #endif
00298
00299 #if !defined(sTSTR)
00300 #define sTSTR(a) getName(typeid(a),0).c_str()
00301 #endif
00302
00303 #if !defined(SSTR)
00304 #define SSTR(a) (a).sizeStr().data
00305 #endif
00306
00307 #if !defined(SSTR_)
00308 #define SSTR_(a) (a)->sizeStr().data
00309 #endif
00310
00311 #if !defined(SSTRM)
00312 #define SSTRM(S,r) (S).toStrf("%d","x",r,"_").data
00313 #endif
00314
00315 #if !defined(SSTRM_)
00316 #define SSTRM_(r,sd,m,sm) Wb::sizeStrM(r,sd,m,sm).data
00317 #endif
00318
00319 #if !defined(QSTR)
00320 #define QSTR(a) (a).qStr().data
00321 #endif
00322
00323 #if !defined(QSTR_)
00324 #define QSTR_(a) (a)->qStr().data
00325 #endif
00326
00327 #if !defined(STR)
00328 #define STR(a) (a).toStr().data

```

```

00329 #endif
00330
00331 #if !defined(STR_)
00332 #define STR_(a) (a)->toStr().data
00333 #endif
00334
00335 #if !defined(STR2)
00336 #define STR2(a,b) (a).toStr(b).data
00337 #endif
00338
00339 #if !defined(STR2_)
00340 #define STR2_(a,b) (a)->toStr(b).data
00341 #endif
00342
00343 #if !defined(cSTR)
00344 #define cSTR(a) Wb::char2Str(a).data
00345 #endif
00346
00347 #if !defined(I2STR)
00348 #define I2STR(a) Wb::int2Str(a).data
00349 #endif
00350
00351 #if !defined(RATS)
00352 #define RATS(a) Wb::rat2Str(__FILE__, __LINE__, a).data
00353 #endif
00354
00355 #if !defined(STRREP)
00356 #define STRREP(str,n) wbsting().initrep(str,n).data
00357 #endif
00358
00359 #if !defined(BITS)
00360 #define BITS(a) Wb::bits(a).data
00361 #endif
00362
00363 #if !defined(POW)
00364
00365     template <class T>
00366     inline T POW(const T &a, const unsigned &n) {
00367         T x = n ? a : 1;
00368         for (unsigned i=1; i<n; i++) x*=n;
00369         return x;
00370     }
00371
00372 #endif
00373 #if !defined(SGN)
00374
00375     template <class T>
00376     inline int SGN(const T &a) {
00377         return ((T(0)<a)-(a<T(0)));
00378     }
00379
00380 #endif
00381 #if !defined(NUMCMP)
00382
00383     template <class T>
00384     inline char NUMCMP(const T &a, const T &b) {
00385         if (a<b) return -1; else
00386         if (a>b) return +1;
00387         return 0;
00388     }
00389
00390     template <class Ta, class Tb>
00391     inline char NUMCMP(const Ta &a, const Tb &b) {
00392         if (a<b) return -1; else
00393         if (a>b) return +1;
00394         return 0;
00395     }
00396
00397 #endif
00398
00399 #define ISCOMPLX_(a) WbUtil<a>::isComplex()
00400 #define ISREAL(a) WbUtil<a>::isReal()
00401
00402 #ifdef TST_MAC
00403     #include "Include/nomex.hh"
00404
00405     #include <Accelerate/Accelerate.h>
00406
00407 #elif defined(NOMEX)
00408     #include "nomex.hh"
00409 #else
00410
00411     #include <mex.h>
00412     #include <mat.h>
00413
00414     #define QS_VERSION 4.0
00415     #define QS_VERSION_SUB 0

```

```

00416     #define QS_VERSION_SUB_ ""
00417
00418     #ifdef MEX_MLVER
00419         #define MEX_MLVER_STR PP_STRFY(MEX_MLVER)
00420     #else
00421         #define MEX_MLVER_STR "(unknown)"
00422     #endif
00423
00424     #ifdef HOST_NAME
00425         #define HOST_NAME_STR PP_STRFY(HOST_NAME)
00426     #else
00427         #define HOST_NAME_STR "(host)"
00428     #endif
00429
00430 #define mxIsNumChar(a) (mxIsNumeric(a) || mxIsChar(a) || mxIsLogical(a))
00431
00432 #endif
00433
00434 #ifdef MAIN
00435     #undef printf
00436 #endif
00437
00438 #ifndef PROG
00439 #ifdef MATLAB_MEX_FILE
00440     #define PROG mexFunctionName()
00441 #else
00442     #define PROG "(?prog?)"
00443 #endif
00444 #endif
00445
00446     enum WBL_COLOR_SCHEME { WLC_OFF,
00447                             WLC_DARK,
00448                             NUM_WBL_COLOR_SCHEME };
00449
00450 namespace Wb {
00451     class myname__ {
00452     public:
00453         myname__() { unsigned i=0, k=0;
00454             #ifdef PROG
00455                 snprintf(data,60,"%s",PROG);
00456             #elif defined MATLAB_MEX_FILE
00457                 snprintf(data,60,"%s",mexFunctionName());
00458             #else
00459                 snprintf(data,60,"(?myname?)");
00460             #endif
00461
00462             for (i=0; data[i]; ++i) { if (data[i]=='/') k=i+1; }
00463             if (k && data[k]) {
00464                 for (i=0; data[k]; ++k, ++i) { data[i]=data[k]; }
00465                 data[i]=0;
00466             }
00467
00468             #ifdef PROG_TAG
00469                 snprintf(data+60,4,"%s",PROG_TAG);
00470             #else
00471                 snprintf(data+60,4,"%s",data);
00472             #endif
00473         };
00474
00475         char data[64];
00476
00477     private:
00478     };
00479
00480     myname__ myName;
00481
00482 #define myname Wb::myName.data
00483 #define mytag Wb::myName.data+60
00484
00485     WBL_COLOR_SCHEME useCol = WLC_DARK;
00486
00487     int get_WB_VERBOSE(const char *F=0, int L=0);
00488     int got_DBSTOP(const char *F=0, int L=0);
00489     int got_DESKTOP();
00490     int is_DEPLOYED(const char *F=0, int L=0);
00491
00492     int envVRB= 0;
00493     int envDKT=-1;
00494
00495     int envDBG= 0;
00496
00497     int my_caller_tid=0;
00498
00499 };
00500
00501 void dbstop(const char* file, int line);
00502 void ExitMsg(const char* s="", char xflag=0);

```



```
00503 void doflush();
00504
00505 #ifdef LOAD_CGC_QSPACE
00506 #ifndef QS_SKIP_MPFR
00507     #define QS_USING_MPFR
00508 #endif
00509 #endif
00510
00511     #define QS_ITAG_ "__"
00512
00513 #ifdef __WBDEBUG__
00514 #define __WB_MEM_CHECK__
00515 #undef WB_SKIP_ASSERT
00516 #elif defined __WB_MEM_CHECK__
00517 #define __WBDEBUG__
00518 #endif
00519
00520 namespace Wb {
00521 class ARGV {
00522 public:
00523     ARGV(va_list *vl=NULL) : args(vl) {};
00524     ~ARGV() { if (args) { va_end(*args); args=NULL; }; }
00525 private:
00526     va_list *args;
00527 };
00528 };
00529 };
00530
00531 namespace wblog {
00532     const int SLEN=256;
00533 };
00534
00535 #ifdef MATLAB_MEX_FILE
00536 #define PRINTF(...) \
00537     { if (omp_get_level()==0) \
00538         { printf(__VA_ARGS__); } \
00539     else { fprintf(stdout,__VA_ARGS__); } \
00540     }
00541 #else
00542 #define PRINTF printf
00543 #endif
00544
00545 #include "wblibx.h"
00546
00547 #include "wblog.h"
00548
00549 #ifdef QS_USING_OMP
00550 #include "wbomp.hh"
00551 #endif
00552
00553 #include "memtrack.hh"
00554
00555 #include "util.hh"
00556 #include "memlib.hh"
00557 #include "mxlib.hh"
00558
00559 #ifdef QS_USING_MPFR
00560 #include <gmp.h>
00561 #include <mpfr.h>
00562 #include "wbmpfr.hh"
00563 #endif
00564
00565 #include "wbcomplex.hh"
00566 #include "gathered.hh"
00567 #include "wbsort.hh"
00568
00569 #include "wbvector.hh"
00570 #include "wbstring.hh"
00571 #include "wbbitset.hh"
00572 #include "wbio.h"
00573
00574 #include "wbperm.hh"
00575 #include "wbindex.hh"
00576
00577 #include "wbclock.hh"
00578 #include "wbMatrix.hh"
00579
00580 #include "mexlib.hh"
00581
00582 #include "wbMatIO.hh"
00583
00584 #include "wbarray.hh"
00585 #include "wbsparray.hh"
00586
00587 #include "wbutil.hh"
00588
00589 #include "wblas.hh"
```

```

00590 #include "wbMatrix_blas.hh"
00591 #include "wbarray_blas.hh"
00592
00593 #include "wbopts.hh"
00594
00595 #ifdef DBG_GCX_LOCKS
00596     Wb::tmpFile LKF(FL, "-lks");
00597 #endif
00598
00599 #ifdef DBG_QSX_BUF
00600     Wb::tmpFile BFF(FL, "-buf");
00601 #endif
00602
00603 #ifdef LOAD_CGC_QSPACE
00604 #include "clebsch.hh"
00605 #include "QSpace_aux.hh"
00606 #include "QSpace.hh"
00607 #include "mpsortho.hh"
00608 #endif
00609
00610 #include "wblog.c"
00611
00612 #ifdef QS_USING_MPFR
00613 #include "wbmpfr.cc"
00614 #endif
00615
00616 #ifdef QS_USING_OMP
00617 #include "wbomp.cc"
00618 #endif
00619
00620 #include "wbsort.cc"
00621 #include "wbstring.cc"
00622 #include "wbbitset.cc"
00623 #include "memlib.cc"
00624 #include "wbcomplex.cc"
00625 #include "gathered.cc"
00626 #include "util.cc"
00627
00628 #ifdef LOAD_CGC_QSPACE
00629 #include "QSpace_aux.cc"
00630 #include "QSpace.cc"
00631 #endif
00632
00633 #include "wbvector.cc"
00634 #include "wbperm.cc"
00635 #include "wbindex.cc"
00636 #include "wbMatrix.cc"
00637 #include "wbarray.cc"
00638 #include "wbsparray.cc"
00639
00640 #include "wbclock.cc"
00641 #include "mexlib.cc"
00642 #include "matlib.cc"
00643 #include "wbMatrix_blas.cc"
00644 #include "wbarray_blas.cc"
00645
00646 #include "wbio.c"
00647 #include "wbmsg.hh"
00648
00649 #ifdef LOAD_CGC_QSPACE
00650 #include "clebsch.cc"
00651 #include "clebsch_aux.cc"
00652 #include "clebsch_io.cc"
00653 #include "clebsch_old.cc"
00654 #include "mpsortho.cc"
00655 #endif
00656
00657 namespace Wb {
00658
00659     class gpara__ {
00660     public:
00661         gpara__() : x(0) {
00662             #ifdef __WBDEBUG__
00663                 wblog(FL, "TST %s (%s)", FCT, myname);
00664             #endif
00665             wb_srand();
00666
00667             #ifdef __WB_MPFR_HH__
00668                 gmp_randinit_default(Wb::wb_rstate);
00669             #endif
00670
00671             envDKT=got_DESKTOP();
00672             my_caller_tid=omp_get_thread_num();
00673
00674             init();
00675         };
00676

```

```

00677     void init() {
00678         memset(str,0,STRLEN+1);
00679         envVRB=get_WB_VERBOSE();
00680         envDBG=get_DBSTOP();
00681
00682         #ifdef LOAD_CGC_QSPACE
00683         #ifdef QS_USING_OMP
00684             Wb::GetNumThreads(FL,OMP_NUM_THREADS,"OMP_NUM_THREADS");
00685             Wb::GetNumThreads(FL,QSP_NUM_THREADS,"QSP_NUM_THREADS");
00686
00687             sp_num_threads=MAX(OMP_NUM_THREADS,QSP_NUM_THREADS);
00688         #endif
00689         #endif
00690
00691         my_caller_tid=omp_get_thread_num();
00692
00693     };
00694
00695     char x;
00696
00697     private:
00698 };
00699
00700 gpara__ gpara;
00701
00702 class Cleanup {
00703 public:
00704     Cleanup() : i(0) {
00705         gpara.init();
00706     };
00707
00708     void Check() {
00709         wblog::check_ERR_pending();
00710     };
00711
00712     ~Cleanup() {
00713         wblog::myIO.clear();
00714         if (gwb_Profs.size()) { Wb::save_and_clear_Profiling(); }
00715     };
00716
00717     int i;
00718 };
00719
00720 };
00721
00722 #ifdef MAIN
00723     #ifdef MATLAB_MEX_FILE
00724         #warning got MAIN with MATLAB_MEX_FILE
00725         #pragma message "\n\n \
00726         \" Don't define MAIN with MEX files, this kills the whole matlab seesion!\n\" \
00727         \" See ExitMsg() in wblib.h (!)\n\" \
00728         ""
00729     #endif
00730
00731     void ExitMsg(const char* s, char xflag) {
00732         if (s && s[0])
00733             { printf("\n%s !? %s\n",shortFL(FL),s); }
00734         else { printf("\n"); }
00735
00736         if (Wb::envDBG) {
00737             dbstop(FL); if (xflag) { exit(-1); }
00738         }
00739         else {
00740             if (xflag) {
00741                 exit(-1);
00742             }
00743             else {
00744                 mexErrMsgIdAndTxt("Wb:MEX:wblog",s);
00745             }
00746         }
00747     }
00748
00749     void doflush() { fflush(0); };
00750
00751 #else
00752
00753     #ifdef DBSTOP
00754         void ExitMsg(const char* s, char xflag) {
00755             if (s && s[0]) printf("%s >%s<\n",shortFL(FL),s ? s:"");
00756             dbstop(FL);
00757             printf("\n%s dbstop() and on we go ...<\n", shortFL(FL));
00758             mexErrMsgIdAndTxt("Wb:ERR:mex",s);
00759         }
00760     #else
00761         void ExitMsg(const char* s, char xflag __attribute__((unused))) {
00762             printf("\n");
00763         }
00764     #endif

```

```

00764         if (Wb::is_DEPLOYED()) {
00765             Wb::print_backtrace(FL, "from deployed code"); printf("\n"); }
00766
00767             mexErrMsgIdAndTxt("Wb:ERR:mex", s);
00768             fprintf(stdout, "%s TST %s()\n", shortFL(FL), FCT);
00769         }
00770     #endif
00771
00772     void doflush() {
00773         fflush(0);
00774
00775         #if 0
00776
00777             if (!omp_in_parallel()) {
00778                 mexEvalString("pause(0);");
00779             }
00780         #endif
00781
00782     };
00783
00784 #endif
00785
00786 #endif
00787

```

5.41 wblibx.h

```

00001 #ifndef __WB_WBLIBX_HH__
00002 #define __WB_WBLIBX_HH__
00003
00004 /* ----- */
00005 /* ----- */
00006
00007     void usage(const char *file=NULL, int line=0, const char* estr=NULL);
00008
00009     int isHelpIndicator(const mxArray *a0);
00010     int checkHelpVersion( const mxArray *a0, mxArray **argout=NULL);
00011
00012     void wbdie(const char *file, int line, const char* istr);
00013
00014     void dbstop(const char* file, int line);
00015
00016     char* memsize2Str(double x, char *s, unsigned l=-1);
00017
00018     void version_info_toMx(mxCArray *S);
00019
00020 #ifndef MEX_EXT
00021 #if __linux__
00022     #define MEX_EXT mexa64
00023 #elif __unix__
00024     #define MEX_EXT mexa64
00025 #elif __APPLE__
00026     #define MEX_EXT mexmac*
00027 #else
00028     #define MEX_EXT unknown
00029 #endif
00030 #endif
00031
00032 namespace Wb {
00033 class VersionInfo {
00034 public:
00035     VersionInfo() {
00036         fctn[0]=tag[0]=matlab[0]=mpfr[0]=compiler[0]=os[0] = 0;
00037         qspace[0]=git[0]=flags[0]=compiled[0] = 0;
00038         init();
00039     };
00040
00041     VersionInfo& init();
00042     wbsting toStr() const;
00043
00044     void print() const;
00045     mxArray* toMx() const;
00046     char fctn[32], tag[8], matlab[16], mpfr[16], compiler[16], os[24];
00047     char qspace[16], git[16], flags[128], compiled[64];
00048
00049 private:
00050
00051     inline unsigned str_cpy(char *s, unsigned n, const char *a);
00052 };
00053
00054
00055 template <class T>
00056 class is_pointer_ { public: char q() { return 0; }; };

```

```

00057
00058 template <class T>
00059 class is_pointer_<T*> { public: char q() { return 1; }; };
00060
00061 template <class T>
00062 class is_pointer_<T**> { public: char q() { return 2; }; };
00063
00064 template <class T>
00065 class is_pointer_<T***> { public: char q() { return 3; }; };
00066
00067 template <class T>
00068 inline char ispointer(const T& x) { return is_pointer_<T>().q(); };
00069
00070 #endif
00071

```

5.42 wblog.c

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wblog (logging routines)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_LOG_C__
00021 #define __WB_LOG_C__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 #ifndef LIBQS_SETUP
00027
00028 void usage(const char *F, int L, const char* estr) {
00029
00030     int l=0;
00031     while (isspace(USAGE[l])) { ++l; }
00032
00033     if (USAGE[l]) { printf("\n%s%s\n",l?" ":"",USAGE); }
00034     else {
00035         const char *f=mexFunctionName();
00036         if (f && f[0]) {
00037             char cmd[256]; l=snprintf(cmd,256,
00038                 "if exist('%s.m','file')==2, fprintf(1,'\n'); help %s; "
00039                 "else fprintf(1,'    NB! usage has been outsourced to m-file"
00040                 " => checkout %s.m\n'); end",f,f,f);
00041
00042             if (l<256) {
00043                 if (F && F[0]) { mexPrintf("\n    %s\n",shortFL(F,L)); }
00044                 mexEvalString(cmd);
00045             }
00046             else wblog(FL,"WRN %s() got f=%s (l=%d/256) !?",FCT,f,l);
00047         }
00048     }
00049
00050     if (estr && estr[0]) { wbdie(F,L,estr); }
00051 };
00052
00053 #endif
00054
00055 const char* shortFL(const char *F, int L,
00056     unsigned n,
00057     const char *P, char sep
00058 ) {
00059
00060     static char fstr[512]="", *fs=fstr;
00061     unsigned i,l, j=0, nx=(P? 40 : 32);
00062     char *sout=fstr, sx[nx];
00063
00064     if (int(n)<0) { n=WBL_HLEN; }
00065     else if (n<16 || n>=nx) {

```

```

00066         if (!n && !F && !L)
00067             { memset(fstr,0,512); return sout; }
00068         else { wblog(FL,"ERR %s() got n=%d/%d\n",FCT,n+1,nx); }
00069     }
00070
00071 #pragma omp critical (using_shortFL)
00072 { if (fs+n-fstr<512)
00073     { sout=fs; fs+=(n+1); }
00074     else { sout=fs=fstr; }
00075 }
00076
00077 if (!F || !F[0]) { sout[0]=0; return sout; }
00078 if (F[0]=='/' || F[0]=='.') { l=strlen(F);
00079     for (i=l-1; i<l; --i) { if (F[i]=='.') { break; }}
00080     for ( ; i<l; --i) { if (F[i]=='/') { j=i+1; break; }}
00081 }
00082
00083 if (P && !P[0]) {
00084     P=mytag; if (!P || !P[0]) { P=myname; }
00085     #ifdef PROG
00086     if (P && P[0]) { l=strlen(P);
00087         if (!strncmp(F+j,P, l>3? 3:l)) { P=NULL; }
00088     }
00089     #endif
00090 }
00091
00092 if (!P || !P[0]) {
00093     if (L>0)
00094         { l=snprintf(sx,nx,"%.24s:%d",F+j,L); }
00095     else { l=snprintf(sx,nx,"%.24s:", F+j ); }
00096 }
00097 else {
00098     if (L>0)
00099         { l=snprintf(sx,nx,"%.3s%c%.20s:%d",P,sep,F+j,L); }
00100     else { l=snprintf(sx,nx,"%.3s%c%.20s:", P,sep,F+j ); }
00101 }
00102
00103 if (l>=nx) wblog(FL,
00104     "WRN %s() string out of bounds (%d/%d)\n%s",FCT,l,nx,sx);
00105
00106 if (l<=n) { strcpy(sout,sx); }
00107 else {
00108     for (i=l-1; i<l; --i) { if (sx[i]=='.') { break; }}
00109     for (j=i; j<l; --j) { if (sx[j]=='_' || sx[j]=='-') { break; }}
00110     if (j<l && (l-j)<n-3) {
00111         i=n+j-1;
00112         strcpy(sout,sx,i);
00113         sout[i]='\n';
00114         strcpy(sout+i+1,sx+j+1);
00115     }
00116     else if (i<l && (l-i)<n-8) {
00117         j=n+i-1;
00118         strcpy(sout,sx,j);
00119         sout[j]='\n';
00120         strcpy(sout+j+1,sx+i+1);
00121     }
00122     else { i=5;
00123         strcpy(sout,sx,i); sout[i]='\n';
00124         strcpy(sout+i+1,sx+l-n+i+1);
00125     }
00126 }
00127 return sout;
00128 };
00129
00130 const char* tmpmatFL(
00131     const char *F, int L, const char *vn, char *s, unsigned n) {
00132
00133     if (!s || int(n)<8) wblog(FL,"ERR %s() invalid input (%p, %d)",FCT,s,n);
00134     unsigned l=snprintf(s,n,"./tmp");
00135
00136     if (l<n && F && F[0]) { unsigned i=0, k=0;
00137         for (; F[i]; ++i) { if (F[i]=='/') { k=i+1; } }
00138         i=l; l+=snprintf(s+l,n-l,"-%s",F+k);
00139         for (; s[i] && i<n; ++i) { if (s[i]=='.') { s[i]=0; l=i; break; }}
00140     }
00141
00142     if (l<n) { l+=snprintf(s+l,n-l,"-%d",L); }
00143 #ifdef QS_USING_OMP
00144     if (l<n) {
00145         int j=omp_get_thread_num();
00146         if (j!=Wb::my_caller_tid) { l+=snprintf(s+l,n-l,".%d",j); }
00147     }
00148 #endif
00149
00150     if (l<n && vn && vn[0]) { l+=snprintf(s+l,n-l,"-%s",vn); }
00151     if (l<n) { l+=snprintf(s+l,n-l,".mat"); }
00152

```

```

00153     if (l>=n) wblog(F_L,"WRN %s() "
00154         "string out of bounds (%d/%d)\n%s",FCT,l,n,s?s:"(null)");
00155     return s;
00156 };
00157
00158 unsigned Wb::VersionInfo::str_cpy(char *s, unsigned n, const char *a) {
00159     unsigned i=0; s[0]=0;
00160     if (a) {
00161         for (; i<n; ++i) { if (!(s[i]=a[i])) break; }
00162         if (i>=n) { wblog(F_L,
00163             "WRN %s() out of bounds (%d/%d)\n%s",FCT,strlen(a),n,a);
00164         }
00165     }
00166     return i;
00167 };
00168
00169 Wb::VersionInfo& Wb::VersionInfo::init() {
00170
00171     unsigned i,l,m,n; char *s;
00172
00173     str_cpy(fctn,sizeof(fctn),myname);
00174     #ifdef PROG_TAG
00175     str_cpy(tag, sizeof(tag), PROG_TAG);
00176     #endif
00177
00178     n=str_cpy(matlab,sizeof(matlab),MEX_MLVER_STR);
00179
00180     s=matlab;
00181     for (l=i=0; i<n; ++i) { if (s[i]=='/') { l=i+1; }}
00182     if (l) {
00183         for (i=l; i<=n; ++i) { if (!(s[i-l]=s[l])) break; }
00184     }
00185
00186     l=sizeof(os);
00187     #if __linux__
00188     snprintf(os,l,"linux / %s", PP_STRFY(MEX_EXT));
00189     #elif __unix__
00190     snprintf(os,l,"unix / %s", PP_STRFY(MEX_EXT));
00191     #elif __APPLE__
00192     snprintf(os,l,"macOS / %s", PP_STRFY(MEX_EXT));
00193     #else
00194     snprintf(os,l,"unknown / %s",PP_STRFY(MEX_EXT));
00195     #endif
00196
00197     #ifdef QS_USING_MPFR
00198     str_cpy(mpfr,sizeof(mpfr),MPFR_VERSION_STRING);
00199     #endif
00200
00201     n=sizeof(compiler); l=6;
00202
00203     #if defined(__clang__)
00204     snprintf(compiler,n,"clang/%d.%d.%d",
00205         __clang_major__, __clang_minor__, __clang_patchlevel__);
00206     #elif defined(__GNUC__)
00207     snprintf(compiler,n,"gcc/%d.%d.%d",
00208         __GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__);
00209     #endif
00210
00211     #ifdef LOAD_CGC_QSPACE
00212     n=sizeof(qspace);
00213     l=snprintf(qspace,n,"%1f",double(QS_VERSION));
00214
00215     i=strlen(QS_VERSION_SUB_);
00216     if (QS_VERSION_SUB != 0) {
00217         { l+=snprintf(qspace+l,n-l,"%g",double(QS_VERSION_SUB)); }
00218         if (i) { l+=snprintf(qspace+l,n-l,"-%s",QS_VERSION_SUB_); }
00219     }
00220     #endif
00221
00222     #ifdef __QS_GIT_BRANCH__
00223     str_cpy(git,sizeof(git),PP_STRFY(__QS_GIT_BRANCH__));
00224     #endif
00225
00226     n=sizeof(flags); l=0;
00227
00228     #ifdef MAIN
00229     l+=snprintf(flags+l,n-l,"%s","MAIN");
00230     #endif
00231     #ifdef MATLAB_MEX_FILE
00232     l+=snprintf(flags+l,n-l,"%s%s",l? " ", ":", "MEX_FILE");
00233     #endif
00234
00235     i=l; m=0;
00236     l+=snprintf(flags+l,n-l," using");
00237
00238     #ifdef LOAD_CGC_QSPACE
00239     if (l<n) { l+=snprintf(flags+l,n-l," QS" ); }; ++m;
00240     #endif

```

```

00240 #ifdef QS_USING_OMP
00241     if (l<n) { l+=snprintf(flags+l,n-1," OMP" ); }; ++m;
00242 #endif
00243 #ifdef QS_USING_MPFR
00244     if (l<n) { l+=snprintf(flags+l,n-1," MPFR"); }; ++m;
00245 #endif
00246
00247     if (m)
00248     { i=1; m=0; if (l+3<n) { strcpy(flags+l," "); l+=2; }}
00249     else { l=i; m=1; }
00250
00251 #ifdef WB_SKIP_ASSERT
00252     if (l<n) {
00253         l+=snprintf(flags+l,n-1,"%s%s",m?" ", ":", "SKIP_ASSERT");
00254     }; ++m;
00255 #endif
00256 #ifdef MEX_CXX_FLAGS
00257     if (l<n) {
00258         l+=snprintf(flags+l,n-1,"%s%s",m?" ", ":", PP_STRFY(MEX_CXX_FLAGS));
00259     }; ++m;
00260 #endif
00261 #ifdef __WBDEBUG__
00262     if (l<n) {
00263         l+=snprintf(flags+l,n-1,"%s%s",m?" ", ":", "DEBUG");
00264     }; ++m;
00265 #endif
00266 #ifdef DBSTOP
00267     if (l<n) {
00268         l+=snprintf(flags+l,n-1,"%s%s",m?" ", ":", "DBSTOP");
00269     }; ++m;
00270 #endif
00271 #ifdef DBG_GCX_LOCKS
00272     if (l<n) { l+=snprintf(flags+l,n-1,
00273         "%s%s=%g",m?" ", ":", "DBG_GCX_LOCKS", double(DBG_GCX_LOCKS));
00274     }; ++m;
00275 #endif
00276 #ifdef DBG_QSX_BUF
00277     if (l<n) { l+=snprintf(flags+l,n-1,
00278         "%s%s=%g",m?" ", ":", "DBG_QSX_BUF", double(DBG_QSX_BUF));
00279     }; ++m;
00280 #endif
00281 #ifdef __WB_MEM_CHECK__
00282     if (l<n) {
00283         l+=snprintf(flags+l,n-1,"%s%s",m?" ", ":", "MEM_CHECK");
00284     }; ++m;
00285 #endif
00286
00287     if (m)
00288     { i=1; m=0; }
00289     else { l=i; }
00290
00291     l=snprintf(compiled,sizeof(compiled),"%s on %s",
00292         #ifdef __DATE_COMP__
00293             PP_STRFY(__DATE_COMP__),
00294         #else
00295             __DATE__,
00296         #endif
00297         HOST_NAME_STR
00298     );
00299
00300     return *this;
00301 };
00302
00303 void Wb::VersionInfo::print() const { PRINTF("\n%s\n",toStr().data); };
00304
00305 wbsting Wb::VersionInfo::toStr() const {
00306
00307     wbsting sout; unsigned i=0,l,n=512;
00308     sout.init(n);
00309
00310     const unsigned m=8; char q[2], *sx=sout.data;
00311
00312     const char *field[m] = {
00313         "Matlab","Compiler","System","MPFR","QSpace","Git-branch","Flags","Compiled"
00314     };
00315     const char *data[m] = {
00316         matlab, compiler, os, mpfr, qspace, git, flags, compiled
00317     };
00318
00319     l=snprintf(sx,n,"Version info for function '%s',fctn);
00320     if (*tag)
00321     { l+=snprintf(sx+l,n-1," (%s)\n\n",tag); }
00322     else { l+=snprintf(sx+l,n-1,"\n\n"); }
00323
00324     q[0]=q[1]=0;
00325
00326     for (; i<m; ++i) { if (data[i] && l<n) {

```



```

00327         if (i==0) { *q='R'; } else
00328         if (i==3 || i==4)
00329             { if (*data[i]) { *q='v'; } else continue; }
00330         else { *q=0; }
00331
00332         l+=snprintf(sx+l,n-l,"    %-13s %s%s\n",field[i],q,data[i]);
00333     }
00334
00335     if (l<n)
00336         { if (l) { sout.len=l; }
00337         else { wblog(FL,"WRN %s() string out of bounds (%d/%d)",FCT,l,n); }
00338
00339     return sout;
00340 };
00341
00342 mxArray* Wb::VersionInfo::toMx() const {
00343
00344     unsigned i=0; const unsigned n=10;
00345
00346     const char *fields[n] = {
00347         "function","tag","matlab","system","MPFR","compiler","QSpace",
00348         "git_branch", "flags", "compiled"
00349     };
00350     mxArray *S=mxCreateStructMatrix(1,1,n,fields);
00351
00352     const char *data[n] = {
00353         fctn, tag, matlab, os, mpfr, compiler, qspace, git,
00354         flags, compiled
00355     };
00356
00357     for (; i<n; ++i) {
00358         mxSetFieldByNumber(S,0,i,wbstring(data[i]).toMx());
00359     }
00360
00361     return S;
00362 };
00363
00364 void banner(unsigned n, const char *s, const char *istr, const char *fstr){
00365
00366     unsigned i,r=0,l=0,m=0, s1=0, s2=0;
00367
00368     if (s && s[0]) r=strlen(s);
00369     else wblog(FL,"ERR cannot repeat empty string!");
00370
00371     if (istr && istr[0]) { s1=strlen(istr); printf("%s",istr); }
00372     if (fstr && fstr[0]) { s2=strlen(fstr); }
00373
00374     m=s1+s2; if (n>=m) l=(n-m)/r;
00375     else wblog(FL,"ERR overall length too short (%d/%d)",n,m);
00376
00377     for (i=0; i<l; i++) printf("%s",s);
00378     for (i=1*r+s1, l=n-s2; i<l; i++) printf("%s",s);
00379
00380     if (fstr && fstr[0]) { printf("%s",fstr); }
00381     printf("\n");
00382 };
00383
00384 void wbdie(const char *F, int L, const char* istr) {
00385
00386     PRINTF("\n%s ERR\n\n%s\n\n", shortFL(F,L), istr);
00387
00388     #ifdef QS_USING_OMP
00389         throw Wb::LogException(ERR);
00390     #else
00391         ExitMsg();
00392     #endif
00393 };
00394
00395 void wbdie(
00396     const char *F, int L,
00397     const char *istr1,
00398     const char *istr2
00399 ){
00400     wblog(F,L,"ERR %s\n%s\n", istr1, istr2);
00401     #ifdef QS_USING_OMP
00402         throw Wb::LogException(ERR);
00403     #else
00404         ExitMsg();
00405     #endif
00406 };
00407
00408 Wb::LogException::LogException(WBLOG_TYPE l, const char *s)
00409 : type(l), nrefs(NULL), count(0), ith(-1), nth(-1) {
00410
00411     if (!s || !s[0]) { istr[0]=0; } else {
00412         if (snprintf(istr,64,"%s",s)>=64) {
00413             istr[61]=istr[62]='.'; istr[63]=0;

```

```

00414     }
00415 }
00416
00417 if (l) { nrefs = new int; (*nrefs)=1; count=1;
00418     ith=omp_get_thread_num();
00419     nth=omp_get_num_threads();
00420     #ifdef DBSTOP
00421     fprintf(stdout,"%s %s dbstop() having Wb::LogException "
00422         "nrefs=%d, istr='%s'; tid=%d/%d\n",
00423         shortFLT,*nrefs,istr,ith,nth); fflush(0);
00424     dbstop(FL);
00425     #endif
00426 }
00427 };
00428
00429 void Wb::LogException::report(const char *F, int L, const char *func) {
00430     if (type || count) {
00431         unsigned l, n=128; char s[n];
00432
00433         l=snprintf(s,n,
00434             "%s ERR %s() at nerr=%d",shortFL(F_L),func?func:FCT,count);
00435         if (l<n) {
00436             if (ith<=1 && nth<=1)
00437                 l+=snprintf(s+l,n-l," (serial @ %d/%d)",ith,nth);
00438             else l+=snprintf(s+l,n-l," (parallel section thread %d/%d)",ith,nth);
00439         }
00440
00441         if (type) { PRINTF("\n%s\n\n",s); throw(*this); } else
00442             if (count) { ExitMsg(s); }
00443     }
00444     init();
00445 };
00446
00447 wbsting Wb::LogException::toStr() const {
00448     wbsting sout(128);
00449
00450     unsigned l, n=sout.len; char *s=sout.data;
00451
00452     l=snprintf(s,n,"%s: ",istr[0] ? istr : "");
00453     if (l<n) {
00454         if (type<NUM_TYPE)
00455             { l+=snprintf(s+l,n-l,"type=%s",WBLOG_TYPE_STR[type]); }
00456         else { l+=snprintf(s+l,n-l,"type=%d !?",unsigned(type)); }
00457     }
00458     if (l<n) {
00459         l+=snprintf(s+l,n-l," %s (i=%d/%d; nerr=%d)",
00460             ith<=1 && nth<=1 ? "serial":"parallel section", ith,nth,count);
00461     }
00462     if (l<n) {
00463         if (nrefs)
00464             { l+=snprintf(s+l,n-l," *nrefs=%d",*nrefs); }
00465         else { l+=snprintf(s+l,n-l," nrefs=null"); }
00466     }
00467     return sout;
00468 };
00469
00470 void init_header(
00471     char *hstr, unsigned hlen,
00472     const char *F, int L,
00473     const char *time_stamp, const char *tag, int xcol,
00474     unsigned lenFL=21
00475 ){
00476     unsigned i,k=0,n;
00477
00478     if (xcol && hlen>12) {
00479         i=Wb::termcolor::ID("hdr");
00480         if (int(i)>0) {
00481             if (i<8)
00482                 { i=snprintf(hstr,hlen,"\\e[3%dm",i); }
00483             else { i=snprintf(hstr,hlen,"\\e[38;5;%dm",i); }
00484         }
00485         if (!i || i>=hlen) { i=sprintf(hstr,"\\e[0m"); }
00486         hstr+=i; hlen-=i;
00487     }
00488
00489     if (F) {
00490         if (F[0]=='/' || (F[0]=='.' && F[1]=='/')) {
00491             for (i=0; F[i]; ++i) { if (F[i]=='/') k=i+1; }
00492         }
00493
00494         if (L==WBL_GOT_SHORTFL)
00495             { i=snprintf(hstr,hlen,"%s.100s", F+k ); }
00496         else { i=snprintf(hstr,hlen,"%s.100s:%d",F+k,L); }
00497
00498         if (i>=hlen) wblog(FL,"ERR %s() "
00499             "string out of bounds (%d/%d)%N%N's'",FCT,i,hlen,hstr);
00500

```

```

00501     n=strlen(hstr); if (n>lenFL) {
00502         i=lenFL/2; hstr[i]=hstr[i+1]='.'; i+=2;
00503         for (k=n-lenFL/2+2; k<n; ++k, ++i) hstr[i]=hstr[k];
00504         hstr[i]=0;
00505     }
00506 }
00507 else { sprintf(hstr,"(null):%d", L); }
00508
00509 i=strlen(hstr); for (; i<lenFL; ++i) { hstr[i]=' '; }
00510
00511 n=strlen(time_stamp+17); if (n>32) n=32;
00512 memcpy(hstr+i,time_stamp+17,n); i+=n;
00513 hstr[i++]=' '; hstr[i]=0;
00514
00515 if (xcol) {
00516     if (xcol>0) {
00517         if (xcol<8)
00518             { i+=sprintf(hstr+i,"\\e[3%dm",xcol); }
00519         else { i+=sprintf(hstr+i,"\\e[38;5;%dm",xcol); }
00520     }
00521     else if (xcol<0) { i+=sprintf(hstr+i,"\\e[0m"); }
00522 }
00523
00524 if (tag && tag[0])
00525     { i+=sprintf(hstr+i, " %s ",tag); }
00526 else { hstr[i]=' '; hstr[i+1]=0; }
00527 };
00528
00529 int check_update_header(
00530     char *hstr, const char* fmt, const char *istr=NULL, int lenFL=21
00531 ) {
00532     int n,k=0,i=0,q=0;
00533
00534     for (; fmt[i]; ++i) {
00535         if (isalpha(fmt[i])) q|=1; else if (fmt[i]=='.' ) q|=2; else
00536         if (isdigit(fmt[i])) q|=8; else if (fmt[i]=='_' ) q|=16; else
00537         if (fmt[i]=='/') { q|=4; k=i+1; } else break;
00538     }
00539
00540     if ((q&3)!=3 || fmt[i]!=':') return 0;
00541     n=i++;
00542
00543     for (; fmt[i]; ++i) if (!isdigit(fmt[i])) break;
00544     if (i==n+1 || isalpha(fmt[i])) return 0;
00545
00546     if (istr && istr[0]) {
00547         strcpy(hstr,istr);
00548         n=strlen(hstr); hstr+=n; lenFL-=n;
00549     }
00550
00551     if (i-k<lenFL) {
00552         memcpy(hstr,fmt+k,i-k);
00553         for (k=i-k; k<lenFL; k++) hstr[k]=' ';
00554     }
00555     else {
00556         memcpy(hstr,fmt+k,lenFL/2); k=lenFL/2;
00557         hstr[k]=hstr[k+1]='.';
00558         memcpy(hstr+k+2, fmt+i-k+2,k-2); k=2*k;
00559     }
00560
00561     if (k<lenFL) {
00562         for (; hstr[k]; k++) if (hstr[k]!=' ') break;
00563         if (k>lenFL) {
00564             for (n=k-lenFL; hstr[k]; k++) hstr[k-n]=hstr[k];
00565             hstr[k-n]=0;
00566         }
00567     }
00568
00569     for (; fmt[i]; ++i) {
00570         if (!isspace(fmt[i]) && fmt[i]!=':' && fmt[i]!='(' && fmt[i]!=')')
00571             break;
00572     }
00573
00574     return i;
00575 };
00576
00577 void wbSetLogLevel(unsigned l) {
00578     wblogf(stdout,0,0,"log_level",l);
00579 };
00580
00581 int wblogf(FILE *fid,
00582     const char* file, int line, const char *fmt, va_list args
00583 ) {
00584     #ifdef QS_USING_OMP
00585         Wb::ompGuard myLK(wblog_lock,l);
00586     #endif
00587

```

```

00588     wblog::SBUF S; int l; Wb::LogException e;
00589
00590     WBL_COLOR_SCHEME xcol=WLC_OFF;
00591     if (Wb::useCol && (fid==stdout || fid==stderr)) { xcol=WLC_DARK; }
00592
00593     try { l=wblogs(S,xcol,file,line,fmt,args); }
00594     catch (Wb::LogException &e_) { e=e_; l=-11; }
00595     catch (...) { l=-12; }
00596
00597     { S.flush(fid, l>1); }
00598
00599     if (e.type) { throw(e); } else
00600     if (l<-10) {
00601         sprintf(str,"ERR wblog.h:%d encountered l=%d !?",__LINE__,l);
00602         ExitMsg(str);
00603     }
00604
00605     return l;
00606 };
00607
00608 int wblog1(const char* file, int line, const char *fmt, ...) {
00609
00610     va_list args; Wb::ARGV wd(&args);
00611     va_start(args,fmt);
00612
00613     WBL_COLOR_SCHEME xcol=WLC_OFF;
00614     if (Wb::useCol) { xcol=WLC_DARK; }
00615
00616     wblog::SBUF S; int l; Wb::LogException e;
00617     try { l=wblogs(S,xcol,file,line,fmt,args,l); }
00618     catch (Wb::LogException &e_) { e=e_; l=-11; }
00619     catch (...) { l=-12; }
00620
00621     S.flush(l>3 ? stderr : stdout,l>1);
00622
00623     if (e.type) { throw(e); } else
00624     if (l<-10) {
00625         sprintf(str,"ERR wblog.h:%d encountered l=%d !?",__LINE__,l);
00626         ExitMsg(str);
00627     }
00628
00629     return l;
00630 };
00631
00632 int wblogs(
00633     wblog::SBUF &Sb, WBL_COLOR_SCHEME xcol_,
00634     const char* file, int line,
00635     const char *fmt, va_list args,
00636     char Hflag
00637 ){
00638     static int log_level=1;
00639     static int then=0;
00640
00641     int rval=0, hlen=128, xcol=0;
00642
00643     unsigned i,j,k,l,m, nesc=0;
00644     const unsigned nt=32;
00645
00646     char isfmt=0, hflag=1, bflag=0, iflag=0, eflag=0, wflag=0, fwd=0;
00647
00648     char fstr[128];
00649     char time_stamp[nt], tag[8];
00650
00651     char c, *cp, cb, log_header[hlen];
00652
00653     if (file==NULL && line<=0) {
00654         if (!strcmp(fmt,"log_level")) {
00655             log_level=(unsigned)va_arg(args,int);
00656             return 0;
00657         }
00658     }
00659
00660     if (log_level<0) { return 0; }
00661
00662     if (!fmt) wblog(FL,"ERR wblog got null fmt");
00663
00664     if (fmt[0]=='\\') { hflag=0; fmt+=1; bflag=1; }
00665
00666     for (; *fmt; ++fmt) { if (*fmt=='\n') Sb.cat("\n"); else break; }
00667
00668     tag[0]=0;
00669
00670     eflag=wblog_check_tag(fmt,"ERR",tag); if (!eflag) {
00671     wflag=wblog_check_tag(fmt,"WRN",tag); if (!wflag) {
00672         if (fmt[0]=='\b') { fmt+=1; }
00673         else {
00674             unsigned i=0, k,l;

```

```

00675         for (; fmt[i]; ++i) { if (fmt[i]!='\n') break; }
00676         for (k=i, l=k+3; i<l && fmt[i]; ++i) {
00677             if (fmt[i]=='%' || !isprint(fmt[i])) break; }
00678
00679         if (i==l && fmt[i]==' ') {
00680             memcpy(tag,fmt+k,3); tag[3]=0; iflag=i+1;
00681         }
00682
00683         if (xcol_ && iflag) {
00684             xcol=Wb::termcolor::ID(tag);
00685         }
00686     }
00687 }
00688 else { iflag=wflag; }
00689
00690 if ((log_level & 15)==0) {
00691     if (!eflag && !iflag) return 0; }
00692
00693 if (xcol_ && !xcol) {
00694     xcol=Wb::termcolor::ID(tag);
00695 }
00696
00697 if (eflag || iflag) fmt+=(eflag+iflag);
00698 if (eflag) Sb.cat("\n");
00699
00700 #pragma omp critical (got_CPTR_TIME)
00701 { time_t curr_time=time(NULL);
00702     struct tm *tblock=localtime(&curr_time);
00703
00704     l=strftime(time_stamp,nt,"%a, %d %b %Y %T",tblock);
00705
00706     if (!l) {
00707         snprintf(time_stamp,nt,"Sat 22 Jul 1972 12:00:00%s",
00708             WBLOG_MSEC ? ".000" : "");
00709     }
00710     else if (WBLOG_MSEC && l+4<nt) {
00711         using namespace std::chrono;
00712         time_point<system_clock> t=system_clock::now();
00713
00714         snprintf(time_stamp+l,nt-l,"%03ld",
00715             size_t(duration_cast<milliseconds>(t.time_since_epoch()).count())%1000);
00716     }
00717
00718     if (then && then!=tblock->tm_yday) {
00719         char dstr[nt];
00720         l=strftime(dstr,nt,"%d-%b-%Y %T", tblock);
00721         Sb.catf("\n» TODAY %s (%s)\n",l? dstr:"??",myname);
00722     }
00723     then=tblock->tm_yday;
00724 }
00725
00726 if (!fmt[0] && !eflag && !iflag) {
00727     return 0;
00728 }
00729
00730 init_header(log_header,hlen,file,line,time_stamp,tag,
00731     xcol ? xcol : (xcol_ ? -1 : 0));
00732
00733 for (;;) {
00734     if (fmt[0]=='%' && fmt[1]!='N') { Sb.cat("\n"); fmt+=2; }
00735     else break;
00736 }
00737
00738 if (hflag) { Sb.cat(log_header); hflag=0; }
00739
00740 i=isfmt=0;
00741
00742 while ((c=(*fmt++)) {
00743     if (c=='\f') { fwd|=1; continue; }
00744     if (c=='\n') { fwd ^= 2; } else
00745     if (c=='\"') { fwd ^= 4; }
00746
00747     fstr[i++]=c;
00748
00749     if (i>124) { fstr[i-1]=0;
00750         snprintf(str,512,
00751             "ERR fstr out of bounds %d/128 (%s'%s)",i,fstr,fmt-1);
00752         ExitMsg(str);
00753     }
00754
00755     if (c=='%') {
00756         if (fmt[0]=='%' && !isfmt) { fstr[i]=fmt[0]; ++fmt; ++i; } else
00757         if (i<2 || fstr[i-2]!='\\') { ++isfmt; }
00758         continue;
00759     }
00760 }
00761

```

```

00762
00763     if (c=='\n') {
00764         fstr[i-1]=0; Sb.catf("%s\n",fstr);
00765         if (fmt[0]=='\n') {
00766             for (j=1; fmt[j]; ++j) { if (fmt[j]!='\n') break; }
00767             if (!fmt[j]) { Sb.cat(fmt); break; }
00768         }
00769         else {
00770             fmt+=check_update_header(log_header,fmt);
00771             Sb.cat(log_header);
00772             i=isfmt=0;
00773         }
00774     }
00775     else if (c=='\e' || (c=='\\' && *fmt=='e')) { ++nesc; }
00776
00777     if (!isfmt || isdigit(c) || strchr("%-+L",c)) { continue; }
00778
00779     if (c=='s') {
00780         fstr[i]=0;
00781         if (i>2 && fstr[i-2]=='%') {
00782             fstr[i-2]=0; Sb.cat(fstr);
00783             strcpy(fstr,"%s"); i=2;
00784         }
00785
00786         char *s0=va_arg(args, char*);
00787
00788         if (!s0) { Sb.catf(fstr,"(null)"); } else {
00789
00790             for (k=0; s0[k]>=30; ++k);
00791
00792             if ((!s0[k] && k<4) || i>2) { Sb.catf(fstr,s0); }
00793             else {
00794                 char ck=s0[k]; if (ck) { k+=strlen(s0+k); }
00795                 char s1_[k+1]; char *s1=s1_;
00796                 strcpy(s1,s0);
00797                 for (k=0; s1[k] || !k;) {
00798                     while (s1[k]) {
00799                         if (s1[k]=='\n') {
00800                             Sb.catf("\n%s",log_header);
00801                             s1+=(k+1); k=0;
00802                         }
00803                         else if (isspace(s1[k])) { ++k; }
00804                         else break;
00805                     }
00806
00807                     k=(fwd ? 0 : check_update_header(log_header,s1,"> "));
00808                     if (k) { s1+=k; k=0;
00809                         if (!ck && *fmt==' ') { ++fmt; }
00810
00811                         Sb.catf("\r%s",log_header);
00812                     }
00813
00814                     for (k=0; s1[k]; ++k) {
00815                         if (s1[k]=='\f') { s1[k]='\n'; } else
00816                         if (s1[k]=='\n') break;
00817                     }
00818
00819                     if (s1[k]) {
00820                         for (j=k+1; s1[j]; ++j) { if (s1[j]!='\n') break; }
00821                         if (!s1[j]) { Sb.catf("%s",s1); break; }
00822
00823                         s1[j-1]=0;
00824                         Sb.catf("%s\n",s1);
00825                         Sb.cat(log_header);
00826                         s1+=j; k=0;
00827                     }
00828                     else {
00829                         j=strlen(s1); if (j) {
00830                             if (s1[j-1]=='\\') { bflag|=2;
00831                                 s1[j-1]=0;
00832                             }
00833                             Sb.catf(fstr,s1); break;
00834                         }
00835                     }
00836                 }
00837                 i=isfmt=0;
00838             }
00839             else if (strchr("dXp",c)) {
00840                 fstr[i]=0;
00841                 m=0; if (i>1) if (fstr[i-2]=='l') m++;
00842                 if (m) Sb.catf(fstr, va_arg(args, long));
00843                 else Sb.catf(fstr, va_arg(args, int ));
00844
00845                 i=isfmt=0;
00846             }
00847             else if (c=='D') {
00848                 char s[nt]; int j,l,q;

```

```

00849
00850     if (i>1 && fstr[i-2]=='l') {
00851         fstr[i-2]='s'; fstr[i-1]=0;
00852         l=snprintf(s,nt,"%ld",va_arg(args,long));
00853     }
00854     else {
00855         fstr[i-1]='s'; fstr[i]=0;
00856         l=snprintf(s,nt,"%d", va_arg(args,int ));
00857     }
00858
00859     j=l+(1-(s[0]!='-' ? 1 : 2))/3;
00860     if (j>1 && j<int(nt)) {
00861         s[j]=0; q=(l-1)%3;
00862         for (--l, --j; l>0 && j>1; --j) {
00863             s[j]=s[l--]; if (l%3==q) { s[--j]=','; }
00864         }
00865     }
00866     Sb.catf(fstr,s);
00867     i=isfmt=0;
00868 }
00869 else if (c=='c') {
00870     fstr[i]=0;
00871     Sb.catf(fstr, va_arg(args, int ));
00872     i=isfmt=0;
00873 }
00874 else if (strchr("eEgf",c)) {
00875     fstr[i]=0;
00876     m=0; if (i>1) if (fstr[i-2]=='L') m++;
00877     if (m) Sb.catf(fstr, va_arg(args, long double));
00878     else Sb.catf(fstr, va_arg(args, double));
00879     i=isfmt=0;
00880 }
00881 else if (c=='N') {
00882     fstr[i-2]=0;
00883     Sb.catf("%s\n",fstr);
00884     i=isfmt=0;
00885 }
00886 else if (c=='Y') {
00887     fstr[i-2]=0; l=16;
00888     char c_; c_=time_stamp[l];
00889     time_stamp[l]=0; Sb.catf("%s%s",fstr,time_stamp);
00890     time_stamp[l]=c_;
00891
00892     i=isfmt=0;
00893 }
00894 else if (c=='R') {
00895     k=i; fstr[--i]=0;
00896
00897     for (; int(i)>=0; --i) { if (fstr[i]=='%') break; }
00898
00899     if (int(i)<0)
00900     Sb.catf("%s:%d ERR Invalid format >%s<\n", FLINE, fmt-k);
00901     else {
00902         fstr[i] = 0;
00903         Sb.catf("%s",fstr);
00904
00905         sscanf(fstr+i+1, "%d", &k);
00906
00907         sprintf(fstr, "%s", va_arg(args, char *));
00908
00909         for (; k>0; k--)
00910             Sb.catf("%s", fstr);
00911     }
00912     i=isfmt=0;
00913 }
00914 else if (c=='B') {
00915
00916     k = i; fstr[--i]=0;
00917
00918     for (; int(i)>=0; i--)
00919         if (fstr[i]=='%') break;
00920
00921     if (int(i)<0)
00922     Sb.catf("%s:%d ERR Invalid format >%s<", FLINE, fmt-k);
00923     else {
00924         fstr[i] = 0;
00925         Sb.catf("%s",fstr);
00926
00927         i = sscanf(fstr+i+1, "%d", &k);
00928         if (i<=0) k=8;
00929
00930         m = va_arg(args, int);
00931         cp = (char*)&m;
00932
00933         if (k>0)
00934             for (i=1; i<((k-1)/8+1); i++) {
00935                 for (cb=1, j=0; j<CHAR_BIT; ++j, cb<=1) {

```

```

00936             if (cp[i] & cb) Sb.catf("l");
00937             else Sb.catf("0");
00938             if ((++l)>=k) break;
00939         }
00940     }
00941 }
00942 i=isfmt=0;
00943 }
00944 else {
00945     fstr[i]=0; i=isfmt=0;
00946     Sb.catf(" »%s« ERR-fmt [..%s]\n",fstr,fmt);
00947     if (xcol) { Sb.cat("\e[0m"); }
00948     return -1;
00949 }
00950 }
00951 fstr[i]=0;
00952
00953 if (i && (fstr[i-1]!='\\')) { fstr[i-1]=0; bflag|=2; }
00954 if (!(bflag & 2)) strcat(fstr,"\n");
00955
00956 Sb.catf("%s",fstr);
00957 if (xcol) { Sb.cat("\e[0m"); }
00958
00959 if (nesc && !xcol_) {
00960     Sb.skipEscCols();
00961 }
00962
00963 if (eflag) { rval|=8; }
00964 if (wflag) { rval|=4; }
00965 if (iflag) { rval|=2; }; rval|=1;
00966
00967 #ifdef DBG_GCX_LOCKS
00968     if (!Hflag && (eflag || wflag)) { LKF.flush(); }
00969 #endif
00970
00971 if (eflag) { if (Hflag) { ++wblog::ERR_pending; } else {
00972     #ifdef DBSTOP
00973     { unsigned n=wblog::myIO.size();
00974         if (n) {
00975             fprintf(stdout,"\n"
00976                 "Got %d pending entr%s in I/O buffer:\n\n",n,n==1?"y":"ies");
00977             for (const wblog::stdio_buf &b : wblog::myIO) {
00978                 b.print_stdout(stdout);
00979             }
00980             Sb.print(FL,"DBSTOP"); fflush(stdout);
00981         }
00982     }
00983     #endif
00984     throw Wb::LogException(ERR);
00985 }}
00986 #ifdef DBSTOP
00987     else if (wflag) {
00988         Sb.print(FL,"DBSTOP");
00989         dbstop(file ? file : __FILE__, file ? line : __LINE__);
00990     }
00991 #endif
00992
00993 if (!Hflag) { wblog::check_ERR_pending(); }
00994
00995 return rval;
00996 };
00997
00998 void wblog::check_ERR_pending() {
00999     if (wblog::ERR_pending) { char s[32];
01000         int q=wblog::ERR_pending; wblog::ERR_pending=0;
01001         snprintf(s,32,"pending error%s (e=%d)",q>1? "s":"",q);
01002         ExitMsg(s);
01003     }
01004 };
01005
01006 unsigned wblog_check_tag(const char *fmt, const char *t0, char *tag) {
01007
01008     const char *s;
01009     int i,k,l=strlen(t0); if (!l) { return 0; }
01010
01011     s=strstr(fmt,t0); if (!s) { return 0; }
01012     k=s-fmt;
01013
01014     if (isalnum(s[l])) { return 0; }
01015     for (i=0; i<k; ++i) {
01016         if (isspace(fmt[i])) { return 0; }
01017     }
01018
01019     memcpy(tag,t0,l); tag[l]=0;
01020
01021     l+=k; if (fmt[l]) { ++l; }
01022

```



```

01023     return l;
01024 };
01025
01026 char wblog_findtoken(const char *istr, const char *tok, int maxoffset) {
01027     char f=1; const char *c=strstr(istr,tok);
01028
01029     if (c==0) return 0;
01030     if (maxoffset>=0) if (c>istr+maxoffset) return 0;
01031
01032     if (c>istr)
01033         if (isalnum(*(c-1))) f=0;
01034         if (isalnum(*(c+3))) f=0;
01035     return f;
01036 }
01037
01038 void wblog::SBUF::flush(FILE *fid, char fflag) {
01039
01040     if (!slen || !sbuf) { return; }
01041
01042     if (l && sbuf[0]) {
01043         #ifdef MATLAB_MEX_FILE
01044         if (fid==stdout || fid==stderr) { int tid=omp_get_thread_num();
01045             if (tid!=Wb::my_caller_tid) {
01046                 wblog::myIO.push_back(wblog::stdio_buf(*this,fid,tid));
01047             }
01048             else {
01049                 wblog::myIO.clear();
01050                 if (Wb::envDKT==1) { mexPrintf("%s",sbuf); }
01051                 else {
01052                     PRINTF("%s",sbuf);
01053                     fflush(stdout);
01054                 }
01055             }
01056         }
01057         else { fprintf(fid,"%s",sbuf); fflush(fid); }
01058         #else
01059         { fprintf(fid,"%s",sbuf); fflush(fid); }
01060         #endif
01061         if (fflag && (fid==stdout || fid==stderr)) { doflush(); }
01062     }
01063     if (sbuf) { sbuf[0]=0; l=0; }
01064 };
01065
01066 void wblog::SBUF::print(const char *F, int L, const char *istr) {
01067     if (!l) return;
01068     if (!slen || l>slen) { fprintf(stderr,
01069         "\n\n%s %s got l=%d/%d !?\n\n",shortFLT,l,slen);
01070         return;
01071     }
01072
01073     unsigned n=l+64, j=(sbuf[0]=='\n' ? 1:0); char s[n];
01074     char *sj=sbuf+j, c=sbuf[l]; sbuf[l]=0;
01075
01076     if (istr) { snprintf(s,n,
01077         "\n%s (%s in %s)\n%s", shortFL(F_L),istr,myname,sj);
01078     }
01079     else if (F) { snprintf(s,n,"\n%s\n%s",shortFL(F,L),sj); }
01080     else { snprintf(s,n,"\rTST> %s",sj); }
01081
01082     PRINTF("\e[3dm%s\e[0m\n",j?1:5,s);
01083     sbuf[l]=c;
01084 };
01085
01086 void wblog::SBUF::increase_size(const char *F, int L, unsigned n) {
01087     if (!slen || !sbuf || l>=slen) {
01088         error_bounds(F_L,FCT,"");
01089         ExitMsg("");
01090     }
01091     else if (wblog::SLEN<=32) {
01092         char s[64]; snprintf(s,64,"\n\n%s ERR got %p @ %d, L=%d !?\n\n",
01093             shortFL(F_L),sbuf,slen,wblog::SLEN);
01094         ExitMsg(s);
01095     }
01096     else if (n>=slen) {
01097         unsigned nx=(1+(n+1)/wblog::SLEN)*wblog::SLEN;
01098         char *sx = new char[nx];

```

```

01110     if (!sx) { char s[64]; snprintf(s, 64,
01111         "\n\n%s ERR allocation error (n=%d)\n\n", shortFL(F_L), nx);
01112         ExitMsg(s);
01113     }
01114     sbuf[slen-1]=0;
01115     strcpy(sx, sbuf); delete [] sbuf; sbuf=sx; slen=nx;
01116
01117     #if 0
01118     nx=strlen(sbuf);
01119     fprintf(stderr, "%s TST increasing sbuf[ %d -> %d ] having "
01120         "l=%d+d (%d)\n", SHORT_FL, nx+1, slen, l, n-1, n);
01121     if (Wb::envVRB & 8)
01122         fprintf(stderr, "%s' (%X)\n\n", sbuf, Wb::envVRB);
01123     #endif
01124 }
01125 };
01126
01127 void wblog::SBUF::error_bounds(
01128     const char *F, int L, const char *fct, const char *fmt) {
01129
01130     if (sbuf && slen) {
01131         sbuf[slen-1]=0;
01132         fprintf(stderr,
01133             "\n\n%s ERR %s() string out of bounds (l=%d%ld/%d):\n\n%s\n\n",
01134             shortFL(F_L), fct?fct:"(fct)", l, fmt ? strlen(fmt):-1, slen, sbuf);
01135         if (fmt) fprintf(stderr, " ERR fmt = '%s'\n\n", fmt);
01136         if (l>=slen) ExitMsg("");
01137     }
01138     else {
01139         char s[128]; snprintf(s, 128,
01140             "\n\n%s ERR %s() got uninitialized %p @ l=%d%ld/%d !?\n\n",
01141             shortFL(F_L), fct?fct:"(fct)", sbuf, l, fmt ? strlen(fmt):-1, slen);
01142         ExitMsg(s);
01143     }
01144 };
01145
01146 int wblog::SBUF::cat(const char *s) {
01147     int n=0;
01148
01149     if (s && s[0]) {
01150         n=strlen(s); if (l+n>=slen) { increase_size(F_L, l+n); }
01151         strcpy(sbuf+l, s); l+=n;
01152     }
01153     return n;
01154 };
01155
01156 int wblog::SBUF::catf(const char *fmt, ...) {
01157
01158     if (!fmt || !fmt[0]) { return 0; }
01159
01160     va_list args; Wb::ARGV wd(&args);
01161     va_start(args, fmt);
01162
01163     unsigned iter=0, n=strlen(fmt)+32;
01164     if (l+n>=slen) { increase_size(F_L, l+n); }
01165     if (l>=slen) {
01166         fprintf(stdout, "%s %s ERR got l=%d/%d (%d)", shortFLT, l, slen, n);
01167         fflush(stdout);
01168     }
01169
01170     while (1) {
01171         n=vsnprintf(sbuf+l, slen-l, fmt, args);
01172         if (l+n<slen) { break; } else increase_size(F_L, l+n);
01173
01174         va_end(args); va_start(args, fmt);
01175         if (++iter>2) fprintf(stdout, "%s %s ERR got iter=%d", shortFLT, iter);
01176     }
01177
01178     if (iter>1) {
01179         fprintf(stderr, "%s TST increasing sbuf[%d->%d] having "
01180             "l=%d%ld (%d)\n", SHORT_FL, wblog::SLEN, slen, l, n, l+n);
01181         if (Wb::envVRB & 8)
01182             fprintf(stderr, "\n`%s' (%X)\n\n", sbuf, Wb::envVRB);
01183     }
01184     l+=n; return n;
01185 };
01186
01187 int wblog::SBUF::skipEscCols() {
01188     int nesc=0, nskip=0;
01189     if (!sbuf) { return (nesc=-1); }
01190     unsigned i=0, k=0, j=0;
01191
01192     for (; sbuf[i]; ++i) {
01193         if (sbuf[i]=='\\e' && sbuf[i+1]=='[') { j=i+2; } else
01194             if (sbuf[i]=='\\' && sbuf[i+1]=='e' && sbuf[i+2]=='[') { j=j+3; }
01195             if (!j) { if (k<i) { sbuf[k]=sbuf[i]; }; ++k; }

```

```

01197     else {
01198         for (; sbuf[j]; ++j) {
01199             if (!isdigit(sbuf[j]) && sbuf[j]!=';') { break; }
01200         }
01201         if (sbuf[j]=='m')
01202             { ++nesc; i=j; }
01203         else { ++nskip; if (k<i) { sbuf[k]=sbuf[i]; }; ++k; }
01204         j=0;
01205     }
01206 }
01207
01208 if (k<i) { sbuf[k]=0; l=k; }
01209
01210 if (!nesc) { nesc=-nskip; }
01211 return nesc;
01212 };
01213
01214 char* Wb::surdStrf(wbstring &s, const char *fmt, ...) {
01215     va_list args; Wb::ARGV wd(&args);
01216     va_start(args,fmt);
01217     if (!s) { s.init(32); }
01218     return surdStrf(s.data,s.len,fmt,args);
01219 };
01220
01221 char* Wb::surdStrf(char *s, unsigned n, const char *fmt, va_list args) {
01222
01223     if (!s || n<8) wblog(FL,"ERR %s() invalid usage (s=%g, n=%d)",FCT,s,n);
01224     if (!fmt || !fmt[0]) { s[0]=0; return s; }
01225
01226     unsigned i,k,l;
01227
01228     k=std::wcerrtomb(s,u'\u221A',NULL);
01229
01230     i=l=k+1;
01231     l+=vsprintf(s+l,n-l,fmt,args);
01232
01233     for (; i<n && s[i]; ++i) { if (s[i]<'0' || s[i]>'9') break; }
01234
01235     if (l+(s[i] ? 1:0)>=n) wblog(FL,
01236         "WRN %s() string out of bounds (%d/%d)\n'%s'",FCT,l,n,s);
01237
01238     if (s[i]) {
01239         s[k]='('; s[l]=')'; i=0;
01240     }
01241     else {
01242         for (i=k; i; --i) { s[i]=s[i-1]; }
01243         s[i++]=' ';
01244     }
01245     return s+i;
01246 };
01247
01248 Wb::termcolor& Wb::termcolor::init(unsigned k) {
01249
01250     if (em) { delete [] em; em=e1=NULL; }
01251
01252     if (!Wb::useCol) { em=e1 = new char[1]; em[0]=0; }
01253     else {
01254         unsigned l, n=(k<8 ? 12 : 18);
01255         em = new char[n];
01256         l=snprintf(em,n,"e[0m");
01257         e1 = em + (++l);
01258
01259         if (k< 8) { l+=snprintf(e1,n-l,"e[3%dm",k); } else
01260         if (k<256) { l+=snprintf(e1,n-l,"e[38;5;%dm",k); }
01261         else wblog(FL,"ERR %s() color index out of bounds (%d)",FCT,k);
01262
01263         if (l>=n) wblog(FL,
01264             "WRN %s() termcolor() string out of bounds (%d/%d)",FCT,l,n);
01265     }
01266
01267     return *this;
01268 };
01269
01270 int Wb::termcolor::ID(const char *tag) {
01271
01272     int i=0;
01273
01274     if (Wb::useCol && tag && tag[0]) {
01275         switch (tag[0]) {
01276             case 'h':
01277                 if (!strcmp(tag,"hdr")) { i=240; }
01278                 break;
01279             case 'E':
01280                 if (!strcmp(tag,"ERR")) { i= 1; } else
01281                 if (!strcmp(tag,"ENV")) { i=240; }
01282                 break;
01283             case 'W':

```

```

01284         if (!strcmp(tag,"WRN")) { i= 9; }
01285         break;
01286     case ' ':
01287         if (!strcmp(tag," *")) { i=243; } else
01288             if (!strcmp(tag," * ")) { i=246; }
01289         break;
01290     case 'T':
01291         if (!strcmp(tag,"TST")) { i=246; }
01292         break;
01293     case 'N':
01294         if (!strcmp(tag,"NB!")) { i= 34; }
01295         break;
01296     case 'D':
01297         if (!strcmp(tag,"DBG")) { i=130; }
01298         break;
01299     case 'X':
01300         if (tag[1]=='X') {
01301             if (tag[2]=='E') { i=1; } else
01302                 if (tag[2]=='W') { i=9; } else
01303                     if (tag[2]=='G') { i=2; } else
01304                         if (tag[2]=='Y') { i=3; } else
01305                             if (tag[2]=='B') { i=4; }
01306         }
01307         break;
01308     }}
01309     return i;
01310 };
01311 };
01312
01313 #endif
01314

```

5.43 wblog.h

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wblog (logging routines)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_LOG_H__
00021 #define __WB_LOG_H__
00022
00023 // e.g. set in matlab via
00024 // » setenv('WB_VERBOSE',num2str(1<10 + 1<21));
00025 // » setenvb WB_VERBOSE 10 21
00026 // read in wblib.h -> envVRB=gathered.cc -> get_WB_VERBOSE()
00027 // default 0xF1; // 0xF1 = 241: bit 0 and 4..7
00028
00029 #define WBL_MSEC__      (1U<10)
00030 #define WBL_TCAST__    (1U<12)
00031
00032 #define WBL_CLK__      (1U<20)
00033 #define WBL_CTR__      (1U<21)
00034 #define WBL_MMEX__     (1U<22)
00035
00036 #define WBL_IO__       (1U<23)
00037 #define WBL_RESIZE__   (1U<24)
00038 #define WBL_RLARGE__   (1U<25)
00039
00040 #define WBLOG_MSEC      (Wb::envVRB & WBL_MSEC__ )
00041 #define WBLOG_TCAST     (Wb::envVRB & WBL_TCAST__ )
00042 #define WBLOG_CLK       (Wb::envVRB & (WBL_CLK__ | 192))
00043 #define WBLOG_CTR       (Wb::envVRB & WBL_CTR__ )
00044 #define WBLOG_MMEX      (Wb::envVRB & WBL_MMEX__ )
00045 #define WBLOG_IO        (Wb::envVRB & WBL_IO__ )
00046 #define WBLOG_RESIZE    (Wb::envVRB & WBL_RESIZE__ )
00047 #define WBLOG_RLARGE    (Wb::envVRB & WBL_RLARGE__ )
00048
00049 #define WBL_HLEN 20

```

```

00050 #define WBL_GOT_SHORTFL -99
00051
00052 const char* shortFL(
00053     const char *F, int L=-1, unsigned n=WBL_HLEN,
00054     const char *P=NULL, char sep=':');
00055
00056 const char* tmpmatFL(
00057     const char *F, int L, const char *vn, char *s, unsigned n);
00058
00059 enum WBLOG_TYPE { OK, WRN, ERR, NUM_TYPE };
00060
00061 const char* WBLOG_TYPE_STR[NUM_TYPE] = { "OK","WRN","ERR" };
00062
00063 namespace Wb {
00064
00065     struct LogException : public exception {
00066     public:
00067         LogException (WBLOG_TYPE l=OK, const char *s=NULL);
00068
00069         LogException (const LogException &e)
00070             : type(e.type), nrefs(e.nrefs), count(e.count), ith(e.ith), nth(e.nth) {
00071             if (nrefs) { ++(*nrefs); ++count;
00072             }
00073             istr[63]=0;
00074             strncpy(istr,e.istr,64);
00075             if (istr[63]) { istr[61]=istr[62]='.'; istr[63]=0; }
00076         };
00077
00078         ~LogException() noexcept {
00079             if (nrefs) {
00080                 if (--(*nrefs)<1) {
00081                     delete nrefs; nrefs=NULL;
00082                 }
00083             } else {
00084             }
00085         };
00086
00087         LogException& operator=(const LogException &e) {
00088             type=e.type; count=e.count; ith=e.ith; nth=e.nth;
00089             nrefs=e.nrefs; if (nrefs) { ++(*nrefs);
00090             }
00091             istr[63]=0;
00092             strncpy(istr,e.istr,64);
00093             if (istr[63]) { istr[61]=istr[62]='.'; istr[63]=0; }
00094             return *this;
00095         };
00096
00097         LogException& operator++() {
00098             ++count;
00099
00100             int n=omp_get_num_threads();
00101             if (nth<n) { nth=n; ith=omp_get_thread_num(); }
00102
00103             return *this;
00104         };
00105
00106         LogException& operator+=(const LogException &e) {
00107             if (!type)
00108                 { (*this)=e; count=1; }
00109             else { ++count; }
00110
00111             int n=omp_get_num_threads();
00112             if (nth<n) { nth=n; ith=omp_get_thread_num(); }
00113
00114             return *this;
00115         };
00116
00117         bool operator!() const { return (!type && !count); }
00118         explicit operator bool() const { return (type || count); }
00119
00120         LogException& init() {
00121             if (nrefs) { --(*nrefs);
00122                 if (*nrefs<1) { delete nrefs; nrefs=NULL; }
00123             }
00124             type=OK; count=ith=nth=0; istr[0]=0;
00125             return *this;
00126         };
00127
00128         const char* what() const throw () { return istr; };
00129
00130         wstring toStr() const;
00131         void report(const char *F=0, int L=0, const char *func=0);
00132
00133         WBLOG_TYPE type;
00134         int *nrefs;
00135         char istr[64];

```

```

00137         unsigned count;
00138         int ith, nth;
00139
00140     private:
00141     };
00142
00143     char* surdStrf(wbstring &s, const char *fmt, ...);
00144     char* surdStrf(char *s, unsigned n, const char *fmt, va_list args);
00145
00146     enum class TCOLS {
00147         BLACK, RED, GREEN, YLW, BLUE, PINK, CYAN, LGRAY, DGRAY,
00148         LRED, LGREEN, LYLW, LBLUE, LPNK, LCYAN, WHITE
00149     };
00150
00151     class termcolor {
00152     public:
00153
00154         termcolor(const char *tag)
00155             : em(NULL), el(NULL) { int k=ID(tag); init(k); };
00156
00157         termcolor(Wb::TCOLS k)
00158             : em(NULL), el(NULL) { init(unsigned(k)); };
00159
00160         termcolor(unsigned k)
00161             : em(NULL), el(NULL) { init(k); };
00162
00163         ~termcolor() {
00164             if (em) { delete [] em; }
00165             em=el=NULL;
00166         }
00167
00168         termcolor& init(unsigned k);
00169
00170         static int ID(const char *tag);
00171
00172         char *em, *el;
00173
00174     private:
00175     };
00176
00177 };
00178
00179 namespace wblog {
00180
00181     class SBUF {
00182     public:
00183         SBUF(unsigned l0=wblog::SLEN) : l(0), slen(0), sbuf(NULL) {
00184             init(l0);
00185         };
00186
00187         ~SBUF() { if (sbuf) {
00188             #pragma omp critical (__ensure_sequential_wblog__)
00189             { flush(stderr,1); }
00190             delete [] sbuf;
00191         } };
00192
00193         SBUF& init(unsigned l0=0) { l=0;
00194             if (sbuf) { delete [] sbuf; sbuf=NULL; }
00195             if ((slen=l0)) {
00196                 sbuf = new char[slen];
00197                 if (!sbuf) { sprintf(str,"ERR %s() "
00198                     "failed to allocate sbuf (l=%d)",__FUNCTION__,slen);
00199                     ExitMsg(str);
00200                 }
00201                 memset(sbuf,0,slen);
00202             }
00203             return *this;
00204         };
00205
00206         int cat(const char *s);
00207         int catf(const char *fmt, ...);
00208
00209         void error_bounds(const char *F, int L, const char *fct, const char *fmt);
00210         void increase_size(const char *F, int L, unsigned n);
00211
00212         void flush(FILE *fid=stdout, char fflag=0);
00213         void print(const char *F=0, int L=0, const char *istr=0);
00214
00215         int skipEscCols();
00216
00217         unsigned l,slen;
00218         char *sbuf;
00219
00220     private:
00221     };
00222
00223     class stdio_buf {

```

```

00224 public:
00225     stdio_buf(SBUF &S, FILE *f_, int t_) : s(NULL), fid(0), tid(0), ref(0) {
00226         if (f_!=stdin && f_!=stdout) {
00227             fprintf(stderr, "\nERR %s() invalid fid=%p\n' %s'\n\n",
00228                 FCT, f_, S.sbuf?S.sbuf:"");
00229         }
00230         if (S.sbuf) {
00231             s=S.sbuf; fid=f_; tid=t_; ref=1;
00232             S.sbuf=0; S.slen=S.l=0;
00233         }
00234     };
00235
00236     stdio_buf(const stdio_buf &b) : s(NULL), fid(0), ref(0) {
00237         if (!b.ref) fprintf(stderr, "\nERR %s() got b.ref=%d\n\n", FCT, b.ref);
00238         s=b.s; fid=b.fid; tid=b.tid;
00239     };
00240
00241     void print_stdout(FILE *f=NULL) const {
00242         if (Wb::envVRB &256) {
00243             unsigned i=0, m=0; for (; s[i]; ++i) {
00244                 if (s[i]!='\n') { if (++m>2) break; } else m=0;
00245             }
00246             if (m>2) { i-=2; char *x = (char*)s;
00247                 if (tid<10) x[i]=tid + '0'; else
00248                 if (tid<36) x[i]=tid-10+'a'; else
00249                 if (tid<62) x[i]=tid-36+'A'; else { x[i]='*'; }; x[i+1]=' ';
00250             }
00251         }
00252         if (f) { fprintf(f, "%s", s); }
00253         else {
00254             if (Wb::envDKT==1)
00255                 { mexPrintf("%s", s); }
00256             else { PRINTF("%s", s); }
00257         }
00258     };
00259
00260     ~stdio_buf() { if (s && !ref) {
00261         if (Wb::my_caller_tid!=omp_get_thread_num()) {
00262             fprintf(stderr, "WRN %s() got tid=%d/%d !?\nWRN '%s'\n",
00263                 FCT, omp_get_thread_num(), Wb::my_caller_tid, s);
00264         }
00265         else {
00266             print_stdout();
00267             delete [] s;
00268             fflush(fid);
00269         }
00270     } };
00271
00272 private:
00273     const char *s;
00274     FILE *fid;
00275     int tid;
00276     char ref;
00277 };
00278
00279     std::deque<stdio_buf> myIO;
00280
00281     void check_ERR_pending();
00282     int ERR_pending=0;
00283
00284 };
00285
00286     void wbSetLogLevel(unsigned l);
00287
00288     int wblogs(wblog::SBUF &S,
00289         WBL_COLOR_SCHEME xcol,
00290         const char* file, int line,
00291         const char *fmt, va_list args,
00292         char Hflag=0
00293     );
00294
00295     int wblogs(
00296         wblog::SBUF &S, WBL_COLOR_SCHEME xcol,
00297         const char* file, int line, const char *fmt, ...
00298     ){
00299         va_list args; Wb::ARGV wd(&args);
00300         va_start(args, fmt);
00301         return wblogs(S, xcol, file, line, fmt, args);
00302     };
00303
00304     int wblogf(FILE *fid,
00305         const char* file, int line, const char *fmt, va_list args);
00306
00307     int wblogf(FILE *fid,
00308         const char* file, int line, const char *fmt, ...) {
00309         va_list args; Wb::ARGV wd(&args);
00310

```

```

00311     va_start(args,fmt);
00312
00313     int l=wblogf(fid,file,line,fmt,args);
00314
00315     return l;
00316 };
00317
00318 int wblog1(const char* file, int line, const char *fmt, ...);
00319
00320 #define wblog(...) wblogf(stdout, __VA_ARGS__)
00321
00322 int wb_printf(const char *fmt, ...) {
00323
00324     unsigned l=0, n=256;
00325     wblog::SBUF S(n); {
00326         va_list args; Wb::ARGV wd(&args);
00327         va_start(args,fmt);
00328         l=vsprintf(S.sbuf,n,fmt,args);
00329     }
00330     if (l>=n) fprintf(stderr,
00331         "\nERR string out of bounds (%d/%d)\nERR %s\n\n",l,n,S.sbuf);
00332
00333     #pragma omp critical (__ensure_sequential_wblog__)
00334     { S.flush(); }
00335
00336     return l;
00337 };
00338
00339 unsigned wblog_check_tag(const char *fmt, const char *t, char *tag);
00340
00341 char wblog_findtoken(const char *istr, const char *tok, int maxoffset=-1);
00342
00343 void banner(
00344     unsigned n, const char *s,
00345     const char *istr=NULL, const char *fstr=NULL
00346 );
00347
00348 #endif
00349

```

5.44 wbMatIO.hh

```

00001 #ifndef __WB_MatIO_HCC__
00002 #define __WB_MatIO_HCC__
00003
00004 // IO handling of mat files (MatLab binaries)
00005
00006 namespace Wb {
00007
00008 class matFile {
00009 public:
00010
00011     matFile() : mfp(NULL) {};
00012
00013     matFile(const char *F, int L, const char *fname, const char *rw
00014 ) : mfp(NULL) { open(F,L,fname,rw,1); };
00015
00016     matFile(const matFile &X) : mfp(NULL) {
00017         wblog(FL,"ERR do not copy matFile object! (0x%lX)",&X);
00018     };
00019
00020 ~matFile() {
00021     init();
00022 };
00023
00024 void close() { init(); };
00025 void init() {
00026     if (mfp) if (matClose(mfp)) {
00027         wblog(F.data,L,"WRN Could not close file %s%s /%s !?",
00028             strlen(fname.data)>20 ? "\n" : "", fname.data, rw.data);
00029     }
00030     mfp=NULL; F.init(); L=0; fname.init(); rw.init();
00031 };
00032
00033 int Open(const char *F, int L, const char *FName, const char *RW
00034 ){ return open(F,L,FName,RW,1); };
00035
00036 int open(
00037     const char *file, int line, const char *FName, const char *RW,
00038     char xflag=0
00039 ){
00040     if (mfp) init();
00041     F=file; L=line; fname=FName;

```



```

00042         if (!RW || !RW[0]) wblog(F, L,
00043             "ERR %s() got empty read/write flag", FCT);
00044
00045         if (!strcmp(RW, "w")) {
00046             rw="w7.3";
00047         }
00048         else if (!strcmp(RW, "w:std")) {
00049             rw="wz";
00050         }
00051         else { rw=RW; }
00052
00053         mfp=matOpen(fname.data, rw.data);
00054         if (!mfp) {
00055             if (xflag) { unsigned n=strlen(fname.data);
00056                 if (n<50)
00057                     wblog(F, data, L, "ERR failed to %s access%s%s"
00058                         "\n(hint: non-existing directory?)",
00059                         rw.data, n<30? " ":" file\n", fname.data);
00060                 else {
00061                     char *x,cwd[256]; cwd[0]=0; x=getcwd(cwd,256);
00062                     wblog(F, data, L, "ERR failed to access (%s) ... %N%N"
00063                         "  mat: %s !?%N"
00064                         "  cwd: %s%N"
00065                         " (hint: non-existing directory or no rw permission)%N",
00066                     rw.data, fname.data, x ? cwd : "(pwd)");
00067                 }
00068             }
00069             init(); return 0;
00070         }
00071         return 1;
00072     };
00073
00074     matFile& put(
00075         const char *file, int line,
00076         const char *vname, const mxArray* a
00077     ){
00078         int i=matPutVariable(mfp,vname,a);
00079         if (i) {
00080             wblog(file ? file : F.data, line ? line : L,
00081                 "ERR matPutVariable() returned %d\n(%s,%s; 0x%lX) !?\n"
00082                 "hint: possibly out of disc space", i,
00083                 fname.data ? fname.data : "(undefined)",
00084                 vname ? vname : "(empty)", mfp
00085             );
00086         }
00087         return *this;
00088     };
00089     matFile& put(const char *vname, const mxArray* a){
00090         return put(0,0,vname,a);
00091     };
00092
00093     int existsVar(const char* vn) const {
00094         mxArray *a=matGetVariableInfo(mfp,vn);
00095         int r=(a!=NULL);
00096         mxDestroyArray(a);
00097         return r;
00098     };
00099
00100     int deleteVar(const char* vn) const {
00101         int i=matDeleteVariable(mfp,vn);
00102         return i;
00103     };
00104
00105     int deleteVar(const char *f, int L, const char* vn) const {
00106         int i=matDeleteVariable(mfp,vn);
00107         if (i) wblog(f,L,
00108             "ERR Failed to delete variable '%s'\n"
00109             "from file '%s' (0x%lX; %d/%d ?)",
00110             vn, fname.data, mfp, existsVar(vn), i);
00111         return i;
00112     };
00113
00114     MATFile *mfp;
00115
00116     wbsting F, fname, rw;
00117     int L;
00118
00119     protected:
00120     private:
00121 };
00122
00123 }
00124
00125 #endif
00126

```

5.45 wbMatrix.cc

```

00001  /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MATRIX_ROW_MAJOR_CC__
00021 #define __WB_MATRIX_ROW_MAJOR_CC__
00022
00023  /* ----- */
00024  /* ----- */
00025  // quick integer data type // Wb,Dec02,11
00026
00027  template<class T> inline
00028  bool wbMatrix<T>::thisIsInt() const { return 0; };
00029
00030  template<> inline
00031  bool wbMatrix<int>::thisIsInt() const { return 1; };
00032
00033  template<> inline
00034  bool wbMatrix<unsigned>::thisIsInt() const { return 1; };
00035
00036  template<> inline
00037  bool wbMatrix<size_t>::thisIsInt() const { return 1; };
00038
00039  template<> inline
00040  bool wbMatrix<char>::thisIsInt() const { return 1; };
00041
00042  template<> inline
00043  bool wbMatrix<unsigned char>::thisIsInt() const { return 1; };
00044
00045  template<>
00046  bool wbMatrix<double>::isNormal() const {
00047      for (size_t n=dim1*dim2, i=0; i<n; ++i)
00048          if (data[i] && !isnormal(data[i])) return 0;
00049      return 1;
00050  };
00051
00052  template<>
00053  bool wbMatrix<float>::isNormal() const {
00054      for (size_t n=dim1*dim2, i=0; i<n; ++i)
00055          if (data[i] && !isnormal(data[i])) return 0;
00056      return 1;
00057  };
00058
00059  template<>
00060  bool wbMatrix<wbcomplex>::isNormal() const {
00061      for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00062          if (data[i].r && !isnormal(data[i].r)) return 0;
00063          if (data[i].i && !isnormal(data[i].i)) return 0;
00064      }
00065      return 1;
00066  };
00067
00068  template<> bool wbMatrix<unsigned>::isNormal() const { return 1; };
00069  template<> bool wbMatrix<size_t>::isNormal() const { return 1; };
00070  template<> bool wbMatrix<int>::isNormal() const { return 1; };
00071  template<> bool wbMatrix<char>::isNormal() const { return 1; };
00072  template<> bool wbMatrix<long>::isNormal() const { return 1; };
00073
00074  template<class T> inline
00075  wbMatrix<T>& wbMatrix<T>::init(
00076      const char *F, int L, const mxArray *a,
00077      char tflag, char ref, char tcheck
00078  ){
00079      if (!a || mxIsEmpty(a)) { init(); return *this; }
00080
00081      unsigned r=mxGetNumberOfDimensions(a);
00082      const size_t *S=mxGetDimensions(a);
00083
00084      if (r!=2 || !mxIsNumChar(a)) wblog(F,L,
00085          "ERR got rank-%d array '%s'",r,mxGetClassName(a));

```

```

00086
00087     size_t d1=(tflag ? S[1]:S[0]), d2=(tflag ? S[0]:S[1]);
00088
00089     Mx::Array<T> A(F_L,a,!tflag);
00090     if (!ref || !A.data) { if (ref) wblog(FL,"WRN invalid ref: %s",STR(A));
00091         RENEW(d1,d2);
00092         A.copyTo(data,tcheck);
00093     }
00094     else { init2ref(d1,d2,A.data); }
00095
00096     return *this;
00097 };
00098
00099 template <class T> inline
00100 wbMatrix<T>& wbMatrix<T>::init2ref(size_t r, size_t c, const T* d) {
00101     if (data && !isref) { init(); }; isref=1;
00102     dim1=r; dim2=c; data=(T*)d;
00103     return *this;
00104 };
00105
00106 template <class T> inline
00107 wbMatrix<T>& wbMatrix<T>::init2ref(const wbvector<T> &v, const char tflag) {
00108     if (data && !isref) { init(); }; isref=1; data=v.data;
00109     if (tflag)
00110         { dim1=v.len; dim2=1; }
00111     else { dim1=1; dim2=v.len; }
00112     return *this;
00113 };
00114
00115 template <class T> inline
00116 wbMatrix<T>& wbMatrix<T>::init2ref(const wbMatrix<T> &M) {
00117     if (data && !isref) { init(); }; isref=1;
00118     dim1=M.dim1; dim2=M.dim2; data=M.data;
00119     return *this;
00120 };
00121
00122 template <class T>
00123 template<class TA, class TB>
00124 wbMatrix<T>& wbMatrix<T>::init(
00125     const char *F, int L,
00126     const wbMatrix<TA> &M, const wbvector<TB> &V, char dim
00127 ){
00128     size_t i,j,n1=M.dim1,n2=M.dim2;
00129     if (abs(dim)==2) {
00130         if (n1!=V.len) wblog(F_L,
00131             "ERR %s() size mismatch %d/%dx%d",FCT,V.len,n1,n2);
00132         RENEW(n1,n2+1,0,0);
00133         if (dim>0) {
00134             for (i=0; i<n1; ++i) {
00135                 for (j=0; j<n2; ++j) (*this)(i,j)=(T)M(i,j);
00136                 (*this)(i,j)=(T)V[i];
00137             }
00138         }
00139         else {
00140             for (i=0; i<n1; ++i) {
00141                 (*this)(i,0)=(T)V[i];
00142                 for (j=0; j<n2; ++j) (*this)(i,j+1)=(T)M(i,j);
00143             }
00144         }
00145     }
00146     else if (abs(dim)==1) {
00147         if (n2!=V.len) wblog(F_L,
00148             "ERR %s() size mismatch %dx%d/%d",FCT,n1,n2,V.len);
00149         RENEW(n1+1,n2,0,0);
00150         if (dim>0) {
00151             for (i=0; i<n1; ++i)
00152                 for (j=0; j<n2; ++j) (*this)(i+1,j)=(T)M(i,j);
00153             for (j=0; j<n2; ++j) (*this)(i,j)=(T)V[j];
00154         }
00155         else {
00156             for (j=0; j<n2; ++j) (*this)(0,j)=(T)V[j];
00157             for (i=0; i<n1; ++i)
00158                 for (j=0; j<n2; ++j) (*this)(i+1,j)=(T)M(i,j);
00159         }
00160     }
00161     else wblog(F_L,"ERR %s() invalid dim=%d",FCT,dim);
00162
00163     return *this;
00164 };
00165
00166 template <class T> inline
00167 int matchSortedIdxU(
00168     const char *F, int L,
00169     C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
00170     widx_t m,
00171     char lex
00172 ){

```

```

00173     widx_t m1=0, m2=0; int i=0;
00174     if (QA.dim2!=QB.dim2) wblog(FL,"ERR %s() dimension mismatch "
00175         "%dx%d <> %d/%d",FCT,QA.dim1,QA.dim2,QB.dim1,QB.dim2);
00176
00177     try {
00178         i=Wb::matchSortedIdx(
00179             QA.data, QA.dim2, QA.dim1,
00180             QB.data, QB.dim2, QB.dim1, Ia,Ib, m, lex, &m1, &m2
00181         );
00182         if (m1 || m2) wblog(F_L,"ERR %s() "
00183             "got non-unique/sorted input records (%d,%d)",FCT,m1,m2);
00184     }
00185     catch (...) {
00186 #ifdef MATLAB_MEX_FILE
00187         MXPput(FL,"q").add(QA,"A").add(QB,"B").add(int(m),"m").add(lex,"lex");
00188         wblog(FL,"ERR %s()",FCT);
00189 #endif
00190     }
00191
00192     return i;
00193 };
00194
00195 template<class T> inline
00196 bool wbMatrix<T>::quickCheckSorted(
00197     char dir,
00198     char lex,
00199     const char *F, int L,
00200     size_t n
00201 ) const {
00202
00203     char c; dir=(dir>0 ? +1 : -1);
00204
00205     for (size_t i=1; i<dim1; i++) {
00206         c=recCompare(i,i-1,-1,lex);
00207         if (c) {
00208             if (c!=dir) {
00209                 if (F) wblog(F,L,
00210                     "ERR %s() input records not sorted %s (%d)",
00211                     FCT, dir>0 ? "ascendingly":"descendingly", i);
00212                 else return 0;
00213             }
00214             if (int(--n)<=0) break;
00215         }
00216     }
00217
00218     return 1;
00219 };
00220
00221 template<>
00222 WBINDEX& wbMatrix<size_t>::toIndex(
00223     WBINDEX &I, const WBINDEX *S) const {
00224
00225     const size_t *idx=data;
00226     if (S==NULL) {
00227         WBINDEX SX(dim2); widx_t *s=SX.data; size_t i=0,j;
00228
00229         for (; i<dim1; ++i, idx+=dim2) {
00230             for (j=0; j<dim2; ++j) {
00231                 if (s[j]<idx[j]) s[j]=idx[j];
00232             }
00233         }
00234         SX+=1; return toIndex(I,&SX);
00235     }
00236
00237     I.init(dim1); if (!dim1) return I;
00238     if (!dim2) wblog(FL,
00239         "ERR %s() got empty index matrix (%dx%d)",FCT,dim1,dim2);
00240
00241     size_t i=0,j, l=dim2-1;
00242     const widx_t *s=S->data;
00243
00244     for (; i<dim1; ++i, idx+=dim2) { widx_t &q=I.data[i];
00245         for (j=l, q=idx[j--]; j<l; --j) { q = q*size_t(s[j]) + size_t(idx[j]); }
00246     }
00247
00248     return I;
00249 };
00250
00251 template<>
00252 WBIDXMAT& wbMatrix<size_t>::toIndex2D(
00253     const WBINDEX &S, const wbindex &Ic,
00254     WBIDXMAT &IJ, char pos
00255 ) const {
00256
00257     wvector<char> M(S.len);
00258     size_t i; char *m=M.data;
00259

```

```

00260     if (S.len!=dim2) wblog(FL,
00261         "ERR %s() size mismatch (%dx%d/%d)",FCT,dim1,dim2,S.len);
00262     if (!dim2) wblog(FL,
00263         "ERR %s() got empty index matrix (%dx%d)",FCT,dim1,dim2);
00264
00265     for (i=0; i<Ic.len; ++i) {
00266         if (Ic[i]>S.len) wblog(FL,"ERR %s() index out of bounds "
00267             "%s; %d",FCT,STR(Ic+1),S.len);
00268         if ((++m[Ic[i]])>1) wblog(FL,"ERR %s() index not unique "
00269             "%s; %d",FCT,STR(Ic+1),S.len
00270         );
00271     }
00272
00273     if ((--pos)>1) wblog(FL,
00274         "ERR %s() index out of bounds (pos=%d)",FCT,pos);
00275
00276     if (dim2==2 && Ic.len==1) {
00277         if (int(Ic[0])==pos) { IJ=(*this); }
00278         else {
00279             IJ.init(dim1,dim2);
00280             size_t d12=dim1*dim2; widx_t *d=IJ.data;
00281             for (i=0; i<d12; i+=2) {
00282                 d[i]=data[i+1];
00283                 d[i+1]=data[i];
00284             }
00285         }
00286         return IJ;
00287     }
00288
00289     IJ.init(dim1,2); if (!dim1) return IJ;
00290
00291     wbindx Ik(S.len-Ic.len);
00292     unsigned j=0, k=0, lk=Ik.len-1, lc=Ic.len-1;
00293     const size_t *idx=data, *s=S.data;
00294     const widx_t *ic=Ic.data; widx_t *ik=Ik.data;
00295
00296     for (i=0; i<S.len; ++i) { if (!m[i]) { ik[j++]=i; }}
00297
00298     for (i=0; i<dim1; ++i, idx+=dim2, k+=2) {
00299         if (Ik.len) {
00300             widx_t &I=IJ.data[k+(1-pos)]; I=idx[ik[lk]];
00301             for (j=lk-1; j<lk; --j) { if (I) I*=s[ik[j]]; I+=idx[ik[j]]; }
00302         }
00303         if (Ic.len) {
00304             widx_t &J=IJ.data[k+pos]; J=idx[ic[lc]];
00305             for (j=lc-1; j<lc; --j) { if (J) J*=s[ic[j]]; J+=idx[ic[j]]; }
00306         }
00307     }
00308
00309     return IJ;
00310 };
00311
00312 template<>
00313 wbMatrix<double>& wbMatrix<double>::SkipTiny_float(double x) {
00314     Wb::chopTiny_float(data,dim1*dim2,x);
00315     return *this;
00316 }
00317
00318 template<>
00319 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::SkipTiny_float(wbcomplex x) {
00320     Wb::chopTiny_float((double*)data,2*dim1*dim2,x.r);
00321     return *this;
00322 }
00323
00324 template <class T> inline
00325 size_t wbMatrix<T>::maxRec(char lex, size_t *m_) const {
00326
00327     if (!dim1 || !dim2) { wblog(FL,
00328         "WRN %s() got empty matrix (%d,%d)",FCT,dim1,dim2);
00329         return 0;
00330     }
00331
00332     size_t i=1, k=0, m=1; char c;
00333     for (; i<dim1; i++) {
00334         c=recCompare(k,i,-1,lex);
00335         if (c<0) { k=i; m=1; }
00336         else if (c==0) { m++; }
00337     }
00338
00339     if (m_) (*m_)=m;
00340     return k;
00341 };
00342
00343 template <> inline
00344 size_t wbMatrix<double>::maxRec_float(
00345     char lex, size_t *m_, double xref
00346 ) const {

```

```

00347
00348     wbMatrix<double> M(*this);
00349     Wb::chopTiny_float(M.data,dim1*dim2,xref);
00350     return M.maxRec(lex,m_);
00351 };
00352
00353 template <class T> inline
00354 T wbMatrix<T>::recMax(size_t r, size_t *k) const {
00355
00356     if (r>=dim1) wblog(FL,
00357         "ERR %s() index out of bounds (%d/%d)",FCT,r,dim1);
00358     if (!dim2) { wblog(FL,
00359         "WRN %s() from empty matrix (%dx%d)",FCT,dim1,dim2);
00360         return 0;
00361     }
00362
00363     T *d=data+r*dim2, x=d[0]; if (k) (*k)=0;
00364
00365     for (size_t i=1; i<dim2; ++i) {
00366         if (x<d[i]) { x=d[i]; if (k) (*k)=i; }
00367     }
00368
00369     return x;
00370 };
00371
00372 template <class T> inline
00373 T wbMatrix<T>::colMax(size_t c, size_t *k) const {
00374
00375     if (c>=dim2) wblog(FL,
00376         "ERR %s() index out of bounds (%d/%d)",FCT,c,dim2);
00377     if (!dim1) { if (k) (*k)=0; wblog(FL,
00378         "WRN %s() from empty matrix (%dx%d)",FCT,dim1,dim2);
00379         return 0;
00380     }
00381
00382     T *d=data+c, x=d[0]; d+=dim2; if (k) (*k)=0;
00383
00384     for (size_t i=1; i<dim1; ++i, d+=dim2) {
00385         if (x<d[0]) { x=d[0]; if (k) (*k)=i; }
00386     }
00387
00388     return x;
00389 };
00390
00391 template <class T> inline
00392 double wbMatrix<T>::recMaxA(size_t r, size_t *k) const {
00393     if (k) (*k)=0;
00394
00395     if (r>=dim1) wblog(FL,
00396         "ERR %s() index out of bounds (%d/%d)",FCT,r,dim1);
00397     if (!dim2) { wblog(FL,
00398         "WRN %s() from empty matrix (%dx%d)",FCT,dim1,dim2);
00399         return 0;
00400     }
00401
00402     T *d=data+r*dim2;
00403     double a=ABS(d[0]), amax=a;
00404
00405     for (size_t i=1; i<dim2; i++) {
00406         a=ABS(d[i]); if (amax<a) { amax=a; if (k) (*k)=i; }
00407     }
00408
00409     return amax;
00410 };
00411
00412 template <class T>
00413 double wbMatrix<T>::maxRelDiff(const wbMatrix<T> &M) const {
00414
00415     size_t i, s=dim1*dim2;
00416     double maxdiff=0., maxval=0.;
00417
00418     if (!sameSize(M)) wblog(FL,
00419         "ERR %s() incompatible matrix objects",FCT);
00420
00421     for (i=0; i<s; i++) {
00422         maxdiff=MAX(maxdiff, fabs(double(M.data[i]-data[i])));
00423         maxval=MAX(maxval,
00424             MAX( fabs(double(data[i])), fabs(double(M.data[i])) ) );
00425     }
00426
00427     return ((maxval==0.) ? 0. : maxdiff/maxval);
00428 };
00429
00430 template <class T>
00431 T wbMatrix<T>::normDiff2(const wbMatrix<T> &M, size_t *k) const {
00432
00433     if (!sameSize(M)) wblog(FL,

```

```

00434         "ERR %s() incompatible matrix objects (%dx%d, %dx%d)",
00435         FCT,dim1,dim2,M.dim1,M.dim2);
00436     T fac=1;
00437
00438     return Wb::rangeNormDiff2(M.data, data, dim1*dim2, fac, k);
00439 };
00440
00441 template <class T>
00442 template <class T2>
00443 double wbMatrix<T>::normDiff(const wbMatrix<T2> &M) const {
00444
00445     if (!sameSize(M)) wblog(FL,
00446         "ERR %s() incompatible matrix objects (%dx%d, %dx%d)",
00447         FCT,dim1,dim2,M.dim1,M.dim2);
00448
00449     double x, x2=0;
00450     for (size_t n=numel(), i=0; i<n; ++i) {
00451         x=(data[i]-M.data[i]); x2+=(x*x);
00452     }
00453
00454     return sqrt(x2);
00455 };
00456
00457 template<class T>
00458 bool wbMatrix<T>::isOrthoRows(T *x, char tnorm, T eps) const {
00459
00460     if (!dim1 || !dim2 || dim1>dim2) return 0;
00461
00462     wbarray<T> E, A(*this,'r');
00463     Wb::MatProd(A,A,E, tnorm? 'T':'C');
00464
00465     if (x)
00466         return E.isProptoId(*x,eps);
00467     else return E.isDiagMatrix();
00468 };
00469
00470 template<class T>
00471 bool wbMatrix<T>::isOrthoCols(
00472     const wbMatrix<T> &B_, T *x, char tnorm, T eps) const {
00473
00474     if (dim1>dim2 || B_.dim1>B_.dim2) return 0;
00475     if (!(*this) && !B_) return 0;
00476
00477     wbarray<T> E, A(*this,'r'), B(B_,'r');
00478     Wb::MatProd(A,B,E, tnorm? 'T':'C');
00479
00480     if (x)
00481         return E.isProptoId(*x,eps);
00482     else return E.isDiagMatrix();
00483 };
00484
00485 template <class T> inline
00486 wbMatrix<T>& wbMatrix<T>::NormalizeCol(size_t k) {
00487
00488     if (k>=dim2) wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
00489     if (!dim1) return *this;
00490
00491     T *d=data+k, x2 = Wb::overlap(d,d,dim1,dim2);
00492     Wb::timesRange(d,1/SQRT(x2),dim1,dim2);
00493
00494     return *this;
00495 };
00496
00497 template <class T> inline
00498 wbMatrix<T>& wbMatrix<T>::set2Cols(const size_t i1, const size_t i2){
00499     const wbMatrix<T> M; save2(M);
00500     M.getCols(i1,i2,*this); return *this;
00501 };
00502
00503 template <class T> inline
00504 wbMatrix<T>& wbMatrix<T>::getCols(
00505     size_t j1, size_t j2,
00506     wbMatrix<T> &M
00507 ) const {
00508
00509     if (&M==this) {
00510         wbMatrix<T> X; getCols(j1,j2,X);
00511         return X.save2(M);
00512     }
00513
00514     if (j1==0 && j2+1==dim2) return M.init(*this);
00515
00516     if (j1>=dim2 || j2>=dim2) {
00517         if (int(j1-j2)==1 && (j1==0 || j1==dim2)) {
00518             return M.init(dim1,0);
00519         }
00520         else wblog(FL,

```

```

00521     "ERR index out of bounds (%dx%d: %d,%d)",dim1,dim2,j1,j2);
00522 }
00523 if (j1>j2) return M.init(dim1,0);
00524
00525     size_t i=0, m=j2-j1+1; const T *p=data+j1;
00526
00527     M.init(dim1,m);
00528
00529     for (; i<dim1; ++i)
00530     MEM_COPY<T>(M.data+i*m, m, p+i*dim2);
00531
00532     return M;
00533 };
00534
00535 template <class T> inline
00536 wbvector<T>& wbMatrix<T>::getCol(
00537     const size_t k, wbvector<T> &v
00538 ) const {
00539
00540     size_t i=0; const T *p=data+k;
00541
00542     if (k>=dim2) wblog(FL,
00543         "ERR %s() index out of bounds (%dx%d/%d)",FCT,dim1,dim2,k);
00544     v.init(dim1);
00545     for (; i<dim1; ++i, p+=dim2) { v[i]=p[0]; }
00546
00547     return v;
00548 };
00549
00550 template <class T>
00551 template <class TI> inline
00552 wbMatrix<T>& wbMatrix<T>::getCols(
00553     size_t n, const TI *I, wbMatrix<T> &M) const {
00554
00555     wbMatrix<T> X(dim1,n); T *d0=data, *d=X.data;
00556     size_t i=0, j;
00557
00558     for (j=0; j<n; ++j) if (I[j]>=dim2) wblog(FL,
00559         "ERR %s() index out of bounds (%d: %d/%d)",FCT,j,I[j],dim2);
00560
00561     for (; i<dim1; ++i, d+=n, d0+=dim2) {
00562         for (j=0; j<n; ++j) d[j]=d0[I[j]]; }
00563
00564     X.save2(M); return M;
00565 };
00566
00567 template <class T> inline
00568 wbvector<T>& wbMatrix<T>::getRec(
00569     const size_t j0, wbvector<T> &v,
00570     char ref
00571 ) const {
00572
00573     if (j0>=dim1) wblog(FL,
00574         "ERR %s() index out of bounds (%d/%d)",FCT,j0,dim1);
00575     return v.init(dim2,rec(j0),ref);
00576 };
00577
00578 template <class T>
00579 wbMatrix<T>& wbMatrix<T>::getRecs(
00580     const WBINDEX &I, wbMatrix<T> &M
00581 ) const {
00582
00583     size_t i=0; M.init(I.len,dim2);
00584     for (; i<I.len; ++i) {
00585         if (I[i]>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",I[i],dim1);
00586         MEM_COPY<T>(M.data+i*dim2, dim2, data+I[i]*dim2);
00587     }
00588     return M;
00589 };
00590
00591 template <class T>
00592 wbMatrix<T>& wbMatrix<T>::getRecs(
00593     size_t i1, size_t i2, wbMatrix<T> &M,
00594     size_t m
00595 ) const {
00596
00597     if (i1>=dim1 || i2>=dim1) wblog(FL,
00598         "ERR index out of bounds (%d,%d/%dx%d)",i1,i2,dim1,dim2);
00599
00600     if (m==0) {
00601         if (i1>i2) { M.init(0,dim2); return M; }
00602         M.init(i2-i1+1,dim2);
00603         MEM_COPY<T>(M.data, M.dim1*dim2, data+i1*dim2);
00604     }
00605     else {
00606         bool lflag=(int(m)<0); m=abs(int(m));
00607         if (m>dim2) wblog(FL,

```



```

00608         "ERR %s() m=%d/%d out of bounds",FCT,m,dim2);
00609
00610     M.init(i1<=i2 ? i2-i1+1 : 0, m);
00611     if (M.dim1) {
00612         const T* d0=data+i1*dim2; if (lflag) d0+=(dim2-m);
00613
00614         for (size_t i=0; i<M.dim1; ++i, d0+=dim2) {
00615             MEM_COPY<T>(M.data+i*m, m, d0);
00616         }
00617     }
00618 }
00619
00620 return M;
00621 };
00622
00623 template <class T> inline
00624 wbMatrix<T>& wbMatrix<T>::getBlocks(
00625     size_t j1, size_t j2, const size_t D,
00626     wbMatrix<T> &M
00627 ) const {
00628
00629     size_t i,isnn, D2=2*D, s=D;
00630     T *p2,*p0;
00631
00632     if (D*(j1+1)>dim2 || D*(j2+1)>dim2 || (D ? dim2%D : 0)) wblog(FL,
00633     "ERR invalid block parameters (%d,%d,%d,%d).",j1, j2, D, dim2);
00634
00635     M.init(dim1,D2);
00636
00637     isnn=(j2==(j1+1));
00638     if (isnn) s+=s;
00639
00640     p0=data+D*j1; p2=M.data;
00641     for (i=0; i<dim1; i++) MEM_COPY<T>(p2+i*D2, s, p0+i*dim2);
00642
00643     if (!isnn) {
00644         p0=data+D*j2; p2=M.data+D;
00645         for (i=0; i<dim1; i++) MEM_COPY<T>(p2+i*D2, s, p0+i*dim2);
00646     }
00647
00648     return M;
00649 }
00650
00651 template <class T>
00652 void wbMatrix<T>::ColKron(
00653     const wbMatrix<T> &B,
00654     int d_
00655 ){
00656     if (&B==this) {
00657         wbMatrix<T> X(B); ColKron(X,d_);
00658         return;
00659     }
00660
00661     size_t i,j, da=dim1, d2=dim2, db=B.dim1;
00662     size_t d=( d_<0 ? B.dim2 : (size_t)d_);
00663
00664     if (d>B.dim2) wblog(FL,
00665     "ERR %s() index out of bounds (%d/%d)", FCT,d,B.dim2);
00666
00667     if (B.isEmpty() || d==0) return;
00668     if (isEmpty()) { *this = B; return; }
00669
00670     Repmat(db,1, 0,B.dim2);
00671
00672     T *dd, *d0;
00673     for (i=0; i<db; ++i) {
00674         d0=B.data+i*B.dim2; dd = data + i*da*dim2 + d2;
00675         for (j=0; j<da; ++j, dd+=dim2) MEM_COPY<T>(dd,d,d0);
00676     }
00677 };
00678
00679 template <class T>
00680 wbMatrix<T>& wbMatrix<T>::repmat(
00681     size_t m, size_t n, wbMatrix<T> &Q,
00682     size_t pad1, size_t pad2
00683 ) const {
00684
00685     if (this==&Q) {
00686         wbMatrix<T> X(*this);
00687         return X.repmat(m,n,Q,pad1,pad2);
00688     }
00689
00690     size_t r,i,j;
00691     size_t d1=dim1+pad1, d2=dim2+pad2, D1=m*d1, D2=n*d2;
00692
00693     Q.init(D1,D2);
00694

```

```

00695     for (i=0; i<m; i++)
00696     for (j=0; j<n; j++) {
00697         T *d = Q.data + (i*n*dl+j)*d2;
00698         for (r=0; r<dim1; r++, d+=D2) MEM_COPY<T>(d, dim2, data+r*dim2);
00699     }
00700
00701     return Q;
00702 }
00703
00704 template <class T> inline
00705 wbMatrix<T>& wbMatrix<T>::getBlock(
00706     size_t j1, const size_t D, wbMatrix<T> &M
00707 ) const {
00708     T *d, *d0=data+D*j1;
00709
00710     if (D*(j1+1)>dim2 || (D ? dim2%D : 0)) wblog(FL,
00711         "ERR %s() invalid block parameters (%d+%d/%d)",FCT,j1,D,dim2);
00712     M.init(dim1,D); d=M.data;
00713
00714     for (size_t i=0; i<dim1; i++, d+=D, d0+=dim2)
00715         MEM_COPY<T>(d,D,d0);
00716
00717     return M;
00718 };
00719
00720 template <class T> inline
00721 wbMatrix<T>& wbMatrix<T>::getBlock(
00722     size_t i, size_t j,
00723     size_t n, size_t m,
00724     wbMatrix<T> &M
00725 ) const {
00726
00727     if (i+n>dim1 || j+m>dim2) wblog(FL,
00728         "ERR %s() index out of bounds (%d..%d/%d; %d..%d/%d)",
00729         i+1,i+n,dim1,j+1,j+m,dim2
00730     );
00731
00732     M.init(n,m); Wb::cpyStride(M.data,ref(i,j),m,n,-1,dim2);
00733     return M;
00734 };
00735
00736 template <class T>
00737 template <class T2>
00738 wbMatrix<T>& wbMatrix<T>::getBlock(
00739     size_t k,
00740     size_t D,
00741     const wbMatrix<T2>& R, size_t k2, size_t D2,
00742     wbMatrix<T> &M
00743 ) const {
00744
00745     size_t i,j;
00746     const T *d0; const T2 *d2; T *d;
00747
00748     if (D*(k+1)>dim2 || !D || dim2%D
00749         || D2*(k2+1)>R.dim2 || !D2 || R.dim2%D2) wblog(FL,
00750         "ERR invalid block setting (%d,%d/%d; %d,%d/%d).",
00751         k,D,dim2, k2,D2,R.dim2);
00752     if (dim1!=R.dim1) wblog(FL,
00753         "ERR %s() dimension mismatch (%d/%d)",FCT,dim1,R.dim1);
00754
00755     M.init(dim1,D+D2); d=M.data; d0=data+D*k; d2=R.data+D2*k2;
00756
00757     for (i=0; i<dim1; i++) {
00758         MEM_COPY<T>(d,D,d0); d0+=dim2; d+=D;
00759         for (j=0; j<D2; j++) d[j]=T(d2[j]);
00760         d2+=R.dim2; d+=D2;
00761     }
00762
00763     return M;
00764 };
00765
00766 template <class T> inline
00767 void wbMatrix<T>::setBlock(
00768     size_t k, const size_t D,
00769     const T* d0
00770 ){
00771     T *d = data+k*D;
00772
00773     if ((k+1)*D>dim2 || (D ? dim2%D : 0)) wblog(FL,
00774         "ERR invalid block parameters (%d,%d,%d).",k, D, dim2);
00775
00776     for (size_t i=0; i<dim1; i++, d+=dim2)
00777         MEM_COPY<T>(d,D,d0);
00778 };
00779
00780 template <class T> inline
00781 void wbMatrix<T>::SetBlock(

```

```

00782     size_t i0, size_t j0,
00783     const wbMatrix<T> &M
00784 ){
00785     if (i0+M.dim1>dim1 || j0+M.dim2>dim2) wblog(FL,"ERR %s() index "
00786         "out of bounds (%d/%d)",FCT,i0+1,M.dim1,dim1,j0+1,M.dim2,dim2);
00787     const T* d0=M.data; T* d=ref(i0,j0);
00788
00789     for (size_t i=0; i<M.dim1; i++, d+=dim2, d0+=M.dim2)
00790         MEM_COPY<T>(d,M.dim2,d0);
00791 };
00792
00793 template <class T>
00794 wbMatrix<T>& wbMatrix<T>::blockSum(size_t D, wbMatrix<T> &Q) const {
00795
00796     size_t i,r,rk;
00797     const T *q0; T *q;
00798
00799     if (&Q==this) wblog(FL,"ERR Output space same as input space!");
00800     if (!D || dim2%D) wblog(FL,"ERR Invalid block size (%d/%d)",D,dim2);
00801
00802     rk=dim2/D; if (rk==1) { Q=(*this); return Q; }
00803
00804     Q.init(dim1,D);
00805
00806     for (i=0; i<dim1; i++) {
00807         q0=rec(i); q=Q.rec(i);
00808         for (r=0; r<rk; r++, q0+=D) Wb::addRange(q0,q,D);
00809     }
00810
00811     return Q;
00812 }
00813
00814 template <class T> inline
00815 void wbMatrix<T>::blockSum(
00816     size_t j1,
00817     size_t j2,
00818     const size_t D,
00819     wbMatrix<double> &Q
00820 ) const {
00821
00822     size_t i,k;
00823     double *q1, *q2, *q;
00824
00825     if (D*(j1+1)>dim2 || D*(j2+1)>dim2 || (D ? dim2%D : 0)) wblog(FL,
00826         "ERR Invalid block parameters (%d,%d,%d,%d).",j1, j2, D, dim2);
00827
00828     Q.init(dim1,D);
00829     for (k=0; k<dim1; k++) {
00830         q=rec(k); q1=q+D*j1; q2=q+D*j2;
00831         q=Q.rec(k);
00832         for (i=0; i<D; i++) q[i]=q1[i]+q2[i];
00833     }
00834 }
00835
00836 template <class T> inline
00837 void wbMatrix<T>::blockSum(
00838     const wbindex &J,
00839     const size_t D,
00840     wbMatrix<double> &Q
00841 ) const {
00842
00843     size_t i,j,r=(D ? dim2/D : 0);
00844     double *q;
00845
00846     if (D ? dim2%D : 0) wblog(FL,"ERR Invalid D=%d (%d)", D, dim2);
00847     for (i=0; i<J.len; i++) if (J[i]>=r)
00848         wblog(FL,"ERR Block index out of range (%d; %d,%d)",J[i],D,dim2);
00849
00850     Q.init(dim1,D);
00851
00852     for (i=0; i<dim1; i++) {
00853         q=Q.rec(i);
00854         for (j=0; j<J.len; j++) Wb::addRange(Q.ref(i,J[j]*D), q, D);
00855     }
00856 }
00857
00858 template <class T> inline
00859 void wbMatrix<T>::recSet(size_t k,
00860     const T* a, const wbindex &ia,
00861     const T* b, const wbindex &ib
00862 ){
00863     if (ia.len+ib.len!=dim2) wblog(FL,"ERR %s() size mismatch "
00864         "%dx%d/(%d: %d+%d)",FCT,dim1,dim2,k,ia.len,ib.len);
00865     if (k>=dim1) wblog(FL,
00866         "ERR %s() index out of bounds (%dx%d: %d)",FCT,dim1,dim2,k);
00867
00868     if (dim2) { size_t i,l=0; T *d=rec(k);

```

```

00869         for (i=0; i<ia.len; ++i, ++l) { d[l]=a[ia[i]]; }
00870         for (i=0; i<ib.len; ++i, ++l) { d[l]=b[ib[i]]; }
00871     }
00872 };
00873
00874 template <class T> inline
00875 void wbMatrix<T>::recSetB(
00876     size_t r, size_t j, size_t D,
00877     const T *q0, const T *q2, const T a
00878 ){
00879     T *q = data + r*dim2 + j*D;
00880
00881     if (r>=dim1 || (j+1)*D>dim2 || (D ? dim2%D : 0)) wblog(FL,
00882         "ERR setRecB() index out of bounds (%d/%d; %d/%d %d)",
00883         r,dim1, j,dim2, D);
00884
00885     if (q2==NULL)
00886         for (size_t i=0; i<D; i++) q[i]=q0[i];
00887     else if (a) {
00888         if (a==+1) for (size_t i=0; i<D; i++) q[i]=q0[i]+q2[i]; else
00889         if (a==+1) for (size_t i=0; i<D; i++) q[i]=q0[i]-q2[i];
00890         else for (size_t i=0; i<D; i++) q[i]=q0[i]+a*q2[i];
00891     }
00892 }
00893
00894 template <class T> inline
00895 void wbMatrix<T>::recSetB(
00896     size_t r, const windex &J, size_t D, const T *d0
00897 ){
00898     size_t i,j,k; const widx_t *jp=J.data;
00899     T *d = data + r*dim2;
00900
00901     if (r>=dim1) wblog(FL,
00902         "ERR %s() index out of bounds (%d/%d)",FCT,r,dim1);
00903     if (J.len*D!=dim2) wblog(FL,
00904         "ERR %s() size mismatch ([%s] %d/%d)",FCT,STR(J+1),D,dim2);
00905
00906     if (D==1) {
00907         for (j=0; j<J.len; j++) d[j]=d0[jp[j]];
00908     }
00909     else {
00910         for (j=0; j<J.len; j++, d+=D) { k=jp[j]*D;
00911             for (i=0; i<D; i++) { d[i]=d0[k+i]; }
00912         }
00913     }
00914 }
00915
00916 template <class T> inline
00917 void wbMatrix<T>::recSet(size_t i1, size_t i2) {
00918
00919     if (i1>=dim1 || i2>=dim1) wblog(FL,
00920         "ERR index out of bounds (%d,%d; %d)", i1, i2, dim1);
00921
00922     if (i1!=i2)
00923         MEM_COPY<T>(data+i1*dim2, dim2, data+i2*dim2);
00924 }
00925
00926 template <class T> inline
00927 void wbMatrix<T>::recSet(size_t i, const wbvector<T> &v) {
00928
00929     if (i>=dim1) wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,i,dim1);
00930     if (v.len!=dim2) wblog(FL,"ERR size mismatch (%d/%d)",v.len,dim2);
00931     MEM_COPY<T>(data+i*dim2, dim2, v.data);
00932 };
00933
00934 template <class T>
00935 template <class T2>
00936 void wbMatrix<T>::recSetT(size_t i, const wbvector<T2> &v) {
00937
00938     size_t j=0, n=(v.len<dim2 ? v.len : dim2); T *d = data + i*dim2;
00939
00940     if (i>=dim1) wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,i,dim1);
00941     if (v.len>dim2) wblog(FL,"ERR size mismatch (%d/%d)",v.len,dim2);
00942
00943     for (; j<n; ++j) { d[j]=T(v.data[j]); }
00944     for (; j<dim2; ++j) { d[j]=T(0); }
00945 };
00946
00947 template <class T> inline
00948 void wbMatrix<T>::recSet (
00949     size_t i, const wbMatrix<T> &M, size_t j
00950 ){
00951     if (i>=dim1 || j>=M.dim1) wblog(FL,
00952         "ERR index out of bounds (%d/%d; %d/%d)", i, dim1, j, M.dim1);
00953     if (M.dim2!=dim2) wblog(FL,
00954         "ERR incompatible objects (%d/%d)", M.dim2, dim2);
00955 }

```

```

00956     MEM_COPY<T>(data+i*dim2, dim2, M.data+j*M.dim2);
00957 }
00958
00959 template <class T> inline
00960 void wbMatrix<T>::recSet (
00961     size_t i, const wbvector<T> &v1, const wbvector<T> &v2
00962 ){
00963     if (i>=dim1) wblog(FL,
00964         "ERR index out of bounds (%d/%d)", i, dim1);
00965     if (v1.len+v2.len!=dim2) wblog(FL,
00966         "ERR incompatible objects (%d+%d; %d)", v1.len, v2.len, dim2);
00967
00968     MEM_COPY<T>(data+i*dim2, dim2, v1.len, v1.data, v2.data);
00969 }
00970
00971 template <class T>
00972 void wbMatrix<T>::recSet(size_t i, const T* v, T x) {
00973     if (i>=dim1) wblog(FL,
00974         "ERR record index out of bounds (%d/%s)", i, SSTR_(this));
00975     T *d=data+i*dim2; i=0;
00976     if (d!=v) {
00977         if (x==T(+1)) { for (; i<dim2; ++i) d[i]= v[i]; } else
00978         if (x==T(-1)) { for (; i<dim2; ++i) d[i]=-v[i]; } else
00979         if (x==T( 0)) { for (; i<dim2; ++i) d[i]= T(0); }
00980         else { for (; i<dim2; ++i) d[i]=x*v[i]; }
00981     }
00982 };
00983
00984 template <class T> inline
00985 void wbMatrix<T>::recSetP(size_t i1, const T* v, size_t n) {
00986     if (i1>=dim1) wblog(FL,
00987         "ERR record index out of bounds (%d/%d)", i1, dim1);
00988     if (long(n)<0) n=dim2; else if (n>dim2) wblog(FL,
00989         "ERR %s() rec-length out of bounds (%d/%d)", FCT, n, dim2);
00990     T *p=data+i1*dim2; if (p!=v) MEM_COPY<T>(p, n, v);
00991 };
00992
00993 template <class T> inline
00994 void wbMatrix<T>::recSetP(size_t i1,
00995     const T *v1, size_t n1,
00996     const T *v2, size_t n2
00997 ){
00998     if (i1>=dim1) {
00999         if (n1 || n2) wblog(FL,
01000             "ERR record index out of bounds (%d/%d)", i1, dim1);
01001         return;
01002     }
01003     if (n1+n2>dim2) wblog(FL,
01004         "ERR %s() size out of bounds (%d+%d/%d)", FCT, n1, n2, dim2);
01005
01006     size_t i; T *p=data+i1*dim2;
01007     for (i=0; i<n1; ++i) { p[i]=v1[i]; }; p+=n1;
01008     for (i=0; i<n2; ++i) { p[i]=v2[i]; };
01009 };
01010
01011 template <class T> inline
01012 void wbMatrix<T>::recSetP(size_t l,
01013     const T *v1, const windex &i1, const T *v2, const windex &i2
01014 ){
01015     if (l>=dim1) {
01016         if (i1.len || i2.len) wblog(FL,
01017             "ERR record index out of bounds (%d/%d)", l, dim1);
01018         return;
01019     }
01020     if (i1.len+i2.len>dim2) wblog(FL,
01021         "ERR %s() size out of bounds (%d+%d/%d)", FCT, i1.len, i2.len, dim2);
01022
01023     size_t i=0; T *x=data+l*dim2; if (i1.len) {
01024         for (; i<i1.len; ++i) { x[i]=v1[i1.data[i]]; }; x+=i1.len; i=0; }
01025         for (; i<i2.len; ++i) { x[i]=v2[i2.data[i]]; };
01026 };
01027
01028 template <class T> inline
01029 void wbMatrix<T>::recAddP(size_t i1, const T* v) {
01030     T *d = data+i1*dim2;
01031     for (size_t i=0; i<dim2; i++) d[i]+=v[i];
01032 }
01033
01034 template <class T>
01035 template<class T2> inline
01036 void wbMatrix<T>::setCol(size_t k, const wbvector<T2> &v) {
01037     if (v.len!=dim1) wblog(FL,
01038         "ERR %s() size mismatch (%d/%d)", FCT, v.len, dim1);
01039     setCol(k, v.data);
01040 };
01041
01042 template <class T>

```

```

01043 template<class T2> inline
01044 void wbMatrix<T2>::setCol(size_t k, const T2 *v, T fac, size_t stride) {
01045
01046     if (k>=dim2) {
01047         if (dim2) wblog(FL,
01048             "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01049         else wblog(FL,
01050             "ERR %s() got empty matrix (%dx%d/%d)",FCT,dim1,dim2,k);
01051     }
01052
01053     T *d=data+k; size_t i=0;
01054     if (fac==1)
01055         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)= T(*v); }}
01056     else if (fac==-1)
01057         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=-T(*v); }}
01058     else { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=fac*T(*v); }}
01059 };
01060
01061 template <class T> inline
01062 void wbMatrix<T>::setCol(size_t k, const T &x) {
01063
01064     if (k>=dim2) {
01065         if (dim2) wblog(FL,
01066             "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01067         else wblog(FL,
01068             "ERR %s() got empty matrix (%dx%d/%d)",FCT,dim1,dim2,k);
01069     }
01070
01071     T *d=data+k;
01072     for (size_t i=0; i<dim1; ++i, d+=dim2) (*d)=x;
01073 };
01074
01075 template <class T>
01076 template <class T2> inline
01077 void wbMatrix<T>::setColT (size_t k, const wbvector<T2> &v) {
01078     if (v.len!=dim1) wblog(FL,
01079         "ERR %s() size mismatch (%d/%d)",FCT,v.len,dim1);
01080     setColT(k,v.data);
01081 };
01082
01083 template <class T>
01084 template <class T2> inline
01085 void wbMatrix<T>::setColT(size_t k, const T2 *v, T fac, size_t stride) {
01086
01087     T *d=data+k; size_t i=0;
01088     if (k>=dim2) wblog(FL,
01089         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01090
01091     if (fac==1)
01092         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)= T(*v); }}
01093     else if (fac==-1)
01094         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=-T(*v); }}
01095     else { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=fac*T(*v); }}
01096 };
01097
01098 template <class T> inline
01099 wbMatrix<T>& wbMatrix<T>::colTimes(size_t i, T x) {
01100
01101     if (i>=dim2) wblog(FL,
01102         "ERR %s() index out of bounds (%s/%d)",FCT,SSTR_(this),i);
01103     if (x!=T(1)) { T *d=data+i; i=0;
01104         if (x==T( 0)) { for (; i<dim1; ++i, d+=dim2) d[0]= T(0); } else
01105         if (x==T(-1)) { for (; i<dim1; ++i, d+=dim2) d[0]=-d[0]; }
01106         else { for (; i<dim1; ++i, d+=dim2) d[0]*=x; }
01107     }
01108     return *this;
01109 };
01110
01111 template <class T> inline
01112 wbMatrix<T>& wbMatrix<T>::rowTimes(size_t i, T x) {
01113
01114     if (i>=dim1) wblog(FL,
01115         "ERR %s() index out of bounds (%d /%s",FCT,i,SSTR_(this));
01116     if (x!=T(1)) { T *d=data+i*dim2; i=0;
01117         if (x==T( 0)) { for (; i<dim2; ++i) d[i]= T(0); } else
01118         if (x==T(-1)) { for (; i<dim2; ++i) d[i]=-d[i]; }
01119         else { for (; i<dim2; ++i) d[i]*=x; }
01120     }
01121     return *this;
01122 };
01123
01124 template <class T>
01125 size_t wbMatrix<T>::skipNanRecs(wbindex &I) {
01126     wblog(FL,"WRN %s() irrelevant for type <%s>",FCT,TSTR(T));
01127     return 0;
01128 }
01129

```

```

01130 template <>
01131 size_t wbMatrix<double>::skipNanRecs(wbindex &I) {
01132
01133     size_t i,j,k=0,skipped=0;
01134     double *d=data;
01135     I.init(dim1);
01136
01137     for (i=0; i<dim1; i++, d+=dim2) {
01138         for (j=0; j<dim2; j++) if (std::isnan(d[j])) break;
01139         if (j<dim2) { skipped++; continue; }
01140
01141         if (k<i) recSet(k,i);
01142         I[k++]=i;
01143     }
01144
01145     if (k==0)
01146         { Resize(0,dim2); I.init(); }
01147     else { dim1=k; I.len=k; }
01148
01149     return skipped;
01150 };
01151
01152 template <class T> inline
01153 wbMatrix<T>& wbMatrix<T>::recPermute(
01154     const wbperm &P, wbMatrix<T> &M, char iflag
01155 ) const {
01156
01157     if (P.len!=dim1 || !P.isValidPerm()) wblog(FL,
01158         "ERR %s() invalid permutation [%s; %d]",FCT,STR(P),dim1);
01159     if (&M==this) wblog(FL,"ERR %s() got same object!",FCT);
01160
01161     if (M.dim1!=dim1 || M.dim2!=dim2) {
01162         M.init(dim1,dim2);
01163     }
01164
01165     if (iflag) {
01166         const T *d=data;
01167         for (size_t i=0; i<P.len; ++i, d+=dim2)
01168             MEM_COPY<T>(M.data+P[i]*dim2, dim2, d);
01169     }
01170     else {
01171         T *d=M.data;
01172         for (size_t i=0; i<P.len; ++i, d+=dim2)
01173             MEM_COPY<T>(d, dim2, data+P[i]*dim2);
01174     }
01175
01176     return M;
01177 };
01178
01179 template <class T> inline
01180 wbMatrix<T>& wbMatrix<T>::colPermute(
01181     const wbperm &P, wbMatrix<T> &M,
01182     char iflag
01183 ) const {
01184
01185     size_t i,j;
01186     const wperm_t *p=P.data;
01187     const T* d0; T* d2;
01188
01189     if (!P.isValidPerm(0,0,dim2)) wblog(FL,
01190         "ERR %s() invalid permutation [%s; %d]", FCT,STR(P), dim2);
01191     M.init(dim1,dim2); d2=M.data; d0=data;
01192
01193     if (iflag) {
01194         for (i=0; i<dim1; i++, d0+=dim2, d2+=dim2)
01195             for (j=0; j<dim2; j++) d2[p[j]]=d0[j];
01196     }
01197     else {
01198         for (i=0; i<dim1; i++, d0+=dim2, d2+=dim2)
01199             for (j=0; j<dim2; j++) d2[j]=d0[p[j]];
01200     }
01201
01202     return M;
01203 };
01204
01205 template <class T> inline
01206 void wbMatrix<T>::blockPermute(const wbperm &P, wbMatrix<T> &M
01207 ) const {
01208
01209     if (this==&M) {
01210         wbMatrix<T> X(*this); X.blockPermute(P,M);
01211         return;
01212     }
01213
01214     size_t i,j,D; const T *d0=data; T *d;
01215
01216     if (!P.len || !P.isValidPerm()) wblog(FL,

```

```

01217     "ERR invalid permutation (%d;%d)", P.len, dim2);
01218     if (dim2%P.len) wblog(FL,
01219         "ERR block dimension mismatch ( %d = %d*?? )", dim2, P.len);
01220
01221     M.init(dim1,dim2); d=M.data; D=dim2/P.len;
01222
01223     for (i=0; i<dim1; i++, d0+=dim2)
01224         for (j=0; j<P.len; j++, d+=D) MEM_COPY<T>(d, D, d0+D*P[j]);
01225 }
01226
01227 template <class T> inline
01228 wbMatrix<T>& wbMatrix<T>::cols2Front(
01229     const WBINDEX &I1, wbMatrix<T> &M
01230 ) const {
01231
01232     wbperm P; P.init2Front(I1,dim2);
01233     return colPermute(P,M);
01234 };
01235
01236 template <class T> inline
01237 wbMatrix<T>& wbMatrix<T>::cols2End(
01238     const WBINDEX &I1, wbMatrix<T> &M
01239 ) const {
01240
01241     wbperm P; P.init2End(I1,dim2);
01242     return colPermute(P,M);
01243 };
01244
01245 template <class T>
01246 WBIDXMAT& wbMatrix<T>::toBlockIndex(
01247     widx_t D, WBIDXMAT &II_,
01248     wbvector< wbMatrix<T> > *QQ_
01249 ) const {
01250
01251     if (!D || dim2%D)
01252         wblog(FL,"ERR %s() invalid blocksize (%d/%d)",dim2,D);
01253
01254     size_t i,m=dim2/D;
01255
01256     wbvector< wbMatrix<T> > QQ;
01257     wbvector< wbindex > II;
01258     WBINDEX dd;
01259     wbperm P;
01260
01261     wbvector< WBINDEX const* > Ip;
01262
01263     QQ.initDef(m);
01264     II.initDef(m); Ip.init(m);
01265
01266     for (i=0; i<m; i++) {
01267         getBlock(i,D,QQ[i]).groupRecs(P,dd);
01268         II[i].BlockIndex(dd).Permute(P,'i');
01269         Ip[i]=(&II[i]);
01270     }
01271
01272     II_.CAT(2,Ip);
01273     if (QQ_) QQ.save2(*QQ_);
01274
01275     return II_;
01276 };
01277
01278 template <class T>
01279 template <class T2>
01280 void wbMatrix<T>::toBlockIndex(widx_t D,
01281     wbMatrix<T2> &R, widx_t DR_,
01282     WBIDXMAT *IB_,
01283     WBIDXMAT *I2_,
01284     WBIDXMAT *SS_,
01285     wbvector< wbMatrix<T> > *QQ_
01286 ) const {
01287
01288     if (dim1!=R.dim1) wblog(FL,
01289         "ERR %s() size mismatch (%d/%d)",FCT,dim1,R.dim1);
01290     if (!D || dim2%D) wblog(FL,
01291         "ERR %s() invalid blocksize (%d/%d)",FCT,dim2,D);
01292
01293     if (dim1==0 || dim2==0) {
01294         if (IB_) { IB_>init(); }
01295         if (I2_) { I2_>init(); }
01296         if (SS_) { SS_>init(); }
01297         if (QQ_) { QQ_>init(); }; return;
01298     }
01299
01300     size_t i,DR, m=dim2/D;
01301
01302     wbvector< wbMatrix<T> > QQ;
01303     wbvector< WBINDEX > IB,I2,SB;

```



```

01304     WBINDEX dd;
01305     wbMatrix<T2> Ri;
01306     wbperm pp;
01307
01308     if (int(DR_)<=0) DR=R.dim2/m; else DR=DR_;
01309     if (m*DR_!=R.dim2) wblog(FL,
01310         "ERR block size mismatch (expecting %d*%d = %d / %d)",
01311         m,DR_,m*DR_,R.dim2
01312     );
01313
01314     QQ.init(m);
01315     IB.init(m); I2.init(m); SB.init(m);
01316
01317     for (i=0; i<m; i++) {
01318         getBlock(i,D, R,i,DR, QQ[i])
01319         .groupRecs(pp,dd, D,
01320             IB[i], I2[i], SB[i],
01321             'p'
01322         );
01323     }
01324
01325     if (IB_) { IB_>CAT(2,IB); }
01326     if (I2_) { I2_>CAT(2,I2); }
01327     if (SS_) { SS_>CAT(2,SB); }; if (QQ_) QQ.save2(*QQ_);
01328 };
01329
01330 template <class T>
01331 void wbMatrix<T>::recPrint(
01332     size_t k, const char *istr0, char mflag) const {
01333
01334     size_t l=0, n=64; char s[n];
01335     wbvector<T> d;
01336
01337     if (k>=dim1) wblog(FL,
01338         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim1);
01339
01340     l=snprintf(s,n,"%32s.rec(%ld)", istr0, k);
01341     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%ld/%ld)",FCT,l,n);
01342
01343     d.init(dim2, (*this).rec(k));
01344     d.print(s,mflag);
01345
01346     return;
01347 };
01348
01349 template <class T>
01350 wbstring wbMatrix<T>::rec2Str(
01351     size_t k,
01352     const char *fmt,
01353     const char *sep,
01354     size_t stride,
01355     const char *sep2
01356 ) const {
01357
01358     if (k>=dim1) wblog(FL,
01359         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim1);
01360     if (!dim2) return "";
01361
01362     wbvector<T> d; d.init2ref(dim2, data+k*dim2);
01363     return d.toStrf(fmt,sep,stride,sep2);
01364 };
01365
01366 template <class T>
01367 wbstring wbMatrix<T>::toStr(
01368     const char *fmt0, const char *sep, const char *rsep,
01369     size_t stride, const char *sep2
01370 ) const {
01371
01372     size_t j, i=16+dim1*dim2*16;
01373     wbstring s,fmt;
01374
01375     if (isComplex()) { i*=2; }
01376     if (i<128) { i=128; }
01377     s.init(i);
01378
01379     if (fmt0 && fmt0[0])
01380         { fmt=fmt0; }
01381     else { fmt.init2Fmt((T)0); }
01382
01383     for (i=0; i<dim1; ++i) { if (i) s.push(FL,rsep);
01384     for (j=0; j<dim2; ++j) { if (j) s.push(FL,sep);
01385         s.pushf(FL,fmt.data, data[i*dim2+j]);
01386         if (stride && ((j+1)%stride)==0 && j+1<dim2) {
01387             s.push(FL,sep2);
01388         }
01389     }
01390 }

```

```

01391
01392     return s;
01393 };
01394
01395 template <>
01396 wbstring wbMatrix<wbcomplex>::toStr(
01397     const char *fmt0, const char *sep, const char *rsep,
01398     size_t stride, const char *sep2
01399 ) const {
01400
01401     wbstring s;
01402     unsigned i,j; char fmt[16], zfmt[8]="%s", s1[32];
01403
01404     snprintf(fmt,16,"%s",fmt0 && fmt0[0] ? fmt0 : "%.4g");
01405
01406     for (i=0; fmt[i]; ++i) { if (fmt[i]!='%') {
01407         if (isdigit(fmt[++i])) { unsigned l=fmt[i]-'0';
01408             for (j=i+1; fmt[j]; ++j) {
01409                 if (isdigit(fmt[j])) { l=10*l+(fmt[j]-'0'); } else break;
01410             }
01411             snprintf(zfmt,8,"%%ds ",l);
01412         }
01413         break;
01414     }}
01415
01416     s.init(32+(4+i)*dim1*dim2);
01417
01418     for (i=0; i<dim1; ++i) { if (i) s.push(FL,rsep);
01419     for (j=0; j<dim2; ++j) { if (j) s.push(FL,sep);
01420         snprintf(s1,32,"%s",(*this)(i,j).toStr(fmt).data);
01421         s.pushf(FL,zfmt,s1);
01422         if (stride && ((j+1)%stride)==0 && j+1<dim2) {
01423             s.push(FL,sep2);
01424         }
01425     }}
01426
01427     return s;
01428 };
01429
01430 template <class T>
01431 void wbMatrix<T>::toMxStruct(mxArray* S, const char *vname) const {
01432
01433     mxArray *a=Mx::Array<double>(dim1,dim2,'t').copyFromTR(data);
01434
01435     int fid = mxAddField2Scalar(FL,S,vname);
01436     mxSetFieldByNumber(S,0,fid, a);
01437 };
01438
01439 template <class T>
01440 void wbMatrix<T>::mat2mxs(mxArray* S, char fid) const {
01441     int n;
01442
01443     if (!S) wblog(FL,"ERR %s() got null",FCT);
01444     if (!mxIsStruct(S)) wblog(FL,
01445         "ERR %s() got non-structure %s",FCT,mxGetClassName(S));
01446     if ((n=mxGetNumberOfElements(S))!=1) wblog(FL,
01447         "ERR %s() single struct required (%d)",FCT,n);
01448     if (fid>=(n=mxGetNumberOfFields(S))) wblog(FL,
01449         "ERR %s() field index out of bounds (%d/%d)",FCT,fid,n);
01450
01451     mxArray *a=Mx::Array<double>(dim2,dim1).copyFromTR(data);
01452     mxSetFieldByNumber(S,0,fid, a);
01453 };
01454
01455 #ifdef QS_USING_OMP
01456
01457 template <class T>
01458 void getSortPerm_OMP(
01459     const wbMatrix<T> &A, wbperm &P, char dir, char lex
01460 ){
01461     if (!A.dim2) { P.init(); return; }
01462
01463     wbRecs<T> R(A,P,dir,lex);
01464
01465     #ifdef WB_CLK_SPARSE
01466         Wb::Clock clk("mat:sort:recs-omp",0);
01467     #endif
01468
01469     if (A.dim1<128) {
01470         sort(P.data, P.data+A.dim1, R);
01471         return;
01472     }
01473
01474     int id=0, i,l, im=0, mp=MIN(
01475         1 << unsigned(floor(log2(double(A.dim1)))-6),
01476         omp_get_max_threads()
01477 );

```

```

01478
01479     double nsub=double(A.dim1)/mp;
01480
01481     wbindx idx(mp+1); wbpem PX(P.len);
01482     for (i=0; i<mp; ++i) { idx[i]=size_t(i*nsub+0.5); }
01483     idx[i]=A.dim1;
01484
01485     #pragma omp parallel for
01486     for (i=0; i<mp; ++i) {
01487         size_t i1=idx.data[i], i2=idx.data[i+1];
01488         sort(P.data+i1, P.data+i2, R);
01489         id=MAX(id,omp_get_thread_num());
01490     }
01491
01492     while (idx.len>2) { int m2=mp/2; P.swap(PX); ++im;
01493
01494     #pragma omp parallel for
01495     for (i=0; i<m2; ++i) {
01496         size_t k=2*i, i1=idx.data[k], i2=idx.data[k+1], i3=idx.data[k+2];
01497         merge(
01498             PX.data+i1, PX.data+i2,
01499             PX.data+i2, PX.data+i3, P.data+i1, R
01500         );
01501         id=MAX(id,omp_get_thread_num());
01502     }
01503
01504     if (mp%2) {
01505         size_t i1=idx.data[mp-3], i2=idx.data[mp-1], i3=idx.data[mp];
01506         memcpy(PX.data+i1,P.data+i1,(i2-i1)*sizeof(size_t));
01507         merge(
01508             PX.data+i1, PX.data+i2,
01509             PX.data+i2, PX.data+i3, P.data+i1, R
01510         );
01511         idx[mp-1]=idx[mp]; idx.len=(mp--);
01512     }
01513
01514     for (l=1, i=2; i<mp; i+=2, ++l) { idx[l]=idx[i]; }
01515     idx[l]=idx[mp]; mp=1; idx.len=l+1;
01516 }
01517
01518 };
01519
01520 #endif
01521
01522 template <class T> inline
01523 wbMatrix<T>& wbMatrix<T>::SortRecs(
01524     wbpem &P, char dir, char lex
01525 ){
01526     if (!dim1) { P.init(); return *this; }
01527     static int use_omp=-1;
01528
01529     #ifdef LOAD_CGC_QSPACE
01530     if (use_omp<0) {
01531         use_omp=(
01532             sp_num_threads > 0 ? 1 : 0
01533         );
01534
01535         if (use_omp && CG_VERBOSE>6)
01536             wblog(FL,"OMP %s() using openMP on top of STL",FCT);
01537     }
01538
01539     char isLarge=(
01540         (dim1>(1<<30) && CG_VERBOSE>5) ||
01541         (dim1>(1<<28) && CG_VERBOSE>8) );
01542
01543     if (isLarge) {
01544         if (use_omp>0) sprintf(str,
01545             "(parallel mode @ %d; %d)",omp_get_max_threads(),use_omp);
01546         else strcpy (str,"(serial mode)");
01547         wblog(FL,"TST %s() got %.3g x %d entries %s",
01548             FCT,double(dim1),int(dim2),str);
01549     }
01550     #else
01551     if (use_omp<0) use_omp=0;
01552     #endif
01553
01554     if (use_omp<=0 || dim1<128) {
01555         Wb::hpsort(data,dim2,dim1, P,dir,lex);
01556         return *this;
01557     }
01558     else {
01559
01560         #ifdef QS_USING_OMP
01561         getSortPerm_OMP(*this,P,dir,lex);
01562         #else
01563         wblog(FL,"ERR %s()",FCT);
01564         #endif

```

```

01565
01566     return recPermute(P);
01567 }
01568
01569 #ifdef LOAD_CGC_QSPACE
01570     if (isLarge) wblog(FL, "TST %s() done", FCT);
01571 #endif
01572     return *this;
01573 };
01574
01575 template <class T>
01576 template <class T2>
01577 void wbMatrix<T>::groupRecs_rdeg(
01578     wbperm &P, WBINDEX &D, const wbMatrix<T2> &R,
01579     wbvector<wid_x_t> *Ib,
01580     wbvector<wid_x_t> *I2,
01581     wbvector<wid_x_t> *Sb,
01582     char iflag
01583 ){
01584     if (dim1==0) { P.init(); D.init(); return; }
01585
01586     SortRecs(P);
01587     groupSortedRecs(D);
01588
01589     if (R.dim1!=P.len) wblog(FL,
01590         "ERR %s() dimension mismatch (%d/%d)", FCT, R.dim1, P.len);
01591     if (R.dim2==0) { wblog(FL,
01592         "WRN %s() got empty reference space", FCT); return;
01593     }
01594
01595     size_t i,j,d,l=0, m=D.len;
01596     WBINDEX dd,v, I(D.max());
01597     wbvector< WBINDEX > SS;
01598     wbvector< wbindex > II;
01599     wbMatrix<T2> Q;
01600     wbperm pp;
01601
01602     if (I2 || Sb) { II.init(m); if (Sb) SS.init(m); }
01603
01604     for (i=0; i<m; ++i, l+=d) { d=D[i];
01605         if (d>1) {
01606             I.len=d; for (j=0; j<d; j++) I[j]=P[l+j];
01607             R.getRecs(I,Q).groupRecs(pp,dd);
01608             for (j=0; j<d; j++) P[l+j]=I[pp[j]];
01609
01610             if (I2 || Sb) {
01611                 II[i].BlockIndex(dd); if (Sb)
01612                 SS[i].init(pp.len).set(dd.len);
01613             }
01614         }
01615         else {
01616             if (d!=1) wblog(FL, "ERR d=%g !?", d);
01617             if (I2 || Sb) {
01618                 II[i].init(1).set(0); if (Sb)
01619                 SS[i].init(1).set(1);
01620             }
01621             continue;
01622         }
01623     }
01624
01625     if (Ib) {
01626         wbindex J; J.BlockIndex(D); J.save2(*Ib);
01627         if (Ib->len!=P.len) wblog(FL, "ERR size mismatch %d/%d", Ib->len, P.len);
01628         if (iflag) Ib->Permute(P, iflag);
01629     }
01630
01631     if (I2) {
01632         wbvector< WBINDEX* > J(II.len);
01633         for (i=0; i<II.len; ++i) { J[i]=(WBINDEX*)(II[i]); }; I2->Cat(J);
01634         if (I2->len!=P.len) wblog(FL, "ERR size mismatch %d/%d", I2->len, P.len);
01635         if (iflag) I2->Permute(P, iflag);
01636     }
01637
01638     if (Sb) {
01639         Sb->Cat(SS);
01640         if (Sb->len!=P.len) wblog(FL, "ERR size mismatch %d/%d", Sb->len, P.len);
01641         if (iflag) Sb->Permute(P, iflag);
01642     }
01643 };
01644
01645 template <class T>
01646 void wbMatrix<T>::groupRecs(
01647     wbperm &P, WBINDEX &D, size_t nc,
01648     WBINDEX &Ib,
01649     WBINDEX &I2,
01650     WBINDEX &Sb,
01651     char iflag

```

```

01652 ){
01653     if (dim1==0) { P.init(); D.init(); return; }
01654
01655     SortRecs(P);
01656
01657     if (nc>dim2) wblog(FL,"ERR dimension out of bounds (%d/%d)",nc,dim2);
01658     groupSortedRecs(D,nc,Ib,I2,Sb);
01659
01660     if (iflag) {
01661         Ib.Permute(P,iflag);
01662         I2.Permute(P,iflag);
01663         Sb.Permute(P,iflag);
01664     }
01665 };
01666
01667 template <class T>
01668 void wbMatrix<T>::groupRecs(
01669     wbpem &P, WBINDEX &D,
01670     size_t m,
01671     char lex,
01672     wbinde *Ig,
01673     wbMatrix<T> *X
01674 ){
01675     if (!dim1) { P.init(); D.init(); return; }
01676     if (!dim2) {
01677         P.init(dim1); D.init(1); D[0]=dim1;
01678         if (Ig) { Ig->init(dim1).set(0); }
01679
01680         if (int(m)<0) dim1=1;
01681         else if (m) wblog(FL,
01682             "ERR %s() m=%d out of bounds (%dx%d)",FCT,m,dim1,dim2);
01683         return;
01684     }
01685
01686     #ifdef WB_CLK_SPARSE
01687     Wb::Clock clk("mat:sort:recs",0);
01688     #endif
01689
01690     if (X) {
01691         if (dim1>1) { X->init(*this); } else { X=NULL; }
01692     }
01693
01694     if (!isSorted(+1,lex))
01695         { SortRecs(P,+1,lex); }
01696     else { P.init(dim1); }
01697
01698     #ifdef WB_CLK_SPARSE
01699     clk.Switch("mat:group:recs");
01700     #endif
01701
01702     if (int(m)<0) { groupSortedRecs(D, 0,-1,lex); }
01703     else if (m) { groupSortedRecs(D,'k',m,lex); }
01704     else { D.init(1); D[0]=dim1; }
01705
01706     if (X && (dim1==X->dim1)) { X->save2(*this);
01707         P.init(dim1);
01708         D.init2val(dim1,1);
01709     }
01710
01711     if (Ig) {
01712         size_t i,j,l=0,d=0; widx_t *ig; wpermt *p=P.data;
01713         Ig->init(P.len); ig=Ig->data;
01714         for (i=0; i<D.len; ++i, l+=d) { d=D.data[i];
01715             for (j=0; j<d; ++j) ig[p[l+j]]=i; }
01716     }
01717 };
01718
01719 template <class T>
01720 void wbMatrix<T>::groupRecs(
01721     wbpem &P, WBINDEX &D,
01722     const WBINDEX *I, char lex
01723 ){
01724     if (dim1==0) { P.init(); D.init(); return; }
01725
01726     SortRecs(P,+1,lex);
01727
01728     if (!I) { groupSortedRecs(D,0,-1,lex); }
01729     else {
01730         wbMatrix<T> X; this->cols2Front(*I,X);
01731         X.groupSortedRecs(D,'k',I->len,lex);
01732     }
01733 };
01734
01735 template <class T>
01736 void wbMatrix<T>::groupSortedRecs(
01737     WBINDEX &d,
01738     char keepall,

```

```

01739     size_t m,
01740     char lex
01741 ){
01742     size_t i,ig,n=dim1;
01743     char c, cref=0;
01744
01745     if (int(m)<0) { m=dim2; }
01746     else if (m>dim2) wblog(FL,"ERR number out of bounds (%d/%d)",m,dim2);
01747     if (!dim2) {
01748         if (!dim1) wblog(FL,
01749             "WRN %s() for %dx%d matrix !?",FCT,dim1,dim2);
01750         d.init(1); d[0]=dim1; init(); return;
01751     }
01752     else if (m==0) {
01753         if (m<dim2 && !keepall) wblog(FL,
01754             "WRN %s() keeping all since (m=%d)<%d",FCT,m,dim2);
01755         d.init(1); d[0]=dim1; return;
01756     }
01757
01758     d.init(n); if (n==0) return;
01759
01760     ig=0; d[ig]++;
01761     for (i=1; i<n; i++) {
01762         c=recCompare(i,i-1,m,lex);
01763         if (c) {
01764             if ((++ig)!=i) { if (!keepall)
01765                 MEM_COPY<T>(data+ig*dim2, dim2, data+i*dim2); }
01766
01767             if (c!=cref) {
01768                 if (cref) { MXPput(FL,"a").add(*this,"M").add(m,"m");
01769                     wblog(FL,"ERR recs not sorted %d/%d (%d, m=%d/%d)",
01770                         c,cref,i,m,dim2);
01771                 }
01772                 else cref=c;
01773             }
01774         }
01775
01776         d[ig]++;
01777     }
01778
01779     d.Resize(ig+1);
01780     if (!keepall) Resize(ig+1,dim2);
01781 };
01782
01783 template <class T>
01784 void wbMatrix<T>::groupSortedRecs(
01785     WBINDEX &d,
01786     size_t m,
01787     char lex,
01788     WBINDEX &Ib,
01789     WBINDEX &I2,
01790     WBINDEX &Sb
01791 ){
01792     size_t i,j,i2,i0=0, ig=0;
01793     char c,c2, cref=0;
01794
01795     if (int(m)<0) { m=dim2; }
01796     else if (!(m && dim2) || m>dim2) wblog(FL,
01797         "ERR number out of bounds (%d/%d)",m,dim2);
01798
01799     if (!dim1 || !dim2) {
01800         d.init(); init(); Ib.init(); I2.init(); Sb.init();
01801         return;
01802     }
01803
01804     Ib.init(dim1); d.init(dim1);
01805     I2.init(dim1);
01806     Sb.init(dim1);
01807
01808     for (d[ig]++, i=1; i<=dim1; i++, d[ig]++) {
01809         if (i<dim1)
01810             c=recCompare(i,i-1,m,lex);
01811         else c=99;
01812
01813         if (c) {
01814             for (i2=0, j=i0+1; j<i; j++) {
01815                 c2=recCompare(j,j-1,-1,lex); if (c2) { i2++;
01816                     if (c2!=cref) {
01817                         if (cref) wblog(FL,
01818                             "ERR input recs not sorted (%d: %d/%d).",i,c2,cref);
01819                         else cref=c2;
01820                     }
01821                 }
01822                 I2[j]=i2;
01823             }
01824             for (i2++, j=i0; j<i; j++) { Ib[j]=ig; Sb[j]=i2; }
01825             i0=i; if (i>=dim1) break;

```

```

01826
01827         if (c!=cref) {
01828             if (cref) wblog(FL,
01829                 "ERR input recs not sorted (%d: %d/%d).",i,c,cref);
01830             else cref=c;
01831         }
01832
01833         if ((++ig)!=i)
01834             MEM_COPY<T>(data+ig*dim2, dim2, data+i*dim2);
01835     }
01836 }
01837
01838 d.len=ig+1;
01839 dim1=ig+1;
01840 };
01841
01842 template <> inline
01843 wbMatrix<double>& wbMatrix<double>::sortRecs_float(wbperm &P, char dir) {
01844
01845     if (dim1==0) { P.init(); return *this; }
01846
01847     char lex=1;
01848     wbMatrix<double> X(*this); X.SkipTiny_float();
01849
01850     X.SortRecs(P,dir,lex);
01851
01852     return Set2Recs(P);
01853 };
01854
01855 template <class T>
01856 char wbMatrix<T>::findRecsInSet(
01857     const wbMatrix<T> &S, wbvector<int> &I) const {
01858
01859     size_t j,l,k,d, e=0; int ns=(int)S.dim1, ifound;
01860     WBINDEX dg;
01861     wbMatrix<T> QQ(*this);
01862     wbperm is;
01863
01864     QQ.groupRecs(is,dg); I.init(dim1);
01865
01866     for (l=k=0; k<QQ.dim1; k++) {
01867         for (ifound=0; ifound<ns; ifound++) {
01868             if (QQ.recEqual(k, S.rec(ifound)))
01869                 break;
01870         }
01871         if (ifound>=ns) { ifound=-1; e++; }
01872
01873         for (d=dg[k], j=0; j<d; j++)
01874             I[is[l++]]=ifound;
01875     }
01876
01877     return e;
01878 }
01879
01880 template <class T>
01881 size_t wbMatrix<T>::findUniqueRecSorted1(size_t n, T eps) const {
01882
01883     if (long(n)>=0 && (!n || n>dim2)) wblog(FL,
01884         "ERR %s() invalid n=%d/%d",FCT,n,dim2);
01885     if (dim1<=1) { return (dim1 ? 0:-1); }
01886
01887     if (eps==0) {
01888         char c=0, c0=0;
01889         for (size_t l=dim1-1, i=0; i<l; ++i) {
01890             c=recCompare(i,i+1,n);
01891             if (!c || !c0) { c0=c; continue; }
01892             else { return i; }
01893         }
01894         if (dim1==2 && c) return 0;
01895     }
01896     else {
01897         T c=0, c0=0;
01898         for (size_t l=dim1-1, i=0; i<l; ++i) {
01899             c=recDiff2(i,i+1,n);
01900             if (c<eps || c0<eps) { c0=c; continue; }
01901             else { return i; }
01902         }
01903         if (dim1==2 && (c>eps)) return 0;
01904     }
01905     return -1;
01906 };
01907
01908 template <class T>
01909 int wbMatrix<T>::findRecSorted(const T* r, size_t n, char lex) const {
01910
01911     if (int(n)>=0 && (!n || n>dim2)) wblog(FL,
01912         "ERR %s() invalid n=%d/%d",FCT,n,dim2);

```

```

01913
01914     if (dim1<3) {
01915         for (size_t i=0; i<dim1; ++i) {
01916             if (!recCompareP(i,r,n,lex)) return i;
01917         }; return -1;
01918     }
01919
01920     size_t k1=0, k2=dim1-1, k=(k2-k1)/2;
01921     char c, cref=recCompare(k1,k2,n,lex);
01922
01923     if (cref==0) {
01924         wblog(FL,"WRN all recs the same (%dx%d) ???",dim1,dim2);
01925         return (recCompareP(0,r,n,lex) ? -1 : 0);
01926     }
01927     for (size_t m=MAX(size_t(1), (dim1-2)/4), i=1; i<dim1; i+=m)
01928         if ((c=recCompare(i-1,i,n,lex))==--cref) wblog(FL,
01929             "ERR records not sorted (%d/%d) !?",c,cref);
01930
01931     c=recCompareP(k1,r,n,lex); if (c!= cref) return (c ? -1 : 0);
01932     c=recCompareP(k2,r,n,lex); if (c!=-cref) return (c ? -1 : dim1-1);
01933     c=recCompareP(k ,r,n,lex);
01934
01935     while (1) {
01936         if (c==cref)
01937             { k1=k; k+=(k2-k1)/2; if (k==k1) return -1; } else
01938             if (c) { k2=k; k-=(k2-k1)/2; if (k==k2) return -1; } else
01939                 return k;
01940
01941         if (recCompare(k1,k2,n,lex)==-cref) wblog(FL,
01942             "ERR records not sorted !?");
01943         c=recCompareP(k,r,n,lex);
01944     }
01945
01946     return -1;
01947 };
01948
01949 template <class T>
01950 int wbMatrix<T>::findRec(const T* r, size_t n, char lex) const {
01951
01952     for (size_t k=0; k<dim1; k++)
01953         if (!recCompareP(k,r,n,lex)) return k;
01954
01955     return -1;
01956 }
01957
01958 template <class T>
01959 size_t wbMatrix<T>::UnionRecs(const wbMatrix<T> &B0, wbindex &Ia){
01960     size_t l=0,ia=0,ib=0, na=(dim2 ? dim1 : 0), nb=B0.dim1;
01961     char c;
01962
01963     wbMatrix<T> A(*this), B(B0);
01964     wbperm P1,P2; wbindex I;
01965
01966     if (A.isEmpty() || B.isEmpty()) { init(); return na; }
01967
01968     if (A.dim2 != B.dim2) wblog(FL,
01969         "ERR %s() severe dimension mismatch (%d/%d)",
01970         FCT,A.dim2,B.dim2);
01971
01972     A.SortRecs(P1); I.init(na);
01973     B.SortRecs(P2);
01974
01975     while (ia<na && ib<nb) { c=A.recCompareP(ia, B.rec(ib));
01976         if (c<0) ia++; else
01977             if (c>0) ib++;
01978         else {
01979             do { I[l++]=ia++; } while (ia<na && A.recEqual(ia-1,ia));
01980             do {         ib++; } while (ib<nb && B.recEqual(ib-1,ib));
01981         }
01982     }
01983
01984     if (l) I.len=l; else I.init();
01985
01986     A.Set2Recs(I); P1.get(I,Ia);
01987
01988     return (na-dim1);
01989 }
01990
01991 template <class T>
01992 bool wbMatrix<T>::gotRecOverlap(const wbMatrix<T> &B0) const {
01993
01994     size_t ia=0,ib=0, na=(dim2 ? dim1 : 0), nb=B0.dim1;
01995     char c;
01996
01997     wbMatrix<T> A(*this), B(B0);
01998     wbperm P1,P2;
01999

```



```

02000     if (A.isEmpty() || B.isEmpty()) return 0;
02001
02002     if (A.dim2!=B.dim2) wblog(FL,
02003     "ERR %s() severe dimension mismatch (%d/%d)",FCT,A.dim2,B.dim2);
02004
02005     A.SortRecs(P1);
02006     B.SortRecs(P2);
02007
02008     while (ia<na && ib<nb) { c=A.recCompareP(ia, B.rec(ib));
02009         if (c<0) ia++; else
02010         if (c>0) ib++; else return 1;
02011     }
02012
02013     return 0;
02014 };
02015
02016 template <class T>
02017 bool wbMatrix<T>::gotRecOverlapSA(const wbMatrix<T> &B) const {
02018
02019     size_t ia=0,ib=0, na=(dim2 ? dim1 : 0), nb=B.dim1;
02020     char c;
02021
02022     if (isEmpty() || B.isEmpty()) return 0;
02023
02024     if (dim2!=B.dim2) wblog(FL,
02025     "ERR %s() severe dimension mismatch (%dx%d/%dx%d)",
02026     FCT,dim1,dim2,B.dim1,B.dim2);
02027
02028     while (ia<na && ib<nb) { c=recCompareP(ia, B.rec(ib));
02029         if (c<0) ia++; else
02030         if (c>0) ib++; else return 1;
02031     }
02032
02033     return 0;
02034 };
02035
02036 template <class T>
02037 size_t wbMatrix<T>::findValsCol(
02038     const size_t k, const wbvector<T> &v, wbindex &Ia
02039 ) const {
02040     wbvector<T> vk; getCol(k,vk);
02041     vk.findValues(v,Ia);
02042     return Ia.len;
02043 }
02044
02045 template <class T> inline
02046 void matchIndexU(const char *F, int L,
02047     const wbMatrix<T> &QA,
02048     const wbMatrix<T> &QB,
02049     wbindex &IB, const char force
02050 ){
02051     size_t ma,mb; wbindex Ia, Ib;
02052
02053     matchIndex(QA,QB,Ia,Ib,1,&ma,&mb);
02054     if (mb) wblog(F,L,"ERR index B not unique (%d)",mb);
02055
02056     if (force) { wbpperm iP;
02057         if (Ia.len!=QA.dim1) wblog(F,L,
02058         "ERR failed to find all records (%d/%d)",Ia.len,QA.dim1);
02059
02060         getIPerm(Ia,iP); Ib.select(iP,IB);
02061     }
02062     else {
02063         IB.init(QA.dim1).set(size_t(-1));
02064         IB.Set(Ia,Ib);
02065     }
02066 }
02067
02068 template <class T>
02069 int matchIndex(
02070     const wbMatrix<T> &QA, const wbMatrix<T> &QB,
02071     wbindex &Ia, wbindex &Ib,
02072     char lex,
02073     widx_t *ma, widx_t *mb,
02074     T eps
02075 ){
02076     size_t i;
02077
02078     wbMatrix<T> Q1(QA), Q2(QB);
02079     wbpperm P1, P2;
02080     wbindex ix1, ix2;
02081
02082     if (QA.dim2!=QB.dim2 && QA.dim2 && QB.dim2) wblog(FL,
02083     "ERR %s severe dimension mismatch (%s <> %s)",
02084     FCT,SSTR(QA),SSTR(QB));
02085
02086     if (QA.isEmpty() || QB.isEmpty()) {

```

```

02087     Ia.init(); Ib.init(); return 0;
02088 }
02089
02090 Q1.SortRecs(P1);
02091 Q2.SortRecs(P2);
02092
02093 i=matchSortedIdx(Q1,Q2,ix1,ix2,-1,lex,ma,mb,eps);
02094
02095 P1.get(ix1,Ia);
02096 P2.get(ix2,Ib);
02097
02098 return (int)i;
02099 };
02100
02101 template <class T>
02102 int wbMatrix<T>::getDiff(
02103     const wbMatrix<T> &B, windex &Ia, windex *Ib) const {
02104
02105     wbMatrix<T> Q1(*this), Q2(B);
02106     wperm P1,P2;
02107     windex ix1,ix2;
02108
02109     if (dim2!=B.dim2) wblog(FL,
02110         "ERR %s() severe dimension mismatch (%d/%d)",
02111         FCT,dim2, B.dim2
02112     );
02113
02114     if (isEmpty() || B.isEmpty()) {
02115         Ia.init(); if (Ib) Ib->init();
02116         return 0;
02117     }
02118
02119     Q1.SortRecs(P1);
02120     Q2.SortRecs(P2);
02121
02122     Q1.getDiffSorted(Q2,ix1, Ib ? &ix2 : NULL);
02123     P1.get(ix1, Ia); if (Ib) {
02124         P2.get(ix2,*Ib); }
02125
02126     return Ia.len;
02127 };
02128
02129 template <class T>
02130 int wbMatrix<T>::getDiffSorted(
02131     const wbMatrix<T> &B, windex &Ia, windex *IB
02132 ) const {
02133
02134     size_t ia,ib,la,lb; widx_t *Ib=NULL; char c;
02135
02136     if (dim2!=B.dim2) wblog(FL,
02137         "ERR %s() dimension mismatch (%d/%d)",FCT,dim2,B.dim2);
02138
02139     Ia.init(dim1);
02140     if (IB) { IB->init(B.dim1); Ib=IB->data; }
02141
02142     for (la=lb=ia=ib=0; ia<dim1 && ib<B.dim1; ) {
02143         c=recCompareP(ia, B.rec(ib));
02144         if (c<0) { Ia[la++]=ia++; } else
02145         if (c>0) { if (Ib) { Ib[lb++]=ib; }; ++ib; }
02146         else {
02147             while (++ia<dim1) {
02148                 c=recCompare(ia-1,ia);
02149                 if (c) {
02150                     if (c>0) wblog(FL,
02151                         "ERR %s() expecting ascending order (%d)",FCT,ia);
02152                     break;
02153                 }
02154             }
02155             while (++ib<B.dim1) {
02156                 c=B.recCompare(ib-1,ib);
02157                 if (c) {
02158                     if (c>0) wblog(FL,
02159                         "ERR %s() expecting ascending order (%d)",FCT,ib);
02160                     break;
02161                 }
02162             }
02163         }
02164     }
02165
02166     while (ia<dim1) { Ia[la++]=ia++; }
02167     Ia.Resize(la);
02168
02169     if (Ib) {
02170         while (ib<B.dim1) { Ib[lb++]=ib++; }
02171         IB->Resize(lb);
02172     }
02173

```

```

02174     return Ia.len;
02175 };
02176
02177 #endif
02178

```

5.46 wbMatrix.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MATRIX_ROW_MAJOR_HH__
00021 #define __WB_MATRIX_ROW_MAJOR_HH__
00022
00023 /* -----
00024  * wbMatrix - row-major matrix class
00025  * AW (C) Jan 2006
00026  * ----- */
00027
00028 template <class T>
00029 class wbMatrix {
00030
00031 public:
00032
00033     wbMatrix(size_t r=0, size_t c=0)
00034         : data(NULL), dim1(r), dim2(c), isdiag(0), isref(0) {
00035         if (r || c) NEW_DATA();
00036     };
00037
00038     wbMatrix(size_t r, size_t c, T *d, const char ref=0)
00039         : data(NULL), dim1(r), dim2(c), isdiag(0), isref(0) {
00040         if (ref) { data=d; isref=1; }
00041         else if (r || c) NEW_DATA(d);
00042     };
00043
00044     wbMatrix(const wbMatrix &M)
00045         : data(NULL), dim1(M.dim1), dim2(M.dim2), isdiag(M.isdiag), isref(0) {
00046         NEW_DATA(M.data);
00047     };
00048
00049     template <class T2>
00050     wbMatrix(const wbMatrix<T2> &M)
00051         : data(NULL), dim1(0), dim2(0), isdiag(0), isref(0) {
00052         if (M.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,M.isdiag);
00053         initT(M);
00054     };
00055
00056     wbMatrix(const mxArray *a, char tflag=0, char ref=0)
00057         : data(NULL), dim1(0), dim2(0), isdiag(0), isref(0) {
00058         init(FL,a,tflag,ref);
00059     };
00060
00061     wbMatrix(const char *F, int L, const mxArray *a)
00062         : data(NULL), dim1(0), dim2(0), isdiag(0), isref(0) {
00063         init(F,L,a);
00064     };
00065
00066     virtual ~wbMatrix() { if (data && !isref) {
00067         WB_DELETE(data);
00068     } };
00069
00070     wbMatrix& init(size_t r=0, size_t c=0, const T* d=NULL) {
00071         if (isref) {
00072             isref=0; dim1=dim2=0; data=0;
00073         }
00074         RENEW(r,c,d); return *this;
00075     };

```

```

00076
00077     wbMatrix& init(
00078         size_t r, size_t c, const T *d0, size_t stride
00079     ){
00080         unsigned i=0, j=0; T* d;
00081
00082         if (int(stride)<0) wblog(FL,
00083             "ERR %s() got stride=%ld !?",FCT,stride);
00084         if (!d0) wblog(FL,"ERR %s() got null space !?",FCT);
00085
00086         RENEW(r,c,NULL,0); d=data;
00087         for (; i<dim1; ++i, d+=dim2, d0+=stride) {
00088             for (j=0; j<dim2; ++j) d[j]=d0[j];
00089         }
00090         return *this;
00091     };
00092
00093     wbMatrix& initIdentity(size_t n) {
00094         size_t i=0, n2=n*n; RENEW(n,n,NULL,0);
00095         for (; i<n2; i+=(n+1)) { data[i]=T(1); }
00096     };
00097
00098     wbMatrix& init2val(size_t r, size_t c, const T &d) {
00099         RENEW(r,c,NULL,0); set(d);
00100         return *this;
00101     };
00102
00103     wbMatrix& initDef(size_t r=0, size_t c=0, T* d=NULL) {
00104         RENEW(r,c,NULL,0);
00105         if (d) for (size_t n=r*c, i=0; i<n; ++i) data[i]=d[i];
00106         return *this;
00107     };
00108
00109     template<class T2>
00110     wbMatrix<T>& initT(size_t r, size_t c, T2* d) {
00111         RENEW(r,c,0,0);
00112         for (size_t n=r*c, i=0; i<n; ++i) data[i]=(T)d[i];
00113         return *this;
00114     };
00115
00116     template<class T2>
00117     wbMatrix<T>& initT(const wbMatrix<T2> &B) {
00118         if (B.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,B.isdiag);
00119         RENEW(B.dim1,B.dim2,0,0);
00120         for (size_t n=dim1*dim2, i=0; i<n; ++i) data[i]=(T)B.data[i];
00121         return *this;
00122     };
00123
00124     template<class T2>
00125     wbMatrix<T>& initT(const char *F, int L, const wbMatrix<T2> &B) {
00126         if (B.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,B.isdiag);
00127         RENEW(B.dim1,B.dim2,0,0);
00128         for (size_t n=dim1*dim2, i=0; i<n; ++i) { data[i]=(T)B.data[i];
00129             if (T2(data[i])!=B.data[i]) wblog(F,L,
00130                 "ERR type conversion changes value (%g,%g)",
00131                 double(B.data[i]), double(T2(data[i])))
00132             );
00133         }
00134         return *this;
00135     };
00136
00137     template<class TA, class TB>
00138     wbMatrix<T>& init(const char *F, int L,
00139         const wbMatrix<TA> &B, const wbvector<TB> &V, char dim=2
00140     );
00141
00142     template<class T2>
00143     wbMatrix<T>& initDim(const wbMatrix<T2> &B) {
00144         RENEW(B.dim1, B.dim2);
00145         return *this;
00146     };
00147
00148     wbMatrix& init(const wbMatrix &B) {
00149         RENEW(B.dim1, B.dim2, B.data); isdiag=B.isdiag;
00150         return *this;
00151     };
00152
00153     wbMatrix& init(
00154         const char *F, int L, const mxArray *a,
00155         char tflag=0, char ref=0, char tcheck=1);
00156
00157     void init(const mxArray *a) { init(FL,a); };
00158
00159     void init0(const mxArray *a) {
00160         init(FL,a,1,0,1);
00161     };
00162

```

```

00163     void initTST() {
00164         size_t i,j,k=0;
00165         for (i=0; i<dim1; ++i)
00166             for (j=0; j<dim2; ++j) data[k++]=10*(i+1)+(j+1);
00167     };
00168
00169     wbMatrix& init2ref(size_t r, size_t c, const T* d);
00170     wbMatrix& init2ref(const wbvector<T> &v, const char tflag=0);
00171     wbMatrix& init2ref(const wbMatrix &M);
00172
00173     wbMatrix& unRef();
00174
00175     void no_ref_(const char *F, int L, const char *fct) {
00176         if (isref) { wblog(F_L,
00177             "ERR %s() got reference (isref=%d)",fct,isref);
00178         }
00179     };
00180
00181     void swap(wbMatrix &B, char ref=0) {
00182         if (this!=&B) {
00183             if (!ref && (isref || B.isref)) wblog(FL,
00184                 "ERR %s() got arrays with isref=(%d,%d)",FCT,isref,B.isref);
00185             SWAP(data, B.data );
00186             SWAP(dim1, B.dim1 );
00187             SWAP(dim2, B.dim2 );
00188             SWAP(isref, B.isref );
00189             SWAP(isdiag, B.isdiag);
00190         }
00191     };
00192
00193     wbMatrix& save2(wbMatrix &A, char ref=0) {
00194         if (this!=&A) {
00195             if (!ref) { no_ref_(FLF); }
00196             this->swap(A,ref); if (data!=A.data) { init(); }
00197         }
00198         return A;
00199     };
00200
00201     void copyStride(T* dd, size_t stride) const {
00202         T *d0=data;
00203         for (size_t i=0; i<dim1; ++i, d0+=dim2, dd+=stride)
00204             MEM_COPY<T>(dd, dim2, d0);
00205     };
00206
00207     wbMatrix& operator= (const wbMatrix &B) {
00208         if (this!=&B) {
00209             RENEW(B.dim1, B.dim2, B.data);
00210             isdiag=B.isdiag;
00211         }
00212         return *this;
00213     };
00214
00215     template<class TB>
00216     wbMatrix& operator= (const wbMatrix<TB> &B) {
00217         init(B.dim1, B.dim2);
00218         for (size_t i=0, n=B.numel(); i<n; ++i) { data[i]=T(B.data[i]); }
00219         isdiag=B.isdiag;
00220         return *this;
00221     };
00222
00223     explicit operator bool() const {
00224         if (data)
00225             { if ( dim1 && dim2) { return 1; }}
00226         else { if (!dim1 || !dim2) { return 0; }}
00227     };
00228     wblog(FL,"ERR unexpected matrix %gx%g (%p)",dim1,dim2,data);
00229     return 0;
00230 };
00231 bool operator! () const { return !bool(*this); }
00232
00233 bool operator==(const T &x) const {
00234     for (size_t n=dim1*dim2, i=0; i<n; ++i)
00235         { if (data[i]!=x) return 0; }
00236     return 1;
00237 };
00238
00239 bool operator==(const wbMatrix &B) const {
00240     if (this!=&B) {
00241         if (dim1!=B.dim1 || dim2!=B.dim2) return 0;
00242         if (data!=B.data) {
00243             for (size_t n=dim1*dim2, i=0; i<n; ++i)
00244                 if (data[i]!=B.data[i]) return 0;
00245         }
00246     }
00247     return 1;
00248 };
00249

```

```

00250     bool operator!=(const wbMatrix &B) const {
00251         return !((*this)==B);
00252     };
00253
00254     bool deepEqualP(const wbMatrix &B) const;
00255
00256     bool anyEqual(const T& x) const {
00257         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00258             if (data[i]==x) return 1; }
00259         return 0;
00260     };
00261
00262     bool allEqual(const T& x) const {
00263         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00264             if (data[i]!=x) return 0; }
00265         return 1;
00266     };
00267
00268     bool anyUnequal(const T& x) const { return !allEqual(x); };
00269
00270     bool anyLT(const T& x) const {
00271         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00272             if (data[i]<x) return 1; }
00273         return 0;
00274     };
00275
00276     bool anyGT(const T& x) const {
00277         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00278             if (data[i]>x) return 1; }
00279         return 0;
00280     };
00281
00282     bool col_anyGT(size_t j, const T& x) const {
00283         if (dim1) {
00284             if (j>=dim2) wblog(FL,
00285                 "ERR %s() index out of bounds (%d/%d)",FCT,j,dim2);
00286             for (size_t i=0; i<dim1; ++i, j+=dim2) {
00287                 if (data[j]>x) return 1;
00288             }
00289         }
00290         return 0;
00291     };
00292
00293     wbMatrix& operator+= (const wbMatrix &B) {
00294         size_t i,s=dim1*dim2;
00295
00296         if (isdiag) if (!B.isdiag) isdiag=0;
00297
00298         if (dim1!=B.dim1 || dim2!=B.dim2) wblog(FL,
00299             "ERR Dimension mismatch (%dx%d + %dx%d).",dim1,dim2,B.dim1,B.dim2);
00300
00301         for (i=0; i<s; ++i) data[i]+=B.data[i];
00302         return *this;
00303     };
00304
00305     wbMatrix operator+ (const wbMatrix &B) const {
00306         wbMatrix Mout(*this); Mout+=B;
00307         return Mout;
00308     };
00309
00310     wbMatrix operator+ (const T &x) const {
00311         wbMatrix Mout(*this); Mout+=x;
00312         return Mout;
00313     };
00314
00315     void operator-= (const wbMatrix &B) {
00316         if (isdiag) if (!B.isdiag) { isdiag=0; }
00317         if (dim1!=B.dim1 || dim2!=B.dim2) wblog(FL,
00318             "ERR size mismatch (%dx%d + %dx%d).",dim1,dim2,B.dim1,B.dim2);
00319
00320         for (size_t i=0, n=dim1*dim2; i<n; ++i) {
00321             data[i]-=B.data[i];
00322         }
00323     };
00324
00325     wbMatrix operator- (const wbMatrix &B) const {
00326         wbMatrix Mout(*this); Mout-=B;
00327         return Mout;
00328     };
00329
00330     void operator*= (const T c) {
00331         size_t i, n=dim1*dim2;
00332
00333         if (c==T(+1)) { return; } else
00334         if (c==T( 0)) { set((T)0); } else
00335         if (c==T(-1)) { for (i=0; i<n; ++i) data[i]=-data[i]; }
00336         else { for (i=0; i<n; ++i) data[i]*=c; }

```

```

00337     };
00338
00339     wbMatrix& operator+= (const T c) { if (c) {
00340         for (size_t n=dim1*dim2, i=0; i<n; ++i) data[i]+=c; }
00341         return *this;
00342     };
00343
00344     void operator-= (const T c) { if (c) {
00345         for (size_t n=dim1*dim2, i=0; i<n; ++i) data[i]-=c; }
00346     };
00347
00348     wbMatrix operator* (const T x) const {
00349         wbMatrix X(*this); if (x!=1) X*=x;
00350         return X;
00351     };
00352
00353     wbMatrix& add (const wbMatrix &B, const T& c=1, const char zflag=0);
00354     wbMatrix& minus(const wbMatrix &B, const T& c=1, const char zflag=0);
00355     wbMatrix& times(T fac, wbMatrix &X) const {
00356         X=*this; X*=fac; return X;
00357     };
00358
00359     wbMatrix& rowTimes(size_t i, T fac);
00360     wbMatrix& colTimes(size_t j, T fac);
00361
00362     T prod() const {
00363         size_t i,n=dim1*dim2; T x=0;
00364
00365         if (n==0) {
00366             wblog(FL,"WRN %s() of null object !?",FCT);
00367             return x;
00368         }
00369
00370         for (x=data[0], i=1; i<n; ++i)
00371             if (x<data[i]) x*=data[i];
00372
00373         return x;
00374     };
00375
00376     T min() const {
00377         size_t i,n=dim1*dim2; T x=0;
00378
00379         if (n==0) { wblog(FL,
00380             "WRN %s() of null object !?",FCT); return x; }
00381         for (x=data[0], i=1; i<n; ++i) { if (x>data[i]) x=data[i]; }
00382
00383         return x;
00384     };
00385
00386     T max() const {
00387         size_t i,n=dim1*dim2; T x=0;
00388
00389         if (n==0) { wblog(FL,
00390             "WRN %s() of null object !?",FCT); return x; }
00391         for (x=data[0], i=1; i<n; ++i) { if (x<data[i]) x=data[i]; }
00392
00393         return x;
00394     };
00395
00396     T aMax() const {
00397         T m=0; if (dim1 && dim2) {
00398             size_t i=0, n=dim1*dim2; T a;
00399             for (; i<n; ++i) { a=ABS(data[i]); if (m<a) { m=a; } }
00400         }
00401         else wblog(FL,"WRN %s() got empty object",FCT);
00402         return m;
00403     };
00404
00405     T aMax(unsigned &i_, unsigned &j_) const {
00406         T m=0, a;
00407         if (!dim1 && !dim2) wblog(FL,"ERR %s() got empty object",FCT);
00408
00409         size_t i,j,l=0;
00410         for (i=0; i<dim1; ++i)
00411             for (j=0; j<dim2; ++j, ++l) { if (m<(a=ABS(data[i]))) {
00412                 m=a; i_=i; j_=j;
00413             } }
00414         return m;
00415     };
00416
00417     T sum() const {
00418         T x=0;
00419         size_t i, n=dim1*dim2;
00420         for (i=0; i<n; ++i) x+=data[i];
00421         return x;
00422     };
00423

```

```

00424 T cumsum0prodRec(wbvector<T> &cs, char xflag=0) const {
00425     wbvector<T> x;
00426     return recProd(x).cumsum_(cs,xflag);
00427 };
00428
00429 T cumsum0prodRec(
00430     wbvector<T> &cs, unsigned i1, unsigned i2, char xflag=0
00431 ) const {
00432
00433     wbvector<T> x(dim1);
00434     if (i2>=dim2) wblog(FL,
00435         "ERR %s() index out of bounds (%d/%d)",FCT,i2+1,dim2);
00436
00437     if (i1<=i2) {
00438         unsigned i=0, j, n=i2-i1+1; const T* d=data+i1;
00439         for (; i<dim1; ++i, d+=dim2) {
00440             x[i]=d[0];
00441             for (j=1; j<n; ++j) { x[i]*=d[j]; }
00442         }
00443     }
00444
00445     return x.cumsum_(cs,xflag);
00446 };
00447
00448 T norm2() const {
00449     T x2=0; size_t i, n=dim1*dim2;
00450     for (i=0; i<n; ++i) x2+=NORM2(data[i]);
00451     return x2;
00452 };
00453 T norm() const { return SQRT(norm2()); }
00454
00455 double normReal() const;
00456 double normImag() const;
00457
00458 T norm2Cols(long j1, long j2) const {
00459     T x2=0; long i,j;
00460
00461     if (j1<0) { j1+=dim2; }
00462     if (j2<0) { j2+=dim2; }; if (j1>j2 || !dim1) { return x2; }
00463     if (j1<0 || j2>=dim2) wblog(FL,
00464         "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,j1,j2,dim2);
00465
00466     for (i=0; i<dim1; ++i)
00467         for (j=j1; j<=j2; ++j) { x2+=NORM2(data[i*dim2+j]); }
00468     return x2;
00469 };
00470
00471 T norm2Recs(long i1, long i2) const {
00472     T x2=0; long i,j;
00473
00474     if (i1<0) { i1+=dim1; }
00475     if (i2<0) { i2+=dim1; }; if (i1>i2 || !dim2) { return x2; }
00476     if (i1<0 || i2>=dim1) wblog(FL,
00477         "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,i1,i2,dim1);
00478
00479     for (i=i1; i<=i2; ++i)
00480         for (j=0; j<dim2; ++j) { x2+=NORM2(data[i*dim2+j]); }
00481     return x2;
00482 };
00483
00484 wbvector<T>& norm2Cols(wbvector<T> &x2_) const {
00485     x2_.init(dim2); T* x2=x2_.data; size_t i,j, l=0;
00486     for (i=0; i<dim1; ++i)
00487         for (j=0; j<dim2; ++j, ++l) { x2[j]+=NORM2(data[l]); }
00488     return x2_;
00489 };
00490
00491 wbvector<T> norm2Cols() const {
00492     wbvector<T> x2; return norm2Cols(x2);
00493 };
00494
00495 wbvector<T> normCols() const {
00496     wbvector<T> x_; norm2Cols(x_); T *x=x_.data;
00497     for (size_t i=0; i<dim2; ++i) { x[i]=SQRT(x[i]); }
00498     return x_;
00499 };
00500
00501 size_t maxRec(char lex=1, size_t *m=NULL) const;
00502 size_t maxRec_float(char lex=1, size_t *m=NULL, T xref=-1) const {
00503     return maxRec(lex,m);
00504 };
00505
00506 T recMax(size_t r, size_t *k=NULL) const;
00507 T colMax(size_t c, size_t *k=NULL) const;
00508
00509 T colMax(size_t c, unsigned &k) const { T x;
00510     size_t K; x=colMax(c,&K); k=K;

```



```

00511         if (size_t(k) != K) wblog(FL,
00512             "ERR %s() unsigned out of bounds (%d/%ld)", FCT, k, K);
00513         return x;
00514     };
00515
00516     wbMatrix& NormalizeCol(size_t k);
00517
00518     bool isOrthoRows(T *x=NULL, char tnorm=0, T eps=1E-14) const;
00519     bool isOrthoRows(const wbMatrix &B,
00520         T *x=NULL, char tnorm=0, T eps=1E-14) const;
00521
00522     double recMaxA(size_t r, size_t *k=NULL) const;
00523
00524     size_t skipNanRecs(wbindex &I);
00525
00526     wbMatrix& ResizeXRecs(size_t r);
00527     wbMatrix& Resize(size_t r, size_t c, const T* dx=NULL);
00528     wbMatrix& resize(size_t m, size_t n, wbMatrix &M) const;
00529
00530     wbMatrix& Resize2Mult(const WBINDEX &M);
00531
00532     wbMatrix& transpose(wbMatrix &M) const;
00533     wbMatrix& Transpose() {
00534         wbMatrix X; save2(X); return X.transpose(*this);
00535     };
00536
00537     wbMatrix& Reshape(size_t r, size_t c);
00538
00539     template <class T2>
00540     wbMatrix<T2>& AddCols(size_t n, const T2* d0=NULL) {
00541         Resize(dim1, dim2+n);
00542         if (d0) Wb::cpyStride(data+dim2-n, d0, n, dim1, dim2);
00543         return *this;
00544     };
00545
00546     template <class T2>
00547     wbMatrix<T2>& AddRows(size_t n, const T2* d0=NULL) {
00548         Resize(dim1+n, dim2);
00549         if (d0) MEM_COPY<T>(data+(dim1-n)*dim2, n*dim2, d0);
00550         return *this;
00551     };
00552
00553     bool isEmpty() const { return data==NULL; };
00554     bool isNormal() const;
00555
00556     bool isFinite() const {
00557         return Wb::is_finite(data, dim1*dim2);
00558     };
00559
00560     bool isSquare() const { return (dim1==dim2); };
00561     bool isSquare(size_t d) const { return (dim1==dim2 && d==dim1); };
00562
00563     bool isScalar() const { return (dim1==1 && dim2==1); };
00564     bool isVector() const { return (dim1==1 || dim2==1); };
00565
00566     bool isDiag() const;
00567     bool isDiagMatrix(T eps=1E-14) const;
00568
00569     bool isIdentity(double eps, double &maxdiff) const;
00570     bool isProptoId(T &x, T eps=1E-14) const;
00571
00572     bool isHConj(
00573         const wbMatrix &B, double eps=1E-12, double *xref=NULL
00574     ) const { return isSym_aux(B, eps, xref, 's'); };
00575
00576     bool isHConj(
00577         double eps=1E-12, double *xref=NULL
00578     ) const { return isSym_aux(*this, eps, xref, 's'); };
00579
00580     bool isAHerm(
00581         const wbMatrix &B, double eps=1E-12, double *xref=NULL
00582     ) const { return isSym_aux(B, eps, xref, 'a'); };
00583
00584     bool isAHerm(
00585         double eps=1E-12, double *xref=NULL
00586     ) const { return isSym_aux(*this, eps, xref, 'a'); };
00587
00588     bool isComplex() const;
00589
00590     bool isUnique() const;
00591
00592     bool isUniqueSorted(char dir=0, char lex=1) const;
00593
00594     int isSorted(
00595         char dir=0,
00596         char lex=1,
00597         size_t m=-1

```

```

00598     ) const;
00599
00600     bool quickCheckSorted(
00601         char dir,
00602         char lex=1,
00603         const char *F=NULL, int L=0,
00604         size_t n=2
00605     ) const;
00606
00607     bool thisIsInt() const;
00608
00609     int getDiff(
00610         const wbMatrix &B, wbindex &Ia, wbindex *Ib=NULL) const;
00611
00612     int getDiffSorted(
00613         const wbMatrix &B, wbindex &Ia, wbindex *Ib=NULL) const;
00614
00615     size_t length() const {
00616         return dim1>dim2 ? (dim2 ? dim1 : 0) : (dim1 ? dim2 : 0);
00617     };
00618
00619     size_t numel() const { return dim1*dim2; };
00620
00621     void flipSign() {
00622         size_t i,s=dim1*dim2;
00623         for (i=0; i<s; ++i) data[i]=-data[i];
00624     };
00625
00626     wbMatrix& Symmetrize() {
00627         size_t i,j,r,s;
00628         if (dim1!=dim2) wblog(FL,
00629             "ERR Symmetrize() called with %dx%d matrix !?",dim1,dim2);
00630
00631         for (j=0; j<dim2; ++j)
00632             for (i=j+1; i<dim1; ++i) {
00633                 r=i*dim2+j; s=j*dim2+i;
00634                 data[r]=data[s]=0.5*(data[r]+data[s]);
00635             }
00636
00637         return *this;
00638     };
00639
00640     void set(const T &x) {
00641         for (size_t s=dim1*dim2, i=0; i<s; ++i) { data[i]=x; }
00642         isdiag=0;
00643     };
00644
00645     wbMatrix& setRand(double fac=1., double shift=0.);
00646     void setDiagRand(double fac=1., double shift=0.);
00647
00648     double maxRelDiff(const wbMatrix &M) const;
00649
00650     template <class T2>
00651     double normDiff(const wbMatrix<T2> &M) const;
00652
00653     T normDiff2(const wbMatrix &M, size_t *k=NULL) const;
00654     T normDiff (const wbMatrix &M, size_t *k=NULL) const {
00655         return SQRT(normDiff2(M,k));
00656     };
00657
00658     bool sameSize(const wbMatrix B) const {
00659         return (dim1==B.dim1 && dim2==B.dim2);
00660     };
00661
00662     WBINDEX& toIndex(WBINDEX &I, const WBINDEX *S=NULL) const {
00663         wblog(FL,"ERR %s() not defined for type '%s'",FCT,
00664             TSTR(I)); return I;
00665     };
00666
00667     WBIDXMAT& toIndex2D(
00668         const WBINDEX &S, const wbindex &ic, WBIDXMAT &IJ,
00669         char pos=1
00670     ) const {
00671         wblog(FL,"ERR %s() not defined for type '%s'",FCT,
00672             TSTR(I)); return IJ;
00673     };
00674
00675     wbMatrix& SkipTiny_float(T ref __attribute__ ((unused)) ==-1) {
00676         return *this;
00677     };
00678
00679     void reset() {
00680         if (dim1 && dim2) MEM_SET<T>(data,dim1*dim2);
00681         isdiag=0;
00682     }
00683
00684     size_t nnz() const {

```

```

00685     size_t n=0, i=0,s=dim1*dim2;
00686     for (i=0; i<s; ++i) { if (data[i]!=T(0)) ++n; }
00687     return n;
00688 };
00689
00690 size_t nnz(T eps) const {
00691     size_t n=0;
00692     if (!eps) { n=nnz(); } else {
00693         size_t i,s=dim1*dim2;
00694         for (i=0; i<s; ++i) { if (ABS(data[i])>eps) ++n; }
00695     }
00696     return n;
00697 };
00698
00699 wbMatrix& CAT(C_UINT d12, const wbMatrix** M, C_UINT len);
00700 wbMatrix& CAT(C_UINT d12, const wbvector< wbMatrix > &M0);
00701 wbMatrix& CAT(C_UINT d12, const wbvector< wbMatrix const* > M);
00702
00703 wbMatrix& CAT(C_UINT d12, const wbvector< wbvector<T> > &M);
00704 wbMatrix& CAT(C_UINT d12, const wbvector< wbvector<T> const* > &M);
00705
00706 wbMatrix& cat(C_UINT d12, const wbMatrix&);
00707 wbMatrix& cat(C_UINT d12, const wbMatrix&, const wbMatrix&);
00708 wbMatrix& cat(C_UINT d12, const wbMatrix&, const wbMatrix&, const wbMatrix&);
00709
00710 wbMatrix& Cat(C_UINT d12,
00711     const wbMatrix&, const wbMatrix&);
00712 wbMatrix& Cat(C_UINT d12,
00713     const wbMatrix&, const wbMatrix&, const wbMatrix&);
00714 wbMatrix& Cat(C_UINT d12,
00715     const wbMatrix&, const wbMatrix&, const wbMatrix&, const wbMatrix&);
00716
00717 wbMatrix<T>& CAT(const unsigned dim,
00718     wbvector< wbMatrix<T> const* > M,
00719     wbvector< WBINDEX const* > I
00720 );
00721
00722 wbMatrix& Cat(const unsigned dim,
00723     const wbMatrix &M1, const wbindex &I1,
00724     const wbMatrix &M2, const wbindex &I2
00725 ){
00726     wbvector< wbMatrix<T> const* > M(2);
00727     wbvector< WBINDEX const* > I(2);
00728     M.data[0]=&M1; I.data[0]=&I1;
00729     M.data[1]=&M2; I.data[1]=&I2;
00730     return CAT(dim,M,I);
00731 };
00732
00733 void split(C_UINT d12, wbvector< wbMatrix* > &M) const;
00734 void split(C_UINT d12, wbMatrix** M, C_UINT len) const;
00735 void split(C_UINT d12, wbMatrix &M1, wbMatrix &M2, wbMatrix &M3) const;
00736
00737 const T& operator() (size_t i, size_t j, const T&x) const {
00738     if (i<dim1 && j<dim2) return data[i*dim2+j];
00739     else return x;
00740 };
00741 T& operator() (size_t i, size_t j, const T&x) {
00742     if (i<dim1 && j<dim2) return data[i*dim2+j];
00743     else return x;
00744 };
00745
00746 const T& operator() (size_t i, size_t j) const { return data[i*dim2+j]; };
00747 T& operator() (size_t i, size_t j) { return data[i*dim2+j]; };
00748
00749 const T& el(size_t i, size_t j) const {
00750     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00751         "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00752     return data[i*dim2+j];
00753 };
00754 T& el(size_t i, size_t j) {
00755     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00756         "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00757     return data[i*dim2+j];
00758 };
00759
00760 const T& els(long i, long j) const {
00761     if (i<0) i+=dim1;
00762     if (j<0) j+=dim2;
00763     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00764         "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00765     return data[i*dim2+j];
00766 };
00767 T& els(long i, long j) {
00768     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00769         "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00770     return data[i*dim2+j];
00771 };

```

```

00772
00773     const T* operator() (size_t i) const { return data+i*dim2; };
00774     T* operator() (size_t i)          { return data+i*dim2; };
00775
00776     const T& operator[] (size_t i) const { return data[i]; };
00777     T& operator[] (size_t i)          { return data[i]; };
00778
00779     const T* ref(size_t i, size_t j=0) const {
00780         if (int(j)<0) j=dim2+j;
00781         if (i>=dim1 || j>dim2) wblog(FL,"ERR %s() "
00782             "index out of bounds (%ld/%ld, %ld/%ld)",FCT,i,dim1,j,dim2);
00783         return data+(i*dim2+j);
00784     };
00785     T* ref(size_t i, size_t j=0) {
00786         if (int(j)<0) j=dim2+j;
00787         if (i>=dim1 || j>dim2) wblog(FL,"ERR %s() "
00788             "index out of bounds (%ld/%ld, %ld/%ld)",FCT,i,dim1,j,dim2);
00789         return data+(i*dim2+j);
00790     };
00791
00792     const T* rec(size_t i) const {
00793         if (i>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",i,dim1);
00794         return data+i*dim2;
00795     };
00796
00797     T* rec(size_t i) {
00798         if (i>=dim1) wblog(FL,
00799             "ERR %s() index out of bounds (%d/%d)",FCT,i,dim1);
00800         return data+i*dim2;
00801     };
00802
00803     void getDiag(wbvector<T> &D) const {
00804         size_t i, n=MIN(dim1,dim2); D.init(n);
00805         for (i=0; i<n; ++i) D[i]=data[i*dim2+i];
00806     };
00807     wbvector<T> getDiag() const {
00808         wbvector<T> D; getDiag(D); return D;
00809     };
00810
00811     void appendRows (size_t n, const T* =NULL);
00812     void appendRow(const wbvector<T> &v);
00813
00814     mxArray* toMx (char raw=0) const;
00815     mxArray* toMxP(char flag=0) const;
00816     mxArray* toMx_base(char raw=0) const;
00817     mxArray* toMx_base_d(char raw=0) const;
00818
00819     mxArray* toMx_Struct() const;
00820     mxArray* toMxP_S() const;
00821     mxArray* toMxP_C() const;
00822
00823     mxArray* toMxT(char raw=0) const { return toMx(); }
00824
00825     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00826     void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
00827
00828     void toMxStruct(mxArray* S, const char *vname) const;
00829     void mat2mx (mxArray* &a) const { a=toMx(); };
00830     void mat2mxs(mxArray* a, char field_nr) const;
00831
00832     void info(const char *istr="ans") const;
00833     void print(const char *istr="", char mflag=0) const;
00834     void Print(const char *istr="", char mflag=0) const;
00835
00836     void recPrint(size_t i, const char *istr="", char mflag=0) const;
00837
00838     void printdata(
00839         const char *istr, const char *dbl_fmt=" %8.4g", const char *rsep="\n") const;
00840
00841     wbstring toStr() const { return toStr("", " ", ";", "0,"); };
00842
00843     wbstring toStr(
00844         const char *fmt, const char *sep=" ", const char *rsep=";",
00845         size_t stride=0, const char *sep2=""
00846     ) const;
00847
00848     wbstring rec2Str(
00849         size_t k, const char *fmt="", const char *sep=" ",
00850         size_t stride=0, const char *sep2=""
00851     ) const;
00852
00853     wbstring sizeStr() const {
00854         wbstring s(16);
00855         size_t l=snprintf(s.data,s.len,"%ldx%ld",dim1,dim2);
00856         if (l>s.len) wblog(FL,
00857             "WRN %s() string out of bounds (%ld/%ld)",FCT,l,s.len);
00858         return s;

```

```

00859     };
00860
00861     void put(const char *F, int L,
00862             const char *vname="ans", const char *ws="caller", const char raw=0
00863     ) const {
00864         mxArray *a=toMx(raw);
00865         mxPutAndDestroy(F_L,a,vname,ws);
00866     };
00867
00868     void put(const char *vname="ans", const char *ws="caller",
00869             const char raw=0) const { put(0,0,vname,ws,raw); }
00870
00871     void putg(const char *vname="ans", const char raw=0
00872     ) const { put(vname, "global",raw); return; };
00873
00874     void putx(const char *vname="ans", const char raw=0
00875     ) const { put(vname, "caller", raw); return; };
00876
00877     void put0(const char *vname="ans", const char *ws="caller") const {
00878         put(vname,ws,'r');
00879     }
00880
00881     void getReal(wbMatrix<double> &R) const;
00882     void getImag(wbMatrix<double> &I) const;
00883     void set(const wbMatrix<double> &R, const wbMatrix<double> &I);
00884     wbMatrix& Conj();
00885
00886     void setRec(size_t r, const T x) {
00887         T* d=data+r*dim2;
00888         if (r>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",r,dim1);
00889         for (size_t i=0; i<dim2; ++i) d[i]=x;
00890     };
00891
00892     void setRec(size_t r, const T x1, const T x2) {
00893         size_t i=0, m=dim2%2, n=dim2-m; T* d=data+r*dim2;
00894
00895         if (r>=dim1) wblog(FL,
00896             "ERR index out of bounds (%d/%d)",r,dim1);
00897
00898         for (; i<n; i+=2) { d[i]=x1; d[i+1]=x2; }
00899         if (m) d[i]=x1;
00900     };
00901
00902     void recSet(size_t idest, size_t isrc);
00903
00904     void recSet(size_t, const wbvector<T>&);
00905
00906     template<class T2>
00907     void recSetT(size_t, const wbvector<T2>&);
00908
00909     void recSet(size_t, const wbMatrix&, size_t);
00910     void recSet(size_t i, const wbvector<T>&, const wbvector<T>&);
00911
00912     void recSet (size_t, const T*, T bfac=1);
00913     void recSetP(size_t, const T*, size_t n=-1);
00914     void recSetP(size_t, const T*, size_t, const T*, size_t);
00915     void recSetP(size_t,
00916         const T*, const wbindex &i1, const T*, const wbindex &i2);
00917
00918     void recSetB(size_t, size_t, size_t D, const T*,
00919         const T* =NULL, const T bfac=1);
00920     void recSetB(size_t, const wbindex&, size_t D, const T*);
00921
00922     void recSet(size_t k,
00923         const wbvector<T> &a, const wbindex &ia,
00924         const wbvector<T> &b, const wbindex &ib
00925     ){ return recSet(a.data,ia,b.data,ib); };
00926
00927     void recSet(size_t k,
00928         const T* a, const wbindex &ia,
00929         const T* b, const wbindex &ib
00930     );
00931
00932     template<class T2>
00933     void setCol(size_t k, const wbvector<T2> &v);
00934
00935     template<class T2>
00936     void setCol(size_t k, const T2 *v, T fac=1, size_t stride=1);
00937
00938     void setCol(size_t k, const T& x);
00939
00940     template <class T2>
00941     void setColT(size_t k, const wbvector<T2> &v);
00942     template <class T2>
00943     void setColT(size_t k, const T2* v, T fac=1, size_t stride=1);
00944
00945     void setLastCol(const T& x) { setCol(dim2-1,x); };

```

```

00946
00947 void recAddP(size_t, const T*);
00948 int recLess (size_t, size_t, char lex=1) const;
00949 int recLessE(size_t, size_t, char lex=1) const;
00950 int recEqual(size_t, size_t) const;
00951 int recEqual(size_t j1, const T *d) const;
00952
00953 T recDiff2(size_t i1, size_t i2, size_t n=-1) const;
00954 T recDiff2P(size_t i1, const T* b, size_t n=-1) const;
00955
00956 bool recAllEqual(const T *v) {
00957     for (size_t i=0; i<dim1; ++i) { if (recCompareP(i,v)) return 0; }
00958     return 1;
00959 };
00960
00961 bool recAllEqual() {
00962     for (size_t i=1; i<dim1; ++i) { if (recCompare(i,0)) return 0; }
00963     return 1;
00964 };
00965
00966 bool recIsZero(size_t r) {
00967     T *d=data+r*dim2;
00968     if (r>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",r,dim1);
00969     for (size_t i=0; i<dim2; ++i) if (d[i]!=0) return 0;
00970     return 1;
00971 };
00972
00973 char recCompare(
00974     size_t, size_t, size_t m=-1, char lex=1, T eps=0) const;
00975 char recCompare(
00976     size_t, const wbvector<T> &r2, char lex=1) const;
00977 char recCompareP(
00978     size_t, const T*, size_t m=-1, char lex=1, T eps=0) const;
00979
00980 void set2Rec(size_t i) { recSet(0,i); Resize(1,dim2); };
00981
00982 wbMatrix& Set2Recs(const WBINDEX &I) {
00983     wbMatrix X; save2(X);
00984     return X.getRecs(I,*this);
00985 };
00986
00987 wbvector<T>& getRec(size_t j0, wbvector<T> &v, char ref=0) const;
00988 wbvector<T> getRec(size_t j0) const {
00989     wbvector<T> v; return getRec(j0,v);
00990 };
00991
00992 size_t UnionRecs(const wbMatrix &B, wbindex &Ia);
00993
00994 bool gotRecOverlap (const wbMatrix &B) const;
00995 bool gotRecOverlapSA(const wbMatrix &B) const;
00996
00997 size_t findValsCol(
00998     const size_t k, const wbvector<T> &v, wbindex &Ia) const;
00999
01000 wbMatrix& getRecs(const WBINDEX &, wbMatrix &M) const;
01001 wbMatrix getRecs(const WBINDEX &I) const {
01002     wbMatrix M; getRecs(I,M);
01003     return M;
01004 };
01005
01006 wbMatrix& getRecs(
01007     size_t j1, size_t j2, wbMatrix &M, size_t m=0) const;
01008
01009 wbMatrix getRecs(
01010     size_t i1, size_t i2, size_t m=0) const {
01011     wbMatrix M; getRecs(i1,i2,M,m);
01012     return M;
01013 };
01014
01015 wbvector<T>& getCol(const size_t c, wbvector<T> &v) const;
01016 wbvector<T> getCol(const size_t c) const {
01017     wbvector<T> v; getCol(c,v); return v;
01018 };
01019
01020 wbMatrix getCols(const size_t j1, const size_t j2) const {
01021     wbMatrix M; getCols(j1,j2,M);
01022     return M;
01023 };
01024
01025 wbMatrix& getCols(size_t j1, size_t j2, wbMatrix &M) const;
01026
01027 template <class TI>
01028 wbMatrix& getCols(size_t n, const TI *I, wbMatrix<T> &M) const;
01029
01030 wbMatrix& getCols(const wbpem &p, wbMatrix &M) const {
01031     return getCols(p.len,p.data,M);
01032 };

```

```

01033
01034     wbMatrix& getCols(
01035         const wbindex &I, wbMatrix &M, char iflag=0) const {
01036
01037         if (!iflag) { return getCols(I.len,I.data,M); }
01038         else {
01039             wbindex I2; I.invert(dim2,I2);
01040             return getCols(I2.len,I2.data,M);
01041         }
01042     };
01043
01044     wbMatrix& set2Cols(
01045         const size_t j1, const size_t j2);
01046
01047     wbMatrix& set2Cols(const wbindex &J, char iflag=0) {
01048         wbMatrix X; this->save2(X); X.getCols(J,*this,iflag);
01049         return *this;
01050     };
01051
01052     wbMatrix& recPermute(const wbperm &P, wbMatrix &M, char iflag=0) const;
01053     wbMatrix& recPermute(const wbperm &P, char iflag=0) {
01054         wbMatrix M(*this);
01055         return M.recPermute(P,*this,iflag);
01056     };
01057
01058     wbMatrix& colPermute(const wbperm &P, wbMatrix &M, char iflag=0) const;
01059     wbMatrix& colPermute(const wbperm &P, char iflag=0){
01060         wbMatrix M; save2(M); return M.colPermute(P,*this,iflag);
01061     };
01062
01063     wbMatrix& FlipRecs() { wbperm P(dim1,'r'); return recPermute(P); };
01064     wbMatrix& FlipCols() { wbperm P(dim2,'r'); return colPermute(P); };
01065
01066     wbMatrix& cols2Front(
01067         const WBINDEX &I1, wbMatrix &M) const;
01068     wbMatrix& cols2End(
01069         const WBINDEX &I1, wbMatrix &M) const;
01070
01071     void blockPermute(const wbperm &P, wbMatrix &M) const;
01072     void BlockPermute(const wbperm &P){
01073         wbMatrix X; save2(X); X.blockPermute(P,*this);
01074     };
01075
01076     wbvector<T>& recProd(wbvector<T> &p) const;
01077     T recProd(size_t) const;
01078
01079     T recSum (size_t) const;
01080     wbvector<T>& recSum(wbvector<T> &) const;
01081     wbvector<T>& recSumA(wbvector<T> &) const;
01082     wbvector<T> recSum() const { wbvector<T> s; return recSum(s); };
01083     T colSum (size_t) const;
01084
01085     wbMatrix& repmat(
01086         size_t m, size_t n, wbMatrix &Q,
01087         size_t pad1=0, size_t pad2=0
01088     ) const;
01089
01090     wbMatrix& Repmat(
01091         size_t m, size_t n,
01092         size_t pad1=0, size_t pad2=0
01093     ){ wbMatrix X(*this); return X.repmat(m,n,*this,pad1,pad2); };
01094
01095     void ColKron(
01096         const wbMatrix &Q,
01097         int d=-1
01098     );
01099
01100     wbMatrix& getBlocks(
01101         size_t j1, size_t j2,
01102         const size_t D, wbMatrix &M
01103     ) const;
01104
01105     wbMatrix& getBlock(
01106         size_t j1,
01107         const size_t D, wbMatrix &M) const;
01108
01109     wbMatrix getBlock(
01110         size_t j1,
01111         const size_t D
01112     ) const {
01113         wbMatrix M; getBlock(j1,D,M);
01114         return M;
01115     };
01116
01117     template <class T2>
01118     wbMatrix<T>& getBlock(
01119         size_t k, size_t D,

```

```

01120         const wbMatrix<T2>& R, size_t k2, size_t D2,
01121         wbMatrix &M
01122     ) const;
01123
01124     wbMatrix& getBlock(
01125         size_t i, size_t j,
01126         size_t n, size_t m,
01127         wbMatrix &M
01128     ) const;
01129
01130     wbMatrix& blockSum(size_t D, wbMatrix&) const;
01131     void blockSum(size_t, size_t, const size_t, wbMatrix<double>&) const;
01132     void blockSum(const wbindex &J, const size_t, wbMatrix<double>&) const;
01133
01134     void setBlock(size_t k, const size_t D, const T*);
01135     void SetBlock(size_t i0, size_t j0, const wbMatrix &M);
01136
01137     WBIDXMAT& toBlockIndex(
01138         widx_t D, WBIDXMAT &II,
01139         wbvector< wbMatrix > *QI=NULL
01140     ) const;
01141
01142     template <class T2>
01143     void toBlockIndex(widx_t D,
01144         wbMatrix<T2> &R, widx_t DR,
01145         WBIDXMAT *II=NULL, WBIDXMAT *IS=NULL,
01146         WBIDXMAT *SS=NULL, wbvector< wbMatrix > *QI=NULL
01147     ) const;
01148
01149     void groupRecs(
01150         wbperm &P, WBINDEX &d,
01151         size_t m=-1,
01152         char lex=1,
01153         wbindex *Ig=NULL,
01154         wbMatrix<T> *X=NULL
01155     );
01156
01157     void groupRecs(wbindex &Ig, widx_t m=-1, char lex=1) {
01158         wbperm P; wbvector<widx_t> d;
01159         groupRecs(P,d,m,lex,&Ig);
01160     };
01161
01162     void groupRecs(
01163         groupIndex<widx_t> &IG, size_t m=-1, char lex=1,
01164         wbindex *Ig=NULL
01165     ){
01166         wbperm P; wbvector<widx_t> d;
01167         groupRecs(P,d,m,lex,Ig);
01168         IG.initX(P,d);
01169     };
01170
01171     void groupRecs(
01172         groupIndex<widx_t> &IG, size_t m, char lex, const wperm_t **p
01173     ){
01174         wbperm P; wbvector<widx_t> d;
01175         groupRecs(P,d,m,lex);
01176         IG.initX(P,d,p);
01177     };
01178
01179     void groupRecs(
01180         wbperm &P, WBINDEX &d, wbMatrix &B,
01181         size_t m=-1, char lex=1, wbindex *Ig=NULL
01182     ) const { B=*this; B.groupRecs(P,d,m,lex,Ig); };
01183
01184     void groupRecs(
01185         wbperm &P, WBINDEX &d,
01186         const WBINDEX *I, char lex=1
01187     );
01188
01189     template <class T2>
01190     void groupRecs_rdeg(wbperm &P, WBINDEX &D, const wbMatrix<T2> &R,
01191         WBINDEX *Ib=NULL,
01192         WBINDEX *I2=NULL,
01193         WBINDEX *Sb=NULL,
01194         char iflag=0
01195     );
01196
01197     void groupRecs(
01198         wbperm &P, WBINDEX &D, size_t nc,
01199         WBINDEX &Ib,
01200         WBINDEX &I2,
01201         WBINDEX &Sb,
01202         char iflag=0
01203     );
01204
01205     void groupSortedRecs(
01206         WBINDEX &d, char keepall=0, size_t m=-1, char lex=1);

```



```

01207
01208     void groupSortedRecs(
01209         WBINDEX &d, const WBINDEX *I);
01210
01211     void groupSortedRecs(
01212         WBINDEX &d,
01213         size_t mc,
01214         char lex,
01215         WBINDEX &Ib,
01216         WBINDEX &I2,
01217         WBINDEX &Sb
01218     );
01219
01220     void makeUnique() { if (dim1<2) return;
01221         wbperm P; WBINDEX D;
01222         groupRecs(P,D);
01223     };
01224     void makeUnique(wbperm &P, WBINDEX &D) {
01225         if (dim1<2) {
01226             P.init(dim1); D.init(dim1); if (dim1) D[0]=1;
01227             return;
01228         }
01229         groupRecs(P,D);
01230     };
01231
01232     void makeUnique(wbindex &I) {
01233         if (dim1<2) { I.init(dim1); }
01234         else {
01235             size_t i=0, l=0, d; wbperm P; WBINDEX D;
01236             groupRecs(P,D); I.init(D.len);
01237             for (; i<D.len; ++i, l+=d) { d=D[i]; I[i]=P[l]; }
01238         }
01239     };
01240
01241     void findUnique(wbindex &I) const {
01242         if (dim1<2) { I.init(dim1); return; }
01243
01244         wbMatrix X(*this);
01245         wbperm P; WBINDEX D; X.groupRecs(P,D);
01246         I.init(D.numEQ(T(1)));
01247         for (size_t d, l=0, j=0, i=0; i<D.len; ++i, l+=d) {
01248             if ((d=D[i])==1) I[j++]=P[l];
01249         }
01250     };
01251
01252     size_t findUniqueRecSorted1(size_t n=-1, T eps=T(1E-12)) const;
01253
01254     void makeUnique_ig(wbindex &Iu) {
01255         wbperm P; wbvector<widxt> D; groupRecs(P,D);
01256         Iu.initGroup(P,D);
01257     };
01258     void makeUnique(groupIndex<widxt> &IG) { groupRecs(IG); };
01259
01260     wbMatrix& SortRecs(wbperm &P,
01261         char dir=+1,
01262         char lex=1
01263     );
01264
01265     wbMatrix& sortRecs(
01266         wbMatrix &B, wbperm &P, char dir=+1, char lex=1
01267     ) {
01268         B=*this;
01269         return B.SortRecs(P,dir,lex);
01270     };
01271
01272     wbMatrix& SortRecs(char dir=+1, char lex=1) {
01273         wbperm P; return SortRecs(P,dir,lex);
01274     };
01275
01276     wbMatrix& sortRecs_float(wbperm &P, char dir=+1) {
01277         P.init(dim1); return *this;
01278     };
01279     wbMatrix& sortRecs_float() { wbperm P; return sortRecs_float(P); };
01280
01281     char findRecsInSet(const wbMatrix &S, wbvector<int> &I) const;
01282     int findRecSorted(const T* r, size_t n=-1, char lex=1) const;
01283     int findRec(const T* r, size_t n=-1, char lex=1) const;
01284
01285     T *data;
01286
01287     size_t dim1, dim2;
01288
01289     char isdiag, isref;
01290
01291 protected:
01292
01293     void NEW_DATA(T* d=NULL) { size_t s=dim1*dim2;

```

```

01294         if (data) { WB_DELETE(data); }
01295     if (s) {
01296         WB_NEW(data,s);
01297         MEM_COPY<T>(data,s,d);
01298     }
01299     else data=NULL;
01300 };
01301
01302 void RENEW(
01303     const size_t &d1, const size_t &d2, const T* dd=NULL, char iflag=1
01304 ){
01305     const size_t s=d1*d2; isdiag=0;
01306
01307     if (isref) {
01308         if (d1==0 || d2==0) {
01309             if (d1 || d2) wblog(FL,
01310                 "WRN wbMatrix is declared as reference "
01311                 "%dx%d => %dx%d; %d",dim1,dim2,d1,d2,isref
01312             );
01313
01314             dim1=d1; dim2=d2; isref=0;
01315
01316             data=NULL;
01317
01318             return;
01319         }
01320         else {
01321             wblog(FL,"ERR wbMatrix is declared as reference (%dx%d; %d)",
01322                 d1,d2, isref);
01323         }
01324     }
01325
01326     if (d1!=dim1 || d2!=dim2) {
01327         dim1=d1; dim2=d2; if (data) {
01328             if (dd==data) wblog(FL,
01329                 "ERR init space equals *this (use Resize instead)");
01330             WB_DELETE(data);
01331         }
01332         if (d1==0 || d2==0) return;
01333         WB_NEW(data,s);
01334     }
01335
01336     if (!iflag && !dd) return;
01337     if (s) MEM_COPY<T>(data,s,dd);
01338 };
01339
01340 private:
01341
01342     bool isSym_aux(
01343         const wbMatrix &B, double eps, double *xref,
01344         const char symflag='s'
01345     ) const;
01346 };
01347
01348 template <class T>
01349 class wbRecs {
01350     const T* data;
01351     size_t n,lda; bool asc, lex;
01352
01353 public:
01354     wbRecs(const wbMatrix<T> &A, wbperm &P, char dir_=1, char lex_=1)
01355         : data(A.data), n(A.dim1), lda(A.dim2),
01356         asc(dir_> 0 ? 1 : 0), lex(lex_> 0 ? 1 : 0)
01357     {
01358         P.init(n);
01359     };
01360
01361     ~wbRecs() {};
01362
01363     bool operator()(const size_t &pa, const size_t &pb) const {
01364         if (pa>=n || pb>=n) wblog(FL,
01365             "ERR %s() index out of bounds (%d,%d; %d)",FCT,pa,pb,n);
01366
01367         const T *a=data+pa*lda, *b=data+pb*lda;
01368         if (lex) { size_t i=0;
01369             if (asc) {
01370                 for (; i<lda; ++i) {
01371                     if (a[i]<b[i]) return 1;
01372                     if (a[i]>b[i]) return 0;
01373                 }
01374             }
01375             else {
01376                 for (; i<lda; ++i) {
01377                     if (a[i]<b[i]) return 1;
01378                     if (a[i]>b[i]) return 0;
01379                 }
01380             }
01381         }
01382     }
01383 };

```

```

01381         for (; i<lda; ++i) {
01382             if (a[i]<b[i]) return 0;
01383             if (a[i]>b[i]) return 1;
01384         }
01385     }
01386 }
01387 else { size_t i=lda-1;
01388     if (asc) {
01389         for (; i<lda; --i) {
01390             if (a[i]<b[i]) return 1;
01391             if (a[i]>b[i]) return 0;
01392         }
01393     }
01394     else {
01395         for (; i<lda; --i) {
01396             if (a[i]<b[i]) return 0;
01397             if (a[i]>b[i]) return 1;
01398         }
01399     }
01400 }
01401 if (asc) return (pa<pb);
01402 else return (pb<pa);
01403 };
01404 };
01405
01406 template <class T>
01407 void getSortPerm_OMP(
01408     const wbMatrix<T> &A, wbperm &P, char dir, char lex);
01409
01410 template <class T>
01411 bool wbMatrix<T>::deepEqualP(const wbMatrix<T> &B) const {
01412     if (dim1!=B.dim1 || dim2!=B.dim2) return 0;
01413
01414     if (data!=B.data) {
01415         size_t i=0, n=numel();
01416         for (; i<n; ++i) { if (data[i]==B.data[i]) continue;
01417             if (!data[i] || !B.data[i]) return 0; else
01418                 if (!(*data[i]==(*B.data[i]))) return 0;
01419         }
01420     }
01421
01422     return 1;
01423 };
01424
01425 template <class T> inline
01426 bool wbMatrix<T>::isDiag() const {
01427     size_t i,j, l=0;
01428
01429     for (i=0; i<dim1; ++i)
01430         for (j=0; j<dim2; ++j, ++l) { if (i!=j && data[l]) return 0; }
01431
01432     return 1;
01433 };
01434
01435 template <class T>
01436 bool wbMatrix<T>::isDiagMatrix(T eps) const {
01437
01438     if (!dim1 || dim1!=dim2) return 0;
01439     size_t i,j, l=0;
01440
01441     for (i=0; i<dim1; ++i)
01442         for (j=0; j<dim2; ++j, ++l) { if (i!=j && ABS(data[l])>eps) return 0; }
01443
01444     return 1;
01445 };
01446
01447 template <class T>
01448 bool wbMatrix<T>::isIdentity(double eps, double &maxdiff) const {
01449
01450     if (!dim1 || dim1!=dim2) return 0;
01451     size_t i,j,l=0; double d; maxdiff=0;
01452
01453     for (i=0; i<dim1; ++i)
01454         for (j=0; j<dim2; ++j, ++l) {
01455             d = ABS( data[l] - (i!=j ? 0 : 1) );
01456             if (maxdiff<d) maxdiff=d;
01457         }
01458
01459     return (maxdiff<eps);
01460 };
01461
01462 template <class T>
01463 bool wbMatrix<T>::isProptoId(T &x, T eps) const {
01464
01465     if (!dim1 || dim1!=dim2) return 0;
01466     size_t i,j,l=0;
01467

```

```

01468     for (i=0; i<dim1; ++i)
01469     for (j=0; j<dim2; ++j, ++l) {
01470         if (ABS(data[l] - (i!=j ? 0 : data[0]))>eps) { return 0; }
01471     }
01472     x=data[0]; return 1;
01473 };
01474
01475 template <class T> inline
01476 bool wbMatrix<T>::isSym_aux(
01477     const wbMatrix<T> &B, double eps, double *xref,
01478     const char symflag
01479 ) const {
01480
01481     const wbMatrix<T> &A>(*this);
01482     char issame = (&A==&B);
01483     size_t i,j,k;
01484     double x;
01485
01486     if (A.dim1!=B.dim2 || A.dim2!=B.dim1) return 0;
01487     if (xref) *xref=0;
01488
01489     if (symflag=='s') {
01490         if (eps==0) {
01491             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01492                 for (j=k; j<dim2; ++j) {
01493                     if (A(i,j)!=CONJ(B(j,i))) {
01494                         if (xref) {
01495                             x = ABS(A(i,j) - CONJ(B(j,i)));
01496                             xref[0] = MAX(*xref,x);
01497                         }
01498                         else return 0;
01499                     }
01500                 }
01501             }
01502         }
01503         else {
01504             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01505                 for (j=k; j<dim2; ++j) {
01506                     x = ABS(A(i,j) - CONJ(B(j,i)));
01507                     if (x>eps) { if (xref) xref[0]=MAX(*xref,x); else return 0; }
01508                 }
01509             }
01510         }
01511     }
01512     else if (symflag=='a') {
01513         if (eps==0) {
01514             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01515                 for (j=k; j<dim2; ++j) {
01516                     if (A(i,j)!=-CONJ(B(j,i))) {
01517                         if (xref) {
01518                             x = ABS(A(i,j) + CONJ(B(j,i)));
01519                             xref[0] = MAX(*xref,x);
01520                         }
01521                         else return 0;
01522                     }
01523                 }
01524             }
01525         }
01526         else {
01527             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01528                 for (j=k; j<dim2; ++j) {
01529                     x = ABS(A(i,j) + CONJ(B(j,i)));
01530                     if (x>eps) { if (xref) xref[0]=MAX(*xref,x); else return 0; }
01531                 }
01532             }
01533         }
01534     }
01535     else wblog(FL,"ERR invalid symflag=%c<%d>",symflag,symflag);
01536
01537     if (xref && (*xref)>eps) return 0;
01538     return 1;
01539 }
01540
01541 template <class T> inline
01542 bool wbMatrix<T>::isComplex() const { return 0; };
01543
01544 template<> inline
01545 bool wbMatrix<wbcomplex>::isComplex() const {
01546     for (size_t s=dim1*dim2, i=0; i<s; ++i)
01547         if (data[i].i!=0.) return 1;
01548     return 0;
01549 }
01550
01551 template <class T> inline
01552 bool wbMatrix<T>::isUnique() const {
01553     wbMatrix<T> Q(*this);
01554     wbperm P;

```

```

01555     if (!dim2) { return (dim1<=1); }
01556
01557     Q.SortRecs(P);
01558     return Q.isUniqueSorted();
01559 };
01560
01561 template <class T> inline
01562 bool wbMatrix<T>::isUniqueSorted(char dir, char lex) const {
01563
01564     if (dim1<=1 || !dim2) return 1;
01565     size_t i=1; char c=0;
01566
01567     if (!dir) { if (dim1<=2) return 1;
01568                 c=recCompare(1,0,-1,lex); if (!c) { return 0; }
01569                 i=2;
01570     }
01571     else if (abs(int(dir))>1) {
01572         if (dir=='A') c=+1; else
01573         if (dir=='D') c=-1; else
01574             wblog(FL,"ERR %s() invalid c='%c'<%d> !?",FCT,c,c);
01575     }
01576     else { c=dir; }
01577
01578     for (; i<dim1; ++i) {
01579         if (recCompare(i,i-1,-1,lex)!=c) return 0;
01580     }
01581
01582     return 1;
01583 };
01584
01585 template <class T> inline
01586 int wbMatrix<T>::isSorted(char dir, char lex, size_t m) const {
01587
01588     if (!dim1 || !dim2) { return -1; }
01589     if (dim1==1) { return 3; }
01590
01591     size_t j=1, ndeg=0; char c=0;
01592
01593     if (!dir) {
01594         for (; j<dim1; ++j, ++ndeg) {
01595             if ((c=recCompare(j,j-1,m,lex))) { dir=c; ++j; break; }
01596         }
01597     }
01598     else if (abs(int(dir))>1) {
01599         wblog(FL,"ERR %s() invalid direction s=%d",FCT,dir);
01600     }
01601
01602     for (; j<dim1; ++j) {
01603         if ((c=recCompare(j,j-1,m,lex))) { if (c!=dir) return 0; }
01604         else { ++ndeg; }
01605     }
01606
01607     return (1 | (ndeg ? 0 : 2) | (c<0 ? 4 : 0));
01608 };
01609
01610 template <class T> inline
01611 wbMatrix<T>& wbMatrix<T>::ResizeXRecs(size_t nr) {
01612     no_ref_(FLF);
01613
01614     if (nr) {
01615         if (nr<=dim1 && nr/double(dim1)>0.10) dim1=nr;
01616         else Resize(nr,dim2);
01617     }
01618     else init(0,dim2);
01619
01620     return *this;
01621 };
01622
01623 template <class T> inline
01624 wbMatrix<T>& wbMatrix<T>::resize(
01625     size_t m, size_t n, wbMatrix<T> &M
01626 ) const {
01627
01628     if (this==&M) return M.Resize(m,n);
01629
01630     M.init(m,n);
01631     m=MIN(m,dim1); n=MIN(n,dim2); if (m==0 || n==0) return M;
01632
01633     for (size_t i=0; i<m; ++i)
01634         MEM_COPY<T>(M.data+i*M.dim2, n, data+i*dim2);
01635
01636     return M;
01637 };
01638
01639 template <class T> inline
01640 wbMatrix<T>& wbMatrix<T>::Resize(size_t nr, size_t nc, const T* dx) {
01641

```

```

01642     size_t d1,d2,s=nr*nc; T *d0=data;
01643     no_ref_(FLF);
01644
01645     d1=MIN(dim1,nr); d2=MIN(dim2,nc);
01646
01647     if (dim1==nr && dim2==nc) return *this;
01648     if (d1==0 || d2==0) { RENEW(nr,nc); return *this; }
01649
01650     WB_NEW(data,s);
01651
01652     if (nc==dim2) {
01653         MEM_COPY<T>(data,d1*dim2,d0);
01654         if (nr>dim1) {
01655             if (dx)
01656                 MEM_COPY<T>(data+dim1*dim2,(nr-dim1)*dim2,dx);
01657             else MEM_SET<T>(data+dim1*dim2,(nr-dim1)*dim2);
01658         }
01659     }
01660     else {
01661         size_t i; T *p=data, *p0=d0;
01662         for (i=0; i<d1; ++i, p+=nc, p0+=dim2) {
01663             MEM_COPY<T>(p,d2,p0); if (d2<nc) {
01664                 MEM_SET<T>(p+d2,nc-d2); }
01665             if (nr>dim1) {
01666                 if (dx) wblog(FL,"ERR %s() "
01667                     "allows no reference data if dim2=%d->%d",FCT,dim2,nc);
01668                 MEM_SET<T>(data+dim1*dim2,(nr-dim1)*dim2);
01669             }
01670         }
01671     }
01672
01673     dim1=nr; dim2=nc;
01674     WB_DELETE(d0);
01675
01676     return *this;
01677 };
01678
01679 template <class T> inline
01680 wbMatrix<T>& wbMatrix<T>::Resize2Mult(const WBINDEX &M) {
01681
01682     size_t m=M.sum();
01683
01684     if (M.len!=dim1) wblog(FL,
01685         "ERR %s() size mismatch (%d/%dx%d)",FCT,M.len,dim1,dim2);
01686     if (!m || !dim2) { return init(m,dim2); } else
01687     if (int(m)<0) wblog(FL,"ERR %s() got m=%d !?",FCT,m);
01688
01689     size_t i,j;
01690     wbMatrix<T> X(m,dim2);
01691     const T *x0=data; T *x=X.data;
01692
01693     for (i=0; i<M.len; ++i, x0+=dim2) { m=M.data[i];
01694         for (j=0; j<m; ++j, x+=dim2) MEM_COPY<T>(x, dim2, x0); }
01695
01696     return X.save2(*this);
01697 };
01698
01699 template <class T> inline
01700 wbMatrix<T>& wbMatrix<T>::unRef() {
01701
01702     if (!isref) return *this;
01703     if (!dim1 || !dim2) {
01704         if (data) wblog(FL,"ERR data=%lX (%dx%d) !?",data,dim1,dim2);
01705         isref=0; return *this;
01706     }
01707
01708     size_t s=dim1*dim2;
01709     T const* const d0=data;
01710
01711     WB_NEW(data,s);
01712
01713     MEM_COPY<T>(data,s,d0);
01714     isref=0;
01715
01716     return *this;
01717 };
01718
01719 template <class T> inline
01720 wbMatrix<T>& wbMatrix<T>::transpose(wbMatrix<T> &M) const {
01721
01722     if (this==&M) return M.Transpose();
01723
01724     size_t i,j,k=0; M.init(dim2,dim1);
01725
01726     if (isdiag) {
01727         M.isdiag=1; k=MIN(dim1,dim2);
01728         for (i=0; i<k; ++i) M.data[i*dim1+i]=data[i*dim2+i];

```

```

01729         return M;
01730     }
01731
01732     for (j=0; j<dim2; ++j)
01733     for (i=0; i<dim1; ++i) M.data[k++]=data[i*dim2+j];
01734
01735     return M;
01736 };
01737
01738 template <class T> inline
01739 wbMatrix<T>& wbMatrix<T>::Reshape(size_t nr, size_t nc) {
01740
01741     if (dim1!=nr || dim2!=nc) {
01742         if (dim1*dim2!=nr*nc) wblog(FL,
01743             "ERR %s() size not preserved (%dx%d => %dx%d)",
01744             FCT,dim1,dim2,nr,nc
01745         );
01746         dim1=nr; dim2=nc;
01747     }
01748     return *this;
01749 };
01750
01751 template <class T> inline
01752 wbMatrix<T>& wbMatrix<T>::add(
01753     const wbMatrix<T> &M,
01754     const T& c,
01755     const char zflag
01756 ){
01757     size_t i,s=dim1*dim2;
01758
01759     if (data==NULL && zflag) {
01760         RENEW(M.dim1, M.dim2, M.data); (*this)*=c;
01761         isdiag=M.isdiag;
01762         return *this;
01763     }
01764
01765     if (dim1!=M.dim1 || dim2!=M.dim2)
01766         wbdie(FL, "Dimension mismatch.");
01767
01768     if (isdiag) if (!M.isdiag) isdiag=0;
01769
01770     if (c==T(+1)) { for (i=0; i<s; ++i) data[i]+=M.data[i]; } else
01771     if (c==T( 0)) { return *this; } else
01772     if (c==T(-1)) { for (i=0; i<s; ++i) data[i]-=M.data[i]; }
01773     else { for (i=0; i<s; ++i) data[i]+=(c * M.data[i]); }
01774
01775     return *this;
01776 }
01777
01778 template <class T> inline
01779 wbMatrix<T>& wbMatrix<T>::minus(
01780     const wbMatrix<T> &M,
01781     const T& c,
01782     const char zflag
01783 ){
01784     size_t i,s=dim1*dim2;
01785
01786     if (data==NULL && zflag) {
01787         RENEW(M.dim1, M.dim2, M.data); (*this)*=(-c);
01788         isdiag=M.isdiag;
01789         return *this;
01790     }
01791
01792     if (dim1!=M.dim1 || dim2!=M.dim2)
01793         wbdie(FL, "Dimension mismatch.");
01794
01795     if (isdiag) if (!M.isdiag) isdiag=0;
01796
01797     if (c==T(+1)) { for (i=0; i<s; ++i) data[i]-=M.data[i]; } else
01798     if (c==T( 0)) { return *this; } else
01799     if (c==T(-1)) { for (i=0; i<s; ++i) data[i]+=M.data[i]; }
01800     else { for (i=0; i<s; ++i) data[i]-=(c * M.data[i]); }
01801
01802     return *this;
01803 }
01804
01805 template <class T> inline
01806 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
01807     const wbMatrix<T> &M2
01808 ){
01809     const wbMatrix<T>* M0[] = {this, &M2};
01810     wbvector< wbMatrix<T> const* > M; M.init(2,M0);
01811     return CAT(dim,M);
01812 };
01813
01814 template <class T> inline
01815 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,

```

```

01816     const wbMatrix<T> &M2, const wbMatrix<T> &M3
01817 ){
01818     const wbMatrix<T>* M0[] = {this, &M2, &M3};
01819     wbvector< wbMatrix<T> const* > M; M.init(3,M0);
01820     return CAT(dim,M);
01821 };
01822
01823 template <class T> inline
01824 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
01825     const wbMatrix<T> &M2, const wbMatrix<T> &M3, const wbMatrix<T> &M4
01826 ){
01827     const wbMatrix<T>* M0[] = {this, &M2, &M3, &M4};
01828     wbvector< wbMatrix<T> const* > M; M.init(4,M0);
01829     return CAT(dim,M);
01830 };
01831
01832 template <class T> inline
01833 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
01834     const wbMatrix<T> &M1, const wbMatrix<T> &M2
01835 ){
01836     wbMatrix<T> const* M0[] = {&M1, &M2};
01837     wbvector< wbMatrix<T> const* > M; M.init(2,M0);
01838     return CAT(dim,M);
01839 };
01840
01841 template <class T> inline
01842 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
01843     const wbMatrix<T> &M1, const wbMatrix<T> &M2,
01844     const wbMatrix<T> &M3
01845 ){
01846     wbMatrix<T> const* M0[] = {&M1, &M2, &M3};
01847     wbvector< wbMatrix<T> const* > M; M.init(3,M0);
01848     return CAT(dim,M);
01849 };
01850
01851 template <class T> inline
01852 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
01853     const wbMatrix<T> &M1, const wbMatrix<T> &M2,
01854     const wbMatrix<T> &M3, const wbMatrix<T> &M4
01855 ){
01856     wbMatrix<T> const* M0[] = {&M1, &M2, &M3, &M4};
01857     wbvector< wbMatrix<T> const* > M; M.init(4,M0);
01858     return CAT(dim,M);
01859 };
01860
01861 template <class T> inline
01862 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01863     wbMatrix<T> const* M[], const unsigned len
01864 ){
01865     wbvector< wbMatrix<T> const* > MM;
01866     MM.init(len,M);
01867
01868     return CAT(dim,MM);
01869 };
01870
01871 template <class T> inline
01872 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01873     const wbvector< wbMatrix<T> > &M0
01874 ){
01875     wbvector< wbMatrix<T> const* > M(M0.len);
01876     for (unsigned i=0; i<M0.len; ++i) M[i]=&M0[i];
01877
01878     return CAT(dim,M);
01879 };
01880
01881 template <class T> inline
01882 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01883     const wbvector< wbvector<T> > &V0
01884 ){
01885     wbvector< wbMatrix<T> > M0(V0.len);
01886     wbvector< wbMatrix<T> const* > M(V0.len);
01887
01888     for (size_t i=0; i<V0.len; ++i) {
01889         if (dim==1)
01890             M0[i].init2ref(1,V0[i].len,V0[i].data);
01891         else M0[i].init2ref(V0[i].len,1,V0[i].data);
01892
01893         M[i]=&M0[i];
01894     }
01895
01896     return CAT(dim,M);
01897 };
01898
01899 template <class T> inline
01900 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01901     const wbvector< wbvector<T> const* > &V0
01902 ){

```



```

01903     wbvector< wbMatrix<T> > M0(V0.len);
01904     wbvector< wbMatrix<T> const* > M(V0.len);
01905
01906     for (size_t i=0; i<V0.len; ++i) {
01907         if (dim==1)
01908             M0[i].init2ref(1,V0[i]->len,V0[i]->data);
01909         else M0[i].init2ref(V0[i]->len,1,V0[i]->data);
01910
01911         M[i]=&M0[i];
01912     }
01913
01914     return CAT(dim,M);
01915 };
01916
01917 template <class T>
01918 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01919     wbvector< wbMatrix<T> const* > M
01920 ){
01921     wbMatrix<T> X;
01922     size_t i,k;
01923
01924     if (isdiag) { wblog(FL,"WRN unsetting isdiag"); isdiag=0; }
01925
01926     if (M.len==0) { this->init(); return *this; }
01927
01928     for (k=i=0; i<M.len; ++i) {
01929         if (M[i]==this) {
01930             if (!(k++)) { this->save2(X); }
01931             M[i]=X;
01932         }
01933     }
01934
01935     for (k=i=0; i<M.len; ++i) {
01936         if (M[i] && (M[i]->dim1 || M[i]->dim2)) {
01937             if (k<i) { M[k]=M[i]; }; ++k;
01938             if (M[i]->isdiag) wblog(FL,
01939                 "ERR %s() got diagonal matrix (i=%d)",FCT, i+1
01940             );
01941         }
01942     }
01943     if (k<M.len) {
01944         if (k) M.len=k;
01945         else { this->init(); return *this; }
01946     }
01947
01948     if (dim==1) {
01949         size_t m,d1,D1,d2=0;
01950
01951         for (D1=k=0; k<M.len; ++k) { D1+=(M[k]->dim1);
01952             if (k==0) d2=(M[k]->dim2); else
01953                 if (d2!=(M[k]->dim2)) wblog(FL,
01954                     "ERR dimension mismatch (%d/%d)",d2,M[k]->dim2
01955                 );
01956         }
01957
01958         this->init(D1,d2);
01959
01960         for (m=k=0; k<M.len; ++k) { d1=(M[k]->dim1);
01961             MEM_COPY<T>(data+d2*m, d1*d2, M[k]->data);
01962             m+=d1;
01963         }
01964     }
01965     else if (dim==2) {
01966         size_t i,m,d1=0,d2,D2;
01967         T *p, *p2;
01968
01969         for (D2=k=0; k<M.len; ++k) { D2+=(M[k]->dim2);
01970             if (k==0) d1=(M[k]->dim1); else
01971                 if (d1!=(M[k]->dim1)) wblog(FL,
01972                     "ERR dimension mismatch (%d/%d)",d1,M[k]->dim1
01973                 );
01974         }
01975
01976         this->init(d1,D2);
01977
01978         for (m=k=0; k<M.len; ++k) { d2=(M[k]->dim2);
01979             p=data+m; p2=(M[k]->data);
01980
01981             for (i=0; i<d1; ++i)
01982                 MEM_COPY<T>(p+D2*i, d2, p2+d2*i);
01983
01984             m+=d2;
01985         }
01986     }
01987     else wblog(FL,"ERR invalid cat dim=%d",dim);
01988
01989     return *this;

```

```

01990 };
01991
01992 template <class T>
01993 wbMatrix<T>& wbMatrix<T>::CAT(
01994     const unsigned dim,
01995     wbvector< wbMatrix<T> const* > M,
01996     wbvector< WBINDEX const* > I
01997 ){
01998     wbMatrix<T> X;
01999     size_t i,k;
02000
02001     if (M.len!=I.len) wblog(FL,
02002         "ERR %s() size mismatch %d/%d",FCT,M.len,I.len);
02003     if (!M.len) { this->init(); return *this; }
02004
02005     for (k=i=0; i<M.len; ++i) {
02006         if (M[i]==this) {
02007             if (!(k++)) { this->save2(X); }
02008             M[i]=&X;
02009         }
02010     }
02011
02012     for (k=i=0; i<M.len; ++i) {
02013         if (M[i] && (M[i]->dim1 || M[i]->dim2 || I[i]->len)) {
02014             if (k<i) { M[k]=M[i]; I[k]=I[i]; }; ++k;
02015             if (M[i]->isdiag) wblog(FL,
02016                 "ERR %s() got diagonal matrix (i=%d)",FCT, i+1
02017             );
02018         }
02019     }
02020     if (k<M.len) {
02021         if (k) { M.len=k; I.len=k; }
02022         else { this->init(); return *this; }
02023     }
02024
02025     if (isdiag) isdiag=0;
02026
02027     if (dim==1) {
02028         size_t d1,d2=0, D1=0;
02029
02030         for (k=0; k<M.len; ++k) { D1+=(I[k]->len);
02031             if (k==0) d2=(M[k]->dim2); else
02032                 if (d2!=(M[k]->dim2)) wblog(FL,
02033                     "ERR dimension mismatch (vcat: dim2=%d/%d)",d2,M[k]->dim2
02034                 );
02035         }
02036
02037         this->init(D1,d2);
02038         T *x=data;
02039
02040         for (k=0; k<M.len; ++k) {
02041             const T* x0=M[k]->data;
02042             const widx_t *Ik=I[k]->data; D1=M[k]->dim1; d1=I[k]->len;
02043
02044             for (i=0; i<d1; ++i, x+=d2) {
02045                 if (Ik[i]>=D1) wblog(FL,
02046                     "ERR %s() index out of bounds (%d/%d)",FCT,Ik[i]+1,D1);
02047                 MEM_COPY<T>(x, d2, x0+Ik[i]*d2);
02048             }
02049         }
02050     }
02051     else if (dim==2) {
02052         size_t i,d1=0,d2,D1,D2=0;
02053
02054         for (k=0; k<M.len; ++k) { D2+=(d2=M[k]->dim2);
02055             if (k==0) d1=(I[k]->len); else
02056                 if (d1!=(I[k]->len)) wblog(FL,
02057                     "ERR dimension mismatch (%d/%d)",d1,I[k]->len
02058                 );
02059         }
02060
02061         this->init(d1,D2);
02062         T *x=data;
02063
02064         for (k=0; k<M.len; ++k) {
02065             const T* x0=M[k]->data;
02066             const widx_t *Ik=I[k]->data; D1=M[k]->dim1; d2=(M[k]->dim2);
02067
02068             for (i=0; i<d1; ++i) {
02069                 if (Ik[i]>=D1) wblog(FL,
02070                     "ERR %s() index out of bounds (%d/%d)",FCT,Ik[i]+1,D1);
02071                 MEM_COPY<T>(x+i*D2, d2, x0+Ik[i]*d2);
02072             }; x+=d2;
02073         }
02074     }
02075     else wblog(FL,"ERR invalid cat dim=%d",dim);
02076

```

```

02077     return *this;
02078 };
02079
02080 template <class T> inline
02081 void wbMatrix<T>::split (
02082     const unsigned dim,
02083     wbvector< wbMatrix<T>* > &M) const {
02084
02085     split(dim,M.data,M.len);
02086     return;
02087 }
02088
02089 template <class T> inline
02090 void wbMatrix<T>::split (
02091     const unsigned dim,
02092     wbMatrix<T> &M1,
02093     wbMatrix<T> &M2,
02094     wbMatrix<T> &M3
02095 ) const {
02096
02097     wbMatrix<T>* M[] = {&M1,&M2,&M3};
02098     split(dim,M,3);
02099     return;
02100 }
02101
02102 template <class T>
02103 void wbMatrix<T>::split (
02104     const unsigned dim, wbMatrix<T>** M, const unsigned len
02105 )const {
02106
02107     unsigned k;
02108
02109     for (k=0; k<len; ++k) {
02110         if (M[k]==this)
02111             wbdie(FL, "This matrix itself is included in split list!");
02112     }
02113
02114     if (dim==1) {
02115         size_t m,d1,D1;
02116
02117         for (D1=k=0; k<len; ++k) D1+=(*M[k]).dim1;
02118
02119         if (D1!=dim1)
02120             wbdie(FL, "Dimension mismatch.");
02121
02122         for (k=0; k<len; ++k) {
02123             if (M[k]->dim2!=dim2) M[k]->init(M[k]->dim1, dim2);
02124         }
02125
02126         for (m=k=0; k<len; ++k) { d1=M[k]->dim1;
02127             MEM_COPY<T>(M[k]->data, d1*dim2, data+dim2*m);
02128             m+=d1;
02129         }
02130     }
02131     else if (dim==2) {
02132         size_t i,m,d2,D2;
02133         T *p, *p2;
02134
02135         for (D2=k=0; k<len; ++k) D2+=M[k]->dim2;
02136
02137         if (D2!=dim2)
02138             wblog(FL,"ERR dimension mismatch (%d/%d)",D2,dim2);
02139
02140         for (k=0; k<len; ++k) {
02141             if (M[k]->dim1!=dim1) M[k]->init(dim1,M[k]->dim2);
02142         }
02143
02144         for (m=k=0; k<len; ++k) {
02145             d2=M[k]->dim2; p=data+m; p2=M[k]->data;
02146
02147             for (i=0; i<dim1; ++i)
02148                 MEM_COPY<T>(p2+d2*i, d2, p+D2*i);
02149
02150             m+=d2;
02151         }
02152     }
02153     else wblog(FL,"ERR invalid cat index (%d).", dim);
02154 };
02155
02156 template <class T> inline
02157 int wbMatrix<T>::recLess(size_t i1, size_t i2, char lex) const {
02158
02159     size_t i;
02160     T *r1=data+i1*dim2, *r2=data+i2*dim2;
02161
02162     if (i1>=dim1 || i2>=dim1) wblog(FL,
02163         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);

```

```

02164
02165     if (lex>0) {
02166         for (i=0; i<dim2; ++i)
02167             if (r1[i]<r2[i]) return 1; else
02168             if (r1[i]>r2[i]) return 0;
02169     }
02170     else {
02171         for (i=dim2-1; i<dim2; --i)
02172             if (r1[i]<r2[i]) return 1; else
02173             if (r1[i]>r2[i]) return 0;
02174     }
02175
02176     return 0;
02177 }
02178
02179 template <class T> inline
02180 int wbMatrix<T>::recLessE(size_t i1, size_t i2, char lex) const {
02181
02182     size_t i;
02183     T *r1=data+i1*dim2, *r2=data+i2*dim2;
02184
02185     if (i1>=dim1 || i2>=dim1) wblog(FL,
02186         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
02187
02188     if (lex) {
02189         for (i=0; i<dim2; ++i)
02190             if (r1[i]<r2[i]) return 1; else
02191             if (r1[i]>r2[i]) return 0;
02192     }
02193     else {
02194         for (i=dim2-1; i<dim2; --i)
02195             if (r1[i]<r2[i]) return 1; else
02196             if (r1[i]>r2[i]) return 0;
02197     }
02198
02199     return 1;
02200 }
02201
02202 template <class T> inline
02203 int wbMatrix<T>::recEqual(size_t i1, const T *d) const {
02204     if (i1>=dim1) wblog(FL,
02205         "ERR %s() index out of bounds (%d/%d)",FCT,i1,dim1);
02206     if (data==d) return 1;
02207     else return (!memcmp(data+i1*dim2, d, dim2*sizeof(T)));
02208 };
02209
02210 template <class T> inline
02211 int wbMatrix<T>::recEqual(size_t i1, size_t i2) const {
02212     if (i1>=dim1 || i2>=dim1) wblog(FL,
02213         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
02214     if (i1==i2) return 1;
02215     else return (!memcmp(data+i1*dim2, data+i2*dim2, dim2*sizeof(T)));
02216 };
02217
02218 template <class T> inline
02219 T wbMatrix<T>::recDiff2(size_t i1, size_t i2, size_t n) const {
02220     if (i1>=dim1 || i2>=dim1) wblog(FL,
02221         "ERR %s() index out of bounds (%d,%d/%d)",i1,i2,dim1);
02222     if (i1==i2) return 0;
02223     return Wb::rangeNormDiff2(
02224         data+i1*dim2, data+i2*dim2, int(n)<0 ? dim2 : n
02225     );
02226 };
02227
02228 template <class T> inline
02229 T wbMatrix<T>::recDiff2P(size_t i1, const T* b, size_t n) const {
02230     if (i1>=dim1) wblog(FL,
02231         "ERR %s() index out of bounds (%d/%d)",i1,dim1);
02232     return Wb::rangeNormDiff2(
02233         data+i1*dim2, b, int(n)<0 ? dim2 : n
02234     );
02235 };
02236
02237 template <class T> inline
02238 char wbMatrix<T>::recCompare(
02239     size_t i1, size_t i2,
02240     size_t m,
02241     char lex,
02242     T eps
02243 ) const {
02244
02245     if (i1>=dim1 || i2>=dim1) wblog(FL,
02246         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
02247     if (!dim2) wblog(FL,
02248         "ERR %s() got nothing to compare (dim2=%d)",FCT,dim2);
02249
02250     if (int(m)<0) m=dim2; else

```

```

02251     if (!m || m>dim2) wblog(FL,"ERR size out of bounds (%d/%d)",m,dim2);
02252
02253     i1*=dim2; i2*=dim2;
02254     if (lex<=0) {
02255         i1+=(dim2-m);
02256         i2+=(dim2-m);
02257     }
02258
02259     if (eps<=T(0))
02260         return Wb::recCompare(data+i1, data+i2, m, lex);
02261     else return Wb::recCompare(data+i1, data+i2, m, lex, eps);
02262 };
02263
02264 template <class T> inline
02265 char wbMatrix<T>::recCompare(
02266     size_t i1, const wbvector<T> &v, char lex
02267 ) const {
02268
02269     if (v.len!=dim2) wblog(FL,
02270         "ERR %s() size mismatch (%d,%d)",FCT,v.len,dim2);
02271     return Wb::recCompare(data+i1*dim2, v.data, dim2, lex);
02272 };
02273
02274 template <class T> inline
02275 char wbMatrix<T>::recCompareP(
02276     size_t i1, const T* v,
02277     size_t m,
02278     char lex,
02279     T eps
02280 ) const {
02281
02282     if (i1>=dim1) wblog(FL,
02283         "ERR %s() index out of bounds (%d/%d)",FCT,i1,dim1);
02284     if (!dim2) wblog(FL,
02285         "ERR %s() got nothing to compare (dim2=%d)",FCT,dim2);
02286
02287     if (int(m)<0) m=dim2; else
02288     if (!m || m>dim2) wblog(FL,"ERR size out of bounds (%d/%d)",m,dim2);
02289
02290     i1*=dim2;
02291     if (lex<=0) i1+=(dim2-m);
02292
02293     return Wb::recCompare(data+i1, v, m, lex, eps);
02294 };
02295
02296 template <class T> inline
02297 wbvector<T>& wbMatrix<T>::recProd(wbvector<T> &pp) const {
02298     size_t i=0, j; T *d=data, p;
02299
02300     pp.init(dim1);
02301
02302     if (!dim2) wblog(FL,"ERR %s() records of length 0",FCT);
02303
02304     for (; i<dim1; ++i, d+=dim2) {
02305         for (p=d[0], j=1; j<dim2; ++j) { p*=d[j]; } pp[i]=p; }
02306     return pp;
02307 };
02308
02309 template <class T> inline
02310 T wbMatrix<T>::recProd(size_t i) const {
02311
02312     if (i>=dim1 || dim2==0) { wblog(FL,
02313         "ERR %s() got empty record length (%dx%d; i)",FCT,dim1,dim2,i);
02314         return 0;
02315     }
02316
02317     const T *d=data+i*dim2; T p=d[0];
02318     for (size_t j=1; j<dim2; ++j) { p*=d[j]; }
02319     return p;
02320 };
02321
02322 template <class T>
02323 inline T wbMatrix<T>::colSum(size_t c) const {
02324     T *d=data+c, x=0;
02325
02326     if (c>=dim2) wblog(FL,"ERR index out of bounds (%d/%d)", c, dim2);
02327     if (dim1==0) wblog(FL,
02328         "WRN Sum over column of length zero (%d/%d).", c, dim2);
02329
02330     for (size_t i=0; i<dim1; ++i, d+=dim2) x+=(*d);
02331     return x;
02332 }
02333
02334 template <class T> inline
02335 T wbMatrix<T>::recSum(size_t i) const {
02336
02337

```

```

02338     if (i>=dim1) wblog(FL,
02339         "ERR %s() index out of bounds (%d,%dx%d)",FCT,i+1,dim1,dim2);
02340     if (!dim2) wblog(FL,
02341         "ERR %s() got empty object (%d,%dx%d)",i,dim1,dim2);
02342
02343     return Wb::addRange(rec(i),dim2);
02344 };
02345
02346 template <class T> inline
02347 wbvector<T>& wbMatrix<T>::recSum(wbvector<T> &s) const {
02348
02349     s.init(dim1);
02350     if (!dim2) { if (dim1) wblog(FL,
02351         "WRN %s() got empty object (%dx%d)",FCT,dim1,dim2);
02352         return s;
02353     }
02354
02355     size_t i=0; const T* d=data;
02356     for (; i<dim1; ++i, d+=dim2) { s[i]=Wb::addRange(d,dim2); }
02357
02358     return s;
02359 };
02360
02361 template <class T> inline
02362 wbvector<T>& wbMatrix<T>::recSumA(wbvector<T> &s) const {
02363
02364     s.init(dim1);
02365     if (!dim2) { if (dim1) wblog(FL,
02366         "WRN %s() got empty object (%dx%d)",FCT,dim1,dim2);
02367         return s;
02368     }
02369
02370     size_t i=0; const T* d=data; double q;
02371     for (; i<dim1; ++i, d+=dim2) {
02372         q=Wb::addRange2(d,dim2);
02373         s[i]=std::sqrt(q);
02374     }
02375
02376     return s;
02377 };
02378
02379 template <class T>
02380 wbMatrix<T>& wbMatrix<T>::setRand(double fac, double shift) {
02381     size_t i,s=dim1*dim2;
02382     static char first_call=1;
02383
02384     if (first_call) { wb_srand(); first_call=0; }
02385     fac/=(double)RAND_MAX; isdiag=0;
02386
02387     if (shift==0.)
02388         for (i=0; i<s; ++i) data[i]=(T)(fac*rand());
02389     else for (i=0; i<s; ++i) data[i]=(T)(fac*rand()+shift);
02390
02391     return *this;
02392 }
02393
02394 template<>
02395 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::setRand(double fac, double shift) {
02396     size_t i,s=dim1*dim2;
02397     static char first_call=1;
02398
02399     if (first_call) { wb_srand(); first_call=0; }
02400     fac/=(double)RAND_MAX; isdiag=0;
02401
02402     if (shift==0.) {
02403         for (i=0; i<s; ++i)
02404             data[i].set(fac*rand(), fac*rand());
02405     }
02406     else {
02407         for (i=0; i<s; ++i)
02408             data[i].set(fac*rand()+shift, fac*rand()+shift);
02409     }
02410
02411     return *this;
02412 }
02413
02414 template<>
02415 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::Symmetrize() {
02416     size_t i,j,r,s;
02417     if (dim1!=dim2) wblog(FL,
02418         "ERR Symmetrize() called with %dx%d matrix !?",dim1,dim2);
02419
02420     for (j=0; j<dim2; ++j)
02421         for (i=j+1; i<dim1; ++i) { r=i*dim2+j; s=j*dim2+i;
02422             data[r]=data[s]=wbcomplex(
02423                 0.5*(data[r].r+data[s].r),
02424                 0.5*(data[r].i-data[s].i)

```

```

02425         );
02426         data[s].Conj();
02427     }
02428
02429     return *this;
02430 };
02431
02432 template <class T>
02433 inline void wbMatrix<T>::setDiagRand(double fac, double shift) {
02434     size_t i, s=MIN(dim1,dim2);
02435     static char first_call=1;
02436
02437     if (data==NULL) return;
02438
02439     if (first_call) { wb_srand(); first_call=0; }
02440     fac/=(double)RAND_MAX;
02441
02442     MEM_SET<T>(data,dim1*dim2);
02443
02444     if (shift==0.)
02445         for (i=0; i<s; ++i) data[i*dim2+i]=(T)(fac*rand());
02446     else for (i=0; i<s; ++i) data[i*dim2+i]=(T)(fac*rand()+shift);
02447
02448     isdiag=1;
02449 }
02450
02451 template <class T>
02452 mxArray* wbMatrix<T>::toMx_base_d(char raw) const {
02453     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02454     return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02455 };
02456
02457 template <class T>
02458 mxArray* wbMatrix<T>::toMx_base(char raw) const {
02459     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02460     return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02461 };
02462
02463 template <>
02464 mxArray* wbMatrix<wbcomplex>::toMx_base(char raw) const {
02465     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02466     return Mx::Array<wbcomplex>(d1,d2,!raw).copyFromTR(data);
02467 };
02468
02469 template <>
02470 mxArray* wbMatrix<char>::toMx_base(char raw) const {
02471     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02472     return Mx::Array<mxChar>(d1,d2,!raw).copyFromTR(data);
02473 };
02474
02475 template <>
02476 mxArray* wbMatrix<long>::toMx_base(char raw) const {
02477     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02478     if (double(aMax())<1E15)
02479         return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02480     else return Mx::Array<int64_T>(d1,d2,!raw).copyFromTR(data);
02481 };
02482
02483 template <>
02484 mxArray* wbMatrix<unsigned long>::toMx_base(char raw) const {
02485     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02486     if (double(aMax())<1E15)
02487         return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02488     else return Mx::Array<uint64_T>(d1,d2,!raw).copyFromTR(data);
02489 };
02490
02491 template <class T> inline
02492 mxArray* wbMatrix<T>::toMx(char raw) const {
02493     if (raw) wblog(FL,
02494         "WRN raw flag will be ignored for typeid=%s",TSTR(T));
02495     return toMx_Struct();
02496 };
02497
02498 template <class T> inline
02499 mxArray* wbMatrix<T>::toMxP(char flag) const {
02500
02501     if (!flag || flag=='s' || flag=='S') { return toMxP_S(); }
02502     if (flag=='c' || flag=='C') { return toMxP_C(); }
02503
02504     wblog(FL,"ERR %s() invalid flag (%s)",FCT,cSTR(flag));
02505     return 0;
02506 };
02507
02508 template <> inline
02509 mxArray* wbMatrix<unsigned>::toMx(char raw) const {
02510     return toMx_base(raw);
02511 };

```

```

02512 template <> inline
02513 mxArray* wbMatrix<unsigned>::toMxT(char raw) const {
02514     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02515     return Mx::Array<uint32_T>(d1,d2,!raw).copyFromTR(data);
02516 };
02517
02518 template <> inline
02519 mxArray* wbMatrix<int>::toMx(char raw) const {
02520     return toMx_base(raw);
02521 };
02522 template <> inline
02523 mxArray* wbMatrix<int>::toMxT(char raw) const {
02524     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02525     return Mx::Array<int32_T>(d1,d2,!raw).copyFromTR(data);
02526 };
02527
02528 template <> inline
02529 mxArray* wbMatrix<unsigned long>::toMx(char raw) const {
02530     return toMx_base(raw);
02531 };
02532 template <> inline
02533 mxArray* wbMatrix<unsigned long>::toMxT(char raw) const {
02534     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02535     return Mx::Array<uint64_T>(d1,d2,!raw).copyFromTR(data);
02536 };
02537
02538 template <> inline
02539 mxArray* wbMatrix<long>::toMx(char raw) const {
02540     return toMx_base(raw);
02541 };
02542 template <> inline
02543 mxArray* wbMatrix<long>::toMxT(char raw) const {
02544     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02545     return Mx::Array<int64_T>(d1,d2,!raw).copyFromTR(data);
02546 };
02547
02548 template <> inline
02549 mxArray* wbMatrix<unsigned char>::toMx(char raw) const {
02550     return toMx_base(raw);
02551 };
02552 template <> inline
02553 mxArray* wbMatrix<unsigned char>::toMxT(char raw) const {
02554     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02555     return Mx::Array<uint8_T>(d1,d2,!raw).copyFromTR(data);
02556 };
02557
02558 template <> inline
02559 mxArray* wbMatrix<char>::toMx(char raw) const {
02560     return toMx_base(raw);
02561 };
02562 template <> inline
02563 mxArray* wbMatrix<char>::toMxT(char raw) const {
02564     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02565     return Mx::Array<int8_T>(d1,d2,!raw).copyFromTR(data);
02566 };
02567
02568 template <> inline
02569 mxArray* wbMatrix<double>::toMx(char raw) const {
02570     return toMx_base(raw);
02571 };
02572
02573 template <> inline
02574 mxArray* wbMatrix<wbcomplex>::toMx(char raw) const {
02575     return toMx_base(raw);
02576 };
02577
02578 template<class T>
02579 mxArray* wbMatrix<T>::toMx_Struct() const {
02580
02581     mxArray *S=data->mxCreatStruct(dim1,dim2);
02582
02583     size_t i,j;
02584     for (i=0; i<dim1; ++i)
02585         for (j=0; j<dim2; ++j) data[j+i*dim2].add2MxStruct(S,i+j*dim1);
02586
02587     return S;
02588 };
02589
02590 template<class T>
02591 mxArray* wbMatrix<T>::toMxP_S() const {
02592
02593     mxArray *S;
02594     if (data) { size_t i,j;
02595         S=(*data)->mxCreatStruct(dim1,dim2);
02596
02597         for (i=0; i<dim1; ++i)
02598             for (j=0; j<dim2; ++j) data[j+i*dim2]->add2MxStruct(S,i+j*dim1);

```



```

02599     }
02600     else { S=mxCreateStructMatrix(dim1,dim2,0,NULL); }
02601
02602     return S;
02603 };
02604
02605 template<class T>
02606 mxArray* wbMatrix<T>::toMxP_C() const {
02607
02608     mxArray *S;
02609
02610     if (data) { size_t i,j;
02611         S=mxCreateCell(dim1,dim2);
02612
02613         for (i=0; i<dim1; ++i)
02614             for (j=0; j<dim2; ++j) data[j+i*dim2]->add2MxCell(S,i+j*dim1);
02615     }
02616     else { S=mxCreateCellMatrix(dim1,dim2); }
02617
02618     return S;
02619 };
02620
02621 template<class T>
02622 mxArray* wbMatrix<T>::mxCreateStruct(unsigned m, unsigned n) const {
02623     return mxCreateCellMatrix(m,n);
02624 }
02625
02626 template<class T>
02627 void wbMatrix<T>::add2MxStruct(mxArray *S, unsigned i, char tst) const {
02628
02629     if (tst) {
02630         size_t s=0;
02631         if (!S || (s=mxGetNumberOfElements(S))<1 || i>=s) wblog(FL,
02632             "ERR %s() must follow mxCreateCell()\n%lx, %d/%d",
02633             FCT,S,i+1,s);
02634     }
02635
02636     mxSetCell(S,i,toMx());
02637 }
02638
02639 template <class T>
02640 void wbMatrix<T>::info(const char *istr) const {
02641
02642     size_t l=0, n=16; char s[n];
02643     snprintf(s,n,"%lix%li",dim1,dim2);
02644     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
02645     fprintf(stdout," %-12s %-10s @ 0x%p double array\n", istr, s, data);
02646 };
02647
02648 template <class T>
02649 void wbMatrix<T>::print(const char *istr, char mflag) const {
02650
02651     mxArray *a=toMx_base(0);
02652
02653     if (istr && istr[0]) {
02654         if (mflag) { wb_printf("\n%s = [\n",istr); }
02655         else {
02656             wb_printf("\n%s = [%dx%d double]\n\n",istr,dim1,dim2);
02657         }
02658     } else { wb_printf("\n"); }
02659
02660     Wb::CallMatlab(0,NULL,1,&a,"disp");
02661     wb_printf("%s\n", mflag && istr && istr[0] ? "];":""");
02662
02663     mxDestroyArray(a);
02664 };
02665
02666 template <class T>
02667 void wbMatrix<T>::Print(const char *istr, char mflag) const {
02668
02669     size_t i,j;
02670
02671     if (!mflag) {
02672         if (istr[0])
02673             printf("\n%s = [%ldx%ld double]\n\n", istr, dim1, dim2);
02674         else printf("\n");
02675     }
02676     else
02677         printf("\n%s = [\n", istr[0] ? istr : "ans");
02678
02679     for (i=0; i<dim1; ++i) {
02680         for (j=0; j<dim2; ++j) printf(" %8.3g", (double)data[i*dim2+j]);
02681         printf("\n");
02682     }
02683
02684     if (mflag) printf("];\n");
02685 }

```

```

02686
02687 template <class T>
02688 void wbMatrix<T>::printdata(
02689     const char *istr, const char *dbl_fmt, const char *rsep
02690 ) const {
02691
02692     size_t i=0,j;
02693     if (istr && istr[0]) printf("%s = [\n ",istr); else printf("[");
02694
02695     for (; i<dim1; ++i) { if (i) printf("%s ",rsep);
02696     for (j=0; j<dim2; ++j) { printf(dbl_fmt, double(data[i*dim2+j])); }
02697
02698     printf(" ]\n");
02699 }
02700
02701 template <class T>
02702 void wbMatrix<T>::appendRow(const wbvector<T> &v) {
02703
02704     if (dim1) {
02705         if (dim2!=v.len) wblog(FL,
02706             "ERR Dimension mismatch (%dx%d : %d)", dim1,dim2, v.len);
02707
02708         appendRows(1, v.data);
02709     }
02710     else RENEW(1,v.len,v.data);
02711 };
02712
02713 template <class T>
02714 void wbMatrix<T>::appendRows(size_t n, const T* v) {
02715
02716     size_t s=dim1*dim2, ds=n*dim2;
02717     T *d0=data;
02718
02719     dim1+=n; if (!n || !dim2) return;
02720
02721     WB_NEW(data,s+ds);
02722     MEM_COPY<T>(data,s+ds,s,d0,v);
02723     WB_DELETE(d0);
02724 };
02725
02726 template<class T>
02727 void wbMatrix<T>::getReal(wbMatrix<double> &R) const {
02728     if (typeid(T)!=typeid(double)) {
02729         R.init(dim1,dim2);
02730         for (size_t n=dim1*dim2, i=0; i<n; ++i) { R.data[i]=double(data[i]); }
02731     }
02732     else { R.init(*this); }
02733     return;
02734 };
02735
02736 template<class T>
02737 void wbMatrix<T>::getImag(wbMatrix<double> &I) const {
02738     wblog(FL,"ERR wbMatrix::getImag not defined for type %s.",
02739         TSTR(T));
02740 };
02741
02742 template<class T>
02743 void wbMatrix<T>::set(const wbMatrix<double> &R, const wbMatrix<double> &I) {
02744     wblog(FL,"ERR wbMatrix::set(R,I) not defined for type %s.",
02745         TSTR(T));
02746 };
02747
02748 template<class T>
02749 wbMatrix<T>& wbMatrix<T>::Conj() { return *this; };
02750
02751 template<class T>
02752 double wbMatrix<T>::normReal() const { return SQRT(norm2()); };
02753
02754 template<class T>
02755 double wbMatrix<T>::normImag() const { return 0; };
02756
02757 template<>
02758 double wbMatrix<wbcomplex>::normReal() const {
02759     double x=0;
02760     for (size_t n=dim1*dim2, i=0; i<n; ++i) x+=NORM2(data[i].r);
02761     return SQRT(x);
02762 };
02763
02764 template<>
02765 double wbMatrix<wbcomplex>::normImag() const {
02766     double x=0;
02767     for (size_t n=dim1*dim2, i=0; i<n; ++i) x+=NORM2(data[i].i);
02768     return SQRT(x);
02769 };
02770
02771 template<>
02772 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::Conj() {

```

```

02773     for (size_t s=dim1*dim2, i=0; i<s; ++i)
02774     data[i].i = -data[i].i;
02775     return *this;
02776 }
02777
02778 template<>
02779 void wbMatrix<wbcomplex>::getReal(wbMatrix<double> &R) const {
02780     R.init(dim1, dim2);
02781
02782     if (isdiag) {
02783         size_t i,k,s=MIN(dim1,dim2);
02784         for (i=0; i<s; ++i) { k=i*dim2+i; R.data[k]=data[k].r; }
02785         R.isdiag=isdiag;
02786     }
02787     else {
02788         for (size_t s=dim1*dim2, i=0; i<s; ++i)
02789             R.data[i]=data[i].r;
02790     }
02791 }
02792
02793 template<>
02794 void wbMatrix<wbcomplex>::getImag(wbMatrix<double> &I) const {
02795     I.init(dim1, dim2);
02796
02797     if (isdiag) {
02798         size_t i,k,s=MIN(dim1,dim2);
02799         for (i=0; i<s; ++i) { k=i*dim2+i; I.data[k]=data[k].i; }
02800         I.isdiag=isdiag;
02801     }
02802     else {
02803         for (size_t s=dim1*dim2, i=0; i<s; ++i)
02804             I.data[i]=data[i].i;
02805     }
02806 }
02807
02808 template<>
02809 void wbMatrix<wbcomplex>::set(
02810     const wbMatrix<double> &R,
02811     const wbMatrix<double> &I
02812 ){
02813     if (!R.sameSize(I)) wblog(FL,
02814         "ERR Dimension mismatch (%dx%d)!", R.dim1, R.dim2, I.dim1, I.dim2);
02815
02816     init(R.dim1, R.dim2);
02817
02818     for (size_t s=dim1*dim2, i=0; i<s; ++i)
02819         data[i].set(R.data[i], I.data[i]);
02820 }
02821
02822 template <class T> inline
02823 void matchIndexU(const char *F, int L,
02824     const wbMatrix<T> &QA, const wbMatrix<T> &QB,
02825     wbindex &Ib, const char force=1
02826 );
02827
02828 template<class T>
02829 int matchIndex(C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
02830     char lex=1,
02831     widx_t *ma=NULL, widx_t *mb=NULL,
02832     T eps=0
02833 );
02834
02835 template <class T> inline
02836 int matchIndex(
02837     const wbvector<T> &qA, const wbvector<T> &qB,
02838     wbindex &Ia, wbindex &Ib,
02839     char lex=1,
02840     widx_t *ma=NULL, widx_t *mb=NULL,
02841     T eps=0
02842 ){
02843     wbMatrix<T> QA, QB; int r;
02844     QA.init2ref(qA, 't');
02845     QB.init2ref(qB, 't'); if (QA.dim2!=1 || QB.dim2!=1) wblog(FL, "ERR");
02846
02847     r=matchIndex(QA, QB, Ia, Ib, lex, ma, mb, eps);
02848     return r;
02849 };
02850
02851 template <class T> inline
02852 int matchSortedIdx(
02853     const T *da, size_t lda, size_t na,
02854     const T *db, size_t ldb, size_t nb, wbindex &Ia, wbindex &Ib,
02855     widx_t m=-1,
02856     char lex=1
02857 ){
02858     widx_t m1=0, m2=0;
02859     return Wb::matchSortedIdx(da, lda, na, db, ldb, nb, Ia, Ib, m, lex, &m1, &m2);

```

```

02860 };
02861
02862 template <class T> inline
02863 int matchSortedIdx(C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
02864     widx_t m=-1,
02865     char lex=1,
02866     widx_t *ma=NULL, widx_t *mb=NULL,
02867     T eps=0
02868 ){
02869     return Wb::matchSortedIdx(
02870         QA.data, QA.dim2, QA.dim1,
02871         QB.data, QB.dim2, QB.dim1, Ia,Ib, m, lex, ma, mb, eps
02872     );
02873 };
02874
02875 template <class T> inline
02876 int matchSortedIdx(C_TMAT &QA, C_TMAT &QB,
02877     wbindex &Ia, wbindex &Ib, wbvector<widx_t> &D,
02878     widx_t m=-1,
02879     char lex=1,
02880     T eps=0
02881 ){
02882     return Wb::matchSortedIdx(
02883         QA.data, QA.dim2, QA.dim1,
02884         QB.data, QB.dim2, QB.dim1, Ia,Ib,D, m, lex, eps
02885     );
02886 };
02887
02888 template <class T> inline
02889 int matchSortedIdxU(
02890     const char *F, int L,
02891     C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
02892     widx_t m=-1, char lex=1
02893 );
02894
02895 template <class T> inline
02896 int matchSortedIdxU(
02897     C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib, widx_t m=-1
02898 ){ return matchSortedIdxU(F,QA,QB,Ia,Ib,m); };
02899
02900 #endif
02901

```

5.47 wbMatrix_blas.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_BLAS_CC__
00021 #define __WB_BLAS_CC__
00022
00023 // ----- //
00024 // ----- //
00025 // wrapper routine
00026
00027 template<class TA, class TB, class TC>
00028 void MMDIAG(
00029     const wbMatrix<TA> &A,
00030     const wbMatrix<TB> &B, wbMatrix<TC> &C,
00031     char aflag='N', char bflag='N',
00032     const TA afac=1., const TC cfac=0.
00033 );
00034
00035 template<class TA, class TB, class TC>
00036 wbMatrix<TC> & MMDIAG(
00037     const wbvector<TA> &A,
00038     const wbMatrix<TB> &B, wbMatrix<TC> &C,

```

```

00039     char aflag='N', char bflag='N',
00040     const TA afac=1., const TC cfac=0.
00041 );
00042
00043 template<class TA, class TB, class TC>
00044 wbMatrix<TC>& MMDIAG(
00045     const wbMatrix<TA> &A,
00046     const wbvector<TB> &B, wbMatrix<TC> &C,
00047     char aflag='N', char bflag='N',
00048     const TA afac=1., const TC cfac=0.
00049 );
00050
00051 template<class TA, class TB, class TC>
00052 void DZGEMM_VEC(
00053     const wbMatrix<TA> &A,
00054     const wbvector<TB> &B,
00055     wbvector<TC> &C, int m, int n,
00056     char aflag='N', const TA afac=1., const TC cfac=0.
00057 );
00058
00059 template<class TD>
00060 void DZGEMM(
00061     const wbMatrix<TD> &A,
00062     const wbMatrix<TD> &B, wbMatrix<TD> &C,
00063     const unsigned k,
00064     char aflag='N', char bflag='N',
00065     const TD afac=1., const TD cfac=0.
00066 );
00067
00068 template<>
00069 inline void DZGEMM<double>(
00070     const wbMatrix<double> &A,
00071     const wbMatrix<double> &B, wbMatrix<double> &C,
00072     const unsigned k,
00073     char aflag, char bflag,
00074     const double afac, const double cfac
00075 ){
00076
00077 #ifdef WB_CLOCK
00078     Wb::Clock clk("dgemm",1);
00079     if (aflag=='N')
00080         stat_dgemm.account(A.dim1*A.dim2*(A.dim2==B.dim1 ? B.dim2 : B.dim1));
00081     else stat_dgemm.account(A.dim1*A.dim2*(A.dim1==B.dim1 ? B.dim2 : B.dim1));
00082 #endif
00083
00084     dgemm(bflag, aflag, int(C.dim2), int(C.dim1), int(k),
00085         afac, B.data, int(B.dim2),
00086         A.data, int(A.dim2),
00087         cfac, C.data, int(C.dim2)
00088     );
00089 }
00090
00091 template<>
00092 inline void DZGEMM<wbcomplex>(
00093     const wbMatrix<wbcomplex> &A,
00094     const wbMatrix<wbcomplex> &B, wbMatrix<wbcomplex> &C,
00095     const unsigned k,
00096     char aflag, char bflag,
00097     const wbcomplex afac, const wbcomplex cfac
00098 ){
00099 #ifdef WB_CLOCK
00100     Wb::Clock clk("zgemm",1);
00101
00102     if (bflag=='N')
00103         stat_dgemm.account(A.dim1*A.dim2*(A.dim2==B.dim1 ? B.dim2 : B.dim1));
00104     else stat_dgemm.account(A.dim1*A.dim2*(A.dim1==B.dim1 ? B.dim2 : B.dim1));
00105 #endif
00106
00107     zgemm (
00108         bflag, aflag, (int)C.dim2, (int)C.dim1, (int)k,
00109         afac, B.data, (int)B.dim2, A.data, (int)A.dim2,
00110         cfac, C.data, (int)C.dim2
00111     );
00112
00113 #ifdef WB_CLOCK
00114     { Wb::Clock *clk;
00115       if (clk.stop())
00116           { Wb::Clock("zgeNX",1,0,&Wb::Clocks,&clk,0); }
00117       else { Wb::Clock("zgeNN",1,0,&Wb::Clocks,&clk,0); }
00118
00119       clk.ncall+=2;
00120       clk.tcpu+= (long unsigned)
00121           (double(A.dim1*A.dim2*B.dim2)*(4E-9*CLOCKS_PER_SEC)+1.5);
00122     }
00123 #endif
00124 }
00125

```

```

00126 namespace Wb {
00127
00128 void MatProd(
00129     const wbMatrix<double> &A,
00130     const wbMatrix<wbcomplex> &B, wbMatrix<wbcomplex> &C,
00131     char aflag='N', char bflag='N',
00132     const double afac=1., const double cfac=0.,
00133     const char i00flag=0
00134 ){
00135     wbMatrix<double> R,I, rC, iC;
00136
00137     B.getReal(R);
00138     B.getImag(I);
00139
00140     if (bflag=='C') I.flipSign();
00141
00142     if (cfac!=0.) { C.getReal(rC); C.getImag(iC); }
00143
00144     MatProd(A, R, rC, aflag, bflag, afac, cfac, i00flag);
00145     MatProd(A, I, iC, aflag, bflag, afac, cfac, i00flag);
00146
00147     C.set(rC,iC);
00148 };
00149
00150 void MatProd(
00151     const wbMatrix<wbcomplex> &A,
00152     const wbMatrix<double> &B, wbMatrix<wbcomplex> &C,
00153     char aflag='N', char bflag='N',
00154     const double afac=1., const double cfac=0.,
00155     const char i00flag=0
00156 ){
00157     wbMatrix<double> R,I, rC, iC;
00158
00159     A.getReal(R);
00160     A.getImag(I);
00161
00162     if (aflag=='C') I.flipSign();
00163
00164     if (cfac!=0.) { C.getReal(rC); C.getImag(iC); }
00165
00166     MatProd(R, B, rC, aflag, bflag, afac, cfac, i00flag);
00167     MatProd(I, B, iC, aflag, bflag, afac, cfac, i00flag);
00168
00169     C.set(rC,iC);
00170 };
00171
00172 template<class T>
00173 void MatProd(
00174     const wbMatrix<T> &A,
00175     const wbMatrix<T> &B, wbMatrix<T> &C,
00176     char aflag, char bflag,
00177     const T afac, const T cfac,
00178     const char i00flag
00179 ){
00180     unsigned a1=A.dim1, a2=A.dim2, b1=B.dim1, b2=B.dim2;
00181     const char flags[]="NTCntc";
00182
00183 #ifdef WB_CLOCK
00184     Wb::Clock clk("MatProd",1);
00185 #endif
00186
00187     if (!strchr(flags,aflag) || !strchr(flags,bflag)) wblog(FL,
00188         "ERR %s() invalid flags %c<%d>, %c<%d>",
00189         FCT,aflag,aflag,bflag,bflag);
00190
00191     if (aflag!='N') SWAP(a1,a2);
00192     if (bflag!='N') SWAP(b1,b2);
00193
00194     if (a2!=b1) {
00195         wblog(FL, "ERR %s() dimension mismatch: (%d,%d) * (%d,%d) ?",
00196             FCT,a1,a2,b1,b2); return;
00197     }
00198
00199     if (cfac!=0.) {
00200         if (C.data==NULL && i00flag) {
00201             if (cfac!=1.) wblog(FL,
00202                 "WRN C = A*B + c*[] with c=%s !?", toStr(cfac).data);
00203             C.init(a1,b2);
00204         }
00205         else if (C.dim1!=a1 || C.dim2!=b2) { wblog(FL,
00206             "ERR %s() dimension mismatch: C=(%d,%d) =? (%d,%d)",
00207             FCT,C.dim1,C.dim2,a1,b2); return;
00208         }
00209     }
00210
00211     if (a1==0 || b2==0) {
00212         if (cfac==0.) C.init(a1,b2);

```

```

00213     return;
00214 }
00215
00216 if (a2==0) { wblog(FL,
00217     "ERR %s() cannot multiply (%dx%d)*(%dx%d)!",
00218     FCT,a1,a2,b1,b2); return; }
00219
00220 if (&C==&A || &C==&B) { wblog(FL,
00221     "ERR I/O spaces must be distinct!\n[%7lX %7lX %7lX]",
00222     &A, &B, &C); return; }
00223
00224 if (cfac==0.)
00225     C.init(a1,b2);
00226
00227 if (A.isdiag || B.isdiag)
00228     MMDIAG(A,B,C, aflag,bflag,afac,cfac);
00229 else DZGEMM(A,B,C,a2,aflag,bflag,afac,cfac);
00230 };
00231
00232 template<class TA, class TB, class TC>
00233 void MatProd(
00234     const wbMatrix<TA> &A,
00235     const wbvector<TB> &B, wbvector<TC> &C,
00236     char aflag, const TC afac, const TC cfac,
00237     const char i00flag
00238 ){
00239     unsigned m=A.dim1, n=A.dim2;
00240     if (aflag!='N') SWAP(m,n);
00241
00242     if (B.len!=n) wblog(FL,
00243         "ERR severe size mismatch Ax=b (%dx%d, %d)",m,n,B.len);
00244
00245     if (!C.data) {
00246         if (cfac!=0. && i00flag) wblog(FL,
00247             "WRN C = A*B + c*[] with c=%s !?",toStr(cfac).data);
00248         C.init(m);
00249     }
00250     else if (C.len!=m) {
00251         if (cfac==0.) C.init(m); else wblog(FL,
00252             "ERR severe size mismatch Ax=b (%d, %d)", m, C.len);
00253     }
00254
00255     DZGEMM_VEC(A,B,C,m,n,aflag,afac,cfac);
00256 };
00257
00258 };
00259
00260 template<>
00261 inline void DZGEMM_VEC(
00262     const wbMatrix<double> &A,
00263     const wbvector<double> &B,
00264     wbvector<double> &C, int m, int n,
00265     char aflag, const double afac, const double cfac
00266 ){
00267     dgemm (
00268         'N', aflag, 1, m, n, afac,
00269         B.data, 1, A.data, A.dim2, cfac,
00270         C.data, 1
00271     );
00272 }
00273
00274 template<>
00275 inline void DZGEMM_VEC(
00276     const wbMatrix<wbcomplex> &A,
00277     const wbvector<wbcomplex> &B,
00278     wbvector<wbcomplex> &C, int m, int n,
00279     char aflag, const wbcomplex afac, const wbcomplex cfac
00280 ){
00281     zgemm (
00282         'N', aflag, 1, m, n, afac,
00283         B.data, 1, A.data, A.dim2, cfac,
00284         C.data, 1
00285     );
00286 }
00287
00288 template<>
00289 inline void DZGEMM_VEC(
00290     const wbMatrix<double> &A,
00291     const wbvector<wbcomplex> &B,
00292     wbvector<wbcomplex> &C, int m, int n,
00293     char aflag, const double afac, const wbcomplex cfac
00294 ){
00295     wbvector<double> b,cr,ci;
00296
00297     if (cfac && cfac!=1.) { C*=cfac; }
00298
00299     B.getReal(b); C.getReal(cr); DZGEMM_VEC(A,b,cr,m,n,aflag,afac,1.);

```

```

00300     B.getImag(b); C.getImag(ci); DZGEMM_VEC(A,b,ci,m,n,aflag,afac,1.);
00301
00302     C.set(cr,ci);
00303 };
00304
00305 inline void DZGEMM_aux(
00306     const wbMatrix<double> &A,
00307     const wbMatrix<double> &B, wbMatrix<double> &C,
00308     const unsigned k,
00309     char aflag='N', char bflag='N',
00310     const double afac=1., const double cfac=0.
00311 ){
00312     if (A.isdiag || B.isdiag)
00313         MMDIAG(A, B, C, aflag, bflag, afac, cfac);
00314     else DZGEMM(A, B, C, k, aflag, bflag, afac, cfac);
00315 }
00316
00317 template<class TA, class TB, class TC>
00318 inline void MMDIAG(
00319     const wbMatrix<TA> &A,
00320     const wbMatrix<TB> &B, wbMatrix<TC> &C,
00321     char aflag, char bflag,
00322     const TA afac, const TC cfac
00323 ){
00324     unsigned i, s=C.dim1*C.dim2, dmax;
00325
00326     if (!A.isdiag && !B.isdiag) wblog(FL,
00327         "ERR Wrong call - elements not diagonal.");
00328
00329     if (A.isdiag && A.dim1>1 && A.dim2>1)
00330         if (A(1,0)!=0. || A(0,1)!=0.) wblog(FL,
00331             "ERR Input A is NOT diag even though isdiag=%d", A.isdiag);
00332
00333     if (B.isdiag && B.dim1>1 && B.dim2>1)
00334         if (B(1,0)!=0. || B(0,1)!=0.) wblog(FL,
00335             "ERR Input B is NOT diag even though isdiag=%d", B.isdiag);
00336
00337     if (cfac!=1.) for (i=0; i<s; i++) C.data[i]*=cfac;
00338
00339     if (A.isdiag && !B.isdiag) {
00340         MMDIAG(A.getDiag(), B, C, aflag, bflag, afac);
00341     }
00342     else
00343         if (!A.isdiag) && B.isdiag {
00344             MMDIAG(A, B.getDiag(), C, aflag, bflag, afac);
00345         }
00346     else {
00347         wbvector<TA> adiag;
00348         wbvector<TB> bdiag;
00349         A.getDiag(adiag); B.getDiag(bdiag);
00350
00351         if (aflag=='C')
00352             for (i=0; i<adiag.len; i++) adiag[i]=conj(adiag[i]);
00353
00354         if (bflag=='C')
00355             for (i=0; i<bdiag.len; i++) bdiag[i]=conj(bdiag[i]);
00356
00357         dmax=MIN( MIN(A.dim1,A.dim2), MIN(B.dim1,B.dim2) );
00358         for (i=0; i<dmax; i++) C(i,i) += afac * adiag[i] * bdiag[i];
00359     }
00360
00361     if (C.isdiag) if (!A.isdiag || !B.isdiag) C.isdiag=0;
00362 }
00363
00364 template<class TA, class TB, class TC>
00365 inline wbMatrix<TC>& MMDIAG(
00366     const wbvector<TA> &A,
00367     const wbMatrix<TB> &B, wbMatrix<TC> &C,
00368     char aflag, char bflag,
00369     const TA afac, const TC cfac
00370 ){
00371     unsigned i,j, m=B.dim1, n=B.dim2;
00372     char acc,btr;
00373     TA dbl;
00374
00375     if (bflag=='C' && typeid(TB)!=typeid(wbcomplex)) bflag='T';
00376
00377     if (!strchr("NTC",aflag) || !strchr("NTC",bflag)) wblog(FL,
00378         "ERR Invalid flag(s) %c<%d>, %c<%d>",aflag,aflag,bflag,bflag);
00379
00380     acc=(aflag=='C');
00381     btr=(bflag!='N'); if (btr) SWAP(m,n);
00382
00383     if (A.len!=m) wblog(FL,
00384         "ERR Size mismatch (diag(%d) * %dx%d)", A.len,m,n);
00385     if (&C==&B) wblog(FL,"ERR Output space C equal to input space.");
00386

```



```

00387     if (C.isEmpty() || (cfac==0. && (C.dim1!=m || C.dim2!=n)))
00388         C.init(m,n);
00389     else {
00390         if (C.dim1!=m || C.dim2!=n) wblog(FL,
00391             "ERR Size mismatch (%dx%d; %dx%d)", C.dim1, C.dim2, m, n);
00392         if (cfac!=1.) {
00393             if (cfac==0.) C.reset();
00394             else C*=cfac;
00395         }
00396     }
00397
00398     for (i=0; i<C.dim1; i++) {
00399         dbl=A[i]; if (acc) dbl=conj(dbl);
00400         dbl*=afac; if (dbl==0.) continue;
00401
00402         switch (bflag) {
00403             case 'N': for (j=0; j<C.dim2; j++) C(i,j)+=(dbl*B(i,j)); break;
00404             case 'T': for (j=0; j<C.dim2; j++) C(i,j)+=(dbl*B(j,i)); break;
00405             case 'C': for (j=0; j<C.dim2; j++) C(i,j)+=(dbl*conj(B(j,i)));
00406         }
00407     }
00408
00409     return C;
00410 }
00411
00412 template<class TA, class TB, class TC>
00413 inline wbMatrix<TC>& MMDIAG(
00414     const wbMatrix<TA> &A,
00415     const wbvector<TB> &B, wbMatrix<TC> &C,
00416     char aflag, char bflag,
00417     const TA afac, const TC cfac
00418 ){
00419     unsigned i,j, m=A.dim1, n=A.dim2;
00420     char atr,bcc;
00421     TB dbl;
00422
00423     if (aflag=='C' && typeid(TA)!=typeid(wbcomplex)) aflag='T';
00424
00425     if (!strchr("NTC",aflag) || !strchr("NTC",bflag)) wblog(FL,
00426         "ERR Invalid flag(s) %c<%d>, %c<%d>", aflag,aflag,bflag,bflag);
00427
00428     atr=(aflag!='N'); if (atr) SWAP(m,n);
00429     bcc=(bflag=='C');
00430
00431     if (B.len!=n) wblog(FL,
00432         "ERR Size mismatch (%dx%d * diag(%d))", m,n, B.len);
00433     if (&C==&A) wblog(FL, "ERR Output space C equal to input space.");
00434
00435     if (C.isEmpty() || (cfac==0. && (C.dim1!=m || C.dim2!=n)))
00436         C.init(m,n);
00437     else {
00438         if (C.dim1!=m || C.dim2!=n) wblog(FL,
00439             "ERR Size mismatch (%dx%d; %dx%d)", C.dim1, C.dim2, m, n);
00440         if (cfac!=1.) {
00441             if (cfac==0.) C.reset();
00442             else C*=cfac;
00443         }
00444     }
00445
00446     for (j=0; j<C.dim2; j++) {
00447         dbl=B[j]; if (bcc) dbl=conj(dbl);
00448         dbl*=afac; if (dbl==0.) continue;
00449
00450         switch (aflag) {
00451             case 'N': for (i=0; i<C.dim1; i++) C(i,j)+=(dbl*A(i,j)); break;
00452             case 'T': for (i=0; i<C.dim1; i++) C(i,j)+=(dbl*A(j,i)); break;
00453             case 'C': for (i=0; i<C.dim1; i++) C(i,j)+=(dbl*conj(A(j,i)));
00454         }
00455     }
00456
00457     return C;
00458 }
00459
00460 inline void WbEigenSymmetric (
00461     const wbMatrix<double> &M, wbMatrix<double> &V, wbvector<double> &E
00462 ){
00463     unsigned n=M.dim1; pINT ni=0, q=0;
00464     double nd=0;
00465
00466     if (M.dim1!=M.dim2) wblog(FL,
00467         "ERR WbEigenSymmetric requires square matrix (%d,%d).",
00468         M.dim1, M.dim2);
00469
00470     V=M; E.init(n); if (n==0) return;
00471
00472     dsyevd('V','U',n,V.data,n,E.data,&nd,-1,&ni,-1,q);
00473     if (nd<1 || ni<1 || q) wblog(FL,

```

```

00474         "ERR DSYEVD returned lwork=%g/%d (e=%d) !?",nd,ni,q);
00475
00476         wbvector<double> wd(nd);
00477         wbvector<pINT> wi(ni);
00478
00479         dsyevd(
00480             'V',
00481             'L',
00482             n,
00483             V.data,
00484             n,
00485             E.data,
00486             wd.data, wd.len,
00487             wi.data, wi.len,
00488             q
00489         );
00490
00491         if (q) wblog(FL,"ERR DSYEV returned e=%d !?", q);
00492     }
00493
00494     inline void WbEigenSymmetric (
00495         const wbMatrix<wbcomplex> &M, wbMatrix<wbcomplex> &V, wbvector<double> &E
00496     ){
00497         unsigned n=M.dim1; pINT q, ni=0;
00498         wbcomplex nz=0; double nr=0;
00499
00500         if (M.dim1!=M.dim2) wblog(FL,
00501             "ERR %s() requires square matrix (%d,%d)",FCT,M.dim1,M.dim2);
00502         if (!M.isHConj(1E-12,&nr)) wblog(FL,
00503             "ERR %s() got non-hermitian matrix (%.3g)",FCT,nr);
00504         nr=0;
00505
00506         V=M; E.init(n); if (!n) return;
00507
00508         zheevd('V','U',n,V.data,n,E.data,&nz,-1,&nr,-1,&ni,-1,q);
00509         if (nz.r<1 || nr<1 || ni<1 || q) wblog(FL,
00510             "ERR ZHEEVD returned lwork=%g/%g/%d (e=%d) !?",nz.r,nr,ni,q);
00511
00512         wbvector<wbcomplex> wz(nz.r);
00513         wbvector<double> wr(nr);
00514         wbvector<pINT> wi(ni);
00515
00516         zheevd(
00517             'V',
00518             'L',
00519             n,
00520             V.data,
00521             n,
00522             E.data,
00523             wz.data, wz.len,
00524             wr.data, wr.len,
00525             wi.data, wi.len,
00526             q
00527         );
00528
00529         if (q<0) { wblog(FL,"ERR %s() ZHEEVD returned e=%d",FCT,q); }
00530         if (q>0) {
00531             V=M; E.init(n);
00532
00533             wblog(FL,"WRN %s() ZHEEVD returned i=%d / falling back to ZHEEV",FCT,q);
00534             MXPut(FL,0,"tmpfile").add(M,"M").add(q,"q");
00535
00536             if (n<128) { ni=2*n-1; }
00537             else {
00538                 ni=ilaenv(1,"zhetrd","U", (pINT)n, (pINT)n, (pINT)n, (pINT)n);
00539                 if (ni>n) wblog(FL,"WRN %s() got lwork=(%d+2)*%d",FCT,ni,n);
00540                 ni=(ni+1)*n;
00541             }
00542             if (size_t(ni)>wz.len) { wz.init(ni); }
00543
00544             nr=3*n-2; if (nr>wr.len) { wr.init(nr); }
00545
00546             zheev(
00547                 'V',
00548                 'L',
00549                 n,
00550                 V.data,
00551                 n,
00552                 E.data,
00553                 wz.data, ni,
00554                 wr.data,
00555                 q
00556             );
00557             wblog(FL,"ERR ZHEEV returned e=%d", q);
00558         }
00559
00560         V.Conj();

```

```

00561 };
00562
00563 template<class T>
00564 inline void GESVD_M(
00565     wbMatrix<T> &A,
00566     wbMatrix<T> &U,
00567     wbvector<double> &S,
00568     wbMatrix<T> &Vt
00569 ){
00570     wblog(FL,"ERR GESVD_M not defined for datatype %s",TSTR(T));
00571 };
00572
00573 template<>
00574 inline void GESVD_M(
00575     wbMatrix<double> &A,
00576     wbMatrix<double> &U,
00577     wbvector<double> &S,
00578     wbMatrix<double> &Vt
00579 ){
00580     unsigned M=A.dim1, N=A.dim2, K=S.len;
00581     wbvector<double> wd;
00582     double nd;
00583     pINT q, ni=8*MIN(M,N);
00584     wbvector<pINT> wi(ni);
00585
00586     #ifdef WB_CLOCK
00587         Wb::Clock clk("dgesvd",1);
00588     #endif
00589
00590     dgesdd('S',N,M,A.data,N,S.data,Vt.data,N,U.data,K,&nd,-1,wi.data,q);
00591     if (nd<1 || q) wblog(FL,
00592         "ERR DGESDD returned lwork=%g (e=%d) !?",nd,q);
00593     wd.init(nd);
00594
00595     dgesdd(
00596         'S', N, M, A.data, N, S.data,
00597         Vt.data, N,
00598         U.data, K,
00599         wd.data, wd.len, wi.data,
00600         q
00601     );
00602
00603     if (q) wblog(FL,"ERR DGESDD returned e=%d !?", q);
00604 };
00605
00606 template<>
00607 inline void GESVD_M(
00608     wbMatrix<wbcomplex> &A,
00609     wbMatrix<wbcomplex> &U,
00610     wbvector<double> &S,
00611     wbMatrix<wbcomplex> &Vt
00612 ){
00613     unsigned M=A.dim1, N=A.dim2, K=S.len;
00614     wbvector<wbcomplex> wz;
00615
00616     pINT q, n=MIN(M,N);
00617     wbcomplex nz;
00618
00619     wbvector<double> wd(n*(5*n+7));
00620     wbvector<pINT> wi(12*n);
00621
00622     if (!M || !N) wblog(FL,"ERR %s() got %dx%d matrix!?",FCT,M,N);
00623
00624     #ifdef WB_CLOCK
00625         Wb::Clock clk("zgesvd",1);
00626     #endif
00627
00628     zgesdd(
00629         'S',N,M,A.data,N,S.data,Vt.data,N,U.data,K,
00630         &nz, -1, wd.data, wi.data, q);
00631     if (nz.r<1 || q) wblog(FL,
00632         "ERR ZGESDD() returned lwork=%g (e=%d) !?",nz.r,q);
00633     wz.init(nz.r);
00634
00635     zgesdd(
00636         'S', N, M, A.data, N, S.data, Vt.data, N, U.data, K,
00637         wz.data, wz.len, wd.data, wi.data,
00638         q
00639     );
00640
00641     if (q) wblog(FL,"ERR ZGESDD returned e=%d !?", q);
00642 };
00643
00644 template<class T>
00645 inline void wbSVD(
00646     const wbMatrix<T> &A0,
00647     wbMatrix<T> &U,

```

```

00648     wbvector<double> &S,
00649     wbMatrix<T> &Vt
00650 ){
00651     wbMatrix<T> A(A0);
00652
00653     unsigned M=A.dim1, N=A.dim2, K=MIN(M,N);
00654
00655     U.init(M,K); S.init(K); Vt.init(K,N);
00656     if (!M || !N) return;
00657
00658     GESVD_M(A,U,S,Vt);
00659
00660 #ifdef SVD_BUG_SAFEGUARD
00661     { double a=SQRT(A0.norm2()), s=SQRT(S.norm2()), e=fabs(a-s)/a;
00662       if (e>1E-10) {
00663         wblog(FL,"WRN %s() need to fix SVD bug: %dx%d @ e=%.3g !?",FCT,M,N,e);
00664       }
00665     }
00666 #endif
00667 };
00668
00669 #endif
00670

```

5.48 wbMatrix_blas.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MATRIX_BLAS_HH__
00021 #define __WB_MATRIX_BLAS_HH__
00022
00023 // ----- //
00024 // ----- //
00025
00026 void WbEigenSymmetric (const wbMatrix<double>&M,
00027     wbMatrix<double>&U, wbvector<double> &E);
00028
00029 namespace Wb {
00030
00031 template<class T>
00032 void MatProd(
00033     const wbMatrix<T> &A,
00034     const wbMatrix<T> &B, wbMatrix<T> &C,
00035     char aflag='N', char bflag='N',
00036     const T afac=1., const T cfac=0.,
00037     const char i00flag=0
00038 );
00039
00040 template<class TA, class TB, class TC>
00041 void MatProd(
00042     const wbMatrix<TA> &A,
00043     const wbvector<TB> &B, wbvector<TC> &C,
00044     char aflag='N', const TA afac=1., const TC cfac=0.,
00045     const char i00flag=0
00046 );
00047
00048 template<class T>
00049 void SVD(
00050     const wbMatrix<T> &A,
00051     wbMatrix<T> &U,
00052     wbMatrix<double> &S,
00053     wbMatrix<T> &Vt
00054 );
00055
00056 };
00057

```

```
00058 #endif
00059
```

5.49 wbmpfr.hh

```
00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace MPFR routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MPFR_HH__
00021 #define __WB_MPFR_HH__
00022
00023 /* ----- *
00024  * wrapper class for MPFR (multiple precision floating point library)
00025  * Wb, Feb01, 12
00026  *
00027  * -----
00028  Docu: see $SRC/MPFR/doc/mpfr.pdf
00029  * -----
00030
00031 MPFR basics
00032  * before assigning, MPFR variables need to be initialized.
00033  * when done with a variable, it needs to be cleared out.
00034  * variable will have the same allocated space during all its life.
00035
00036  * MPFR functions may create intermediate caches
00037  => use mpfr_free_cache() to free
00038  It is strongly advised to do that before terminating a thread
00039  to avoid memory leaks.
00040
00041  * The semantics of a calculation in MPFR is specified as follows:
00042  Compute the requested operation exactly (with "infinite accuracy"),
00043  and round the result to the precision of the destination variable,
00044  with the given rounding mode.
00045
00046  * if two variables are used to store only a few significant bits,
00047  and their product is stored in a variable with large precision,
00048  then MPFR will still compute the result with full precision.
00049
00050  * -----
00051 General rules:
00052  * all MPFR functions expect output arguments before input arguments;
00053  typically: output var, input vars ..., rounding
00054  * ok to use the same variable for both input and output in the same expression
00055  * functions returning int: ternary value
00056  0: value stored in output is the exact result of given mathematical function.
00057  pos. (neg): value stored in output is greater (lower) than the exact result.
00058
00059  * -----
00060 MPFR rounding modes (similar to GMP rounding modes)
00061 GMP_RNDD, GMP_RNDU, GMP_RNDN, ...):
00062
00063 MPFR_RNDN: nearest: round to nearest (roundTiesToEven in IEEE 754-2008)
00064 MPFR_RNDZ: zero : round toward zero (roundTowardZero in IEEE 754-2008)
00065 MPFR_RNDU: up : round toward plus infity (roundTowardPositive in IEEE 754-2008)
00066 MPFR_RNDD: down : round toward minus infity (roundTowardNegative in IEEE 754-2008)
00067 MPFR_RNDA: away 0 : round away from zero.
00068
00069  * -----
00070 Initialization routines can clearance
00071
00072 void mpfr_init(mpfr_t x)
00073 => initialize x to default precision and set its value to NaN
00074
00075 void mpfr_inits2(mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
00076 => initialize set of variables to default precision
00077 => must be terminated by null-pointer (whose type must also be mpfr_ptr)
00078
```

```

00079 void mpfr_init2(mpfr_t x, mpfr_prec_t prec)
00080 void mpfr_inits2(mpfr_prec_t prec, mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
00081 => same as previous init() functions, but with specified exact precision
00082
00083 void mpfr_clear(mpfr_t x)
00084 => free the space occupied by the significand of x.
00085 Make sure to call this function for all mpfr_t variables
00086 when you are done with them.
00087
00088 void mpfr_clears(mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
00089 => clear sequence of variables
00090 => must be terminated by null-pointer (whose type must also be mpfr_ptr)
00091
00092 void mpfr_set_default_prec(mpfr_prec_t prec)
00093 => if MPFR was compiled with --enable-thread-safe,
00094 the default precision is local to each thread.
00095
00096 mpfr_prec_t mpfr_get_default_prec(void)
00097 => return the current default MPFR precision in bits.
00098
00099 void mpfr_set_prec (mpfr_t x, mpfr_prec_t prec)
00100 mpfr_prec_t mpfr_get_prec (mpfr_t x)
00101 => set/get variable specific precision
00102
00103 * ----- */
00104
00105 #define DEF_RND GMP_RNDN
00106
00107 namespace Wb {
00108
00109 gmp_randstate_t wb_rstate;
00110
00111 template <unsigned P=128>
00112 class mpfr__ {
00113
00114 public:
00115
00116 mpfr__(double val=0) {
00117     mpfr_init2(f,P); init(FL,val);
00118     #pragma omp atomic
00119     ++num_vars;
00120 };
00121
00122 mpfr__(const char *F, int L, double val) {
00123     mpfr_init2(f,P); init(F_L,val);
00124     #pragma omp atomic
00125     ++num_vars;
00126 };
00127
00128 mpfr__(const mpfr__& b) {
00129     mpfr_init2(f,P); mpfr_set(f,b.f,DEF_RND);
00130     #pragma omp atomic
00131     ++num_vars;
00132 };
00133
00134 ~mpfr__() {
00135     mpfr_clear(f);
00136
00137     #pragma omp atomic
00138     --num_vars;
00139
00140     if (!num_vars) {
00141         Cleanup(FL);
00142     }
00143 };
00144
00145 void Cleanup(const char *F=NULL, int L=0) const;
00146
00147 mpfr__& init (const char *F, int L, double val, char lflag=0);
00148 mpfr__& init_s(const char *F, int L, const char *s, unsigned base=10);
00149 mpfr__& init_d(const char *F, int L, double val, char dcheck=1);
00150
00151 mpfr__& set_pi () { mpfr_const_pi (f,DEF_RND); return *this; };
00152 mpfr__& set_log2() { mpfr_const_log2(f,DEF_RND); return *this; };
00153 mpfr__& set_e () { init(FL,1); mpfr_exp(f,f,DEF_RND); return *this; };
00154
00155 mpfr__& set_nan () { mpfr_set_nan(f); return *this; };
00156 mpfr__& set_inf (int s=+1) { mpfr_set_inf(f,s); return *this; };
00157 mpfr__& set_zero(int s=+1) { mpfr_set_zero(f,s); return *this; };
00158
00159 int isnan () const { return mpfr_nan_p(f); };
00160 int isinf () const { return mpfr_inf_p(f); };
00161 int iszero () const { return mpfr_zero_p(f); };
00162 int isnumber () const { return mpfr_number_p(f); };
00163 int isnonzero() const { return mpfr_regular_p(f); };
00164
00165 int sign() { return mpfr_sgn(f); };

```

```

00166
00167 void swap(mpr_ &b) { mpr_swap(f,b.f); };
00168
00169 int cmp(const mpr_ &b ) const { return mpr_cmp (f,b.f); };
00170 int cmp(const double &val) const { return mpr_cmp_d (f,val); };
00171 int cmp(const int &val) const { return mpr_cmp_si (f,val); };
00172 int cmp(const long &val) const { return mpr_cmp_si (f,val); };
00173 int cmp(const unsigned &val) const { return mpr_cmp_ui (f,val); };
00174 int cmp(const unsigned long &val) const { return mpr_cmp_ui (f,val); };
00175
00176 bool operator> (const mpr_ &x) const { return (cmp(x)> 0); };
00177 bool operator> (const double &x) const { return (cmp(x)> 0); };
00178 bool operator> (const int &x) const { return (cmp(x)> 0); };
00179 bool operator> (const long &x) const { return (cmp(x)> 0); };
00180 bool operator> (const unsigned &x) const { return (cmp(x)> 0); };
00181 bool operator> (const unsigned long &x) const { return (cmp(x)> 0); };
00182
00183 bool operator< (const mpr_ &x) const { return (cmp(x)< 0); };
00184 bool operator< (const double &x) const { return (cmp(x)< 0); };
00185 bool operator< (const int &x) const { return (cmp(x)< 0); };
00186 bool operator< (const long &x) const { return (cmp(x)< 0); };
00187 bool operator< (const unsigned &x) const { return (cmp(x)< 0); };
00188 bool operator< (const unsigned long &x) const { return (cmp(x)< 0); };
00189
00190 bool operator>= (const mpr_ &x) const { return (cmp(x)>=0); };
00191 bool operator>= (const double &x) const { return (cmp(x)>=0); };
00192 bool operator>= (const int &x) const { return (cmp(x)>=0); };
00193 bool operator>= (const long &x) const { return (cmp(x)>=0); };
00194 bool operator>= (const unsigned &x) const { return (cmp(x)>=0); };
00195 bool operator>= (const unsigned long &x) const { return (cmp(x)>=0); };
00196
00197 bool operator<= (const mpr_ &x) const { return (cmp(x)<=0); };
00198 bool operator<= (const double &x) const { return (cmp(x)<=0); };
00199 bool operator<= (const int &x) const { return (cmp(x)<=0); };
00200 bool operator<= (const long &x) const { return (cmp(x)<=0); };
00201 bool operator<= (const unsigned &x) const { return (cmp(x)<=0); };
00202 bool operator<= (const unsigned long &x) const { return (cmp(x)<=0); };
00203
00204 bool operator!= (const mpr_ &x) const { return (cmp(x)!=0); };
00205 bool operator!= (const double &x) const { return (cmp(x)!=0); };
00206 bool operator!= (const int &x) const { return (cmp(x)!=0); };
00207 bool operator!= (const long &x) const { return (cmp(x)!=0); };
00208 bool operator!= (const unsigned &x) const { return (cmp(x)!=0); };
00209 bool operator!= (const unsigned long &x) const { return (cmp(x)!=0); };
00210
00211 bool operator== (const mpr_ &x) const { return (cmp(x)==0); };
00212 bool operator== (const double &x) const { return (cmp(x)==0); };
00213 bool operator== (const int &x) const { return (cmp(x)==0); };
00214 bool operator== (const long &x) const { return (cmp(x)==0); };
00215 bool operator== (const unsigned &x) const { return (cmp(x)==0); };
00216 bool operator== (const unsigned long &x) const { return (cmp(x)==0); };
00217
00218 mpr_ & operator= (double val) { return init(FL,val); };
00219 mpr_ & operator= (const mpr_ &b) {
00220     mpr_set(f,b.f,DEF_RND);
00221     return *this;
00222 };
00223
00224 mpr_ & times(const mpr_ &b, mpr_ &c) const {
00225     mpr_mul(c.f,f,b.f,DEF_RND); return c; };
00226 mpr_ & divby(const mpr_ &b, mpr_ &c) const {
00227     mpr_div(c.f,f,b.f,DEF_RND); return c; };
00228
00229 mpr_ & operator*= (const mpr_ &b) {
00230     mpr_mul(f,f,b.f,DEF_RND); return *this; };
00231 mpr_ & operator/= (const mpr_ &b) {
00232     mpr_div(f,f,b.f,DEF_RND); return *this; };
00233 mpr_ & operator+= (const mpr_ &b) {
00234     mpr_add(f,f,b.f,DEF_RND); return *this; };
00235 mpr_ & operator-= (const mpr_ &b) {
00236     mpr_sub(f,f,b.f,DEF_RND); return *this; };
00237
00238 mpr_ & operator*= (double val) { mpr_ b(FL,val); return (*this)*=b; };
00239 mpr_ & operator/= (double val) { mpr_ b(FL,val); return (*this)/=b; };
00240 mpr_ & operator+= (double val) { mpr_ b(FL,val); return (*this)+=b; };
00241 mpr_ & operator-= (double val) { mpr_ b(FL,val); return (*this)-=b; };
00242
00243 mpr_ operator* (const mpr_ &b) const {
00244     mpr_ x; mpr_mul(x.f,f,b.f,DEF_RND); return x; };
00245 mpr_ operator/ (const mpr_ &b) const {
00246     mpr_ x; mpr_div(x.f,f,b.f,DEF_RND); return x; };
00247 mpr_ operator+ (const mpr_ &b) const {
00248     mpr_ x; mpr_add(x.f,f,b.f,DEF_RND); return x; };
00249 mpr_ operator- (const mpr_ &b) const {
00250     mpr_ x; mpr_sub(x.f,f,b.f,DEF_RND); return x; };
00251
00252 mpr_ operator* (int n) const {

```

```

00253     mpfr__ x; mpfr_mul_si(x.f,f,n,DEF_RND); return x; };
00254 mpfr__ operator/ (int n) const {
00255     mpfr__ x; mpfr_div_si(x.f,f,n,DEF_RND); return x; };
00256 mpfr__ operator+ (int n) const {
00257     mpfr__ x; mpfr_add_si(x.f,f,n,DEF_RND); return x; };
00258 mpfr__ operator- (int n) const {
00259     mpfr__ x; mpfr_sub_si(x.f,f,n,DEF_RND); return x; };
00260
00261 mpfr__ operator* (unsigned n) const {
00262     mpfr__ x; mpfr_mul_ui(x.f,f,n,DEF_RND); return x; };
00263 mpfr__ operator/ (unsigned n) const {
00264     mpfr__ x; mpfr_div_ui(x.f,f,n,DEF_RND); return x; };
00265 mpfr__ operator+ (unsigned n) const {
00266     mpfr__ x; mpfr_add_ui(x.f,f,n,DEF_RND); return x; };
00267 mpfr__ operator- (unsigned n) const {
00268     mpfr__ x; mpfr_sub_ui(x.f,f,n,DEF_RND); return x; };
00269
00270 mpfr__ operator* (long n) const {
00271     mpfr__ x; mpfr_mul_si(x.f,f,n,DEF_RND); return x; };
00272 mpfr__ operator/ (long n) const {
00273     mpfr__ x; mpfr_div_si(x.f,f,n,DEF_RND); return x; };
00274 mpfr__ operator+ (long n) const {
00275     mpfr__ x; mpfr_add_si(x.f,f,n,DEF_RND); return x; };
00276 mpfr__ operator- (long n) const {
00277     mpfr__ x; mpfr_sub_si(x.f,f,n,DEF_RND); return x; };
00278
00279 mpfr__ operator* (unsigned long n) const {
00280     mpfr__ x; mpfr_mul_ui(x.f,f,n,DEF_RND); return x; };
00281 mpfr__ operator/ (unsigned long n) const {
00282     mpfr__ x; mpfr_div_ui(x.f,f,n,DEF_RND); return x; };
00283 mpfr__ operator+ (unsigned long n) const {
00284     mpfr__ x; mpfr_add_ui(x.f,f,n,DEF_RND); return x; };
00285 mpfr__ operator- (unsigned long n) const {
00286     mpfr__ x; mpfr_sub_ui(x.f,f,n,DEF_RND); return x; };
00287
00288 mpfr__ & Sqrt() { mpfr_sqrt(f,f,DEF_RND); return *this; };
00289 mpfr__ & Exp() { mpfr_exp(f,f,DEF_RND); return *this; };
00290 mpfr__ & Log() { mpfr_log(f,f,DEF_RND); return *this; };
00291 mpfr__ & Log2() { mpfr_log2(f,f,DEF_RND); return *this; };
00292 mpfr__ & Atan() { mpfr_atan(f,f,DEF_RND); return *this; };
00293
00294 mpfr__ & Tanh() { mpfr_tanh(f,f,DEF_RND); return *this; };
00295 mpfr__ & Atanh() { mpfr_atanh(f,f,DEF_RND); return *this; };
00296
00297 mpfr__ & Rand() {
00298     mpfr_urandomb(f,wb_rstate);
00299     return *this;
00300 };
00301
00302 mpfr__ & Log10() { mpfr_log10(f,f,DEF_RND); return *this; };
00303 mpfr__ & Round() { mpfr_round(f,f); return *this; };
00304
00305 mpfr__ sqrt() const {
00306     mpfr__<P> x(*this); mpfr_sqrt(x.f,x.f,DEF_RND); return x; };
00307 mpfr__ exp() const {
00308     mpfr__<P> x(*this); mpfr_exp(x.f,x.f,DEF_RND); return x; };
00309 mpfr__ log() const {
00310     mpfr__<P> x(*this); mpfr_log(x.f,x.f,DEF_RND); return x; };
00311 mpfr__ log2() const {
00312     mpfr__<P> x(*this); mpfr_log2(x.f,x.f,DEF_RND); return x; };
00313 mpfr__ atan() const {
00314     mpfr__<P> x(*this); mpfr_atan(x.f,x.f,DEF_RND); return x; };
00315 mpfr__ log10() const {
00316     mpfr__<P> x(*this); mpfr_log10(x.f,x.f,DEF_RND); return x; };
00317 mpfr__ round() const {
00318     mpfr__<P> x(*this); mpfr_round(x.f,x.f); return x; };
00319
00320 explicit operator double() const { return mpfr_get_d(f,DEF_RND); };
00321
00322 explicit operator wbcomplex() const;
00323
00324 explicit operator bool() const {
00325     return (mpfr_zero_p(f) ? 0 : 1);
00326 };
00327
00328 double getval_d() const { return mpfr_get_d(f,DEF_RND); };
00329
00330 bool operator< (double val) const { return (getval_d()< val); };
00331 bool operator<= (double val) const { return (getval_d()<=val); };
00332 bool operator> (double val) const { return (getval_d()> val); };
00333 bool operator>= (double val) const { return (getval_d()>=val); };
00334
00335 int prec(unsigned base=2) const {
00336     int p=mpfr_get_prec(f);
00337     if (base!=2) { p = p* (::log(2)/::log(base)) + 1; }
00338     return p;
00339 };

```



```

00340
00341     mpfr__ eps() {
00342         mpfr__ x;
00343         mpfr_set_uj_2exp(x.f, 1, -prec(2), DEF_RND);
00344         return x;
00345     };
00346
00347     mpfr__ & FlipSign() {
00348         mpfr_setsign(f,f, mpfr_signbit(f) ? 0 : 1, DEF_RND);
00349         return *this;
00350     };
00351
00352     mpfr__ & Abs() {
00353         mpfr_setsign(f,f, 0, DEF_RND);
00354         return *this;
00355     };
00356
00357     mpfr__ abs() const { return mpfr__(*this).Abs(); };
00358     mpfr__ fabs() const { return mpfr__(*this).Abs(); };
00359     mpfr__ abs2() const {
00360         mpfr__ x(*this); x*=(*this);
00361         return x;
00362     };
00363
00364     void print(const char *vname="", const char *istr=0, char sep=0) const {
00365         if (sep) printf("-----"
00366             "0-----+-----1-----+-----2-----+-----3-----+-----4-----+-----"
00367             "5-----+-----6-----+-----7-----\n"
00368         );
00369         if (vname && vname[0]) printf("%6s = ", vname);
00370         else printf("          ");
00371
00372         mpfr_out_str (stdout,
00373             10,
00374             0,
00375             f,
00376             GMP_RNDU
00377         );
00378
00379         if (!istr || !istr[0]) {
00380             long p,q; unsigned niter; double r, x=getval_d();
00381             int e=Rational(x,p,q,&r,&niter,NULL,6,1000000,1E-10,1E-12);
00382             if (!e && p) { char s[32];
00383                 if (q!=1) snprintf(s,32," = %ld/%ld",p,q);
00384                 else snprintf(s,32," = %ld",p);
00385                 istr=s;
00386             }
00387         }
00388         printf(" %s\n", istr && istr[0] ? istr : "");
00389     };
00390
00391     size_t to_str(char *s, size_t n_, int base=10) const;
00392     wstring to_str(int base=10) const;
00393
00394     void toStr(
00395         char *s, unsigned n, int base=10, int nd=-1, char align=0) const;
00396
00397     wstring& toStr(
00398         wstring &sout,int base=10, int nd=-1, char align=0) const;
00399
00400     mpfr_t f;
00401
00402     static unsigned long num_vars;
00403
00404 private:
00405
00406 };
00407
00408 typedef mpfr__<128> quad;
00409 typedef mpfr__<128> quad2;
00410
00411 typedef mpfr__<192> quad3;
00412 typedef mpfr__<256> quad4;
00413 typedef mpfr__<256> qquad;
00414
00415 template<>
00416 unsigned long quad::num_vars=0;
00417
00418 quad mpfr_dummy_quad;
00419
00420 template <unsigned P>
00421 mpfr__<P> operator+(const mpfr__<P> &a) { return mpfr__<P>(a); };
00422
00423 template <unsigned P>
00424 mpfr__<P> operator-(const mpfr__<P> &a) { return mpfr__<P>(a).FlipSign(); };
00425
00426 template <class T, unsigned P>

```

```

00427 mpfr__<P> operator*(const T& x_, const mpfr__<P> &a) {
00428     mpfr__<P> x(x_); return (x*=a);
00429 };
00430
00431 template <class T, unsigned P>
00432 mpfr__<P> operator+(const T& x_, const mpfr__<P> &a) {
00433     mpfr__<P> x(x_); return (x+=a);
00434 };
00435
00436 template <class T, unsigned P>
00437 mpfr__<P> operator-(const T& x_, const mpfr__<P> &a) {
00438     mpfr__<P> x(x_); return (x-=a);
00439 };
00440
00441 template <class T, unsigned P>
00442 mpfr__<P> operator/(const T& x_, const mpfr__<P> &a) {
00443     mpfr__<P> x(x_); return (x/=a);
00444 };
00445
00446 template <unsigned P> inline
00447 int isnan(const mpfr__<P> &a) { return a.isnan(); };
00448
00449 template <unsigned P> inline
00450 int isinf(const mpfr__<P> &a) { return a.isinf(); };
00451
00452 template <unsigned P> inline
00453 int iszero(const mpfr__<P> &a) { return a.iszero(); };
00454
00455 template <unsigned P> inline
00456 int isnumber(const mpfr__<P> &a) { return a.isnumber(); };
00457
00458 template <unsigned P> inline
00459 int isfinite(const mpfr__<P> &a) { return a.isfinite(); };
00460
00461 template <unsigned P>
00462 mpfr__<P> fabs(const mpfr__<P> &x) { return x.fabs(); };
00463
00464 template <unsigned P>
00465 mpfr__<P> sqrt(const mpfr__<P> &x) { return x.sqrt(); };
00466
00467 template <unsigned P>
00468 mpfr__<P> exp(const mpfr__<P> &x) { return x.exp(); }
00469
00470 template <unsigned P>
00471 mpfr__<P> log(const mpfr__<P> &x) { return x.log(); }
00472
00473 template <unsigned P>
00474 mpfr__<P> log2(const mpfr__<P> &x) { return x.log2(); }
00475
00476 template <unsigned P>
00477 mpfr__<P> atan(const mpfr__<P> &x) { return x.atan(); }
00478
00479 template <unsigned P>
00480 mpfr__<P> log10(const mpfr__<P> &x) { return x.log10(); }
00481
00482 template <unsigned P>
00483 mpfr__<P> round(const mpfr__<P> &x) { return x.round(); }
00484
00485 class cc_quad {
00486 public:
00487
00488     cc_quad(long double l=0) : f(l) { if (l!=0) {
00489         long double r1=std::sqrt(l), r2=std::sqrt(f);
00490
00491         printf("1> %22.20Lf => %22.20Lf (%.3Lg)\n",l,r1,r1*r1-l);
00492         printf("2> %22.20Lf => %22.20Lf (%.3Lg)\n", f,r2,r2*r2-f);
00493
00494         r2=std::sqrt(f);
00495         printf("3> %22.20Lf => %22.20Lf (%.3Lg)\n\n",f,r2,r2*r2-f);
00496     } }
00497
00498     };
00499
00500     cc_quad sqrt() const { cc_quad x(*this); x.f=std::sqrt(f); return x; };
00501     cc_quad fabs() const { cc_quad x(*this); x.f=std::fabs(f); return x; };
00502     cc_quad abs() const { cc_quad x(*this); x.f=std::fabs(f); return x; };
00503
00504     cc_quad operator+(const cc_quad& b) const { return (f+b.f); };
00505     cc_quad operator-(const cc_quad& b) const { return (f-b.f); };
00506     cc_quad operator*(const cc_quad& b) const { return (f*b.f); };
00507     cc_quad operator/(const cc_quad& b) const { return (f/b.f); };
00508
00509     cc_quad operator+=(const cc_quad& b) { return (f+=b.f); };
00510     cc_quad operator-=(const cc_quad& b) { return (f-=b.f); };
00511     cc_quad operator*=(const cc_quad& b) { return (f*=b.f); };
00512     cc_quad operator/=(const cc_quad& b) { return (f/=b.f); };
00513

```

```

00514     long double f;
00515
00516     private:
00517
00518 };
00519
00520     cc_quad sqrt(const cc_quad &a) { return a.sqrt(); }
00521
00522 };
00523
00524 #endif
00525

```

5.50 wbmshg.hh

```

00001 #ifndef __WB_MSG_HH__
00002 #define __WB_MSG_HH__
00003
00004 /* https://stackoverflow.com/questions/24785230/difference-between-msgget-and-mq-open
00005  * Wb, Oct06, 22
00006
00007     The msgXXX family is inherited from SysV. The mq_XXX family was
00008     created by POSIX. Both families are however part of POSIX.
00009
00010     The two families have somewhat different feature sets the most
00011     important being that the SysV variant allows for the specification
00012     of different message types. This provides for flexibility in the
00013     way messages are processed and in assigning priorities. The POSIX
00014     family in contrast always returns the oldest message with the
00015     highest priority.
00016
00017     As a rule the POSIX family is the better choice unless there is
00018     something about SysV that you really need to use. The POSIX
00019     interface is also less cumbersome. An added bonus (on Linux) is
00020     that the POSIX mqd_t type representing a queue identifier is
00021     actually a file descriptor. This allows for the extremely
00022     convenient use of POSIX queues with select, poll and similar
00023     facilities.
00024
00025     Basicly, msgget, msgsnd, msgrcv are System V IPC, while mq_open,
00026     mq_send, mq_receive are POSIX IPC.
00027
00028     POSIX IPC is designed after System V IPC. So a lot of old systems
00029     support only System V IPC while new systems begin to support POSIX
00030     IPC as well. And, because POSIX IPC can learn the advantages and
00031     disadvantages from System V IPC, POSIX IPC may be designed and
00032     implemented better. A notable difference is that
00033     all the POSIX IPC interfaces are thread safe.
00034
00035     Comments: SysV interfaces are also thread safe. There would be no
00036     point to them otherwise
00037     Well, I didn't confirm it but just read it from
00038     docs.oracle.com/cd/E19455-01/806-4750/6jddqdf1tf/index.html that
00039     SysV IPC may not thread safe: Unlike the System V IPC interfaces,
00040     the POSIX IPC interfaces are all multithread safe
00041
00042 # IPC benchmarking
00043 # https://stackoverflow.com/questions/2854004/ipc-speed-and-compare
00044 Pipe benchmark                36,539 msg/s
00045 FIFOs (named pipes) benchmark 26,246 msg/s
00046 Message Queues benchmark      67,920 msg/s
00047 Shared Memory benchmark       3,821,893 msg/s 100x faster!
00048 TCP sockets benchmark         22,483 msg/s
00049 Unix domain sockets benchmark 40,683 msg/s
00050 ZeroMQ benchmark              15,414 msg/s
00051
00052 https://www.qnx.com/developers/docs/6.5.0SP1.update/...
00053 com.qnx.doc.neutrino_sys_arch/ipc.html#Shared_memory
00054
00055 Shared memory offers the highest bandwidth IPC available. Once a
00056 shared-memory object is created, processes with access to the
00057 object can use pointers to directly read and write into it
00058
00059 */
00060
00061 // ----- //
00062 // Messaging class -> automates deletion of message queue!!
00063 // Wb, Dec03, 2003
00064
00065 #define WB_MSG_PRIV 0666
00066
00067 typedef struct __wb_msgbuf {
00068     long    type;

```

```

00069         time_t   time;
00070         char      text[128];
00071         double    val;
00072     } wbMsgBuf;
00073
00074     const size_t WBMSGSZ = sizeof(wbMsgBuf) - sizeof(long);
00075
00076     class wbMsg {
00077
00078     public:
00079
00080         wbMsg (const char* file, int line,
00081             const char *istr="Message queue", int key_=0
00082         ){
00083             info=istr;
00084
00085             key = (key_>0) ? key_ : getpid();
00086             msgflg = WB_MSG_PRIV;
00087
00088             if ((msqid = msgget(key, IPC_CREAT | msgflg)) < 0) {
00089                 wblog(FL,"ERR msgget returns %d", -msqid);
00090                 exit(-1);
00091             }
00092
00093             if (istr && istr[0]) wblog(file,line,"%NMSG create queue "
00094                 "%s (key=%d (0x%lX), qid=%d)", info.data,key,key,msqid
00095             );
00096         };
00097
00098         ~wbMsg () {
00099             if (info.data && info.data[0]) wblog (FL,"MSG remove queue "
00100                 "%s (key=%d (0x%lX), qid=%d)", info.data,key,key,msqid);
00101             msgctl (msqid, IPC_RMID, 0);
00102         };
00103
00104         int Rec(const char*,int, wbMsgBuf &rbuf,
00105             unsigned flag=IPC_NOWAIT
00106         );
00107
00108         int Snd(const char*,int, wbMsgBuf &sbuf,
00109             const char *istr, double val=0., unsigned type=1
00110         );
00111
00112         wstring info;
00113         int msqid, msgflg;
00114         key_t key;
00115
00116     protected:
00117     private:
00118     };
00119
00120     int wbMsg::Rec (
00121         const char* file, int line,
00122         wbMsgBuf &rbuf,
00123         unsigned flag
00124     ){
00125
00126         memset (&rbuf,0,sizeof(wbMsgBuf));
00127
00128         int i = msgrcv(msqid, &rbuf, WBMSGSZ, 1, flag);
00129         if (i < 0) {
00130             if (i!=-1)
00131                 wblog(file,line,
00132                     "ERR can't read message port %d (%d)", msqid, i);
00133             return -i;
00134         }
00135
00136         sprintf(str,"%s", ctime(&rbuf.time));
00137         str[strlen(str)-1] = 0;
00138
00139         wblog (file,line,"%NNB! RECEIVED MESSAGE %s (%d; 0x%lX %d)\n\n"
00140             "    time : %s\n"
00141             "    text  : %s\n"
00142             "    val   : %g\n"
00143             "    type  : %ld\n",
00144             info.data ? info.data : "", key, key, msqid,
00145             str, rbuf.text, rbuf.val, rbuf.type);
00146
00147         return 0;
00148     }
00149
00150     int wbMsg::Snd (
00151         const char* file, int line,
00152         wbMsgBuf &sbuf,
00153         const char *istr, double val,
00154         unsigned type
00155     ){

```

```

00156
00157     int i;
00158
00159     memset(&sbuf,0,sizeof(wbMsgBuf));
00160
00161     sbuf.type = type;
00162     sbuf.time = time((time_t*)NULL);
00163     strncpy(sbuf.text,istr,127);
00164     sbuf.val = val;
00165
00166     i = msgsnd(msqid, &sbuf, WBMSGSZ, IPC_NOWAIT);
00167     if (i<0) {
00168         wblog(FL,"ERR can't send to message port %d (err=%d)", msqid, i);
00169         return -1;
00170     }
00171
00172     sprintf(str,"%s", ctime(&sbuf.time));
00173     str[strlen(str)-1] = 0;
00174
00175     wblog (file,line,
00176         "Message sent ...\n"
00177         "   type  : %ld\n"
00178         "   time  : %s\n"
00179         "   text  : %s\n"
00180         "   val   : %g",
00181         sbuf.type, str, sbuf.text, sbuf.val);
00182
00183     return 0;
00184 }
00185
00186 int WbMSGsnd (
00187     const char* file, int line,
00188     unsigned key,
00189     char *text, double val,
00190     unsigned type=1
00191 ){
00192
00193     wbMsgBuf sbuf;
00194     int i, msqid;
00195
00196     if ((msqid = msgget(key, IPC_CREAT | WB_MSG_PRIV)) < 0) {
00197         wblog(FL,"ERR can't read message port %d (err=%d)", key, -msqid);
00198         return -1;
00199     }
00200
00201     sbuf.type = type;
00202     sbuf.time = time((time_t*)NULL);
00203     strcpy (sbuf.text, text);
00204     sbuf.val = val;
00205
00206     i = msgsnd(msqid, &sbuf, WBMSGSZ, IPC_NOWAIT);
00207     if (i<0) {
00208         wblog(FL,"ERR can't send to message port %d (err=%d)", msqid, i);
00209         return -1;
00210     }
00211
00212     sprintf (str, "%s", ctime(&sbuf.time));
00213     str[strlen(str)-1] = 0;
00214
00215     wblog(file,line,
00216         "Message sent (%d; 0x%lX %d) ...\n"
00217         "   type  : %ld\n"
00218         "   time  : %s\n"
00219         "   text  : %s\n"
00220         "   val   : %g", key, key, msqid,
00221         sbuf.type, str, sbuf.text, sbuf.val);
00222
00223     return 0;
00224 }
00225
00226 #endif
00227

```

5.51 wbomp.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace OMP routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.

```

```

00009  * You may obtain a copy of the License at
00010  *
00011  *      http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_OMP_CC__
00021 #define __WB_OMP_CC__
00022
00023 #ifdef QS_USING_OMP
00024
00025 wbststring Wb::ompID2Str(char vflag) {
00026     wbststring s(8);
00027     unsigned k=(vflag ? 16 : 0),
00028     i=omp_get_thread_num(), n=omp_get_num_threads(), l=omp_get_level();
00029
00030     if (!i && n==1 && !l) { s.init(8+k); strcpy(s.data,"serial"); l=6; }
00031     else if (l!=1)
00032     { s.init(16+k); l=snprintf(s.data,s.len,"OMP-%d/%d@%d",i,n,l); }
00033     else { s.init(12+k); l=snprintf(s.data,s.len,"OMP-%d/%d", i,n ); }
00034
00035     if (k && l<s.len) {
00036         l+=snprintf(s.data+l, s.len-l, " %s",Wb::hostid(vflag).data);
00037     }
00038
00039     return s;
00040 };
00041
00042 int Wb::ompStatus(const char *F, int L) {
00043     unsigned l=0, n=1024;
00044     char fmt[]=" %-30s %2d\n", s[n];
00045
00046     if (l<n) l+=snprintf(s+l,n-l,fmt,
00047         "omp_get_active_level", omp_get_active_level());
00048     if (l<n) l+=snprintf(s+l,n-l,fmt,
00049         "omp_get_cancellation", omp_get_cancellation());
00050     if (l<n) l+=snprintf(s+l,n-l,fmt,
00051         "omp_get_default_device", omp_get_default_device());
00052     if (l<n) l+=snprintf(s+l,n-l,fmt,
00053         "omp_get_dynamic", omp_get_dynamic());
00054     if (l<n) l+=snprintf(s+l,n-l,fmt,
00055         "omp_get_level", omp_get_level());
00056     if (l<n) l+=snprintf(s+l,n-l,fmt,
00057         "omp_get_max_task_priority", omp_get_max_task_priority());
00058     if (l<n) l+=snprintf(s+l,n-l,fmt,
00059         "omp_get_max_threads", omp_get_max_threads());
00060     if (l<n) l+=snprintf(s+l,n-l,fmt,
00061         "omp_get_max_active_levels", omp_get_max_active_levels());
00062     if (l<n) l+=snprintf(s+l,n-l,fmt,
00063         "omp_get_num_procs", omp_get_num_procs());
00064     if (l<n) l+=snprintf(s+l,n-l,fmt,
00065         "omp_get_num_teams", omp_get_num_teams());
00066     if (l<n) l+=snprintf(s+l,n-l,fmt,
00067         "omp_get_num_threads", omp_get_num_threads());
00068
00069     if (l<n) l+=snprintf(s+l,n-l,fmt,
00070         "omp_get_proc_bind", omp_get_proc_bind());
00071     if (l<n) l+=snprintf(s+l,n-l,fmt,
00072         "omp_get_team_num", omp_get_team_num());
00073     if (l<n) l+=snprintf(s+l,n-l,"%n");
00074     if (l<n) l+=snprintf(s+l,n-l,fmt,
00075         "omp_get_thread_num", omp_get_thread_num());
00076     if (l<n) l+=snprintf(s+l,n-l,fmt,
00077         "omp_in_parallel", omp_in_parallel());
00078     if (l<n) l+=snprintf(s+l,n-l,fmt,
00079         "omp_in_final", omp_in_final());
00080     if (l<n) l+=snprintf(s+l,n-l,fmt,
00081         "omp_is_initial_device", omp_is_initial_device());
00082
00083     fprintf(stdout,"\n%-18s %s -> %s()\n\n",shortFL(F_L),myname,FCT);
00084     if (l<n)
00085         fprintf(stdout,"%s\n",s);
00086     else fprintf(stdout,"%s ... (%d/%d)\n",s,l,n);
00087
00088     return 0;
00089 };
00090
00091 int Wb::ompNLock::acquire() {
00092
00093     omp_set_nest_lock(&nlk);
00094     owner=omp_get_thread_num(); ++ntot; ++level;
00095

```

```

00096 #if defined(DBG_GCX_LOCKS) && ( DBG_GCX_LOCKS & 12 )
00097     LKF.blogf(FL, " + %s%s", STR_(this), check_BUF_size().data);
00098 #endif
00099
00100     return level;
00101 };
00102
00103 int Wb::ompNLock::test_acquire() {
00104     int l=omp_test_nest_lock(&_nlk);
00105     if (!l) return l;
00106
00107     owner=omp_get_thread_num(); ++ntot; ++level;
00108
00109     return level;
00110 };
00111
00112 int Wb::ompNLock::release() {
00113
00114     if (owner!=omp_get_thread_num()) {
00115         wstring sx=get_istr(); bool q=sx.strlen()<10;
00116         wblog(PFL, "WRN %s() "
00117             "lock%s%s @ l=%d/%d/%d by non-owner %d/%d/%d !?%s%s",
00118             FCT, q? " ":"", q? sx.data:" ", level, nactive, ntot,
00119             owner, omp_get_thread_num(), omp_get_num_threads(),
00120             q? "":"\n", q? "":sx.data
00121         );
00122     }
00123
00124     if (level<=0) wblog(FL, "WRN %s() %s", FCT, STR_(this));
00125
00126     int l=(--level);
00127     if (l<=0) owner=-1;
00128
00129     #if defined(DBG_GCX_LOCKS) && ( DBG_GCX_LOCKS & 12 )
00130         LKF.blogf(FL, " - %s%s", STR_(this), check_BUF_size().data);
00131     #endif
00132
00133     omp_unset_nest_lock(&_nlk);
00134
00135     return (l>0? l : 0);
00136 };
00137
00138 #ifdef LOAD_CGC_QSPACE
00139
00140 wstring Wb::ompNLock::check_BUF_size() const {
00141
00142     wstring s;
00143
00144     if (istr) { s.init(16);
00145
00146         if (!strcmp(istr, "CG_locks"))
00147             snprintf(s.data, s.len, " @ %ld", gCS.BUF.size()); else
00148             if (!strcmp(istr, "XS_buf"))
00149                 snprintf(s.data, s.len, " @ %ld", gXS.XBUF.size());
00150     }
00151     else { s.init(1); s.data[0]=0; }
00152
00153     return s;
00154 };
00155
00156 #endif
00157
00158 wstring Wb::ompNLock::get_istr() const {
00159     wstring s(128);
00160     if (istr) {
00161         Wb::termcolor Tc("WRN");
00162         snprintf(s.data, s.len, "%s%s%s", Tc.el, istr, Tc.em);
00163     }
00164     else {
00165         snprintf(s.data, s.len, "%lx", (unsigned long)this);
00166     }
00167     return s;
00168 };
00169
00170 wstring Wb::ompNLock::toStr(const char *istr_) const {
00171
00172     wstring sout(128);
00173     unsigned l=0, n=sout.len; char *s=sout.data;
00174
00175     if ( nactive && !ntot ) l+=snprintf(s, n, "%d/%d", level, nactive); else
00176     if (!nactive && ntot>9) l+=snprintf(s, n, "%d/%03ld", level, ntot );
00177     else l+=snprintf(s, n, "%d/%d/%ld", level, nactive, ntot);
00178
00179     l+=Wb::strpad_(s+l, 8-1);
00180     l+=snprintf(s+l, n-l, " %2d", owner);
00181
00182     if (l<n) {

```

```

00183         if (istr && istr[0]) l+=snprintf(s+l, n-l, "%s", istr );
00184     else if (istr_)         l+=snprintf(s+l, n-l, "%s'", istr_);
00185     else {
00186         l+=snprintf(s+l, n-l, " (null)");
00187         wblog(FL, "WRN %s() got null istr", FCT);
00188     }
00189 }
00190 if (l<n && (!istr || !istr[0])) {
00191     l+=snprintf(s+l, n-l, "%p", this);
00192 }
00193 return sout;
00194 };
00195
00196 void Wb::ompNLock::set_istr(const char *s) {
00197
00198     if ((!istr || !istr[0]) && s && s[0]) {
00199         if (istr) { delete [] istr; }
00200         istr = new char[strlen(s)+1];
00201         strcpy(istr, s);
00202     }
00203 };
00204
00205 void Wb::ompGuard::acquire(const char *F, int L, ompNLock &lk, int lmax) {
00206
00207     if (lp) wblog(FL, "ERR %s() already initialized with lock", FCT);
00208
00209     if (lk.level<lmax) { lp=&lk;
00210         lk.acquire();
00211
00212         if (lk.ntot<=1 && F) {
00213             Wb::termcolor Tc("WRN");
00214             fprintf(stdout, "\n %sTST %s %s() "
00215                 "using ompCritical '%s'\n", Tc.el, shortFL(F_L), FCT,
00216                 lk.istr? lk.istr: "(unnamed ompNLock)", Tc.em);
00217             fflush(0);
00218         }
00219     }
00220     else if (lk.level>lmax) fprintf(stdout, "%s %s "
00221         "WRN got lk.level=%d/%d\n", shortFLT, lk.level, lmax);
00222 };
00223
00224 #ifdef LOAD_CGC_QSPACE
00225
00226 namespace CG {
00227
00228     Wb::ompNLock* Guard::find(size_t h) {
00229         auto it = CG::lock_map.find(h);
00230         if (it==CG::lock_map.end())
00231             { return NULL; }
00232         else { return it->second; }
00233     };
00234
00235     template<class TQ>
00236     wstring Guard::Status(const QSet<TQ> &Q, const char *tag) {
00237         size_t h1=QHash<TQ>()(Q, tag); wstring istr=Q.toStr();
00238         if (tag && tag[0]) {
00239             unsigned l=strlen(tag)+4; char sx[l];
00240             snprintf(sx, l, "%s'", tag); istr+=sx;
00241         }
00242         return Status(h1, istr.data);
00243     };
00244
00245     template<class TM>
00246     wstring Guard::Status(const cgc_contract_id<TM> &idc) {
00247         size_t h1=MHash<TM>()(idc); wstring istr=idc.toStr();
00248         return Status(h1, istr.data);
00249     };
00250
00251     wstring Guard::Status(size_t h1, const char *istr) {
00252         wstring sout;
00253         const Wb::ompNLock *lk=Guard::find(h1);
00254         if (lk) { sout=lk->toStr(istr); }
00255         else {
00256             unsigned n=(istr? strlen(istr):0); n=MIN(2U, n)+16; sout.init(n);
00257             snprintf(sout.data, n, "no entry for %s", istr && istr[0]? istr: "");
00258         }
00259         return sout;
00260     };
00261
00262     int Guard::toStr(wstring &s, size_t h, const char *istr_) {
00263         const Wb::ompNLock *lk=Guard::find(h);
00264         if (lk) { s=lk->toStr(istr_); return 1; }
00265         else if (istr_ && istr_[0]) { s.init(128);
00266             if (h)
00267                 { snprintf(s.data, s.len, "(undefined) %s", istr_); }
00268             else { snprintf(s.data, s.len, "(undefined; h=0 !?) %s", istr_); }
00269         }
00270     }

```



```

00270     else { s.init(32);
00271         if (h)
00272             { snprintf(s.data,s.len,"(undefined; h=0x%lX)",h); }
00273         else { snprintf(s.data,s.len,"(undefined; h=0)"); }
00274     }
00275     return 0;
00276 };
00277
00278 wbststring Guard::toStr(size_t h, const char *istr_) {
00279     wbststring s;
00280     Guard::toStr(s,h,istr_);
00281     return s;
00282 };
00283
00284 wbststring Guard::toStr() const {
00285     wbststring sout; if (!hid.len) return sout;
00286
00287     unsigned i=0,j,k=0,n=128*hid.len; wbststring sl; char *s;
00288     sout.init(n); s=sout.data;
00289
00290     for (; k<hid.len; ++k) {
00291         Guard::toStr(sl,hid[k]);
00292         for (j=0; sl.data[j]; ++j, ++i) { if (i<n) s[i]=sl.data[j]; }
00293         if (i<n && k<hid.len) { s[i++]='\n'; }
00294     }
00295     s[i<n ? i:n-1]=0;
00296     if (i>=n) wblog(FL,"ERR string out of bounds (%d/%d)\n%s",i,n,s);
00297     return sout;
00298 };
00299 };
00300
00301 int CG::Guard::print_CG_locks() {
00302     unsigned i=0;
00303
00304     fprintf(stdout,"level/nactive/nuse, owner_thread   info_string\n");
00305     for (auto I=CG::lock_map.begin(); I!=CG::lock_map.end(); ++I) {
00306         fprintf(stdout,"%2d. %s\n",++i,STR_(I->second));
00307     }
00308
00309     return 0;
00310 };
00311
00312 const char* CG::Guard::sprintf_locks(
00313     wbststring &sout, const int* got, const char *ss[], const char *indent
00314 ){
00315     if (hid.len) {
00316         unsigned i=0,l=0,n=256*hid.len; char *s; wbststring sx;
00317         if (sout.len<n) { sout.init(n); } else { n=sout.len; }
00318         s=sout.data;
00319
00320         for (; i<hid.len; ++i) {
00321             if (!Guard::toStr(sx,hid[i],"")) { sx.init(); }
00322             if (l<n)
00323                 l+=sprintf(s+l,n-l,"%s%d. %-8s %s\n", indent? indent:"",
00324                     i+1, got[i]? "ok.":"rejected", sx.len? " " : "", ss[i]);
00325             if (l<n && sx.len)
00326                 l+=sprintf(s+l,n-l,"%s %s\n", indent? indent:"", sx.data);
00327         }
00328         if (l>=n) wblog(FL,
00329             "WRN %s() string out of bounds (%d/%d)%N%N`s'`s'`N", FCT,l,n,s);
00330     }
00331     else { sout=""; }
00332
00333     return sout.data;
00334 };
00335
00336 int CG::Guard::deal_with_cond(
00337     const char *F, int L, size_t h0, const char *cond) {
00338
00339     int ntry=-1; if (!cond || !cond[0]) return ntry;
00340     unsigned i=0;
00341
00342     if (cond[0]=='?' ) {
00343         { Wb::ompGuard gLK(CG_locks);
00344             const auto lp=CG::lock_map.find(h0);
00345             if (lp==CG::lock_map.end() || !lp->second->level) { ntry=0; }
00346             ++i;
00347         }
00348     }
00349     if (ntry && cond[i]) {
00350         if (isdigit(cond[i])) { sscanf(cond+i,"%d",&ntry); }
00351         else wblog(F_L,"ERR %s() invalid cond='%s'",FCT,cond);
00352     }
00353
00354     return ntry;
00355 };
00356 int CG::Guard::acquire(

```

```

00357     const char *F, int L, const CG::FileLock &flk,
00358     const char *tag, const char *cond
00359 ) {
00360     if (omp_get_num_threads() <= 1) { return 0; }
00361
00362     if (hid.len) wblog(FL,
00363         "ERR Guard() already initialized (len=%d)", hid.len);
00364
00365     size_t h0; unsigned l, l2, n=64;
00366     int ntry=-1; const char *sR; char s[n]; const char* ss[1]={s};
00367
00368     flk.ensureFOpen(FL);
00369
00370     sR=Wb::rstrstr(flk.fname.data, "/RCS");
00371     if (sR) { ++sR; }
00372     else { const char *s0=flk.fname.data;
00373         sR=Wb::rstrstr(s0, "Store");
00374         if (sR>s0) { --sR; }
00375         else { sR=s0; }
00376     }
00377     l2=strlen(sR);
00378
00379     h0 = Wb::string_hash(tag ? tag : mytag)
00380         ^ Wb::string_hash(sR);
00381
00382     if (cond && cond[0]) {
00383         ntry=deal_with_cond(F, L, h0, cond);
00384         if (!ntry) { return 0; }
00385     }
00386
00387     if (tag && tag[0])
00388         { l=snprintf(s, n, "%s", shortFL(F_L, -1, tag, '*')); }
00389     else { l=snprintf(s, n, "%s", shortFL(F_L, -1, mytag, '/')); }
00390     for (; l<24; ++l) { s[l]=' '; }
00391
00392     if (l+l2+l>=n) wblog(FL,
00393         "ERR %s() string out of bounds (%d+%d+%d)\n%s %s", FCT, l, l2, n, s, sR);
00394     l+=snprintf(s+l, n-l, " %s", sR);
00395
00396     hid.init(l, &h0);
00397
00398     return acquire_set(ss, ntry);
00399 };
00400
00401 template<class TQ>
00402 int CG::Guard::acquire(
00403     const char *F, int L, const QSet<TQ> &Q, const char *tag,
00404     const char *cond
00405 ) {
00406     if (omp_get_num_threads() <= 1) { return 0; }
00407
00408     size_t h0=QHash<TQ>() (Q, tag);
00409
00410     int ntry=-1;
00411     if (cond && cond[0]) {
00412         ntry=deal_with_cond(F, L, h0, cond); if (!ntry) return 0; }
00413
00414     unsigned l, n=64; char s[n]; const char* ss[1]={s};
00415     if (hid.len) wblog(FL,
00416         "ERR Guard() already initialized (len=%d)", hid.len);
00417
00418     if (tag && tag[0])
00419         { l=snprintf(s, n, "%s", shortFL(F_L, -1, tag, '*')); }
00420     else { l=snprintf(s, n, "%s", shortFL(F_L, -1, mytag, '/')); }
00421
00422     if (l<n) {
00423         for (; l<24; ++l) { s[l]=' '; }
00424         l+=snprintf(s+l, n-l, " %s", STR(Q));
00425     }
00426     if (l>=n) wblog(FL,
00427         "ERR %s() string out of bounds (%d/%d)\n%s", FCT, l, n, s);
00428
00429     hid.init(l, &h0);
00430
00431     return acquire_set(ss, ntry);
00432 };
00433
00434 template<class TM>
00435 int CG::Guard::acquire(
00436     const char *F, int L, const cgc_contract_id<TM> &idc,
00437     const char *cond
00438 ) {
00439     if (omp_get_num_threads() <= 1) return 0;
00440
00441     int ntry=-1; size_t h0=MHash<TM>() (idc);
00442     if (cond && cond[0]) {
00443         ntry=deal_with_cond(F, L, h0, cond); if (!ntry) return 0;

```

```

00444     }
00445
00446     unsigned n=128; char s[n]; const char* ss[1]={s};
00447     if (hid.len) wblog(FL,
00448         "ERR Guard() already initialized (len=%d)",hid.len);
00449
00450     snprintf(s,n,"%-20s %s",
00451         shortFL(F_L,-1,mytag,'/'),
00452         STR2(idc,0)
00453     );
00454
00455     hid.init(1,&h0);
00456
00457     return acquire_set(ss,ntry);
00458 };
00459
00460 template<class TQ>
00461 void CG::Guard::acquire(
00462     const char *F, int L, const QSet<TQ> *Q[], unsigned n
00463 ){
00464     if (omp_get_num_threads()<=1) return;
00465
00466     unsigned i=0, k=0, slen=64; char* ss[n];
00467     if (hid.len) wblog(FL,
00468         "ERR Guard() already initialized (len=%d)",hid.len);
00469     hid.init(n);
00470
00471     for (i=0; i<n; ++i) { if (Q[i]) {
00472         if (!Q[i]->isEmpty()) {
00473             hid[k]=QHash<TQ>() (*Q[i]);
00474             ss[k] = new char[slen];
00475             snprintf(ss[k++],slen,
00476                 "%-20s %s",shortFL(F_L,-1,mytag,'/'), STR_(Q[i])
00477             );
00478         }
00479         else wblog(FL,"WRN %s() got empty QSet (%d/%d)",FCT,i+1,n);
00480     }}
00481
00482     if (!k) wblog(FL,
00483         "ERR Guard() got no relevant QSet data (%d/%d)",k,n);
00484     hid.len=k; for (; k<n; ++k) { ss[k]=NULL; }
00485
00486     acquire_set((const char**)ss);
00487
00488     for (i=0; i<n; ++i) { if (ss[i]) { delete [] ss[i]; }}
00489 };
00490
00491 int CG::Guard::acquire_set_iter(
00492     int *got, Wb::ompNLock *lk[], const char *ss[], unsigned iter
00493 ){
00494
00495     int nlks=0;
00496     unsigned i=0, gotnew;
00497
00498     { Wb::ompGuard gLK(CG_locks);
00499     for (; i<hid.len; ++i) {
00500         Wb::ompNLock *lp=CG::lock_map[hid[i]];
00501         if ((gotnew=(lp ? 0 : 1))) {
00502             lp = new Wb::ompNLock(ss[i]);
00503         }
00504         lk[i]=lp;
00505
00506         if (!iter) { ++(lp->nactive); }
00507         got[i] = lp->test_acquire();
00508
00509         if (got[i]) { ++nlks; }
00510
00511         #if defined(DBG_GCX_LOCKS) && ( DBG_GCX_LOCKS & 8 )
00512         if (got[i] || gotnew) {
00513             LKF.bblogf(FL, gotnew? "add %s":" + %s", STR2_(lk[i],ss[i]));
00514         }
00515         #endif
00516     } }
00517
00518     if ((unsigned)nlks<hid.len) {
00519         for (i=0; i<hid.len; ++i) { if (got[i]) lk[i]->release(); }}
00520
00521     return ((unsigned)nlks>=hid.len ? nlks: nlks-hid.len);
00522 };
00523
00524 int CG::Guard::acquire_set_wait(
00525     int *got, Wb::ompNLock *lk[], const char *ss[], int ntry) {
00526
00527     int nlks=0; unsigned i, missed=0, ndead=0, nlog=0;
00528     int iter=1;
00529
00530     double wt=-1,

```

```

00531     wt1=60,
00532     wt2=600;
00533
00534     if (ntry<=0) {
00535         if (!ntry) wblog(FL,"WRN %s() got ntry=%d !?",FCT,ntry);
00536         ntry=1200;
00537     }
00538
00539     Wb::thread_xlink xl;
00540
00541     for (; iter<ntry; ++iter) {
00542         nlks=acquire_set_iter(got,lk,ss,iter);
00543
00544         missed=(nlks<=0 ? 1 : 0);
00545         if (!missed) break;
00546
00547         xl.init(FL);
00548         for (i=0; i<hid.len; ++i) { if (!got[i]) xl.add(FL,lk[i]); }
00549
00550         if (wt<=0) {
00551             wt=0.01*std::rand()/double(RAND_MAX); if (wt<1E-4) wt=1E-4; }
00552         else {
00553             double x=1+std::rand()/double(RAND_MAX);
00554             if ((wt*x)>wt2) { wt=wt2; }
00555         }
00556
00557         i=Wb::thread_xlink::check_deadlocks();
00558         ndead=(i? ndead+1 : 0);
00559
00560         if (wt>wt1 || (i && wt>1)) {
00561             Wb::termcolor Tc("WRN");
00562             wbststring sout; sprintf_locks(sout,got,ss, " ");
00563
00564             fprintf(stdout,"\n%-12s %s "
00565                 "itry %d/%d @ dt=%4.3gs %s(ith=%d/%d)%s\n%s",
00566                 shortFLT, iter,ntry,wt, Tc.el,xl.t0,xl.nt,Tc.em, sout.data);
00567
00568             unsigned l, n=64; char sx[n];
00569             l=snprintf(sx,n,"%d/%d OMP deadlocks? ",i,ndead);
00570             snprintf(sx+l,n-l,"(ith=%d/%d)",xl.t0,xl.nt);
00571
00572             if (ndead) { char xflag=(wt>wt1 && ndead>10);
00573                 if (++nlog==1 || xflag) {
00574                     Wb::termcolor Tc("WRN");
00575                     fprintf(stdout,
00576                         "%s %s %sWRN %d deadlocks?%s (%ld locks total)\n\n",
00577                         shortFLT, Tc.el,i,Tc.em, CG::lock_map.size());
00578                     print_CG_locks();
00579                 }
00580
00581                 if (xflag)
00582                     { wblog(FL,"ERR %s %.3g, %d",sx,wt,ndead); }
00583                 else { wblog(FL,"WRN %s",sx); }
00584             }
00585             else wblog(FL,
00586                 "WRN waiting for OMP lock%ss %s", hid.len==1? "":"s",sx+l);
00587         }
00588         Wb::pause(wt);
00589     }
00590
00591     if ((missed && ntry>99)
00592         #if defined(DBG_GCX_LOCKS) && ( DBG_GCX_LOCKS & 8 )
00593         || (iter>1)
00594         #endif
00595     ) {
00596         char sx[64]; wbststring sout; sprintf_locks(sout,got,ss);
00597
00598         snprintf(sx,64,"%ld lock%ss (%d %s @ %.3g; %d/%d)", hid.len,
00599             hid.len==1?"":"s",iter,iter!=1?"tries":"try",wt,xl.t0,xl.nt);
00600
00601         #if defined(DBG_GCX_LOCKS) && ( DBG_GCX_LOCKS & 8 )
00602         LKF.blogf(FL, missed? "ERR %s\n%s":"ok. %s\n%s",sx,sout.data);
00603         #endif
00604
00605         if (missed) fprintf(stderr,"%-12s %s "
00606             "ERR failed to acquire %s\n%s",shortFLT,sx,sout.data);
00607     }
00608
00609     return nlks;
00610 };
00611
00612 void CG::Guard::release() {
00613     if (!hid.len) { active=false; return; }
00614
00615     unsigned i=0; char err[hid.len];
00616
00617     map <size_t, Wb::ompNLock*>::iterator it;

```

```

00618     Wb::ompNLock *lp;
00619 { Wb::ompGuard gLK(CG_locks);
00620
00621     for (i=0; i<hid.len; ++i) { err[i]=0;
00622         it=CG::lock_map.find(hid[i]);
00623         if (it!=CG::lock_map.end()) { lp=it->second;
00624             if (lp) {
00625                 if (active) { lp->release(); }
00626                 else if (lp->owner==omp_get_thread_num()) { err[i]=1; }
00627
00628                 if ((--lp->nactive)<=0) {
00629                     #if defined(DBG_GCX_LOCKS) && ( DBG_GCX_LOCKS & 8 )
00630                         LKF.blogf(FL,"rm  %s",STR_(lp));
00631                     #endif
00632
00633                     delete lp; CG::lock_map.erase(it);
00634                 }
00635                 } else err[i]=2;
00636             } else err[i]=4;
00637         }
00638
00639     for (i=0; i<hid.len; ++i) { if (err[i]) { if (i) { err[0]=err[i]; }
00640         if (err[i] & 1) wblog(FL,"ERR %s() "
00641             "still owner of released lock %d/%d #%lx",FCT,i+1,hid.len,hid[i]);
00642         else if (err[i] & 2) wblog(FL,"ERR %s() "
00643             "got null lock %d/%d #%lx",FCT,i+1,hid.len,hid[i]);
00644         else if (err[i] & 4) wblog(FL,"ERR %s() "
00645             "lock %d/%d #%lx no longer exists !?",FCT,i+1,hid.len,hid[i]);
00646     }; if (err[0]) fflush(0);
00647
00648     hid.init(); active=false;
00649 };
00650
00651 void CG::Guard::print(const char *F, int L) {
00652
00653     if (!hid.len) {
00654         wblog(F_L," *  %s() no locks in place (#hid=%d)",FCT,hid.len);
00655         return;
00656     }
00657
00658     if (hid.len==1)
00659         wblog(F_L," *  CG::Guard() got single lock entry");
00660     else wblog(F_L," *  CG::Guard() contains %d locks",hid.len,hid.len);
00661
00662     Wb::ompGuard gLK(CG_locks);
00663     for (unsigned i=0; i<hid.len; ++i) {
00664         auto it=CG::lock_map.find(hid[i]);
00665         wblog(FL," *  %d. %-24s #%lx",i+1,
00666             it==CG::lock_map.end() ? "(not in locks)" :
00667             (it->second ? it->second->istr : "(null)"), hid[i]
00668         );
00669     }
00670 };
00671
00672 int Wb::thread_xlink::add(const char *F, int L, size_t h) {
00673     const auto im = CG::lock_map.find(h);
00674     if (im==CG::lock_map.end()) wblog(FL,
00675         "ERR %s() non existing lock CG::lock_map[%x]",FCT,h);
00676     return add(F_L,im->second);
00677 };
00678
00679 #endif
00680 namespace Wb {
00681
00682 int thread_xlink::add(const char *F, int L, const ompNLock *lk) {
00683
00684     unsigned t2=0, n=8*sizeof(tx); size_t b=1;
00685
00686     assert(lk!=NULL);
00687
00688     t2=lk->owner;
00689     if (int(t2)<0) {
00690         return (lk->level<=0 ? -1 : 0);
00691     }
00692
00693     if (t0>=n || t2>=n) wblog(F_L,
00694         "ERR %s() thread ID out of bounds %d/%d/%d/%d\n%s",
00695         FCT,t0,t2,nt,n,STR_(lk));
00696
00697     if (tx & (b<=t2)) { return 0; }
00698     else { tx|=b; return 1; }
00699 };
00700
00701 int check_deadlock_rec(unsigned i0, size_t tt, const thread_xlink *x) {
00702
00703     unsigned r=0, i=0; size_t b=1, q=x->tx;
00704

```

```

00705     for (; q && !r; ++i, b<<=1, q>>=1) { if (q & 1) {
00706         if (tt & b) {
00707             if (i==i0) ++r;
00708         }
00709         else {
00710             const auto it=txmap.find(i);
00711             if (it!=txmap.end()) {
00712                 r+=check_deadlock_rec(i0, tt | b, it->second);
00713             }
00714         }
00715     }}
00716
00717     return r;
00718 };
00719
00720 int thread_xlink::check_deadlocks() {
00721
00722     unsigned i, r=0, n=8*sizeof(size_t);
00723
00724     #pragma omp critical (thread_xlink__)
00725     for (auto it=txmap.begin(); it!=txmap.end(); ++it) {
00726         if ((i=it->first)<n)
00727             r+=check_deadlock_rec(i, (size_t(1)<<i, it->second);
00728         else wblog(FL,
00729             "WRN %s() thread ID out of bounds (%d/%d)",FCT,i,n);
00730     }
00731
00732     return r;
00733 };
00734
00735 };
00736
00737 #endif
00738
00739 #endif
00740

```

5.52 wbomp.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace OMP routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_OMP_HH__
00021 #define __WB_OMP_HH__
00022
00023 #ifdef QS_USING_OMP
00024
00025 namespace Wb {
00026
00027     wstring ompID2Str(char vflag=0);
00028
00029     int ompStatus(const char *F, int L);
00030
00031     class ompNLock {
00032     public:
00033
00034         ompNLock(const char *s=NULL)
00035             : level(0), owner(-1), istr(0), nactive(0), ntot(0) {
00036             omp_init_nest_lock(&_nlk);
00037             if (s && s[0]) { set_istr(s); }
00038         };
00039
00040         ~ompNLock() {
00041             if (level || nactive) {
00042                 wblog(FL, "WRN %s() got lock @ l=%d, n=%d (id=%d/%d) !?",
00043                     FCT, level, nactive, omp_get_thread_num(), omp_get_num_threads());
00044             }
00045         }
00046     };
00047
00048 }
00049
00050 #endif
00051

```

```

00045         if (level==1 && !nactive) { release(); }
00046     }
00047     if (istr) { delete [] istr; istr=0; }
00048     owner=-1;
00049     omp_destroy_nest_lock(&_nlk);
00050 };
00051
00052 void set_istr(const char *s);
00053
00054 wbststring toStr(const char *istr=NULL) const;
00055
00056 wbststring get_istr() const;
00057
00058 int acquire();
00059
00060 int test_acquire();
00061
00062 int release();
00063
00064 int level;
00065 int owner;
00066
00067 char *istr;
00068
00069 int nactive;
00070
00071 long ntot;
00072
00073 private:
00074
00075     omp_nest_lock_t _nlk;
00076
00077     ompNLock(const ompNLock &);
00078     void operator=(const ompNLock &);
00079
00080 #ifdef LOAD_CGC_QSPACE
00081     wbststring check_BUF_size() const;
00082 #endif
00083 };
00084
00085 class ompGuard {
00086 public:
00087     ompGuard() : lp(0) { };
00088
00089     ompGuard(const char *F, int L, ompNLock &lk, int lmax=99)
00090         : lp(0) {
00091         if (omp_get_num_threads()>1) {
00092             acquire(F,L,lk,lmax);
00093         }
00094     };
00095
00096     ompGuard(ompNLock &lk, int lmax=99) : lp(0) {
00097         if (omp_get_num_threads()>1) {
00098             acquire(0,0,lk,lmax);
00099         }
00100     };
00101
00102     void acquire(ompNLock &lk, int lmax=99) { acquire(0,0,lk,lmax); };
00103     void acquire(const char *F, int L, ompNLock &lk, int lmax=99);
00104
00105     ~ompGuard() { if (lp) { lp->release(); } };
00106
00107     int level() { return (lp ? lp->level : -1); };
00108
00109 private:
00110
00111     ompNLock *lp;
00112
00113     ompGuard(const ompGuard &) {
00114         wblog(FL,"ERR %s() forbidden copy",FCT); };
00115     void operator=(const ompGuard &) {
00116         wblog(FL,"ERR %s() forbidden copy",FCT); };
00117 };
00118
00119 class nthreads_aux {
00120 public:
00121     nthreads_aux() : np_curr(0), np_max(0) { };
00122
00123     nthreads_aux& increment(const char *f1) {
00124         int q=omp_get_num_threads();
00125         if (np_max>q) {
00126             Wb::termcolor Tc(1);
00127             fprintf(stdout,"TST %s%s() having nq=%d->%d%s\n",
00128                 Tc.el,f1?" f1:" (null)",np_max,q,Tc.em);
00129             np_max=q;
00130         }
00131     };

```

```

00132     }
00133     if (np_max<(++np_curr)) {
00134         Wb::termcolor Tc(5);
00135         fprintf(stdout,"TST %s%s() having nq=%d->%d%s\n",
00136             Tc.el,fl? fl:"(null)",np_max,np_curr,Tc.em);
00137         np_max=np_curr;
00138     }
00139     return *this;
00140 };
00141
00142 nthreads_aux& decrement(const char *fl) {
00143     if (--np_curr<0) { Wb::termcolor Tc(5);
00144         fprintf(stdout,"%sWRN %s() having nq=%d/%d%s\n",
00145             Tc.el,fl? fl:"(null)",np_curr,np_max,Tc.em);
00146     }
00147     return *this;
00148 };
00149
00150 private:
00151     int np_curr, np_max;
00152 };
00153
00154 map< std::string, nthreads_aux > np_hist;
00155
00156 class checkMaxThreads {
00157 public:
00158
00159     checkMaxThreads(const char *F, int L) {
00160         std::string fl=shortFL(F,L);
00161         #pragma omp critical (checkMaxThreads__)
00162         { np_hist[fl].increment(fl.c_str()); im=np_hist.find(fl); }
00163     };
00164
00165     ~checkMaxThreads() {
00166         #pragma omp critical (checkMaxThreads__)
00167         { im->second.decrement(im->first.c_str()); }
00168     };
00169
00170 private:
00171
00172     map< std::string, nthreads_aux >::iterator im;
00173 };
00174
00175 class thread_xlink;
00176 map< int, thread_xlink* > txmap;
00177
00178 class thread_xlink {
00179 public:
00180
00181     thread_xlink(const char *F=NULL, int L=0) : tx(0) {
00182         t0=omp_get_thread_num();
00183         nt=omp_get_num_threads();
00184
00185         if (F) { fl=shortFL(F,L); }
00186     };
00187
00188     ~thread_xlink() {
00189         auto it = txmap.find(t0);
00190         if (it!=txmap.end()) {
00191             #pragma omp critical (thread_xlink__)
00192             txmap.erase(t0);
00193         }
00194     };
00195
00196     void init(const char *F=NULL, int L=0) {
00197         if (nt<=1) {
00198             t0=omp_get_thread_num();
00199             nt=omp_get_num_threads();
00200         }
00201         else {
00202             unsigned t2=omp_get_thread_num(); if (t2!=t0) {
00203                 wblog(F_L,"WRN %s() thread-id changed %d->%d / %d->%d",
00204                     FCT, t0,t2,nt,omp_get_num_threads());
00205             }
00206         }
00207
00208         if (!tx) { if (F || fl.empty()) fl=shortFL(F,L);
00209             #pragma omp critical (thread_xlink__)
00210             txmap[t0]=this;
00211         }
00212         tx=(1<t0);
00213     };
00214
00215     int add(const char *F, int L, size_t h);
00216     int add(const char *F, int L, const ompNLock *lk);
00217
00218     static int check_deadlocks();

```



```

00219
00220     unsigned t0, nt;
00221
00222     size_t tx;
00223     std::string fl;
00224
00225 private:
00226 };
00227
00228 };
00229
00230 Wb::ompNLock wblog_lock("wblog");
00231 Wb::ompNLock rclog_lock("rclog");
00232 Wb::ompNLock mxapi_lock("mxapi");
00233
00234 Wb::ompNLock XS_buf("XS_buf");
00235
00236 Wb::ompNLock CS_buf("CS_buf");
00237
00238 Wb::ompNLock CG_locks("CG_locks");
00239
00240 #else
00241 #endif
00242
00243 #endif
00244

```

5.53 wbopts.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace MEX routine options class
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_OPTS_HCC__
00021 #define __WB_OPTS_HCC__
00022
00023 /* ----- */
00024 /* ----- */
00025 /* class to manage optional input arguments */
00026
00027 class OPTS {
00028
00029 public:
00030
00031     OPTS () : mto(0) {};
00032
00033     OPTS (const mxArray** ain, const unsigned n)
00034     : mto(0) { init(ain,n); };
00035
00036     OPTS (const char *fname)
00037     : mto(0) { init(fname); };
00038
00039     void init (const mxArray** ain, const unsigned n);
00040     void init (const char *fname);
00041
00042     ~OPTS () { init(); }
00043     void init() {
00044         name.init();
00045
00046         if (MAT.mfp && aa.len)
00047             for (unsigned i=0; i<aa.len; ++i) mxDestroyArray(aa[i]);
00048
00049         aa.init();
00050     };
00051
00052     int len() { return ( MAT.mfp ? -1 : (int)aa.len ); };
00053
00054     template <class T>

```

```

00055     bool getOpt(const char*, int,
00056                const char* vn, T &x, char force=0, const char *istr=0);
00057
00058     template <class T>
00059     bool getOpt(const char*, int,
00060                const char* vn, wbvector<T> &x, char force=0, const char *istr=0);
00061
00062     template <class T>
00063     bool getOpt(const char*, int,
00064                const char* vn, wbMatrix<T> &x, char force=0, const char *istr=0);
00065
00066     bool getOpt(const char*, int, const char* vn, const char *istr=0);
00067
00068     template <class T>
00069     bool getOpt(const char* vn, T &x, char force=0, const char *istr=0) {
00070         return getOpt(NULL,0, vn,x,force,istr);
00071     };
00072
00073     template <class T>
00074     bool getOpt(const char* vn,
00075                wbvector<T> &x, char force=0, const char *istr=0) {
00076         return getOpt(NULL,0,x,force,istr);
00077     };
00078
00079     template <class T>
00080     bool getOpt(const char* vn,
00081                wbMatrix<T> &x, char force=0, const char *istr=0) {
00082         return getOpt(NULL,0,x,force,istr);
00083     };
00084
00085     bool getOpt(const char* vn, const char *istr=0) {
00086         return getOpt(NULL,0,vn,istr);
00087     };
00088
00089     void checkAnyLeft(const char *MAT=NULL, int L=0);
00090
00091 protected:
00092 private:
00093
00094     wbvector<mxArray*> aa;
00095     wbvector<wbstring> name;
00096
00097     unsigned mto;
00098
00099     Wb::matFile MAT;
00100
00101     int findOpt(const char* vn);
00102 };
00103
00104 int OPTS::findOpt(const char* vn) {
00105
00106     unsigned i, n=name.len;
00107     int k=-1;
00108
00109     for (i=0; i<n; ++i)
00110         if (name[i].len) if (!strcmp(name[i].data,vn)) {
00111             if (k<0) k=i; else ++mto;
00112         };
00113
00114     return k;
00115 }
00116
00117 void OPTS::init(const mxArray** ain, const unsigned l) {
00118
00119     unsigned n=128; char s[n];
00120     if (MAT.mfp) wblog(FL,
00121         "ERR OPTS already initialized by file\n%s",MAT.fname.data);
00122
00123     aa.init(l, (mxArray**)ain);
00124     name.init(l);
00125
00126     for (unsigned i=0; i<aa.len; ++i) {
00127         if (!mxIsChar(aa[i])) { continue; } else
00128             if (mxGetString(aa[i],s,n)) { wblog(FL,
00129                 "WRN failed to read OPTS string (len<%d?)",n);
00130                 continue;
00131             }
00132
00133         name[i].init(s);
00134         aa[i]=NULL;
00135     }
00136 };
00137
00138 void OPTS::init (const char *fname) {
00139
00140     if (aa.len) wblog(FL,
00141         "ERR OPTS already set by mxArray[] (n=%d)\ngot %s",aa.len,fname);

```

```

00142
00143     MAT.Open(FL, fname, "r");
00144 };
00145
00146 bool OPTS::getOpt(
00147     const char *F, int L, const char *vn, const char *istr
00148 ){
00149     unsigned r=0;
00150
00151     if (MAT.mfp) {
00152         mxArray *a = matGetVariable(MAT.mfp, vn); r=(a!=NULL);
00153         if (a) {
00154             double dbl=0.;
00155             if (mxGetNumber(a,dbl)) { wblog(FL,"ERR %s",str); }
00156             if (dbl==0) r=0;
00157         }
00158         mxDestroyArray(a);
00159     }
00160     else {
00161         int i=findOpt(vn); r=(i>=0);
00162         if (r) name[i].init();
00163     }
00164
00165     if (r && F) {
00166         if (istr && istr[0])
00167             { wblog(F,L," * %s",istr); }
00168         else { wblog(F,L," * %s",vn); }
00169     }
00170
00171     return r;
00172 };
00173
00174 template <class T>
00175 bool OPTS::getOpt(
00176     const char *F, int L, const char* vn, T &x0,
00177     char force, const char *istr
00178 ){
00179     unsigned k,e=0;
00180     mxArray *a; T x=0;
00181
00182     if (MAT.mfp) {
00183         a = matGetVariable(MAT.mfp, vn);
00184         if (!a) {
00185             if (force) wblog(F_L,
00186                 "ERR %s() option '%s' required (%s)",FCT,vn, istr?istr:"");
00187             return 0;
00188         }
00189     }
00190     else {
00191         int i=findOpt(vn);
00192         if (i<0) {
00193             if (force) wblog(F_L,
00194                 "ERR %s() option '%s' required (%s)",FCT,vn, istr?istr:"");
00195             return 0;
00196         }
00197
00198         k=(unsigned) (i+1);
00199
00200         if (k>=name.len) { ++e; } else
00201         if (aa[k]==NULL) { ++e; }
00202         if (e) wblog(F_L,"ERR %s() missing value for '%s'",FCT,vn);
00203
00204         a=aa[k];
00205
00206         name[k-1].init(); aa[k]=NULL;
00207     }
00208
00209     if (mxGetNumber(a,x)) wblog(F_L,
00210         "ERR %s() failed to read value for '%s'\n%s",FCT,vn,str);
00211
00212     if (F && x0!=x) {
00213         if (istr && istr[0])
00214             { wblog(F,L," * %-8s: %s [%s]", vn, NSTR(x), istr); }
00215         else { wblog(F,L," * %-8s: %s", vn, NSTR(x)); }
00216     }
00217
00218     x0=x;
00219
00220     if (MAT.mfp) mxDestroyArray(a);
00221
00222     return 1;
00223 };
00224
00225 template<>
00226 bool OPTS::getOpt(const char *F, int L,
00227     const char* vn,
00228     wstring &x, char force, const char *istr

```

```

00229 ){
00230     int lflag=0;
00231     if (MAT.mfp) { unsigned n=128; char s[n]; s[0]=0;
00232         mxArray *a = matGetVariable(MAT.mfp, vn);
00233         if (!a) {
00234             if (force) wblog(F_L,"ERR %s() missing input for '%s'",FCT,vn);
00235             return 0;
00236         }
00237
00238         if (!mxIsChar(a)) wblog(F_L,
00239             "ERR %s() invalid type '%s' for '%s'",FCT,mxGetClassName(a),vn);
00240         if (mxGetString(a,s,n)) wblog(F_L,
00241             "ERR %s() failed to read string for '%s' (len<%d?)",FCT,vn,n);
00242
00243         if (x.data && !strcmp(x.data,s)) { ++lflag; }
00244         x=s; mxDestroyArray(a);
00245     }
00246     else {
00247         int i; unsigned k,e=0;
00248
00249         i=findOpt(vn);
00250         if (i<0) {
00251             if (force) wblog(F_L,"ERR %s() option '%s' required",FCT,vn);
00252             return 0;
00253         }
00254         k=unsigned(i+1);
00255
00256         if (k>=name.len) { ++e; } else
00257         if (aa[k]!=NULL) { ++e; }
00258         if (e) wblog(F_L,"ERR %s() missing value for option '%s'",FCT,vn);
00259
00260         if (x.data && !strcmp(x.data,name[k].data)) ++lflag;
00261         x=name[k];
00262
00263         name[k-1].init(); name[k].init();
00264     }
00265
00266     if (F && lflag) {
00267         if (istr && istr[0])
00268             { wblog(F,L," * %-8s: %s [%s]",vn, x.data, istr); }
00269         else { wblog(F,L," * %-8s: %s", vn, x.data); }
00270     }
00271
00272     return 1;
00273 };
00274
00275 template<>
00276 bool OPTS::getOpt(const char *F, int L,
00277     const char* vn, mxArray* &a, char force, const char *istr
00278 ){
00279     if (MAT.mfp) {
00280         a=matGetVariable(MAT.mfp, vn);
00281         if (a) aa.Append((mxArray*)a);
00282     }
00283     else {
00284         int i=findOpt(vn);
00285
00286         if (i<0) a=NULL;
00287         else {
00288             unsigned e=0, k=unsigned(i+1);
00289
00290             if (k>=name.len) { ++e; } else
00291             if (aa[k]==NULL) { ++e; }
00292             if (e) wblog(F_L,"ERR %s() value missing for '%s'",FCT,vn);
00293
00294             a=aa[k];
00295
00296             name[k-1].init(); aa[k]=NULL;
00297         }
00298     }
00299
00300     if (!a) {
00301         if (force) wblog(F_L,"ERR %s() missing value for '%s'",FCT,vn);
00302         return 0;
00303     }
00304
00305     if (F) {
00306         if (istr && istr[0])
00307             { wblog(F,L," * %-8s: (%s) [%s]",vn, mxGetClassName(a), istr); }
00308         else { wblog(F,L," * %-8s: (%s)", vn, mxGetClassName(a)); }
00309     }
00310
00311     return 1;
00312 };
00313
00314 template<>
00315 bool OPTS::getOpt(

```

```

00316     const char *F __attribute__ ((unused)),
00317     int L __attribute__ ((unused)),
00318     const char* vn, itag_ &t, char force, const char *istr
00319 ){
00320     wbsttring q;
00321     getOpt(vn,q,force,istr);
00322     if (q) { t.init(0,0,q.data); return 1; }
00323     return 0;
00324 };
00325
00326 template<class T>
00327 bool OPTS::getOpt(const char *F, int L,
00328     const char* vn, wvector<T>&x_, char force, const char *istr
00329 ){
00330     int lflag=0;
00331     if (MAT.mfp) {
00332         mxArray *a = matGetVariable(MAT.mfp, vn);
00333         if (!a) {
00334             if (force) wblog(F_L,"ERR %s() missing input for '%s'",FCT,vn);
00335             return 0;
00336         }
00337
00338         if (!mxIsDouble(a)) wblog(F_L,
00339             "ERR %s() invalid type `%s' for '%s'",FCT,mxGetClassName(a),vn);
00340
00341         wvector<T> x; x.init(FL,a);
00342         if (x!=x_) { x.save2(x_); ++lflag; }
00343         mxDestroyArray(a);
00344     }
00345     else {
00346         int i; unsigned k,e=0;
00347
00348         i=findOpt(vn);
00349         if (i<0) {
00350             if (force) { wblog(F_L,"ERR %s() missing input %s",FCT,vn); }
00351             return 0;
00352         }
00353         k=unsigned(i+1);
00354
00355         if (k>=name.len) { ++e; } else
00356         if (aa[k]==NULL) { ++e; }
00357         if (e) wblog(F_L,"ERR %s() missing value for '%s' (vector)",FCT,vn);
00358
00359         wvector<T> x; x.init(FL,aa[k]);
00360         if (x!=x_) { x.save2(x_); ++lflag; }
00361
00362         name[k-1].init(); aa[k]=NULL;
00363     }
00364
00365     if (F && lflag) {
00366         if (istr && istr[0])
00367             { wblog(F,L," * %-8s: [%s] (%s)", vn, x_.toStr().data, istr); }
00368         else { wblog(F,L," * %-8s: [%s]", vn, x_.toStr().data); }
00369     }
00370
00371     return 1;
00372 }
00373
00374 template<class T>
00375 bool OPTS::getOpt(const char *F, int L,
00376     const char* vn, wMatrix<T>&x_, char force, const char *istr
00377 ){
00378     int lflag=0;
00379     if (MAT.mfp) {
00380         mxArray *a = matGetVariable(MAT.mfp, vn);
00381         if (!a) {
00382             if (force) wblog(F_L,
00383                 "ERR %s() missing value for %s (matrix)",FCT,vn);
00384             return 0;
00385         }
00386
00387         if (!mxIsDouble(a)) wblog(F_L,
00388             "ERR %s() invalid type `%s' for '%s'",FCT,mxGetClassName(a),vn);
00389
00390         wMatrix<T> x; x.init(FL,a);
00391         if (x!=x_) { x.save2(x_); ++lflag; }
00392         mxDestroyArray(a);
00393     }
00394     else {
00395         int i; unsigned k,e=0;
00396
00397         i=findOpt(vn);
00398         if (i<0) {
00399             if (force) wblog(F_L,
00400                 "ERR %s() missing input %s (matrix)",FCT,vn);
00401             return 0;
00402         }

```

```

00403     k=unsigned(i+1);
00404
00405     if (k>=name.len) { ++e; } else
00406     if (aa[k]==NULL) { ++e; }
00407     if (e) wblog(F_L,
00408         "ERR %s() missing input data for '%s' (matrix)",FCT,vn);
00409
00410     wbMatrix<T> x; x.init(FL,aa[k]);
00411     if (x!=x_) { x.save2(x_); ++lflag; }
00412
00413     name[k-1].init(); aa[k]=NULL;
00414 }
00415
00416 if (F && lflag) {
00417     if (istr && istr[0])
00418         { wblog(F,L," * %-8s: [%s] (%s)", vn, x_.toStr().data, istr); }
00419     else { wblog(F,L," * %-8s: [%s]",      vn, x_.toStr().data); }
00420 }
00421
00422 return 1;
00423 }
00424
00425 void OPTS::checkAnyLeft(const char *F, int L) {
00426
00427     unsigned i,e=0;
00428
00429     if (MAT.mfp) {
00430         MAT.init();
00431         for (i=0; i<aa.len; ++i) mxDestroyArray(aa[i]);
00432         return;
00433     }
00434
00435     for (i=0; i<name.len; ++i) { if (name[i].len || aa[i]) {
00436         if ((++e)=1) printf("\n");
00437
00438         if (name[i].len) {
00439             printf("%5d: '%s'\n", i+1, name[i].data);
00440         }
00441         else if (aa[i]) {
00442             int m=mxGetM(aa[i]), n=mxGetN(aa[i]);
00443             if (mxIsCell(aa[i]))
00444                 printf("%5d: { %dx%d %s }\n", i+1,m,n,mxGetClassName(aa[i]));
00445             else printf("%5d: { %dx%d %s }\n", i+1,m,n,mxGetClassName(aa[i]));
00446         }
00447     }}
00448
00449     if (e) { PRINTF("\n"); if (mto) {
00450         wblog(F_L,"WRN some options appear more than once (m=%d)",mto); }
00451         wblog(F_L,"ERR unused/unrecognized options");
00452     }
00453 };
00454
00455 #endif
00456

```

5.54 wbpem.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace permute routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_PERM_CC__
00021 #define __WB_PERM_CC__
00022
00023 // ----- //
00024 // ----- //
00025
00026 char wbpem::isValidPerm(const char *F, int L, wperm_t r) const {

```

```

00027
00028     char rval=0, id=0;
00029
00030     if (swperm_t(r)>=0 && r!=len) { if (F) wblog(F,L,
00031         "ERR invalid permutation [%s; %d]",toStr().data,r);
00032         return rval;
00033     }
00034
00035     if (len) { wbvector<char> mark(len);
00036         for (wperm_t i=0; i<len; ++i) {
00037             if (i==data[i]) { ++id; } else
00038                 if (data[i]>=len || mark[data[i]]++) { if (F) wblog(F,L,
00039                     "ERR invalid permutation [%s]",toStr().data;
00040                     return rval;
00041                 }
00042         }
00043     }
00044     return (rval=( unsigned(id)==len? 2:1));
00045 };
00046
00047 wbperm& wbperm::init(const wbperm &P, char iflag, unsigned r) {
00048     if (&P!=this) {
00049         if (iflag)
00050             { P.invert(*this); }
00051         else { RENEW(P.len,P.data); }
00052     }
00053     else if (iflag) { Invert(); }
00054
00055     if (int(r)>=0) { Extend(r); }
00056
00057     return *this;
00058 };
00059
00060 wbperm& wbperm::invert(wbperm &iP) const {
00061     if (&iP==this) {
00062         wbperm X; this->invert(X).save2(iP);
00063         return iP;
00064     }
00065
00066     #ifndef SKIP_WB_ASSERT
00067     isValidPerm(FL);
00068     #endif
00069
00070     iP.RENEW(len);
00071
00072     for (wperm_t *p=iP.data, i=0; i<len; ++i) {
00073         if (data[i]>=len) wblog(FL,"ERR %s() "
00074             "permutation out of bounds (%d/%d)",FCT,data[i],len);
00075         p[data[i]]=i;
00076     }
00077     return iP;
00078 };
00079
00080 wbperm& wbperm::Extend(wperm_t r) {
00081
00082     if (r!=len) {
00083         if (int(r)<=0) wblog(FL,"ERR %s() requesting r=%d/%d",FCT,r,len);
00084
00085         if (r>len) {
00086             wbperm P(r);
00087             if (len) { memcpy(P.data,data,len*sizeof(wperm_t)); }
00088             P.save2(*this);
00089         }
00090         else {
00091             for (unsigned i=r; i<len; ++i) {
00092                 if (data[i]!=i) wblog(FL,
00093                     "ERR %s() invalid r=%d having %s",FCT,r,STR_(this));
00094             }
00095             Shorten2(r);
00096         }
00097     }
00098     return *this;
00099 };
00100
00101 wbperm& wbperm::Complete(wperm_t l) {
00102     if (l<len) {
00103         if (l) {
00104             unsigned i=0; wbvector<char> mark(len);
00105             for (; i<l; ++i) {
00106                 if (data[i]>=len) wblog(FL,"ERR %s() invalid partial perm\n"
00107                     "(index out of bounds i=%d: %d/%d)",FCT,i+1,data[i],len);
00108                 if (++mark[data[i]]!=1) wblog(FL,"ERR %s() "
00109                     "invalid partial perm (non-unique: %s)",FCT,STR_(this));
00110             }
00111             for (i=0; i<len; ++i) { if (!mark[i]) { data[l++]=i; } }
00112         }
00113         else { init(l=len); }
00114     }

```

```

00114     else if (l>len) wblog(FL,
00115         "ERR %s() invalid usage (l=%d/%d)",FCT,l,len);
00116
00117     return *this;
00118 };
00119
00120 int wbperm::initStr(
00121     const char *F, int L, const char *s, wperm_t offset
00122 ){
00123
00124     if (!s) { init(); return -1; }
00125     while (isspace(*s) && *s) ++s;
00126     if (!*s){ init(); return 0; }
00127
00128     unsigned i=0, n=0;
00129     while (isdigit(s[n])) ++n;
00130     for (i=n; isspace(s[i]); ++i) {};
00131
00132     if (!s[i]) { init(n);
00133         for (i=0; i<n; ++i) { data[i]=s[i]-'0'; }
00134         if (int(offset)>=0) {
00135             for (i=0; i<n; ++i) {
00136                 if (data[i]<offset) { if (F) wblog(FL,"ERR %s() "
00137                     "invalid offset ('%s': %d !?)",FCT,s,offset);
00138                     else return -2;
00139                 }
00140                 data[i]-=offset;
00141             }
00142         }
00143         else {
00144             char mark[n+1]; memset(mark,0,n+1);
00145             for (i=0; i<n; ++i) {
00146                 if (data[i]>n || ++mark[data[i]]!=1) { return -4; }
00147             }
00148             if (!mark[0]) {
00149                 for (i=0; i<n; ++i) { --data[i]; }
00150             }
00151             return len;
00152         }
00153     }
00154     else {
00155         if (isdigit(s[i]) || isspace(s[i]) || s[i]=='\'' || s[i]=='\"';') {
00156             i=Str2Idx(s,*this,offset);
00157         }
00158         else { i=-(i+1); }
00159
00160         if (int(i)<0) { if (F) wblog(FL,
00161             "ERR %s() invalid permutation (i=%d, offset=%d) "
00162             "%N%N s = '%s' %N",FCT,-i,offset,s);
00163             else return -3;
00164         }
00165     }
00166
00167     i=isValidPerm(F ? F : NULL,F ? L : 0);
00168
00169     return (i ? len : -4);
00170 };
00171
00172 wbperm& wbperm::init_trafo(const wbperm &p1, const wbperm &p2) {
00173
00174     if (!p1.len) {
00175         if (p2.len) return init(p2);
00176         else return init();
00177     }
00178     if (!p2.len) { return init(p1,'i'); }
00179
00180     if (p1.len!=p2.len) wblog(FL,
00181         "ERR %s() length mismatch (%d/%d)",FCT,p1.len,p2.len);
00182
00183     if (this==&p1 || this==&p2) {
00184         wbperm X; X.init_trafo(p1,p2);
00185         X.save2(*this); return *this;
00186     }
00187
00188     init(p1,'i').Permute(p2);
00189
00190     return *this;
00191 };
00192
00193 wbperm& wbperm::init2Front(const wbvector<wperm_t> &I, wperm_t N) {
00194
00195     wperm_t i,j;
00196     wbvector<char> mark(N); char *m=mark.data;
00197     if (I.len>N) wblog(FL,"ERR index too long (%d/%d)",I.len,N);
00198
00199     init(N);
00200     for (i=0; i<I.len; i++) { j=I[i];

```



```

00201         if (j>=N) wblog(FL,"ERR index out of bounds (%d/%d)",j,N);
00202         data[i]=j; if ((++m[j])>1) {
00203             wblog(FL,"ERR %s() index not unique (%d/%d)",FCT,j,N); }
00204     }
00205
00206     if (i<N)
00207     for (j=0; j<N; j++) { if (!m[j]) data[i++]=j; }
00208
00209     return *this;
00210 };
00211
00212 wbperm& wbperm::init2End(const wbvector<wperm_t> &I, wperm_t N) {
00213     wperm_t i,j, l=N-I.len;
00214     wbvector<char> mark(N); char *m=mark.data;
00215     if (I.len>N) wblog(FL,"ERR index too long (%d/%d)",I.len,N);
00216
00217     init(N);
00218     for (i=0; i<I.len; i++) { j=I[i];
00219         if (j>=N) wblog(FL,"ERR index out of bounds (%d/%d)",j,N);
00220         data[l+i]=j; if ((++m[j])>1) {
00221             wblog(FL,"ERR %s() index not unique (%d/%d)",FCT,j,N); }
00222     }
00223
00224     if (l)
00225     for (i=j=0; j<N; j++) { if (!m[j]) data[i++]=j; }
00226
00227     return *this;
00228 };
00229
00230 wbperm& wbperm::init2FrontB(wperm_t m, wperm_t N) {
00231
00232     if (!m || m>N) wblog(FL,
00233         "ERR %s() m out of bounds (%d/%d)",FCT,m,N);
00234
00235     wperm_t i,l=N-m; init(N);
00236
00237     for (i=0; i<m; ++i) { data[i]=i+l; }
00238     for ( ; i<N; ++i) { data[i]=i-m; }
00239
00240     return *this;
00241 };
00242
00243 wbperm& wbperm::init2EndB(wperm_t m, wperm_t N) {
00244
00245     if (!m || m>N) wblog(FL,
00246         "ERR %s() m out of bounds (%d/%d)",FCT,m,N);
00247
00248     wperm_t i,l=N-m; init(N);
00249
00250     for (i=0; i<l; ++i) { data[i]=i+m; }
00251     for ( ; i<N; ++i) { data[i]=i-l; }
00252
00253     return *this;
00254 };
00255
00256 wbperm& wbperm::init2Front(wperm_t k, wperm_t N) {
00257     if (k>=N) wblog(FL,
00258         "ERR %s() index out of bounds (%d/%d)",FCT,k+1,N);
00259     wperm_t i=0; RENEW(N); data[0]=k;
00260     for (; i<k; ++i) { data[i+1]=i; }; ++i;
00261     for (; i<N; ++i) { data[i ]=i; };
00262     return *this;
00263 };
00264
00265 wbperm& wbperm::init2End(wperm_t k, wperm_t N) {
00266     if (k>=N) wblog(FL,
00267         "ERR %s() index out of bounds (%d/%d)",FCT,k+1,N);
00268     wperm_t i=0; RENEW(N); data[N-1]=k;
00269     for (; i<k; ++i) { data[i ]=i; }; ++i;
00270     for (; i<N; ++i) { data[i-1]=i; };
00271     return *this;
00272 };
00273
00274 wbperm& wbperm::Cycle(wperm_t k1, wperm_t k2) {
00275     if (k1>=len || k2>=len) wblog(FL,
00276         "ERR %s() index out of bounds [%d %d]/%d !?",FCT,k1,k2,len);
00277     if (k1==k2) return *this;
00278
00279     wperm_t x1=data[k1], k=k1;
00280     if (k1<k2)
00281         { for (; k<k2; ++k) { data[k]=data[k+1]; }}
00282     else { for (; k>k2; --k) { data[k]=data[k-1]; }}
00283     data[k]=x1;
00284
00285     return *this;
00286 };
00287

```

```

00288 wperm_t wperm::initCycle(wperm_t r, wperm_t k1, wperm_t k2) {
00289
00290     if (k1>=r || k2>=r) wblog(FL,
00291 "ERR %s() index out of bounds [%d %d]/%d !?",FCT,k1,k2,len);
00292     if (k1==k2) { return Index(r); }
00293
00294     RENEW(r);
00295
00296     if (k1<k2) { wperm_t k=0;
00297         for (; k<k1; ++k) { data[k]=k; }
00298         for (; k<k2; ++k) { data[k]=k+1; }; data[k++]=k1;
00299         for (; k<r; ++k) { data[k]=k; }
00300     }
00301     else { wperm_t k=r-1;
00302         for (; k>k1; --k) { data[k]=k; }
00303         for (; k>k2; --k) { data[k]=k-1; }; data[k--]=k1;
00304         for (; k<r; --k) { data[k]=k; }
00305     }
00306
00307     return *this;
00308 };
00309
00310 wperm_t wperm::Rotate(swperm_t k) {
00311
00312     if (!len) wblog(FL,
00313 "ERR %s() got empty permutation %d/%d !?",FCT,k,len);
00314     if ((k%len)<0) k+=len;
00315     if (k) {
00316         wperm P0(*this);
00317         for (wperm_t i=0; i<len; ++i) { data[(i+k)%len]=P0[i]; }
00318     }
00319     return *this;
00320 };
00321
00322 wperm_t wperm::initRotate(wperm_t r, swperm_t k) {
00323
00324     if (!r) {
00325         if (k) wblog(FL,
00326 "ERR %s() got empty permutation %d/%d !?",FCT,k,r);
00327         return Index(r);
00328     }
00329
00330     if ((k%r)<0) k+=r;
00331     if (!k) return Index(r);
00332
00333     RENEW(r);
00334     for (wperm_t i=0; i<len; ++i) { data[(i+k)%len]=i; }
00335     return *this;
00336 };
00337
00338 bool wperm::sameAs(const wperm &b) const {
00339     if (!len) {
00340         if (b.len)
00341             { return b.isIdentityPerm(); }
00342         else { return 1; }
00343     }
00344     if (!b.len)
00345         { return isIdentityPerm(); }
00346     else { return (*this)==b; }
00347 };
00348
00349 bool wperm::isIdentityPerm() const {
00350     bool id=1; if (len) {
00351         for (unsigned i=0; i<len; ++i) { if (data[i]!=i) { id=0; break; } };
00352     }
00353     return id;
00354 };
00355
00356 bool wperm::isIdentityPerm(const char *F, int L, wperm_t r) const {
00357     if (int(r)>=0 && len!=r) { if (F) wblog(F,L,
00358 "ERR %s() invalid permutation (len=%d/%d)",FCT,len,r);
00359         return 0;
00360     }
00361     bool q=1; char mark[len]; memset(mark,0,len);
00362     wperm_t i=0;
00363
00364     for (; i<len; ++i) {
00365         if (data[i]==i) { ++mark[i]; } else { q=0; break; }
00366     }
00367     for (; i<len; ++i) {
00368         if (data[i]>=len) wblog(F,L, "ERR %s() invalid permutation\n"
00369 "(index out of range %d/%d)",FCT,data[i],len);
00370         if ((++mark[data[i]])>1) wblog(F,L,"ERR %s() "
00371 "invalid permutation (non-unique index)",FCT);
00372     }
00373     return q;
00374 };

```

```

00375
00376 bool wbp perm::isReversePerm() const {
00377     if (len==0) return 0;
00378     for (wperm_t m=len-1, i=0; i<len; i++) if (data[i]!=m-i) return 0;
00379     return 1;
00380 };
00381
00382 bool wbp perm::isCyclic2F(wperm_t m, wperm_t n,
00383     char iflag
00384 ) const {
00385
00386     if (swperm_t(n)<0) { n = (len>m ? (len-m) : 0); }
00387     if (!len) { return (m || n ? 0 : 1); }
00388     if (len!=m+n) wblog(FL,
00389         "ERR %s() length mismatch (%d / %d + %d)",FCT,len,m,n);
00390
00391     if (iflag) { SWAP(m,n); }
00392
00393     wperm_t i=0;
00394     for (; i<m; ++i) { if (data[i]!=i+n) { return 0; }}
00395     for (; i<len; ++i) { if (data[i]!=i-m) { return 0; }}
00396     return 1;
00397 };
00398
00399 bool wbp perm::isOpTranspose() const {
00400
00401     if (len!=3 && len%2) { return 0; }
00402
00403     for (wperm_t i=0, r=len/2; i<r; ++i) {
00404         if (data[i]!=i+r || data[i+r]!=i) return 0;
00405     }
00406     return 1;
00407 };
00408
00409 wbp perm& wbp perm::Permute(const wbp perm &P, char iflag, wperm_t r){
00410
00411     if (P.len && !P.isIdentityPerm(FL,r)) {
00412         if (!len || isIdentityPerm(FL,r)) {
00413             if (iflag)
00414                 { P.invert(*this); }
00415             else { init(P); }
00416         }
00417         else {
00418             wbp perm X(*this);
00419             wperm_t i=0, *x=X.data, *p=P.data;
00420
00421             if (iflag)
00422                 { for (; i<r; ++i) { data[p[i]] = x[i]; }}
00423             else { for (; i<r; ++i) { data[i] = x[p[i]]; }}
00424         }
00425     }
00426     return *this;
00427 };
00428
00429 #endif
00430

```

5.55 wbp perm.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbvector (template vector class)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_PERM_HH__
00021 #define __WB_PERM_HH__
00022
00023 // ----- //
00024 // permutation class

```

```

00025 // Wb,Apr19,06
00026 // ----- //
00027
00028 class wbperm : public wbvector<wperm_t> {
00029
00030 public:
00031
00032     wbperm() {} ;
00033
00034     wbperm(wperm_t n, char reverse=0) { Index(n,reverse); };
00035
00036     wbperm(const wbperm &P, char iflag=0, unsigned r=-1)
00037         : WBPERM() { init(P,iflag,r); };
00038
00039     wbperm(const wbperm &P, size_t r) { init_pad(P,r); };
00040
00041     template<class T>
00042     wbperm(const wbvector<T> &b, wperm_t offset=0) {
00043         init(FL,b,offset);
00044     };
00045
00046     wbperm(const wbindex &P);
00047
00048     wbperm(const char *s, wperm_t offset=1) {
00049         initStr(FL,s,offset);
00050     }
00051
00052     wperm_t ell(wperm_t i) const {
00053         if (len) {
00054             if (i>=len) wblog(FL,
00055                 "ERR element index out of bounds (%d/%d)",i+1/len);
00056             return data[i];
00057         }
00058         else return i;
00059     };
00060
00061     wbperm& operator= (const char *s) {
00062         initStr(FL,s,0); return *this;
00063     };
00064
00065     int initStr(
00066         const char *F, int L, const char *s, wperm_t offset=1);
00067
00068     wbperm& init(const wbperm &P, char iflag=0, unsigned r=-1);
00069     wbperm& Extend(wperm_t r);
00070
00071     wbperm& init_trafo(const wbperm &p1_, const wbperm &p2_);
00072
00073     wbperm& init_pad(const wbperm &P, wperm_t r) {
00074         if (r<P.len) wblog(FL,
00075             "ERR invalid %s construction (%d/%d)",FCT,r,P.len);
00076         RENEW(r); memcpy(data,P.data,P.len*sizeof(P.data[0]));
00077         for (wperm_t i=P.len; i<r; i++) data[i]=i;
00078         if (!isValidPerm()) dispInvalidPerm(FL);
00079         return *this;
00080     };
00081
00082     wbperm& init(unsigned n=0, char reverse=0) {
00083         return Index(n,reverse);
00084     };
00085
00086     wbperm& init(unsigned n, const wperm_t* d, char ref=0) {
00087         wbvector<wperm_t>::init(n,d,ref); isValidPerm(FL);
00088         return *this;
00089     };
00090
00091     wbperm& init(const char *F, int L, const mxArray* a){
00092         return init(F,L,a,mxIsChar(a) ? 1 : 0);
00093     };
00094
00095     wbperm& init(const char *F, int L,
00096         const mxArray* a, wperm_t offset
00097     ){
00098         if (mxIsNumeric(a)) {
00099             WBPERM::init(F,L,a); if (offset) { operator+=(offset); }
00100             isValidPerm(F_L);
00101         }
00102         else if (mxIsChar(a)) {
00103             int l=mxGetNumberOfElements(a)+1;
00104             char s[l]; mxGetString(a,s,l);
00105             initStr(F_L,s,offset);
00106         }
00107         else wblog(FL,
00108             "ERR %s() invalid input (%s) !?",FCT,mxGetClassName(a));
00109         return *this;
00110     };
00111

```

```

00112     template<class T>
00113     wbpemr& init(
00114         const char *F, int L, const wbpemr<T> &b, wperm_t offset=0
00115     ){
00116         initT(b.len,b.data);
00117         if (offset) { (*this)--offset; }; isValidPerm(F_L);
00118         return *this;
00119     };
00120
00121     wbpemr& initTranspose(wperm_t r) {
00122         WBPemr::init(r); wperm_t r2=r/2;
00123         if (r%2) wblog(FL,"ERR %s() even input rank required (%d)",FCT,r);
00124         for (wperm_t i=0; i<r2; ++i) { data[i]=r2+i; data[r2+i]=i; }
00125         return *this;
00126     };
00127
00128     wbpemr& initPerm2(wperm_t r) {
00129         WBPemr::init(r); wperm_t r2=r/2;
00130         if (r%2) wblog(FL,"ERR %s() even input rank required (%d)",FCT,r);
00131         for (wperm_t l=0,i=0; i<r2; ++i) {
00132             data[l++]=i;
00133             data[l++]=r2+i;
00134         }
00135         return *this;
00136     };
00137
00138     wbpemr& initFirstTo (wperm_t k, wperm_t N) {
00139         Index(N); mvFirstTo(k);
00140         return *this;
00141     };
00142
00143     wbpemr& initLastTo (wperm_t k, wperm_t N) {
00144         Index(N); mvLastTo(k);
00145         return *this;
00146     };
00147
00148     wbpemr& init2Front(const WBPemr &I, wperm_t N);
00149     wbpemr& init2End  (const WBPemr &I, wperm_t N);
00150
00151     wbpemr& init2FrontB(wperm_t m, wperm_t N);
00152     wbpemr& init2EndB  (wperm_t m, wperm_t N);
00153
00154     wbpemr& init2Front(wperm_t k, wperm_t N);
00155     wbpemr& init2End  (wperm_t k, wperm_t N);
00156
00157     wbpemr& Cycle(wperm_t k1, wperm_t k2);
00158     wbpemr& initCycle(wperm_t n, wperm_t k1, wperm_t k2);
00159
00160     wbpemr& Index(wperm_t n, char reverse=0) {
00161         RENEW(n); if (n==0) return *this;
00162         if (reverse)
00163             for (wperm_t l=n-1, i=0; i<n; ++i) data[i]=l-i;
00164         else for (wperm_t i=0; i<n; ++i) data[i]=i;
00165         return *this;
00166     };
00167
00168     wbpemr& Index(wperm_t i1, wperm_t i2) {
00169         if (i1>i2) { RENEW(0); return *this; }
00170
00171         RENEW(i2-i1+1);
00172         for (wperm_t i=0; i<len; ++i) data[i]=i+i1;
00173
00174         return *this;
00175     };
00176
00177     char isValidPerm(const char *F, int L, wperm_t r=-1) const;
00178     char isValidPerm(wperm_t r=-1) const { return isValidPerm(0,0,r); };
00179
00180     wbpemr& Complete(wperm_t l);
00181
00182     wbpemr& Invert() {
00183         wbpemr iP; this->invert(iP).save2(*this);
00184         return *this;
00185     };
00186
00187     wbpemr& invert(wbpemr &iP) const;
00188
00189     wbpemr& Rotate(swperm_t l);
00190     wbpemr& initRotate(wperm_t n, swperm_t k);
00191
00192     wbpemr& Permute(const wbpemr &P, char iflag, wperm_t r);
00193     wbpemr& Permute(const wbpemr &P, char iflag=0) {
00194         return Permute(P, iflag, len>P.len ? len : P.len);
00195     };
00196
00197     bool sameAs(const wbpemr &b) const;
00198     bool isIdentityPerm() const;

```

```

00199     bool isIdentityPerm(const char *F, int L, wperm_t r=-1) const;
00200
00201     bool isReversePerm() const;
00202     bool isCyclic2F(wperm_t m, wperm_t n=-1, char iflag=0) const;
00203
00204     bool isOpTranspose() const;
00205
00206     explicit operator bool() const {
00207         return (isEmpty() || isIdentityPerm() ? 0 : 1); };
00208     bool operator! () const { return !bool(*this); }
00209
00210     wperm& times(const wperm &p2, wperm &Pout) const {
00211         if (!isValidPerm()) wblog(FL,
00212             "ERR %s() invalid permutation (%s)",__FUNCTION__,toStr().data);
00213         if (!p2.isValidPerm()) wblog(FL,
00214             "ERR %s() invalid permutation (%s)",__FUNCTION__,STR(p2));
00215         if (len!=p2.len) wblog(FL,
00216             "ERR %s() incompatible permutations (%d/%d)",
00217             __FUNCTION__,len,p2.len);
00218
00219         wperm P(len);
00220         for (wperm_t i=0; i<len; i++) P.data[i]=data[p2.data[i]];
00221
00222         P.save2(Pout); return Pout;
00223     };
00224
00225     wperm& flipIdx(wperm &P) const;
00226     wperm& FlipIdx();
00227     wperm flipIdx() const {
00228         wperm P; flipIdx(P);
00229         return P;
00230     };
00231
00232     void ToRaw() { ToRaw(len); };
00233     void ToRaw(wperm_t n) {
00234         if (n<len) wblog(FL,"WRN ToRaw() with n=%d/%d !?",n,len);
00235         if (n==0) return; else n--;
00236
00237         Flip();
00238
00239         for (wperm_t i=0; i<len; i++) {
00240             if (i>n) wblog(FL,"ERR wperm - invalid n=%d (%d)",n,i);
00241             data[i]=n-data[i];
00242         }
00243     }
00244
00245     wstring toStr() const {
00246
00247         wstring s_;
00248         unsigned i=0; char *s;
00249
00250         if (len<10) {
00251             s_.init(len+1); s=s_.data;
00252             for (; i<len; ++i) { s[i]='1'+data[i]; }
00253             s[i]=0;
00254         }
00255         else {
00256             unsigned l=0, n=len*(log10(double(len))+2);
00257             s_.init(n); s=s_.data;
00258             for (; i<len && l<n; ++i) {
00259                 l+=snprintf(s+l,n-l, " %d", (int)(data[i]+1));
00260             }
00261             if (l+3>=n) wblog(FL,"WRN %s() "
00262                 "string out of bounds (%d/%d; %d/%d)",FCT,l,n,i+1,len);
00263             else { s[l]=' '; s[l+1]=0; }
00264         }
00265         return s_;
00266     };
00267
00268     mxArray* toMx(const char tflag=0) const {
00269         return toMx_offset(1,tflag);
00270     };
00271
00272 protected:
00273 private:
00274
00275     void dispInvalidPerm(const char* F, int L) const {
00276         wblog(F,L,"WRN invalid permutation [%s]",toStr().data); };
00277
00278 };
00279
00280 int isValidPerm(const char *s, wperm_t r=-1) {
00281     int q=0; wperm P; int i;
00282     if ((i=P.initStr(0,0,s))<0) { q=-1; } else
00283     if (P.isValidPerm(r)) { q=P.len; } else { q=-2; }
00284     return q;
00285 };

```

```

00286
00287 int isValidPerm(const mxArray *a, wperm_t r=-1) {
00288     int q=0;
00289
00290     if (!a) { q=-11; } else
00291     if (mxIsChar(a)) {
00292         wbsting S(FL,a); if (S) {
00293             q=isValidPerm(S.data,r);
00294         }
00295     }
00296     else if (!mxIsNumeric(a)) { q=-12; }
00297     else if (!Mx::IsVector(a)) {
00298         if (mxGetNumberOfElements(a)) q=-13;
00299     }
00300     else {
00301         wvector<double> X(FL,a);
00302         if (int(r)>=0 && X.len!=r) { q=-3; } else
00303         if (X) {
00304             unsigned i=0, j, n=X.len; double *x=X.data;
00305             wvector<char> m(n+1);
00306             for (; i<n; ++i) { j=x[i];
00307                 if (x[i]!=j || j>n || ++m[j]!=1) { break; }
00308             }
00309             q=(i<n || (m[0] && m[n]) ? -4 : n);
00310         }
00311     }
00312     return q;
00313 };
00314
00315 wperm& wperm::flipIdx(wperm &P) const {
00316
00317     if (&P==this) wblog(FL,
00318         "ERR flipIdx(P) calls itself\nused flipIdx() instead!");
00319
00320     P.RENEW(len); if (!len) return P;
00321
00322     for (wperm_t m=len-1, i=0; i<len; i++)
00323     P.data[i]=m-data[i];
00324
00325     return P;
00326 };
00327
00328 inline wperm& wperm::FlipIdx() {
00329
00330     for (wperm_t m=len-1, i=0; i<len; ++i) {
00331         if (data[i]>m) wblog(FL,"ERR index out of bounds (%d/%d)",data[i],m);
00332         data[i]=m-data[i];
00333     }
00334
00335     return *this;
00336 };
00337
00338 #endif
00339

```

5.56 wbsort.cc

```

00001 #ifndef __WB_REC_HPSORT_CC__
00002 #define __WB_REC_HPSORT_CC__
00003
00004 /* ----- *
00005  * Heap sort algorithm for vector array
00006  * Reference: Numerical Recipies C, p337f
00007  * AW (C) Jan 2006
00008  * ----- *
00009  * mergesort (ranks #2, vs. #3 for hpsort)
00010  *   + on average faster than hpsort
00011  *   + stable (does not reverse order within degenerate subspaces)
00012  *   - requires worst case an additional space of size N
00013  *   + in-place merge-sort: implemented in C++ STL (!) [stable_sort]
00014  *   + easily parallelizes!
00015  *   while quicksort is faster in typical cases, it is
00016  *     (i) not stable, and (ii) has a worst-case performance of N^2
00017  * hpsort, for comparsion
00018  *   is NOT stable (ie. may reverse order within degenerate subspaces)
00019  *   slower on average, while in-place
00020  *   still performs MEM_COPY even on already sorted array (!?)
00021  *   reverse sort is actually faster than sorting a sorted array (!?)
00022  * Wb,Dec29,11
00023  * ----- */
00024
00025 // by including P, this ensures to keep initial order of degenerate
00026 // subspaces, in that what is actually is sorted is [ra,(1:N)']

```

```

00027
00028 template<class T>
00029 void Wb::hpsort(
00030     wbvector<T> &ra,
00031     wbperm &P,
00032     char dir
00033 ){
00034     size_t i,j,l,ir, N=ra.len; wperm_t ia,*p;
00035     T rra;
00036
00037     char q, Q=(dir>0 ? +1 : -1);
00038
00039     P.Index(N); p=P.data; if (N<2) return;
00040
00041     l = (N>1);
00042     ir= N-1;
00043
00044     for (;;) {
00045         if(l>0) {
00046             rra=ra[--l];
00047             ia=p[l];
00048         } else {
00049             rra=ra[ir];
00050             ra[ir]=ra[0];
00051
00052             ia=p[ir]; p[ir]=p[0];
00053
00054             if (--ir==0) {
00055                 ra[0]=rra;
00056                 p[0]=ia;
00057                 break;
00058             }
00059         }
00060
00061         i=1;
00062
00063         j=l+l+1;
00064         while (j<=ir) {
00065             if (j<ir) {
00066                 q=NUMCMP (ra[j],ra[j+1]); if (q==0) q=NUMCMP (p[j],p[j+1]);
00067                 if (q!=Q) j++;
00068             }
00069
00070             q=NUMCMP (rra,ra[j]); if (q==0) q=NUMCMP (ia,p[j]);
00071             if (q!=Q) {
00072                 ra[i]=ra[j];
00073                 p[i]=p[j];
00074
00075                 i = j;
00076                 j = ((j+1)<<1) - 1;
00077             }
00078             else break;
00079         }
00080         ra[i]=rra;
00081         p[i]=ia;
00082     }
00083
00084     return;
00085 };
00086
00087 template<class T>
00088 int Wb::hpsort(
00089     T *A,
00090     size_t lda,
00091     size_t N,
00092     wbperm &P,
00093     char dir,
00094     char lex
00095 ){
00096     size_t i,j,l, ia,ir, *p;
00097
00098     char q, Q=(dir>0 ? +1 : -1);
00099
00100     P.Index(N); p=P.data; if (N<2) return 0;
00101
00102     l=(N>1); ir=N-1;
00103     T x[lda];
00104
00105     for (;;) {
00106         if (l>0) {
00107             ia=p[--l]; MEM_COPY<T>(x,lda,A+l*lda);
00108         }
00109         else {
00110             ia=p[ir]; MEM_COPY<T>(x,lda,A+ir*lda);
00111             p[ir]=p[0]; MEM_COPY<T>(A+ir*lda,lda,A);
00112
00113             if (--ir==0) {

```



```

00114         p[0]=ia; MEM_COPY<T>(A,lda,x);
00115         break;
00116     }
00117 }
00118
00119     i=1; j=2*1+1;
00120
00121     while (j<=ir) { T *aj=A+j*lda;
00122         if (j<ir) {
00123             q=Wb::recCompare(aj,aj+lda,lda,lex);
00124             if (q==0) q=NUMCMP(p[j],p[j+1]);
00125             if (q!=Q) { ++j; aj+=lda; }
00126         }
00127
00128         q=Wb::recCompare(x,aj,lda,lex); if (q==0) q=NUMCMP(ia,p[j]);
00129         if (q!=Q) {
00130             MEM_COPY<T>(A+i*lda,lda,A+j*lda);
00131             p[i]=p[j];
00132
00133             i=j;
00134             j=((j+1)<<1)-1;
00135         }
00136         else break;
00137     }
00138     p[i]=ia; MEM_COPY<T>(A+i*lda,lda,x);
00139 }
00140
00141     return 0;
00142 };
00143
00144 template<class T>
00145 int Wb::hpsort(
00146     wbMatrix<T> &ra, wbperm &P, char dir, char lex
00147 ){ return hpsort(ra.data, ra.dim2, ra.dim1, P, dir, lex); };
00148
00149 template<class T>
00150 void Wb::hpsort(wbvector<T> &ra) { wbperm P; hpsort(ra,P); }
00151
00152 template <class T> inline
00153 size_t Wb::matchSortedIdx_grp(
00154     const T *A, int lda, size_t na,
00155     const T *B, int ldb, size_t nb,
00156     wbindex &gra, wbindex &grb, size_t &mtot,
00157     int m,
00158     char lex,
00159     T eps
00160 ){
00161     size_t ia,ib,iga=0,igb=0,ng=0,m1,m2,*p1,*p2;
00162     int nwrn=0; char c, cr=0;
00163     const T *a,*b;
00164
00165     if (m<0) { m=lda;
00166         if (lda!=ldb) wblog(FL,
00167             "WRN %s() got different record length (%d/%d)",FCT,lda,ldb);
00168     }
00169
00170     if (!m) wblog(FL,"ERR finite m required "
00171         " (m=%d; %dx%d <> %dx%d)",m,lda,na,ldb,ldb);
00172     if (m>lda || m>ldb || lda<=0 || ldb<=0) wblog(FL,
00173         "ERR m out of bounds (m=%d; %d/%d)",m,lda,ldb);
00174     if (!A || !B) wblog(FL,
00175         "ERR %s() got null data (0x%lX, 0x%lX)",FCT,A,B);
00176
00177     gra.init(2*na); grb.init(2*nb); mtot=0;
00178
00179     if (lex<=0) {
00180         A+=(lda-m);
00181         B+=(ldb-m);
00182     }
00183
00184     a=A; b=B;
00185     for (ia=1; ia<na; ++ia, a+=lda) {
00186         c=Wb::recCompare(a,a+lda,m,lex,eps,&nwrn);
00187         if (c) { cr=c; break; }
00188     }
00189     if (!cr) {
00190         for (ib=1; ib<nb; ++ib, b+=ldb) {
00191             c=Wb::recCompare(b,b+ldb,m,lex,eps,&nwrn);
00192             if (c) { cr=c; break; }
00193         };
00194         if (!cr) { cr=-1; }
00195     }
00196     for (a=A,b=B, ia=ib=0; ia<na && ib<nb;) {
00197         c=Wb::recCompare(a,b,m,lex,eps,&nwrn);
00198
00199         if (c== cr) { ++ia; a+=lda; continue; } else
00200         if (c== -cr) { ++ib; b+=ldb; continue; } else

```

```

00201         if (c) wblog(FL,"ERR %s() c=%d/%d !?",FCT,c,cr);
00202
00203         m1=m2=1; ++ng;
00204         p1=gra.data+(iga++); gra[iga++]=ia++; a+=lda;
00205         p2=grb.data+(igb++); grb[igb++]=ib++; b+=ldb;
00206
00207         while (ia<na) {
00208             c=Wb::recCompare(a-lda,a,m,lex,eps,&nwrn);
00209             if (!c) { gra[iga++]=ia++; a+=lda; ++m1; }
00210             else {
00211                 if (c!=cr) wblog(FL,
00212                     "ERR %s() got unsorted recs (%d: %d/%d)",FCT,ia+1,c,cr);
00213                 break;
00214             }
00215         }
00216         while (ib<nb) {
00217             c=Wb::recCompare(b-ldb,b,m,lex,eps,&nwrn);
00218             if (!c) { grb[igb++]=ib++; b+=ldb; ++m2; }
00219             else {
00220                 if (c!=cr) wblog(FL,
00221                     "ERR %s() got unsorted recs (%d: %d/%d)",FCT,ib+1,c,cr);
00222                 break;
00223             }
00224         }
00225         mtot+=m1*m2; (*p1)=m1; (*p2)=m2;
00226     }
00227
00228     gra.Shorten2(iga);
00229     grb.Shorten2(igb);
00230
00231     return ng;
00232 };
00233
00234 template <class T> inline
00235 size_t Wb::matchSortedIdx(
00236     const T *A, int lda, size_t na,
00237     const T *B, int ldb, size_t nb, wbindex &Ia, wbindex &Ib,
00238     int m, char lex,
00239     widx_t *ma_,
00240     widx_t *mb_,
00241     T eps
00242 ){
00243     size_t i,j,k=0, ig=0,iga=0,igb=0, m1,m2,mt=0,ma=0,mb=0, ng,*p1,*p2;
00244     wbindex gra,grb;
00245
00246     ng=Wb::matchSortedIdx_grp(A,lda,na,B,ldb,nb,gra,grb,mt,m,lex,eps);
00247     Ia.init(mt); Ib.init(mt); mt=0;
00248
00249     for (; ig<ng; ++ig) { m1=gra[iga]; m2=grb[igb];
00250         if (m1==1 && m2==1) {
00251             Ia[k]=gra[iga+1]; iga+=2;
00252             Ib[k]=grb[igb+1]; igb+=2; ++k;
00253         }
00254         else {
00255             p1=gra.data+(iga+1); iga+=(m1+1);
00256             p2=grb.data+(igb+1); igb+=(m2+1);
00257
00258             ma+=(m1-1);
00259             mb+=(m2-1); mt+=(m1*m2-1);
00260
00261             for (i=0; i<m1; ++i)
00262                 for (j=0; j<m2; ++j, ++k) { Ia[k]=p1[i]; Ib[k]=p2[j]; }
00263         }
00264     }
00265
00266     if (ma_) { (*ma_)=ma; }
00267     if (mb_) { (*mb_)=mb; }
00268
00269     return mt;
00270 };
00271
00272 template <class T> inline
00273 size_t Wb::matchSortedIdx(
00274     const T *A, int lda, size_t na,
00275     const T *B, int ldb, size_t nb,
00276     wbindex &Ia, wbindex &Ib, wbvector<widx_t> &D,
00277     int m, char lex, T eps
00278 ){
00279     size_t i,j,k=0, ig=0,iga=0,igb=0, m1,m2,mt=0, ng,*p1,*p2;
00280     wbindex gra, grb;
00281
00282     ng=Wb::matchSortedIdx_grp(A,lda,na,B,ldb,nb,gra,grb,mt,m,lex,eps);
00283     Ia.init(mt); Ib.init(mt); D.init(ng); mt=0;
00284
00285     for (; ig<ng; ++ig) { m1=gra[iga]; m2=grb[igb];
00286         if (m1==1 && m2==1) {
00287             Ia[k]=gra[iga+1]; iga+=2;

```

```

00288         Ib[k]=grb[igb+1]; igb+=2; D[ig]=1; ++k;
00289     }
00290     else {
00291         p1=gra.data+(iga+1); iga+=(m1+1);
00292         p2=grb.data+(igb+1); igb+=(m2+1);
00293
00294         D[ig]=m1*m2; mt+=(D[ig]-1);
00295
00296         for (i=0; i<m1; ++i)
00297             for (j=0; j<m2; ++j, ++k) { Ia[k]=p1[i]; Ib[k]=p2[j]; }
00298     }
00299 }
00300
00301 return mt;
00302 };
00303
00304 #endif
00305

```

5.57 wbsort.hh

```

00001 #ifndef __WB_REC_HPSORT_HH__
00002 #define __WB_REC_HPSORT_HH__
00003
00004 namespace Wb {
00005
00006 /* ----- */
00007 // generalized matrix/record sort
00008 // sorted by rows/records of length m, with actual leading length lda
00009 /* ----- */
00010
00011 template<class T>
00012 int hpsort(
00013     T *A,
00014     size_t lda,
00015     size_t N,
00016     wbperm &P,
00017     char dir=+1,
00018     char lex=+1
00019 );
00020
00021 template<class T>
00022 int hpsort(
00023     wbMatrix<T> &ra,
00024     wbperm &P,
00025     char dir=+1,
00026     char lex=+1
00027 );
00028
00029 template<class T>
00030 void hpsort(
00031     wbvector<T> &ra,
00032     wbperm &P,
00033     char dir=+1
00034 );
00035
00036 template<class T>
00037 void hpsort(wbvector<T> &ra);
00038
00039 template <class T> inline
00040 size_t matchSortedIdx_grp(
00041     const T *da, int lda, size_t na,
00042     const T *db, int ldb, size_t nb,
00043     wbindex &gra, wbindex &grb, size_t &mtot,
00044     int m,
00045     char lex,
00046     T eps=0
00047 );
00048
00049 template <class T> inline
00050 size_t matchSortedIdx(
00051     const T *da, int lda, size_t na,
00052     const T *db, int ldb, size_t nb, wbindex &Ia, wbindex &Ib,
00053     int m, char lex,
00054     widx_t *ma=NULL,
00055     widx_t *mb=NULL,
00056     T eps=0
00057 );
00058
00059 template <class T> inline
00060 size_t matchSortedIdx(
00061     const T *da, int lda, size_t na,
00062     const T *db, int ldb, size_t nb,

```

```

00063     wbindex &Ia, wbindex &Ib, wbvector<widx_t> &D,
00064     int m, char lex, T eps=0
00065 );
00066
00067 };
00068
00069 #endif
00070

```

5.58 wbsparray.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbsparray (sparse array of arbitrary dimension)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SPARRAY_CC__
00021 #define __WB_SPARRAY_CC__
00022
00023 // ----- //
00024 // check whether A=this fits into B for all dimensions
00025 // return value
00026 // 0 A=this does not fit into B (is larger in at least one dimension)
00027 // else A fits, where
00028 // 1 A has same SIZE.len (strict)
00029 // 2 B.SIZE.len is longer than in A (B has non-zero extra dimension)
00030 // 3 A.SIZE.len is longer than in B (A has trailing singleton dimensions)
00031 // 4 A and B have diagonal format
00032 // Wb,Jun22,20
00033
00034 template <class TD>
00035 char wbsparray<TD>::fitsSize(const wbsparray<TD> &B, char strict) const {
00036
00037     bool i1=isDiag(), i2=B.isDiag();
00038     if (i1 || i2) {
00039         if (!i1 || !i2) wblog(FL,
00040             "ERR %s() got mixed diag format (%d/%d)",FCT,i1,i2);
00041         return (D.len<=B.D.len ? 4 : 0);
00042     }
00043
00044     unsigned i=0, n=MIN(SIZE.len,B.SIZE.len);
00045     const SPIDX_T *s1=SIZE.data, *s2=B.SIZE.data;
00046
00047     for (; i<n; ++i) { if (s1[i]>s2[i]) return 0; }
00048
00049     if (SIZE.len==B.SIZE.len) { return 1; }
00050     if (strict) { return 0; }
00051
00052     if (SIZE.len<B.SIZE.len) {
00053         for (n=B.SIZE.len; i<n; ++i) { if (!s2[i]) return 0; }
00054         return 2;
00055     }
00056     else {
00057         for (n=SIZE.len; i<n; ++i) { if (s1[i]>1) return 0; }
00058         return 3;
00059     }
00060 };
00061
00062 template <class TD>
00063 char wbsparray<TD>::sameSize(const wbsparray<TD> &B, char strict) const {
00064
00065     if (SIZE.len!=B.SIZE.len && !strict) {
00066         if ( isDiag(FL)) return (B.isSMatrix(FL, D.len) ? 11 : 0);
00067         if (B.isDiag(FL)) return ( isSMatrix(FL,B.D.len) ? 12 : 0);
00068
00069         char s=sameSizeUp2Singletons(B.SIZE);
00070         if (s) {
00071             if (s>2)
00072                 return 23;

```

```

00073         else {
00074             return (SIZE.len>B.SIZE.len ? 21 : 22);
00075         }
00076     }
00077     else return 0;
00078 }
00079
00080 if (SIZE==B.SIZE) {
00081     if (!SIZE.len && D.len)
00082         return (D.len==B.D.len ? 13 : 0);
00083     else return 1;
00084 }
00085
00086 return 0;
00087 };
00088
00089 template <class TD> inline
00090 char wbsparray<TD>::sameSizeUp2Singletons(
00091     const wbvector<SPIDX_T> &S, const char strict) const {
00092
00093     if (SIZE==S) return 1;
00094     if (numel()!=S.prod(0)) return 0;
00095
00096     if (isDiag()) {
00097         if (S.len<2 || S[0]!=S[1] || S[0]!=D.len) return 0;
00098         if (S.len==2) return 1;
00099         if (!strict) {
00100             for (unsigned i=2; i<S.len; ++i) if (S[i]!=1) return 0;
00101             return 2;
00102         }
00103         return 0;
00104     }
00105
00106     { char ok=1;
00107       unsigned n,N, i=0; const SPIDX_T *s;
00108
00109       if (SIZE.len<S.len)
00110           { n=SIZE.len; N= S.len; s= S.data; }
00111       else { n= S.len; N=SIZE.len; s=SIZE.data; }
00112
00113       for (; i<n; ++i) { if (SIZE[i]!=S[i]) { ok=0; break; } }
00114       if (ok) {
00115           for (; i<N; ++i) if (s[i]!=1) { ok=0; break; } }
00116
00117       if (ok) return 2;
00118     }
00119
00120     if (!strict) {
00121         unsigned i=0, j=0, ra=SIZE.len, rb=S.len;
00122         const SPIDX_T *sa=SIZE.data, *sb=S.data;
00123
00124         for (; i<ra; ++i) { if (sa[i]!=1) {
00125             for (; j<rb; ++j) { if (sb[j]!=1) break; }
00126             if (j<rb && sa[i]==sb[j]) { ++j; }
00127             else return 0;
00128         } }
00129         for (; j<rb; ++j) { if (sb[j]!=1) return 0; }
00130         return 3;
00131     }
00132
00133     return 0;
00134 };
00135
00136 template <class TD> inline
00137 char wbsparray<TD>::matchWithSingletons(
00138     const char *F, int L, const wbvector<SPIDX_T> &S, wbvector<int> &M
00139 ) const {
00140
00141     unsigned i=0, j=0, e=0, ra=SIZE.len, rb=S.len;
00142     const SPIDX_T *sa=SIZE.data, *sb=S.data;
00143
00144     if (!S.len) {
00145         if (SIZE.len) wblog(FL,"ERR %s() got %s <> ()",FCT,SSTR_(this));
00146         M.init(); return 0;
00147     }
00148
00149     M.init(S.len).set(-1);
00150
00151     for (; i<ra; ++i) { if (sa[i]!=1) {
00152         for (; j<rb; ++j) { if (sb[j]!=1) break; }
00153         if (j<rb && sa[i]==sb[j]) { M[j++]=i; }
00154         else { ++e; break; }
00155     } }
00156
00157     if (!e)
00158         for (; j<rb; ++j) { if (sb[j]!=1) { ++e; break; } }
00159

```

```

00160     if (e) {
00161         if (F) wblog(F,L,"ERR %s() got size mismatch (%s <> %s)",
00162             FCT, SSTR_(this), SSTR(S));
00163         return e;
00164     }
00165
00166     return 0;
00167 };
00168
00169 template <class TD> inline
00170 bool wbsparray<TD>::hasSize(SPIDX_T d1) const {
00171     return (SIZE.len==1 && SIZE[0]==d1);
00172 };
00173
00174 template <class TD> inline
00175 bool wbsparray<TD>::hasSize(SPIDX_T d1, SPIDX_T d2) const {
00176     return (SIZE.len==2 && SIZE[0]==d1 && SIZE[1]==d2);
00177 };
00178
00179 template <class TD> inline
00180 bool wbsparray<TD>::hasSize(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const {
00181     return (SIZE.len==3 && SIZE[0]==d1 && SIZE[1]==d2 && SIZE[2]==d3);
00182 };
00183
00184 template <class TD> inline
00185 wbsparray<TD>& wbsparray<TD>::setRand(double x, char sflag){
00186
00187     SPIDX_T n=numel(); if (!n) return *this;
00188     n=MIN(n,SPIDX_T(x*pow(double(n),1./rank(FL))));
00189
00190     init_nnz(n); if (!n) return *this;
00191
00192     SPIDX_T i,j, m=SIZE.len, *I=IDX.data;
00193     wbindx I1; Wb::rand R;
00194
00195     for (i=0; i<n; ++i, I+=m) {
00196         for (j=0; j<m; ++j) {
00197             R.rand_(I[j], SIZE[j]);
00198             if (I[j]>=SIZE[j]) wblog(FL,"ERR %s() index out of bounds "
00199                 "(%d,%d: %d; %s)",FCT,i+1,j+1,I[j],SSTR_(this)
00200             );
00201         }
00202     }
00203
00204     IDX.makeUnique(I1); D.Select(I1);
00205     setRand_data(x,sflag);
00206
00207     Compress(); return *this;
00208 };
00209
00210 template <class TD> inline
00211 wbsparray<TD>& wbsparray<TD>::setRand_data(double x, char sflag) {
00212
00213     if (D.len) {
00214         SPIDX_T i=0, n=D.len; Wb::rand R;
00215
00216         if (sflag) {
00217             if (sflag=='d' || sflag=='D') {
00218                 for (; i<n; ++i) { R.rands(D.data[i]); }
00219             }
00220             else {
00221                 for (; i<n; ++i) { R.randb(D.data[i]); }
00222             }
00223         }
00224         else {
00225             for (; i<n; ++i) { R.rand_(D.data[i]); }
00226         }
00227
00228         if (x!=1) { R.scale_rand(D.data,D.len, x); }
00229     }
00230
00231     return *this;
00232 };
00233
00234 template <class TD> inline
00235 wbsparray<TD>& wbsparray<TD>::Resize(const wbvector<SPIDX_T> &S_) {
00236
00237     if (SIZE.len!=S_.len) wblog(FL,
00238         "ERR %s() invalid size (%d/%d)",FCT,S_.len,SIZE.len);
00239     if (!D.len) { return init(S_); }
00240     if (SIZE.len!=IDX.dim2) wblog(FL,
00241         "ERR %s() got size %s !?",FCT,SSTR_(this));
00242
00243     SPIDX_T i,j, l=0, N=IDX.dim1, m=IDX.dim2;
00244     const SPIDX_T *idx=IDX.data, *s=S_.data;
00245     wbsparray<TD> X; TD *x=D.data;
00246

```

```

00247     for (j=0; j<m; ++j) { if (s[j]<SIZE[j]) break; }
00248     if (j==m) {
00249         SIZE=S_; return *this;
00250     }
00251
00252     for (i=0; i<N; ++i, idx+=m) {
00253         for (j=0; j<m; ++j) { if (idx[j]>=s[j]) break; }
00254         if (j==m && x[i]!=0) ++l;
00255     }
00256
00257     if (l) {
00258         save2(X); init(S_,l); l=0; idx=X.IDX.data; x=X.D.data;
00259         for (i=0; i<N; ++i, idx+=m) {
00260             for (j=0; j<m; ++j) { if (idx[j]>=s[j]) break; }
00261             if (j==m && x[i]!=0) {
00262                 for (j=0; j<m; ++j) { IDX(l,j)=idx[j]; }
00263                 D[l++]=x[i];
00264             }
00265         }
00266     }
00267     else init(S_);
00268
00269     return *this;
00270 };
00271
00272 template <class TD> inline
00273 SPIDX_T wbsparray<TD>::findRecSortedP(
00274     SPIDX_T *idx, SPIDX_T n, char lex
00275 ){
00276     unsigned i=0;
00277
00278     if (long(n)<0) n=IDX.dim2;
00279     else if (n!=SIZE.len) {
00280         if (lex>0) {
00281             if (n<SIZE.len) {
00282                 for (i=n; i<SIZE.len; ++i) { if (SIZE[i]!=1) { i=-1; break; }}
00283             }
00284             else if (n>SIZE.len) {
00285                 for (i=SIZE.len; i<n; ++i) { if (idx[i]!=0) { i=-1; break; }}
00286                 n=SIZE.len;
00287             }
00288         }
00289         else {
00290             wblog(FL,"ERR %s() got n=%d/%d (%d)",FCT,n,SIZE.len,lex);
00291         }
00292     }
00293
00294     if (int(i)<0 || !n || !SIZE.len) wblog(FL,
00295         "ERR %s() invalid index [%s] (%s)",
00296         FCT, wvector<SPIDX_T>(n,idx,'r'), SSTR_(this)
00297     );
00298
00299     return IDX.findRecSorted(idx,n,lex);
00300 };
00301
00302 template<class TD> inline
00303 char wbsparray<TD>::sameUptoFac(
00304     const wbsparray<TD> &B, TD *fac_, char lex, TD eps
00305 ) const {
00306
00307     widx_t i=-1, n=D.len; TD a0,b0,fac=1;
00308     const TD *a=D.data, *b=B.D.data;
00309     windex Ia,Ib;
00310     char iA=0, iB=0, sab=sameSize(B);
00311
00312     if (!sab) return 1;
00313     if (!n) {
00314         if (fac_) { (*fac_)=(B.D.len ? 0. : 1.); }
00315         return (B.D.len ? 0 : 2);
00316     }
00317
00318     if (sab>=20) wblog(FL,"ERR %s() got singletons (%s <> %s)",
00319         FCT,SSTR_(this), SSTR(B));
00320
00321     if (sab>=10) {
00322         iA=((sab-10) & 1);
00323         iB=((sab-10) & 2);
00324     }
00325
00326     a0=D.aMax(&i);
00327
00328     if (swidx_t(i)<0) wblog(FL,
00329         "ERR %s() got zero data (%g;%d)",FCT,double(a0),i
00330     );
00331
00332     if (iB) {
00333         if (iA) b0=B.D[i];

```

```

00334     else {
00335         const SPIDX_T *I=IDX.ref(i);
00336         if (IDX.dim2!=2) wblog(FL,
00337             "ERR %s() %dx%d !?",FCT,IDX.dim1,IDX.dim2);
00338         b0=(I[0]==I[1] ? B.D[I[0]] : TD(0.));
00339     }
00340 }
00341 }
00342 else {
00343     b0=B.value(IDX.rec(i),IDX.dim2);
00344 }
00345 }
00346 if (a0<=eps) {
00347     if (fac_) { (*fac_)=((a0==0 && b0) || ABS(b0)>eps ? 0. : 1.); }
00348     return (ABS(b0)>eps ? 0 : 3);
00349 }
00350 if (ABS(b0)<=eps) { return 4; }
00351
00352 fac=a[i]/b0; if (fac_) (*fac_)=fac;
00353
00354 if (sab>=10) {
00355     if (iA && iB) {
00356         for (i=0; i<D.len; ++i) { if (ABS(a[i]-fac*b[i])>eps) return 11; }
00357         return 0;
00358     }
00359     else if (iA) {
00360         const SPIDX_T *I=B.IDX.data; wbvector<char> mark(D.len);
00361         if (B.IDX.dim2!=2) wblog(FL,
00362             "ERR %s() %dx%d !?",FCT,B.IDX.dim1,B.IDX.dim2);
00363         for (i=0; i<B.D.len; ++i, I+=2) {
00364             if (I[0]==I[1]) { mark.el(I[0])=1;
00365                 if (ABS(a[I[0]]-fac*b[i])>eps) return 12; }
00366             else if (ABS(b[i])>eps) return 13;
00367         }
00368         for (i=0; i<D.len; ++i) {
00369             if (!mark[i] && ABS(a[i])>eps) return 14;
00370         }
00371         return 0;
00372     }
00373     else {
00374         const SPIDX_T *I=IDX.data; wbvector<char> mark(B.D.len);
00375         if (IDX.dim2!=2) wblog(FL,
00376             "ERR %s() %dx%d !?",FCT,IDX.dim1,IDX.dim2);
00377         for (i=0; i<D.len; ++i, I+=2) {
00378             if (I[0]==I[1]) { mark.el(I[0])=1;
00379                 if (ABS(a[i]-fac*b[I[0]])>eps) return 21; }
00380             else if (ABS(a[i])>eps) return 22;
00381         }
00382         for (i=0; i<B.D.len; ++i) {
00383             if (!mark[i] && ABS(b[i])>eps) return 23;
00384         }
00385         return 0;
00386     }
00387 }
00388
00389 matchSortedIdxU(FL,IDX,B.IDX,Ia,Ib,-1,lex);
00390 for (n=Ia.len, i=0; i<n; ++i) {
00391     if (ABS(a[Ia[i]]-fac*b[Ib[i]])>eps) return 31;
00392 }
00393
00394 Ia.Invert( IDX.dim1,'u');
00395 for (i=0; i<Ia.len; ++i) { if (ABS(a[Ia[i]])>eps) return 32; }
00396
00397 Ib.Invert(B.IDX.dim1,'u');
00398 for (i=0; i<Ib.len; ++i) { if (ABS(b[Ib[i]])>eps) return 33; }
00399
00400 return 0;
00401 };
00402
00403 template<class TD> inline
00404 bool wbsparray<TD>::isZero(TD eps, char bflag) const {
00405
00406     if (!D.len) return 1;
00407     Wb::scale_eps(eps,D.data,D.len);
00408
00409     if (eps==0) {
00410         for (SPIDX_T i=0; i<D.len; ++i) if (D[i]!=0) return 0;
00411     }
00412     else {
00413         if (bflag) { return (D.norm())<eps; }
00414         else {
00415             for (SPIDX_T i=0; i<D.len; ++i) {
00416                 if (ABS(D[i])>eps) return 0; }
00417             }
00418     }
00419
00420     return 1;

```



```

00421 };
00422
00423 template <class TD>
00424 bool wbsparray<TD>::isDiagMatrix(TD eps) const {
00425
00426     if (!isDiag()) { unsigned r=rank(FL);
00427
00428         if (r!=2) return 0;
00429         if (IDX.dim2==1) return 1;
00430
00431         SPIDX_T i=0; const SPIDX_T *id=IDX.data;
00432
00433         if (IDX.dim2!=2) sperror_this(FLF);
00434         Wb::scale_eps(eps,D.data,D.len);
00435
00436         if (eps==0) {
00437             for (; i<IDX.dim1; ++i, id+=2) { if (id[0]!=id[1]) return 0; }
00438         }
00439         else {
00440             for (; i<IDX.dim1; ++i, id+=2) if (id[0]!=id[1]) {
00441                 if (ABS(D[i])>eps) return 0;
00442             }
00443         }
00444     }
00445
00446     return 1;
00447 };
00448
00449 template <class TD>
00450 wbvector<TD>& wbsparray<TD>::getDiag(
00451     const char *F, int L, wbvector<TD> &dd) const {
00452
00453     if (isDiag(F_L)) { dd.init(D); }
00454     else {
00455         SPIDX_T i=0, n=-1; const SPIDX_T *I=IDX.data;
00456
00457         if (!isSMatrix(F_L,&n) || IDX.dim2!=2) wblog(F_L,
00458             "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR_(this));
00459         dd.init(n);
00460
00461         for (; i<IDX.dim1; ++i, I+=2) {
00462             if (*I==I[1]) { dd.el(*I) += D[i]; }
00463         }
00464     }
00465     return dd;
00466 };
00467
00468 template <class TD>
00469 TD wbsparray<TD>::getDiag(const char *F, int L, SPIDX_T k) const {
00470
00471     TD x=0;
00472
00473     if (isDiag(F_L)) {
00474         if (sSPIDX_T(k)<0) { k+=D.len; }
00475         if (k>=D.len) wblog(FL,
00476             "ERR %s() index out of bounds (%d/%d)",FCT,k,D.len);
00477         x=D[k];
00478     }
00479     else {
00480         SPIDX_T i=0, n=-1; const SPIDX_T *I=IDX.data;
00481
00482         if (!isSMatrix(F_L,&n) || IDX.dim2!=2) wblog(F_L,
00483             "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR_(this));
00484         if (sSPIDX_T(k)<0) { k+=n; }
00485         if (k>=n) wblog(FL,
00486             "ERR %s() index out of bounds (%d/%d)",FCT,k,n);
00487
00488         for (; i<IDX.dim1; ++i, I+=2) {
00489             if (*I==I[1] && *I==k) { x+=D[i]; }
00490         }
00491     }
00492     return x;
00493 };
00494
00495 template <class TD>
00496 wbsparray<TD>& wbsparray<TD>::diag2reg(
00497     const char *F, int L, wbsparray<TD> &A, unsigned r
00498 ) const {
00499
00500     if (int(r)<0 || r==2) {
00501         if (!D.len) { checkSize(F_L); A=(*this); return A; }
00502         if (!isDiag(FL)) sperror_this(F_LF);
00503
00504         SPIDX_T i=0, n=D.len, *I; const SPIDX_T s[]={n,n};
00505         wbvector<SPIDX_T> S(2,s);
00506
00507         A.init(S,n); A.D=D; I=A.IDX.data;

```

```

00508         for (; i<n; ++i, I+=2) { I[0]=I[1]=i; }
00509     }
00510     else {
00511         if (!r) wblog(FL,"ERR %s() invalid rank (%d)",FCT,r);
00512         if (!isScalar()) sperror_this(F_LF);
00513         A.SIZE.init(r).set(1);
00514         A.IDX.init(D.len,r); A.D=D; A.isref=0;
00515     }
00516
00517     return A;
00518 };
00519
00520 template <class TD>
00521 wbsparray<TD>& wbsparray<TD>::diag2reg(
00522     const char *F, int L, unsigned r
00523 ){
00524     if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00525
00526     if (int(r)<0 || r==2) {
00527         if (!D.len) { checkSize(F_L); return *this; }
00528         if (!isDiag(FL)) sperror_this(F_LF);
00529
00530         SPIDX_T i=0, n=D.len, *I;
00531         SIZE.init(2); SIZE[0]=SIZE[1]=n;
00532         IDX.init(n,SIZE.len);
00533
00534         for (I=IDX.data; i<n; ++i, I+=2) { I[0]=I[1]=i; }
00535     }
00536     else {
00537         if (!r) wblog(FL,"ERR %s() invalid rank (%d)",FCT,r);
00538         if (!isScalar()) sperror_this(F_LF);
00539         SIZE.init(r).set(1);
00540         IDX.init(D.len,r);
00541     }
00542
00543     return *this;
00544 };
00545
00546 template <class TD>
00547 bool wbsparray<TD>::isProptoId(TD &q, TD eps) const {
00548     if (!D.len) return 0;
00549     if (isDiag(FL)) {
00550         if (q==0) q=D[0];
00551         for (SPIDX_T i=1; i<D.len; ++i) { if (ABS(D[i]-q)>eps) return 0; }
00552         return 1;
00553     }
00554
00555     unsigned j, r=rank(FL), r2=r/2;
00556     SPIDX_T i=0,d=1; const SPIDX_T *id=IDX.data, *s=SIZE.data;
00557
00558     if (IDX.dim2!=r) sperror_this(FLF);
00559     Wb::scale_eps(eps,D.data,D.len);
00560
00561     if (!r || r%2) return 0;
00562     for (j=0; j<r2; ++j) { d*=s[j];
00563         if (!s[j] || s[j]!=s[j+r2]) return 0;
00564     }
00565
00566     if (double(eps)<=0) {
00567         if (D.len!=d) return 0;
00568         for (; i<IDX.dim1; ++i, id+=r) {
00569             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) return 0; }
00570             if (i || q!=0) { if (ABS(D[i]-q)>eps) return 0; }
00571             else q=D[i];
00572         }
00573     }
00574
00575     else {
00576         SPIDX_T n=0;
00577         for (; i<IDX.dim1; ++i, id+=r) {
00578             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) {
00579                 if (ABS(D[i])>eps) return 0;
00580                 else break;
00581             }}
00582             if (j==r2) {
00583                 if ((++n)!=1 || q!=0) { if (ABS(D[i]-q)>eps) return 0; }
00584                 else q=D[i];
00585             }
00586         }
00587         if (!n || n!=d) return 0;
00588     }
00589
00590     return 1;
00591 };
00592
00593 template <class TD>
00594 char wbsparray<TD>::isIdentity(TD eps) const {

```

```

00595
00596     if (!D.len) return 1;
00597     if (isDiag(FL)) {
00598         for (SPIDX_T i=1; i<D.len; ++i) { if (ABS(D[i]-1)>eps) return 2; }
00599         return 0;
00600     }
00601
00602     unsigned r=rank(FL), r2=r/2;
00603     SPIDX_T j,i=0,d=1; const SPIDX_T *id=IDX.data, *s=SIZE.data;
00604
00605     if (IDX.dim2!=r) sperror_this(FLF);
00606     Wb::scale_eps(eps,D.data,D.len);
00607
00608     if (!r || r%2) return 3;
00609     for (j=0; j<r2; ++j) { d*=s[j];
00610         if (!s[j] || s[j]!=s[j+r2]) return 4;
00611     }
00612
00613     if (eps<=0) {
00614         if (D.len!=d) return 5;
00615         for (; i<IDX.dim1; ++i, id+=r) {
00616             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) return 6; }
00617             if (ABS(D[i]-1)>eps) return 7;
00618         }
00619     }
00620     else {
00621         SPIDX_T n=0;
00622         for (; i<IDX.dim1; ++i, id+=r) {
00623             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) {
00624                 if (ABS(D[i])>eps) return 8;
00625                 else break;
00626             } }
00627             if (j==r2) { ++n;
00628                 if (ABS(D[i]-1)>eps) return 9;
00629             }
00630         }
00631         if (!n || n!=d) return 10;
00632     }
00633     return 0;
00634 };
00635
00636 template <class TD>
00637 char wbsparray<TD>::isIdentity(
00638     const wbpem &P, TD *dval_, TD eps) const {
00639
00640     TD dval=1;
00641
00642     if (!D.len) return 1;
00643     if (isDiag(FL)) {
00644         if (dval_) { (*dval_)=dval=D.avg(); eps*=dval; }
00645         for (SPIDX_T i=1; i<D.len; ++i) { if (ABS(D[i]-dval)>eps) return 2; }
00646         return 0;
00647     }
00648
00649     unsigned j1,j2, r=rank(FL);
00650     const SPIDX_T *id=IDX.data;
00651
00652     if (IDX.dim2!=r) sperror_this(FLF);
00653
00654     if (P.len) {
00655         if (P.len!=r || r<2) wblog(FL,"ERR %s() "
00656             "invalid permutation P=[%s] (r=%d)",FCT,STR(P),r);
00657         j1=P[0]; j2=P[1];
00658     }
00659     else {
00660         unsigned i=0; j1=j2=SIZE.len;
00661         for (; i<SIZE.len; ++i) { if (SIZE[i]>1) { j1=i; break; } }
00662         for (++i; i<SIZE.len; ++i) { if (SIZE[i]>1) { j2=i; break; } }
00663         for (++i; i<SIZE.len; ++i) { if (SIZE[i]>1) { break; } }
00664         if (j1==SIZE.len) {
00665             if (SIZE.allEqual(1)) {
00666                 if (D.len!=1) return 3;
00667                 dval=D[0]; if (dval_) { (*dval_)=dval; }
00668                 return 0;
00669             }
00670             else return 4;
00671         }
00672         if (j2==SIZE.len || i<SIZE.len) return 5;
00673     }
00674
00675     if (j1>SIZE.len || j2>SIZE.len || j1==j2) wblog(FL,
00676         "ERR %s() got (%d,%d/%d)",FCT,j1,j2,SIZE.len);
00677     if (SIZE[j1]!=SIZE[j2]) return 6;
00678
00679     Wb::scale_eps(eps,D.data,D.len);
00680
00681

```

```

00682     if (dval_) {
00683         SPIDX_T i=0, l=0; dval=0;
00684         for (; i<IDX.dim1; ++i, id+=r) {
00685             if (id[j1]==id[j2]) { dval+=D[i]; ++l; }
00686         }
00687         if (l!=SIZE[j1]) return 7;
00688         if (l) { dval/=l; eps*=dval; }; (*dval_)=dval;
00689         id=IDX.data;
00690     }
00691
00692     if (eps<=TD(0)) { if (D.len!=SIZE[j1]) return 8;
00693         for (SPIDX_T i=0; i<IDX.dim1; ++i, id+=r) {
00694             if (id[j1]!=id[j2]) return 12;
00695             if (ABS(D[i]-dval)>eps) return 11;
00696         }
00697     }
00698     else {
00699         SPIDX_T i=0, l=0;
00700         for (; i<IDX.dim1; ++i, id+=r) {
00701             if (id[j1]==id[j2]) { ++l;
00702                 if (ABS(D[i]-dval)>eps) return 11; }
00703             else { if (ABS(D[i])>eps) return 12; }
00704         }
00705         if (!l || l!=SIZE[j1]) return 7;
00706     }
00707
00708     return 0;
00709 };
00710
00711 template <class TD>
00712 bool wbsparray<TD>::isSym_aux(
00713     const char *F, int L, const char *fct,
00714     const wbsparray<TD> &B0, TD eps, TD* xref,
00715     const char symflag,
00716     const char lflag
00717 ) const {
00718
00719     wbsparray<TD> B; B0.transpose(F_L,B);
00720     SPIDX_T i,r,s; widxt m,ma,mb;
00721     wbindx Ia,Ib;
00722     wvector<SPIDX_T> Ja(IDX.dim1), Jb(B.IDX.dim1);
00723     TD x;
00724
00725     checkSize(FL,"A: "); if (&B!=this) checkSize(FL,"B: ");
00726
00727     if (IDX.dim2!=B.IDX.dim2 || IDX.dim2%2) {
00728         if (lflag) sprintf(str,
00729             "%s %s() only applies to even-rank objects (%ld;%ld).",
00730             shortFL(F,L),fct, SIZE.len, B.SIZE.len);
00731         return 0;
00732     }
00733     if (isEmpty() && B.isEmpty()) return 1;
00734
00735     m=matchIndex(IDX,B.IDX,Ia,Ib,1,&ma,&mb);
00736     if (m) wblog(FL,"ERR %s() input sparse matrices "
00737         "not compressed (%d,%d,%d) !?",FCT,m,ma,mb);
00738
00739     if (xref) (*xref)=0;
00740
00741     if (symflag=='s') {
00742         for (i=0; i<Ia.len; ++i) { r=Ia[i]; s=Ib[i];
00743             if ((++Ja[r])>1 || (++Jb[s])>1) {
00744                 wblog(FL,"ERR %s()",FCT);
00745             }
00746             x=ABS(D[r]-CONJ(B.D[s])); if (xref) *xref=MAX(*xref,x);
00747             if (x>eps) return 0;
00748         }
00749     }
00750     else if (symflag=='a') {
00751         for (i=0; i<Ia.len; ++i) { r=Ia[i]; s=Ib[i];
00752             if ((++Ja[r])>1 || (++Jb[s])>1) {
00753                 wblog(FL,"ERR %s()",FCT);
00754             }
00755             x=ABS(D[r]+CONJ(B.D[s])); if (xref) *xref=MAX(*xref,x);
00756             if (x>eps) return 0;
00757         }
00758     }
00759     else wblog(FL,"ERR invalid symflag=%c<%d>",symflag,symflag);
00760
00761     for (i=0; i<Ja.len; ++i) { if (!Ja[i] && ABS(D[i])>eps) return 0; }
00762
00763     if (&B0!=this)
00764         for (i=0; i<Jb.len; ++i) { if (!Jb[i] && ABS(B.D[i])>eps) return 0; }
00765
00766     return 1;
00767 };
00768

```

```

00769 template <class TD>
00770 TD wbsparray<TD>::dotProd(
00771     const char *F, int L, const wbsparray<TD> &B, char tnorm
00772 ) const {
00773
00774     TD x=TD();
00775     char cflag = ((tnorm || WbUtil<TD>::hasNoConj()) ? 0 : 1);
00776     char iA=isDiag(), iB=B.isDiag();
00777
00778     if (iA || iB) {
00779         if ((!iA && SIZE.len!=2) || (!iB && B.SIZE.len!=2) || dim()!=B.dim())
00780             wblog(FL,"ERR %s() size mismatch (%s <> %s)",
00781                 FCT,SSTR_(this),SSTR(B)
00782             );
00783
00784         if (iA && iB) {
00785             const TD *a=D.data, *b=B.D.data; SPIDX_T i=0;
00786             if (cflag)
00787                 { for (; i<D.len; ++i) x+=CONJ(a[i])*b[i]; }
00788             else { for (; i<D.len; ++i) x+=      a[i] *b[i]; }
00789         }
00790         else if (iA)
00791             { x=B.dotProd_full_diag(*this, cflag ? 2:0); }
00792         else { x= dotProd_full_diag(B,      cflag ? 1:0); }
00793
00794         return x;
00795     }
00796
00797     if (SIZE!=B.SIZE) wblog(F_L,"ERR %s() "
00798         "got size mismatch (%s <> %s)",FCT,SSTR_(this),SSTR(B));
00799     if (!SIZE.len) { return x; }
00800
00801     SPIDX_T ia=0, ib=0, na=IDX.dim1, nb=B.IDX.dim1, l=0, lnext=0;
00802     const SPIDX_T *Ia=IDX.data, *Ib=B.IDX.data, Davg=SQRT(SIZE.prod());
00803     const TD *a=D.data, *b=B.D.data;
00804     unsigned m=SIZE.len;
00805     char q=0, lex=0;
00806
00807     while (ia<na && ib<nb) {
00808         q=Wb::recCompare(Ia,Ib,m,lex);
00809
00810         if (q<0) { ++ia; Ia+=m; } else
00811         if (q>0) { ++ib; Ib+=m; } else {
00812             if (cflag)
00813                 { x+=CONJ(a[ia])*b[ib]; }
00814             else { x+=      a[ia] *b[ib]; }
00815             ++ia; ++ib; Ia+=m; Ib+=m;
00816
00817             if ((++l)>=lnext) { lnext+=Davg;
00818                 if (ia<na) { q=Wb::recCompare(Ia-m,Ia,m,lex);
00819                     if (q>=0) wblog(F_L,"ERR %s() A: "
00820                         "input sparse IDX not sorted (%ld/%ld) !?",FCT,ia,na); }
00821                 if (ib<nb) { q=Wb::recCompare(Ib-m,Ib,m,lex);
00822                     if (q>=0) wblog(F_L,"ERR %s() B: "
00823                         "input sparse IDX not sorted (%ld/%ld) !?",FCT,ib,nb);
00824                 }
00825             }
00826         }
00827     }
00828
00829     return x;
00830 };
00831
00832 template <class TD> inline
00833 TD wbsparray<TD>::dotProd_full_diag(const wbsparray<TD> &B, char cflag
00834 ) const {
00835
00836     const TD *a=D.data, *b=B.D.data;
00837     const SPIDX_T *I=IDX.data;
00838     TD x=TD();
00839
00840     if (IDX.dim2!=2 || B.IDX.dim1 || B.D.len!=SIZE[0]) wblog(FL,
00841         "ERR %s() %s <> %s !?",FCT, SSTR_(this), SSTR(B)
00842     );
00843
00844     if (cflag==0) {
00845         for (SPIDX_T i=0; i<D.len; ++i, I+=2) {
00846             if (I[0]==I[1]) { x += a[i] * b[I[0]]; }
00847         }
00848     }
00849     else if (cflag==1) {
00850         for (SPIDX_T i=0; i<D.len; ++i, I+=2) {
00851             if (I[0]==I[1]) { x += CONJ(a[i]) * b[I[0]]; }
00852         }
00853     }
00854     else if (cflag==2) {
00855         for (SPIDX_T i=0; i<D.len; ++i, I+=2) {

```

```

00856         if (I[0]==I[1]) { x += a[i] * CONJ(b[I[0]]); }
00857     }
00858 }
00859 else wblog(FL,"ERR %s() invalid cflag=%d",FCT,cflag);
00860
00861 return x;
00862 };
00863
00864 template <class TD>
00865 wbsparray<TD>& wbsparray<TD>::TimesEl(
00866     const wbsparray<TD> &B, char tnorm
00867 ){
00868     char cflag = ((tnorm || WbUtil<TD>::hasNoConj()) ? 0 : 1);
00869     char iA=0, iB=0, sab=sameSize(B);
00870
00871     if (!sab || sab>=20) wblog(FL,"ERR %s() "
00872         "size mismatch (%s <> %s)",FCT,SSTR_(this),SSTR(B));
00873
00874     if (sab>=10) { wbsparray<TD> X;
00875
00876         iA=((sab-10) & 1);
00877         iB=((sab-10) & 2);
00878
00879         if (iA && iB) {
00880             SPIDX_T i=0; const TD *a=D.data; TD *x;
00881             X=B; x=X.D.data;
00882             if (cflag)
00883                 { for (; i<D.len; ++i) x[i]*=CONJ(a[i]); }
00884             else { for (; i<D.len; ++i) x[i]*= a[i]; }
00885         }
00886         else if (iA)
00887             { B.timesEl_full_diag(*this,X, cflag ? 2:0); }
00888         else { timesEl_full_diag(B, X, cflag ? 1:0); }
00889     }
00890     else {
00891         wbindex Ia,Ib; SPIDX_T i=0, m,ma=0,mb=0;
00892         const TD *a=D.data, *b=B.D.data;
00893
00894         m=matchIndex(IDX,B.IDX,Ia,Ib,1,&ma,&mb);
00895         if (m) wblog(FL,"ERR %s() input sparse "
00896             "objects not compressed (%d,%d,%d) !?",FCT,m,ma,mb);
00897
00898         wbsparray<TD> X(SIZE,Ia.len);
00899         TD *x=X.D.data;
00900
00901         for (i=0; i<Ia.len; ++i) { X.IDX.recSetP(i, IDX.rec(Ia[i])); }
00902         if (cflag)
00903             { for (i=0; i<Ia.len; ++i) { x[i]=CONJ(a[Ia[i]])*b[Ib[i]]; } }
00904         else { for (i=0; i<Ia.len; ++i) { x[i]= a[Ia[i]] *b[Ib[i]]; } }
00905
00906         X.save2(*this).Compress();
00907     }
00908
00909     return *this;
00910 };
00911
00912 template <class TD> inline
00913 wbsparray<TD>& wbsparray<TD>::timesEl_full_diag(
00914     const wbsparray<TD> &B, wbsparray<TD> &X, char cflag
00915 ) const {
00916
00917     const TD *a=D.data, *b=B.D.data;
00918     const SPIDX_T *I=IDX.data;
00919
00920     X.initDiag(B.D.len); TD *x=X.D.data;
00921
00922     if (IDX.dim2!=2 || B.IDX.dim1 || B.D.len!=IDX.dim1) wblog(FL,
00923         "ERR %s() (%d,%d)x%d <> (%d,%d)x%d !?",FCT,
00924         D.len, IDX.dim1,IDX.dim2, B.D.len, B.IDX.dim1,B.IDX.dim2
00925     );
00926
00927     if (cflag==0) {
00928         for (SPIDX_T i=0; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1]) {
00929             x[I[0]]=a[i]*b[I[0]];
00930         } }
00931     }
00932     else if (cflag==1) {
00933         for (SPIDX_T i=0; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1]) {
00934             x[I[0]]=CONJ(a[i])*b[I[0]];
00935         } }
00936     }
00937     else if (cflag==2) {
00938         for (SPIDX_T i=0; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1]) {
00939             x[I[0]]=a[i]*CONJ(b[I[0]]);
00940         } }
00941     }
00942     else wblog(FL,"ERR %s() invalid cflag=%d",FCT,cflag);

```

```

00943     return X;
00944 };
00945
00946 template <class TD> inline
00947 wbsparray<TD>& wbsparray<TD>::setCol(
00948     const char *F, int L, SPIDX_T k, const wbsparray<TD> &v
00949 ){
00950     unsigned rv=v.rank(F_L);
00951
00952     if (rank(F_L)!=2 || rv<1 || rv>2 || SIZE[0]!=v.SIZE[0] ||
00953         (rv==2 && v.SIZE[1]!=1)
00954         ) wblog(F_L,"ERR %s() invalid input (%s <> %s)",
00955             FCT, SSTR_(this), SSTR(v)
00956         );
00957
00958     if (k>=SIZE[1]) wblog(F_L,
00959         "ERR %s() index out of bounds (%d/%d)",FCT,k,SIZE[1]);
00960     if ((IDX.dim1 && IDX.dim2!=2) || v.IDX.dim2!=rv) wblog(F_L,
00961         "ERR %s() %dx%d <> %dx%d/%d !?",FCT,IDX.dim1,IDX.dim2,
00962         v.IDX.dim2,v.IDX.dim2,rv);
00963
00964     SPIDX_T i=0, l=0, l2, n=IDX.dim1, *I2=IDX.data+1;
00965
00966     for (; i<n; ++i, I2+=2) { if ((*I2)!=k) {
00967         if (l<i) { setRec(l,i); }; ++l;
00968     } }
00969
00970     if (l<i && F) wblog(F_L,
00971         "WRN %s() *this already contains data at col=%d/%d (%d/%d)",
00972         FCT,k,SIZE[1],l,SIZE[0]
00973     );
00974     if (!l) {
00975         if (v.IDX.dim2==1)
00976             { v.IDX.resize(v.IDX.dim1, 2, IDX); }
00977         else { IDX=v.IDX; }
00978         IDX.setLastCol(k);
00979
00980         D=v.D; return *this;
00981     }
00982
00983     n=v.D.len; l2=l+n;
00984     if (l2) {
00985         if (l2<=IDX.dim1)
00986             { REDSIZE_NNZ(l2); }
00987         else { IDX.Resize(l2,IDX.dim2); D.Resize(l2); }
00988     }
00989     else { IDX.init(0,IDX.dim2); D.init(); }
00990
00991     if (n) {
00992         const SPIDX_T *Iv=v.IDX.data;
00993         Wb::cpyRange(D.data+1, v.D.data, n); I2=IDX.rec(l);
00994         for (i=0; i<n; ++i, I2+=2, Iv+=rv) {
00995             I2[0]=(*Iv);
00996             I2[1]=k;
00997         }
00998     }
00999     return *this;
01000 };
01001
01002 template <class TD> inline
01003 TD wbsparray<TD>::Normalize(char tnorm, char qflag){
01004     TD x = SQR(Tb::overlap(D.data,D.data,D.len,1,tnorm));
01005
01006     if (ABS(x)>TD(1E-15)) Wb::timesRange(D.data,TD(1)/x,D.len);
01007     else if (!qflag) {
01008         wblog(F_L,"ERR %s() got vector with norm %.3g !?",FCT,double(x));
01009     }
01010
01011     return x;
01012 };
01013
01014 template <class TD>
01015 TD wbsparray<TD>::NormalizeCol(SPIDX_T k, char tnorm, char qflag){
01016
01017     if (rank() !=2 || IDX.dim2!=2) wblog(F_L,
01018         "ERR %s() requires rank-2 (got %s)",FCT,SSTR_(this));
01019     if (k>=SIZE[1]) wblog(F_L,
01020         "ERR %s() index out of bounds (%d/%d)",FCT,k,SIZE[1]);
01021     checkSize(F_LF);
01022
01023     SPIDX_T i=0, n=IDX.dim1, *I2=IDX.data+1;
01024     opFlags<TD> xflags(tnorm ? 'N' : 'C');
01025     TD x=0;
01026
01027     if (xflags.conj())
01028         { for (; i<n; ++i, I2+=2) if ((*I2)==k) { x+=CONJ(D[i])*D[i]; } }
01029     else { for (; i<n; ++i, I2+=2) if ((*I2)==k) { x+=D[i]*D[i]; } }

```

```

01030
01031     x=SQRT(x);
01032     if (ABS(x)>1E-15) { x=1/x;
01033         for (i=0; i<n; ++i, I2+=2) if ((*I2)==k) { D[i]*=x; }
01034     }
01035     else if (!qflag) {
01036         wblog(FL,"ERR %s() got vector with norm %.3g !?",FCT,x);
01037     }
01038
01039     return x;
01040 };
01041
01042 template <class TD>
01043 wbspararray<TD>& wbspararray<TD>::QRdecomp(const char *F, int L,
01044     wbspararray<TD> &Q,
01045     char posR,
01046     TD eps
01047 ){
01048     if (rank()!=2 || SIZE.len!=2 || IDX.dim2!=2) wblog(F_L,
01049         "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR_(this));
01050
01051     SPIDX_T d1=SIZE[0], d2=SIZE[1], *idx=IDX.data;
01052     if (!d1 || !d2) { Q.init(); return *this; }
01053
01054     SPIDX_T i,l, k=0, k_, l1=0, l2=0, iu=0, N=IDX.dim1;
01055     TD x2, c2=0, e2=0, eps2=eps*eps;
01056     wbspararray<TD> u,X;
01057     int dk=0;
01058
01059     eps2*=SQRT(TD(MAX(d1,d2)));
01060
01061     if (eps2>TD(1E-8)) wblog(F_L,
01062         "ERR %s() got eps=%.3g !?",FCT,double(eps));
01063
01064     if (!N) { SPIDX_T one=1;
01065         Q.initIdentity(d1,one); init(one,d2);
01066         return *this;
01067     }
01068
01069     if (N==1) {
01070         wvector<SPIDX_T> s(2); wbspararray<TD> R;
01071         s[0]=d1; s[1]=1; Q.init(s,1); Q.IDX(0,0)=IDX(0,0);
01072         s[0]=1; s[1]=d2; R.init(s,1); R.IDX(0,1)=IDX(0,1);
01073
01074         if (posR && D[0]<0)
01075             { Q.D[0]=-1; R.D[0]=-D[0]; }
01076         else { Q.D[0]= 1; R.D[0]= D[0]; }
01077
01078         return R.save2(*this);
01079     }
01080
01081     wvector< wbspararray<TD> > U(d2);
01082
01083     wvector<char> sR;
01084     if (posR) { posR=1; sR.init2val(d2,1); }
01085
01086     wbspararray<TD> A_(*this);
01087
01088     for (; k<=d2; ++k) {
01089         N=IDX.dim1;
01090
01091         if (k) {
01092             if (l1>=N) l1=N-1;
01093             if (IDX(l1,1)>=k_) {
01094                 while (l1>0) { if (IDX(--l1,1)<k_) { ++l1; break; } }
01095             }
01096             if (IDX(l1,1)!=k_) wblog(FL,
01097                 "ERR %s() QR out of pace (%ld/%ld) !?",FCT,IDX(l1,1),k_);
01098             l2=l1;
01099         }
01100
01101         idx=IDX.rec(l2); k_=idx[1]; dk=-1;
01102
01103         while (l2<N) {
01104             while (l2<N && idx[0]<k && idx[1]==k_) { ++l2; idx+=2; }
01105
01106             if ((++dk)==0 && posR && k) {
01107                 const TD &rk=D[l2-1];
01108
01109                 if (idx[-2]!=k-1) {
01110                     MXPut(FL,"q").add(A_, "A")
01111                         .add(*this, "R").add(IDX, "IDX").add(D, "D")
01112                         .add(l1, "l1").add(l2, "l2").add(k, "k").add(k_, "k_")
01113                         .add(c2, "c2").add(e2, "e2").add(eps, "eps");
01114                     wblog(FL,
01115                         "ERR %s() got R(%ld/%ld,%ld)=%.4g (l1=%ld, l2=%ld)",
01116                         FCT,idx[-2],k-1,k_,double(rk),l1,l2-1

```



```

01117         );
01118     }
01119     if (NORM2(rk) <= eps2) {
01120         sprintf(str, "got R(%ld,%ld)=%.4g", k-1, k_, double(rk));
01121         if (rk) wblog(FL, "WRN %s() %s", FCT, str);
01122         else wblog(FL, "ERR %s() %s", FCT, str);
01123     }
01124     if (rk < 0) { sR[k-1] = -1; posR = -1; }
01125 }
01126
01127 if (l2 > N && idx[-2] < k) { l1 = l2; break; }
01128 if (idx[l1] != k_) {
01129     k_ = idx[l1]; continue;
01130 }
01131
01132 l1 = l2; c2 = NORM2(D[l2]); idx += 2; ++l2;
01133 while (l2 < N && idx[l1] == k_) { idx += 2; c2 += NORM2(D[l2++]); }
01134
01135 if (c2 <= eps2 || (l1 && dk == 0)) { e2 += c2;
01136     if (c2 > 100 * eps2) wblog(FL, "ERR %s() "
01137         "got c2=%.3g (%.3g) !?", FCT, double(c2), double(eps2));
01138     for (l = l1; l < l2; ++l) D[l] = 0;
01139     if (l2 < N) {
01140         k_ = idx[l1]; continue;
01141     }
01142     else { l1 = l2; break; }
01143 }
01144 else {
01145     break;
01146 }
01147 }; if (l1 >= N) break;
01148
01149 if (k_ < k || k >= d2) {
01150     MXPut(FL, "q").add(A_, "A")
01151         .add(*this, "R").add(IDX, "IDX").add(D, "D")
01152         .add(l1, "l1").add(l2, "l2").add(k, "k").add(k_, "k_")
01153         .add(c2, "c2").add(e2, "e2").add(eps, "eps");
01154     wblog(FL, "ERR %s() k_=%ld/%ld/%ld !?", FCT, k_, k, d2);
01155 }
01156
01157 x2 = getHouseholderVec(FL, k, k_, u, eps2);
01158
01159 if (x2 < eps2 * 100) {
01160     if (x2 > eps2) wblog(FL, "WRN %s() got x2 @ %.3g / %.3g "
01161         "[%ldx%ld; %d]", FCT, double(x2), double(eps2), d1, d2, dk);
01162     continue;
01163 }
01164
01165 LProject1(u, X, -2);
01166 Plus(FL, X);
01167 u.save2(U[iu++]);
01168 }
01169
01170 if (!k || k > d2) wblog(FL, "ERR %s() got k=%ld/%ld !?", FCT, k, d2);
01171
01172 if (k < d1) {
01173     N = IDX.dim1; idx = IDX.data; c2 = 0;
01174     for (l = 0; l < N; ++l, idx += 2) {
01175         if (idx[0] >= k) { c2 += NORM2(D[l]); }
01176     }
01177     if (c2 > eps2) { MXPut(FL, "q").add(*this, "R").add(c2, "c2");
01178         wblog(FL, "ERR %s() got non-triangular R matrix (%.3g) !?",
01179             FCT, double(c2));
01180     }
01181     SPIDX_T s[2] = {k, d2};
01182     Resize(wbvector<SPIDX_T>(2, s));
01183 }
01184 e2 += SkipTiny(eps);
01185
01186 Q.initIdentity(d1, MIN(d1, k));
01187 if (iu) {
01188     for (k = iu - 1; k < iu; --k) {
01189         Q.LProject1(U[k], X, -2);
01190         Q.Plus(FL, X);
01191     }
01192 }
01193 e2 += Q.SkipTiny(eps);
01194
01195 if (posR < 0) {
01196     TD *x = Q.D.data; N = Q.IDX.dim1; idx = Q.IDX.data;
01197     for (i = 0; i < N; ++i, idx += 2) { if (sR[idx[1]] < 0) x[i] = -x[i]; }
01198     x = D.data; N = IDX.dim1; idx = IDX.data;
01199     for (i = 0; i < N; ++i, idx += 2) { if (sR[idx[0]] < 0) x[i] = -x[i]; }
01200 }
01201
01202 return *this;
01203 };

```

```

01204
01205 template <class TD>
01206 wbspararray<TD>& wbspararray<TD>::OrthoNormalizeColsQR(
01207     const char *F, int L,
01208     TD eps
01209 ){
01210     wbspararray<TD> Q;
01211
01212     QRdecomp(F_L,Q,'+',eps);
01213
01214     if (SIZE.len==2 || D.len) {
01215         SPIDX_T l, n=D.len, d2=SIZE[1]; char e=0; int lmax=-1;
01216         wvector<TD> dd(d2); TD eps=1E-12;
01217         wvector<int> id; id.init2val(d2,-1);
01218
01219         for (l=0; l<n; ++l) { if (NORM(D[l])>eps) {
01220             dd[IDX(l,l)]=D[l];
01221             id[IDX(l,l)]=IDX(l,0);
01222         }}
01223
01224         for (l=0; l<d2 && !e; ++l) {
01225             if (id[l]>lmax) {
01226                 if (id[l]==lmax+1) { lmax=id[l];
01227                     if (dd[l]<0) { e=1;
01228                         sprintf(str,"dd[%ld]=%.3g",l,double(dd[l]));
01229                     }
01230                 }
01231                 else { e=2; sprintf(str,"%ld: %d -> %d",l,lmax,id[l]); }
01232             }
01233         }
01234
01235         if (e) { MXPut(FL,"q")
01236             .add(*this,"R").add(Q,"Q").add(SIZE,"S")
01237             .add(IDX,"IDX").add(D,"D").add(dd,"dd").add(id,"id");
01238             wblog(FL,"ERR %s() %s (eps=%g) !?",FCT,str,double(eps));
01239         }
01240     }
01241
01242     return Q.save2(*this);
01243 };
01244
01245 template <class TD>
01246 wbspararray<TD>& wbspararray<TD>::OrthoNormalizeColsGS(
01247     const char *F, int L,
01248     char tnorm,
01249     char qflag,
01250     TD eps,
01251     unsigned np
01252 ){
01253     if (rank()!=2) wblog(FL,
01254         "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR(this));
01255
01256     SPIDX_T i,j,ip;
01257     char xflag=(qflag=='x' || qflag=='X');
01258     wvector< wbspararray<TD> > X;
01259     wvector<SPIDX_T> I0;
01260     TD a,z;
01261
01262     splitSparseCM(F_L,X, xflag ? eps : TD(0));
01263
01264     WbUtil<TD>().adjust_tnorm(tnorm);
01265
01266     for (i=0; i<X.len; ++i) {
01267         for (ip=0; ip<np; ++ip) {
01268             for (j=0; j<i; ++j) { if (X[j].D.len) {
01269                 z=X[i].dotProd(FL,X[j],tnorm); if (z!=0) {
01270                     X[i].Plus(FL,X[j],-z); }
01271             }}
01272         }
01273
01274         z=X[i].norm2(tnorm); a=ABS(z);
01275
01276         if (a>eps) { X[i]*=(TD(1)/SQRT(a)); }
01277         else {
01278             if (qflag) {
01279                 X[i].initz();
01280             }
01281             else if (!xflag) {
01282                 MXPut(FL,"x").add(*this,"V").add(X,"X")
01283                     .add(qflag,"qflag").add(xflag,"xflag")
01284                     .add(i+1,"i").add(a,"a").add(eps,"eps");
01285                 wblog(F_L,"ERR |U(:,%d/%d)| = %.3g (%g) %s !?", i+1, X.len,
01286                     double(a), double(eps), tnorm ? " (using t-norm)" : "");
01287             }
01288         }
01289     }
01290 }

```

```

01291
01292     wbvector< wbsparray<TD>* > xp(X.len);
01293     if (xflag) {
01294         for (j=i=0; i<X.len; ++i) { if (X[i].D.len) {
01295             xp[j++]=(X[i]);
01296         }}
01297         if (j<i) {
01298             if (!j) wblog(F_L, "ERR %s() got all null-vectors", FCT);
01299             xp.len=j;
01300         }
01301     }
01302     else {
01303         for (i=0; i<X.len; ++i) { xp[i]=(X[i]); }
01304     }
01305
01306     initCAT(FL, xp, &I0);
01307     if (!IDX.dim1) wblog(F_L, "ERR %s() got all null-vectors", FCT);
01308
01309     return *this;
01310 };
01311
01312 template <class TD>
01313 template <class T2>
01314 wbvector<T2>& wbsparray<TD>::norm2vec(unsigned l,
01315     wbvector<T2> &a, char tnorm
01316 ) const {
01317
01318     if (l>=SIZE.len) wblog(FL,
01319         "ERR %s() index out of bounds (l=%d/%d)", FCT, l, SIZE.len);
01320     a.init(SIZE[l]);
01321
01322     unsigned m=IDX.dim2;
01323     SPIDX_T i=0, *ip=IDX.data+l;
01324     T2 *ad=a.data;
01325
01326     if (tnorm)
01327         for (; i<D.len; ++i, ip+=m) { ad[*ip] += D[i]*D[i]; }
01328     else { for (; i<D.len; ++i, ip+=m) { ad[*ip] += CONJ(D[i])*D[i]; } }
01329
01330     return a;
01331 };
01332
01333 template <class TD>
01334 TD wbsparray<TD>::trace() const {
01335
01336     TD x=0;
01337
01338     unsigned r=rank(FL); if (!r) return x;
01339
01340     if (r==2 && !IDX.dim1) return D.sum();
01341     if (r%2 || r!=SIZE.len || r!=IDX.dim2) sperror_this(FLF);
01342
01343     SPIDX_T i=0; unsigned r2=r/2, n=r2*sizeof(SPIDX_T);
01344     const SPIDX_T *idx=IDX.data;
01345
01346     for (; i<r2; ++i) {
01347         if (SIZE[i]!=SIZE[i+r2]) wblog(FL,
01348             "ERR %s() got non-symmetric tensor (%s)", FCT, SSTR_(this));
01349     }
01350
01351     for (i=0; i<IDX.dim1; ++i, idx+=r) {
01352         if (memcmp(idx, idx+r2, n)==0) x+=D[i];
01353     }
01354
01355     return x;
01356 };
01357
01358 template <class TD>
01359 template <class T2>
01360 wbvector<T2>& wbsparray<TD>::trace(
01361     unsigned k, wbvector<T2> &A
01362 ) const {
01363
01364     SPIDX_T i=0;
01365     unsigned r=IDX.dim2, r2=(r-1)/2, n=r2*sizeof(SPIDX_T);
01366     const SPIDX_T *idx=IDX.data+(k?0:1), *sz=SIZE.data+(k?0:1);
01367     int l=(k ? r-1 : -1);
01368
01369     if (k && k+1!=SIZE.len) wblog(FL,
01370         "ERR %s() only accepts k=1 or k=rank (%d/%d)", FCT, k+1, r);
01371     if (SIZE.len%2!=1) wblog(FL, "ERR %s() "
01372         "requires odd-rank tensor (r=%d; k=%d)", FCT, SIZE.len, k);
01373     if (SIZE.len!=IDX.dim2 || IDX.dim1!=D.len) sperror_this(FLF);
01374
01375     A.init(SIZE[k]);
01376     T2 *a=A.data;
01377

```

```

01378     for (; i<r2; ++i) {
01379         if (sz[i]!=sz[i+r2]) wblog(FL, "ERR %s() "
01380             "got non-symmetric tensor (%s; k=%d)", FCT, SSTR_(this), k+1);
01381     }
01382
01383     for (i=0; i<IDX.dim1; ++i, idx+=r) {
01384         if (memcmp(idx, idx+r2, n)==0) {
01385             if (idx[l]>=A.len) wblog(FL, "ERR %s() "
01386                 "index out of bounds (%d,%d: %d/%d)", FCT, i, l, idx[l], A.len);
01387             a[idx[l]]+=D[i];
01388         }
01389     }
01390
01391     return A;
01392 };
01393
01394 bool mxIsWbssparray(
01395     const char *F, int L, const mxArray *a, unsigned k
01396 ){
01397
01398     if (!a || mxIsEmpty(a)) { return 0; }
01399
01400     SPIDX_T n=mxGetNumberOfElements(a);
01401     if (k>=n) { if (F) wblog(F, L,
01402         "ERR sparse::%s() index exceeds dimension (%d/%d)", FCT, k, n);
01403         return 0;
01404     }
01405
01406     int e[3] {
01407         mxGetFieldNumber(a, "S"),
01408         mxGetFieldNumber(a, "idx"), mxGetFieldNumber(a, "data")
01409     };
01410
01411     if (e[0]<0 || e[1]<0 || e[2]<0) { if (F) wblog(FL,
01412         "ERR %s() invalid sparse array (missing fields%s%s)", FCT,
01413         e[0]?" S":"", e[1]?" idx":"", e[2]?" data":"" );
01414         return 0;
01415     }
01416
01417     return 1;
01418 };
01419
01420 bool mxIsWbssparray(
01421     const char *F, int L, const mxArray *a, unsigned k,
01422     const mxArray **as_, const mxArray **ad_, const mxArray **ai_
01423 ){
01424
01425     if (!a || mxIsEmpty(a)) { return 0; }
01426
01427     SPIDX_T n=mxGetNumberOfElements(a);
01428     if (k>=n) { if (F) wblog(F, L,
01429         "ERR sparse::%s() index exceeds dimension (%d/%d)", FCT, k, n);
01430         return 0;
01431     }
01432
01433     const mxArray
01434         *as=mxGetField(a, k, "S"),
01435         *ai=mxGetField(a, k, "idx"),
01436         *ad=mxGetField(a, k, "data");
01437
01438     if (!as || !ad) { if (F) {
01439         if (as && mxGetNumberOfElements(as)) wblog(F_L,
01440             "ERR %s() invalid sparse array (0x%lX, 0x%lX: S (%d)",
01441             FCT, as, ad, as ? mxGetNumberOfElements(as):-1);
01442         if (ad && mxGetNumberOfElements(ad)) wblog(F_L,
01443             "ERR %s() invalid sparse array (0x%lX, 0x%lX: data (%d)",
01444             FCT, as, ad, ad ? mxGetNumberOfElements(ad):-1);
01445     }
01446         return 0;
01447     }
01448
01449     if (as_) (*as_)=as;
01450     if (ad_) (*ad_)=ad;
01451     if (ai_) (*ai_)=ai;
01452
01453     return 1;
01454 };
01455
01456 template <class TD>
01457 wbsparray<TD>& wbsparray<TD>::toScalar(const char *F, int L, unsigned r) {
01458
01459     if (D.len>1 || (SIZE.len && !SIZE.allEqual(1))) wblog(F_L,
01460         "ERR %s() invalid scalar %s", FCT, info2Str().data);
01461
01462     if (!D.len) { D.init(1); D[0]=0; }
01463
01464     if (int(r)<=0) {

```

```

01465         if (SIZE.data) SIZE.init();
01466         if (IDX.dim1 || IDX.dim2) IDX.init();
01467     }
01468     else {
01469         if (SIZE.len) { if (SIZE.len!=r) wblog(FL,
01470             "ERR %s() got rank mismatch (%d/%d)",FCT,SIZE.len,r); }
01471         else {
01472             SIZE.init(r).set(1);
01473             IDX.init(1,r);
01474         }
01475     }
01476 }
01477 return *this;
01478 };
01479
01480 template <class TD>
01481 TD wbsparray<TD>::getScalar(const char *F, int L) const {
01482
01483     if (D.len>1 || (SIZE.len && !SIZE.allEqual(1))) wblog(F_L,
01484         "ERR %s() invalid scalar %s",FCT,info2Str().data);
01485
01486     return (D.len ? D[0] : TD(0));
01487 };
01488
01489 template <class TD>
01490 wbvector<TD>& wbsparray<TD>::getCol(SPIDX_T k, wbvector<TD> &v) const {
01491
01492     if (isDiag(FL)) {
01493         if (sSPIDX_T(k)<0) { k+=D.len; }
01494         if (k>=D.len) wblog(FL,
01495             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01496         v.init(D.len); v[k]=D[k];
01497     }
01498     else {
01499         if (SIZE.len!=2) wblog(FL,"ERR %s() invalid S=%d",FCT,SSTR_(this));
01500         if (sSPIDX_T(k)<0) { k+=SIZE[1]; }
01501         if (k>=SIZE[1]) wblog(FL,
01502             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01503
01504         SPIDX_T i=0, n=SIZE[0]; const SPIDX_T *I=IDX.data;
01505         v.init(n);
01506
01507         for (; i<IDX.dim1; ++i, I+=2) { if (I[1]==k) {
01508             if (*I>=n) wblog(FL,"ERR %s() "
01509                 "index out of bounds (%d/%d/ %s)",FCT,*I,n,SSTR_(this));
01510             v[*I]+=D[i];
01511         }}
01512     }
01513     return v;
01514 };
01515
01516 template <class TD>
01517 wbvector<TD>& wbsparray<TD>::getRow(SPIDX_T k, wbvector<TD> &v) const {
01518
01519     if (isDiag(FL)) {
01520         if (sSPIDX_T(k)<0) { k+=D.len; }
01521         if (k>=D.len) wblog(FL,
01522             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01523         v.init(D.len); v[k]=D[k];
01524     }
01525     else {
01526         if (SIZE.len!=2) wblog(FL,"ERR %s() invalid S=%d",FCT,SSTR_(this));
01527         if (sSPIDX_T(k)<0) { k+=SIZE[0]; }
01528         if (k>=SIZE[0]) wblog(FL,
01529             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01530
01531         SPIDX_T i=0, n=SIZE[1]; const SPIDX_T *I=IDX.data;
01532         v.init(n);
01533
01534         for (; i<IDX.dim1; ++i, I+=2) { if (*I==k) {
01535             if (I[1]>=n) wblog(FL,"ERR %s() "
01536                 "index out of bounds (%d/%d/ %s)",FCT,I[1],n,SSTR_(this));
01537             v[I[1]]+=D[i];
01538         }}
01539     }
01540     return v;
01541 };
01542
01543 template <class TD>
01544 int wbsparray<TD>::checkTrailingSingletons(
01545     const char *F, int L, unsigned *r
01546 ) const {
01547
01548     if (SIZE.len!=IDX.dim2) wblog(F_L,
01549         "ERR %s() size mismatch (%d/%d)",FCT,SIZE.len,IDX.dim2);
01550     if (!SIZE.len) {
01551         if (r && int(*r)>=0) wblog(FL,

```

```

01552         "ERR %s() out of bounds (%d/%d)",FCT,*r,SIZE.len);
01553     return 0;
01554 }
01555
01556 unsigned l=SIZE.len-1;
01557 for (; l<SIZE.len; --l) { if (SIZE[l]!=1) break; }; ++l;
01558
01559 if (r) {
01560     if (int(*r)>=0) {
01561         if ((*r)>SIZE.len) wblog(FL,
01562             "ERR %s() out of bounds (%d/%d)",FCT,*r,SIZE.len);
01563         if (l>(*r)) wblog(FL,"ERR %s() "
01564             "got non-singletons for dim>=%d (%d,%d)",FCT,*r,l,SIZE.len);
01565         l=(*r);
01566     }
01567     else (*r)=l;
01568 }
01569
01570 return (SIZE.len-1);
01571 };
01572
01573 template <class TD>
01574 int wbsparray<TD>::skipTrailingSingletons(
01575     const char *F, int L, wbsparray<TD> &C, unsigned r
01576 ) const {
01577
01578     C.init();
01579
01580     if (SIZE.len && checkTrailingSingletons(F_L,&r)) {
01581         unsigned m=SIZE.len-r;
01582
01583         #ifndef WB_SKIP_ASSERT
01584         SPIDX_T i=0, n=IDX.dim1, zero=0;
01585         for (; i<n; ++i) if (Wb::anyUnequal(IDX.ref(i,r),m,zero)) {
01586             wblog(F_L,"ERR %s() IDX out of bounds (%d/%dx%d: %d/%d) !?",
01587                 FCT,i+1,IDX.dim1,IDX.dim2,r,SIZE.len);
01588         }
01589         #endif
01590
01591         C.SIZE.init(r,SIZE.data);
01592         IDX.resize(IDX.dim1,r, C.IDX);
01593         C.D=D;
01594
01595         return m;
01596     }
01597
01598     if (int(r)>=0) wblog(FL,
01599         "ERR %s() out of bounds (%d/%d)",FCT,r,SIZE.len);
01600
01601     return 0;
01602 };
01603
01604 template <class TD>
01605 int wbsparray<TD>::SkipTrailingSingletons(
01606     const char *F, int L, unsigned r
01607 ){
01608
01609     if (SIZE.len && checkTrailingSingletons(F_L,&r)) {
01610         unsigned m=SIZE.len-r;
01611
01612         #ifndef WB_SKIP_ASSERT
01613         SPIDX_T i=0, n=IDX.dim1, zero=0;
01614         for (; i<n; ++i) if (Wb::anyUnequal(IDX.ref(i,r),m,zero)) {
01615             wblog(F_L,"ERR %s() IDX out of bounds (%d/%dx%d: %d/%d) !?",
01616                 FCT,i+1,IDX.dim1,IDX.dim2,r,SIZE.len);
01617         }
01618         #endif
01619
01620         if (r) SIZE.len=r; else SIZE.init();
01621         IDX.Resize(IDX.dim1,r);
01622
01623         return m;
01624     }
01625
01626     if (int(r)>=0) wblog(FL,
01627         "ERR %s() out of bounds (%d/%d)",FCT,r,SIZE.len);
01628
01629     return 0;
01630 };
01631
01632 template <class TD> inline
01633 bool wbsparray<TD>::hasSingletons(const wbindex &I) const {
01634
01635     if (!I.len) wblog(FL,"ERR %s() got empty index set",FCT);
01636     for (unsigned i=0; i<I.len; ++i) {
01637         if (I[i]>=SIZE.len || SIZE[I[i]]!=1) return 0;
01638     }

```

```

01639
01640     return 1;
01641 };
01642
01643 template <class TD>
01644 wbsparray<TD>& wbsparray<TD>::skipSingletons(
01645     const char *F, int L, const wbindex &I, wbsparray<TD> &A
01646 ) const {
01647
01648     if (!I.len) { A=*this; return A; }
01649     if (&A==this) {
01650         wbsparray<TD> X; A.save2(X);
01651         return X.skipSingletons(F,L,I,A);
01652     }
01653
01654     if (!hasSingletons(I)) wblog(F_L,"ERR %s() got S=(%) "
01655         "given I=[%s] !?", FCT,SSTR_(this), STR(I));
01656
01657     wbindex I2; I.invert(IDX.dim2,I2,'u');
01658
01659     SIZE.select(I2,A.SIZE); A.isref=0;
01660     IDX.getCols(I2,A.IDX); A.D=D;
01661
01662     return A;
01663 };
01664
01665 template <class TD>
01666 wbsparray<TD>& wbsparray<TD>::AddTrailingSingletons(
01667     const char *F, int L, unsigned r
01668 ){
01669     unsigned rk=rank(F_L); if (r==rk) { return *this; }
01670
01671     if (int(r)<0) wblog(F_L,"ERR %s() invalid rank (%d/%d)",FCT,r,rk);
01672     if (r<rk) wblog(F_L,"ERR %s() cannot reduce rank (%d/%d)",FCT,r,rk);
01673
01674     if (isDiag()) {
01675         if (IDX.data) wblog(F_L,
01676             "ERR %s() got IDX data (%s) !?",FCT,IDX.dim1,IDX.dim2);
01677         SIZE.init(r); SIZE[0]=SIZE[1]=D.len;
01678         for (unsigned i=2; i<r; ++i) SIZE[i]=1;
01679         IDX.init(D.len,r);
01680         for (SPIDX_T *I=IDX.data, i=0; i<D.len; ++i, I+=r) {
01681             I[0]=I[1]=i;
01682         }
01683     }
01684     else {
01685         if (SIZE.len) {
01686             unsigned j=SIZE.len; SIZE.Resize(r);
01687             for (; j<r; ++j) SIZE[j]=1;
01688         }
01689
01690         if (IDX.dim1) {
01691             if (!SIZE.len) wblog(F_L,
01692                 "ERR %s() got empty size (%d/%d)",FCT,SIZE.len,r);
01693             IDX.Resize(IDX.dim1,r);
01694         }
01695     }
01696
01697     return *this;
01698 };
01699
01700 template <class TD>
01701 char wbsparray<TD>::checkSize2(
01702     const char *F, int L, const wbsparray<TD> &B
01703 ) const {
01704
01705     char sab=sameSize(B);
01706     this->checkSize(F_L,"A:"); B.checkSize(F_L,"B:");
01707
01708     if (sab==1) {
01709         if (IDX.dim2!=B.IDX.dim2){ wblog(F_L,
01710             "ERR sparse::%s() IDX size mismatch (%dx%d <> %dx%d)",
01711             FCT,IDX.dim1,IDX.dim2,B.IDX.dim1,B.IDX.dim2);
01712             return -1;
01713         }
01714     }
01715     else if (sab<=0) {
01716         wblog(F_L,"ERR %s() mismatch %s <> %s (%d)",
01717             FCT,SSTR_(this),SSTR(B),sab);
01718     }
01719
01720     return sab;
01721 };
01722
01723 template <class TD> inline
01724 char wbsparray<TD>::checkSize(
01725     const char *F, int L, const char *fct, const char *istr

```

```

01726 ) const {
01727
01728     if (SIZE.len) {
01729         if (IDX.data) {
01730             if (IDX.dim1!=D.len || IDX.dim2!=SIZE.len) {
01731                 if (F) sperror_this(F_L_F,istr); else return 2;
01732             }
01733         }
01734         else if (D.len) {
01735             if (SIZE.len==2 && SIZE[0]==D.len && SIZE[1]==D.len) {
01736                 wblog(FL,"WRN %s() got SIZE for diag-format (%s; d=%d)",
01737                     FCT,SSTR_(this),D.len);
01738             }
01739             else { if (F) sperror_this(F_L_F,istr); else return 3; }
01740         }
01741     }
01742     else {
01743         if (!IDX.isEmpty()) {
01744             if (F) sperror_this(F_L_F,istr); else return 1;
01745         }
01746     }
01747     return 0;
01748 };
01749
01750 template <class TD> inline
01751 void wbspararray<TD>::check_IDX_range(const char *F, int L) const {
01752
01753     if (!SIZE.len) {
01754         if (IDX.dim1 || IDX.dim2) sperror_this(F_L_F);
01755         return;
01756     }
01757
01758     if (IDX.dim1!=D.len ||
01759         (IDX.dim2!=SIZE.len && (IDX.dim1 || IDX.dim2))
01760     ) sperror_this(F_L_F);
01761
01762     if (IDX.dim1) {
01763         SPIDX_T i=0; unsigned j=0, m=IDX.dim2;
01764         const SPIDX_T *s=SIZE.data, *I=IDX.data;
01765
01766         for (; i<IDX.dim1; ++i, I+=m) {
01767             for (j=0; j<m; ++j) {
01768                 if (I[j]>=s[j]) wblog(FL,
01769                     "ERR %s() index out of bounds (%d: %d <> %s)",
01770                     FCT,j+1,I[j]+1, SSTR_(this)
01771                 );
01772             }
01773         }
01774     }
01775 };
01776
01777 template <class TD>
01778 wbspararray<TD>& wbspararray<TD>::make2D(const wbindex &ic, wbspararray<TD> &B,
01779     char pos
01780 ) const {
01781
01782     if (B.D.len) { B.init(); }; B.D=D;
01783     IDX.toIndex2D(SIZE,ic,B.IDX,pos);
01784
01785     SPIDX_T s2, s1=SIZE.prod(ic.data,ic.len,s2);
01786
01787     if (pos==1)
01788         { SPIDX_T s[2]={s1,s2}; B.SIZE.init(2,s); }
01789     else { SPIDX_T s[2]={s2,s1}; B.SIZE.init(2,s); }
01790
01791     B.Compress(); return B;
01792 };
01793
01794 template <class TD>
01795 sparseIndex2D<SPIDX_T>& wbspararray<TD>::toIndex2D(
01796     const wbindex &ic, sparseIndex2D<SPIDX_T> &a,
01797     char pos,
01798     char use_rmaj
01799 ) const {
01800
01801     unsigned i,j;
01802     if (pos==1) { i=0; j=1; } else { i=1; j=0; }
01803
01804     a.S.init(2);
01805     a.S[i]=SIZE.prod(ic.data,ic.len,a.S[j]);
01806
01807 #ifndef WB_SKIP_ASSERT
01808     { unsigned k; double s1=1, s2=1; wvector<char> mark(SIZE.len);
01809         for (k=0; k<ic.len; ++k) { ++mark[ic[k]]; }
01810         for (k=0; k<SIZE.len; ++k) { (mark[k] ? s1 : s2)*=SIZE[k]; }
01811         if (double(a.S[i])!=s1 || double(a.S[j])!=s2) wblog(FL,
01812             "ERR %s() type range out of bounds !?\n{gx%g <> %ldx%ld}",

```



```

01813         FCT,s1,s2,a.S[0],a.S[1]
01814     );
01815 }
01816 #endif
01817
01818     IDX.toIndex2D(SIZE,ic,a.IJ,pos);
01819
01820     return a.Setup(use_rmaj);
01821 };
01822
01823 template <class TD>
01824 mxArray* wbsparray<TD>::toMxSp() const {
01825
01826     if (!Wb::isBaseType(typeid(TD))) {
01827         double x;
01828         for (SPIDX_T i=0; i<D.len; ++i) { x=double(D[i]);
01829             if (fabs(double(TD(x)-D[i])/x)>1E-20) {
01830                 MXPt(FL,"ans").add(*this,"data").add(i+1,"i").add(x,"x");
01831                 wblog(FL,"ERR %s() got data type `%s' (d[%d]=%g @ %.3g)",
01832                     FCT,TSTR(TD), i+1,x,double(TD(x)-D[i])
01833                 );
01834             }
01835         }
01836     }
01837
01838     if (isScalar()) {
01839         if (D.len) {
01840             if (D.len>1) sperror_this(FLF);
01841             return numtoMx(D[0]);
01842         }
01843         return numtoMx(0);
01844     }
01845
01846     if (isDiag(FL)) {
01847         wbMatrix<SPIDX_T> IJ; SPIDX_T d1=1,d2=1;
01848         get2DIndex(IJ,d1,d2);
01849
01850         return Wb::mxCreateSparse(FL,d1,d2,IJ, D);
01851     }
01852
01853     if (!SIZE.len) {
01854         if (IDX.isEmpty() && D.isEmpty())
01855             return Wb::mxCreateSparse(FL,0,0,IDX,D);
01856         else wblog(FL,
01857             "ERR %s() invalid wbsparray (%s;%d)",FCT,SSTR_(this),D.len);
01858     }
01859
01860     if (SIZE.len==2) {
01861         return Wb::mxCreateSparse(FL,SIZE[0],SIZE[1],IDX,D);
01862     }
01863
01864     if (SIZE.len==1) {
01865         wbMatrix<SPIDX_T> IX(IDX.dim1,2);
01866         SPIDX_T i=0, *I=IX.data;
01867
01868         if (IDX.dim2!=1) sperror_this(FLF);
01869         for (; i<IDX.dim1; ++i, I+=2) {
01870             I[0]=IDX.data[i]; I[1]=0;
01871         }
01872
01873         return Wb::mxCreateSparse(FL,SIZE[0],1,IX,D);
01874     }
01875
01876     wblog(FL,"ERR %s() requires rank-2 wbsparray (%s)",
01877         FCT,SSTR_(this)); return 0;
01878 };
01879
01880 template <class TD>
01881 mxArray* wbsparray<TD>::mxCreateStructX(
01882     unsigned m, unsigned n, int ma
01883 ) const {
01884     const char *fields[]={"S","idx","data","info","A"};
01885     return mxCreateStructMatrix(m,n,ma!=-99 ? 5:4,fields);
01886 };
01887
01888 template <class TD>
01889 void wbsparray<TD>::add2MxStructX(mxArray *a, unsigned i, int ma) const {
01890
01891     char isId=isIdentityMatrix(), isc=isScalar();
01892
01893     mxSetFieldByNumber(a,i,0, SIZE.toMx());
01894
01895     if (isc) {
01896         mxSetFieldByNumber(a,i,1, IDX.toMx());
01897         mxSetFieldByNumber(a,i,2, D.toMx('t'));
01898         mxSetFieldByNumber(a,i,3, wbsting("scalar").toMx());
01899     }

```

```

01900     else if (!IDX.isEmpty() && IDX.allEqual(0) && D.allEqual(0)) {
01901         mxSetFieldByNumber(a,i,1,mxCreateSparse(IDX.dim1,IDX.dim2,0,mxREAL));
01902         mxSetFieldByNumber(a,i,2,mxCreateSparse(1,D.len,0,mxREAL));
01903         mxSetFieldByNumber(a,i,3,wbstring("init").toMx());
01904     }
01905     else if (isId) {
01906         mxSetFieldByNumber(a,i,2, numtoMx(D[0]));
01907         mxSetFieldByNumber(a,i,3,wbstring("identity").toMx());
01908     }
01909     else if (isDiag(FL)) {
01910         mxSetFieldByNumber(a,i,2, D.toMx('t'));
01911         mxSetFieldByNumber(a,i,3,wbstring("diag").toMx());
01912     }
01913     else if (IDX.dim2!=1 && isDiagMatrix()) {
01914         mxSetFieldByNumber(a,i,1, IDX.getCols(0,0).toMx());
01915         mxSetFieldByNumber(a,i,2, D.toMx('t'));
01916     }
01917     else {
01918         mxSetFieldByNumber(a,i,1, IDX.toMx());
01919         mxSetFieldByNumber(a,i,2, D.toMx('t'));
01920     };
01921
01922     if (ma!=-99) {
01923         wbMatrix<SPIDX_T> IJ; SPIDX_T d1=1,d2=1;
01924
01925         if (!SIZE.len) { d1=d2=0; }
01926         else ma=get2DIndex(IJ,d1,d2,ma);
01927
01928         mxSetFieldByNumber(a,i,4,
01929             Wb::mxCreateSparse(FL,d1,d2,IJ,D)
01930         );
01931     }
01932 };
01933
01934 template <class TD>
01935 mxArray* wbsparray<TD>::mxCreateSTRUCT(
01936     unsigned m, unsigned n
01937 ) const {
01938     const char *fields[]={"S","idx","data","A"};
01939     return mxCreateStructMatrix(m,n,4,fields);
01940 };
01941
01942 template <class TD>
01943 mxArray* wbsparray<TD>::add2MxSTRUCT(mxArray *a, unsigned i) const {
01944
01945     wbMatrix<SPIDX_T> IJ; SPIDX_T d1=1,d2=1; char tflag='t';
01946     get2DIndex(IJ,d1,d2);
01947
01948     mxSetFieldByNumber(a,i,0, SIZE.toMx());
01949     mxSetFieldByNumber(a,i,1, IDX.toMx());
01950
01951 #ifdef QS_USING_MPFR
01952     if (typeid(TD)==typeid(Wb::quad)) {
01953         tflag=30;
01954     }
01955 #endif
01956
01957     mxSetFieldByNumber(a,i,2, D.toMx(tflag));
01958
01959     mxSetFieldByNumber(a,i,3,Wb::mxCreateSparse(FL,d1,d2,IJ,D));
01960
01961     return a;
01962 };
01963
01964 template <class TD> inline
01965 mxArray* wbsparray<TD>::mxCreateStruct(unsigned m, unsigned n) const {
01966     const char *fields[]={"S","data"};
01967     return mxCreateStructMatrix(m,n,2,fields);
01968 };
01969
01970 #ifdef QS_USING_MPFR
01971
01972 template <> inline mxArray*
01973 wbsparray<Wb::quad>::mxCreateStruct(unsigned m, unsigned n) const {
01974     const char *fields[]={"S","idx","data"};
01975     return mxCreateStructMatrix(m,n,3,fields);
01976 };
01977
01978 #endif
01979
01980 template <class TD>
01981 void wbsparray<TD>::add2MxStruct(mxArray *a, unsigned i, int ma) const {
01982
01983     wbMatrix<SPIDX_T> IJ;
01984     SPIDX_T d1,d2; get2DIndex(IJ,d1,d2,ma);
01985     int e=0;
01986

```

```

01987     mxSetFieldByNumber(a,i,0, SIZE.toMx());
01988     mxSetFieldByNumber(a,i,1, Wb::mxCreateSparse(FL,d1,d2,IJ,D,NULL,&e));
01989
01990     if (e && !isDiag()) {
01991         static unsigned ic=0; char s[8];
01992         snprintf(s,8,"%i02d",ic++);
01993         MXPut(FL,s).add(*this,"A").addP(toMX(),"A0").add(e,"e");
01994         wblog(FL,"ERR %s() got e=%d",FCT,e);
01995     }
01996 };
01997
01998 #ifndef QS_USING_MPFR
01999
02000 template <>
02001 void wbsparray<Wb::quad>::add2MxStruct(
02002     mxArray *a, unsigned i, int ma __attribute__((unused))
02003 ) const {
02004
02005     mxSetFieldByNumber(a,i,0, SIZE.toMx());
02006     mxSetFieldByNumber(a,i,1, IDX.toMxT());
02007     mxSetFieldByNumber(a,i,2, D.toMx());
02008 };
02009
02010 #endif
02011
02012 template <class TD> inline
02013 mxArray* wbsparray<TD>::mxCreateCell(unsigned m, unsigned n) const {
02014     return mxCreateCellMatrix(m,n);
02015 };
02016
02017 template <class TD>
02018 void wbsparray<TD>::add2MxCell(mxAarray *a, unsigned i) const {
02019
02020     if (rank()>2) wblog(FL,
02021         "ERR %s() cell array expects rank <=2 (%s)",FCT,SSTR_(this));
02022     mxSetCell(a,i,toMxSp());
02023
02024 };
02025
02026 template <class TD>
02027 mxArray* wbsparray<TD>::IDtoMx() const {
02028
02029     const char *fields[]={"S","nnz","norm","stat"};
02030     mxArray *S=mxCreateStructMatrix(1,1,4,fields);
02031
02032     const char *field2[]={"istat","idx","d"};
02033     mxArray *as=mxCreateStructMatrix(1,1,3,field2);
02034
02035     if (isDiag()) {
02036         SPIDX_T s[2]={D.len,D.len};
02037         mxSetFieldByNumber(S,0,0, wvector<SPIDX_T>(2,s).toMx());
02038     } else
02039         mxSetFieldByNumber(S,0,0, SIZE.toMx());
02040
02041     mxSetFieldByNumber(S,0,1, numtoMx(nnz()));
02042     mxSetFieldByNumber(S,0,2, numtoMx(double(norm())));
02043
02044     if (D.len>3) {
02045         unsigned i=0; const unsigned m=3;
02046         const TD *d=D.data; TD a;
02047
02048         wbindx I(m); widxt *k=I.data;
02049         wvector<double> x_(m); double *x=x_.data;
02050         x[0]=d[0];
02051         x[1]=ABS(d[0]);
02052         x[2]=d[0];
02053
02054         for (++i; i<D.len; ++i) { a=ABS(d[i]);
02055             if (x[0]>d[i]) { x[0]=d[i]; k[0]=i; }
02056             if (x[1]>a) { x[1]=a; k[1]=i; }
02057             if (x[2]<d[i]) { x[2]=d[i]; k[2]=i; }
02058         }
02059
02060         mxSetFieldByNumber(as,0,0, I.toMx());
02061         mxSetFieldByNumber(as,0,1, IDX.getRecs(I).toMx());
02062         mxSetFieldByNumber(as,0,2, x_.toMx());
02063     }
02064     else if (D.len) {
02065         mxSetFieldByNumber(as,0,1, IDX.toMx());
02066         mxSetFieldByNumber(as,0,2, D.toMx());
02067     }
02068
02069     mxSetFieldByNumber(S,0,3,as);
02070
02071     return S;
02072 };
02073

```

```

02074 template <class TD>
02075 wbspararray<TD>& wbspararray<TD>::initCAT(
02076     const char *F, int L, wbvector< wbspararray<TD>* > &X,
02077     wbvector<SPIDX_T> *I0, char del
02078 ) {
02079     SPIDX_T i, id, l, r=0, i0=-1, i2=0, n=0, nz=0;
02080     wbvector<SPIDX_T> S;
02081
02082     for (i=0; i<X.len; ++i) {
02083         if (!X[i] || X[i]->isEmpty()) continue;
02084         if (X[i]==this) {
02085             wbspararray<TD> A; A.initCAT(F,L,X,I0,del);
02086             return A.save2(*this);
02087         }
02088         X[i]->checkSize(F_LF);
02089         nz+=X[i]->nnz();
02090
02091         if ((++n)==1) { i0=i; r=X[i]->SIZE.len+1;
02092             S.init(r,X[i]->SIZE);
02093         }; i2=i;
02094
02095         if (i!=i0 && !X[i0]->sameSize(*X[i])) wblog(F_L,
02096             "ERR %s() severe size mismatch (%d: %s <> %d: %s)",FCT,
02097             i0+1, SSTR_(X[i0]), i+1, SSTR_(X[i])
02098         );
02099     }
02100     if (long(i0)<0) { init(); return *this; }
02101
02102     char gotI0=(I0 && I0->len);
02103     if (gotI0 && I0->len!=X.len+1) wblog(F_L,
02104         "ERR %s() size mismatch (%d/%d+1)",FCT,I0->len,X.len);
02105
02106     S.last()=(gotI0 ? I0->last() : n);
02107     init(S,nz);
02108
02109     for (id=-1, l=0, i=i0; i<=i2; ++i) {
02110         if (X[i] && !X[i]->isEmpty()) {
02111             if (gotI0) { id=I0->data[i];
02112                 if (id>=S.last()) { wblog(F_L,
02113                     "ERR %s() index out of bounds (%d/%d)",FCT,id,S.last());
02114                 }
02115             }
02116             else ++id;
02117
02118             if (X[i]->nnz()) l+=catRecs(l,*X[i], id);
02119         }
02120     }
02121
02122     if (l!=D.len) wblog(F_L,"ERR %s() %d/%d !?",FCT,l,D.len);
02123
02124     if (del) for (i=0; i<X.len; ++i) {
02125         if (X[i]) { WB_DELETE_1(X[i]); }
02126         else break;
02127     }
02128
02129     Compress(); return *this;
02130 };
02131
02132 template <class TD>
02133 SPIDX_T wbspararray<TD>::catRecs(
02134     SPIDX_T i0, const wbspararray<TD> &a, SPIDX_T id
02135 ) {
02136     SPIDX_T n=a.IDX.dim1;
02137
02138     if (IDX.dim2!=a.IDX.dim2+1) wblog(F_L,
02139         "ERR %s() size mismatch (%d+1/%d)",FCT,IDX.dim2,a.IDX.dim2);
02140     if (i0+n>IDX.dim1) wblog(F_L,
02141         "ERR %s() index out of bounds (%d+%d = %d ?)",FCT,i0,n,IDX.dim1);
02142
02143     if (!n && a.D.len==1) {
02144         if (IDX.dim2!=1 || IDX.dim1!=D.len) wblog(F_L,"ERR %s() "
02145             "%s <> %dx%d (%d)",FCT,SSTR_(this),IDX.dim1,IDX.dim2,i0);
02146         if (a.D[0])
02147             { IDX(i0,0)=id; D[i0]=a.D[0]; return 1; }
02148         else { return 0; }
02149     }
02150
02151     if (a.D.len!=n) wblog(F_L,"ERR %s() %s <> %dx%d (%d)",
02152         FCT, SSTR_(a), a.IDX.dim1, a.IDX.dim2, i0);
02153
02154     Wb::cpyRange(D.data+i0, a.D.data, n);
02155     Wb::cpyStride(
02156         IDX.rec(i0), a.IDX.data, a.IDX.dim2, a.IDX.dim1,
02157         IDX.dim2
02158     );
02159
02160     for (SPIDX_T l=IDX.dim2-1, i=0; i<n; ++i) {

```

```

02161     IDX(i0+i,1)=id;
02162 }
02163
02164 return n;
02165 };
02166
02167 template <class TD>
02168 wbsparray<TD>& wbsparray<TD>::Cat(
02169     const char *F, int L,
02170     const wbsparray<TD> &B, unsigned k
02171 ){
02172     if (this==&B) wblog(FL,"ERR %s() got same object !?",FCT);
02173     if ( isEmpty()) { (*this)=B; return *this; }
02174     if (B.isEmpty()) { return *this; }
02175
02176     if (!SIZE.len || !B.SIZE.len) wblog(F_L,
02177         "ERR %s() got empty size (%s <> %s @ %d) !?",
02178         FCT, SSTR_(this), SSTR(B), k);
02179     checkSize(FLF); B.checkSize(FLF);
02180
02181     if (!k || (k>SIZE.len && k>99)) wblog(F_L,
02182         "ERR %s() k out of bounds (%d/%d)",FCT,k,SIZE.len);
02183     if (k>SIZE.len) AddTrailingSingletons(FL,k);
02184     --k;
02185
02186     SPIDX_T i=0, n=MIN(SIZE.len, B.SIZE.len); int e=0;
02187     SPIDX_T D1=SIZE[k], d1=D.len, d2=B.D.len, d12=d1+d2;
02188
02189     for (; i<n; ++i) { if (i!=k) {
02190         if (SIZE[i]!=B.SIZE[i]) { e=1; break; }
02191     } }
02192     if (e==0) { if (i==k) ++i;
02193         for (; i< SIZE.len; ++i) { if ( SIZE[i]!=1) { e=2; break; } }
02194         for (; i<B.SIZE.len; ++i) { if (B.SIZE[i]!=1) { e=3; break; } }
02195     }
02196     if (e) wblog(F_L,
02197         "ERR %s() size mismatch (%s <> %s @ %d; e=%d)",
02198         FCT, SSTR_(this), SSTR(B), k, e
02199     );
02200
02201     SIZE[k]+=(B.SIZE.len>k ? B.SIZE[k] : 1);
02202     D.Resize(d12, B.D.data);
02203
02204     if (!IDX.dim1) {
02205         if (B.IDX.dim2==SIZE.len)
02206             { IDX=B.IDX; }
02207         else { B.IDX.resize(B.IDX.dim1,SIZE.len, IDX); }
02208     }
02209     else {
02210         if (B.IDX.dim2==IDX.dim2) {
02211             IDX.Resize(d12,IDX.dim2, B.IDX.data);
02212         }
02213         else {
02214             SPIDX_T j, *a, n=MIN(IDX.dim2,B.IDX.dim2);
02215             const SPIDX_T *b=B.IDX.data;
02216
02217             IDX.Resize(d12,IDX.dim2); a=IDX.ref(d1,0);
02218             for (i=0; i<d2; ++i, a+=IDX.dim2, b+=B.IDX.dim2) {
02219                 for (j=0; j<n; ++j) { a[j]=b[j]; }
02220             }
02221             for (SPIDX_T i=d1; i<d12; ++i) IDX(i,k)+=D1;
02222         }
02223     }
02224     Compress(); return *this;
02225 };
02226
02227 template <class TD>
02228 wbsparray<TD>& wbsparray<TD>::initCAT(
02229     const char *F, int L, const wvector< const wbsparray<TD>* > &X,
02230     unsigned k
02231 ){
02232     if (!X.len) { init(); return *this; }
02233
02234     SPIDX_T i,j, i0=-1, i2=0, n=0;
02235     wvector<SPIDX_T> S0, DD(X.len), dd(X.len);
02236
02237     for (i=0; i<X.len; ++i) { if (X[i] && !X[i]->isEmpty()) {
02238         const wvector<SPIDX_T> &S2=X[i]->SIZE;
02239         if (X[i]==this) {
02240             wbsparray<TD> A; A.initCAT(F,L,X,k);
02241             return A.save2(*this);
02242         }
02243
02244         if (S2.len!=X[i]->IDX.dim2) X[i]->sperror_this(F_L);
02245         X[i]->checkSize(F_LF);
02246
02247         if ((++n)==1) { i0=i; S0=S2;

```

```

02248         if (!k || k>S2.len) wblog(FL,"ERR %s() k out of bounds "
02249             " (%s; %d)",FCT,SSTR(S2),k);
02250         --k;
02251     }
02252     else {
02253         if (S0.len!=S2.len) wblog(FL,"ERR %s() rank mismatch "
02254             " (%d: %s <> %d/%d: %s; %d)",FCT,i0+1,SSTR(S0),
02255             i+1,X.len, SSTR(S2), k+1);
02256         for (j=0; j<S0.len; ++j) {
02257             if (j!=k && S0[j]!=S2[j]) wblog(FL,"ERR %s() size mismatch "
02258                 " (%d: %s <> %d/%d: %s; %d/%d)",FCT,i0+1,SSTR(S0),
02259                 i+1,X.len, SSTR(S2), j+1,k+1);
02260         }
02261     }
02262
02263     DD[i]=S2[k];
02264     dd[i]=X[i]->D.len; i2=i;
02265 }}
02266
02267 SPIDX_T dall=dd.sum(), Dall=DD.sum();
02268
02269 if (!n) { init(); return *this; }
02270 S0[k]=Dall; init(S0,dall); if (!dall) return *this;
02271
02272 SPIDX_T *pi=IDX.data,N,s, d1=0, d2, D1=0, m=S0.len;
02273 TD *pd=D.data;
02274
02275 for (i=i0; i<=i2; ++i, d1+=n, D1+=N) { n=dd[i]; N=DD[i];
02276     s=n*m; MEM_COPY<SPIDX_T>(pi,s,X[i]->IDX.data); pi+=s;
02277     s=n ; MEM_COPY<TD>(pd,s,X[i]->D.data); pd+=s;
02278     for (d2=d1+n, j=d1; j<d2; ++j) IDX(j,k)+=D1;
02279 }
02280 if (d1!=IDX.dim1) wblog(FL,"ERR %s() %d/%d",FCT,d1,IDX.dim1);
02281
02282 Compress(); return *this;
02283 };
02284
02285 template <class TD>
02286 wbspararray<TD>& wbspararray<TD>::init(
02287     const char *F, int L, const mxArray *a, unsigned k
02288 ){
02289
02290     init(); if (!a) {
02291         if (k) wblog(FL,"ERR %s() got k=%d for null mxArray !?",FCT,k);
02292         return *this;
02293     }
02294
02295     if (mxIsEmpty(a) && mxIsDouble(a)) {
02296         SPIDX_T i,r=mxGetNumberOfDimensions(a);
02297         if (r) {
02298             const size_t *sp=mxGetDimensions(a);
02299             wvector<SPIDX_T> S(r);
02300
02301             for (i=0; i<r; ++i) S[i]=(SPIDX_T)sp[i];
02302             init(S,0);
02303         }
02304         return *this;
02305     }
02306
02307     if (mxIsStruct(a)) {
02308         const mxArray *as,*ad,*ai;
02309         if (!mxIsWbspararray(F_L,a,k,&as,&ad,&ai)) wblog(FL,
02310             "ERR %s() invalid sparse array",FCT);
02311
02312         SIZE.init(F_L,as);
02313         IDX .init(F_L,ai);
02314         D   .init(F_L,ad);
02315
02316         check_IDX_range(F_L);
02317
02318         if (IDX.isEmpty()) {
02319             if (!SIZE.isEmpty() && !IDX.dim2) {
02320                 if (IDX.dim1) wblog(FL,
02321                     "ERR %s() got size %s !?",FCT,SSTR_(this));
02322                 IDX.init(IDX.dim1,SIZE.len);
02323             }
02324         }
02325         if (!isDiag()) {
02326             if (IDX.dim1!=D.len || (IDX.dim1 && IDX.dim2!=SIZE.len))
02327                 sperror_this(F_L);
02328         }
02329
02330         Compress();
02331         return *this;
02332     }
02333
02334     if (mxIsCell(a)) {

```

```

02335     a=mxGetCell(a,k); if (!a) return *this;
02336 }
02337 else if (k) wblog(FI,"ERR %s() got k=%d !?",FCT,k);
02338
02339 if (mxIsSparse(a)) {
02340
02341     SPIDX_T j,k,l,m,n,d, nnz=mxGetNzmax(a);
02342     mwIndex *Ir,*Jc;
02343
02344     if (!mxIsDouble(a) || mxGetNumberOfDimensions(a)!=2) wblog(FI,
02345         "ERR %s() invalid input (got %s)",FCT,mxTypeSize2Str(a).data);
02346
02347     m=mxGetM(a); n=mxGetN(a); initz(m,n,nnz);
02348     Ir=mxGetIr(a);
02349     Jc=mxGetJc(a);
02350
02351     if (mxIsComplex(a)) {
02352         if (typeid(TD)!=typeid(wbcomplex)) wblog(FI,
02353             "ERR %s() got complex sparse array for %s",FCT,TSTR(TD));
02354         wbcomplex *dz=(wbcomplex*)mxGetComplexDoubles(a);
02355         for (l=j=0; j<n; ++j) { d=SPIDX_T(Jc[j+1]-Jc[j]);
02356             for (k=0; k<d; ++k,++l) { D[l]=dz[l];
02357                 IDX(l,0)=SPIDX_T(Ir[l]); IDX(l,1)=j;
02358             }
02359         }
02360     }
02361     else {
02362         double *dr=mxGetDoubles(a);
02363         for (l=j=0; j<n; ++j) { d=SPIDX_T(Jc[j+1]-Jc[j]);
02364             for (k=0; k<d; ++k,++l) { D[l]=dr[l];
02365                 IDX(l,0)=SPIDX_T(Ir[l]); IDX(l,1)=j;
02366             }
02367         }
02368     }
02369     if (l && l!=nnz) wblog(FI,
02370         "ERR severe sparse inconsistency (mex: %d,%d)",l,nnz);
02371 }
02372 else {
02373
02374     if (!mxIsDouble(a)) wblog(FI,
02375         "ERR %s() got '%s' data",FCT,mxGetClassName(a));
02376     if (typeid(TD)==typeid(wbcomplex)) wblog(FI,
02377         "WRN %s() ignores complex input data",FCT);
02378
02379     SPIDX_T r=mxGetNumberOfDimensions(a);
02380     const size_t *sp=mxGetDimensions(a);
02381     const double *dr=mxGetDoubles(a);
02382
02383     wbvector<SPIDX_T> S(r), I(r);
02384     SPIDX_T i,k,N, nz=0, iz=0, l=r-1, *s=S.data, *ip=I.data;
02385
02386     for (i=0; i<r; ++i) S[i]=(SPIDX_T)sp[i];
02387
02388     N=S.prod(0);
02389     for (i=0; i<N; ++i) { if (dr[i]!=0) ++nz; }
02390     init(S,nz);
02391
02392     for (i=0; i<N; ++i) {
02393         if (dr[i]!=0) {
02394             if (iz>=nz) wblog(FI,
02395                 "ERR %s() index out of bounds (%d/%d) !?",FCT,iz+1,nz);
02396             D[iz]=dr[i]; IDX.recSetP(iz,ip); ++iz;
02397         }
02398
02399         k=0; ++ip[0];
02400         while (ip[k]>=s[k] && k<l) { ip[k]=0; ++ip[++k]; }
02401     }
02402 }
02403
02404 return *this;
02405 };
02406
02407 template <class TD>
02408 wbsparray<TD>& wbsparray<TD>::init(
02409     const char *F, int L, const wbararray<TD> &A, TD eps
02410 ){
02411     unsigned k, r=A.rank(), l=r-1;
02412     wbvector<SPIDX_T> I(r);
02413     SPIDX_T i,N, nz=0, iz=0, *ip=I.data; const size_t *s=A.SIZE.data;
02414
02415     N=A.numel();
02416     for (i=0; i<N; ++i) { if (ABS(A.data[i])>eps) ++nz; }
02417     init(A.SIZE,nz);
02418
02419     for (i=0; i<N; ++i) {
02420         if (ABS(A.data[i])>eps) {
02421             if (iz>=nz) wblog(FI,L,

```

```

02422         "ERR %s() index out of bounds (%d/%d) !?",FCT,iz+1,nz);
02423         D[iz]=A.data[i]; IDX.recSetP(iz,ip); ++iz;
02424     }
02425
02426     k=0; ++ip[0];
02427     while (ip[k]>=s[k] && k<l) { ip[k]=0; ++ip[++k]; }
02428 }
02429
02430 return *this;
02431 };
02432
02433 template <class TD>
02434 template <class TA>
02435 wbarray<TA>& wbsparray<TD>::toFull(wbarray<TA> &A) const {
02436
02437     if (isDiag()) { A.initDiag(D); }
02438     else {
02439         unsigned k, r=rank(FL), l=r-1;
02440         SPIDX_T i,j; const SPIDX_T *s=SIZE.data, *ip=IDX.data;
02441
02442         A.init(SIZE);
02443         if (IDX.dim1 && A.data!=NULL) {
02444             for (i=0; i<IDX.dim1; ++i, ip+=IDX.dim2) {
02445                 for (j=ip[l], k=l-1; k<r; --k) { j=j*s[k]+ip[k]; }
02446                 A.data[j]=TA(D.data[i]);
02447             }
02448         }
02449     }
02450     return A;
02451 };
02452
02453 template <class TD>
02454 template <class T2>
02455 wbvector<T2>& wbsparray<TD>::toFull(wbvector<T2> &A) const {
02456
02457     if (SIZE.len) {
02458         unsigned m=IDX.dim2, r=rank(FL);
02459         SPIDX_T i=0; const SPIDX_T *s=SIZE.data, *ip=0;
02460
02461         for (; i<r; ++i) { if (s[i]!=1) {
02462             if (!ip) { ip=IDX.data+i; }
02463             else wblog(FL,
02464                 "ERR %s() invalid usage (got %s tensor)",
02465                 FCT,SSTR_(this)
02466             );
02467         } }
02468         if (!ip) ip=IDX.data;
02469
02470         A.init(numel());
02471         for (i=0; i<D.len; ++i, ip+=m) {
02472             if ((*ip)>=A.len) wblog(FL,
02473                 "ERR %s() invalid sparse setting (%d/%d)",FCT,*ip,A.len);
02474             if (A[*ip]) wblog(FL,"ERR %s() "
02475                 "invalid sparse setting (%d: %g)",FCT,*ip,double(A[*ip]));
02476             A[*ip]=D.data[i];
02477         }
02478     }
02479     else { A.init(1);
02480         if (D.len) {
02481             if (D.len>1) wblog(FL,
02482                 "ERR %s() invalid scalar (len=%d)",FCT,D.len);
02483             A.data[0]=D.data[0];
02484         }
02485         else { A.data[0]=0; }
02486     }
02487     return A;
02488 };
02489
02490 template <class TD> inline
02491 wbsparray<TD>& wbsparray<TD>::reshape(
02492     const wbvector<SPIDX_T> &S, wbsparray<TD> &A
02493 ) const {
02494
02495     if (S==SIZE) { A=*this; return A; }
02496
02497     if (!S.len && (SIZE.allEqual(1) && D.len<=1)) {
02498         A.init(); A.D=D;
02499         return A;
02500     }
02501
02502     if (isDiag()) {
02503         wbsparray<TD> X; diag2reg(FL,X);
02504         return X.reshape(S,A);
02505     }
02506
02507     if (sameSizeUp2Singletons(S)) {
02508         unsigned j; SPIDX_T i=0, *I; const SPIDX_T *I0=IDX.data; int *ms;

```



```

02509
02510     wvector<int> Ms; matchWithSingletons(FL,S,Ms);
02511     A.init(S); A.D=D; A.IDX.init(IDX.dim1,S.len);
02512     ms=Ms.data; I=A.IDX.data;
02513
02514     for (; i<IDX.dim1; ++i, I+=S.len, IO+=IDX.dim2) {
02515         for (j=0; j<S.len; ++j) {
02516             if (ms[j]>=0) { I[j]=IO[ms[j]]; }
02517         }
02518     }
02519     return A;
02520 }
02521
02522 if (numel()!=S.prod(0)) wblog(FL,
02523     "ERR %s() got size mismatch (%s => %s)",
02524     FCT,SSTR_(this), SSTR(S)
02525 );
02526
02527 A.init(S); A.D=D; A.IDX.init(IDX.dim1,S.len);
02528
02529 if (!IDX.isEmpty()) {
02530     SPIDX_T i,k, *I=A.IDX.data; unsigned r0=SIZE.len, r=S.len;
02531     const SPIDX_T *IO=IDX.data, *S0=SIZE.data, *S=A.SIZE.data;
02532
02533     for (i=0; i<IDX.dim1; ++i, IO+=r0, I+=r) {
02534         k=Wb::sub2ind(S0,IO,r0);
02535         Wb::ind2sub(k,S,I,r);
02536     }
02537 }
02538
02539 return A;
02540 };
02541
02542 template <class TD>
02543 template <class DB> inline
02544 wbsparray<TD>& wbsparray<TD>::init(
02545     const char *F, int L,
02546     SPIDX_T d1, SPIDX_T d2, const wbsparray<DB> &B
02547 ){
02548     SPIDX_T s[] = {d1,d2};
02549     wvector<SPIDX_T> S; S.init(2,s);
02550
02551     if (B.numel()!=S.prod(0)) wblog(F_L,"ERR %s() size mismatch "
02552         "(%s <> %s)",FCT,SSTR(S),SSTR(B));
02553     (*this)=B; return Reshape(S);
02554 };
02555
02556 template <class TD>
02557 template <class DB> inline
02558 wbsparray<TD>& wbsparray<TD>::init(
02559     const char *F, int L,
02560     SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const wbsparray<DB> &B
02561 ){
02562     SPIDX_T s[] = {d1,d2,d3};
02563     wvector<SPIDX_T> S; S.init(3,s);
02564
02565     if (B.numel()!=S.prod(0)) wblog(F_L,"ERR %s() size mismatch "
02566         "(%s <> %s)",FCT,SSTR(S),SSTR(B));
02567     (*this)=B; return Reshape(S);
02568 };
02569
02570 template <class TD>
02571 template <class DB> inline
02572 wbsparray<TD>& wbsparray<TD>::init(
02573     const char *F, int L,
02574     SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const wbsparray<DB> &B
02575 ){
02576     SPIDX_T s[] = {d1,d2,d3,d4};
02577     wvector<SPIDX_T> S; S.init(4,s);
02578
02579     if (B.numel()!=S.prod(0)) wblog(F_L,"ERR %s() size mismatch "
02580         "(%s <> %s)",FCT,SSTR(S),SSTR(B));
02581     (*this)=B; return Reshape(S);
02582 };
02583
02584 template <class TD>
02585 wbsparray<TD>& wbsparray<TD>::splitLast(SPIDX_T k, wbsparray &a) const {
02586
02587     if (&a==this) {
02588         wblog(FL,"ERR %s() output same as input space",FCT);
02589     }
02590
02591     unsigned r=SIZE.len, l=r-1;
02592     SPIDX_T *I=IDX.data+l, i=0, n=0;
02593
02594     if (r<2) {
02595         wblog(FL,"ERR %s() got S=%s (k=%d)",FCT,SSTR_(this),k);

```

```

02596     }
02597
02598     if (k>=SIZE[l]) wblog(FL,
02599         "ERR %s() index out of bounds (%s /%d)",FCT,SSTR_(this),k);
02600     if (IDX.dim2!=r) wblog(FL,"ERR %s() %d/%d",FCT,IDX.dim2,r);
02601
02602     for (; i<IDX.dim1; ++i, I+=r) { if (*I==k) ++n; }
02603
02604     a.SIZE.init(l,SIZE.data);
02605     a.init(a.SIZE,n);
02606
02607     if (n) {
02608     for (I=IDX.data+l, n=i=0; i<IDX.dim1; ++i, I+=r) {
02609         if (*I==k) {
02610             a.IDX.recSetP(n,IDX.rec(i));
02611             a.D[n++]=D[i];
02612         }
02613     }}
02614
02615     return a;
02616 };
02617
02618 template <class TD>
02619 wbspararray<TD>& wbspararray<TD>::select0(
02620     const WBINDEX &P, unsigned dim,
02621     wbspararray<TD> &X
02622 ) const {
02623
02624     if (&X==this) {
02625         wbspararray<TD> Q; X.save2(Q);
02626         return Q.select0(P,dim,X);
02627     }
02628     if (dim>=SIZE.len) wblog(FL,
02629         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02630
02631     SPIDX_T i,l, d0=SIZE[dim], *j;
02632     sSPIDX_T *I;
02633
02634     wbvector<sSPIDX_T> I_(d0); I_.set(-1); I=I_.data;
02635
02636     for (i=0; i<P.len; ++i) {
02637         if (P[i]>=d0) wblog(FL,
02638             "ERR %s() index out of bounds (%d/%d)",FCT,P[i],d0);
02639         if (I[P[i]]>=0) wblog(FL,
02640             "ERR %s() input index not unique (%d/%d)",FCT,P[i],d0);
02641         I[P[i]]=i;
02642     }
02643
02644     for (j=IDX.data+dim, l=i=0; i<IDX.dim1; ++i, j+=IDX.dim2) {
02645         if (I[*j]>=0) ++l;
02646     }; X.init(SIZE,l); X.SIZE[dim]=P.len;
02647
02648     for (j=IDX.data+dim, l=i=0; i<IDX.dim1; ++i, j+=IDX.dim2) {
02649         if (I[*j]>=0) {
02650             X.IDX.recSetP(l,IDX.rec(i)); X.IDX(l,dim)=I[*j];
02651             X.D[l++]=D[i];
02652         }
02653     }
02654
02655     return X;
02656 };
02657
02658 template <class TD>
02659 wbspararray<TD>& wbspararray<TD>::getBlock(const char *F, int L,
02660     SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2,
02661     wbspararray<TD> &B
02662 ) const {
02663
02664     SPIDX_T k1,k2,dim1,dim2, i=0, l=0, n=IDX.dim1;
02665
02666     if (!isRank(2)) wblog(FL,
02667         "ERR %s() only applies to rank-2 tensors (%d)",FCT,SIZE.len);
02668
02669     if (!SIZE.len) { dim1=dim2=D.len; }
02670     else {
02671         if (SIZE.len!=2 || IDX.dim2!=2) wblog(FL,"ERR %s() "
02672             "got invalid rank-2 tensor (%ld,%ld)!",FCT,SIZE.len,IDX.dim2);
02673         dim1=SIZE[0]; dim2=SIZE[1];
02674     }
02675
02676     if (sSPIDX_T(i2)<0) i2+=dim1;
02677     if (sSPIDX_T(j2)<0) j2+=dim2;
02678
02679     if (sSPIDX_T(i1)<0 || i2>=dim1) wblog(FL,"ERR %s() \n"
02680         "index out of bounds (l: %ld .. %ld / %ld)", FCT,i1,i2,dim1);
02681     if (sSPIDX_T(j1)<0 || j2>=dim2) wblog(FL,"ERR %s() \n"
02682         "index out of bounds (l: %ld .. %ld / %ld)", FCT,j1,j2,dim2);

```

```

02683
02684     k1=(i2>=i1 ? i2-i1+1 : 0);
02685     k2=(j2>=j1 ? j2-j1+1 : 0);
02686
02687     B.init(k1,k2); if (!k1 || !k2) return B;
02688
02689     if (!SIZE.len) {
02690         k1=MAX(i1,j1); k2=MIN(i2,j2); l=(k2>k1 ? k2-k1 : 0);
02691         if (l) { B.init_nnz(l); SPIDX_T *idx=B.IDX.data;
02692             for (; i<l; ++i, ++k1, idx+=2) {
02693                 idx[0]=k1-i1;
02694                 idx[1]=k1-j1;
02695                 B.D.data[i]=D.data[k1];
02696             }
02697         }
02698         return B;
02699     }
02700
02701     const SPIDX_T *i0=IDX.data; SPIDX_T *idx;
02702
02703     for (; i<n; ++i, i0+=2) {
02704         if (i0[1]>=j1) {
02705             if (i0[1]<=j2) { if (i0[0]>=i1 && i0[0]<=i2) ++l; }
02706             else break;
02707         }
02708     }
02709
02710     if (l) { B.init_nnz(l); idx=B.IDX.data; i0=IDX.data; l=0;
02711         for (i=0; i<n; ++i, i0+=2) {
02712             if (i0[1]>=j1) {
02713                 if (i0[1]<=j2) { if (i0[0]>=i1 && i0[0]<=i2) {
02714                     idx[0]=i0[0]-i1;
02715                     idx[1]=i0[1]-j1;
02716                     B.D.data[l]=D.data[i]; idx+=2; ++l;
02717                 }} else break;
02718             }
02719         }
02720     }
02721
02722     return B;
02723 };
02724
02725 template <class TD>
02726 wbsparray<TD>& wbsparray<TD>::splitBlock(const char *F, int L,
02727     sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2,
02728     wbsparray<TD> &B
02729 ){
02730
02731     SPIDX_T k1,k2,dim1,dim2, j,i_, i=0, l=0, n=IDX.dim1;
02732
02733     if (!isRank(2) || SIZE.len!=2) wblog(FL,
02734         "ERR %s() only applies to rank-2 tensors (%d)",FCT,SIZE.len);
02735
02736     B.init(SIZE); if (!D.len) return B;
02737     dim1=SIZE[0]; dim2=SIZE[1];
02738
02739     if (i2<0) i2+=dim1;
02740     if (j2<0) j2+=dim2;
02741
02742     if (i1<0 || i2>=dim1) wblog(FL,"ERR %s() \n"
02743         "index out of bounds (l: %ld .. %ld / %ld)", FCT,i1,i2,dim1);
02744     if (j1<0 || j2>=dim2) wblog(FL,"ERR %s() \n"
02745         "index out of bounds (l: %ld .. %ld / %ld)", FCT,j1,j2,dim2);
02746
02747     k1=(i2>=i1 ? i2-i1+1 : 0);
02748     k2=(j2>=j1 ? j2-j1+1 : 0); if (!k1 || !k2) return B;
02749
02750     SPIDX_T *j0, *i0=IDX.data, *idx;
02751
02752     for (; i<n; ++i, i0+=2) { if (i0[1]>=j1) break; }; i_=i;
02753     for (; i<n; ++i, i0+=2) {
02754         if (i0[1]<=j2) { if (i0[0]>=i1 && i0[0]<=i2) ++l; }
02755         else break;
02756     }
02757
02758     if (l) {
02759         B.init_nnz(l); idx=B.IDX.data; l=0; j=i_; i0=IDX.data+2*i;
02760         for (; i<n; ++i, i0+=2) {
02761             if (i0[1]<=j2 && i0[0]>=i1 && i0[0]<=i2) {
02762                 idx[0]=i0[0];
02763                 idx[1]=i0[1]; idx+=2;
02764                 B.D.data[l]=D.data[i]; ++l;
02765             }
02766             else {
02767                 if (j<i) { j0=IDX.data+2*j;
02768                     j0[0]=i0[0];
02769                     j0[1]=i0[1]; D[j]=D[i];

```

```

02770         }; ++j;
02771     }
02772 }
02773 }
02774
02775 REDSIZE_NNZ(j);
02776 return B;
02777 };
02778
02779 template <class TD>
02780 TD wbsparray<TD>::getHouseholderVec(const char *F, int L,
02781     SPIDX_T k,
02782     SPIDX_T j,
02783     wbsparray<TD> &u,
02784     TD eps2
02785 ) const {
02786     TD x2=0, b2=0;
02787
02788     if (!isRank(2) || SIZE.len!=2 || IDX.dim2!=2) wblog(F_L,
02789         "ERR %s() only applies to rank-2 tensors (%s; %d,%d)",
02790         FCT,SSTR_(this),SIZE.len,IDX.dim2);
02791
02792     SPIDX_T dim1=SIZE[0], dim2=SIZE[1], i1,i2, i=0, l=0, n=IDX.dim1;
02793     const SPIDX_T *i0=IDX.data;
02794     char gotk=0;
02795
02796     if (k>=dim2) wblog(F_L,"ERR %s() "
02797         "col-index out of bounds (%d/%d)",FCT,k,dim2);
02798     u.init(dim1); if (!dim1) return x2;
02800
02801     for (; i<n; ++i, i0+=2) { if (i0[1]>=j) break; }
02802     for (; i<n; ++i, i0+=2) { if (i0[0]>=k) break; }
02803     i1=i; if (i<n && i0[0]==k && i0[1]==j) gotk=1;
02804     for (; i<n; ++i, i0+=2) {
02805         if (i0[1]==j) { b2+=NORM2(D[i]); ++l; }
02806         else break;
02807     }
02808     i2=i;
02809
02810     if (b2<=eps2) {
02811         if (b2==0) {
02812             wblog(F_L,"WRN %s() got 0 vector (%s) !?",FCT,SSTR_(this));
02813             return x2;
02814         }
02815         wblog(F_L,"WRN %s() got b2=%.3g !?",FCT,double(b2));
02816     }
02817
02818     u.init_nnz(l+(gotk?0:1));
02819
02820     SPIDX_T *idx=u.IDX.data; i0=IDX.rec(i1);
02821
02822     if (gotk) { l=0; } else { u.D[0]=0; idx[0]=k; l=1; }
02823
02824     for (i=i1; i<i2; ++i, i0+=2) {
02825         u.D[l]=D.data[i]; idx[l]=i0[0]; ++l;
02826     }
02827
02828     x2=NORM2(u.D[0]);
02829     if (u.D[0]<0) { u.D[0]=-SQRT(b2); } else { u.D[0]+=SQRT(b2); }
02830     x2=b2-x2+NORM2(u.D[0]);
02831
02832     u*=(TD(1)/SQRT(x2));
02833
02834     return x2;
02835 };
02836
02837 template <class TD>
02838 wbsparray<TD>& wbsparray<TD>::IDX_Shift(sSPIDX_T i1, sSPIDX_T i2) {
02839
02840     if (!isRank(2) || SIZE.len!=2 || IDX.dim2!=2) wblog(F_L,
02841         "ERR %s() only applies to rank-2 tensors (%s; %d,%d)",
02842         FCT,SSTR_(this),SIZE.len,IDX.dim2
02843     );
02844
02845     SPIDX_T i=0, n=IDX.dim1, *I=IDX.data;
02846     SPIDX_T dim1=SIZE[0]+i1, dim2=SIZE[1]+i2; SIZE[0]=dim1; SIZE[1]=dim2;
02847
02848     if (i1>=0 && i2>=0) {
02849         for (; i<n; ++i, I+=2) { I[0]+=i1; I[1]+=i2; }
02850     }
02851     else {
02852         if (sSPIDX_T(dim1)<0) wblog(F_L,
02853             "ERR %s() index out of bounds (%ld/%ld)",FCT,i1,SIZE[0]);
02854         if (sSPIDX_T(dim2)<0) wblog(F_L,
02855             "ERR %s() index out of bounds (%ld/%ld)",FCT,i2,SIZE[1]);
02856     }

```

```

02857         for (; i<n; ++i, I+=2) { I[0]+=i1; I[1]+=i2;
02858             if (I[0]>=dim1 || I[1]>=dim2) wblog(FL,
02859                 "ERR %s() index out of bounds (%ld,%ld; %s)",
02860                 FCT,I[0],I[1],SSTR_(this)
02861             );
02862         }
02863     }
02864
02865     return *this;
02866 };
02867
02868 template <class TD>
02869 SPIDX_T wbsparray<TD>::LContractVec_omp(
02870     const wbsparray<TD> &b, wbvector<TD> &x, char tnorm) const {
02871
02872     SPIDX_T i, k0=0, d2, l=0;
02873
02874     if (!isMatrix()) wblog(FL,
02875         "ERR %s() only applies to rank-2 tensors (%d)",FCT,SIZE.len);
02876
02877     if (!isEmpty() && !b.isEmpty()) {
02878         if (b.IDX.dim2!=1 || b.SIZE.len!=1) wblog(FL,
02879             "ERR %s() expecting sparse VECTOR\n(got b: IDX=%s; SIZE=[%s])",
02880             FCT,SSTR(b.IDX),STR(b.SIZE));
02881         if (dim0(0)!=b.SIZE[0]) wblog(FL,
02882             "ERR %s() size mismatch (%d/%d)",FCT,SSTR_(this),SSTR(b));
02883     }
02884
02885     if (!D.len || !b.D.len) { x.init(); return l; }
02886
02887     if (SIZE.len) {
02888         indexSparseRef<TD> ISP(FL,*this);
02889         d2=ISP.numCols(); x.init(d2);
02890
02891         #ifndef WB_SKIP_ASSERT
02892         SPIDX_T dl=ISP.numRows();
02893         if (dl!=SIZE[0] || d2!=SIZE[1] || !dl || !d2) wblog(FL,
02894             "ERR %s() got size inconsistency (%dx%d / %s)",
02895             FCT,dl,d2,SSTR_(this));
02896         #endif
02897
02898         int np=1;
02899         if (sp_num_threads>1) {
02900             np=IDX.dim1/1024;
02901             if (np>sp_num_threads) np=sp_num_threads;
02902             if (np>int(d2)) np=d2;
02903         }
02904
02905         for (; k0<d2; ++k0) { if (ISP.nnz(k0)) break; }
02906
02907         #pragma omp parallel for num_threads(np) reduction(+:l)
02908         for (SPIDX_T k=k0; k<d2; ++k) {
02909             x[k]=ISP.LContractVec(b,k,tnorm);
02910             if (x[k]) l+=1;
02911         }
02912
02913         if (l!=x.nnz()) {
02914             MXPut(FL,"g").add(x,"x").add(l,"l"); SPIDX_T n=0;
02915             for (i=0; i<x.len; ++i) {
02916                 if (x[i]) printf("%6ld: %3ld : %+.4g\n",i,++n,double(x[i])); else
02917                     if (x[i]!=0) printf("%6ld: %3ld : %+.4g (!!)\n",i,++n,double(x[i]));
02918             }
02919             wblog(FL,"ERR %s() %ld/%ld !?",FCT,l,x.nnz());
02920         }
02921     }
02922     else {
02923         const SPIDX_T *idx=b.IDX.data;
02924         d2=D.len;
02925         x.init(d2); k0=b.IDX[0]; l=b.IDX.dim1;
02926         for (i=0; i<l; ++i) {
02927             if (idx[i]>=d2 || (i && idx[i-1]>=idx[i])) wblog(FL,
02928                 "ERR %s() b.IDX out of bounds (%ld/%ld) !?",FCT,idx[i],d2);
02929             x[idx[i]]=b.D[i]*D[idx[i]];
02930         }
02931     }
02932
02933     return l;
02934 };
02935
02936 template <class TD>
02937 wbsparray<TD>& wbsparray<TD>::LProject1(
02938     const wbsparray<TD> &B, wbsparray<TD> &X, TD xfac, char tnorm) const {
02939
02940     if (!D.len) { X=(*this); return X; }
02941     if (!B.D.len) { X.init(SIZE); return X; }
02942
02943     wbvector<TD> x;

```

```

02944     SPIDX_T i, d1,d2;
02945     SPIDX_T l=LContractVec_omp(B,x,tnorm);
02946
02947     if (xfac!=1) {
02948         if (xfac) x*=xfac;
02949         else wblog(FL,"ERR %s() got xfac=0 !?",FCT);
02950     }
02951
02952     d1=B.D.len; d2=x.len;
02953     if (SIZE.len) { X.init(SIZE, 1*d1); }
02954     else {
02955         SPIDX_T s[2]={d2,d2};
02956         X.init(wbvector<SPIDX_T>(2,s), 1*d1);
02957     }
02958
02959     const SPIDX_T *I=B.IDX.data;
02960     SPIDX_T *idx=X.IDX.data, r=X.IDX.dim2; l=0;
02961     const TD *b=B.D.data; TD *xb=X.D.data;
02962
02963     for (SPIDX_T k=0; k<d2; ++k) { if (x[k]) {
02964         for (i=0; i<d1; ++i, ++l, idx+=r) {
02965             xb[l]=x[k]*b[i]; idx[0]=I[i]; idx[1]=k;
02966         }
02967     }}
02968
02969     if (l!=X.IDX.dim1) wblog(FL,"ERR %s() %ld/%ld !?",FCT,l,X.IDX.dim1);
02970     return X;
02971 };
02972
02973 template <class TD> inline
02974 wbspararray<TD>& wbspararray<TD>::Permute(const wbperm &P, char iflag){
02975
02976     if (!SIZE.len && !IDX.dim1 && D.len<=1) return *this;
02977     if (!P.isEmpty()) {
02978         SIZE.Permute(P,iflag); IDX.colPermute(P,iflag);
02979         Sort();
02980     }
02981     return *this;
02982 };
02983
02984 template <class TD> inline
02985 wbspararray<TD>& wbspararray<TD>::permute(
02986     const wbperm &P, wbspararray<TD>&B, char iflag
02987 ) const {
02988
02989     if (this==&B) return B.Permute(P,iflag);
02990
02991     if (P.isEmpty() || (!SIZE.len && !IDX.dim1 && D.len<=1)) { B=(*this); }
02992     else {
02993         SIZE.permute(B.SIZE,P,iflag); B.D=D; B.isref=0;
02994         IDX.colPermute(P,B.IDX,iflag);
02995         B.Sort();
02996     }
02997     return B;
02998 };
02999
03000 template <class TD> inline
03001 wbspararray<TD>& wbspararray<TD>::Permute(const char* s, char iflag){
03002     return Permute(Str2Idx(s,1),iflag);
03003 };
03004
03005 template <class TD> inline
03006 wbspararray<TD>& wbspararray<TD>::permute(
03007     const char* s, wbspararray<TD>&Q, char iflag
03008 ) const { return permute(Q,Str2Idx(s,1),iflag); };
03009
03010 template <class TD> inline
03011 wbspararray<TD>& wbspararray<TD>::ColPermute(const wbperm &P0, char iflag){
03012
03013     if (isIdentityPerm(P0)) return *this;
03014
03015     if (SIZE.len!=2 || IDX.dim2!=2) wblog(FL,
03016         "ERR %s() requires rank-2 (%s; %d)",FCT,SSTR_(this),IDX.dim2);
03017     if (P0.len!=SIZE[1]) wblog(FL,
03018         "ERR %s() size mismatch (%d/%d)",FCT,P0.len,SIZE[1]);
03019
03020     SPIDX_T k,i=0, l=IDX.dim2-1; wbperm P;
03021
03022     if (!iflag)
03023         P0.invert(P);
03024     else P.wbvector<wperm_t>::init2ref(P0);
03025
03026     for (; i<IDX.dim1; ++i) { k=IDX(i,1);
03027         if (k>=P.len) wblog(FL,"ERR %s() %d/%d !?",FCT,k,P.len);
03028         IDX(i,1)=P[k];
03029     }
03030     return Sort();

```

```

03031 };
03032
03033 template <class TD> inline
03034 wbsparray<TD>& wbsparray<TD>::MatPermute(const wbperm &P0, char iflag){
03035
03036     if (isIdentityPerm(P0)) return *this;
03037
03038     SPIDX_T d=-1;
03039     if (!isSMatrix(FL,&d)) wblog(FL,
03040 "ERR %s() requires rank-2 (%s; %d)",FCT,SSTR_(this),IDX.dim2);
03041     if (P0.len!=d) wblog(FL,
03042 "ERR %s() size mismatch (%d/%d)",FCT,P0.len,d);
03043
03044     if (isDiag()) { D.Permute(P0); }
03045     else {
03046         wbperm P;
03047         if (!iflag)
03048             P0.invert(P);
03049         else P.wbvector<wperm_t>::init2ref(P0);
03050
03051         SPIDX_T *I=IDX.data, n=IDX.numel(); wperm_t *p=P.data;
03052         for (SPIDX_T i=0; i<n; ++i) {
03053             if (I[i]>=P.len) wblog(FL,"ERR %s() %d/%d !?",FCT,I[i],P.len);
03054             I[i]=p[I[i]];
03055         }
03056     }
03057
03058     return Sort();
03059 };
03060
03061 template <class TD>
03062 void wbsparray<TD>::setSize_kron(
03063     const char *F, int L,
03064     const wbvector<SPIDX_T> &sa, const wbvector<SPIDX_T> &sb,
03065     char kflag
03066 ){
03067     wbvector<SPIDX_T> S;
03068     if (sa.len!=sb.len) wblog(F,L,
03069 "ERR %s() rank mismatch (%d/%d)",FCT,sa.len,sb.len);
03070
03071     if (kflag) { S.init(sa.len); SPIDX_T *s=S.data;
03072         for (unsigned i=0; i<sa.len; ++i) {
03073             s[i]=sa[i]*sb[i];
03074         }
03075     }
03076     else { S.init(sa.len+sb.len); SPIDX_T *s=S.data;
03077         for (unsigned i=0, l=0; i<sa.len; ++i) {
03078             s[l++]=sa[i];
03079             s[l++]=sb[i];
03080         }
03081     }
03082     S.save2(SIZE);
03083 };
03084
03085 template <class TD>
03086 void wbsparray<TD>::setSize_kron(const char *F, int L, char kflag) {
03087
03088     if (kflag) {
03089         unsigned l=0, i=0, r=IDX.dim2; SPIDX_T *s=SIZE.data;
03090         if (SIZE.len!=2*r) wblog(F,L,
03091 "ERR %s() unexpected size (%dx%d <> %d/(%d/2))",
03092 FCT,IDX.dim1,IDX.dim2,D.len,SIZE.len
03093 );
03094         if (r) {
03095             for (; i<r; ++i,l+=2) { s[i]=s[l]*s[l+1]; }
03096             SIZE.len=r;
03097         }
03098     }
03099     else if (SIZE.len!=IDX.dim2) sperror_this(F,L);
03100 };
03101
03102 template <class TD>
03103 wbsparray<TD>& wbsparray<TD>::setRec_kron(
03104     SPIDX_T l, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2,
03105     unsigned kflag
03106 ){
03107     if (l>IDX.dim1) wblog(FL,"ERR %s() index "
03108 "out of bounds (%d,%d <> %dx%d)",FCT,l,r,IDX.dim1,IDX.dim2);
03109
03110     if (kflag) {
03111         if (l==0 && (SIZE.len!=2*r || IDX.dim2!=r))
03112             wblog(FL,"ERR %s() 2*%d/2*%d/%d",FCT,r,IDX.dim2,SIZE.len);
03113         SPIDX_T *q=IDX.rec(l);
03114         for (unsigned i=0; i<r; ++i) {
03115             q[i]=i1[i] + SIZE[2*i]*i2[i];
03116         }
03117     }

```

```

03118     else {
03119         if (l==0 && (SIZE.len!=2*r || SIZE.len!=IDX.dim2))
03120             wblog(FL,"ERR %s() index out of bounds (%d,%d <> %dx%d)",
03121                 FCT,l,r,IDX.dim1,IDX.dim2
03122             );
03123
03124         SPIDX_T *q=IDX.rec(l);
03125         for (unsigned j=0, i=0; i<r; ++i) {
03126             q[j++]=i1[i];
03127             q[j++]=i2[i];
03128         }
03129     }
03130     D[l]=x; return *this;
03131 };
03132
03133 template <class TD>
03134 wbsparray<TD>& wbsparray<TD>::transpose(
03135     const char *F, int L, wbsparray<TD> &A
03136 ) const {
03137
03138     unsigned r=rank(); wbperr P;
03139
03140     if (r%2) wblog(F_L,"ERR %s() "
03141         "applies to even-rank arrays only! (%s)",FCT,SSTR_(this));
03142
03143     P.initTranspose(r);
03144     permute(P,A);
03145
03146     return A.Sort();
03147 };
03148
03149 template <class TD> inline
03150 wbsparray<TD>& wbsparray<TD>::tensorProd(const char *F, int L,
03151     const wbsparray<TD>& B0, wbsparray<TD>& C,
03152     char aflag0, char bflag0,
03153     char kflag
03154 ) const {
03155
03156     char iA=isDiag(FL), iB=B0.isDiag(FL);
03157
03158     if (iA || iB) {
03159         if (iA && iB) {
03160             if (kflag) {
03161                 C.initDiag(D.len*B0.D.len); D.tensorProd(B0.D,C,D);
03162                 return C;
03163             }
03164             wbsparray<TD> A2; diag2reg(FL,A2);
03165             wbsparray<TD> B2; B0.diag2reg(FL,B2);
03166             return A2.tensorProd(F,L,B2,C,aflag0,bflag0,kflag);
03167         }
03168         else if (iA) {
03169             wbsparray<TD> A2; diag2reg(FL,A2, B0.rank());
03170             return A2.tensorProd(F,L,B0,C,aflag0,bflag0,kflag);
03171         }
03172         else {
03173             wbsparray<TD> B2; B0.diag2reg(FL,B2, rank());
03174             return tensorProd(F,L,B2,C,aflag0,bflag0,kflag);
03175         }
03176     }
03177
03178     if ((void*)this==(void*)(&C) || (void*)(&B0)==(void*)(&C)) {
03179         wbsparray<TD> X; tensorProd(F_L,B0,X,aflag0,bflag0,kflag);
03180         return X.save2(C);
03181     }
03182
03183     unsigned ra=rank(), rb=B0.rank();
03184
03185     if (!ra || ra%2 || ra!=rb) wblog(F_L,"ERR %s() invalid input to "
03186         "kron(%s,%s)",FCT,SSTR_(this),SSTR(B0));
03187     if (SIZE.len!=IDX.dim2 || B0.SIZE.len!=B0.IDX.dim2) wblog(FL,
03188         "ERR %s() data inconsistency (%s -> %dx%d, %s -> %dx%d)",FCT,
03189         SSTR_(this), IDX.dim1, IDX.dim2,
03190         SSTR(B0),B0.IDX.dim1,B0.IDX.dim2);
03191
03192     opFlags<TD> aflag(aflag0), bflag(bflag0);
03193     wbsparray<TD> A(*this),B(B0);
03194
03195     if (!A.isCompressed()) wblog(FL,"ERR %s() A not sorted/unique",FCT);
03196     if (!B.isCompressed()) wblog(FL,"ERR %s() B not sorted/unique",FCT);
03197
03198     aflag.apply(F_L,A);
03199     bflag.apply(F_L,B);
03200
03201     SPIDX_T i,j, l=0, na=A.IDX.dim1, nb=B.IDX.dim1;
03202     C.init_kron(SIZE,B.SIZE,na*nb,kflag);
03203
03204     if (C.IDX.dim1) {

```



```

03205     for (i=0; i<na; ++i)
03206     for (j=0; j<nb; ++j,++l) {
03207         C.setRec_kron(
03208             l, A.D[i]*B.D[j], ra, A.IDX.rec(i), B.IDX.rec(j), kflag
03209         );
03210     }
03211 }
03212
03213 C.setSIZE_kron(F_L,kflag);
03214 C.Compress (FL);
03215
03216 return C;
03217 };
03218
03219 template <class TD> inline
03220 wbsparray<TD>& wbsparray<TD>::tensorProdX(const char *F, int L,
03221     const wbsparray& B, wbsparray& C,
03222     char conja, const TD &afac, char conjb, const TD &cfac,
03223     const wbpem *Pfinal
03224 ) const {
03225
03226     if (cfac && cfac!=1) { C.D*=cfac; }
03227     if (afac==0) { return C; }
03228
03229     char iA=isDiag(FL), iB=B.isDiag(FL), noconj=1;
03230     unsigned l=0, ra=(iA ? 2 : SIZE.len), rb=(iB ? 2 : B.SIZE.len);
03231
03232     const SPIDX_T *Ia=IDX.data, *Ib=B.IDX.data;
03233     SPIDX_T ia=0, ib=0, ic=0, na=D.len, nb=B.D.len, *Ic;
03234     wbsparray<TD> X;
03235
03236     if ((conja || conjb)) {
03237         if (ISCOMPLX_(TD)) { noconj=0; }
03238         else { conja=conjb=0; }
03239     }
03240
03241     X.SIZE.init(ra+rb); Ic=X.SIZE.data;
03242     if (iA) { Ic[0]=Ic[l]=na; }
03243     else { for (; l<ra; ++l) { Ic[l]=SIZE[l]; }; l=0; }; Ic+=ra;
03244     if (iB) { Ic[0]=Ic[l]=nb; }
03245     else { for (; l<rb; ++l) { Ic[l]=B.SIZE[l]; }}
03246     X.init_nnz(na*nb); Ic=X.IDX.data;
03247
03248     for (; ib<nb; ++ib) {
03249         for (ia=0; ia<na; ++ia, ++ic) {
03250             if (iA) { Ic[0]=Ic[l]=ia; }
03251             else { for (l=0; l<ra; ++l) { Ic[l]=Ia[l]; }; Ia+=ra; }
03252             Ic+=ra;
03253
03254             if (iB) { Ic[0]=Ic[l]=ib; }
03255             else { for (l=0; l<rb; ++l) { Ic[l]=Ib[l]; }; Ib+=rb; }
03256             Ic+=rb;
03257
03258             if (noconj) { X.D[ic]=D[ia]*B.D[ib]; }
03259             else { X.D[ic] =
03260                 (conja ? CONJ( D[ia]) : D[ia])
03261                 * (conjb ? CONJ(B.D[ib]) : B.D[ib]);
03262             }
03263         }
03264     }
03265
03266     if (Pfinal && !Pfinal->isIdentityPerm()) { X.Permute(*Pfinal); }
03267
03268     if (afac!=1) { X*=afac; }
03269     if (cfac) {
03270         if (X.SIZE!=C.SIZE) wblog(FL,"ERR %s() "
03271             "got size mismatch (%s <> %s) !?",FCT,SSTR(X),SSTR(C));
03272         C.Plus(F_L,X);
03273     }
03274     else X.save2(C);
03275
03276     return C;
03277 };
03278
03279 template <class TD>
03280 wbsparray<TD>& wbsparray<TD>::Plus(
03281     const char *F, int L,
03282     const wbsparray<TD> &B, TD bfac, TD afac,
03283     TD eps
03284 ){
03285     if (isref) wblog(FL,"ERR %s() got isref=%d !?",FCT,isref);
03286
03287     if ((void*)this==(void*)&B) {
03288         TD x=afac+bfac;
03289         if (x== 0) { D.init(); IDX.init(0,IDX.dim2); isref=0; } else
03290         if (x== -1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]=-D[i]; } else
03291         if (x!= +1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]*=x; }

```

```

03292     return *this;
03293 }
03294
03295 if (isEmpty()) { (*this)=B; (*this)*=bfac; return *this; }
03296 if (B.isEmpty()) { (*this)*=afac; return *this; }
03297
03298 char ida=isDiag(), idb=B.isDiag();
03299 if (ida || idb) {
03300     if (ida && idb) {
03301         if (D.len!=B.D.len) wblog(FL,
03302             "ERR %s() size mismatch (%ld/%ld)",FCT,D.len,B.D.len);
03303         for (SPIDX_T i=0; i<D.len; ++i) { D[i]=afac*D[i]+bfac*B.D[i]; }
03304         return *this;
03305     }
03306     else if (ida) { diag2reg();
03307         return Plus(F,L,B,bfac,afac,eps);
03308     }
03309     else if (idb) {
03310         wbsparray<TD> B2; B.diag2reg(FL,B2);
03311         return Plus(F,L,B2,bfac,afac,eps);
03312     }
03313 }
03314
03315 if (SIZE!=B.SIZE || !SIZE.len) wblog(FL,
03316     "ERR %s() got size mismatch (%s <> %s)",FCT,SSTR_(this),SSTR(B));
03317
03318 if (afac!=1) {
03319     if (afac== 0) { D.init(); IDX.init(0,IDX.dim2); isref=0; } else
03320     if (afac==-1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]=-D[i]; } else
03321     if (afac!=+1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]*=afac; }
03322 }
03323
03324 if (bfac==0) return *this;
03325
03326 SPIDX_T i=0, ia=0, ib=0, l=0, nx, na=IDX.dim1, nb=B.IDX.dim1;
03327 const SPIDX_T *Ia=IDX.data, *Ib=B.IDX.data;
03328 unsigned m=SIZE.len;
03329 char lex=0;
03330
03331 wbsparray<TD> X(SIZE);
03332 const TD *a=D.data, *b=B.D.data; TD *x, e2=0;
03333
03334 wbvector<char> qq_(na+nb); char q, *qq=qq_.data;
03335
03336 #ifndef WB_SKIP_ASSERT
03337     SPIDX_T Dmax=SIZE.max();
03338     for (ia=1; ia<na; ia+=Dmax) {
03339         if (Wb::recCompare(Ia+(ia-1)*m,Ia+ia*m,m,lex)>=0) wblog(FL,
03340             "ERR %s() A: input sparse IDX not sorted (%ld/%ld) !?",FCT,ia,na);
03341     }
03342     for (ib=1; ib<nb; ib+=Dmax) {
03343         if (Wb::recCompare(Ib+(ib-1)*m,Ib+ib*m,m,lex)>=0) wblog(FL,
03344             "ERR %s() B: input sparse IDX not sorted (%ld/%ld) !?",FCT,ib,nb);
03345     }
03346     ia=ib=0;
03347 #endif
03348
03349 while (ia<na && ib<nb) {
03350     qq[l++]=q=Wb::recCompare(Ia+ia*m,Ib+ib*m,m,lex);
03351
03352     if (q<=0) { ++ia; }
03353     if (q>=0) { ++ib; }
03354 }
03355
03356 if (ia<na)
03357     { for (; ia<na; ++ia) { qq[l++]=-1; } }
03358 else { for (; ib<nb; ++ib) { qq[l++]="+1; } }
03359
03360 X.init_nnz(1); x=X.D.data; nx=X.IDX.dim1; ia=ib=1=0;
03361
03362 for (; i<nx; ++i) {
03363     if (qq[i]<0) {
03364         x[l]=a[ia]; if (afac!=1) x[l]*=afac;
03365         if (ABS(x[l])>eps) X.IDX.recSetP(l++,Ia); else e2+=NORM2(x[l]);
03366         ++ia; Ia+=m;
03367     }
03368     else if (qq[i]>0) {
03369         x[l]=b[ib]; if (bfac!=1) x[l]*=bfac;
03370         if (ABS(x[l])>eps) X.IDX.recSetP(l++,Ib); else e2+=NORM2(x[l]);
03371         ++ib; Ib+=m;
03372     }
03373     else {
03374         x[l]=a[ia]; if (afac!=1) x[l]*=afac;
03375         x[l]+=(bfac!=1 ? bfac*b[ib] : b[ib]);
03376         if (ABS(x[l])>eps) X.IDX.recSetP(l++,Ia); else e2+=NORM2(x[l]);
03377         ++ia; ++ib; Ia+=m; Ib+=m;
03378     }
}

```

```

03379     }
03380
03381     X.REDSIZE_NNZ(1);
03382
03383     return X.save2(*this);
03384 };
03385
03386 template <class TD>
03387 double wbsparray<TD>::Compress(
03388     const char *F, int L, TD eps, char lex, const wbpem *Pc
03389 ) {
03390     wbpem P; WBINDEX dg; rank(F_L);
03391
03392     checkSize(F_L);
03393     if (!IDX.dim1) {
03394         double e2=0;
03395         if (Pc && Pc->len!=2) wblog(FL,
03396             "ERR %s() invalid permutation (%s)",FCT,Pc->toStr().data);
03397         if (double(eps)>0) {
03398             for (SPIDX_T k=0; k<D.len; ++k) {
03399                 TD &x=D.data[k];
03400                 if (x && ABS(x)<=eps) { e2+=double(NORM2(x)); x=0; }
03401             }
03402         }
03403         return e2;
03404     }
03405
03406     if (!Pc || Pc->isIdentityPerm()) {
03407         if (double(eps)<=0 && IDX.isUniqueSorted(+1,lex)) { return 0; }
03408         IDX.groupRecs(P,dg,-1,lex);
03409     }
03410     else {
03411         wbMatrix<SPIDX_T> X; IDX.colPermute(*Pc,X);
03412         X.groupRecs(P,dg,-1,lex);
03413     }
03414
03415 #ifdef WB_CLK_SPARSE
03416     Wb::Clock clk("sparse:cmpr",0);
03417 #endif
03418
03419     SPIDX_T i,j,k,d; wperm_t *p=P.data;
03420     wbvector<TD> D0; D.save2(D0); D.init(dg.len);
03421     const TD *d0=D0.data;
03422
03423     for (j=k=0; j<dg.len; ++j,++k) {
03424         TD &x=D.data[k]; x=d0[p[0]];
03425         for (d=dg.data[j], i=1; i<d; ++i) { x+=d0[p[i]]; }
03426
03427         p+=d;
03428         if (x==0) { --k; } else if (k<j) { IDX.recSet(k,j); }
03429     }
03430
03431     REDSIZE_NNZ(k);
03432
03433     return (double)SkipTiny(eps);
03434 };
03435
03436 template <class TD>
03437 int wbsparray<TD>::splitSparseCM(
03438     const char *F, int L, wbvector< wbsparray<TD> > &X,
03439     TD eps,
03440     char ref,
03441     char tnorm
03442 ) {
03443     iterSparseCMref<TD> ISP(F_L,*this);
03444     SPIDX_T i=0, n=ISP.numIter(); if (!n) { X.init(); return X.len; }
03445
03446     if (eps>0) {
03447         wbvector<char> mark(n); SPIDX_T l=0;
03448         for (; i<n; ++i) { if (ISP.norm2(i,tnorm)>=eps) { mark[i]=1; ++l; }}
03449         X.init(l); i=l=0;
03450         for (; i<n; ++i) { if (mark[i]) { ISP.getIter(i,X[l++],ref); }}
03451     }
03452     else {
03453         X.init(n);
03454         for (; i<n; ++i) { ISP.getIter(i,X[i],ref); }
03455     }
03456
03457     return X.len;
03458 };
03459
03460 template <class T>
03461 inline int checkContract(const char* F, int L,
03462     const wbvector<T> &SA, const ctrIdx &ica,
03463     const wbvector<T> &SB, const ctrIdx &ich,
03464     const wbpem *pfinal, wbvector<T> *SC
03465 ) {

```

```

03466     int e=0;
03467
03468     if (ica.len!=icb.len) {
03469         if (F) wblog(F,L,
03470             "ERR %s() invalid index set (length mismatch, [%s], [%s])",
03471             FCT,STR(ica),STR(icb));
03472         return 1;
03473     }
03474     if (!ica.len) {
03475         if (F) wblog(F,L,"ERR contract() got empty ic[ab] !?");
03476         return 1;
03477     }
03478
03479     for (unsigned i=0; i<ica.len; ++i) {
03480         if ((ica[i]<SA.len ? SA[ica[i]] : 1) !=
03481             (icb[i]<SB.len ? SB[icb[i]] : 1))
03482             wblog(F,L,
03483                 "ERR invalid contraction @ i=%d/%d ... !?\n"
03484                 "having: %s (%s) * %s (%s)",i+1,ica.len,
03485                 SSTR(SA), STR(ica), SSTR(SB), STR(icb)
03486             );
03487     }
03488
03489     if (SC) {
03490         wvector<T> sC; sC.initI(SA,ica,SB,icb);
03491
03492         if (SC->isEmpty()) { sC.save2(*SC); }
03493         else {
03494             if (pfinal && pfinal->len) sC.Permute(*pfinal);
03495             if ((*SC)!=sC) { e=6; if (F) wblog(F,L,
03496                 "ERR %s() severe size mismatch (C: %s <> %s)",
03497                 FCT, SSTR(sC), SSTR_(SC));
03498             }
03499         }
03500
03501         if (!SC->prod(0)) return -1;
03502     }
03503
03504     return e;
03505 };
03506
03507 template <class TD> inline
03508 char wbsparray<TD>::contract_scalar(
03509     const char* F, int L, const ctrIdx &ica,
03510     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03511     const wperm &pfinal, const TD& afac, const TD& cfac
03512 ) const {
03513
03514     char isa=isScalar(), isb=B.isScalar(), gotC=(cfac && !C.isEmpty());
03515     unsigned r=-1;
03516     if (!isa || !isb) return 0;
03517
03518     TD x=(D.len && B.D.len ? afac*D[0]*B.D[0] : TD(0));
03519     if (gotC && C.D.len) { x+=cfac*C.D[0]; }
03520
03521     if (ica.len!=icb.len) wblog(F,L,"ERR %s() length mismatch: "
03522         "%s <> %s",FCT, STR(ica), STR(icb));
03523     if (gotC && (!C.isScalar() || C.D.len>1)) wblog(F,L,
03524         "ERR %s() invalid scalar %s",FCT,C.info2Str("C").data);
03525     if (SIZE.len!=IDX.dim2 || B.SIZE.len!=B.IDX.dim2) wblog(F,L,
03526         "ERR %s() invalid scalars %s <> %s", FCT,
03527         info2Str("A").data, B.info2Str("B").data);
03528
03529     if (SIZE.len) {
03530         if (B.SIZE.len) {
03531             checkContract(F_L, SIZE,ica,B.SIZE,icb,
03532                 (gotC? &pfinal : NULL), (gotC? &C.SIZE : NULL));
03533             r=(SIZE.len + B.SIZE.len - 2*ica.len);
03534         }
03535         else {
03536             wvector<SPIDX_T> Sb(2); Sb[0]=Sb[1]=B.D.len;
03537             checkContract(F_L, SIZE,ica,Sb,icb,
03538                 (gotC? &pfinal : NULL), (gotC? &C.SIZE : NULL));
03539             r=(SIZE.len + 2 - 2*ica.len);
03540         }
03541     }
03542     else {
03543         if (B.SIZE.len) {
03544             wvector<SPIDX_T> Sa(2); Sa[0]=Sa[1]=D.len;
03545             checkContract(F_L,Sa,ica,B.SIZE,icb,
03546                 (gotC? &pfinal : NULL), (gotC? &C.SIZE : NULL));
03547             r=(2 + B.SIZE.len - 2*ica.len);
03548         }
03549     }
03550
03551     C.initScalar(x,r);
03552

```

```

03553     return 1;
03554 };
03555
03556 template <class TD>
03557 wbsparray<TD>& wbsparray<TD>::contract_diag_diag(
03558     const char* F, int L, const ctrIdx &ica,
03559     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03560     const wbpperm &pfinal, const TD& afac, const TD& cfac
03561 ) const {
03562     SPIDX_T i=0, ra=2, rb=2; int e=0;
03563
03564     if (SIZE.len || B.SIZE.len) wblog(F_L,
03565         "ERR %s() (%s) <> (%s)", FCT, SSTR_(this), SSTR(B));
03566     if (D.len!=B.D.len) wblog(F_L,
03567         "ERR %s() size mismatch (%d/%d)", FCT, D.len, B.D.len);
03568
03569     if (!ica.len) e=-1; else
03570     if (ica.len!=icb.len || ica.len>2) e=-2; else {
03571         for (; i<ica.len; ++i) if (ica[i]>ra || icb[i]>rb) e=3;
03572     }
03573     if (!e && ica.len==2 && (ica[0]==ica[1] || icb[0]==icb[1])) e=4;
03574     if (e) wblog(F_L,
03575         "ERR %s() invalid contraction %s (%s) <> %s (%s)",
03576         FCT, SSTR_(this), STR(ica), SSTR(B), STR(icb)
03577     );
03578
03579     if (ISCOMPLX(TD) && (ica.conj || icb.conj))
03580     wblog(F_L, "ERR %s() complex not yet implemented", FCT);
03581
03582     wvector<TD> x(D.len);
03583     if (afac) {
03584         for (i=0; i<D.len; ++i) { x[i]=D[i]*B.D[i]; }
03585     }
03586
03587     if (ica.len==2) { TD q=afac*x.sum();
03588         if (cfac) {
03589             if (!C.isScalar()) wblog(F_L,
03590                 "ERR %s() invalid scalar C (%s)", FCT, SSTR(C));
03591             C.D*=cfac; C[0]+=q;
03592         }
03593         else C.initScalar(q);
03594     }
03595     if (!pfinal.isEmpty()) wblog(F_L,
03596         "ERR %s() got pfinal=[%s] for scalar !?", FCT, STR(pfinal)
03597     );
03598 }
03599
03600 else {
03601     if (cfac) {
03602         if (!C.isDiag() || C.D.len!=D.len) wblog(F_L,
03603             "ERR %s() invalid diag C (%s; %d) @ cfac=%.3g",
03604             FCT, SSTR(C), D.len, double(cfac));
03605         C.D*=cfac; C.D.Plus(x, afac);
03606     }
03607     else {
03608         if (afac) x*=afac;
03609         C.initDiag(D.len); x.save2(C.D);
03610     }
03611
03612     if (!pfinal.isEmpty() && !pfinal.isValidPerm(0,0,2))
03613     wblog(F_L, "ERR %s() got pfinal=[%s] !?", FCT, STR(pfinal));
03614 }
03615
03616 return C;
03617 };
03618
03619 template <class TD>
03620 char wbsparray<TD>::contract_check_2full(
03621     const char* F, int L, const ctrIdx &ica,
03622     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03623     const wbpperm &pfinal, const TD& afac, const TD& cfac,
03624     char flag, TD eps, SPIDX_T *nnz_C
03625 ) const {
03626
03627     if (flag=='s' || flag=='S'
03628         || typeid(TD)!=typeid(double))
03629     return 0;
03630
03631     if (flag && flag!='f' && flag!='F')
03632     wblog(F_L, "ERR %s() invalid flag=%c<%d>", FCT, flag, flag);
03633
03634     if (!flag || nnz_C!=NULL) {
03635         SPIDX_T NA=numel(), NB=B.numel(), M=SIZE.prod(ica.data, ica.len);
03636         SPIDX_T NC=(NA/M)*(NB/M);
03637         double pa=nnz()/double(NA), pb=B.nnz()/double(NB);
03638
03639         double pc=M*pa*pb; if (pc>1) pc=1;

```

```

03640
03641     if (!NA || !NB || !M) wblog(F_L,
03642         "ERR %s() got\n%s [%s] <> %s [%s] (%d)", FCT, SSTR_(this),
03643         ica.toStr().data, SSTR(B), STR(icb), M);
03644
03645     if ((pc>0.50 && NC<(1<<24)) ||
03646         (pc>0.20 && NC<(1<<16)) ||
03647         (pc>0.10 && NC<(1<<12))
03648     ) { flag='F';
03649         wvector<SPIDX_T> sC; sC.initI(SIZE,ica,B.SIZE,icb);
03650     }
03651
03652     if (nnz_C) {
03653         double nc=pc*NC;
03654         if (NC>128) (nc)*=1.20; else
03655         if (NC> 32) (nc)*=1.50; else
03656         if (NC>  4) (nc)*=2; else nc=NC;
03657         (*nnz_C)=MIN(SPIDX_T(nc),NC);
03658     }
03659 }
03660
03661 if (!flag) return 0;
03662
03663 #ifdef WB_CLK_SPARSE
03664 Wb::Clock clk("sparse:ctr-full",0);
03665 #endif
03666
03667 wbararray<TD> Af,Bf,ABf; this->toFull(Af); B.toFull(Bf);
03668
03669 Af.contract(F_L,ica,Bf,icb,ABf,pfinal,afac);
03670
03671 if (cfac!=0 && C.D.len) { C*=cfac;
03672     wbararray<TD> Cf; C.toFull(Cf);
03673     ABf+=Cf;
03674 }
03675
03676 C.init(F_L,ABf,eps); return 1;
03677 };
03678
03679 template <class TD>
03680 wbsparray<TD>& wbsparray<TD>::contract(
03681     const char* F, int L, const ctrIdx &ica,
03682     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03683     const wperm &pfinal, const TD& afac, const TD& cfac,
03684     char sflag,
03685     TD eps
03686 ) const {
03687     static int nvlog=0;
03688
03689 #ifdef WB_CLK_SPARSE
03690 Wb::Clock clk("sparse:ctr:all",0);
03691 #endif
03692
03693 if (D.len<=1 && B.D.len<=1) {
03694     if (contract_scalar(F_L,ica,B,icb,C,pfinal,afac,cfac)) {
03695         return C;
03696     }
03697 }
03698
03699 if (!sflag && typeid(TD)!=typeid(double)) sflag='s';
03700
03701 if (this==&C || &B==&C) { wbsparray<TD> X(C);
03702     contract(F,L,ica,B,icb,X,pfinal,afac,cfac,sflag,eps);
03703     return X.save2(C);
03704 }
03705
03706 char iA=isDiag(FL), iB=B.isDiag(FL);
03707 if (iA || iB) {
03708     if (iA && iB) {
03709         if ((ica.len && ica.len<=2) && (icb.len && icb.len<=2))
03710             return contract_diag_diag(F_L,ica,B,icb,C,pfinal,afac,cfac);
03711         else {
03712             if (ica.len || icb.len) wblog(FL,
03713                 "ERR %s() got %d/%d !?",FCT,ica.len,icb.len);
03714             return tensorProdX(FL,B,C,ica.conj,afac,icb.conj,cfac,&pfinal);
03715         }
03716     }
03717     else if (iA) {
03718         wbsparray<TD> A2; diag2reg(FL,A2);
03719         return A2.contract(F,L,ica,B,icb,C,pfinal,afac,cfac,sflag,eps);
03720     }
03721     else if (iB) {
03722         wbsparray<TD> B2; B.diag2reg(FL,B2);
03723         return contract(F,L,ica,B2,icb,C,pfinal,afac,cfac,sflag,eps);
03724     }
03725 }
03726 }

```

```

03727
03728     wvector<SPIDX_T> sC; if (cfac) sC=C.SIZE; int e=0;
03729
03730     if ((e=checkContract(F_L,SIZE,ica,B.SIZE,icb, &pfinal, &sC))) {
03731         if (e<0) {
03732             if (sC.len) {
03733                 C.init(sC);
03734                 return C;
03735             }
03736         }
03737         else wblog(FL,"ERR %s()",FCT);
03738     }
03739
03740     if (!D.len || !B.D.len) {
03741         if (cfac) { C.D*=cfac; } else { C.init(sC); }
03742         if (e<0) {
03743             if (C.SIZE.len || C.IDX.dim2) wblog(FL,"ERR %s() !?",FCT);
03744             if (!C.D.len) { C.D.init(1); C.D[0]=0; }
03745         }
03746         return C;
03747     }
03748
03749     if (contract_check_2full(F_L,
03750         ica,B,icb,C,pfinal,afac,cfac,sflag,eps)) { return C; }
03751
03752     wbsparray<TD> C0;
03753
03754     if (cfac!=0) { C.D*=cfac; }
03755     if (afac==0) {
03756         if (!cfac) C.init(sC.Permute(pfinal));
03757         return C;
03758     }
03759     if (cfac!=0) C.save2(C0);
03760
03761     if (B.IDX.dim2==1) {
03762         if (B.SIZE.len!=1 || ica.len!=1) wblog(FL,
03763             "ERR %s() got S=[%s] !?",FCT,SSTR(B));
03764         wvector<TD> b; B.toFull(b);
03765         contract(FL,ica[0],b,C);
03766
03767         C.Permute(pfinal); if (afac!=1) C*=afac;
03768         if (!C0.isEmpty()) { C+=C0; }
03769
03770         return C;
03771     }
03772
03773 #ifdef WB_CLK_SPARSE
03774     Wb::Clock cl2("sparse:ctr:2",0);
03775 #endif
03776
03777     sparseIndex2D<SPIDX_T> a2, b2;
03778
03779     toIndex2D(ica,a2,2,'r');
03780     B.toIndex2D(icb,b2,1);
03781
03782     if (!a2.IJ.dim1 || a2.IJ.dim2!=2 || !a2.cIdx.len ||
03783         !b2.IJ.dim1 || b2.IJ.dim2!=2 || !b2.cIdx.len)
03784         wblog(FL,"ERR %s() unexpected IJ data (%s [%ld]; %s [%ld])",FCT,
03785             SSTR(a2.IJ), a2.cIdx.len,
03786             SSTR(b2.IJ), b2.cIdx.len
03787         );
03788
03789     if (b2.rmaj) wblog(FL,
03790         "ERR %s() got unexpected sparse dimensions\n B: %s @ [%s]",
03791         FCT,SSTR(B), SSTR(icb)
03792     );
03793
03794     int np=1; if (sp_num_threads>1) {
03795         np=(IDX.dim1+B.IDX.dim1)/1024;
03796         if (np>sp_num_threads) { np=sp_num_threads; }
03797         if (unsigned(np)>b2.S[1]) { np=b2.S[1]; }
03798         if (np<1) { np=1; }
03799     }
03800
03801 #ifdef LOAD_CGC_QSPACE
03802     if (np<sp_num_threads && CG_VERBOSE>5) {
03803         static time_t tlast=0;
03804         time_t tnow=time(NULL);
03805         if (tnow>tlast+60) { size_t s1=(1<26);
03806             if (numel(>s1 || B.numel(>s1 || sC.prod(0)>s1) {
03807                 wblog(FL,"CTR %12s * %12s @ %d/%d threads",
03808                     SSTR(a2), SSTR(b2), np, sp_num_threads
03809                 );
03810                 tlast=tnow;
03811             }
03812         }
03813     }

```

```

03814 #endif
03815
03816 if (a2.rmaj) {
03817     size_t s1=a2.S[0], s2=b2.S[1], la=a2.IJ.dim1-1;
03818     const size_t nk=b2.cIdx.len-1;
03819     const SPIDX_T *pa=a2.P.data, *pb=b2.P.data;
03820
03821     if (s1*s2>(1<<20)) wblog(FL,
03822         "ERR %s() got a.rmaj=%d having %dx%d * %dx%d !?!",FCT,
03823         a2.S[0], a2.S[1], b2.S[0], b2.S[1]
03824     );
03825
03826     wbararray<TD> Cx(s1,s2);
03827     Wb::LogException ex;
03828
03829     #pragma omp parallel for num_threads(np)
03830     for (unsigned k=0; k<nk; ++k) { if (!ex) { try {
03831         size_t ib=b2.cidx(k), lb=b2.cidx(k+1);
03832
03833         if (lb>ib) { --lb;
03834             size_t ia=0, ib_=ib;
03835             SPIDX_T *Ia=a2.IJ.data, *Ib=b2.IJ.data+2*ib;
03836             char c;
03837
03838             while (ib<=lb) {
03839                 c=NUMCMP (Ia[1],Ib[0]);
03840
03841                 if (c==0) {
03842                     if (ia<la) {
03843                         if (Ia[0]!=Ia[2]) c=-2;
03844                     }
03845                     else {
03846                         c+=3;
03847                     }
03848
03849                     if (ib<lb) {
03850                         if (!c && Ib[1]!=Ib[3]) c+=2;
03851                     }
03852
03853                     if (Ia[0]>=s1 || Ib[1]>=s2) wblog(FL,
03854                         "ERR %s() index out of bounds (%ld/%ld; %ld/%ld)",
03855                         FCT, Ia[0], s1, Ib[1], s2);
03856
03857                     Cx(Ia[0],Ib[1]) += D[pa[ia]] * B.D[pb[ib]];
03858                 }
03859
03860                 if (ia<la && (c<0 || ib==lb)) {
03861                     if (Ia[0]==Ia[2]) { if (Ia[1]>=Ia[3]) wblog(FL,
03862                         "ERR %s() a.IJ not sorted or unique !?",FCT); }
03863                     else {
03864                         if (Ia[0]>Ia[2]) wblog(FL,
03865                             "ERR %s() a.IJ not sorted !?",FCT);
03866                         Ib-=2*(ib-ib_); ib=ib_;
03867                     }
03868                     ++ia; Ia+=2;
03869                 }
03870                 else if (c) {
03871                     if (ib<lb) {
03872                         if (Ib[1]==Ib[3]) { if (Ib[0]>=Ib[2]) wblog(FL,
03873                             "ERR %s() b.IJ not sorted or unique !?",FCT); }
03874                         else {
03875                             if (Ib[1]>Ib[3]) wblog(FL,
03876                                 "ERR %s() b.IJ not sorted !?",FCT);
03877                             ib_=ib+1; Ia-=2*ia; ia=0;
03878                         }
03879                     }
03880                     ++ib; Ib+=2;
03881                 }
03882                 else {
03883                     if (ia<la && Ia[1]>=Ia[3])
03884                         wblog(FL,"ERR %s() a.IJ not sorted !?",FCT);
03885                     if (ib<lb && Ib[0]>=Ib[2])
03886                         wblog(FL,"ERR %s() b.IJ not sorted !?",FCT);
03887
03888                     ++ia; Ia+=2;
03889                     ++ib; Ib+=2;
03890                 }
03891             }}}
03892             catch (Wb::LogException &e_) { ex+=e_; }
03893             catch (...) { ++ex; }
03894         }}
03895     ex.report (FLF);
03896
03897     C.init (FL,Cx).Reshape (sC);
03898     if (!C.SIZE.len && !C.D.len) {
03899         C.D.init (1); C.D[0]=0;
03900     }

```



```

03901
03902     C.Permute(pfinal); if (afac!=1) C*=afac;
03903     if (!C0.isEmpty()) { C+=C0; }
03904
03905     return C;
03906 }
03907
03908 wbMatrix<SPIDX_T> MARK(np,a2.S[0]);
03909 groupIndex<widx_t> kc; { kc.init(b2.S[1]); }
03910
03911 const SPIDX_T *pa=a2.P.data, *pb=b2.P.data;
03912 SPIDX_T k=0, ltot=0, nnzc=0;
03913
03914 #pragma omp parallel for num_threads(np) reduction(+:ltot)
03915 for (k=0; k<b2.S[1]; ++k) {
03916     int id=omp_get_thread_num();
03917     widx_t *mark=MARK.ref(id,0);
03918     SPIDX_T i, i_, i1, i2, j, j_, j1=b2.cidx(k), j2=b2.cidx(k+1), nnzc=0;
03919     for (j_=j1; j_<j2; ++j_) { j=b2.ridx(j_);
03920         i1=a2.cidx(j); i2=a2.cidx(j+1);
03921         for (i_=i1; i_<i2; ++i_) { i=a2.ridx(i_);
03922             if (mark[i]<k+1) { mark[i]=k+1; ++nnzc; }
03923         }
03924     }
03925     kc.setD(k,nnzc);
03926     ltot+=nnzc;
03927 }
03928
03929 nnzc=kc.init_ic();
03930 if (nnzc!=ltot) wblog(FL,"ERR %s() "
03931     "nnz inconsistency (%ld/%ld) !?",FCT,ltot,nnzc);
03932
03933 #ifdef WB_CLK_SPARSE
03934     cl2.Switch("sparse:ctr:3");
03935 #endif
03936
03937 if (!nnzc) {
03938     if (C0.isEmpty()) C.init(sC); else C0.save2(C);
03939     return C;
03940 }
03941
03942 wbMatrix<TD> W(np,a2.S[0]);
03943 wbMatrix<SPIDX_T> IC_(np,a2.S[0]), IC(np,a2.S[0]);
03944 MARK.set(0);
03945
03946 ctrIdx ika, ikb;
03947 int largeD=0;
03948
03949 ica.invert( SIZE.len,ika);
03950 icb.invert( B.SIZE.len,ikb);
03951
03952 { size_t DX=(1<<26);
03953
03954     if (D.len>DX || B.D.len>DX || nnzc>DX) { largeD=2; nvlog=1; }
03955     else {
03956         DX=(1<<20);
03957         if (D.len>DX || B.D.len>DX || nnzc>DX) { largeD=1;
03958             if ((++nvlog)%24==1) largeD|=2;
03959         }
03960     }
03961
03962     if (largeD&2) {
03963         wblog(FL,"TST %s() large sparse arrays (%d/%d threads)\n"
03964             " A: %-20s (%9.3g) @ %s\n B: %-20s (%9.3g) @ %s\n"
03965             "> C: %-20s (%9.3g)",FCT, np, sp_num_threads,
03966             SSTR_(this), D.len/double( numel()), STR(ica),
03967             SSTR(B), B.D.len/double(B.numel()), STR(icb),
03968             STR(sC), nnzc/double(sC.prod(1)));
03969     }
03970     else if (largeD) {
03971         wblog(FL,"CTR %s_%s * %s_%s = %s (np=%d/%d)", SSTR_(this),
03972             STR(ica), SSTR(B), STR(icb), SSTR(sC), np, sp_num_threads);
03973     }
03974     if (largeD) { Wb::MemStat(FL); doflush(); }
03975 }
03976
03977 C.init(sC,nnzc);
03978 ltot=0;
03979
03980 #pragma omp parallel for num_threads(np) reduction(+:ltot)
03981 for (k=0; k<b2.S[1]; ++k) {
03982     int id=omp_get_thread_num();
03983     wbvector<SPIDX_T> ic(IC.dim2,IC.ref(id,0),'r');
03984     SPIDX_T *ic_=IC_.ref(id,0), is=0;
03985     widx_t *mark=MARK.ref(id,0);
03986     TD *w=W.ref(id,0);
03987 }

```

```

03988     SPIDX_T i, i_, i1, i2, j, j_, j1=b2.cidx(k), j2=b2.cidx(k+1);
03989     bool sortflag=((j2-j1)<SPIDX_T(ceil(0.2*MARK.dim2)));
03990
03991     for (j_=j1; j_<j2; ++j_) { j=b2.ridx(j_);
03992         const TD &Bjk=B.D[pb[j_]];
03993         i1=a2.cidx(j); i2=a2.cidx(j+1);
03994         for (i_=i1; i_<i2; ++i_) { i=a2.ridx(i_);
03995             if (mark[i]<k+1) { mark[i]=k+1; ic_[i]=pa[i_];
03996                 if (sortflag) ic[is++]=i;
03997                 w[i] = D[pa[i_]] * Bjk;
03998             }
03999             else {
04000                 w[i]+=(D[pa[i_]] * Bjk);
04001             }
04002         }
04003     }
04004
04005     SPIDX_T l=kc.getGroup(k);
04006
04007     if (sortflag) {
04008         if (is) {
04009             ic.len=is; ic.Sort();
04010             for (i_=0; i_<ic.len; ++i_) { i=ic[i_];
04011                 C.IDX.recSetP(l,IDX.rec(ic_[i]),ika,B.IDX.rec(pb[j1]),ikb);
04012                 C.D[l++]=w[i];
04013             }
04014             ic.len=MARK.dim2;
04015         }
04016         ltot+=is;
04017     }
04018     else {
04019         for (i=0; i<MARK.dim2; ++i) {
04020             if (mark[i]==k+1) {
04021                 C.IDX.recSetP(l,IDX.rec(ic_[i]),ika,B.IDX.rec(pb[j1]),ikb);
04022                 C.D[l++]=w[i]; ++ltot;
04023             }
04024         }
04025     }
04026 }
04027
04028 if (ltot!=C.D.len) wblog(FL,
04029     "ERR %s() %d/%d/%d !?",FCT,ltot,C.D.len,nnc);
04030
04031 if (largeD) {
04032     size_t s=C.memSize();
04033     if (s>(1<<30)) wblog(FL,
04034         "TST %s() size(C) = %s",FCT,Wb::size2Str(s).data);
04035     Wb::MemStat(FL);
04036 }
04037 #ifdef WB_CLK_SPARSE
04038     cl2.stop();
04039 #endif
04040
04041 C.Permute(pfinal); if (afac!=1) C*=afac;
04042 if (!C0.isEmpty()) { C+=C0; }
04043
04044 return C;
04045 };
04046
04047 template <class TD>
04048 wbspararray<TD>& wbspararray<TD>::contract(
04049     const char *F, int L,
04050     unsigned k, const wvector<TD> &B, wbspararray<TD> &C
04051 ) const {
04052
04053     if (k>=SIZE.len) {
04054         if (B.len!=1) wblog(F_L,"ERR %s() "
04055             "size mismatch (%s ; %d @ %d)",FCT,SSTR_(this),B.len,k+1);
04056         C=*this; C*=B.data[0];
04057         if (ABS(B.data[0])<TD(1E-8)) C.Compress();
04058         return C;
04059     }
04060
04061     if (SIZE[k]!=B.len) wblog(F_L,
04062         "ERR %s() size mismatch (%s ; %d @ %d)",FCT,SSTR_(this),B.len,k+1);
04063     if (IDX.dim1!=D.len || IDX.dim2!=SIZE.len) wblog(FL,
04064         "ERR %s() size inconsistency (%dx%d <> %dx%d) !?",
04065         FCT,IDX.dim1,IDX.dim2,D.len,SIZE.len);
04066
04067     if (IDX.dim2>1) {
04068         const SPIDX_T *idx=IDX.data+k;
04069         const TD *b=B.data; TD *c; windex Ik;
04070
04071         Ik.Index_ex(IDX.dim2,k); C.init();
04072         IDX.getCols(Ik,C.IDX); C.D=D; c=C.D.data;
04073         SIZE.select(Ik,C.SIZE);
04074     }

```

```

04075         for (SPIDX_T n=IDX.dim2, i=0; i<D.len; ++i, idx+=n) {
04076             if ((*idx)>=B.len) wblog(F_L,
04077                 "ERR %s() index out of bounds (%d/%d)",FCT,(*idx)+1,B.len);
04078             c[i]*=b[*idx];
04079         }
04080
04081         C.Compress();
04082     }
04083     else if (IDX.dim2==1) {
04084         if (int(k)>=0 && k) wblog(F_L,
04085             "ERR %s() ctrIdx out of bounds (%d/1)",FCT,k+1);
04086         C.initScalar();
04087
04088         const SPIDX_T *idx=IDX.data;
04089         const TD *a=D.data, *b=B.data;
04090         TD &c=C.D[0]; c=0;
04091
04092         for (SPIDX_T i=0; i<D.len; ++i) {
04093             if (idx[i]>=B.len) wblog(F_L,
04094                 "ERR %s() index out of bounds (%d/%d)",FCT,idx[i]+1,B.len);
04095             c+=a[i]*b[idx[i]];
04096         }
04097     }
04098     else {
04099         info(F_L); wblog(F_L,"ERR %s()",FCT);
04100     }
04101
04102     return C;
04103 };
04104
04105 template <class TD>
04106 wbararray<TD>& wbsparray<TD>::contract(
04107     const char* F, int L, const ctrIdx &ica,
04108     const wbsparray<TD> &B, const ctrIdx &icb, wbararray<TD> &C,
04109     const wbpperm &pfinal, const TD& afac, const TD& cfac
04110 ) const {
04111
04112     wbararray<TD> Af,Bf,X;
04113
04114     if (cfac && !C.isEmpty()) { C.save2(X); X*=cfac; }
04115
04116     this->toFull(Af); B.toFull(Bf);
04117     Af.contract(F_L,ica,Bf,icb,C,pfinal,afac);
04118
04119     if (!X.isEmpty()) {
04120         if (!X.sameSize(C)) wblog(F_L,
04121             "ERR %s() severe size mismatch (%s <> %s; %g)",
04122             FCT, SSTR(C), SSTR(X), cfac);
04123         C+=X;
04124     }
04125     return C;
04126 };
04127
04128 template <class TD>
04129 wbararray<TD>& contract(const char* F, int L,
04130     const wbararray<TD> &A, const wbindex &ic1,
04131     const wbsparray<TD> &B, const wbindex &ic2, wbararray<TD> &C,
04132     const wbpperm &pfinal, const TD& afac, const TD& cfac
04133 ){
04134     unsigned m=ic1.len, r1=A.rank(), r2=B.rank(F_L); int e=0;
04135     wbararray<TD> X;
04136
04137     if ((void*)&A==(void*)&C) wblog(F_L,
04138         "ERR sparse::%s() must have distinct target space",FCT);
04139
04140     wvector<SPIDX_T> sC; if (cfac) sC=C.SIZE;
04141     if ((e=checkContract(F_L,A.SIZE,ic1,B.SIZE,ic2, &pfinal, &sC))) {
04142         if (e<0) { C.init(sC); if (sC.len) return C; }
04143         else wblog(F_L,"ERR %s()",FCT);
04144     }
04145
04146     if (afac==0 || !B.D.len) {
04147         if (cfac!=0) { C*=cfac; } else { C.init(sC.Permute(pfinal)); }
04148         return C;
04149     }
04150
04151     if (cfac!=0) { C*=cfac; C.save2(X); }
04152     C.init(sC);
04153
04154     wvector<SPIDX_T> s1,D2;
04155     wMatrix<SPIDX_T> I2;
04156     wbpperm Pa,P2;
04157
04158     A.SIZE.getI(ic1,s1); Pa.init2End(ic1,r1);
04159     B.IDX.cols2End(ic2,I2).groupRecs(P2,D2,r2-m);
04160
04161     wbindex Ia(r1),Ic(sC.len);

```

```

04162     SPIDX_T i, ig,d,k,l, k1=r1-ic1.len, k2=r2-ic2.len;
04163     SPIDX_T *i2, *ia=Ia.data, *ic=Ic.data, *pa=Pa.data;
04164     wbindex K1(s1);
04165
04166     for (l=ig=0; ig<D2.len; ++ig) { d=D2[ig];
04167     for (k=0; k<d; ++k,++l) {
04168         TD b=B.D.data[P2[l]]; if (b==0) continue; if (afac!=1) b*=afac;
04169
04170         i2=I2.ref(l); if (k==0) {
04171             for (i=0; i<k2; ++i) ic[k1+i]=i2[i]; }
04172             for (i=0; i<m; ++i) ia[pa[k1+i]]=i2[k2+i];
04173
04174             for (K1.reset(); ++K1;) {
04175                 for (i=0; i<k1; ++i) { ia[pa[i]]=ic[i]=K1.data[i]; }
04176                 if (b== 1) { C(Ic)+=A(Ia); } else
04177                 if (b== -1) { C(Ic)-=A(Ia); } else { C(Ic)+=b*A(Ia); }
04178             }
04179         }}
04180
04181     C.Permute(pfinal); if (!X.isEmpty()) C+=X;
04182     return C;
04183 };
04184
04185 template <class TD>
04186 wbararray<TD>& contract(const char* F, int L,
04187     const wbsparray<TD> &A, const wbindex &ic1,
04188     const wbararray<TD> &B, const wbindex &ic2, wbararray<TD> &C,
04189     const wbpperm &pfinal, const TD& afac, const TD& cfac
04190 ){
04191     unsigned m=ic1.len, r1=A.rank(FL), r2=B.rank(); int e=0;
04192     wbararray<TD> X;
04193
04194     if ((void*)(&B)==(void*)(&C)) wblog(F,L,
04195         "ERR sparse::%s() must have distinct target space",FCT);
04196
04197     wvector<SPIDX_T> sC; if (cfac) sC=C.SIZE;
04198     if ((e=checkContract(F_L,A.SIZE,ic1,B.SIZE,ic2, &pfinal, &sC))) {
04199         if (e<0) { C.init(sC); if (sC.len) return C; }
04200         else wblog(FL,"ERR %s()",FCT);
04201     }
04202
04203     if (afac==0 || !A.D.len) {
04204         if (cfac!=0) { C*=cfac; } else { C.init(sC.Permute(pfinal)); }
04205         return C;
04206     }
04207
04208     if (cfac!=0) { C*=cfac; C.save2(X); }
04209     C.init(sC);
04210
04211     wvector<SPIDX_T> s2,D1;
04212     wMatrix<SPIDX_T> I1;
04213     wbpperm Pb,P1;
04214
04215     A.IDX.cols2End(ic1,I1).groupRecs(P1,D1,r1-m);
04216     B.SIZE.getI(ic2,s2); Pb.init2End(ic2,r2);
04217
04218     wbindex Ib(r2),Ic(sC.len);
04219     SPIDX_T i, ig,d,k,l, k1=r1-ic1.len, k2=r2-ic2.len, *i1,
04220     *ib=Ib.data, *ic=Ic.data, *pb=Pb.data;
04221     wbindex K2(s2);
04222
04223     for (l=ig=0; ig<D1.len; ++ig) { d=D1[ig];
04224     for (k=0; k<d; ++k,++l) {
04225         TD a=A.D.data[P1[l]]; if (a==0) continue; if (afac!=1) a*=afac;
04226
04227         i1=I1.ref(l); if (k==0) {
04228             for (i=0; i<k1; ++i) ic[i]=i1[i]; }
04229             for (i=0; i<m; ++i) ib[pb[k2+i]]=i1[k1+i];
04230
04231             for (K2.reset(); ++K2;) {
04232                 for (i=0; i<k2; ++i) { ib[pb[i]]=ic[k1+i]=K2.data[i]; }
04233                 if (a== 1) { C(Ic)+=B(Ib); } else
04234                 if (a== -1) { C(Ic)-=B(Ib); } else { C(Ic)+=a*B(Ib); }
04235             }
04236         }}
04237
04238     C.Permute(pfinal); if (!X.isEmpty()) C+=X;
04239     return C;
04240 };
04241
04242 template <class TD>
04243 TD Wb::VMatVprod(
04244     const wbsparray<TD> &v1, const wbsparray<TD> &M, const wbsparray<TD> &v2
04245 ){
04246     wbsparray<TD> x,X; wbindex i1(1), i2(1);
04247     i1[0]=0; i2[0]=1;
04248

```

```

04249     M.contract (FL,i2,v2,i1,X);
04250     v1.contract (FL,i1,X,i1,x);
04251
04252     x.checkSize(FL);
04253     if (x.D.len && !x.isScalar()) {
04254         MXPut (FL,"q").add(M,"M").add(v1,"v1").add(v2,"v2")
04255             .add(X,"X").add(x,"x");
04256         wblog(FL,"ERR %s() got %s",FCT,SSTR(x));
04257     }
04258
04259     return (x.D.len ? x[0] : 0.);
04260 };
04261
04262 template <class TD>
04263 wbsparray<TD>& Wb::MatProd(
04264     const wbsparray<TD> &A, const wbsparray<TD> &B, wbsparray<TD> &C,
04265     char aflag0, char bflag0, TD afac, TD cfac
04266 ){
04267     if (&C==&A || &C==&B) {
04268         wbsparray<TD> X; if (cfac) X=C;
04269         Wb::MatProd(A,B,X,aflag0,bflag0,afac,cfac);
04270         return X.save2(C);
04271     }
04272
04273     int ra=A.rank(), rb=B.rank(), e=0;
04274     if (ra!=2) { if (ra || !A.isEmpty()) e+=1; }
04275     if (rb!=2) { if (rb || !B.isEmpty()) e+=2; }
04276
04277     if (e) wblog(FL,
04278         "ERR %s() expects matrices (%s; %s)",FCT,SSTR(A),SSTR(B));
04279     if (!ra || !rb) {
04280         if (!C.isEmpty() && cfac!=1) C*=cfac;
04281         return C;
04282     }
04283
04284     opFlags<TD> aflag(aflag0), bflag(bflag0);
04285     const wbsparray<TD>
04286         &AX = aflag.applyConjOrRef(FL,A),
04287         &BX = bflag.applyConjOrRef(FL,B);
04288     ctrIdx ia(1),ib(1);
04289
04290     ia[0]=(aflag.trans() ? 0:1);
04291     ib[0]=(bflag.trans() ? 1:0);
04292
04293     AX.contract (FL,ia,BX,ib,C, wbperm(), afac, cfac);
04294
04295     if (&AX!=&A) { delete &AX; }
04296     if (&BX!=&B) { delete &BX; }; return C;
04297 };
04298
04299 template <class TD>
04300 wbsparray<TD>& wbsparray<TD>::comm(const char *F, int L,
04301     const wbsparray<TD> &B, wbsparray<TD> &C, char aflag, char bflag
04302 ) const {
04303
04304     if (&C==this || &C==&B) {
04305         wbsparray<TD> X; comm(F,L,B,X,aflag,bflag);
04306         return X.save2(C);
04307     }
04308
04309     SPIDX_T d=-1;
04310     if (!isSMatrix(FL,&d) || !B.isSMatrix(FL,&d)) wblog(F_L,
04311         "ERR %s() invalid operators for [A,B] (%s; %s)", FCT,
04312         SSTR_(this), SSTR(B)
04313     );
04314
04315     Wb::MatProd(*this,B,C,aflag,bflag);
04316     Wb::MatProd(B,*this,C,bflag,aflag,TD(-1.),TD(1.));
04317
04318     return C;
04319 };
04320
04321 #endif
04322

```

5.59 wbsparray.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbsparray (sparse array of arbitrary dimension)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *

```

```

00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SPARRAY_HH__
00021 #define __WB_SPARRAY_HH__
00022
00023 // #define WB_CLK_SPARSE
00024
00025 int sp_num_threads = 0;
00026
00027 bool mxIsWbsparray(
00028     const char *F, int L, const mxArray *a, unsigned k=0);
00029
00030 bool mxIsWbsparray(
00031     const char *F, int L, const mxArray *a, unsigned k,
00032     const mxArray **as, const mxArray **ad, const mxArray **ai
00033 );
00034
00035 bool mxIsWbsparray(const mxArray *a) { return mxIsWbsparray(0,0,a); }
00036
00037 #define wbsparray    wbsparray<double>
00038 #define wbsparrayTD  wbsparray<TD>
00039
00040 template<class TD>
00041 class wbsparray {
00042
00043 public:
00044
00045     wbsparray() : isref(0) {};
00046
00047     wbsparray(SPIDX_T d1) : isref(0) { if (d1) init(d1); };
00048     wbsparray(SPIDX_T d1, SPIDX_T d2) : isref(0) { init(d1,d2); };
00049     wbsparray(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) : isref(0) { init(d1,d2,d3); };
00050     wbsparray(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
00051         : isref(0) { init(d1,d2,d3,d4); };
00052
00053     wbsparray(const wvector<SPIDX_T> &d, SPIDX_T l=0)
00054         : isref(0) { init(d,l); };
00055
00056     wbsparray(char *sidx, SPIDX_T l=0)
00057         : isref(0) { init(Str2Idx(sidx),l); };
00058
00059     wbsparray(const wbsparray &a)
00060         : isref(0) { SIZE=a.SIZE; IDX=a.IDX; D=a.D; };
00061
00062     wbsparray(const char *F, int L, const mxArray *a)
00063         : isref(0) { init(F,L,a); };
00064
00065     wbsparray(const mxArray *a) : isref(0) { init(FL,a); };
00066
00067     wbsparray& init(
00068         const char *F, int L, const warray<TD> &A, TD eps=0
00069     );
00070
00071     template<class TA>
00072     warray<TA>& toFull(warray<TA> &A) const;
00073
00074     template<class TA>
00075     wvector<TA>& toFull(wvector<TA> &A) const;
00076
00077     template<class DB>
00078     wbsparray& init(const wbsparray<DB> &a) {
00079         if ((void*)this!=(void*)&a) {
00080             SIZE=a.SIZE; IDX=a.IDX; D=a.D;
00081             isref=0;
00082         }
00083         return *this;
00084     };
00085
00086     SPIDX_T catRecs(SPIDX_T i0, const wbsparray &a, SPIDX_T id);
00087
00088     wbsparray& init(const wvector<SPIDX_T> &S0, SPIDX_T l=0) {
00089         SIZE=S0; D.init(l);
00090         if (S0.len) IDX.init(l,S0.len);
00091         else {
00092             if (l>1) wblog(FL,
00093                 "WRN %s() use initDiag() with S=[] and nz=%d",FCT,l);

```

```

00094         IDX.init();
00095     }
00096     return *this;
00097 };
00098
00099 template<class I2>
00100 wbsparray& init(const wbvector<I2> &S_, SPIDX_T l=0) {
00101     wbvector<SPIDX_T> S0(S_);
00102     return init(S0,l);
00103 };
00104
00105 wbsparray& init() {
00106     SIZE.init(); IDX.init(); D.init(); isref=0;
00107     return *this;
00108 };
00109
00110 wbsparray& init(SPIDX_T d1) {
00111     if (d1) SIZE.init(1,&d1); else SIZE.init();
00112     IDX.init(); D.init(); isref=0;
00113     return *this;
00114 };
00115
00116 wbsparray& init(SPIDX_T d1, SPIDX_T d2) {
00117     SPIDX_T s[] = {d1,d2}; SIZE.init(2,s); IDX.init(); D.init();
00118     return *this;
00119 };
00120
00121 wbsparray& init(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) {
00122     SPIDX_T s[] = {d1,d2,d3}; SIZE.init(3,s); IDX.init(); D.init();
00123     return *this;
00124 };
00125
00126 wbsparray& init(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4) {
00127     SPIDX_T s[] = {d1,d2,d3,d4}; SIZE.init(4,s); IDX.init(); D.init();
00128     return *this;
00129 };
00130
00131 wbsparray& initz() {
00132     IDX.init(0,SIZE.len); D.init();
00133     return *this;
00134 };
00135
00136 wbsparray& initz(SPIDX_T d1, SPIDX_T n) {
00137     SIZE.init(1,&d1); IDX.init(n,SIZE.len); D.init(n);
00138     return *this;
00139 };
00140
00141 wbsparray& initz(SPIDX_T d1, SPIDX_T d2, SPIDX_T n) {
00142     SPIDX_T s[] = {d1,d2}; SIZE.init(2,s); IDX.init(n,SIZE.len); D.init(n);
00143     return *this;
00144 };
00145
00146 wbsparray& initz(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T n) {
00147     SPIDX_T s[] = {d1,d2,d3}; SIZE.init(3,s); IDX.init(n,SIZE.len); D.init(n);
00148     return *this;
00149 };
00150
00151 wbsparray& init_rzd(SPIDX_T r, SPIDX_T n) {
00152     SIZE.init(r); IDX.init(n,r); D.init(n); isref=0;
00153     return *this;
00154 };
00155
00156 wbsparray& init_nnz(SPIDX_T n=0, char kflag=0) {
00157     if (isref) {
00158         if (SIZE.isref) wblog(FL,"ERR %s() got isref "
00159             "%d,%d,%d,%d",FCT,isref,SIZE.isref,IDX.isref,D.isref);
00160         isref=0;
00161     }
00162     if (!n) {
00163         if (!SIZE.len) { IDX.init(); }
00164         else {
00165             if (kflag) { IDX.init(n,SIZE.len/2); }
00166             else { IDX.init(n,SIZE.len ); }
00167         }
00168         D.init(); return *this;
00169     }
00170
00171     if (!SIZE.len) wblog(FL,
00172         "ERR sparse::%s() got empty size SIZE!",FCT);
00173     if (kflag) {
00174         if (SIZE.len%2) wblog(FL,
00175             "ERR %s() even rank expected (%d)",FCT,SIZE.len);
00176         IDX.init(n,SIZE.len/2);
00177     }
00178     else { IDX.init(n,SIZE.len); }
00179
00180     D.init(n); return *this;

```

```

00181     };
00182
00183     wbsparray& setRand(double x=1, char sflag=0);
00184     wbsparray& setRand_data(double x=1, char sflag=0);
00185
00186     template<class DB>
00187     wbsparray& init(const char *F, int L,
00188         SPIDX_T d1, SPIDX_T d2, const wbsparray<DB> &B);
00189
00190     template<class DB>
00191     wbsparray& init(const char *F, int L,
00192         SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const wbsparray<DB> &B);
00193
00194     template<class DB>
00195     wbsparray& init(const char *F, int L,
00196         SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const wbsparray<DB> &B);
00197
00198     wbsparray& initEmpty(const wbsparray &a) {
00199         if (this!=&a) SIZE=a.SIZE;
00200         IDX.init(0,SIZE.len); D.init();
00201         return *this;
00202     };
00203
00204     wbsparray& init(
00205         const char *F, int L, const mxArray *a, unsigned k=0);
00206
00207     wbsparray& init2ref(const wbsparray &a) {
00208         SIZE=a.SIZE; IDX.init2ref(a.IDX); D.init2ref(a.D);
00209         isref=1; return *this;
00210     };
00211
00212     wbsparray& unRef() {
00213         if (!isref) return *this;
00214         IDX.unRef(); D.unRef(); isref=0;
00215         return *this;
00216     };
00217
00218     wbsparray& initCAT(const char *F, int L,
00219         wvector< wbsparray* > &X,
00220         wvector<SPIDX_T> *IO=NULL, char del=0
00221     );
00222
00223     wbsparray& initCAT(const char *F, int L,
00224         wvector< wbsparray > &X, const wvector<SPIDX_T> *IO=NULL
00225     ){
00226         wvector< wbsparray* > xp(X.len);
00227         for (unsigned i=0; i<X.len; ++i) xp[i]=(&X[i]);
00228         return initCAT(F,L,xp,IO);
00229     };
00230
00231     wbsparray& initCAT(
00232         const char *F, int L, const wvector< const wbsparray* > &X,
00233         unsigned dim
00234     );
00235
00236     wbsparray& initCAT(
00237         const char *F, int L, const wvector< wbsparray > &X,
00238         unsigned dim
00239     ){
00240         wvector< const wbsparray* > xp(X.len);
00241         for (unsigned i=0; i<X.len; ++i) xp[i]=(&X[i]);
00242         return initCAT(F,L,xp,dim);
00243     };
00244
00245     wbsparray& Cat(const char *F, int L,
00246         const wbsparray &B, unsigned dim
00247     );
00248
00249     wbsparray& init_kron(
00250         const wvector<SPIDX_T> &sa, const wvector<SPIDX_T> &sb,
00251         SPIDX_T nab, char kflag
00252     ){
00253         setSIZE_kron(FL,sa,sb);
00254         return init_nnz(nab,kflag);
00255     };
00256
00257     void setSIZE_kron(const char *F, int L,
00258         const wvector<SPIDX_T> &sa, const wvector<SPIDX_T> &sb,
00259         char kflag=0);
00260     void setSIZE_kron(const char *F, int L, char kflag=0);
00261
00262     wbsparray& setRec(SPIDX_T i, SPIDX_T j) {
00263         if (i>=D.len || j>=D.len || IDX.dim1!=D.len) wblog(FL,"ERR %s() "
00264             "index out of bounds (%d,%d/%d,%d",FCT,i,j,IDX.dim1,D.len);
00265         if (i!=j) { D[i]=D[j]; IDX.recSet(i,j); }
00266         return *this;
00267     };

```



```

00268
00269     template<class T2>
00270     wbsparray& setRec(SPIDX_T i, SPIDX_T i1, SPIDX_T i2, const T2& x) {
00271         if (IDX.dim2!=2) wblog(FL,"ERR %s() "
00272             "rank-2 tensor required (%d: %s)",FCT,SIZE.len,SSTR_(this));
00273         if (i>=D.len || IDX.dim1!=D.len) wblog(FL,"ERR %s() "
00274             "index out of bounds (%d/%d,%d)",FCT,i,IDX.dim1,D.len);
00275         SPIDX_T *I=IDX.rec(i);
00276         I[0]=i1; I[1]=i2; D[i]=x;
00277         return *this;
00278     };
00279
00280     wbsparray& setRecP(SPIDX_T i, const SPIDX_T *I, const TD &x) {
00281         if (i>=D.len && IDX.dim1!=D.len) wblog(FL,"ERR %s() "
00282             "index out of bounds (%d /%d,%d)",FCT,i,IDX.dim1,D.len);
00283         IDX.recSetP(i,I); D[i]=x;
00284         return *this;
00285     };
00286
00287     wbsparray& setRec_kron(
00288         SPIDX_T l, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2,
00289         unsigned kflag=0
00290     );
00291
00292     wbsparray& Resize(SPIDX_T l) {
00293         IDX.Resize(l,IDX.dim2); D.Resize(l);
00294         return *this;
00295     };
00296
00297     wbsparray& REDSIZE_NNZ(SPIDX_T l) {
00298         if (l) {
00299             if (IDX.dim1) {
00300                 if (l>IDX.dim1) wblog(FL,"ERR %s() "
00301                     "range out of bounds (IDX: %ld/%ld)",FCT,l,IDX.dim1);
00302                 IDX.dim1=l;
00303             }
00304             if (D.len) {
00305                 if (l>D.len) wblog(FL,"ERR %s() "
00306                     "range out of bounds (D: %ld/%ld)",FCT,l,D.len);
00307                 D.len=l;
00308             }
00309         }
00310         else { IDX.init(0,SIZE.len); D.init(); }
00311         return *this;
00312     };
00313
00314     wbsparray& Resize(const wbvector<SPIDX_T> &S);
00315
00316     wbsparray& operator= (const wbsparray &a) {
00317         if ((void*)this!=(void*)&a) init(a);
00318         return *this;
00319     };
00320     template<class DB>
00321     wbsparray& operator= (const wbsparray<DB> &a) {
00322         if ((void*)this!=(void*)&a) init(a);
00323         return *this;
00324     };
00325
00326     wbsparray& save2(wbsparray &a, char ref=0) {
00327         if (this!=&a) {
00328             SIZE.save2(a.SIZE); IDX.save2(a.IDX,ref); D.save2(a.D);
00329             a.isref=isref; isref=0;
00330         }
00331         return a;
00332     };
00333
00334     SPIDX_T sub2ind(SPIDX_T k) const {
00335         return Wb::sub2ind(SIZE.data,IDX.rec(k),SIZE.len);
00336     };
00337
00338     void ind2sub(SPIDX_T k, SPIDX_T *I) const {
00339         Wb::ind2sub(k, SIZE.data, I, SIZE.len);
00340     };
00341
00342     void ind2sub(SPIDX_T k, wbindex &I) const {
00343         if (I.len!=SIZE.len) I.init(SIZE.len);
00344         Wb::ind2sub(k, SIZE.data, I.data, SIZE.len);
00345     };
00346
00347     wbsparray<TD>& make2D(const wbindex &ic, wbsparray<TD> &B,
00348         char pos=1
00349     ) const;
00350
00351     sparseIndex2D<SPIDX_T>& toIndex2D(
00352         const wbindex &ic, sparseIndex2D<SPIDX_T> &a,
00353         char pos,
00354         char use_rmaj=0

```

```

00355     ) const;
00356
00357     wbsparray& initDiag(SPIDX_T d) {
00358         D.init(d); SIZE.init(); IDX.init();
00359         return *this;
00360     };
00361
00362     wbsparray& initDiag(const wbvector<TD> &dd) {
00363         D.init(dd); SIZE.init(); IDX.init();
00364         return *this;
00365     };
00366
00367     wbsparray& initDiag2(SPIDX_T d, TD dval=1) {
00368         initz(d,d, d); D.set(dval); {
00369             SPIDX_T i=0, l=d-1, *idx=IDX.data;
00370             for (; i<d; ++i, idx+=2) { idx[0]=l-i; idx[1]=i; }
00371         }
00372         return *this;
00373     };
00374
00375     bool isDiagMatrix(TD eps=1E-14) const;
00376
00377     bool isDiag(const char *F=0, int L=0) const {
00378         if (SIZE.len) { checkSize(F_LF); }
00379         return (!IDX.data && D.len ? 1 : 0);
00380     };
00381
00382     wbvector<TD>& getDiag(const char *F, int L, wbvector<TD> &dd) const;
00383
00384     TD getDiag(const char *F, int L, SPIDX_T k) const;
00385
00386     wbsparray& getDiag(const char *F, int L, wbsparray &A) const {
00387         A.init(); getDiag(F,L,A.D);
00388         return A;
00389     };
00390
00391     wbsparray& diag2reg(const char *F=0, int L=0, unsigned r=-1);
00392     wbsparray& diag2reg(
00393         const char *F, int L, wbsparray &A, unsigned r=-1) const;
00394
00395     wbsparray& toDiag(const char *F=0, int L=0) {
00396         wbsparray X; getDiag(F_L,X);
00397         return X.save2(*this);
00398     };
00399
00400     int toDiagSafe(const char *F, int L, TD eps=1E-14) {
00401         if (!isDiagMatrix(eps)) { if (F) {
00402             wblog(F_L, "ERR %s() got non-diagonal matrix",FCT); }
00403             return 0;
00404         }
00405         wbsparray X; getDiag(F_L,X);
00406         X.save2(*this); return 1;
00407     };
00408
00409     wbsparray& initIdentity(SPIDX_T d, TD dval=1){
00410         initDiag(d); D.set(dval);
00411         return *this;
00412     };
00413
00414     wbsparray& initIdentity(SPIDX_T d1, SPIDX_T d2, sSPIDX_T m=0, TD dval=1){
00415         if (d1==d2 && !m) { return initIdentity(d1,dval); }
00416
00417         sSPIDX_T k1=0, k2=0, i1=0, i2=d1, j1=0, j2=d2;
00418         SPIDX_T l, s[] = {d1,d2};
00419         if (m>=0) { j1=-m; j2=-m; } else { i1=m; i2+=m; }
00420
00421         k2=MIN(i2,j2); l=(k2>k1 ? k2-k1 : 0);
00422         SIZE.init(2,s); IDX.init(l,2); D.init2val(l,dval);
00423
00424         if (l) { SPIDX_T *idx=IDX.data;
00425             for (SPIDX_T i=0; i<l; ++i, ++k1, idx+=2) {
00426                 idx[0]=k1-i1;
00427                 idx[1]=k1-j1;
00428             }
00429         }
00430
00431         return *this;
00432     };
00433
00434     wbsparray& initIdentity(
00435         const wbvector<SPIDX_T> &S0, int m=-1, sSPIDX_T k=0, char fflag='f');
00436     wbsparray& initIdentityB3(
00437         SPIDX_T d1, SPIDX_T d2, SPIDX_T D,
00438         sSPIDX_T i0=0, char fflag='f'
00439     );
00440
00441     unsigned rank(const char *F=NULL, int L=0) const {

```

```

00442     checkSize(F_LF);
00443     if (!SIZE.len) {
00444         return (isDiag() ? 2 : 0);
00445     }
00446     return SIZE.len;
00447 };
00448
00449 SPIDX_T dim() const {
00450     if (isDiag()) return D.len;
00451     if (SIZE.len!=2 || SIZE[0]!=SIZE[1]) wblog(FL,
00452         "ERR %s() got %s array",FCT,SSTR_(this));
00453     return SIZE[0];
00454 };
00455
00456 SPIDX_T dim(unsigned k) const {
00457     if (!k) wblog(FL,
00458         "ERR %s() input dim not 1-based (%d)",FCT,k);
00459     return dim0(--k);
00460 };
00461
00462 SPIDX_T dim0(unsigned k) const {
00463     if (k<2 && isDiag()) { return D.len; }
00464     if (k<SIZE.len) { return SIZE[k]; }
00465     else wblog(FL,"ERR %s() "
00466         "index out of bounds (%d having %s)",FCT,k+1,SSTR_(this));
00467     return 0;
00468 };
00469
00470 SPIDX_T matDim(unsigned k=1) {
00471     if (!k || k>2) wblog(FL,
00472         "ERR %s() index out of bounds (%d/%d)",FCT,k);
00473     if (isDiag(FL)) { return D.len; } else
00474     if (k==1) { return Wb::prodRange(SIZE.data,SIZE.len/2); }
00475     else {
00476         unsigned l=SIZE.len/2;
00477         return Wb::prodRange(SIZE.data+l,SIZE.len-l);
00478     }
00479 };
00480
00481 SPIDX_T maxDim(unsigned r=-1) const {
00482     SPIDX_T d=0;
00483
00484     if (int(r)<0) {
00485         if (SIZE.len ) { d=SIZE.max(); } else
00486         if (isDiag(FL)) { d=D.len; }
00487     }
00488     else if (SIZE.len) {
00489         if (!r || r>SIZE.len) wblog(FL,
00490             "ERR %s() invalid dim range (%d/%d)",FCT,r,SIZE.len);
00491         d=SIZE[0];
00492         for (unsigned i=1; i<r; ++i) { if (d<SIZE[i]) d=SIZE[i]; }
00493     }
00494     else if (isDiag(FL)) {
00495         if (!r || r>2) wblog(FL,
00496             "ERR %s() invalid dim range (%d/2)",FCT,r);
00497         d=D.len;
00498     }
00499     else if (r) {
00500         wblog(FL,"ERR %s() invalid dim range (%d/0)",FCT,r);
00501     }
00502
00503     return d;
00504 };
00505
00506 TD norm2(char tnorm=0) const {
00507     return Wb::overlap(D.data,D.data,D.len,1,tnorm);
00508 };
00509
00510 wstring sizeStr() const {
00511     wstring s;
00512     if (SIZE.len) { s=SIZE.toStrf("", "x"); }
00513     else if (D.len) {
00514         s.init(16); s.pushf(FL, "[%dx%d]",D.len,D.len);
00515     }
00516     else { s="[]"; }
00517     return s;
00518 };
00519
00520 TD SkipTinyCols(TD eps=1E-14) {
00521     unsigned r=rank(); if (!r && isEmpty()) return 0;
00522     SPIDX_T l=0, i=0; TD e2=0;
00523     wvector<TD> a2; norm2vec(r-1,a2);
00524     WBINDEX P(a2.len);
00525
00526     for (; i<a2.len; ++i) {
00527         if (a2[i]>eps) { P[l++]=i; } else { e2+=a2[i]; }
00528     }

```

```

00529         if (l) P.len=1; else P.init();
00530         Select0(P,r-1);
00531
00532         return e2;
00533     };
00534
00535     TD SkipTiny(TD eps=1E-14) {
00536         TD e2=0; SPIDX_T l=0, i=0;
00537         if (eps) {
00538             Wb::scale_eps(eps,D.data,D.len);
00539             for (i=0; i<D.len; ++i) {
00540                 if (ABS(D[i])>eps) { if (l<i) { setRec(l,i); }; ++l; }
00541                 else { e2+=NORM2(D[i]); }
00542             }
00543         }
00544         else {
00545             for (i=0; i<D.len; ++i) {
00546                 if (D[i]!=eps) { if (l<i) { setRec(l,i); }; ++l; }
00547             }
00548         }
00549
00550         if (l<i) { IDX.ResizeXRecs(l); D.Shorten2(l); }
00551         return e2;
00552     };
00553
00554     bool isVector(const char *F=0, int L=0) const {
00555         unsigned r=rank(F_L); if (r==1) return 1;
00556         return (SIZE.count(1)>=(r-1));
00557     };
00558
00559     bool isMatrix(const char *F=0, int L=0) const { return rank(F_L)==2; };
00560     bool isRank(unsigned r) const { return (rank(FL)==r); };
00561
00562     bool isSMatrix(const char *F=0, int L=0, SPIDX_T *d=NULL) const {
00563         unsigned r=rank(F_L);
00564         if (r!=2 || (SIZE.len && SIZE[0]!=SIZE[1])) return 0;
00565         if (d) {
00566             SPIDX_T d0=(SIZE.len ? SIZE[0] : D.len);
00567             if (sSPIDX_T(*d)>=0) { if ((*d)!=d0) return 0; }
00568             else (*d)=d0;
00569         }
00570         return 1;
00571     };
00572
00573     bool isSMatrix(const char *F, int L, SPIDX_T d) const {
00574         unsigned r=rank(F_L); SPIDX_T d0=(SIZE.len ? SIZE[0] : D.len);
00575         if (r!=2 || (SIZE.len && SIZE[0]!=SIZE[1]) || d!=d0) return 0;
00576         return 1;
00577     };
00578
00579     bool isEmpty() const {
00580         return (SIZE.len || IDX.dim1 || D.len ? 0 : 1);
00581     };
00582
00583     bool isEmpty_() const {
00584         if (SIZE.len || IDX.dim1 || D.len) {
00585             if (IDX.dim1!=D.len || IDX.dim2!=SIZE.len) wblog(FL,
00586                 "ERR %s() size mismatch (%dx%d / %s)",
00587                 FCT,D.len,SIZE.len,SSTR(IDX));
00588             return 0;
00589         }
00590         return 1;
00591     };
00592
00593     explicit operator bool() const { return !isEmpty_(); }
00594     bool operator! () const { return isEmpty_(); }
00595
00596     bool isZero(TD eps=0, char flag=0) const;
00597
00598     SPIDX_T nnz() const {
00599         if (isScalar(FL))
00600             return (D.len==1 && D.data[0]!=0 ? 1 : 0);
00601         else return D.len;
00602     };
00603
00604     SPIDX_T nnz(const TD eps1, const TD eps2=1E9) const {
00605         SPIDX_T i=0, n=0; TD a,e=0;
00606         for (i=0; i<D.len; ++i) {
00607             a=ABS(D[i]); if (a>eps1) ++n;
00608             else if (a>eps2 && e<a) e=a;
00609         }
00610         if (e) wblog(FL,
00611             "WRN %s() got small value(s) %.3g (%g;%g)",FCT,
00612             double(e), double(eps1), double(eps2));
00613         return n;
00614     };
00615

```

```

00616     SPIDX_T nnz_diag() const {
00617         if (isDiag(FL)) return D.nnz();
00618         if (IDX.dim2!=2) wblog(FL,
00619             "ERR %s() requires rank-2 tensor (S=%s)",FCT,SSTR_(this));
00620         SPIDX_T i=0, n=0; const SPIDX_T *I=IDX.data;
00621         for (; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1] && D[i]!=0) ++n; }
00622         return n;
00623     };
00624
00625     SPIDX_T numel() const {
00626         return (isDiag(FL) ? D.len*D.len : SIZE.prod(0));
00627     };
00628
00629     TD norm(char tnorm=0) const {
00630         return Sqrt(Wb::overlap(D.data,D.data,D.len,1,tnorm));
00631     };
00632
00633     template<class T2>
00634     wbvector<T2>& norm2vec(unsigned k,
00635         wbvector<T2> &A, char tnorm=0
00636     ) const;
00637
00638     void swap(wbsparray &B) {
00639         if (this!=&B) {
00640             if (isref || B.isref) wblog(FL,
00641                 "ERR %s() got arrays with isref=(%d,%d)",FCT,isref,B.isref);
00642             SIZE.swap(B.SIZE);
00643             IDX.swap(B.IDX);
00644             D.swap(B.D);
00645         }
00646     };
00647
00648     wbsparray& TimesEl(const wbsparray &B, char tnorm=0);
00649
00650     TD dotProd(
00651         const char *F, int L, const wbsparray &B, char tnorm=0) const;
00652
00653     TD froNorm2(const wbsparray &B) const { return dotProd(0,0,B); };
00654
00655     double normDiff2(const wbsparray &B) const {
00656         wbsparray X(*this); X-=B; return (double)X.norm2();
00657     };
00658     double normDiff (const wbsparray &B) const {
00659         wbsparray X(*this); X-=B; return (double)X.norm();
00660     };
00661
00662     TD Normalize(char tnorm=0, char qflag=0);
00663     TD NormalizeCol(SPIDX_T k, char tnorm=0, char qflag=0);
00664
00665     wbsparray& QRdecomp(const char *F, int L,
00666         wbsparray &Q,
00667         char posR=0,
00668         TD eps=1E-14
00669     );
00670
00671     wbsparray& OrthoNormalizeColsQR(
00672         const char *F=NULL, int L=0,
00673         TD eps=1E-14
00674     );
00675
00676     wbsparray& OrthoNormalizeColsGS(
00677         const char *F=NULL, int L=0, char tnorm=0,
00678         char qxflag=0, TD eps=1E-14, unsigned np=1
00679     );
00680
00681     wbsparray& setCol(
00682         const char *F, int L, SPIDX_T k, const wbsparray &v);
00683
00684     wbsparray& setCol(SPIDX_T k, const wbsparray &v) {
00685         return setCol(0,0,k,v);
00686     };
00687
00688     double sparsity() const {
00689         if (!D.len) return 0;
00690         if (isScalar(FL)) return (D.len==1 && D.data[0]!=0 ? 1 : 0);
00691         return D.len/double(numel());
00692     };
00693
00694     int checkTrailingSingletons(const char *F=0, int L=0, unsigned *r=NULL) const;
00695
00696     int SkipTrailingSingletons(const char *F=0, int L=0, unsigned r=-1);
00697     int skipTrailingSingletons(
00698         const char *F, int L, wbsparray &c, unsigned r=-1) const;
00699
00700     bool hasSingletons(const wbindex &I) const;
00701
00702     wbsparray& skipSingletons(

```

```

00703     const char *F, int L, const wbindex &I, wbsparray &A
00704 ) const;
00705
00706 wbsparray& AddTrailingSingletons(
00707     const char *F, int L, unsigned r);
00708
00709 wbsparray& initScalar(const TD x=0, unsigned r=-1) {
00710     if (int(r)<=0) {
00711         if (D.len!=1) { D.init(1); }; D[0]=x;
00712         if (SIZE.len) { SIZE.init(); }
00713         IDX.init();
00714     }
00715     else {
00716         SIZE.init(r).set(1);
00717         if (x) {
00718             if (D.len!=1) { D.init(1); }; D[0]=x;
00719             IDX.init(1,r);
00720         }
00721         else { IDX.init(0,r); D.init(); }
00722     }
00723     isref=0; return *this;
00724 };
00725
00726 char isScalar(const char *F=0, int L=0, char zflag=0) const {
00727     if (!SIZE.len) {
00728         if (IDX.dim1 || IDX.dim2) sperror_this(F,L);
00729         return (zflag ? D.len<=1 : D.len==1);
00730     }
00731     return SIZE.allEqual(1);
00732 };
00733
00734 wbsparray& toScalar(const char *F=0, int L=0, unsigned r=-1);
00735
00736 TD getScalar(const char *F=0, int L=0) const;
00737
00738 char checkSize(
00739     const char *F=0, int L=0, const char *fct=0, const char *istr=0
00740 ) const;
00741
00742 char checkSize2(const char *F, int L, const wbsparray &B) const;
00743
00744 void check_IDX_range(const char *F=0, int L=0) const;
00745
00746 bool isProptoId() const { TD q=0; return isProptoId(q); }
00747 bool isProptoId(TD &q,TD eps=1E-14) const;
00748
00749 bool isIdentityMatrix(TD eps=1E-14) const {
00750     TD q=1; return isProptoId(q,eps);
00751 };
00752
00753 char isIdentity(TD eps=1E-14) const;
00754 char isIdentity(const wbpperm &P, TD *dval=NULL, TD eps=1E-14) const;
00755
00756 bool hasSize(SPIDX_T d1) const;
00757 bool hasSize(SPIDX_T d1, SPIDX_T d2) const;
00758 bool hasSize(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const;
00759
00760 template<class T>
00761 wbvector<T>& getSize(wbvector<T> &S) const {
00762     if (SIZE.len) { return S.initT(SIZE); }
00763     if (D.len && isDiag()) { return S.init(2).set(D.len); }
00764     return S.init();
00765 };
00766
00767 char fitsSize(const wbsparray &, char strict=0) const;
00768
00769 char sameSize(const wbsparray &, char strict=0) const;
00770 char sameSizeUp2Singletons(
00771     const wbvector<SPIDX_T> &S, char strict=0) const;
00772
00773 char matchWithSingletons(const char *F, int L,
00774     const wbvector<SPIDX_T> &S, wbvector<int> &M
00775 ) const;
00776
00777 char sameUptoFac(
00778     const wbsparray &B, TD *fac=NULL, char lex=-1,
00779     TD eps=1E-12) const;
00780
00781 bool operator== (const wbsparray &B) const {
00782     if (this==&B) return 1;
00783     if (SIZE!=B.SIZE) return 0;
00784     if (IDX==B.IDX && D==B.D) return 1;
00785     return (normDiff2(B)<1E-20);
00786 };
00787
00788 bool operator!= (const wbsparray &B) const {
00789     return !(*this==B);

```

```

00790     };
00791
00792     wbsparray& Plus(const char *F, int L,
00793         const wbsparray &B, TD bfac=1, TD afac=1, TD eps=0
00794     );
00795
00796     TD sum() const { return D.sum(); };
00797     TD aMin() const { return D.aMin(1); };
00798     TD aMax() const { return D.aMax(); };
00799
00800     SPIDX_T findRecSortedP(SPIDX_T *idx, SPIDX_T n,
00801         char lex=-1
00802     );
00803
00804     wbsparray& Eldiv(const char *F, int L, const wbvector<TD> &x_) {
00805         SPIDX_T i=0, j;
00806         const SPIDX_T *idx=IDX.data;
00807         const TD *x=x_.data;
00808
00809         if (!isVector()) wblog(F_L,
00810             "ERR %s() invalid vector (%s)",FCT,SSTR_(this));
00811         for (; i<SIZE.len; ++i) { if (SIZE[i]>1) { idx+=i; break; } }
00812
00813         for (i=0; i<IDX.dim1; ++i, idx+=IDX.dim2) { j=(*idx);
00814             if (j>=x_.len) wblog(F_L,
00815                 "ERR %s() index out of bounds (%d/%d)",FCT,j+1,x_.len);
00816             if (x[j]==0) wblog(F_L,
00817                 "ERR %s() div/0 !? (%g)",FCT,double(x[j]));
00818             D.data[i]/=x[j];
00819         }
00820         return *this;
00821     };
00822
00823     wbsparray& operator+=(const wbsparray &B) { return Plus(F_L,B,+1); }
00824     wbsparray& operator-=(const wbsparray &B) { return Plus(F_L,B,-1); }
00825
00826     wbsparray& Conj() { D.Conj(); return *this; }
00827
00828     wbsparray& operator*=(TD x) {
00829         if (isref) {
00830             D*=x;
00831         }
00832         else {
00833             if (x!=0) { if (x!=1) D*=x; }
00834             else init_nnz();
00835         }
00836         return *this;
00837     };
00838
00839     wbsparray& operator/=(TD x) {
00840         if (x==0) wblog(F_L,"ERR %s() div/0",FCT);
00841         if (x!=1) D/=x;
00842         return *this;
00843     };
00844
00845     const TD& operator[](SPIDX_T i) const {
00846         if (i>=D.len) {
00847             wblog(F_L,"ERR %s() index out of bounds (%d/%d; %s)",
00848                 FCT,i,D.len,SSTR_(this));
00849         }
00850         return D.data[i];
00851     };
00852
00853     TD& operator[](SPIDX_T i) {
00854         if (i>=D.len) {
00855             if (i==0 && IDX.dim1==0) {
00856                 if (SIZE.len && SIZE.allEqual(1)) {
00857                     IDX.init(1,SIZE.len);
00858                     D.init(1); return D.data[i];
00859                 }
00860                 else if (!SIZE.len && IDX.dim2==0) {
00861                     IDX.init();
00862                     D.init(1); return D.data[i];
00863                 }
00864             }
00865             wblog(F_L,"ERR %s() index out of bounds (%d/%d; %s)",
00866                 FCT,i,D.len,SSTR_(this));
00867         }
00868         return D.data[i];
00869     };
00870
00871     TD value(const SPIDX_T *idx, SPIDX_T n, char lex=-1
00872     ) const {
00873         SPIDX_T i=IDX.findRecSorted(idx,n,lex);
00874         return (sSPIDX_T(i)<0 ? TD(0.) : D.data[i]);
00875     };
00876

```

```

00877     wstring info2Str(const char *istr=0) const {
00878         wstring s(32);
00879         if (istr) {
00880             s.pushf(FL,"%s(%s; %dx%d; %d)",istr,
00881                 SSTR_(this), IDX.dim1,IDX.dim2,D.len);
00882         }
00883         else {
00884             s.pushf(FL,"%s; %dx%d; %d",
00885                 SSTR_(this), IDX.dim1,IDX.dim2,D.len);
00886         }
00887         return s;
00888     };
00889
00890     size_t memSize() const {
00891         size_t m=sizeof(TD);
00892 #ifdef QS_USING_MPFR
00893         if (typeid(TD)==typeid(Wb::quad)) {
00894             m+=16;
00895         }
00896 #endif
00897         return ( sizeof(isref)
00898             + sizeof(SIZE) + SIZE.len*sizeof(SPIDX_T)
00899             + sizeof(IDX ) + IDX.numel()*sizeof(SPIDX_T)
00900             + sizeof(D ) + D.len*m
00901         );
00902     };
00903
00904     void info(const char* istr="") const { return info(0,0,istr); };
00905     void info(const char* F, int L, const char* "=") const;
00906     void print(const char* istr="", const char *fmt="") const;
00907
00908     void print(
00909         const char *F, int L,
00910         const char *istr="", const char *fmt=""
00911     ) const;
00912
00913     void printdata(const char *istr="ans", const char *fmt="") const;
00914
00915     void sperror_this(
00916         const char *F, int L, const char *fct=0, const char *istr=0
00917     ) const {
00918         wblog(F_L,"ERR %s() %s S=%s (%dx%d; %d) %s",
00919             fct ? fct : "sparse", istr ? istr : "got",
00920             SSTR_(this), IDX.dim1,IDX.dim2,D.len,
00921             isref ? " **REF** ":"");
00922     };
00923 };
00924
00925     TD trace() const;
00926
00927     template<class T2>
00928     wvector<T2>& trace(unsigned k, wvector<T2> &A) const;
00929
00930     bool isSym_aux(
00931         const char *file, int line, const char *fct,
00932         const wbsparray &B, TD eps, TD* xref,
00933         const char symflag='s',
00934         const char lflag=1
00935     ) const;
00936
00937     bool isHConj(
00938         const wbsparray &B, TD eps=1E-12, TD* xref=NULL
00939     ) const { return isSym_aux(FLF, B, eps, xref, 's'); };
00940
00941     bool isHConj(
00942         TD eps=1E-12, TD* xref=NULL
00943     ) const { return isSym_aux(FLF,*this,eps,xref,'s'); };
00944
00945     bool isAHerm(
00946         const wbsparray &B, TD eps=1E-12, TD* xref=NULL
00947     ) const { return isSym_aux(FLF, B, eps, xref, 'a'); };
00948
00949     bool isAHerm(
00950         TD eps=1E-12, TD* xref=NULL
00951     ) const { return isSym_aux(FLF,*this,eps,xref,'a'); };
00952
00953     wbsparray& Sort(char lex=-1) {
00954         if (IDX.data) {
00955             if (IDX.dim1!=D.len) wblog(FL,
00956                 "ERR %s() %d/%d !?",FCT,IDX.dim1,D.len);
00957             wbperm P; IDX.SortRecs(P,+1,lex); D.Select(P);
00958         }
00959         return *this;
00960     };
00961
00962     double Compress(
00963         const char *F=0, int L=0, TD eps=0,

```



```

00964     char lex=-1,
00965     const wbpem *P=NULL
00966 );
00967
00968 bool isCompressed(
00969     char lex=-1, char dir=0
00970 ) const { return IDX.isUniqueSorted(dir,lex); };
00971
00972 int splitSparseCM(
00973     const char *F, int L, wvector< wbsparray > &X,
00974     TD eps=0, char ref=0, char tnorm=0);
00975
00976 wbsparray& reshape(
00977     const wvector<SPIDX_T> &S, wbsparray &A) const;
00978
00979 wbsparray& Reshape(const wvector<SPIDX_T> &S){
00980     wbsparray X; reshape(S,X); return X.save2(*this);
00981 };
00982
00983 wbsparray& Reshape(SPIDX_T s1, SPIDX_T s2){
00984     wbsparray X; wvector<SPIDX_T> S(2); S[0]=s1; S[1]=s2;
00985     reshape(S,X); return X.save2(*this);
00986 };
00987
00988 wbsparray& Reshape(SPIDX_T s1, SPIDX_T s2, SPIDX_T s3){
00989     wbsparray X; wvector<SPIDX_T> S(3); S[0]=s1; S[1]=s2; S[2]=s3;
00990     reshape(S,X); return X.save2(*this);
00991 };
00992
00993 wvector<TD>& getCol(SPIDX_T k, wvector<TD> &v) const;
00994 wvector<TD>& getRow(SPIDX_T k, wvector<TD> &v) const;
00995
00996 wbsparray& getBlock(const char *F, int L,
00997     SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2,
00998     wbsparray<TD> &B
00999 ) const;
01000
01001 wbsparray& getCol(SPIDX_T k, wbsparray<TD> &b) {
01002     if (k>=dim0(k)) wblog(FL,"ERR %s() "
01003         "col-index out of bounds (%ld / %s)",FCT,k,SSTR_(this));
01004     return getBlock(FL,0,-1,k,k);
01005 };
01006
01007 wbsparray& splitBlock(const char *F, int L,
01008     sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2,
01009     wbsparray<TD> &B
01010 );
01011
01012 wbsparray& SplitQR(sSPIDX_T k, sSPIDX_T k_, wbsparray<TD> &X) {
01013     save2(X); X.splitBlock(FL,k,-1,k_,-1,*this);
01014     return *this;
01015 };
01016
01017 TD getHouseholderVec(const char *F, int L,
01018     SPIDX_T k, SPIDX_T j, wbsparray<TD> &b,
01019     TD eps2=0
01020 ) const;
01021
01022 wbsparray& IDX_Shift(sSPIDX_T i1, sSPIDX_T i2);
01023
01024 wbsparray<TD>& LProject1(
01025     const wbsparray<TD> &b, wbsparray<TD> &X,
01026     TD xfac=1, char tnorm=0) const;
01027
01028 SPIDX_T LContractVec_omp(
01029     const wbsparray<TD> &b, wvector<TD> &x, char tnorm) const;
01030
01031 wbsparray& splitLast(SPIDX_T k, wbsparray &a) const;
01032
01033 wbsparray& select0(
01034     const WBINDEX &,
01035     unsigned dim,
01036     wbsparray&
01037 ) const;
01038
01039 wbsparray& Select0(const WBINDEX &I, unsigned dim) {
01040     wbsparray X; save2(X); X.select0(I,dim,*this);
01041     return *this;
01042 };
01043
01044 wbsparray& Permute(const char* s, char iflag=0);
01045 wbsparray& Permute(const wbpem&, char iflag=0);
01046 wbsparray& permute(const char* s, wbsparray&, char iflag=0) const;
01047 wbsparray& permute(const wbpem&, wbsparray&, char iflag=0) const;
01048
01049 wbsparray& transpose(const char *F, int L, wbsparray &A) const;
01050 wbsparray& Transpose(const char *F=0, int L=0) {

```

```

01051         wbsparray X; transpose(F,L,X); X.save2(*this);
01052         return *this;
01053     };
01054
01055     wbsparray& htranspose(const char *F, int L, wbsparray &A) const {
01056         transpose(F,L,A); A.D.Conj();
01057         return A;
01058     };
01059     wbsparray& hTranspose(const char *F=0, int L=0) {
01060         wbsparray X; htranspose(F,L,X); X.save2(*this);
01061         return *this;
01062     };
01063
01064     wbsparray& ColPermute(const wbpperm &P, char iflag=0);
01065     wbsparray& MatPermute(const wbpperm &P, char iflag=0);
01066
01067     wbsparray& mtimes(
01068         const char *F, int L, const wbsparray&B, wbsparray&C,
01069         char sflag=0, const TD eps=0
01070     ){
01071         const wbsparray &A=*this;
01072         if (A.rank()!=2 || B.rank()!=2) wblog(FL,
01073             "ERR %s() requires rank-2 wbsparrays (%s * %s)",
01074             FCT, SSTR(A), SSTR(B));
01075         wbindx ia(1), ib(1); ia[0]=1; ib[0]=0;
01076
01077         return contract(F,L,ia,B,ib,C,wbpperm(),1,0,sflag,eps);
01078     };
01079
01080     char contract_scalar(
01081         const char* F, int L, const ctrIdx &ica,
01082         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01083         const wbpperm &pfinal=wbpperm(), const TD& afac=1., const TD& cfac=0.
01084     ) const;
01085
01086     wbsparray& contract_diag_diag(
01087         const char* F, int L, const ctrIdx &ica,
01088         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01089         const wbpperm &pfinal, const TD& afac, const TD& cfac
01090     ) const;
01091
01092     char contract_check_2full(
01093         const char* F, int L, const ctrIdx &ica,
01094         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01095         const wbpperm &pfinal, const TD& afac, const TD& cfac,
01096         char flag, TD eps, SPIDX_T *nnz_C=NULL
01097     ) const;
01098
01099     wbsparray& contract(const char *F, int L, const ctrIdx &ica,
01100         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01101         const wbpperm &pfinal=wbpperm(), const TD& afac=1., const TD& cfac=0.,
01102         char sflag=0, const TD eps=0
01103     ) const;
01104
01105     wbsparray& contract(const char *F, int L, const char *ica,
01106         const wbsparray &B, const char *icb, wbsparray &C,
01107         const wbpperm &pfinal=wbpperm(), const TD& afac=1., const TD& cfac=0.,
01108         char sflag=0, const TD eps=0
01109     ) const {
01110
01111         return contract(F,L,
01112             ctrIdx(FL,ica), B, ctrIdx(FL,icb), C,
01113             pfinal,afac,cfac,sflag,eps
01114         );
01115     };
01116
01117     wbarray<TD>& contract(const char* F, int L, const ctrIdx &ica,
01118         const wbsparray &B, const ctrIdx &icb, wbarray<TD> &C,
01119         const wbpperm &pfinal, const TD& afac, const TD& cfac
01120     ) const;
01121
01122     wbsparray& contract(const char *F, int L,
01123         unsigned il, const wbvector<TD> &B, wbsparray &C
01124     ) const;
01125
01126     wbsparray& comm(const char *F, int L,
01127         const wbsparray &B, wbsparray &C,
01128         char aflag='N', char bflag='N'
01129     ) const;
01130
01131     wbsparray& tensorProdX(
01132         const char *F, int L, const wbsparray& B, wbsparray& C,
01133         char conja=0, const TD &afac=1, char conjb=0, const TD &cfac=0,
01134         const wbpperm *Pfinal=NULL
01135     ) const;
01136
01137     wbsparray& tensorProd(

```

```

01138     const char *F, int L, const wbsparray& B, wbsparray& C,
01139     char aflag='N', char bflag='N', char kflag=0
01140 ) const;
01141
01142 wbsparray& Kron(const wbsparray& B,
01143     char aflag='N', char bflag='N', char kflag='k'
01144 ) {
01145     wbsparray X; tensorProd(FL,B,X,aflag,bflag,kflag);
01146     return X.save2(*this);
01147 };
01148
01149 wbsparray& kron(
01150     const char *F, int L, const wbsparray& B, wbsparray& C,
01151     char aflag='N', char bflag='N', char kflag='k'
01152 ) const {
01153     return tensorProd(F,L,B,C,aflag,bflag,kflag);
01154 };
01155
01156 mxArray* toMxSp() const;
01157
01158 mxArray* toMx(int m=-99) const {
01159     unsigned r=rank(FL);
01160
01161     if (m== -99 && r<=2 && Wb::isBaseType(typeid(TD))) {
01162         return toMxSp();
01163     }
01164
01165     mxArray *a=mxCreateStruct(1,1);
01166     add2MxStruct(a,0,m);
01167     return a;
01168 };
01169
01170 mxArray* IDtoMx() const;
01171
01172 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01173 void add2MxStruct(mxArray *a, unsigned i, int ma=-99) const;
01174
01175 mxArray* mxCreateCell(unsigned m, unsigned n) const;
01176 void add2MxCell(mxArray *a, unsigned i) const;
01177
01178 mxArray* toMx0(int m=-99) const {
01179     mxArray *a=mxCreateStructX(1,1,m);
01180     add2MxStructX(a,0,m);
01181     return a;
01182 };
01183 mxArray* mxCreateStructX(unsigned m, unsigned n, int ma=-99) const;
01184 void add2MxStructX(mxArray *a, unsigned i, int ma=-99) const;
01185
01186 mxArray* toMX() const {
01187     mxArray *a=mxCreateSTRUCT(1,1);
01188     return add2MxSTRUCT(a,0);
01189 };
01190 mxArray* mxCreateSTRUCT(unsigned m, unsigned n) const;
01191 mxArray* add2MxSTRUCT(mxArray *a, unsigned i) const;
01192
01193 void put(const char *vname, unsigned ma=-99, const char *ws="caller"
01194 ) const { put(0,0,vname,ma,ws); };
01195
01196 void put0(const char *F, int L,
01197     const char *vname, const char *ws="caller"
01198 ) const { put(F,L,vname,-99,ws,'X'); };
01199
01200 void put(const char *F, int L, const char *vname,
01201     unsigned ma=-99, const char *ws="caller", char flag=0
01202 ) const {
01203     mxArray *a=0; {
01204         if (!flag) a=toMx(ma); else
01205         if (flag=='X') a=toMX(); else
01206         if (flag=='0') a=toMx0(); else
01207         wblog(FL,"ERR %s() invalid flag=%s",FCT,Wb::char2Str(flag).data);
01208     }
01209     int i=mexPutVariable(ws,vname,a);
01210
01211     if (i) wblog(F_L,
01212         "ERR failed to write variable `%s' (%d)",vname,i);
01213     if (F) wblog(F,L,
01214         "I/O putting variable '%s' to %s",vname,ws);
01215     mxDestroyArray(a);
01216 };
01217
01218 wvector<SPIDX_T> SIZE;
01219 wbMatrix<SPIDX_T> IDX;
01220
01221 wvector<TD> D;
01222
01223 char isref;

```

```

01225
01226     protected:
01227
01228         unsigned get2DSize(
01229             const char *F, int L,
01230             SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const;
01231
01232         unsigned get2DIndex(
01233             wbMatrix<SPIDX_T> &IJ, SPIDX_T &d1, SPIDX_T &d2,
01234             int m=-1) const;
01235
01236     private:
01237
01238         TD dotProd_full_diag(const wbsparray &B, char cflag) const;
01239
01240         wbsparray& timesEl_full_diag(
01241             const wbsparray &B, wbsparray &X, char cflag
01242         ) const;
01243
01244     };
01245
01246     template<class TD> inline
01247     wbsparray<TD> operator+(const wbsparray<TD> &A) { return A; };
01248
01249     template<class TD> inline
01250     wbsparray<TD> operator-(const wbsparray<TD> &A) {
01251
01252         wbsparray<TD> X(A); TD *x=X.D.data;
01253         for (SPIDX_T n=X.D.len, i=0; i<n; ++i) { x[i]=-x[i]; }
01254         return X;
01255     };
01256
01257     template<class TD>
01258     class indexSparseRef {
01259     public:
01260
01261         indexSparseRef(const wbsparray<TD> &A_, unsigned M2=1)
01262             : M(0), A(NULL) { init(FL,A_,M2); };
01263
01264         indexSparseRef(
01265             const char *F, int L, const wbsparray<TD> &A_, unsigned M2=1)
01266             : M(0), A(NULL) { init(F,L,A_,M2); };
01267
01268         indexSparseRef& init(const char *F, int L,
01269             const wbsparray<TD> &A_, unsigned M2=1);
01270
01271         SPIDX_T numCols() const;
01272         SPIDX_T numRows() const;
01273         SPIDX_T nnz(SPIDX_T k) const;
01274
01275         TD LContractVec(
01276             const wbsparray<TD> &b, SPIDX_T k, char tnorm=0);
01277
01278         mxArray* toMx() const;
01279
01280         unsigned M;
01281
01282         wbvector<SPIDX_T> JA;
01283
01284         wbvector<sSPIDX_T> JJ;
01285
01286         const wbsparray<TD> *A;
01287     protected:
01288     private:
01289
01290     };
01291
01292     template <class TD>
01293     indexSparseRef<TD>& indexSparseRef<TD>::init(
01294         const char *F, int L, const wbsparray<TD> &A_, unsigned M2
01295     ){
01296         unsigned i, r=A_.rank(FL);
01297
01298         if (r<2 || A_.SIZE.len!=r || A_.IDX.dim2!=r) wblog(F_L,
01299             "ERR %s() got empty or diagonal wbsparray !? (%s)",FCT,SSTR(A_));
01300
01301         if (!M2 || M2>=r) wblog(F_L,
01302             "ERR %s() M2 out of bounds (%d/%d)",FCT,M2,r);
01303
01304         M=r-M2; A=(&A_);
01305
01306         const SPIDX_T *s=A_.SIZE.data, *idx=A_.IDX.data;
01307         SPIDX_T l, n=A_.IDX.dim1, D1=s[0], D2=s[M];
01308
01309         sSPIDX_T j,I,J, Ip=-1, Jp=-1;

```

```

01312
01313     for (i=1; i<M; ++i) D1*=s[i];
01314     for (++i; i<r; ++i) D2*=s[i];
01315
01316     #ifndef WB_SKIP_ASSERT
01317     { double d1=s[0], d2=s[M];
01318       for (i=1; i<M; ++i) d1*=s[i];
01319       for (++i; i<r; ++i) d2*=s[i];
01320       if ((1+ceil(log2(d1))) >= 8*sizeof(SPIDX_T) ||
01321           (1+ceil(log2(d2))) >= 8*sizeof(SPIDX_T)) wblog(FL,"ERR %s() "
01322           "got possible indexSparseRef overrun (%.3g, %.3g / %.3g)",
01323           FCT, log2(d1), log2(d2), 8*sizeof(SPIDX_T)
01324       );
01325     }
01326     #endif
01327
01328     if (!D1 || !D2) wblog(FL,"ERR %s() got empty wbsparray !?",FCT);
01329
01330     JA.init(D2+1);
01331     JJ.init2val(MIN(D1,D2),-1);
01332
01333     for (l=0; l<n; ++l, idx+=r) {
01334         I=Wb::sub2ind(s,idx,M); J=Wb::sub2ind(s+M,idx+M,M2);
01335
01336         if (J>Jp) { for (j=Jp+1; j<=J; ++j) JA[j]=1; }
01337         else {
01338             bool i=(J==Jp); if ((i && I<=Ip) || !i) wblog(FL,
01339             "ERR %s() wbsparray not sorted/col-major !?",FCT);
01340         }
01341
01342         if (I==J) JJ[J]=1;
01343
01344         Ip=I; Jp=J;
01345     }
01346
01347     J=D2; if (J>Jp) { for (j=Jp+1; j<=J; ++j) JA[j]=1; }
01348
01349     return *this;
01350 };
01351
01352 template <class TD> inline
01353 SPIDX_T indexSparseRef<TD>::numCols() const {
01354     if (!A || JA.len<1) wblog(FL,
01355     "ERR %s() not yet initialized !?",FCT);
01356     return (JA.len-1);
01357 };
01358
01359 template <class TD> inline
01360 SPIDX_T indexSparseRef<TD>::numRows() const {
01361     if (!A || !M || A->SIZE.len<=M) wblog(FL,
01362     "ERR %s() got uninitialized object ?",FCT);
01363     const SPIDX_T *S=A->SIZE.data; SPIDX_T D1=S[0];
01364     for (unsigned i=1; i<M; ++i) D1*=S[i];
01365     return D1;
01366 };
01367
01368 template <class TD> inline
01369 SPIDX_T indexSparseRef<TD>::nnz(SPIDX_T k) const {
01370     if (k+1>JA.len) wblog(FL,
01371     "ERR %s() index out of bounds (%d/%d) !?",FCT,k,JA.len);
01372     return (JA[k+1]-JA[k]);
01373 };
01374
01375 template <class TD> inline
01376 TD indexSparseRef<TD>::LContractVec(
01377     const wbsparray<TD> &B, SPIDX_T k, char tnorm
01378 ) {
01379     TD x=0;
01380
01381     if (!A) wblog(FL,"ERR %s() got empty A !?",FCT);
01382     if (A->SIZE.len!=2 || M!=1) wblog(FL,"ERR %s() "
01383     "only applies to rank-2 tensors (%s; %d)",FCT,SSTR_(A),M);
01384     if (k+1>JA.len) wblog(FL,
01385     "ERR %s() index out of bounds (%d/%d) !?",FCT,k,JA.len);
01386
01387     SPIDX_T d1=A->SIZE[0], d2=A->SIZE[1];
01388     SPIDX_T i=0, i2=B.IDX.dim1, j=JA[k], j2=JA[k+1];
01389
01390     if (k>=d2) wblog(FL,
01391     "ERR %s() index out of bounds (%d/%d) !?",FCT,k,d2);
01392     if (B.SIZE.len!=1 || B.SIZE[0]!=d1) wblog(FL,
01393     "ERR %s() size mismatch (%s <> %s)",FCT,SSTR_(A),SSTR_(B));
01394     if (j==j2) return x;
01395
01396     SPIDX_T r=2, jr=j*r;
01397     char cflag = (tnorm || WbUtil<TD>::hasNoConj()) ? 0 : 1);
01398     const SPIDX_T *I=B.IDX.data, *J=A->IDX.data;

```

```

01399     const TD *a=A->D.data, *b=B.D.data;
01400
01401     while (i<i2 && j<j2) {
01402         if (I[i ]<J[jr]) { ++i; } else
01403         if (J[jr]<I[i ]) { ++j; jr+=r; } else {
01404             if (cflag)
01405                 { x+=CONJ(b[i])*a[j]; }
01406             else { x+=      b[i] *a[j]; }
01407
01408             ++i; ++j; jr+=r;
01409
01410             if (i<i2 && I[i]<=I[i-1]) wblog(FL,
01411 "ERR %s() A: input sparse IDX not sorted !?",FCT);
01412             if (j<j2 && J[jr]<=J[jr-r]) wblog(FL,
01413 "ERR %s() B: input sparse IDX not sorted !?",FCT);
01414         }
01415     }
01416
01417     return x;
01418 };
01419
01420 template <class TD>
01421 mxArray* indexSparseRef<TD>::toMx() const {
01422
01423     if (A==NULL) wblog(FL,"ERR %s() A not yet initialized !?",FCT);
01424
01425     const char *fields[]={ "SIZE","M","JA","JJ"};
01426     mxArray *a=mxCreateStructMatrix(1,1,4,fields);
01427
01428     mxSetFieldByNumber(a,0, 0, A->SIZE.toMx());
01429     mxSetFieldByNumber(a,0, 1, numtoMx(M));
01430     mxSetFieldByNumber(a,0, 2, JA.toMx());
01431     mxSetFieldByNumber(a,0, 3, JJ.toMx());
01432
01433     return a;
01434 };
01435
01436 template<class TD>
01437 class iterSparseCMref {
01438
01439 public:
01440
01441     iterSparseCMref(const char *F, int L, wbspararray<TD> &A);
01442
01443     SPIDX_T next(wbspararray<TD> &a) {
01444         if ((++it)+1>=I0.len) { it=-1; return (it+1); }
01445         init(it,a,'r'); return (it+1);
01446     };
01447
01448     wbspararray<TD>& getIter(SPIDX_T i, wbspararray<TD> &a, char ref=0) const;
01449     TD norm2(SPIDX_T i, char tnorm=0) const;
01450
01451     SPIDX_T numIter() const {
01452         if (!IX.len) wblog(FL,
01453 "ERR %s() got non-initialized/empty object",FCT);
01454         return IX.len-1;
01455     };
01456
01457     bool gotLast() const {
01458         if (sSPIDX_T(it)<0 || it+1>=IX.len) wblog(FL,
01459 "ERR %s() not within loop (%d/%d)",FCT,it,IX.len-1);
01460         return (it+2==IX.len);
01461     };
01462
01463     SPIDX_T index() const {
01464         if (sSPIDX_T(it)<0 || it+1>=IX.len) wblog(FL,
01465 "ERR %s() not within loop (%d/%d)",FCT,it,IX.len-1);
01466         return IX[it];
01467     };
01468
01469     mxArray* toMx() const;
01470
01471     SPIDX_T it;
01472
01473     wbspararray<SPIDX_T> I0;
01474
01475     wbspararray<SPIDX_T> IX;
01476
01477     wbMatrix<SPIDX_T> IDX;
01478
01479 protected:
01480 private:
01481
01482     unsigned r;
01483     const SPIDX_T *S;
01484     const TD* data;
01485 };

```

```

01486
01487 template <class TD>
01488 mxArray* iterSparseCMref<TD>::toMx() const {
01489
01490     const char *fields[]={ "SIZE", "r", "it", "I0", "IX", "IDX", "data" };
01491     mxArray *a=mxCreateStructMatrix(1,1,7,fields);
01492
01493     mxSetFieldByNumber(a,0, 0, wbvector<SPIDX_T>(r,S,'r').toMx());
01494     mxSetFieldByNumber(a,0, 1, numtoMx(r));
01495     mxSetFieldByNumber(a,0, 2, numtoMx(it));
01496     mxSetFieldByNumber(a,0, 3, I0.toMx());
01497     mxSetFieldByNumber(a,0, 4, IX.toMx());
01498     mxSetFieldByNumber(a,0, 5, IDX.toMx());
01499     mxSetFieldByNumber(a,0, 6, wbvector<TD>(IDX.dim1,data,'r').toMx());
01500
01501     return a;
01502 };
01503
01504 template <class TD>
01505 iterSparseCMref<TD>::iterSparseCMref(
01506     const char *F, int L, wbsparray<TD> &A) : r(A.SIZE.len) {
01507
01508     unsigned l=r-1; SPIDX_T D;
01509     if (r<2 || ! (D=A.SIZE.last())) wblog(F_L,
01510         "ERR %s() got %s (%d)",FCT,SSTR(A),D);
01511
01512     A.Compress(F_L);
01513     S=A.SIZE.data; data=A.D.data; it=-1;
01514
01515     if (A.IDX.dim1!=A.D.len || A.SIZE.len!=A.IDX.dim2)
01516         A.sperror_this(F_LF);
01517
01518     if (A.IDX.dim1==0) {
01519         I0.init(1);
01520         IX.init(1); IX[0]=D;
01521         IDX.init(0,r-1); return;
01522     }
01523     if (A.IDX.dim1==1) {
01524         I0.init(2); I0[1]=1;
01525         IX.init(2); IX[0]=A.IDX(0,1); IX[1]=D;
01526         IDX.init2ref(1,1,A.IDX.data);
01527         return;
01528     }
01529
01530     SPIDX_T i0=0, i=1, k=1, *I=A.IDX.ref(1,1);
01531     const int d2=A.IDX.dim2;
01532     wbvector<SPIDX_T> Dg;
01533
01534     for (k=1; i<A.IDX.dim1; ++i, I+=d2) { if (I[0]!=I[-d2]) ++k; }
01535     Dg.init(k); IX.init(k+1);
01536
01537     k=0; I=A.IDX.ref(0,1);
01538     IX[0]=I[0]; I+=d2;
01539
01540     for (i=1; i<A.IDX.dim1; ++i, I+=d2) {
01541         if (I[0]!=I[-d2]) { Dg.data[k++]=i-i0; i0=i; IX[k]=I[0]; }
01542     }
01543     Dg.data[k++]=i-i0; IX[k]=D;
01544
01545     Dg.cumsum_(I0,'N');
01546     A.IDX.getCols(0,A.IDX.dim2-2,IDX);
01547 };
01548
01549 template <class TD>
01550 wbsparray<TD>& iterSparseCMref<TD>::getIter(
01551     SPIDX_T i, wbsparray<TD> &a, char ref) const {
01552
01553     if (i+1>=I0.len) wblog(F_L,
01554         "ERR %s() index out of bounds (%d/%d)",FCT,i,I0.len);
01555
01556     SPIDX_T i0=I0[i], n=I0[i+1]-i0;
01557     if (sSPIDX_T(n)<=0) wblog(F_L,"ERR %s() got I0(%d:%d) !?",FCT,i0,i0+n);
01558
01559     if (ref=='r' || ref=='R') {
01560         a.SIZE.init2ref(r-1,S);
01561         a.D.init2ref(n, data+i0);
01562         a.IDX.init2ref(n,IDX.dim2,IDX.rec(i0));
01563         a.isref=1;
01564     }
01565     else {
01566         a.SIZE.init(r-1,S);
01567         a.D.init(n, data+i0);
01568         a.IDX.init(n,IDX.dim2,IDX.rec(i0));
01569     }
01570
01571     return a;
01572 };

```

```

01573
01574 template <class TD>
01575 TD iterSparseCMref<TD>::norm2(SPIDX_T i, char tnorm) const {
01576
01577     if (i+1>=I0.len) wblog(FL,
01578         "ERR %s() index out of bounds (%d/%d)",FCT,i,I0.len);
01579
01580     SPIDX_T i0=I0[i], n=I0[i+1]-i0;
01581
01582     return Wb::overlap(data+i0,data+i0,n,1,tnorm);
01583 };
01584
01585 template <class T>
01586 inline int checkContract(const char* F, int L,
01587     const wbvector<T> &SA, const ctrIdx &ica,
01588     const wbvector<T> &SB, const ctrIdx &icb,
01589     const wbperm *pfinal=NULL, wbvector<T> *SC=NULL
01590 );
01591
01592 template <class TD>
01593 wbarray<TD>& contract(const char *F, int L,
01594     const wbsparray<TD>&, const wbindex&,
01595     const wbarray<TD>&, const wbindex&, wbarray<TD>&,
01596     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01597 );
01598
01599 template <class TD>
01600 wbarray<TD>& contract(const char *F, int L,
01601     const wbarray<TD>&, const wbindex&,
01602     const wbsparray<TD>&, const wbindex&, wbarray<TD>&,
01603     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01604 );
01605
01606 namespace Wb {
01607
01608     template <class TD>
01609     TD VMatVprod(
01610         const wbsparray<TD> &v1, const wbsparray<TD> &M,
01611         const wbsparray<TD> &v2
01612     );
01613
01614     template <class TD>
01615     wbsparray<TD>& MatProd(
01616         const wbsparray<TD> &A, const wbsparray<TD> &B, wbsparray<TD> &C,
01617         char aflag='N', char bflag='N', TD afac=TD(1.), TD cfac=TD(0.)
01618     );
01619
01620 };
01621
01622 template <class TD> inline
01623 wbarray<TD>& contract(const char *F, int L,
01624     const wbsparray<TD> &A, const char* s1,
01625     const wbsparray<TD> &B, const char* s2, wbsparray<TD> &C,
01626     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01627 ){
01628     return
01629     A.contract(F_L, wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
01630 };
01631
01632 template <class TD> inline
01633 wbarray<TD>& contract(const char *F, int L,
01634     const wbsparray<TD> &A, const char* s1,
01635     const wbarray<TD> &B, const char* s2, wbarray<TD> &C,
01636     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01637 ){
01638     return
01639     contract(F,L, A,wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
01640 };
01641
01642 template <class TD> inline
01643 wbarray<TD>& contract(const char *F, int L,
01644     const wbarray<TD> &A, const char* s1,
01645     const wbsparray<TD> &B, const char* s2, wbarray<TD> &C,
01646     const wbperm &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01647 ){
01648     return
01649     contract(F,L, A,wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
01650 };
01651
01652 template<class TD> inline
01653 void wbsparray<TD>::info(const char* F, int L, const char* istr) const {
01654
01655     SPIDX_T nz=D.len; checkSize(F_L);
01656
01657     if (!D.len) {
01658         if (SIZE.len) {
01659             wblog(F_L,"sparse %3s %12s (all zero)",

```



```

01660         istr && istr[0] ? istr:"", SSTR_(this));
01661     }
01662     else {
01663         wblog(F_L,"sparse %3s (empty)",
01664             istr && istr[0] ? istr:"", SSTR_(this));
01665     }
01666     return;
01667 }
01668
01669 if (isDiag(F_L)) {
01670     if (nz>4) {
01671         wblog(F_L,"sparse %3s %12s (diag)",
01672             istr && istr[0] ? istr:"", SSTR_(this));
01673     }
01674     else if (nz>1) {
01675         wblog(F_L,"sparse %3s %8sdiag( [%s] )",
01676             istr && istr[0] ? istr:"", SSTR_(this), STR(D));
01677     }
01678     else {
01679         wblog(F_L,"sparse %3s %5s(scalar) %-8.4g",
01680             istr && istr[0] ? istr:"", SSTR_(this), double(D[0]));
01681     }
01682     return;
01683 }
01684
01685 if (nz>4) {
01686     wblog(F_L,"sparse %3s %12s (nz=%d @ %-6.3g)",
01687         istr && istr[0] ? istr:"", SSTR_(this), nnz(), sparsity());
01688 }
01689 else {
01690     wblog(F_L,"sparse %3s %12s {%s}",
01691         istr && istr[0] ? istr:"", SSTR_(this), D.toStrf("%.4g", " ").data);
01692 }
01693 };
01694
01695 template<class TD> inline
01696 void wbsparray<TD>::printdata(const char *istr, const char *fmt0) const {
01697     SPIDX_T i; const char *fmt; wstring FMT;
01698
01699     if (fmt0 && fmt0[0])
01700         { fmt=fmt0; }
01701     else { FMT.init2Fmt((TD)0); fmt=FMT.data; }
01702
01703     checkSize(FLF);
01704
01705     if (isScalar()) {
01706         printf("%s = %4s (scalar)\n",istr,D.len ? NSTrf(D[0],fmt) : 0);
01707     }
01708     else if (isDiag()) {
01709         printf("%s = diag([",istr);
01710         for (i=0; i<D.len; ++i) printf(" %4s",NSTrf(D[i],fmt));
01711         printf(" ] (D=%ld)\n",D.len);
01712     }
01713     else if (SIZE.len==1) {
01714         printf("%s = [",istr);
01715         for (i=0; i<IDX.dim1; ++i) {
01716             printf(" %d %.8g;",IDX(i,1)+1,double(D[i]));
01717         }
01718         printf(" ] (vector, D=%ld)\n",SIZE[0]);
01719     }
01720     else if (SIZE.len==2) {
01721         printf("%s = [ % ( %ld x %ld )\n",istr,SIZE[0],SIZE[1]);
01722         for (i=0; i<IDX.dim1; ++i) {
01723             printf(" %5d %4d %.48g\n",IDX(i,0)+1,IDX(i,1)+1,double(D[i]));
01724         }
01725         printf("];\n");
01726     }
01727     else { wblog(FL,
01728         "ERR %s() only applies to rank<=2 (%s)",FCT,SSTR_(this));
01729     }
01730 };
01731 };
01732
01733 template<class TD> inline
01734 unsigned wbsparray<TD>::get2DSize(
01735     const char *F, int L,
01736     SPIDX_T &d1, SPIDX_T &d2, int m_,
01737     char fflag
01738 ) const {
01739     const SPIDX_T* const s=SIZE.data;
01740     unsigned m=0, i=0, r=rank(F_L);
01741
01742     if (isDiag()) wblog(FL,"ERR %s() got DIAG_REP",FCT);
01743
01744     if (!r) { d1=d2=0; return 0; }
01745
01746

```

```

01747     if (m_<0) {
01748         if (r%2 && fflag) wblog(F,L,"ERR sparse::%s() "
01749             "even rank or 2nd argument required (%s)",FCT,SSTR_(this));
01750         else {
01751             SPIDX_T dm=SIZE[0], d=std::sqrt(double(numel()));
01752             for (m=1; m<SIZE.len; ++m) {
01753                 if (dm>=d) break; else dm*=SIZE[m];
01754             }
01755         }
01756     }
01757 }
01758 else { m=m_;
01759     if (m>r) wblog(F,L,"ERR sparse::%s() "
01760         "invalid m=%d (%s)",FCT,m,SSTR_(this));
01761 }
01762
01763 if (m>0) { for (d1=s[i++]; i<m; ++i) { d1*=s[i]; }} else d1=1;
01764 if (m<r) { for (d2=s[i++]; i<r; ++i) { d2*=s[i]; }} else d2=1;
01765
01766     return m;
01767 };
01768
01769 template<class TD> inline
01770 unsigned wbsparray<TD>::get2DIndex(
01771     wbMatrix<SPIDX_T> &IJ, SPIDX_T &d1, SPIDX_T &d2, int m_
01772 ) const {
01773
01774     if (isDiag(FL) || !D.len) {
01775         d1=d2=D.len; IJ.init(d1,2);
01776         for (SPIDX_T *I=IJ.data, i=0; i<d1; ++i, I+=2) { I[0]=I[1]=i; }
01777         return 1;
01778     }
01779     if (!IDX.dim2) sperror_this(FLF);
01780
01781     SPIDX_T i,j,k, n=IDX.dim1;
01782     unsigned r=rank(FL), l=r-1, m=get2DSize(FL,d1,d2,m_);
01783
01784     IJ.init(IDX.dim1,2);
01785
01786     if (!d1 || !d2 || !r) {
01787         if (IJ.data || D.data) wblog(FL,
01788             "ERR %s() got d1=%d, d2=%d, yet S=[%s] (%d) !?",
01789             FCT,d1,d2,STR(SIZE),D.len);
01790         return m;
01791     }
01792
01793     SPIDX_T *ij=IJ.data; char e=0;
01794     const SPIDX_T *idx=IDX.data, *s=SIZE.data;
01795
01796     if (!m || m==r) {
01797         for (i=0; i<n; ++i, ij+=2, idx+=r) {
01798             for (j=1, k=idx[j--]; j<r; --j) { k=k*s[j]+idx[j]; };
01799             if (m==r)
01800                 { ij[0]=k; if (k>=d1) { e=1; break; }; }
01801             else { ij[1]=k; if (k>=d2) { e=2; break; }; }
01802         }
01803     }
01804     else {
01805         for (i=0; i<n; ++i, ij+=2, idx+=r) {
01806             j=1;
01807
01808             for (k=idx[j--]; j>=m; --j) { k=k*s[j]+idx[j]; };
01809             ij[1]=k;
01810
01811             for (k=idx[j--]; j<r; --j) { k=k*s[j]+idx[j]; };
01812             ij[0]=k;
01813
01814             if (ij[0]>=d1 || ij[1]>=d2) { e=3; break; }
01815         }
01816     }
01817
01818     if (e) wblog(FL,
01819         "ERR %s() index out of bounds (%d: (%d,%d) %dx%d; %s: %d; %d)",
01820         FCT, i+1, ij[0]+1, ij[1]+1, d1,d2,SSTR_(this),m, e
01821     );
01822
01823     return m;
01824 };
01825
01826 template<class TD> inline
01827 wbsparray<TD>& wbsparray<TD>::initIdentity(
01828     const wbvector<SPIDX_T> &S0,
01829     int m,
01830     sSPIDX_T id,
01831     char fflag
01832 ) {
01833     SPIDX_T d1=1,d2=1;

```

```

01834     wbindx I(S0.len); SIZE=S0; init(S0);
01835
01836     get2DSize(FL,d1,d2,m);
01837
01838     if (d1 && d2) { SPIDX_T i=0, n=0;
01839         if (id==0) {
01840             n=MIN(d1,d2); init_nnz(n); D.set(1);
01841             for (; i<n; ++i) ind2sub(i+d1*i,IDX.rec(i));
01842         }
01843         else if (id>0) { SPIDX_T k=id; if (k<d2) {
01844             n=MIN(d1,d2-k); init_nnz(n); D.set(1);
01845             for (; i<n; ++i) ind2sub(i+d1*(i+k),IDX.rec(i));
01846         }
01847         else if (id<0) { SPIDX_T k=-id; if (k<d1) {
01848             n=MIN(d1-k,d2); init_nnz(n); D.set(1);
01849             for (; i<n; ++i) ind2sub((i+k)+d1*i,IDX.rec(i));
01850         }
01851     }
01852
01853     if (!D.len && fflag)
01854         wblog(FL,"ERR sparse::%s() to empty (k=%d given %s)",
01855             FCT, id, SSTR_(this)
01856         );
01857
01858     return *this;
01859 };
01860
01861 template<class TD> inline
01862 wbsparray<TD>& wbsparray<TD>::initIdentityB3(
01863     SPIDX_T d1, SPIDX_T d2, SPIDX_T D,
01864     sSPIDX_T i0,
01865     char fflag
01866 ){
01867     SPIDX_T s[3]={d1,d2,D}; SIZE.init(3,s);
01868     return initIdentity(SIZE,2,i0,fflag);
01869 };
01870
01871 template<class TD>
01872 class sparseCollector2D {
01873
01874 public:
01875
01876     sparseCollector2D(SPIDX_T m=0, SPIDX_T n=0)
01877         : M(m), N(n), nnz(0), NIJ(NULL), data(NULL) { SPIDX_T MN=M*N;
01878         if (MN) {
01879             WB_NEW(data,MN); MEM_SET<TD>(data,MN);
01880             WB_NEW(NIJ,MN); memset(NIJ,0,MN*sizeof(SPIDX_T));
01881         }
01882     };
01883
01884     ~sparseCollector2D() { clear(); }
01885
01886     void clear() {
01887         if ((data==NULL) ^ (NIJ==NULL)) wblog(FL,
01888             "ERR %s() got partial initialization (%lX, %lX) !?",FCT,data,NIJ);
01889         if (data) { WB_DELETE(data); }
01890         if (NIJ) { WB_DELETE(NIJ); }
01891         M=N=nnz=0;
01892     };
01893
01894     void init(SPIDX_T m=0, SPIDX_T n=0) { SPIDX_T MN=M*N;
01895         clear(); M=m; N=n; nnz=0;
01896         if (MN) {
01897             WB_NEW(data,MN); MEM_SET<TD>(data,MN);
01898             WB_NEW(NIJ,MN); memset(NIJ,0,MN*sizeof(SPIDX_T));
01899         }
01900     };
01901
01902     sparseCollector2D& add(
01903         const char *F, int L, SPIDX_T i, SPIDX_T j, const TD& a
01904     ){
01905         if (i>=M || j>=N || !data) wblog(F_L,
01906             "ERR %s() index out of bounds (%d,%d)/(%dx%d) [%lX]",
01907             FCT,i,j,M,N,data);
01908         SPIDX_T k=i*N+j;
01909         data[k]+=a; if ((++NIJ[k])==1) ++nnz;
01910         return *this;
01911     };
01912
01913     void getData(wbMatrix<SPIDX_T> &IJ, wbvector<TD> &X) {
01914         SPIDX_T i,j, k=0, l=0;
01915         IJ.init(nnz,2); X.init(nnz);
01916
01917         for (i=0; i<M; ++i)
01918             for (j=0; j<N; ++j, ++l) if (data[l]!=0) {
01919                 if (k>=nnz || !NIJ[l]) wblog(FL,"ERR %s() "
01920                     "possibly incorrect nnz (%g/%g; %d) !?",FCT,k,nnz,NIJ[l]);

```

```

01921         X.data[k]=data[l];
01922         IJ(k,0)=i; IJ(k,1)=j; ++k;
01923     }
01924     if (nnz && k<nnz) {
01925         if (k) { IJ.dim1=k; X.len=k; }
01926         else {
01927             wblog(FL,"WRN %s() got all-zero data",FCT);
01928             IJ.init(0,2); X.init();
01929         }
01930     }
01931 };
01932
01933 mxArray* toMx() const {
01934     const char *fields[]={"M","N","nnz","data","NIJ"};
01935     mxArray *S=mxCreateStructMatrix(1,1,5,fields);
01936
01937     mxSetFieldByNumber(S,0,0, numtoMx(M));
01938     mxSetFieldByNumber(S,0,1, numtoMx(N));
01939     mxSetFieldByNumber(S,0,2, numtoMx(nnz));
01940     mxSetFieldByNumber(S,0,3, wbMatrix<TD>(M,N,data,'r').toMx());
01941     mxSetFieldByNumber(S,0,4, wbMatrix<SPIDX_T>(M,N,NIJ,'r').toMx());
01942
01943     return S;
01944 };
01945
01946 protected:
01947 private:
01948
01949     SPIDX_T M,N,nnz,*NIJ;
01950     TD* data;
01951 };
01952
01953 #endif
01954

```

5.60 wbstring.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbstring
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_STRING_CC__
00021 #define __WB_STRING_CC__
00022
00023 // ----- //
00024 // tags: memsize fsize kB MB GB
00025
00026 char* Wb::memsize2Str(double x, char *s, unsigned l) {
00027     if (int(l)<0) {
00028         if (x<5E3) sprintf(s,"%g bytes",x);           else
00029         if (x<1E6) sprintf(s,"%g kB",x/(1<10));      else
00030         if (x<1E9) sprintf(s,"%gM",x/(1<20));        else
00031         sprintf(s,"%gG",x/(1<30));
00032     }
00033     else {
00034         unsigned i=0; if (l) {
00035             if (x<5E3) i=snprintf(s,l,"%g bytes",x);           else
00036             if (x<1E6) i=snprintf(s,l,"%g kB",x/(1<10));      else
00037             if (x<1E9) i=snprintf(s,l,"%gM",x/(1<20));        else
00038             i=snprintf(s,l,"%gG",x/(1<30));
00039         }
00040         if (i>=l) wblog(FL,
00041             "WRN %s() string out of bounds (%s; %d/%d)",
00042             FCT,s,strlen(s),l
00043         );
00044     }
00045     return s;
00046 };

```

```

00047
00048 wbststring Wb::repHome(const wbststring &file) {
00049     return Wb::repHome(file.data);
00050 };
00051
00052 namespace Wb {
00053
00054 template<class T>
00055 wbststring num2Str(const T &x, const char *f, unsigned l) {
00056     wbststring s(int(l)<16 ? 16 : l), fmt;
00057     if (!f || f[0]) {
00058         fmt.init2Fmt(x,-1,4); f=fmt.data;
00059     }
00060     l=snprintf(s.data,s.len,f,x);
00061     if (l>=s.len) wblog(FL,"ERR %s() string out of bounds "
00062         "%d/%d\n%s '%s' -> '%s'",FCT,l,s.len,TSTR(T),f,s.data);
00063     return s;
00064 };
00065
00066 template<>
00067 wbststring num2Str(const wbcomplex &x, const char *f,
00068     unsigned l __attribute__((unused)))
00069 ){ return x.toStr(f && f[0] ? f : "%.4g"); };
00070
00071 #ifdef QS_USING_MPFR
00072 template<>
00073 wbststring num2Str(const Wb::quad &x, const char *f, unsigned l
00074 ){ return num2Str(double(x),f,l); };
00075 #endif
00076
00077 template<>
00078 wbststring num2Str(const int &x, const char *f, unsigned l) {
00079     return int2Str(x,f,l);
00080 };
00081 template<>
00082 wbststring num2Str(const unsigned &x, const char *f, unsigned l) {
00083     return int2Str(x,f,l);
00084 };
00085 template<>
00086 wbststring num2Str(const long &x, const char *f, unsigned l) {
00087     return int2Str(x,f,l);
00088 };
00089 template<>
00090 wbststring num2Str(const unsigned long &x, const char *f, unsigned l) {
00091     return int2Str(x,f,l);
00092 };
00093
00094 template<>
00095 wbststring num2Str(const char &x,
00096     const char *f __attribute__((unused)), unsigned l) {
00097     return char2Str(x,l);
00098 };
00099 template<>
00100 wbststring num2Str(const unsigned char &x,
00101     const char *f __attribute__((unused)), unsigned l) {
00102     return char2Str(x,l);
00103 };
00104
00105 wbststring int2Str(long x, const char *f, unsigned l) {
00106     wbststring s_(l?l:32);
00107     unsigned i=0, j,k=0, m=0, n=0; char *s=s_.data;
00108
00109     l=snprintf(s,s_.len,f && f[0]? f:"%ld",x);
00110     for (; s[i]; ++i) {
00111         if (!isdigit(s[i])) { if (s[i]!='-') ++m; else ++k; }
00112         else {
00113             for (++n, ++i; s[i]; ++i) { if (isdigit(s[i])) ++n; else break; }
00114             break;
00115         }
00116     }
00117     if (s[i] || i!=l || !n || k>1) wblog(FL,
00118         "ERR %s() invalid int '%s' !?",FCT,s);
00119
00120     i=(n-1)/3;
00121     j=1; if (i>m) { j+=(i-m); }
00122     if (j>=s_.len) wblog(FL,
00123         "ERR %s() string out of bounds (%ld / %d)",FCT,x,s_.len);
00124
00125     if (n>3) { s[j]=0; --n;
00126         for (i=1; i<=l; --n) { if (i && j) {
00127             s[--j]=s[--i];
00128             if (j>1 && n<1 && (1-i)%3==0) { s[--j]=','; }
00129             } else { break; } }
00130         if (j<i) wblog(FL,
00131             "ERR %s() invalid int %ld '%s' (%d/%d/%d) !?",FCT,x,s,i,j,n);
00132     }
00133     return s_;

```

```

00134 };
00135
00136 wstring char2Str(int x, unsigned l) {
00137     wstring s(l?l:16);
00138
00139     if (isalnum(x)) l=snprintf(s.data,s.len,"%c",x); else
00140     if (isprint(x)) l=snprintf(s.data,s.len,"%c",x);
00141     else l=snprintf(s.data,s.len,"%d",x);
00142
00143     if (l>=s.len) wblog(FL,
00144         "WRN %s() string out of bounds `%s' (%d/%d)",FCT,s.data,l,s.len);
00145     return s;
00146 };
00147
00148 };
00149
00150 int Wb::findRegEx(const char *str, const char *pat, char icase) {
00151
00152     int q=0; regex_t r;
00153
00154     if (!str) wblog(FL,"ERR %s() got null input !?",FCT);
00155     if (!pat || !pat[0]) wblog(FL,
00156         "ERR %s() got invalid regexp (empty)",FCT);
00157
00158     if (strstr(pat,"\\d")) wblog(FL,
00159         "ERR %s() does not support \\d for digits (use [0-9], instead)",FCT);
00160
00161     q=(REG_EXTENDED | REG_NOSUB); if (icase) { q|=REG_ICASE; }
00162     if ((q=regcomp(&r,pat, q))) wblog(FL,
00163         "ERR %s() invalid regexp '%s' (e=%d)",FCT,pat,q);
00164     if (!regex(&r,str,0,NULL,0)) { q=1; }
00165
00166     regfree(&r);
00167     return q;
00168 };
00169
00170 template<class T>
00171 wstring& wstring::operator<< (const T& x) {
00172     wblog(FL,"ERR %s() type `%s' not specialized",FCT,TSTR(x));
00173     return *this;
00174 };
00175
00176 template<class T>
00177 wstring& wstring::operator<< (const T* x) {
00178     wblog(FL,"ERR %s() type `%s' not specialized",FCT,TSTR(*x));
00179     return *this;
00180 };
00181
00182 template<>
00183 wstring& wstring::operator<< (const char* s) {
00184     return (*this)+=s;
00185 };
00186 template<>
00187 wstring& wstring::operator<< (const char& x) {
00188     return (*this)+=Wb::char2Str(x);
00189 };
00190
00191 template<>
00192 wstring& wstring::operator<< (const int& x) {
00193     return (*this)+=Wb::int2Str(x);
00194 };
00195 template<>
00196 wstring& wstring::operator<< (const unsigned& x) {
00197     return (*this)+=Wb::int2Str(x);
00198 };
00199 template<>
00200 wstring& wstring::operator<< (const long& x) {
00201     return (*this)+=Wb::int2Str(x);
00202 };
00203 template<>
00204 wstring& wstring::operator<< (const unsigned long& x) {
00205     return (*this)+=Wb::int2Str(x);
00206 };
00207
00208 template<>
00209 wstring& wstring::operator<< (const double& x) {
00210     return (*this)+=Wb::num2Str(x);
00211 };
00212 template<>
00213 wstring& wstring::operator<< (const float& x) {
00214     return (*this)+=Wb::num2Str(x);
00215 };
00216 template<>
00217 wstring& wstring::operator<< (const wcomplex& x) {
00218     return (*this)+=x.toStr();
00219 };
00220

```

```

00221 size_t wbstring::toHash(unsigned offset) const {
00222
00223     size_t h=0;
00224
00225     if (offset) {
00226         if (!data) wblog(FL,
00227             "ERR %s() got offset=%d for null string",FCT,offset);
00228         if (offset>=len) { wblog(FL,
00229             "WRN %s() offset out of bounds (%d/%d)",FCT,offset,len);
00230             return h;
00231         }
00232         for (unsigned i=0; i<offset; ++i) { if (!data[i]) wblog(FL,
00233             "ERR %s() string terminating prior to offset "
00234             "(i=%d/%d, len=%d)\n%s",FCT,i+1,offset,len,data);
00235         }
00236     }
00237
00238     if (data) {
00239         h=Wb::string_hash(data+offset,len-offset); }
00240     return h;
00241 };
00242
00243 size_t Wb::string_hash(const char *s, unsigned n,
00244     size_t hid
00245 ) {
00246     if (s) { unsigned i=0;
00247         if (int(n)<0) {
00248             for (; s[i]; ++i) { hid = ((hid<5) + hid) ^ s[i]; }
00249         }
00250         else {
00251             for (; i<n && s[i]; ++i) { hid = ((hid<5) + hid) ^ s[i]; }
00252             if (s[i]) { wblog(FL,
00253                 "WRN %s() string not zero-terminated (n=%d)",FCT,n);
00254             }
00255         }
00256     }
00257     else if (n) wblog(FL,"ERR %s() got n=%d for null string",FCT,n);
00258     return hid;
00259 };
00260
00261 inline wbstring& wbstring::push(
00262     const char *F, int L, const char* s, char extend) {
00263
00264     if (!s || !s[0]) { return *this; }
00265
00266     unsigned n1=(data ? strlen(data) : 0), n2=strlen(s);
00267
00268     if (n1+n2>=len && extend) {
00269         unsigned l=n1+n2+1, m=(l%32); if (m) { l=l+(32-m); }
00270         if (WBLOG_RESIZE) wblog(F_L,
00271             "TST %s() resizing %d -> %d",FCT,len,l);
00272         Resize(l);
00273     }
00274
00275     if (n1+n2<len) { strcpy(data+n1,s); }
00276     else if (!len) wblog(F_L,
00277         "ERR %s() uninitialized string (%d; %p)",FCT,len,data);
00278     else {
00279         if (n1+l<len) { strncpy(data+n1,s,len-n1); data[len-1]=0; }
00280         wblog(F_L,
00281             "WRN %s() string out of bounds (%d+%d/%d)%N%N> %s%N",
00282             FCT,n1,n2,len,data);
00283     }
00284
00285     return *this;
00286 };
00287
00288 wbstring& wbstring::pushf(const char *F, int L, const char *fmt, ...) {
00289
00290     if (fmt && fmt[0]) {
00291         size_t l, n1=strlen(data), n2=len-n1;
00292         if (len<=l) wblog(F_L,"ERR %s() got empty string (%d)",FCT,len);
00293         if (n1>=len) wblog(F_L,
00294             "ERR %s() got string without null-terminator (%d)!",FCT,len);
00295
00296         for (int retry=1; retry<=2; ++retry) {
00297             #pragma omp critical (got_va_list)
00298             { va_list args;
00299                 va_start(args,fmt);
00300                 l=vsprintf(data+n1,n2,fmt,args);
00301                 va_end(args);
00302             }
00303             if (l<n2) { break; }
00304             if (retry==1) {
00305                 l+=(n1+1); l+=(-1 % (l>128 ? 256 : 64));
00306                 if (WBLOG_RESIZE) wblog(F_L,
00307                     "TST %s() resizing %d -> %d",FCT,len,l);

```

```

00308         Resize(n1+l+1); n2=len-n1;
00309     }
00310     else { data[len-1]=0; wblog(F_L,
00311         "ERR %s() string out of bounds (%d->%d/%d)%N%N> %s%N",
00312         FCT,n1,n1+l,len,data);
00313     }
00314 }
00315 }
00316 return *this;
00317 };
00318
00319 int Wb::String::pushf(const char *fmt, ...) {
00320
00321     if (!fmt || !fmt[0]) { return -1; }
00322     if (len<=1) wblog(F_L,
00323         "ERR %s got empty string (%d)",fline? fline:FCT, len);
00324     if (ipos+strlen(fmt)>=len) wblog(F_L,
00325         "ERR %s string out of bounds (%d+%d/%d)",fline? fline:FCT,
00326         ipos,strlen(fmt),len);
00327
00328     unsigned ip0=ipos;
00329
00330     #pragma omp critical (got_va_list)
00331     { va_list args;
00332       va_start(args,fmt);
00333       ipos+=vsprintf(data+ipos,len-ipos,fmt,args);
00334       va_end(args);
00335     }
00336
00337     if (ipos>=len) { data[len-1]=0; wblog(F_L,
00338         "ERR %s() string out of bounds (%d->%d/%d)%N%N> %s%N",
00339         fline? fline:FCT, ip0,ipos,len,data);
00340     }
00341
00342     return (ipos-ip0);
00343 };
00344
00345 inline int wbstring::printf(const char *F, int L,
00346     const char *fmt, ...
00347 ) {
00348     int l=-1;
00349     if (fmt && fmt[0]) {
00350         if (len<=1) wblog(F_L,"ERR %s() got empty string (%d)",FCT,len);
00351
00352         va_list args;
00353         va_start(args,fmt);
00354         l=vsprintf(data,len,fmt,args);
00355         va_end(args);
00356
00357         if (unsigned(l)>=len) { data[len-1]=0; wblog(F_L,
00358             "ERR %s() string out of bounds (%d/%d)%N%N> %s%N",FCT,l,len,data);
00359         }
00360     }
00361     return l;
00362 };
00363
00364 inline wbstring& wbstring::cpy(const char *F, int L,
00365     const char *s
00366 ) {
00367     if (s) {
00368         size_t l=strlen(s);
00369         if (l+1>=len) wblog(F_L,
00370             "ERR %s() string out of bounds (%s; %d/%d)",FCT,s,l,len);
00371         if (s[0]) strcat(data,s);
00372         else data[0]=0;
00373     }
00374     else wblog(F_L,"ERR %s() got NULL string",FCT);
00375
00376     return *this;
00377 };
00378
00379 int wbstring::RegEx_replace(
00380     const char *F, int L, const char *pat, const char *rep,
00381     char icase,
00382     char gflag
00383 ) {
00384
00385     int rval=0;
00386     if (!data || !data[0]) { return rval; }
00387
00388     auto rflags = std::regex_constants::ECMAScript;
00389     if (icase) { rflags |= std::regex_constants::icase; }
00390
00391     try {
00392         string sout=regex_replace(data,regex(pat,rflags),rep,
00393             gflag ? std::regex_constants::match_default :
00394             std::regex_constants::format_first_only);

```



```

00395         const char *s=sout.data();
00396         rval=strcmp(data,s);
00397         if (rval) { init(s); }
00398     }
00399     catch (...) {
00400         wblog(F_L,"ERR regex_replace() invalid regex_t = '%s'",pat);
00401     }
00402
00403     return rval;
00404 };
00405
00406 #endif
00407

```

5.61 wbstring.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbstring
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_STRING_HH__
00021 #define __WB_STRING_HH__
00022
00023 namespace Wb {
00024
00025     char* memsize2Str(double x, char *s, unsigned l);
00026
00027     wbstring repHome(const wbstring &file);
00028
00029     int findRegEx(const char *s, const char *pat, char icase=0);
00030
00031     size_t string_hash(const char *s, unsigned n=-1, size_t hid=0);
00032 };
00033
00034 class wbstring : public wbvector<char> {
00035 public:
00036
00037     wbstring (unsigned n=0) : wbvector<char>(n) {};
00038
00039     wbstring (unsigned n, const char c)
00040         : wbvector<char>(n+1) { set(c); data[len-1]=0; };
00041
00042     wbstring (const char* s,unsigned n) { init(s,n); }
00043
00044     wbstring (const wbstring &s) : wbvector<char>() { init(s.data); };
00045     wbstring (const char* s1) : wbvector<char>() { init(s1); };
00046
00047     wbstring (
00048         const char* s1, const char* s2,
00049         const char* s3=NULL, const char* s4=NULL
00050     ) : wbvector<char>() {
00051         unsigned
00052             l1 = (s1? strlen(s1) : 0),
00053             l2 = (s2? strlen(s2) : 0),
00054             l3 = (s3? strlen(s3) : 0),
00055             l4 = (s4? strlen(s4) : 0);
00056
00057         RENEW(l1+l2+l3+l4+1);
00058         if (l1) { strcpy(data, s1); }
00059         if (l2) { strcpy(data+l1, s2); }
00060         if (l3) { strcpy(data+l1+l2, s3); }
00061         if (l4) { strcpy(data+l1+l2+l3,s4); }
00062         data[len-1]=0;
00063     };
00064
00065 #ifndef NOMEX
00066     wbstring(const mxArray *a) : wbvector<char>() { init(0,0,a); };
00067

```

```

00068     wbsttring& init(const mxArray *a) { return init(0,0,a); }
00069 #endif
00070
00071     wbsttring (const char *F, int L, const mxArray *a)
00072     : wbvector<char>() { init(F,L,a); };
00073
00074 #if 0
00075     ~wbstring() {
00076         if (len) {
00077             if (data[len-1]) {
00078                 unsigned i=0; for (; i<len && data[i]; ++i) { };
00079                 if (i>=len) { wblog(FL,
00080                     "WRN %s() string out of bounds? (len=%d/%d)",FCT,i,len);
00081                     fprintf(stdout,"\n");
00082                     for (i=0; i<len && data[i]; ++i) fputc(data[i],stdout);
00083                     fprintf(stdout,"\n\n");
00084                 }
00085             }
00086         }
00087     };
00088 #endif
00089
00090     wbsttring& init(unsigned n=0) { RENEW(n); return *this; };
00091
00092     wbsttring& init(const char *s) {
00093         if (s) {
00094             if (s[0])
00095                 { RENEW(strlen(s)+1, s); data[len-1]=0; }
00096             else { RENEW(1); data[0]=0; }
00097         } else { RENEW(0); }
00098         return *this;
00099     };
00100
00101     wbsttring& init(const char* s, unsigned n) {
00102         unsigned l=(s? strlen(s) : -1);
00103         RENEW(n+1,s,l); data[n]=0;
00104         return *this;
00105     };
00106
00107     wbsttring& init(const char *F, int L, const mxArray *a) {
00108         int i,n;
00109         if (!a || !(n=mxGetNumberOfElements(a))) {
00110             RENEW(0); return *this;
00111         };
00112
00113         if (!mxIsChar(a)) wblog(F_L,
00114             "ERR initializing string with type '%s'",mxGetClassName(a));
00115
00116         RENEW(++n);
00117         i=mxGetString(a,data,n);
00118
00119         if (i) wblog(F_L,"ERR failed to read string >%s<",data);
00120         return *this;
00121     };
00122
00123     wbsttring& initrep(const char* s, unsigned n) {
00124         if (s && s[0] && n) {
00125             unsigned l=strlen(s); init(l*n+1);
00126             if (l==1) { memset(data,s[0],n); data[len-1]=0; }
00127             else {
00128                 unsigned i=0; char *d=data;
00129                 for (; i<n; ++i, d+=l) { strcpy(d,s); }
00130             }
00131         }
00132         else { init(); }
00133         return *this;
00134     };
00135
00136     wbsttring& initrep(char c, unsigned n) {
00137         if (n) {
00138             if (!isprint(c)) wblog(FL,
00139                 "WRN %s() got non-print char=%c<%d>",FCT,c,c);
00140             init(n+1); data[len-1]=0; memset(data,c,n);
00141         }
00142         else { init(); }
00143         return *this;
00144     };
00145
00146     void set(char c) {
00147         if (len) {
00148             memset(data, c, len-1); data[len-1]=0; }
00149     };
00150
00151     wbsttring& operator= (const char* s) { return init(s); }
00152
00153     bool gotTerm0() {

```

```

00155         for (unsigned i=0; i<len; ++i) { if (data[i]==0) return 1; }
00156         return 0;
00157     };
00158
00159     bool strchr(const char *s) {
00160         if (data && data[0]) {
00161             if (!s || !s[0]) wblog(FL,"WRN %s() got empty char set !?",FCT);
00162             else { for (; *s; ++s) { if (::strchr(data,*s)) return 1; }}
00163         }
00164         return 0;
00165     };
00166
00167     wbststring& push(const char *F, int L, const char* s, char extend=1);
00168     wbststring& push(const char* s, char extend=1) {
00169         return push(0,0,s,extend); };
00170
00171     wbststring& pushf(const char *F, int L, const char *fmt, ...);
00172     wbststring& cpy(const char *F, int L, const char *s);
00173
00174     int RegEx_replace(const char *F, int L, const char *pat, const char *rep,
00175         char icalse=0, char gflag=1);
00176
00177     int printf(const char *F, int L, const char *fmt, ...);
00178
00179     wbststring& time() { return time(::time(NULL)); };
00180     wbststring& time(const time_t &t) { RENEW(32);
00181         #pragma omp critical (got_CPTR_TIME)
00182         { ctime_r(&t,data); }
00183         return *this;
00184     };
00185
00186     wbststring& time_sys() { return time_sys(::time(NULL)); };
00187     wbststring& time_sys(const time_t &t) { RENEW(32);
00188         #pragma omp critical (got_CPTR_TIME)
00189         { strftime(data,31,"%a %b %d %H:%M:%S %Z %Y",localtime(&t)); }
00190         return *this;
00191     };
00192
00193     wbststring& Upper() {
00194         for (unsigned n=strlen(data), i=0; i<n; i++)
00195             data[i]=std::toupper(data[i]);
00196         return *this;
00197     };
00198
00199     wbststring& Lower() {
00200         for (unsigned n=strlen(data), i=0; i<n; i++)
00201             data[i]=std::tolower(data[i]);
00202         return *this;
00203     };
00204
00205     wbststring toupper() const {
00206         wbststring sout(*this); sout.Upper(); return sout; };
00207
00208     wbststring tolower() const {
00209         wbststring sout(*this); sout.Lower(); return sout; };
00210
00211     wbststring& Chomp() {
00212         if (data && data[0]) { int i=strlen(data);
00213             for (; i>=0; --i) {
00214                 if (data[i]>20 && data[i]!=' ') { data[i+1]=0; break; }
00215             }
00216             if (i<0) data[0]=0;
00217         }
00218         return *this;
00219     };
00220
00221     char getopt(char c) {
00222         for (unsigned i=0; data[i]; i++) {
00223             if (data[i]==c) {
00224                 data[i]=-1;
00225                 return 1;
00226             }
00227         }; return 0;
00228     };
00229
00230     char getopt() {
00231         for (unsigned i=0; data[i]; i++) {
00232             if (data[i]>0) { return data[i]; }
00233         }; return 0;
00234     }
00235
00236     const char* basename(const char c='/') const {
00237         if (!data) return data;
00238         return Wb::basename(data,c);
00239     };
00240
00241     mxArray* toMx() const {

```

```

00242         return mxCreateString(data ? data : "(null)");
00243     };
00244
00245     size_t toHash(unsigned offset=0) const;
00246
00247     void add2MxStruct(mxArray *S, unsigned i) const {
00248         mxSetCell(S,i,toMx());
00249     };
00250
00251     template <class T>
00252     char* init2Fmt(const T& x __attribute__((unused)), int n=-1, int p=-1){
00253         (*this)=wbstring(Wb::num2Fmt<T>(FL,n,p));
00254         return data;
00255     };
00256
00257     template <class T> wbstring& operator« (const T* x);
00258     template <class T> wbstring& operator« (const T& x);
00259
00260     wbstring operator+ (const wbstring &s) const { return (*this)+s.data; };
00261     wbstring operator+ (const char *s) const {
00262         wbstring sout;
00263         unsigned n1=data? strlen(data):0, n2=s && s[0]? strlen(s):0;
00264         sout.init(n1+n2);
00265         if (n1) { strcpy(sout.data, data); }
00266         if (n2) { strcpy(sout.data+n1, s); }
00267         return sout;
00268     };
00269
00270     wbstring& operator+=(const char *s) {
00271         if (data && s && s[0]) {
00272             unsigned n1=strlen(data), n2=strlen(s);
00273             if (n2) { Resize(n1+n2); strcpy(data+n1, s); }
00274         }
00275         else if (s) { (*this)=s; }
00276         return *this;
00277     };
00278
00279     wbstring& operator+=(const wbstring &s) {
00280         if (s.data) { (*this)+=s.data; }
00281         return *this;
00282     };
00283
00284     unsigned cat(unsigned &k, const char *s) {
00285         if (s) { unsigned i=0;
00286             for (; s[i]!=0 && k<len; ++i) { data[k++]=s[i]; }
00287             if (s[i]) wblog(FL,"ERR wbstring::cat out of bounds (%d/%d)",k,len);
00288         }
00289         data[k]=0; return k;
00290     };
00291
00292     wbstring toStr(char level=0) const {
00293         if (data) {
00294             if (isPrint()>level)
00295                 { return *this; }
00296             else { return wbvector<char>::toStr(); }
00297         }
00298         else { return wbstring("(null)"); }
00299     };
00300
00301     char& operator[] (unsigned i) const { return data[i]; };
00302
00303     unsigned strlen_() const {
00304         return (data && data[0] ? strlen(data) : 0); };
00305
00306     bool isEmpty() const { return (data && data[0] ? 0 : 1); };
00307
00308     char isPrint() const {
00309         if (data) { unsigned i=0, n=0;
00310             for (; i<len && data[i]; ++i) {
00311                 if (!isprint(data[i])) { return 0; }}
00312             for (; i<len; ++i) { if (data[i]) {
00313                 if (isprint(data[i])) { ++n; } else { return 0; }
00314             }}
00315             return (n ? 2 : 1);
00316         }
00317         return 0;
00318     };
00319
00320     bool operator! () const { return (data && data[0] ? 0 : 1); }
00321     explicit operator bool() const { return (data && data[0] ? 1 : 0); };
00322
00323     unsigned isName(int L=-1) const {
00324         unsigned i, n=strlen(data);
00325         if (L>=0 && L<(int)n) return 0;
00326         for (i=0; i<n; i++) if (!isalnum(data[i])) return 0;
00327         return n;
00328     };

```

```

00329
00330     int operator== (const char *s) const {
00331         return (strcmp(data, s)==0);
00332     };
00333     int operator!= (const char *s) const {
00334         return (strcmp(data, s)!=0);
00335     };
00336
00337     int operator== (const wbststring &s) const {
00338         return strcmp(data, s.data)==0;
00339     };
00340     int operator!= (const wbststring &s) const {
00341         return strcmp(data, s.data)!=0;
00342     };
00343     int operator< (const wbststring &s) const {
00344         return strcmp(data, s.data)<0;
00345     };
00346     int operator<= (const wbststring &s) const {
00347         return strcmp(data, s.data)<=0;
00348     };
00349     int operator> (const wbststring &s) const {
00350         return strcmp(data, s.data)>0;
00351     };
00352     int operator>= (const wbststring &s) const {
00353         return strcmp(data, s.data)>=0;
00354     };
00355
00356     int firstLocation (const wbststring &, unsigned k=0);
00357     int firstLocation (const char*, unsigned offset=0);
00358
00359     std::istream & getline (std::istream &istr, char c='\n');
00360
00361     protected:
00362     private:
00363 };
00364
00365 std::istream & wbststring::getline (std::istream &istr, char c) {
00366     const unsigned maxlen=256;
00367     char tmp[maxlen];
00368     istr.getline(tmp, maxlen-1, c);
00369
00370     init(strlen(tmp)+1); strcpy (data, tmp);
00371
00372     return istr;
00373 }
00374
00375 int wbststring::firstLocation (const wbststring &s, unsigned k) {
00376     return firstLocation(s.data,k);
00377 }
00378
00379 int wbststring::firstLocation (const char *s, unsigned offset) {
00380     int i, j, n=strlen(data), m=strlen(s);
00381
00382     for (j=0, i=offset; i<n && j<m; i++, j++) {
00383         if (data[i] != s[j]) {
00384             i-=j;
00385             j=-1;
00386         }
00387     }
00388
00389     if (j==m) return i-m;
00390     else return -1;
00391 }
00392
00393 namespace Wb {
00394     template<class T>
00395     wbststring num2Str(const T &x, const char *fmt=0, unsigned l=0);
00396
00397     wbststring int2Str(long x, const char *fmt=0, unsigned l=0);
00398     wbststring char2Str(int x, unsigned l=0);
00399
00400     wbststring repStr(const char* s_, unsigned n) {
00401         wbststring s;
00402         return s.initrep(s_,n);
00403     };
00404
00405     class String : public wbvector<char> {
00406     public:
00407
00408         String(unsigned n)
00409             : wbvector<char>(n), ipos(0), fline(NULL) { if (n) { data[n]=0; } };
00410
00411         String(const char *F, int L, unsigned n)
00412             : wbvector<char>(n), ipos(0), fline(NULL) { if (n) { data[n]=0; }
00413             if (F && F[0]) { fline=shortFL(F,L); }
00414             };
00415

```

```

00416     int pushf(const char *fmt, ...);
00417
00418     protected:
00419     private:
00420
00421     unsigned ipos;
00422     const char *fline;
00423 };
00424
00425 };
00426
00427 #include <queue>
00428
00429 class wblogbuf_struct {
00430
00431     public:
00432
00433     wblogbuf_struct() : file(NULL), line(0), text(NULL) {};
00434
00435     wblogbuf_struct(const char *s) : file(NULL), line(0), text(NULL) {
00436         if (s) {
00437             unsigned n=strlen(s);
00438             WB_NEW(text,n+1); strcpy(text,s);
00439         }
00440     };
00441
00442     wblogbuf_struct(const char *F, int L, const char *s)
00443     : file(NULL), line(L), text(NULL) { unsigned i=0,k=0,n;
00444         if (F) {
00445             for (; F[i]; i++) { if (F[i]=='/' || F[i]=='\\') k=i+1; }
00446             n=strlen(F+k); WB_NEW(file,n+1); strcpy(file,F+k);
00447         }
00448         if (s) { n=strlen(s); WB_NEW(text,n+1); strcpy(text,s); }
00449     };
00450
00451     wblogbuf_struct(const wblogbuf_struct &S)
00452     : file(S.file), line(S.line), text(S.text)
00453     ){
00454     };
00455
00456     ~wblogbuf_struct () {
00457         if (file) { WB_DELETE(file); }
00458         if (text) { WB_DELETE(text); }
00459         line=0;
00460     };
00461
00462     void unset() { file=NULL; line=0; text=NULL; };
00463
00464     char *file; int line; char *text;
00465
00466     protected:
00467     private:
00468 };
00469
00470 class wblogbuffer {
00471
00472     public:
00473
00474     void push(const char *F, int L, const char *s) {
00475         wblogbuf_struct S(F,L,s);
00476         buf.push(S); S.unset();
00477     };
00478
00479     void flush() {
00480         while (!buf.empty()) {
00481             wblogbuf_struct S=buf.front();
00482             wblog(
00483                 S.file ? S.file : __FILE__,
00484                 S.line ? S.line : __LINE__, S.text
00485             );
00486             S.unset();
00487             buf.pop();
00488         }
00489         doflush();
00490     };
00491
00492     protected:
00493     private:
00494
00495     queue<wblogbuf_struct> buf;
00496 };
00497
00498     wblogbuffer wblogBuf;
00499
00500 #endif
00501

```

5.62 wbutil.hh

```

00001  /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : QSpace utility routines
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WBUTIL_HCC__
00021 #define __WBUTIL_HCC__
00022
00023 // NB! util.hh cannot use wstring etc. as members of class
00024 // definitions here since wstring.hh etc. need to be fully
00025 // defined (if an object is used as a member of a class,
00026 // the compiler wants to know its full definition, so the
00027 // forward declarations in wlib.h do not suffice)
00028 // Wb, Sep03, 16
00029
00030 int OMP_NUM_THREADS = 0;
00031
00032 namespace Wb {
00033
00034 class Path {
00035 public:
00036
00037     Path() {} ;
00038     Path(const char *s) { init(FL,s); };
00039     Path(const char *F, int L, const char *s) { init(F,L,s); };
00040
00041     int init() { buf.init(); idx.init(); return 0; };
00042
00043     int init(
00044         const char *F, int L, const char *s, const char *delim=":",
00045         unsigned n_=-1
00046     );
00047
00048     int append(
00049         const char *F, int L, const char *s, const char *delim=":"
00050     );
00051
00052     char* operator[] (unsigned i) {
00053         if (i+1>=idx.len) wblog(FL,
00054             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,idx.len-1);
00055         return buf.data+idx[i];
00056     };
00057     const char* operator[] (unsigned i) const {
00058         if (i+1>=idx.len) wblog(FL,
00059             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,idx.len-1);
00060         return buf.data+idx[i];
00061     };
00062
00063     const char* last() const {
00064         if (idx.len<2) {
00065             if (!idx.len)
00066                 wblog(FL,"ERR %s() path got uninitialized path !?",FCT);
00067             else wblog(FL,"ERR %s() path got path with idx.len=%d !?",FCT,idx.len);
00068         }
00069         return (*this)[idx.len-2];
00070     };
00071
00072     unsigned numel() const {
00073         if (idx.len<2) {
00074             if (!idx.len) wblog(FL,
00075                 "ERR %s() got uninitialized Path (%d,%d)",FCT,idx.len,buf.len);
00076             else wblog(FL,"ERR %s() got Path with idx.len=%d",FCT,idx.len);
00077         }
00078         return (idx.len-1);
00079     };
00080
00081     Wb::iterator<Path,char*> begin() {
00082         return Wb::iterator<Path,char*>(this);
00083     };
00084     Wb::citerator<Path,char*> cbegin() const {
00085         return Wb::citerator<Path,char*>(this);

```

```

00086     };
00087
00088     void print(const char *istr="" const;
00089
00090     wbsting buf;
00091     wbvector<unsigned> idx;
00092
00093     protected:
00094     private:
00095
00096 };
00097
00098 void Path::print(const char *istr) const {
00099     unsigned i=0, n=numel();
00100
00101     if (istr && istr[0]) wblog(FL,"==> Contents of %s%N",istr);
00102     for (; i<n; ++i) { PRINTF(" %4d. %s\n",i+1,(*this)[i]); }
00103     PRINTF("\n");
00104 };
00105
00106 int Path::init(
00107     const char *F, int L, const char *s, const char *delim,
00108     unsigned n_
00109 ){
00110
00111     if (!s || !s[0]) {
00112         if (int(n_)>0) buf.init(n_); else buf.init();
00113         idx.init(); return 0;
00114     }
00115
00116     if (!delim || !delim[0]) wblog(F_L,
00117         "ERR %s() invalid delim '%s' !?",FCT,delim?delim:"(null)");
00118
00119     unsigned i,m0,m=0, n=strlen(s); char *b,*b0;
00120
00121     if (int(n_)<0) { buf=s; }
00122     else {
00123         if (n<n) wblog(F_L,"ERR %s() "
00124             "input path length out of bounds %d/%d !?",FCT,n,n_);
00125         buf.init(n_); strcpy(buf.data,s);
00126     }
00127
00128     b0=b=buf.data;
00129
00130     while ((b0=strsep(&b,delim)) { ++m; }
00131
00132     idx.init(m+1); m0=m; b=buf.data;
00133
00134     for (m=i=0; i<=n; ++i) {
00135         idx[m++]=i;
00136
00137         if (!b[i]) continue;
00138
00139         do { ++i; } while (b[i] && i<=n);
00140     }
00141
00142     if (m!=m0) wblog(FL,"ERR %s() m=%d/%d !?",FCT,m,m0);
00143     idx[m]=n;
00144
00145     if (idx.last()>=buf.len) wblog(FL,
00146         "ERR %s() index out of bounds (%d/%d) !?",FCT,idx.last(),buf.len);
00147     return m;
00148 };
00149
00150 int Path::append(
00151     const char *F, int L, const char *s, const char *delim
00152 ){
00153     if (!idx.len) { return init(F,L,s,delim); }
00154
00155     if (!s || !s[0]) { buf.init(); idx.init(); return 0; }
00156     if (!delim || !delim[0]) wblog(FL,
00157         "ERR %s() invalid delim '%s' !?",FCT,delim?delim:"(null)");
00158
00159     unsigned i,l=idx.len-1,m0,m=0, n=strlen(s);
00160     unsigned i0=idx[l]+1;
00161     char *b=buf.data+i0, *b0=b;
00162
00163     if (i0+n>=buf.len) wblog(FL,
00164         "ERR %s() index out of bounds (%d+%d/%d) !?",FCT,i0,n,buf.len);
00165     strcpy(b0,s);
00166
00167     while ((b0=strsep(&b,delim)) { ++m; }
00168
00169     idx.Resize(idx.len+m);
00170     m0=m; b=buf.data+i0;
00171
00172     for (m=i=0; i<=n; ++i) {

```



```

00173         idx[l+(m++)]=i0+i;
00174
00175         if (!b[i]) continue;
00176
00177         do { ++i; } while (b[i] && i<=n);
00178     }
00179
00180     if (m!=m0) wblog(FL,"ERR %s() m=%d/%d !?",FCT,m,m0);
00181     idx[l+m]=n;
00182
00183     if (idx.last()>=buf.len) wblog(FL,
00184         "ERR %s() index out of bounds (%d/%d) !?",FCT,idx.last(),buf.len);
00185
00186     return m;
00187 };
00188
00189 double getTimeNow() {
00190
00191     struct timespec now;
00192     int e=clock_gettime(CLOCK_REALTIME,&now);
00193     if (e<0) wblog(FL,"ERR %s() gettimeofday returned error (e=%d)",FCT,e);
00194
00195     return (now.tv_sec + (now.tv_nsec/1e9));
00196 };
00197
00198 template<class T>
00199 void getPerms(wbMatrix<T> &PP,unsigned r, bool l2r=0) {
00200     if (r<=2) {
00201         if (!r) { PP.init(); } else
00202         if (r==1) { PP.init(1,1); }
00203         else {
00204             const T pp[4] = {0, 1, 1, 0};
00205             PP.init(2,2,pp);
00206         }; return;
00207     }
00208     else if (r>12) {
00209         double N=1, i=0; for (; i<r; ++i) { N*=(i+1); }
00210         wblog(FL,"ERR %s() n=%d exceeds bounds (=> %.4g perms)",FCT,r,N);
00211     }
00212
00213     unsigned i,j,l,n,p,k,k_,b=0; size_t N=1;
00214
00215     for (i=0; i<r; ++i) { N*=size_t(i+1); }
00216     PP.init(N,r);
00217
00218     if (l2r) {
00219         for (i=0; i<r; ++i) { PP(0,i)=PP(1,i)=i; };
00220         SWAP(PP(1,0),PP(1,1)); l=2;
00221
00222         for (p=2; p<r; ++p) { n=1;
00223             for (j=0; j<n; ++j, ++b) {
00224                 for (k=0; k<p; ++k, ++l) {
00225                     k_=( b%2 ? k: p-k-1 );
00226
00227                     if (l>=N) wblog(FL,"ERR %s() index out of bounds %d/%d",FCT,l,N);
00228                     for (i=0; i<k_; ++i) { PP(1,i)=PP(j,i); }; PP(1,i)=p;
00229                     for (++i; i<=p; ++i) { PP(1,i)=PP(j,i-1); }
00230                     for ( ; i<r; ++i) { PP(1,i)=i; }
00231                 }}
00232             }
00233         }
00234     else {
00235         for (i=0; i<r; ++i) { PP(0,i)=PP(1,i)=i; };
00236         SWAP(PP(1,i-2),PP(1,i-1));
00237         l=2;
00238
00239         for (p=r-3; p<r; --p) { n=1;
00240             for (j=0; j<n; ++j, ++b) {
00241                 for (k=p+1; k<r; ++k, ++l) {
00242                     k_=( b%2 ? p+r-k : k);
00243
00244                     if (l>=N) wblog(FL,"ERR %s() index out of bounds %d/%d",FCT,l,N);
00245                     for (i=0; i<p; ++i) { PP(1,i)=i; }
00246                     for ( ; i<k_; ++i) { PP(1,i)=PP(j,i+1); }; PP(1,i)=p;
00247                     for (++i; i<r; ++i) { PP(1,i)=PP(j,i); }
00248                 }}
00249             }
00250         }
00251     if (l!=N) wblog(FL,"WRN %s() index mismatch %d/%d",FCT,l,N);
00252 };
00253
00254 class counter {
00255 public:
00256     counter() : n(0) {};
00257
00258     counter(const char *F, int L, const char *fmt_) : n(0) {
00259         file=F; line=L; fmt=fmt_;

```

```

00260     };
00261
00262     ~counter() { if (n) {
00263         if (!fmt) wblog(FL, "ERR %s() got undefined fmt", FCT);
00264         unsigned n=128; char s[n];
00265         snprintf(s, n, fmt.data, n);
00266         if (file)
00267             { wblog(file.data, line, "--> %s", s); }
00268         else { wblog(FL,                "--> %s", s); }
00269     } };
00270
00271     counter& operator++() { ++n; return *this; }
00272
00273     long n;
00274     int line;
00275     wstring file, fmt;
00276
00277     protected:
00278     private:
00279 };
00280
00281 };
00282
00283 #endif

```

5.63 wbvector.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbvector (template vector class)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SIMPLE_VECTOR_CC__
00021 #define __WB_SIMPLE_VECTOR_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 template <class T>
00027 wbvector<T>& wbvector<T>::set(const wbvector<size_t> &IO, const T &x) {
00028     const size_t *I=IO.data;
00029     for (size_t i=0; i<IO.len; ++i) {
00030         if (I[i]>=len) wblog(FL,
00031             "ERR %s() index out of bounds (%d/%d)", FCT, I[i], len);
00032         data[I[i]]=x;
00033     }
00034     return *this;
00035 };
00036
00037 template <class T>
00038 wbvector<T>& wbvector<T>::add(const wbvector<size_t> &IO, const T &x) {
00039     const size_t *I=IO.data;
00040     for (size_t i=0; i<IO.len; ++i) {
00041         if (I[i]>=len) wblog(FL,
00042             "ERR %s() index out of bounds (%d/%d)", FCT, I[i], len);
00043         data[I[i]]+=x;
00044     }
00045     return *this;
00046 };
00047
00048 template <class T>
00049 bool wbvector<T>::anyUnequal(
00050     const wbvector<size_t> &IO, const T &x, size_t *k
00051 ) const {
00052
00053     const size_t *I=IO.data;
00054     for (size_t i=0; i<IO.len; ++i) {
00055         if (I[i]>=len) wblog(FL,
00056             "ERR %s() index out of bounds (%d/%d)", FCT, I[i], len);

```

```

00057         if (data[I[i]]!=x) { if (k) (*k)=I[i]; return 1; }
00058     }
00059     return 0;
00060 };
00061
00062 template <class T>
00063 bool wbvector<T>::anyEqual(
00064     const wbvector<size_t> &I0, const T &x, size_t *k
00065 ) const {
00066
00067     const size_t *I=I0.data;
00068     for (size_t i=0; i<I0.len; ++i) {
00069         if (I[i]>=len) wblog(FL,
00070             "ERR %s() index out of bounds (%d/%d)",FCT,I[i],len);
00071         if (data[I[i]]==x) { if (k) (*k)=I[i]; return 1; }
00072     }
00073     return 0;
00074 };
00075
00076 template <class T>
00077 wbvector<T>& wbvector<T>::Set(
00078     const wbvector<size_t> &I, const wbvector<T> &v
00079 ){
00080     if (!I.isUnique()) wblog(FL,"ERR index not unique");
00081     if (I.len!=v.len) wblog(FL,
00082         "ERR length mismatch (%d/%d)",I.len,v.len);
00083
00084     for (size_t j,i=0; i<I.len; i++) { j=I[i];
00085         if (j>=len) wblog(FL,
00086             "ERR index out of bounds (%d: %d/%d)",i,j,len);
00087
00088         data[j]=v[i];
00089     }
00090     return *this;
00091 };
00092
00093 template <class T>
00094 bool wbvector<T>::isUnique() const {
00095     wbvector v(*this); wbsperm p; Wb::hpsort(v,p);
00096     for (size_t i=1; i<v.len; ++i) if (v[i]==v[i-1]) return 0;
00097     return 1;
00098 };
00099
00100 template <>
00101 bool wbvector<double>::isNormal() const {
00102     for (size_t i=0; i<len; ++i)
00103         if (data[i] && !isnormal(data[i])) return 0;
00104     return 1;
00105 };
00106
00107 template <>
00108 bool wbvector<float>::isNormal() const {
00109     for (size_t i=0; i<len; ++i)
00110         if (data[i] && !isnormal(data[i])) return 0;
00111     return 1;
00112 };
00113
00114 template <> bool wbvector<unsigned>::isNormal() const { return 1; };
00115 template <> bool wbvector<size_t> >::isNormal() const { return 1; };
00116 template <> bool wbvector<int> >::isNormal() const { return 1; };
00117 template <> bool wbvector<char> >::isNormal() const { return 1; };
00118 template <> bool wbvector<long> >::isNormal() const { return 1; };
00119
00120 template <class T>
00121 T wbvector<T>::normDiff2(const wbvector<T> &v, T fac) const {
00122     if (len!=v.len) wblog(FL,
00123         "ERR %s() vector length mismatch (%d/%d)",FCT,len,v.len);
00124     return Wb::rangeNormDiff2(data,v.data,len,fac);
00125 };
00126
00127 template <class T>
00128 double wbvector<T>::normDiff2_(const wbvector<T> &v, double fac) const {
00129     double n2=0; unsigned i=0;
00130
00131     if (len!=v.len) wblog(FL,
00132         "ERR %s() vector length mismatch (%d/%d)",FCT,len,v.len);
00133
00134     if (fac==1) {
00135         for (; i<len; ++i) { n2+=data[i].normDiff2(v.data[i]); }
00136     }
00137     else if (fac!=0) { T x;
00138         for (; i<len; ++i) {
00139             x=v.data[i]; x*=fac; n2+=data[i].normDiff2(x);
00140         }
00141     }
00142     return n2;
00143 };

```

```

00144
00145 template <class T>
00146 T wbvector<T>::cumsum_(wbvector<T> &S, char xflag) const {
00147     T Stot=0;
00148
00149     S.init( xflag? len+1 : len);
00150     if (len>1) {
00151         size_t i=0, l=S.len-1; T *x=S.data;
00152         for (; i<l; ++i) { x[i+1]=x[i]+data[i]; }
00153         Stot=(xflag ? x[l] : x[l]+data[l]);
00154     }
00155     else if (len) {
00156         if (xflag) { S[l]=data[0]; }
00157         Stot=data[0];
00158     }
00159     return Stot;
00160 };
00161
00162 template <class T>
00163 T wbvector<T>::Cumsum(char zflag) const {
00164     if (!len) return 0;
00165     if (zflag) {
00166         T q,x=data[0]; data[0]=0;
00167         for (size_t i=1; i<len; ++i) {
00168             q=data[i]; data[i]=data[i-1]+x; x=q;
00169         }
00170         return data[len-1]+x;
00171     }
00172     else {
00173         for (size_t i=1; i<len; ++i) { data[i]+=data[i-1]; }
00174         return data[len-1];
00175     }
00176 };
00177
00178 template <class T>
00179 template <class T2>
00180 T wbvector<T>::cumsum0prod(wbvector<T2> &b, wbvector<T> &cs, char xflag) const {
00181     size_t i=0, l=(xflag ? len+1 : len);
00182     if (b.len!=len) wblog(FL,
00183         "ERR %s() got length mismatch (%d/%d)",FCT,len,b.len);
00184     cs.init(l); if (!len) return 0;
00185
00186     for (--l; i<l; ++i) { cs[i+1]=cs[i]+data[i]*b.data[i]; }
00187     return (xflag ? cs[i] : cs[i]+data[i]*b.data[i]);
00188 };
00189
00190 template <class T>
00191 wbvector<T>& wbvector<T>::Eldiv(const wbvector<T> &v) const {
00192     if (len!=v.len) wblog(FL,
00193         "ERR %s() size mismatch (%d/%d)",FCT,len,v.len);
00194     for (size_t i=0; i<len; ++i) {
00195         data[i] /= v.data[i];
00196         if (v.data[i]==0) wblog(FL,
00197             "ERR %s() div/0 !? (%g)",FCT,double(data[i])
00198         );
00199     }
00200     return *this;
00201 };
00202
00203 template <class T>
00204 wbvector<T>& wbvector<T>::tensorProd(
00205     const wbvector<T> &v2, wbvector<T> &vv) const {
00206
00207     if (&vv==this || &vv==&v2) {
00208         wbvector<T> x;
00209         return this->tensorProd(v2,x).save2(vv);
00210     }
00211
00212     vv.init(len*v2.len);
00213     if (vv) {
00214         size_t i, j=0, l=0;
00215         const T *d2=v2.data; T *d=vv.data;
00216
00217         for (; j<v2.len; ++j) {
00218             for (i=0; i<len; ++i, ++l) { d[l]=data[i]*d2[j]; }
00219         }
00220         return vv;
00221     };
00222
00223 template <class T>
00224 wbvector<T>& wbvector<T>::RevertSigns() {
00225
00226     size_t i=0;
00227     for (; i<len; ++i) { if (data[i]) break; }
00228     if (i<len) {
00229         if (data[i]>0 && (-data[i]>0)) wblog(FL,"ERR %s() "
00230             "got unsigned data type %s",FCT,TSTR(T));

```

```

00231         for (; i<len; ++i) data[i]=-data[i];
00232     }
00233     return *this;
00234 };
00235
00236 template <class T>
00237 void wbvector<T>::selectSU(const WBINDEX &I) {
00238     size_t i,n;
00239     WBINDEX mark(len);
00240     for (i=0; i<I.len; i++) {
00241         if (I[i]>=len)
00242             wblog(FL,"ERR index out of bounds (%d/%d).", i,len);
00243         mark[I[i]]++;
00244     }
00245     for (n=i=0; i<mark.len; i++)
00246         if (mark[i]) mark[n++]=i;
00247     mark.Resize(n);
00248     Select(mark);
00249 };
00250
00251
00252 template <class T>
00253 wbvector<T>& wbvector<T>::select(
00254     const WBINDEX &I, wbvector<T> &a) const {
00255     a.init(I.len); if (!I.len) { return a; }
00256     for (size_t i=0; i<a.len; ++i) {
00257         if (I[i]>=len) wblog(FL,
00258             "ERR %s() index out of bounds (%d/%d)",FCT,I[i],len);
00259         a.data[i]=data[I[i]];
00260     }
00261     return a;
00262 };
00263
00264
00265 template <class T>
00266 void wbvector<T>::select(const WBINDEX &I, T* r) const {
00267     if (I.len==0) return;
00268     for (size_t i=0; i<I.len; i++) {
00269         if (I[i]>=len) wblog(FL,
00270             "ERR Index out of bounds (%d/%d).", I[i],len);
00271         r[i]=data[I[i]];
00272     }
00273 };
00274
00275
00276 template <class T>
00277 wbvector<T>& wbvector<T>::Select(const WBINDEX &I) {
00278     if (isref) wblog(FL,"ERR must not resize vector reference!");
00279     size_t i; T* d0=data;
00280     if (I.len==0) { init(); return *this; }
00281     WB_NEW(data,I.len);
00282     for (i=0; i<I.len; ++i) {
00283         if (I[i]>=len) wblog(FL,
00284             "ERR index out of bounds (%d/%d)", I[i],len);
00285         data[i]=d0[I[i]];
00286     }
00287     len=I.len;
00288     WB_DELETE(d0); return *this;
00289 };
00290
00291
00292 template <class T>
00293 wbvector<T>& wbvector<T>::BlockSelect(
00294     const WBINDEX &I, const WBINDEX &D
00295 ) {
00296     WBINDEX dc;
00297     size_t i,j,l,n; T *d0=data, *d;
00298     if (isref) wblog(FL,"ERR shall not resize vector reference!");
00299     if (I.len==0) { init(); return *this; }
00300     for (n=i=0; i<I.len; i++) {
00301         j=I[i]; if (j>=D.len) wblog(FL,
00302             "ERR index out of bounds (%d/%d).",j,D.len);
00303         n+=D[j];
00304     }
00305     i=D.cumsum_(dc);
00306     if (i!=len) wblog(FL,"ERR severe size mismatch (%d/%d; %d)",i,len,n);
00307     WB_NEW(data,n); len=n;
00308 }

```

```

00318     for (n=l=i=0; i<I.len; ++i, l+=n) { j=I[i];
00319         d=d0+dc[j]; n=D[j];
00320         for (j=0; j<n; j++) data[l+j]=d[j];
00321     }
00322
00323     WB_DELETE(d0); return *this;
00324 };
00325
00326 template <class T>
00327 template <class T1, class T2, class T3, class T4>
00328 wbvector<T>& wbvector<T>::Cat(
00329     const wbvector<T1> *v1, const wbvector<T2> *v2,
00330     const wbvector<T3> *v3, const wbvector<T4> *v4
00331 ){
00332     return Cat(
00333         v1 ? v1->data : (T1*)0, v1 ? v1->len : 0,
00334         v2 ? v2->data : (T2*)0, v2 ? v2->len : 0,
00335         v3 ? v3->data : (T3*)0, v3 ? v3->len : 0,
00336         v4 ? v4->data : (T4*)0, v4 ? v4->len : 0
00337     );
00338 };
00339
00340 template <class T>
00341 template <class T1, class T2, class T3, class T4>
00342 wbvector<T>& wbvector<T>::Cat(
00343     const T1* v1, size_t l1, const T2* v2, size_t l2,
00344     const T3* v3, size_t l3, const T4* v4, size_t l4
00345 ){
00346     wbvector<T> X(l1+l2+l3+l4);
00347     size_t i; T *x=X.data;
00348
00349     for (i=0; i<l1; ++i) { x[i]=T(v1[i]); }; x+=l1;
00350     for (i=0; i<l2; ++i) { x[i]=T(v2[i]); }; x+=l2;
00351     for (i=0; i<l3; ++i) { x[i]=T(v3[i]); }; x+=l3;
00352     for (i=0; i<l4; ++i) { x[i]=T(v4[i]); };
00353
00354     return X.save2(*this);
00355 };
00356
00357 template <class T>
00358 wbvector<T>& wbvector<T>::Cat(const wbvector<wbvector<T> > &vv) {
00359     size_t i,n=0; T *d;
00360
00361     for (i=0; i<vv.len; i++) n+=vv[i].len;
00362     init(n); d=data;
00363
00364     for (i=0; i<vv.len; i++) {
00365         memcpy(d,vv[i].data,sizeof(T)*vv[i].len); d+=vv[i].len;
00366     }
00367     return *this;
00368 };
00369
00370 template <class T>
00371 wbvector<T>& wbvector<T>::Cat(const wbvector<wbvector<T>* > &vv) {
00372     size_t i,n=0; T *d;
00373
00374     for (i=0; i<vv.len; i++) n+=(vv[i] ? vv[i]->len : 0);
00375     init(n); d=data;
00376
00377     for (i=0; i<vv.len; i++) { if (vv[i]) {
00378         const wbvector<T> &v=*vv[i];
00379         memcpy(d,v.data,sizeof(T)*v.len); d+=v.len;
00380     } }
00381     return *this;
00382 };
00383
00384 template <class T>
00385 void wbvector<T>::info(const char *istr) const {
00386
00387     size_t l=0, n=32; char sstr[n];
00388     size_t b=len*sizeof(T);
00389     wbsting tstr;
00390
00391     if (typeid(T)==typeid(double)) tstr="double";
00392     else tstr.init(TSTR(T));
00393
00394     if (b<(1<10)) l=snprintf(sstr,n,"%ld ", b); else
00395     if (b<(1<20)) l=snprintf(sstr,n,"%3g kB",b/double(1<10)); else
00396         l=snprintf(sstr,n,"%3g MB",b/double(1<20));
00397     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00398
00399     printf(" %12s %10ld %12s @ %p %s vector%s\n",
00400         istr, len, sstr, (void*)data, tstr.data, isref ? " ***ISREF***" : "");
00401 }
00402
00403 template <class T>
00404 void wbvector<T>::print(const char *istr, char mflag) const {

```

```

00405
00406     mxArray *a;
00407
00408     if (mflag==1) {
00409         char fmt[8];
00410
00411         if (typeid(T)==typeid(double))
00412             strcpy(fmt, "%8g");
00413         else strcpy(fmt, "%4d");
00414
00415         if (istr[0]) printf("%s:", istr);
00416
00417         for (size_t i=0; i<len; i++) printf(fmt, data[i]);
00418         printf("%s\n", isref ? " ***ISREF***" : "");
00419
00420         return;
00421     }
00422
00423     a=mxCreateDoubleMatrix(1, len, mxREAL);
00424
00425     double *d=mxGetDoubles(a);
00426     for (size_t i=0; i<len; i++) { d[i]=(double)data[i]; }
00427
00428     if (!mflag) {
00429         if (istr[0])
00430             { wb_printf("\n%s = [1x%d double]\n\n", istr, len); }
00431         else { wb_printf("\n"); }
00432     }
00433     else {
00434         wb_printf("\n%s = [%s\n",
00435             istr[0]? istr:"ans", isref? " *isref*:"");
00436     }
00437
00438     Wb::CallMatlab(0, NULL, 1, &a, "disp");
00439
00440     if (mflag>1) wb_printf("];\n");
00441
00442     mxDestroyArray(a);
00443 };
00444
00445 template <class T> inline
00446 wbindex& wbvector<T>::find(const T &x, wbindex &I, char iflag) const {
00447
00448     if (!len) { I.init(); }
00449     else {
00450         size_t i=0, l=0;
00451         for (; i<len; ++i) { if (data[i]==x) ++l; }
00452
00453         if (!iflag) { I.init(l); if (I.len) { l=i=0;
00454             for (; i<len; ++i) { if (data[i]==x) I[l++]=i; }}
00455         }
00456         else { I.init(len-l); if (I.len) { l=i=0;
00457             for (; i<len; ++i) { if (data[i]!=x) I[l++]=i; }}
00458         }
00459     }
00460     return I;
00461 };
00462
00463 template <class T> inline
00464 wbindex& wbvector<T>::find(const T &x, wbindex &I, wbindex &Ix) const {
00465
00466     if (!len) { I.init(); Ix.init(); }
00467     else {
00468         size_t i=0, l=0, k=0;
00469         for (; i<len; ++i) { if (data[i]==x) ++l; }
00470         I.init(l); Ix.init(len-l); l=i=0;
00471         for (; i<len; ++i) { if (data[i]==x) I[l++]=i; else Ix[k++]=i; }
00472     }
00473     return I;
00474 };
00475
00476 template <class T> inline
00477 wbindex& wbvector<T>::findGT(const T &x, wbindex &I, char iflag) const {
00478
00479     if (!len) { I.init(); }
00480     else {
00481         size_t i=0, l=0;
00482         for (; i<len; ++i) { if (data[i]>x) ++l; }
00483
00484         if (!iflag) { I.init(l); if (I.len) { l=i=0;
00485             for (; i<len; ++i) { if (data[i]>x) I[l++]=i; }}
00486         }
00487         else { I.init(len-l); if (I.len) { l=i=0;
00488             for (; i<len; ++i) { if (!(data[i]>x)) I[l++]=i; }}
00489         }
00490     }
00491     return I;

```

```

00492 };
00493
00494 template <class T> inline
00495 wbindex& wbvector<T>::findGT(const T &x, wbindex &I, wbindex &Ix) const {
00496     if (!len) { I.init(); Ix.init(); }
00497     else {
00498         size_t i=0, l=0, k=0;
00499         for (; i<len; ++i) { if (data[i]>x) ++l; };
00500         I.init(l); Ix.init(len-l); l=i=0;
00501         for (; i<len; ++i) { if (data[i]>x) I[l++]=i; else Ix[k++]=i; }
00502     }
00503     return I;
00504 };
00505
00506 template <class T> inline
00507 size_t wbvector<T>::numZeros(T eps) const {
00508     size_t n=0, i=0;
00509     if (!eps) {
00510         for (; i<len; ++i) { if (!data[i]) { ++n; } }
00511     }
00512     else if (eps>0) {
00513         for (; i<len; ++i) { if (ABS(data[i])<eps) { ++n; } }
00514     }
00515     else {
00516         wblog(FL,"WRN %s() got eps=%3g (-> 0) !?",FCT,double(eps));
00517         for (; i<len; ++i) { if (!data[i]) { ++n; } }
00518     }
00519     return n;
00520 };
00521
00522 template <class T> inline
00523 size_t wbvector<T>::numEQ(const T &x) const {
00524     size_t n=0, i=0;
00525     for (; i<len; ++i) { if (data[i]==x) ++n; }
00526     return n;
00527 };
00528
00529 template <class T> inline
00530 size_t wbvector<T>::numGT(const T &x) const {
00531     size_t n=0, i=0;
00532     for (; i<len; ++i) { if (data[i]>x) ++n; }
00533     return n;
00534 };
00535
00536 template <class T> inline
00537 size_t wbvector<T>::numLT(const T &x) const {
00538     size_t n=0, i=0;
00539     for (; i<len; ++i) { if (data[i]<x) ++n; }
00540     return n;
00541 };
00542
00543 template <class T> inline
00544 wbindex& wbvector<T>::findRange(
00545     const T &x1, const T &x2, wbindex &I, const char *w
00546 ) const {
00547     if (len==0) { I.init(); return I; }
00548
00549     bool i1=1,i2=1; size_t k=0;
00550     if (w) {
00551         if ((w[0]!='[' && w[0]!='']') || (w[1]!='[' && w[1]!='']') || w[2])
00552             wblog(FL,"ERR invalid interval specification '%s'",w);
00553         i1=(w[0]=='[');
00554         i2=(w[1]==']');
00555     }
00556     I.init(len);
00557     for (size_t i=0; i<len; i++) { const T &x=data[i];
00558         if (x<x1 || x>x2 || (x==x1 && !i1) || (x==x2 && !i2)) continue;
00559         I[k++]=i;
00560     }
00561     if (k) I.len=k; else I.init();
00562     return I;
00563 };
00564
00565 template <class T>
00566 size_t wbvector<T>::findValues(const wbvector<T> &B0, wbindex &Ia) const {
00567     size_t l=0,i=0,j=0, m=len, n=B0.len;
00568     char c;
00569     wbvector<T> A(*this), B(B0);
00570     wperm P1,P2; wbindex I;
00571     T *a, *b;
00572

```



```

00579     if (A.isEmpty() || B.isEmpty()) { Ia.init(); return 0; }
00580
00581     A.Sort(P1);
00582     B.Sort(P2); I.init(m); a=A.data; b=B.data;
00583
00584     while (i<m && j<n) { c=NUMCMP(a[i],b[j]);
00585         if (c<0) i++; else
00586             if (c>0) j++;
00587             else {
00588                 do { I[l++]=i++; } while (i<m && a[i-1]==a[i]);
00589                 do { j++; } while (j<n && b[j-1]==b[j]);
00590             }
00591     }
00592
00593     if (l) I.len=l; else I.init();
00594
00595     P1.get(I,Ia);
00596
00597     return Ia.len;
00598 }
00599
00600 template <class T> inline
00601 wbvector<T>& wbvector<T>::getI(size_t k, wbvector<T> &v) const {
00602
00603     if (k>=len) wblog(FL,
00604         "ERR Index out of bounds (%d/%d) !?",k,len);
00605
00606     v.init(len-l);
00607     for (size_t l=0, i=0; i<len; i++) if (i!=k) v.data[l++]=data[i];
00608     return v;
00609 };
00610
00611 template <class T> inline
00612 wbvector<T>& wbvector<T>::getI(
00613     const wbindex &I, wbvector<T> &v, char uflag
00614 ) const {
00615
00616     wbindex J; I.invert(len,J,uflag);
00617     return select(J,v);
00618 };
00619
00620 template <class T> inline
00621 wbvector<T>& wbvector<T>::initI(
00622     const wbvector<T> &a, const wbindex &ia,
00623     const wbvector<T> &b, const wbindex &ib, char uflag
00624 ){
00625     if (!a.len && !b.len) {
00626         if (ia.len || ib.len) wblog(FL,
00627             "ERR %s() index out of bounds (%d/%d)",FCT,ia.len,ib.len);
00628         return init();
00629     }
00630
00631     size_t i=0, j=0, na=a.len, nb=b.len, n=na+nb;
00632     char ma[n], *mb=ma+na; memset(ma,0,n*sizeof(char));
00633
00634     if (uflag) {
00635         for (; i<ia.len; ++i) { j=ia[i];
00636             if (j>=na) wblog(FL,
00637                 "ERR %s() index out of bounds (%d/%d)",FCT,j+1,a.len);
00638             if ((++ma[j])>1) wblog(FL,
00639                 "ERR %s() index not unique (%d/%d)",FCT,j+1,a.len);
00640             };
00641         }
00642         for (i=0; i<ib.len; ++i) { j=ib[i];
00643             if (j>=nb) wblog(FL,
00644                 "ERR %s() index out of bounds (%d/%d)",FCT,j+1,b.len);
00645             if ((++mb[j])>1) wblog(FL,
00646                 "ERR %s() index not unique (%d/%d)",FCT,j+1,b.len);
00647             };
00648         }
00649     na-=ia.len; nb-=ib.len;
00650 }
00651 else {
00652     for (; i<ia.len; ++i) { j=ia[i];
00653         if (j>=a.len) wblog(FL,
00654             "ERR %s() index out of bounds (%d/%d)",FCT,j+1,a.len);
00655         if ((++ma[j])==1) --na; else ma[j]=1;
00656     }
00657     for (i=0; i<ib.len; ++i) { j=ib[i];
00658         if (j>=nb) wblog(FL,
00659             "ERR %s() index out of bounds (%d/%d)",FCT,j+1,b.len);
00660         if ((++mb[j])==1) --nb; else mb[j]=1;
00661     }
00662 }
00663
00664     init(na+nb);
00665

```

```

00666     if (len) { j=0;
00667         if (na) {
00668             for (i=0; i<a.len; ++i) { if (!ma[i]) { data[j]=a[i]; ++j; } }
00669         }
00670         if (nb) {
00671             for (i=0; i<b.len; ++i) { if (!mb[i]) { data[j]=b[i]; ++j; } }
00672         }
00673         if (j!=len) wblog(FL,"ERR %s() %d/%d !?",FCT,j,len);
00674     }
00675     return *this;
00676 };
00677
00678 template <class T>
00679 template <class T2>
00680 wbvector<T>& wbvector<T>::initT(
00681     const char *F, int L, const wbvector<T2> &v) {
00682     RENEW(v.len);
00683     for (size_t i=0; i<len; ++i) {
00684         data[i]=T(v.data[i]);
00685         if (T2(data[i])!=v.data[i]) wblog(F_L,
00686             "ERR %s() got rounded value: %g (%s) -> %g (%s)",
00687             FCT, double(data[i]), TSTR(T2),
00688             double(v.data[i]), TSTR(T)
00689         );
00690     }
00691     return *this;
00692 };
00693
00694 template <class T>
00695 template <class TD> inline
00696 wbvector<T>& wbvector<T>::initT(const wbspararray<TD> &S) {
00697     if (!S.isVector(FL)) wblog(FL,
00698         "ERR %s() invalid sparse vector (%s)",FCT,S.sizeStr().data);
00699     size_t i=0, j=0, n=S.IDX.dim1, m=S.IDX.dim2;
00700     const SPIDX_T *idx=S.IDX.data;
00701     for (; i<S.SIZE.len; ++i) { if (S.SIZE[i]>1) { idx+=i; break; } }
00702     RENEW(S.numel());
00703     for (i=0; i<n; ++i, idx+=m) { j=(*idx);
00704         if (j>=len) wblog(FL,
00705             "ERR %s() index out of bounds (%d/%d)",FCT,j+1,len);
00706         data[j]=T(S.D.data[i]);
00707     }
00708     return *this;
00709 };
00710
00711 bool mxIsWbvector(
00712     const char *F, int L, const mxArray *a,
00713     size_t *n_, const char *istr, char dflag
00714 ){
00715     size_t n=0;
00716     if (a && (n=mxGetNumberOfElements(a))) { int i=0;
00717         if (istr && istr[0])
00718             i=Mx::IsDblVector(0,0,a,0,dflag);
00719         else i=Mx::IsDblVector(F,L,a,0,dflag);
00720         if (!i) {
00721             if (F) wblog(F,L,
00722                 "ERR %s() %s%snumeric vector required\n%s (%s; %d)", FCT,
00723                 istr?istr:"", istr?" ":"", str, mxGetClassName(a), n);
00724             return 0;
00725         }
00726     }
00727     if (n_) { (*n_)=n; }
00728     return n;
00729 };
00730
00731 template <class T>
00732 int wbvector<T>::init_base(
00733     const char *F, int L,
00734     const mxArray *a, const char *istr, char tcheck, char ref
00735 ){
00736     size_t n=0;
00737     if (!a || !mxIsWbvector(F_L,a,&n,istr,'*')) {
00738         init(); return 1;
00739     }
00740 }

```

```

00753     Mx::Array<T> A(a);
00754     if (ref && A.data)
00755         { init2ref(n,A.data); }
00756     else { init(n); A.copyTo(data,tcheck); }
00757
00758     if (data && !Wb::is_finite(data,1)) wblog(FL,
00759         "WRN %s() encountered nan or inf in mex input data",FCT);
00760
00761     return 0;
00762 };
00763
00764 #ifdef QS_USING_MPFR
00765
00766 template<>
00767 int wbvector<Wb::quad>::init_mpfr(
00768     const char *F, int L, const mxArray *a, char base,
00769     const char *istr, char tcheck
00770 ){
00771     if (!a) { init();
00772         if (F) wblog(FL,"ERR %s() got null mxArray",FCT);
00773         return -1;
00774     }
00775     if (!mxIsNumChar(a)) { init();
00776         if (F) wblog(FL,
00777             "ERR %s() invalid type `%s'",FCT,mxGetClassName(a));
00778         return -2;
00779     }
00780     if (mxIsEmpty(a)) { init(); return 0; }
00781
00782     if (base<=0 && mxIsWbvector(0,0,a) && mxIsDouble(a)) {
00783         wbvector<double> X; X.init(F_L,a,istr,tcheck);
00784         this->initT(X); return 0;
00785     }
00786
00787     if (base<=0) base=30; else
00788     if (base==1 || base>62) wblog(FL,
00789         "ERR %s() invalid base=%d<%c>\n"
00790         "(interpreted as base for Wb::quad)",FCT,base,base);
00791
00792     unsigned n=0, np=1; int e=0;
00793
00794     unsigned r=mxGetNumberOfDimensions(a);
00795     const size_t *sz=mxGetDimensions(a);
00796
00797     wbvector<char> ss; char *sd=NULL;
00798
00799     if (r!=2) {
00800         if (F) wblog(FL,"ERR invalid rank-%d array",r);
00801         init(); return -3;
00802     }
00803     if (!sz[0] || sz[0]>62 || (sz[0]==1 && (!sz[1] || sz[1]>62))) {
00804         if (F) wblog(FL,"ERR invalid %ldx%ld array",sz[0],sz[1]);
00805         init(); return -4;
00806     }
00807
00808     if (mxIsInt8(a)) {
00809         Mx::Array<char> A(FL,a);
00810         sd=A.data;
00811
00812         if (sz[0]>1)
00813             { n=sz[0]; init(sz[1]); }
00814         else { n=sz[1]; init(sz[0]); }
00815     }
00816     else if (mxIsChar(a) && (sz[0]==1 || sz[1]==1)) {
00817         Mx::Array<mxChar> A(FL,a);
00818
00819         if (sz[0]>1)
00820             { n=sz[0]; init(sz[1]); }
00821         else { n=sz[1]; init(sz[0]); }
00822
00823         ss.initT(n,A.data);
00824         sd=ss.data;
00825     }
00826     else {
00827         MXPut(FL,"Idbg")
00828         .addP(mxDuplicateArray(a),"a").add(wbstring(istr),"istr");
00829         wblog(FL,"ERR %s() invalid input (%s: %d x %d)",
00830             FCT,mxGetClassName(a),sz[0],sz[1]);
00831     }
00832
00833     if (len>=64) {
00834         np=MAX( QSP_NUM_THREADS, OMP_NUM_THREADS );
00835     }
00836
00837     #pragma omp parallel for num_threads(np)
00838     for (size_t k=0; k<len; ++k) { if (!e) {
00839

```

```

00840     const char *sk=sd+k*n; char cx=0;
00841     unsigned i=0, i0=0;
00842
00843     try {
00844         while (i<n && sk[i]!=' ') { ++i; }; i0=i;
00845         while (i<n && sk[i]!=' ' && sk[i]) { ++i; }
00846
00847         if (i0<i && i<n) {
00848             cx=sk[i]; ((char*)sk)[i]=0;
00849             data[k].init_s(F_L,sk+i0,base); ((char*)sk)[i]=cx;
00850         }
00851         else { char s[n+1]; ++e; strncpy(s,sk,n); s[n]=0;
00852             wblog(FL,"... error i=%4ld/%ld: '%s' (%d..%d/%d)",
00853                 k,len,s,i0,i,n);
00854         }
00855     }
00856     catch (...) {
00857         ++e; if (cx) { ((char*)sk)[i]=cx; }
00858     }
00859 }
00860 if (e) wblog(FL,"ERR %s() e=%d",FCT,e);
00861
00862 return 0;
00863 };
00864
00865 template<>
00866 int wbvector<Wb::quad>::init(
00867     const char *F, int L,
00868     const mxArray *a, const char *istr, char tcheck,
00869     char ref __attribute__((unused)))
00870 ){
00871     unsigned base=(tcheck<2 ? 30 : tcheck);
00872
00873     if (base>30) wblog(FL,
00874         "WRN %s() using tcheck=%d as mpfr base !?",FCT,tcheck);
00875
00876     return init_mpfr(F,L,a,base,istr,tcheck);
00877 };
00878
00879 #endif
00880
00881 template <class T>
00882 int wbvector<T>::init_Struct(
00883     const char *F, int L, const mxArray *a, const char *istr, char ref
00884 ){
00885     if (!a || !mxIsStruct(a)) { if (F) wblog(F,L,
00886         "ERR %s() invalid input %s",FCT,istr?istr:"");
00887         init(); return 1;
00888     }
00889
00890     unsigned i=0, n=mxGetNumberOfElements(a);
00891     init(n);
00892
00893     for (; i<n; ++i) { data[i].init(F_L,a,ref,i); }
00894
00895     return 0;
00896 };
00897
00898 template <class T> inline
00899 void wbvector<T>::Sort(char dir) {
00900     wbpem p; Wb::hpsort(*this,p,dir);
00901 };
00902
00903 template <class T> inline
00904 void wbvector<T>::sort(wbvector &v2) const {
00905     wbpem p; v2=(*this); Wb::hpsort(v2,p);
00906 };
00907
00908 template <class T> inline
00909 wbpem& wbvector<T>::pSort(wbpem &p) const {
00910     if (!isSorted()) { wbvector x(*this); Wb::hpsort(x,p); }
00911     else p.init();
00912     return p;
00913 };
00914
00915 template <class T>
00916 inline wbvector<T>& wbvector<T>::Permute(const wbpem &P, char iflag) {
00917     const wperm_t *p=P.data; char q=0;
00918
00919     if (!P.len || (P.len<=len && (q=P.isValidPerm())>1)) { return *this; }
00920
00921     if (P.len!=len) {
00922         wbpem P_; P_.init(P,iflag,len);
00923         return Permute(P_);
00924     }
00925
00926     if (q) {

```

```

00927     size_t i=0; wbvector<T> v(*this);
00928     if (!iflag)
00929         { for (; i<len; ++i) data[i]=v.data[p[i]]; }
00930     else { for (; i<len; ++i) data[p[i]]=v.data[i]; }
00931 }
00932 else if (P.len<12) wblog(FL,
00933     "ERR invalid permutation [%s] (len=%d/%d)",STR(P),P.len,len);
00934 else wblog(FL,"ERR invalid permutation (len=%d/%d)", P.len, len);
00935
00936 return *this;
00937 };
00938
00939 template <class T>
00940 inline wbvector<T>& wbvector<T>::permute(
00941     wbvector<T> &v, const wbperm &P, char iflag
00942 ) const {
00943
00944     if (P.isEmpty()) { v=*this; return v; }
00945     const wperm_t *p=P.data;
00946
00947     if (len!=P.len || !validPerm(P)) {
00948         if (P.len<12) wblog(FL,
00949             "ERR invalid permutation [%s] (%d/%d)",STR(P),len,P.len);
00950         else wblog(FL,"ERR invalid permutation (%d/%d)",len,P.len);
00951     }
00952
00953     if (!P.isIdentityPerm()) { v.init(len);
00954         if (iflag==0) {
00955             for (size_t i=0; i<len; i++) v.data[i]=data[p[i]]; }
00956         else { for (size_t i=0; i<len; i++) v.data[p[i]]=data[i]; }
00957     }
00958     else v=(*this);
00959
00960     return v;
00961 };
00962
00963 template <class T> inline
00964 wbvector<T>& wbvector<T>::blockPermute(
00965     const wbperm &P, wbvector<T> &v, char iflag
00966 ) const {
00967
00968     if (isEmpty() || P.isEmpty() || P.isIdentityPerm()) {
00969         if (P.len && len%P.len) wblog(FL,
00970             "ERR %s() data mismatch (%d mod %d !?)",FCT,len,P.len);
00971         v=*this; return v;
00972     }
00973     if (&v==this) {
00974         wbvector<T> x(*this);
00975         return x.blockPermute(P,v,iflag);
00976     }
00977
00978     if (len%P.len || !validPerm(P)) wblog(FL,
00979         "ERR invalid permutation (%d/%d)", len, P.len);
00980     v.init(len);
00981
00982     size_t i,j, m=len/P.len;
00983     const wperm_t *p=P.data; const T *d0=data; T *d=v.data;
00984
00985     if (iflag==0) {
00986         for (i=0; i<P.len; ++i, d+=m) { d0=data+m*p[i];
00987             for (j=0; j<m; ++j) d[j]=d0[j]; }
00988     }
00989     else {
00990         for (i=0; i<P.len; ++i, d0+=m) { d=v.data+m*p[i];
00991             for (j=0; j<m; ++j) d[j]=d0[j]; }
00992     }
00993
00994     return v;
00995 };
00996
00997 template <class T> inline
00998 int wbvector<T>::blockCompare(
00999     const wbperm &Pa, const wbvector &B, const wbperm &Pb
01000 ) const {
01001
01002     if (len!=B.len ||
01003         (Pa.len && len%Pa.len) || (Pb.len && B.len%Pb.len))
01004         wblog(FL,"ERR %s() got length mismatch (%d@%d, %d@%d)",
01005             FCT,len,Pa.len,B.len,Pb.len
01006         );
01007
01008     if ((!Pa.len && !Pb.len) || Pa.len==len || Pb.len==B.len) {
01009         if (Pa.len) {
01010             if (Pb.len) {
01011                 if (Pa.len!=Pb.len) wblog(FL,
01012                     "ERR %s() got length mismatch (%d@%d, %d@%d)",
01013                     FCT,len,Pa.len,B.len,Pb.len

```

```

01014         );
01015
01016         const wperm_t *pa=Pa.data, *pb=Pb.data;
01017         for (size_t i=0; i<len; ++i) {
01018             if (data[pa[i]]!=B.data[pb[i]]) {
01019                 return (data[pa[i]]<B.data[pb[i]] ? -1 : +1);
01020             }
01021         }
01022     }
01023     else {
01024         const wperm_t *pa=Pa.data;
01025         for (size_t i=0; i<len; ++i) {
01026             if (data[pa[i]]!=B.data[i]) {
01027                 return (data[pa[i]]<B.data[i] ? -1 : +1);
01028             }
01029         }
01030     }
01031 }
01032 else {
01033     if (Pb.len) {
01034         const wperm_t *pb=Pb.data;
01035         for (size_t i=0; i<len; ++i) {
01036             if (data[i]!=B.data[pb[i]]) {
01037                 return (data[i]<B.data[pb[i]] ? -1 : +1);
01038             }
01039         }
01040     }
01041     else {
01042         for (size_t i=0; i<len; ++i) {
01043             if (data[i]!=B.data[i]) {
01044                 return (data[i]<B.data[i] ? -1 : +1);
01045             }
01046         }
01047     }
01048 }
01049 }
01050 else {
01051     if (Pa.len) {
01052         if (Pb.len) {
01053             const wperm_t *pa=Pa.data, *pb=Pb.data;
01054             size_t j, i=0, m=len/Pa.len;
01055             const T *a, *b;
01056
01057             if (Pa.len!=Pb.len) wblog(FI,
01058                 "ERR %s() got invalid permutations (%d@%d, %d@%d)",
01059                 FCT,len,Pa.len,B.len,Pb.len
01060             );
01061
01062             for (; i<Pa.len; ++i) { a=data+m*pa[i]; b=B.data+m*pb[i];
01063                 for (j=0; j<m; ++j) {
01064                     if (a[j]!=b[j]) return (a[j]<b[j] ? -1 : +1);
01065                 }
01066             }
01067         }
01068         else {
01069             const wperm_t *pa=Pa.data;
01070             size_t j, i=0, m=len/Pa.len;
01071             const T *a, *b=B.data;
01072
01073             for (; i<Pa.len; ++i, b+=m) { a=data+m*pa[i];
01074                 for (j=0; j<m; ++j) {
01075                     if (a[j]!=b[j]) { return (a[j]<b[j] ? -1 : +1); }
01076                 }
01077             }
01078         }
01079     }
01080     else {
01081         if (Pb.len) {
01082             const wperm_t *pb=Pb.data;
01083             size_t j, i=0, m=B.len/Pb.len;
01084             const T *a=data, *b;
01085
01086             for (; i<Pb.len; ++i, a+=m) { b=B.data+m*pb[i];
01087                 for (j=0; j<m; ++j) {
01088                     if (a[j]!=b[j]) { return (a[j]<b[j] ? -1 : +1); }
01089                 }
01090             }
01091         }
01092     }
01093 }
01094
01095 return 0;
01096 };
01097
01098 template <class T>
01099 inline int wbvector<T>::set2Group(
01100     const wperm &P, const WBINDEX &D,

```

```

01101     const T* S0,
01102     char iflag
01103 ){
01104     size_t i,j,l,d,n=D.len; int e=0;
01105     wbvector<T> X;
01106
01107     if (!S0 && len!=P.len) wblog(FL,
01108     "ERR %s() severe size mismatch (%d/%d)", FCT, len, P.len);
01109
01110     if (n==0) {
01111         if (P.len) wblog(FL,
01112         "ERR empty object inconsistency (%d,%d)", P.len, D.len);
01113         return 0;
01114     }
01115
01116     if (S0==NULL || S0==data) { save2(X); S0=X.data; }
01117
01118     init(n);
01119
01120     for (l=i=0; i<n; i++, l+=d) { d=D[i];
01121         data[i]=S0[P[l]];
01122
01123         for (j=1; j<d; j++) if (S0[P[l+j]]!=data[i]) {
01124             e++; if (iflag)
01125                 sprintf(str,"%g/%g", (double)data[i], (double)S0[P[l+j]]);
01126         }
01127     }
01128
01129     return e;
01130 }
01131
01132 template<class T>
01133 mxArray* wbvector<T>::mxCreateStruct(unsigned m, unsigned n) const {
01134     return mxCreateCellMatrix(m,n);
01135 }
01136
01137 template<class T>
01138 void wbvector<T>::add2MxStruct(mxArray *S, unsigned i, char tflag) const {
01139     mxSetCell(S,i,toMx(tflag));
01140 }
01141
01142 template<class T>
01143 mxArray* wbvector<T>::mxCreateCell(unsigned m, unsigned n) const {
01144     return mxCreateCellMatrix(m,n);
01145 }
01146
01147 template<class T>
01148 void wbvector<T>::add2MxCell(mxArray *S, unsigned i, char tflag) const {
01149     mxSetCell(S,i,toMx(tflag));
01150 }
01151
01152 template<class T> inline
01153 void wbvector<T>::add2MxStruct(
01154     mxArray *S, const char *vname, char tflag) const {
01155
01156     int fid; mxArray *a;
01157
01158     fid = mxAddField2Scalar(FL,S,vname);
01159
01160     if (!tflag)
01161         a=mxCreateDoubleMatrix(1,len,mxREAL);
01162     else a=mxCreateDoubleMatrix(len,1,mxREAL);
01163
01164     double *d=mxGetDoubles(a);
01165     for (size_t i=0; i<len; ++i) d[i]=(double)data[i];
01166
01167     mxSetFieldByNumber(S,0,fid, a);
01168 };
01169
01170 template<class T> inline
01171 wbsting wbvector<T>::sizeStr() const {
01172     wbsting s;
01173     if (len) { s=toStrf("","x"); } else { s="[]"; }
01174     return s;
01175 };
01176
01177 template<class T> inline
01178 wbsting wbvector<T>::toStr() const { return toStr(-1," "); }
01179
01180 template<class T> inline
01181 wbsting wbvector<T>::toStr(int n, const char *sep) const {
01182
01183     wbsting s(MAX(size_t(32), len*MAX(16,int(n)+6))), fmt;
01184     fmt.init2Fmt((T)0,n);
01185
01186     for (size_t i=0; i<len; ++i) {
01187         if (i>0) { s.push(FL,sep); }

```

```

01188         s.pushf(FL,fmt.data,data[i]);
01189     }
01190     return s;
01191 };
01192
01193 template<> inline
01194 wbsting wvector<wbcomplex>::toStr(int n, const char *sep) const {
01195
01196     wbsting s(MAX((size_t)32, len*MAX(16,abs(n)+6))), fmz, fmd;
01197     fmz.init2Fmt(wbcomplex(0),n); fmd.init2Fmt(double(0),n);
01198
01199     for (size_t i=0; i<len; ++i) { if (i) { s.push(FL,sep); }
01200         if (data[i].r && data[i].i)
01201             s.pushf(FL,fmz.data,data[i].r,data[i].i);
01202         else if (data[i].r)
01203             { s.pushf(FL,fmd.data,data[i].r); }
01204         else { s.pushf(FL,fmd.data,data[i].i).push("i"); }
01205     }
01206     return s;
01207 };
01208
01209 #ifdef QS_USING_MPFR
01210 template<> inline
01211 wbsting wvector<Wb::quad>::toStr(int n, const char *sep) const {
01212
01213     wbsting s(MAX((size_t)32, len*MAX(16,int(n)+6))), fmt;
01214     fmt.init2Fmt((double)0,n);
01215
01216     for (size_t i=0; i<len; ++i) {
01217         if (i>0) { s.push(FL,sep); }
01218         s.pushf(FL,fmt.data,double(data[i]));
01219     }
01220
01221     return s;
01222 };
01223 #endif
01224
01225 template<class T> inline
01226 wbsting wvector<T>::toStrf (
01227     const char *fmt_, const char *sep, unsigned stride, const char *sep2
01228 ) const {
01229
01230     wbsting s(MAX(32U, unsigned(len)*16)), fmt;
01231     unsigned i_, isep=0;
01232
01233     if (fmt_ && fmt_[0])
01234         { fmt=fmt_; }
01235     else { fmt.init2Fmt(T(0)); }
01236
01237     for (unsigned i=0; i<len; ++i) { if (++isep>1) {
01238         s.push (FL,sep); }
01239         s.pushf(FL,fmt.data,data[i]); if (stride) { i_=i+1;
01240             if (!(i_%stride) && i_<len) { s.push(FL,sep2); isep=0; } }
01241     }
01242
01243     return s;
01244 };
01245
01246 template<> inline
01247 wbsting wvector<wbcomplex>::toStrf(
01248     const char *fmt_, const char *sep, unsigned stride, const char *sep2
01249 ) const {
01250
01251     wbsting s;
01252     unsigned w,i=0; char fmt[16], s1[32], flag=1;
01253
01254     snprintf(fmt,16,"%s",fmt_ && fmt_[0] ? fmt_ : "%.4g");
01255     w=2 + sprintf(str,fmt, sqrt(2.))
01256         + sprintf(str,fmt,-sqrt(2.)*1e-20);
01257
01258     s.init(32+(4+i)*len);
01259
01260     for (size_t i=0; i<len; ++i) {
01261         if (i>0) { if (flag) { s.push(FL,sep); } else { flag=1; } }
01262
01263         snprintf(s1,32,"%s",data[i].toStr(fmt).data);
01264         s.pushf(FL,"%*s",w,s1);
01265         if (stride && ((i+1)%stride)==0 && i+1<len) {
01266             s.push(FL,sep2); flag=0;
01267         }
01268     }
01269
01270     return s;
01271 };
01272
01273 #ifdef QS_USING_MPFR
01274

```



```

01275 template<> inline
01276 wbstring wbvector<Wb::quad>::toStrf(
01277     const char *fmt_, const char *sep, unsigned stride, const char *sep2
01278 ) const {
01279
01280     wbvector<double> x; x.init(*this);
01281     return x.toStrf(fmt_, sep, stride, sep2);
01282 };
01283
01284 #endif
01285
01286 #ifdef __WB_MPFR_HH__
01287
01288 template <> inline
01289 mxArray* wbvector< Wb::quad >::toMx(char base) const {
01290
01291     if (base<=0) base=30; else
01292     if (base==1 || base>62) wblog(FL,
01293         "ERR %s() invalid base=%d<%c>\n"
01294         "(interpreted as base for Wb::quad)", FCT, base, base);
01295
01296     unsigned nmax=0, n=0; size_t S[2];
01297     mxArray *a;
01298
01299     wbstring s_; char *s=NULL;
01300     if (len) { data[0].toStr(s_, base); n=s_.len; s=s_.data; }
01301
01302     if (len!=1)
01303     { S[0]=n; S[1]=len; }
01304     else { S[0]=len; S[1]=n; }
01305
01306     if (len==1 && data[0]==Wb::quad(int(double(data[0]))) {
01307         unsigned i=0, l=0;
01308
01309         while (i<n && s[i] && s[i]!='.') { ++i; }
01310         if (i<n && s[i]) { l=i;
01311             for (++i; i<n && s[i]; ++i) { if (s[i]!='0')
01312                 wblog(FL, "ERR %s() expecting integer (%s) !?", FCT, s);
01313             }
01314             s[l]=0; S[1]=l+1;
01315
01316             return a=Mx::Array<mxChar>(2, S).copyFromTR(s);
01317         }
01318     }
01319
01320     char *cdat=NULL;
01321     a=Mx::Array<char>(2, S).Return(cdat);
01322
01323     if (!len) { return a; }
01324
01325     unsigned np=1; int e=0;
01326     if (len>=64) {
01327         np=MAX( QSP_NUM_THREADS, OMP_NUM_THREADS );
01328     }
01329
01330     #pragma omp parallel for num_threads(np)
01331     for (size_t k=0; k<len; ++k) { if (!e) {
01332         unsigned i=0, i0=0;
01333         char *sk=cdat+k*n;
01334
01335         try { data[k].toStr(sk, n, base, -1, 'a'); }
01336         catch (...) { ++e; }
01337
01338         while (i<n && s[i]==' ') { ++i; }; i0=i;
01339         while (i<n && s[i]!=' ' && s[i]) { ++i; }
01340
01341         if (i<2 || i0>=i || s[i] || (++i)>=n) { ++e;
01342             char s[n+1]; strncpy(s, sk, n); s[n]=0;
01343             wblog(FL, "... error i=%4ld/%ld: '%s' (%d..%d/%d)",
01344                 k, len, s, i0, i, n);
01345         }
01346
01347         #pragma omp critical (mpfr_collecting_nmax)
01348         { if (nmax<i) nmax=i; }
01349     } }
01350
01351     if (e) wblog(FL, "ERR %s() e=%d", FCT, e);
01352
01353     if (nmax==n) wblog(FL,
01354         "WRN %s<Wb::quad> got n=%d/%d (len=%ld)", FCT, nmax, n, len);
01355     if (nmax>n) wblog(FL,
01356         "ERR %s<Wb::quad> got n=%d/%d (len=%ld)", FCT, nmax, n, len);
01357
01358     return a;
01359 };
01360
01361 #endif

```

```

01362
01363 template <class T>
01364 inline wbvector<T>& wbvector<T>::Move2_FE(
01365     const WBINDEX &I,
01366     char iflag
01367 ){
01368     size_t i,k;
01369     wbvector<char> mark(len);
01370     wbvector<T> x;
01371
01372     if (I.isEmpty()) return *this;
01373     if (!isUniqueIdxSet(I,len)) {
01374         wblog(FL, "ERR Move2_FE Index out of bounds [%s; %d]",
01375             I.toStr().data, len); return *this;
01376     }
01377
01378     for (i=0; i<I.len; i++) mark[I[i]]++;
01379
01380     select(I,x);
01381
01382     if (iflag=='E') {
01383         for (k=i=0; i<len; i++)
01384             if (mark[i]==0) {
01385                 if (k!=i) data[k]=data[i];
01386                 k++;
01387             }
01388         memcpy(data+k, x.data, x.len*sizeof(T));
01389     }
01390     else
01391         if (iflag=='F') {
01392             for (k=i=len-1; int(i)>=0; i--)
01393                 if (mark[i]==0) {
01394                     if (k!=i) data[k]=data[i];
01395                     k--;
01396                 }
01397             memcpy(data, x.data, x.len*sizeof(T));
01398         }
01399     else wblog(FL, "ERR Move2_FE - Invalid flag %c<%d>", iflag, iflag);
01400
01401     return *this;
01402 }
01403
01404 template <class T>
01405 inline wbvector<T>& wbvector<T>::Skip(size_t il) {
01406
01407     if (il>=len) {
01408         wblog(FL, "ERR Skip - index out of range (%d,%d)\n[%s; %d]",
01409             il, len, toStr().data, il); return *this;
01410     }
01411
01412     for (size_t i=il+1; i<len; i++) data[i-1]=data[i];
01413     Resize(len-1);
01414
01415     return *this;
01416 }
01417
01418 template <class T>
01419 inline wbvector<T>& wbvector<T>::Skip(
01420     const WBINDEX &I,
01421     char rflag
01422 ){
01423     size_t i,k;
01424     wbvector<char> mark(len); char *m=mark.data;
01425
01426     for (k=0; k<I.len; k++) {
01427         if (I[k]<len) m[I[k]]++;
01428         else {
01429             wblog(FL, "ERR Skip - index out of range (%d,%d)\n[%s; %d]",
01430                 I[k]+1, len, toStr().data, k+1); return *this;
01431         }
01432     }
01433
01434     for (k=i=0; i<len; ++i) {
01435         if (m[i]==0) { if (k!=i) { data[k]=data[i]; }
01436             ++k;
01437         }
01438     }
01439
01440     if (!rflag) Resize(k);
01441     else {
01442         if (k) len=k; else init();
01443     }
01444
01445     return *this;
01446 };
01447
01448 #endif

```

01449

5.64 wbvector.hh

```

00001  /* -----
00002  * Project : QSpace tensor library (v4.0 pre-release)
00003  * Class   : wbvector (template vector class)
00004  *
00005  * Copyright 2022 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SIMPLE_VECTOR_HCC__
00021 #define __WB_SIMPLE_VECTOR_HCC__
00022
00023 template <class T> class wbvector;
00024
00025 WBINDEX Index(widx_t i1, widx_t i2);
00026 char isUniqueIdxSet(const WBINDEX &I, widx_t imax);
00027 char validPerm(const WBPPerm &P, wperm_t d=-1);
00028 void getIPerm(const WBPPerm &P, WBPPerm &iP);
00029
00030 bool mxIsWbvector(
00031     const char *F, int L, const mxArray *a,
00032     size_t *n=NULL, const char *istr=NULL, char dflag='d');
00033
00034 bool mxIsWbvector(const mxArray *a) {
00035     return mxIsWbvector(0,1,a);
00036 };
00037
00038 template <class T>
00039 class wbvector {
00040
00041 public:
00042
00043     wbvector() : data(NULL), len(0), isref(0) {};
00044
00045     wbvector(size_t l, T x) : data(NULL), len(0), isref(0) {
00046         RENEW(l,NULL,0,0);
00047         set(x);
00048     };
00049
00050     wbvector(size_t l, const T* d0=NULL, char ref=0)
00051         : data(NULL), len(0), isref(0) {
00052         if (ref && d0 && l)
00053             { init2ref(l,d0); }
00054         else { RENEW(l,d0); }
00055     };
00056
00057     wbvector(const wbvector &v)
00058         : data(NULL), len(0), isref(0) { RENEW(v.len, v.data); };
00059
00060     template <class T2>
00061     wbvector(const wbvector<T2> &v)
00062         : data(NULL), len(0), isref(0) { initT(v.len,v.data); }
00063
00064     wbvector(const wbvector &v1, const wbvector &v2)
00065         : data(NULL), len(0), isref(0) {
00066
00067         size_t i, k=0;
00068         RENEW(v1.len+v2.len);
00069
00070         for (i=0; i<v1.len; ++i, ++k) data[k]=v1[i];
00071         for (i=0; i<v2.len; ++i, ++k) data[k]=v2[i];
00072     };
00073
00074     wbvector(const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
00075         : data(NULL), len(0), isref(ref) {
00076         init(F,L,a,NULL,check_type,ref);
00077     };
00078
00079     wbvector(const mxArray *a, char check_type=1, char ref=0)

```

```

00080     : data(NULL), len(0), isref(ref) {
00081         init(FL,a,NULL,check_type,ref);
00082     };
00083
00084     virtual ~wbvector() {
00085         if (!isref && data) { WB_DELETE(data); }
00086     };
00087
00088     wbvector& init(size_t l=0) { return RENEW(l); };
00089     wbvector& init(size_t l, const T* d0) { return RENEW(l,d0); };
00090
00091     wbvector& init(size_t l, const T* d0, char ref) {
00092         if (ref && d0 && l)
00093             { return init2ref(l,d0); }
00094         else { return RENEW(l,d0); }
00095     };
00096
00097     wbvector& initp(size_t l, const T* d0, size_t n,
00098         char init=1
00099     ){ return RENEW(l,d0,n,init); };
00100
00101     wbvector& init2val(size_t l, const T& x) {
00102         return RENEW_VAL(l,x); };
00103
00104     wbvector& initStride(size_t l, const T* d0, size_t stride) {
00105         RENEW(l,NULL,0,0);
00106         for (size_t i=0; i<len; ++i, d0+=stride) { data[i]=*d0; }
00107         return *this;
00108     };
00109
00110     wbvector& init_stride(
00111         const T* d0, size_t n, size_t m, size_t D0
00112     ){
00113         RENEW(n*m);
00114         Wb::cpyStride(data, d0, n, m, -1, D0);
00115         return *this;
00116     };
00117
00118     wbvector& init(const wbvector &v) {
00119         RENEW(v.len);
00120         for (size_t i=0; i<len; ++i) data[i]=v.data[i];
00121         return *this;
00122     };
00123
00124     wbvector& init(size_t n, const wbvector &v) {
00125         RENEW(n); if (n>v.len) n=v.len;
00126         for (size_t i=0; i<n; ++i) data[i]=v.data[i];
00127         return *this;
00128     };
00129
00130     wbvector& init_data(const T *d0) {
00131         for (size_t i=0; i<len; ++i) { data[i]=d0[i]; }
00132         return *this;
00133     };
00134
00135     template<class T0>
00136     wbvector& init_data(const T0 *d0) {
00137         for (size_t i=0; i<len; ++i) { data[i]=T(d0[i]); }
00138         return *this;
00139     };
00140
00141     template<class T0>
00142     wbvector& init_data(const char *F, int L, const T0 *d0) {
00143         for (size_t i=0; i<len; ++i) { data[i]=T(d0[i]);
00144             if (T0(data[i])!=d0[i]) wblog(FL,
00145                 "ERR type conversion changes value (%g->%g) !?",
00146                 double(d0[i]), double(data[i])
00147             );
00148         }
00149         return *this;
00150     };
00151
00152     wbvector<T>& init(const wbspararray<T> &S) { return initT(S); };
00153
00154     template <class T2>
00155     wbvector<T>& initT(const wbspararray<T2> &S);
00156
00157     template <class T2>
00158     wbvector<T>& initT(size_t l, const T2* v) {
00159         RENEW(l);
00160         for (size_t i=0; i<len; ++i) data[i]=T(v[i]);
00161         return *this;
00162     };
00163
00164     template <class T2>
00165     wbvector<T>& initT(const wbvector<T2> &v) {
00166         RENEW(v.len);

```

```

00167         for (size_t i=0; i<len; ++i) data[i]=T(v.data[i]);
00168         return *this;
00169     };
00170
00171     template <class T2>
00172     wbvector<T>& initT(const wbvector<T2> &v) {
00173         RENEW(v.len);
00174         for (size_t i=0; i<len; ++i) data[i]=T(v.data[i], '!');
00175         return *this;
00176     };
00177
00178     template <class T2>
00179     wbvector<T>& initT(const char *F, int L, const wbvector<T2> &v);
00180
00181     wbvector& init2ref(const wbvector &v) {
00182         return init2ref(v.len, v.data);
00183     };
00184
00185     wbvector& init2ref(size_t l, const T* d0) {
00186         if (l && !d0) wblog(FL, "ERR %s() got null ref (n=%d)", FCT, l);
00187         if (data && !isref) { RENEW(0); }
00188         isref=1; len=l; data=(len ? (T*)d0 : NULL);
00189         return *this;
00190     };
00191
00192     wbvector& unRef();
00193
00194     wbvector& init(size_t l1, const T* d1, size_t l2, const T* d2) {
00195         RENEW(l1+l2); MEM_COPY<T>(data, len, l1, d1, d2);
00196         return *this;
00197     };
00198
00199     int init(
00200         const char *F, int L, const mxArray *a, const char *istr=NULL,
00201         char check_type __attribute__((unused)) =1,
00202         char ref=0
00203     ) { return init_Struct(F, L, a, istr, ref); };
00204
00205     int init_Struct(
00206         const char *F, int L, const mxArray *a,
00207         const char *istr=NULL, char ref=0
00208     );
00209
00210     int init_base(
00211         const char *F, int L, const mxArray *a,
00212         const char *istr=NULL, char check_type=1, char ref=0
00213     );
00214
00215     int init(
00216         const mxArray *a, const char *istr=NULL, char check_type=1
00217     ){ return init(0, 0, a, istr, check_type); };
00218
00219     int init_mpfr(
00220         const char *F, int L, const mxArray *a, char base=-1,
00221         const char *istr="", char tcheck=1
00222     );
00223
00224     void initDef(size_t n) {
00225         if (data) { WB_DELETE(data); }
00226         len=n; if (n) { WB_NEW(data, n); }
00227     };
00228
00229     void initDef(const wbvector &v) {
00230         if (len!=v.len) initDef(v.len);
00231         for (size_t i=0; i<v.len; ++i) data[i]=v[i];
00232     };
00233
00234     wbvector& operator= (const wbvector &v) {
00235         if (this!=&v) { RENEW(v.len, v.data); }
00236         return *this;
00237     };
00238
00239     template<class TB>
00240     wbvector<T>& operator= (const wbvector<TB> &v) {
00241         init(v.len);
00242         for (size_t i=0; i<v.len; ++i) { data[i]=T(v.data[i]); }
00243         return *this;
00244     };
00245
00246     wbvector& set(const T &x) {
00247         for (size_t i=0; i<len; ++i) { data[i]=x; }
00248         return *this;
00249     };
00250
00251     wbvector& setMin(const T &x) {
00252         for (size_t i=0; i<len; ++i) { if (data[i]<x) data[i]=x; }
00253         return *this;

```

```

00254     };
00255     wbvector& setMax(const T &x) {
00256         for (size_t i=0; i<len; ++i) { if (data[i]>x) data[i]=x; }
00257         return *this;
00258     };
00259
00260     wbvector& set(const wbvector<size_t> &I, const T &x);
00261     wbvector& add(const wbvector<size_t> &I, const T &x);
00262
00263     wbvector& Set(const wbvector<size_t> &I, const wbvector &v);
00264
00265     void Sort(wbperm &p, char dir=+1) { Wb::hpsort(*this,p,dir); };
00266     void sort(wbvector &v2, wbperm &p) const { v2=(*this); Wb::hpsort(v2,p); };
00267
00268     void Sort(char dir=+1);
00269     void sort(wbvector &v2) const;
00270
00271     wbperm& pSort(wbperm &p) const;
00272
00273     bool isSorted() const {
00274         for (size_t i=1; i<len; ++i) if (data[i-1]> data[i]) return 0;
00275         return 1;
00276     };
00277     bool isSortedU() const {
00278         for (size_t i=1; i<len; ++i) if (data[i-1]>=data[i]) return 0;
00279         return 1;
00280     };
00281
00282     bool isUnique() const;
00283
00284     int contains(const T& x) const {
00285         for (size_t i=0; i<len; ++i) { if (data[i]==x) return 1; }
00286         return 0;
00287     };
00288
00289     bool isNormal() const;
00290
00291     bool isFinite() const {
00292         return Wb::is_finite(data,len);
00293     };
00294
00295     size_t findI(const T& x) const {
00296         for (size_t i=0; i<len; ++i) { if (data[i]==x) return i; }
00297         return -1;
00298     };
00299     size_t findI() const {
00300         for (size_t i=0; i<len; ++i) { if (data[i]) return i; }
00301         return -1;
00302     };
00303
00304     size_t count(const T& x) const {
00305         size_t i=0, n=0;
00306         for (; i<len; ++i) if (data[i]==x) ++n;
00307         return n;
00308     };
00309
00310     size_t icount(const T& x) const {
00311         size_t i=0,n=0;
00312         for (; i<len; ++i) if (data[i]!=x) ++n;
00313         return n;
00314     };
00315
00316     size_t nnz() const {
00317         size_t i=0, n=0;
00318         for (; i<len; ++i) if (data[i]) ++n;
00319         return n;
00320     };
00321     size_t nnz(T eps) const {
00322         if (eps==0) { return nnz(); }
00323         else { return len-numZeros(eps); }
00324     };
00325
00326     size_t numZeros(T eps=T(1E-14)) const;
00327     size_t numEQ(const T&) const;
00328     size_t numGT(const T&) const;
00329     size_t numLT(const T&) const;
00330
00331     wbindex& find(const T &x, wbindex &I, char iflag=0) const;
00332     wbindex& find(const T &x, wbindex &I, wbindex &Ix) const;
00333     wbindex& findGT(const T &x, wbindex &I, char iflag=0) const;
00334     wbindex& findGT(const T &x, wbindex &I, wbindex &Ix) const;
00335
00336     wbindex& findzeros(wbindex &I) const { return find(0,I); }
00337     wbindex& find(wbindex &I) const { return find(0,I,'i'); }
00338
00339     wbindex& find(wbindex &I, wbindex &Ix) const {
00340         find(0,Ix,I); return I; }

```

```

00341
00342     wbvector<size_t> find() const {
00343         wbvector<size_t> I;
00344         return find(I);
00345     };
00346
00347     wbindex& findRange(
00348         const T &x1, const T &x2, wbindex &I, const char *w=NULL
00349     ) const;
00350
00351     int findClosestSorted(T r) const;
00352     int findClosest(T r) const;
00353
00354     size_t findValues(const wbvector &v, wbindex &I) const;
00355
00356     wbvector& setp(const T *x) {
00357         for (size_t i=0; i<len; ++i) { data[i]=x[i]; }
00358         return *this;
00359     };
00360
00361     void set(size_t offset, const T *x, size_t n) {
00362         if (offset+n>=len) wblog(FL,
00363             "ERR index out of bounds (%d+%d; %d)",offset,n,len);
00364         MEM_COPY<T>(data+offset,n,x);
00365     };
00366
00367     void reset() { if (len) MEM_SET<T>(data,len); };
00368
00369     void cpy(const T* v) {
00370         if (len) MEM_COPY<T>(data,len,v);
00371     };
00372
00373     void cpy(const T *x, size_t m) {
00374         if (m>len) { wblog(FL,"ERR index out of bounds (%d/%d)",m,len); m=len; }
00375         if (m) MEM_COPY<T>(data,m,x);
00376     };
00377
00378     wbvector& save2(wbvector &v) {
00379         swap(v); if (data!=v.data) { init(); }
00380         return v;
00381     };
00382
00383     void swap(wbvector &v) {
00384         if (this!=&v) {
00385             SWAP(len, v.len );
00386             SWAP(data, v.data );
00387             SWAP(isref, v.isref);
00388         }
00389     };
00390
00391     wbvector& swap(size_t i, size_t j) {
00392         if (i<len && j<len)
00393             { T x=data[i]; data[i]=data[j]; data[j]=x; }
00394         else { wblog(FL,
00395             "ERR %s() index out of bounds (%d,%d/%d)",FCT,i+1,j+1,len); }
00396         return *this;
00397     };
00398
00399     wbvector& Collect(T& A1, T& A2) {
00400         init(2); A1.save2(data[0]); A2.save2(data[1]);
00401         return *this;
00402     };
00403
00404     wbvector& Collect(T& A1, T& A2, T& A3) {
00405         init(3); A1.save2(data[0]); A2.save2(data[1]); A3.save2(data[2]);
00406         return *this;
00407     };
00408
00409     wbvector& Skip(size_t);
00410     wbvector& Skip(const WBINDEX &, char rflag=0);
00411
00412     wbvector& Move2_FE (const WBINDEX &I, char FE_flag);
00413     wbvector& Move2front(const WBINDEX &I) { return Move2_FE(I,'F'); }
00414     wbvector& Move2end (const WBINDEX &I) { return Move2_FE(I,'E'); }
00415
00416     wbvector& Move2front (size_t k) {
00417         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00418         T xk=data[k];
00419         for (size_t i=k; i>0; i--) data[i]=data[i-1];
00420         data[0]=xk; return *this;
00421     }
00422
00423     wbvector& Move2end (size_t k) {
00424         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00425         T xk=data[k];
00426         for (size_t i=k+1; i<len; i--) data[i-1]=data[i];
00427         data[len-1]=xk; return *this;

```

```

00428     }
00429
00430     wbvector& mvLastTo (size_t k) {
00431         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00432         T xk=data[len-1];
00433         for (size_t i=len-1; i>k; i--) data[i]=data[i-1];
00434         data[k]=xk; return *this;
00435     }
00436
00437     wbvector& mvFirstTo (size_t k) {
00438         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00439         T xk=data[0];
00440         for (size_t i=0; i<k; i++) data[i]=data[i+1];
00441         data[k]=xk; return *this;
00442     }
00443
00444     bool checkSameLength(
00445         const wbvector &a,
00446         const char *F=NULL, int L=0, const char *istr=""
00447     ) const;
00448
00449     bool checkSameLength(
00450         const wbvector &a,
00451         const wbvector &b,
00452         const char *F=NULL, int L=0, const char *istr=""
00453     ) const;
00454
00455     template <class T_>
00456     void operator+= (const T_ &a) {
00457         if (a==T(0)) return;
00458         for (size_t i=0; i<len; ++i) data[i]+=a;
00459     };
00460
00461     template <class T_>
00462     void operator-= (const T_ &a) {
00463         if (a==T(0)) return;
00464         for (size_t i=0; i<len; ++i) data[i]-=a;
00465     };
00466
00467     template <class T_>
00468     wbvector& operator*= (const T_ &a) {
00469         if (len && a!=T_(1)) {
00470             if (isref) wblog(FL,"ERR %s() reference is considered const",FCT);
00471             if (!a) { MEM_SET<T>(data,len); }
00472             else { size_t i=0;
00473                 if (a==T_(-1))
00474                     for (; i<len; ++i) { data[i]=-data[i]; }
00475                 else for (; i<len; ++i) { data[i]*=a; }
00476             }
00477         }
00478         return *this;
00479     };
00480
00481     template <class T_>
00482     wbvector& operator/= (const T_ &a) {
00483         if (len && a!=T_(1)) { size_t i=0;
00484             if (isref) wblog(FL,"ERR %s() reference is considered const",FCT);
00485             if (!a) wblog(FL,"ERR got div/0");
00486
00487             if (a==T_(-1)) { for (; i<len; ++i) data[i]=-data[i]; } else
00488             if (len<4 || WbUtil<T>::isInt()) { for (; i<len; ++i) data[i]/=a; }
00489             else {
00490                 T x=1/T(a);
00491                 if (fabs(double(x*a-1))>1E-12)
00492                     { for (; i<len; ++i) { data[i]/=a; }}
00493                 else { for (; i<len; ++i) { data[i]*=x; }}
00494             }
00495         }
00496         return *this;
00497     };
00498
00499     template <class T_>
00500     wbvector operator* (const T_ &a) const {
00501         wbvector u(*this); if (a!=1) u*=a;
00502         return u;
00503     };
00504
00505     template <class T_>
00506     wbvector operator+ (const T_ &a) const {
00507         wbvector u(*this);
00508         if (a!=0) for (size_t i=0; i<u.len; ++i) { u.data[i]+=a; }
00509         return u;
00510     };
00511
00512     template <class T_>
00513     wbvector operator- (const T_ &a) const {
00514         wbvector u(*this);

```



```

00515         if (a!=0) for (size_t i=0; i<u.len; ++i) { u.data[i]-=a; }
00516         return u;
00517     };
00518
00519     wbvector operator+ (const wbvector &b) const {
00520         wbvector u(*this); checkSameLength(b,FLF);
00521         for (size_t i=0; i<u.len; i++) u.data[i]+=b.data[i];
00522         return u;
00523     };
00524
00525     wbvector operator- (const wbvector &b) const {
00526         wbvector u(*this); checkSameLength(b,FLF);
00527         for (size_t i=0; i<u.len; i++) u.data[i]-=b.data[i];
00528         return u;
00529     };
00530
00531     void operator+= (const wbvector &b) {
00532         checkSameLength(b,FLF);
00533         for (size_t i=0; i<len; ++i) data[i]+=b.data[i];
00534     };
00535
00536     void operator-= (const wbvector &b) {
00537         checkSameLength(b,FLF);
00538         for (size_t i=0; i<len; ++i) data[i]-=b.data[i];
00539     };
00540
00541     wbvector& Plus (const T* b) {
00542         for (size_t i=0; i<len; ++i) data[i]+=b[i];
00543         return *this;
00544     };
00545
00546     wbvector& Plus (const T* b, T bfac) {
00547         size_t i=0;
00548         if (bfac==+1) for (; i<len; ++i) data[i]+=b[i]; else
00549         if (bfac== -1) for (; i<len; ++i) data[i]-=b[i]; else
00550         if (bfac!= 0) for (; i<len; ++i) data[i]+=(bfac*b[i]);
00551         return *this;
00552     };
00553
00554     wbvector& Plus (const wbvector &b, T bfac=1) {
00555         checkSameLength(b,FLF);
00556         return Plus(b.data,bfac);
00557     };
00558
00559     void plus (
00560         const wbvector &b, wbvector &c, T bfac=1, T cfac=0
00561     ) const {
00562         if (cfac) {
00563             checkSameLength(b,c,FLF); c*=cfac; c.Plus(*this); }
00564         else { checkSameLength(b, FLF); c=*this; }
00565
00566         c.Plus(b,bfac);
00567     };
00568
00569     wbvector& times(T fac, wbvector &x) const {
00570         x=*this; x*=fac; return x;
00571     };
00572
00573     wbvector& RevertSigns();
00574
00575     int isum() const {
00576         int s=0; if (len) {
00577             size_t i=1; s=data[0]; for (; i<len; ++i) { s+=data[i]; }
00578         }; return s;
00579     };
00580
00581     T sum() const {
00582         T s=T(); if (len) {
00583             size_t i=1; s=data[0]; for (; i<len; ++i) { s+=data[i]; }
00584         }; return s;
00585     };
00586
00587     T sum(size_t i1, const char *estr) const {
00588         if (!estr || strcmp(estr,"end")) wblog(FL,
00589             "ERR invalid usage (got '%s')", estr);
00590         return sum(i1,len-1);
00591     };
00592
00593     T sum(size_t i1, size_t i2) const {
00594         T s=T();
00595
00596         if (!len) { if (!i1 && !i2) return s; }
00597         if (i1>=len || i2>=len) wblog(FL,
00598             "ERR index out of bounds (%d,%d; %d)",i1,i2,len);
00599         if (i1<=i2) {
00600             size_t i=i1+1; s=data[i1]; for (; i<=i2; ++i) { s+=data[i]; }
00601         }

```

```

00602     return s;
00603 };
00604
00605 T cumsum_(wbvector &S, char xflag=0) const;
00606 T Cumsum(char zflag=0) const;
00607
00608 template <class T2>
00609 T cumsum0prod(wbvector<T2> &b, wbvector<T> &cs, char xflag) const;
00610
00611 wbvector& Eldiv( const wbvector &v) const;
00612
00613 wbvector& tensorProd(const wbvector &v2, wbvector &vv) const;
00614
00615 T scalarProd(const wbvector &v) const {
00616     if (len!=v.len) wblog(FL,
00617         "ERR scalarProd - length mismatch (%d/%d)",len,v.len);
00618
00619     if (len) return scalarProd(v.data);
00620     else return 0;
00621 };
00622
00623 T scalarProd(const T *v) const {
00624     T s=T(); for (size_t i=0; i<len; ++i) s+=(data[i]*CONJ(v[i]));
00625     return s;
00626 };
00627
00628 T norm() const { return SQRT(norm2()); }
00629
00630 T norm2() const {
00631     T s=0; if (len==0) return s;
00632     s=NORM2(data[0]);
00633
00634     for (size_t i=1; i<len; i++) s+=NORM2(data[i]);
00635     return s;
00636 };
00637
00638 double normDiff(const wbvector &v, T fac=1) const {
00639     return std::sqrt(double(normDiff2(v,fac)));
00640 };
00641
00642 T normDiff2 (const wbvector &v, T fac=1) const;
00643 double normDiff2_(const wbvector &v, double fac=1) const;
00644
00645 T GSProject(const T *v, char isnorm=0, char tflag=0) const {
00646     return gs_project_range(data,v,len,1,isnorm,tflag);
00647 };
00648
00649 wbvector& Normalize(const char *F=NULL, int L=0) {
00650     double n=std::sqrt(norm2());
00651     if (n==0) {
00652         if (F) wblog(F,L,"WRN normalize zero vector!? - randomize");
00653         setRand(1,-0.5); return Normalize(F,L);
00654     }
00655     (*this)*=(1./n); return *this;
00656 };
00657
00658 wbvector& add(const T* d) {
00659     for (size_t i=0; i<len; ++i) data[i]+=d[i];
00660     return *this;
00661 };
00662
00663 wbvector& subtract(const T* d) {
00664     for (size_t i=0; i<len; ++i) data[i]-=d[i];
00665     return *this;
00666 };
00667
00668 void getReal(wbvector<double> &R) const;
00669 void getImag(wbvector<double> &I) const;
00670 void set(const wbvector<double> &R, const wbvector<double> &I);
00671 wbvector& Conj() { return *this; };
00672
00673 wbvector& applyFlag(char &flag, const T &fac, wbvector &a);
00674
00675 bool allEqual() const {
00676     for (size_t i=1; i<len; ++i) if (data[i]!=data[0]) return 0;
00677     return 1;
00678 }
00679
00680 bool allZero() const { return allEqual(T(0)); };
00681 bool allOnes() const { return allEqual(T(1)); };
00682
00683 bool allEqual(const T &a) const {
00684     bool q=1; size_t i=0;
00685     if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
00686     for (; i<len; ++i) { if (data[i]!=a) { q=0; break; } }
00687     return q;
00688 }

```

```

00689
00690     bool allEqual2(const T &a, T eps=1E-15) const {
00691         if (!eps) { return allEqual(a); }
00692         else {
00693             T x, x2=0; if (a!=0) eps*=a;
00694             if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
00695             for (size_t i=0; i<len; ++i) { x=data[i]-a; x2+=(x*x); }
00696             return (ABS(x2)<eps);
00697         }
00698     };
00699
00700     bool allUnequal(const T &a) const {
00701         bool q=1; size_t i=0;
00702         if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
00703         for (; i<len; ++i) { if (data[i]==a) { q=0; break; } }
00704         return q;
00705     }
00706
00707     bool any() const { bool q=0; size_t i=0;
00708         for (; i<len; ++i) { if (data[i]) { q=1; break; } }
00709         return q;
00710     }
00711
00712     bool anyUnequal(const T &a) const {
00713         bool q=0; size_t i=0;
00714         for (; i<len; ++i) { if (data[i]!=a) { q=1; break; } }
00715         return q;
00716     }
00717
00718     bool anyEqual(const T &a, size_t *il=NULL) const {
00719         bool q=0; size_t i=0;
00720         for (; i<len; ++i) { if (data[i]==a) {
00721             if (il) { (*il)=i; }
00722             q=1; break;
00723         } }
00724         return q;
00725     };
00726
00727     bool anyGT(const T &a) const {
00728         bool q=0; size_t i=0;
00729         for (; i<len; ++i) { if (data[i]> a) { q=1; break; } }
00730         return q;
00731     };
00732     bool anyGE(const T &a) const {
00733         bool q=0; size_t i=0;
00734         for (; i<len; ++i) { if (data[i]>=a) { q=1; break; } }
00735         return q;
00736     };
00737
00738     bool anyLT(const T &a) const {
00739         bool q=0; size_t i=0;
00740         for (; i<len; ++i) { if (data[i]< a) { q=1; break; } }
00741         return q;
00742     };
00743     bool anyLE(const T &a) const {
00744         bool q=0; size_t i=0;
00745         for (; i<len; ++i) { if (data[i]<=a) { q=1; break; } }
00746         return q;
00747     };
00748
00749     bool allGT(const T &a) const { return !anyLE(a); }
00750     bool allGE(const T &a) const { return !anyLT(a); }
00751     bool allLE(const T &a) const { return !anyGT(a); }
00752     bool allLT(const T &a) const { return !anyGE(a); }
00753
00754     bool allIn(const T &a1, const T &a2) const {
00755         bool q=1; size_t i=0;
00756         if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
00757         for (; i<len; ++i) { if (data[i]<a1 || data[i]>a2) { q=0; break; } }
00758         return q;
00759     };
00760
00761     bool anySmallVals(const T eps=1E-15) const {
00762         size_t i=0; if (eps>0)
00763             { for (; i<len; ++i) { if (ABS(data[i])<eps) return 1; } }
00764         else { for (; i<len; ++i) { if (data[i]) return 1; } }
00765         return 0;
00766     };
00767
00768     bool isEqual(const T* v) const {
00769         return (!memcmp(data, v, len*sizeof(T)));
00770     }
00771
00772     bool anyEqual(const wbvector<size_t> &I, const T &x,
00773         size_t *k=NULL) const;
00774     bool anyUnequal(const wbvector<size_t> &I, const T &x,
00775         size_t *k=NULL) const;

```

```

00776
00777     bool isEmpty() const {
00778         if ((!len) ^ (!data)) wblog(FL,
00779             "ERR severe data inconsistency (%ld,%p)", len, data);
00780         return (data ? 0 : 1);
00781     };
00782
00783     explicit operator bool() const { return !isEmpty(); }
00784     bool operator! () const { return isEmpty(); }
00785
00786     bool operator==(const wbvector &v) const { return isEqual(v); };
00787     bool operator!=(const wbvector &v) const { return !isEqual(v); };
00788
00789     bool isEqual(const wbvector &v) const {
00790         bool q=1;
00791         if (this!=&v) {
00792             if (len!=v.len) { q=0; } else
00793                 if (data!=v.data) {
00794                     for (size_t i=0; i<len; ++i) {
00795                         if (data[i]!=v.data[i]) { q=0; break; }
00796                     }
00797                 }
00798             }
00799         return q;
00800     };
00801
00802     bool isEqualR(const wbvector &v) const {
00803         if (len!=v.len) return 0;
00804         for (size_t l=len-1, i=0; i<len; ++i) {
00805             if (data[i]!=v.data[l-i]) return 0;
00806         }
00807         return 1;
00808     };
00809
00810     bool operator^(const wbvector &v) const {
00811         if (this==&v) { return (len ? 0 : 1); }
00812         if (len!=v.len) return 0;
00813
00814         for (size_t i=0; i<len; ++i) {
00815             if (!(data[i]==0) ^ (v.data[i]==0)) return 0; }
00816
00817         return 1;
00818     };
00819
00820     bool deepEqualP(const wbvector &B) const;
00821
00822     char cmp(const wbvector &v) const;
00823
00824     bool operator< (const wbvector &v) const { return cmp(v)< 0; };
00825     bool operator<= (const wbvector &v) const { return cmp(v)<=0; };
00826     bool operator> (const wbvector &v) const { return cmp(v)> 0; };
00827     bool operator>= (const wbvector &v) const { return cmp(v)>=0; };
00828
00829     int operator< (const T &r) const {
00830         for (size_t i=0; i<len; ++i) if (data[i]>=r) return 0;
00831         return (len ? 1 : 0);
00832     };
00833
00834     T& el(size_t i) const {
00835         if (i>=len) wblog(FL, "ERR element index out of bounds (%d/%d)", i+1, len);
00836         return data[i];
00837     };
00838
00839     T elx(size_t i, const T &x) const {
00840         return (i<len ? data[i] : x);
00841     };
00842
00843     const T& operator[] (size_t i) const { return data[i]; };
00844     T& operator[] (size_t i) { return data[i]; };
00845
00846     T& last() const {
00847         if (!len) wblog(FL, "ERR %s() got empty %s vector", FCT, TSTR(T));
00848         return data[len-1];
00849     };
00850
00851     T& last(size_t &i) const {
00852         if (!len) wblog(FL, "ERR %s() got empty %s vector", FCT, TSTR(T));
00853         return data[i=len-1];
00854     };
00855
00856     template<class TI>
00857     T& last(TI &i_) const { size_t i=len-1; i_=i;
00858         if (!len) wblog(FL, "ERR %s() got empty %s vector", FCT, TSTR(T));
00859         return data[i];
00860     };
00861
00862     T& p2last() const {

```

```

00863         if (!len) wblog(FL,"ERR %s() got empty %s vector",FCT,TSTR(T));
00864         return data+(len-1);
00865     };
00866
00867     T* ref(size_t i) const { return (data+i); };
00868
00869     wbvector& setRand(double fac=1., double shift=0.);
00870     void setIdx (double shift=0., double fac=1.);
00871
00872     int set2Group(const wbperm &P,
00873                 const WBINDEX &D, const T* S0=NULL, char iflag=0
00874     );
00875
00876     wbvector& Resize_(size_t n) {
00877         if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00878
00879         if (n) {
00880             if (n<=len && (len<32 || (4*n)>len))
00881                 { len=n; }
00882             else { Resize(n); }
00883         }
00884         else init();
00885
00886         return *this;
00887     };
00888
00889     wbvector& Shorten2(size_t n) {
00890         if (!n) {
00891             if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00892             init();
00893         }
00894         else if (n<len && (len<32 || (4*n)>len)) { len=n; }
00895         else if (n!=len) {
00896             if (n>len) wblog(FL,
00897                 "WRN %s() got increasing size %ld -> %ld",FCT,len,n);
00898             Resize(n);
00899         }
00900         return *this;
00901     };
00902
00903     wbvector& Resize(size_t n, const T* d=NULL) {
00904         if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00905
00906         if (n==len) { return *this; }
00907         if (!len) { return RENEW(n,d); };
00908         if (!n) { len=0; if (data) WB_DELETE(data); return *this; }
00909
00910         T *d0=data;
00911         WB_NEW(data,n);
00912         MEM_COPY<T>(data, n, (n<len ? n : len), d0, d);
00913         len=n;
00914         WB_DELETE(d0); return *this;
00915     };
00916
00917     wbvector& resize(size_t n, wbvector& X) const {
00918         if (!n) { X.init(); } else
00919         if (n==len) { X=(*this) ; }
00920         else {
00921             WB_NEW(X.data,n); X.len=n;
00922             if (len) { MEM_COPY<T>(X.data, MIN(n,len), data); }
00923             if (n>len) { MEM_SET<T>(X.data+len,n-len); }
00924         }
00925         return X;
00926     };
00927
00928     wbvector& cat(const wbvector &v2) {
00929         Resize(len+v2.len, v2.data);
00930         return *this;
00931     };
00932     wbvector& Cat(const wbvector &v1, const wbvector &v2) {
00933         RENEW(v1.len+v2.len);
00934         MEM_COPY<T>(data,len,v1.len,v1.data,v2.data);
00935         return *this;
00936     };
00937     wbvector& Cat(const wbvector<wbvector > &vv);
00938     wbvector& Cat(const wbvector<wbvector* > &vv);
00939
00940     template <class T1, class T2, class T3, class T4>
00941     wbvector& Cat(
00942         const T1* v1, size_t l1, const T2* v2, size_t l2,
00943         const T3* v3=0, size_t l3=0,
00944         const T4* v4=0, size_t l4=0
00945     );
00946
00947     template <class T1, class T2, class T3, class T4>
00948     wbvector& Cat (
00949         const wbvector<T1> *v1,const wbvector<T2> *v2,

```

```

00950         const wbvector<T3> *v3=0,
00951         const wbvector<T3> *v4=0
00952     );
00953
00954     wbvector& Append(const wbvector &v2) {
00955         Resize(len+v2.len, v2.data);
00956         return *this;
00957     };
00958     wbvector& Append(const T &x) {
00959         Resize(len+1, &x);
00960         return *this;
00961     };
00962     wbvector& Append(size_t d2, const T *x) {
00963         Resize(len+d2, x);
00964         return *this;
00965     };
00966
00967     wbvector append(const wbvector &v) const {
00968         wbvector v2; append(v.data,v.len,v2);
00969         return v2;
00970     };
00971
00972     wbvector& append(const wbvector &v2, wbvector &vout) const {
00973         append(v2.data, v2.len, vout);
00974         return vout;
00975     };
00976     void append(const T *x, size_t d2, wbvector &vout) const {
00977         vout.RENEW(len+d2);
00978         MEM_COPY<T>(vout.data,vout.len,len,data,x);
00979     };
00980
00981     int recLess (const T* v) const;
00982     int recLessE(const T* v) const;
00983     int recCompare(const T* v) const;
00984
00985     T max() const;
00986     T max(size_t *k) const;
00987     T max(unsigned &k) const { T x;
00988         size_t l=0; x=max(&l); Wb::safeConvert (FL,l,k);
00989         return x;
00990     };
00991
00992     T max_(T x) const;
00993
00994     T min() const;
00995     T min(size_t *k) const;
00996     T min(unsigned &k) const { T x;
00997         size_t l=0; x=min(&l); Wb::safeConvert (FL,l,k);
00998         return x;
00999     };
01000     T min_(T x) const;
01001
01002     T aMax(size_t *k=NULL) const;
01003
01004     T aMax(unsigned &k) const { T x;
01005         size_t k_; x=aMax(&k_); Wb::safeConvert (FL,k_,k);
01006         return x;
01007     };
01008
01009     T aMin(char zflag=0, size_t *k=NULL) const;
01010
01011     T maxneg(size_t *k=NULL) const;
01012     T minpos(size_t *k=NULL) const;
01013
01014     T avg() const;
01015     T std() const;
01016
01017     void range(T &xmin, T&xmax) const;
01018     T range() const;
01019
01020     wbvector& Permute(const wbperm &P, char iflag=0);
01021     wbvector& permute(wbvector &, const wbperm &P, char iflag=0) const;
01022
01023     wbvector permute(const wbperm &P, char iflag=0) const {
01024         wbvector<T> X; permute(X,P,iflag);
01025         return X;
01026     };
01027
01028     wbvector& blockPermute(
01029         const wbperm &P, wbvector &, char iflag=0) const;
01030
01031     wbvector& BlockPermute(const wbperm &P, char iflag=0) {
01032         wbvector<T> x(*this);
01033         return x.blockPermute(P,*this,iflag);
01034     };
01035
01036     int blockCompare(

```

```

01037         const wperm &pa, const wbvector &b, const wperm &pb) const;
01038
01039 void get(const WBINDEX &I0, wbvector &v) const {
01040     const widx_t *I=I0.data;
01041     size_t i=0, n=I0.len; v.init(n);
01042
01043     for (; i<n; i++) {
01044         if (I[i]>=len) wblog(FL,
01045             "ERR index out of bounds (%d/%d) !?",I[i],len);
01046         v.data[i]=data[I[i]];
01047     }
01048 };
01049
01050 void get(const wbvector<size_t> &I, T* v) const {
01051     for (size_t i=0; i<I.len; i++) {
01052         if (I[i]>=len)
01053             wblog(FL,"ERR index out of bounds (%d/%d) !?", I[i],len);
01054
01055         v[i]=data[I[i]];
01056     }
01057     return;
01058 };
01059
01060 void get(size_t i1, size_t i2, wbvector &v) const {
01061
01062     if (i1>=len || i2>=len) wbdie(FL,"index out of range.");
01063     if (i2<i1)
01064         v.init();
01065     else v.init(i2-i1, data+i1);
01066 };
01067
01068 void selectSU(const WBINDEX &I);
01069 void select(const WBINDEX &I, T* r) const;
01070 wbvector& select(const WBINDEX &I, wbvector &a) const;
01071 wbvector& Select(const WBINDEX &I);
01072 wbvector& BlockSelect(const WBINDEX &I, const WBINDEX &D);
01073
01074 wbvector select(const WBINDEX &I) const {
01075     wbvector dI; select(I,dI);
01076     return dI;
01077 };
01078
01079 wbvector& getI(size_t k, wbvector &v) const;
01080 wbvector& getI(const windex &k, wbvector &v, char uflag=1) const;
01081
01082 wbvector& getI(const WBINDEX &k, wbvector &v, char uflag=1) const {
01083     return getI((const windex&)k,v,uflag);
01084 };
01085
01086 wbvector& initI(const wbvector &a, const windex &ia){
01087     return a.getI(ia,*this); };
01088 wbvector& initI(
01089     const wbvector &a, const windex &ia,
01090     const wbvector &b, const windex &ib,
01091     char uflag=1
01092 );
01093
01094 wbvector& Flip() {
01095     size_t i, m2=len/2, l=len-1;
01096     for (i=0; i<m2; i++) SWAP(data[i],data[l-i]);
01097     return *this;
01098 };
01099
01100 void flip(wbvector &a) const {
01101     if (&a==this) { a.Flip(); return; }
01102     a.init(len);
01103     for (size_t l=len-1, i=0; i<len; ++i) a.data[i]=data[l-i];
01104 };
01105
01106 wbvector flip() const {
01107     wbvector a; flip(a);
01108     return a;
01109 };
01110
01111 wbvector operator[] (const wbvector<size_t>& I) const {
01112     wbvector v(I.len);
01113     size_t i=0, *const k=I.data;
01114
01115     for (; i<I.len; ++i) {
01116         if (k[i]>=len) wblog(FL,
01117             "ERR index out of bounds (%d: %d/%d)",i,I.max(),len);
01118         v[i]=data[k[i]];
01119     }
01120
01121     return v;
01122 };
01123

```

```

01124 T prod() const {
01125     if (!len) wblog(FL,"WRN %s() empty data set (returning 0)",FCT);
01126     return prod(0);
01127 };
01128
01129 T prod(T x) const {
01130     if (len) {
01131         size_t i=1; x=data[0]; for (; i<len; ++i) { x*=data[i]; }
01132     }; return x;
01133 };
01134
01135 T prod(size_t i, size_t i2) const {
01136     T x=0;
01137     if (i<len && i<=i2) {
01138         if ((++i2)>len) { i2=len; }
01139         for (x=data[i++]; i<i2; ++i) { x*=data[i]; }
01140     }
01141     else wblog(FL,
01142         "ERR empty index set [%d,%d] / %d (returning 0)",i,i2,len);
01143     return x;
01144 };
01145
01146 T prod(size_t i, size_t i2, T x) const {
01147     if (long(i)<0) { i =len+i; }
01148     if (long(i2)<0) { i2=len+i2; }
01149     if (i<len && i<=i2) {
01150         if ((++i2)>len) { i2=len; }
01151         for (x=data[i++]; i<i2; ++i) { x*=data[i]; }
01152     }
01153     return x;
01154 };
01155
01156 template <class TI>
01157 T prod(const TI *I, size_t n) const {
01158     T x=0; if (n) { size_t i=1;
01159         x=el(I[0]); for (; i<n; ++i) { x*=el(I[i]); }
01160     }
01161     return x;
01162 };
01163
01164 template <class T1, class T2>
01165 T2 prod(const T1 *I, size_t n, T2 &s2) const {
01166     size_t i,k;
01167     char m[len]; for (i=0; i<len; ++i) m[i]=0;
01168     for (i=0; i<n; ++i) { k=I[i];
01169         if (k>len) wblog(FL,
01170             "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
01171         if ((++m[k])>1) wblog(FL,"ERR %s() index not unique",FCT);
01172     }
01173     if (!len) { s2=0; return 0; }
01174     else {
01175         T2 s1=1; s2=1;
01176         for (i=0; i<len; ++i) { (m[i] ? s1 : s2) *= data[i]; }
01177         return s1;
01178     }
01179 };
01180
01181 void prod2(T &p1, T &p2, const T &p0=0) const {
01182     size_t l=len/2; if (len%2) wblog(FL,
01183         "ERR cannot split vector into two equal parts (%d)",len);
01184
01185     if (len==0) { p1=p0; p2=p0; return; }
01186     p1=data[0]; p2=data[1];
01187
01188     for (size_t i=1; i<l; i++) { p1*=data[i]; p2*=data[i+1]; }
01189 }
01190
01191 void info(const char *istr) const;
01192 void print(const char *istr, char mflag=0) const;
01193 void printdata(const char *istr=0) const {
01194     if (istr && istr[0]) printf("%s = ",istr);
01195     printf("[ %s ]\n",STR_(this));
01196 };
01197
01198 void put(
01199     const char *F, int L,
01200     const char *vname="ans", char tflag=0, const char *ws="base"
01201 ) const;
01202
01203 void put(const char *vname="ans", char tflag=0, const char *ws="base"
01204 ) const { put(0,0,vname,tflag,ws); }
01205
01206 void dispWithMsg(const char* file, int line,
01207     const char *msg, const char* vname="") const {
01208     print(vname);
01209     wblog(file,line,msg);
01210 }

```



```

01211
01212 void mat2mx(mxArray* &a, char dim=2) const;
01213 void mat2mxs(mxArray* a, char field_nr, char dim=2) const;
01214
01215 mxArray* toMx_Struct (const char tflag=0) const;
01216 mxArray* toMxP_S(const char tflag=0) const;
01217
01218 mxArray* toMx (const char tflag=0) const { return toMx_Struct (tflag); };
01219 mxArray* toMxP(const char tflag=0) const { return toMxP_S(tflag); };
01220
01221 mxArray* toMx_base(const char tflag=0) const {
01222     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01223     return Mx::Array<double>(m,n).copyFromTR(data);
01224 };
01225
01226 mxArray* toMx_base_d(const char tflag=0) const {
01227     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01228     return Mx::Array<double>(m,n).copyFromTR(data);
01229 };
01230
01231 mxArray* toMx_offset(const T& offset, const char tflag=0) const;
01232
01233 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01234 void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const;
01235 void add2MxStruct(mxArray *S, const char *vname, char tflag=0) const;
01236
01237 mxArray* mxCreateCell(unsigned m, unsigned n) const;
01238 void add2MxCell(mxArray *S, unsigned i, char tflag=0) const;
01239
01240 void putg(const char *vname="ans", char tflag=0) const {
01241     put(vname,tflag,"global");
01242     return;
01243 };
01244
01245 void putx(const char *vname="ans", char tflag=0) const {
01246     put(vname,tflag,"base");
01247     return;
01248 };
01249
01250 wbsting toStr(int n, const char *sep) const;
01251 wbsting toStr() const;
01252
01253 wbsting sizeStr() const;
01254
01255 wbsting toStrf(
01256     const char *fmt0="",
01257     const char *sep=" ",
01258     unsigned stride=0,
01259     const char *sep2=""
01260 ) const;
01261
01262 T *data;
01263
01264 size_t len;
01265
01266 char isref;
01267
01268 protected:
01269
01270 wbvector& RENEW(size_t n, const T* d=NULL, size_t l=-1, char init=1) {
01271     if (data) {
01272         if (isref) { data=NULL; } else
01273             if (n!=len) { WB_DELETE(data); }
01274     }
01275     isref=0; len=n; if (!n) { return *this; }
01276
01277     if (!data) { WB_NEW(data,len); }
01278
01279     if (long(l)<0) { l=len; }
01280
01281     if (d && l) {
01282         MEM_COPY<T>(data,l<len ? l:len,d); if (l<len && init) {
01283             MEM_COPY<T>(data+l,len-l,NULL); }
01284     }
01285     else if (init) {
01286         MEM_COPY<T>(data, len, NULL);
01287     }
01288
01289     return *this;
01290 };
01291
01292 wbvector& RENEW_VAL(size_t n, const T& x) {
01293     if (isref) { data=NULL; isref=0; } else
01294         if (n!=len && data) { WB_DELETE(data); }
01295
01296     len=n;
01297     if (len) {

```

```

01298         if (!data) { WB_NEW(data,len); }
01299         for (size_t i=0; i<len; ++i) { data[i]=x; }
01300     }
01301     return *this;
01302 };
01303
01304 private:
01305
01306 };
01307
01308 template <class T>
01309 wbvector<T>& wbvector<T>::unRef() {
01310
01311     if (!isref) return *this;
01312     if (!len) {
01313         if (data) wblog(FL,"ERR data=%lX (%d)",data,len);
01314         isref=0; return *this;
01315     }
01316
01317     T const* const d0=data;
01318
01319     WB_NEW(data,len); MEM_COPY<T>(data,len,d0);
01320     isref=0;
01321
01322     return *this;
01323 };
01324
01325 template <class T>
01326 bool wbvector<T>::deepEqualP(const wbvector<T> &B) const {
01327     if (len!=B.len) return 0;
01328
01329     if (data!=B.data) {
01330         for (size_t i=0; i<len; i++) { if (data[i]==B.data[i]) continue;
01331             if ((!data[i] || !B.data[i])) return 0; else
01332                 if (!(*data[i]==(*B.data[i]))) return 0;
01333         }
01334     }
01335
01336     return 1;
01337 };
01338
01339 template <class T> inline
01340 char wbvector<T>::cmp(const wbvector<T> &v) const {
01341
01342     size_t i=0, n=MIN(len,v.len); const T *b=v.data;
01343     for (; i<n; ++i) {
01344         if (data[i]<b[i]) return -1;
01345         if (data[i]>b[i]) return +1;
01346     }
01347
01348     if (len<v.len) return -1;
01349     if (len>v.len) return +1; else return 0;
01350 };
01351
01352 template <class T> inline
01353 bool wbvector<T>::checkSameLength(
01354     const wbvector<T> &a,
01355     const char *F, int L, const char *istr
01356 ) const {
01357
01358     bool i=(len==a.len);
01359     if (!i && F) {
01360         if (istr && istr[0]) sprintf(str,"%s() - ",istr); else str[0]=0;
01361         wblog(F,L, "ERR %svector length mismatch (%d/%d)",str,len,a.len);
01362     }
01363     return i;
01364 };
01365
01366 template <class T> inline
01367 bool wbvector<T>::checkSameLength(
01368     const wbvector<T> &a,
01369     const wbvector<T> &b,
01370     const char *F, int L, const char *istr
01371 ) const {
01372
01373     bool i=(len==a.len && len==b.len);
01374     if (!i && F) {
01375         if (istr && istr[0]) sprintf(str,"%s() - ",istr); else str[0]=0;
01376         wblog(F,L, "ERR %svector length mismatch (%d,%d/%d)",str,a.len,b.len,len);
01377     }
01378     return i;
01379 };
01380
01381 template <class T> inline
01382 void wbvector<T>::getReal(wbvector<double> &R) const {
01383     R.init(len);
01384     for (size_t i=0; i<len; ++i) { R.data[i]=double(data[i]); }

```

```

01385 };
01386
01387 template <> inline
01388 void wbvector<wbcomplex>::getReal(wbvector<double> &R) const {
01389     R.init(len);
01390     for (size_t i=0; i<len; ++i) { R.data[i]=data[i].r; }
01391 };
01392
01393 template <class T> inline
01394 void wbvector<T>::getImag(wbvector<double> &I) const {
01395     I.init(len);
01396 };
01397
01398 template <> inline
01399 void wbvector<wbcomplex>::getImag(wbvector<double> &I) const {
01400     I.init(len);
01401     for (size_t i=0; i<len; ++i) { I.data[i]=data[i].i; }
01402 };
01403
01404 template <> inline
01405 void wbvector<wbcomplex>::set(
01406     const wbvector<double> &R, const wbvector<double> &I) {
01407     size_t i=0; const double *re=R.data, *im=I.data;
01409     init(R.len ? R.len : I.len);
01411     if (R.len) {
01413         if (I.len) {
01414             if (R.len!=I.len) wblog(FL,
01415                 "ERR %s() got length inconsistency (%d/%d)",FCT,R.len,I.len);
01416             for (; i<len; ++i) { data[i].init(re[i],im[i]); }
01417         }
01418         else { for (; i<len; ++i) { data[i].init(re[i],0); } }
01419     }
01420     else if (I.len) { for (; i<len; ++i) { data[i].init(0,im[i]); } }
01421 };
01422
01423 template <> inline
01424 wbvector<wbcomplex>& wbvector<wbcomplex>::Conj() {
01425     for (size_t i=0; i<len; ++i) data[i].Conj();
01426     return *this;
01427 };
01428
01429 template <class T> inline
01430 wbvector<T>& wbvector<T>::applyFlag(
01431     char &flag, const T &fac, wbvector<T> &a
01432 ) {
01433     if (!strchr("NCTc",flag))
01434         wblog(FL,"ERR %s() invalid flag %c<%d>",__FUNCTION__,flag,flag);
01435     if (flag=='c') flag='C';
01437     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='N';
01438     if (fac!=1 || flag=='C') { a=(*this);
01440         if (flag=='C') a.Conj();
01441         if (fac!=1) a*=fac;
01442     }
01443     else a<<*this;
01444 };
01445
01446 template <class T> inline
01447 T wbvector<T>::min() const {
01448     if (len) { size_t i=1; T x=data[0];
01450         for (; i<len; ++i) { if (x>data[i]) { x=data[i]; } }
01451         return x;
01452     }
01453     wblog(FL,"ERR %s() got empty vector",FCT);
01454     return 0;
01455 };
01456
01457 template <class T> inline
01458 T wbvector<T>::min_(T x) const {
01459     if (len) { size_t i=1; x=data[0];
01460         for (; i<len; ++i) { if (x>data[i]) { x=data[i]; } }
01461     }
01462     return x;
01463 };
01464
01465 template <class T> inline
01466 T wbvector<T>::min(size_t *k) const {
01467     if (!k) { return min(); }
01468     if (len) { size_t i=1; T x=data[0]; *k=0;
01469         for (; i<len; ++i) { if (x>data[i]) { x=data[i]; *k=i; } }
01470         return x;
01471     }

```

```

01472     wblog(FL,"ERR %s() got empty vector",FCT);
01473     return 0;
01474 };
01475
01476 template <class T> inline
01477 T wbvector<T>::max() const {
01478     if (len) { unsigned i=1; T x=data[0];
01479         for (; i<len; ++i) { if (x<data[i]) { x=data[i]; } }
01480         return x;
01481     }
01482     wblog(FL,"ERR %s() got empty vector",FCT);
01483     return 0;
01484 };
01485
01486 template <class T> inline
01487 T wbvector<T>::max_(T x) const {
01488     if (len) { size_t i=1; x=data[0];
01489         for (; i<len; ++i) { if (x<data[i]) { x=data[i]; } }
01490         return x;
01491     }
01492 };
01493
01494 template <class T> inline
01495 T wbvector<T>::max(size_t *k) const {
01496     if (!k) { return max(); }
01497     if (len) { size_t i=1; T x=data[0]; *k=0;
01498         for (; i<len; ++i) { if (x<data[i]) { x=data[i]; *k=i; } }
01499         return x;
01500     }
01501     wblog(FL,"ERR %s() got empty vector",FCT);
01502     return 0;
01503 };
01504
01505 template <class T> inline
01506 T wbvector<T>::minpos(size_t *k) const {
01507     if (len) {
01508         T x=0; size_t i=0; if (k) { (*k)=-1; }
01509         for (; i<len; ++i) { if (data[i]>0) {
01510             x=data[i]; if (k) { (*k)=i; }; ++i; break;
01511         } }
01512         for (; i<len; ++i) { if (data[i]>0 && x>data[i]) {
01513             x=data[i]; if (k) { (*k)=i; }
01514         } }
01515         return x;
01516     }
01517     wblog(FL,"WRN %s() of empty vector",FCT);
01518     return 0;
01519 };
01520
01521 template <class T> inline
01522 T wbvector<T>::maxneg(size_t *k) const {
01523     if (len) {
01524         T x=0; size_t i=0; if (k) { (*k)=-1; }
01525         for (; i<len; ++i) { if (data[i]<0) {
01526             x=data[i]; if (k) { (*k)=i; }; ++i; break;
01527         } }
01528         for (; i<len; ++i) { if (data[i]<0 && x<data[i]) {
01529             x=data[i]; if (k) { (*k)=i; }
01530         } }
01531         return x;
01532     }
01533     wblog(FL,"WRN %s() of empty vector",FCT);
01534     return 0;
01535 };
01536
01537 template <class T> inline
01538 T wbvector<T>::aMax(size_t *k_) const {
01539     if (len) {
01540         size_t k=-1, i=0; T a,x=0;
01541         for (; i<len; ++i) {
01542             a=ABS(data[i]); if (x<a) { x=a; k=i; }
01543         }
01544         if (k_) (*k_)=k;
01545         return x;
01546     }
01547     else {
01548         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got null vector",FCT); }
01549         return 0;
01550     }
01551 };
01552
01553 template <class T> inline
01554 T wbvector<T>::aMin(char zflag, size_t *k_) const {

```

```

01559
01560     if (!len) {
01561         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got null vector",FCT); }
01562         return 0;
01563     }
01564
01565     size_t i=0, k=-1; double a,x=0;
01566
01567     if (zflag==0) {
01568         x=ABS(data[i++]); k=0; if (x!=0) {
01569             for (; i<len; ++i) { a=ABS(data[i]); if (x>a) {
01570                 x=a; k=i; if (x==0) break;
01571             }}
01572         }
01573     }
01574     else {
01575         for (; i<len; ++i) {
01576             a=ABS(data[i]); if (a) { x=a; k=i; break; }
01577         }
01578         for (; i<len; ++i) { a=ABS(data[i]);
01579             if (a && x>a) { x=a; k=i; }
01580         }
01581     }
01582     if (k_) (*k_)=k;
01583     return x;
01584 };
01585
01586 template <class T> inline
01587 void wbvector<T>::range(T &xmin, T&xmax) const {
01588     if (len==0) {
01589         xmin=xmax=0;
01590         wblog(FL,"WRN %s() of null vector",FCT);
01591     }
01592     else {
01593         xmin=xmax=data[0];
01594         for (size_t i=1; i<len; i++) {
01595             if (xmin>data[i]) xmin=data[i];
01596             if (xmax<data[i]) xmax=data[i];
01597         }
01598     }
01599 };
01600
01601 template <class T> inline
01602 T wbvector<T>::range() const {
01603     T xmin, xmax; range(xmin,xmax);
01604     return (xmax-xmin);
01605 };
01606
01607 template <class T> inline
01608 T wbvector<T>::avg() const {
01609
01610     if (!len) {
01611         wblog(FL,"WRN %s() of null vector !?",FCT);
01612         return 0;
01613     }
01614     else {
01615         T x=data[0];
01616         for (size_t i=1; i<len; ++i) { x+=data[i]; }
01617         return (x/T(len));
01618     }
01619 };
01620
01621 template <class T> inline
01622 T wbvector<T>::std() const {
01623
01624     if (len<2) {
01625         wblog(FL,"WRN %s() of vector of length %d !?",FCT,len);
01626         return 0;
01627     }
01628     else {
01629         T x=data[0], x2=data[0]*data[0];
01630         for (size_t i=1; i<len; ++i) { x+=data[i]; x2+=data[i]*data[i]; }
01631         return SQRT( ( x2 - (x*x)/T(len) ) / T(len-1) );
01632     }
01633 };
01634
01635 template <class T> inline
01636 int wbvector<T>::findClosest(T r) const {
01637     if (len==0) return -1;
01638
01639     T dbl, eps=ABS(data[0]-r); size_t i,k=0;
01640
01641     for (i=1; i<len; i++) {
01642         dbl = ABS(data[i]-r);
01643         if (dbl<eps) { k=i; eps=dbl; if (eps==0) break; }
01644     }
01645

```

```

01646     return k;
01647 }
01648
01649 template <class T>
01650 int wbvector<T>::findClosestSorted(T r) const {
01651     if (len==0) return -1;
01652
01653     size_t i, k, n1=0, n2=len, e=0;
01654     char c, cref;
01655
01656     if (len==0) return -1;
01657     if (len==1) return 0;
01658
01659     cref=NUMCMP(data[0], data[len-1]);
01660
01661     if (cref) {
01662         if (len>6) {
01663             for (i=1; i<4; i++) if (NUMCMP(data[i-1],data[i])==-cref) e++;
01664             for (i=len-3; i<len; i++)
01665                 if (NUMCMP(data[i-1],data[i])==-cref) e++;
01666         }
01667         else
01668             for (i=1; i<len; i++) if (NUMCMP(data[i-1],data[i])==-cref) e++;
01669     }
01670     else for (i=1; i<len; i++) if (NUMCMP(data[i-1],data[i])) e++;
01671
01672     if (e) wblog(FL,"ERR Data not sorted! (%d)", e);
01673     if (cref==0) { wblog(FL,
01674         "WRN find closest: all data constant!\n(%g; %g %dx)",
01675         (double)r, (double)data[0], len); return 0;
01676     }
01677
01678     c=NUMCMP(data[len-1],r); if (c!=-cref) return len-1;
01679     c=NUMCMP(data[0],r); if (c!= cref) return 0;
01680
01681     for (k=0;;) {
01682         if (c==cref) {
01683             n1=n2/2; n2-=n1; if (n1) k+=n1; else {
01684                 if (k+1<len && (ABS(r-data[k]) > ABS(r-data[k+1]))) k++;
01685                 break;
01686             }
01687         }
01688         else if (c) {
01689             n2=n1/2; n1-=n2; if (n2) k-=n2; else {
01690                 if (k) if (ABS(r-data[k]) > ABS(r-data[k-1])) k--;
01691                 break;
01692             }
01693         }
01694         else break;
01695
01696         c=NUMCMP(data[k],r);
01697     }
01698
01699     return k;
01700 }
01701
01702 template <class T>
01703 wbvector<T>& wbvector<T>::setRand(double fac, double shift) {
01704     size_t i;
01705     static char first_call=1;
01706
01707     if (first_call) { wb_srand(); first_call=0; }
01708     fac/=(double)RAND_MAX;
01709
01710     if (shift==0.)
01711         for (i=0; i<len; ++i) data[i]=(T) (fac*rand());
01712     else for (i=0; i<len; ++i) data[i]=(T) (fac*rand()+shift);
01713
01714     return *this;
01715 };
01716
01717 template<>
01718 wbvector<wbcomplex>& wbvector<wbcomplex>::setRand(double fac, double shift) {
01719     size_t i;
01720     static char first_call=1;
01721
01722     if (first_call) { wb_srand(); first_call=0; }
01723     fac/=(double)RAND_MAX;
01724
01725     if (shift==0.) {
01726         for (i=0; i<len; ++i)
01727             data[i].set(fac*rand(), fac*rand());
01728     }
01729     else {
01730         for (i=0; i<len; ++i)
01731             data[i].set(fac*rand()+shift, fac*rand()+shift);
01732     }
01733 }

```

```

01733
01734     return *this;
01735 };
01736
01737 template <class T> inline
01738 void wbvector<T>::setIdx(double shift, double fac) {
01739     size_t i;
01740
01741     if (shift==0.) {
01742         if (fac==1.)
01743             for (i=0; i<len; i++) data[i]=(T)(i);
01744         else for (i=0; i<len; i++) data[i]=(T)(fac*i);
01745     }
01746     else {
01747         if (fac==1.)
01748             for (i=0; i<len; i++) data[i]=(T)(i+shift);
01749         else for (i=0; i<len; i++) data[i]=(T)(fac*i+shift);
01750     }
01751 };
01752
01753 template <class T>
01754 void wbvector<T>::mat2mx(mxArray* &a, char dim) const {
01755
01756     if (dim==1) a=mxCreateDoubleMatrix(len,1,mxREAL); else
01757     if (dim==2) a=mxCreateDoubleMatrix(1,len,mxREAL);
01758     else wblog(FL,"ERR %s() invalid dim=%d",FCT,dim);
01759
01760     double *d=mxGetDoubles(a);
01761     for (size_t i=0; i<len; ++i) d[i]=(double)data[i];
01762 };
01763
01764 template <class T>
01765 void wbvector<T>::mat2mxs(mxArray* a, char fid, char dim) const {
01766
01767     size_t i=-1, m=1, n=len;
01768
01769     if (!mxIsStruct(a) || (i=mxGetNumberOfElements(a))!=1) wblog(FL,
01770 "ERR %s() invalid input structure (n=%d)",FCT,i);
01771     if (fid>=int(i=mxGetNumberOfFields(a))) wblog(FL,
01772 "ERR %s() field index out of bounds (%d/%d)",FCT,fid,i);
01773
01774     if (dim==1) { m=len; n=1; } else
01775     if (dim==2) { m=1; n=len; } else
01776         wblog(FL,"ERR %s() invalid dim=%d",FCT,dim);
01777
01778     mxArray *b=mxCreateDoubleMatrix(m,n,mxREAL);
01779     double *d=mxGetDoubles(b);
01780
01781     for (i=0; i<len; ++i) { d[i]=(double)data[i]; }
01782
01783     mxSetFieldByNumber(a,0,fid,b);
01784 };
01785
01786 template <class T> inline
01787 int wbvector<T>::recLess(const T* v) const {
01788
01789     for (size_t i=0; i<len; i++)
01790         if (data[i]<v[i]) return 1; else
01791         if (data[i]>v[i]) return 0;
01792
01793     return 0;
01794 }
01795
01796 template <class T> inline
01797 int wbvector<T>::recLessE(const T* v) const {
01798
01799     for (size_t i=0; i<len; i++)
01800         if (data[i]<v[i]) return 1; else
01801         if (data[i]>v[i]) return 0;
01802
01803     return 1;
01804 }
01805
01806 template <class T> inline
01807 int wbvector<T>::recCompare(const T* v) const {
01808
01809     for (size_t i=0; i<len; i++)
01810         if (data[i]<v[i]) return -1; else
01811         if (data[i]>v[i]) return +1;
01812
01813     return 0;
01814 }
01815
01816 template<>
01817 mxArray* wbvector<long>::toMx_base(const char tflag) const {
01818     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01819     return Mx::Array<long>(m,n).copyFromTR(data);

```

```

01820 };
01821
01822 template<>
01823 mxArray* wbvector<size_t>::toMx_base(const char tflag) const {
01824     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01825     return Mx::Array<size_t>(m,n).copyFromTR(data);
01826 };
01827
01828 template<>
01829 mxArray* wbvector<wbcomplex>::toMx_base(const char tflag) const {
01830     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01831     return Mx::Array<wbcomplex>(m,n).copyFromTR(data);
01832 };
01833
01834 template<> int wbvector<double>::init(
01835     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01836 { return init_base(F,L,a,istr,tcheck,ref); }
01837
01838 template<> int wbvector<wbcomplex>::init(
01839     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01840 { return init_base(F,L,a,istr,tcheck,ref); }
01841
01842 template<> int wbvector<char>::init(
01843     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01844 { return init_base(F,L,a,istr,tcheck,ref); }
01845
01846 template<> int wbvector<int>::init(
01847     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01848 { return init_base(F,L,a,istr,tcheck,ref); }
01849
01850 template<> int wbvector<unsigned>::init(
01851     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01852 { return init_base(F,L,a,istr,tcheck,ref); }
01853
01854 template<> int wbvector<size_t>::init(
01855     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01856 { return init_base(F,L,a,istr,tcheck,ref); }
01857
01858 template<> mxArray* wbvector<double>::toMx(const char tflag) const {
01859     return toMx_base(tflag); }
01860 template<> mxArray* wbvector<wbcomplex>::toMx(const char tflag) const {
01861     return toMx_base(tflag); }
01862
01863 template<> mxArray* wbvector<unsigned>::toMx(const char tflag) const {
01864     return toMx_base(tflag); };
01865 template<> mxArray* wbvector<int>::toMx(const char tflag) const {
01866     return toMx_base(tflag); }
01867 template<> mxArray* wbvector<char>::toMx(const char tflag) const {
01868     return toMx_base(tflag); }
01869 template<> mxArray* wbvector<uint8_T>::toMx(const char tflag) const {
01870     return toMx_base(tflag); }
01871 template<> mxArray* wbvector<short>::toMx(const char tflag) const {
01872     return toMx_base(tflag); };
01873 template<> mxArray* wbvector<uint16_T>::toMx(const char tflag) const {
01874     return toMx_base(tflag); };
01875 template<> mxArray* wbvector<long>::toMx(const char tflag) const {
01876     return toMx_base(tflag); };
01877 template<> mxArray* wbvector<size_t>::toMx(const char tflag) const {
01878     return toMx_base(tflag); };
01879
01880 template<class T>
01881 mxArray* wbvector<T>::toMx_Struct(const char tflag) const {
01882
01883     mxArray *S;
01884
01885     if (tflag)
01886         S=data->mxCreatStruct(len,1);
01887     else S=data->mxCreatStruct(1,len);
01888
01889     for (size_t i=0; i<len; ++i) { data[i].add2MxStruct(S,i); }
01890     return S;
01891 }
01892
01893 template<class T>
01894 mxArray* wbvector<T>::toMxP_S(const char tflag) const {
01895
01896     mxArray *S;
01897
01898     if (tflag)
01899         S=(*data)->mxCreatStruct(len,1);
01900     else S=(*data)->mxCreatStruct(1,len);
01901
01902     for (size_t i=0; i<len; ++i) { data[i]->add2MxStruct(S,i); }
01903     return S;
01904 };
01905
01906 template<class T>

```



```

01907 mxArray* wbvector<T>::toMx_offset(const T& offset, const char tflag)
01908 const {
01909     if (!len) { return mxCreateDoubleMatrix(0,0,mxREAL); }
01910
01911     mxArray *a=(tflag ?
01912         mxCreateDoubleMatrix(len,1,mxREAL) :
01913         mxCreateDoubleMatrix(1,len,mxREAL)
01914     );
01915
01916     double *d=mxGetDoubles(a);
01917     for (size_t i=0; i<len; ++i) { d[i]=double(data[i]+offset); }
01918
01919     return a;
01920 };
01921
01922 template <class T>
01923 void wbvector<T>::put(
01924     const char *file, int line, const char *vname, char tflag,
01925     const char* ws
01926 ) const {
01927
01928     size_t i;
01929     mxArray *a=toMx(tflag);
01930
01931     if (file) wblog(file,line,
01932         "I/O putting vector '%s' to %s%s",vname,ws,tflag ? " (t)" : ".");
01933
01934     i=mexPutVariable(ws, vname, a);
01935
01936     if (i==1) {
01937         info("this"); wblog(__FILE__,
01938             "ERR failed to put variable %s into workspace %s.",vname,ws);
01939     }
01940
01941     mxDestroyArray(a);
01942     return;
01943 }
01944
01945 inline WBINDEX Index(size_t i1, size_t i2) {
01946     WBINDEX ii;
01947
01948     if (i1>i2) return ii;
01949
01950     ii.init(i2-i1+1);
01951     for (size_t i=0; i<ii.len; i++) ii[i]=i+i1;
01952
01953     return ii;
01954 };
01955
01956 inline char isUniqueIdxSet(const WBINDEX &I0, widx_t imax) {
01957
01958     size_t i, n=I0.len, e=0;
01959     const widx_t *I=I0.data;
01960     wbvector<char> mark(imax); char *m=mark.data;
01961
01962     for (i=0; i<n; i++) if (I[i]<imax) m[I[i]]++; else e++;
01963     for (i=0; i<imax; i++) if (m[i]>1) e++;
01964
01965     return (e==0);
01966 }
01967
01968 char validPerm(const WBPERM &P, wperm_t d) {
01969     char rval=0, id=0; size_t i=0, n=P.len;
01970     const wperm_t *p=P.data;
01971     wbvector<char> mark(n);
01972
01973     if (swperm_t(d)>=0) { if (P.len!=d) return rval; }
01974     for (i=0; i<n; ++i) {
01975         if (p[i]==i) { ++id; } else
01976             if (p[i]>=n || mark[p[i]]++) { return rval; }
01977     }
01978
01979     return (rval=(unsigned(id)==n? 2:1));
01980 };
01981
01982 inline void getIPerm(const WBPERM &P, WBPERM &iP) {
01983
01984     size_t i,n=P.len;
01985
01986     #ifdef __WBDEBUG__
01987         if (!validPerm(P))
01988             wbdie(FLINE,"Invalid permutation.");
01989     #endif
01990
01991     iP.init(n);
01992
01993

```

```

01994     for (i=0; i<n; i++) iP[P[i]]=i;
01995
01996     return;
01997 }
01998
01999 char isIdentityPerm(const WBPERM &P) {
02000
02001     for (size_t i=0; i<P.len; i++) if (P[i]!=i) return 0;
02002     return 1;
02003 }
02004
02005 #endif
02006

```

5.65 xmap.hh

```

00001 #ifndef __WB_XMAP_HCC__
00002 #define __WB_XMAP_HCC__
00003
00004 /* ----- *
00005  * class XMAP - map to store contraction pattern
00006  * AW (C) Jun 2006
00007  * ----- */
00008
00009 // ----- //
00010 // ----- //
00011 // central element for contraction map
00012
00013 class XMAP_Node {
00014
00015 public:
00016
00017     XMAP_Node ()
00018     : Z(NULL), R(NULL), I(NULL),
00019       tflag(0), ic(0), il(0), i2(0), isref(0), z(1.) {};
00020
00021     ~XMAP_Node () {};
00022
00023     free(){
00024         if (isref) { Z=R=NULL; }
00025         else {
00026             if (Z) { delete Z; Z=NULL; }
00027             if (R) { delete R; R=NULL; }
00028         }
00029     };
00030
00031     void init(const mxArray *a);
00032
00033     wbvector<unsigned> size() const {
00034         if (Z) return (*Z).SIZE; else
00035         if (R) return (*R).SIZE;
00036         else return wbvector<unsigned>();
00037     };
00038
00039     unsigned rank() const {
00040         if (Z) return (*Z).rank(); else
00041         if (R) return (*R).rank();
00042         else return 0;
00043     };
00044
00045     bool sameSize(const XMAP_Node &B) const {
00046         if ((!R ^ !Z) || (!B.R ^ !B.Z)) wblog(FL,
00047         "ERR Severe XMAP inconsistency.");
00048
00049         return (
00050             (R ? (*R).SIZE : (*Z).SIZE) !=
00051             (B.R ? (*B.R).SIZE : (*B.Z).SIZE)
00052         );
00053     };
00054
00055     template<class T>
00056     void set(
00057         const wbArray<T> &A,
00058         unsigned ic, unsigned il, unsigned i2, wbcomplex z, char ref=1
00059     );
00060
00061     void set(
00062         unsigned ic, unsigned il, unsigned i2, wbcomplex z, char ref=1
00063     );
00064
00065     wbArray<wbcomplex> *Z;
00066     wbArray<double> *R;
00067

```

```

00068     char tflag;
00069     wbpem P;
00070
00071     unsigned ic;
00072     unsigned il;
00073     unsigned i2;
00074
00075     unsigned isref;
00076
00077     wbcomplex z;
00078
00079 protected:
00080 private:
00081 };
00082 };
00083
00084 class XMAP : public wvector<XMAP_Node> {
00085 public:
00086     XMAP(unsigned n) : isref(1) { initDef(n); };
00087
00088     ~XMAP() {};
00089
00090     free() {
00091         for (unsigned i=0; i<len; i++)
00092             data[i].free();
00093     };
00094
00095     void init(unsigned n) { free(); initDef(n); };
00096
00097     template<class T>
00098     void set(
00099         unsigned k, const wArray<T> &A,
00100         unsigned ic, unsigned il, unsigned i2, wbcomplex z
00101     ) { data[k].set(A,ic,il,i2,z, isref); };
00102
00103     void set(
00104         unsigned k,
00105         unsigned ic, unsigned il, unsigned i2, wbcomplex z
00106     ) { data[k].set(ic,il,i2,z, isref); };
00107
00108     char isref;
00109
00110 protected:
00111 private:
00112 };
00113
00114 void XMAP_Node::init(const mxArray *a) {
00115
00116     if (!a) wblog(FL,
00117         "ERR Input mxArray is NULL.");
00118
00119     if (!mxIsDouble(a) || mxIsSparse(a)) wblog(FL,
00120         "ERR Need full double array upon input.");
00121
00122     free();
00123     (*( mxIsComplex(a) ? Z : R )).init(a);
00124 }
00125
00126 template<class T>
00127 void XMAP_Node::set(
00128     unsigned k, const wArray<T> &A,
00129     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
00130 ) {
00131     wblog(FL, "ERR XMAP::set() not available for data type %s",
00132         TSTR(T));
00133 };
00134
00135 void XMAP_Node::set(
00136     unsigned k, const wArray<wbcomplex> &A,
00137     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
00138 ) {
00139     ic=ic_; il=il_; i2=i2_; z=z_;
00140
00141     if (A.rank()!=2) wblog(FL,
00142         "ERR expecting rank-2 tensors to contract.");
00143
00144     if (ref) {
00145         Z=&A; isref=1;
00146     }
00147     else {
00148         if (!Z) Z = new wArray<wbcomplex>;
00149         (*Z)=A; (*Z)*=z;
00150         z=1.;
00151     }
00152 };
00153
00154 void XMAP_Node::set(

```

```

00155     const wbArray<double> &A,
00156     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
00157 ){
00158     ic=ic_; il=il_; i2=i2_; z=z_;
00159
00160     if (A.rank()!=2) wblog(FL,
00161     "ERR expecting rank-2 tensors to contract.");
00162
00163     if (ref) {
00164         R=&A; isref=1;
00165     }
00166     else {
00167         if (z.i==0.) {
00168             if (!R) R = new wbArray<double>;
00169             (*R)=A; (*R)*=z.r;
00170         }
00171         else {
00172             if (!Z) Z = new wbArray<wbcomplex>;
00173             (*Z).set(A, wbArray<double>());
00174             (*Z)*=z;
00175         }
00176         z=1.;
00177     }
00178 };
00179
00180 void XMAP_Node::set(
00181     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_
00182 ){
00183     ic=ic_; il=il_; i2=i2_; z=z_;
00184
00185     if (Z && R) wblog(FL,
00186     "ERR Both Z and R are set (0x%lX, 0x%lX) ???", Z, R);
00187
00188     if (Z) (*Z)*=z; else
00189     if (R) {
00190         if (z.i==0) (*R)*=z.r;
00191         else {
00192             Z = new wbArray<wbcomplex>;
00193             Z.set(R, wbArray<double>()); (*Z)*=z;
00194             delete R; R=NULL;
00195         }
00196     };
00197
00198     z=1.;
00199 };
00200
00201 void XMAP::init(const mxArray *a) {
00202     unsigned i,N;
00203
00204     if (mxIsCell(a)) {
00205         N=mxGetNumberOfElements(a);
00206         init(N);
00207
00208         for (i=0; i<N; i++)
00209             data[i].init(mxGetCell(a,i));
00210     }
00211     else
00212     if (mxIsDouble(a)) {
00213         N=1; init(N); data[0].init(a);
00214     }
00215
00216     for (i=0; i<N; i++) {
00217         if (data[i].rank()!=2) wblog(FL,
00218         "ERR Expecting rank-2 tensors (%d)", data[i].rank());
00219     }
00220 }
00221
00222 #endif
00223

```

Index

[__wb_msgbuf](#), 11

[blockSpaceQS< TQ, TD >](#), 20

[cdata< TD >](#), 20

[CData< TQ, TD >](#), 27

[CG::FileLock](#), 55

[cgc_contract_id< TM >](#), 31

[cgdStatus](#), 37

[cgrType](#), 38

[CPAT< TQ, TD >](#), 42

[CPAT_Set< TQ, TD >](#), 43

[cpu_time](#), 43

[CRef< TQ >](#), 44

[CStore< TQ >](#), 46

[ctrlIdx](#), 48

[DY::Symmetry](#), 132

[DY::weight_info< TQ >](#), 206

[DY::Weights< TQ >](#), 206

[genRG_base< TQ, TD >](#), 55

[genRG_struct< TQ, TD >](#), 56

[groupIndex< T >](#), 57

[icFlags](#), 58

[indexSparseRef< TD >](#), 58

[is_pointer_< T >](#), 59

[is_pointer_< T * >](#), 59

[is_pointer_< T ** >](#), 60

[is_pointer_< T *** >](#), 60

[itag_](#), 60

[iTags](#), 61

[IterOM_DATA< TD >](#), 70

[iterSparseCMref< TD >](#), 70

[MEM_SET< T >](#), 72

[MEM_SET< T * >](#), 73

[MHash< TM >](#), 73

[Mx::Array< T >](#), 12

[mxArray_buf](#), 76

[MXPut](#), 77

[mxStruct](#), 78

[NRGData< TQ, TD >](#), 79

[NRGIndex< TQ, TD >](#), 81

[opFlags< T >](#), 82

[OPTS](#), 82

[pairPatternCG< T >](#), 83

[QBlock< TQ, TD, TE >](#), 84

[QDir](#), 85

[QHash< TQ >](#), 92

[QHash_](#), 92

[QMap< TQ >](#), 93

[QSet< TQ >](#), 100

[qset< TQ >](#), 93

[QSpace< TQ, TD >](#), 102

[QType](#), 109

[QVec](#), 110

[RCStore](#), 118

[RIXS_para](#), 119

[RSpecData< TS >](#), 120

[RStore< TQ, TD >](#), 120

[sparseCollector2D< TD >](#), 122

[sparseIndex2D< T >](#), 122

[Spectral< TS >](#), 123

[STAT_DGEMM](#), 124

[SVD_Data< TQ, TD >](#), 131

[TDSpectral< TS >](#), 133

[tensor_ref_](#), 133

[tensorRef_](#), 134

[tensorRefs](#), 135

[Wb::ARGV](#), 11

[Wb::Bitset](#), 13

[Wb::bitset< T >](#), 19

[Wb::cc_quad](#), 20

[Wb::citerator< T, T2 >](#), 39

[Wb::citerator< T, T2 * >](#), 40

[Wb::CleanUp](#), 40

[Wb::Clock](#), 40

[Wb::ClockSet](#), 41

[Wb::counter](#), 41

[Wb::gpara_](#), 57

[Wb::IOstat](#), 59

[Wb::iterator< T, T2 >](#), 69

[Wb::iterator< T, T2 * >](#), 69

[Wb::line_stat_](#), 71

[Wb::LogException](#), 71

[Wb::matFile](#), 72

[Wb::mpfr_< P >](#), 73

[Wb::MREC](#), 76

[Wb::myname_](#), 78

[Wb::num2Fmt< T >](#), 81

[Wb::Path](#), 83

[Wb::Profile](#), 84

Wb::rand, [118](#)
Wb::SigHandler, [121](#)
Wb::sptr< T >, [124](#)
Wb::String, [125](#)
Wb::termcolor, [141](#)
Wb::tmpFile, [142](#)
Wb::VersionInfo, [142](#)
Wb::WbListMTRACK, [167](#)
wbarray< T >, [143](#)
wbarrRef< T >, [151](#)
wbcomplex, [151](#)
wbdebug_dummy, [153](#)
wbhist, [153](#)
wbIndex, [160](#)
wbindex, [153](#)
wblog::SBUF, [121](#)
wblog::stdio_buf, [125](#)
wblogbuf_struct, [168](#)
wblogbuffer, [168](#)
wbMatrix< T >, [169](#)
wbMsg, [176](#)
wbperm, [176](#)
wbRecs< T >, [184](#)
wbREF< TX >, [184](#)
wbsparray< TD >, [184](#)
wbstring, [190](#)
wbtopy, [198](#)
WbUtil< T >, [199](#)
wbvector< T >, [199](#)

X3Map< TQ, TD >, [208](#)
x3map< TQ, TD >, [207](#)
XMAP, [208](#)
XMAP_CMat, [215](#)
XMAP_Node, [215](#)